ClusterControl Reference Manual

Release 2.4.0

Severalnines

Jun 18, 2026

Abstract

This is the ClusterControl Reference Manual. It documents ClusterControl through version 2.4.0, including installation, configuration, cluster deployment, monitoring, and management features.

For notes detailing the changes in each release, see the ClusterControl Release Notes.

For help with using ClusterControl, please visit the Severalnines support channels, where you can discuss your issues with other users.

Document generated on: June 18, 2026

TABLE OF CONTENTS

1 Getting Started **20**
1.1 Getting Started with ClusterControl . . . 20
1.1.1 What is ClusterControl? . . . 20
1.1.2 Why choose ClusterControl? . . . 21
1.1.3 Editions and releases . . . 21
1.1.4 Target users . . . 21
1.2 How it Works? . . . 22
1.2.1 Components . . . 23
1.2.2 Architecture . . . 24
1.2.3 Management and automation operations . . . 26
1.2.4 Monitoring operation . . . 26
1.2.5 Multiple controllers management . . . 28
1.3 Supported databases . . . 29
1.3.1 Database servers . . . 29
1.3.2 Auxiliary tools . . . 31
1.4 Key Features . . . 31
1.4.1 All cluster types . . . 31
1.4.2 Cluster-specific features . . . 32
1.5 Requirements . . . 40
1.5.1 Hardware . . . 40
1.5.2 Operating system . . . 42
1.5.3 Software dependencies . . . 42
1.5.4 Firewall and security groups . . . 43
1.5.5 Operating system user . . . 44
1.5.6 SSH key-based authentication . . . 47
1.5.7 Timezone . . . 50
1.6 General . . . 50
1.6.1 Is ClusterControl free? . . . 50
1.6.2 Is there a live demo site where I can try out ClusterControl? . . . 50
1.6.3 Can I purchase the license for the first year only? . . . 50
1.6.4 My 30-day demo license has expired but I haven't had enough time to test ClusterControl. Can I request for an extension? . . . 50
1.6.5 I would like to subscribe as a paid user but have a limited budget. What should I do? 51
1.6.6 Can I resell your product? . . . 51
1.6.7 Who uses ClusterControl? . . . 51
1.7 Technical . . . 51
1.7.1 Do I need to be a DBA to use ClusterControl? . . . 51
1.7.2 Does ClusterControl run on Windows? . . . 51

1.7.3 How do I upgrade ClusterControl? . . . 51
1.7.4 I'm using ClusterControl community version. Which support channel should I use? 51
1.7.5 Can I install ClusterControl on RHEL-based system but my database cluster is running on Debian? . . . 51
1.7.6 Can I use ClusterControl in an offline environment (without Internet)? . . . 51
1.7.7 Can I have a mix of different SSH users per cluster? . . . 51
1.7.8 Can I use a non-privileged SSH user for ClusterControl? . . . 52
1.7.9 What is difference of MySQL Galera Cluster, Percona XtraDB Cluster and MariaDB Galera Cluster? . . . 52
1.7.10 I found a bug. Where should I report it? . . . 52
1.7.11 My cluster is down. What should I do? . . . 52
1.7.12 How can I submit a feature request? . . . 52
1.8 Quickstart . . . 52
1.8.1 Step 1: Install ClusterControl . . . 52
1.8.2 Step 2: Create the first admin user . . . 54
1.8.3 Step 3: Set up SSH key-based authentication . . . 55
1.8.4 Step 4: Deploy or import database cluster/server . . . 55
1.8.5 Step 5: Start managing . . . 57
1.8.6 Step 6: Additional resources . . . 58
1.9 Online Installation . . . 58
1.9.1 Installer script (install-cc) . . . 58
1.9.2 Helm Chart . . . 60
1.9.3 Ansible Role . . . 63
1.9.4 Puppet module . . . 64
1.10 Offline Setup . . . 67
1.10.1 Step 1: Build and export the repository . . . 67
1.10.2 Step 2: Remove default repositories . . . 75
1.10.3 Step 3: Import the repository to the air-gapped host . . . 76
1.10.4 Step 4: Install ClusterControl in offline mode . . . 78
1.10.5 Step 5: Post-installation . . . 80
1.11 Troubleshooting Issues . . . 81
1.11.1 Error page "Not Found" after installation . . . 81
1.12 Manual Installation . . . 82
1.12.1 Requirements . . . 83
1.12.2 Installation Steps . . . 83
1.12.3 Troubleshooting Issues . . . 90
1.13 ClusterControl repository . . . 93
1.14 ClusterControl CLI repository . . . 94
1.15 Legacy GUI Installation . . . 95
1.15.1 Caveats . . . 95
1.15.2 Installation steps . . . 95
1.16 ClusterControl on Marketplaces . . . 104
1.16.1 Current Marketplaces . . . 104
1.16.2 Passwordless SSH connections . . . 104
1.16.3 Launching the ClusterControl Application . . . 104
1.16.4 Getting the Public SSH Key . . . 105
1.16.5 Launching the Hosts for the Database Nodes . . . 105
1.16.6 Deploying a Database Cluster . . . 105
1.17 Tutorials . . . 105
1.17.1 Day-1 Operations . . . 105
1.17.2 Day-2 Operations . . . 106

1.18 Standalone MySQL (MariaDB, Percona, Oracle) . . . 107
1.18.1 Prerequisites . . . 107
1.18.2 Architecture . . . 108
1.18.3 Step 1: Set up SSH key-based authentication . . . 108
1.18.4 Step 2: Deploy a new cluster (single node) . . . 109
1.18.5 Step 3: Monitor your cluster . . . 110
1.18.6 Step 4: Import data . . . 110
1.18.7 Step 5: (Optional) Deploy a new load balancer . . . 110
1.18.8 Step 6: Connect to the database server . . . 111
1.18.9 Step 7: Enable automatic backups . . . 111
1.18.10 Step 8: Configure alerts . . . 112
1.18.11 Step 9: Manage your node . . . 112
1.18.12 Step 10: Scale your cluster . . . 113
1.18.13 Conclusion . . . 114
1.19 MySQL Replication (MariaDB, Percona, Oracle) . . . 114
1.19.1 Prerequisites . . . 115
1.19.2 Architecture . . . 115
1.19.3 Step 1: Set up SSH key-based authentication . . . 116
1.19.4 Step 2: Deploy a new cluster . . . 117
1.19.5 Step 3: Monitor your cluster . . . 118
1.19.6 Step 4: Import data . . . 119
1.19.7 Step 5: Deploy load balancers . . . 119
1.19.8 Step 6: Deploy virtual IP address . . . 121
1.19.9 Step 7: Connect to the database cluster . . . 121
1.19.10 Step 8: Enable automatic backups . . . 122
1.19.11 Step 9: Configure alerts . . . 123
1.19.12 Step 10: Manage your cluster and load balancers . . . 124
1.19.13 Step 11: Scale your cluster . . . 124
1.19.14 Conclusion . . . 125
1.20 MySQL Group Replication . . . 126
1.20.1 Prerequisites . . . 126
1.20.2 Architecture . . . 126
1.20.3 Step 1: Set up SSH key-based authentication . . . 127
1.20.4 Step 2: Deploy a new cluster . . . 128
1.20.5 Step 3: Monitor your cluster . . . 129
1.20.6 Step 4: Import data . . . 130
1.20.7 Step 5: Deploy a new load balancer . . . 131
1.20.8 Step 6: Connecting to the database cluster . . . 132
1.20.9 Step 7: Enable automatic backups . . . 133
1.20.10 Step 8: Configure alerts . . . 134
1.20.11 Step 9: Manage your cluster and load balancer . . . 134
1.20.12 Step 10: Scale your cluster . . . 135
1.20.13 Conclusion . . . 136
1.21 Percona XtraDB Cluster . . . 136
1.21.1 Prerequisites . . . 137
1.21.2 Architecture . . . 137
1.21.3 Step 1: Set up SSH key-based authentication . . . 138
1.21.4 Step 2: Deploy a new cluster . . . 139
1.21.5 Step 3: Monitor your cluster . . . 140
1.21.6 Step 4: Import data . . . 141
1.21.7 Step 5: Deploy a new load balancer . . . 142

1.21.8 Step 6: Connect to the database cluster . . . 143
1.21.9 Step 7: Enable automatic backups . . . 143
1.21.10 Step 8: Configure alerts . . . 145
1.21.11 Step 9: Manage your cluster and load balancer . . . 145
1.21.12 Step 10: Scale your cluster . . . 146
1.21.13 Conclusion . . . 147
1.22 MariaDB Galera Cluster . . . 147
1.22.1 Prerequisites . . . 148
1.22.2 Architecture . . . 148
1.22.3 Step 1: Set up SSH key-based authentication . . . 149
1.22.4 Step 2: Deploy a new cluster . . . 149
1.22.5 Step 3: Monitor your cluster . . . 151
1.22.6 Step 4: Import data . . . 151
1.22.7 Step 5: Deploy load balancers and virtual IP . . . 152
1.22.8 Step 6: Connect to the database cluster . . . 154
1.22.9 Step 7: Enable automatic backups . . . 154
1.22.10 Step 8: Configure alerts . . . 156
1.22.11 Step 9: Manage your cluster and load balancer . . . 156
1.22.12 Step 10: Scale your cluster . . . 157
1.22.13 Conclusion . . . 158
1.23 PostgreSQL Streaming Replication . . . 158
1.23.1 Prerequisites . . . 158
1.23.2 Architecture . . . 159
1.23.3 Step 1: Set up SSH key-based authentication . . . 160
1.23.4 Step 2: Deploy a new cluster . . . 160
1.23.5 Step 3: Monitor your cluster . . . 162
1.23.6 Step 4: Import data . . . 162
1.23.7 Step 5: Deploy a new load balancer . . . 162
1.23.8 Step 6: Connecting to the database cluster . . . 164
1.23.9 Step 7: Enable automatic backups . . . 165
1.23.10 Step 8: Configure alerts . . . 166
1.23.11 Step 9: Manage your cluster and load balancer . . . 166
1.23.12 Step 10: Scale your cluster . . . 167
1.23.13 Conclusion . . . 168
1.24 Prerequisites . . . 168
1.25 Architecture . . . 168
1.26 Step 1: Set up SSH key-based authentication . . . 169
1.27 Step 2: Deploy a new cluster . . . 170
1.28 Step 3: Monitor your cluster . . . 171
1.29 Step 4: Deploy PgBouncer . . . 171
1.30 Step 5: Creating connection pool for PgBouncer . . . 172
1.31 Step 6: Connecting to the database cluster . . . 173
1.32 Step 7: Connecting to the database cluster via PgBouncer . . . 173
1.33 Step 8: Enable automatic backups . . . 173
1.34 Step 9: Configure alerts . . . 174
1.35 Step 10: Manage your cluster . . . 175
1.36 Conclusion . . . 175
1.37 PostgreSQL Logical Replication . . . 175
1.37.1 Introduction . . . 176
1.37.2 Architecture . . . 176
1.37.3 Prerequisites . . . 177

1.37.4 Step 1: Set up SSH key-based authentication . . . 177
1.37.5 Step 2: Deploy a new cluster . . . 178
1.37.6 Step 3: Publication and Subscription . . . 180
1.38 Prerequisites . . . 181
1.39 Architecture . . . 181
1.40 Step 1: Set up SSH key-based authentication . . . 182
1.41 Step 2: Deploy MongoDB replica set . . . 182
1.42 Step 3: Monitor your cluster . . . 184
1.43 Step 4: Schedule automatic backups . . . 184
1.44 Step 5: Configure alerts and notifications . . . 185
1.45 Step 6 Connect to the cluster . . . 185
1.46 Step 7: Cluster management operations . . . 185
1.47 Conclusion . . . 186
1.48 Prerequisites . . . 186
1.49 Architecture . . . 187
1.50 Step 1: Set up SSH key-based authentication . . . 187
1.51 Step 2: Deploy MongoDB Enterprise Replica Set . . . 188
1.52 Step 3: Monitor your cluster . . . 189
1.53 Step 4: Schedule automatic backups . . . 189
1.54 Step 5: Configure alerts and notifications . . . 190
1.55 Step 6: Connect to the cluster . . . 190
1.56 Step 7: Verify Enterprise installation . . . 191
1.57 Step 8: Cluster management operations . . . 191
1.58 Conclusion . . . 191
1.59 Prerequisites . . . 192
1.60 Architecture . . . 192
1.61 Step 1: Set up SSH key-based authentication . . . 192
1.62 Step 2: Deploy MongoDB Sharded Cluster . . . 193
1.63 Step 3: Monitor your cluster . . . 195
1.64 Step 4: Schedule automatic backups . . . 195
1.65 Step 5: Configure alerts and notifications . . . 196
1.66 Step 6 Connect to the cluster . . . 196
1.67 Step 7: Cluster management operations . . . 196
1.68 Conclusion . . . 197
1.69 Redis Replication with Sentinel . . . 197
1.69.1 Prerequisites . . . 197
1.69.2 Architecture . . . 198
1.69.3 Step 1: Set up SSH key-based authentication . . . 198
1.69.4 Step 2: Deploy a Redis Sentinel Cluster . . . 198
1.69.5 Step 3: Monitor your cluster . . . 200
1.69.6 Step 4: Connecting to the database cluster . . . 200
1.69.7 Step 5: Enable automatic backups . . . 200
1.69.8 Step 6: Configure alerts . . . 202
1.69.9 Step 7: Manage your cluster . . . 202
1.69.10 Step 8: Scale your cluster . . . 203
1.69.11 Conclusion . . . 204
1.70 Valkey Replication with Sentinel . . . 204
1.70.1 Prerequisites . . . 204
1.70.2 Architecture . . . 204
1.70.3 Step 1: Set up SSH key-based authentication . . . 205
1.70.4 Step 2: Deploy a Valkey Sentinel Cluster . . . 205

1.70.5 Step 3: Monitor your cluster . . . 207
1.70.6 Step 4: Connecting to the database cluster . . . 207
1.70.7 Step 5: Enable automatic backups . . . 207
1.70.8 Step 6: Configure alerts . . . 208
1.70.9 Step 7: Manage your cluster . . . 209
1.70.10 Step 8: Scale your cluster . . . 209
1.70.11 Conclusion . . . 210
1.71 Redis Cluster . . . 210
1.71.1 Prerequisites . . . 211
1.71.2 Architecture . . . 211
1.71.3 Step 1: Set up SSH key-based authentication . . . 212
1.71.4 Step 2: Deploy a Redis Cluster . . . 212
1.71.5 Step 3: Monitor your cluster . . . 214
1.71.6 Step 4: Connecting to the database cluster . . . 214
1.71.7 Step 5: Enable automatic backups . . . 215
1.71.8 Step 6: Configure alerts . . . 216
1.71.9 Step 7: Manage your cluster . . . 217
1.71.10 Step 8: Scale your cluster . . . 217
1.71.11 Conclusion . . . 218
1.72 Valkey Cluster . . . 218
1.72.1 Prerequisites . . . 219
1.72.2 Architecture . . . 219
1.72.3 Step 1: Set up SSH key-based authentication . . . 219
1.72.4 Step 2: Deploy a Valkey Cluster . . . 220
1.72.5 Step 3: Monitor your cluster . . . 222
1.72.6 Step 4: Connecting to the database cluster . . . 222
1.72.7 Step 5: Enable automatic backups . . . 222
1.72.8 Step 6: Configure alerts . . . 223
1.72.9 Step 7: Manage your cluster . . . 224
1.72.10 Step 8: Scale your cluster . . . 225
1.72.11 Conclusion . . . 225
1.73 SQL Server standalone . . . 226
1.73.1 Prerequisites . . . 226
1.73.2 Architecture . . . 226
1.73.3 Step 1: Set up SSH key-based authentication . . . 227
1.73.4 Step 2: Deploy SQL Server standalone . . . 227
1.73.5 Step 3: Monitor your cluster . . . 228
1.73.6 Step 4: Connecting to the database cluster . . . 229
1.73.7 Step 5: Enable automatic backups . . . 229
1.73.8 Step 6: Configure alerts . . . 230
1.73.9 Step 7: Manage your cluster . . . 230
1.73.10 Conclusion . . . 231
1.74 SQL Server Always On availability group . . . 231
1.74.1 Prerequisites . . . 231
1.74.2 Architecture . . . 232
1.74.3 Step 1: Set up SSH key-based authentication . . . 232
1.74.4 Step 2: Deploy a SQL Server Always On availability group . . . 233
1.74.5 Step 3: Connecting to the database cluster . . . 234
1.74.6 Step 4: Create a database and add into availability group . . . 235
1.74.7 Step 5: Monitor your cluster . . . 235
1.74.8 Step 6: Enable automatic backups . . . 236

1.74.9 Step 7: Configure alerts . . . 237
1.74.10 Step 8: Manage your cluster . . . 237
1.74.11 Step 9: Scale your cluster . . . 238
1.74.12 Conclusion . . . 239
1.75 Elasticsearch Standalone Cluster . . . 239
1.75.1 Prerequisites . . . 239
1.75.2 Architecture . . . 239
1.75.3 Step 1: Set up SSH key-based authentication . . . 240
1.75.4 Step 2: Deploy a new cluster (single node) . . . 241
1.75.5 Step 3: Monitor your cluster . . . 242
1.75.6 Step 4: Import data . . . 242
1.75.7 Step 5: Connect to the database . . . 244
1.75.8 Step 6: Enable automatic backups . . . 244
1.75.9 Step 7: Configure alerts . . . 245
1.75.10 Step 8: Manage your node . . . 245
1.75.11 Conclusion . . . 246
1.76 Elasticsearch High-Availability Cluster . . . 246
1.76.1 Prerequisites . . . 247
1.76.2 Architecture . . . 247
1.76.3 Step 1: Set up SSH key-based authentication . . . 248
1.76.4 Step 2: Deploy a new cluster (high-availability) . . . 248
1.76.5 Step 3: Monitor your cluster . . . 250
1.76.6 Step 4: Import data . . . 250
1.76.7 Step 5: Connect to the database . . . 251
1.76.8 Step 6: Enable automatic backups . . . 252
1.76.9 Step 7: Configure alerts . . . 252
1.76.10 Step 8: Manage your node . . . 253
1.76.11 Step 9: Scale your cluster . . . 253
1.76.12 Conclusion . . . 255
1.77 ClusterControl and Database Clusters Deployment in an Offline Environment . . . 255
1.77.1 Setting up Satellite Repository Server . . . 255
1.77.2 Setting Up Database Nodes . . . 263
1.77.3 ClusterControl Offline Installation . . . 267
1.77.4 Offline Database Deployment using ClusterControl . . . 268
1.78 Use cases . . . 271
1.79 Deploying a Secondary Cluster . . . 272
1.79.1 Replicate from Primary . . . 272
1.79.2 Replicate from Backups . . . 273
1.79.3 Clone cluster . . . 273
1.80 Failover . . . 274
1.81 Cluster Replication Status using CLI . . . 274
1.82 Use cases . . . 275
1.83 Important considerations . . . 275
1.84 Deploying a PostgreSQL cluster . . . 275
1.85 Major upgrades for PostgreSQL . . . 276
1.85.1 Upgrade methods . . . 276
1.85.2 Monitoring upgrade process . . . 276
1.86 Using the ClusterControl CLI . . . 277
1.86.1 Monitoring the upgrade . . . 277
1.87 Use cases . . . 278
1.88 Configuring Cloud Storage . . . 278

1.89 Creating a Backup in the Cloud . . . 279
1.90 Restoring from the Cloud . . . 279
1.91 Automating Backups . . . 279
1.92 Verifying Backups . . . 280
1.93 Accessing Cloud Backups via CLI . . . 280
1.94 Use cases . . . 281
1.95 Configuring point-in-time recovery . . . 281
1.95.1 Enable transaction log archiving . . . 281
1.95.2 Enable full backup . . . 282
1.96 Performing point-in-time recovery . . . 282
1.96.1 Using the ClusterControl GUI . . . 282
1.96.2 Using the ClusterControl CLI . . . 283
1.96.3 Verifying the recovery . . . 283
1.97 Use cases . . . 284
1.98 Creating a read-only user . . . 284
1.99 CLI client configuration . . . 286
1.100 Accessing the RPC API . . . 286
1.101 Limitations of This Method . . . 288
1.102 Pre-Migration Checks . . . 288
1.103 Installing ClusterControl v2 . . . 289
1.104 Importing Existing Clusters into ClusterControl v2 . . . 289
1.104.1 Prerequisites . . . 289
1.104.2 Importing a MySQL/MariaDB standalone or replication . . . 290
1.104.3 Importing a PostgreSQL/TimescaleDB cluster . . . 292
1.104.4 Importing a MongoDB replica set . . . 294
1.105 Importing Configurations from ClusterControl v1 . . . 296
1.105.1 Users and Access . . . 297
1.105.2 Notification Settings . . . 298
1.105.3 SSL Certificates . . . 299
1.105.4 Backup Schedules . . . 300
1.105.5 Scheduled Operational Reports . . . 301
1.106 Post-Migration Checks . . . 302
1.107 User and Access Management . . . 304
1.107.1 List Functions . . . 304
1.107.2 Example Prompt . . . 304
1.108 Cluster Inventory and Health . . . 305
1.108.1 List Functions . . . 305
1.108.2 Example Prompt . . . 305
1.109 Performance Analysis . . . 306
1.109.1 List Functions . . . 306
1.109.2 Example Prompt . . . 306
1.110 Jobs and Operations . . . 307
1.110.1 List Functions . . . 307
1.110.2 Example Prompt . . . 307
1.111 Backups and Scheduling . . . 307
1.111.1 List Functions . . . 307
1.111.2 Example Prompt . . . 307
1.112 ProxySQL . . . 308
1.112.1 List Functions . . . 308
1.112.2 Example Prompt . . . 308
1.113 Configuration . . . 308

1.113.1 List Functions . . . 308
1.113.2 Example Prompt . . . 309
1.114 Maintenance, Audit, and Other Tools . . . 309
1.114.1 List Functions . . . 309
1.114.2 Example Prompt . . . 309

2 User Guide **311**
2.1 ClusterControl User Guide . . . 311
2.2 Overview . . . 311
2.2.1 ClusterControl GUI . . . 312
2.2.2 ClusterControl CLI . . . 313
2.2.3 ClusterControl RPC API . . . 314
2.2.4 Terraform provider for ClusterControl . . . 314
2.3 ClusterControl GUI . . . 315
2.3.1 Accessing the GUI . . . 316
2.3.2 User registration page . . . 316
2.3.3 Login page . . . 316
2.3.4 Home . . . 317
2.3.5 Navigation bar . . . 318
2.3.6 Sidebar . . . 319
2.3.7 Footer . . . 326
2.3.8 Give Us Feedback . . . 326
2.4 ClusterControl CLI . . . 326
2.4.1 Overview . . . 327
2.4.2 Installation . . . 327
2.4.3 Configuring CLI user . . . 328
2.4.4 Using the CLI . . . 331
2.5 Accessing the RPC API . . . 342
2.5.1 Local access . . . 342
2.5.2 Remote access . . . 342
2.6 Authenticating with the API . . . 343
2.7 Successful request . . . 344
2.8 Failed request . . . 344
2.9 Deployment . . . 345
2.10 Create Database Cluster . . . 345
2.10.1 Support Matrix . . . 346
2.10.2 Prerequisites . . . 346
2.10.3 MySQL Replication . . . 346
2.10.4 MySQL Galera . . . 351
2.10.5 MySQL Group Replication . . . 355
2.10.6 PostgreSQL Streaming . . . 359
2.10.7 PostgreSQL Logical . . . 364
2.10.8 TimescaleDB . . . 369
2.10.9 MongoDB Replica Set . . . 373
2.10.10 MongoDB Sharded Cluster . . . 378
2.10.11 Valkey Sentinel . . . 384
2.10.12 Redis Sentinel . . . 388
2.10.13 Valkey Cluster . . . 392
2.10.14 Redis Cluster . . . 396
2.10.15 SQL Server . . . 400
2.10.16 Elasticsearch . . . 404

2.11 Import Database Cluster . . . 407
2.11.1 Support Matrix . . . 408
2.11.2 Prerequisites . . . 408
2.11.3 MySQL Replication . . . 408
2.11.4 MySQL Galera . . . 412
2.11.5 PostgreSQL Streaming . . . 415
2.11.6 TimescaleDB . . . 419
2.11.7 MongoDB Replica Set . . . 421
2.11.8 MongoDB Sharded Cluster . . . 424
2.11.9 Valkey Sentinel . . . 427
2.11.10 Valkey Cluster . . . 430
2.11.11 Redis Sentinel . . . 433
2.11.12 Redis Cluster . . . 436
2.11.13 Microsoft SQL Server . . . 439
2.11.14 Elasticsearch . . . 441
2.11.15 MySQL Group Replication . . . 444
2.12 Create Load Balancer . . . 447
2.12.1 Support Matrix . . . 447
2.12.2 Prerequisites . . . 448
2.12.3 ProxySQL . . . 448
2.12.4 HAProxy . . . 451
2.12.5 MariaDB MaxScale . . . 455
2.12.6 Garbd . . . 457
2.12.7 PgBouncer . . . 458
2.12.8 Keepalived . . . 460
2.13 Import Load Balancer . . . 461
2.13.1 Prerequisites . . . 462
2.13.2 Support Matrix . . . 462
2.13.3 ProxySQL . . . 462
2.13.4 HAProxy . . . 463
2.13.5 MariaDB MaxScale . . . 464
2.13.6 Garbd . . . 466
2.13.7 PgBouncer . . . 467
2.13.8 Keepalived . . . 468
2.14 Observability . . . 469
2.15 Database Status & Health Checks . . . 469
2.15.1 Cluster Summary . . . 470
2.15.2 Dashboard Metrics . . . 470
2.15.3 Performance Advisors . . . 472
2.16 Cluster Node Summary . . . 472
2.17 Node Health Status . . . 473
2.17.1 CPU Utilization . . . 473
2.17.2 Memory Utilization . . . 474
2.17.3 Disk IO Utilization . . . 474
2.18 Node Menu . . . 474
2.19 Configure Slow Query Log . . . 475
2.20 Slow Query Monitor . . . 475
2.21 RealTime Query Monitor . . . 475
2.22 ProxySQL for Query Monitor . . . 476
2.23 Reset Query Monitoring . . . 476
2.24 DB Growth . . . 476

2.25 DB Status . . . 477
2.26 DB Variables . . . 477
2.27 Advisors . . . 478
2.28 Schema Analyzer . . . 478
2.29 Transaction Deadlocks . . . 478
2.30 Running Operations . . . 479
2.31 System Logs . . . 479
2.32 Audit Logs . . . 479
2.33 Audit Logging . . . 479
2.33.1 MariaDB . . . 480
2.33.2 PostgreSQL . . . 480
2.34 Watchlist - Monitoring dashboards in kiosk-mode . . . 481
2.34.1 Access Watchlist management section . . . 482
2.34.2 Add clusters directly into watchlist . . . 482
2.34.3 Create watchlist . . . 483
2.34.4 Monitor in kiosk-mode . . . 483
2.34.5 Share watchlist with others . . . 483
2.34.6 Delete watchlist . . . 484
2.35 Alarms . . . 484
2.36 Jobs . . . 485
2.37 Audit Log . . . 486
2.38 Report templates . . . 486
2.39 Generating a report . . . 487
2.40 Listing and viewing reports . . . 489
2.41 Scheduling a report . . . 489
2.42 Management . . . 490
2.43 Failover and Recovery . . . 491
2.43.1 Node Recovery . . . 491
2.43.2 Cluster Recovery . . . 492
2.44 Cluster Management . . . 495
2.44.1 Cluster Actions . . . 496
2.44.2 Cluster-specific actions . . . 508
2.45 Nodes Management . . . 509
2.45.1 Access the Nodes section . . . 510
2.45.2 Node Actions . . . 510
2.45.3 Replication actions . . . 514
2.45.4 Read-only and binary logging . . . 518
2.45.5 Galera-specific actions . . . 519
2.45.6 MongoDB-specific actions . . . 520
2.45.7 PostgreSQL-specific actions . . . 521
2.45.8 ProxySQL-specific actions . . . 522
2.45.9 Schedule maintenance for a node . . . 523
2.46 Configuration Management . . . 524
2.46.1 Access the configuration management section . . . 525
2.46.2 Refresh configuration files . . . 525
2.46.3 View and edit configuration files . . . 525
2.46.4 Compare new configuration with existing configuration . . . 526
2.46.5 Apply configuration changes across nodes . . . 527
2.46.6 Monitor configuration consistency . . . 528
2.46.7 Configuration template . . . 528
2.47 Upgrades and Patches . . . 532

2.47.1 Upgrade procedures . . . 532
2.47.2 Minor upgrade . . . 533
2.47.3 Major upgrade . . . 535
2.48 Alerts and Notifications . . . 537
2.48.1 Alarms . . . 537
2.48.2 Email notifications . . . 538
2.48.3 Forward alarms to third-party services . . . 541
2.49 Database Users and Privileges . . . 541
2.49.1 Best practices . . . 541
2.49.2 Access the database users management . . . 542
2.49.3 MySQL/MariaDB-based clusters . . . 542
2.49.4 PostgreSQL-based clusters . . . 551
2.49.5 Redis-based clusters . . . 558
2.50 MySQL/MariaDB Replication . . . 564
2.51 MySQL Galera . . . 565
2.52 MySQL Cluster (NDB) . . . 565
2.53 MongoDB ReplicaSet . . . 566
2.54 PostgreSQL/TimeScaleDB Replication . . . 566
2.55 Maintenance . . . 567
2.55.1 Type of maintenance mode . . . 567
2.55.2 Configuring maintenance mode . . . 568
2.55.3 Checking maintenance status . . . 569
2.55.4 Deactivating maintenance mode . . . 569
2.56 Certificate Management . . . 570
2.56.1 Generate Self-signed Certificate . . . 570
2.56.2 Generate Client/Server Certificate . . . 571
2.56.3 Import . . . 572
2.57 Backup and Restore . . . 573
2.58 Overview . . . 573
2.58.1 What you can do . . . 574
2.58.2 Backup jobs . . . 574
2.58.3 Backup encryption and decryption . . . 575
2.58.4 Backup retention . . . 576
2.58.5 Supported backup methods . . . 576
2.59 Create Backup . . . 581
2.59.1 Overview of Backup on Demand . . . 581
2.59.2 Support Matrix . . . 582
2.59.3 Prerequisites . . . 582
2.59.4 Creating the backup . . . 582
2.60 Schedule Backup . . . 618
2.60.1 Support Matrix . . . 618
2.60.2 Prerequisites . . . 619
2.60.3 Scheduling the backup . . . 619
2.61 Restore Backup . . . 650
2.61.1 Overview of Restore Backup . . . 650
2.61.2 Support Matrix . . . 650
2.61.3 Prerequisites . . . 651
2.61.4 MySQL Replication . . . 651
2.61.5 MySQL Galera . . . 654
2.61.6 PostgreSQL/TimescaleDB . . . 656
2.61.7 MongoDB Replica Set . . . 657

2.61.8 MongoDB Sharded Cluster . . . 658
2.61.9 Redis Sentinel Cluster . . . 659
2.61.10 Redis/Valkey Cluster . . . 660
2.61.11 SQL Server . . . 661
2.61.12 ElasticSearch . . . 662
2.62 Restore External Backup . . . 663
2.62.1 Difference with restoring internal vs external backup . . . 663
2.62.2 Support Matrix . . . 664
2.62.3 Prerequisites . . . 664
2.62.4 MySQL Replication . . . 665
2.62.5 MySQL Galera . . . 667
2.63 Backup Verification . . . 669
2.63.1 Overview of Backup Verification . . . 669
2.63.2 Support Matrix . . . 670
2.63.3 Prerequisites . . . 670
2.63.4 MySQL Replication . . . 670
2.63.5 MySQL Galera . . . 673
2.63.6 PostgreSQL/TimescaleDB . . . 676
2.63.7 Redis Sentinel Cluster . . . 679
2.63.8 ElasticSearch . . . 681
2.64 Create Cluster from Backup . . . 682
2.64.1 Support Matrix . . . 682
2.64.2 Prerequisites . . . 682
2.64.3 Creating the Cluster from Backup . . . 683
2.65 Upload Backup to Cloud . . . 686
2.65.1 Uploading backups to the cloud . . . 686
2.65.2 Support Matrix . . . 686
2.65.3 Prerequisites . . . 687
2.65.4 Default Configuration . . . 687
2.65.5 Steps to Upload a Backup . . . 687
2.65.6 Steps to Re-upload a Backup (for backups already in the cloud) . . . 688
2.66 Download Backup from Cloud . . . 689
2.66.1 Support Matrix . . . 690
2.66.2 Prerequisites . . . 690
2.66.3 Default Configuration . . . 690
2.66.4 Steps to Download a Backup . . . 690
2.67 Create Replica Cluster . . . 691
2.67.1 Support Matrix . . . 692
2.67.2 Prerequisites . . . 692
2.67.3 Create Replica Cluster Options . . . 692
2.67.4 Galera Cluster . . . 692
2.67.5 PostgreSQL . . . 695
2.67.6 TimescaleDB . . . 698
2.68 Add Database Node . . . 701
2.68.1 Support Matrix . . . 701
2.68.2 Prerequisites . . . 702
2.68.3 MySQL Galera . . . 702
2.68.4 MongoDB Replica Set . . . 704
2.68.5 MongoDB Sharded Cluster . . . 706
2.68.6 Redis Sentinel . . . 709
2.68.7 Redis/Valkey Cluster . . . 710

2.68.8 SQL Server . . . 712
2.69 Add Replica Node . . . 714
2.69.1 Support Matrix . . . 714
2.69.2 Prerequisites . . . 714
2.69.3 MySQL Replication . . . 715
2.69.4 Galera Cluster . . . 718
2.69.5 PostgreSQL . . . 721
2.69.6 TimescaleDB . . . 723
2.69.7 Redis/Valkey Cluster . . . 725
2.69.8 Post Deployment . . . 727
2.70 Remove Database Node (or Remove Node) . . . 727
2.70.1 Support Matrix . . . 727
2.70.2 Prerequisites . . . 728
2.70.3 Deployment steps . . . 728
2.71 Single-Controller vs Multi-Controller . . . 730
2.72 Activating ClusterControl Ops-C . . . 730
2.73 Controller Management . . . 731
2.73.1 Add Controller . . . 732
2.73.2 Edit Controller . . . 733
2.73.3 Remove Controller . . . 734
2.74 Monitoring Dashboard . . . 735
2.75 ClusterControl Manager . . . 736
2.76 ClusterControl Kubernetes proxy . . . 737
2.77 GitOps . . . 737
2.77.1 Argo CD integration . . . 740
2.77.2 GitHub integration . . . 740
2.78 Deploy your first Kubernetes cluster . . . 740
2.78.1 1. Install ClusterControl Ops-C . . . 741
2.78.2 2. Install the required tools . . . 741
2.78.3 3. Create a Kubernetes cluster . . . 741
2.78.4 4. Connect to a Kubernetes cluster . . . 741
2.78.5 5. Configure the agent . . . 742
2.78.6 6. Set up the repository . . . 742
2.78.7 7. Install Argo CD . . . 743
2.78.8 8. Install the agent . . . 744
2.78.9 9. Wait for environment detection . . . 745
2.78.10 10. Pull requests . . . 745
2.78.11 11. GitOps synchronization . . . 746
2.78.12 12. Confirm activation . . . 746
2.79 Create a Kubernetes namespace . . . 746
2.80 Deploy a database operator . . . 746
2.80.1 Supported operators . . . 747
2.81 Create a new database cluster . . . 747
2.81.1 Database connection details . . . 749
2.82 Resource and database configuration templates . . . 750
2.83 Retrieve and view logs . . . 750
2.84 Purge resources . . . 750
2.85 Troubleshooting issues . . . 751
2.85.1 Restart agent . . . 751
2.85.2 Agent with debug log . . . 751
2.85.3 Basic troubleshooting commands . . . 751

2.86 Example issues and solutions . . . 752
2.86.1 Failed database operators . . . 752
2.86.2 Failed backups . . . 756
2.86.3 Node maintenance and troubleshooting . . . 757
2.87 Cloud providers . . . 758
2.87.1 Prerequisites . . . 758
2.87.2 Creating a New S3 Integration . . . 758
2.88 Modifying existing integrations . . . 765
2.89 Requirements . . . 765
2.90 Configuration . . . 765
2.91 Starting up . . . 766
2.92 Reverting the Vault integration . . . 766
2.93 Limitations . . . 766
2.94 Requirements . . . 767
2.95 Providers . . . 767
2.96 Resources . . . 767
2.97 Quick start . . . 767
2.97.1 Installing and configuring ClusterControl for API access . . . 767
2.97.2 Setting up the Terraform provider for ClusterControl . . . 768
2.97.3 Deploying database clusters using Terraform . . . 769
2.97.4 Destroying a deployed database cluster . . . 770
2.98 LDAP settings . . . 773
2.99 Map LDAP group . . . 774
2.100 Delete mapping group . . . 774
2.101 Supported integrations . . . 775
2.102 Notification behavior . . . 775
2.103 Set up notification . . . 776
2.103.1 Slack . . . 776
2.103.2 Telegram . . . 777
2.103.3 ServiceNow . . . 778
2.103.4 PagerDuty . . . 778
2.103.5 Splunk On-Call (formerly known as VictorOps) . . . 779
2.103.6 OpsGenie . . . 780
2.103.7 ilert . . . 781
2.104 Webhook integration . . . 782
2.105 Events . . . 784
2.105.1 Warning Events . . . 784
2.105.2 Critical Events . . . 787
2.106 Setting up dedicated Prometheus node . . . 789
2.106.1 Host Selection . . . 792
2.106.2 Scrape Interval . . . 792
2.106.3 Data Retention Size . . . 792
2.106.4 Data Directory . . . 792
2.106.5 Scrape Interval . . . 792
2.106.6 Arguments . . . 792
2.106.7 Scrape Interval . . . 793
2.106.8 Arguments . . . 793
2.107 Logging . . . 796
2.108 Debugging . . . 796
2.108.1 Debugging ClusterControl Controller (CMON) . . . 796
2.108.2 Install debugging components . . . 796

2.108.3 Debugging Steps . . . 797
2.109 CMON on Foreground . . . 798
2.110 ClusterControl GUI diagnostic . . . 799
2.111 New deployment abd import cluster . . . 799
2.112 Existing database cluster . . . 799
2.113 Overview . . . 800
2.114 Generating an Error Report . . . 800
2.115 ClusterControl GUI diagnostic . . . 801
2.116 Common Issues . . . 802
2.116.1 ClusterControl Installation . . . 802
2.117 Known Limitations . . . 807
2.117.1 General . . . 807
2.117.2 SSH . . . 807
2.117.3 MySQL/MariaDB . . . 808
2.117.4 Containerization . . . 808
2.117.5 Monitoring . . . 808

3 Reference Manuals **809**
3.1 What You'll Find in This Manual . . . 809
3.1.1 ClusterControl Components . . . 809
3.1.2 Feature Reference . . . 809
3.1.3 Command and API Reference . . . 809
3.1.4 Troubleshooting and Support . . . 809
3.2 Components . . . 809
3.3 ClusterControl Controller . . . 810
3.3.1 Command Line Arguments . . . 810
3.3.2 Startup File . . . 811
3.3.3 Configuration Options . . . 812
3.3.4 LDAP . . . 835
3.3.5 CMON Database . . . 839
3.4 ClusterControl GUI . . . 839
3.4.1 Introduction . . . 839
3.4.2 Files and location . . . 840
3.4.3 Managing multiple controllers . . . 840
3.5 ClusterControl CLI . . . 842
3.5.1 s9s-account . . . 842
3.5.2 s9s-alarm . . . 845
3.5.3 s9s-backup . . . 846
3.5.4 s9s-cluster . . . 856
3.5.5 s9s-container . . . 876
3.5.6 s9s-controller . . . 881
3.5.7 s9s-job . . . 883
3.5.8 s9s-log . . . 888
3.5.9 s9s-maintenance . . . 890
3.5.10 s9s-metatype . . . 892
3.5.11 s9s-node . . . 894
3.5.12 s9s-process . . . 903
3.5.13 s9s-replication . . . 905
3.5.14 s9s-report . . . 907
3.5.15 s9s-script . . . 909
3.5.16 s9s-server . . . 911

3.5.17 s9s-tree . . . 914
3.5.18 s9s-user . . . 918
3.6 ClusterControl SSH . . . 925
3.7 ClusterControl Notifications . . . 926
3.8 ClusterControl Cloud . . . 926
3.8.1 Commands . . . 927
3.8.2 Options . . . 927
3.9 ClusterControl Cloud File Manager . . . 927
3.9.1 Usage . . . 928
3.9.2 Commands . . . 928
3.9.3 Options . . . 928
3.9.4 Examples . . . 928
3.10 ClusterControl Proxy . . . 929
3.10.1 ClusterControl Manager . . . 929
3.10.2 ClusterControl Manager Admin CLI . . . 930
3.10.3 Configuration file . . . 931
3.10.4 Configure custom SSL certificates . . . 933
3.10.5 Web server: headers and simple examples . . . 933
3.11 ClusterControl Agent . . . 935
3.11.1 Query Monitor Agents . . . 935
3.12 ClusterControl CMON RPC v2 API Reference . . . 937
3.12.1 Introduction . . . 937
3.12.2 Alarms . . . 939
3.12.3 Information about Controller's audit log . . . 943
3.12.4 User Authentication . . . 945
3.12.5 Information about Backups . . . 950
3.12.6 Certificate Authority (CA) . . . 959
3.12.7 Cloud credentials API . . . 968
3.12.8 Cluster Information and Manipulation . . . 973
3.12.9 Configuration . . . 993
3.12.10 Controllers and HA . . . 1013
3.12.11 Discovery . . . 1017
3.12.12 Hosts and Containers . . . 1026
3.12.13 Job Information and Manipulation . . . 1035
3.12.14 Log Related Calls . . . 1046
3.12.15 PostgreSQL Logical Replication . . . 1053
3.12.16 Maintenance . . . 1059
3.12.17 Mysql databaseperformance . . . 1066
3.12.18 Reports . . . 1068
3.12.19 Repositories . . . 1076
3.12.20 Cmon Events . . . 1078
3.12.21 Statistical Data Calls . . . 1079
3.12.22 CMON Directory Tree . . . 1099
3.12.23 User Database manipulation . . . 1108
3.12.24 Controllers manipulation on database . . . 1135
3.12.25 Watchlists manipulation on database . . . 1137
3.13 ClusterControl Domain Specific Language (DSL) . . . 1140
3.13.1 Introduction . . . 1140
3.13.2 Language basics . . . 1140
3.13.3 Functions . . . 1143
3.13.4 Examples . . . 1163

3.14 ClusterControl Client SDK . . . 1169
3.14.1 SDK for various programming languages . . . 1170
3.14.2 Generating Sources from the API definition . . . 1170
3.15 Provider Configuration . . . 1172
3.16 Resources . . . 1172
3.16.1 clustercontrol_db_cluster . . . 1172
3.16.2 clustercontrol_db_cluster_backup . . . 1177
3.16.3 clustercontrol_db_cluster_backup_schedule . . . 1178
3.16.4 clustercontrol_db_cluster_maintenance . . . 1179
3.17 Common fields in the resource definition . . . 1179
3.17.1 Resource – clustercontrol_db_cluster . . . 1179
3.17.2 Scheduling Backups using the clustercontrol_db_cluster_backup_scheduler resource 1179
3.17.3 Taking adhoc backups using the clustercontrol_db_cluster_backup resource . . . 1180
3.17.4 Setting a maintenance window using the clustercontrol_db_cluster_maintenance resource . . . 1180
3.18 Supported backup methods for the respective database types (and vendors) . . . 1181
3.19 Toggling cluster auto-recovery option . . . 1181
3.20 Build . . . 1181
3.21 Create a CMON user for the cmon_exporter . . . 1181
3.22 How to run . . . 1182
3.22.1 Running in foreground . . . 1182
3.22.2 Systemd . . . 1182
3.22.3 Docker . . . 1182
3.23 Access the metrics . . . 1182
3.24 Current metrics . . . 1182
3.25 Install . . . 1185
3.26 Options . . . 1185
3.27 Usage . . . 1185
3.28 Run as systemd . . . 1186
3.29 Sample prometheus.yml . . . 1187
3.30 ClusterControl MCP . . . 1188
3.30.1 How It Works . . . 1188
3.30.2 Configuration . . . 1189
3.30.3 Connecting to AI Clients . . . 1191
3.30.4 Verifying the Connection . . . 1193

CHAPTER

ONE

GETTING STARTED

1.1 Getting Started with ClusterControl

Welcome to the **ClusterControl** documentation! This guide will help you get started with ClusterControl, a powerful and user-friendly tool designed to simplify the deployment, management, and monitoring of high-availability database clusters. Whether you're working with open-source databases such as MySQL, MariaDB, PostgreSQL, TimescaleDB, MongoDB, Redis, Valkey as well as proprietary options like SQL Server, MongoDB Enterprise, and EDB. ClusterControl provides a unified interface and advanced features to automate and secure your database operations.

1.1.1 What is ClusterControl?

ClusterControl is an all-in-one database management system that provides robust features for monitoring, scaling, and automating database clusters developed by Severalnines.

It supports multiple database technologies and offers a single platform for:

- **Automated Deployment**: Deploy various database clusters with a few clicks, supporting topologies like master-slave, Galera clusters, and sharded environments.
- **Comprehensive Monitoring**: Gain real-time visibility into cluster health, resource usage, and performance metrics with customizable dashboards and alerts.
- **Backup and Recovery**: Schedule automatic backups, configure retention policies, and restore clusters with ease.
- **Scaling and Load Balancing**: Automatically or manually scale clusters to handle workload spikes while ensuring stability and high availability.
- **Security and Compliance**: Manage user permissions, enforce SSL, and track access logs to maintain a secure database environment.

With its focus on ease of use and powerful automation, ClusterControl is ideal for both seasoned database administrators and those new to database cluster management.

1.1.2 Why choose ClusterControl?

Managing a database cluster can be complex, involving manual configuration, monitoring, and scaling to ensure that applications run smoothly. ClusterControl simplifies these processes by centralizing them in intuitive interfaces (GUI, CLI, API), with support for essential features:

- **High Availability**: ClusterControl's clustering and failover capabilities minimize downtime and keep databases available.
- **Centralized Management**: Handle all cluster configurations, monitoring, and updates from a single control panel.
- **Automation and Efficiency**: Reduce manual effort with automated tasks like backups, recovery, scaling, and upgrades.
- **Multi-Database Support**: A wide range of supported databases and cloud integrations make ClusterControl a versatile tool.

1.1.3 Editions and releases

ClusterControl comes in 3 editions - **Community**, **Advanced** and **Enterprise** editions, within the same binary. Please review the ClusterControl product page for features comparison between these editions.

All new ClusterControl installations will be automatically configured with a 30-day fully functional Enterprise edition trial license (1). When the license expires, ClusterControl defaults to the Community Edition. To upgrade from Community to Advanced or Enterprise, you need a valid software license. For commercial information, please contact us. { .annotate }

1. This requires an internet connection for automatic license retrieval and activation. For an offline environment, reach out to our Sales department at sales@severalnines.com to request for a free trial license.

Starting from ClusterControl 2.2.0 (September 2024), ClusterControl provides 2 releases:

Release branch	Version	Release date
Yearly long-term support (LTS)	2.4.0	February 2026
Latest (default)	2.4.0	February 2026

See Release Notes for details of all ClusterControl's releases.

1.1.4 Target users

ClusterControl is a database management and automation platform designed for DBAs (Database Administrators), DevOps teams, system administrators, and IT operations professionals who are responsible for deploying, managing, and maintaining database clusters. Its target users typically have a need for high-availability, scalability, performance monitoring, and automation in database environments.

Target user	Purpose	Use case
DBAs (Database Administrators)	ClusterControl simplifies tasks like database deployment, configuration, backup, and scaling. It also provides monitoring, failure recovery, and performance optimization, reducing the manual work DBAs have to do.	Automating repetitive tasks like backups, failover management, and upgrades for complex database systems.
DevOps Engineers	DevOps teams benefit from ClusterControl's ability to automate infrastructure-as-code (IaC) tasks, enabling rapid provisioning and scaling of databases.	Deploying and managing database clusters as part of continuous integration/continuous deployment (CI/CD) pipelines.
System Administrators / IT Operations	For IT admins responsible for managing infrastructure, ClusterControl offers a unified dashboard for monitoring database health, setting up alerts, and troubleshooting database issues across multiple environments.	Monitoring and maintaining database uptime, setting up automated alerts, and ensuring disaster recovery protocols are in place.
Development Teams (for large-scale applications)	Developers working on applications with high database demands use ClusterControl to ensure that their databases are scalable and reliable, supporting high-performance queries and transactions.	Scaling databases to handle increased traffic as application usage grows.
Companies Managing Multiple Databases	Organizations running multiple database engines (like MySQL, PostgreSQL, MongoDB, etc.) can use ClusterControl to centrally manage these varied database environments.	Managing hybrid database environments with multiple technologies under one platform.
Enterprises with High-Availability Requirements	Enterprises that require high availability and failover solutions for mission-critical databases use ClusterControl for its robust failover, replication, and clustering capabilities.	Ensuring zero-downtime for databases running business-critical applications.

1.2 How it Works?

At its core, ClusterControl operates as a central management layer, integrating with multiple database technologies to streamline complex tasks and ensure high availability and performance. Using automation and intelligent monitoring, it allows you to deploy, scale, back up, and recover clusters with minimal manual intervention.

This page details the underlying mechanisms and processes that allow ClusterControl to function as a reliable, secure, and efficient solution for managing and optimizing your entire database infrastructure.

1.2.1 Components

Primarily, ClusterControl consists of the following components:

Component	Package name	Description
ClusterControl Controller (cmon)	clusterc controll	The brain of ClusterControl. A backend service performing automation, management, monitoring and scheduling tasks on the managed nodes.
ClusterControl GUI	clusterc mcc	A modern web user interface to visualize and manage the cluster. It also supports for centralized management of multiple ClusterControl Controller instances, also known as ClusterControl Operations Center (Ops-C). It interacts with the ClusterControl controller via the `cmon-proxy` service and remote procedure call (RPC).
ClusterControl CLI	s9s-tools	A command-line tool to manage and monitor clusters provisioned by ClusterControl.
ClusterControl SSH	clusterc ssh	A web-based SSH client module implemented using JavaScript and WebSockets.
ClusterControl Notifications	clusterc notificat	A service and interface module for integration with third-party messaging and incident management tools.
ClusterControl Cloud	clusterc cloud	A service and interface module for integration with cloud providers.
ClusterControl Cloud File Manager	clusterc clud	A command-line interface module to interact with cloud storage objects.
ClusterControl Proxy	clusterc proxy	A controller proxying service for ClusterControl GUI.
ClusterControl Kubernetes Proxy	clusterc kuber-proxy	A proxying service to interact with database clusters running on Kubernetes platform.

For a large deployment, ClusterControl provides an additional suite to provision multiple ClusterControl instances at once, called ClusterControl Operations Center (Ops-C). This feature is now merged within ClusterControl GUI and can be activated under the **License** section.

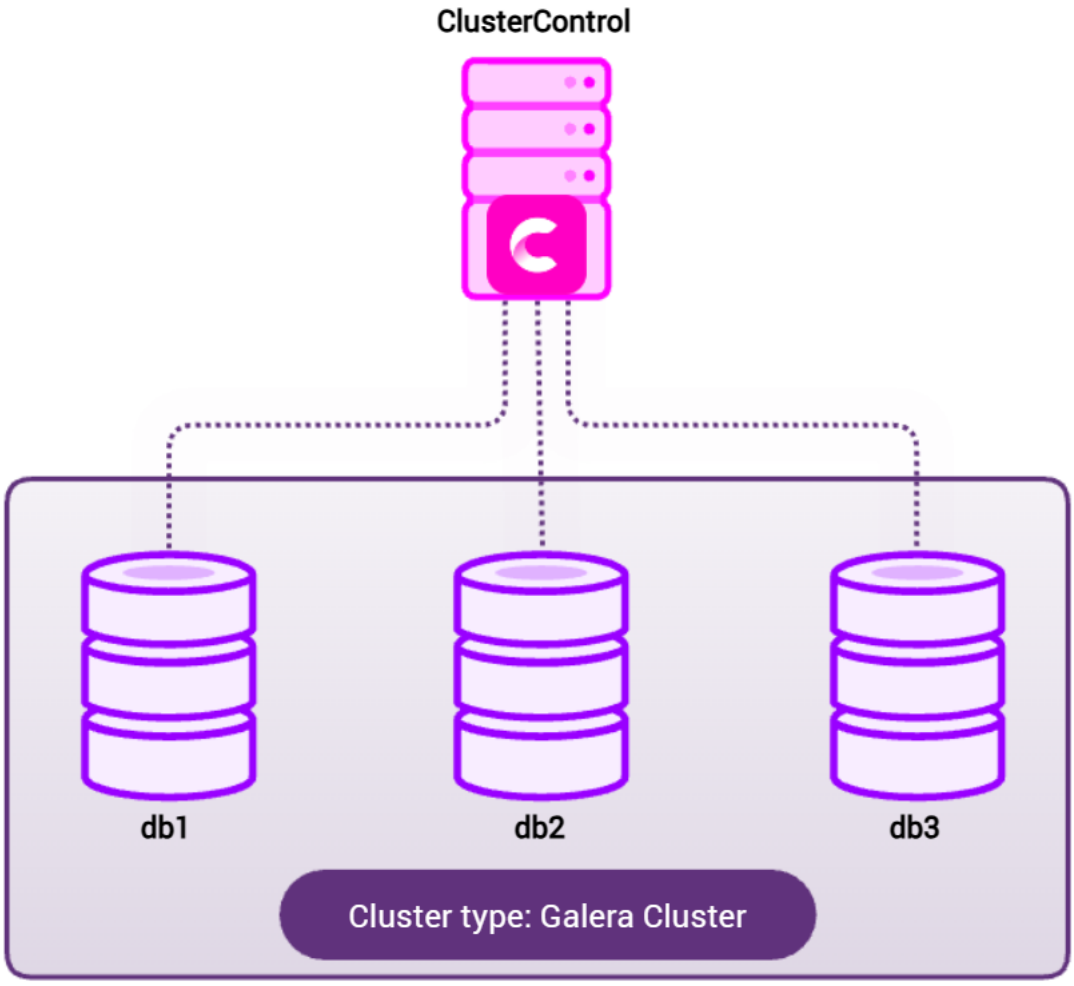
ClusterControl also provides a number of integration components:

Component	Description
Terraform Provider for ClusterControl	Terraform provider that enables interaction with ClusterControl database platform.
cmon_exporter	Prometheus exporter for ClusterControl Controller (cmon) service.
cmon_sd	An HTTP endpoint that exposes monitoring targets for external Prometheus service discovery.

See *Components* for more details on every ClusterControl component.

1.2.2 Architecture

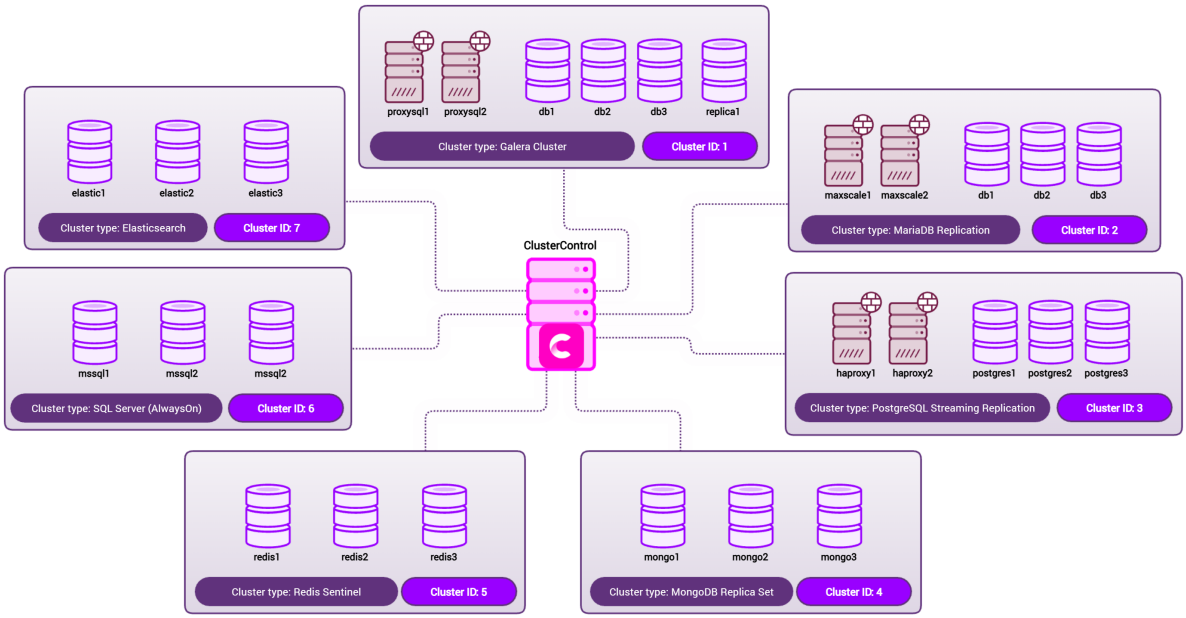
ClusterControl components must reside on a node separate from the database cluster. For instance, a three-node Galera Cluster requires ClusterControl on a fourth, independent node. Following is an example deployment of having a Galera cluster with ClusterControl:



Once the cmon service is initiated, it loads its configuration parameters into the CMON database from `/etc/cmon.cnf` and `/etc/cmon.d/cmon_*.cnf`. Each configuration file corresponds to a unique cluster ID. The service then registers hosts, gathers information, and performs regular checks and scheduled tasks on all managed nodes via SSH. This agentless management relies on passwordless SSH (key-based authentication), utilizing the `ssh_identity` key and `ssh_user` defined in the CMON configuration. For

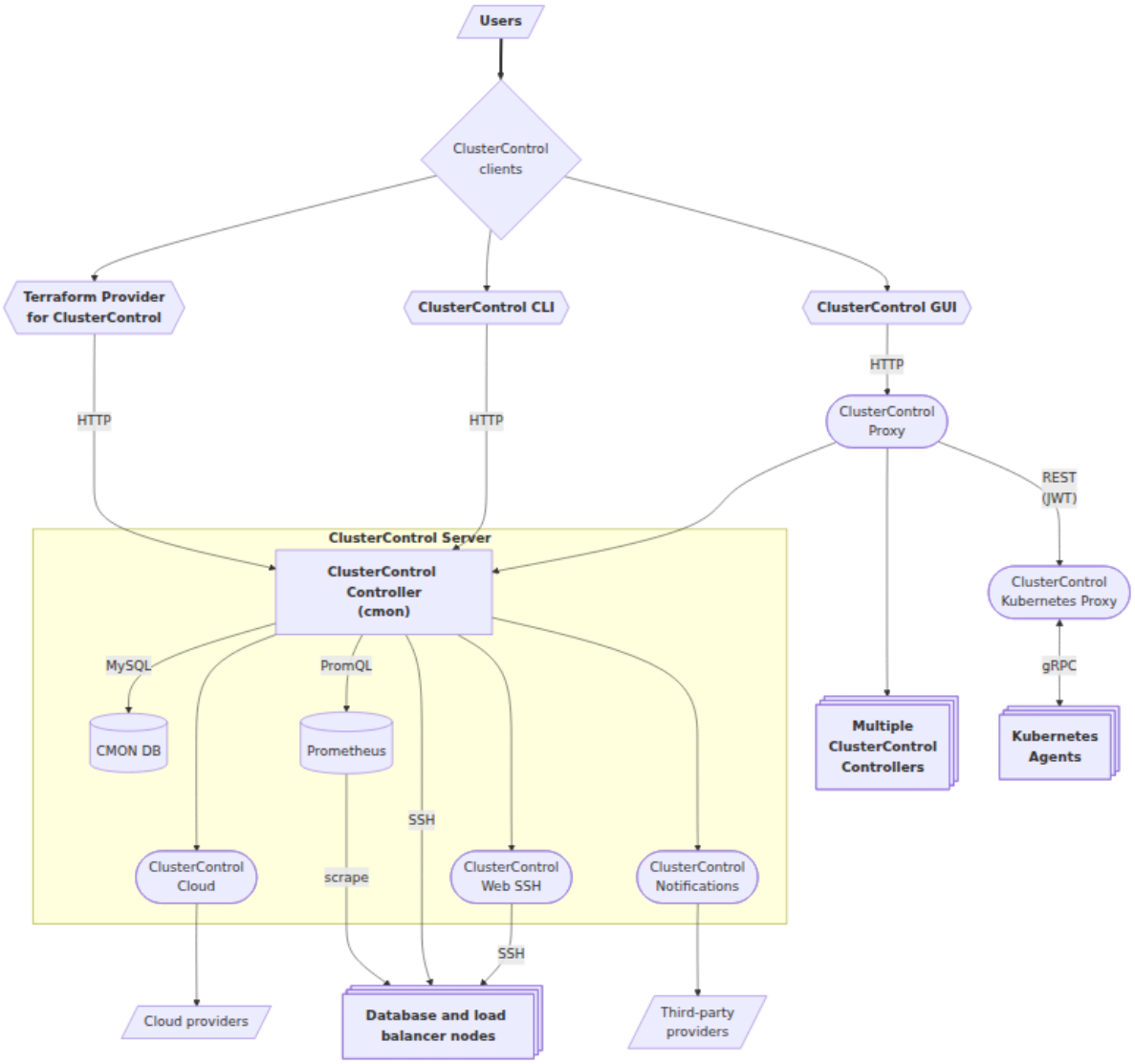
monitoring, ClusterControl employs an agent-based approach using Prometheus and exporters, including `cmonagent` for query monitoring.

Management of the database infrastructure is performed through the ClusterControl GUI, accessible at `https://<ClusterControl_host>`, or via the ClusterControl CLI. Users can begin by importing an existing database cluster or creating a new database server or cluster, either on-premises or in the cloud, provided the target nodes are reachable via SSH. A single ClusterControl instance supports managing multiple clusters of various types, as shown in the following figure:



ClusterControl controller exposes all functionalities through remote procedure calls (RPC) on port 9500 (authenticated by an RPC token), port 9501 (RPC with TLS) and integrates with a number of modules like notifications (9510), cloud (9518) and web SSH (9511). The client components, ClusterControl GUI or ClusterControl CLI interact with those interfaces to retrieve monitoring data (cluster load, host status, alarms, backup status, etc.) or to send management commands (add/remove nodes, run backups, upgrade a cluster, etc.).

The following diagram illustrates the internal architecture of ClusterControl and how the components interact with each other:



ClusterControl has minimal performance impact and will not cause any downtime to your database server or cluster. In fact, it will perform automatic recovery (if enabled) when it finds a failed database node or cluster. See *Failover and Recovery*.

1.2.3 Management and automation operations

For tasks such as management and deployment, ClusterControl executes jobs by sending remote commands via SSH to the designated target node. Users only need to install the ClusterControl controller package on the ClusterControl host, and ensure the SSH key-based authentication and the CMON database user grants are properly set up on each of the managed hosts.

This mechanism requires no agent which simplifies the configuration of the whole infrastructure. Agent-based mode of operation is only supported for monitoring jobs, as described in the next section.

1.2.4 Monitoring operation

For monitoring operation, ClusterControl uses Prometheus and exporters to store time-series monitoring data. For query monitoring, ClusterControl uses either a query agent (ClusterControl Agent, `cmnd`) or SSH pulling, depending on the cluster type.

Other operations like management, scaling, and deployment are performed through an agentless approach

as described in the *Management and Deployment Operations*.

Info

ClusterControl used to support both agentless (SSH sampling) and agent-based monitoring. However, since v1.9.8, ClusterControl defaults to agent-based monitoring using Prometheus. Agentless monitoring is no longer recommended.

Monitoring tools

ClusterControl uses a Prometheus server (default to port 9090) to store time-series monitoring data, and all monitored nodes will be configured with at least three exporters (depending on the node's role):

Node type	Exporter name	Port	Package
All	Process exporter	9011	process-exporter
	Node/system metrics exporter	9100	node_exporter
MySQL/MariaDB	MySQL exporter	9104	mysqld_exporter
MongoDB	MongoDB exporter	9216	mongodb_exporter
PostgreSQL/TimescaleDB	Postgres exporter	9187	postgres_exporter
Redis/Valkey	Redis exporter	9121	redis_exporter
SQL Server	SQL exporter	9999	sql_exporter
Elasticsearch	Elasticsearch exporter	9114	elasticsearch_exporter
ProxySQL	ProxySQL exporter	42004	proxysql_exporter
HAProxy	HAProxy exporter	9101	haproxy_exporter

On every monitored host, ClusterControl will configure and daemonize the exporter process using `systemd`. It is recommended to have an Internet connection when installing the necessary packages and automate the Prometheus deployment. For offline installation, the packages must be pre-downloaded into `/var/cache/cmon/packages` on the ClusterControl node. For the list of required packages and links, please refer to `/usr/share/cmon/templates/packages.conf`.

Apart from the Prometheus scrape process, ClusterControl also connects to the process exporter via HTTP calls to determine the process state of the node, and PromQL to visualize and trend the monitoring data.

Note

Note ClusterControl depends on a working Prometheus for accurate reporting on management and monitoring data. Therefore, Prometheus and exporter processes are managed by the internal process manager thread. A non-operational Prometheus will have a significant impact on the CMON process.

ClusterControl uses a conservative approach to prevent port conflicts when monitoring multiple instances of the same process on a single host (currently limited to PostgreSQL-based clusters). It automatically assigns an incremental port number for each subsequent instance's exporter.

Example

If a host has two PostgreSQL instances (e.g., on ports 5432 and 5437), the first instance's exporter will use the default port (e.g., 9187). The second instance's exporter will be automatically configured to use

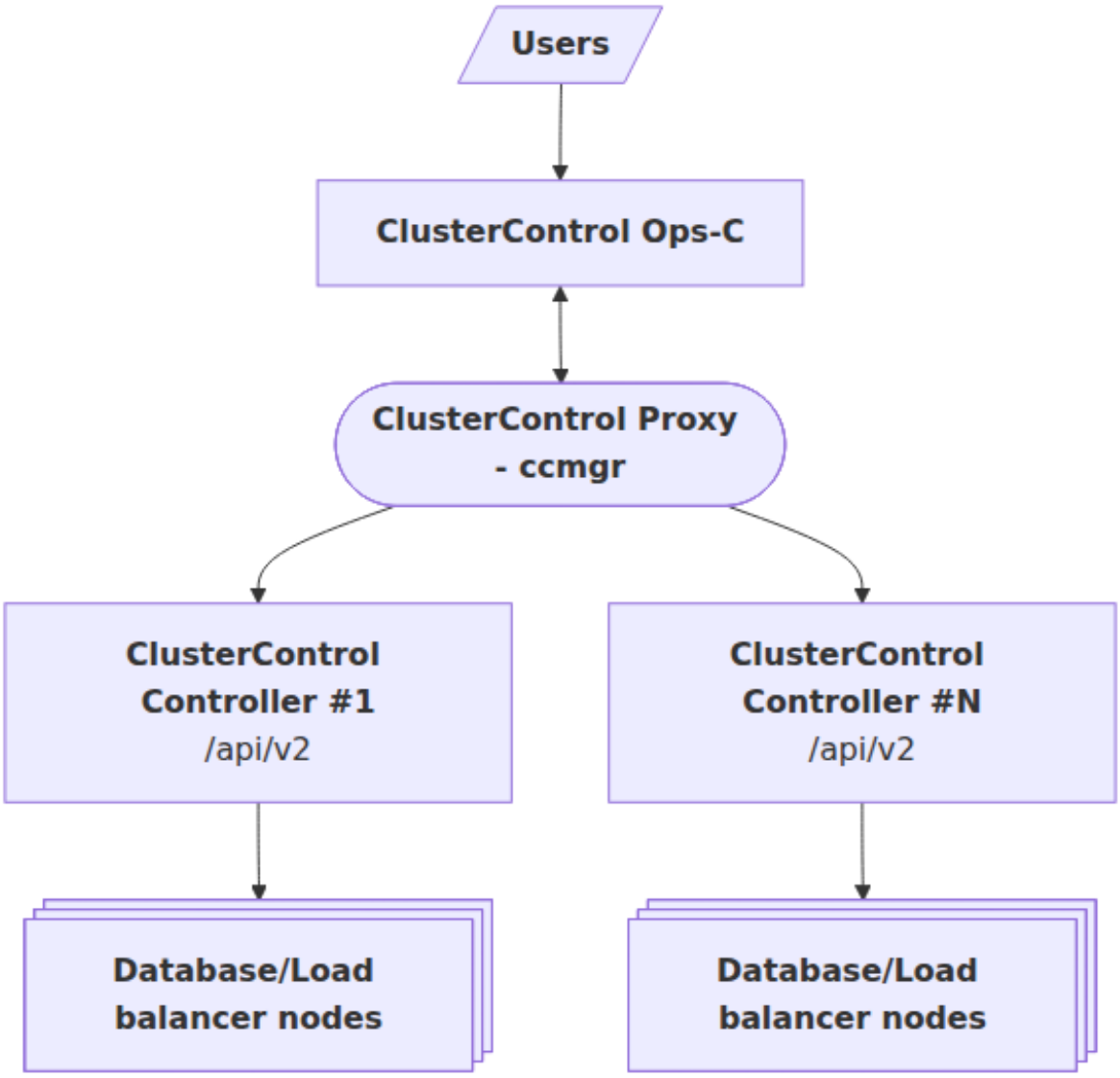
the next available port (e.g., 9188).

1.2.5 Multiple controllers management

For large deployments, it is possible to have multiple ClusterControl instances managing your fleet of database clusters. For example, for 1000+ database nodes, you probably need around 5 to 8 ClusterControl instances (depending on the hardware size) to manage and automate the whole set. You can use ClusterControl Ops-C (shortform of "Operation Center") to have a centralized management system to oversee the whole clusters.

ClusterControl Ops-C utilizing the ClusterControl Proxy service (`ccmgr`) to connect to multiple ClusterControl instances to retrieve status and information about all clusters managed by every ClusterControl controller. It aggregates and visualizes overall view of all clusters in real-time, allowing you to have a centralized view of what is happening without the hassle to check and find the corresponding ClusterControl controller.

The following diagram shows how a single ClusterControl Ops-C installation ommunicates with multiple ClusterControl instances:



By default, the ClusterControl Ops-C GUI is disabled and can be activated under the ***License*** section. When activating, additional configuration is required to create a privileged user to securely integrate and manage multiple ClusterControl controllers. Apart from the standard ClusterControl users authentication method, it also supports authenticating using the existing directory service like LDAP or Active Directory. Simply add a controller by specifying the username and password of ClusterControl user (must belong to admins group) and you are good to go.

See Also

- *User Guide → Integration → ClusterControl Ops-C*
- *User Guide → Overview → ClusterControl GUI*
- *Reference Manuals → ClusterControl Proxy*

1.3 Supported databases

ClusterControl provides comprehensive support for a wide range of open-source and proprietary databases, enabling users to manage, monitor, and automate their database clusters from a unified interface.

1.3.1 Database servers

The following table shows the supported database product with its respective vendor and supported major version:

Database type	Product name	Vendor	Supported major version	Recommended minimum number of nodes
MySQL	MySQL Community Server	Oracle	8.0, 8.4	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 primary/replica) Group replication: 3 nodes (1 primary + 2 replicas)
	Percona Server for MySQL	Percona	8.0, 8.4	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 primary/replica)
	Percona Server for MySQL Pro	Percona	8.0, 8.4	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 primary/replica)
	Percona XtraDB Cluster	Percona	8.0, 8.4	Cluster: 3 nodes (3 primaries)
	Percona XtraDB Cluster Pro	Percona	8.4	Cluster: 3 nodes (3 primaries)
MariaDB	MariaDB Community Server	MariaDB	10.5, 10.6, 10.11, 11.4, 11.8	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 primary/replica) Cluster: 3 nodes (3 primaries)
PostgreSQL	PostgreSQL Server	PostgreSQ	12, 13, 14, 15, 16, 17, 18	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 replica)
	PostgreSQL Server	EDB	12, 13, 14, 15, 16, 17	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 replica)
	TimescaleDB	Times	12, 13, 14, 15, 16, 17, 18	Standalone: 1 node (1 primary) Replication: 2 nodes (1 primary + 1 replica)
MongoDB	MongoDB Community Server	MongoDB	4.4, 5.0, 6.0, 7.0, 8.0	Replica set: 3 nodes (1 primary + 2 replicas) Sharded cluster: 7 nodes (3 config servers + 3 shard servers + 1 mongos)
	MongoDB Enterprise Server	MongoDB	4.4, 5.0, 6.0, 7.0, 8.0	Replica set: 3 nodes (1 primary + 2 replicas) Sharded cluster: 7 nodes (3 config servers + 3 shard servers + 1 mongos)
	Percona Server for MongoDB	Percona	4.4, 5.0, 6.0, 7.0, 8.0	Replica set: 3 nodes (1 primary + 2 replicas) Sharded cluster: 7 nodes (3 config servers + 3 shard servers + 1 mongos)
Redis	Redis Community Edition	Redis	6, 7, 8	Sentinel: 3 nodes (1 primary with Sentinel + 2 replicas with Sentinel) Cluster: 6 nodes (3 primaries + 3 replicas)
	Valkey	LF Projec	7, 8	Sentinel: 3 nodes (1 primary with Sentinel + 2 replicas with Sentinel) Cluster: 6 nodes (3 primaries + 3 replicas)
	Valkey	Canonical	7	Sentinel: 3 nodes (1 primary with Sentinel + 2 replicas with Sentinel) Cluster: 6 nodes (3 primaries + 3 replicas)
SQL Server	Microsoft SQL Server for Linux	Microsof	2019 (15.x), 2022 (16.x)	Standalone: 1 node (1 primary) Always On availability group: 2 nodes (1 primary + 1 replica)
Elastic-search	Elasticsearch	Elastic	7.17, 8.15, 8.3	Standalone cluster: 1 node (1 eligible-master/data node) High availability cluster: 5 nodes (3 eligible-master nodes + 2 data nodes)

Note

Note ClusterControl only supports managing x86-64 architecture hosts (x86_64, x64, AMD64, Intel64).

Warning

Attention

Some database products are only supported on selected operating systems. See *Operating System*.

1.3.2 Auxiliary tools

Apart from the database software, ClusterControl also supports the following open-source auxiliary tools:

Type	Product name	Repository
Load balancer	ProxySQL	Percona
	MariaDB MaxScale	MariaDB
	HAProxy	EPEL/OS
	Keepalived	EPEL/OS
	garbd	Percona/MariaDB
Connection pooler	PgBouncer	PGDG
Backup	pgBackRest	PGDG
	Percona Backup for MongoDB	Percona
Monitoring	Prometheus	Prometheus

1.4 Key Features

Explore ClusterControl's key features below to understand how it can support your database management needs from end to end.

1.4.1 All cluster types

The following are features that are available across all cluster types supported by ClusterControl:

Security and control

- LDAP/Active Directory authentication. Enterprise
- Key and certificate management. Enterprise Advanced
- Role-based access control.
- Integration with Hashicorp Vault. Enterprise
- Multi-controller management. Enterprise

Monitoring and Alerting

- Email notification with fine-grained delivery configuration based on severity. Enterprise Advanced
- Integration with messaging tools like Slack, Telegram and webhook. Enterprise Advanced
- Integration with incident management platform like Pagerduty, Splunk On-Call (VictorOps), ServiceNow, OpsGenie and ilert. Enterprise Advanced
- Jobs monitoring.
- Automatic alarms for various critical and warning events.
- Watchlist - Monitoring dashboards in full-screen view (kiosk-mode).

Cloud storage

- Amazon S3. Enterprise Advanced
- Google Cloud Storage. Enterprise Advanced
- Microsoft Azure Blob Storage. Enterprise Advanced
- Any S3-compatible storage providers. Enterprise Advanced

Node management

- Schedule maintenance mode and tracking.
- Web-based SSH console.
- Reboot host. Enterprise Advanced
- Stop/Start/Restart services. Enterprise Advanced

Audit and reporting

- Operational reports (system report, database availability report, backup report, upgrade report, database growth report, schema change report).
- ClusterControl audit log and tracking.
- Activity center.
- Error reports.
- Centralized database and load balancer logs.

1.4.2 Cluster-specific features

MySQL/MariaDB/Percona

- Deployment automation:
  - Deploy MySQL/MariaDB standalone, Galera Cluster, replication (primary-replica and primary-primary).
  - Deploy MySQL group replication.
  - Support for various MySQL flavors (Percona Server for MySQL, Percona Server for MySQL Pro, Oracle MySQL Community Server, MariaDB Community Server).
  - Import existing MySQL/MariaDB standalone, replication or Galera Cluster.

    - Clone cluster (Galera Cluster). Enterprise
    - Cluster-to-Cluster replication (Galera Cluster). Enterprise
    - Deploy MySQL replication on Kubernetes (Technical preview).
- Backup management: Enterprise Advanced
    - On-demand or scheduled logical or physical backups (partial, full and incremental).
    - Support for mysqldump, Percona Xtrabackup, Mariabackup.
    - Point-in-Time Recovery (PITR).
    - Backup to local disk and remote storage (S3-compatible object storage, NFS).
    - Backup verification.
    - Backup binary logs.
    - Restore from local backup or cloud.
    - Restore external backup.
- Failover and recovery: Enterprise Advanced
    - Automated failover for primary-replica and Galera clusters.
    - Node auto-recovery and healing.
    - Promote replica to primary.
    - Rebuild replica from primary.
    - Change replication source.
    - Read-only and read-write management.
- Scaling: Enterprise Advanced
    - Scale-out by adding new database nodes (replicas, delayed replicas or Galera nodes).
    - Scale-out load balancer nodes.
    - Scaling support for MySQL Replication and Galera.
- Monitoring and alerting:
    - Topology visualizer.
    - Host monitoring.
    - Database stats monitoring.
    - Load balancer stats monitoring.
    - Track replication lag, Galera state, disk space, queries per second.
    - Customizable threshold and alerting.
    - Database and load balancer logs.
- Load balancing: Enterprise Advanced
    - Integrates with HAProxy, ProxySQL and MariaDB MaxScale for automated load balancing.
    - ProxySQL manager (query rules, users, query cache, routing)

    - Integrates with Keepalived for virtual IP address.
    - Integrates with garbd (Galera arbitrator daemon).
- Security: Enterprise
    - SSL encryption for client-server communication.
    - SSL encryption between server-server communication (Galera Cluster and replication).
    - User management, audit logging, and access control.
    - Backup encryption.
- Performance management:
    - Query monitoring, slow query detection, schema analyzer, query optimizer.
    - Database growth.
    - Advisors for tuning performance and query improvement.
    - Database and load balancer configuration management. Enterprise Advanced
    - Database user management. Enterprise Advanced
- Database upgrades and patching: Enterprise Advanced
    - Automated rolling upgrades for minor versions.
    - Schema upgrade and migration management.

PostgreSQL/TimescaleDB/EDB

- Deployment automation:
    - Deploy PostgreSQL standalone or streaming replication (primary-replica).
    - Deploy PostgreSQL logical replication with possibility of bi-directional replication.
    - Support for various PostgreSQL flavors (PostgreSQL, TimescaleDB, EDB Postgres Server).
    - Import existing PostgreSQL standalone and streaming replication.
    - Cluster-to-Cluster replication.Enterprise
    - Deploy PostgreSQL streaming replication on Kubernetes (Technical preview).
- Backup management: Enterprise Advanced
    - On-demand and scheduled logical or physical backups (partial, full and incremental).
    - Support for pgdump, pg_basebackup, PgBackRest.
    - Point-in-Time Recovery (PITR).
    - Backup to local disk and remote storage (S3-compatible object storage, NFS).
    - Backup verification.
    - Restore from local backup or cloud.
    - WAL archiving.
    - Dedicated backup repository host.
- Failover and recovery: Enterprise Advanced

    - Automated failover for primary-replica streaming replication clusters.
    - Node automatic recovery and healing.
    - Promote replica to primary.
    - Rebuild replica from primary.
    - Change replication source.
- Scaling: Enterprise Advanced
    - Scale-out by adding new database nodes (replicas).
    - Scale-out load balancer nodes.
    - Scaling support for streaming replication.
- Monitoring and alerting:
    - Topology visualizer.
    - Host monitoring.
    - Database stats monitoring.
    - Load balancer stats monitoring.
    - Track replication lag, database state, disk space, queries per second.
    - Customizable threshold and alerting.
    - Database and load balancer logs.
- Load balancing: Enterprise Advanced
    - Integrates with HAProxy for automated load balancing.
    - Integrates with Keepalived for virtual IP address.
    - pgBouncer pool management.
- Security: Enterprise
    - SSL encryption for client-server communication.
    - SSL encryption between server-server communication (replication).
    - User management, audit logging, and access control.
    - Backup encryption.
- Performance management: Enterprise Advanced
    - Query monitoring, slow query detection, query optimizer.
    - Database growth.
    - Advisors for tuning performance and query improvement.
    - Database and load balancer configuration management.
    - Database user management.
- Database upgrades and patching: Enterprise Advanced
    - Automated rolling upgrades for major and minor versions.

- Plugins/extensions management:
  - TimescaleDB
  - pgvector

MongoDB

- Deployment automation:
  - Deploy a new MongoDB standalone, replica set or sharded cluster (single or multiple shards).
  - Support for various MongoDB flavors (MongoDB Community, MongoDB Enterprise, Percona Server for MongoDB).
  - Import existing MongoDB Replica Set or Sharded Cluster.
- Backup management: Enterprise Advanced
  - On-demand and scheduled logical backups.
  - Support for mongodump and Percona Backup for MongoDB.
  - Backup to local disk and remote storage (S3-compatible object storage, NFS).
  - Backup verification.
  - Restore from local backup or cloud.
- Failover and recovery: Enterprise Advanced
  - Automated failover for primary-replica streaming replication clusters.
  - Node auto-recovery and healing.
  - Step down primary for auto election.
  - Freeze node.
- Scaling: Enterprise Advanced
  - Scale-out by adding new mongod or arbiter (standalone or replica set)
  - Scale-out by adding new mongodb, config or mongos nodes (sharded cluster).
  - Scale-out by adding new replica set shards (sharded cluster)
- Monitoring and alerting:
  - Topology visualizer.
  - Host monitoring.
  - Database stats monitoring.
  - Mongos router monitoring.
  - Track replication lag, database state, disk space, queries per second.
  - Customizable threshold and alerting.
- Security: Enterprise Advanced
  - SSL encryption for client-server communication.
  - SSL encryption between server-server communication (replication).

    - User management, audit logging, and access control.
    - Backup encryption.
- Performance management:
    - Ops monitoring.
    - Database growth.
    - Advisors for tuning performance and query improvement.
    - Database configuration management. Enterprise Advanced
- Database upgrades and patching: Enterprise Advanced
    - Automated rolling upgrades for minor versions.

Redis/Valkey

- Deployment automation:
    - Deploy a new Redis/Valkey Sentinel or cluster.
    - Support for various Redis flavors (Redis Community and Valkey).
    - Import existing Redis/Valkey Sentinel or cluster.
- Backup management: Enterprise Advanced
    - On-demand and scheduled RDB (Redis Database) and AOF (Append Only File) backups.
    - Backup to local disk and remote storage (S3-compatible object storage, NFS).
    - Restore from local backup or cloud.
- Failover and recovery: Enterprise Advanced
    - Automated failover for primary-replica replication clusters (Sentinel).
    - Node auto-recovery and healing.
    - Promote replica to primary.
    - Change replication source.
- Scaling: Enterprise Advanced
    - Scale-out by adding new Redis node (sentinel or cluster mode)
- Monitoring and alerting:
    - Topology visualizer.
    - Host monitoring.
    - Database stats monitoring.
    - Track database state, disk space, queries per second.
    - Customizable threshold and alerting.
- Security: Enterprise Advanced
    - SSL encryption for client-server communication.
    - SSL encryption between server-server communication.

    - User management, audit logging, and access control.
    - Backup encryption.
- Performance management:
    - Database growth.
    - Advisors for tuning performance and query improvement.
    - Database configuration management. Enterprise Advanced
    - Database user management. Enterprise Advanced
- Database upgrades and patching: Enterprise Advanced
    - Automated rolling upgrades for minor versions.

SQL Server

- Deployment automation:
    - Deploy a new SQL Server standalone or Always On Availability Group cluster.
    - Import existing SQL Server standalone or Always On Available Group cluster.
- Backup management: Enterprise Advanced
    - On-demand and scheduled physical backups.
    - Support for full, differential, and transaction log backups.
    - Backup to local disk and remote storage (S3-compatible object storage, NFS).
    - Restore from local backup or cloud.
    - Point-in-time recovery.
- Failover and recovery: Enterprise Advanced
    - Automated failover for primary-replica AlwaysOn availability group.
    - Node auto-recovery and healing.
    - Promote replica to primary.
    - Rebuild replica.
- Scaling: Enterprise Advanced
    - Scale-out by adding new SQL Server node.
- Monitoring and alerting:
    - Topology visualizer.
    - Host monitoring.
    - Database stats monitoring.
    - Track database state, disk space, queries per second.
    - Customizable threshold and alerting.
- Security: Enterprise Advanced
    - SSL encryption for client-server communication.

    - SSL encryption between server-server communication.
    - User management, audit logging, and access control.
- Performance management:
    - Database growth.
    - Advisors for tuning performance and query improvement.
    - Database configuration management. Enterprise Advanced

Elasticsearch

- Deployment automation:
    - Deploy a new Elasticsearch single-node cluster or high-availability cluster.
    - Import existing Elasticsearch single-node cluster or high-availability cluster.
- Backup management: Enterprise Advanced
    - On-demand and scheduled Elastic snapshot backup.
    - Backup to local disk and remote storage (S3-compatible object storage, NFS).
    - Restore from local backup or cloud.
- Failover and recovery: Enterprise Advanced
    - Automated failover for high availability setup.
    - Node auto-recovery and healing.
- Scaling: Enterprise Advanced
    - Scale-out by adding new data node.
- Monitoring and alerting:
    - Topology visualizer.
    - Host monitoring.
    - Database stats monitoring.
    - Track database state, disk space, queries per second.
    - Customizable threshold and alerting.
- Security: Enterprise Advanced
    - SSL encryption for client-server communication.
    - SSL encryption between server-server communication.
    - User management, audit logging, and access control.
- Performance management:
    - Database growth.
    - Advisors for tuning performance and query improvement.
    - Database configuration management. Enterprise Advanced

See also

- Feature comparison: ClusterControl product page

1.5 Requirements

Before setting up ClusterControl, it's essential to ensure that your environment meets the necessary requirements for a smooth installation and optimal performance. ClusterControl is designed to work across a variety of infrastructure setups, from on-premise servers to cloud environments, and supports multiple database technologies.

This page outlines the hardware, software, network, and database-specific requirements needed to successfully deploy and run ClusterControl.

1.5.1 Hardware

The following table shows the recommended hardware specifications for a ClusterControl host:

Aspect	Minimum	Small (3-15 nodes)	Medium (15-50 nodes)	Large (50-150 nodes)
CPU architecture	x86_64 only			
CPU	2 cores	4 cores	16 cores	24 cores
RAM	2 GB	8 GB	16 GB	32 GB
Disk space	20 GB	50 GB	150 GB	300 GB

Tip

For very large deployments, you probably need more than one ClusterControl instances to manage the whole fleet. Use ClusterControl Operational Center (Ops-C) to manage these ClusterControl instances. See *Integration → ClusterControl Ops-C*.

Capacity planning

In regards to capacity planning, to simply put into perspective, we have seen real-world use-case where a single ClusterControl server in production with 24 CPUs, 32GB RAM, 150 GB disk space (currently 92% usage) to manage 67 clusters with around 130 nodes (database + load balancers).

We have also covered optimizing the performance of ClusterControl in this blog post, How to Optimize Performance of ClusterControl and Its Components.

CPU

The CPU capacity of the ClusterControl server plays a crucial role in handling various tasks such as monitoring, managing, and orchestrating database clusters. To accommodate a significant workload and ensure scalability for future growth, we recommend a minimum of 8 to 24 CPU cores with multi-threading capabilities.

RAM

Memory availability directly impacts the performance and responsiveness of ClusterControl, especially when handling a large number of database clusters and nodes. For optimal operation, we recommend 8 to 32 GB of RAM. However, considering the increasing complexity of database environments and potential future expansions, provisioning additional memory is advisable.

During the installation stage, the installer script tunes the necessary parameters for MySQL and Apache according to the amount of RAM in the host. If you have increased the RAM after the installation, it is recommended to revisit the tuning by looking at this blog post, How to Optimize Performance of ClusterControl and Its Components.

Disk space

Disk space requirements primarily depend on the size of the monitoring data, logs, backups, and other associated files generated by ClusterControl. Given the example use-case disk utilization of 92% for 150GB, it is recommended to allocate at least 300GB of disk space for the ClusterControl server. This ensures sufficient headroom for data growth and temporary storage needs.

If you use ClusterControl as the centralized backup repository, consider adding more disk space, adding another disk or mounting a networked file system like NFS, SMB or iSCSI to cater to this.

If you use Prometheus and agent-based monitoring, you may also use a dedicated Prometheus server (by default Prometheus will be installed on the ClusterControl server) to scale up the monitoring resources. Prometheus defaults to 15 days retention period and if you require longer retention period, adjust the disk allocation for `/var/lib/prometheus` partition accordingly.

Network bandwidth

ClusterControl relies on network communication for monitoring, provisioning, and managing database clusters. Therefore, it's essential to ensure adequate network bandwidth to support the expected workload. A Gigabit Ethernet connection or higher is recommended for optimal performance. For inter-WAN communication, consider placing the ClusterControl server at the nearest of the most critical segment (production/primary).

Additional considerations

High availability

For mission-critical environments, implementing high availability (HA) for the ClusterControl server is highly recommended. This involves deploying redundant ClusterControl nodes in an active-passive or active-active configuration to ensure continuous operation and fault tolerance. See Redundancy & High Availability.

Storage performance

While sufficient disk space is essential, equally important is the performance of the storage subsystem. Utilizing fast and reliable storage technologies such as solid-state drives (SSDs) or high-performance RAID arrays can significantly improve the responsiveness of ClusterControl, especially during data-intensive operations like backups, restores, scaling and reporting.

Monitoring and maintenance

Regular monitoring of hardware resources, including CPU, RAM, disk space, and network utilization, is essential to identify potential bottlenecks or capacity issues proactively. Additionally, periodic maintenance tasks such as disk cleanup, log rotation, and performance tuning should be scheduled to optimize the ClusterControl server's performance and stability.

1.5.2 Operating system

ClusterControl

ClusterControl has been tested on the following operating systems:

- Red Hat Enterprise Linux 8.x/9.x
- Rocky Linux 8.x/9.x
- AlmaLinux 8.x/9.x
- Ubuntu 22.04/24.04 LTS
- Debian 10.x/11.x/12.x

Monitored nodes

For the monitored nodes, the following operating systems are supported:

- Red Hat Enterprise Linux 8.x/9.x
- Rocky Linux 8.x/9.x
- AlmaLinux 8.x/9.x
- Ubuntu 20.04/22.04/24.04 LTS
- Debian 10.x/11.x/12.x

Should the cluster deployment or import job fail due to unsupported operating system, an informational message will display the operating systems supported by ClusterControl for the selected database technology.

Warning

Attention

- Mixing Red Hat-based and Debian-based nodes in a database cluster is not supported.

1.5.3 Software dependencies

The following software and packages are required by ClusterControl:

- MySQL client/server (5.7 or later) or MariaDB client/server (10.0 or later)
- OpenSSH server/client
- NTP server – All servers' time must be synced under one time zone
- `socat` or `netcat` – for streaming backups

Note

Note If ClusterControl is installed via installation script (install-cc) or package manager (yum/dnf/apt), all dependencies will be automatically satisfied.

1.5.4 Firewall and security groups

It is important to secure the ClusterControl host and the database cluster. It is recommended for users to isolate the database infrastructure from the public Internet and just whitelist the known hosts or networks to reach the database cluster.

ClusterControl requires ports used by the following services to be accessible or enabled:

- ICMP (echo reply/request).
- SSH (default is 22).
- HTTPS (default is 443).
- MySQL (default is 3306).
- CMON RPC (default is 9500).
- CMON RPC TLS (default is 9501).
- CMON Events (default is 9510).
- CMON SSH (default is 9511).
- CMON Cloud (default is 9518).
- Streaming port for backups through socat/netcat (default is 9999, configurable under ***Backup Settings***).
- CMON Proxy (default is 19501) - Only for ClusterControl Ops-C (multi-controller).

ClusterControl supports various database and application vendors and each has its own set of standard ports that need to be reachable. Following ports and services need to be reachable from ClusterControl node to the managed nodes (unless the target services are deliberately configured on non-default ports):

Node type	Default port (Service)
All managed nodes	- 22 (SSH) - ICMP (echo reply/request) - 9100 (Node exporter) - 9011 (Process exporter)
MySQL/MariaDB (standalone and replication)	- 3306 (MySQL) - 9104 (MySQL exporter) - 4433 (query monitor)
Galera Cluster (MariaDB Galera Cluster/Percona XtraDB Cluster)	- 3306 (MySQL) - 4444 (SST) - 4567 TCP/UDP (Galera) - 4568 (Galera IST) - 9200 (HAProxy health check) - 9104 (MySQL exporter) - 4433 (query monitor)
MongoDB replica set	- 27017 (mongod) - 9126 (MongoDB exporter)
MongoDB sharded cluster	- 27018 (mongod) - 27017 (mongos) - 27019 (config server) - 9216 (MongoDB exporter)
PostgreSQL/TimescaleDB	- 5432 (PostgreSQL) - 9200 (HAProxy health check) - 9187 (PostgreSQL exporter) - 4433 (query monitor)
HAProxy	- 9600 (HAProxy stats) - 3307 (MySQL load-balanced) - 3308 (MySQL load-balanced read-only) - 5433 (PostgreSQL load-balanced) - 5434 (PostgreSQL load-balanced read-only) - 9101 (HAProxy exporter)
MariaDB MaxScale	- 6603 (MaxCtrl - CLI) - 4006 (Round robin listener) - 4008 (Read/Write split listener) - 4442 (Debug information)
Keepalived	- IP protocol 112 (VRRP)
Galera Arbitrator (garbd)	- 4567 (Galera)
ProxySQL	- 6032 (ProxySQL Admin) - 6033 (MySQL load-balanced) - 42004 (ProxySQL exporter)
Prometheus	- 9090 (Prometheus)
Microsoft SQL Server	- 1433 (SQL Server) - 9999 (SQL Server exporter)
Redis/Valkey (Sentinel)	- 6379 (Redis) - 26379 (Sentinel) - 9121 (Redis exporter)
Redis/Valkey (Cluster)	- 6379 (Redis) - 16379 (Cluster bus) - 9121 (Redis exporter)
Elasticsearch	- 9200 (Elastic HTTP/Transfer) - 9114 (Elasticsearch exporter)

1.5.5 Operating system user

ClusterControl controller (cmon) process requires a dedicated operating system user to perform various management and monitoring commands on the managed nodes. The value of `os_user` or `sshuser` in CMON configuration file, must exist on all managed nodes and it should have the ability to perform superuser commands.

You are recommended to install ClusterControl as 'root', and running as root is the easiest option. If you perform the installation using another user other than 'root', the following must be true:

- The OS user must exist on all nodes
- The OS user must not be `mysql`
- The `sudo` or `pbrun` or `doas` program must be installed on all hosts
- The OS user must be allowed to do sudo, i.e, it must be in sudoers
- The OS user must be configured with the proper `PATH` environment variable. The following environment `PATH` is expected for user `myuser`:

```
PATH=/
↪usr/local/bin:/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/myuser/.local/bin:/hom
```

Warning

Attention

ClusterControl requires full access of sudo (all commands) for full functionality. Restricting the commands would cause some of the operations to fail (cluster recovery, failover, backup restoration, service control and cluster deployment).

For sudoers, using passwordless sudo is recommended (privilege escalation without password). To set up a passwordless sudo user, open `/etc/sudoers` via text editor and add the following line at the end. Replace `<os_user>` with the sudo username of your choice:

```
  <os_user> ALL=(ALL) NOPASSWD: ALL
```

Open a new terminal to verify if it works. You should now be able to run the following command without entering a password:

```
sudo ls /usr
```

Note

Note If using passwordless sudo is not an option, do not forget to specify the sudo password when deploying a new or importing an existing database cluster.

You can also verify this with the SSH command line used by CMON (assuming SSH key-based authentication has been set up correctly):

```
ssh -qt <os_user>@<ip_address_or_hostname> "sudo ls /usr"
```

Where `<os_user>` is the name of the user you intend to use by ClusterControl for management purposes, and `<ip_address_or_hostname>` is the IP address or hostname of a node in your cluster.

For pbrun, a commercial access management solution that provides controlled elevation of privileges in a Unix/Linux environment, implementing strong passwords for users is a best practice. The installation of pbrun as follow:

```
# Extract installation packages
  tar -xzvf powerbroker-x.x.x.tar.gz

# Run Installer
  cd powerbroker-x.x.x

# Verify installation
  pbrun -v

# Initialize configuration
  pbrun --init
```

Roles and users are defined in `/etc/pb.conf`. This involves configuring `pbrun` and registering `<os_user>` with `pbrun`.

```
# Add user to PowerBroker
  pb_useradd os_user

# Assign roles
  pb_roleadd os_user role_name
```

Verify if the `<os_user>` can run some root command using pbrun:

```
ssh user@remote-host "pbrun /usr/bin/systemctl restart nginx"
```

Where `<os_user>` is the name of the user you intend to use by ClusterControl for management purposes, and `<ip_address_or_hostname>` is the IP address or hostname of a node in your cluster.

For doas, a lightweight command-line tool, similar to `sudo`, for running commands as another user (typically `root`). It is simpler, more secure, uses a minimal `/etc/doas.conf` file, and is common on OpenBSD and minimalist Linux. The installation of `doas` as follow:

Red Hat/CentOS

Installation of `doas` command line utility in Redhat based:

1. Install the package from repository.

   ```
   dnf install opendoas
   ```

2. Create config file.

   ```
   touch /etc/doas.conf
   chmod 600 /etc/doas.conf
   ```

3. Add `<os_user>` in the `doas.conf` config file.

   ```
   permit os_user
   ```

Debian/Ubuntu

Installation of `doas` command line utility in Debian based:

1. Install the package from repository.

   ```
   apt update
   apt install doas
   ```

   If the package not available, try:

   ```
   apt install opendoas
   ```

2. Create config file.

   ```
   touch /etc/doas.conf
   chmod 600 /etc/doas.conf
   ```

3. Add <os_user> in the doas.conf config file.

```
permit os_user
```

Verify if the <os_user> can execute a root command:

```
ssh os_user@remote-host "doas systemctl restart nginx"
```

Where <os_user> is the name of the user you intend to use by ClusterControl for management purposes, and <ip_address_or_hostname> is the IP address or hostname of a node in your cluster.

1.5.6 SSH key-based authentication

Proper SSH key-based authentication setup from ClusterControl node to all managed nodes is mandatory. Before performing any operation on the managed node, the node must be accessible via SSH without using a password by using the key-based authentication instead.

ClusterControl uses libssh (a multiplatform C library using SSHv2 protocol) which supports the following public-key algorithms:

- ssh-rsa
- rsa-sha2-512
- rsa-sha2-256
- ssh-dss
- ssh-ed25519
- ecdsa-sha2-nistp256
- ecdsa-sha2-nistp384
- ecdsa-sha2-nistp521

Note

Note Take note that ClusterControl is fully tested with the RSA public key. Other supported key types should work in most cases.

Setting up key-based authentication

To set up an SSH key-based authentication, make sure you generate SSH keys (private and public keys) and copy the public key from the ClusterControl host as the designated user to the target host.

Tip

It is not necessary to setup two-way key-based authentication SSH, e.g: from the managed database node to the ClusterControl node.

To generate an SSH key, use the ssh-keygen command which is available with the OpenSSH-client package. On ClusterControl node:

```
$ whoami
root
$ ssh-keygen -t rsa      # press Enter on all prompts
```

The above command will generate SSH RSA private and public keys under the user's home directory, `/root/.ssh/`. The private key, `id_rsa` has to be kept secure on the node. The public key, `id_rsa.pub` should be copied over to all nodes that want to be accessed by ClusterControl passwordlessly.

The next step is to copy the SSH public key to all nodes. You may use the `ssh-copy-id` command to achieve this if the destination node supports password authentication:

```
ssh-copy-id <os_user>@<target_node>
```

Example

```
$ whoami
root
$ ls -1 ~/.ssh/id*
/root/.ssh/id_rsa
/root/.ssh/id_rsa.pub
$ ssh-copy-id root@192.168.0.10     # specify the root password of␣
↪192.168.0.10 if prompted
```

The command `ssh-copy-id` will simply copy the public key from the source server and add it into the destination server's authorized key list, default to `~/.ssh/autohorized_keys` of the authenticated SSH user. If password authentication is disabled, and the target node requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.0.10
```

Alternatively, you can manually copy the public key to the target nodes. On ClusterControl node, copy the content of SSH public key located at `~/.ssh/id_rsa.pub` and paste it into `~/.ssh/authorized_keys` of all target nodes.

Example

The following example shows how a root user on the ClusterControl host (192.168.0.10) generates and copies an SSH key to databases hosts (192.168.0.11, 192.168.0.12, 192.168.0.13):

```
$ whoami
root
$ ssh-keygen -t rsa            # press Enter on all prompts
$ ls -1 ~/.ssh/id*
/root/.ssh/id_rsa
/root/.ssh/id_rsa.pub
$ ssh-copy-id 192.168.0.11     # specify the root password of 192.168.0.11␣
↪if prompted
$ ssh-copy-id 192.168.0.12     # specify the root password of 192.168.0.12␣
↪if prompted
```

```
$ ssh-copy-id 192.168.0.13    # specify the root password of 192.168.0.13␣
↪if prompted
```

If you are running as a sudo user e.g "sysadmin", here is an example:

```
$ whoami
sysadmin
$ ssh-keygen -t rsa           # press Enter on all prompts
$ ls -1 ~/.ssh/id*
/home/sysadmin/.ssh/id_rsa
/home/sysadmin/.ssh/id_rsa.pub
$ ssh-copy-id 192.168.0.11    # specify the sysadmin password of␣
↪192.168.0.11 if prompted
$ ssh-copy-id 192.168.0.12    # specify the sysadmin password of␣
↪192.168.0.12 if prompted
$ ssh-copy-id 192.168.0.13    # specify the sysadmin password of␣
↪192.168.0.13 if prompted
```

You should be able to SSH from ClusterControl to the other server(s) without a password:

```
ssh sysadmin@192.168.0.11
ssh sysadmin@192.168.0.12
ssh sysadmin@192.168.0.13
```

For cloud users, you can use the corresponding key pair generated by the cloud provider by uploading it onto the ClusterControl host and specify the physical path when configuring the SSH-related parameters in the ClusterControl UI (deploy a cluster, import nodes, etc). ClusterControl will then use this key to perform tasks that require SSH key-based authentication and store the path via `ssh_identity` variable inside CMON configuration file:

```
ssh_identity=/path/to/keypair/cloud.pem
```

If you use other public-key algorithms (ClusterControl defaults to RSA), make sure the public key generated on the ClusterControl node is copied and allowed on all managed nodes under `~/.ssh/autohorized_keys`. You can use `ssh-copy-id` command (as shown in the examples above), or simply copy the public key to all managed nodes manually.

Sudo password

Sudo with or without a password is possible. If undefined, ClusterControl will escalate to sudoer without a password. When deploying a new or importing an existing cluster into ClusterControl, the user will be asked to specify the sudo password in the deployment dialog. The specified sudo password is then stored inside the CMON configuration file under `sudo` variable:

```
sudo="echo 'thesudopassword' | sudo -S 2>/dev/null"
```

Warning

Attention

Having `2>/dev/null` in the sudo command is compulsory to strip out stderr from the response.

1.5.7 Timezone

ClusterControl requires all servers' time to be synchronized and to run within the same time zone. Verify this by using the following command:

```
$ date
Mon Sep 17 22:59:24 UTC 2013
```

To change time zone, e.g from UTC to Pacific time:

```
rm /etc/localtime
ln -sf /usr/share/zoneinfo/US/Pacific localtime
```

UTC is however recommended. Configure NTP client for each host with a working time server to avoid time drifting between hosts which could cause inaccurate reporting or incorrect graphs plotting. To immediately sync a server's time with a time server, use the following command:

```
ntpdate -u time.apple.com     # or any other NTP server close to the server
```

Frequently asked questions related to ClusterControl and Severalnines.

1.6 General

1.6.1 Is ClusterControl free?

ClusterControl has a Community Edition that is free forever, available features include deployment and monitoring of clusters/servers. See the ClusterControl product page or *Key Features*.

After installation, you will get a 30-day free trial license (no credit card or signing required) with all enterprise features enabled. After the trial license expires, it will default back to the Community edition.

1.6.2 Is there a live demo site where I can try out ClusterControl?

There is no live demo site available at the moment, although we have some webinars that show the product in detail. We recommend users to install ClusterControl on a test server/VM and give it a try.

You can also request a demo by filling up our Request Demo form so we can prepare a demo for you – usually, we can do this within 24 to 48 hours after the initial inquiry.

1.6.3 Can I purchase the license for the first year only?

Subscriptions are annual and include software licenses, all updates, and technical support. At the end of the subscription period, you can choose to renew the subscription or continue with the Community Edition. When the license expires, ClusterControl defaults to the Community Edition.

1.6.4 My 30-day demo license has expired but I haven't had enough time to test ClusterControl. Can I request for an extension?

Of course. Please use our contact us page or email directly to sales[at]severalnines.com.

1.6.5 I would like to subscribe as a paid user but have a limited budget. What should I do?

We'll be happy to understand your specific requirements. Please contact us at sales[at]severalnines.com.

1.6.6 Can I resell your product?

Yes, we have a reseller program. Please email us at sales[at]severalnines.com.

1.6.7 Who uses ClusterControl?

We have over 100 paying customers, including the likes of Cisco, BT, CNRS, Ping Identity, Technicolor, and Eurovision. A list of references and case studies can be found on the Case Studies page. See also *Target users*.

1.7 Technical

1.7.1 Do I need to be a DBA to use ClusterControl?

No. We build ClusterControl to be used by DBAs as well as SysAdmins and DevOps to easily manage their databases.

1.7.2 Does ClusterControl run on Windows?

No. ClusterControl only runs on RHEL-based and Debian-based with x86-64 architecture.

1.7.3 How do I upgrade ClusterControl?

Follow the upgrade steps as described in our offical documentation page. See Upgrading ClusterControl.

1.7.4 I'm using ClusterControl community version. Which support channel should I use?

You can open a thread in our public forum. In case you have to share files or logs that might contain sensitive information, please submit a support request.

1.7.5 Can I install ClusterControl on RHEL-based system but my database cluster is running on Debian?

Yes. Starting from 1.2.11, ClusterControl does not depend on the DB nodes' operating system any longer.

1.7.6 Can I use ClusterControl in an offline environment (without Internet)?

Yes. However some features might not be working well, as explained in *Offline Installation* page. Also, see this tutorial, *Deploy Database Cluster in Offline Environment*.

1.7.7 Can I have a mix of different SSH users per cluster?

It's possible but not recommended since it will make the deployment more complex. If you choose root as the SSH user, you should use root user for all nodes across the cluster.

1.7.8 Can I use a non-privileged SSH user for ClusterControl?

Unfortunately, no. ClusterControl requires super-privileged SSH user to perform tasks like deployment, administration and management of the database nodes. See *Operating system user*.

1.7.9 What is difference of MySQL Galera Cluster, Percona XtraDB Cluster and MariaDB Galera Cluster?

The Galera technology is developed by Codership Oy and is available as a patch for standard MySQL and InnoDB. Percona and MariaDB leverage the Galera library in Percona XtraDB Cluster (PXC) and MariaDB Galera Cluster respectively.

Since they all leverage the same Galera library, replication performance should be fairly similar. The Codership build usually has the latest version of Galera, although that could change in the future. See Galera Cluster Comparison – Codership vs Percona vs MariaDB.

1.7.10 I found a bug. Where should I report it?

Please report the bug by submitting a support request. For Enterprise customers, bug fixes are provided soonest possible with custom builds.

1.7.11 My cluster is down. What should I do?

Please submit a support request at our support site. If you are a paying customer, please do this with your company email address so the ticket can be assigned to the appropriate support SLA. Please also include an *Error Report*.

1.7.12 How can I submit a feature request?

You can either use our Feature Request page or submit a ticket at our support site.

1.8 Quickstart

Follow these steps to quickly get ClusterControl up and running in your infrastructure.

1.8.1 Step 1: Install ClusterControl

On the ClusterControl host, as root or sudo user, run:

```
wget https://severalnines.com/downloads/cmon/install-cc
chmod +x install-cc
sudo ./install-cc
```

Info

All new installations come with a 30-day fully functional Enterprise edition trial license (online installation only). Once expired, ClusterControl will default to Community edition.

Example

On Ubuntu 22.04 LTS:

```
    $ wget https://severalnines.com/downloads/cmon/install-cc
    Resolving severalnines.com (severalnines.com)... 104.21.94.86,␣
↪172.67.221.146, 2606:4700:3034::6815:5e56, ...
    Connecting to severalnines.com␣
↪(severalnines.com)|104.21.94.86|:443... connected.
    HTTP request sent, awaiting response... 200 OK
    Length: unspecified [text/plain]
    Saving to: 'install-cc'
    install-cc                                  [ <=>                  ␣
↪                                               ]  79.83K  --.-KB/s    ␣
↪in 0.07s
    2024-09-17 07:47:12 (1.05 MB/s) - 'install-cc' saved [81747]

    $ chmod +x install-cc

    $ sudo ./install-cc
    2024-09-17 07:47:17 UTC -- Installing lsb-release ...
    Reading package lists...
    Building dependency tree...
    Reading state information...
    lsb-release is already the newest version (11.1.0ubuntu4).
    lsb-release set to manually installed.
    0 upgraded, 0 newly installed, 0 to remove and 37 not upgraded.
    !!
    2024-09-17 07:47:29 UTC -- Only RHEL/RockyLinux/AlmaLinux 8|9, Debian␣
↪11|12, Ubuntu 20.04|22.04|24.04 LTS versions are supported
    2024-09-17 07:47:29 UTC -- System RAM is > 1.5G
    2024-09-17 07:47:29 UTC -- Setting MySQL innodb_buffer_pool_size to␣
↪50% of system RAM
    2024-09-17 07:47:29 UTC -- MySQL innodb_buffer_pool_size set to 1955M

    2024-09-17 07:47:29 UTC -- Severalnines would like your help␣
↪improving our installation process.
    2024-09-17 07:47:29 UTC -- Information such as OS, memory and install␣
↪success helps us improve how we onboard our users.
    2024-09-17 07:47:29 UTC -- None of the collected information␣
↪identifies you personally.
    2024-09-17 07:47:29 UTC -- !!
    => Would you like to help us by sending diagnostics data for the␣
↪installation? (Y/n): Y

    This script will add Severalnines repository server for deb and rpm␣
↪packages and
    install the ClusterControl Web Applicaiton and Controller.
    An Apache and MySQL server will also be installed. An existing MySQL␣
↪Server on this host can be used.

    => Enter a MySQL root password to use: 2024-09-17 08:31:53 UTC --
    => Enter the MySQL root password again:
    => Enter a CMON MySQL user password to use: 2024-09-17 08:31:58 UTC --
```

```
    => Enter the CMON user password again:

    ...
    ...

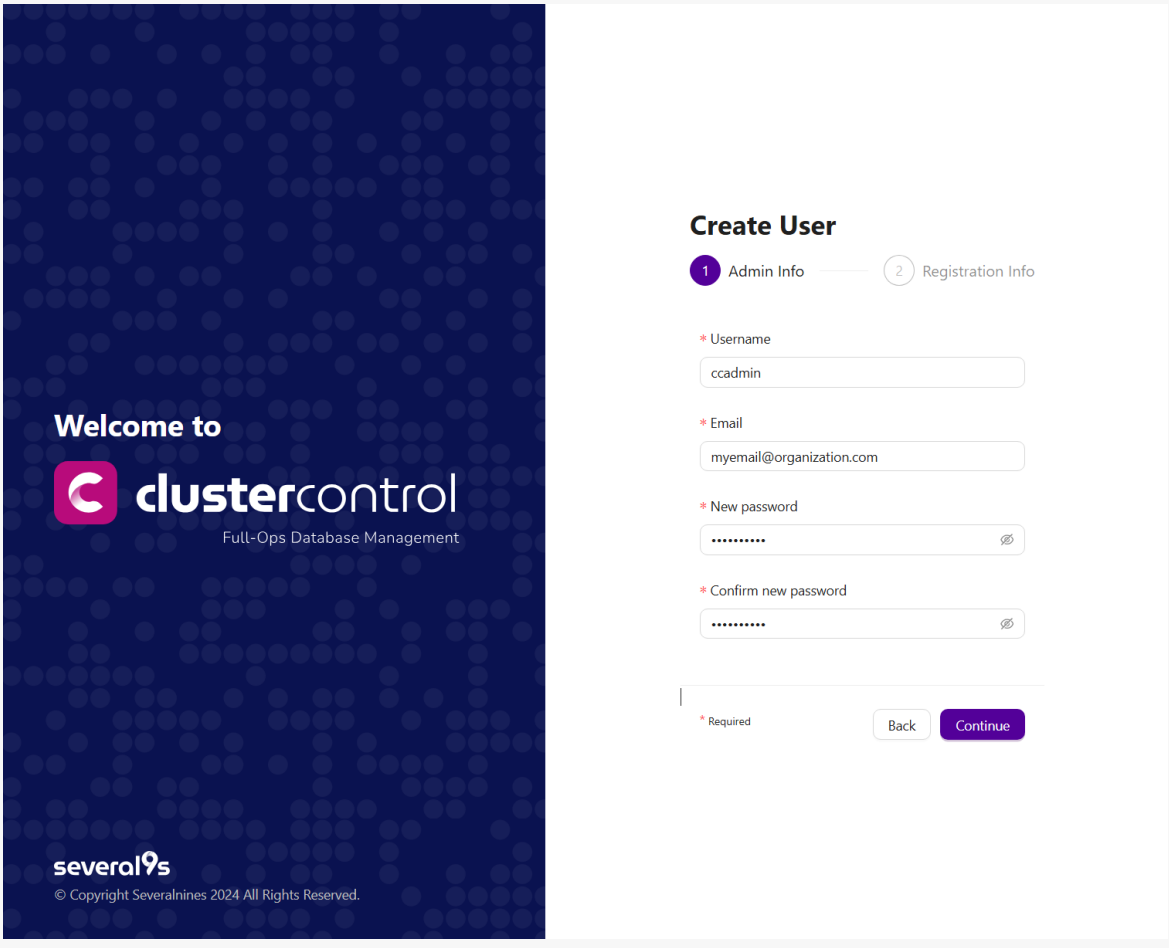
    2024-09-17 08:37:12 UTC -- ClusterControl installation completed!
    Open your web browser to https://178.62.210.68 and create a default␣
↪Admin User.
    If you want to uninstall ClusterControl then please follow the␣
↪instructions here,␣
↪https://severalnines.com/docs/administration.html#uninstall
```

For more information on the installation methods, see *Installation*.

1.8.2 Step 2: Create the first admin user

Open `https://<ClusterControl_host>/` in your browser to create the first admin user.

Example



1.8.3 Step 3: Set up SSH key-based authentication

Set up passwordless SSH (key-based authentication) to all target nodes. Run the following commands on the ClusterControl server:

```
# as root or sudo user
ssh-keygen -t rsa                                          # press Enter on␣
↪all prompts
ssh-copy-id -i ~/.ssh/id_rsa {target_node_IP_address}    # repeat this for␣
↪all target nodes
```

???+ Note

```
You could also add the generated public key, `~/.ssh/id_rsa.pub` to all␣
↪target nodes' `~/.ssh/authorized_keys` file manually.
```

As root

```
    $ whoami
    root
    $ ssh-keygen -t rsa
    $ ssh-copy-id -i /root/.ssh/id_rsa root@192.168.17.101    # db1
    $ ssh-copy-id -i /root/.ssh/id_rsa root@192.168.17.102    # db2
    $ ssh-copy-id -i /root/.ssh/id_rsa root@192.168.17.103    # db3
```

As sudoer (ubuntu)

```
    $ whoami
    ubuntu
    $ ssh-keygen -t rsa
    $ ssh-copy-id -i /home/ubuntu/.ssh/id_rsa ubuntu@192.168.17.101    # db1
    $ ssh-copy-id -i /home/ubuntu/.ssh/id_rsa ubuntu@192.168.17.102    # db2
    $ ssh-copy-id -i /home/ubuntu/.ssh/id_rsa ubuntu@192.168.17.103    # db3
```

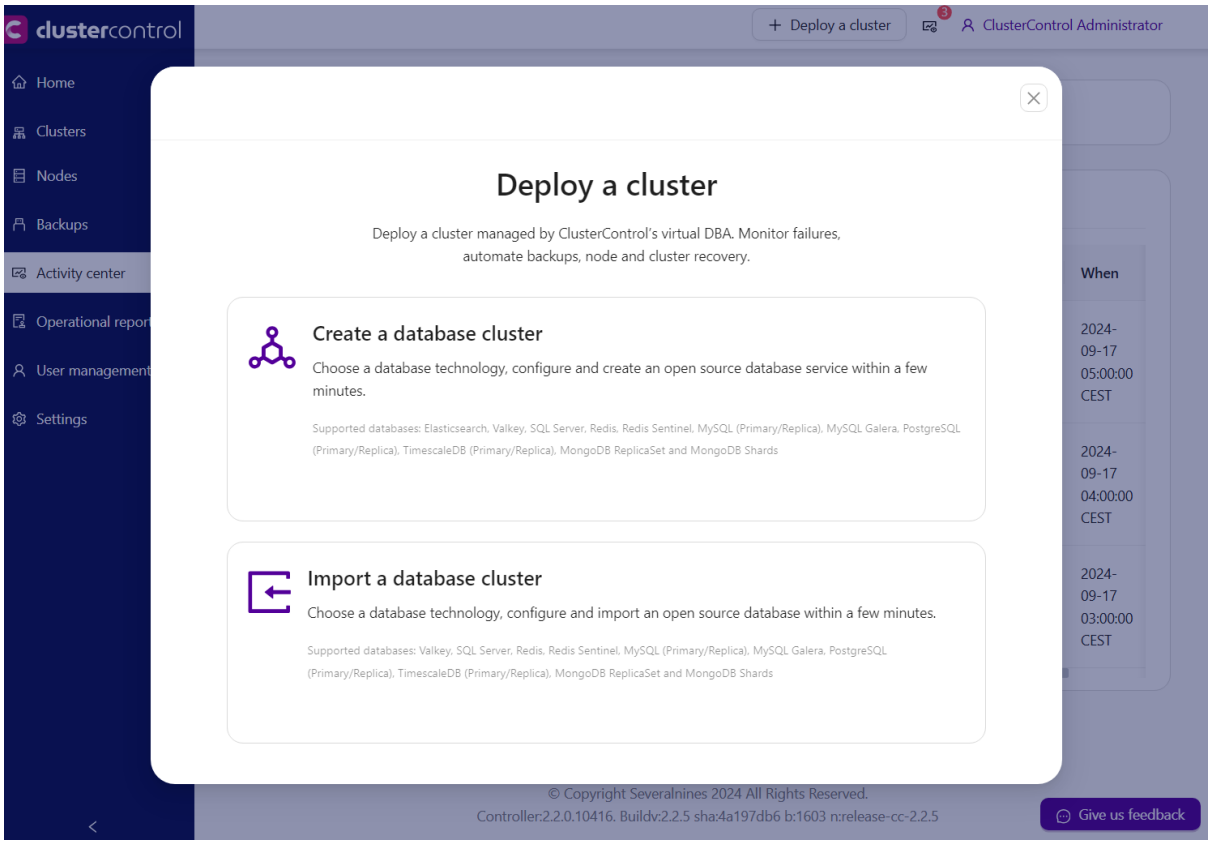
???+ Note

```
For cloud instances, you can use the corresponding private key pair␣
↪generated by the cloud provider and upload it onto the ClusterControl␣
↪host and specify the physical path when configuring the SSH-related␣
↪parameters in the ClusterControl GUI (deploy a cluster, import nodes,␣
↪etc).
```

1.8.4 Step 4: Deploy or import database cluster/server

You can deploy a new database cluster or import an existing database cluster into ClusterControl.

ClusterControl GUI



ClusterControl CLI

```
# On ClusterControl node
$ s9s cluster --create \
    --cluster-type=galera \
    --nodes="192.168.17.101;192.168.17.102;192.168.17.103" \
    --vendor=percona \
    --provider-version=8.0 \
    --db-admin-passwd='Str0nGpa$$word' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='Percona XtraDB Cluster 5.7'
```

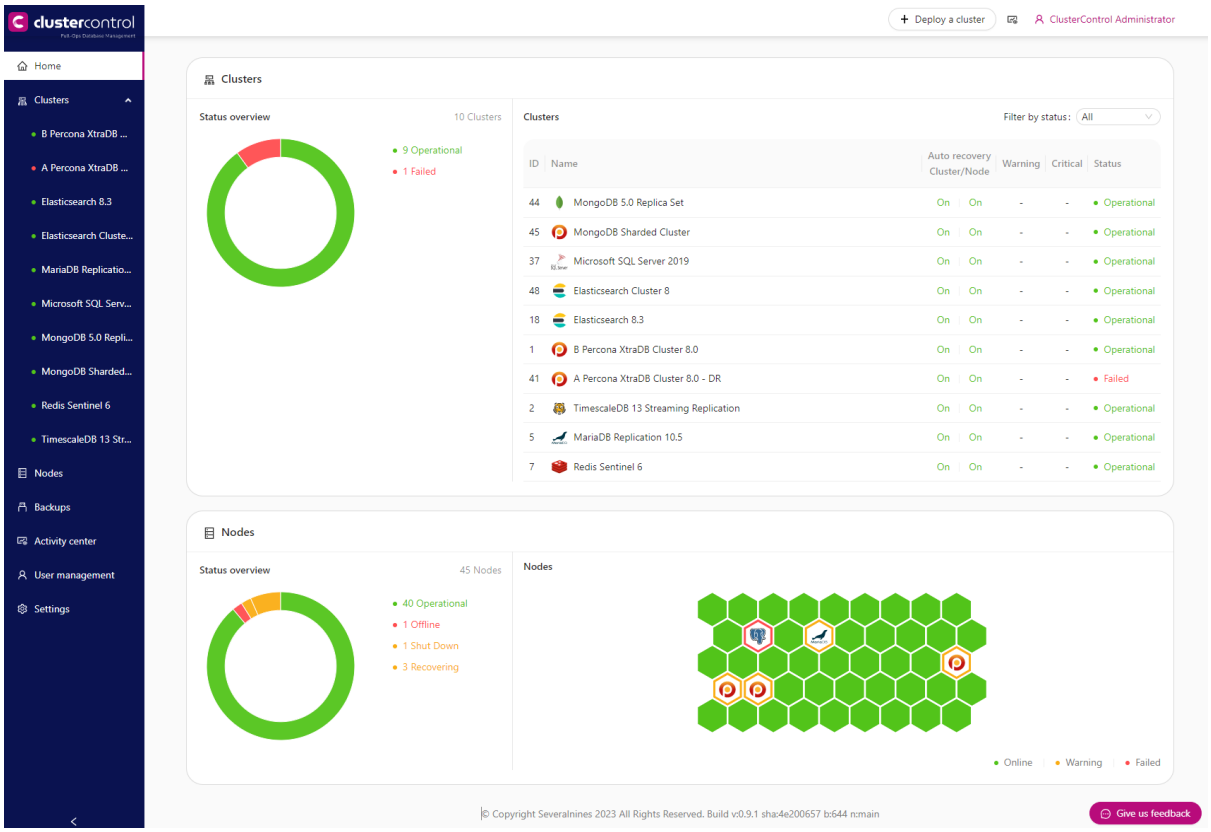
More info

- See the Tutorials section.
- *User Guide → Deployment → Create Database Cluster*
- *User Guide → Deployment → Import Database Cluster*

1.8.5 Step 5: Start managing

Use the ClusterControl GUI or CLI to monitor and manage your database cluster/server.

ClusterControl GUI



ClusterControl CLI

```
$ s9s cluster --list --long
$ s9s node --list --long
$ s9s cluster --add-node \
    --nodes=192.168.17.104 \
    --cluster-id=1 \
    --wait
$ s9s cluster --rolling-restart \
    --cluster-id=1 \
    --schedule="$(date -d 'now + 20 min')"
```

More info

- *User Guide*
- *Administration Guide*

1.8.6 Step 6: Additional resources

- *User Guide*
- *Administration Guide*
- Follow ClusterControl blog
- Contact Support
- Contact Sales

1.9 Online Installation

There are several ways to get ClusterControl installed and running in your infrastructure:

- Installer script (install-cc)
- ClusterControl Helm Charts
- Ansible Role
- Puppet Module

1.9.1 Installer script (install-cc)

The installer script is the recommended way to install ClusterControl, and it is the easiest way to get ClusterControl up and running on the supported operating system. For example, see *Quickstart*.

The script must be downloaded and executed on the ClusterControl node, which performs all necessary steps to install and configure ClusterControl's packages and dependencies on that particular host. It also supports offline installation with `OFFLINE=1` variable exported, however, you need to have a mirrored repository enabled beforehand. See *Offline Installation* for details. The script assumes that the host can install all dependencies via the operating system's repositories.

On the ClusterControl server, run the following commands:

```
wget https://severalnines.com/downloads/cmon/install-cc
chmod +x install-cc
sudo ./install-cc     # omit sudo if you run as root
```

??? Info

```
The installation script will attempt to automate the following tasks:

- Install and configure a MySQL or MariaDB server.
- Install ClusterControl dependencies via the package manager.
- Install and configure ClusterControl packages via package manager.
- Configure ClusterControl RPC API user and token.
- Configure ClusterControl Controller (CMON) with minimal configuration␣
```

(continues on next page)

(continued from previous page)

```
↪options.
- Enable the ClusterControl-related services on boot and start them up.
```

After the installation completes, open a web browser and go to `https://<ClusterControl_host>/` and create the default admin user by specifying a username (username "admin" is reserved) and password on the welcome page.

Environment variables

The installer script also understands a number of environment variables if defined. Supported environment variables are:

Variables	Description
S9S_CMON_PASSWORD	MySQL cmon user password.
S9S_ROOT_PASSWORD	MySQL root user password.
S9S_DB_PORT	MySQL port for cmon to connect.
HOST	Primary IP address or FQDN of the host. Useful if the host has multiple IP addresses.
OFFLINE	Special flag to indicate an offline installation. A mirrored repository must be enabled.
INNODB_BUFFER_POOL_SI	MySQL InnoDB buffer pool size to be configured on the host. The default is 50% of the host's RAM.
CLUSTERCONTROL_BUILD	ClusterControl builds (other than the controller). Separate each package with space.
CONTROLLER_BUILD	ClusterControl controller build.
S9S_TOOLS_BUILD	ClusterControl CLI (a.k.a s9s) build.

The environment variable can be set through the `export` command or by prefixing the install command as shown in the *Example Use Cases* section.

Example use cases

If you have multiple network interface cards, you may immediately assign a primary IP address using the `HOST` variable as per the example below (otherwise the script would prompt use to choose one primary interface):

```
HOST=192.168.1.10 ./install-cc
```

By default, the script will allocate 50% of the host's RAM to the InnoDB buffer pool. You may change this by assigning a value in MB for `INNODB_BUFFER_POOL_SIZE` variable as the example below:

```
INNODB_BUFFER_POOL_SIZE=512 ./install-cc
```

If you want to perform a single-liner non-interactive installation, you can assign each mandatory variable with its value beforehand, similar to the example below:

```
S9S_CMON_PASSWORD=cmonP4ss \
S9S_ROOT_PASSWORD=root123 \
S9S_DB_PORT=3306 \
```

(continues on next page)

(continued from previous page)

```
HOST=10.10.10.10 \
./install-cc
```

If you want to install a specific version instead of the latest in the repository, you can use `CLUSTERCONTROL_BUILD`, `CONTROLLER_BUILD` and `S9S_TOOLS_BUILD` environment variables. You can get the available package name and version from the ClusterControl download site.

Examples are as follows:

Red Hat/CentOS/AlmaLinux/Rocky Linux

```
CLUSTERCONTROL_BUILD="clustercontrol-1.7.1-5622␣
↪clustercontrol-cloud-1.7.1-163 clustercontrol-clud-1.7.1-163␣
↪clustercontrol-notifications-1.7.1-159 clustercontrol-ssh-1.7.1-70" \
CONTROLLER_BUILD="clustercontrol-controller-1.7.1-2985" \
S9S_TOOLS_BUILD="s9s-tools-1.7-93.1" \
./install-cc
```

Debian/Ubuntu

```
CLUSTERCONTROL_BUILD="clustercontrol-1.7.1-5622␣
↪clustercontrol-cloud-1.7.1-163 clustercontrol-clud-1.7.1-163␣
↪clustercontrol-notifications-1.7.1-159 clustercontrol-ssh-1.7.1-70" \
CONTROLLER_BUILD="clustercontrol-controller-1.7.1-2985" \
S9S_TOOLS_BUILD="s9s-tools-1.7.20190117-release1" \
./install-cc
```

1.9.2 Helm Chart

This Helm chart is designed to provide everything you need to get ClusterControl running in a vanila Kubernetes cluster. This includes dependencies like:

- Ingress-Nginx Controller
- MySQL Operator
- VictoriaMetrics Operator

If you do not wish to install any of those, see *Helm chart dependencies*.

Installation using Helm chart

1. Add a helm chart repository with the following commands:

   ```
   helm repo add s9s https://severalnines.github.io/helm-charts/
   helm repo update
   ```

2. Create a namespace for ClusterControl. It's also required for MySQL Operator to run in a custom namespace (not default):

   ```
   kubectl create ns clustercontrol
   kubectl config set-context --current --namespace=clustercontrol
   ```

3. Install ClusterControl using Helm:

```
helm install clustercontrol s9s/clustercontrol --devel
```

The installation will generate an example SSH key for you to use to access your database nodes. However, you can use your own SSH key as described below.

Providing your own SSH keys for ClusterControl to use

The ClusterControl Helm chart provides an example SSH key for you to use, however, you should provide your SSH keys for ClusterControl to use and connect to your target machines. These should already be configured on the target server's `authorized_keys`.

1. Create k8s secrets with your SSH keys:

```
kubectl create secret generic my-ssh-keys --from-file=key1=/
↪path/to/my/.ssh/id_rsa
```

`key1` is the filename of your SSH key in ClusterControl - this will be created under `/root/.ssh-keys-user`

> **Note**
>
> Note You can use multiple `--from-file` - be sure to provide unique key names - `key1`, `key2`, `key3`

2. Install or upgrade ClusterControl by providing `cmon.sshKeysSecretName` value with our secret name created above:

```
helm upgrade --install clustercontrol s9s/clustercontrol --set␣
↪cmon.sshKeysSecretName=my-ssh-keys --devel
```

Custom configuration via values.yaml

To create your own override file, export it from Helm using the show command:

```
helm show values s9s/clustercontrol > values.yaml
```

Look at the generated `values.yaml` and customize your configuration here. To install or upgrade using your custom `values.yaml`, include the override file with the `-f` flag:

```
helm install clustercontrol s9s/clustercontrol -f values.yaml --devel
```

> **Note**
>
> Note - ClusterControl CMON RPC API is accessible within the cluster via `cmon-master:9501`. - ClusterControl GUI is accessible within the cluster via `cmon-master:3000`. - We highly recommend ingress as ClusterControl GUI requires CMON API to be exposed and available externally.

Helm chart dependencies

Oracle MySQL Operator or NGINX ingress controller

If you already have Oracle MySQL Operator or NGINX ingress controller installed, you can set the following flags:

```
helm install clustercontrol s9s/clustercontrol --debug \
  --set fqdn=clustercontrol.example.com \
  --set installMysqlOperator=false \
  --set ingressController.enabled=false --devel
```

This Helm chart has dependencies that make ClusterControl easier to install. None of these is necessary if you provide your equivalent or you already have it installed:

- oracle-mysql-operator

```
- Oracle MySQL operator is required for running the MySQL database␣
↪within the Kubernetes cluster.

- You can disable this by setting: `installMysqlOperator: false`
```

- oracle-mysql-innodbcluster

```
- A MySQL InnoDB Cluster is required for ClusterControl.
- You must provide a different MySQL/MariaDB for ClusterControl to␣
↪use. Please refer to the [official Helm chart documentation for␣
↪MySQL InnoDB␣
↪Cluster](https://github.com/mysql/mysql-operator/blob/trunk/helm/mysql-innodbcluste
- You can disable this by setting: `createDatabases: false`.
```

- nginx-ingress-controller

```
- An ingress controller to access ClusterControl.
- For more information, please refer to the official Helm chart␣
↪documentation for the [NGINX-Ingress␣
↪Controller](https://github.com/kubernetes/ingress-nginx/tree/main/charts/ingress-n
- If you already have an ingress controller installed or wish to use a␣
↪different one, you can disable this by:

  ```yaml
  ingressController:
  enabled: false
  ```
```

VictoriaMetrics or other Prometheus-compatible monitoring

If you wish to use your VictoriaMetrics or other Prometheus-compatible monitoring systems, please refer to victoria-metrics-single parameters. These defaults provide a minimal need for ClusterControl metrics and dashboards to work. Feel free to adjust as needed, however, keep in mind the required labels annotations, and service discovery. If you already have your own VictoriaMetrics or Prometheus cluster and don't want to install this, you can disable it by setting the following inside `values.yaml`:

```
prometheusHostname: my-prometheus-server
monitoring:
  enabled: false
```

1.9.3 Ansible Role

If you are automating your infrastructure using Ansible, we have created a role for this purpose and it is available at Ansible Galaxy. This role also supports deploy a new cluster and import an existing cluster into ClusterControl automatically, as shown under *Example Playbooks*.

See also

ClusterControl Ansible Github page.

Getting the role is as easy as:

```
ansible-galaxy install severalnines.clustercontrol
```

Usage

1. Get the ClusterControl Ansible role from Ansible Galaxy or Github.

Ansible Galaxy

```
ansible-galaxy install severalnines.clustercontrol
```

Github

```
git clone https://github.com/severalnines/ansible-clustercontrol
cp -rf ansible-clustercontrol /etc/ansible/roles/severalnines.clustercontrol
```

2. Create a playbook. See *Example playbooks* section.
3. Run the playbook.

   ```
   ansible-playbook cc.playbook
   ```

4. Once ClusterControl is installed, go to `https://<ClusterControl_host>/` and create the default admin user/password.
5. On ClusterControl node, set up SSH key-based authentication to all target DB nodes. For example, if ClusterControl node is 192.168.0.10 and DB nodes are 192.168.0.11, 192.168.0.12 and 192.168.0.13:

   ```
   ssh-copy-id 192.168.0.11    # DB1
   ssh-copy-id 192.168.0.12    # DB2
   ssh-copy-id 192.168.0.13    # DB3
   ```

6. Start to deploy a new database cluster or add an existing cluster.

Example playbooks

The simplest playbook would be:

```
- hosts: clustercontrol-server
  roles:
    - { role: severalnines.clustercontrol }
```

If you would like to specify custom configuration values as explained above, create a file called `vars/main.yml` and include it inside the playbook:

```
- hosts: 192.168.10.15
  vars:
    - vars/main.yml
  roles:
    - { role: severalnines.clustercontrol, tags: controller }
```

Inside `vars/main.yml`:

```
controller: true
mysql_root_username: admin
mysql_root_password: super-user-password
cmon_mysql_password: super-cmon-password
cmon_mysql_port: 3307
```

If you are running as another user, ensure the user has the ability to escalate as a superuser via sudo. Example playbook for Ubuntu 20.04 with sudo password enabled:

```
- hosts: ubuntu@192.168.10.100
  become: yes
  become_user: root
  roles:
    - { role: severalnines.clustercontrol, tags: controller }
```

Then, execute the command with the `--ask-become-pass` flag, for example:

```
$ ansible-playbook cc.playbook --ask-become-pass
```

1.9.4 Puppet module

This module installs ClusterControl for your new database node/cluster deployment or on top of your existing database node/cluster. The Puppet module for ClusterControl automates the following actions based on the scope of this module:

- Setup ClusterControl required repositories
- Download and installs dependencies. These dependencies are required by its compontents such as Apache, PHP, OpenSSL, MySQL/MariaDB are among its basic packages required.
- Install ClusterControl components such as the controller, front-end, CMON cloud, CMON SSH, and CMON Clud (upload/download cloud link)
- Automates configuration for Apache

    - configures the for port 80 and port 443 (for SSL/TLS). This includes its designated rewrite rules needed
    - ensures port 443 is enabled
    - enables header module setting *X-Frame-Options: sameorigin*
    - check permission for ClusterControl UI and install SSL.
- Automates MySQL/MariaDB installation.
    - create CMON DB, grant cmon user and configure DB for ClusterControl UI.

Requirements

Make sure you meet following criteria prior to the deployment:

- ClusterControl node must run on a clean dedicated host with internet connection.
- If you are running as non-root user, make sure the user is able to escalate to root with sudo command.

Installation using Puppet

Our ClusterControl module for Puppet is available either on Puppet Forge or using this Severalnines Puppet by cloning or downloading it as a zip. Then place it under puppetlabs `modulepath` directory and make sure to name your module directory as `clustercontrol`, for example `/etc/puppetlabs/code/environments/production/modules/clustercontrol`.

1. Generate an API Token. To do this, go to $modulepath/clustercontrol/files, then run the following. For example,

```
$ cd /
↪etc/puppetlabs/code/environments/production/modules/clustercontrol
$ files/s9s_helper.sh --generate-token
efc6ac7fbea2da1b056b901541697ec7a9be6a77
```

   Reserve that token which you will use that as an input parameter for your manifests file.

2. Let say you have the following hosts:

```
clustercontrol.local          192.168.1.10
galera1.local                      192.168.1.11
galera2.local                      192.168.1.12
galera3.local                      192.168.1.13
```

3. Create a manifests file, let say we named it `clustercontrol.pp`:

```
cd /etc/puppetlabs/code/environments/production
ls -alth manifests/clustercontrol.pp
-rw-r--r-- 1 root root 1.4K Oct 23 15:00 manifests/clustercontrol.pp
```

4. Have the following example puppet agent node below where the hostname of the target to install ClusterControl using puppet is 192.168.40.90. Below is the definition for ClusterControl as follows:

```
node 'clustercontrol.puppet.local' {      # Applies only to mentioned␣
↪node. If nothing mentioned, applies to all.
```

(continues on next page)

(continued from previous page)

```
        class { 'clustercontrol':
            is_controller => true,
            cc_hostname => '192.168.40.90',
            mysql_cmon_password => 'R00tP@55',
            api_token => 'efc6ac7fbea2da1b056b901541697ec7a9be6a77',
            ssh_user => 'vagrant',
            ssh_user_group => 'vagrant',
            only_cc_v2 => true
        }
}
```

5. Run the following command on the target clustercontrol, which is the host `clustercontrol.puppet.local` for this example installation:

```
puppet agent -t
```

Once deployment is complete, open the ClusterControl GUI at `https://<ClusterControl_host>/` and create the first admin user. You can start to add existing database node/cluster, or deploy a new one. Ensure that SSH key-based authentication is configured properly from ClusterControl node to all DB nodes beforehand.

This module only supports bootstrapping MySQL servers with IP address only (it expects `skip-name-resolve` is enabled on all MySQL nodes).

More info

- ClusterControl Puppet Module Github page.
- For more examples of deployments using Puppet, see Puppet Module for ClusterControl - Adding Management and Monitoring to your Existing Database Clusters.

The `install-cc` official installation script has been updated with several helper functions to simplify the installation of ClusterControl in air-gapped (offline) environments. These functions enable the creation of `.deb` or `.rpm` repositories that can be imported into the target system.

The process involves first running the script on a server or virtual machine that matches the Linux distribution of the air-gapped server and has internet access. The script performs the following tasks:

- Downloads the necessary ClusterControl packages along with their dependencies, such as the MySQL server or `gnuplot` for the CMON Controller.
- Builds a local file-based `.deb` or `.rpm` package repository based on the Linux distribution.
- Generates a tarball containing the repository directory and the repository metadata file, which can then be transferred and imported onto another server.

Take note that the following ClusterControl features will not work without an Internet connection:

- ***Backup → Create/Schedule Backup → Upload to Cloud*** – requires a connection to cloud providers.
- ***Integrations → Cloud Providers*** – requires a connection to cloud providers.
- ***Manage → Load Balancer*** – requires a connection to EPEL, ProxySQL, HAProxy, MariaDB repository.

- ***Manage → Upgrades*** – requires a connection to the provider's repository.
- ***Deploy Database Cluster*** – requires a connection to the database provider's repository.

1.10 Offline Setup

1.10.1 Step 1: Build and export the repository

The offline repository package is configured on a temporary host that has internet access. The installer script is able to build the local repository for offline usage on RHEL/Rocky Linux/AlmaLinux 8/9, Debian 11/12 and Ubuntu 20.04/22.04/24.04.

Download the installer script on the temporary host that has an Internet access:

```
wget https://severalnines.com/downloads/cmon/install-cc
chmod +x install-cc
```

Build the local repository for ClusterControl by using the `--airgap-repo` flag:

```
./install-cc --airgap-repo
```

Example

The following is an example output of the temporary server (the staging host that has an Internet connection) on Ubuntu 24.04:

```
[root@staging]:/home/vagrant# ./install-cc --airgap-repo
This script will configure the Severalnines repository server for both␣
↪deb and rpm packages, and install the ClusterControl Web Application␣
↪and Controller.
It will install a new MySQL server or utilize an existing MySQL server on␣
↪the host.
2025-05-30 05:52:23 UTC -- Only RHEL/RockyLinux/AlmaLinux 8|9, Debian␣
↪11|12, Ubuntu 20.04|22.04|24.04 LTS versions are supported
2025-05-30 05:52:23 UTC -- Waiting for locks ...
2025-05-30 05:52:23 UTC -- Update distro repos ...
141 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists...
Building dependency tree...
Reading state information...
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 141 not upgraded.
2025-05-30 05:52:56 UTC -- DOWNLOAD_DIR=/home/vagrant/cc_packages
2025-05-30 05:52:56 UTC -- REPO_DIR=/usr/local/repo/cc_repo
2025-05-30 05:52:56 UTC -- EXPORT_FILE=cc_repo.tar.gz
2025-05-30 05:52:56 UTC -- IMPORT_REPO_DIR=/usr/local/repo/cc_repo
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
The following additional packages will be installed:
```

```
  dirmngr gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client gpgconf␣
↪gpgsm gpgv keyboxd
Suggested packages:
  pinentry-gnome3 tor parcimonie xloadimage gpg-wks-server scdaemon
The following packages will be upgraded:
  dirmngr gnupg gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client␣
↪gpgconf gpgsm gpgv keyboxd
11 upgraded, 0 newly installed, 0 to remove and 130 not upgraded.
Need to get 2291 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪gpg-wks-client amd64 2.4.4-2ubuntu17.2 [70.9 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪dirmngr amd64 2.4.4-2ubuntu17.2 [323 kB]
Get:3 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪gnupg-utils amd64 2.4.4-2ubuntu17.2 [109 kB]
Get:4 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 gpgsm␣
↪amd64 2.4.4-2ubuntu17.2 [232 kB]
Get:5 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪gpg-agent amd64 2.4.4-2ubuntu17.2 [227 kB]
Get:6 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 gpg␣
↪amd64 2.4.4-2ubuntu17.2 [565 kB]
Get:7 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪gpgconf amd64 2.4.4-2ubuntu17.2 [103 kB]
Get:8 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 gnupg␣
↪all 2.4.4-2ubuntu17.2 [359 kB]
Get:9 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪keyboxd amd64 2.4.4-2ubuntu17.2 [78.3 kB]
Get:10 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64 gpgv␣
↪amd64 2.4.4-2ubuntu17.2 [158 kB]
Get:11 http://us.archive.ubuntu.com/ubuntu noble-updates/main amd64␣
↪gnupg-l10n all 2.4.4-2ubuntu17.2 [66.1 kB]
Fetched 2291 kB in 3s (674 kB/s)
(Reading database ... 50146 files and directories currently installed.)
Preparing to unpack .../0-gpg-wks-client_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gpg-wks-client (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../1-dirmngr_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking dirmngr (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../2-gnupg-utils_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gnupg-utils (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../3-gpgsm_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gpgsm (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../4-gpg-agent_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gpg-agent (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../5-gpg_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gpg (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../6-gpgconf_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gpgconf (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../7-gnupg_2.4.4-2ubuntu17.2_all.deb ...
Unpacking gnupg (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
```

```
Preparing to unpack .../8-keyboxd_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking keyboxd (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Preparing to unpack .../9-gpgv_2.4.4-2ubuntu17.2_amd64.deb ...
Unpacking gpgv (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Setting up gpgv (2.4.4-2ubuntu17.2) ...
(Reading database ... 50146 files and directories currently installed.)
Preparing to unpack .../gnupg-l10n_2.4.4-2ubuntu17.2_all.deb ...
Unpacking gnupg-l10n (2.4.4-2ubuntu17.2) over (2.4.4-2ubuntu17) ...
Setting up gnupg-l10n (2.4.4-2ubuntu17.2) ...
Setting up gpgconf (2.4.4-2ubuntu17.2) ...
Setting up gpg (2.4.4-2ubuntu17.2) ...
Setting up gnupg-utils (2.4.4-2ubuntu17.2) ...
Setting up gpg-agent (2.4.4-2ubuntu17.2) ...
Setting up gpgsm (2.4.4-2ubuntu17.2) ...
Setting up dirmngr (2.4.4-2ubuntu17.2) ...
Setting up keyboxd (2.4.4-2ubuntu17.2) ...
Setting up gnupg (2.4.4-2ubuntu17.2) ...
Setting up gpg-wks-client (2.4.4-2ubuntu17.2) ...
Processing triggers for install-info (7.1-3build2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d␣
 ↪instead (see apt-key(8)).
OK
deb http://repo.severalnines.com/s9s-tools/noble/ ./
--2025-05-30 05:53:55-- ␣
 ↪http://repo.severalnines.com/severalnines-repos.asc
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
 ↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 963 [application/octet-stream]
Saving to: 'STDOUT'

-                           ␣
 ↪100%[=================================================>]     963  --.-
 ↪KB/s    in 0s

2025-05-30 05:53:55 (53.1 MB/s) - written to stdout [963/963]
```

```
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d␣
↪instead (see apt-key(8)).
OK
deb [arch=amd64] http://repo.severalnines.com/deb ubuntu main
2025-05-30 05:53:59 UTC -- Added /etc/apt/sources.list.d/s9s-repo.list
2025-05-30 05:53:59 UTC -- Updating repo ...
Get:1 http://repo.severalnines.com/deb ubuntu InRelease [3973 B]
Get:2 http://repo.severalnines.com/s9s-tools/noble ./ InRelease [1876 B]
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu noble InRelease
Hit:5 http://us.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://us.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:7 http://repo.severalnines.com/deb ubuntu/main amd64 Packages [71.9␣
↪kB]
Get:8 http://repo.severalnines.com/s9s-tools/noble ./ Packages [785 B]
Fetched 78.5 kB in 9s (8410 B/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
130 packages can be upgraded. Run 'apt list --upgradable' to see them.
W: http://repo.severalnines.com/deb/dists/ubuntu/InRelease: Key is stored␣
↪in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the␣
↪DEPRECATION section in apt-key(8) for details.
W: http://repo.severalnines.com/s9s-tools/noble/./InRelease: Key is␣
↪stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the␣
↪DEPRECATION section in apt-key(8) for details.
Hit:1 http://us.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://repo.severalnines.com/deb ubuntu InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://repo.severalnines.com/s9s-tools/noble ./ InRelease
Hit:5 http://us.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
130 packages can be upgraded. Run 'apt list --upgradable' to see them.
W: http://repo.severalnines.com/deb/dists/ubuntu/InRelease: Key is stored␣
↪in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the␣
↪DEPRECATION section in apt-key(8) for details.
W: http://repo.severalnines.com/s9s-tools/noble/./InRelease: Key is␣
↪stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the␣
↪DEPRECATION section in apt-key(8) for details.
2025-05-30 05:54:34 UTC -- Downloading .deb packages and dependencies...
2025-05-30 05:54:34 UTC -- Processing clustercontrol-mcc...

WARNING: apt does not have a stable CLI interface. Use with caution in␣
↪scripts.

--2025-05-30 05:54:38-- ␣
```

```
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-cloud/clustercontrol-cloud
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 16809906 (16M) [application/octet-stream]
Saving to: 'clustercontrol-cloud_2.3.2-423_x86_64.deb'

clustercontrol-cloud_2.3.2-42␣
↪100%[=================================================>]  16.03M ␣
↪1.05MB/s    in 16s

2025-05-30 05:54:55 (1002 KB/s) -
↪'clustercontrol-cloud_2.3.2-423_x86_64.deb' saved [16809906/16809906]

--2025-05-30 05:54:55-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-clud/clustercontrol-clud_2
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13383006 (13M) [application/octet-stream]
Saving to: 'clustercontrol-clud_2.3.2-423_x86_64.deb'

clustercontrol-clud_2.3.2-423␣
↪100%[=================================================>]  12.76M ␣
↪1.67MB/s    in 8.8s

2025-05-30 05:55:04 (1.45 MB/s) -
↪'clustercontrol-clud_2.3.2-423_x86_64.deb' saved [13383006/13383006]

--2025-05-30 05:55:04-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-proxy/clustercontrol-proxy
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7532004 (7.2M) [application/octet-stream]
Saving to: 'clustercontrol-proxy_2.3.2-78_x86_64.deb'

clustercontrol-proxy_2.3.2-78␣
↪100%[=================================================>]   7.18M ␣
↪1.34MB/s    in 6.0s

2025-05-30 05:55:11 (1.20 MB/s) -
↪'clustercontrol-proxy_2.3.2-78_x86_64.deb' saved [7532004/7532004]

--2025-05-30 05:55:11-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-kuber-proxy/clustercontrol
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
```

```
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 15708012 (15M) [application/octet-stream]
Saving to: 'clustercontrol-kuber-proxy-0.1.0-39-Linux.deb'

clustercontrol-kuber-proxy-0.␣
↪100%[=================================================>]  14.98M ␣
↪2.37MB/s    in 8.3s

2025-05-30 05:55:19 (1.81 MB/s) -
↪'clustercontrol-kuber-proxy-0.1.0-39-Linux.deb' saved␣
↪[15708012/15708012]

--2025-05-30 05:55:19-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-notifications/clustercontr
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3291698 (3.1M) [application/octet-stream]
Saving to: 'clustercontrol-notifications_2.3.2-373_x86_64.deb'

clustercontrol-notifications_␣
↪100%[=================================================>]   3.14M ␣
↪1.03MB/s    in 3.0s

2025-05-30 05:55:23 (1.03 MB/s) -
↪'clustercontrol-notifications_2.3.2-373_x86_64.deb' saved␣
↪[3291698/3291698]

--2025-05-30 05:55:23-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-ssh/clustercontrol-ssh_2.3
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3156366 (3.0M) [application/octet-stream]
Saving to: 'clustercontrol-ssh_2.3.2-213_x86_64.deb'

clustercontrol-ssh_2.3.2-213_␣
↪100%[=================================================>]   3.01M ␣
↪1.14MB/s    in 2.6s

2025-05-30 05:55:26 (1.14 MB/s) - 'clustercontrol-ssh_2.3.2-213_x86_64.deb
↪' saved [3156366/3156366]

--2025-05-30 05:55:26-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-mcc/clustercontrol-mcc_2.3
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
```

```
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6834822 (6.5M) [application/octet-stream]
Saving to: 'clustercontrol-mcc_2.3.2-457_amd64.deb'

clustercontrol-mcc_2.3.2-457_␣
↪100%[=================================================>]   6.52M ␣
↪1.17MB/s    in 6.3s

2025-05-30 05:55:33 (1.03 MB/s) - 'clustercontrol-mcc_2.3.2-457_amd64.deb
↪' saved [6834822/6834822]

2025-05-30 05:55:33 UTC -- Processing clustercontrol-controller...

WARNING: apt does not have a stable CLI interface. Use with caution in␣
↪scripts.

--2025-05-30 05:55:36-- ␣
↪http://repo.severalnines.com/deb/pool/main/c/clustercontrol-controller/clustercontrol-
Resolving repo.severalnines.com (repo.severalnines.com)... 54.247.53.237
Connecting to repo.severalnines.com␣
↪(repo.severalnines.com)|54.247.53.237|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42128170 (40M) [application/octet-stream]
Saving to: 'clustercontrol-controller-2.3.2-12981-x86_64.deb'

clustercontrol-controller-2.3␣
↪100%[=================================================>]  40.18M ␣
↪1.37MB/s    in 23s

2025-05-30 05:55:59 (1.76 MB/s) -
↪'clustercontrol-controller-2.3.2-12981-x86_64.deb' saved␣
↪[42128170/42128170]

--2025-05-30 05:55:59-- ␣
↪http://us.archive.ubuntu.com/ubuntu/pool/main/f/fonts-android/fonts-droid-fallback_6.0
Resolving us.archive.ubuntu.com (us.archive.ubuntu.com)... 91.189.91.82,␣
↪91.189.91.81, 91.189.91.83, ...
Connecting to us.archive.ubuntu.com␣
↪(us.archive.ubuntu.com)|91.189.91.82|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1805156 (1.7M) [application/vnd.debian.binary-package]
Saving to: 'fonts-droid-fallback_6.0.1r16-1.1build1_all.deb'

fonts-droid-fallback_6.0.1r16␣
↪100%[=================================================>]   1.72M  ␣
↪871KB/s    in 2.0s

2025-05-30 05:56:02 (871 KB/s) -
```

```
↪'fonts-droid-fallback_6.0.1r16-1.1build1_all.deb' saved␣
↪[1805156/1805156]

--2025-05-30 05:56:02-- ␣
↪http://us.archive.ubuntu.com/ubuntu/pool/main/g/gcc-14/libgomp1_14.2.0-4ubuntu2~24.04_
Resolving us.archive.ubuntu.com (us.archive.ubuntu.com)... 91.189.91.82,␣
↪91.189.91.83, 91.189.91.81, ...
Connecting to us.archive.ubuntu.com␣
↪(us.archive.ubuntu.com)|91.189.91.82|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 147940 (144K) [application/vnd.debian.binary-package]
Saving to: 'libgomp1_14.2.0-4ubuntu2~24.04_amd64.deb'

libgomp1_14.2.0-4ubuntu2~24.0␣
↪100%[=================================================>] 144.47K  ␣
↪142KB/s    in 1.0s

2025-05-30 05:56:04 (142 KB/s) - 'libgomp1_14.2.0-4ubuntu2~24.04_amd64.deb
↪' saved [147940/147940]

--2025-05-30 05:56:04-- ␣
↪http://us.archive.ubuntu.com/ubuntu/pool/main/f/fftw3/libfftw3-double3_3.3.10-1ubuntu3
Resolving us.archive.ubuntu.com (us.archive.ubuntu.com)... 91.189.91.82,␣
↪91.189.91.83, 91.189.91.81, ...
Connecting to us.archive.ubuntu.com␣
↪(us.archive.ubuntu.com)|91.189.91.82|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 838254 (819K) [application/vnd.debian.binary-package]
Saving to: 'libfftw3-double3_3.3.10-1ubuntu3_amd64.deb'

libfftw3-double3_3.3.10-1ubun␣
↪100%[=================================================>] 818.61K  ␣
↪440KB/s    in 1.9s

2025-05-30 05:56:06 (440 KB/s) -
↪'libfftw3-double3_3.3.10-1ubuntu3_amd64.deb' saved [838254/838254]

--2025-05-30 05:56:06-- ␣
↪http://us.archive.ubuntu.com/ubuntu/pool/main/f/fonts-dejavu/fonts-dejavu-mono_2.37-8_
Resolving us.archive.ubuntu.com (us.archive.ubuntu.com)... 91.189.91.83,␣
↪91.189.91.81, 91.189.91.82, ...
Connecting to us.archive.ubuntu.com␣
↪(us.archive.ubuntu.com)|91.189.91.83|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 502232 (490K) [application/vnd.debian.binary-package]
Saving to: 'fonts-dejavu-mono_2.37-8_all.deb'

fonts-dejavu-mono_2.37-8_all.␣
↪100%[=================================================>] 490.46K  ␣
↪301KB/s    in 1.6s
```

```
2025-05-30 05:56:09 (301 KB/s) - 'fonts-dejavu-mono_2.37-8_all.deb' saved␣
↪[502232/502232]

...
...
...

dpkg-scanpackages: info: Wrote 140 entries to output Packages file.
2025-05-30 06:04:58 UTC -- Debian repository created successfully in /
↪usr/local/repo/cc_repo
2025-05-30 06:04:58 UTC -- Exporting repository to /
↪usr/local/repo/cc_repo.tar.gz...
2025-05-30 06:05:29 UTC -- Repository exported!
```

The process of creating a local repository by the installer script involves several steps:

1. Create the local repository
2. Download all packages required by ClusterControl, including dependencies
3. Export the repository into a compressed file before transferring it to the air-gapped host.

The tarball is called `cc_repo.tar.gz` and stored at one level above the repository directory (`/usr/local/repo`). The filename can be changed with the `EXPORT_FILE` environment variable.

1.10.2 Step 2: Remove default repositories

To ensure a smooth installation process, remove the default repository files on the ClusterControl host so that the imported offline repository (built on step 1) will be used as a default repository.

RHEL/RockyLinux/AlmaLinux

The default repositories are defined under `/etc/yum.repos.d` directory. Remove all repository definition files under `/etc/yum.repos/d` as below:

```
rm -f /etc/yum.repos.d/*.repo
```

Or, disable the repositories:

```
sudo sed -i 's/^enabled=1/enabled=0/' /etc/yum.repos.d/*.repo
```

Debian/Ubuntu

The default repositories are defined inside `/etc/apt/sources.list`. Remove the default lines by either deleting or commenting them out:

```
sudo sed -i '/^deb /d' /etc/apt/sources.list
```

1.10.3 Step 3: Import the repository to the air-gapped host

Importing a repository tarball takes two steps:

1. Copy the `install-cc` and `cc_repo.tar.gz` file to the target host. You may use remote copy tools like SFTP, SCP, rsync or use media device like USB drive or DVD.
2. Run the `install-cc` script with the `--import-repo` parameter.

Before running the `--import-repo` command in the air-gapped environment, disable the default repository since we are going to replace the existing repositories with the one that we prepared in the previous step.

The tarball (`cc_repo.tar.gz`) is located in the directory where the installer script is located. Run the repository import process with the following command:

```
./install-cc --import-repo
```

Example

Tranfer the installer script, `install-cc` and the local repository tarball, `cc_repo.tar.gz` from the staging host (with Internet) to the offline ClusterControl host (without Internet):

```
[root@staging ~]# sftp vagrant@10.10.10.11
vagrant@10.10.10.11's password:
Connected to 10.10.10.11.
sftp> put install-cc
Uploading install-cc to /home/vagrant/install-cc
install-cc-dev                                                ␣
↪             100%   59KB   1.4MB/s   00:00
sftp> exit
[root@staging ~]# cd /usr/local/repo/
[root@staging ~]# sftp vagrant@10.10.10.11
vagrant@10.10.10.11's password:
Connected to 10.10.10.11.
sftp> put cc_repo.tar.gz
Uploading cc_repo.tar.gz to /home/vagrant/cc_repo.tar.gz
cc_repo.tar.gz                                                ␣
↪             100%  266MB  15.5MB/s   00:17
```

Import the repository on the ClusterControl host:

```
[root@clustercontrol ~]:/home/vagrant# ./install-cc --import-repo
This script will configure the Severalnines repository server for both␣
↪deb and rpm packages, and install the ClusterControl Web Application␣
↪and Controller.
It will install a new MySQL server or utilize an existing MySQL server on␣
↪the host.
2025-05-30 06:09:51 UTC -- Only RHEL/RockyLinux/AlmaLinux 8|9, Debian␣
↪11|12, Ubuntu 20.04|22.04|24.04 LTS versions are supported
2025-05-30 06:09:51 UTC -- Waiting for locks ...
2025-05-30 06:09:51 UTC -- Update distro repos ...
141 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists...
Building dependency tree...
```

```
Reading state information...
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 141 not upgraded.
2025-05-30 06:10:13 UTC -- Extracting /home/vagrant/cc_repo.tar.gz to /
↪usr/local/repo/cc_repo...
2025-05-30 06:10:14 UTC -- Configuring Debian repository...
Get:1 file:/usr/local/repo/cc_repo stable InRelease
Ign:1 file:/usr/local/repo/cc_repo stable InRelease
Get:2 file:/usr/local/repo/cc_repo stable Release [626 B]
Get:2 file:/usr/local/repo/cc_repo stable Release [626 B]
Get:3 file:/usr/local/repo/cc_repo stable Release.gpg
Ign:3 file:/usr/local/repo/cc_repo stable Release.gpg
Get:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages
Ign:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Get:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages
Ign:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Get:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages
Ign:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Get:4 file:/usr/local/repo/cc_repo stable/main amd64 Packages [45.9 kB]
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Get:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Ign:5 file:/usr/local/repo/cc_repo stable/main Translation-en
Get:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Ign:6 file:/usr/local/repo/cc_repo stable/main amd64 Components
Hit:7 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:8 http://us.archive.ubuntu.com/ubuntu noble InRelease
```

```
Get:9 http://us.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:10 http://us.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 126 kB in 4s (32.2 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
141 packages can be upgraded. Run 'apt list --upgradable' to see them.
W: No Hash entry in Release file /
 ↪var/lib/apt/lists/partial/_usr_local_repo_cc%5frepo_dists_stable_Release␣
 ↪which is considered strong enough for security purposes
2025-05-30 06:10:20 UTC -- Debian repository successfully configured.
2025-05-30 06:10:20 UTC -- Repository imported!
```

1.10.4 Step 4: Install ClusterControl in offline mode

Once the local ClusterControl repository is imported, install ClusterControl without any access to the public internet. The installation process remains the same — simply run the installation script while setting the `OFFLINE=1` environment variable:

```
OFFLINE=1 INNODB_BUFFER_POOL_SIZE={db_memory_allocation} \
  SEND_DIAGNOSTICS=0 \
  S9S_CMON_PASSWORD={cmon_password} \
  S9S_ROOT_PASSWORD={root_password} ./install-cc
```

Adjust the `INNODB_BUFFER_POOL_SIZE` parameter based on the hardware or VM specifications you are using and fill in the `cmon_password` and `root_password` according to your password policy.

To configure Severalnines repository to use HTTPS URL instead, specify `USE_REPO_TLS=1`.

Example

Perform the offline ClusterControl installation on the ClusterControl host:

```
[root@clustercontrol ~]# OFFLINE=1 INNODB_BUFFER_POOL_SIZE=512␣
 ↪SEND_DIAGNOSTICS=0 S9S_CMON_PASSWORD=cmon123 S9S_ROOT_PASSWORD=root123 .
 ↪/install-cc
This script will configure the Severalnines repository server for both␣
 ↪deb and rpm packages, and install the ClusterControl Web Application␣
 ↪and Controller.
It will install a new MySQL server or utilize an existing MySQL server on␣
 ↪the host.
2025-05-30 06:12:33 UTC -- Only RHEL/RockyLinux/AlmaLinux 8|9, Debian␣
 ↪11|12, Ubuntu 20.04|22.04|24.04 LTS versions are supported
Reading package lists...
Building dependency tree...
Reading state information...
lsb-release is already the newest version (12.0-2).
0 upgraded, 0 newly installed, 0 to remove and 141 not upgraded.
2025-05-30 06:12:34 UTC -- System RAM is > 1.5G
2025-05-30 06:12:34 UTC -- Setting MySQL innodb_buffer_pool_size to 40%␣
 ↪of system RAM
```

```
2025-05-30 06:12:34 UTC -- MySQL innodb_buffer_pool_size set to 512M
2025-05-30 06:12:34 UTC -- Run with 'INNODB_BUFFER_POOL_SIZE=512 ./
↪install-cc' to customize the innodb buffer pool setting.

2025-05-30 06:12:34 UTC -- Severalnines would like your help improving␣
↪our installation process.
2025-05-30 06:12:34 UTC -- Information such as OS, memory and install␣
↪success helps us improve how we onboard our users.
2025-05-30 06:12:34 UTC -- None of the collected information identifies␣
↪you personally.
2025-05-30 06:12:34 UTC -- !!

This script will configure the Severalnines repository server for both␣
↪deb and rpm packages, and install the ClusterControl Web Application␣
↪and Controller.
It will install a new MySQL server or utilize an existing MySQL server on␣
↪the host.


2025-05-30 06:12:34 UTC -- Installing required packages ...
Reading package lists...
Building dependency tree...
Reading state information...

...
...
...

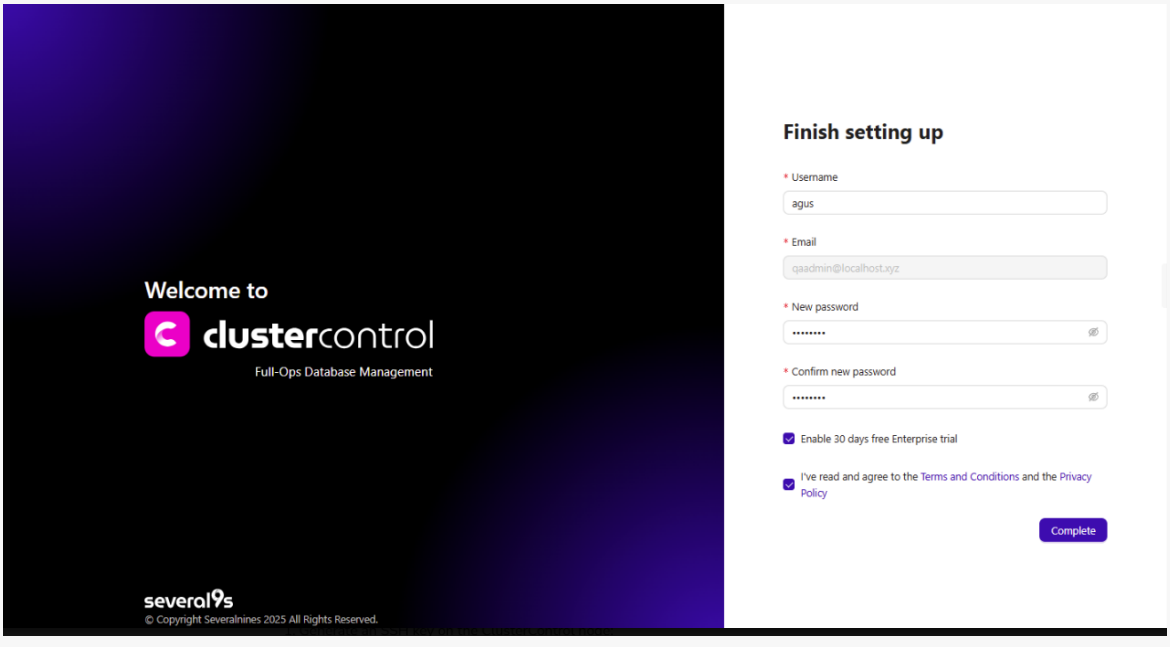
No VM guests are running outdated hypervisor (qemu) binaries on this host.
=> Waiting 60s until the CMON Controller completes its startup cycle ...
2025-05-30 06:18:22 UTC -- Create temporary cc setup user ...
User created.
2025-05-30 06:18:22 UTC -- *** Restarting the Controller process to␣
↪generate CMON API rpc_tls files.
Synchronizing state of cmon-events.service with SysV service script with /
↪usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cmon-events
Created symlink /
↪etc/systemd/system/multi-user.target.wants/cmon-events.service -> /
↪etc/systemd/system/cmon-events.service.
Synchronizing state of cmon-ssh.service with SysV service script with /
↪usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cmon-ssh
Created symlink /
↪etc/systemd/system/multi-user.target.wants/cmon-ssh.service -> /
↪etc/systemd/system/cmon-ssh.service.
Synchronizing state of cmon-cloud.service with SysV service script with /
↪usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cmon-cloud
Created symlink /
```

```
↪etc/systemd/system/multi-user.target.wants/cmon-cloud.service -> /
↪etc/systemd/system/cmon-cloud.service.
2025-05-30 06:18:43 UTC -- Configuring ClusterControl MCC / Ops-C ...
2025-05-30 06:18:43 UTC -- Configuring ccmgr (cmon-proxy) ...
ClusterControl Manager - admin CLI v2.2
Controller 127.0.0.1:9501 registered successfully
Changing port from 19051 to 443
Changing frontend_path from /app to /var/www/html/clustercontrol-mcc
File /var/www/html/clustercontrol-mcc/config.js updated successfully
Configuration /usr/share/ccmgr/ccmgr.yaml updated successfully
Please restart 'cmon-proxy' service to apply changes
2025-05-30 06:18:43 UTC -- ClusterControl MCC / Ops-C frontend installed .
↪..
2025-05-30 06:18:43 UTC -- ClusterControl installation completed!
Open your web browser to https://10.10.10.11 to start using␣
↪ClusterControl.
If you want to uninstall ClusterControl then please follow the␣
↪instructions here,␣
↪https://severalnines.com/docs/administration.html#uninstall
```

1.10.5 Step 5: Post-installation

Access the ClusterControl GUI at `https://<ClusterControl_host>` and create a new admin user.

Example



Once ClusterControl is up and running, import your existing cluster or deploy a new database cluster and start managing them from one place. Make sure SSH key-based authentication is configured from the ClusterControl node to the database nodes.

1. Generate an SSH key on the ClusterControl node:

```
ssh-keygen -t rsa # press Enter on all prompts
```

2. Setup SSH key-based authentication from ClusterControl to the database and load balancer nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa <os_user>@<IP address/hostname>
```

Repeat step 2 for every database/load balancer host that you are going to manage.

1.11 Troubleshooting Issues

1.11.1 Error page "Not Found" after installation

In highly restricted environments, after an offline installation of ClusterControl is complete, users may encounter a "Not Found" error when attempting to access the ClusterControl GUI at `https://{ClusterControl_host}/`. This issue typically arises due to incorrect directory permissions within `/var/www/html/clustercontrol-mcc`, mostly originated from a strict default umask.

Example

Check the permission for group and others in every directory under `/var/www/html/clustercontrol-mcc`. The directories' permission should be `drwxr-xr-x`. Run the following command on the ClusterControl server:

```
$ chmod 755 /var
$ chmod 755 /var/www
$ chmod 755 /var/www/html
$ chmod 755 /var/www/html/clustercontrol-mcc
```

Example

Here is an example output to verify the permission and ownership:

```
$ ls -al /var | grep www
drwxr-xr-x  3 root root   4096 Oct 23 15:26 www
$ ls -al /var/www | grep html
drwxr-xr-x  4 root root 4096 Oct 23 15:29 html
$ ls -al /var/www/html
total 16
drwxr-xr-x 4 root     root     4096 Oct 23 15:29 .
drwxr-xr-x 3 root     root     4096 Oct 23 15:26 ..
drwxr-xr-x 3 root     root     4096 Jan  9 12:44 clustercontrol-mcc
drwxr-x--- 2 www-data www-data 4096 Oct 23 15:29 cmon-repos
```

After the permission for group and others already are correct, restart the `cmon-proxy` service:

```
systemctl restart cmon-proxy
```

Open the ClusterControl GUI in the web browser to verify if the issue is resolved.

1.12 Manual Installation

If you want to have more control over the installation process, you may perform a manual installation. ClusterControl requires a number of packages to be installed and configured, as described in the following list:

- `clustercontrol-mcc` – ClusterControl graphical user interface (GUI).
- `clustercontrol-controller` – ClusterControl CMON controller.
- `clustercontrol-notifications` – ClusterControl notification module, to forward alarms and notifications to third-party tools like PagerDuty and Slack.
- `clustercontrol-ssh` – ClusterControl web-based SSH module, to access the host via SSH directly from ClusterControl GUI.
- `clustercontrol-cloud` – ClusterControl cloud module, to integrate with your cloud providers from ClusterControl GUI.
- `clustercontrol-proxy` – ClusterControl controller proxying service for ClusterControl web user interface (GUI).
- `clustercontrol-kuber-proxy` - ClusterControl module for integration to the Kubernetes environment.
- `clustercontrol-clud` – ClusterControl cloud file manager module, to upload and download backups from cloud storage. It requires `clustercontrol-cloud`.
- `s9s-tools` – ClusterControl command-line interface (CLI).

Note

Note Installing and uninstalling ClusterControl should not bring any downtime to the managed database cluster.

1.12.1 Requirements

Make sure the following is ready prior to this installation:

- The ClusterControl host must be running on the supported operating system. See *Operating System*.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*
- You must have an internet connection on the ClusterControl node during the installation process. Otherwise, see *Offline Installation*.

1.12.2 Installation Steps

The steps described in the following sections should be performed on the ClusterControl node unless specified otherwise.

Red Hat/CentOS/Rocky Linux/AlmaLinux

1. Set up *ClusterControl repository*.
2. Set up *ClusterControl CLI repository*.
3. Disable SElinux and open required ports (or stop firewall):

```
sed -i 's|SELINUX=enforcing|SELINUX=disabled|g' /etc/selinux/config
setenforce 0
systemctl stop firewalld
```

4. Install required packages via package manager:

```
dnf -y install wget dmidecode hostname python3 mariadb mariadb-server
alternatives --set python /usr/bin/python3
```

5. Install EPEL packages:

```
# RHEL/CentOS/Rocky/Alma 9
dnf -y install epel-release

# RHEL/Alma/Rocky 8
dnf config-manager --set-enabled powertools
dnf -y install epel-release epel-next-release
```

6. Install ClusterControl packages:

```
dnf -y install clustercontrol-mcc \
            clustercontrol-controller \
            clustercontrol-ssh \
            clustercontrol-notifications \
            clustercontrol-cloud \
            clustercontrol-clud \
            clustercontrol-proxy \
```

(continues on next page)

(continued from previous page)

```
            clustercontrol-kuber-proxy \
            s9s-tools
```

7. Start the MariaDB server, enable it on boot and set the database root password:

```
systemctl start mariadb
systemctl enable mariadb
mysqladmin -uroot password 'themysqlrootpassword'
```

8. Create a database user called `cmon`, and grant proper privileges:

```
mysql -uroot -p -e 'GRANT ALL PRIVILEGES ON *.* TO "cmon"@"localhost"␣
↪IDENTIFIED BY "{cmonpassword}" WITH GRANT OPTION'
mysql -uroot -p -e 'GRANT ALL PRIVILEGES ON *.* TO "cmon"@"127.0.0.1"␣
↪IDENTIFIED BY "{cmonpassword}" WITH GRANT OPTION'
mysql -uroot -p -e 'GRANT ALL PRIVILEGES ON *.* TO
↪"cmon"@"{controller_ip_address}" IDENTIFIED BY "{cmonpassword}" WITH␣
↪GRANT OPTION'
mysql -uroot -p -e 'FLUSH PRIVILEGES'
```

   Replace `{cmonpassword}` with respective value and `{controller_ip_address}` with the valid FQDN or IP address of the ClusterControl node.

9. Generate a ClusterControl key to be used by the `--rpc-token` option further below:

```
$ uuidgen | tr -d '-'
6856d96a19d049aa8a7f4a5ba57a34740b3faf57
```

10. Initialize the ClusterControl Controller service, `cmon` by running the following command:

```
cmon --init \
     --mysql-hostname="127.0.0.1" \
     --mysql-port="3306" \
     --mysql-username="cmon" \
     --mysql-password="{cmonpassword}" \
     --mysql-database="cmon" \
     --hostname="{ClusterControl Primary IP Address}" \
     --rpc-token="{ClusterControl API key as generated above}" \
     --controller-id="clustercontrol"
```

Example

```
$ cmon --init \
       --mysql-hostname="127.0.0.1" \
       --mysql-port="3306" \
       --mysql-username="cmon" \
       --mysql-password="xxxx" \
       --mysql-database="cmon" \
       --hostname="10.10.10.13" \
       --rpc-token="dcd17b14e88b47f8ac7f25cd85508fb0" \
       --controller-id="clustercontrol"
```

```
   The --init option received, initializing the controller.
   Verifying the Cmon Database...
   Cmon Database connect success, the database is not yet created,␣
↪ok.
   Checking the Cmon Database schema...
   Cmon Database does not exist, will be created now.
   Applying modifications from
↪'cmon_db.sql,cmon_db_mods_hotfix.sql,cmon_data.sql'.

   Verifying connection...
   Initializing the user manager.
   User manager is creating system users.
   Checking that the system users exist.
   Creating system groups: admins
   Creating system groups: users
   Creating system groups: nobody
   Creating system user.
   Creating nobody user.
  Creating admin user.
```

Warning

Attention

```
The value of the `hostname` must be either a valid FQDN or IP address of␣
↪the ClusterControl node. If the host has multiple IP addresses, pick␣
↪the primary IP address of the host. The `cmon` user password taken from␣
↪previous step when creating the user.
```

11. ClusterControl event and cloud modules require their service definition inside `/etc/default/cmon`. Create the file and add the following lines:

```
EVENTS_CLIENT="http://127.0.0.1:9510"
CLOUD_SERVICE="http://127.0.0.1:9518"
```

12. Initialize the ClusterControl web application to be started on port 443:

```
ccmgradm init --local-cmon -p 443 -f /var/www/html/clustercontrol-mcc
```

Example

```
$ ccmgradm init --local-cmon -p 443 -f /
↪var/www/html/clustercontrol-mcc
ClusterControl Manager - admin CLI v2.2
Controller 127.0.0.1:9501 registered successfully
Changing frontend_path from /app to /var/www/html/clustercontrol-mcc
File /var/www/html/clustercontrol-mcc/config.js updated successfully
Configuration /usr/share/ccmgr/ccmgr.yaml updated successfully
Please restart 'cmon-proxy' service to apply changes
```

Tip

If you want to use your own SSL certificate, update the `tls_key` and `tls_cert` values inside `/usr/share/ccmgr/ccmgr.yaml` accordingly.

13. Enable ClusterControl daemons on boot and start them:

```
systemctl enable cmon cmon-ssh cmon-events cmon-cloud cmon-proxy␣
↪kuber-proxy
systemctl start cmon cmon-ssh cmon-events cmon-cloud cmon-proxy␣
↪kuber-proxy
```

14. Create a user called `ccsetup`, for new registration purposes (if this user exists, ClusterControl GUI first run will default to registration page instead):

```
export S9S_USER_CONFIG=/tmp/ccsetup.conf
s9s user --create --new-password=admin --group=admins --email-address="
↪{your_email_address}" --controller="https://localhost:9501" ccsetup
unset S9S_USER_CONFIG
```

15. Open ClusterControl GUI at `https://<ClusterControl_host>/` and create the default admin user by specifying a username (username "admin" is reserved) and password on the welcome page.

16. Generate an SSH key to be used by ClusterControl when connecting to all managed hosts. In this example, we are using the root user to connect to the managed hosts. To generate an SSH key for the root user, do:

```
$ whoami
  root
$ ssh-keygen -t rsa # Press enter for all prompts
```

Note

Note If you are running as sudoer, the default SSH key will be located under `/home/$USER/.ssh/id_rsa`. See *Operating System User*.

17. Before creating or importing a database server/cluster into ClusterControl, set up passwordless SSH from the ClusterControl host to the database host(s). Use the following command to copy the SSH key to the target hosts:

```
ssh-copy-id -i ~/.ssh/id_rsa <SSH user>@<IP address of the target node>
```

Replace `<SSH user>` and `<IP address of the target node>` with appropriate values. Repeat the command for all target hosts.

The installation is complete and you can start to import existing or deploy a new database cluster. See *User Guide* to start using ClusterControl.

Debian/Ubuntu

The following steps should be performed on the ClusterControl node unless specified otherwise. Omit sudo if you are installing as the root user.

1. Set up *ClusterControl repository*.
2. Set up *ClusterControl CLI repository*.
3. If you have AppArmor running, disable it and open the required firewall ports (or stop iptables):

```
sudo systemctl stop apparmor
sudo systemctl disable apparmor
sudo systemctl mask apparmor
sudo systemctl stop ufw   # or nftables or iptables
```

4. Install ClusterControl dependencies:

```
sudo apt-get update
sudo apt-get install -y python3 software-properties-common␣
↪mysql-client mysql-server
update-alternatives --install /usr/bin/python python /usr/bin/python3 1
```

5. Install the ClusterControl package:

```
sudo apt-get install -y clustercontrol-controller \
     clustercontrol-mcc \
     clustercontrol-ssh \
     clustercontrol-notifications \
     clustercontrol-cloud \
     clustercontrol-clud \
     clustercontrol-proxy \
     clustercontrol-kuber-proxy \
     s9s-tools
```

6. Start MySQL server, enable it on boot and set a MySQL root password by using the `mysql_secure_installation` script:

```
systemctl start mysql
systemctl enable mysql
mysql_secure_installation
```

7. Create a database user called `cmon` and grant the right database privileges:

```
mysql -uroot -p -e 'GRANT ALL PRIVILEGES ON *.* TO "cmon"@"localhost"␣
↪IDENTIFIED BY "{cmonpassword}" WITH GRANT OPTION'
mysql -uroot -p -e 'GRANT ALL PRIVILEGES ON *.* TO "cmon"@"127.0.0.1"␣
↪IDENTIFIED BY "{cmonpassword}" WITH GRANT OPTION'
mysql -uroot -p -e 'GRANT ALL PRIVILEGES ON *.* TO
↪"cmon"@"{controller_ip_address}" IDENTIFIED BY "{cmonpassword}" WITH␣
↪GRANT OPTION'
mysql -uroot -p -e 'FLUSH PRIVILEGES'
```

8. Generate a ClusterControl key to be used by `--rpc-token` option further below:

```
$ uuidgen | tr -d '-'
6856d96a19d049aa8a7f4a5ba57a34740b3faf57
```

9. Initialize the ClusterControl Controller service called `cmon` by running the following command:

```
cmon --init \
     --mysql-hostname="127.0.0.1" \
     --mysql-port="3306" \
     --mysql-username="cmon" \
     --mysql-password="{cmonpassword}" \
     --mysql-database="cmon" \
     --hostname="{ClusterControl Primary IP Address}" \
     --rpc-token="{ClusterControl API key as generated above}" \
     --controller-id="clustercontrol"
```

Example

```
$ cmon --init \
       --mysql-hostname="127.0.0.1" \
       --mysql-port="3306" \
       --mysql-username="cmon" \
       --mysql-password="xxxx" \
       --mysql-database="cmon" \
       --hostname="10.10.10.13" \
       --rpc-token="dcd17b14e88b47f8ac7f25cd85508fb0" \
       --controller-id="clustercontrol"
   The --init option received, initializing the controller.
   Verifying the Cmon Database...
   Cmon Database connect success, the database is not yet created,␣
↪ok.
   Checking the Cmon Database schema...
   Cmon Database does not exist, will be created now.
   Applying modifications from
↪'cmon_db.sql,cmon_db_mods_hotfix.sql,cmon_data.sql'.

   Verifying connection...
   Initializing the user manager.
   User manager is creating system users.
   Checking that the system users exist.
   Creating system groups: admins
   Creating system groups: users
   Creating system groups: nobody
   Creating system user.
   Creating nobody user.
  Creating admin user.
```

Warning

Attention

```
The value of the `hostname` must be either a valid FQDN or IP address of␣
↪the ClusterControl node. If the host has multiple IP addresses, pick␣
↪the primary IP address of the host. The `cmon` user password taken from␣
↪previous step when creating the user.
```

10. ClusterControl's event and cloud modules require `/etc/default/cmon` for service definition. Create the file and add the following lines:

```
EVENTS_CLIENT="http://127.0.0.1:9510"
CLOUD_SERVICE="http://127.0.0.1:9518"
```

11. Initialize the ClusterControl web application to be started on port 443:

```
ccmgradm init --local-cmon -p 443 -f /var/www/html/clustercontrol-mcc
```

Example

```
$ ccmgradm init --local-cmon -p 443 -f /
↪var/www/html/clustercontrol-mcc
ClusterControl Manager - admin CLI v2.2
Controller 127.0.0.1:9501 registered successfully
Changing frontend_path from /app to /var/www/html/clustercontrol-mcc
File /var/www/html/clustercontrol-mcc/config.js updated successfully
Configuration /usr/share/ccmgr/ccmgr.yaml updated successfully
Please restart 'cmon-proxy' service to apply changes
```

Tip

If you want to use your own SSL certificate, update the `tls_key` and `tls_cert` values inside `/usr/share/ccmgr/ccmgr.yaml` accordingly.

12. Enable ClusterControl on boot and start them:

```
systemctl daemon-reload
systemctl enable cmon cmon-ssh cmon-events cmon-cloud cmon-proxy␣
↪kuber-proxy
systemctl restart cmon cmon-ssh cmon-events cmon-cloud cmon-proxy␣
↪kuber-proxy
```

13. Create a user called `ccsetup`, for new registration purposes (if this user exists, ClusterControl GUI first run will default to registration page instead):

```
export S9S_USER_CONFIG=/tmp/ccsetup.conf
s9s user --create --new-password=admin --group=admins --email-address="
```

(continues on next page)

(continued from previous page)

```
↪{your_email_address}" --controller="https://localhost:9501" ccsetup
unset S9S_USER_CONFIG
```

14. Open ClusterControl GUI at `https://<ClusterControl_host>/` and create the default admin user by specifying a username (username "admin" is reserved) and password on the welcome page.

15. Generate an SSH key to be used by ClusterControl when connecting to all managed hosts. In this example, we are using the "root" user to connect to the managed hosts. To generate an SSH key for the root user, do:

```
$ whoami
  root
$ ssh-keygen -t rsa # Press enter for all prompts
```

> **Note**
>
> Note If you are running as sudoer, the default SSH key will be located under `/home/$USER/.ssh/id_rsa`. See *Operating System User*.

16. Before importing a database server/cluster into ClusterControl or deploy a new cluster, set up passwordless SSH from ClusterControl host to the database host(s). Use the following command to copy the SSH key to the target hosts:

```
ssh-copy-id -i ~/.ssh/id_rsa <SSH user>@<IP address of the target node>
```

Replace `<SSH user>` and `<IP address of the target node>` with appropriate values. Repeat the command for all target hosts.

The installation is complete and you can start to import existing or deploy a new database cluster. See *User Guide* to start using ClusterControl.

1.12.3 Troubleshooting Issues

Failed to create `ccsetup` user

In some cases, the `ccsetup` user was failed to be created with the following error:

```
s9s user --create --new-password=admin --group=admins --email-address=
↪"user@example.com" --controller="https://localhost:9501" ccsetup
Connect to localhost:9501 failed(111): Connection refused.
```

Check the ClusterControl Controller log messages in `/var/log/cmon.log`, and see if there is the following error:

```
2025-08-21T07:33:33.296Z : (INFO) Checking if CmonDb access is working␣
↪properly.
2025-08-21T07:33:33.297Z : (WARNING) Cmon DB connection error: Access␣
↪denied for user 'cmon'@'localhost' (using password: YES) (errno: 1045)
2025-08-21T07:33:33.298Z : (INFO) CmonDb connection or query failure. Error␣
↪code: 1045, Message: No connection (Access denied for user 'cmon'@
↪'localhost' (using password: YES))
```

(continues on next page)

(continued from previous page)

```
2025-08-21T07:33:33.298Z : (INFO) Lets block and wait for working CmonDb␣
↪connection.
```

Remove the existing `/etc/cmon.cnf` configuration, and run again `cmon --init` script.

```
$ rm -rf /etc/cmon.cnf
$ cmon --init \
     --mysql-hostname="127.0.0.1" \
     --mysql-port="3306" \
     --mysql-username="cmon" \
     --mysql-password="xxxx" \
     --mysql-database="cmon" \
     --hostname="10.10.10.13" \
     --rpc-token="dcd17b14e88b47f8ac7f25cd85508fb0" \
     --controller-id="clustercontrol"
The --init option received, initializing the controller.
Generating cmon configuration...
------8<------8<------8<------8<------8<------8<------8<------
#
# Configuration file for the Cmon Controller.
#

#
# The name or IP address of the Cmon Controller.
#
hostname=10.10.10.13

#
# Cmon Database credentials. The controller will use
# this database to store its own data structures.
#
mysql_hostname=127.0.0.1
mysql_port=3306
mysql_password='xxxx'
cmon_user=cmon
cmon_db=cmon
rpc_key=dcd17b14e88b47f8ac7f25cd85508fb0

------8<------8<------8<------8<------8<------8<------8<------
Verifying the Cmon Database...
Cmon Database connect is successful, database exists.
Checking the Cmon Database schema...
Applying modifications from
↪'cmon_db_mods-2.2.3-2.3.2.sql,cmon_db_mods_hotfix.sql'.

Verifying connection...
Initializing the user manager.
User manager is creating system users.
Checking that the system users exist.
```

After that, re-create the `ccsetup` user using ClusterControl CLI:

```
s9s user --create --new-password=admin --group=admins --email-address=
↪"user@example.com" --controller="https://localhost:9501" ccsetup
```

Error page "Not Found" after installation

In highly restricted environments, after a manual installation of ClusterControl is complete, users may encounter a "Not Found" error when attempting to access the ClusterControl GUI at `https://{ClusterControl_host}/`. This issue typically arises due to incorrect directory permissions within `/var/www/html/clustercontrol-mcc`, mostly originated from a strict default umask.

Example

Check the permission for group and others in every directory under `/var/www/html/clustercontrol-mcc`. The directories' permission should be `drwxr-xr-x`. Run the following command on the ClusterControl server:

```
$ chmod 755 /var
$ chmod 755 /var/www
$ chmod 755 /var/www/html
$ chmod 755 /var/www/html/clustercontrol-mcc
```

Example

Here is an example output to verify the permission and ownership:

```
$ ls -al /var | grep www
drwxr-xr-x  3 root root   4096 Oct 23 15:26 www
$ ls -al /var/www | grep html
drwxr-xr-x  4 root root 4096 Oct 23 15:29 html
```

```
$ ls -al /var/www/html
total 16
drwxr-xr-x 4 root     root     4096 Oct 23 15:29 .
drwxr-xr-x 3 root     root     4096 Oct 23 15:26 ..
drwxr-xr-x 3 root     root     4096 Jan  9 12:44 clustercontrol-mcc
drwxr-x--- 2 www-data www-data 4096 Oct 23 15:29 cmon-repos
```

After the permission for group and others already are correct, restart the `cmon-proxy` service:

Severalnines provides RPM-based (yum/dnf) and DEB-based package repositories accessible at https://repo.severalnines.com. The automatic installation procedure (as described in *Online Installation*) automatically configures the ClusterControl node with the respective package repository.

1.13 ClusterControl repository

ClusterControl repository offers ClusterControl-related packages that should be installed on the ClusterControl server. The repository provides the following packages:

- ClusterControl Controller
- ClusterControl GUI
- ClusterControl Notifications
- ClusterControl Web SSH
- ClusterControl Cloud
- ClusterControl Cloud File Manager
- ClusterControl Ops-C
- ClusterControl Proxy
- ClusterControl Agent

yum/dnf

1. Add the Severalnines repository public key into your RPM keyring:

```
rpm --import http://repo.severalnines.com/severalnines-repos.asc
```

2. You can download the repository definition from Severalnines download page:

```
wget http://www.severalnines.com/downloads/cmon/s9s-repo.repo -P /
↪etc/yum.repos.d/
```

Or, you can create the repository definition manually:

```
cat - > /etc/yum.repos.d/s9s-repo.repo << REPOEND
[s9s-repo]
name = Severalnines Release Repository
baseurl = http://repo.severalnines.com/rpm/os/x86_64
enabled = 1
gpgkey = http://repo.severalnines.com/severalnines-repos.asc
```

(continues on next page)

(continued from previous page)

```
gpgcheck = 1
REPOEND
```

3. Look for ClusterControl packages:

```
yum search clustercontrol
```

apt

1. Add Severalnines repository public key into your APT keyring:

```
apt-get -y install gnupg2
wget http://repo.severalnines.com/severalnines-repos.asc -O- | sudo␣
↪apt-key add -
```

2. You can download the repository definition from Severalnines download page:

```
sudo wget http://www.severalnines.com/downloads/cmon/s9s-repo.list -P /
↪etc/apt/sources.list.d/
```

Or, add the Severalnines APT source list manually:

```
echo 'deb [arch=amd64] http://repo.severalnines.com/deb ubuntu main' |␣
↪sudo tee /etc/apt/sources.list.d/s9s-repo.list
```

3. Update package list:

```
sudo apt-get update
```

4. Look for ClusterControl packages:

```
sudo apt-cache search clustercontrol
```

Manual download

Severalnines offers a dedicated download page for manual download at Severalnines Download page. Use the filter box to find the right packages and versions quickly.

1.14 ClusterControl CLI repository

ClusterControl CLI repository offers ClusterControl client-related packages that should be installed on the client host or workstation. It supports more operating systems than the *ClusterControl repository*. The repository provides the following packages:

- ClusterControl CLI (`s9s`)
- ClusterControl CLI library

Package repository (yum/dnf/apt)

Use the `install-s9s-tools.sh` script to install ClusterControl CLI and its library. The script will detect the operating system, configure the necessary GPG key, repository definition, download and install the necessary packages and also initialize the CLI user.

1. Download the repository installer script from here:

```
wget https://repo.severalnines.com/s9s-tools/install-s9s-tools.sh
```

2. Run the installer script:

```
bash install-s9s-tools.sh
```

Manual download

To repository page is accessible publicly at https://repo.severalnines.com/s9s-tools/. You can download the ClusterContol CLI packages directly from there.

1.15 Legacy GUI Installation

Starting from ClusterControl 1.9.7 (September 2023), ClusterControl GUI v2 is the default frontend graphical user interface for ClusterControl, accessible via `https://{ClusterControl_host}/`. For new installations, the installer script will only install GUI v2, skipping GUI v1. If you would like to have GUI v1, follow the instructions as written in this article.

1.15.1 Caveats

Before continuing with the installation, do mind the following important points:

- ClusterControl GUI v1 requires PHP. It only supports the outdated PHP 7. It does not support the latest version of PHP 8 at the time of this writing. If the operating system comes with PHP 8 installed by default, you have to downgrade to PHP 7 to continue using it. ClusterControl GUI v2 does not have this requirement.
- It requires a MySQL/MariaDB database called "dcps".
- Only the Apache web server is supported due to its dependency on .htaccess and Apache configuration files.
- ClusterControl GUI v1 has reached the end of the development cycle and is considered a feature-freeze product. All new developments will be happening on ClusterControl GUI v2.
- To run ClusterControl GUI v1 on a Red Hat-based distribution, SELinux must be turned off. The installer script only configures the SELinux policy for ClusterControl GUI v2.

1.15.2 Installation steps

The following installation steps are written based on Ubuntu 22.04 LTS and Rocky Linux 9.2. It is assumed that you have already performed a ClusterControl installation using the installer script, `install-cc` which by default would give you ClusterControl GUI v2. See the *Quickstart* section for details.

Rocky Linux 9.2

1. If you have created an admin user during the first login of ClusterControl GUI v2, proceed to step 2. Otherwise, create a ClusterControl admin user if you haven't done so by accessing ClusterControl GUI v2.

2. Disable SELinux by configuring the following line inside `/etc/selinux/config`:

```
SELINUX=disabled
```

   Deactivate SELinux during runtime:

```
setenforce 0
```

3. Install ClusterControl GUI v1 package:

```
dnf install -y clustercontrol
```

4. Copy the provided Apache configuration files to the Apache configuration directory:

```
cp -f /var/www/html/clustercontrol/app/tools/apache2/s9s*.conf /
↪etc/httpd/conf.d/
```

5. Add `Listen {HTTPS port}` at the top of the `s9s-ssl.conf` file (note that port 443 is already taken by GUI v2, so we have to pick another port):

```
vim /etc/httpd/conf.d/s9s-ssl.conf
```

   Add the following in the first line:

```
Listen 9443
```

   In the same file, configure the same HTTPS port as defined above on line 3:

```
<VirtualHost _default_:9443>
```

   Also in the same file, configure the SSL certificate and key with the provided cert and key (lines 55 and 56, after you have added the first line above):

```
            SSLCertificateFile      /etc/ssl/certs/s9server.crt
            SSLCertificateKeyFile /etc/ssl/private/s9server.key
```

   Restart the Apache web server to apply the changes:

```
systemctl restart httpd
```

6. Check the provided PHP module of the operating system. For Rocky Linux 9.2, the PHP module provided by the default AppStream repository is 8.1, which is not supported. We can check using the following command:

```
dnf module list php
```

7. If PHP7 is unavailable as in this case, we will configure the Remi repository:

```
dnf install dnf-utils http://rpms.remirepo.net/enterprise/remi-release-
↪9.rpm -y
```

8. Verify that the PHP 7 module is listed (in this example, it is `remi-7.4`):

```
$ dnf module list php -y

Rocky Linux 9 - AppStream
Name            Stream              Profiles                            ␣
↪Summary
php             8.1                 common [d], devel, minimal          ␣
↪PHP scripting language

Remi's Modular repository for Enterprise Linux 9 - x86_64
Name            Stream              Profiles                            ␣
↪Summary
php             remi-7.4            common [d], devel, minimal          ␣
↪PHP scripting language
php             remi-8.0            common [d], devel, minimal          ␣
↪PHP scripting language
php             remi-8.1            common [d], devel, minimal          ␣
↪PHP scripting language
php             remi-8.2            common [d], devel, minimal          ␣
↪PHP scripting language
php             remi-8.3            common [d], devel, minimal          ␣
↪PHP scripting language
```

9. Switch PHP to use `remi-7.4` module:

```
dnf module -y switch-to php:remi-7.4
```

10. Install the required PHP7 packages:

```
dnf install -y php php-mysql php-ldap php-gd php-curl php-xml php-fpm
```

11. Start the php-fpm service and restart Apache web server:

```
systemctl start php-fpm
systemctl enable php-fpm
systemctl restart httpd
```

12. Create `bootstrap.php` from the provided template:

```
cp /var/www/html/clustercontrol/bootstrap.php.default /
↪var/www/html/clustercontrol/bootstrap.php
```

13. Configure `/var/www/html/clustercontrol/bootstrap.php` and set the password, `DBPASS`, port, `DBPORT` and `RPCTOKEN` accordingly:

```
define('DB_PASS', 'DBPASS');
define('DB_PORT', 'DBPORT');
define('RPC_TOKEN','RPCTOKEN');
```

> **Note**
>
> Note

Set the password and port used when installing CCv2 for the cmon MySQL user. You can also retrieve the information from `/etc/cmon.cnf` (including the RPC token). ::: :::{admonition} Example output :class: dropdown

```
```php
...
define('DB_HOST', '127.0.0.1');
define('DB_LOGIN', 'cmon');
define('DB_PASS', 'PjsL23&H7dNsN$b');
define('DB_NAME', 'dcps');
define('DB_PORT', '3306');
...
define('RPC_TOKEN','80e0a3c8894e931f8b26d6f196c52a9818e513ba');
```
:::
```

14. Login to MySQL and create the 'dcps' database:

```
mysql -ucmon -p -e 'CREATE DATABASE dcps'
```

> **Note**
>
> Note

We skip configuring MySQL user privilege since the "cmon" MySQL user has a global admin privilege. :::
15. Import the `dcps` database schema structure from the provided SQL file:

```
```bash
mysql -ucmon -p dcps < /var/www/html/clustercontrol/sql/dc-schema.sql
```
```

16. Finally, add the following database flag to skip the registration and admin user creation page since we have done it via ClusterControl GUI v2 when creating the first admin user:

```
$ mysql -ucmon -p
mysql> INSERT INTO dcps.settings (name, type) VALUES (
↪'registration_complete', 'settings');
```

17. The installation is now complete. ClusterControl GUI v1 should be accessible at `https://{ClusterControl_host}:9443/` . The final thing to do is to import the existing clusters (if you have any) into ClusterControl GUI v1, by executing this script:

```
#!/bin/bash

function import() {
for i in /etc/cmon.d/cmon_*.cnf; do
```

(continues on next page)

(continued from previous page)

```
    NAME=$(grep ^name= $i | cut -d "=" -f2 | tr -d "'")
    TYPE=$(grep ^cluster_type= $i | cut -d "=" -f2)
    TOKEN=$(grep ^rpc_key= $i | cut -d "=" -f2 | tr -d "'")
    CREATED_BY=$(grep ^owner= $i | cut -d "=" -f2)
    CID=$(grep ^cluster_id= $i | cut -d "=" -f2)
    DBUSER=cmon
    DBPASS=$(grep ^mysql_password= $i | cut -d "=" -f2 | tr -d "'")


    echo "Importing cluster ID = $CID, from $i"
    MYSQL_PWD="$DBPASS" mysql -u$DBUSER -f -A -Bse \
        "INSERT IGNORE INTO dcps.clusters SET company_id = 1, \
            id = ${CID}, \
            cluster_id = ${CID}, \
            name = '${NAME}', \
            token = '${TOKEN}', \
            cmon_db = 'cmon', \
            type = '${TYPE}', \
            status = 100, \
            created_by = $CREATED_BY"
    retval=$?

    if [ $retval -eq 0 ]; then
        echo 'OK'
    else
        echo 'ERROR. Fix the script and try again.'
        exit 1
    fi
done

echo 'Import complete.'
exit 0
}

echo 'Use this script to import clusters into ClusterControl GUI v1'
echo 'after GUI v1 is installed and running (v1.9.7 only).'
echo 'This script can be run multiple times if any error occurs.'
echo
read -p "Do you wish to continue? [Y/n] " yn
case $yn in
        [Yy]* ) import; break;;
        * ) echo 'Exiting..'; exit 1;;
esac
```

Copy the above code inside the ClusterControl server and save it as `import_cluster_guiv1.sh`. Example output as below:

```
$ vim import_cluster_guiv1.sh # paste the code & save
$ chmod 755 import_cluster_guiv1.sh
```

(continues on next page)

(continued from previous page)

```
$ ./import_cluster_guiv1.sh
Use this script to import clusters into ClusterControl GUI v1
after GUI v1 is installed and running (v1.9.7 only).
This script can be run multiple times if any error occurs.

Do you wish to continue? [Y/n] Y
Importing cluster ID = 2, from /etc/cmon.d/cmon_2.cnf
OK
Importing cluster ID = 3, from /etc/cmon.d/cmon_3.cnf
OK
Importing cluster ID = 4, from /etc/cmon.d/cmon_4.cnf
OK
Importing cluster ID = 7, from /etc/cmon.d/cmon_7.cnf
OK
```

Ubuntu 22.04

1. If you have created an admin user during the first login of ClusterControl GUI v2, proceed to step 2. Otherwise, create a ClusterControl admin user if you haven't done so by accessing ClusterControl GUI v2.
2. Install ClusterControl GUI v1 package:

```
apt update -y
apt install -y clustercontrol
```

3. Copy the provided Apache configuration files to the Apache configuration directory:

```
cp -f /var/www/html/clustercontrol/app/tools/apache2/s9s*.conf /
↪etc/apache2/sites-enabled/
```

4. Add `Listen {HTTPS port}` at the top of the `s9s-ssl.conf` file (note that port 443 is already taken by GUI v2, so we have to pick another port):

```
vim /etc/apache2/sites-enabled/s9s-ssl.conf
```

Add the following in the first line:

```
Listen 9443
```

In the same file, configure the same HTTPS port as defined above on line 3:

```
<VirtualHost _default_:9443>
```

Also in the same file, configure the SSL certificate and key with the provided cert and key (lines 55 and 56, after you have added the first line above):

```
            SSLCertificateFile      /etc/ssl/certs/s9server.crt
            SSLCertificateKeyFile /etc/ssl/private/s9server.key
```

Restart the Apache web server to apply the changes:

```
systemctl restart apache2
```

5. By default, the installer script configures the PHP7 repository called “ondrej”. You may verify by using the following command:

```
apt-cache policy | grep ondrej
```

6. If the repository is not listed in the above output, you may install it by using the following commands:

```
apt install -y software-properties-common apt-transport-https
add-apt-repository -y ppa:ondrej/php
apt update -y
```

7. Install the required PHP7 packages:

```
apt install -y php7.4-mysql php7.4-gd libapache2-mod-php7.4␣
↪php7.4-curl php7.4-ldap php7.4-xml
```

Note

Note

By default, the installer script will install the above packages. In that case, the above command will return those packages as installed. ::: 8. Create `bootstrap.php` from the provided template:

```
```bash
cp /var/www/html/clustercontrol/bootstrap.php.default /
↪var/www/html/clustercontrol/bootstrap.php
```
```

9. Configure `/var/www/html/clustercontrol/bootstrap.php` and set the password, `DBPASS`, port, `DBPORT` and `RPCTOKEN` accordingly:

```
define('DB_PASS', 'DBPASS');
define('DB_PORT', 'DBPORT');
define('RPC_TOKEN','RPCTOKEN');
```

Note

Note

Set the password and port used when installing CCv2 for the cmon MySQL user. You can also retrieve the information from `/etc/cmon.cnf` (including the RPC token). ::: :::{admonition} Example output :class: dropdown

```
```php
...
define('DB_HOST', '127.0.0.1');
define('DB_LOGIN', 'cmon');
```

(continues on next page)
```

(continued from previous page)

```
define('DB_PASS', 'PjsL23&H7dNsN$b');
define('DB_NAME', 'dcps');
define('DB_PORT', '3306');
...
define('RPC_TOKEN','80e0a3c8894e931f8b26d6f196c52a9818e513ba');
```
:::

10. Login to MySQL and create the 'dcps' database:

```
mysql -ucmon -p -e 'CREATE DATABASE dcps'
```

> **Note**
>
> Note

We skip configuring MySQL user privilege since the "cmon" MySQL user has a global admin privilege. :::
11. Import the `dcps` database schema structure from the provided SQL file:

```
```bash
mysql -ucmon -p dcps < /var/www/html/clustercontrol/sql/dc-schema.sql
```
```

12. Finally, add the following database flag to skip the registration and admin user creation page since we have done it via ClusterControl GUI v2 when creating the first admin user:

```
$ mysql -ucmon -p
mysql> INSERT INTO dcps.settings (name, type) VALUES (
↪'registration_complete', 'settings');
```

13. The installation is now complete. ClusterControl GUI v1 should be accessible at `https://{ClusterControl_host}:9443/` . The final thing to do is to import the existing clusters (if you have any) into ClusterControl GUI v1, by executing this script:

```
#!/bin/bash

function import() {
for i in /etc/cmon.d/cmon_*.cnf; do
    NAME=$(grep ^name= $i | cut -d "=" -f2 | tr -d "'")
    TYPE=$(grep ^cluster_type= $i | cut -d "=" -f2)
    TOKEN=$(grep ^rpc_key= $i | cut -d "=" -f2 | tr -d "'")
    CREATED_BY=$(grep ^owner= $i | cut -d "=" -f2)
    CID=$(grep ^cluster_id= $i | cut -d "=" -f2)
    DBUSER=cmon
    DBPASS=$(grep ^mysql_password= $i | cut -d "=" -f2 | tr -d "'")


    echo "Importing cluster ID = $CID, from $i"
    MYSQL_PWD="$DBPASS" mysql -u$DBUSER -f -A -Bse \
```

(continues on next page)

(continued from previous page)

```
        "INSERT IGNORE INTO dcps.clusters SET company_id = 1, \
            id = ${CID}, \
            cluster_id = ${CID}, \
            name = '${NAME}', \
            token = '${TOKEN}', \
            cmon_db = 'cmon', \
            type = '${TYPE}', \
            status = 100, \
            created_by = $CREATED_BY"
    retval=$?

    if [ $retval -eq 0 ]; then
        echo 'OK'
    else
        echo 'ERROR. Fix the script and try again.'
        exit 1
    fi
done

echo 'Import complete.'
exit 0
}

echo 'Use this script to import clusters into ClusterControl GUI v1'
echo 'after GUI v1 is installed and running (v1.9.7 only).'
echo 'This script can be run multiple times if any error occurs.'
echo
read -p "Do you wish to continue? [Y/n] " yn
case $yn in
        [Yy]* ) import; break;;
        * ) echo 'Exiting..'; exit 1;;
esac
```

Copy the above code inside the ClusterControl server and save it as `import_cluster_guiv1.sh`. Example output as below:

```
$ vim import_cluster_guiv1.sh # paste the code & save
$ chmod 755 import_cluster_guiv1.sh

$ ./import_cluster_guiv1.sh
Use this script to import clusters into ClusterControl GUI v1
after GUI v1 is installed and running (v1.9.7 only).
This script can be run multiple times if any error occurs.

Do you wish to continue? [Y/n] Y
Importing cluster ID = 2, from /etc/cmon.d/cmon_2.cnf
OK
Importing cluster ID = 3, from /etc/cmon.d/cmon_3.cnf
OK
Importing cluster ID = 4, from /etc/cmon.d/cmon_4.cnf
```

(continues on next page)

(continued from previous page)

```
OK
Importing cluster ID = 7, from /etc/cmon.d/cmon_7.cnf
OK
```

1.16 ClusterControl on Marketplaces

An online/cloud marketplace provides users with access to applications and services that are built on, integrated with, or complement the service provider's offering. Most offer pre-configured one-click provisioning of applications and services.

ClusterControl will be offered on a number of marketplaces, allowing you to easily deploy a ClusterControl instance that you can use to manage your database nodes on that service provider's platform. The marketplace requirements on how to create an application differ and in some cases this will require that a few extra steps are taken by you to get started.

1.16.1 Current Marketplaces

- DigitalOcean
- gridscale.io
- Vultr
- Linode
- Google Cloud Platform
- Stay tuned … more to come!

1.16.2 Passwordless SSH connections

ClusterControl requires password-less SSH access to the hosts where the database nodes will be managed. The ClusterControl application can in some marketplaces automatically generate a private and public SSH key for the root user while in others you will need to perform this step manually.

The public SSH key is needed when you launch your hosts to be used for the database nodes. It must be included/selected as one of the SSH keys that will be used when the cloud instances are created otherwise ClusterControl will not be able to connect and manage the database nodes.

1.16.3 Launching the ClusterControl Application

On all marketplaces, there is a 1-click button to launch ClusterControl. You are then presented with a launch form where you might need to provide the following:

- OS image to use – Only Ubuntu 18.04 and Ubuntu 20.04 is supported for the ClusterControl node.
- MySQL root user password – If there is no input field for it then a random password will be generated.
- CMON (DB) user password – If there is no input field for it then a random password will be generated.
- Public SSH key – The SSH key to use when launching the image. This is not the same SSH key that ClusterControl will need later.
- Enable private IP – Enable the use of private IP if that option is shown.

1.16.4 Getting the Public SSH Key

On some marketplaces, the ClusterControl image is allowed to generate a password-less SSH key automatically at boot up. You can then copy the SSH public key by SSH into the ClusterControl instance where the public SSH key is displayed on login.

If that is not the case you will need to generate the password-less SSH key for the default SSH user by SSH into the ClusterControl instance and perform the following (ex for a root user):

```
$ ssh-keygen -b 2048 -t rsa -f /root/.ssh/id_rsa -q -N ""
$ ssh-keygen -y -f /root/.ssh/id_rsa > /root/.ssh/id_rsa.pub
$ cat /root/.ssh/id_rsa.pub
```

Copy the public SSH key and use it later when the hosts/cloud instances for the database nodes are provisioned.

1.16.5 Launching the Hosts for the Database Nodes

Depending on the cloud provider that you are using, you can add the previously created public SSH key to your 'security settings' or add it when the cloud instances are launched. The selected public SSH key(s) will be inserted into the cloud instance's `~/.ssh/authorized_keys` file so that the ClusterControl instance is able to SSH into the host.

You will also need to make sure that the cloud instances are on the same private network as the ClusterControl instance and enable 'private IPs' if available when launching the cloud instances.

1.16.6 Deploying a Database Cluster

After you have launched your cloud instances for the database nodes then proceed to open your web browser to `https://<ClusterControl_instance_IP_address>` and enter the **User registration** information for the initial admin user.

Start to deploy a database cluster onto the previously launched hosts by following the *deployment guide*.

1.17 Tutorials

This section provides hands-on tutorials to help you master ClusterControl. Each tutorial is designed to give you practical experience and a deeper understanding of ClusterControl's key features and workflows.

1.17.1 Day-1 Operations

"Your First Cluster" Tutorial Series

MySQL/MariaDB

- *Standalone →*
- *Replication →*
- *Group Replication →*
- *Percona XtraDB Cluster →*
- *MariaDB Cluster →*

PostgreSQL/TimescaleDB

- *PostgreSQL Streaming Replication →*
- *TimescaleDB Streaming Replication →*
- *PostgreSQL Logical Replication →*

MongoDB

- *MongoDB Replica Set →*
- *MongoDB Enterprise Replica Set →*
- *MongoDB Sharded Cluster →*

Redis/Valkey

- *Redis Sentinel →*
- *Redis Cluster →*
- *Valkey Sentinel →*
- *Valkey Cluster →*

SQL Server

- *Standalone →*
- *Always-On Availability Group →*

Elasticsearch

- *Single-node Cluster →*
- *High Availability Cluster →*

See also

- *Deploy Database Cluster in Offline Environment →*

1.17.2 Day-2 Operations

See also

- *Deploy and Failover to Secondary Cluster →*
- *Perform Major Upgrade for PostgreSQL →*
- *Backup to and Restore from Cloud →*
- *Database Point-in-time Recovery →*
- *Connect to ClusterControl RPC API as Read-only User →*

- *Migrating Existing Clusters from ClusterControl v1 to ClusterControl v2 →*
- *Managing Clusters using ClusterControl MCP →*

1.18 Standalone MySQL (MariaDB, Percona, Oracle)

This guide will take you through the process of setting up your first single-node MySQL/MariaDB/Percona server using ClusterControl. By following this tutorial, you'll have a fully functional (but non-replicated) database node that you can monitor, manage, and later scale if your requirements change.

A standalone MySQL/MariaDB server is the simplest deployment which perfect for development, testing, or lightweight production workloads. If you decide to expand to replication or a high availability (HA) cluster, you can do so seamlessly from this single-node foundation.

This tutorial applies to the following MySQL variants:

- MariaDB Server
- Percona Server for MySQL
- Percona Pro Server for MySQL
- Oracle MySQL Community Server

See also

- MySQL (Oracle) documentation
- MariaDB documentation
- Percona Server for MySQL documentation
- Percona Distribution for MySQL documentation

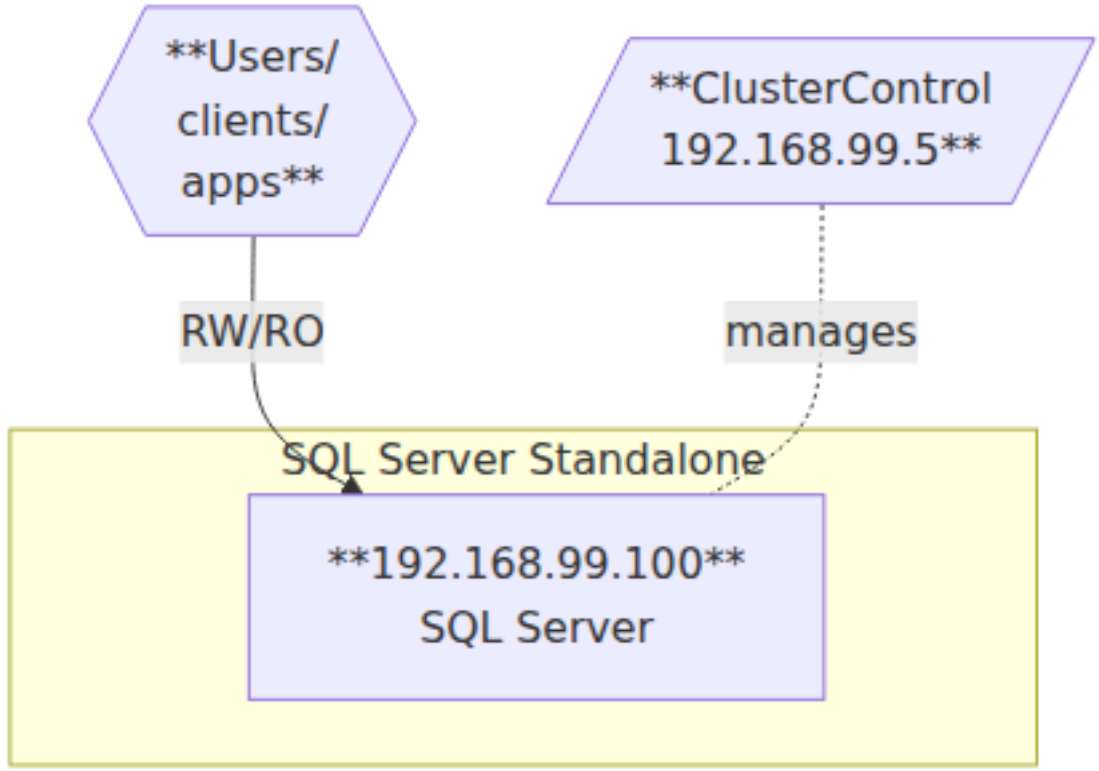
1.18.1 Prerequisites

Before proceeding, ensure you have:

- ClusterControl installed and running. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- At least two hosts (bare-metal or virtual machines):
  - One for the ClusterControl server.
  - One for the MySQL/MariaDB server.
- SSH access to all servers.
- Internet access on the database host to install required packages.
- Network Time Protocol (NTP) is configured and running on both hosts to keep their clocks synchronized.

1.18.2 Architecture

Below is a simplified diagram of the final architecture for a standalone deployment:



No load balancers or replicas are present here. You have a single MySQL/MariaDB server that directly serves both read and write requests from your applications.

1.18.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

Copy the public key to your standalone database node (replace 192.168.99.101 with your node's IP/hostname):

```
```bash
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101
```
```

If the target node uses a custom SSH key or port, you can add options:

```
```bash
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.101
```
```

2. Test passwordless SSH from the ClusterControl server:

```
ssh root@192.168.99.101 "ls /root"
```

Ensure there is no password prompt. If the command returns directory contents, you're set.

1.18.4 Step 2: Deploy a new cluster (single node)

1. Open a web browser and go to the ClusterControl server's IP or hostname.
2. On the ClusterControl dashboard, click ***Deploy a cluster (top-right) → Create a database cluster***. This opens the **Deploy a cluster** panel.
3. Choose "MySQL Replication" from the **Database** dropdown, choose "MariaDB" or your preferred vendor from the **Vendor** dropdown and also the database major version from the **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Replication** wizard, configure the database cluster as below:

Deploy MySQL Replication

(1) Cluster details

- **Name**: For example, `Standalone MySQL`.
- **Tags**: (Optional) e.g., `standalone`, `production`, `dc1`.

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: `22`
- **SSH sudo password**: (leave blank if you rely on key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: `3306` (default)
- **Server data directory**: `/var/lib/mysql` (default)
- **Admin/Root user**: `root`
- **Admin/Root password**: Password to be assigned to the database `root` user
- **Repository**: Use vendor repositories (default)
- **Version**: Choose the minor version for your selected vendor
- **Configuration template**: A default `my.cnf` or `my.cnf.replication`
- **Enable SSL encryption**: Optional (default off if not required)

(4) Add node

- **Standalone node**: Enter `192.168.99.101` and press ++enter++.
- Wait for the node to turn **green**. If an error (red) appears, check the logs in Activity center for details.

(5) Preview

- Review your configuration. You can go back and adjust if needed.

5. Click **Finish** to start deployment.
6. ClusterControl will install and configure MySQL on 192.168.99.101. You can track progress in the **Activity center**. After a few minutes, your new single-node cluster will appear on the **Home** page.

1.18.5 Step 3: Monitor your cluster

Once deployed, you'll see:

- Cluster health: The **Home** page provides the cluster state, even though it's a single node, it's treated as a "cluster" of size one.
- Node health: Hover on the honeycomb diagram or check the Nodes tab. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → {cluster} → Dashboards***.
- Recent alarms: Any triggered alarms will appear if there are configuration or resource issues.
- Automatic recovery status: If enabled, ClusterControl can attempt to restart a crashed MySQL server automatically.
- Topology viewer: You'll see a very simple topology with only one node.

1.18.6 Step 4: Import data

1. You can import data into the standalone server in a variety of ways:

Using MySQL dump

```
mysql -u root -p -h database_name < dumpfile.sql
```

Using `LOAD DATA INFILE`

```
LOAD DATA INFILE '/path/to/file.csv'
INTO TABLE table_name
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

2. If you have a large dataset (hundreds of GB), ensure you have enough disk space and memory.

1.18.7 Step 5: (Optional) Deploy a new load balancer

A load balancer (e.g., ProxySQL, HAProxy) is not necessary for a single-node environment. Typically, it is used for multi-node or high availability setups to split traffic or handle failover. You can skip this entirely if you only have one node.

Tip

If you plan to add replicas or switch to a high-availability cluster later, a load balancer can be introduced at that point. See *Create Load Balancers* for more info.

1.18.8 Step 6: Connect to the database server

Your application or client will connect directly to the standalone server:

- Host: 192.168.99.101
- Port: 3306
- User/Password: The credentials specified during the deployment

1.18.9 Step 7: Enable automatic backups

Even for a single node, regular backups are critical. You can create scheduled backups and optionally enable point-in-time recovery (PITR) by keeping binary logs.

1. Go to ***ClusterControl GUI → {cluster} → Backups***.
2. Click ***Create Backup → Schedule a Backup***. The Create a backup schedule wizard will open and proceed to configure your backup as below:

***Create Backup → Schedule a Backup**

(1) Configuration

- **Schedule name**: `Daily backup - mysqldump`
- **Backup method**: `mysqldump` (or `xtrabackup`/`mariabackup` for physical)
- **Dump type**: `Complete PITR-compatible`
- **Upload backup to cloud**: Off (default)

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: `6`
- **Enable encryption**: (optional)
- **Verify backup**: Off (default)
- **Retention**: On (default)
- **Failover backup if node is down**: On (default)

(3) Storage

- **Store on controller**: (default)
- **Storage directory**: `/root/backups` (default)
- **Backup subdirectory**: `BACKUP-%I` (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at `02:00`
- **Timezone**: your local timezone

(5) Preview

- Verify all settings. You can go back to adjust if needed.

3. Click **Create** to schedule it. The backup schedule will appear under the **Schedules** tab.

ClusterControl will now automatically perform your backups. You can also enable backup verification, encryption, or point-in-time recovery under the same Backups section.

1.18.10 Step 8: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. ***ClusterControl GUI → {cluster} → Settings → Email Notifications***.
2. Select a User group (your group) from the **User group** dropdown.
3. Select your email address to from the **Users in the selected group** dropdown.
4. Click ***Enable***.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures timely notifications when something goes wrong.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.18.11 Step 9: Manage your node

ClusterControl offers a range of management features even for a standalone setup:

- Database upgrades and patching: Perform minor version upgrade to keep your cluster up to date. This feature is available at ***ClusterControl GUI → {cluster} → Actions → Upgrades***.

- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → {cluster} → Nodes → Actions***.
- ProxySQL management: Perform ProxySQL management related to query rules, query cache, proxy user management, export configuration, synchronize instances (in cluster mode), restart service and reboot host. These features are available at ***ClusterControl GUI → {cluster} → Nodes → Actions*** and ***ClusterControl GUI → {cluster} → Nodes → ProxySQL***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → {cluster} → Manage → Configuration***.
- Database user management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → {cluster} → Manage → DB Users***.
- Backup management: Create, schedule, restore, perform point-in-time recovery, upload backup to cloud, verify an existing backup, and set retention period. These features are available at **ClusterControl GUI → {cluster} → Backups → Actions_** and **ClusterControl GUI → {cluster} → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → SSH Console***.

1.18.12 Step 10: Scale your cluster

Scaling your standalone database server into a multi-node setup is straightforward with ClusterControl. You can add new replica nodes (for read scaling or backup offloading), configure a load balancer for improved performance and failover, or even switch to a synchronous replication topology (like Galera) for high availability, all from within the ClusterControl GUI. This means you don't have to redeploy your database from scratch. Instead, you can gradually expand as your workloads and traffic demands increase.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → {cluster} → Actions → Add new → Replica node → Create a replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 3306
- **Data directory**: /var/lib/mysql
- **Configuration template**: my.cnf.repl80
- **Install software**: On (default)
- **Disable firewall**: Checked (default)

- **Disable SELinux/AppArmor**: Checked (default)

(2) Advanced settings

- **Rebuild from a backup**: Off (default)
- **Include in load balancer set (if exists)**: On
- **Delay the replica node**: Off (default)

(3) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press "Enter".

(4) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.18.13 Conclusion

Congratulations! You've deployed, monitored, and managed a standalone MySQL/MariaDB server using ClusterControl. Although you now have only one node, you can effortlessly evolve this environment to replication or high availability in the future. Remember to:

- Keep your backups current and tested.
- Monitor performance and resource usage.
- Secure your node by restricting access and using strong passwords.
- Expand to multi-node topologies if your needs grow.

1.19 MySQL Replication (MariaDB, Percona, Oracle)

This guide will lead you through the process of setting up your first MySQL replication topology (primary-replica), complete with ProxySQL load balancers and Keepalived for high availability, using ClusterControl. By following this tutorial, you'll have a fully functional database cluster that you can monitor, manage, and scale.

MySQL Replication is a well-established feature that enables data from a primary (also known as a source) MySQL server to be copied automatically to one or more replicas. This allows you to scale out read workloads, offload backups and reporting, and provide basic high availability with manual or automatic failover strategies.

This tutorial applies to the following MySQL distributions:

- MariaDB Server

- Percona Server for MySQL
- Percona Pro Server for MySQL
- Oracle MySQL Community Server

See also

- MySQL Replication (Oracle) documentation
- MariaDB Replication documentation
- Percona Server for MySQL documentation
- Percona Distribution for MySQL documentation

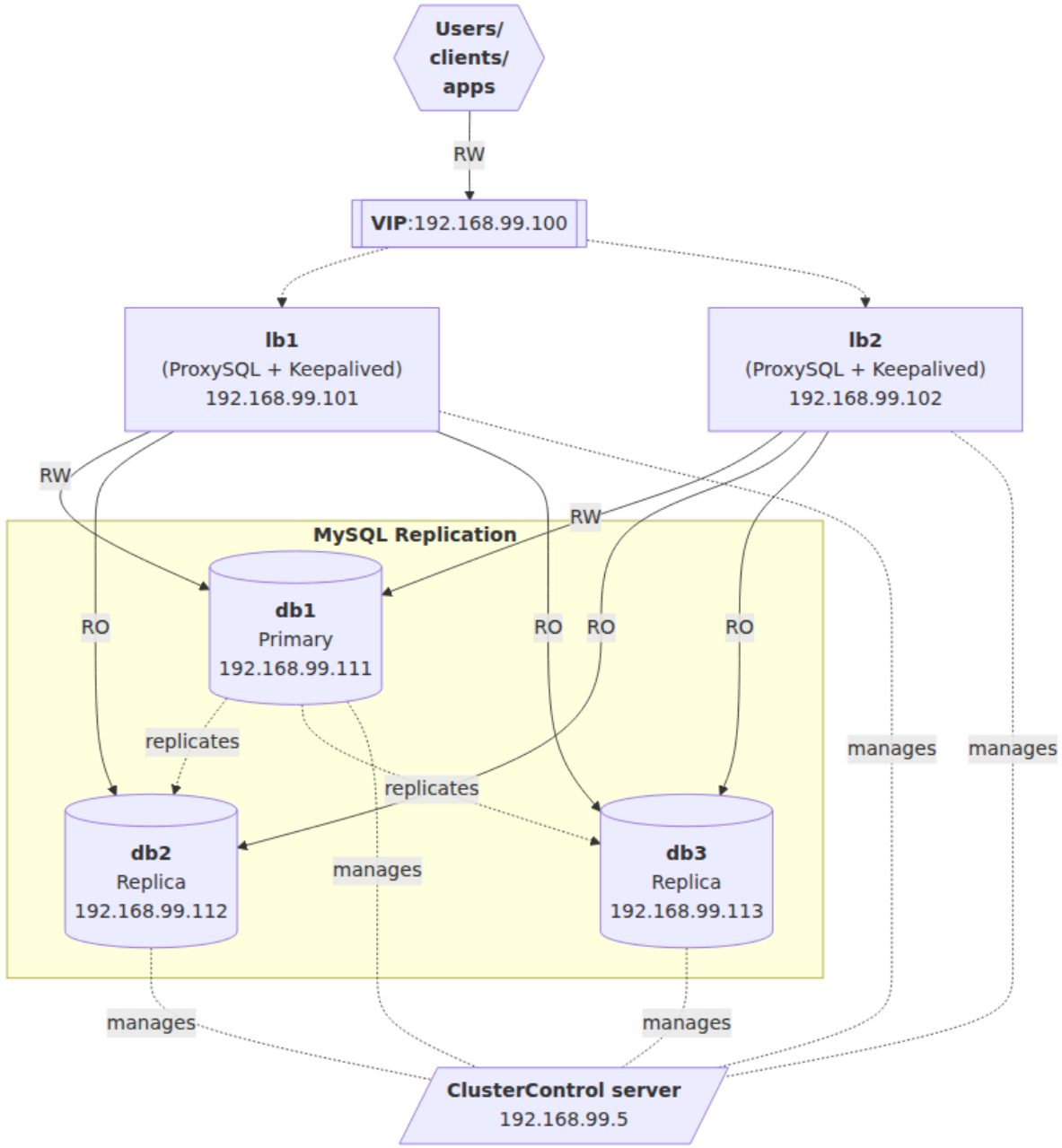
1.19.1 Prerequisites

Before proceeding, ensure the following requirements are met:

- ClusterControl is installed and running. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- At least four hosts (bare-metal or virtual machines) with a supported OS:
  - A host for the primary MySQL server.
  - Two hosts for the replica MySQL servers.
  - Two hosts for the load balancer (ProxySQL).
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- Network Time Protocol (NTP) is configured and running on both hosts to keep their clocks synchronized.

1.19.2 Architecture

Below is a simplified diagram of the final architecture:



1.19.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes (specify the password if prompted):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101  # lb1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # lb2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.111  # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.112  # db2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.113  # db3
```

If the target node only requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.101  # lb1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.102  # lb2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.111  # db1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.112  # db2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.113  # db3
```

3. Perform some tests and make sure you can execute the following commands from the ClusterControl server without any password prompt:

```
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.111 "ls /root"
ssh root@192.168.99.112 "ls /root"
ssh root@192.168.99.113 "ls /root"
```

1.19.4 Step 2: Deploy a new cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on the **Deploy a cluster** button in the top-right corner and choose **Create a database cluster**. This will open the **Deploy a cluster** panel.
3. Choose "MySQL Replication" from the **Database** dropdown, choose "Oracle" or your preferred vendor from the **Vendor** dropdown and also the database major version from the **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Replication** wizard, configure the database cluster as below:

Deploy MySQL Replication

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g, My First Replication Cluster).
- **Tags**: Enter one or more tags separate by pressing "Enter" (e.g, production, replication, dc1).

(2) SSH configuration

- **SSH user**: root
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave it blank)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install sofware**: On (default)

- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 3306 (default)
- **Server data directory**: `/var/lib/mysql` (default)
- **Admin/Root user**: root
- **Admin/Root password**: Enter the password for the database admin/root
- **Repository**: Use vendor repositories (default)
- **Version**: Choose a database minor version.
- **Configurion template**: my.cnf.replication (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Galera node**:
  - Specify "192.168.99.111" and press ++enter++, followed by "192.168.99.112" and press ++enter++, followed by "192.168.99.113" and press ++enter++.
  - Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
  - You can only proceed to the next step if all of the specified nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. Click **Finish** to start the deployment.
6. ClusterControl will provision your MySQL Replication cluster. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the cluster will appear in the **Home** page.

1.19.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → {cluster} → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm

but the recovery procedures will be skipped.

- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state.

1.19.6 Step 4: Import data

When importing data in a primary-replica (asynchronous) replication setup, you must import the data on the primary server, and it will replicate to all replicas.

1. Import data on the primary (db1). Depending on your data format, you can use different utilities:

Using MySQL dump

```
mysql -u root -p -h database_name < dumpfile.sql
```

Using `LOAD DATA INFILE`

```
LOAD DATA INFILE '/path/to/file.csv'
INTO TABLE table_name
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

2. Check the cluster status in ClusterControl. Go to ***ClusterControl GUI → Clusters → {cluster} → Nodes → Topology***. Ensure db2 and db3 are both green (running) and in sync with db1.
3. Once your data import finishes and replicas catch up, you can proceed with normal read/write operations on the primary, and replicas shall stay up to date.

1.19.7 Step 5: Deploy load balancers

Using ProxySQL in front of a MySQL replication allows read-write splitting, enhances performance, scalability, and high availability by providing intelligent query routing, load balancing, connection pooling, failover handling, and security. It allows you to leverage the power of cluster by managing traffic efficiently while ensuring the application remains connected even during node failures or maintenance events.

1. Go to ***ClusterControl GUI → Clusters → {cluster} → Actions → Add new → Load balancer → Create balancer → ProxySQL → Continue***.
2. In the **Create a load balancer** wizard, configure the ProxySQL as below:

Create a load balancer

(1) Where to install

- **Version**: 2.x
- **Server address**: Specify "192.168.99.101" and press ++enter++. Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
- **Admin port**: 6032 (default)
- **Listening port**: 6033 (default)
- **Disable firewall**: Checked (default)

- **Disable SELinux/AppArmor**: Checked (default)
- **Import configuration**: Unchecked (default)
- **Use native clustering**: Unchecked (default) (1) { .annotate }

    1. Check this box if you want to deploy two or more ProxySQL instances. Or, if you plan to add more ProxySQL instance in the future.

(2) Configuration

- **Administration user**: proxysql-admin (default)
- **Administration password**: Specify the password for ProxySQL admin user. ClusterControl will create this user.
- **Monitor user**: proxysql-monitor (default)
- **Monitor password**: Specify the password for ProxySQL monitor user. ClusterControl will create this user.
- **Create new user**: Selected
- **Database username**: Specify the database username that will be load balanced via ProxySQL. ClusterControl will create this user.
- **Database password**: Specify the database password for the database username above.
- **Database name**: Specify the database and/or table to be granted for this database user. If you want to grant this user to all tables for database example, specify `example.*`.
- **MySQL privilege(s)**: ALL PRIVILEGES

(3) Server instances

- **192.168.99.111:3306 (Primary)**: On (default)
- **192.168.99.112:3306 (Primary)**: On (default)
- **192.168.99.113:3306 (Primary)**: On (default)
- Leave the **Max replication lag**, **Max connection** and **Weight** as default.
- **Are you using implicit transactions?**: Off (default) (1) { .annotate }

    1. Toggle on if the application relies on `SET autocommit=0` to create a transaction for you.

(4) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

3. To deploy the load balancer, click **Finish**.
4. ClusterControl will start the ProxySQL deployment job. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the load balancer instance will appear in the **Nodes** page.
5. Repeat the same steps above to add the secondary ProxySQL instance, 192.168.99.102 (lb2).

1.19.8 Step 6: Deploy virtual IP address

A Virtual IP address (VIP) allows your application to connect to a single endpoint (IP address), where the IP address floats between two or more load balancer hosts. Only the active node holds the VIP, while the backup node takes over the VIP if the active node becomes unavailable.

Before deploying the virtual IP address, ensure that you have completed *Step 5* and that both ProxySQL instances are listed under the **Nodes** section. Additionally, prepare an unused routable IP address to be used as the VIP.

1. Go to ***ClusterControl GUI → Clusters → {cluster} → Actions → Add new → Load balancer → Create balancer → Keepalived → Continue***.
2. In the **Create Keepalived service** wizard, configure as below:

Create Keepalived service

(1) Configuration

- **Load balancer type**: ProxySQL
- **Keepalived**: Choose "192.168.99.101:6032 (ProxySQL)" and "192.168.99.102:6032 (ProxySQL)" from the dropdown. Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Host configuration

- **Virtual IP**: 192.168.99.100
- **Network interface**: Specify which network interface that the virtual IP address will be associated with. You can use the `ifconfig` or `ip` command to get the correct interface name. The default value is "eth0".

(3) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

3. To deploy the Keepalived, click **Finish**.
4. ClusterControl will start the deployment job. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the Keepalived instances will appear in the **Nodes** page.

1.19.9 Step 7: Connect to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- Connect to the virtual IP address 192.168.99.100 on port 6033. This is the recommended method for high availability. It provides a load-balanced MySQL connection via ProxySQL through the virtual IP address, which exists on only one of the load balancers at any given time. Use the database user credentials created in *Step 5*.
- Connect to any of the load balancer instances (192.168.99.101 or 192.168.99.102) on port 6033.

This also provides a load-balanced MySQL connection via ProxySQL. Use the database user credentials created in *Step 5*.

- In a replication setup, only the primary node can serve read and write workloads, while replicas operate in read-only mode. Connect directly to the primary node (192.168.99.111) on port 3306 for read-write workloads, or connect to the replica nodes (192.168.99.112 or 192.168.99.113) on port 3306 for read-only workloads.

Note

Note With ProxySQL in the topology, it is recommended to create a new database user using ClusterControl via ***ClusterControl GUI → {cluster} → Manage → DB Users → Create DB User***. ClusterControl ensures that the database user is created across all database nodes as well as ProxySQL nodes. If you create the database user manually, you must also create the same user on both ProxySQL hosts.

1.19.10 Step 8: Enable automatic backups

To protect your data, set up automatic backups. ClusterControl supports both logical and physical backups for MySQL replication. For a small dataset, a daily logical backup should be sufficient since by default, ClusterControl configures every database server with binary logs enabled that can be used for point-in-time recovery.

1. Go to ***ClusterControl GUI → {cluster} → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily backup - mysqldump
- **Cluster**: (leave as it is)
- **Backup host**: db3
- **Backup method**: mysqldump
- **Dump type**: Complete PITR-compatible
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Enable encryption**: On
- **Verify backup**: Off (default)
- **Retention**: On (default)
- **Use extended insert**: On (default)

- **Failover backup if node is down**: On

(3) Storage

- **Storage location**: Store on controller (default)
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. ClusterControl also support advanced backup features like partial backup, backup encryption, backup verification, backup throttling, back up to cloud and point-in-time recovery.

1.19.11 Step 9: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → {cluster} → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.19.12 Step 10: Manage your cluster and load balancers

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform minor version rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → {cluster} → Actions → Upgrades***.
- Database node management: Perform actions such as start, stop, and restart nodes; reboot hosts; bootstrap the cluster; activate or deactivate read-only mode; resync nodes; change the primary source; and rebuild replicas. These features are available at ***ClusterControl GUI → {cluster} → Nodes → {node} → Actions***.
- ProxySQL management: Manage ProxySQL components such as query rules, query cache, proxy users, configuration export, instance synchronization (in cluster mode), service restarts, and host reboots. These features are available at ***ClusterControl GUI → {cluster} → Nodes → {node} → Actions*** and ***ClusterControl GUI → {cluster} → Nodes → ProxySQL***.
- Configuration management: Apply database configuration changes globally or to individual database nodes. This feature is available at ***ClusterControl GUI → {cluster} → Manage → Configuration***.
- Database user management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → {cluster} → Manage → DB Users***.
- Backup management: Create, schedule, and restore backups, perform point-in-time recovery, upload backups to the cloud, verify backups, and set retention policies. These features are available at **ClusterControl GUI → {cluster} → Backups → {backup} → Actions_** and **ClusterControl GUI → {cluster} → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from the ClusterControl GUI via the web-based SSH console. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → {node} → Actions → SSH Console***.

1.19.13 Step 11: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster and scaling your MySQL replication is straightforward. You can scale out the cluster with another database node or a read-only replica (delayed or real-time replication and asynchronous or semi-synchronous replication). Common use cases are to use the replica node for a dedicated read-only workloads, backups, reporting or analytics purposes.

Adding a new database node or replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node if you stream directly from the primary. You may opt to stream from an existing backup to minimize this risk.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → {cluster} → Actions → Add new → Replica node → Create a replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 3306
- **Data directory**: /var/lib/mysql
- **Configuration template**: my.cnf.repl80
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Advanced settings

- **Rebuild from a backup**: Off (default)
- **Include in load balancer set (if exists)**: On
- **Delay the replica node**: Off (default)

(3) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press ++enter++.

(4) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.19.14 Conclusion

Congratulations! You've successfully deployed, monitored, managed, and scaled your first MySQL replication cluster using ClusterControl. Explore additional features like failover management, advanced monitoring, backup management, and more to optimize your database infrastructure for high availability and performance.

1.20 MySQL Group Replication

In this guide, we will walk you through setting up your first database cluster running on MySQL Group Replication with a HAProxy load balancer using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

MySQL Group Replication, an open-source plugin by Oracle, offers built-in high availability and fault tolerance for MySQL (version 5.7.17 and later). This replication solution enables multiple MySQL servers to form a clustered environment, ensuring data redundancy and automatic failover.

Utilizing a Paxos-based consensus protocol, MySQL Group Replication guarantees that transactions are replicated and confirmed by a majority of servers before commitment. Key features include automatic failover, distributed recovery and seamless addition of new nodes. This makes it a robust choice for mission-critical MySQL deployments demanding high uptime and fault tolerance.

See also

- MySQL Group Replication documentation

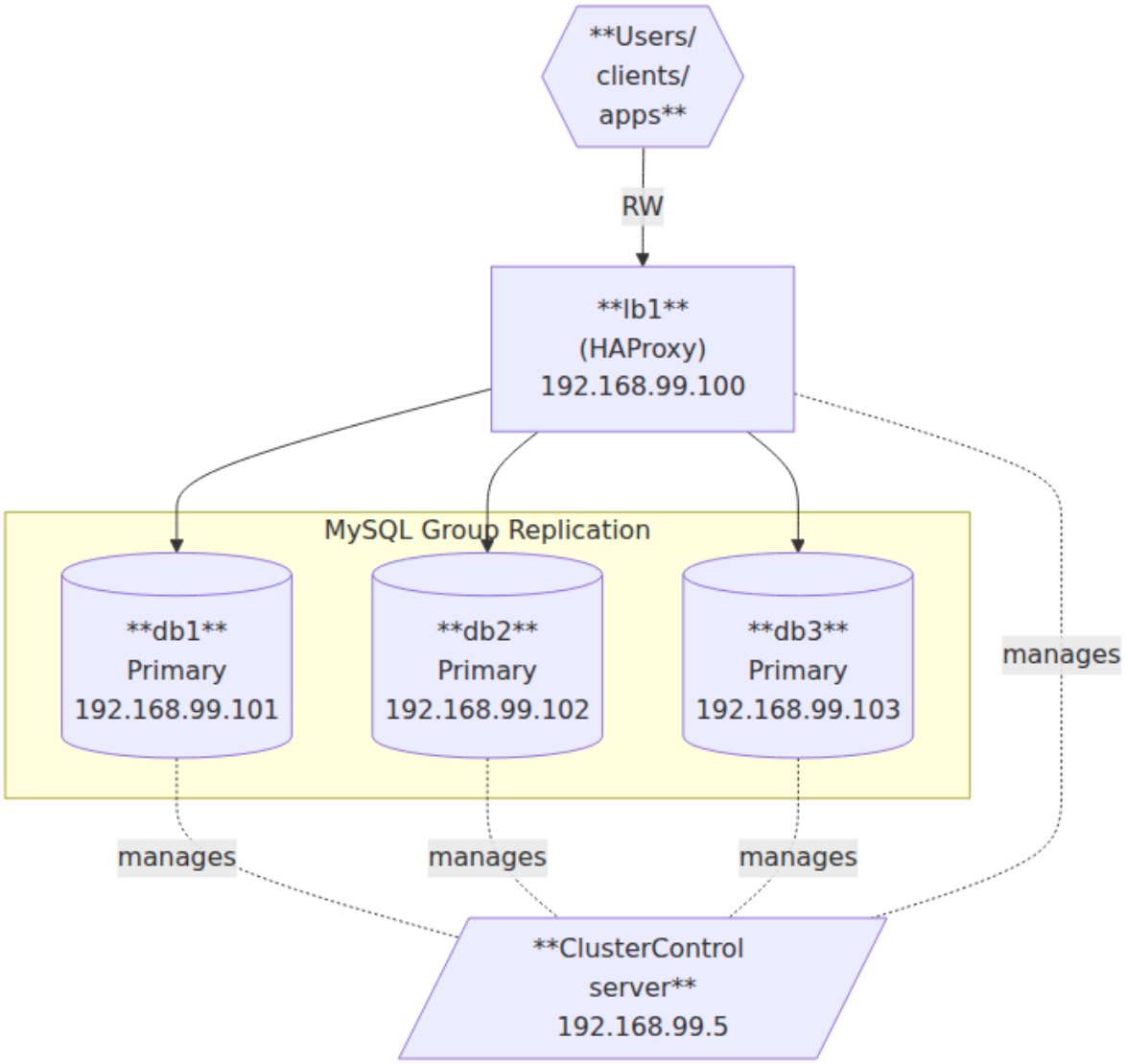
1.20.1 Prerequisites

Before you start, ensure that you have the following:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quick-start* or use the *Installer Script*.
- You need at least four hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*. One host is for database load balancer and another three hosts are for MySQL Group Replication nodes.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.20.2 Architecture

Our final system architecture will look like this:



1.20.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes (specify the password if prompted):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.100   # lb1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101   # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102   # db2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102   # db3
```

If the target node only requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.100   # lb1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
```

(continues on next page)

(continued from previous page)

```
↪root/myprivatekey.pem' root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.102  # db2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.103  # db3
```

3. Perform some tests and make sure you can execute the following commands from the ClusterControl server without any password prompt:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.103 "ls /root"
```

1.20.4 Step 2: Deploy a new cluster

1. Open your browser and go to the IP address or domain name where your ClusterControl is installed.
2. From the ClusterControl dashboard, click on the **Deploy a cluster** button in the top-right corner and choose **Create a database cluster**. This will open the **Deploy cluster** wizard.
3. Choose "MySQL Group Replication" from the **Database** dropdown, choose "Oracle" from the **Vendor** dropdown and also the database major version from the **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Group Replication** wizard, configure the database cluster as below:

Deploy MySQL Group Replication

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g, My First Group Replication).
- **Tags**: Enter one or more tags separate by pressing "Enter" (e.g, production, group replication, dc1).

(2) SSH configuration

- **SSH user**: root
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave it blank)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install sofware**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 3306 (default)
- **Server data directory**: `/var/lib/mysql` (default)

- **Admin/Root user**: root
- **Admin/Root password**: Enter the password for the database admin/root
- **Repository**: Use vendor repositories (default)
- **Version**: Choose a database minor version.
- **Configurion template**: my.cnf.grouprepl (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **MySQL node**:
  - Specify "192.168.99.101" and press "Enter", followed by "192.168.99.102" and press "Enter", followed by ""192.168.99.103" and press ++enter++.
  - Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
  - You can only proceed to the next step if all of the specified nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. To deploy the cluster, click **Finish**.
6. ClusterControl will now start provisioning the MySQL Group Replication. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the cluster will appear in the **Home** page.

1.20.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.20.6 Step 4: Import data

To import data into a MySQL Group Replication, you need to take into account that MySQL Group Replication uses synchronous replication and all changes are replicated across all nodes. Performing restore using `mysqldump` or `LOAD DATA INFILE` will have a significant impact to the cluster and it is a slow operation since all inserts will be cerfified by the group replication. To avoid this, you should perform the restore on one of the database nodes.

Here are the steps to safely import data into a MySQL Group Replication using ClusterControl:

1. Stop two out of three nodes. Firstly, stop db2. Go to ***ClusterControl GUI → Nodes → locate db2 (1) → Actions → Stop node → Stop***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.102`, `Port: 3306`, `Status: Operational`, `Type: MySQL Group Replication` and `Role: Primary` from the nodes list.
2. Next, stop db3. Go to ***ClusterControl GUI → Nodes → locate db3 (1) → Actions → Stop node → Stop***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.103`, `Port: 3306`, `Status: Operational`, `Type: MySQL Group Replication` and `Role: Primary` from the nodes list.
3. At this point, the database should be running as a single-node cluster with only db1 is running. Depending on the format of your data, you can use different MySQL utilities to import data:

Using MySQL dump

If you are importing from a dump file, transfer the dump file to db1 and use the following command to restore:

```
mysql -u root -p -h database_name < dumpfile.sql
```

Using `LOAD DATA INFILE`

If you are importing from a CSV or similar file format, transfer the file to db1 and run the following SQL statement from the MySQL console:

```
LOAD DATA INFILE '/path/to/file.csv'
INTO TABLE table_name
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

4. Once imported, start db2 using **Start Node**. This will automatically sync the transaction write-set through Group Communication System (GCS) in MySQL Group Replication. Go to ***ClusterControl GUI → Nodes → locate db2 (1) → Actions → Start Node***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.102`, `Port: 3306`, `Status: Shut Down`, `Type: MySQL Group Replication` and `Role: Primary` from the nodes list.
5. Start db3 using **Resync Node**. This will automatically sync the transaction write-set through Group Communication System (GCS) in MySQL Group Replication. Go to ***ClusterControl GUI → Nodes → locate db3 (1) → Actions → Start Node***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.103`, `Port: 3306`, `Status: Shut Down`, `Type: MySQL Group Replication` and `Role: Primary` from the nodes list.

6. Once all nodes up and running, verify that all nodes should be green from the **Home** page. The import exercise is complete.

1.20.7 Step 5: Deploy a new load balancer

Using HAProxy in front of a MySQL Group Replication enhances performance, scalability, and high availability by providing intelligent query routing, load balancing, connection pooling, failover handling, and security. It allows you to fully leverage the power of cluster by managing traffic efficiently while ensuring the application remains connected even during node failures or maintenance events.

1. Go to ***ClusterControl GUI → Clusters → locate the cluster → Actions → Add new → Load balancer → Create balancer → HAProxy → Continue***.
2. In the **Create a load balancer** wizard, configure HAProxy as below:

Create a load balancer

(1) Where to install

- **Server address**: `192.168.99.100`. Press "Enter" and wait until it turns green.
- **Policy**: leastconn (default) (1) { .annotate }
    1. You can choose the policy that suits your requirement.
- **Listen port (read/write)**: 5433 (default)
- **Listen port (read-only)**: 5434 (default)
- **Install for read/write splitting (master-slave replication)**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Installation settings

- **Overwrite existing /usr/local/sbin/mysqlchk_rw_split on targets**: On (default)

(3) Advanced settings

- **Stats socket**: /var/run/haproxy.socket (default)
- **Admin port**: 9600 (default)
- **Admin user**: admin
- **Admin password**: Enter the password for the Admin user
- **Backend name (RW)**: haproxy_5433_rw (default)
- **Backend name (RO)**: haproxy_5434_ro (default)
- **Timeout server (in seconds)**: 10800 (default)
- **Timeout client (in seconds)**: 10800 (default)
- **Max connections frontend**: 8192 (default)
- **Max connections backend (per instance)**: 1024
- **xinted allow connections from**: 0.0.0.0/0 (default)

(4) Server instances in the load balancer

- **192.168.99.101:3306 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.101]
- **192.168.99.102:3306 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.102]
- **192.168.99.103:3306 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.103]

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

3. To deploy the load balancer, click **Finish**.
4. ClusterControl will now start provisioning the HAProxy instance. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the load balancer instance will appear in the **Nodes** page.

!!! Tips

```
For production environment, it is recommended to have at least two load␣
↪balancers to eliminate single-point of failure. You can deploy another␣
↪HAProxy instance and tie them together with a virtual IP address. See␣
↪[Create Load␣
↪Balancers](../../../../../user-guide/deployment/create-load-balancer.md).
```

1.20.8 Step 6: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- Load balancer instance, 192.168.99.100 on port 3307. This is a load-balanced MySQL connection via HAProxy. Use the database user credentials that you created at *Step 5*.
- If there are two or more load balancers, tie them together with a virtual IP address provided by Keepalived. Use the virtual IP address in your connection string instead of the load balancers' primary IP address.

1.20.9 Step 7: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For MySQL Group Replication, ClusterControl supports both logical and physical backups. For a small dataset, a daily logical backup should be sufficient since by default, ClusterControl configures every database server with binary logs enabled that can be used for point-in-time recovery.

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily backup - mysqldump
- **Cluster**: (leave as it is)
- **Backup host**: db3
- **Backup method**: mysqldump
- **Dump type**: Complete PITR-compatible
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Enable encryption**: On
- **Verify backup**: Off (default)
- **Retention**: On (default)
- **Use extended insert**: On (default)
- **Failover backup if node is down**: On

(3) Storage

- **Storage location**: Store on controller (default)
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. ClusterControl also support advanced backup features like partial backup, backup encryption, backup verification, backup throttling, back up to cloud and point-in-time recovery. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size and perform restoration if necessary.

1.20.10 Step 8: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.20.11 Step 9: Manage your cluster and load balancer

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → choose the cluster → Actions → Upgrades***.

- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- HAProxy management: Perform HAProxy management related disable nodes, see the detailed status, restart service and reboot host. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions*** and ***ClusterControl GUI → choose the cluster → Nodes → HAProxy***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Database user management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → DB Users***.
- Backup management: Create, schedule, restore, perform point-in-time recovery, upload backup to cloud, verify an existing backup, and set retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions**_ and **ClusterControl GUI → choose the cluster → Backups → More**_.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.20.12 Step 10: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For MySQL Group Replication, you can scale out the cluster with another database node.

Adding a new database node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node if you stream directly from the primary. You may opt to stream from an existing backup to minimize this risk.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new node, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Node → Create a database node***.
2. In the **Create a database node** wizard, configure the following:

Create a database node

(1) Node Configuration

- **Port**: 3306
- **Data directory**: /var/lib/mysql
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Advanced settings

- **Rebuild from a backup**: Off (default)
- **Include in load balancer set (if exists)**: On

(3) Add node

- **Node**: Specify the IP address or hostname of the node that you want to add and press "Enter".

(4) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.20.13 Conclusion

Congratulations! You've successfully deployed, monitored, managed and scaled your first MySQL Group Replication using ClusterControl. From here, you can explore additional features such as failover management, advanced monitoring, advanced backup management, database restore, cluster cloning and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.21 Percona XtraDB Cluster

In this guide, we will walk you through setting up your first database cluster running on Percona XtraDB Cluster with a ProxySQL load balancer using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

Percona XtraDB Cluster (PXC) is an open-source, high-availability solution for MySQL, based on the Galera replication technology. It provides a synchronous multi-master replication system where all nodes in the cluster hold the same data, ensuring strong consistency and zero data loss. PXC is designed to deliver automatic failover, seamless scalability, and load balancing across multiple nodes, making it highly resilient to node failures. It includes features like automatic node provisioning, parallel replication, and integrated backup support, making it a robust solution for mission-critical MySQL environments that require high uptime and fault tolerance.

See also

- Percona XtraDB Cluster product page
- Percona XtraDB Cluster documentation

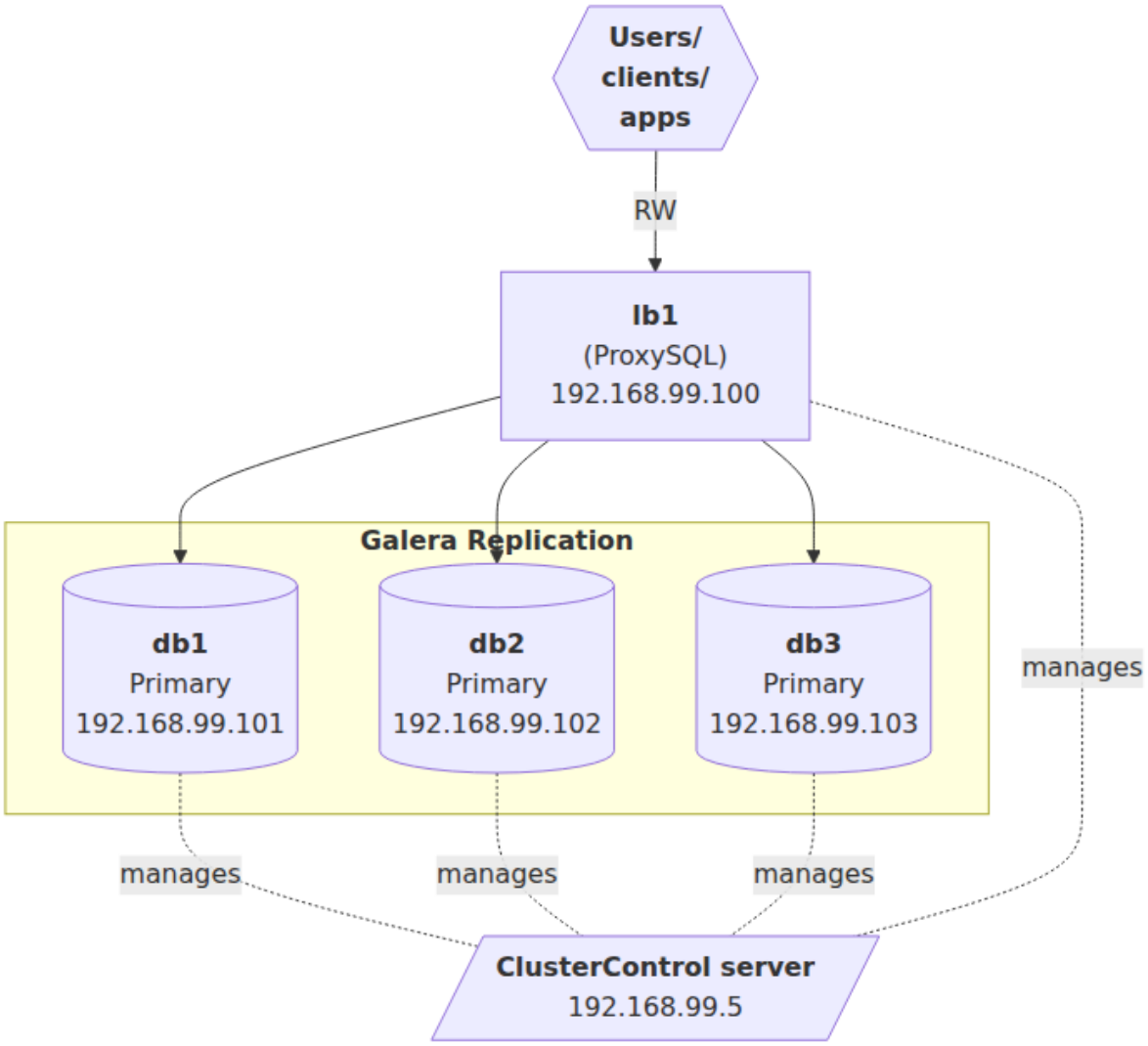
1.21.1 Prerequisites

Before you start, ensure that you have the following:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need at least four hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*. One host is for database load balancer and another three hosts are for Percona XtraDB Cluster.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.21.2 Architecture

Our final system architecture will look like this:



1.21.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes (specify the password if prompted):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.100  # lb1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # db2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # db3
```

If the target node only requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.100  # lb1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
```

(continues on next page)

(continued from previous page)

```
↪root/myprivatekey.pem' root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.102  # db2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.103  # db3
```

3. Perform some tests and make sure you can execute the following commands from the ClusterControl server without any password prompt:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.103 "ls /root"
```

1.21.4 Step 2: Deploy a new cluster

1. Open your browser and go to the IP address or domain name where your ClusterControl is installed.
2. From the ClusterControl dashboard, click on the **Deploy a cluster** button in the top-right corner and choose **Create a database cluster**. This will open the **Deploy cluster** wizard.
3. Choose "MySQL Galera" from the **Database** dropdown, choose "Percona XtraDB" from the **Vendor** dropdown and also the database major version from the **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Galera Cluster** wizard, configure the database cluster as below:

Deploy MySQL Galera Cluster

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g, My First Galera Cluster).
- **Tags**: Enter one or more tags separate by pressing "Enter" (e.g, production, galera, dc1).

(2) SSH configuration

- **SSH user**: root
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave it blank)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install sofware**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 3306 (default)
- **Server data directory**: `/var/lib/mysql` (default)

- **Admin/Root user**: root
- **Admin/Root password**: Enter the password for the database admin/root
- **Repository**: Use vendor repositories (default)
- **Version**: Choose a database minor version.
- **Configurion template**: my.cnf.80-pxc (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Galera node**:
  - Specify "192.168.99.101" and press "Enter", followed by "192.168.99.102" and press "Enter", followed by ""192.168.99.103" and press ++enter++.
  - Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
  - You can only proceed to the next step if all of the specified nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. To deploy the cluster, click **Finish**.
6. ClusterControl will now start provisioning the Percona XtraDB Cluster. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the cluster will appear in the **Home** page.

1.21.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → {cluster} → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → {cluster} → Nodes → Topology***.

1.21.6 Step 4: Import data

To import data into a Percona XtraDB Cluster (PXC), you need to take into account that PXC uses synchronous replication and all changes are replicated across all nodes. Performing restore using `mysqldump` or `LOAD DATA INFILE` will have a significant impact to the cluster and it is a slow operation since all inserts will be cerfified by Galera write-set replication. To avoid this, you should perform the restore on one of the database nodes.

Here are the steps to safely import data into a PXC cluster using ClusterControl:

1. Stop two out of three nodes. Firstly, stop db2. Go to ***ClusterControl GUI → Nodes → locate db2 (1) → Actions → Stop node → Stop***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.102`, `Port: 3306`, `Status: Operational`, `Type: XtraDB` and `Role: Primary` from the nodes list.
2. Next, stop db3. Go to ***ClusterControl GUI → Nodes → locate db3 (1) → Actions → Stop node → Stop***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.103`, `Port: 3306`, `Status: Operational`, `Type: XtraDB` and `Role: Primary` from the nodes list.
3. At this point, the database should be running as a single-node cluster with only db1 is running. Depending on the format of your data, you can use different MySQL utilities to import data:

Using MySQL dump

If you are importing from a dump file, transfer the dump file to db1 and use the following command to restore:

```
mysql -u root -p -h database_name < dumpfile.sql
```

Using `LOAD DATA INFILE`

If you are importing from a CSV or similar file format, transfer the file to db1 and run the following SQL statement from the MySQL console:

```
LOAD DATA INFILE '/path/to/file.csv'
INTO TABLE table_name
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

4. Once imported, trigger **Resync Node** for db2. This will initiate a full resynchronization using State Snapshot Transfer (SST), and the node will automatically start once the resync completes. Go to ***ClusterControl GUI → Nodes → db2 (1) → Actions → Resync Node → Resync***.
{ .annotate }
    1. Find the node with `Hostname: 192.168.99.102`, `Port: 3306`, `Status: Shut Down`, `Type: XtraDB` and `Role: Primary` from the nodes list.
5. Repeat the same step for db3. Trigger **Resync Node**, which will perform a full SST, and the node will automatically start after the process completes. Go to ***ClusterControl GUI → Nodes → db3 (1) → Actions → Resync Node → Resync***.
{ .annotate }
    1. Find the node with `Hostname: 192.168.99.103`, `Port: 3306`, `Status: Shut Down`, `Type: XtraDB` and `Role: Primary` from the nodes list.

6. After all nodes are resynced, verify from the **Home** page that all nodes are in green (**Operational**) status. The import process is now complete.

1.21.7 Step 5: Deploy a new load balancer

Using ProxySQL in front of a Percona XtraDB Cluster enhances performance, scalability, and high availability by providing intelligent query routing, load balancing, connection pooling, failover handling, and security. It allows you to fully leverage the power of cluster by managing traffic efficiently while ensuring the application remains connected even during node failures or maintenance events.

1. Go to ***ClusterControl GUI → Clusters → locate the cluster → Actions → Add new → Load balancer → Create balancer → ProxySQL → Continue***.
2. In the **Create a load balancer** wizard, configure the ProxySQL as below:

Create a load balancer

(1) Where to install

- **Version**: 2.x
- **Server address**: Specify "192.168.99.100" and press "Enter". Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
- **Admin port**: 6032 (default)
- **Listening port**: 6033 (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)
- **Import configuration**: Unchecked (default)
- **Use native clustering**: Unchecked (default) (1) { .annotate }

    1. Check this box if you want to deploy two or more ProxySQL instances. Or, if you plan to add more ProxySQL instance in the future.

(2) Configuration

- **Administration user**: proxysql-admin (default)
- **Administration password**: Specify the password for ProxySQL admin user. ClusterControl will create this user.
- **Monitor user**: proxysql-monitor (default)
- **Monitor password**: Specify the password for ProxySQL monitor user. ClusterControl will create this user.
- **Create new user**: Selected
- **Database username**: Specify the database username that will be load balanced via ProxySQL. ClusterControl will create this user.
- **Database password**: Specify the database password for the database username above.
- **Database name**: Specify the database and/or table to be granted for this database user. If you want to grant this user to all tables for database example, specify `example.*`.

- **MySQL privilege(s)**: ALL PRIVILEGES

(3) Server instances

- **192.168.99.101:3306 (Primary)**: On (default)
- **192.168.99.102:3306 (Primary)**: On (default)
- **192.168.99.103:3306 (Primary)**: On (default)
- Leave the **Max replication lag**, **Max connection** and **Weight** as default.
- **Are you using implicit transactions?**: Off (default) (1) { .annotate }
    1. Toggle on if the application relies on `SET autocommit=0` to create a transaction for you.

(4) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

3. To deploy the load balancer, click **Finish**.
4. ClusterControl will now start provisioning the ProxySQL instance. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the load balancer instance will appear in the **Nodes** page.

!!! Tips

```
For production environment, it is recommended to have at least two load␣
↪balancers to eliminate single-point of failure. You can deploy another␣
↪ProxySQL instance and tie them together with a virtual IP address. See␣
↪[Load␣
↪Balancers](../../../../../user-guide/deployment/create-load-balancer.md).
```

1.21.8 Step 6: Connect to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- Load balancer instance, 192.168.99.100 on port 6033. This is a load-balanced MySQL connection via ProxySQL. Use the database user credentials that you created at *Step 5*.
- If there are two or more load balancers, tie them together with a virtual IP address provided by Keepalived. Use the virtual IP address in your connection string instead of the load balancers' primary IP address.
- One of the database nodes on port 3306. However, you need to create a new database user using ClusterControl. A user created during the ProxySQL deployment stage is only allowed from the ProxySQL host.

1.21.9 Step 7: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For Percona XtraDB Cluster, ClusterControl supports both logical and physical backups. For a small dataset, a daily logical backup should be sufficient since by default, ClusterControl configures every database server with binary logs enabled that can be used for point-in-time recovery.

1. Go to ***ClusterControl GUI → {cluster} → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily backup - mysqldump
- **Cluster**: (leave as it is)
- **Backup host**: db3
- **Backup method**: mysqldump
- **Dump type**: Complete PITR-compatible
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Enable encryption**: On
- **Verify backup**: Off (default)
- **Retention**: On (default)
- **Use extended insert**: On (default)
- **Failover backup if node is down**: On

(3) Storage

- **Storage location**: Store on controller (default)
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. Clus-

terControl also support advanced backup features like partial backup, backup encryption, backup verification, backup throttling, back up to cloud and point-in-time recovery. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size and perform restoration if necessary.

1.21.10 Step 8: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → {cluster} → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.21.11 Step 9: Manage your cluster and load balancer

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → {cluster} → Actions → Upgrades***.
- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → {cluster} → Nodes → Actions***.
- ProxySQL management: Perform ProxySQL management related to query rules, query cache, proxy user management, export configuration, synchronize instances (in cluster mode), restart service and reboot host. These features are available at ***ClusterControl GUI → {cluster} → Nodes → Actions*** and ***ClusterControl GUI → {cluster} → Nodes → ProxySQL***.

- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → {cluster} → Manage → Configuration***.
- Database user management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → {cluster} → Manage → DB Users***.
- Backup management: Create, schedule, restore, perform point-in-time recovery, upload backup to cloud, verify an existing backup, and set retention period. These features are available at **ClusterControl GUI → {cluster} → Backups → Actions_** and **ClusterControl GUI → {cluster} → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → SSH Console***.

1.21.12 Step 10: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For Percona XtraDB Cluster, you can scale out the cluster with another database node or a read-only replica (delayed or real-time replication and asynchronous or semi-synchronous replication). Common use cases are to use the replica node for a dedicated read-only workloads, backups, reporting or analytics purposes.

Adding a new database node or replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node if you stream directly from the primary. You may opt to stream from an existing backup to minimize this risk.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → {cluster} → Actions → Add new → Replica node → Create a replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 3306
- **Data directory**: /var/lib/mysql
- **Configuration template**: my.cnf.repl80
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Advanced settings

- **Rebuild from a backup**: Off (default)

- **Include in load balancer set (if exists)**: On
- **Delay the replica node**: Off (default)

(3) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press "Enter".

(4) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.21.13 Conclusion

Congratulations! You've successfully deployed, monitored, managed and scaled your first Percona XtraDB Cluster using ClusterControl. From here, you can explore additional features such as failover management, advanced monitoring, advanced backup management, database restore, cluster cloning and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.22 MariaDB Galera Cluster

This guide will lead you through the process of setting up your first MariaDB Galera Cluster, complete with HAProxy load balancers and Keepalived for high availability, using ClusterControl. By following this tutorial, you'll have a fully functional database cluster that you can effortlessly monitor, manage, and scale.

MariaDB Galera Cluster is an open-source solution designed for high availability in MariaDB environments. It leverages Galera replication technology to offer synchronous multi-master replication, ensuring data consistency across all nodes and preventing data loss. The cluster supports automatic failover, seamless scaling, and effective load balancing, providing resilience against node failures. With features such as automatic node provisioning, parallel replication, and built-in backup options, it's an excellent choice for critical MariaDB deployments that demand maximum uptime.

See also

- MariaDB Galera Cluster product page
- MariaDB Galera Cluster documentation
- HAProxy product page
- HAProxy documentation page

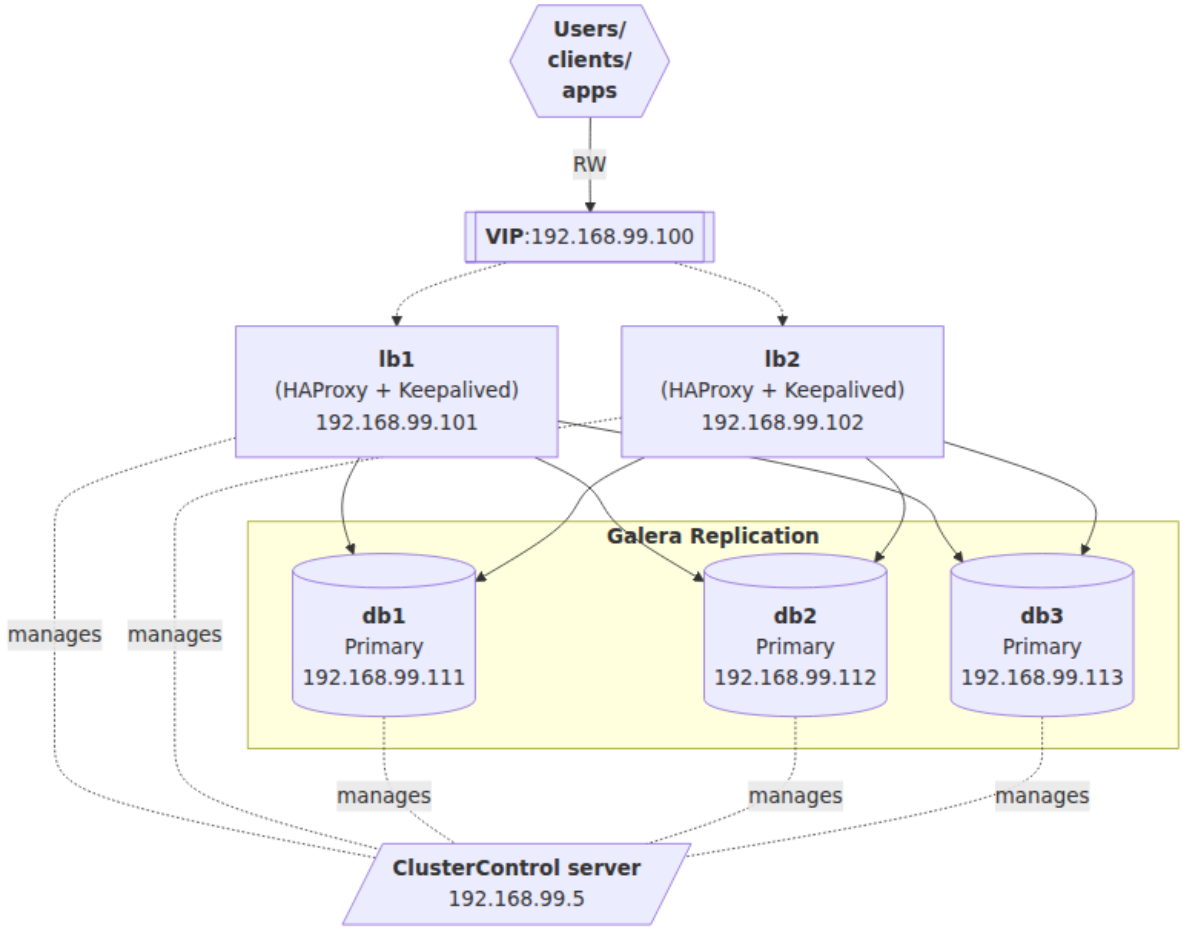
1.22.1 Prerequisites

Before proceeding, ensure the following requirements are met:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quick-start* or use the *Installer Script*.
- You need at least five hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*. Two hosts are for database load balancer and another three hosts are for MariaDB Galera Cluster.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.22.2 Architecture

Our final system architecture will look like this:



1.22.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes (specify the password if prompted):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101  # lb1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # lb2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.111  # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.112  # db2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.113  # db3
```

If the target node only requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.101  # lb1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.102  # lb2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.111  # db1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.112  # db2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.113  # db3
```

3. Perform some tests and make sure you can execute the following commands from the ClusterControl server without any password prompt:

```
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.111 "ls /root"
ssh root@192.168.99.112 "ls /root"
ssh root@192.168.99.113 "ls /root"
```

1.22.4 Step 2: Deploy a new cluster

1. Open your browser and go to the IP address or domain name where your ClusterControl is installed.
2. From the ClusterControl dashboard, click on the **Deploy a cluster** button in the top-right corner and choose **Create a database cluster**. This will open the **Deploy cluster** wizard.
3. Choose "MySQL Galera" from the **Database** dropdown, choose "MariaDB" from the **Vendor** dropdown and also the database major version from the **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Galera Cluster** wizard, configure the database cluster as below:

Deploy MySQL Galera Cluster

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g, My First Galera Cluster).

- **Tags**: Enter one or more tags separate by pressing "Enter" (e.g, production, galera, dc1).

(2) SSH configuration

- **SSH user**: root
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave it blank)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install sofware**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 3306 (default)
- **Server data directory**: `/var/lib/mysql` (default)
- **Admin/Root user**: root
- **Admin/Root password**: Enter the password for the database admin/root
- **Repository**: Use vendor repositories (default)
- **Version**: Choose a database minor version.
- **Configurion template**: my.cnf.mdb106+-galera (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Galera node**:
  - Specify "192.168.99.111" and press ++enter++, followed by "192.168.99.112" and press ++enter++, followed by ""192.168.99.113" and press ++enter++.
  - Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
  - You can only proceed to the next step if all of the specified nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. To deploy the cluster, click **Finish**.
6. ClusterControl will now start provisioning the MariaDB Galera Cluster. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the cluster will appear in the **Home** page.

1.22.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → {cluster} → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state.

1.22.6 Step 4: Import data

When importing data into a MariaDB Galera Cluster, consider the impact of synchronous replication. For large datasets, it's recommended to import data on a single node and then resynchronize the cluster.

Here are the steps to safely import data into a MariaDB Galera cluster using ClusterControl:

1. Stop two out of three nodes. Firstly, stop db2. Go to ***ClusterControl GUI → Nodes → locate db2 (1) → Actions → Stop node → Stop***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.112`, `Port: 3306`, `Status: Operational`, `Type: MariaDB` and `Role: Primary` from the nodes list.
2. Next, stop db3. Go to ***ClusterControl GUI → Nodes → locate db3 (1) → Actions → Stop node → Stop***. { .annotate }
    1. Find the node with `Hostname: 192.168.99.113`, `Port: 3306`, `Status: Operational`, `Type: MariaDB` and `Role: Primary` from the nodes list.
3. At this point, the database should be running as a single-node cluster with only db1 is running. Depending on the format of your data, you can use different MySQL utilities to import data:

Using MySQL dump

If you are importing from a dump file, transfer the dump file to db1 and use the following command to restore:

```
mysql -u root -p -h database_name < dumpfile.sql
```

Using `LOAD DATA INFILE`

If you are importing from a CSV or similar file format, transfer the file to db1 and run the following SQL statement from the MySQL console:

```
LOAD DATA INFILE '/path/to/file.csv'
INTO TABLE table_name
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

4. Once imported, start db2 using **Resync Node**. This will trigger a full resyncing job using State Snapshot Tranfer (SST). Go to ***ClusterControl GUI → Nodes → locate db2 (1) → Actions → Resync node → Resync***. { .annotate }

    1. Find the node with `Hostname: 192.168.99.112`, `Port: 3306`, `Status: Shut Down`, `Type: MariaDB` and `Role: Primary` from the nodes list.

5. Start db3 using **Resync Node**. This will trigger a full resyncing job using State Snapshot Tranfer (SST). Go to ***ClusterControl GUI → Nodes → locate db3 (1) → Actions → Resync node → Resync***. { .annotate }

    1. Find the node with `Hostname: 192.168.99.113`, `Port: 3306`, `Status: Shut Down`, `Type: MariaDB` and `Role: Primary` from the nodes list.

6. Once resynced, verify that all nodes should be green from the **Home** page. The import exercise is complete.

1.22.7 Step 5: Deploy load balancers and virtual IP

Integrating HAProxy and Keepalived with MariaDB Galera Cluster enhances performance, load balancing, and high availability.

1. Go to ***ClusterControl GUI → Clusters → locate the cluster → Actions → Add new → Load balancer → Create balancer → HAProxy → Continue***.

2. In the **Create a load balancer** wizard, configure the HAProxy as below:

Create a load balancer

(1) Where to install

- **Server address**: Specify "192.168.99.101" and press ++enter++. Wait until everything is green. If red notice appears, inspect the error seen by ClusterControl and fix it.
- **Policy**: leastconn (default) (1) { .annotate }

    1. You can choose the policy that suits your requirement.

- **Listen port (read/write)**: 3307 (default)
- **Install for read/write splitting (master-slave replication)**: Off (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)
- **Import configuration**: Unchecked (default)

(2) Installation settings

- **Overwrite existing /usr/local/sbin/mysqlchk on target**: On (default)

(3) Advanced settings

- **Stats socket**: /var/run/haproxy.socket (default)
- **Admin port**: 9600 (default)
- **Admin user**: admin
- **Admin password**: Enter the password for the Admin user

- **Backend name (RW)**: haproxy_3307_rw (default)
- **Backend name (RO)**: haproxy_3308_ro (default)
- **Timeout server (in seconds)**: 10800 (default)
- **Timeout client (in seconds)**: 10800 (default)
- **Max connections frontend**: 8192 (default)
- **Max connections backend (per instance)**: 64 (default)
- **xinted allow connections from**: 0.0.0.0/0 (default)

(4) Server instances in the load balancer

- **192.168.99.111:3306 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.111]
- **192.168.99.112:3306 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.112]
- **192.168.99.113:3306 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.113]

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

3. To deploy the load balancer, click **Finish**.
4. ClusterControl will now start provisioning the HAProxy instance. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, proceed to the next step to add Keepalived.
5. Repeat the same steps above to add the second HAProxy instance, 192.168.99.102 (lb2).
6. Go to ***ClusterControl GUI → Clusters → locate the cluster → Actions → Add new → Load balancer → Create balancer → Keepalived → Continue***.
7. In the **Create a load balancer** wizard, configure the Keepalived as below:

Create a load balancer

(1) Configuration

- **Load balancer type**: Select HAProxy
- **Keepalived**: Select **192.168.99.101:9600** and **192.168.99.102:9600**. Ensure everything is green.(1) { .annotate }

    1. Two HAProxy instances are required to configure the Keepalived.

- **Security configuration**
    - **Disable firewall**: Checked (default)
    - **Disable SELinux/AppArmor**: Checked (default)

(2) Host configuration

- **Virtual IP**: Enter the Virtual IP, as in this case is 192.168.99.100.
- **Network interface**: Specify the network interface for the Virtual IP. In this case is eth0.

(3) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

8. ClusterControl will now start provisioning the Keepalived instance. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the load balancer instance will appear in the **Nodes** page.
9. To check the HAProxy status, in the **Nodes** page click on the HAProxy under **Type** column and it will open the **Node details** page for the selected HAProxy. Verify everything is green here.

1.22.8 Step 6: Connect to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- Virtual IP (VIP) managed by Keepalived, 192.168.99.100 on port 3307. Use the virtual IP address in your connection string instead of the load balancers' primary IP address. This VIP handles all read and write traffic to the database nodes, ensuring seamless failover and high availability.
- Direct connection to a database node on port 3306.

With Keepalived managing read and write traffic through the VIP, applications can seamlessly interact with the database without needing to handle individual HAProxy instances. This setup significantly enhances redundancy and ensures that the database remains accessible at all times.

1.22.9 Step 7: Enable automatic backups

To protect your data, set up automatic backups. ClusterControl supports both logical and physical backups for MariaDB Galera Cluster. For a small dataset, a daily logical backup should be sufficient since by default, ClusterControl configures every database server with binary logs enabled that can be used for point-in-time recovery.

1. Go to ***ClusterControl GUI → {cluster} → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily backup - mysqldump
- **Cluster**: (leave as it is)
- **Backup host**: db3
- **Backup method**: mysqldump
- **Dump type**: Complete PITR-compatible
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Enable encryption**: On
- **Verify backup**: Off (default)
- **Retention**: On (default)
- **Use extended insert**: On (default)
- **Failover backup if node is down**: On

(3) Storage

- **Storage location**: Store on controller (default)
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. ClusterControl also support advanced backup features like partial backup, backup encryption, backup verification, backup throttling, back up to cloud and point-in-time recovery.

1.22.10 Step 8: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → {cluster} → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.22.11 Step 9: Manage your cluster and load balancer

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling minor version upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → {cluster} → Actions → Upgrades***.
- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → {cluster} → Nodes → Actions***.
- HAProxy management: Perform HAProxy management related disable nodes, see the detailed status, restart service and reboot host. These features are available at ***ClusterControl GUI → {cluster} → Nodes → Actions*** and ***ClusterControl GUI → {cluster} → Nodes → HAProxy***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → {cluster} → Manage → Configuration***.
- Database user management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → {cluster} → Manage → DB Users***.

- Backup management: Create, schedule, restore, perform point-in-time recovery, upload backup to cloud, verify an existing backup, and set retention period. These features are available at **ClusterControl GUI → {cluster} → Backups → Actions_** and **ClusterControl GUI → {cluster} → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → {cluster} → Nodes → Actions → SSH Console***.

1.22.12 Step 10: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster and scaling your MariaDB Galera Cluster is straightforward. You can scale out the cluster with another database node or a read-only replica (delayed or real-time replication and asynchronous or semi-synchronous replication). Common use cases are to use the replica node for a dedicated read-only workloads, backups, reporting or analytics purposes.

Adding a new database node or replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node if you stream directly from the primary. You may opt to stream from an existing backup to minimize this risk.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → {cluster} → Actions → Add new → Replica node → Create a replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 3306
- **Data directory**: /var/lib/mysql
- **Configuration template**: my.cnf.repl80
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Advanced settings

- **Rebuild from a backup**: Off (default)
- **Include in load balancer set (if exists)**: On
- **Delay the replica node**: Off (default)

(3) Add node

- **Primary node**: Choose one of the primary node from the dropdown.

- **Node**: Specify the IP address or hostname of the node that you want to add and press ++enter++.

(4) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.22.13 Conclusion

Congratulations! You've successfully deployed, monitored, managed, and scaled your first MariaDB Galera Cluster using ClusterControl. Explore additional features like failover management, advanced monitoring, backup management, and more to optimize your database infrastructure for high availability and performance.

1.23 PostgreSQL Streaming Replication

This guide will lead you through the process of setting up your first PostgreSQL streaming replication cluster (primary-replica), complete with an HAProxy load balancer, using ClusterControl. By following this tutorial, you'll have a fully functional database cluster that you can effortlessly monitor, manage, and scale.

PostgreSQL streaming replication is a built-in feature that allows data from a primary PostgreSQL server to be streamed in real-time to one or more replica servers. This architecture improves read scalability, offers data redundancy, and provides a basis for high availability when combined with a failover mechanism. This tutorial applies to major versions of PostgreSQL starting from 10 and above.

See also

- PostgreSQL Documentation on Streaming Replication

1.23.1 Prerequisites

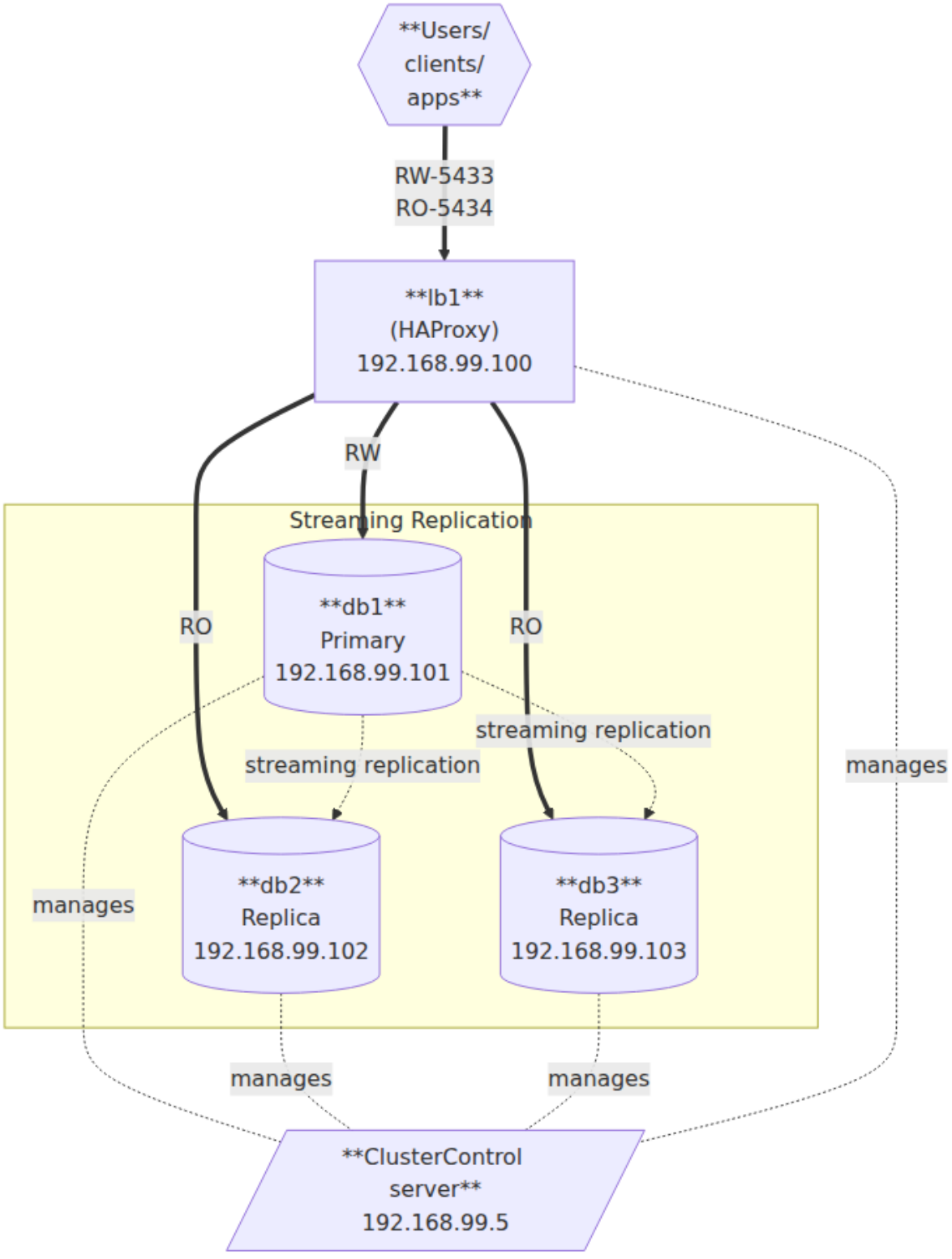
Before proceeding, ensure the following requirements are met:

- ClusterControl is installed and running. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- At least four hosts (bare-metal or virtual machines) with a supported OS:
  - One host for the PostgreSQL primary server.
  - Two hosts for the PostgreSQL replica servers.
  - One host for the load balancer (HAProxy).
- SSH access to all servers involved.

- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to keep their clocks synchronized.

1.23.2 Architecture

Below is a high-level architecture diagram of the final architecture:



1.23.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes (specify the password if prompted):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.100  # lb1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # db2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # db3
```

If the target node only requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.100  # lb1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.102  # db2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.103  # db3
```

3. Perform some tests and make sure you can execute the following commands from the ClusterControl server without any password prompt:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.103 "ls /root"
```

1.23.4 Step 2: Deploy a new cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose "PostgreSQL Streaming" from the **Database** dropdown and the appropriate **Vendor** and **Version** (e.g., "PostgreSQL" and "16") from the wizard. Click **Continue**.
4. In the **Deploy PostgreSQL wizard**, configure the cluster as follows:

Deploy PostgreSQL wizard

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g., My First PostgreSQL Cluster).
- **Tags**: Enter one or more tags, separated by pressing "Enter" (e.g., production, replication, dc1).

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave blank if using key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server data directory**: `/var/lib/pgsql` (default varies by distro)
- **Server port**: 5432 (default)
- **User**: `postgres`
- **Password**: Enter a secure password for the PostgreSQL `postgres` superuser
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Primary node**:
  - Specify "192.168.99.101" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.102" as replica and press ++enter++.
  - Specify "192.168.99.103" as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Turn "On" **Synchronous Replication** if required.
- Proceed only if all nodes are reachable (shown in green).

(5) Extensions

- Leave the toggle button "Off" if you don't require the extensions, else turn "On" "pgvector" or "PostGIS" if you want to enable them.

(6) Preview

- Review the summary. You can go back to modify any configuration. The settings persist until you exit the wizard.

5. Click **Finish** to start the deployment.
6. ClusterControl will now start provisioning the PostgreSQL streaming replication cluster. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the cluster will appear in the **Home** page.

1.23.5 Step 3: Monitor your cluster

After deployment, you are redirected to the **Home** page. This page provides an at-a-glance view of the cluster's status:

- Cluster health: The **Home** page provides the cluster state.
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.
- Verify the cluster status from the **ClusterControl GUI** by checking that the **Primary node** is active, **Replica nodes** are connected and streaming correctly, replication lag remains low, and no alarms or failed jobs are present. This helps confirm that **PostgreSQL streaming replication** is healthy and ready for normal operations.

1.23.6 Step 4: Import data

In a primary-replica (asynchronous) streaming replication setup, you must import the data on the primary, and changes will replicate automatically to the replicas.

To import data from a dump file on the primary (db1: 192.168.99.101), you can use `psql` command as below:

```
psql -U postgres -h 192.168.99.101 -f dumpfile.sql
```

Once the import completes and the replicas have caught up, you can proceed with normal read/write operations on the primary, while replicas remain in sync.

1.23.7 Step 5: Deploy a new load balancer

Integrating HAProxy with PostgreSQL improves high availability and load balancing. ClusterControl allows you to automate the deployment of HAProxy.

1. Go to ***ClusterControl GUI → Clusters → locate the cluster → Actions → Add new → Load balancer → Create balancer → HAProxy → Continue***.
2. In the **Create a load balancer** wizard, configure HAProxy as below:

Create a load balancer

(1) Where to install

- **Server address**: `192.168.99.100`. Press "Enter" and wait until it turns green.
- **Policy**: leastconn (default) (1) { .annotate }
    1. You can choose the policy that suits your requirement.

- **Listen port (read/write)**: 5433 (default)
- **Listen port (read-only)**: 5434 (default)
- **Install for read/write splitting (master-slave replication)**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Installation settings

- **Overwrite existing /usr/local/sbin/postgreschk_rw_split on targets**: On (default)

(3) Advanced settings

- **Stats socket**: /var/run/haproxy.socket (default)
- **Admin port**: 9600 (default)
- **Admin user**: admin
- **Admin password**: Enter the password for the Admin user
- **Backend name (RW)**: haproxy_5433_rw (default)
- **Backend name (RO)**: haproxy_5434_ro (default)
- **Timeout server (in seconds)**: 10800 (default)
- **Timeout client (in seconds)**: 10800 (default)
- **Max connections frontend**: 8192 (default)
- **Max connections backend (per instance)**: 1024
- **xinted allow connections from**: 0.0.0.0/0 (default)

(4) Server instances in the load balancer

- **192.168.99.101:5432 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.101]
- **192.168.99.102:5432 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.102]
- **192.168.99.103:5432 (Primary)**: On (default)
  - **Advance options**
    - **Role**: Active (default)
    - **Connection address**: External Address [192.168.99.103]

(4) Preview

- Review the summary. You can go back to any previous section to modify configurations if needed.

3. Click **Finish** to deploy HAProxy.
4. ClusterControl will now start provisioning the HAProxy instance. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the load balancer instance will appear in the **Nodes** page.

!!! Tips

```
For production environment, it is recommended to have at least two load␣
↪balancers to eliminate single-point of failure. You can deploy another␣
↪HAProxy instance and tie them together with a virtual IP address. See␣
↪[Load␣
↪Balancers](../../../../../user-guide/deployment/create-load-balancer.md).
```

1.23.8 Step 6: Connecting to the database cluster

Start by creating a new database user by going to ***ClusterControl GUI → Clusters → choose the cluster → Manage → DB Users → Create DB User***. Ensure the **Hostname** is the IP address of the application/client/user. Under the **Privileges** section, set a grant on the database and tables and add more privileges for the user by clicking **Add privileges** if necessary. Click **Create** and the database user will be created and granted on all of the database nodes.

Tip

You can create a new database by going to ***ClusterControl GUI → Clusters → choose the cluster → Actions → Create database***.

Next, the application/client/user can access the database cluster via:

- Load balancer instance, 192.168.99.100 on port 5433 for read-write workloads. For read-only workloads, send the database connections to port 5434. These are load-balanced PostgreSQL connections via HAProxy.
- If there are two or more load balancers, tie them together with a virtual IP address provided by Keepalived. Use the virtual IP address in your connection string instead of the load balancers' primary IP address.
- For read-write workloads, send the database connections to the PostgreSQL primary server, 192.168.99.101 on port 5432. For read-only workloads, send the database connections to either of the PostgreSQL replica servers, 192.168.99.102 or 192.168.99.103 on port 5432.

Note

Note If a failover happens, 192.168.99.101 might not be running as primary, and you have to find the new primary from ClusterControl GUI (or by querying directly to the database server) and send the read-write workloads there. That is why having a load balancer is a better approach for your application.

1.23.9 Step 7: Enable automatic backups

To protect your data, set up automatic backups. ClusterControl supports both logical (`pg_dumpall`) and physical (`pg_basebackup`, `pgBackRest`) backups for PostgreSQL.

1. Go to ***ClusterControl GUI → Clusters → choose the cluster → Backups*** to open the backup overview.
2. Go to ***Create Backup → Schedule a Backup***. The backup wizard appears.
3. In the **Create a backup schedule** wizard:

Create a backup schedule

(1) Configuration

- **Schedule name**: e.g., `Daily Backup`
- **Cluster**: (auto-selected)
- **Backup host**: `db3` (select one of the replicas to offload backup load)
- **Backup method**: `pg_dump` or `pg_basebackup`
- **PITR enabled**: On (default for other than `pg_basebackup`)
- **Upload backup to cloud**: Off (default, can be enabled if needed)

(2) Advanced settings

- **Compression**: On
- **Compression level**: 6
- **Enable encryption**: Off or On (optional)
- **Verify backup**: Off (default)
- **Retention**: On, configure how many backups to keep
- **Failover backup if node is down**: Off (default)

(3) Storage

- **Storage location**: `Store on controller` (default)
- **Storage directory**: `/root/backups` (default)
- **Backup subdirectory**: `BACKUP-%I` (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Timezone**: Your preferred timezone

(5) Preview

- Review the summary. You can always go back to any previous section to modify settings.

4. Click Create to set the backup schedule.

ClusterControl will handle the scheduled backups automatically. Advanced backup features like partial backups, encryption, verification, and point-in-time recovery can also be configured.

1.23.10 Step 8: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.23.11 Step 9: Manage your cluster and load balancer

ClusterControl offers many features for ongoing management:

- **PostgreSQL upgrades**: Perform rolling major or minor version upgrade to keep the cluster up-to-date. This feature is available at ***ClusterControl GUI → choose cluster → Actions → Upgrades***.
- **Node management**: Start/stop/restart a node, promote a replica to primary, reboot host, etc. These features are available at ***ClusterControl GUI → choose cluster → Nodes → Actions***.

- **Load balancer management**: Manage and configure HAProxy backends, frontend ports, perform service restarts, and advanced routing. These features are available at ***ClusterControl GUI → choose cluster → Nodes → Actions*** (for HAProxy nodes).
- **Configuration management**: Adjust PostgreSQL settings globally or on individual nodes. This feature is available at ***ClusterControl GUI → choose cluster → Manage → Configuration***.
- **User management**: Manage PostgreSQL roles and privileges. These features are available at ***ClusterControl GUI → choose cluster → Manage → DB Users***.
- **Backup management**: Create, schedule, restore, perform point-in-time recovery, upload backups to cloud, verify backups, and set retention. These features are available at ***ClusterControl GUI → choose cluster → Backups***.
- **Maintenance mode**: Activate or schedule maintenance for nodes. These features are available at ***ClusterControl GUI → choose cluster → Nodes → Actions → Schedule maintenance***.
- **SSH console**: Directly access nodes via a web-based SSH console. This feature is available at ***ClusterControl GUI → choose cluster → Nodes → Actions → SSH Console***.

1.23.12 Step 10: Scale your cluster

A key advantage of ClusterControl is easy scalability. You can add more replica nodes to handle greater read workloads or to distribute backups, analytics, or failover targets.

1. To add a new replica, go to ***ClusterControl GUI → choose cluster → Actions → Add new → Replica node***.
2. In the **Create a replica node** wizard, configure:

Create a replica node

(1) Node configuration

- **Port**: 5432 (or your chosen port)
- **Data directory**: `/var/lib/pgsql`
- **Install software**: On
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(2) Advanced settings

- **Include in load balancer set**: On (if you want it in HAProxy)
- **Synchronous replication**: 0ff (default)
- **Instance name**: Enter instance name
- **Version**: Pre-selected by default

(3) Add node

- **Primary node**: Pick the primary node from the dropdown `192.168.99.101`.
- **Node**: Type the new replica node IP, then `Enter` and wait until green:

(4) Preview

- Confirm your deployment summary.

3. Click **Finish** to trigger the job.
4. Monitor the deployment from **Activity center**. When complete, the new replica node appears in **Nodes** page.

1.23.13 Conclusion

Congratulations! You've successfully deployed, monitored, managed, and scaled your first PostgreSQL Streaming Replication cluster with ClusterControl. Explore additional features like failover management, advanced monitoring, backup management, and more to further optimize your PostgreSQL infrastructure for high availability and performance.

This guide will lead you through setting up your first TimescaleDB streaming replication cluster (primary-replica) with PgBouncer as a load balancer using ClusterControl. By following this tutorial, you'll have a fully functional database cluster that you can effortlessly monitor, manage, and scale.

TimescaleDB Streaming Replication is a built-in feature that allows data from a primary TimescaleDB server to be streamed in real-time to one or more replica servers. This architecture improves read scalability, provides data redundancy, and enhances high availability when combined with a failover mechanism.

See also

- TimescaleDB Documentation
- PgBouncer Documentation

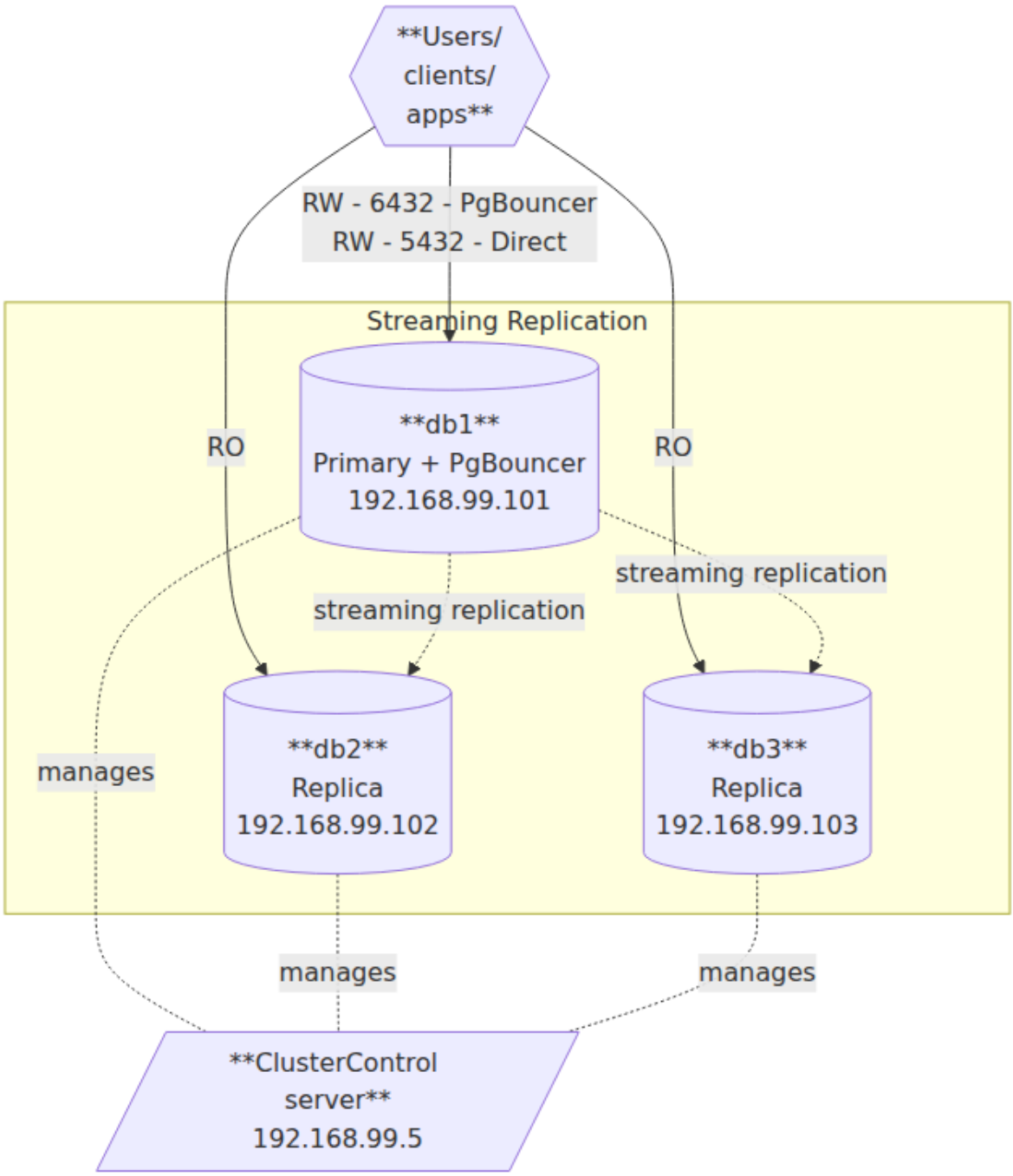
1.24 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl is installed and running. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- At least four hosts (bare-metal or virtual machines) with a supported OS:
  - One host for the TimescaleDB primary server
  - Two hosts for the TimescaleDB replica servers
  - One host for the PgBouncer load balancer
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to keep their clocks synchronized.

1.25 Architecture

Below is a simplified diagram of the final architecture:



1.26 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes:

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.100  # lb1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102  # db2
```

(continues on next page)

(continued from previous page)

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.103  # db3
```

3. Test connectivity from the ClusterControl server to each host:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.103 "ls /root"
```

1.27 Step 2: Deploy a new cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose "TimescaleDB" from the **Database** dropdown and the appropriate **Vendor** and **Version** (e.g., "TimescaleDB" and "16") from the wizard. Click **Continue**.
4. In the **Deploy TimescaleDB wizard**, configure the cluster as follows:

Deploy TimescaleDB wizard

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g., My First TimescaleDB Cluster).
- **Tags**: Enter one or more tags, separated by pressing "Enter" (e.g., production, replication, dc1).

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave blank if using key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server data directory**: `/var/lib/pgsql` (default varies by distro)
- **Server port**: 5432 (default)
- **User**: `postgres`
- **Password**: Enter a secure password for the TimescaleDB `postgres` superuser
- **Repository**: Use vendor repositories (default)

- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Primary node**:
  - Specify "192.168.99.101" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.102" as replica and press ++enter++.
  - Specify "192.168.99.103" as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Turn "On" **Synchronous Replication** if required.
- Proceed only if all nodes are reachable (shown in green).

(5) Extensions

- Leave the toggle button "Off" if you don't require the extensions, else turn "On" "pgvector" or "PostGIS" if you want to enable them.

(6) Preview

- Review the summary. You can go back to modify any configuration. The settings persist until you exit the wizard.

5. Click Finish to start the deployment.
6. ClusterControl will provision your TimescaleDB streaming replication cluster. Monitor progress from **Activity center**. This process can take a few minutes. Once complete, the cluster appears on the Home page.

1.28 Step 3: Monitor your cluster

After deployment, the **Home** page provides an overview of your cluster's health, node status, alarms, automatic recovery status, and topology visualization.

1.29 Step 4: Deploy PgBouncer

Integrating PgBouncer with TimescaleDB improves connection pooling and scalability. Follow these steps to deploy PgBouncer with ClusterControl:

1. Navigate to ***ClusterControl → Clusters → locate the cluster → Actions → Add new → Load balancer → Create balancer → PgBouncer → Continue***.
2. In the **Create a load balancer** wizard, configure PgBouncer:

Create a load balancer

(1) Authentication

- **PgBouncer admin user**: Enter a PgBouncer admin user. For example: `pgbouncer`.
- **PgBouncer admin password**: : Enter the password.

(2) Add nodes

- **PgBouncer node**: Select the TimescaleDB node from the dropdown menu and wait until the status turned green.
- **Listen port**: 6432 (default).

(4) Preview

- Review the summary. You can go back to any previous section to modify configurations if needed.

3. Click **Finish** to deploy PgBouncer.
4. ClusterControl provisions the PgBouncer instance. Monitor the progress from Activity center. Once done, the load balancer appears under **Nodes**.

1.30 Step 5: Creating connection pool for PgBouncer

Now that the PgBouncer is deployed, the next step is to create the connection pool. To create a connection pool:

1. Navigate to **ClusterControl → Clusters → Nodes**, click on the node that is showing PgBouncer under **Type** category.
2. In the **Nodes details** page, click on **Create new pool**:
3. In the **Create new pool** wizard, configure connection pool:

Create new pool

Create new pool

- **PgBouncer host name**: Pick the PgBouncer node.
- **Pool Name**: : Enter connection pool name.
- **Username**: select the database user from the dropdown menu.
- **Pool Mode**: Select one of the following modes:
  - `session`: Releases server after client disconnects.
  - `transaction`: Releases server after transaction completes.
  - `statement`: Releases server after each query. Multi-statement transactions are not allowed in this mode.
- **Pool Size**: Leave empty to use the default.
- **Max Connections**: Enter the maximum number of connections.

4. Click **Add** to create the connection pool.

Once created, PgBouncer will begin managing connections based on your configured pool parameters.

1.31 Step 6: Connecting to the database cluster

For read-write workloads, connect to the primary TimescaleDB serrver, for example:

```
psql -U postgres -h 192.168.99.101 -p 5432
```

1.32 Step 7: Connecting to the database cluster via PgBouncer

After creating a connection pool in ClusterControl, you can connect to that specific pool using the format:

```
psql -U <username> -h <pgbouncer_host> -p <pgbouncer_port> -d <pool_name>
```

Example: If you created a pool named `app_pool` for user `appuser`, and PgBouncer runs on 192.168.99.101:

```
psql -U appuser -h 192.168.99.101 -p 6432 -d app_pool
```

!!! important The `-d` parameter must match the pool name you defined during pool creation (not necessarily the actual database name).

Parameters:

- `U`: The database user assigned to the pool
- `h`: PgBouncer host/IP
- `p`: PgBouncer listening port (default: 5432)
- `d`: The pool name (acts like the database name to PgBouncer)

1.33 Step 8: Enable automatic backups

To protect your data, set up automatic backups. ClusterControl supports both logical (pg_dump) and physical (pg_basebackup, pgBackRest) backups for TimescaleDB.

1. Go to ***ClusterControl → Clusters → choose the cluster → Backups*** to open the backup overview.
2. Go to ***Create Backup → Schedule a Backup***. The backup wizard appears.
3. In the **Create a backup schedule** wizard:

Create a backup schedule

(1) Configuration

- **Schedule name**: e.g., `Daily Backup`
- **Cluster**: (auto-selected)
- **Backup host**: `db3` (select one of the replicas to offload backup load)
- **Backup method**: `pg_dump` or `pg_basebackup`
- **PITR enabled**: On (default for other than pgbasebackup)
- **Upload backup to cloud**: Off (default, can be enabled if needed)

(2) Advanced settings

- **Compression**: On
- **Compression level**: 6
- **Enable encryption**: Off or On (optional)
- **Verify backup**: Off (default)
- **Retention**: On, configure how many backups to keep
- **Failover backup if node is down**: Off (default)

(3) Storage

- **Storage location**: `Store on controller` (default)
- **Storage directory**: `/root/backups` (default)
- **Backup subdirectory**: `BACKUP-%I` (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Timezone**: Your preferred timezone

(5) Preview

- Review the summary. You can always go back to any previous section to modify settings.

4. Click Create to set the backup schedule.

ClusterControl will handle the scheduled backups automatically. Advanced backup features like partial backups, encryption, verification, throttling, and point-in-time recovery can also be configured.

1.34 Step 9: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.

6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.35 Step 10: Manage your cluster

ClusterControl offers many features for ongoing management:

- **PostgreSQL upgrades**: Perform rolling version upgrade to keep the cluster up-to-date. This feature is available at ***ClusterControl GUI → choose cluster → Actions → Upgrades***.
- **Node management**: Start/stop/restart a node, promote a replica to primary, reboot host, etc. These features are available at ***ClusterControl GUI → choose cluster → Nodes → Actions***.
- **Connection pool management**: Manage and configure PgBouncer connection pools and databases. These features are available at ***ClusterControl GUI → choose cluster → Nodes → Actions*** (for PgBouncer nodes).
- **Configuration management**: Adjust TimescaleDB settings globally or on individual nodes. This feature is available at ***ClusterControl GUI → choose cluster → Manage → Configuration***.
- **User management**: Manage TimescaleDB roles and privileges. These features are available at ***ClusterControl GUI → choose cluster → Manage → DB Users***.
- **Backup management**: Create, schedule, restore, perform point-in-time recovery, upload backups to cloud, verify backups, and set retention. These features are available at ***ClusterControl GUI → choose cluster → Backups***.
- **Maintenance mode**: Activate or schedule maintenance for nodes. These features are available at ***ClusterControl GUI → choose cluster → Nodes → Actions → Schedule maintenance***.
- **SSH console**: Directly access nodes via a web-based SSH console. This feature is available at ***ClusterControl GUI → choose cluster → Nodes → Actions → SSH Console***.

1.36 Conclusion

Congratulations! You have successfully deployed a TimescaleDB streaming replication with PgBouncer as a load balancer using ClusterControl. Explore additional features such as automatic failover, monitoring, and backup strategies to enhance your database's reliability and performance.

1.37 PostgreSQL Logical Replication

This tutorial introduces how to implement PostgreSQL Logical Replication in a bi-directional architecture using ClusterControl. It is particularly suited for environments that require multi-region write availability, zero-downtime upgrades, or active-active application architectures.

1.37.1 Introduction

PostgreSQL supports two replication paradigms:

- **Streaming Replication** — built-in, synchronous or asynchronous binary-level replication from primary to read-only standbys.
- **Logical Replication** — table-level, row-based, asynchronous replication using a publish/subscribe model.

Logical replication is ideal when:

- You want granular control (replicating specific tables or schemas).
- You need to replicate between different PostgreSQL versions.
- You require multi-primary setups, like bi-directional replication.

Warning

Warning!

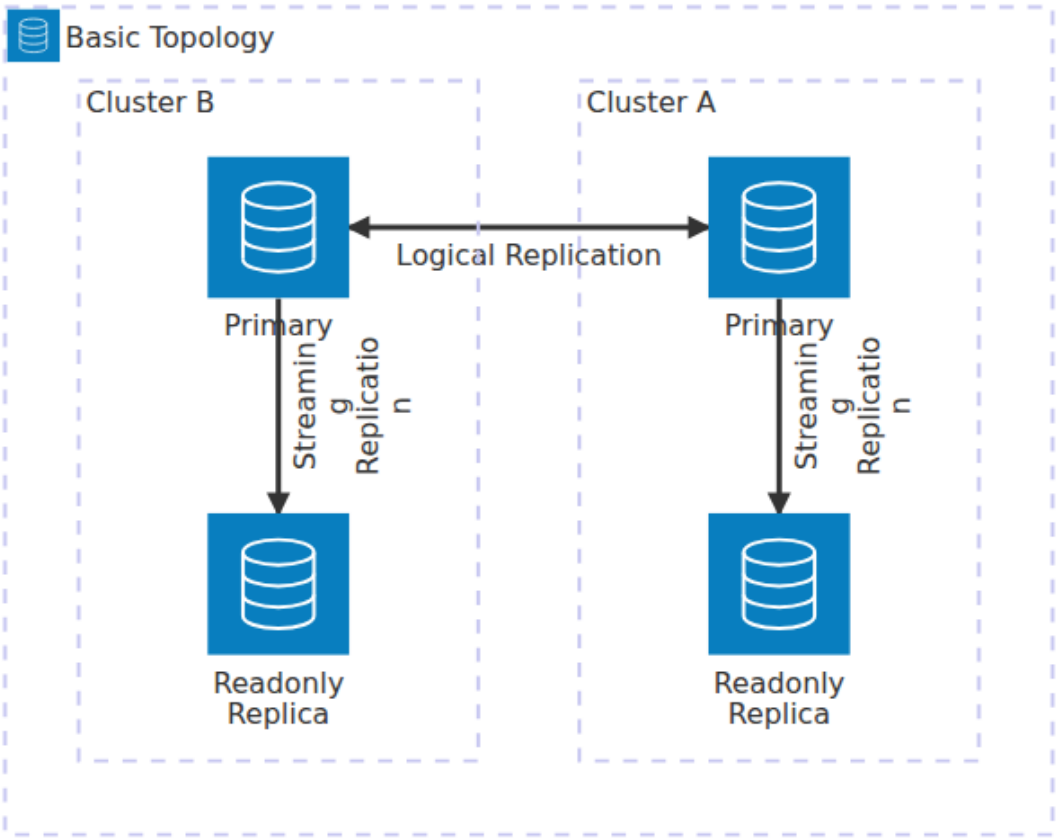
Logical replication does not replicate DDL changes or automatically resolve conflicts.

1.37.2 Architecture

The bi-directional setup supported by ClusterControl consists of two PostgreSQL streaming replication clusters, each configured with:

- One primary node
- One or more read-only replicas
- Logical replication enabled between the two primary nodes

Below is a high-level architecture diagram of the final architecture:



1.37.3 Prerequisites

Ensure you have:

- ClusterControl version 2.3.1 or higher. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- The number of nodes that you want to include in the clusters with a supported OS.
- Network connectivity between all the database nodes and ClusterControl.
- SSH access to all the nodes.

1.37.4 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes (specify the password if prompted):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.101   # db1
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.102   # db2
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.103   # db3
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.99.104   # db4
```

If the target node only requires a key-based authentication, you can use the `-o` flag to customize the SSH option:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.101  # db1
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.102  # db2
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.103  # db3
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.99.104  # db4
```

3. Perform some tests and make sure you can execute the following commands from the ClusterControl server without any password prompt:

```
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.103 "ls /root"
ssh root@192.168.99.104 "ls /root"
```

1.37.5 Step 2: Deploy a new cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose "PostgreSQL Logical" from the **Database** dropdown and the appropriate **Vendor** and **Version** (e.g., "PostgreSQL" and "17") from the wizard. Click **Continue**.
4. In the **Deploy PostgreSQL Logical cluster wizard**, configure the cluster as follows:

Deploy PostgreSQL Logical cluster wizard

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g., My First PostgreSQL Cluster).
- **Tags**: Enter one or more tags, separated by pressing "Enter" (e.g., production, replication, dc1).

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave blank if using key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server data directory**: `/var/lib/pgsql` (default varies by distro)
- **Server port**: 5432 (default)
- **User**: `postgres`
- **Password**: Enter a secure password for the PostgreSQL `postgres` superuser
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Cluster 1**: It means the first PostgreSQL Streaming Replication cluster.
- **Primary node**:
  - Specify "192.168.99.101" as the primary node for Cluster 1 and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.102" as replica and press ++enter++.
  - You can add more replica nodes here.
  - Turn "On" **Synchronous Replication** if required.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- **Cluster 2**: It means the second PostgreSQL Streaming Replication cluster.
- **Primary node**:
  - Specify "192.168.99.103" as the primary node for Cluster 2 and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.104" as replica and press ++enter++.
  - You can add more replica nodes here.
  - Turn "On" **Synchronous Replication** if required.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(5) Extensions

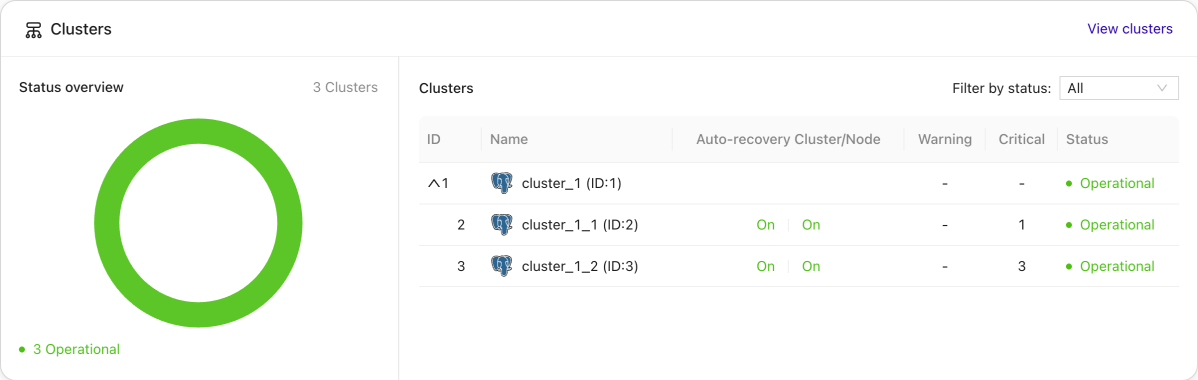
- Leave the toggle button "Off" if you don't require the extensions, else turn "On" "pgvector" or "PostGIS" if you want to enable them.

(6) Preview

- Review the summary. You can go back to modify any configuration. The settings persist until you exit the wizard.

5. Click **Finish** to start the deployment.
6. ClusterControl will now start provisioning both PostgreSQL streaming replication clusters. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the clusters will appear in the

Home page.



1.37.6 Step 3: Publication and Subscription

Now, for PostgreSQL Logical Replication, you have to create a Publication (a set of tables to be replicated), and a Subscription to access this Publication and stream changes.

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. Choose your PostgreSQL Logical cluster from the list.
3. Go to **Create publications / subscriptions**.
4. Create a new **publication** to publish changes to a set of tables, which can then be consumed by logical replication subscribers when creating a **subscription**:

publication

(1) Create publication

- **Logical cluster**: The selected PostgreSQL Logical cluster.
- **Publisher cluster**: Choose one of the PostgreSQL Streaming Replication clusters.
- **Database**: Choose the databases to be replicated.
- **Publication name**: Create a name for the new **publication**.
- **Include all tables**: On (default).
- Then, press on **Create publication**.

(2) Create subscription

- **Logical cluster**: The selected PostgreSQL Logical cluster.
- **Subscriber cluster**: Choose one of the PostgreSQL Streaming Replication clusters.
- **Subscription name**: Create a name for the new **subscription**.
- **Publication**: Choose an existing **publication**.
- **Database**: Choose the database where to replicate data.
- **Copy data**: On (default). This flag should be false when creating second subscription on a bi-directional link. Specifies whether to copy pre-existing data in the publications that are being subscribed to when the replication starts.

- **Origin**: **None** (default). Set in **None** means the subscription will only replicate changes that do not have an associated origin (i.e., changes made locally). If origin is set in **Any**, the subscription will replicate all changes regardless of their origin, including those already replicated from other nodes.
- Then, press on **Create subscription**.
- **WARNING: Setting origin=any can lead to a potential infinite loop in bi-directional replication setups, as changes replicated from one node can be sent back to the originating node.**

For more information about each PostgreSQL Streaming Replication setup, refer to *PostgreSQL Streaming Replication*

This guide will lead you through deploying your first MongoDB ReplicaSet using ClusterControl. You'll end up with a fully functional, production-ready MongoDB cluster that's equipped for monitoring, automatic failover, backups, scaling, and alerting.

MongoDB ReplicaSet is a built-in high availability feature that replicates data from a primary node to one or more secondary nodes. ClusterControl simplifies the setup process, offering automation, visibility, and lifecycle management.

See also

- MongoDB Replica Set Docs

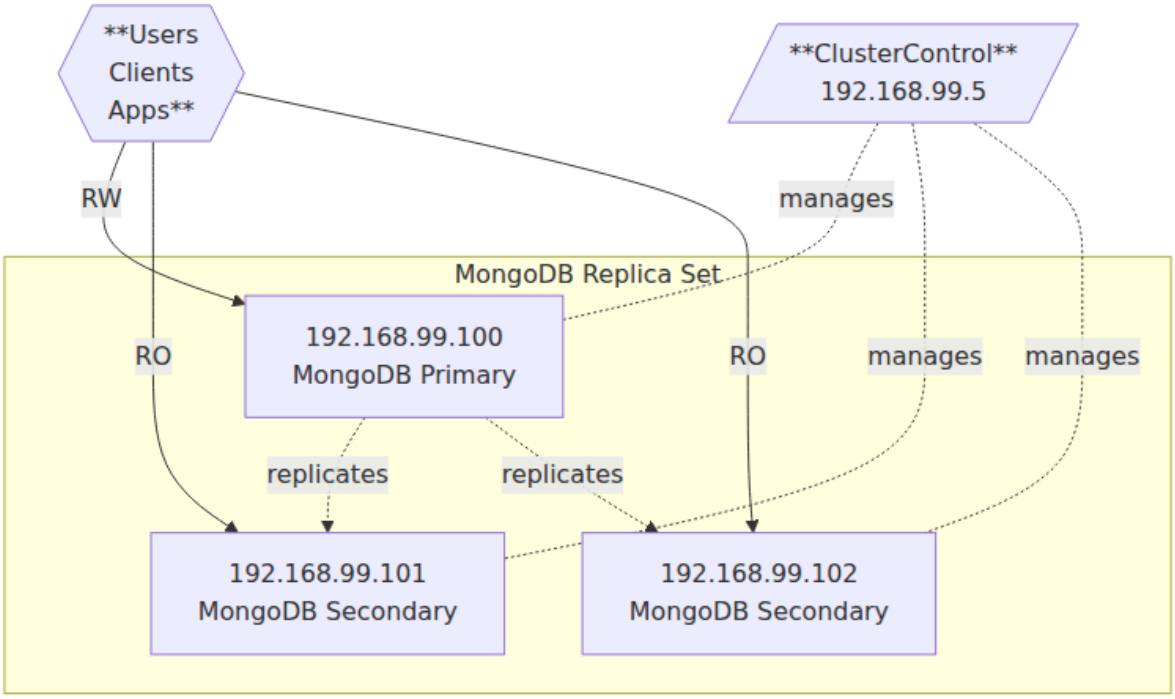
1.38 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
  - One host for the MongoDB primary server.
  - Two hosts for the MongoDB replica servers.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to keep their clocks synchronized.

1.39 Architecture

Below is the architecture diagram of the replica set:



1.40 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.100  # Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.101  # Secondary 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.102  # Secondary 2
```

3. Test connectivity from the ClusterControl server to each host:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
```

1.41 Step 2: Deploy MongoDB replica set

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose "MongoDB" from the **Database** dropdown and the appropriate **Vendor** and **Version** (e.g., "MongoDB" and "7.0") from the wizard. Click **Continue**.
4. In the **Deploy MongoDB wizard**, configure the cluster as follows:

Deploy MongoDB wizard

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g., My First MongoDB Cluster).
- **Tags**: Enter one or more tags, separated by pressing "Enter" (e.g., production, replication, dc1).

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave blank if using key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 27017 (default)
- **Server data directory**: `/var/lib/mongodb`
- **User**: Enter MongoDB admin user eg: `mongoadm`
- **Password**: Enter a secure password for the MongoDB admin
- **ReplicaSet name**: Enter your ReplicaSet name
- **Configuration template**: mongodb.conf.7.0.percona (default)
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Primary node**:
  - Specify "192.168.99.100" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.101" as replica and press ++enter++.
  - Specify "192.168.99.102" as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(6) Preview

- Review the summary. You can go back to modify any configuration. The settings persist until you exit the wizard.

5. Click Finish to start the deployment.

6. ClusterControl will provision your MongoDB ReplicaSet cluster. Monitor progress from **Activity center**. This process can take a few minutes. Once complete, the cluster appears on the Home page.

1.42 Step 3: Monitor your cluster

After deployment, the **Home** page provides an overview of your cluster's health, node status, alarms, automatic recovery status, and topology visualization.

1.43 Step 4: Schedule automatic backups

To protect your data, set up automatic backups:

1. Go to ***ClusterControl → Clusters → choose the cluster → Backups*** to open the backup overview.
2. Go to ***Create Backup → Schedule a Backup***. The backup wizard appears.
3. In the **Create a backup schedule** wizard:

Create a backup schedule

(1) Configuration

- **Schedule name**: Enter the backup name, eg: `DailyBackup`
- **Cluster**: Auto selected by default.
- **Backup method**: mongodump.
- **Upload to cloud**: Off (enable if configured)

(2) Advanced settings

- **Retention**: 31 Days (default)
- **Enable encryption**: `Off` (default)

(3) Configure storage

- **Storage host**: Select your storage host
- **Storage directory**: Set accordingly
- **Backup subdirectory**: Set accordingly

(5) Schedule

- **Set backup schedule**: Simple
- **Every**: Your preferred time
- **Timezone**: Your preferred timezone

(4) Preview

- Review and click **Create**

4. Click **Create** to set the backup schedule.

1.44 Step 5: Configure alerts and notifications

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.45 Step 6 Connect to the cluster

Use the Mongo shell or your application driver:

```
mongosh -u <mongo_admin> -p <password> --authenticationDatabase admin --
↪host 192.168.99.100
```

Check replica set status:

```
rs.status()
```

1.46 Step 7: Cluster management operations

ClusterControl supports:

- Adding/removing replica nodes
- Performing rolling upgrades
- Resyncing lagging nodes

- Manual or automatic failover
- Customizing MongoDB configuration
- Upgrading MongoDB versions
- Deploying sharded clusters (optional)
- Managing backup schedules and restores
- User and role management

All operations can be executed safely from the UI without logging into individual nodes.

1.47 Conclusion

You have successfully deployed a MongoDB ReplicaSet using ClusterControl. Your cluster is now fault-tolerant, secure, and manageable through a unified interface.

This guide will lead you through deploying your first MongoDB Enterprise Replica Set using ClusterControl. You'll end up with a fully functional, production-ready MongoDB cluster that's equipped for monitoring, automatic failover, backups, scaling, and alerting.

MongoDB Community and MongoDB Enterprise use the same core database engine and offer identical functionality for data modeling, querying, replication, sharding, and transactions, but they differ significantly in security, compliance, and operational features. MongoDB Enterprise adds capabilities required in regulated and corporate environments, such as encryption at rest, auditing, LDAP and Kerberos authentication, FIPS compliance, enterprise key management, and commercial support, none of which are available in the Community edition.

MongoDB Enterprise Replica Set is a built-in high availability feature that replicates data from a primary node to one or more secondary nodes. ClusterControl simplifies the setup process, offering automation, visibility, and lifecycle management.

See also

- MongoDB Enterprise Replica Set Docs

1.48 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
  - One host for the MongoDB primary server.
  - Two hosts for the MongoDB replica servers.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to keep their clocks synchronized.

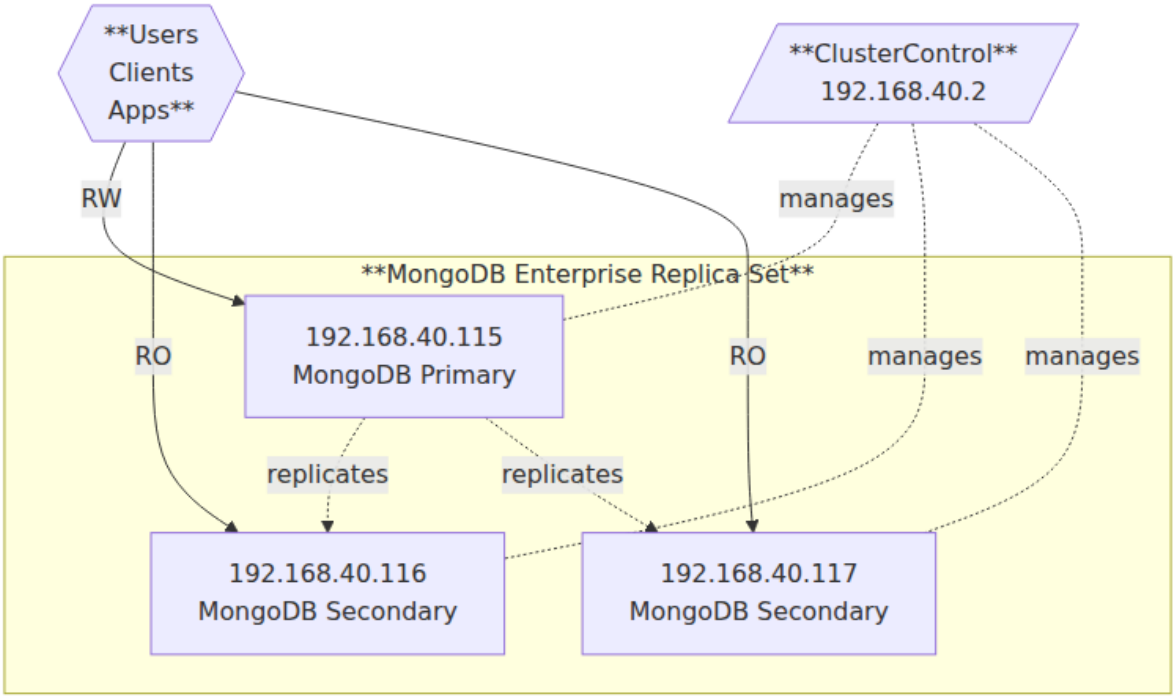
Warning

Attention

For production usage, ensure you have a valid MongoDB Enterprise commercial subscription license for your cluster. MongoDB Enterprise can be installed without a license but only permitted for evaluation purposes.

1.49 Architecture

Below is the architecture diagram of the replica set:



1.50 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.115  # Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.116  # Secondary 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.117  # Secondary 2
```

3. Test connectivity from the ClusterControl server to each host:

```
ssh root@192.168.40.115 "ls /root"
ssh root@192.168.40.116 "ls /root"
ssh root@192.168.40.117 "ls /root"
```

1.51 Step 2: Deploy MongoDB Enterprise Replica Set

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose "MongoDB" from the **Database** dropdown and the appropriate **Vendor** and **Version** (e.g., "MongoDB" and "7.0") from the wizard. Click **Continue**.
4. In the **Deploy MongoDB wizard**, configure the cluster as follows:

Deploy MongoDB wizard

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g., My First MongoDB Cluster).
- **Tags**: Enter one or more tags, separated by pressing "Enter" (e.g., production, replication, dc1).

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave blank if using key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 27017 (default)
- **Server data directory**: `/var/lib/mongodb`
- **User**: Enter MongoDB admin user eg: `mongoadm`
- **Password**: Enter a secure password for the MongoDB admin
- **Replica Set name**: Enter your Replica Set name
- **Configuration template**: mongodb.conf.7.0.percona (default)
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: On (default)

(4) Add nodes

- **Primary node**:
  - Specify "192.168.40.115" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.40.116" as replica and press ++enter++.

- Specify “192.168.40.117” as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(6) Preview

- Review the summary. You can go back to modify any configuration. The settings persist until you exit the wizard.

5. Click Finish to start the deployment.
6. ClusterControl will provision your MongoDB Enterprise Replica Set cluster. Monitor progress from **Activity center**. This process can take a few minutes. Once complete, the cluster appears on the Home page.

1.52 Step 3: Monitor your cluster

After deployment, the **Home** page provides an overview of your cluster's health, node status, alarms, automatic recovery status, and topology visualization.

1.53 Step 4: Schedule automatic backups

To protect your data, set up automatic backups:

1. Go to ***ClusterControl → Clusters → choose the cluster → Backups*** to open the backup overview.
2. Go to ***Create Backup → Schedule a Backup***. The backup wizard appears.
3. In the **Create a backup schedule** wizard:

Create a backup schedule

(1) Configuration

- **Schedule name**: Enter the backup name, eg: `DailyBackup`
- **Cluster**: Auto selected by default.
- **Backup method**: mongodump.
- **Upload to cloud**: Off (enable if configured)

(2) Advanced settings

- **Retention**: 31 Days (default)
- **Enable encryption**: `Off` (default)

(3) Configure storage

- **Storage host**: Select your storage host
- **Storage directory**: Set accordingly
- **Backup subdirectory**: Set accordingly

(5) Schedule

- **Set backup schedule**: Simple
- **Every**: Your preferred time
- **Timezone**: Your preferred timezone

(4) Preview

- Review and click **Create**

4. Click **Create** to set the backup schedule.

1.54 Step 5: Configure alerts and notifications

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.55 Step 6: Connect to the cluster

Use the Mongo shell or your application driver:

```
mongosh -u <mongo_admin> -p <password> --authenticationDatabase admin --
↪host 192.168.40.115
```

Check replica set status:

```
rs.status()
```

1.56 Step 7: Verify Enterprise installation

Based on step 6, as you are still connected through `mongosh`, you can verify the MongoDB node if it is an Enterprise version by running the following command in:

```
db.serverBuildInfo().modules
```

1.57 Step 8: Cluster management operations

For MongoDB Enterprise replica set, ClusterControl supports:

- Adding/removing replica nodes
- Performing rolling upgrades
- Resyncing lagging nodes
- Manual or automatic failover
- Customizing MongoDB configuration
- Upgrading MongoDB versions
- Deploying sharded clusters (optional)
- Managing backup schedules and restores
- User and role management

All operations can be executed safely from the GUI without logging into individual nodes.

1.58 Conclusion

You have successfully deployed a MongoDB Enterprise Replica Set using ClusterControl. Your cluster is now fault-tolerant, secure, and manageable through a unified interface.

This guide will lead you through deploying your first MongoDB Sharded Cluster using ClusterControl. By the end, you'll have a production-ready sharded setup that's scalable, fault-tolerant, and fully managed through ClusterControl.

A MongoDB sharded cluster is designed to scale horizontally by partitioning data across multiple shard nodes. Each shard can be a replica set to ensure high availability. ClusterControl automates the deployment, configuration, and ongoing management of a sharded cluster.

See also

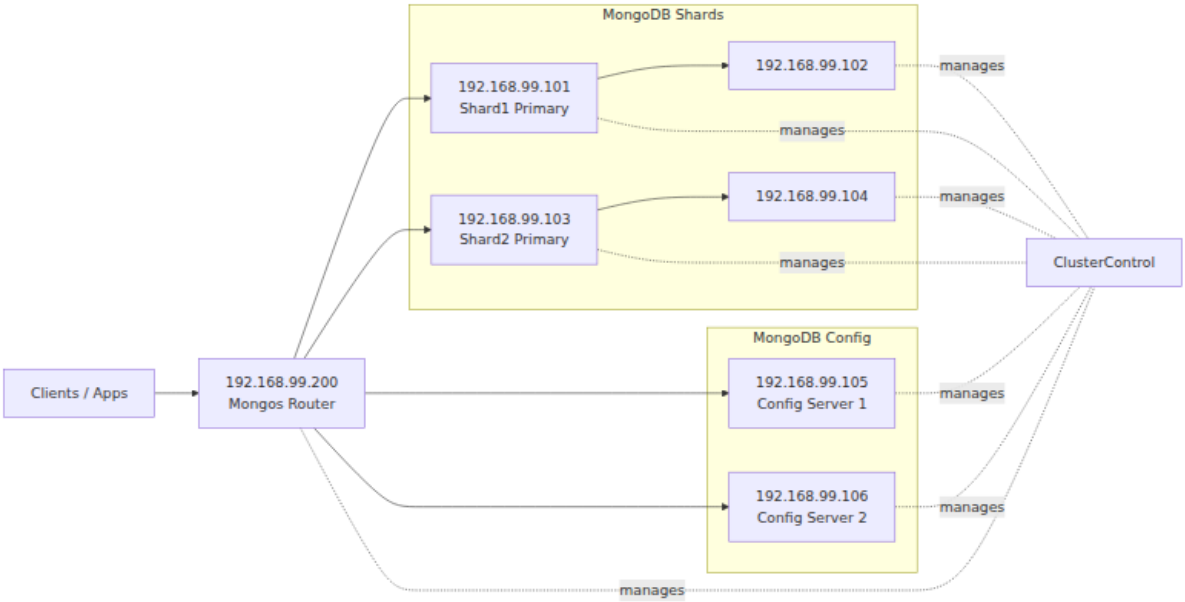
- MongoDB Sharding Docs

1.59 Prerequisites

Make sure you have the following before you begin:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quick-start* or use the *Installer Script*.
- At least six hosts (bare-metal or virtual machines):
  - Three hosts for MongoDB shard replica sets (at least 2 nodes per shard)
  - Two config server nodes
  - One mongos router node (can share with ClusterControl)
- All nodes must have internet access during deployment.
- SSH key-based access from ClusterControl to all target hosts.
- NTP service must be enabled and synchronized on all nodes.

1.60 Architecture



1.61 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

2. Copy the SSH public key to all target nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.200  # Mongos Router
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.101  # Shard 1 Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.102  # Shard 1 Secondary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.103  # Shard 2 Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.104  # Shard 2 Secondary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.105  # Config Server 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.106  # Config Server 2
```

3. Test connectivity from the ClusterControl server to each host:

```
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
ssh root@192.168.99.103 "ls /root"
ssh root@192.168.99.104 "ls /root"
ssh root@192.168.99.105 "ls /root"
ssh root@192.168.99.106 "ls /root"
```

1.62 Step 2: Deploy MongoDB Sharded Cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose "MongoDB Shards" from the **Database** dropdown and the appropriate **Vendor** and **Version** (e.g., "MongoDB" and "7.0") from the wizard. Click **Continue**.
4. In the **Deploy MongoDB wizard**, configure the cluster as follows:

Deploy MongoDB wizard

(1) Cluster details

- **Name**: Enter a name for your cluster (e.g., My First MongoDB Shard Cluster).
- **Tags**: Enter one or more tags, separated by pressing "Enter" (e.g., production, replication, dc1).

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo password**: (leave blank if using key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **Server port**: 5432 (default)
- **Server data directory**: `/var/lib/mongodb`
- **User**: Enter MongoDB admin user eg: `mongoadm`
- **Password**: Enter a secure password for the MongoDB admin
- **Router Configuration template**: mongodb.conf.7.0.percona (default)
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: On (default)

(4) Add Configuration Servers & Routers

- **Configuration Servers**:
  - Specify "192.168.99.105" and "192.168.99.106" as the Configuration Servers and press ++enter++.
- **Router/Mongos node**:
  - Specify "192.168.99.200" as Mongo Router and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(5) Add Shard

- **Define shard name**:
  - Specify the shard name for the First Shard.
- **Nodes Shard**:
  - Specify "192.168.99.101" as Primary Shard and press ++enter++.
  - Specify "192.168.99.102" as Secondary/Replica Shard and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).
- **Define another shard name**:
  - Press ***Add another shard*** and specify the shard name for the Second Shard.
- **Nodes Shard**:
  - Specify "192.168.99.103" as Primary Shard and press ++enter++.
  - Specify "192.168.99.104" as Secondary/Replica Shard and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(6) Preview

- Review the summary. You can go back to modify any configuration. The settings persist until you exit the wizard.

5. Click Finish to start the deployment.

6. ClusterControl will provision your MongoDB Sharded Cluster. Monitor progress from **Activity center**. This process can take a few minutes. Once complete, the cluster appears on the Home page.

1.63 Step 3: Monitor your cluster

After deployment, the **Home** page provides an overview of your cluster's health, node status, alarms, automatic recovery status, and topology visualization.

1.64 Step 4: Schedule automatic backups

ClusterControl uses Percona Backup for MongoDB as default backup method for MongoDB Sharded Cluster. Before installing Percona Backup for MongoDB and configuring the backup, ensure that the servers have mounted the shared storage on all data nodes and config servers. The articles will not cover how to set up the NFS server and client.

To protect your data, set up automatic backups:

1. Go to ***ClusterControl → Clusters → choose the cluster → Backups*** to open the backup overview.
2. Go to ***Create Backup → Schedule a Backup***. The backup wizard appears.
3. In the **Create a backup schedule** wizard:

Create a backup schedule

(1) Configuration

- **Schedule name**: Enter the backup name, eg: `DailyBackup`
- **Cluster**: Auto selected by default.
- **Backup method**: percona-backup-mongodb.

We must install the PBM (Percona Backup for MongoDB) for the first setup. just **Continue** and fill the shared storage path directory.

(2) Advanced settings

- **Retention**: 31 Days (default)
- **Enable encryption**: `Off` (default)

(3) Schedule

- **Set backup schedule**: Simple
- **Every**: Your preferred time
- **Timezone**: Your preferred timezone

(4) Preview

- Review and click **Create**

4. Click **Create** to set the backup schedule.

1.65 Step 5: Configure alerts and notifications

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.66 Step 6 Connect to the cluster

Use the Mongo shell or your application driver:

```
mongosh -u <mongo_admin> -p <password> --authenticationDatabase admin --
↪host 192.168.99.200
```

Check shard cluster status:

```
sh.status()
```

1.67 Step 7: Cluster management operations

ClusterControl supports:

- Adding/Remove shards
- Adding/remove mongos router & config server
- Add new nodes in shards

- Performing minor rolling upgrades
- Resyncing lagging nodes
- Manual or automatic failover
- Customizing MongoDB configuration
- Upgrading MongoDB versions
- Managing backup schedules and restores
- User and role management

All operations can be executed safely from the UI without logging into individual nodes.

1.68 Conclusion

You have successfully deployed a MongoDB Sharded Cluster using ClusterControl. Your cluster is now fault-tolerant, secure, and manageable through a unified interface.

1.69 Redis Replication with Sentinel

In this guide, we will walk you through setting up your first database cluster running on Redis replication with Sentinel using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

Redis is an open-source, in-memory data store that combines the speed of a cache with the flexibility of a database and the messaging capabilities of a broker. It holds data entirely in RAM for sub-millisecond read and write performance, while offering optional persistence to disk so your datasets survive restarts. Beyond simple key/value pairs, Redis natively supports rich data types — lists, sets, sorted sets, hashes, bitmaps, hyperloglogs, and streams — making it easy to implement use cases like caching, real-time analytics, leaderboards, session storage, and pub/sub messaging.

With built-in replication, automatic failover via Redis Sentinel, and sharding in Redis Cluster, it can scale seamlessly from a single-server prototype to a highly available, distributed system serving millions of operations per second.

See also

- Redis Community Edition - Getting Started
- Redis Sentinel Docs

1.69.1 Prerequisites

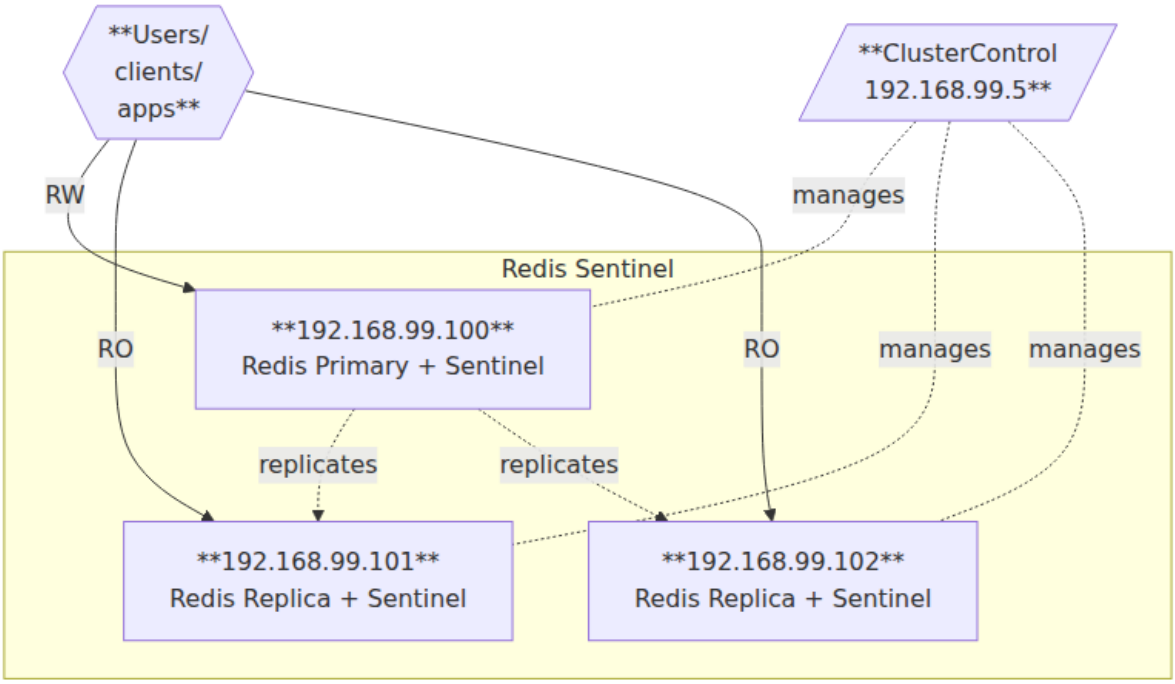
Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.

- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.69.2 Architecture

Below is a simplified diagram of the final architecture:



1.69.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key:

```
ssh-keygen -t rsa
```

2. Copy the SSH key to all Redis nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.100  # Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.101  # Replica 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.102  # Replica 2
```

3. Test the connection:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
```

1.69.4 Step 2: Deploy a Redis Sentinel Cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.

3. Choose **Redis Sentinel** from the **Database** dropdown and the **Vendor** and **Version** (e.g., "Redis" and "7.0") from the wizard. Click **Continue**.
4. In the **Deploy Redis Sentinel cluster wizard**, configure the cluster as follows:

Deploy Redis Sentinel cluster wizard

(1) Cluster details

- **Name**: My Redis Sentinel Cluster
- **Tags**: redis, sentinel, ha

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: Enabled
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(3) Node configuration

- **Redis Port**: `6379` (default)
- **Redis Sentinel Port**: `26379` (default)
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: `On` (default)
- **Authentication**
  - **Password**: Set or copy the generated password for your reference.

(4) Add nodes

- **Primary node**:
  - Specify "192.168.99.100" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.101" as replica and press ++enter++.
  - Specify "192.168.99.102" as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. Click **Finish** to begin the deployment.
6. ClusterControl will start to provision the hosts, install Redis, configure replication, and install Sentinel. Monitor the progress from **Activity Center → Jobs**.

1.69.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.69.6 Step 4: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- The primary Redis server 192.168.99.100 on port 6379. Since by default ClusterControl enforces Redis instance with TLS and authentication, you need to get the key and certificate (commonly under `/var/lib/redis` of the database node) and use the password specified during deployment with the default username "admin" to access.
- For read-only workloads, you may send the database connections to the replica nodes, 192.168.99.101 or 192.168.99.102 on port 6379.

Note

Note Most of the Redis drivers and application connectors come with a built-in support for Redis replication, clusters and Sentinel. Check your application connector's documentation for details.

Tip

You can use popular Redis database management clients like phpRedisAdmin or Redis Insight to perform database administration tasks.

1.69.7 Step 5: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For Redis replication with Sentinel, ClusterControl supports both RDB and AOF backups. To schedule a backup:

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.

2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily full backup - Redis
- **Cluster**: (leave as it is)
- **Backup method**: (leave as it is)
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Use PIGZ for parallel gzip**: Off
- **Enable encryption**: On
- **Retention**: On (default)

(3) Storage

- **Storage location**: Store on controller (default)
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size and perform restoration if necessary.

1.69.8 Step 6: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.69.9 Step 7: Manage your cluster

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → choose the cluster → Actions → Upgrades***.
- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Database user and access control management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Access control***.
- Backup management: Create, schedule, restore, upload backup to cloud, and set retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions_** and **ClusterControl GUI → choose the cluster → Backups → More_**.

- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.69.10 Step 8: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For Redis replication wiht Sentinel, you can scale out the cluster with another replica. Common use cases are to use the replica node for to scale out the read-only workloads, backups, reporting or analytics purposes.

Adding a new replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node due to initial RDB syncing from the primary.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 6379
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press ++enter++.

(3) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.69.11 Conclusion

Congratulations! You've successfully deployed, monitored, managed and scaled your first Redis replication cluster with Sentinel using ClusterControl. From here, you can explore additional features such as failover management, access control list (ACL) rules, advanced monitoring, advanced backup management, database restore and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.70 Valkey Replication with Sentinel

In this guide, we will walk you through setting up your first database cluster running on Valkey replication with Sentinel using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

Valkey is an open-source, in-memory key-value datastore designed for ultra-low-latency reads and writes, combining the roles of a cache, message broker, and primary database in a single engine. Originally forked from Valkey in 2024 (from Redis 7.2.4) under the stewardship of the Linux Foundation, Valkey retains a BSD license and is implemented in C for Unix-like systems, ensuring both performance and broad compatibility

With built-in replication, automatic failover via Valkey Sentinel, and sharding in Valkey Cluster, it can scale seamlessly from a single-server prototype to a highly available, distributed system serving millions of operations per second.

See also

- Valkey Documentation
- Valkey Documentation - Sentinel

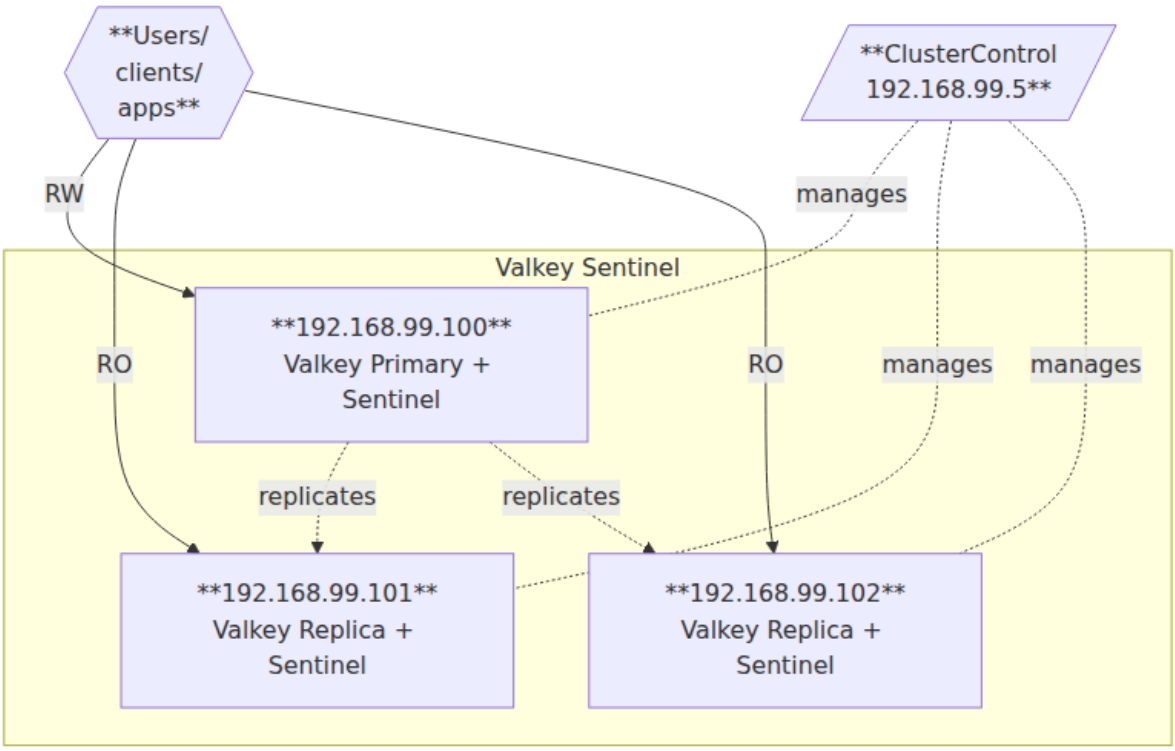
1.70.1 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.70.2 Architecture

Below is a simplified diagram of the final architecture:



1.70.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key:

```
ssh-keygen -t rsa
```

2. Copy the SSH key to all Valkey nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.100  # Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.101  # Replica 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.102  # Replica 2
```

3. Test the connection:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
```

1.70.4 Step 2: Deploy a Valkey Sentinel Cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose **Valkey Sentinel** from the **Database** dropdown and the **Vendor** and **Version** (e.g., "Valkey" and "7.0") from the wizard. Click **Continue**.
4. In the **Deploy Valkey Sentinel cluster wizard**, configure the cluster as follows:

Deploy Valkey Sentinel cluster wizard

(1) Cluster details

- **Name**: My Valkey Sentinel Cluster
- **Tags**: valkey, sentinel, ha

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: Enabled
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(3) Node configuration

- **Valkey Port**: 6379 (default)
- **Valkey Sentinel Port**: 26379 (default)
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: On (default)
- **Authentication**
  - **Password**: Set or copy the generated password for your reference.

(4) Add nodes

- **Primary node**:
  - Specify "192.168.99.100" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.99.101" as replica and press ++enter++.
  - Specify "192.168.99.102" as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. Click **Finish** to begin the deployment.
6. ClusterControl will start to provision the hosts, install Valkey, configure replication, and install Sentinel. Monitor the progress from **Activity Center → Jobs**.

1.70.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.70.6 Step 4: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- The primary Valkey server 192.168.99.100 on port 6379. Since by default ClusterControl enforces Valkey instance with TLS and authentication, you need to get the key and certificate (commonly under `/var/lib/valkey` of the database node) and use the password specified during deployment with the default username "admin" to access.
- For read-only workloads, you may send the database connections to the replica nodes, 192.168.99.101 or 192.168.99.102 on port 6379.

Note

Note Most of the Valkey drivers and application connectors come with a built-in support for Valkey replication, clusters and Sentinel. Check your application connector's documentation for details.

1.70.7 Step 5: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For Valkey replication with Sentinel, ClusterControl supports both RDB and AOF backups. To schedule a backup:

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily full backup - Valkey

- **Cluster**: (leave as it is)
- **Backup method**: (leave as it is)
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Use PIGZ for parallel gzip**: Off
- **Enable encryption**: On
- **Retention**: On (default)

(3) Storage

- **Storage location**: Store on controller (default)
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size and perform restoration if necessary.

1.70.8 Step 6: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.

3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.70.9 Step 7: Manage your cluster

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → choose the cluster → Actions → Upgrades***.
- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Database user and access control management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Access control***.
- Backup management: Create, schedule, restore, upload backup to cloud, and set retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions_** and **ClusterControl GUI → choose the cluster → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.70.10 Step 8: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For Valkey replication wiht Sentinel, you can scale out the cluster with another replica. Common use cases are to use the replica node for to scale out the read-only workloads, backups, reporting or analytics purposes.

Adding a new replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node due to initial RDB syncing from the primary.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 6379
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press ++enter++.

(3) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.70.11 Conclusion

Congratulations! You've successfully deployed, monitored, managed and scaled your first Valkey replication cluster with Sentinel using ClusterControl. From here, you can explore additional features such as failover management, access control list (ACL) rules, advanced monitoring, advanced backup management, database restore and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.71 Redis Cluster

In this guide, we will walk you through setting up your first database cluster running on Redis Cluster using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

Redis is a high-performance, in-memory data store used for caching, key-value storage, messaging, and real-time data processing. It delivers microsecond-level speed because all data is stored in RAM. Redis supports flexible data structures such as strings, lists, hashes, sets, sorted sets, streams, and JSON (via modules), making it useful for workloads like session storage, API rate limiting, leaderboards, and real-time analytics.

In standalone mode, Redis runs with a single primary node (and optional replicas). This setup is simple but has limitations: write operations cannot scale beyond a single server, all data resides on one machine, and failover requires external tooling like Sentinel.

Redis Cluster addresses these limitations by distributing data across multiple nodes using hash-slot sharding, enabling horizontal scaling and higher throughput. It also provides built-in high availability through automatic failover, where replicas can promote themselves when a primary fails. With its decentralized architecture, Redis Cluster eliminates single points of failure and is suitable for large, production-grade deployments.

See also

- Redis Community Edition - Getting Started
- Redis Cluster Specification

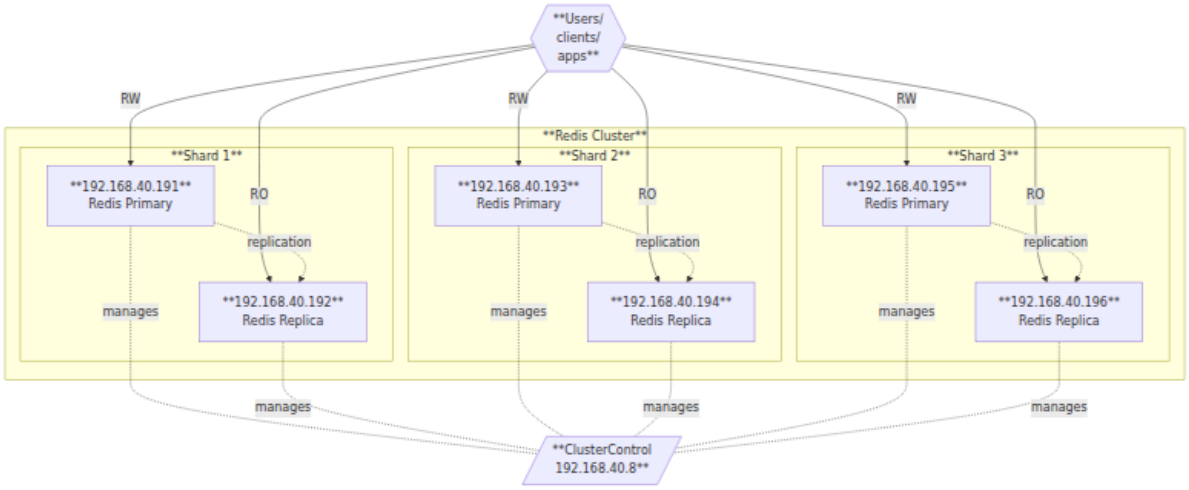
1.71.1 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.71.2 Architecture

Below is a simplified diagram of the final architecture:



1.71.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key:

```
ssh-keygen -t rsa
```

2. Copy the SSH key to all Redis nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.191  # Primary 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.192  # Replica 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.193  # Primary 2
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.194  # Replica 2
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.195  # Primary 3
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.196  # Replica 3
```

3. Test the connection:

```
ssh root@192.168.40.191 "ls /root"
ssh root@192.168.40.192 "ls /root"
ssh root@192.168.40.193 "ls /root"
ssh root@192.168.40.194 "ls /root"
ssh root@192.168.40.195 "ls /root"
ssh root@192.168.40.196 "ls /root"
```

1.71.4 Step 2: Deploy a Redis Cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose **Redis Cluster** from the **Database** dropdown and the **Vendor** and **Version** (e.g., "Redis" and "8") from the wizard. Click **Continue**.
4. In the **Deploy Redis Cluster** cluster wizard, configure the cluster as follows:

Deploy Redis Cluster

(1) Cluster details

- **Name**: My Redis Cluster
- **Tags**: redis, first-deployment

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: Enabled
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(3) Node configuration

- **Redis Port**: `6379` (default)
- **Cluster bus port **: `16379` (default)
- **Node timeout (ms)**: `15000` (default)
- **Replica validity factor**: `10` (default)
- **Configuration template**: `redis8-tls.conf` (default)
- **Repository**: Use vendor repositories (default)
- **Username**: Specify the Redis admin username. ClusterControl will create this user if not exists.
- **Password**: Specify the password for the Redis admin user.

(4) Shard 1

- **Primary node**:
  - Specify “192.168.40.191” as the primary node and press ++enter++.
- **Replica node**:
  - Specify “192.168.40.192” as replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Click on **Add another shard**. This will add another step to add the second shard.

(5) Shard 2

- **Primary node**:
  - Specify “192.168.40.193” as the primary node and press ++enter++.
- **Replica node**:
  - Specify “192.168.40.194” as replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.

- Click on **Add another shard**. This will add another step to add the third shard.

(6) Shard 3

- **Primary node**:
  - Specify "192.168.40.195" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.40.196" as replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Click **Continue**.

(7) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. Click **Finish** to begin the deployment.
6. ClusterControl will start to provision the hosts, install Redis, and configure replication. Monitor the progress from **Activity Center → Jobs**.

1.71.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.71.6 Step 4: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- Any primary Redis server (192.168.40.191, 192.168.40.193 or 192.168.40.195) on port 6379 for read-write workloads. In cluster-mode, the connection will be transparently follow redirects, even though the particular Redis node does not own the cluster slot.
- For read-only workloads, you may send the database connections to the replica nodes (192.168.40.192, 192.168.40.194 or 192.168.40.196) on port 6379.

- Since by default ClusterControl enforces Redis instance with TLS and authentication, you need to get the key and certificate (commonly under `/var/lib/redis` of the database node) and use the password specified during the deployment.

Note

Note Most of the Redis drivers and application connectors come with a built-in support for Redis replication, clusters and Sentinel. Check your application connector's documentation for details.

Tip

You can use popular Redis database management clients like phpRedisAdmin or Redis Insight to perform database administration tasks.

1.71.7 Step 5: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For Redis Cluster, ClusterControl supports both RDB and AOF backups. To schedule a backup:

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily full backup - Redis
- **Cluster**: (leave as it is)
- **Backup method**: (leave as it is)
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Use PIGZ for parallel gzip**: Off
- **Enable encryption**: On
- **Retention**: On (default)

(3) Storage

- **Storage location**: Store on controller (default). You can choose the storage host from the drop down list.
- **Storage directory**: /root/backups (default)

- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size and perform restoration if necessary.

1.71.8 Step 6: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.71.9 Step 7: Manage your cluster

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → choose the cluster → Actions → Upgrades***.
- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Database user and access control management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Access control***.
- Backup management: Create, schedule, restore, upload backup to cloud, and set retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions_** and **ClusterControl GUI → choose the cluster → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.71.10 Step 8: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For Redis Cluster, you can scale out the cluster with another replica. Common use cases are to use the replica node for to scale out the read-only workloads, backups, reporting or analytics purposes.

Adding a new replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node due to initial RDB syncing from the primary.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 6379
- **Install software**: On (default)
- **Disable firewall**: Checked (default)

- **Disable SELinux/AppArmor**: Checked (default)

(2) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press ++enter++.

(3) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.71.11 Conclusion

Congratulations! You've successfully deployed, monitored, managed and scaled your first Redis Cluster using ClusterControl. From here, you can explore additional features such as failover management, access control list (ACL) rules, advanced monitoring, advanced backup management, database restore and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.72 Valkey Cluster

In this guide, we will walk you through setting up your first database cluster running on Valkey Cluster using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

Valkey is an open-source, community-driven, in-memory key-value datastore designed for ultra-low-latency reads and writes, combining the roles of a cache, message broker, and primary database in a single engine. Originally forked from Valkey in 2024 (from Redis 7.2.4) under the stewardship of the Linux Foundation, Valkey retains a BSD license and is implemented in C for Unix-like systems, ensuring both performance and broad compatibility. It is essentially a drop-in replacement for Redis, created after Redis changed its license.

With built-in replication, automatic failover via Valkey Sentinel, and sharding in Valkey Cluster, it can scale seamlessly from a single-server prototype to a highly available, distributed system serving millions of operations per second.

With Valkey Cluster, it provides a way to run a Valkey installation where data is automatically sharded across multiple Valkey nodes. Valkey Cluster also provides some degree of availability during partitions, to continue operations when some nodes fail or are unable to communicate. However, the cluster will become unavailable in the event of larger failures (for example, when the majority of primaries are unavailable).

So, with Valkey Cluster, you get the ability to:

- Automatically split your dataset among multiple nodes.
- Continue operations when a subset of the nodes are experiencing failures or are unable to communicate with the rest of the cluster.

See also

- Valkey Documentation
- Valkey Documentation - Cluster

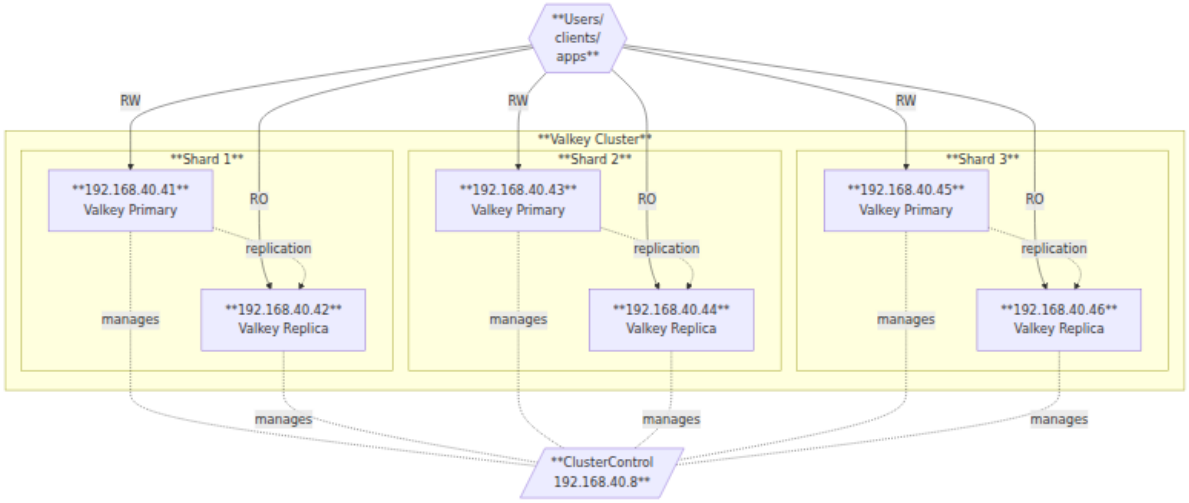
1.72.1 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quick-start* or use the *Installer Script*.
- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.72.2 Architecture

Below is a simplified diagram of the final architecture:



1.72.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key:

```
ssh-keygen -t rsa
```

2. Copy the SSH key to all Valkey nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.41  # Primary 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.42  # Replica 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.43  # Primary 2
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.44  # Replica 2
```

(continues on next page)

(continued from previous page)

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.45  # Primary 3
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.40.46  # Replica 3
```

3. Test the connection:

```
ssh root@192.168.40.41 "ls /root"
ssh root@192.168.40.42 "ls /root"
ssh root@192.168.40.43 "ls /root"
ssh root@192.168.40.44 "ls /root"
ssh root@192.168.40.45 "ls /root"
ssh root@192.168.40.46 "ls /root"
```

1.72.4 Step 2: Deploy a Valkey Cluster

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose **Valkey Cluster** from the **Database** dropdown and the **Vendor** and **Version** (e.g., "Valkey" and "8") from the wizard. Click **Continue**.
4. In the **Deploy Valkey Cluster** wizard, configure the cluster as follows:

Deploy Valkey Cluster

(1) Cluster details

- **Name**: My Valkey Cluster
- **Tags**: Valkey, first-deployment

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: Enabled
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(3) Node configuration

- **Valkey Port**: `6379` (default)
- **Cluster bus port **: `16379` (default)
- **Node timeout (ms)**: `15000` (default)
- **Replica validity factor**: `10` (default)
- **Configuration template**: `valkey8-tls.conf` (default)

- **Repository**: Use vendor repositories (default)
- **Username**: Specify the Valkey admin username. ClusterControl will create this user if not exists.
- **Password**: Specify the password for the Valkey admin user.

(4) Shard 1

- **Primary node**:
  - Specify "192.168.40.41" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.40.42" as replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Click on **Add another shard**. This will add another step to add the second shard.

(5) Shard 2

- **Primary node**:
  - Specify "192.168.40.43" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.40.44" as replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Click on **Add another shard**. This will add another step to add the third shard.

(6) Shard 3

- **Primary node**:
  - Specify "192.168.40.45" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "192.168.40.46" as replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(7) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

5. Click **Finish** to begin the deployment.
6. ClusterControl will start to provision the hosts, install Valkey, and configure replication. Monitor the progress from **Activity Center → Jobs**.

1.72.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm but the recovery procedures will be skipped.
- Topology viewer: Roll over on each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.72.6 Step 4: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- Any primary Valkey server (192.168.40.41, 192.168.40.43 or 192.168.40.45) on port 6379 for read-write workloads. In cluster-mode, the connection will be transparently follow redirects, even though the particular Valkey node does not own the cluster slot.
- For read-only workloads, you may send the database connections to the replica nodes (192.168.40.42, 192.168.40.44 or 192.168.40.46) on port 6379.
- Since by default ClusterControl enforces Valkey instance with TLS and authentication, you need to get the key and certificate (commonly under `/var/lib/valkey` of the database node) and use the password specified during the deployment.

Note

Note Most of the Valkey drivers and application connectors come with a built-in support for Valkey replication, clusters and Sentinel. Check your application connector's documentation for details.

1.72.7 Step 5: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For Valkey Cluster, ClusterControl supports both RDB and AOF backups. To schedule a backup:

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily full backup - Valkey
- **Cluster**: (leave as it is)
- **Backup method**: (leave as it is)
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Compression level**: 6
- **Use PIGZ for parallel gzip**: Off
- **Enable encryption**: On
- **Retention**: On (default)

(3) Storage

- **Storage location**: Store on controller (default). You can choose the storage host from the drop down list.
- **Storage directory**: /root/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size and perform restoration if necessary.

1.72.8 Step 6: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed

on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group where you are belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with summarized warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.72.9 Step 7: Manage your cluster

ClusterControl provides many other features for managing your cluster, including:

- Database upgrades and patching: Perform rolling upgrades to keep your cluster up to date without downtime. This feature is available at ***ClusterControl GUI → choose the cluster → Actions → Upgrades***.
- Database node management: Perform start/stop/restart node, reboot host, bootstrap cluster, activate/deactivate read-only, resync node, change primary source and rebuild replica. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Database user and access control management: Manage database users and privileges globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Access control***.
- Backup management: Create, schedule, restore, upload backup to cloud, and set retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions_** and **ClusterControl GUI → choose the cluster → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.72.10 Step 8: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For Valkey Cluster, you can scale out the cluster with another replica. Common use cases are to use the replica node for to scale out the read-only workloads, backups, reporting or analytics purposes.

Adding a new replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however expect increasing load on the source node due to initial RDB syncing from the primary.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 6379
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Add node

- **Primary node**: Choose one of the primary node from the dropdown.
- **Node**: Specify the IP address or hostname of the node that you want to add and press ++enter++.

(3) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.72.11 Conclusion

Congratulations! You've successfully deployed, monitored, managed and scaled your first Valkey Cluster using ClusterControl. From here, you can explore additional features such as failover management, access control list (ACL) rules, advanced monitoring, advanced backup management, database restore and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant. Valkey

1.73 SQL Server standalone

In this guide, we will walk you through setting up your first database cluster running on SQL Server standalone using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

SQL Server is a relational database management system (RDBMS). Applications and tools connect to a SQL Server instance or database, and communicate using Transact-SQL (T-SQL).

See also

- SQL Server on Linux

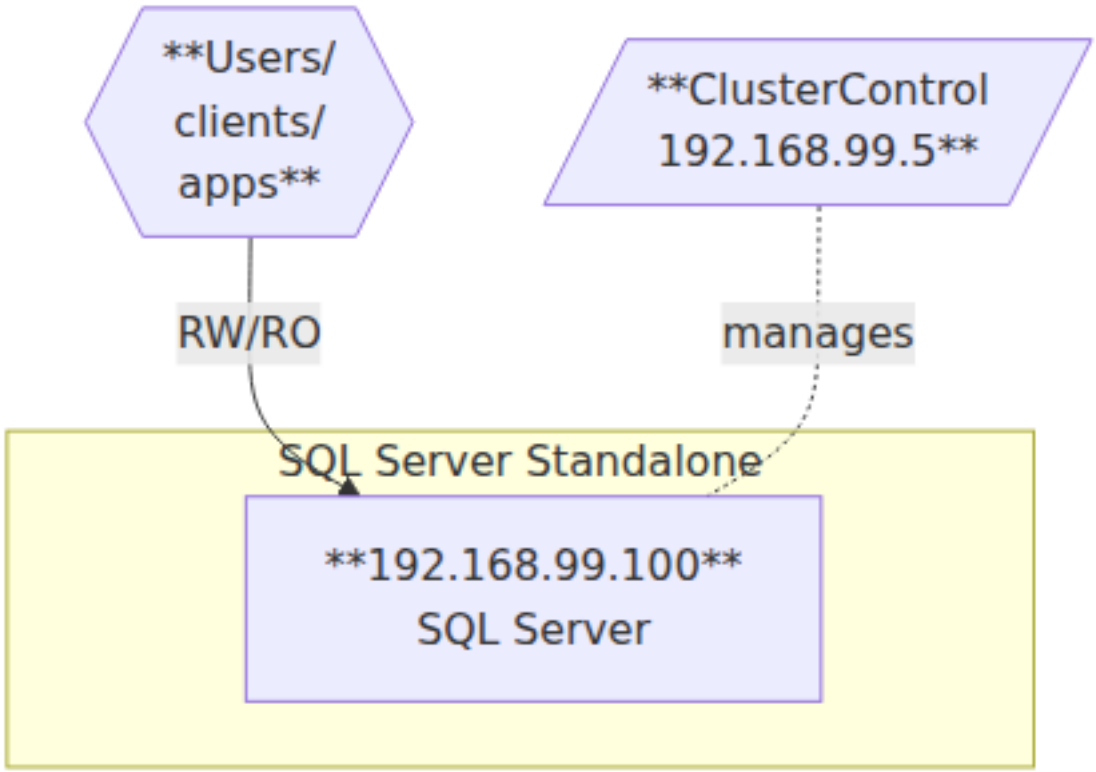
1.73.1 Prerequisites

Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- You need one host (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
- SSH access to the server involved.
- The node must have internet access during the deployment stage.

1.73.2 Architecture

Below is a simplified diagram of the final architecture:



1.73.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key:

```
ssh-keygen -t rsa
```

2. Copy the SSH key to the SQL Server node:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.100
```

3. Test the connection:

```
ssh root@192.168.99.100 "ls /root"
```

4. Host mapping - The first prerequisite is updating the hosts file in all the nodes, including the ClusterControl node, with the hostnames and IP addresses of the database nodes that will participate in the availability group. The following is the expected content of `/etc/hosts` on every host:

```
127.0.1.1 ClusterControl
127.0.0.1 localhost

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
192.168.99.100 SQL-Server
```

1.73.4 Step 2: Deploy SQL Server standalone

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose **SQL Server** from the **Database** dropdown and the **Vendor** and **Version** (e.g., "Microsoft" and "2022") from the wizard. Click **Continue**.
4. In the **Deploy SQL Server cluster**, configure the cluster as follows:

Deploy SQL Server cluster

(1) Cluster details

- **Name**: My SQL Server 2022 Cluster
- **Tags**: sql, ag,

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`

- **Install software**: Enabled
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(3) Node configuration

- **Admin username**: `SQLServerAdmin` (default)
- **Admin username**: Set or copy the generated password for your reference.
- **Server Port**: `1433` (default)
- **Repository**: Use vendor repositories (default)
- **Enable SSL encryption**: `On` (default)

(4) Add nodes

- **Primary node**: (hostname is required as IP is not supported)
  - Specify "SQL-Server" as the primary node and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if the node is reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

6. Click **Finish** to begin the deployment.
7. ClusterControl will start to provision the host and install SQL Server. Monitor the progress from **Activity Center → Jobs**.

1.73.5 Step 3: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of the node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View the most recent alarms of your cluster.
- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm, but the recovery procedures will be skipped.
- Topology viewer: Roll over each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.73.6 Step 4: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- The SQL Server 192.168.99.100 is on port 1433. We can use SQL Server Management Studio (SSMS), Azure Data Studio, or the SQLCMD tool to connect to the SQL Server. For the SSMS connection, authentication, choose SQL Server Authentication, and use the SQL credentials specified during deployment.

Tip

You can use popular SQL Server Management Studio (SSMS), Azure Data Studio, or SQLCMD tool to perform database administration tasks.

1.73.7 Step 5: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For SQL Server standalone, ClusterControl supports full, differential, and transaction log backups. To schedule a backup:

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily full backup - SQL Server
- **Cluster**: (leave as it is)
- **Backup method**: (choose between full, differential, and transaction log)
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Retention**: On (default)
- **Include system databases**: On (default)

(3) Storage

- **Storage location**: Store on node (default)
- **Storage directory**: /var/lib/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00

- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size, and perform restoration if necessary.

1.73.8 Step 6: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks, and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however, a mail transfer agent (sendmail, postfi,x or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group that you belong to from the **User group** dropdown.
3. Choose your email address to from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with a summary of warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie), or web hooks. See *Integration → Notification Services*.

1.73.9 Step 7: Manage your cluster

ClusterControl provides many other features for managing your cluster, including:

- Database node management: Perform start/stop/restart node, reboot host. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.

- Configuration management: Perform database configuration changes. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Backup management: Create, schedule, restore, upload backup to the cloud, and set a retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions_** and **ClusterControl GUI → choose the cluster → Backups → More_**.
- Maintenance management: Activate, deactivate, remark, and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from the ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.73.10 Conclusion

Congratulations! You've successfully deployed, monitored, managed, and scaled your first SQL Server Standalone using ClusterControl. From here, you can explore additional features such as failover management, access control list (ACL) rules, advanced monitoring, advanced backup management, database restore, and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.74 SQL Server Always On availability group

In this guide, we will walk you through setting up your first database cluster running on SQL Server Always On availability group using ClusterControl. By the end of this tutorial, you will have a fully operational database cluster that you can monitor, manage, and scale using ClusterControl.

SQL Server is a relational database management system (RDBMS). Applications and tools connect to a SQL Server instance or database, and communicate using Transact-SQL (T-SQL).

The Always On availability groups feature is a high-availability and disaster-recovery solution that provides an enterprise-level alternative to database mirroring. Always On availability groups maximize the availability of a set of user databases for an enterprise. An availability group supports a failover environment for a discrete set of user databases, known as availability databases, that fail over together. An availability group supports a set of read-write primary databases and one to eight sets of corresponding secondary databases. Optionally, secondary databases can be made available for read-only access and/or some backup operations.

See also

- SQL Server on Linux
- SQL Server Always On availability group

1.74.1 Prerequisites

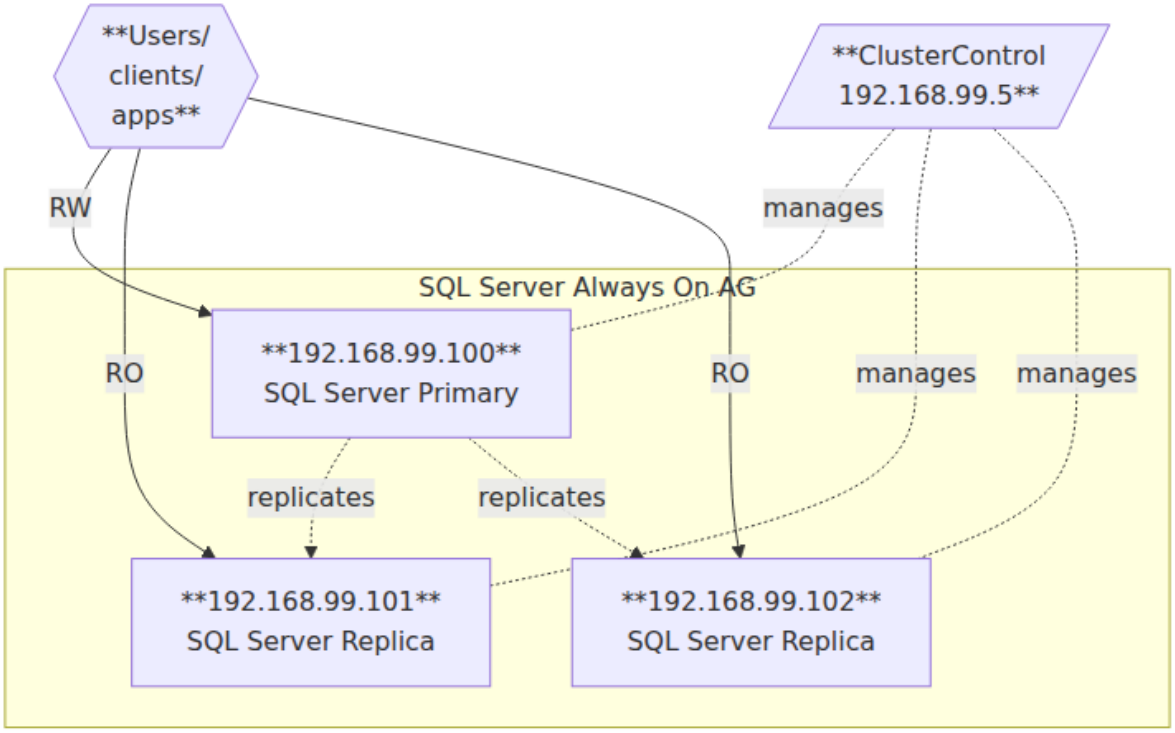
Ensure the following requirements are met before proceeding:

- ClusterControl should be installed and running. If not, follow the installation instructions in *Quickstart* or use the *Installer Script*.
- ClusterControl only supports deploying and managing SQL Server for Linux.

- You need at least three hosts (bare-metal or virtual machines) installed with the supported operating system. See *Operating System*.
- SSH access to all servers involved.
- All nodes must have internet access during the deployment stage.
- NTP is configured and running on all hosts to ensure that all nodes in the cluster maintain synchronized time.

1.74.2 Architecture

Below is a simplified diagram of the final architecture:



1.74.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key:

```
ssh-keygen -t rsa
```

2. Copy the SSH key to all SQL Server nodes:

```
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.100  # Primary
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.101  # Replica 1
ssh-copy-id -i ~/.ssh/id_rsa root@192.168.99.102  # Replica 2
```

3. Test the connection:

```
ssh root@192.168.99.100 "ls /root"
ssh root@192.168.99.101 "ls /root"
ssh root@192.168.99.102 "ls /root"
```

4. Host mapping - The first prerequisite is updating the hosts file in all the nodes, including the ClusterControl node, with the hostnames and IP addresses of the database nodes that will participate in the availability group. The following is the expected content of `/etc/hosts` on every host:

```
127.0.1.1 ClusterControl
127.0.0.1 localhost

# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
192.168.99.100 SQL-primary                # Hostname for node1
192.168.99.101 SQL-replica1                # Hostname for node2
192.168.99.102 SQL-replica2                # Hostname for node3
```

1.74.4 Step 2: Deploy a SQL Server Always On availability group

1. Open your browser and navigate to your ClusterControl server's IP or domain name.
2. From the ClusterControl dashboard, click on **Deploy a cluster** in the top-right corner and choose **Create a database cluster**.
3. Choose **SQL Server** from the **Database** dropdown and the **Vendor** and **Version** (e.g., "Microsoft" and "2022") from the wizard. Click **Continue**.
4. In the **Deploy SQL Server cluster**, configure the cluster as follows:

Deploy SQL Server cluster

(1) Cluster details

- **Name**: My SQL Server 2022 Cluster
- **Tags**: sql, ag,

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa`
- **SSH port**: 22
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: Enabled
- **Disable firewall**: Checked
- **Disable SELinux/AppArmor**: Checked

(3) Node configuration

- **Admin username**: `SQLServerAdmin` (default)
- **Admin username**: Set or copy the generated password for your reference.
- **Server Port**: `1433` (default)
- **Repository**: Use vendor repositories (default)

- **Enable SSL encryption**: `On` (default)

(4) Add nodes

- **Primary node**: (hostname is required as IP is not supported)
  - Specify "SQL-primary" as the primary node and press ++enter++.
- **Replica node**:
  - Specify "SQL-replica1" as replica and press ++enter++.
  - Specify "SQL-replica2" as another replica and press ++enter++.
- Wait until everything is green. If a red warning appears, inspect the error and fix it.
- Proceed only if all nodes are reachable (shown in green).

(5) Preview

- Review the summary of your deployment. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

6. Click **Finish** to begin the deployment.
7. ClusterControl will start to provision the hosts, install SQL Server, and configure the Always On availability group. Monitor the progress from **Activity Center → Jobs**.

1.74.5 Step 3: Connecting to the database cluster

At this point, your application should be able to access the database cluster via the following endpoints:

- The primary SQL Server is 192.168.99.100 on port 1433. We can use SQL Server Management Studio (SSMS), Azure Data Studio, or the `sqlcmd` tool to connect to the SQL Server. For the SSMS connection, authentication, choose "SQL Server Authentication", and use the database credentials specified during the deployment.
- If you want to connect to the SQL server locally in the server, SSH to the server and use the following command:

```
/opt/mssql-tools18/bin/sqlcmd -U SQLServerAdmin -P '
↪<SQLServerAdmin_password>' -C
```

- For read-only workloads, you may send the database connections to the replica nodes, 192.168.99.101 or 192.168.99.102, on port 1433.

Note

Note Most of the SQL Server drivers and application connectors come with a built-in support for Always On availability group. Check your application connector's documentation for details.

1.74.6 Step 4: Create a database and add into availability group

When a new database is created on the primary server, it is not automatically included in the Always On availability group and, therefore, not replicated to the replicas. Replication to a replica only occurs after the database has been added to the Always On availability group. ClusterControl, by default, establishes an availability group named `s9s_ag1`. This step outlines the process of creating a database and subsequently adding it to the `s9s_ag1` availability group to enable replication to the replica servers.

1. SSH to the primary SQL server and connect to the SQL Server by using `sqlcmd`:

```
/opt/mssql-tools18/bin/sqlcmd -U SQLServerAdmin -P '
↪<SQLServerAdmin_password>' -C
```

2. Create a new database called `myshop`:

```
CREATE DATABASE [myshop]
GO
```

2. Add the database into the availability group:

```
ALTER DATABASE [myshop] SET HADR AVAILABILITY GROUP = [s9s_ag1]
GO
```

3. Verify that the database is inside the availability group:

```
SELECT * FROM [master].[sys].[availability_databases_cluster]
GO
```

Tip

Remember to perform these steps if you create a new database, and you want it to be replicated to the replica servers.

1.74.7 Step 5: Monitor your cluster

After the cluster has been deployed, you will be redirected to the **Home** page. This page provides an at-a-glance view of your cluster's status and important aspects of a database, including:

- Cluster health: The **Home** page provides the cluster state
- Node health: Rollover on the honeycomb diagram and get a more detailed view of a particular node. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: View the most recent alarms of your cluster.

- Automatic recovery status: Green means ClusterControl will raise an alarm and perform recovery procedures if it detects the database service is down. Red means ClusterControl will raise an alarm, but the recovery procedures will be skipped.
- Topology viewer: Roll over each cluster entry to see the summary of the cluster visualized in topology view to better understand your overall database state. Also available under ***ClusterControl GUI → Clusters → choose the cluster → Nodes → Topology***.

1.74.8 Step 6: Enable automatic backups

To ensure your cluster data is protected, you should schedule an automatic backup. For SQL Server Always On availability group, ClusterControl supports full, differential, and transaction log backups. To schedule a backup:

1. Go to ***ClusterControl GUI → choose the cluster → Backups*** to open the backup overview page of the cluster.
2. Go to ***Create Backup → Schedule a Backup*** to schedule an automated backup. It will open a backup wizard to follow.
3. In the **Create a backup schedule** wizard, configure the backup as below:

Create a backup schedule

(1) Configuration

- **Schedule name**: Daily full backup - SQL Server
- **Cluster**: (leave as it is)
- **Backup method**: (choose between full, differential, and transaction log)
- **Upload backup to cloud**: Off

(2) Advanced settings

- **Compression**: On (default)
- **Retention**: On (default)
- **Include system databases**: On (default)

(3) Storage

- **Storage location**: Store on node (default)
- **Storage directory**: /var/lib/backups (default)
- **Backup subdirectory**: BACKUP-%I (default)

(4) Schedule

- **Set backup schedule**: Simple
- **Every**: day at 2:00
- **Select timezone**: Use your preferred timezone

(5) Preview

- Review the summary of the backup configuration. You can always go back to any previous section to modify your configurations if you wish. The backup configuration settings will be kept until you exit the wizard.

4. Click **Create** to schedule the backup.

Once configured, ClusterControl will take care of the automated backups, ensuring your data is safe. All successful backups will be listed in the ***Backups → All Backups*** page, where you can see the backup logs, size, and perform restoration if necessary.

1.74.9 Step 7: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks, and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail; however, a mail transfer agent (sendmail, postfix, or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. Go to ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Choose a user group that you belong to from the **User group** dropdown.
3. Choose your email address from the **Users in the selected group** dropdown.
4. Click **Enable**.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures you are always notified when critical issues arise and can act quickly to resolve them. You will also get an email with a summary of warning events every day at the configured delivery time.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie), or web hooks. See *Integration → Notification Services*.

1.74.10 Step 8: Manage your cluster

ClusterControl provides many other features for managing your cluster, including:

- Database node management: Perform start/stop/restart node, reboot host, promote replica, rebuild replica. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.

- Configuration management: Perform database configuration changes globally or on individual database node. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Backup management: Create, schedule, restore, upload backup to the cloud, and set a retention period. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions**_ and **ClusterControl GUI → choose the cluster → Backups → More**_.
- Maintenance management: Activate, deactivate, remark, and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from the ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.74.11 Step 9: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. For SQL Server Always On availability group, you can scale out the cluster with another replica. Common use cases are to use the replica node for to scale out the read-only workloads, backups, reporting, or analytics purposes.

Adding a new replica node is an online and non-blocking operation. It shall not cause downtime to the running cluster however, it will increase load on the source node due to initial syncing from the primary.

ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

1. To add a new replica, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Replica node***.
2. In the **Create a replica node** wizard, configure the following:

Create a replica node

(1) Node Configuration

- **Port**: 1433
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Add node

- **Primary node**: Choose the primary node from the dropdown.
- **Node**: Specify the node's hostname that you want to add and press ++enter++.

(3) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.

4. ClusterControl will start provisioning the new replica node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.74.12 Conclusion

Congratulations! You've successfully deployed, monitored, managed, and scaled your first SQL Server Always On availability group using ClusterControl. From here, you can explore additional features such as failover management, access control list (ACL) rules, advanced monitoring, advanced backup management, database restore, and many more. As you grow, ClusterControl will help ensure that your database infrastructure remains highly available and performant.

1.75 Elasticsearch Standalone Cluster

This guide will take you through the process of setting up your first standalone (also known as single-node) Elasticsearch cluster using ClusterControl. By following this tutorial, you'll have a fully functional (but non-replicated) database node that you can monitor, manage, and later scale if your requirements change.

A standalone Elasticsearch node is the simplest deployment option, but it isn't recommended for production workloads or day-to-day operations in medium- to large-scale organisations. Use it in production only for low-stakes workloads where downtime and potential data loss are acceptable.

This tutorial applies to the deployment of a database cluster for Elasticsearch by Elastics.

See also

- Elasticsearch: One-node clusters

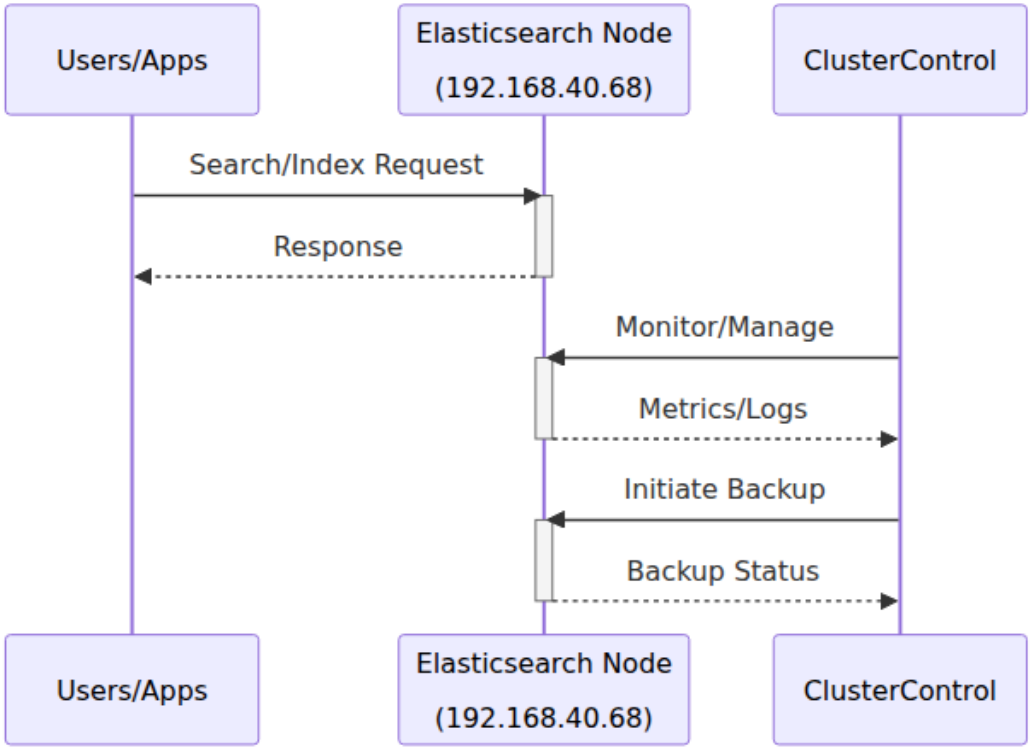
1.75.1 Prerequisites

Before proceeding, ensure you have:

- ClusterControl installed and running. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- At least two hosts (bare-metal or virtual machines):
  - One for the ClusterControl server.
  - One for the Elasticsearch single-node or server.
- SSH access to all servers.
- Internet access on the database host to install required packages.
- Network Time Protocol (NTP) configured and running on both hosts to keep their clocks synchronized.

1.75.2 Architecture

Below is a sequence diagram of the final architecture for a single-node deployment:



Elasticsearch in ClusterControl is deployed without any external supporting load balancers to be added. Although Elasticsearch has its load balancing and high-availability capabilities or mechanisms that can act to provide its functionality. But for a single node, it's goes to its default node type which can function as as a master node, data node, ingest node, or even a coordinating node.

1.75.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

Copy the public key to your standalone database node (replace 192.168.40.68 with your node's IP/hostname):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.40.68
```

If the target node uses a custom SSH key or port, you can add options:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.40.68
```

For some advance setup where the user (for example `mymainacc`) is only allowed to access non-root with only public keys are allowed and password challenge is disabled but has sudo privileges, as `root` OS user, you can do this:

```
[root@pupnode7 ~]# ssh -i /home/mymainacc/.ssh/id_rsa␣
↪mymainacc@192.168.40.68   "sudo bash -c '
umask 077
mkdir -p   /root/.ssh
```

(continues on next page)

(continued from previous page)

```
cat >>     /root/.ssh/authorized_keys
'" < ~/.ssh/id_rsa.pub
```

this copies your root public key to the target node you will be setting up for single-node Elasticsearch cluster deployment

2. Test passwordless SSH from the ClusterControl server:

```
ssh root@192.168.40.68 "stat \$PWD/"
```

Ensure there is no password prompt. If the command returns directory system status, you're set.

1.75.4 Step 2: Deploy a new cluster (single node)

1. Open a web browser and go to the ClusterControl server's IP or hostname.
2. On the ClusterControl dashboard, click ***Deploy a cluster (top-right) → Create a database cluster***. This opens the Deploy cluster wizard.
3. Select **Elasticsearch** from the **Database** dropdown. Specify also the version you desired to choose by clicking **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Replication** wizard, configure the database cluster as below:

Deploy MySQL Replication

(1) Cluster details

- **Name**: For example, `ES-stand-alone-node`.
- **Tags**: (Optional) e.g., `standalone`, `production`, `dc1`.

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa` (ClusterControl will also autofill this field)
- **SSH port**: `22` (default port)
- **SSH sudo password**: (leave blank if you rely on key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **HTTP port**: `9200` (disabled text-field with default port specified)
- **Transfer port**: `9200` (disabled text-field with default port specified)
- **Admin user**: `admin`
- **Admin password**: Password to be assigned to the database `admin` user

- **Repository**: Use vendor repositories (default)

(4) Add node

- **Eligible master**: (Fill in the IP/hostname or FQDN of the node. For example, 192.168.40.65 and press ++enter++)
- **Data nodes**: (leave it empty)

(5) Snapshot storage configuration

- **Repository name**: (Fill in the repository name you want. For example, `es-s9s-repo`)
- **Storage host**: (Click the drop-down field and choose your single-node IP/hostname or FQDN entry)
- **Default snapshot location**: (Path of the shared filesystem use to store your snapshots. For example, `/mnt/data/backups/es-snapshot-repositories`)
- **Configure shared filesystem**: On (default)

(5) Preview

- Review your configuration. You can go back and adjust if necessary.

5. Click **Finish** to start deployment.
6. ClusterControl will now install and configure Elasticsearch standalone cluster on 192.168.40.68. You can track progress in the **Activity center**. After a few minutes, your new single-node cluster will appear on the **Home** page.

1.75.5 Step 3: Monitor your cluster

Once deployed, you'll see:

- Cluster health: The **Home** page provides the cluster state, even though it's a single node, it's treated as a "cluster" of size one.
- Node health: Hover on the honeycomb diagram or check the Nodes tab. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: Any triggered alarms will appear if there are configuration or resource issues.
- Automatic recovery status: If enabled, ClusterControl can attempt to restart a crashed MySQL server automatically.
- Topology viewer: You'll see a very simple topology with only one node.

1.75.6 Step 4: Import data

- You can import data into the standalone server in a variety of ways:

For bulk import, using NDJSON file

NDJSON stands for newline delimited JSON. For example, loading data from Wikimedia Foundation from here, I can load data as such:

Adding index first to the file,

```
jq -c --arg idx enwikiquote '{index:{_index:$idx,_id: .id}}, .'␣
↪first1000-enwikiquote.json > first1000-enwikiquote.ndjson
```

Then load with NDJSON

```
curl --cacert /etc/elasticsearch/certs/elasticsearch-ca.pem -u␣
↪elastic-s9s:myPassw0rd -H 'Content-Type: application/x-ndjson' --
↪data-binary @first1000-enwikiquote.ndjson -X POST
↪'https://192.168.40.68:9200/_bulk?pretty'
```

Using `elasticdump`

Install `elasticdump` first.

For example, you have migrated from another host (in this example, its 192.168.10.100), you can dump your data using `elasticdump` as follows:

```
~/node_modules/elasticdump/bin/elasticdump \
     --cert /etc/elasticsearch/certs/elasticsearch-ca.pem \
     --input=https://elastic-s9s:myPassw0rd@192.168.10.100:9200/enwikiquote␣
↪\
     --output=/backups/enwikiquote.json --type=data \
     --limit=10000
```

Then create an empty index from your target node, in this example, its 192.168.40.65

```
curl --cacert /etc/elasticsearch/certs/elasticsearch-ca.pem \
     -u elastic-s9s:myPassw0rd \
     -H 'Content-Type: application/json' -X PUT
↪'https://192.168.40.68:9200/enwikiquote_play' \
     -d '{ "settings": { "number_of_replicas": 0 } }'
```

Then it's now ready to dump to the target node,

```
NODE_TLS_REJECT_UNAUTHORIZED=0 ~/node_modules/elasticdump/bin/elasticdump \
  --input=/backups/enwikiquote.json \
  --
↪output=https://elastic-s9s:myPassw0rd@192.168.40.68:9200/enwikiquote_play␣
↪\
  --type=data \
  --limit=10000 \
  --concurrency=4
```

```
  You can also pass the NODE_TLS_REJECT_UNAUTHORIZED=0 if you are using a␣
↪self-signed certificate
```

Always remember, when importing data to your single node, keep in mind that ClusterControl shall mark your standalone node to degraded when it detects indices health are **Yellow** whenever your indices are set to have replicas. Make sure when creating your indices during import to mark your replicas as 0 by doing the following example (where index is named `enwikiquote`):

```
curl --cacert /etc/elasticsearch/certs/elasticsearch-ca.pem -u␣
↪elastic-s9s:myPassw0rd -X PUT
↪'https://192.168.40.68:9200/enwikiquote/_settings?pretty' -H
↪'Content-Type: application/json' -d'
{
  "index" : {
    "number_of_replicas" : 0
  }
}
'
```

1.75.7 Step 5: Connect to the database

Your application or client will connect directly to the standalone server:

- Host: 192.168.40.68
- Port: 9200
- User/Password: The credentials specified during deployment. You can also inspect the file `/etc/cmon.d/cmon_$CID.cnf` where $CID is the cluster id of your Elasticsearch deployment.

No load balancer or additional ports are involved.

1.75.8 Step 6: Enable automatic backups

Elasticsearch does not offer the traditional sense of backup. Instead, it uses snapshots which have more capability and robust way to create a consistent backup copy. It is the most recommended method for creating consistent point-in-time backups for an entire cluster or specific indices, and its native to Elasticsearch.

Even a single-node, standalone Elasticsearch cluster needs reliable backups, and ClusterControl makes it easy to schedule them automatically.

1. Go to ***ClusterControl GUI → choose the cluster → Backups***.
2. Click ***Create Backup → Schedule a Backup***. The Create a backup schedule wizard will open and proceed to configure your backup as below:

***Create Backup → Schedule a Backup**

(1) Configuration

- **Schedule name**: `Daily ES-standalone cluster snapshot`
- **Cluster**: (defaults to your Elasticsearch standalone cluster)
- **Repository**: (defaults to the repository you created during deployment)
- **Backup method**: `elasticsearch-snapshot` (default) [disabled text field]

(2) Advanced settings

- **Retention**: On (default)
- **Retention [textfield]**: 4 (Set to your desired number of days to retain backup)

(3) Schedule

- **Set backup schedule**: Simple
- **Every**: day at `02:00`
- **Timezone**: select your local timezone

(4) Preview

- Verify all settings. You can go back to adjust if needed.

3. Click Create to schedule it.

ClusterControl will now automatically perform your backups. You may also restore your backups from the elasticsearch snapshots in the future if necessary for disaster recovery.

1.75.9 Step 7: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Select a User group (your group) from the **User group** dropdown.
3. Select your email address to from the **Users in the selected group** dropdown.
4. Click ***Enable***.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures timely notifications when something goes wrong.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.75.10 Step 8: Manage your node

ClusterControl provides monitoring for your cluster and system overview which displays workloads based on metrics. Once your Elasticsearch cluster is done deployed, agents using Prometheus exporters are deployed to gather metrics and provide more granular monitoring of your cluster and system workload for your standalone cluster in Elastitcsearch.

Apart from the monitoring, you can manage your node with the most available options for this cluster:

- Database node management: Perform start/stop/restart node, reboot host. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- Configuration management: Perform database configuration changes globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Backup management: Create, schedule, restore, store snapshot in an off-cluster storage location such as AWS S3 or any S3 Compatiable storage, while allowing you to set retention period for your backup snapshots. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions_** and **ClusterControl GUI → choose the cluster → Backups → More_**.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.75.11 Conclusion

Congratulations! You've deployed, monitored, and managed a standalone Elasticsearch server using ClusterControl. Currently, a single node or standalone node in ClusterControl for Elasticsearch is limited to only one node, without any feature to scale, but make sure you follow for such updates or request for features if necessary for your requirements. While it limits you to some extent in your single node, its available functions does not stop you from doing your favorite things to do. Make sure you always practice to do the following:

- Keep your backups snapshots current and up to date.
- Monitor performance and resource usage.
- Secure your node by restricting access and using strong passwords.
- Expand to multi-node topologies if your needs grow (this time by deploying a multi-node or high-availability cluster in Elasticsearch)

Enjoy your streamlined single-node database deployment for Elasticsearch, backed by ClusterControl's powerful operations for backups, alerts, scaling, and more!

1.76 Elasticsearch High-Availability Cluster

This guide will take you through the process of setting up your first high-availability cluster, Elasticsearch cluster using ClusterControl. By following this tutorial, you'll have a fully functional high-availability Elasticsearch cluster that you can monitor, manage, and later scale if your requirements change.

A high-availability cluster in Elasticsearch offers you service ensuring your cluster to keep available when individual nodes, racks, or whole zones go down. A highly available cluster in Elasticsearch provides no single-point of failure (SPOF), data redundancy, automatic failover and self-healing (also resiliency), zone/rack awareness, rolling maintenance and upgrade, horizontal scalability, consistent cluster state, and durable snapshots and fast restore.

This tutorial applies to the deployment of a database cluster for Elasticsearch by Elastics.

See also

- Elasticsearch: Backup, high availability, and resilience tools

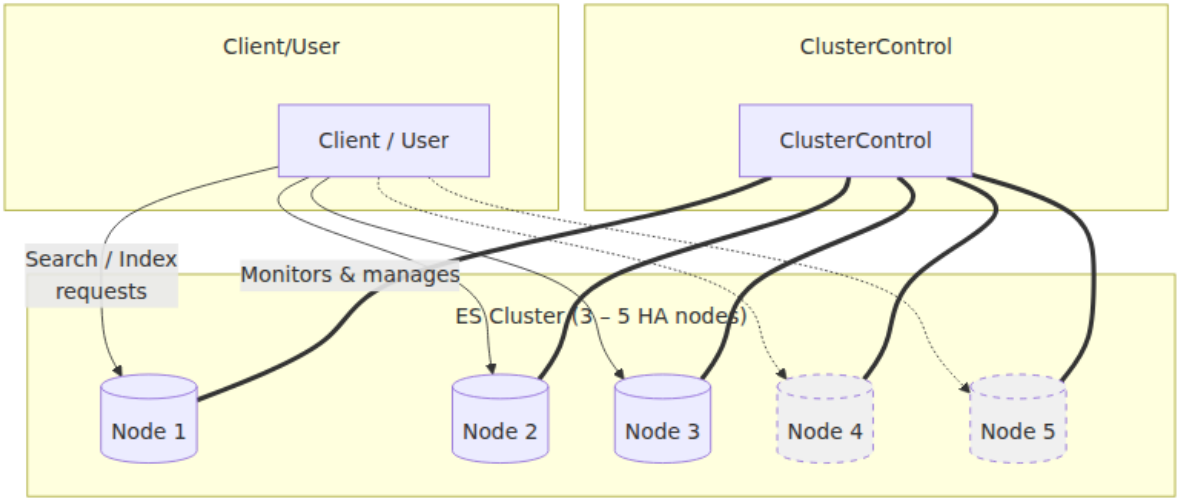
1.76.1 Prerequisites

Before proceeding, ensure you have:

- ClusterControl installed and running. If not, follow the instructions in *Quickstart* or use the *Installer Script*.
- At least two hosts (bare-metal or virtual machines):
  - One for the ClusterControl server.
  - One for the Elasticsearch single-node or server.
- SSH access to all servers.
- Internet access on the database host to install required packages.
- Network Time Protocol (NTP) configured and running on both hosts to keep their clocks synchronized.

1.76.2 Architecture

Below is a simple flow diagram on how shall our deployment managed and monitored with ClusterConrol while the **Client/User** interacts with our Elasticsearch nodes (or master/data nodes) emphasizing a high-availability cluster.



Elasticsearch in ClusterControl is deployed without any additional supporting load balancers. Elasticsearch has its built-in capability for load balancing but its not further discussed or is related for this topic.

On the diagram above, the nodes ES1, ES2, ES3, ES4, and ES5 shall have the following hostnames for this example deployment:

- ES1: 192.168.40.70 (master-data)
- ES2: 192.168.40.71 (master-data)
- ES3: 192.168.40.72 (master-data)

- ES4: 192.168.40.73 (data)
- ES5: 192.168.40.74 (data)

1.76.3 Step 1: Set up SSH key-based authentication

1. On the ClusterControl server, generate a new SSH key as the root user:

```
ssh-keygen -t rsa
```

Copy the public key to your standalone database node (replace 192.168.40.68 with your node's IP/hostname):

```
ssh-copy-id -i /root/.ssh/id_rsa root@192.168.40.70
```

If the target node uses a custom SSH key or port, you can add options:

```
ssh-copy-id -i /root/.ssh/id_rsa -p 22 -o 'IdentityFile /
↪root/myprivatekey.pem' root@192.168.40.670
```

For some advance setup where the user (for example `mymainacc`) is only allowed to access non-root with only public keys are allowed and password challenge is disabled but has sudo privileges, as `root` OS user, you can do this:

```
[root@pupnode7 ~]# ssh -i /home/mymainacc/.ssh/id_rsa␣
↪mymainacc@192.168.40.70   "sudo bash -c '
umask 077
mkdir -p   /root/.ssh
cat >>     /root/.ssh/authorized_keys
'" < ~/.ssh/id_rsa.pub
```

this copies your root public key to the target node you will be setting up for single-node Elasticsearch cluster deployment

2. Test passwordless SSH from the ClusterControl server:

```
ssh root@192.168.40.70 "stat \$PWD/"
```

Ensure there is no password prompt. If the command returns directory system status, you're set.

3. Apply the same steps to the rest of the ndoes from nodes ES2, ES3, ES4, and ES5

1.76.4 Step 2: Deploy a new cluster (high-availability)

1. Open a web browser and go to the ClusterControl server's IP or hostname.
2. On the ClusterControl dashboard, click ***Deploy a cluster (top-right) → Create a database cluster***. This opens the Deploy cluster wizard.
3. Select **Elasticsearch** from the **Database** dropdown. Specify also the version you desired to choose by clicking **Version** dropdown. Click **Continue**.
4. In the **Deploy MySQL Replication** wizard, configure the database cluster as below:

Deploy MySQL Replication

(1) Cluster details

- **Name**: For example, `Elasticsearch-HA`.
- **Tags**: (Optional) e.g., `standalone`, `production`, `dc1`.

(2) SSH configuration

- **SSH user**: `root`
- **SSH user key path**: `/root/.ssh/id_rsa` (ClusterControl will also autofill this field)
- **SSH port**: `22` (default port)
- **SSH sudo password**: (leave blank if you rely on key-based auth)
- **SSH sudo / OS elevation command**: Choose either uses `sudo`(default) or `doas` or `pbrun`
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(3) Node configuration

- **HTTP port**: `9200` (disabled text-field with default port specified)
- **Transfer port**: `9200` (disabled text-field with default port specified)
- **Admin user**: `admin`
- **Admin password**: Password to be assigned to the database `admin` user
- **Repository**: Use vendor repositories (default)

(4) Add node

- **Eligible master**: (Fill in the IP/hostname or FQDN of the master nodes. In this exercise, input ES1, ES2, and ES3)
- **Data nodes**: (Fill in the IP/hostname or FQDN of the data nodes. In this exercise, input ES4, ES5)

(5) Snapshot storage configuration

- **Repository name**: (Fill in the repository name you want. For example, `es-s9s-repo`)
- **Storage host**: (Click the drop-down field and choose your desired node to host the NFS shared directory using IP/hostname or FQDN entry)
- **Default snapshot location**: (Path of the shared filesystem use to store your snapshots. For example, `/mnt/data/backups/es-snapshot-repositories`)
- **Configure shared filesystem**: On (default)

(5) Preview

- Review your configuration. You can go back and adjust if necessary.

5. Click **Finish** to start deployment.
6. ClusterControl will now install and configure your Elasticsearch High-Availability (HA) Cluster.

You can track progress in the **Activity center**. After a few minutes, your Elasticsearch HA Cluster will appear on the **Home** page.

1.76.5 Step 3: Monitor your cluster

Once deployed, you'll see:

- Cluster health: The **Home** page provides the cluster state.
- Node health: Hover on the honeycomb diagram or check the Nodes tab. You can also see more detailed histograms under ***ClusterControl GUI → Clusters → choose the cluster → Dashboards***.
- Recent alarms: Any triggered alarms will appear if there are configuration or resource issues.
- Automatic recovery status: If enabled, ClusterControl can attempt to restart a crashed MySQL server automatically.
- Topology viewer: You'll see a very simple topology with only one node.

1.76.6 Step 4: Import data

- You can import data into the standalone server in a variety of ways:

For bulk import, using NDJSON file

NDJSON stands for newline delimited JSON. For example, loading data from Wikimedia Foundation from here, I can load data as such:

Adding index first to the file,

```
```

```
        $ wget␣
↪https://dumps.wikimedia.org/other/cirrussearch/20250414/abwiki-20250414-cirrussearch-co
        $ gzip -d abwiki-20250414-cirrussearch-content.json.gz
        $ jq -nc --arg idx abwiki '
          foreach inputs as $doc (0;                # counter starts at 0
            . + 1;                                  # next counter value
            ({index:{_index:$idx,_id:.}}),           # emit meta line
            $doc                                     # emit original line
          )
        '  abwiki-20250414-cirrussearch-content.json > abwiki.ndjson
    ```
    Then load with NDJSON
    ```bash
    curl --cacert /etc/elasticsearch/certs/elasticsearch-ca.pem -u␣
↪elastic-s9s:myPassw0rd -H 'Content-Type: application/x-ndjson' --
↪data-binary @abwiki.ndjson -X POST
↪'https://192.168.40.72:9200/_bulk?pretty'
    ```
```
```

Using `elasticdump`

Install `elasticdump` first.

For example, you have migrated from another host (in this example, its 192.168.10.100), you can dump your data using `elasticdump` as follows:

```
~/node_modules/elasticdump/bin/elasticdump \
     --cert /etc/elasticsearch/certs/elasticsearch-ca.pem \
     --input=https://elastic-s9s:myPassw0rd@192.168.10.100:9200/abwiki \
     --output=/backups/abwiki.json --type=data \
     --limit=10000
```

Then create an empty index from your target node, in this example, its 192.168.40.72

```
curl --cacert /etc/elasticsearch/certs/elasticsearch-ca.pem \
     -u elastic-s9s:myPassw0rd \
     -H 'Content-Type: application/json' -X PUT
↪'https://192.168.40.72:9200/abwiki_play' \
     -d '{ "settings": { "number_of_replicas": 1 } }'
```

Then it's now ready to dump to the target node,

```
NODE_TLS_REJECT_UNAUTHORIZED=0 ~/node_modules/elasticdump/bin/elasticdump \
  --input=/backups/abwiki.json \
  --output=https://elastic-s9s:myPassw0rd@192.168.40.72:9200/abwiki_play \
  --type=data \
  --limit=10000 \
  --concurrency=4
```

```
  You can also pass the NODE_TLS_REJECT_UNAUTHORIZED=0 if you are using a␣
↪self-signed certificate
```

For high-availability cluster, make sure your indices are replicated so that it shall always have a copy in case the primary goes down.

1.76.7 Step 5: Connect to the database

Your application or client will connect directly to any of your nodes (master or data nodes). In this example, we go and connect to node ES1,

- Host: 192.168.40.70
- Port: 9200
- User/Password: The credentials specified during deployment. You can also inspect the file `/etc/cmon.d/cmon_$CID.cnf` where $CID is the cluster id of your Elasticsearch deployment.

No load balancer or additional ports are involved.

1.76.8 Step 6: Enable automatic backups

Elasticsearch does not offer the traditional sense of backup. Instead, it uses snapshots which have more capability and robust way to create a consistent backup copy. It is the most recommended method for creating consistent point-in-time backups for an entire cluster or specific indices, and its native to Elasticsearch.

For high-availability cluster, it is very important to always have a reliable backup and a copy of your data in case disaster or catastrophe occurs. In that regard, ClusterControl makes it easy to schedule them automatically.

1. Go to ***ClusterControl GUI → choose the cluster → Backups***.
2. Click ***Create Backup → Schedule a Backup***. The Create a backup schedule wizard will open and proceed to configure your backup as below:

***Create Backup → Schedule a Backup**

(1) Configuration

- **Schedule name**: `Daily ES-HA cluster snapshot`
- **Cluster**: (defaults to your Elasticsearch standalone cluster)
- **Repository**: (defaults to the repository you created during deployment)
- **Backup method**: `elasticsearch-snapshot` (default) [disabled text field]

(2) Advanced settings

- **Retention**: On (default)
- **Retention [textfield]**: 4 (Set to your desired number of days to retain backup)

(3) Schedule

- **Set backup schedule**: Simple
- **Every**: day at `02:00`
- **Timezone**: select your local timezone

(4) Preview

- Verify all settings. You can go back to adjust if needed.

3. Click Create to schedule it.

ClusterControl will now automatically perform your backups. You may also restore your backups from the elasticsearch snapshots in the future if necessary for disaster recovery.

1.76.9 Step 7: Configure alerts

To keep track of any issues or incidents in your cluster, it's important to set up alerting. ClusterControl supports sending alarms and alerts to email, web hooks and third-party notification services like Slack or Telegram. In this example, we are going to use email.

Firstly, configure the mail server. Go to ***ClusterControl GUI → Settings → You don't have a mail server configured. Configure now → SMTP Server***. Fill up all necessary information about the SMTP server. You can also opt for Sendmail, however a mail transfer agent (sendmail, postfix or exim) must be installed on the ClusterControl server.

Once configured, we can configure the alert and recipients as below:

1. ***ClusterControl GUI → choose the cluster → Settings → Email Notifications***.
2. Select a User group (your group) from the **User group** dropdown.
3. Select your email address to from the **Users in the selected group** dropdown.
4. Click ***Enable***.
5. Set the **Digest delivery time** when you want a digested (summarized events) to be sent to you every day.
6. Set all **Critical events** to "Deliver" (default), all **Warning events** to "Digest" and you may ignore the **Info** events.

This ensures timely notifications when something goes wrong.

Tip

You can also configure alarms to be sent to third-party notification systems (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie) or web hooks. See *Integration → Notification Services*.

1.76.10 Step 8: Manage your node

ClusterControl provides monitoring for your cluster and system overview which displays workloads based on metrics. Once your Elasticsearch cluster is done deployed, agents using Prometheus exporters are deployed to gather metrics and provide more granular monitoring of your cluster and system workload for your standalone cluster in Elastitcsearch.

Apart from the monitoring, you can manage your node with the most available options for this cluster:

- Database node management: Perform start/stop/restart node, reboot host. These features are available at ***ClusterControl GUI → choose the cluster → Nodes → Actions***.
- Configuration management: Perform database configuration changes globally. This feature is available at ***ClusterControl GUI → choose the cluster → Manage → Configuration***.
- Backup management: Create, schedule, restore, store snapshot in an off-cluster storage location such as AWS S3 or any S3 Compatiable storage, while allowing you to set retention period for your backup snapshots. These features are available at **ClusterControl GUI → choose the cluster → Backups → Actions**_ and **ClusterControl GUI → choose the cluster → Backups → More**_.
- Maintenance management: Activate, deactivate, remark and schedule maintenance mode for all nodes. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → Schedule maintenance***.
- SSH console: Access your nodes directly from ClusterControl GUI via web SSH console. This feature is available at ***ClusterControl GUI → choose the cluster → Nodes → Actions → SSH Console***.

1.76.11 Step 9: Scale your cluster

One of the key benefits of using ClusterControl is the ease of scaling your cluster. Scaling your cluster in ClusterControl can be done by either scaling out or scaling in (or done horizontal scaling). When you remove nodes, this means you want to scale in, and this can be done in ClusterControl by just going to ***ClusterControl GUI → choose the cluster → Nodes → Actions → Remove node ***. While in this section, we are going to focus on adding a node, which means in this case, we are going to scale out.

When scaling out in Elasticsearch using ClusterControl, you are required to choose and define which role you should have this node carries. The following roles available to choose from are the following:

- master-data
- master
- data
- coordinator

Adding a node to your cluster increases the high-availability of your cluster and also allows more sharding or replication for your data. Giving a large number of nodes in the cluster incurs also penalties such as processing time or overhead that could be the source of bottlenecks in the future. So in this case, plan ahead and make sure that the scaling of cluster is properly setup.

Lastly, ClusterControl assumes that the new node that you want to add meets the requirements as decribed under *Prerequisites* and configured with a proper SSH key-based authentication as shown under *Step 1: Set up SSH key-based authentication*.

Let's go over on how you can scale out your cluster in Elasticsearch:

1. To add a new node, go to ***ClusterControl GUI → choose the cluster → Actions → Add new → Replica node → Add node***.
2. In the **Create a database node** wizard, configure the following:

Create a database node

(1) Node Configuration

- **Port**: 9200 (default)
- **Install software**: On (default)
- **Disable firewall**: Checked (default)
- **Disable SELinux/AppArmor**: Checked (default)

(2) Add node

- **Node**: Specify the IP address or hostname or FQDN of the node that you want to add and press "Enter".
- **Node role**: Displays after the **Node** is specified. Select in this drop-down item to which role you would like this node assigned.

(4) Preview

- Review the summary of the deploymnent. You can always go back to any previous section to modify your configurations if you wish. The deployment configuration settings will be kept until you exit the wizard.

3. Click **Finish** to trigger the deployment job.
4. ClusterControl will start provisioning the new database node. You can monitor the progress from **Activity center**, where you will see detailed logs of the deployment. This process can take a few minutes. Once the deployment is complete, the node will appear in the **Nodes** page.

1.76.12 Conclusion

Congratulations! You've deployed, monitored, and managed a high-availability Elasticsearch cluster using ClusterControl. With high-availability cluster, you can scale in or scale out depends in your necessity and requirements, as these subjects are dynamically changes depending on the demand. Aside from that, you can manage your cluster by scheduling backups or executing backups based on-demand. You can monitor the nodes, setup maintenance mode when version upgrade is necessary. While utilizing ClusterControl makes it more easier and convenient for you, make sure not to miss such any updates or file a request for features if necessary for your requirements. We also advise that you always practice to do the following:

- Keep your backups snapshots current and up to date.
- Monitor performance and resource usage.
- Secure your node by restricting access and using strong passwords.
- Scale-in or scale-out your cluster.

Enjoy your streamlined high-availability Elasticsearch cluster database deployment, backed by ClusterControl's powerful operations for backups, alerts, scaling, and more!

1.77 ClusterControl and Database Clusters Deployment in an Offline Environment

This tutorial outlines how to perform firstly the offline installation of Severalnines ClusterControl and secondly deploying database clusters on hosts (virtual machines or bare-metal servers) that do not have an internet connection.

The document is divided into two parts where part one contains instructions on setting up a software repository host that will serve up a database, third-party tools, and OS software packages to database hosts. Part two contains instructions on how to set up ClusterControl on a host that does not have an internet connection. All hosts in this article are running on CentOS/RockyLinux/AlmaLinux 8 & 9, which uses the YUM/DNF package manager.

Part one and part two can be performed independently of each other. However, the repo server must be ready prior to deploying database clusters from ClusterControl.

We will use the following naming convention for the hosts:

- Satellite repository server (connected to the Internet, reachable by ClusterControl and database servers via local network)
- ClusterControl server (offline)
- Database servers (offline)

1.77.1 Setting up Satellite Repository Server

The satellite server is considered the offline repository server to be used by ClusterControl when deploying database servers and the corresponding tools. This server should have an internet connection since we need to pull the necessary repositories into it. Let's call this "repo server". This server will store and host thousands of packages so do allocate sufficient disk space to the repository directory (`/var/www/html/repos`) in advance.

Setting up OS, OS tools and third-party utility repositories

1. Set up a web server on the repo server to serve up packages to the yum package manager remotely:

```
$ dnf install -y nginx
$ systemctl start nginx
$ systemctl enable nginx
$ systemctl status nginx
```

2. Set up software required for a remote repository server:

```
$ yum install -y createrepo yum-utils
$ yum -y install epel-release
```

3. Configure the nginx in the Satellite Server:

```
$ sudo vi /etc/nginx/conf.d/repos.conf
server {
        listen   80;
        server_name  192.168.100.100;
        root   /var/www/html/repos;
    index index.html;
    location / {
                autoindex on;
        }
}
```

4. Restart the nginx service:

```
$ systemctl restart nginx
$ chcon -Rt httpd_sys_content_t /var/www/html/repos/
```

5. Set up repository directory to be served by the web server:

```
$ mkdir -p /var/www/html/repos/rhel
```

6. Perform repository syncing with the appropriate repositories (the following commands will consume time to finish):

```
$ dnf reposync -g --delete -p /var/www/html/repos/rhel/ --
↪repoid=baseos --newest-only --download-metadata
$ dnf reposync -g --delete -p /var/www/html/repos/rhel/ --
↪repoid=appstream--newest-only --download-metadata
$ dnf reposync -g --delete -p /var/www/html/repos/rhel/ --
↪repoid=extras --newest-only --download-metadata
-- For AlmaLinux/RockyLinux/CentOS 9 (CodeReadyBuilder)
$ dnf config-manager set-enabled crb
$ dnf reposync -g --delete -p /var/www/html/repos/rhel/ --
↪repoid=crb--newest-only --download-metadata
-- For AlmaLinux/RockyLinux/CentOS 8 (Powertools)
$ dnf config-manager set-enabled powertools
$ dnf reposync -g --delete -p /var/www/html/repos/rhel/ --
↪repoid=powertools--newest-only --download-metadata
```

Adding Severalnines repository to the repo server

1. Import the public key of Severalnines

```
$ rpm --import http://repo.severalnines.com/severalnines-repos.asc
```

2. Download the s9s-repo fom the Severalnines Repository

```
$ wget http://www.severalnines.com/downloads/cmon/s9s-repo.repo -P /
↪etc/yum.repos.d/
```

3. Edit the `s9s-repo.repo` and add the following content:

```
[s9s-tools]
name=s9s-tools (RHEL_8)
type=rpm-md
baseurl=http://repo.severalnines.com/s9s-tools/{os_codename}
gpgcheck=1
gpgkey=http://repo.severalnines.com/s9s-tools/{os_codename}/repodata/repomd.xml.key
enabled=1
```

> **Note**
>
> Note Replace `{os_codename}` based on the list of Operating System code below: `RHEL_7`,`RHEL_8`,`RHEL_9`,`CentOS_7`,`CentOS_8`,`CentOS_9`.

4. Create directory and perform repository sync:

```
$ mkdir -p /var/www/html/repos/s9s
$ dnf reposync -g --delete -p /var/www/html/repos/s9s/ --
↪repoid=s9s-repo --newest-only --download-metadata
$ dnf reposync -g --delete -p /var/www/html/repos/s9s/ --
↪repoid=s9s-tools --newest-only --download-metadata
```

Adding PostgreSQL repository to the repo server

1. Download the PostgreSQL release package from this page:

```
$ yum install https://download.postgresql.org/pub/repos/yum/reporpms/
↪{os_code}-x86_64/pgdg-redhat-repo-latest.noarch.rpm
```

> **Note**
>
> Note Replace the `{os_code}` for each verion of Operating System; `EL-7`, `EL-8`, `EL-9`.

2. Clear the repository cache and list out the repositories, make sure the newly added PostgreSQL repository is in the list:

```
$ yum clean all
$ yum repolist
```

3. Set up the repository directory to be served by the web server for PostgreSQL:

```
$ mkdir -p /var/www/html/repos/postgresql
```

4. Perform repository syncing:

```
$ dnf reposync --delete -p /var/www/html/repos/postgresql --
↪repoid=pgdg-common --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/postgresql --
↪repoid=pgdg16 --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/postgresql --
↪repoid=pgdg15 --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/postgresql --
↪repoid=pgdg14 --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/postgresql --
↪repoid=pgdg13 --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/postgresql --
↪repoid=pgdg12 --newest-only --download-metadata
```

Adding MongoDB repository to the repo server

The package name of the MongoDB server as seen in the yum command is `yum install mongodb-org`. Therefore, the `/etc/yum.repos.d/mongodb-org.repo` can only have one entry pointing to the appropriate version of MongoDB (one and only one of either 4.4, 5.0 or 6.0).

1. Create a repo file located at `/etc/yum.repos.d/mongodb-org.repo` and make sure it has the following lines (the example shows we activate MongoDB 6.0):

```
# cat mongodb-org.repo
[mongodb-org-6.0-latest]
name=MongoDB lastest 6.0 Repository
baseurl=https://repo.mongodb.org/yum/redhat/$releasever/
↪mongodb-org/6.0/x86_64/
gpgcheck=0
enabled=1
gpgkey=https://www.mongodb.org/static/pgp/server-6.0.asc

#[mongodb-org-5.0]
#name=MongoDB Repository

↪#baseurl=https://repo.mongodb.org/yum/redhat/$releasever/mongodb-org/5.0/x86_64/
#gpgcheck=1
#enabled=1
#gpgkey=https://www.mongodb.org/static/pgp/server-5.0.asc
```

Note

Note Disable GPGkey checking if you don't have an internet connection. Furthermore, only one version (i.e., either 6.0, 5.0, 4.4) can be supported at any given time. The reason is, that all the versions are called "mongodb-org".

2. Clear the repository cache and list out the repositories, make sure the newly added MongoDB repository is in the list:

```
$ yum clean all
$ yum repolist
```

3. Set up the repository directory to be served by the web server for MongoDB:

```
$ mkdir -p /var/www/html/repos/mongodb
```

4. Perform repository syncing:

```
$ dnf reposync --delete -p /var/www/html/repos/mongodb --
↪repoid=mongodb-org-6.0-latest --newest-only --download-metadata
```

Adding Percona MongoDB repository to the repo server

The package name of the Percona MongoDB server as seen in the yum command is `yum install percona-server-mongodb`. Therefore, the `/etc/yum.repos.d/mongodb-org.repo` can only have one entry pointing to the appropriate version of MongoDB (one and only one of either 4.4, 5.0 or 6.0).

1. Create a repo file located at `/etc/yum.repos.d/percona-mongodb.repo` and make sure it has the following lines (the example shows we activate Percona MongoDB 6.0):

```
[psmdb-60-release-x86_64]
name = Percona Server for MongoDB 6.0 release/x86_64 YUM repository
baseurl =␣
↪http://repo.percona.com/psmdb-60/yum/release/$releasever/RPMS/x86_64
enabled = 1
gpgcheck = 0
#gpgkey = file:///etc/pki/rpm-gpg/PERCONA-PACKAGING-KEY

[psmdb-60-release-noarch]
name = Percona Server for MongoDB 6.0 release/noarch YUM repository
baseurl =␣
↪http://repo.percona.com/psmdb-50/yum/release/$releasever/RPMS/noarch
enabled = 1
gpgcheck = 0
#gpgkey = file:///etc/pki/rpm-gpg/PERCONA-PACKAGING-KEY

[psmdb-60-release-sources]
name = Percona Server for MongoDB 6.0 release/sources YUM repository
baseurl =␣
↪http://repo.percona.com/psmdb-60/yum/release/$releasever/SRPMS
enabled = 0
gpgcheck = 0
#gpgkey = file:///etc/pki/rpm-gpg/PERCONA-PACKAGING-KEY

[tools-release-x86_64]
name = Percona Tools release/x86_64 YUM repository
baseurl =␣
↪http://repo.percona.com/tools/yum/release/$releasever/RPMS/x86_64
```

(continues on next page)

(continued from previous page)

```
enabled = 1
gpgcheck = 0
#gpgkey = file:///etc/pki/rpm-gpg/PERCONA-PACKAGING-KEY

[tools-release-noarch]
name = Percona Tools release/noarch YUM repository
baseurl =␣
↪http://repo.percona.com/tools/yum/release/$releasever/RPMS/noarch
enabled = 1
gpgcheck = 0
#gpgkey = file:///etc/pki/rpm-gpg/PERCONA-PACKAGING-KEY

[tools-release-sources]
name = Percona Tools release/sources YUM repository
baseurl = http://repo.percona.com/tools/yum/release/$releasever/SRPMS
enabled = 0
gpgcheck = 0
#gpgkey = file:///etc/pki/rpm-gpg/PERCONA-PACKAGING-KEY
```

Note

Note Disable GPGkey checking if you don't have an internet connection. Furthermore, only one version (i.e., either 6.0, 5.0, 4.4) can be supported at any given time. The reason is, that all the versions are called "percona-server-mongodb".

2. Clear the repository cache and list out the repositories, make sure the newly added MongoDB repository is in the list:

```
$ yum clean all
$ yum repolist
```

3. Set up the repository directory to be served by the web server for Percona:

```
$ mkdir -p /var/www/html/repos/percona
```

4. Perform repository syncing:

```
$ dnf reposync --delete -p /var/www/html/repos/percona --
↪repoid=psmdb-60-release-x86_64 --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/percona --
↪repoid=tools-release-x86_64 --newest-only --download-metadata
```

Adding Oracle MySQL repository to the repo server

1. Download the appropriate Oracle MySQL community release package from this page:

```
$ yum install https://dev.mysql.com/get/mysql84-community-release-
↪{os_code}-1.noarch.rpm
```

Note

Note Replace the `{os_code}` for each verion of Operating System; `el7`, `el8`, `el9`.

2. Clear the repository cache and list out the repositories, make sure the newly added MySQL 8 repository is in the list:

```
$ yum clean all
$ yum repolist
```

3. Set up the repository directory to be served by the web server for Oracle:

```
$ mkdir -p /var/www/html/repos/oracle
```

4. Perform repository syncing:

```
$ dnf reposync --delete -p /var/www/html/repos/oracle --
↪repoid=mysql80-community --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/oracle --
↪repoid=mysql-connectors-community --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/oracle --
↪repoid=mysql-tools-community --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/oracle --
↪repoid=mysql-cluster-8.0-community --newest-only --download-metadata
```

Adding Percona MySQL repository to the repo server

1. Download the appropriate Percona MySQL release package from this page:

```
$ yum install␣
↪https://repo.percona.com/yum/percona-release-latest.noarch.rpm
```

2. Enable the Percona Server for MSQL 8.0 (the latest version at the time of this writing):

```
$ percona-release setup ps80
```

3. Clear the repository cache and list out the repositories, make sure the newly added Percona Server repository is in the list:

```
$ yum clean all
$ yum repolist
```

4. Set up the repository directory to be served by the web server for Percona (if not exists):

```
$ mkdir -p /var/www/html/repos/percona
```

5. Perform repository syncing:

```
$ dnf reposync --delete -p /var/www/html/repos/percona --
↪repoid=ps-80-release-x86_64 --newest-only --download-metadata
```

Adding MariaDB repository to the repo server

1. Install the MariaDB repository by using the following script, taken from this page:

```
$ curl -LsS https://r.mariadb.com/downloads/mariadb_repo_setup | sudo␣
↪bash
```

2. Clear the repository cache and list out the repositories, make sure the newly added MariaDB repository is in the list:

```
$ yum clean all
$ yum repolist
```

3. Set up the repository directory to be served by the web server for MariaDB:

```
$ mkdir -p /var/www/html/repos/mariadb
```

4. Perform repository syncing:

```
$ dnf reposync --delete -p /var/www/html/repos/mariadb --
↪repoid=mariadb-main --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/mariadb --
↪repoid=mariadb-maxscale --newest-only --download-metadata
$ dnf reposync --delete -p /var/www/html/repos/mariadb --
↪repoid=mariadb-tools --newest-only --download-metadata
```

Creating Repository

Now we have downloaded all the necessary packages, it is time to create the repository on the repo server:

```
$ createrepo /var/www/html/repos/rhel
$ createrepo /var/www/html/repos/s9s
$ createrepo /var/www/html/repos/postgresql
$ createrepo /var/www/html/repos/mongodb
$ createrepo /var/www/html/repos/percona
$ createrepo /var/www/html/repos/oracle
$ createrepo /var/www/html/repos/mariadb
```

Just to give some heads up on the disk usage, here is how much you need for the repository directory:

```
$ du -sh /var/www/html/repos
38G   /var/www/html/repos/
```

At this point, the repo server should be ready with all the necessary packages for OS, OS tools, third-party dependencies (Perl, Python, wget, tar, socat, net-tools, etc) and databases such as MySQL, MariaDB, PostgreSQL, MongoDB (from MongoDB Org and Percona). We should see the following directory listing under `/var/www/html/repos`:

```
$ tree -d -L 2 /var/www/html/repos
.
├── mariadb
│   ├── mariadb-main
```

(continues on next page)

(continued from previous page)

```
|   ├── mariadb-maxscale
|   ├── mariadb-tools
|   └── repodata
├── mongodb
|   ├── mongodb-org-6.0-latest
|   └── repodata
├── oracle
|   ├── mysql57-community
|   ├── mysql80-community
|   ├── mysql-cluster-7.6-community
|   ├── mysql-cluster-8.0-community
|   ├── mysql-connectors-community
|   ├── mysql-tools-community
|   └── repodata
├── percona
|   ├── ps-80-release-x86_64
|   ├── psmdb-60-release-x86_64
|   ├── repodata
|   └── tools-release-x86_64
├── postgresql
|   ├── pgdg13
|   ├── pgdg14
|   ├── pgdg-common
|   └── repodata
└── rhel
    ├── base
    ├── epel
    ├── repodata
    └── updates
```

1.77.2 Setting Up Database Nodes

Next, we need to set up the following things on all database nodes:

- Configure passwordless SSH from the ClusterControl node to all database nodes.
- Configure the package repository to be pointing to the satellite server.

Suppose there are 3 database nodes to deploy a PostgreSQL Streaming Replication, with the following host details:

- 192.168.100.100 – Repo server
- 192.168.100.200 – ClusterControl
- 192.168.100.211 – PostgreSQL db1
- 192.168.100.212 – PostgreSQL db2
- 192.168.100.213 – PostgreSQL db3

Configure Passwordless SSH

Make sure the appropriate user (root or sudo user) can SSH from the ClusterControl node to the database nodes.

1. On the ClusterControl server, generate an SSH key to be used for passwordless SSH:

```
$ whoami
root
$ ssh-keygen -t rsa # press Enter on all prompts
```

2. Copy this SSH key to all the database nodes:

```
$ ssh-copy-id root@192.168.100.211 # db1
$ ssh-copy-id root@192.168.100.212 # db2
$ ssh-copy-id root@192.168.100.213 # db3
```

   If the database server does not support SSH password authentication, you need to copy the SSH public key located at `/root/.ssh/id_rsa.pub` on the ClusterControl server into `/root/.ssh/authorized_keys` on every database node. See *SSH Key-based Authentication* for more details.

3. Verify that you should be able to execute the following command without error:

```
$ ssh root@192.168.100.211 "ls -l /sbin"
```

We have now configured the passwordless SSH.

Configure OS Repository

Perform the following commands on all nodes (ClusterControl and all database nodes):

1. Create a backup directory to existing the repository definition files:

```
$ mkdir ~/repos.d.backup
```

2. Move existing repository definition files to the backup directory:

```
$ mv /etc/yum.repos.d/*.repo ~/repos.d.backup
```

3. Create a repository definition file pointing to the repo server created in Setting up Satellite Repository Server.

```
$ vi /etc/yum.repos.d/satellite.repo
```

   And add the following lines:

```
[rhel]
name=RHEL Local Repo Server
baseurl=http://192.168.100.100/repos/rhel
enabled=1
gpgcheck=0
```

4. Refresh the repository list:

```
$ sudo yum clean all
$ sudo yum repolist # make sure only the above repository is listed
```

Configure Severalnines Repository

Perform the following commands on ClusterControl nodes:

1. Create a repository definition file pointing to the repo server created in Setting up Satellite Repository Server.

```
$ vi /etc/yum.repos.d/s9s.repo
```

And add the following lines:

```
```ini
[s9s-repo]
name = Severalnines Release Repository
baseurl=http://192.168.100.100/s9s-repo/
enabled=1
gpgcheck=0

[s9s-tools]
name = Severalnines Tools Repository
baseurl=http://192.168.100.100/s9s-tools/
enabled=1
gpgcheck=0
```
```

2. Refresh the repository list:

```
$ sudo yum clean all
$ sudo yum repolist   # make sure only the above repository is listed
```

Configure Database Repository

Perform the following commands on all database nodes, based on the database type that each node wants to deploy.

1. Create a backup directory to existing the repository definition files:

```
$ mkdir ~/repos.d.backup
```

2. Move existing repository definition files to the backup directory:

```
$ mv /etc/yum.repos.d/*.repo ~/repos.d.backup
```

3. Create a repository definition file pointing to the repo server created in *Setting up Satellite Repository Server*.

   For MariaDB (MariaDB Server, MariaDB Galera):

```
$ vi /etc/yum.repos.d/mariadb.repo
```

And add the following lines:

```
[mariadb]
name=MariaDB Local Repo Server
baseurl=http://192.168.100.100/repos/mariadb
enabled=1
gpgcheck=0
```

For MongoDB Inc (MongoDB):

```
$ vi /etc/yum.repos.d/mongodb.repo
```

And add the following lines:

```
[mongodb]
name=MongoDB Local Repo Server
baseurl=http://192.168.100.100/repos/mongodb
enabled=1
gpgcheck=0
```

For Percona (Percona Server, Percona XtraDB Cluster, Percona Server for MongoDB):

```
$ vi /etc/yum.repos.d/percona.repo
```

And add the following lines:

```
[percona]
name=Percona Local Repo Server
baseurl=http://192.168.100.100/repos/percona
enabled=1
gpgcheck=0
```

For PostgreSQL:

```
$ vi /etc/yum.repos.d/postgresql.repo
```

And add the following lines:

```
[postgresql]
name=PostgreSQL Local Repo Server
baseurl=http://192.168.100.100/repos/postgresql
enabled=1
gpgcheck=0
```

For Oracle (MySQL Replication, MySQL Cluster):

```
$ vi /etc/yum.repos.d/oracle.repo
```

And add the following lines:

```
[oracle]
name=Oracle Local Repo Server
baseurl=http://192.168.100.100/repos/oracle
```

(continues on next page)

(continued from previous page)

```
enabled=1
gpgcheck=0
```

4. Refresh the repository list:

```
$ sudo yum clean all
$ sudo yum repolist # make sure only the above repository is listed
```

The repository configuration is now complete. We may proceed to perform ClusterControl installation and subsequently deploy our database clusters without an Internet connection.

1.77.3 ClusterControl Offline Installation

For offline installation, ClusterControl now support using the `install-cc` script with offline mode:

1. Install MySQL or MariaDB server for CMON database
2. Download `install-cc` and install the ClusterControl

Install MySQL/MariaDB

MariaDB will be used by ClusterControl to store configuration and monitoring data for its operation.

1. On the ClusterControl server, run the following commands to install MariaDB and configure the MariaDB root password:

```
$ dnf clean all
$ dnf repolist
$ dnf install mariadb mariadb-server
$ systemctl enable mariadb
$ systemctl start mariadb
$ mysqladmin -uroot password yourR00tP4ssw0rd
```

   Or you can install the MySQL and configure the password:

```
$ dnf install mysql-community-server
$ systemctl start mysqld
$ systemctl status mysqld
$ systemctl enable mysqld
```

2. Check the default `root` password for MySQL in `/var/log/mysqld.log`. Later you can reset the root password based on the policy.

```
mysql -u root -p
mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY 'xxx';
```

Install ClusterControl

1. Download the `install-cc` from this link and put the file into ClusterControl hosts. Give execute permission to the file:

```
$ chmod +x install-cc
```

2. The `install-cc` support offline installation by adding the parameter `OFFLINE=1`. Execute the following in ClusterControl host for the installation:

```
$ S9S_ADMIN_EMAIL="user@example.com" OFFLINE=1 S9S_ROOT_PASSWORD="XXX"␣
↪S9S_CMON_PASSWORD="YYY" ./install-cc
```

> **Note**
>
> Note Fill the `S9S_ADMIN_EMAIL` using your email, `S9S_ROOT_PASSWORD` with the correct password and `S9S_CMON_PASSWORD` you defined.

3. Run the installer script to finish configuring ClusterControl and all of its components (we prefixed the command with a variable `S9S_ADMIN_EMAIL` to be passed to the script to simplify the installation).

   Answer all the questions to complete the installation. Specify the MySQL/MariaDB root password when asked, as configured under the Install MySQL/MariaDB section.

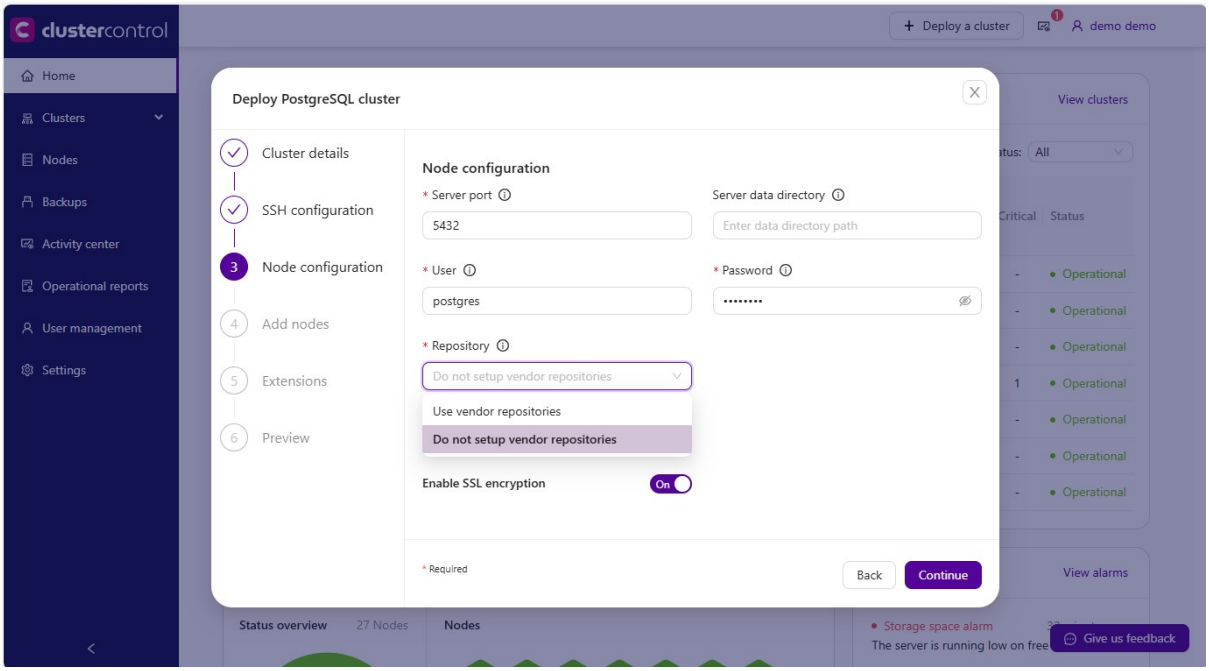
   You should see the following lines:

```
ClusterControl installation completed!
Open your web browser to https://10.0.2.15 and create a default Admin␣
↪User.
If you want to uninstall ClusterControl then please follow the␣
↪instructions here,␣
↪https://severalnines.com/docs/administration.html#uninstall
```

   You may ignore the error because the installer script was trying to reach out to the internet for reverse lookup which is expected. Open the ClusterControl UI and register a super admin user to start deploying database clusters.

1.77.4 Offline Database Deployment using ClusterControl

Once you are logged in to ClusterControl UI, you may proceed to deploy a database cluster by going to ***ClusterControl → Deploy a cluster*** and filling up all necessary details. One thing in particular for this kind of environment, you have to choose **"Do Not Setup Vendor Repositories"** under the ***Repository*** dropdown, since we already preconfigured the repository definition as shown in the *Configure Database Repository* section:



If you would like to deploy a cluster by using CluserControl CLI for the deployment, do not forget to specify `--use-internal-repos` flag, as shown below:

```
$ s9s cluster --create \
        --cluster-type=postgresql \
        --nodes=
↪"192.168.100.211?master;192.168.100.212?slave;192.168.100.213?slave;" \
        --db-admin="postgres" \
        --db-admin-passwd="mySuperStongP455w0rd" \
        --cluster-name=ft_replication_23986 \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --provider-version=13 \
        --use-internal-repos \
        --log
```

That's it. The database deployment should be able to use the satellite repository server for an offline installation.

Setting up Prometheus exporters

For the offline installation of Prometheus and its exporters, you need to download all the required packages first and put the packages in the ClusterControl node (put in under `/var/cache/cmon/packages`).

The URL of the packages can be found in `/usr/share/cmon/templates/packages.conf` as shown below:

```
$ cat /usr/share/cmon/templates/packages.conf
#
# ClusterControl central URL database for
# 3rd party packages installed directly
# (not using APT/YUM repos)
#
```

(continues on next page)

(continued from previous page)

```
[maxscale]
# clustercontrol will substitute the following keywords:
# @VENDOR@  : debian/ubuntu...
# @RELEASE@ : 5,6,7,xenial,wily...
url_deb=
↪"https://downloads.mariadb.com/MaxScale/2.5.7/packages/@VENDOR@/@RELEASE@/maxscale-2.5.
tected]_64.deb"
url_rpm=
↪"https://downloads.mariadb.com/files/MaxScale/2.5.7/packages/rhel/@RELEASE@/maxscale-2.
tected]_64.rpm"

[haproxy]
url_source="http://www.haproxy.org/download/1.8/src/haproxy-1.8.9.tar.gz"

[keepalived]
url_source="http://www.keepalived.org/software/keepalived-1.2.24.tar.gz"

[epel]
# To be substituted:
# @RELEASE@ : 6,7,8
url_rpm=
↪"http://dl.fedoraproject.org/pub/epel/epel-release-latest-@[email pro-
tected]"

[prometheus]
# requiredVersion: if such or greater version found
# clustercontrol will use the existing installed package
requiredVersion=2.29
url=
↪"https://github.com/prometheus/prometheus/releases/download/v2.29.2/prometheus-2.29.2.l

[haproxy_exporter]
requiredVersion=0.9
url=
↪"https://github.com/prometheus/haproxy_exporter/releases/download/v0.9.0/haproxy_export

[node_exporter]
requiredVersion=1.0.1
url=
↪"https://github.com/prometheus/node_exporter/releases/download/v1.0.1/node_exporter-1.0

[mysqld_exporter]
requiredVersion=0.13
url=
↪"https://github.com/prometheus/mysqld_exporter/releases/download/v0.13.0/mysqld_exporte

[postgres_exporter]
requiredVersion=0.4.7
```

(continues on next page)

(continued from previous page)

```
url=
↪"https://github.com/wrouesnel/postgres_exporter/releases/download/v0.4.7/postgres_expor

[pgbouncer_exporter]
requiredVersion=0.4.0
url=
↪"https://github.com/prometheus-community/pgbouncer_exporter/releases/download/v0.4.0/pg

[proxysql_exporter]
requiredVersion=1.1
url=
↪"https://github.com/percona/proxysql_exporter/releases/download/v1.1.0/proxysql_exporte

[process_exporter]
requiredVersion=0.10.10
url=
↪"https://github.com/kedazo/process_exporter/releases/download/0.10.10/process_exporter-

[mongodb_exporter]
requiredVersion=0.11.0
url=
↪"https://github.com/kedazo/mongodb_exporter/releases/download/v0.11.0/mongodb_exporter-

[redis_exporter]
requiredVersion=1.15.0
url=
↪"https://github.com/oliver006/redis_exporter/releases/download/v1.15.0/redis_exporter-v

## for MSSQL
[mssql_exporter]
requiredVersion=0.5.4
url=
↪"https://github.com/severalnines/mssql_exporter/releases/download/0.5.4/mssql_exporter-

[daemon]
url_rpm="http://libslack.org/daemon/download/daemon-0.6.4-1.x86_64.rpm"
```

After the dependencies exist on the server, you may proceed to enable Prometheus agent-based monitoring by going to ***ClusterControl → Dashboards → Enable Agent-based Monitoring***.

In this article, we explore how to deploy a secondary cluster and failover with ClusterControl. A secondary cluster provides high availability and disaster recovery, ensuring minimal downtime in case of primary cluster failure.

1.78 Use cases

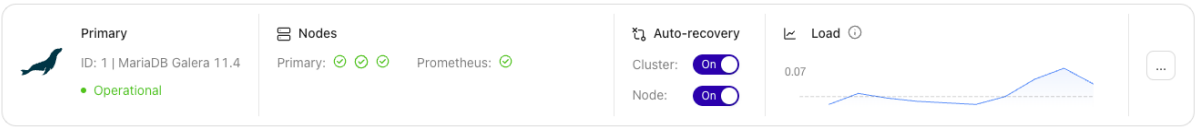
Deploying and failing over to a secondary cluster is useful in the following scenarios:

- Ensuring high availability in case of primary cluster failure.
- Implementing disaster recovery strategies for business continuity.

- Performing planned maintenance without downtime.
- Distributing read traffic across multiple clusters.
- Enabling quick recovery from unexpected infrastructure issues.

1.79 Deploying a Secondary Cluster

To deploy a Secondary Cluster from ClusterControl, first, you need to have a Primary cluster in place.

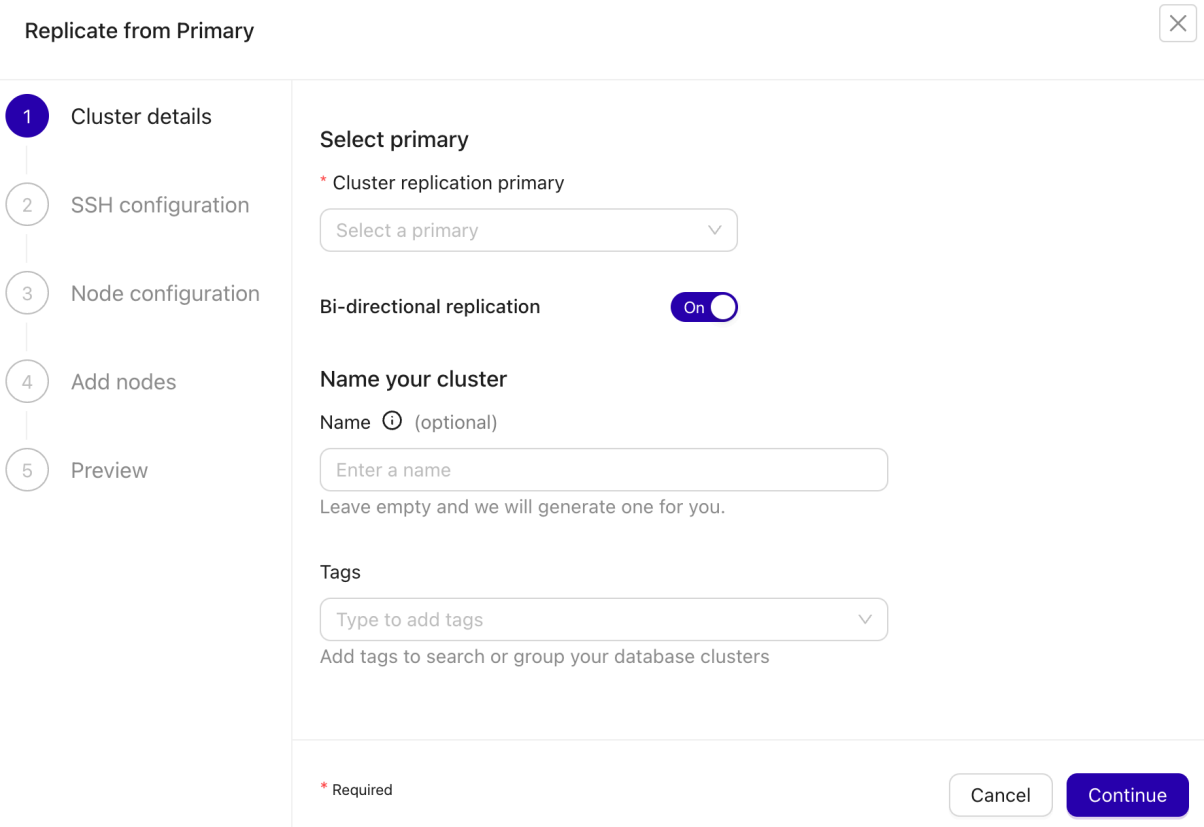


For this, follow the corresponding instructions here: *Create Database Cluster*.

After you have the Primary cluster deployed, go to ***ClusterControl GUI → Clusters → Cluster Actions → Clone/Replicate the cluster → Create replica cluster***.

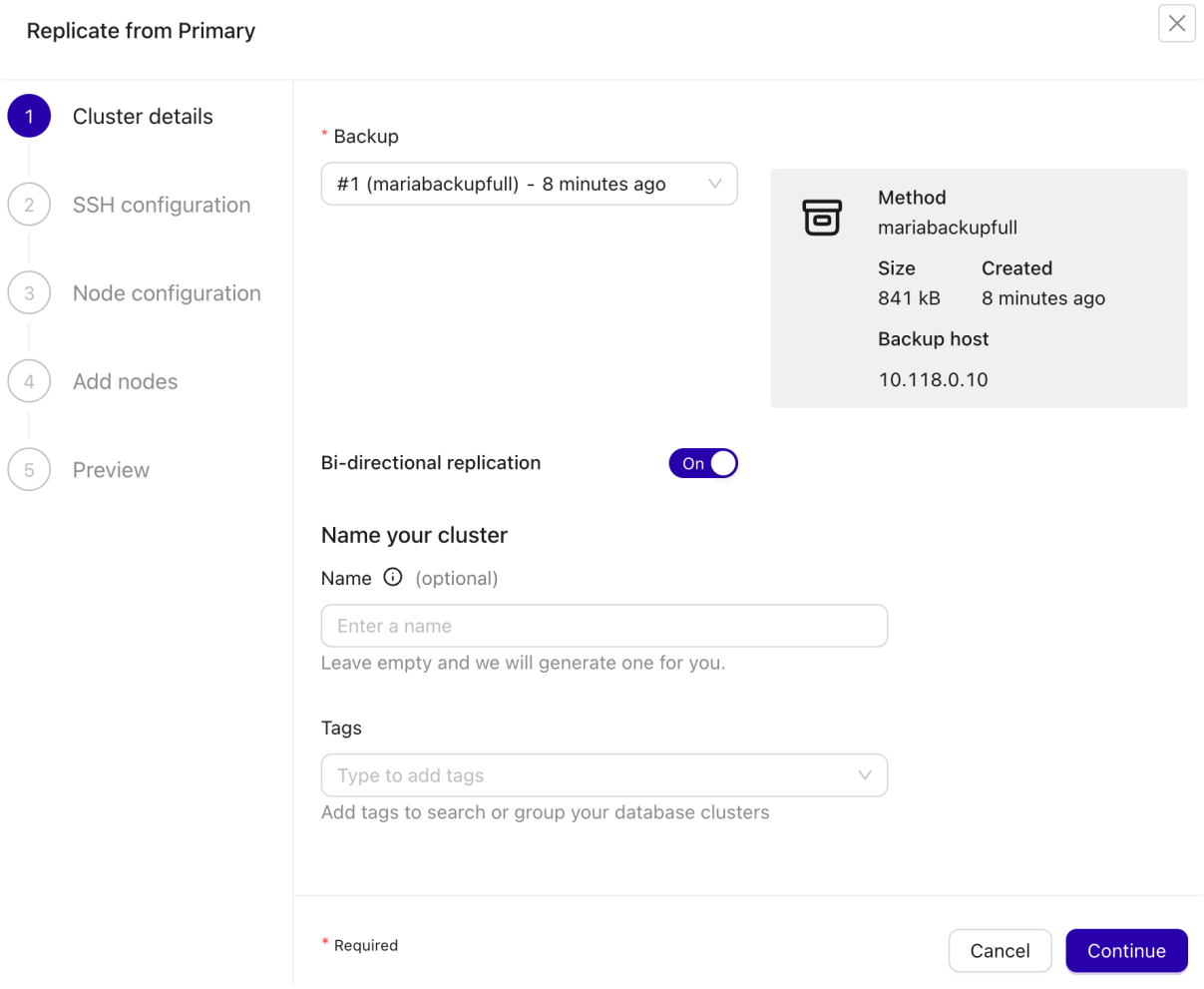
There, you can choose if you want to get data from a primary cluster or from an existing backup.

1.79.1 Replicate from Primary



1. When replicating from a primary cluster, choose the node from where you are going to replicate data from and specify if it is a bidirectional replication.
2. Configure the SSH credentials and security configuration.
3. Specify the node configuration like server port, data directory, version, etc.
4. Add the node IP address or hostname to be added to the secondary cluster.

1.79.2 Replicate from Backups

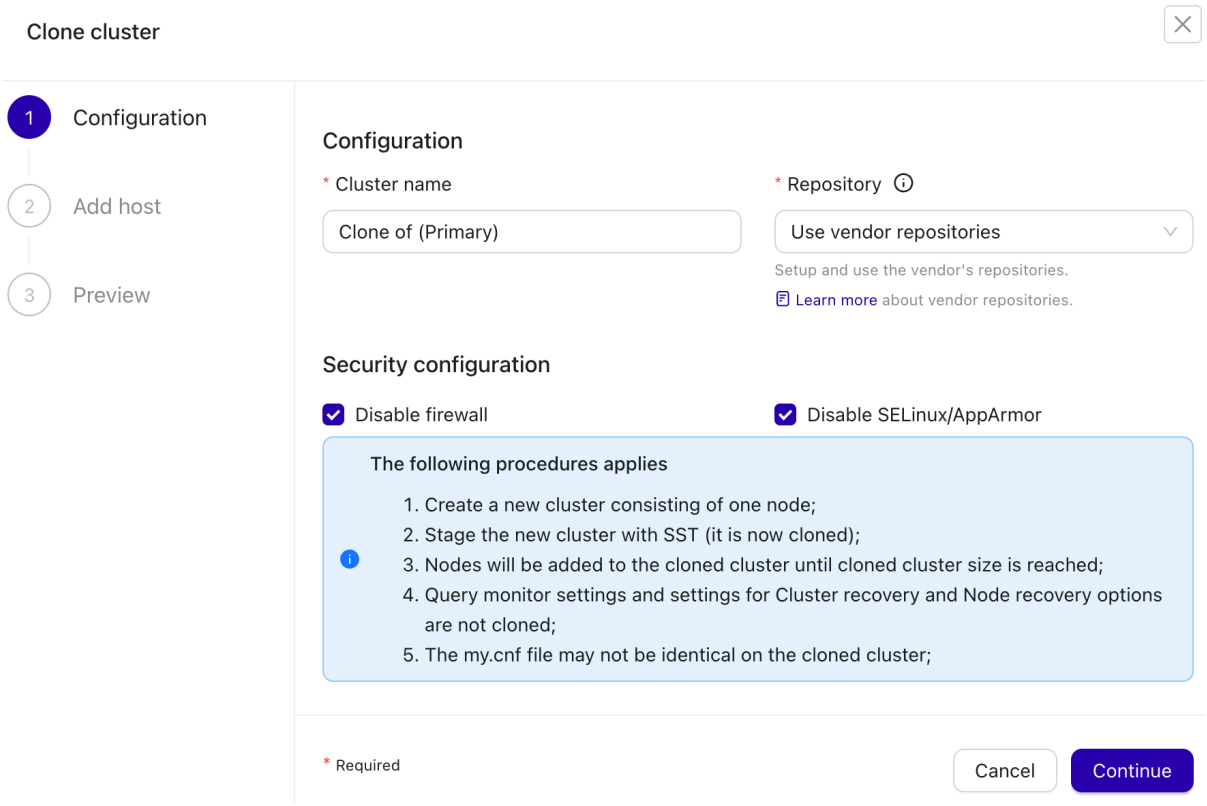


1. When replicating from backups, choose the backup to be restored in the new cluster and specify if it is a bidirectional replication.
2. Configure the SSH credentials and security configuration.
3. Specify the node configuration like server port, data directory, version, etc.
4. Add the node IP address or hostname to be added to the secondary cluster.

1.79.3 Clone cluster

In case you don't want to have the latest data replicated to the new cluster, you can just clone the existing cluster and keep it as a separated cluster for testing or datawarehousing.

For this, after you have the primary cluster deployed, go to ***ClusterControl → Clusters → Cluster Actions → Clone/Replicate the cluster → Clone cluster***.



This will create a exact copy of the primary cluster without configuring a replication between the primary and the new cluster.

1.80 Failover

To failover to the Secondary cluster, you need to disable read-only to be able to write data on it, and change the endpoint to use the new cluster.

Note

Note If you are using a load balancer, depending on the configuration, most probably the endpoint change won't be necessary.

When the Primary cluster is back, if the replication configured between the clusters was bi-directional, the cluster will get the latest data when the replication lag reach 0. Otherwise, you have to rebuild the Primary cluster from the Secondary one and enable read-only after failover back, then update the endpoint again.

1.81 Cluster Replication Status using CLI

To see the Cluster replication status in the CLI, run:

```
s9s replication --list --long
```

You will get an output like this:

```
CID      SLAVE                MASTER               STATUS  MASTER_CLUSTER  LAG
1        10.10.10.122:3306    10.10.10.125:3306    Online  3               0
```

(continues on next page)

(continued from previous page)

```
3       10.10.10.125:3306   10.10.10.122:3306   Online  1               0
Total: 2 replication link(s)
```

In this article, we explore how to perform a PostgreSQL major version upgrade using ClusterControl. A major upgrade involves moving from one major release series to another (for example, PostgreSQL 15 to 16). Because such upgrades may introduce new system catalog structures and changes to internal mechanisms, it is crucial to plan carefully and follow best practices to minimize downtime and avoid data corruption.

1.82 Use cases

Here are some common scenarios in which a major PostgreSQL upgrade is necessary:

- **Feature requirements**: New PostgreSQL versions often include performance improvements, new SQL features, and improved security defaults.
- **End-of-Life (EOL)**: Upgrading ensures you stay on a supported version of PostgreSQL to receive security patches and critical bug fixes.
- **Regulatory and compliance**: Some organizations have strict rules about running supported, up-to-date software versions.
- **Performance enhancements**: Certain major versions bring substantial improvements in indexing, partitioning, and query optimizers.

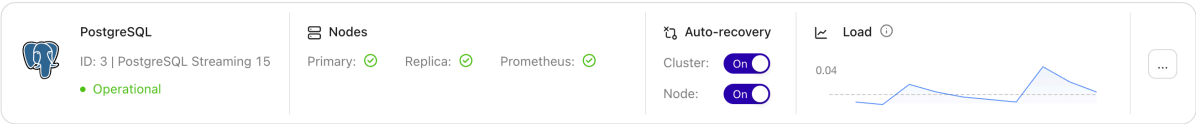
1.83 Important considerations

Before undertaking a major upgrade, consider the following points:

- **Backup everything**: Ensure you have a full backup of all databases.
- **Plan downtime**: Major upgrades typically require some downtime or a strategic failover approach.
- **Validate extensions**: Extensions, custom functions, and plugins might need new versions compatible with the target PostgreSQL release.
- **Test**: Always test the upgrade on a non-production environment first.
- **Roll-back plan**: The safest rollback approach is to restore from a full backup of the old version or keep an untouched replica on the old version until the new version is fully validated.

1.84 Deploying a PostgreSQL cluster

First, you need to have a PostgreSQL cluster *deployed by ClusterControl* or *imported into ClusterControl*.



After you have the PostgreSQL cluster in place, go to ***ClusterControl → Clusters → Cluster Actions → Upgrades*** where you can chose between `minor` or `major` upgrades.

1.85 Major upgrades for PostgreSQL

Choose **Major Upgrades** to performs an upgrade to a later major version for the database software and data files. The upgrade job will upgrade all the nodes in the cluster by default. During the steps, you have to specify temporary ports and upgrade methods.

Warning

Attention

Depending on the upgrade method you might require the storage capacity to hold two data sets.

1.85.1 Upgrade methods

There are three methods for upgrading your PostgreSQL cluster in ClusterControl:

- **copy**: This is the default method with `pg_upgrade`. The `--copy` option is used which will copy files at a time. It is faster than doing regular backup and restore. This requires disk storage capacity to hold both the old and the new files.

Warning

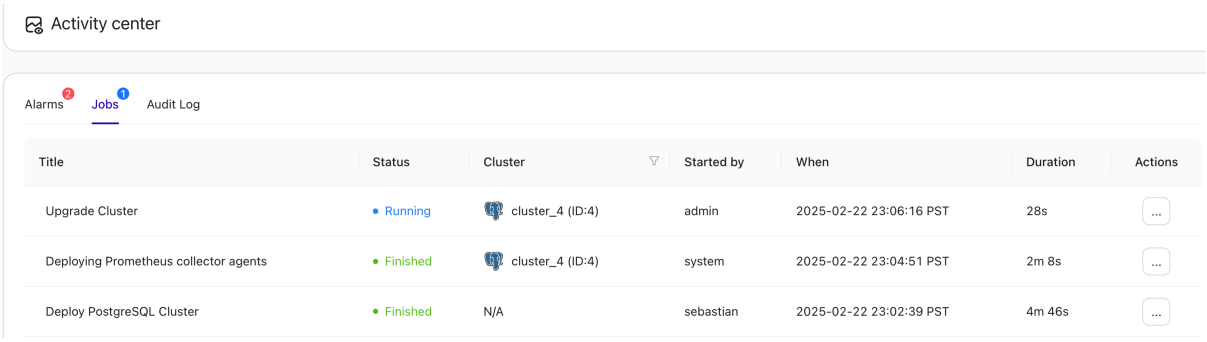
Attention

```
Data stored in user-defined tablespaces is not being copied to the new␣
↪cluster during an upgrade. It remains in its original file system␣
↪location but is duplicated into a subdirectory indicating the version␣
↪number of the new cluster. To manually relocate files that are stored␣
↪in a tablespace after upgrading, move the files to the new location and␣
↪update the symbolic links to point to the files.
```

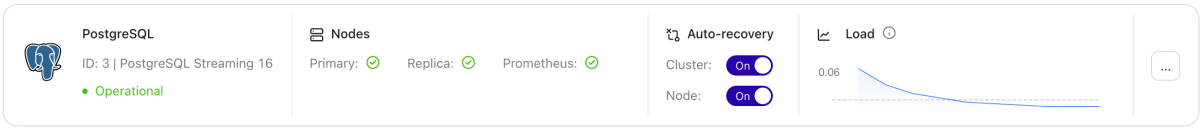
- **link**: This method uses the `--link` option with `pg_ugprade`. Hard links will be used instead of copying files which is faster and requires no extra disk storage. It is crucial to refrain from starting the old cluster after this step to avert any potential data corruption with the new cluster since they share the data.
- **pgdumpall**: It uses pgdumpall to copy files by performing a backup on the old cluster and then restoring it on the new cluster. This requires disk storage capacity to hold the backup.

1.85.2 Monitoring upgrade process

To monitor the upgrade job in ClusterControl, go to ***ClusterControl → Activity center → Jobs*** where you can see the running jobs in the UI.



When the job in completed, you will see the cluster upgraded in the ClusterControl UI.



1.86 Using the ClusterControl CLI

You can also perform major upgrades by using the ClusterControl CLI from the ClusterControl node.

```
s9s cluster \
    --upgrade-cluster \
    --upgrade-to-version=16 \
    --upgrade-method=link \
    --cluster-id=1 \
    --log
```

This command will upgrade cluster packages to a newer major version 16 for cluster ID 1 with `link` as upgrade method.

1.86.1 Monitoring the upgrade

```
s9s job \
    --list \
    --long
```

You will get something like this:

```
Upgrade Cluster
Installing software                                                    [
                                                                    20.00%

Created   : 2025-02-23 07:06:16    ID    : 19         Status : RUNNING
Started   : 2025-02-23 07:06:16    User  : admin      Host   : 127.0.0.1
Ended     :                       Group : admins     Cluster: 4
Tags      : -
RPC       : 2.0
```

Note

Note During the upgrade process, the cluster will not be operational. A major upgrade is performed at your own risk. Severalnines will not support any breaking issues that might occur during this operation.

In this article, we explore how to configure and manage backups in the cloud using ClusterControl. Storing and retrieving your database backups offsite helps ensure robust disaster recovery strategies, reduces on-premise storage needs, and simplifies the process of restoring data in the event of a system failure or data corruption.

1.87 Use cases

Storing backups in the cloud is useful for various scenarios, including:

- **Disaster Recovery**: Keeping copies of your data offsite in cloud storage helps ensure you have reliable restore points in the event of on-premise system failures.
- **Storage Optimization**: Offloading backup files to cloud storage can help reduce local disk usage and associated costs.
- **Compliance Requirements**: Many organizations must maintain offsite backups to meet compliance mandates.
- **Scalability**: Cloud storage services can easily scale as your backup size grows, eliminating the complexity of managing additional on-premise infrastructure.
- **Testing & Development**: Quickly restoring backups from cloud storage to dev/test environments makes it easy to replicate real-world data for application testing.

1.88 Configuring Cloud Storage

Before you can perform backups to or restores from the cloud, you need to configure storage settings in ClusterControl. Depending on your preferred cloud provider (**Amazon S3**, **Google Cloud Storage**, **Azure Blob Storage**), the general steps are similar.

1. **Obtain Your Cloud Credentials**
    - For **Amazon S3**, gather the `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_REGION`.
    - For **Google Cloud**, you should have a service account key (JSON file) that you can point to.
    - For **Azure**, you need to create an storage account and get information such as the `application_id`, `client_secret`, `tenant_id`, etc.
    - For **S3-Compatible Storage Provider**, required information is similar to the **Amazon S3** one.
2. **Configure Cloud storage credentials in ClusterControl**

In the ClusterControl web interface, navigate to Settings > Cloud storage credentials, and add a new storage profile for your cloud provider. Here you can specify:

- Provider: For example, S3
- Credential/Keyfile: GCP JSON key, or AWS key references
- Region: if needed

Once saved, ClusterControl will be aware of your cloud storage location for backup.

1.89 Creating a Backup in the Cloud

Once the cloud storage is configured, perform a backup by following these steps:

1. Navigate to Backup > Create backup. (Or select your cluster and go to Backups)
2. Choose Backup on Demand
3. Select your database cluster
4. Choose the backup method (`mysqldump`, `pg_basebackup`, `xtrabackup`)
5. Enable Upload backup to cloud
6. Follow the next steps
7. Click Create

Monitor the backup process from the Backup tab. Once completed, your backup will be available in the configured cloud storage provider.

1.90 Restoring from the Cloud

To restore a backup from the cloud:

1. Navigate to Backup > Backup Storage and verify the stored backups.
2. Select the backup you want to restore.
3. Click Restore and choose the target database cluster.
4. Confirm the restore process.

ClusterControl will download the backup from the cloud and restore it to the specified database cluster.

1.91 Automating Backups

To automate backups, set up a schedule backup:

1. Navigate to Backup > Create backup
2. Choose Schedule a Backup
3. Select your database cluster
4. Choose the backup method (`mysqldump`, `pg_basebackup`, `xtrabackup`)
5. Enable Upload backup to cloud
6. Follow the next steps
7. Set the schedule (`daily`, `weekly`, etc)
8. Click Create

ClusterControl will automatically create backups at the specified intervals and store them in the cloud.

1.92 Verifying Backups

It is a best practice to periodically verify that your backups can be restored successfully. To automate this, use the Verify backup feature in ClusterControl, or perform the following steps manually:

1. Spinning up a temporary replica or development environment
2. Download the backup from the cloud (it can be done from the ClusterControl UI)
3. Restoring the cloud backup
4. Performing integrity checks on the restored data

1.93 Accessing Cloud Backups via CLI

Below is an example of how to create and restore backups from the command line. These commands should be run on the ClusterControl server.

- List clusters to find the `--cluster-id`

```
s9s cluster --list --long
```

- Create a backup job to the configured cloud storage

```
s9s backup \
    --create \
    --cluster-id=10 \
    --backup-method=xtrabackupfull \
    --cloud-only \
    --cloud-provider="aws" \
    --s3-bucket="my-aws-bucket-eu-west-1" \
    --credential-id=2 \
    --wait
```

- List Available Backups

```
s9s backup --list --long
```

- Restore a Backup

```
s9s backup \
    --restore \
    --backup-id=5 \
    --cluster-id=10 \
        --wait
```

In this article, we explore how to perform a **Database Point-in-time Recovery (PITR)** using ClusterControl. Point-in-time recovery allows you to restore your database to a specific time, often just before a disruptive event (for example, accidental data deletion or corruption). By combining full backups with transaction logs (`binlogs`, `WAL`, or equivalent), ClusterControl can help you recover to a precise moment to minimize data loss.

1.94 Use cases

Point-in-time recovery is useful in scenarios such as:

- **Accidental Data Modification**: A user or application drops a table or updates critical rows by mistake.
- **Data Corruption**: Software or hardware errors that corrupt database pages or files, requiring you to roll back to a clean state.
- **Ransomware or Security Incidents**: Restoring to the moment before an intrusion or malicious activity was detected.
- **Testing & Compliance**: Validating that your backups and binlogs/WAL archives can reconstruct the database to any point in time to meet audit or compliance requirements.

1.95 Configuring point-in-time recovery

1.95.1 Enable transaction log archiving

Depending on your database type, you must ensure the transaction logs are retained long enough to allow PITR. Below are examples for MySQL-based and PostgreSQL-based systems:

MySQL/MariaDB

- Make sure binary logging is enabled and that the retention is appropriately set in the MySQL configuration file (usually `my.cnf`):

```
[mysqld]
log_bin                = binlog
binlog_format          = ROW
expire_logs_days       = 7
server_id              = 100
```

- Restart the MySQL/MariaDB server for changes to take effect.

PostgreSQL

- Configure continuous WAL archiving by editing `postgresql.conf`:

```
wal_level = replica
archive_mode = on
archive_command = 'test ! -f /mnt/server/archivedir/%f && cp %p /
↪mnt/server/archivedir/%f'
```

- Ensure sufficient storage space for WAL archives and that the retention policy aligns with your PITR requirements.
- Restart PostgreSQL for changes to take effect.

Note

Note You can also enable binary logging from the ClusterControl UI.

1.95.2 Enable full backup

Before PITR can be performed, you need consistent base backups:

Choose a backup method

- In the ClusterControl UI, create or scheduled a backup using a supported method (`mysqldump`, `xtrabackup`, `pg_basebackup`, etc).
- Navigate to **Backups** on your cluster.
- Create or schedule a backup job.
- Verify the backup storage location (local or/and cloud).

Validate backups

- Check that backups are running successfully (in **Jobs** or **Backups** sections).
- Periodically restore test backups to ensure they are valid (ClusterControl Verify backup feature helps on this task).

1.96 Performing point-in-time recovery

Once full backups and transaction log archiving are in place, initiate a PITR from ClusterControl.

1.96.1 Using the ClusterControl GUI

1. **Identify the desired recovery time**
   - Determine the timestamp just before an unwanted change or event.
   - Alternatively, note the binlog or WAL position if you have that information.
2. **Navigate to the Backups section**
   - Under the **Backups** section, locate a full backup that predates your desired recovery point.
   - Select **Restore**.
3. **Confirm how to restore the backup**
   - Restore on node or on standalone host.
4. **Specify the recovery point**
   - Enable PITR and provide a timestamp or binlog/WAL sequence.
   - Adjust any additional settings as needed (`Temporary directory`).
5. **Initiate the PITR job**
   - ClusterControl will restore the base backup.
   - After the base restore completes, it will replay the transaction logs or WAL files up until the specified time.
   - Monitor the **Jobs** tab or **Logs** to track progress.

1.96.2 Using the ClusterControl CLI

Below is an example flow for using ClusterControl CLI for MySQL/MariaDB point-in-time recovery tasks:

1. List clusters and backups:

```
s9s backup --list --long
```

   Identify the `--cluster-id` (**CID**) and `--backup-id` (**ID**) of your full backup.

2. Initiate PITR:

```
s9s backup \
        --restore \
        --cluster-id=2 \
        --backup-id=3 \
        --pitr-stop-time="2025-02-10 23:20:09" \
        --nodes="10.118.0.3:3306" \
        --log
```

   Where:

   - `--cluster-id`: ID for the cluster from the previous step.
   - `--backup-id`: ID for the backup from the previous step.
   - `--pitr-stop-time`: the timestamp in `YYYY-MM-DD HH:MM:SS` format to which you want to roll forward.
   - `--nodes`: specify which node to restore on.
   - `--log`: Watch the logs in real time.

3. Monitor the restore job:

```
s9s job --list --long
```

   Check the job status to confirm when the PITR operation completes.

1.96.3 Verifying the recovery

After the point-in-time recovery job completes:

1. In the ClusterControl UI or CLI logs, ensure there are no errors or warnings.
2. Query your restored database to confirm data is as expected for the specified point in time.
3. Reconnect your application to the restored node/cluster and validate functionality.

In this article, we explore how to configure a dedicated read-only user for integration with the ClusterControl platform through the RPC interface. Establishing a read-only user is essential for secure and efficient access, allowing ClusterControl to retrieve necessary information without risk to data integrity or unauthorized modifications.

1.97 Use cases

Creating a read-only user is useful in the following example use cases:

- Using a dedicated read-only user for ClusterControl prevents interference with other user accounts or privileges that might be required for application functions.
- The read-only user can run non-invasive health checks on the database, like checking replication status, server load, and resource utilization, to provide valuable insights for proactive management.
- The read-only user can be used to programatically integrate with your existing monitoring and alerting tools.
- For environments with strict compliance requirements, such as financial or healthcare databases, having a read-only user enables ClusterControl integration without violating access policies.
- The read-only user can access ClusterControl via GUI, CLI and RPC API and suitable for non-administrative personnel for various reasons such as assessing the monitored infrastructure, analyzing the current state and trend, auditing and reporting.

1.98 Creating a read-only user

Suppose there is a two-node PostgreSQL streaming replication being managed and monitored by a ClusterControl server, 192.168.20.10.

The following commands should be executed on the ClusterControl server.

1. Add the primary IP address of the ClusterControl node inside `/etc/default/cmon`:

```
RPC_BIND_ADDRESSES="192.168.20.10,127.0.0.1"
```

2. Restart `cmon` service to make sure ClusterControl listens on port 9501 for both IP addresses defined above:

```
systemctl restart cmon
```

3. Create a read-only user called “readeruser”:

```
s9s user \
    --create \
    --group=readergroup \
    --create-group \
    --generate-key \
    --new-password=s3cr3tP455 \
    --email-address=reader@somewhere.com \
    --first-name=Reader \
    --last-name=User \
    --batch \
    readeruser
```

> **Note**
>
> Note A new private and public key will be created under `~/.s9s` directory called `readeruser.key` and `readeruser.pub`.

4. List out the user and make sure it is there:

```
$ s9s user --list --long
A ID UNAME                        GROUPS      EMAIL                 ␣
↪REALNAME
- 1 system                        admins      - System User
- 2 nobody                        nobody      - Default User
- 3 admin                         admins      - Default User
- 4 s9s-error-reporter-vagrant    admins      - -
A 5 dba                           admins      - -
- 6 readeruser                    readergroup reader@somewhere.com␣
↪Reader User
```

5. Get the Cmon Directory Tree (CDT) value for your cluster (imagine this as `ls -al` in UNIX). In this example, our cluster name is "PostgreSQL 12", as shown below:

```
$ s9s tree --list --long
MODE        SIZE OWNER                          GROUP  NAME
crwxrwx---+    - system                         admins PostgreSQL 12
srwxrwxrwx     - system                         admins localhost
drwxrwxr--  1, 0 system                         admins groups
urwxr--r--     - admin                          admins admin
urwxr--r--     - dba                            admins dba
urwxr--r--     - nobody                         admins nobody
urwxr--r--     - readeruser                     admins readeruser
urwxr--r--     - s9s-error-reporter-vagrant     admins␣
↪s9s-error-reporter-vagrant
urwxr--r--     - system                         admins system
Total: 22 object(s) in 4 folder(s).
```

6. Assign read permission for readeruser and readergroup for the cluster that you want. Our CDT path is "/PostgreSQL 12" (always start with a "/", similar to UNIX):

```
$ s9s tree --add-acl --acl="group:readergroup:r--" "/PostgreSQL 12"
Acl is added.
$ s9s tree --add-acl --acl="user:readeruser:r--" "/PostgreSQL 12"
Acl is added.
```

> **Note**
>
> Note You could also assign permission to a specific path like for example, `"/PostgreSQL 12/192.168.20.62:5432"`, which allows "readeruser" to read the objects of host 192.168.20.62 under this particular cluster.

The configuration is complete. Next, we need to configure the `s9s` client in the reader's workstation.

1.99 CLI client configuration

The following commands should be executed on the reader's workstation (client).

1. Install ClusterControl CLI. See *Installation*.
2. Create a `.s9s` directory and copy the `readeruser.key` and `readeruser.pub` from the ClusterControl server into it:

```
mkdir ~/.s9s
cd ~/.s9s
scp root@192.168.20.10:~/.s9s/readeruser.key .
scp root@192.168.20.10:~/.s9s/readeruser.pub .
```

3. Create an s9s configuration file at `~/.s9s/s9s.conf`:

```
[global]
cmon_user=readeruser
controller=https://192.168.20.10:9501
```

4. At this point, you should be able to run the following commands:

```
$ s9s cluster --ping
PING Ok 4ms
$ s9s user --whoami
readeruser
```

At this point, the configuration is complete on the client-side. You may run the following commands to list out the cluster's objects (from the reader's workstation):

```
s9s cluster --cluster-id=10 --stat
s9s cluster --cluster-id=10 --stat --print-json     # if you want the␣
↪output to be printed in JSON
```

1.100 Accessing the RPC API

If you would like to access the cluster object via HTTP, send the request to the RPCv2 API accessible on port 9501 (TLS enabled). Use one of the following two methods:

1. Pass the username and password inside a nested `authenticate` method in an operation:

```
curl -k 'https://192.168.20.10:9501/v2/clusters' \
-XPOST -d \
```

(continues on next page)

(continued from previous page)

```
'{"operation": "getAllClusterInfo",
"authenticate":
{"password": "s3cr3tP455","username": "readeruser"}}'
```

2. This is the recommended way and is used by all our applications to connect to the ClusterControl controller. Users need to maintain cookies/sessions after the first authentication and send other requests with the corresponding cookies/session. To obtain a session, use the `authenticateWithPassword` operation and connect to the `v2/auth` endpoint:

```
curl -k 'https://192.168.20.10:9501/v2/auth' \
-XPOST -d \
'{"operation":"authenticateWithPassword",
"user_name":"readeruser",
"password":"s3cr3tP455"}' \
-c cookies.jar
```

Then, include cookies.jar with every subsequent read-only request:

```
curl -k 'https://192.168.20.10:9501/v2/clusters' \
-XPOST -d \
'{"operation":"getallclusterinfo"}' \
-b cookies.jar
```

If the RPC call is successful, you should receive a response with `"request_status": "Ok"`, similar to the following output:

```
{
    "controller_id": "683e2b4b-038f-4cf7-93a0-4c3f95b3db70",
    "request_processed": "2024-11-07T19:33:12.701Z",
    "request_status": "Ok",
    "request_user_id": 5,
    "total": 3,
    "clusters":
    [
        ...json content...
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 'readeruser'."
    ]
}
```

If the user is trying to execute a non-read-only request, for example, sending a stop node job that requires an “execute” permission:

```
curl -k 'https://192.168.20.10:9501/v2/jobs' \
-XPOST -d '{"cluster_id": 21, "job": {"class_name": "CmonJobInstance",
"job_spec":
{"command": "stop",
"job_dat":{"clusterid": 21,
```

(continues on next page)

(continued from previous page)

```
"node":{"hostname": "192.168.20.53"}}}},
"operation": "createJobInstance"}' \
-b cookies.jar
```

The reply should return `"request_status": "AccessDenied"`, similar to the following output:

```
{
    "controller_id": "9eb8b1d1-f9d1-439e-945e-108982812bee",
    "error_string": "Execute access to cluster 21 for user readeruser is␣
↪denied.",
    "request_processed": "2021-05-04T07:01:38.342Z",
    "request_status": "AccessDenied",
    "request_user_id": 7,
    "debug_messages":
    [
        "RPC V2 authenticated user is 'readeruser'."
    ]
}
```

For details on the list of operations supported by ClusterControl, see *ClusterControl RPC API reference manual*.

In this article, we explore the procedure for migrating existing database clusters from the ClusterControl GUI v1 to the new ClusterControl GUI v2. This migration strategy is designed to ensure continuous operational integrity and minimize downtime throughout the transition. This approach is particularly advantageous for organizations and users who prefer a phased adoption strategy, allowing for thorough testing and validation of the new ClusterControl v2 interface and its enhanced functionalities in a controlled environment before a full-scale deployment. By following these guidelines, users can seamlessly transition their critical database infrastructure, leveraging the advancements offered by ClusterControl v2 while maintaining stability and performance.

1.101 Limitations of This Method

Before proceeding, it's important to understand the key limitations:

- All operations that depend on Cluster ID may be impacted due to the change of the IDs.
- You begin with new monitoring data as historical metrics and logs are not migrated.
- Manual tuning may be required for certain settings after migration (e.g., failover thresholds).
- Custom scripts and hooks from GUI v1 require manual recreation, as they will not be imported automatically.
- No rollback. Reverting to GUI v1 after migration and disabling it is not straightforward.

1.102 Pre-Migration Checks

Thorough pre-migration checks are essential for a seamless and successful migration. They mitigate risks, prevent data loss, minimize downtime, and guarantee system integrity, averting complications, delays, increased costs, and disruptions:

- In ClusterControl GUI v1, verify the health and stability of your clusters.

- Review custom alerting, operational reports schedules, and backup schedules.
- Make a list of integrations: email, LDAP, cloud backups, notifications, 3rd party integrations etc.
- Make a list of users who log in to ClusterControl GUI v1.
- Ensure SSH access to all nodes is functional for the user running ClusterControl in your new environment with ClusterControl GUI v2.

1.103 Installing ClusterControl v2

Follow these steps:

1. Prepare a new host or virtual machine.
2. Download and install ClusterControl GUI v2 from the official installer or repository. On the ClusterControl server, run the following commands:

```
wget https://severalnines.com/downloads/cmon/install-cc
chmod +x install-cc
sudo ./install-cc # omit sudo if you run as root
```

3. Configure passwordless SSH access to the database nodes. See *SSH Key-based Authentication*.

1.104 Importing Existing Clusters into ClusterControl v2

The *Import Database Cluster* feature in ClusterControl enables users to bring existing database clusters under the existing ClusterControl v1 management with minimal effort. The following cluster types can be imported from ClusterControl v1 to v2:

- MySQL/MariaDB/Percona Server standalone
- MySQL/MariaDB/Percona Server replication
- MariaDB Galera Cluster
- Percona XtraDB Cluster
- PostgreSQL streaming replication
- TimescaleDB streaming replication
- MongoDB replica set
- MongoDB sharded cluster

The rest of the supported cluster types like Redis/Valkey (Sentinel and Cluster), PostgreSQL logical replication, SQL Server and Elasticsearch are only available on ClusterControl v2.

1.104.1 Prerequisites

Before importing a database cluster, several prerequisites must be met:

1. Passwordless SSH (SSH using key-based authentication) is configured from the new ClusterControl v2 node to all database nodes. See *SSH Key-based Authentication*.
2. The target cluster must not be in a degraded state. For example, if you have a three-node Galera cluster, all nodes must be alive, accessible, and in sync.

Below are examples for each supported database type:

1.104.2 Importing a MySQL/MariaDB standalone or replication

For this to work, ClusterControl expects all instances within a group to use the same MySQL root password. It will also automatically try to identify each server's role (primary, replica, multi, or standalone).

ClusterControl GUI

1. To import an existing MySQL replication cluster, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "MySQL Replication".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster. Once imported, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters.
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl will use to connect to all database nodes.
   - **Server data directory**: The database server data directory path.
   - **Admin/Root user**: The database admin username that ClusterControl will use to connect to all database nodes. This user must be granted global SUPER privilege and the GRANT option for localhost only.
   - **Admin/Root password**: The password for **Admin/Root user**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.

- **information_schema queries**: By default, this is set to off. When enabled, this shall allow cmon or ClusterControl to query from `information_schema` to collect metrics from your MySQL database nodes.
- **Cluster auto-recovery**: By default, this is set to off. If toggled on, the cluster auto-recovery will be enabled allowing for degraded or failing cluster will be attempted to be recovered by ClusterControl.
- **Node auto-recovery**: By default, this is set to off. If toggled on, the node auto-recovery will be enabled allowing for failing nodes such as accidentally terminated system process will be tried to be started to get back to the cluster.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify your existing target database nodes and you are allowed to add more than one node depending on your existing MySQL Replication cluster you need to register to ClusterControl. Since this is an existing database cluster which means it is already setup, ClusterControl will be able to identify and scan the type of topology for your database cluster.
   - **Node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The settings will be kept until you exit the import wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a three-node Oracle MySQL Replication 8.0 cluster, with operating system user "ubuntu". Regardless of the order, ClusterControl will be able to identify the primary node(s) through `read_only` and `super_read_only` variables:

```
s9s cluster --register \
    --cluster-type=mysqlreplication \
    --nodes="10.0.10.20;10.0.10.21;10.0.10.22" \
    --vendor=oracle \
    --provider-version=8.0 \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name='PROD - MySQL Replication 8.0' \
```

(continues on next page)

(continued from previous page)

```
    --wait \
    --log
```

1.104.3 Importing a PostgreSQL/TimescaleDB cluster

For this to work, ClusterControl expects all instances within a group to use the same database administrator password. It will also automatically discover the database role of each node (primary, replica, TimescaleDB extension).

ClusterControl GUI

1. To import an existing PostgreSQL streaming replication, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "PostgreSQL Streaming" for PostgreSQL or "TimescaleDB" for TimescaleDB.
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster.
   - **Tags**: Add tags to search or group your database clusters.
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl will use to connect to all database nodes.
   - **User**: The database admin username that ClusterControl will use to connect to all database nodes. This user must be a global super user.
   - **Password**: The password for **User**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not

setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to import:
   - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
   - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note If you have more than one secondary or replica nodes, make sure to add more under **Replica nodes** text field. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The settings will be kept until you exit the import wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a three-node PostgreSQL 16 streaming replication, with operating system user "ubuntu" (the first node is the primary) with tags `production` and `postgres`. Then wait until the job is done while showing logs during the deployment process:

```
s9s cluster --register \
    --cluster-type=postgresql \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=postgresql \
    --provider-version=16 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --with-tags='production;postgres' \
    --cluster-name='PostgreSQL 16 - Streaming Replication' \
```

```
                --wait --log
    ```
```

### 1.104.4 Importing a MongoDB replica set

For this to work, ClusterControl expects all instances within a group to use the same MongoDB admin user password. It will also automatically discover the database role of each node (mongos, config server, shard server, primary, secondary, arbiter).

#### ClusterControl GUI

1. To import an existing MongoDB replica set, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MongoDB ReplicaSet".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Vendor**: This is under **Vendor and version** label in the panel. This is the kind of vendor that your database cluster binaries for MongoDB ReplicaSet is deployed. This is required and you need to choose from *Percona*, *MongoDB*, and *MongoDB Enterprise*.
   - **Name**: This is optional. This shall be the name of your cluster.
   - **Tags**: Add tags to search or group your database clusters.
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl will use to connect to all database nodes.
   - **User**: The database admin username that ClusterControl will use to connect to all database nodes. This should have global superuser role (preferably root) and allowed for localhost only.
   - **Password**: The password for **User**.
   - **MongoDB Auth DB**: The authentication database to authenticate to.
```

- **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to import:
    - **Node**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The settings will be kept until you exit the import wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a three-node MongoDB Replica Set using Percona binaries, you only need to specify the primary node. For example, if you have primary running on 192.168.40.190 and you have replica running on 192.168.40.191, there is no need to specify the replica as it will be auto-detected and will be imported by ClusterControl as well. The following command demonstrates this using `ubuntu` as its OS username, using `ed25519` digital signature algorihtm and print the logs of the job deployment:

```
      s9s cluster --register \
              --cluster-type=mongodb \
              --nodes="192.168.40.190" \
              --vendor=percona \
              --provider-version='7.0' \
              --os-user=ubuntu \
              --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
              --db-admin='mydbuser' \
              --db-admin-passwd='mydbPassw0rd' \
              --cluster-name='MongoDB Percona Server ReplicaSet 7.0' \
              --wait --log
  ```
```

(continues on next page)
```

(continued from previous page)

```
- To import a three-node MongoDB Enterprise Replica Set 7.0, with operating␣
↪system user "root" and let the deployment job run in the foreground:

    ```bash
    s9s cluster --register \
        --cluster-type=mongodbenterprise \
        --nodes="192.168.40.155" \
        --vendor=mongodbenterprise \
        --provider-version='7.0' \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --db-admin='mydbuser' \
        --db-admin-passwd='mydbPassw0rd' \
        --cluster-name='MongoDB Enterprise ReplicaSet 7.0'
    ```

 - To import a MongoDB Sharded Cluster using Percona binaries consisting of␣
↪3-node router nodes using `ubuntu` as its OS username, using ed25519␣
↪digital signature algorihtm and print the logs of the job deployment:

    ```bash
        s9s cluster --register \
                --cluster-type=mongodb \
                --
↪nodes="mongos://192.168.1.11;mongos://192.168.1.12;mongos://192.168.1.12;"
                --vendor=percona \
                --provider-version='7.0' \
                --os-user=ubuntu \
                --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
                --db-admin='mydbuser' \
                --db-admin-passwd='mydbPassw0rd' \
                --cluster-name='MongoDB Percona Server Shards 7.0' \
                --wait --log
    ```
```

1.105 Importing Configurations from ClusterControl v1

While cluster monitoring is imported during the steps above, the following configurations must be manually migrated or re-created in ClusterControl v2:

1.105.1 Users and Access

- Re-create ClusterControl admin/operator accounts and groups:

ClusterControl GUI

To create a user, go to ***Create user or team → Create user*** and fill up all the following information:

Field	Description
Details	
First name	The first name of the user.
Last name	The last name of the user.
Username	The username of the user. In CC v2, the username shall be used when logging in to the login page, in contrast to CC v1 (old GUI) where you had to use e-mail address and password combination in order to login.
Password	The user's password.
Email	Email address of the user. This should be a valid email address (despite not mandatory), for password reset and receiving alerts via email.
Timezone	Choose a timezone from the dropdown list. This timezone will reflect the date/time presentation of the ClusterControl events and monitoring data points.
Team	
Team	Choose a team from the dropdown. This is considered the primary group for this user. A user can be assigned to multiple groups (teams).

ClusterControl CLI

- List all users:

```
s9s user --list --long
```

Example

```
$ s9s user --list --long
A ID UNAME      GNAME  EMAIL REALNAME
-  1 system     admins -     System User
-  2 nobody     nobody -     -
A  3 dba        users  -     -
-  4 remote_dba users  -     -
```

- Create a user called "john" with administrative privilege:

```
s9s user \
    --create \
    --group=admins \
    --generate-key \
```

(continues on next page)

(continued from previous page)

```
    --new-password=s3cr3tP455 \
    --email-address=john@example.com \
    --first-name=John \
    --last-name=Doe \
    --batch \
    john
```

```
See [Users and teams](../../../admin-guide/users-teams.md) for details.
```

- LDAP/AD integration: Replicate the LDAP configuration. See LDAP for details.

1.105.2 Notification Settings

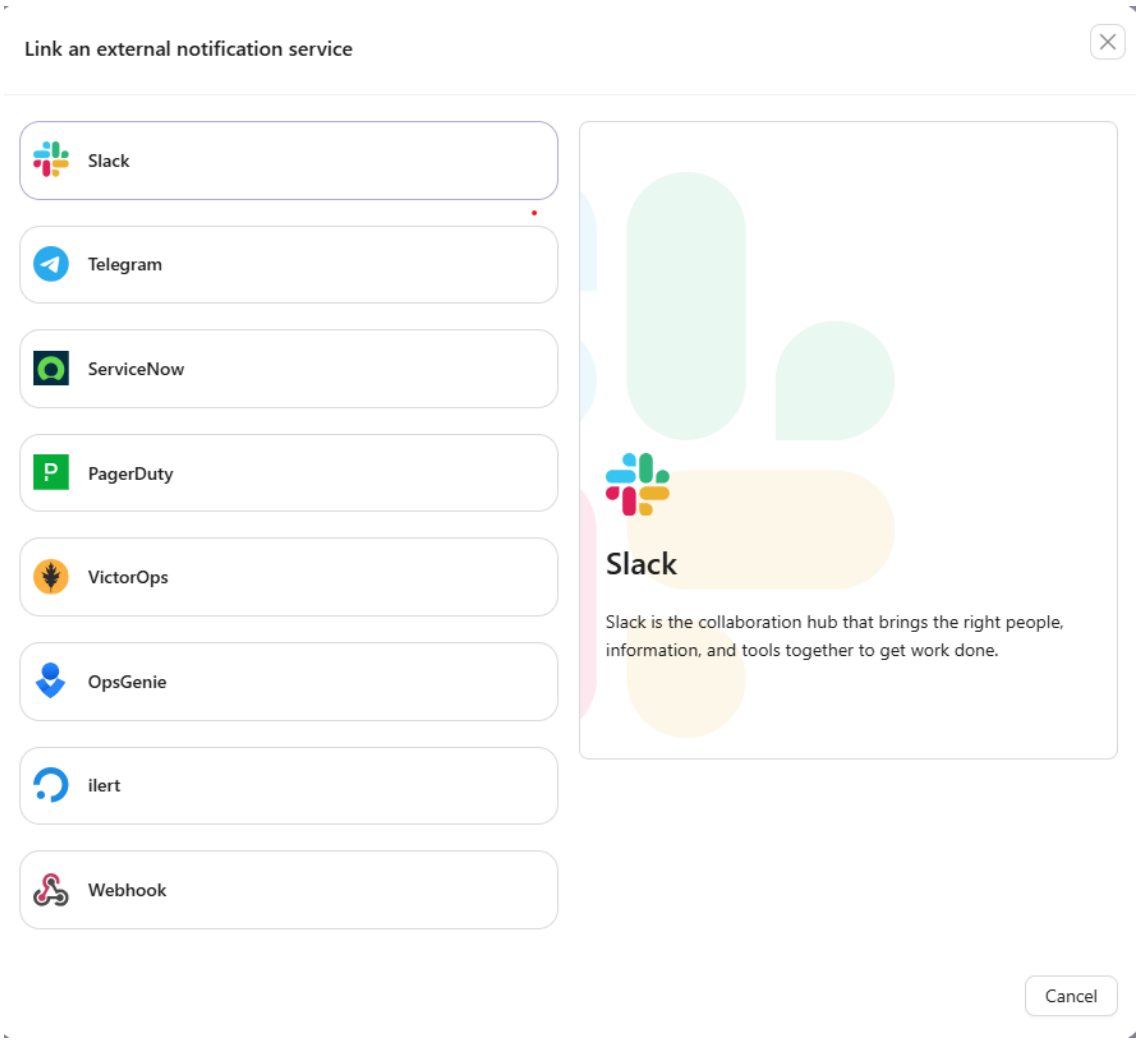
- For email recipients, add them manually, copying from what you have on ClusterControl v1:

ClusterControl GUI

1. To configure an SMTP mail server, go to ***ClusterControl GUI → Settings → Configure now***. It will open a wizard called "Configure mail server". Choose **SMTP Server**.
2. In the "Configure mail server: SMTP" window, configure the following:
   - **Server Address**: The SMTP mail server address that you are going to use to send the email.
   - **Port**: The SMTP port for the mail server. Usually, this value is 25 or 587, depending on your SMTP mail server configuration.
   - **User name**: The SMTP user name. Leave empty if no authentication required.
   - **Password**: The SMTP password for **User name**. Leave empty if no authentication required.
   - **Reply to/from**: Specify the sender of the email. This will appear in the 'From' field of the mail header.
   - **TLS/SSL required**: Check this box if ou want to use TLS/SSL for extra security. The mail server must support TLS/SSL.
3. Check the **Send test email** to test out the SMTP configuration. If you receive an email from ClusterControl, click **Save** and proceed to configure the mail recipients.

```
See [Email␣
↪Notifications](../../../user-guide/management/alerts-notifications.md#email-notificatio
↪for details.
```

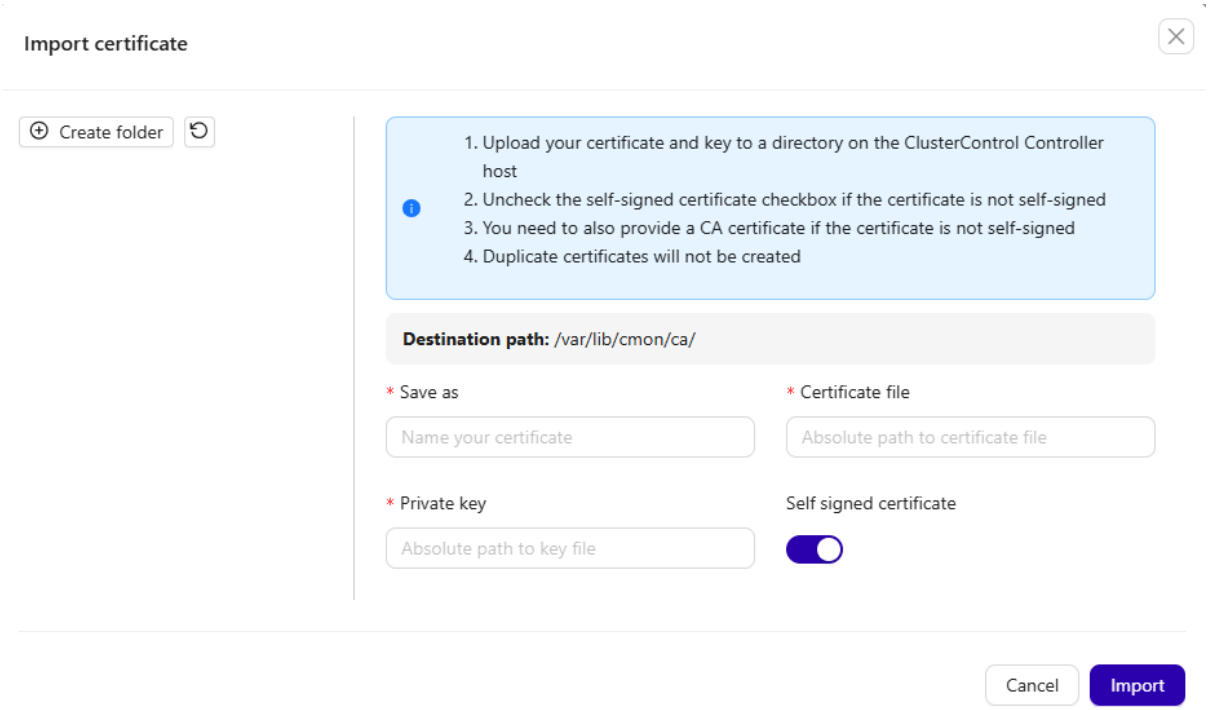
- Third-party integrations (Slack, PagerDuty): Configure again by going to **ClusterControl GUI → Settings → Notification services → Add new integration**. This will give you a dialog box to choose the notification service to use:



See *Notification services* for details.

1.105.3 SSL Certificates

Import SSL keys and certificates into ClusterControl's certificate repository. To access this go to **ClusterControl GUI → Settings → Certificate management → More → Import certificate**:



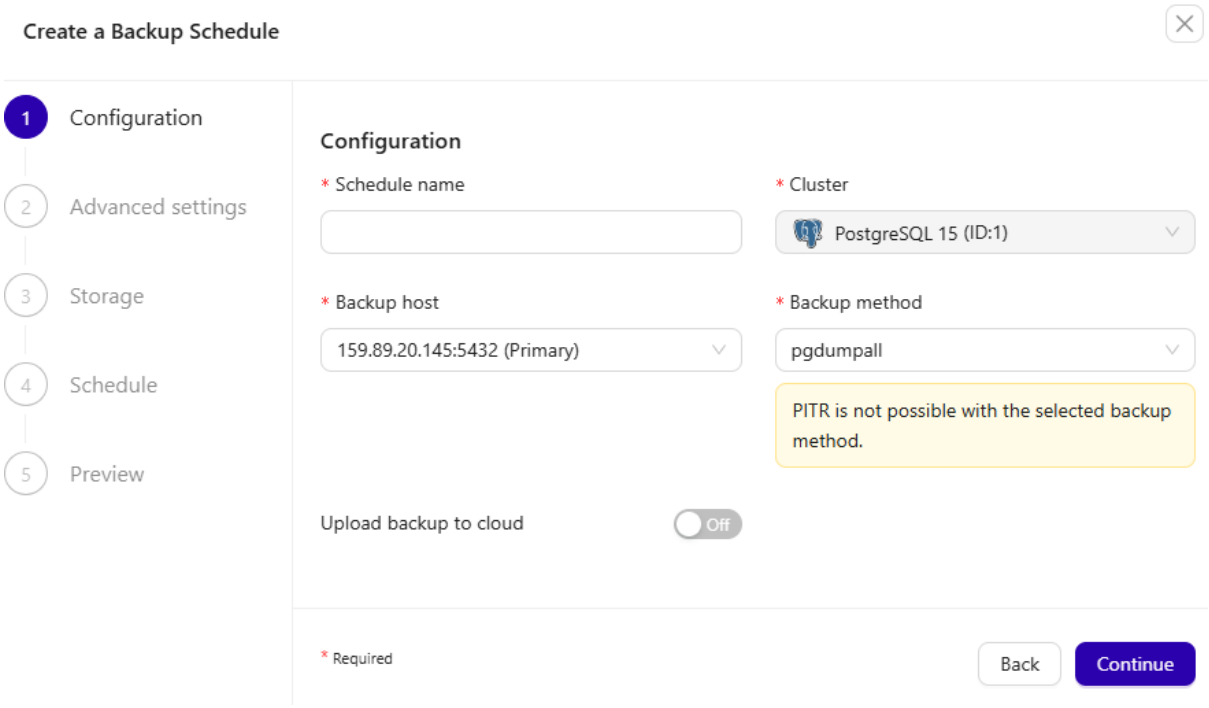
The imported keys and certificates can then be used to enable SSL encryption for server-client connection, replication, or backup at a later stage. Before you perform the important action, bear in mind to:

1. Upload your certificate and key to a directory in the ClusterControl Controller host.
2. Uncheck the **Self-signed Certificate** checkbox if the certificate is not self-signed.
3. You need to also provide a CA certificate if the certificate is not self-signed.
4. Duplicate certificates will not be created.

Field	Description
Destination Path	Where you want the certificate to be imported to. Click on the file explorer window on the left to change the path.
Save As	Certificate name.
Certificate File	Absolute path to the certificate file. For example: `/home/user/ssl/file.crt`.
Private Key File	Absolute path to the key file. For example: `/home/user/ssl/file.key`.
Self-signed Certificate	Uncheck the checkbox if the certificate is not self-signed.
Import	Start the import process.

1.105.4 Backup Schedules

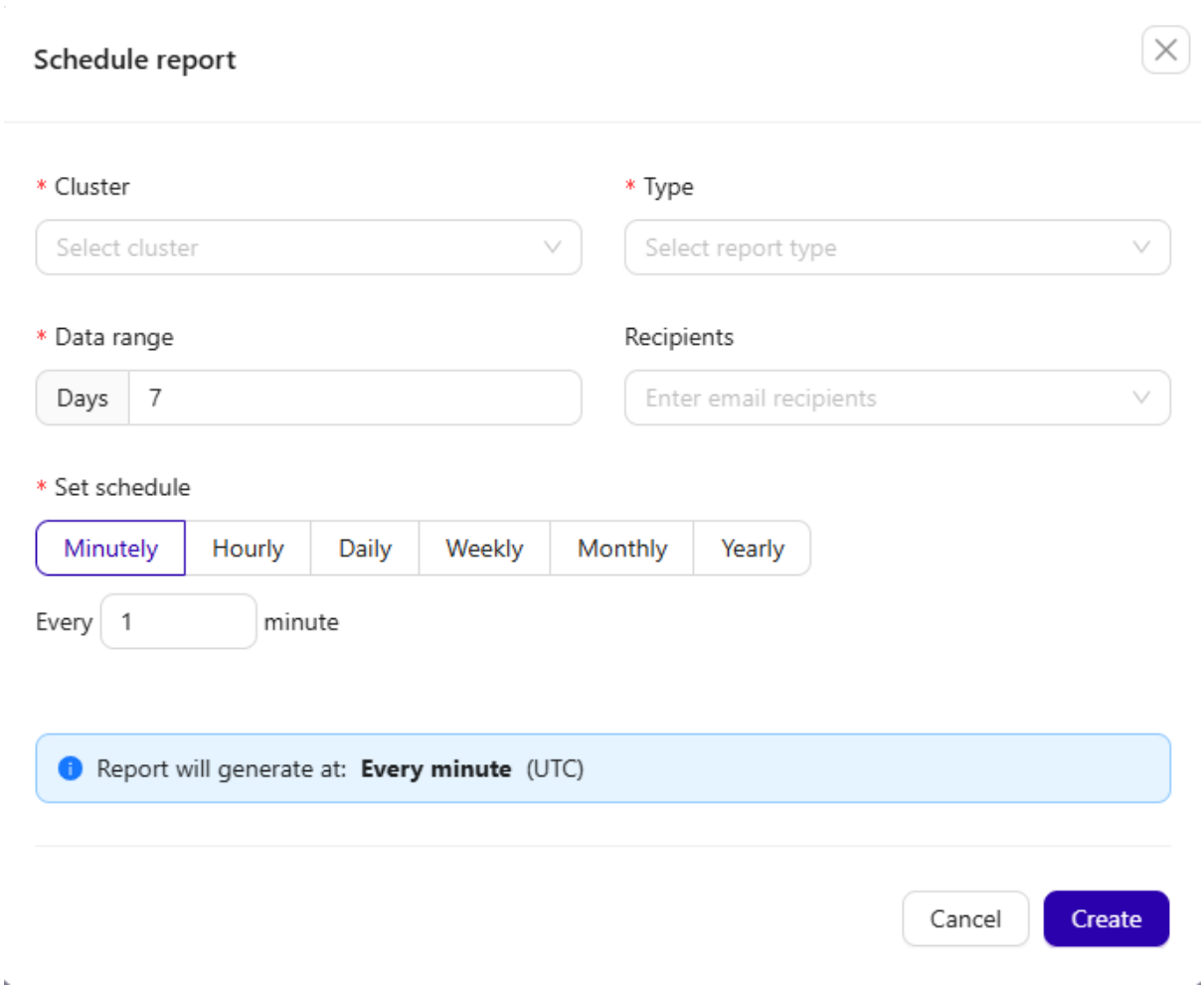
Use the UI or CLI (`s9s backup`) to re-create your backup schedules. To access this go to **ClusterControl GUI → Select cluster → Backups → Create backup → Schedule backup**:



See *Schedule Backup* for details.

1.105.5 Scheduled Operational Reports

Set up Scheduled Operational reports (weekly/monthly) and delivery email addresses as needed:



ClusterControl GUI

1. In the ClusterControl UI, go to ***Sidebar → Operational Reports***.
2. Choose the **Schedules** tab, and create schedule button.
3. Choose the cluster, type of the template reports, time ranges, and fill the email recipients.
4. Set the schedule for delivery of the report.

See *Operational Reports* for details.

1.106 Post-Migration Checks

After migration, it is crucial to perform a series of post-migration checks and optimizations to ensure the new system is operating efficiently and reliably. This comprehensive process typically involves:

1. Cluster functionality
   - Regularly verify monitoring systems are operational, accurately collecting data, and capable of timely anomaly notifications. This includes checking dashboards and alert configurations to ensure all metrics are tracked.
   - In GUI v2, proceed to validate the topology diagrams. This crucial step ensures that the visual representation of your system's architecture accurately reflects its actual configuration and intended design.

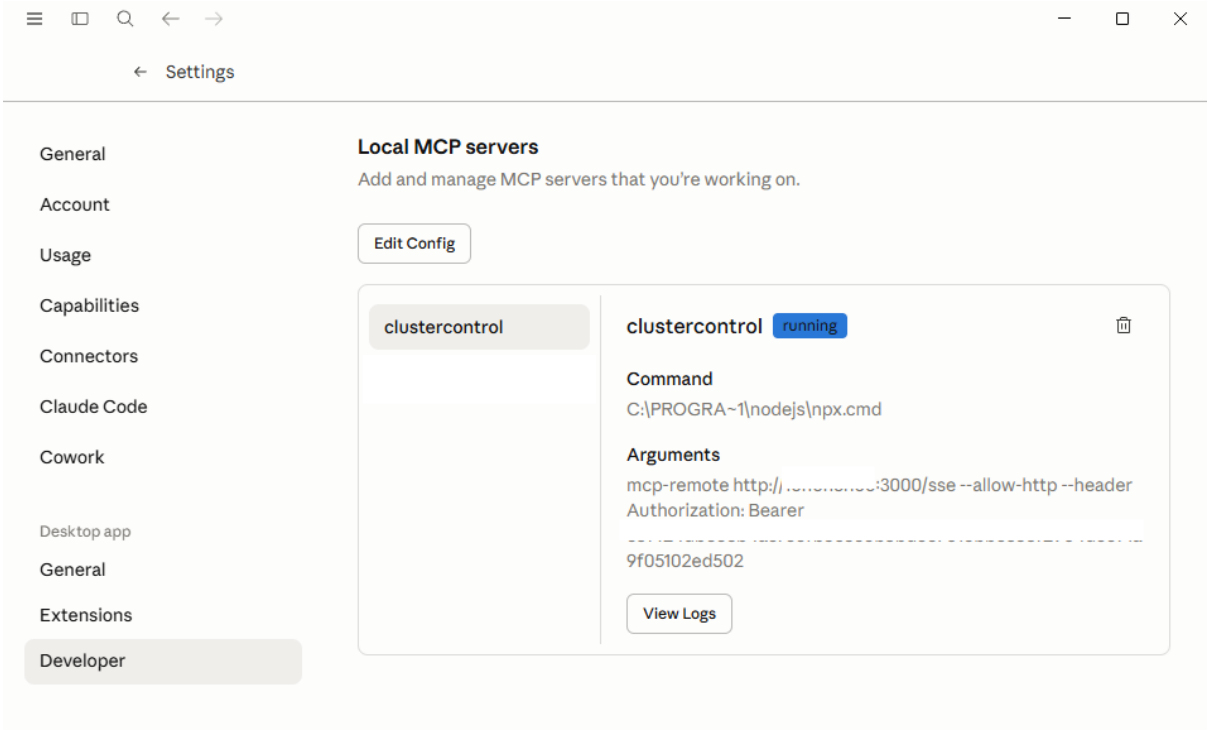
- Simulate a failover to test resilience, observe transition time, and client reconnection. However, before that, ensure data synchronization, cluster health, and application awareness.

2. The Cluster ID will be different from that in GUI v1, requiring immediate updates to all external tools and scripts that rely on it. Failure to update will cause errors.
3. Check alert thresholds, backup schedules, and user privileges:
   - **Alert Thresholds:** Fine-tune CPU, memory, and disk space alerts to ensure effective early warnings without false positives. Verify notification channels and escalation procedures.
   - **Backup Schedules:** Confirm backup frequency/retention, regularly test integrity/restorability, monitor completion, and verify secure offsite storage.
   - **User Privileges:** Conduct access reviews for all the users. Confirming the privileges assigned to users.

In this section, we explore how to manage your database clusters using natural language through the ClusterControl MCP Server. Instead of navigating dashboards or running CLI commands, you simply describe what you want to your AI assistant and it handles the rest.

If you haven't set up the ClusterControl MCP Server yet, please go through the *ClusterControl MCP* documentation first before continuing.

If you have already installed and integrated it with your AI assistant, make sure the integration is working properly before proceeding. For example, if you are using Claude Desktop, go to **Settings → Developer** and confirm that the ClusterControl MCP is listed and running. See screenshot below



Once you can see the MCP Server is running, you are ready to start managing your clusters. The examples below cover the most common operations you can perform through your AI assistant.

Any action that changes something like creating a job, adding a user, scheduling maintenance, restoring a backup, setting node or controller config, etc. Works in two steps:

- Dry run (default): AI Assistants describes exactly what it would do, without touching your CMON controller.

- Execute: After you confirm, AI Assistants calls the tool again with dry_run=false to carry out the action.

High-risk operations (node restart, failover, drop cluster, drop user, delete backup schedule, restore backup, delete ClusterControl account) include an additional warning in the dry-run output so nothing destructive ever runs without your explicit approval.

1.107 User and Access Management

Manage database accounts, review privileges, create or revoke access across your clusters.

1.107.1 List Functions

Below are the functions that currently supported by clustercontrol-mcp for User and Access Management.

Function	Purpose
`list_db_users`	List all database accounts on a cluster (Required Cluster ID)
`create_db_user`	Create a new database user account (Required Cluster ID)
`drop_db_user`	Delete a database user account (Required Cluster ID)
`grant_db_user`	Grant privileges to a database user (Required Cluster ID)
`revoke_db_user`	Revoke specific privileges from a database user (Required Cluster ID)
`revoke_all_db_user`	Revoke all privileges from a database user (Required Cluster ID)
`list_cmon_users`	List all ClusterControl controller user accounts (Required Cluster ID)
`list_cmon_groups`	List all ClusterControl controller user groups (Required Cluster ID)
`create_cmon_user`	Create a new ClusterControl controller user account (Required Cluster ID)
`set_cmon_user`	Modify a ClusterControl controller user profile (Required Cluster ID)
`drop_cmon_user`	Delete a ClusterControl controller user account (Required Cluster ID)

1.107.2 Example Prompt

Here are some examples of using clustercontrol-mcp for User and Access Management:

- List existing users and groups using prompt

```
> List all users and groups that exists in clustercontrol and mask the␣
↪email

<br>
![ClusterControl MCP␣
↪Response](../../../images/clustercontrol-mcp-usman-cmon-1.png)
<br>
```

- Create ClusterControl user and attach to group using prompt

```
> Create a Clustercontrol user with username support and email␣
↪support@severalnines.com using password severalNINES999 in group␣
↪admins

<br>
![ClusterControl MCP␣
```

(continues on next page)

(continued from previous page)

```
↪Response](../../../images/clustercontrol-mcp-usman-cmon-2.png)
<br>
```

1.108 Cluster Inventory and Health

Get a complete picture of your clusters, nodes, topology, and active alarms.

1.108.1 List Functions

Function	Purpose
list_cluster	Full fleet overview with cluster types, states, and versions
get_cluster	Detailed view of a specific cluster including all hosts and their roles (Required Cluster ID)
get_cluster_	Normalized topology view showing nodes and replication links (Required Cluster ID)
list_nodes	Flattened node inventory across all clusters or one cluster (Optional Cluster ID)
list_alarms	Active alarms for a specific cluster (Required Cluster ID)
list_all_ala	Active alarms across the entire fleet
get_controll	CMON version, RPC version, and connectivity status
get_cluster_	Contents of one collected log file, addressed by (cluster_id, hostname, filename). Tail of last 200 lines by default; optional output_path writes the full file locally (Required Cluster ID)
get_cmon_log	CMON daemon log entries for a cluster. New optional filters: severity, component, hostname, since
get_cmon_log	Summary counts of CMON log entries by severity and component
list_cluster	Inventory of log files CMON has collected from cluster hosts (mysqld.log, postgresql-*.log, prometheus.log, agent_internal_logs.log, cmnd.log)

1.108.2 Example Prompt

Here are some examples of using clustercontrol-mcp for Cluster Inventory and Health:

- Get information about Cluster databases

```
> List all database clusters

> Get details for cluster `cluster_id` or `cluster_name`

> Get the logs for cluster `cluster_id` or `cluster_name`

> List active alarms for cluster `cluster_id` or `cluster_name`

<br>
![ClusterControl MCP␣
↪Response](../../../images/clustercontrol-mcp-cluster-1.png)
<br>
<br>
```

(continues on next page)

(continued from previous page)

```
![ClusterControl MCP␣
↪Response](../../../images/clustercontrol-mcp-cluster-2.png)
<br>
```

1.109 Performance Analysis

Identify slow queries, long-running transactions, deadlocks, and other performance bottlenecks across your database fleet.

1.109.1 List Functions

Function	Purpose
`get_cluster_stats`	Time-series CPU, memory, disk, network, SQL, and DB stats (Required Cluster ID)
`get_top_queries`	Top resource-consuming query digests (Required Cluster ID)
`get_sql_processes`	Live process list across all nodes (Required Cluster ID)
`kill_db_process`	Terminate runaway queries (Required Cluster ID)
`get_db_growth`	Historical database size trends for capacity planning (Required Cluster ID)
`check_no_primary_keys`	Find tables missing a primary key (Required Cluster ID)
`check_redundant_index`	Find redundant indexes (Required Cluster ID)
`check_myisam_tables`	Find tables using the MyISAM storage engine (Required Cluster ID)
`get_tx_deadlocks`	Transaction deadlock analysis with blocking SQL

1.109.2 Example Prompt

Here are some examples of using clustercontrol-mcp for Performance Analysis:

- Show current processlist from the cluster.

```
> Show currently running SQL queries on cluster mariasn (mariadb)

<br>
![ClusterControl MCP␣
↪Response](../../../images/clustercontrol-mcp-job-1.png)
<br>
```

- Kill the running job

```
> Kill the query with pid #14927

<br>
![ClusterControl MCP␣
↪Response](../../../images/clustercontrol-mcp-job-2.png)
<br>
```

1.110 Jobs and Operations

Monitor running and completed jobs, track task progress, and review operational history across your clusters.

1.110.1 List Functions

Function	Purpose
`list_jobs`	Recent jobs with optional status and command filters (Required Cluster ID)
`get_job_lo`	Full log output for a specific job (Required Cluster ID and Job id)
`create_job`	Trigger backup, restart, failover, and other cluster operations (Required Cluster ID and command string)
`kill_job`	Terminate stuck jobs (Required Cluster ID)

1.110.2 Example Prompt

Here are some examples of using clustercontrol-mcp for Jobs and Operations:

> List recent jobs on cluster `cluster_id` or `cluster_name`
>
> Show log for job `job_id` on cluster `cluster_id` or `cluster_name`

1.111 Backups and Scheduling

Manage backup jobs, review backup history, create schedules, and restore from existing backups.

1.111.1 List Functions

Function	Purpose
`list_backups`	Recent backups with status and method filters (Required Cluster ID)
`get_latest_backup`	Most recent backup, optionally only successful ones (Required Cluster ID)
`list_backup_schedules`	View all configured backup schedules (Required Cluster ID)
`get_backup_schedule_summa`	Summary of backup schedules for a cluster (Required Cluster ID)
`create_backup_schedule`	Create a new recurring backup schedule (Required Cluster ID)
`update_backup_schedule`	Update an existing backup schedule (Required Cluster ID)
`delete_backup_schedule`	Delete a backup schedule (Required Cluster ID)
`delete_backup`	Delete a backup record (Required Cluster ID)
`restore_backup`	Restore from an existing backup (Required Cluster ID)

1.111.2 Example Prompt

Here are some examples of using clustercontrol-mcp for Backups and Scheduling:

> When was the last successful backup on cluster `cluster_id` or `cluster_name`?
>
> Create a MariaDB backup schedule on cluster `cluster_id` or `cluster_name` every day at 4:25 AM using the mariabackup(full) backup method.

!!! Tips

```
To successfully create a backup schedule, please mention the exact backup␣
↪method name that you want to use, for example mariabackup(full) for a␣
↪full backup. If you only mention use mariabackup method, the backup␣
↪schedule job will be created but it will fail because the backup method␣
↪is not recognized by ClusterControl.
```

1.112 ProxySQL

Inspect and manage ProxySQL instances including backend servers, query routing rules, and traffic statistics.

1.112.1 List Functions

Function	Purpose
`list_proxysql_top_querie`	Top queries by execution time through the proxy (Required Cluster ID)
`list_proxysql_rules`	Query routing rules (Required Cluster ID)
`list_proxysql_servers`	Backend server health and hostgroup configuration (Required Cluster ID)

1.112.2 Example Prompt

Here are some examples of using clustercontrol-mcp for ProxySQL :

> List ProxySQL backend servers on cluster `cluster_id` or `cluster_name`
>
> List ProxySQL query routing rules on cluster `cluster_id` or `cluster_name`
>
> Show top queries through ProxySQL on cluster `cluster_id` or `cluster_name`

1.113 Configuration

View and modify configuration parameters at the cluster, node, and controller level.

1.113.1 List Functions

Function	Purpose
`get_cluster_confi`	View cluster-level database settings (Required Cluster ID)
`get_node_config`	View node-level database settings (Required Cluster ID and Node)
`set_node_config`	Modify a node configuration parameter (with dry-run safety) (Required Cluster ID and Node)
`un-set_node_config`	Reset a node configuration parameter to default (with dry-run safety) (Required Cluster ID and Node)
`get_controller_co`	View CMON controller runtime parameters (Required Cluster ID)
`set_controller_co`	Modify CMON controller runtime parameters (with dry-run safety) (Required Cluster ID)

1.113.2 Example Prompt

Here are some examples of using clustercontrol-mcp for Configuration :

> Get configuration for node [node] on cluster `cluster_id` or `cluster_name`
>
> Set `max_connections` to `150` on node [node] in cluster `cluster_id` or `cluster_name`
>
> Reset `max_connections` to default on node [node] in cluster `cluster_id` or `cluster_name`

1.114 Maintenance, Audit, and Other Tools

Schedule maintenance windows, review audit logs, generate error reports, and perform other administrative tasks.

1.114.1 List Functions

Function	Purpose
`list_maintenance`	List maintenance windows for a cluster (Required Cluster ID)
`list_all_maintenance`	List maintenance windows across all clusters (Required Cluster ID)
`add_maintenance_window`	Schedule a maintenance window for a host (Required Cluster ID)
`remove_maintenance_window`	Remove a maintenance window by UUID (Required Cluster ID)
`remove_maintenance_windows_by`	Bulk remove maintenance windows by cluster or hostname (Required Cluster ID)
`list_audit_log`	Who did what, when, and from where (Required Cluster ID)
`acknowledge_alarm`	Acknowledge a single alarm (Required Cluster ID)
`acknowledge_alarms_by_filter`	Bulk acknowledge alarms by cluster or filter (Required Cluster ID)
`list_schemas`	List all database schemas on a cluster (Required Cluster ID)
`create_schema`	Create a new database schema (Required Cluster ID)
`list_publications`	PostgreSQL logical replication publications (Required Cluster ID)
`list_subscriptions`	PostgreSQL logical replication subscriptions (Required Cluster ID)
`list_error_reports`	List generated error reports for a cluster (Required Cluster ID)
`generate_error_report`	Generate a new error report (Required Cluster ID)
`download_error_report`	Download a generated error report archive (Required Cluster ID)
`unpack_error_report`	Extract a downloaded error report archive (Required Cluster ID)

1.114.2 Example Prompt

Here are some examples of using clustercontrol-mcp for Maintenance, Audit, and Other Tools :

> Show audit logs across all clusters

List error reports for cluster `cluster_id` or `cluster_name`

Generate an error report for cluster `cluster_id` or `cluster_name`

!!! Tips

```
The `download_error_report` tool will download the error report to the host␣
↪where `clustercontrol-mcp` is installed. When using the AI Assistant, you␣
↪are not restricted to English, feel free to use your local language. You␣
↪can also create your own custom prompts tailored to your specific needs,␣
↪and integrate ClusterControl MCP with any automation tools you already␣
↪have.
```

Warning

Attention

Please use ClusterControl MCP with AI Assistants wisely and be cautious about the information you share with them. Avoid sharing sensitive or confidential data such as API keys, or any personal information. ClusterControl MCP is designed to simplify the management of clusters maintained by ClusterControl, with a scope limited to ClusterControl-related operations only.

CHAPTER

TWO

USER GUIDE

2.1 ClusterControl User Guide

The ClusterControl User Guide is your complete resource for mastering the platform, from initial setup to advanced database management. It is designed to provide both new and experienced users with a comprehensive understanding of ClusterControl's core features and functionalities.

This detailed guide will teach you how to:

- Navigate the ClusterControl user interface.
- Efficiently deploy, monitor, and manage your database clusters.
- Execute crucial routine operations, including backups, restores, and upgrades.
- Monitor performance metrics, configure alerts, and optimize your database environments.
- Utilize automation features to minimize manual tasks and ensure high availability.

Focused on streamlining database operations, the ClusterControl User Guide offers practical instructions, helpful tips, and best practices. Each section is structured to help you gain confidence and proficiency in managing your database infrastructure with ClusterControl.

For step-by-step examples and deployment walkthroughs, we also recommend visiting the *Tutorials* section.

2.2 Overview

ClusterControl provides several user interfaces to interact with the ClusterControl Controller service:

Interface	Description	Package name
ClusterControl GUI v1 (deprecated)	The legacy graphical user interface runs as a web application, also known as ClusterControl GUI v1. This is deprecated since ClusterControl 1.9.8 and is no longer supported.	`clustercontrol`
ClusterControl GUI	A modern web user interface to visualize and manage the cluster. It also supports for centralized management of multiple ClusterControl Controller instances, also known as ClusterControl Operations Center (Ops-C). It interacts with the ClusterControl controller via the `cmon-proxy` service and remote procedure call (RPC).	`clustercontrol-mcc`
ClusterControl CLI	The command-line user interface, also known as `s9s`.	`s9s-tools`
ClusterControl RPC API	A programmatic remote procedure call (RPC) interface exposed by the ClusterControl Controller, used by the GUI, CLI, and external integrations to automate and integrate cluster management operations.	`clustercontrol-contro`
Terraform provider for ClusterControl	A Terraform provider to manage database clusters through Infrastructure as Code (IaC).	`terraform-provider-cl`

This user guide provides steps to perform management, monitoring and automation actions via ClusterControl GUI, ClusterControl CLI and Terraform Provider for ClusterControl (where applicable).

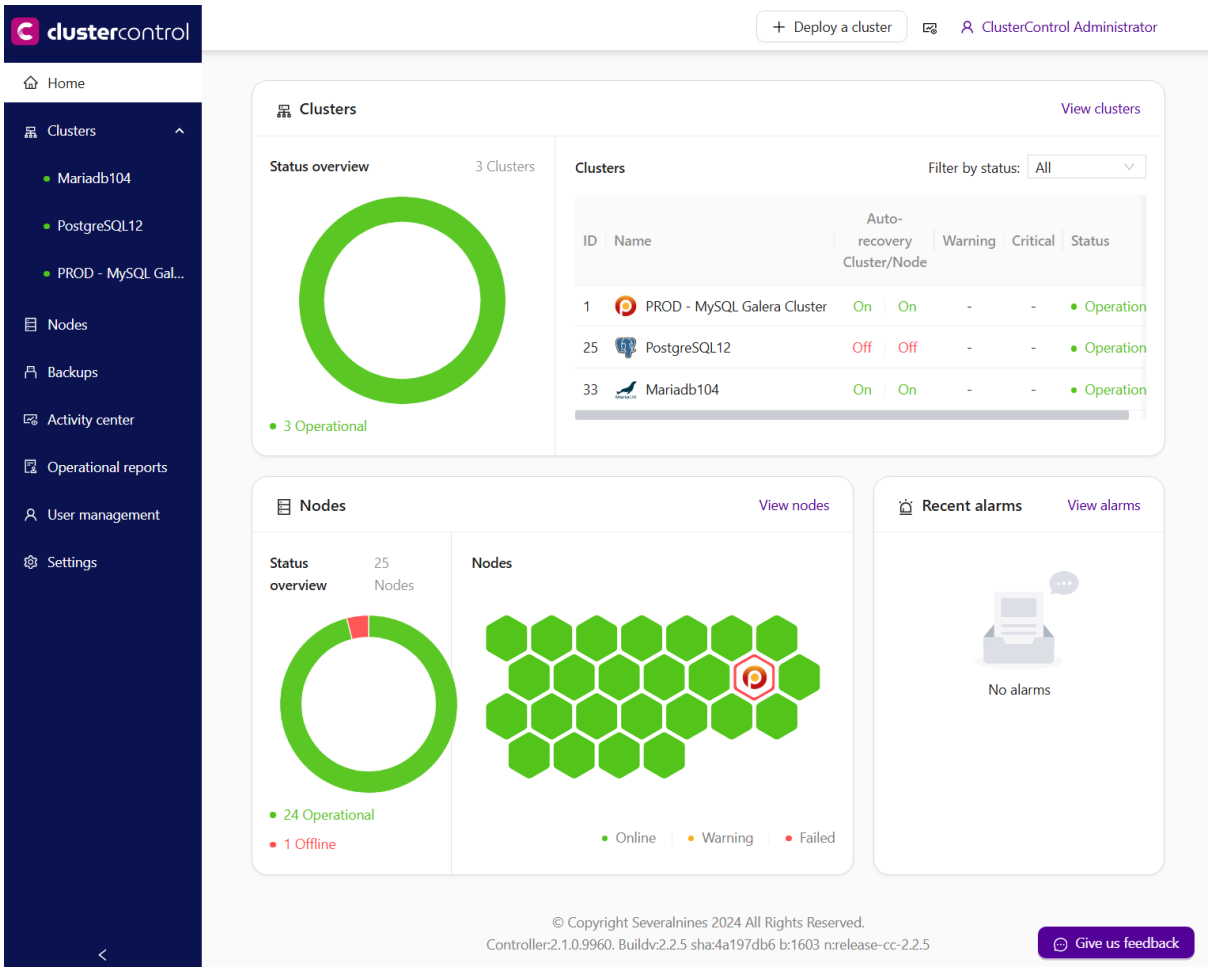
2.2.1 ClusterControl GUI

Warning

Attention

Starting from ClusterControl 1.9.7 (September 2023), ClusterControl GUI is the default frontend graphical user interface for ClusterControl. For new installations, the installer script will only install GUI, skipping GUI v1. If you would like to run GUI v1, see *Legacy GUI Installation*. If you are upgrading from an older version, the GUI v1 will remain available and usable as usual.

The ClusterControl GUI is the graphical user interface to manage and monitor your database clusters. Installation using the installer script will include ClusterControl GUI installation as well, in a package called `clustercontrol-mcc`. It is accessible via `https://{ClusterControl_host}/` and interacts with the ClusterControl Controller (CMON) via remote procedure calls (RPC) on ports 9500 and 9501 for TLS (default).



To use ClusterControl GUI to manage your cluster, see *User Guide → Overview → ClusterControl GUI*. To learn more about ClusterControl GUI, see *Reference Manuals → Components → ClusterControl GUI*.

2.2.2 ClusterControl CLI

Also known as `s9s`, ClusterControl CLI is a command-line tool to interact, control, and manage database clusters using the ClusterControl Database Platform. Installation using the installer script will include ClusterControl CLI installation as well, in a package called `s9s-tools`. You can also install it on another host or workstation to manage the database cluster remotely. The commands are basically JSON messages

being sent over to the ClusterControl Controller (CMON) RPC interface. Communication between the s9s (the command-line tool) and the controller process (CMON) is encrypted using TLS and requires port 9501 to be opened on the controller to be reached by the client host.

The following screenshot shows an example of ClusterControl CLI output from the terminal:

```
[root@docs-cc clustercontrol]# s9s node --list --long --cluster-id=1
STAT VERSION     CID CLUSTER                 HOST           PORT COMMENT
coC- 2.2.0.10416   1 PROD - MySQL Galera Cluster 164.90.192.155 9500 Up and running.
Po-- 2.29.2        1 PROD - MySQL Galera Cluster 164.90.192.155 9090 Process 'prometheus' is running.
yo-- 2.6.5         1 PROD - MySQL Galera Cluster 192.168.99.13  6032 Process 'proxysql' is running.
koM- 2.2           1 PROD - MySQL Galera Cluster 192.168.99.13   112 Process 'keepalived' is running.
slV- 8.0.36        1 PROD - MySQL Galera Cluster 192.168.99.15  3306 Server is unreachable (read-only).
goS- 8.0.36        1 PROD - MySQL Galera Cluster 192.168.99.3   3306 Up and running (read-write).
Ao-- 1.0.0         1 PROD - MySQL Galera Cluster 192.168.99.3   4433 Process 'cmnd' is running.
goM- 8.0.36        1 PROD - MySQL Galera Cluster 192.168.99.4   3306 Up and running (read-write).
Ao-- 1.0.0         1 PROD - MySQL Galera Cluster 192.168.99.4   4433 Process 'cmnd' is running.
goM- 8.0.36        1 PROD - MySQL Galera Cluster 192.168.99.5   3306 Up and running (read-write).
Ao-- 1.0.0         1 PROD - MySQL Galera Cluster 192.168.99.5   4433 Process 'cmnd' is running.
yo-- 2.4.4         1 PROD - MySQL Galera Cluster 192.168.99.6   6032 Process 'proxysql' is running.
koM- 2.2           1 PROD - MySQL Galera Cluster 192.168.99.6    112 Process 'keepalived' is running.
ho-- 2.4.22        1 PROD - MySQL Galera Cluster 192.168.99.7   9600 Process 'haproxy' is running.
Total: 14
```

Not all functionalities are available on both user interfaces. For instance, there is no practicality for the command-line client (ClusterControl CLI) to have all the advanced monitoring features offered by the web application client (ClusterControl GUI). The command-line client is focused on automation and adoption in management, deployment, and scaling operations while the web UI client focuses more on structural visualization with a guided approach. Occasionally, new management features will be introduced in the CLI before being incorporated into ClusterControl UI to turn it into a polished, finalized feature.

Installation using the installer script will include ClusterControl CLI installation as well on the ClusterControl server. The default "admin" user will be created automatically and is configured only for the root user.

To begin managing your clusters using the ClusterControl CLI, see *User Guide → Overview → ClusterControl CLI*. For a comprehensive list of all supported commands, options, and examples, see *Reference Manuals → Components → ClusterControl CLI*.

2.2.3 ClusterControl RPC API

All ClusterControl user interfaces (GUI, CLI, and Terraform provider) communicate with the ClusterControl Controller through a Remote Procedure Call (RPC) API. The RPC layer acts as a single, authoritative interface for executing management operations such as provisioning, monitoring, backups, failover, and upgrades.

The ClusterControl GUI (Ops-C) uses the `cmon-proxy` service to issue RPC calls, while the CLI and Terraform provider invoke the same underlying RPC methods programmatically. The Controller (cmon) executes these requests and applies changes to the managed database clusters. From an operational standpoint, this architecture centralizes control logic, simplifies auditing and automation, and makes it possible to integrate ClusterControl cleanly into GitOps, CI/CD, and Infrastructure-as-Code workflows.

To begin managing your clusters using the ClusterControl RPC API, see *User Guide → Overview → ClusterControl RPC API*. For a comprehensive list of all supported remote procedure calls and endpoints, see *Reference Manuals → API/SDK → ClusterControl RPC API*.

2.2.4 Terraform provider for ClusterControl

The Terraform Provider for Severalnines ClusterControl offers a seamless integration between the powerful infrastructure management tool, Terraform, and the robust database management capabilities of ClusterControl. This provider empowers users to effortlessly provision, configure, and manage database clusters

through Infrastructure as Code (IaC), streamlining the deployment process and enhancing the efficiency of managing complex database environments.

The following is an example of the Terraform resource definition for a PostgreSQL streaming replication deployment:

```
resource "clustercontrol_db_cluster" "test-postgresql" {

    db_cluster_create = true
    db_cluster_name = "PostgreSQL Cluster"
    db_cluster_type = "pg-replication"
    db_version = "15"
    db_vendor = "postgresql"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "test-primary"
    }
    db_host {
        hostname = "test-replica"
    }
    ssh_key_file = "/root/.ssh/id_rsa"
    ssh_user = "root"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

This project is hosted in Github, terraform-provider-clustercontrol. To learn more about this provider, see *Reference Manuals → Tools → Terraform provider for ClusterConttrol*.

2.3 ClusterControl GUI

The ClusterControl GUI is a web-based interface that simplifies the deployment, monitoring, and management of database clusters. Developed by Severalnines, it offers a unified console for common database technologies such as MySQL, MariaDB, PostgreSQL, and MongoDB. Instead of juggling multiple command-line tools or separate dashboards, database administrators can perform end-to-end cluster lifecycle tasks from a single, intuitive environment.

With the GUI, you can:

- Provision new clusters through step-by-step wizards
- Monitor health and performance via real-time metrics, customizable dashboards, and alerts
- Automate backups and upgrades to maintain high availability
- Scale out by adding nodes and load balancers through a few clicks
- Integrate security best practices with user management and audit logs

Whether you're managing a single small cluster or multiple enterprise-grade deployments, the ClusterControl GUI provides the automation, observability, and orchestration needed to keep your databases running optimally — all from one central location.

2.3.1 Accessing the GUI

After ClusterControl is successfully installed, you may access the ClusterControl GUI at `https://<ClusterControl_IP_address_or_hostname>/` using a web browser.

If ClusterControl is installed using the *Helm chart*, you need to connect to the FQDN value defined in the `values.yaml` (or by using the `--set` parameter). For example, in the following ClusterControl installation:

```
helm install clustercontrol s9s/clustercontrol \
  --set fqdn=clustercontrol.example.com
```

After the deployment succeeds, you can access the ClusterControl GUI by going to `https://clustercontrol.example.com`, provided this FQDN is resolvable to Kubernetes load balancer EXTERNAL-IP value of `clustercontrol-ingress-nginx-controller` service. If `fqdn` flag is undefined, it defaults to `localhost`. Hence, you have to do DNS mapping of host `localhost` to resolve to the Kubernetes load balancer IP address and access via browser using `https://localhost`.

Warning

Attention

For Helm chart deployment, the ClusterControl GUI must be accessed via domain name, hostname or FQDN (configurable by `--set fqdn=<domain_name/fqdn>`). Accessing directly using load balancer IP address or Kubernetes `port-forward` option will result with a "page not found" error, since the GUI access is proxied by an ingress controller service and not a web server.

2.3.2 User registration page

For a new installation, you will be redirected to the user registration page. This is where you set up the first admin user, with a valid email address to receive alerts via email. In the next section, provide personal information and consent with the product's terms and condition and privacy policy. Click **Complete** to complete the user registration.

Once registered, you will be automatically logged in as the newly created admin user into ClusterControl.

2.3.3 Login page

If there is no active user session (or the session has expired), ClusterControl will default on viewing the login page. Enter a valid username and password to login.

If this is a new installation, you will be redirected to the user registration page instead. See *User registration page*.

Forgot password

If you forget your password, you can use the ClusterControl CLI to reset the user's password. This action must be performed by any user under the ClusterControl admin group.

Connect to the ClusterControl server via SSH, and run the following command:

```
s9s user --change-password --new-password=<new_password> <username>
```

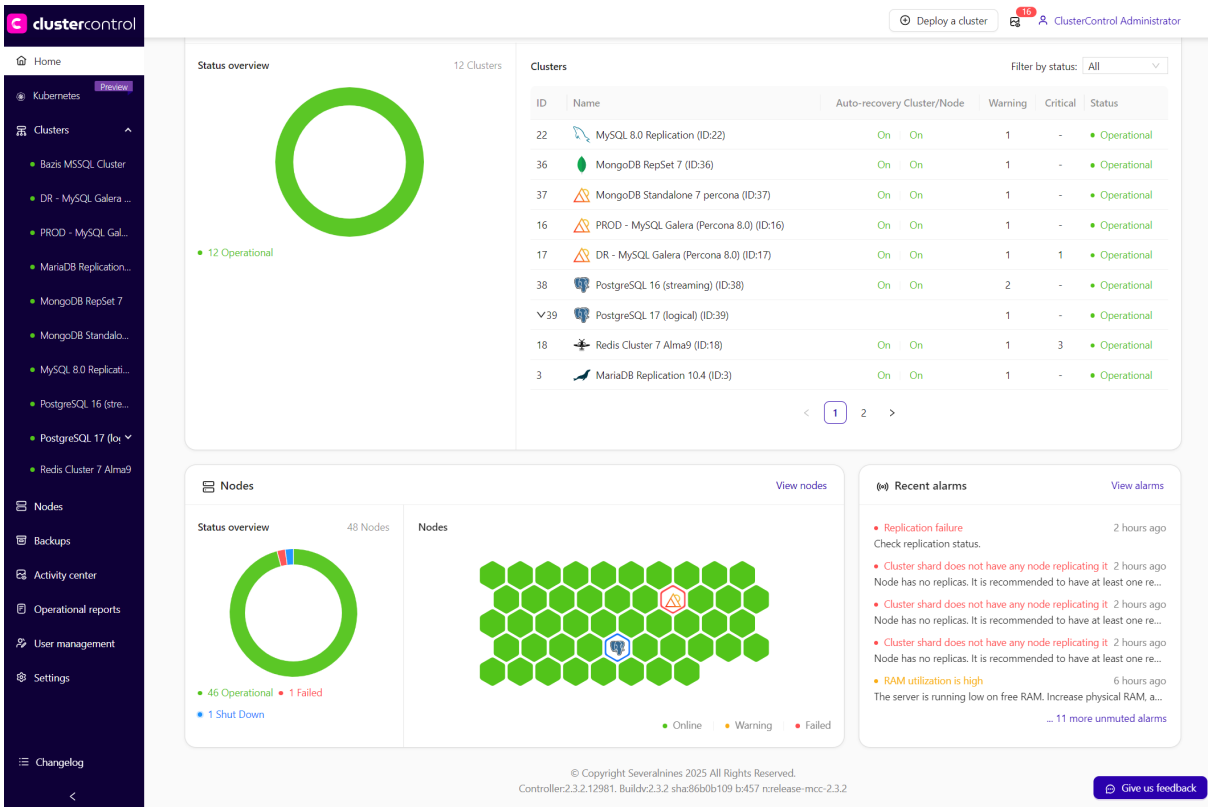
Example output

```
$ s9s user --change-password --new-password='K7hY24$2WeC#' dbadmin
Ok.
```

Once done, you may login again using the new password set above.

2.3.4 Home

This is the default start-up page after a user has successfully logged in. It presents a summary of database clusters and nodes managed by ClusterControl, bounded by the user's group permission. In the main panel, you will see 3 main sections for clusters, nodes and alarms.



In the **Home** page, here are its key sections:

Home

Clusters Status (top-left card)

- The donut chart summarizes the clusters' state.
- A compact table listing clusters by ID, name, and status:
  - Auto-recovery: Whether ClusterControl can automatically recover the cluster or node if a failure occurs. See *Failover and Recovery*.
  - Warnings/Critical: Number of issues requiring attention.
  - Status: Cluster status.

Nodes Status (bottom-left card)

- The donut chart shows a summary of total nodes and how many are online or have warnings/failures.
- A honeycomb grid representing each node's current state (green for online, red for failed and yellow for warning).
- Rollover on a honeycomb cell will pop up a card displaying more information about the node.

Recent Alarms (right card)

- Lists active alerts or alarms.
- If everything is running smoothly, you might see "No alarms".

Each card often has a **View..** or **Details..** link, taking you to a more detailed page. For example, **View clusters**, **View nodes**, or **View alarms**.

!!! Tips

```
* Hover and Click: Many charts or icons are interactive - hover to see node␣
↪details, click to drill down.
* Filtering and Sorting: In lists (e.g., the Clusters list), you can often␣
↪filter by status or sort by name/ID.
* Always Check the Events/Alarms: They're a quick way to spot issues before␣
↪they become critical.
```

2.3.5 Navigation bar

The top navigation bar provides quick access to ClusterControl's common functionalities. It also displays the breadcrumbs for cluster list and notification banners. If you click sidebar's **Clusters** and click on a cluster, the breadcrumbs will appear on the navigation bar visualizing the current path you are at.

Subscribe Now

Clicking on the **Subscribe Now** button at the top-right area will redirect you to our Severalnine's self-serve web page.

What is self-serve?

Self-serve is a new license offering by Severalnines for our ClusterControl customers. This allows you to gain freedom from being locked in an annual payment or yearly contracts if the budget is tight or you wanted to experience the features and functionality that ClusterControl can offer. There are doubts and skepticism when utilizing ClusterControl for your environment (production, QA, or staging/dev environment), so this can an ideal option if you want to have a monthly contract. Please checkout the self-serve web page for more details.

Deploy a cluster

Clicking on the **Deploy a Cluster** button at the top-right area will give users two more options, as shown in the following screenshot:

- **Create a database cluster** – Use this deployment wizard to deploy a new database cluster or server. See *Create Database Cluster*.

- **Import a database cluster** – Use this deployment wizard to import your existing database cluster or server. See *Import Database Cluster*.

Activity center

Next to the **Deploy a Cluster** button is an icon (commonly appeared with a small red counter) called **Activity center**. Clicking on it will expand the quick view of activity center where you can see the latest alarms, jobs and audit logs. There are also **Actions** button associated with every entry where you can perform certain actions according to the entry's type.

User profile

The top-right corner provides quick access to the current user's profile. It provides the timezone information, email address and the full name (in first name, last name format) with quick access to profile management, license management and the logout page.

2.3.6 Sidebar

The left sidebar provides all ClusterControl global menus. It is collapsible and expandable by clicking the < or > icon at the bottom.

Kubernetes

Info

This feature is in technical preview in v2.3.2.

Clicking on **Kubernetes** will expand the list of all database clusters managed by ClusterControl which running on Kubernetes platform. See *Integration → Kubernetes*.

This feature can be disabled by going to ***ClusterControl GUI → Settings → License → Disable Kubernetes***.

Clusters

Clicking on **Clusters** will expand the list of all database clusters managed by ClusterControl. Click on one of the clusters will open the individual cluster section on the main panel.

Nodes

Provides global overview of all nodes for all clusters managed by ClusterControl.

You can filter the list by cluster and node's status, by clicking on the filter icon in the respective columns. You can also perform node's management tasks by clicking on the **Actions** button.

Read more

- *User Guide → Management → Nodes Management*
- *User Guide → Observability → Status and Healthchecks*

Backups

The Backups page in ClusterControl provides a centralized view of all backup tasks, schedules, and storage repositories in your environment. It allows you to create new backups, manage existing ones, and review completed jobs. This page is essential for ensuring data safety and for streamlining backup operations.

Here are its key sections:

- **Create backup** – Clicking this opens a wizard or form to initiate a new backup job, schedule a backup or create a new Elasticsearch snapshot repository (specific to Elasticsearch backups). See *Create Backup*.
- **All Backups** – Lists all completed or in-progress backups for all clusters. You can customize the display columns by clicking on the **View** button.
- **Schedules** – Shows the schedules configured for automatic or recurring backups for all clusters.
- **Elastic Repositories** – (Specific to Elasticsearch backups) Lists all Elasticsearch snapshot repositories.

Read more

- *User Guide → Backup and Restore → Create Backup*.
- *User Guide → Backup and Restore → Schedule Backup*.

Activity center

Provides global view of all alarms, jobs and audit logs for all clusters managed by ClusterControl.

Operational reports

The operational reports feature provides a comprehensive overview of your database cluster's performance, health, and key metrics, delivering valuable insights to ensure optimal database operations. This feature generates detailed, time-based reports that highlight crucial aspects of your database environment, allowing you to monitor and review system performance, resource usage, and database health at a glance.

With the operational reports, you can:

- Gain insights into resource utilization, such as CPU, memory, disk, and network usage.
- Monitor cluster performance metrics, including query throughput, replication lag, and load.
- Track backup operations, alerting on failures or potential risks in the backup schedule.
- Receive a summary of database status, including uptime, availability, and incident occurrences.
- Customize reports to focus on specific time periods or key performance indicators.

These reports empower database administrators to make informed decisions by providing critical information for troubleshooting, capacity planning, and performance tuning. They can be scheduled to run automatically or generated on-demand, making it easy to stay on top of your database operations and ensure the long-term reliability and efficiency of your database clusters.

Read more

- *User Guide → Observability → Operational Reports*.

User management

Manage ClusterControl users, teams and authentication integration with directory services (LDAP or Active Directory).

Read more

- Administration Guide → Users and Teams

Settings

Provides global settings for the ClusterControl instance.

Profile

Manage the current user's profile and time zone settings. The user's time zone reflects how ClusterControl displays the date and timestamp in the application.

Updating user's profile

Click on **Settings** will open

Change password

Click on the **Change password** and specify the current password and the new password accordingly. Click **Change** to proceed.

If you want to reset the user's password instead, login as a user belongs to "admins" group and go to ***ClusterControl → User management → choose a user → Actions → Change password***.

Cloud storage credentials

The Cloud storage credentials tab provides a centralized location for managing credentials used to store and retrieve backups from supported cloud providers. By configuring valid credentials here, you can automate backup uploads to, and downloads from, various cloud platforms.

The supported object storage platforms are:

- AWS Simple Storage Service (S3)
- Azure Blob Storage
- Google Cloud Storage
- S3-compatible object storage like MinIO, DigitalOcean Spaces, Openstack Swift and SeaweedFS.

Cloud storage integration relies on other ClusterControl modules called `cmon-cloud` and `clud`. These binaries are provided by `clustercontrol-cloud` and `clustercontrol-clud` packages.

Read more

- *User Guide → Integration → Cloud Providers*.
- *User Guide → Backup and Restore → Upload Backup to Cloud*.

- *User Guide → Backup and Restore → Create Backup*.
- *User Guide → Backup and Restore → Schedule Backup*.

Notification services

Manage the integration with third-party notification and messaging platform on events triggered by ClusterControl. This feature allows you to integrate ClusterControl alerts into your organization's communication channels, incident response systems, workflow management systems or event-driven applications via webhook calls. You can configure the notifications to be sent out to all or specific clusters, or to all or specific alert severity like error or warning alerts.

Notification service relies on another ClusterControl module called `cmon-events`. This binary is provided by package `clustercontrol-notifications`.

Read more

- *User Guide → Integration → Notification Services*.
- *Reference Manuals → Components → ClusterControl Notifications*

Certificate management

Manage all keys and certificates of all clusters managed by ClusterControl. In this section, you can generate a new certificate authority (CA), generate a new self-signed certificate or import existing certificates into the certificate repository. The certificates imported or generated here can then be assigned to any cluster managed by ClusterControl.

Read more

- *User Guide → Management → Certificates*.

License

View and manage the details of your ClusterControl license, as well as activating enterprise and experimental features. From this section, you can check if your license is active, identify how many nodes you can manage under the current license, and update the license key as needed.

The following table explains the fields shown in this section:

Field	Description
License	Displays the edition of ClusterControl (e.g., ClusterControl Enterprise).
Status	Indicates whether the license is currently Active or Expired.
License owner	Shows the organization or individual that owns the license.
Expiration date	The date and time when the license will expire. Once this date passes, ClusterControl may revert to a restricted mode unless renewed or updated.
License owner's email	Contact email for the entity that owns the license. Usually used for support or renewal discussions.
Nodes limit	Displays how many database server nodes are currently managed compared to the total limit available under the license (e.g., 12/unlimited nodes). See *Licensed nodes*

!!! Notes "Licensing Types"

```
ClusterControl offers the following licensing types that customers can␣
↪avail:

- Community - Free license offering
- Advanced (Self-Serve) - This is a new offering by Severalnines. Checkout␣
↪our page
- Advanced - Commercial license with support options.
- Enterprise - Commercial license that offers more support options.

You may check [Plan comparison detail](https://severalnines.com/pricing/)␣
↪for complete details.
```

Enable multiple controllers

The multi-controller feature enables centralized management of several ClusterControl installations from a single user interface. It supports environments with over 1000 nodes by horizontal scaling through the addition of multiple ClusterControl controllers.

From a unified dashboard, you can monitor, manage, and gain insights into all controllers, clusters, and nodes, with the ability to switch between active controllers for targeted operations.

Read more

- *User Guide → Integration → ClusterControl Ops-C*.

Updating license key

For users with a valid subscription (Advanced and Enterprise), enter your license key by clicking on the **Enter a license key** button to unlock additional features based on the subscription. The license string contains information about the license type, company/affiliation, email, expiration date, and the total number of licensed nodes.

The following example is license information that one would get from us:

```
Email: test@severalnines.com
Company: Severalnines AB
Expiration Date: 2019-04-23
Licensed Nodes: 1
License Key:␣
↪ewogICAgImNvbXBhbnkiOiAiU2V2ZXJhbG5pbmPkIEFCIiwKICAgICJlbWFpbF9hZGRyZXNzIjog
InRlc3RAc2VRZXJhbG5pbmVzLmNvbSIsCiAgICAiZXhwaXJhdGlvbl9kYXRlIjogIjIwMTktMDQt
MjNUMDA6MDA6MDAuMDAxWiIsCiAgICAibGljZW5zZWRfbm9kZXMiOiAtMSwKICAgICJ0eXBlIjog
IkVudGVycHJpc2UiCn0Ke3M5cy1zaWduYXR1cmUtc2VwYXJhdG9yfQp0RUew5OZKV8SqmmwiQEzT
P+qTNnmCphirIVm7MriA/aCdlJYQcr1NJr4nvTNcSmgu4uFVf3Ufv4olHr4wrBq0/Js9Rm8bJWZo
BO8svHzQhCmIVEWcTYub362krjRyREnOGXaqWwUnvkZ0uUCT+WDaM1P9qn/HawoNd0e8E0+7WiZK
CpwjH+ESqSEppeu/Ewzf3p0C0e8WZbwHtZ9UmX2qJNQq9NDlByrO8FtbVjOOL4zTbc8jV0W2DWzY
1swzOgeyk+7N2eGVRWfdUSzudQSXkT3LA4cdV2HAsU5QLnmKxSCgg+jq+RQJiPwdPXEr3gzjzJzV
+qhmOZ5tTN+WABPy9l3kpztlCbkfzO84/4lM7Z3c4rQ8snMTu6RvD2M+oh/4lhvR8M9RrQQcl8JF
RX2Ak1ZAKxAXkJ97Z5U7nIzuyUGuMTCXdKGEtQkBXzpIcYFvXDeWu0MUks+EULpqG+OFnl+rSZa0
nNTSW3mR/f9B+4e2mK4y2OpJhh4rWPXR1DLpLVLk/2p0o64aEizA+IPe0TP+ox7bFzEfAXirVWfC
/Ol7m1k6arRbl8PSV1DRRcefM+UsABa6jypoiit+JXNPOajdjY1WBgEekCn/jeXBBoPM2k26274u
br0BuHULLkxGSpC8I2/nW6s84E653FO1Kpbvyx+2SKJxwUxLiuEZ2g==
```

Only paste the license key string (starting after the `License Key:` part) into the license text field. Click **Apply license key** to activate the license into runtime. No service restart required.

Licensed nodes

The following nodes' type will be counted as licensed nodes by ClusterControl:

- MySQL/MariaDB/Galera Cluster database nodes
- PostgreSQL/TimescaleDB/EDB database nodes(1) { .annotate }

    1. EDB requires appropriate licensing from the vendor. Kindly get the database license ready prior to the deployment.

- ProxySQL nodes
- Redis database (or shard) nodes
- Valkey database (or shard) nodes
- MongoDB Community/Enterprise database (or shard) nodes(1) { .annotate }

    1. MongoDB Enterprise requires appropriate licensing from the vendor. Kindly get the database license ready prior to the deployment.

- SQL Server database nodes(1) { .annotate }

    1. SQL Server requires appropriate licensing from the vendor. Kindly get the database license ready prior to the deployment.

- Elasticsearch master-eligible and data nodes

A "node" is an instance of database or proxy process. For PostgreSQL, ClusterControl supports managing multiple instances per host. Each PostgreSQL instance (`host:port` pair) is considered as one node.

The following nodes' type are NOT counted as licensed nodes:

- Load balancers/Connection pooler - HAProxy, PgBouncer, MariaDB MaxScale and Keepalived nodes
- MongoDB sharded cluster - MongoDB mongos, config server and arbiter nodes
- Monitoring - Prometheus, VictoriaMetrics nodes
- Backup-related nodes - Backup verification server, Elasticsearch snapshot repository nodes, pgBackRest repository nodes, Percona MongoDB Backup shared storage nodes
- ClusterControl nodes and ClusterControl multi-controller nodes

Best Practices and Notes

1. When the license expires, ClusterControl defaults back to the Community Edition. For features comparison, please refer to the ClusterControl product page. Your database cluster will remain operational after a license is expired.
2. If you would like to force the existing Enterprise edition to the Community edition (commonly to test out and compare different editions during trial), you can truncate the license table on the ClusterControl host manually. On the ClusterControl server, run:

```
$ mysql -uroot -p cmon -e "TRUNCATE TABLE cmon.license"
$ systemctl restart cmon
```

   !!! Danger "Warning"

```
 Once a trial license is truncated and the `cmon` service is␣
↪restarted, there is no way to re-activate the same trial license␣
↪anymore.
```

3. ClusterControl will raise an alarm if a license is about to expire within 30 days (as well as a reminder banner when you logged in ClusterControl). Configure the mail server accordingly to receive the alarm via email. Ensure you renew your license before expiration to avoid interruptions in production management.
4. If you exceed your node limit, you may not be able to add additional nodes. Contact your license provider or Severalnines account manager to upgrade if you anticipate growth beyond the current limit. Severalnines also offers volume license and custom licensing options depending on the use cases and requirements.
5. If you experience any issues related to licensing, you may contact Severalnines support directly and provide the license's owner email address.

6. After updating your license key, consider refreshing your browser to confirm that all license-related features are enabled and loaded correctly.

Diagnostics

Enable 'UI Audit Data Collection' to record and capture the necessary data while reproducing an issue in the user interface. Once the issue has been replicated, download the audit data for analysis. Remember to switch off the setting afterward to stop audit data collection.

Read more

- *User Guide → Troubleshooting → Error Report*.

Changelog

Open the ClusterControl changelog page in Severalnines documentation center. It often shows release highlights, version-specific changes, release notes, and recent updates.

2.3.7 Footer

The footer displays information about the components' version that you are running on. For example:

```
             © Copyright Severalnines 2025 All Rights Reserved.
 Controller:2.3.0.11678. Buildv:2.3.0 sha:a60dc4e5 b:1788␣
↪n:release-cc-2.3.0
```

The above means in this particular setup, the running components are:

- ClusterControl Controller v2.3.0 and build number is 11678
- ClusterControl GUI v2.3.0, commit hash is a60dc4e5 and build number is 1788

2.3.8 Give Us Feedback

This form provides quick access to let us know your feedbacks of the product. If submitted, the form will be sent over via a HTTP call back to our feedback collection system, where it will reach the ClusterControl product manager inbox anonymously.

It is a one-way submission system and no futher information or follow-up is required. The submission requires an Internet connection and without it, the submission will fail.

2.4 ClusterControl CLI

The ClusterControl CLI is a command-line tool to interact, control, and manage database clusters using the ClusterControl Database Platform. By default, the installer script will automatically install and configure this package on the ClusterControl node. You can also install it on another computer or workstation to manage the database cluster remotely. This command-line project is open source and publicly available at GitHub.

ClusterControl CLI opens a new door for cluster automation where you can easily integrate it with existing deployment automation tools like Ansible, Puppet, Chef, or Salt. Not all functionalities

2.4.1 Overview

The command-line tool is invoked by executing a binary called `s9s`. The commands are basically JSON messages being sent over to the ClusterControl Controller (CMON) RPC interface. Communication between the ClusterControl CLI (`s9s`) and the `cmon` process (ClusterControl Controller) is encrypted using TLS and requires port 9501 to be accessible on the controller node.

2.4.2 Installation

The ClusterControl CLI is automatically installed and configured if ClusterControl was installed via one of the methods described under *Online Installation*. You usually want to install the ClusterControl CLI manually if you want to use the CLI outside of the ClusterControl host, or if you installed ClusterControl using the manual or offline installation method.

Installer script

We have built an installer script for `s9s-tools` package available at `http://repo.severalnines.com/s9s-tools/install-s9s-tools.sh`. On the ClusterControl host (or any client host):

```
$ wget http://repo.severalnines.com/s9s-tools/install-s9s-tools.sh
$ chmod 755 install-s9s-tools.sh
$ sudo ./install-s9s-tools.sh
```

If you would like to install manually, see *Package manager* or *Compile from source*.

Package manager

The package list is available on the s9s-tools repository page.

Note

Red Hat/CentOS/Rocky Linux/AlmaLinux

The repository definition file for each distribution can be downloaded directly from:

- RHEL 7: http://repo.severalnines.com/s9s-tools/RHEL_7/s9s-tools.repo
- RHEL 8: http://repo.severalnines.com/s9s-tools/RHEL_8/s9s-tools.repo
- RHEL 9: http://repo.severalnines.com/s9s-tools/RHEL_9/s9s-tools.repo

Installation steps are straight-forward:

```
# Rocky Linux 9
wget http://repo.severalnines.com/s9s-tools/RHEL_9/s9s-tools.repo -P /
↪etc/yum.repos.d
dnf install s9s-tools
```

Debian/Ubuntu

To install, simply do:

```
# Ubuntu 24.04
wget -qO - http://repo.severalnines.com/s9s-tools/$(lsb_release -sc)/
↪Release.key | sudo apt-key add -
echo "deb http://repo.severalnines.com/s9s-tools/$(lsb_release -sc)/ ./"␣
↪| sudo tee /etc/apt/sources.list.d/s9s-tools.list
sudo apt-get update
sudo apt-get install s9s-tools
```

Compile from source

It is possible to build the ClusterControl CLI on Linux and Mac OS/X. To build from source, you may require additional packages and tools to be installed:

1. Get the source code from Github:

```
git clone https://github.com/severalnines/s9s-tools.git
```

2. Navigate to the source code directory:

```
cd s9s-tools
```

3. Install development packages such as C/C++ compiler, `autotools` and `openssl-devel`:

Note

Red Hat/CentOS/Rocky Linux/AlmaLinux

```
yum groupinstall "Development Tools"
yum install automake git openssl-devel
```

Ubuntu/Debian

```
sudo apt-get install build-essential automake git libssl-dev byacc flex␣
↪bison
```

4. Compile the source code:

```
./autogen.sh
./configure
make
make install
```

5. ClusterControl CLI should be installed. Verify by running the following command:

```
s9s --version
```

2.4.3 Configuring CLI user

The first thing that must be done is to create a user that is allowed to connect to and use the controller. Communication between the `s9s` command-line client and the controller (the `cmon` process) is encrypted using TLS on port 9501. Public and private RSA key pairs associated with a username are used to encrypt the communication. The `s9s` command-line client is responsible to set up the user and the required private and public keys.

Local access

During a ClusterControl installation, a user called "admin" is automatically created and configured inside `/etc/s9s.conf` (that is the reason "admin" username is reserved) for ClusterControl CLI usage. To verify this, invoke the `--whoami` flag in the `user` command:

```
s9s user --whoami
```

Example output

```
$ s9s user --whoami
admin
```

If you don't want to use the default `admin` user for ClusterControl CLI, you can create another user as below:

```
s9s user --create \
  --generate-key \
  --controller="https://localhost:9501" \
  --group=admins <username>
```

Example output

Create a user called "dba" to access CLI on the same host as the ClusterControl server:

```
$ s9s user --create \
  --generate-key \
  --controller="https://localhost:9501" \
  --group=admins dba
Grant user 'dba' succeeded.
```

Warning

Attention

Group "admins" has the right to perform all operations on the managed clusters. For a fine-grained access level, please create a different group and assign it with proper ACLs. See s9s-tree.

You will notice the following files will be generated under `~/.s9s/` directory:

```
-rw-------   1 root root 1675 Jan 31 13:29 dba.key
-rw-------   1 root root  426 Jan 31 13:29 dba.pub
-rw-r--r--.  1 root root  286 Jan 31 13:29 s9s.conf
```

Commonly, after running the `--whoami` command, you will become the new user "dba" because `~/.s9s/s9s.conf` (if exists) will take precendence over the global `/etc/s9s.conf`:

```
$ s9s user --whoami
dba
```

To force ClusterControl CLI to use the "admin" user (or to use other user's configuration file located elsewhere), you can prefix the command with `S9S_USER_CONFIG` environment variable:

```
$ S9S_USER_CONFIG=/etc/s9s.conf s9s user --whoami
admin
```

Remote access

The steps to set up the ClusterControl CLI for remote access is similar as for *local access*, except:

- The `s9s` command-line client must exist on the remote server.
- The ClusterControl controller (cmon) must be accepting TLS connections from the remote server.
- The remote server can connect to the controller with key-based authentication (no password). This is required only during the user creation of a private/public key setup for encryption purposes.

The following shows how to prepare ClusterControl to accept remote user access and for a remote user called `remote_dba`.

1. Setup the bind address for ClusterControl controller (cmon) process as follow:

   ```
   vi /etc/default/cmon
   ```

   Add the line:

   ```
   RPC_BIND_ADDRESSES="127.0.0.1,192.168.99.12"
   ```

   Here we assume the public IP address of the ClusterControl host (controller) is 192.168.99.12.

Warning

Attention

```
Ideally, you should lock down this IP address with firewall rules, only␣
↪allowing access from the remote servers you wish to access the␣
↪controller from.
```

2. Restart the controller and check the log:

   ```
   systemctl restart cmon
   ```

3. Verify the ClusterControl Controller is listening to the configured IP address on port 9501:

   ```
   $ cat /var/log/cmon.log | grep -i tls
   2016-11-29 12:34:04 : (INFO) Server started at tls://127.0.0.1:9501
   2016-11-29 12:34:04 : (INFO) Server started at tls://192.168.99.12:9501
   ```

4. On the remote server/computer, we have to enable key-based authentication and create a user called `remote_dba`:

   ```
   useradd -m remote_dba
   ```

5. As the current user (root or sudoer for example) in the remote server, set up a passwordless SSH to the ClusterControl host. Generate one SSH key if you don't have one:

```
ssh-keygen -t rsa    # press 'Enter' for all prompts
```

6. Copy the SSH public key to the ClusterControl controller host, for example, 192.168.99.12:

```
ssh-copy-id root@192.168.99.12
```

7. Create the ClusterControl CLI client user:

```
s9s user --generate-key --create --group=admins --controller=
↪"https://192.168.99.12:9501" remote_dba
```

Example output

```
$ s9s user --generate-key --create --group=admins --controller=
↪"https://192.168.99.12:9501" remote_dba
Warning: Permanently added '192.168.99.12' (ECDSA) to the list of␣
↪known hosts.
Connection to 192.168.99.12 closed.
Grant user 'remote_dba' succeeded.
```

8. Ensure the config file located at `~/.s9s/s9s.conf` looks like this (take note the IP address of the controller may be different):

```
[global]
cmon_user = remote_dba
controller_host_name = 192.168.99.12
controller_port = 9501
rpc_tls = true
```

9. Finally, test the connection:

```
s9s cluster --cluster-id=0 --ping  # you should see 'Ping Ok ..'
s9s cluster --list  # you should see a list of cluster name
```

The configuration for ClusterControl CLI remote user is complete.

2.4.4 Using the CLI

In most sections of the user guide, there are many examples on how to perform actions using ClusterControl CLI (alongside ClusterControl GUI and Terraform Provider for ClusterControl). Find the **ClusterControl CLI** tab to understand how the commands are constructred and manipulated by looking at the examples.

Example

The following is an example tabbed content in the user guide section which highlights the CLI examples:

ClusterControl GUI

.. Some step-by-step guidance ..

ClusterControl CLI

.. Some example commands using ClusterControl CLI .. (1) { .annotate }

1. Look at this tab for ClusterControl CLI examples.

Terraform Provider for ClusterControl

.. Some example definitions using Terraform HCL ..

For full list of all commands and options, see *ClusterControl CLI Reference Manuals*.

Getting started

To check the CLI version, use the `-V` or `--version` flag:

```
s9s --version
```

Example output

```
$ s9s --version
      ___              _                 _
 ___ / _ \ ___       | |_ ___    ___ | |___
/ __| (_) / __|_____| __/ _ \ / _ \| / __|
\__ \\__, \__ \_____| || (_) | (_) | \__ \
|___/  /_/|___/       \__\___/ \___/|_|___/

s9s version 1.9.2024112020 (Sweden)
BUILD (1.9.2024112020-release) 2024-11-21 18:34:39+00:00
Copyright (C) 2016-2022 Severalnines AB
```

To list out all available commands and options, use the `--help` flag:

```
s9s --help
```

Example output

```
$ s9s --help
Usage:
s9s COMMAND [OPTION...]

Where COMMAND is:
```

```
    account - to manage accounts on clusters.
      alarm - to manage alarms.
     backup - to view, create and restore database backups.
    cluster - to list and manipulate clusters.
 controller - to manage Cmon controllers.
        job - to view jobs.
maintenance - to view and manipulate maintenance periods.
   metatype - to print metatype information.
       node - to handle nodes.
    process - to view processes running on nodes.
replication - to monitor and control data replication.
   dbschema - to view database schemas.
     report - to manage reports.
     script - to manage and execute scripts.
     server - to manage hardware resources.
      sheet - to manage spreadsheets.
       user - to manage users.

Generic options:
    -c, --controller=URL       The URL where the controller is found.
    --config-file=PATH         Specify the configuration file for the␣
↪program.
    --help                     Show help message and exit.
    -P, --controller-port INT  The port of the controller.
    -p, --password=PASSWORD    The password for the Cmon user.
    --private-key-file=FILE    The name of the file for authentication.
    --rpc-tls                  Use TLS encryption to controller.
    -u, --cmon-user=USERNAME   The username on the Cmon system.
    -v, --verbose              Print more messages than normally.
    -V, --version              Print version information and exit.

Formatting:
    --batch                    No colors, no human readable, pure data.
    --color=always|auto|never  Sets if colors should be used in the␣
↪output.
    --date-format=FORMAT       The format of the dates printed.
    -l, --long                 Print the detailed list.
    --log-file=PATH            The path where the s9s client puts its␣
↪logs.
    --no-header                Do not print headers.
    --only-ascii               Do not use UTF8 characters.
    --print-json               Print the sent/received JSon messages.
    --print-request            Print the sent JSon request message.

Job related options:
    --job-tags=LIST            Set job tags when creating a new job.
    --log                      Wait and monitor job messages.
    --recurrence=CRONTABSTRING Timing information for recurring jobs.
    --schedule=DATE&TIME       Run the job at the specified time.
    --timeout=SECONDS          Timeout value for the entire job.
```

```
    --wait                      Wait until the job ends.
```

You can also view the user manual by using the `man` command:

```
man s9s
```

For every command, there are help information available with the `--help` flag:

```
s9s cluster --help
s9s node --help
s9s backup --help
s9s user --help
```

Alternatively, run the user manual command and specify the `s9s-<command>` as the chapter name, for example:

```
man s9s-cluster
man s9s-node
man s9s-backup
```

After a command, specify the arguments (with its respective value) or flags. In the following example, we are listing out all nodes belong to cluster ID 36 (Percona XtraDB Cluster 8.0) in a detailed format (`--long`):

```
s9s node --cluster-id=36 --list --long # (1)!
```

1. Get the cluster ID from ClusterControl GUI or ClusterControl CLI using `s9s cluster --list --long`.

Tip

Use the `--cluster-name` argument to identify a cluster with string, instead of `--cluster-id` with integer. For example:

```
s9s node --cluster-name="PROD - PXC 8.0" --list --long
```

Example output

```
$ s9s node --cluster-id=36 --list --long
STAT VERSION     CID CLUSTER        HOST            PORT COMMENT
coC- 2.3.0.11678  36 PROD - PXC 8.0 192.168.99.2    9500 Up and running.
Po-- 2.29.2       36 PROD - PXC 8.0 192.168.99.2    9090 Process
 ↪'prometheus' is running.
goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.3    3306 Up and running␣
 ↪(read-write).
Ao-- -            36 PROD - PXC 8.0 192.168.99.3    4433 Process 'cmnd' is␣
 ↪running.
goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.4    3306 Up and running␣
 ↪(read-write).
```

```
Ao-- -            36 PROD - PXC 8.0 192.168.99.4   4433 Process 'cmnd' is␣
↪running.
goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.5   3306 Up and running␣
↪(read-write).
Ao-- 1.0.0        36 PROD - PXC 8.0 192.168.99.5   4433 Process 'cmnd' is␣
↪running.
yo-- 2.6.5        36 PROD - PXC 8.0 192.168.99.6   6032 Process 'proxysql
↪' is running.
koM- 2.2          36 PROD - PXC 8.0 192.168.99.6    112 Process
↪'keepalived' is running.
yl-- 2.6.5        36 PROD - PXC 8.0 192.168.99.7   6032 Processes with␣
↪name 'proxysql' has ended.
k?-- 2.2          36 PROD - PXC 8.0 192.168.99.7    112 Process
↪'keepalived' is running.
```

By default, ClusterControl CLI will authenticate as the user configured inside `~/.s9s/s9s.conf`. If it does not exist, it will use the global configuration located at `/etc/s9s.conf`. To bypass these configuration files, specify the user's configuration as arguments:

```
# using private key to authenticate
s9s node --cluster-id=36 --list --long \
  --cmon-user=dba \
  --private-key-file=~/.s9s/dba.key \
  --controller=https://localhost:9501 \
  --rpc-tls

# using password to authenticate
s9s node --cluster-id=36 --list --long \
  --cmon-user=dba \
  --password='mysUp3rRsecr3tP455' \
  --controller=https://localhost:9501 \
  --rpc-tls
```

To let ClusterControl CLI load another user configuration file, use the `--config-file` option or `S9S_USER_CONFIG` environment variables:

```
# using --config-file
s9s node --cluster-id=36 --list --long --config-file=/
↪home/dba/my-custom-s9s.conf

# using prefixed environment variable
S9S_USER_CONFIG=/home/dba/my-custom-s9s.conf s9s node --cluster-id=36 --
↪list --long

# using exported environment variable
export S9S_USER_CONFIG=/home/dba/my-custom-s9s.conf
s9s node --cluster-id=36 --list --long
```

Initial check

There are multiple ways to check whether ClusterControl CLI is able to connect to the controller service correctly, as show below:

- Ping cluster ID 0 (a special cluster ID):

```
s9s cluster --cluster-id=0 --ping  # make sure you get 'PING Ok ...'
```

- Get the current active user:

```
s9s user --whoami   # make sure you see your own username
```

- Get the command's exit-code:

```
s9s user --whoami -v | grep exit   # 0 means OK. Not 0 means failed or␣
↪error.
```

Output and manipulation

For more verbosity on the request being sent to the controller, append `-v` flag to any command:

```
s9s node --cluster-id=36 --list --long -v
```

Example output

```
    $ s9s node --cluster-id=36 --list --long -v
    Command line options processed.
    Preparing to send request.
    URI is '/v2/auth'

    +++ Connecting to localhost:9501...
    Connected.
    Initiate TLS...
    TLS handshake finished (version: TLSv1.3, cipher:␣
↪TLS_AES_256_GCM_SHA384).
    Sending:
    POST /v2/auth HTTP/1.0
    Host: localhost:9501
    User-Agent: s9s-tools/1.0
    Connection: close
    Accept: application/json
    Transfer-Encoding: identity
    Content-Type: application/json
    Content-Length: 201

    {
        "operation": "authenticateWithPassword",
        "password": "xxxxxxxxxxxxxxxxxxx",
        "request_created": "2025-01-31T13:06:21.100Z",
        "request_id": 1,
        "user_name": "admin"
```

```
    }

    Controller version: 2.3.0.11678
    Preparing to send request.
    URI is '/v2/clusters/'

    +++ Connecting to localhost:9501...
    Connected.
    Initiate TLS...
    TLS handshake finished (version: TLSv1.3, cipher:␣
↪TLS_AES_256_GCM_SHA384).
    Sending:
    POST /v2/clusters/ HTTP/1.0
    Host: localhost:9501
    User-Agent: s9s-tools/1.0
    Connection: close
    Accept: application/json
    Transfer-Encoding: identity
    Cookie: cmon-sid=8fe6911b1dc84ceca65c90b00fe005b8
    Content-Type: application/json
    Content-Length: 155

    {
        "cluster_id": 36,
        "operation": "getClusterInfo",
        "request_created": "2025-01-31T13:06:21.118Z",
        "request_id": 2,
        "with_hosts": true
    }

    STAT VERSION     CID CLUSTER        HOST           PORT COMMENT
    coC- 2.3.0.11678  36 PROD - PXC 8.0 192.168.99.2   9500 Up and␣
↪running.
    Po-- 2.29.2       36 PROD - PXC 8.0 192.168.99.2   9090 Process
↪'prometheus' is running.
    goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.3   3306 Up and␣
↪running (read-write).
    Ao-- -            36 PROD - PXC 8.0 192.168.99.3   4433 Process 'cmnd
↪' is running.
    goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.4   3306 Up and␣
↪running (read-write).
    Ao-- -            36 PROD - PXC 8.0 192.168.99.4   4433 Process 'cmnd
↪' is running.
    goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.5   3306 Up and␣
↪running (read-write).
    Ao-- 1.0.0        36 PROD - PXC 8.0 192.168.99.5   4433 Process 'cmnd
↪' is running.
    yo-- 2.6.5        36 PROD - PXC 8.0 192.168.99.6   6032 Process
↪'proxysql' is running.
    koM- 2.2          36 PROD - PXC 8.0 192.168.99.6    112 Process
```

```
↪'keepalived' is running.
    yl-- 2.6.5          36 PROD - PXC 8.0 192.168.99.7   6032 Processes␣
↪with name 'proxysql' has ended.
    k?-- 2.2            36 PROD - PXC 8.0 192.168.99.7    112 Process
↪'keepalived' is running.
    Total: 12
    Exiting with exitcode 0.
```

Use the `--print-json` flag to get an extended output in JSON format. For example:

```
s9s node --cluster-name="PROD - PXC 8.0" --list --long --print-json
```

If a command triggers a job, you can choose to wait for the job to finish and a progress bar will be displayed (otherwise, a job will be registered and running in background):

```
s9s node --cluster-id=36 --nodes=192.168.99.4 --restart --wait
```

Example output

```
$ s9s node --cluster-id=36 --nodes=192.168.99.4 --restart --wait
Restarting Node
/ Job 1630 RUNNING    [████        ]  40% Restarting node
```

To see and follow the triggered job messages, specify `--log` (it will display similar output as ClusterControl GUI's job messages):

```
s9s node --cluster-id=36 --nodes=192.168.99.4 --restart --log
```

Example output

```
$ s9s node --cluster-id=36 --nodes=192.168.99.4 --restart --log
CMON version 2.3.0.11678.
Using SSH credentials from cluster.
Cluster ID is 36.
The creds name is 'ssh_cred_cluster_36_6245'.
The username is 'root'.
The keyfile is '/root/.ssh/id_rsa'.
Preparing to restart host.
192.168.99.4: Checking ssh/sudo with credentials ssh_cred_cluster_36_6245.
Saving cluster_autorecovery: true, node_autorecovery: true settings.
Setting cluster_autorecovery: false, node_autorecovery: false settings.
192.168.99.4:3306: Stopping mysqld (timeout=600, force stop after␣
↪timeout=false).
192.168.99.4: Stopping MySQL service.
192.168.99.4:  mysql.service - Percona XtraDB Cluster
     Loaded: loaded␣
↪(]8;;file://docs-pxc-prod-02/usr/lib/systemd/system/mysql.service/usr/lib/systemd/syst
↪enabled; preset: disabled)
```

```
     Active: active (running) since Tue 2025-02-11 10:05:31 UTC; 55s ago
    Process: 92799 ExecStartPre=/usr/bin/mysql-systemd start-pre␣
 ↪(code=exited, status=0/SUCCESS)
    Process: 92826 ExecStartPre=/bin/sh -c systemctl unset-environment _
 ↪WSREP_START_POSITION (code=exited, status=0/SUCCESS)
    Process: 92827 ExecStartPre=/bin/sh -c VAR=`bash /
 ↪usr/bin/mysql-systemd galera-recovery`; [ $? -eq 0 ] && systemctl␣
 ↪set-environment _WSREP_START_POSITION=$VAR || exit 1 (code=exited,␣
 ↪status=0/SUCCESS)
    Process: 93681 ExecStartPost=/bin/sh -c systemctl unset-environment _
 ↪WSREP_START_POSITION (code=exited, status=0/SUCCESS)
    Process: 93683 ExecStartPost=/usr/bin/mysql-systemd start-post
 ↪$MAINPID (code=exited, status=0/SUCCESS)
Main PID: 92862 (mysqld)
     Status: "Server is operational"
      Tasks: 55 (limit: 10866)
     Memory: 609.5M
        CPU: 4.763s
     CGroup: /system.slice/mysql.service
             92862 /usr/sbin/mysqld --
 ↪wsrep_start_position=8fad3f53-7f17-11ef-a5d5-5f0a8f95055b:4637068

Feb 11 10:05:23 docs-pxc-prod-02 systemd[1]: Starting Percona XtraDB␣
 ↪Cluster...
Feb 11 10:05:25 docs-pxc-prod-02 systemd[1]: mysql.service: Got␣
 ↪notification message from PID 93296, but reception only permitted for␣
 ↪main PID 92862
Feb 11 10:05:31 docs-pxc-prod-02 mysql-systemd[93683]:  SUCCESS!
Feb 11 10:05:31 docs-pxc-prod-02 systemd[1]: Started Percona XtraDB␣
 ↪Cluster.
192.168.99.4: Stopping the service mysqld.
...
```

In an output with a list of entries, the left-most column (`STAT`) displays some letters representation of the object. Here is an example:

```
$ s9s node --cluster-id=36 --list --long
STAT VERSION     CID CLUSTER        HOST           PORT COMMENT
coC- 2.3.0.11678  36 PROD - PXC 8.0 192.168.99.2   9500 Up and running.
Po-- 2.29.2       36 PROD - PXC 8.0 192.168.99.2   9090 Process 'prometheus
 ↪' is running.
goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.3   3306 Up and running␣
 ↪(read-write).
Ao-- -            36 PROD - PXC 8.0 192.168.99.3   4433 Process 'cmnd' is␣
 ↪running.
goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.4   3306 Up and running␣
 ↪(read-write).
Ao-- -            36 PROD - PXC 8.0 192.168.99.4   4433 Process 'cmnd' is␣
 ↪running.
```

(continues on next page)

(continued from previous page)

```
goM- 8.0.37       36 PROD - PXC 8.0 192.168.99.5   3306 Up and running␣
↪(read-write).
Ao-- 1.0.0        36 PROD - PXC 8.0 192.168.99.5   4433 Process 'cmnd' is␣
↪running.
yo-- 2.6.5        36 PROD - PXC 8.0 192.168.99.6   6032 Process 'proxysql'␣
↪is running.
koM- 2.2          36 PROD - PXC 8.0 192.168.99.6    112 Process 'keepalived
↪' is running.
yl-- 2.6.5        36 PROD - PXC 8.0 192.168.99.7   6032 Processes with name
↪'proxysql' has ended.
k?-- 2.2          36 PROD - PXC 8.0 192.168.99.7    112 Process 'keepalived
↪' is running.
Total: 12
```

This information can be demystified by looking at the user manual page of the `node` command, under the `STAT` section:

```
$ man s9s-node | grep -A21 -- 'STAT   '
            STAT   Some  status information represented as individual␣
↪letter. This field contains the following char-
                   acters:

                nodetype
                       This is the type of the node. It can be c for␣
↪controller, g for Galera node, x  for  MaxScale
                       node,  k for Keepalived node, p for PostgreSQL, m␣
↪for Mongo, e for MemCached, y for ProxySql,
                       h for HaProxy, b for PgBouncer, B for PgBackRest, t␣
↪for PBMAgent, a for Garbd,  r  for  group
                       replication  host, A for cmon agent, P for␣
↪Prometheus, s for generic MySQL nodes, S for Redis
                       sentinel, R for Redis, E for Elasticsearch, and ?␣
↪for unknown nodes.

                hoststatus
                       The status of the node. It can be o for on-line, l␣
↪for off-line, f for failed  nodes,  r  for
                       nodes performing recovery, - for nodest that are␣
↪shut down and ? for nodes in unknown state.

                role   This  field shows the role of the node in the␣
↪cluster. This can be M for master, S for Slave,
                       U for multi (master and slave), C for controller, V␣
↪for backup verification node, A  for  ar-
                       biter, R for backup repository host D for␣
↪Elasticsearch data host c for Elasticsearch coordi-
                       nator_only host and - for everything else.

                maintenance
                       This field shows if the node is in maintenance␣
```

(continues on next page)

(continued from previous page)

```
↪mode. The character is M for nodes in  maintenance mode and - for nodes that are not in␣
↪maintenance mode.
```

By default, ClusterControl CLI displays a human-friendly output with colouring, header and some statistics. You can use the `--batch` flag to return a plain data, which is useful for manipulation by other commands or automation script:

```
s9s node --cluster-name="PROD - PXC 8.0" --list --long --batch
```

Example output

With `--batch`, it prints data without syntax highlight, header and totals. Only pure table to be processed using filters:

```
$ s9s node --cluster-name="PROD - PXC 8.0" --list --long --batch
coC- 2.3.0.11678 36 PROD - PXC 8.0 192.168.99.2   9500 Up and running.
Po-- 2.29.2      36 PROD - PXC 8.0 192.168.99.2   9090 Process 'prometheus
↪' is running.
goM- 8.0.37      36 PROD - PXC 8.0 192.168.99.3   3306 Up and running␣
↪(read-write).
Ao-- -           36 PROD - PXC 8.0 192.168.99.3   4433 Process 'cmnd' is␣
↪running.
goM- 8.0.37      36 PROD - PXC 8.0 192.168.99.4   3306 Up and running␣
↪(read-write).
Ao-- -           36 PROD - PXC 8.0 192.168.99.4   4433 Process 'cmnd' is␣
↪running.
goM- 8.0.37      36 PROD - PXC 8.0 192.168.99.5   3306 Up and running␣
↪(read-write).
Ao-- 1.0.0       36 PROD - PXC 8.0 192.168.99.5   4433 Process 'cmnd' is␣
↪running.
yo-- 2.6.5       36 PROD - PXC 8.0 192.168.99.6   6032 Process 'proxysql'␣
↪is running.
koM- 2.2         36 PROD - PXC 8.0 192.168.99.6    112 Process 'keepalived
↪' is running.
yl-- 2.6.5       36 PROD - PXC 8.0 192.168.99.7   6032 Processes with␣
↪name 'proxysql' has ended.
k?-- 2.2         36 PROD - PXC 8.0 192.168.99.7    112 Process 'keepalived
↪' is running.
```

To redirect the output to another command, it is recommended to do it in batch mode:

```
$ s9s node --cluster-name="PROD - PXC 8.0" --list --long --batch | grep ^
↪coC | awk {'print $9'}
9500
```

With `--print-json`, we can redirect the output to a JSON processor like `jq` and perform JSON object filtering:

```
$ s9s user --whoami --print-json | jq '.user.groups[].group_name'
"admins"
```

Combine with `diff`, we can compare the current database configurations (from the configuration file) side-by-side between two PostgreSQL database nodes, 192.168.99.7 and 192.168.99.8:

```
diff -y <(s9s node --list-config --nodes=192.168.99.7:5432) <(s9s node --
↪list-config --nodes=192.168.99.8:5432)
```

ClusterControl provides an *RPC API interface* for the ClusterControl controller (CMON).

Please note that the RPCv2 API calls are only available via secure (TLS) connections, where encryption is mandatory. The RPCv2 also uses an authentication call, and very few of the calls are available to be used without authenticating first.

By default, the encrypted communication can be used through port 9501 and the RPC v2 requests are server served on the `/v2` path (e.g. `/v2/clusters`).

2.5 Accessing the RPC API

By default, the ClusterControl RPC API interface (port 9501) listens to localhost. This means, only local user is able to access the API interface and additional steps are required to expose this interface publicly. See *Remote access*.

2.5.1 Local access

1. Access the ClusterControl IP address on port 9501 as shown in the example below:

   ```
   curl -k https://127.0.0.1:9501/v2
   ```

2. You should get the following reply:

   ```
   {
       "controller_id": "c469a80f-8ad9-4e7b-8f5d-82346c671609",
       "error_string": "Invalid operation '' on '/v2/info'.",
       "request_operation": "",
       "request_path": [
           "v2"
       ],
       "request_processed": "2025-04-15T17:55:49.409Z",
       "request_status": "InvalidRequest"
   }
   ```

Despite the error, it means that you are able to connect to the interface and get a proper reply. See *Authenticating with the API* to start sending a proper request.

2.5.2 Remote access

To expose the RPC API remotely, include the IP address of the host in the `RPC_BIND_ADDRESSES` variables inside `/etc/default/cmon`. The following commands should be executed on the ClusterControl server.

1. Add the primary IP address of the ClusterControl node inside `/etc/default/cmon`:

```
RPC_BIND_ADDRESSES="192.168.20.10,127.0.0.1"
```

2. Restart `cmon` service to make sure ClusterControl listens on port 9501 for both IP addresses defined above:

```
systemctl restart cmon
```

3. Make sure you see port 9501 is listening to the configured IP addresses.

```
$ netstat -tulpn | grep 9501
tcp        0      0 127.0.0.1:9501          0.0.0.0:*               ␣
↪LISTEN      1681928/cmon
tcp        0      0 192.168.20.10:9501      0.0.0.0:*               ␣
↪LISTEN      1681928/cmon
```

4. Access the ClusterControl IP address on port 9501 as shown in the example below:

```
curl -k https://192.168.61.210:9501/v2
```

5. You should get the following reply:

```
{
    "controller_id": "c469a80f-8ad9-4e7b-8f5d-82346c671609",
    "error_string": "Invalid operation '' on '/v2/info'.",
    "request_operation": "",
    "request_path": [
        "v2"
    ],
    "request_processed": "2025-04-15T17:55:49.409Z",
    "request_status": "InvalidRequest"
}
```

Despite the error, it means that you are able to connect to the interface and get a proper reply. See *Authenticating with the API* to start sending a proper request.

2.6 Authenticating with the API

If you would like to access the cluster object via HTTP, send the request to the RPCv2 API accessible on port 9501 (TLS enabled). Use one of the following two methods:

1. Pass the username and password inside a nested `authenticate` method in an operation:

```
curl -k 'https://192.168.20.10:9501/v2/clusters' \
-XPOST -d \
'{"operation": "getAllClusterInfo",
"authenticate":
{"password": "s3cr3tP455","username": "readeruser"}}'
```

2. This is the recommended way and is used by all our applications to connect to the ClusterControl controller. Users need to maintain cookies/sessions after the first authentication and send other requests with the corresponding cookies/session. To obtain a session, use the `authenticateWithPassword` operation and connect to the `v2/auth` endpoint:

```
curl -k 'https://192.168.20.10:9501/v2/auth' \
-XPOST -d \
'{"operation":"authenticateWithPassword",
"user_name":"readeruser",
"password":"s3cr3tP455"}' \
-c cookies.jar
```

Then, include cookies.jar with every subsequent read-only request:

```
curl -k 'https://192.168.20.10:9501/v2/clusters' \
-XPOST -d \
'{"operation":"getallclusterinfo"}' \
-b cookies.jar
```

2.7 Successful request

If the RPC call is successful, you should receive a response with `"request_status": "Ok"`, similar to the following output:

```
{
    "controller_id": "683e2b4b-038f-4cf7-93a0-4c3f95b3db70",
    "request_processed": "2024-11-07T19:33:12.701Z",
    "request_status": "Ok",
    "request_user_id": 5,
    "total": 3,
    "clusters":
    [
        ...json content...
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 'readeruser'."
    ]
}
```

2.8 Failed request

In this example, the user that we use is only having a read-only permission. If the user is trying to execute a non-read-only request, for example, sending a stop node job that requires an "execute" permission:

```
curl -k 'https://192.168.20.10:9501/v2/jobs' \
    -XPOST -d '{"cluster_id": 21, "job": {"class_name": "CmonJobInstance",
    "job_spec":
    {"command": "stop",
    "job_dat":{"clusterid": 21,
    "node":{"hostname": "192.168.20.53"}}}},
    "operation": "createJobInstance"}' \
    -b cookies.jar
```

The reply should return `"request_status": "AccessDenied"`, similar to the following output:

```
{
    "controller_id": "9eb8b1d1-f9d1-439e-945e-108982812bee",
    "error_string": "Execute access to cluster 21 for user readeruser is␣
↪denied.",
    "request_processed": "2021-05-04T07:01:38.342Z",
    "request_status": "AccessDenied",
    "request_user_id": 7,
    "debug_messages":
    [
        "RPC V2 authenticated user is 'readeruser'."
    ]
}
```


See also

For a comprehensive list of all supported remote procedure calls and endpoints, see *Reference Manuals → API/SDK → ClusterControl RPC API*.

2.9 Deployment

This section covers key strategies and tools for deploying your database clusters with ClusterControl. Learn how to streamline setup, ensure consistency across environments, and enable rapid scaling as your infrastructure grows.

See also

- *Create Database Cluster →*
- *Import Database Cluster →*
- *Create Load Balancer →*
- *Import Load Balancer →*

2.10 Create Database Cluster

The Create Database Cluster feature in ClusterControl allows users to easily deploy new database clusters with just a few clicks, eliminating the complexities of manual database setup. This feature provides an intuitive workflow for configuring high-availability setups, replication, load balancing, and more.

With Create Database Cluster, you can:

- Automatically deploy single or multi-node database clusters.
- Configure the topology and replication mechanisms during deployment.
- Fine-tune the database server with custom configuration template.
- Configure encryption to improve security.

This feature ensures a streamlined process, making it simple for administrators to quickly set up robust, highly available database environments that are ready for production use. Whether you are a beginner or

an experienced DBA, ClusterControl's guided deployment process ensures consistency, reliability, and best practices in database management.

2.10.1 Support Matrix

The following database cluster types, vendors and topology are supported:

Database	Vendor	Topology
MySQL	Percona, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
MySQL Group Replication	Oracle	Single-primary, multi-primary
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication, logical replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.10.2 Prerequisites

There are prerequisites that need to be fulfilled prior to the deployment for all database clusters:

- Make sure the target database nodes are running on a supported architecture platform and operating system. See *Hardware* and *Operating System*.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to all database nodes. See *SSH Key-based Authentication*.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.

2.10.3 MySQL Replication

A minimum of two nodes is required for MySQL/MariaDB replication. If only one database IP address or hostname is provided, ClusterControl will deploy it as a standalone MySQL/MariaDB server with binary log enabled.

The following vendors and versions are supported for a new deployment:

- Oracle MySQL - 8.0 and 8.4.
- Percona Server for MySQL - 8.0 and 8.4.
- Percona Server for MySQL Pro - 8.0 and 8.4.
- MariaDB Server - 10.5, 10.6, 10.11, 11.4, 11.8.

ClusterControl supports configuring both replication scheme - asynchronous or semisynchronous. It is possible to set up a primary-primary replication (circular replication) under the **Add nodes** section. You can also add more replicas later after the deployment is completed.

Default configuration

By default, ClusterControl deploys MySQL/MariaDB replication with the following configurations:

- MySQL GTID with `log_slave_updates` enabled (MySQL and Percona only).
- MariaDB GTID with `log_slave_updates` enabled (MariaDB only).
- All database nodes will be configured with `read_only=ON` and `super_read_only=ON` (if supported) for protection against accidental write. The chosen primary will be promoted by disabling the read-only in the runtime.
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- ClusterControl will create and grant necessary privileges for the following database users:

```
| Database user      | Purpose                                          |
| ------------------ | ------------------------------------------------ |
| `cmon`             | Management and automation.                       |
| `cmonexporter`     | Prometheus exporter for database monitoring.     |
| `cmonagent`        | ClusterControl query monitoring agent.           |
| `cmon_replication` | MySQL/MariaDB replication.                       |
| `backupuser`       | Backup and restore management.                   |
```

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

ClusterControl sets `read_only=ON` on all slaves but a privileged user (SUPER) can still write to a slave (except for MySQL versions that support `super_read_only`).

Deployment steps

ClusterControl GUI

1. To create a new MySQL replication cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MySQL Replication".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy MySQL Replication Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

Tip

A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.
7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
   - **Admin/Root user**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPER privilege and the GRANT option for localhost only.
   - **Admin/Root password**: The password for **Admin/Root user**.

- **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
- **Configuration template**: The dropdown loads all MySQL configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
- **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
- **Semi-synchronous replication**: Toggle on (default) will instruct ClusterControl to configure a semi-synchronous replication. Toggle off will instruct ClusterControl to configure an asynchronous replication.

9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication. If you want to configure a multi-primary replication (circular replication), toggle on ***Use multi-primary replication***.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.
    - **Use multi-primary replication**: Toggle off (default) will instruct ClusterControl to create the replication cluster as a single-primary replication. Toggling on will open up another set of **Primary node** and **Replica nodes** form field to be filled up for multi-primary replication. ClusterControl will set up the secondary primary node using circular replication but with read-only enabled.

    Note

    Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node Oracle MySQL replication 8.0, with operating system user "ubuntu" (the first node is the primary):

```
s9s cluster --create \
    --cluster-type=mysqlreplication \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=oracle \
    --provider-version=8.0 \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name='PROD - MySQL Replication 8.0'
```

- Create a four-node MariaDB 11.4 multi-primary replication, using operating system user "root", with multiple primaries and replicas (note the `?` sign to identify the node's role in the `--nodes` parameter, where the second primary will be a backup-primary) and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=mysqlreplication \
    --nodes=
↪"192.168.5.11?master;192.168.5.12?slave;192.168.7.11?master;192.168.7.12?slave"␣
↪\
    --vendor=mariadb \
    --provider-version=11.4 \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='System A - MariaDB 11.4' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 3-node Percona Server for MySQL replication 8.0 with semi-synchronous replication, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-replication-cluster" {

    db_cluster_create = true
    db_cluster_name = "PROD - MySQL Replication 8.0"
    db_cluster_type = "replication"
    db_version = "8.0"
    db_vendor = "percona"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
```

(continues on next page)

(continued from previous page)

```
        hostname_internal = "db1.mydb.local"
    }
    db_host {
        hostname = "10.10.10.12"
        hostname_internal = "db2.mydb.local"
    }
    db_host {
        hostname = "10.10.10.13"
        hostname_internal = "db3.mydb.local"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true
    db_semi_sync_replication = true

}
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.10.4 MySQL Galera

A minimal setup is comprised of one Galera node (no high availability, but this can later be scaled with more nodes). However, a minimum of 3 nodes is recommended for high availability. Garbd (an arbitrator) can be added later after the deployment completes.

The following vendors and versions are supported for a new deployment:

- Percona XtraDB Cluster - 8.0 and 8.4.
- Percona XtraDB Cluster Pro - 8.4.
- MariaDB Cluster - 10.5, 10.6, 10.11, 11.4 and 11.8.

Default configuration

By default, ClusterControl deploys Galera Cluster with the following configurations:

- Use `xtrabackup-v2` or `mariabackup` (depending on the vendor chosen) for `wsrep_sst_method`.
- Binary logging is enabled.

- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	MySQL/MariaDB replication.
`backupuser`	Galera SST, backup and restore management.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Deployment steps

ClusterControl GUI

1. To create a new MySQL Galera Cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MySQL Galera".
2. Choose a vendor from the **Vendor** dropdown.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy MySQL Galera Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

> **Tip**
>
> A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.

- **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
- **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
   - **Admin/Root user**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPER privilege and the GRANT option for localhost only.
   - **Admin/Root password**: The password for **Admin/Root user**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
   - **Version**: Choose a supported minor version from the dropdown. If ignored, ClusterControl will install the latest minor version of the current major version available at the moment.
   - **Configuration template**: The dropdown loads all MySQL configuration templates file under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
   - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step. This option is ignored if the chosen vendor is Percona. Percona XtraDB Cluster 8.0 enables SSL by default regardless of this setting.
9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:

- **Galera nodes**: Specify the IP address or hostname of the database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. A minimum of 3 nodes is recommended, and try to stick with odd number of nodes in total to avoid split-brain situation.

  Note

  Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node Percona XtraDB Cluster 8.0, with operating system user "ubuntu":

```
s9s cluster --create \
    --cluster-type=galera \
    --nodes="10.10.10.101;10.10.10.102;10.10.10.103" \
    --vendor=percona \
    --provider-version=8.0 \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name='Percona XtraDB Cluster 8.0'
```

- Create a three-node MariaDB Cluster 11.4 and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=galera \
    --nodes="192.168.5.11;192.168.5.12;192.168.5.13" \
    --vendor=mariadb \
    --provider-version=11.4 \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='System A - MariaDB Cluster 11.4' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 3-node Percona XtraDB Cluster 8.0, with operating system user "ubuntu":

```
resource "clustercontrol_db_cluster" "my-galera-cluster" {

    db_cluster_create = true
    db_cluster_name = "Percona XtraDB Cluster 8.0"
    db_cluster_type = "galera"
    db_version = "8.0"
    db_vendor = "percona"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.101"
    }
    db_host {
        hostname = "10.10.10.102"
    }
    db_host {
        hostname = "10.10.10.103"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.10.5 MySQL Group Replication

A minimal setup for is comprised of one node (no high availability - can be scaled later with more nodes). A recommended setup should be minimum of 3 nodes for high availability.

The following vendors and versions are supported for a new deployment:

- Oracle MySQL Group Replication - 8.0 and 8.4

Default configuration

By default, ClusterControl deploys Group Replication Cluster with the following configurations:

- By default, Clustercontrol parameter `auto_manage_readonly` is set to false.
- Set `loose-group_replication_single_primary_mode=OFF`, which means it is configured for multi-primary mode by default.
- Additional `my.cnf` settings:
  - `loose-group_replication_exit_state_action=ABORT_SERVER`
  - `loose-group_replication_ssl_mode=REQUIRED`
  - `loose-group_replication_recovery_use_ssl=ON`
- Binary logging is enabled.
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`rpl_user`	MySQL group replication.
`backupuser`	Backup and restore management.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Deployment steps

ClusterControl GUI

1. To create a new MySQL Group Replication cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MySQL Group Replication".
2. Choose a vendor from the **Vendor** dropdown.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy MySQL Group Replication** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

> **Tip**
>
> A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
    - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
    - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
    - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
    - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
    - **SSH sudo / OS elevation command**: Select the OS command used to elevate user privileges for executing system-level operations. You can choose from `sudo`, `doas`, `pbrun`
    - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
    - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
    - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.
7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
    - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
    - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
    - **Admin/Root user**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPER privilege and the GRANT option for localhost only.
    - **Admin/Root password**: The password for **Admin/Root user**.
    - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.

- **Version**: Choose a supported minor version from the dropdown. If ignored, ClusterControl will install the latest minor version of the current major version available at the moment.
- **Configuration template**: The dropdown loads all MySQL configuration templates file under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type, by default ClusterControl uses `my.cnf.grouprepl` template. Choose one configuration template for the deployment. See *Configuration Template*.
- **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step. This option is ignored if the chosen vendor is Percona. Percona XtraDB Cluster 8.0 enables SSL by default regardless of this setting.

9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Node**: Specify the IP address or hostname of the database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. A minimum of 3 nodes is recommended, and try to stick with odd number of nodes in total to avoid lose quorum.

      Note

      Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node Oracle MySQL Group Replication Cluster 8.0, with operating system user "ubuntu":

```
s9s cluster --create \
    --cluster-type=group_replication \
    --nodes="192.168.40.194;192.168.40.195;192.168.40.196" \
    --vendor=oracle \
    --provider-version=8.0 \
    --db-admin='root' \
    --db-admin-passwd='mydbPassw0rd' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name='My Group Replication Cluster v8.0'
```

Terraform Provider for ClusterControl

- Deploy a 3-node MySQL Group Replication Cluster 8.0, with operating system user "ubuntu":

```
resource "clustercontrol_db_cluster" "my-group-replication-cluster" {

    db_cluster_create = true
    db_cluster_name = "MySQL Group Replication Cluster 8.0"
    db_cluster_type = "group_replication"
    db_version = "8.0"
    db_vendor = "oracle"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "192.168.40.194"
    }
    db_host {
        hostname = "192.168.40.195"
    }
    db_host {
        hostname = "192.168.40.196"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.10.6 PostgreSQL Streaming

A minimum of two nodes is required for PostgreSQL streaming replication. If only one database IP address or hostname is provided, ClusterControl will deploy it as a standalone PostgreSQL server. The following vendors and versions are supported for a new deployment:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, 16, 17 and 18.
- PostgreSQL from EDB repository - 12, 13, 14, 15, 16, 17 and 18 (requires a valid EDB repository token).

Default configuration

By default, ClusterControl deploys PostgreSQL instances with the following configurations:

- Configure and load the `pg_stat_statements` module.
- The WAL level is set to `replica`.
- All replica will be set with `hot_standby`.
- Configure the `cluster_name` value.
- ClusterControl will configure the PostgreSQL instance with SSL encryption for client-server connections.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.
`backupuser`	Backup and restore management.

Tip

ClusterControl supports pgvector and PostGIS extensions which can be enabled with our PostgreSQL deployment wizard through an additional ***Extensions*** step.

Deployment steps

ClusterControl GUI

1. To create a new PostgreSQL streaming replication, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "PostgreSQL Streaming".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment. For "PostgreSQL Enterprise DB", it requires a valid EDB repository token.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy PostgreSQL cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags. If you choose "PostgreSQL Enteprise DB" as the vendor, provide the EDB repository token where you can retrieve it from here. Click **Continue** to proceed to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
    - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
    - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
    - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
    - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
    - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
    - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
    - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.
7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
    - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
    - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically. If blank, ClusterControl will use the default path.
    - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
    - **Password**: The password for **User**.
    - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
    - **Configuration template**: The dropdown loads all PostgreSQL configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl

node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.

- **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.

9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.
        - **Synchronous Replication**: Toggle off (default) will instruct ClusterControl to skip the synchronous replication. Toggling on will instruct ClusterControl to configure synchronous replication to the corresponding replica. You can have a mixed of synchronous and asynchronous replication for multiple replicas in a cluster.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Extensions** page. In this section, you can toggle on extension from the list you want to be installed together with the PostgreSQL database. Once the cluster is deployed, you will need to enable the extension by using the `CREATE EXTENSION` statement.

12. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.
13. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node PostgreSQL 16 streaming replication, with operating system user "ubuntu" (the first node is the primary) with tags `production` and `postgres`:

```
s9s cluster --create \
    --cluster-type=postgresql \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
```

(continues on next page)

(continued from previous page)

```
    --vendor=postgresql \
    --provider-version=16 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --with-tags='production;postgres' \
    --cluster-name='PostgreSQL 16 - Streaming Replication'
```

- Create a four-node PostgreSQL EDB 16 streaming replication, using operating system user "root" (the first node is the primary) and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=postgresql \
    --nodes="192.168.5.11;192.168.5.12;192.168.5.13;192.168.5.14" \
    --vendor=enterprisedb \
    --provider-version=11.4 \
    --db-admin='postgres' \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --enterprise-token='XXXXXXXXXXXXXXXXXXXXXXXXX' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='System A - PostgreSQL EDB 16 - Streaming␣
↪Replication' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 3-node PostgreSQL streaming replication 16.0, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-postgresql-cluster" {

    db_cluster_create = true
    db_cluster_name = "PostgreSQL Streaming Replication 16"
    db_cluster_type = "postgresql"
    db_version = "16"
    db_vendor = "postgresql"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
    db_host {
        hostname = "10.10.10.12"
    }
    db_host {
        hostname = "10.10.10.13"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
```

(continues on next page)

(continued from previous page)

```
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.10.7 PostgreSQL Logical

Deploys a new PostgreSQL logical replication cluster from ClusterControl. A PostgreSQL logical cluster is built up of multiple PostgreSQL streaming clusters. Logical replication will be between these streaming clusters, and each streaming cluster can have both publications and subscriptions. A minimum of two nodes is required for PostgreSQL logical replication, which is represented in the **Cluster 1** and **Cluster 2** section under the **Add Nodes**. Cluster 1 can be the target publisher and Cluster 2 can be the target subscriber (or vice versa). Multi-primary replication is possible, by configuring two-way publication and subscription links of the same supported database objects.

The following vendors and versions are supported for a new deployment:

- PostgreSQL from postgresql.org repository - 16 and 17.
- PostgreSQL from EDB repository - 16 and 17 (requires a valid EDB repository token).

Default configuration

By default, ClusterControl deploys PostgreSQL instances with the following configurations:

- Configure and load the `pg_stat_statements` module.
- The WAL level is set to `logical`.
- Configure the `cluster_name` value.
- ClusterControl will configure the PostgreSQL instance with SSL encryption for client-server connections.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.
`cmon_logical_replication`	PostgreSQL logical replication.
`backupuser`	Backup and restore management.

Tip

ClusterControl supports pgvector and PostGIS extensions which can be enabled with our PostgreSQL deployment wizard through an additional ***Extensions*** step.

Deployment steps

ClusterControl GUI

1. To create a new PostgreSQL logical replication, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "PostgreSQL Logical".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment. For "PostgreSQL Enterprise DB", it requires a valid EDB repository token.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy PostgreSQL Logical cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags. If you choose "PostgreSQL Enteprise DB" as the vendor, provide the EDB repository token where you can retrieve it from here. Click **Continue** to proceed to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:

   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.

- **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
- **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
- **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically. If blank, ClusterControl will use the default path.
   - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
   - **Password**: The password for **User**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
   - **Configuration template**: The dropdown loads all PostgreSQL configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
   - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Cluster 1**

- **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
- **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.
  - **Synchronous Replication**: Toggle off (default) will instruct ClusterControl to skip the synchronous replication. Toggling on will instruct ClusterControl to configure synchronous replication to the corresponding replica. You can have a mixed of synchronous and asynchronous replication for multiple replicas in a cluster.

- **Cluster 2**
  - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
  - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.
    - **Synchronous Replication**: Toggle off (default) will instruct ClusterControl to skip the synchronous replication. Toggling on will instruct ClusterControl to configure synchronous replication to the corresponding replica. You can have a mixed of synchronous and asynchronous replication for multiple replicas in a cluster.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Extensions** page. In this section, you can toggle on extension from the list you want to be installed together with the PostgreSQL database. Once the cluster is deployed, you will need to enable the extension by using the `CREATE EXTENSION` statement.

12. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.
13. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a two-node PostgreSQL 16 logical replication, with operating system user "ubuntu" (the first node is the primary) with tags `production` and `postgres`:

```
s9s cluster --create \
    --cluster-type=postgresql_logical \
    --nodes="10.10.10.11;10.10.10.12" \
    --vendor=postgresql \
    --provider-version=16 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --with-tags='production;postgres' \
    --cluster-name='PostgreSQL 16 - Logical Replication'
```

- Create a two-node PostgreSQL EDB 16 logical replication, using operating system user "root" (the first node is the primary) and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=postgresql_logical \
    --nodes="192.168.5.11;192.168.5.12;" \
    --vendor=enterprisedb \
    --provider-version=11.4 \
    --db-admin='postgres' \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --enterprise-token='XXXXXXXXXXXXXXXXXXXXXXXXXX' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='System A - PostgreSQL EDB 16 - Logical Replication
↪' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 2-node PostgreSQL logical replication 17.0, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-postgresql-cluster" {

    db_cluster_create = true
    db_cluster_name = "PostgreSQL Logical Replication 16"
    db_cluster_type = "postgresql_logical"
    db_version = "17"
    db_vendor = "postgresql"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
    db_host {
        hostname = "10.10.10.12"
```

(continues on next page)

(continued from previous page)

```
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.10.8 TimescaleDB

Deploys a new TimescaleDB standalone or streaming replication cluster from ClusterControl. A minimum of two nodes is required for TimescaleDB streaming replication. If only one database IP address or hostname is provided, ClusterControl will deploy it as a standalone TimescaleDB server. The following vendors and versions are supported for a new deployment:

- TimescaleDB - 12, 13, 14, 15, 16 and 17.

Default configuration

By default, ClusterControl deploys TimescaleDB with the following configurations:

- Configure and load the `pg_stat_statements` module and `timescaledb` extension.
- The WAL level is set to `replica`.
- All replica will be set with `hot_standby`.
- Configure the `cluster_name` value.
- ClusterControl will configure the PostgreSQL instance with SSL encryption for client-server connections.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.
`backupuser`	Backup and restore management.

Note

Note You can also deploy a PostgreSQL and convert it to TimescaleDB at a later stage. However, this action will be irreversible and ClusterControl will treat the cluster as TimescaleDB onwards.

Deployment steps

ClusterControl GUI

1. To create a new TimescaleDB streaming replication, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "TimescaleDB".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy TimescaleDB cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags. Click **Continue** to proceed to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled,

ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.

- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically. If blank, ClusterControl will use the default path.
   - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
   - **Password**: The password for **User**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
   - **Configuration template**: The dropdown loads all TimescaleDB configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
   - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for streaming replication.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.
      - **Synchronous Replication**: Toggle off (default) will instruct ClusterControl to skip the synchronous replication. Toggling on will instruct ClusterControl to configure syn-

chronous replication to the corresponding replica. You can have a mixed of synchronous and asynchronous replication for multiple replicas in a cluster.

Note

Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Extensions** page. In this section, you can toggle on extension from the list you want to be installed together with the TimescaleDB database. Once the cluster is deployed, you will need to enable the extension by using the `CREATE EXTENSION` statement.

12. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.
13. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node TimescaleDB 16 streaming replication, with operating system user "ubuntu" (the first node is the primary) with tags `production` and `timescaledb`:

```
s9s cluster --create \
    --cluster-type=postgresql \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=postgresql \
    --provider-version=16 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --with-tags='production;timescaledb' \
    --cluster-name='TimescaleDB 16 - Streaming Replication' \
    --with-timescaledb
```

- Create a standalone TimescaleDB 15, with operating system user "root" with tags `test` and `timescaledb`, and wait until the job finishes:

```
s9s cluster --create \
    --cluster-type=postgresql \
    --nodes="192.168.99.11" \
    --vendor=postgresql \
    --provider-version=15 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --with-tags='test;timescaledb' \
    --cluster-name='TimescaleDB 16 - standalone' \
```

(continues on next page)

(continued from previous page)

```
    --with-timescaledb \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 3-node TimescaleDB streaming replication 16.0, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-timescaledb-cluster" {

    db_cluster_create = true
    db_cluster_name = "TimescaleDB Streaming Replication 16"
    db_cluster_type = "postgresql"
    db_version = "16"
    db_vendor = "postgresql"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
    db_host {
        hostname = "10.10.10.12"
    }
    db_host {
        hostname = "10.10.10.13"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true
    db_enable_timescale = true

}
```

2.10.9 MongoDB Replica Set

Deploys a new MongoDB standalone or replica set. The database cluster will be automatically added to ClusterControl once deployed. A minimum of three nodes (including MongoDB arbiter) is recommended for a replica set. If only one node is specified, ClusterControl will deploy it as a standalone MongoDB node. The following vendors and versions are supported for a new deployment:

- MongoDB Community - 7.0 and 8.0
- MongoDB Enterprise - 7.0 and 8.0
- Percona Server for MongoDB - 7.0 and 8.0

Default configuration

By default, ClusterControl deploys MongoDB Replica Set members with the following configurations:

- Configure `setParameter.enableLocalhostAuthBypass: true` inside MongoDB configuration file.
- ClusterControl will create and grant necessary roles for additional MongoDB users - `admin.cmon_backup` for backup and restore purposes and `admin.cmonexporter` for query monitoring.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

It is possible to deploy only 2 MongoDB nodes (without an arbiter). The caveat of this approach is no automatic failover. If the primary node goes down then manual failover is required to make the other server run as primary. Automatic failover works fine with 3 nodes and more.

Deployment steps

ClusterControl GUI

1. To create a new MongoDB replica set, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MongoDB ReplicaSet".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy MongoDB ReplicaSet cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.

- **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
- **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
- **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
- **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
   - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted the built-in root role.
   - **Password**: The password for **User**.
   - **ReplicaSet name**: Specify the name of the replica set, similar to `replication.replSetName` option in MongoDB.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part.
   - **Configuration template**: The dropdown loads all MongoDB configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
   - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
9. Click **Continue** to proceed to the next step.

10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:

    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for a replica set configuration.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes. If this field is left blank, ClusterControl will deploy the primary node as a standalone node.
        - **Act as an arbiter**: Toggle off (default) will instruct ClusterControl to configure the node as a standard MongoDB replica. Toggling on will instruct ClusterControl to configure the node as a MongoDB arbiter. An arbiter participates in elections for primary but it does not have a copy of the data set and cannot become a primary.

    Note

    Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node MongoDB Enterprise replica set 7.0, with operating system user "ubuntu" (the first node is the primary) and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=mongodbenterprise \
    --nodes="192.168.99.13;192.168.99.15;192.168.99.19" \
    --vendor=mongodbenterprise \
    --provider-version='7.0' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --db-admin='admin' \
    --db-admin-passwd='MyS3cr3tPass' \
    --cluster-name='MongoDB Enteprise ReplicaSet 7.0' \
    --wait
```

- Create a three-node MongoDB Community replica set 7.0, with operating system user "root" (the first node is the primary and the last node is an arbiter) and let the deployment job run in the fore-

ground:

```
s9s cluster --create \
    --cluster-type=mongodb \
    --nodes="192.168.99.13;192.168.99.15;192.168.99.19?arbiter" \
    --vendor=mongodb \
    --provider-version='7.0' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --db-admin='admin' \
    --db-admin-passwd='MyS3cr3tPass' \
    --cluster-name='MongoDB ReplicaSet 7.0' \
    --wait
```

- Create a standalone MongoDB Community server 6.0, using operating system user "root" and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=mongodb \
    --nodes="192.168.5.11" \
    --vendor=mongodb \
    --provider-version='6.0' \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='MongoDB 6.0 - standalone - devtest' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 3-node replica set of Percona Server for MongoDB 7.0, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-mongodb-replicaset" {

    db_cluster_create = true
    db_cluster_name = "Percona MongoDB Replica Set 7.0"
    db_cluster_type = "mongodb"
    db_version = "7.0"
    db_vendor = "percona"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
    db_host {
        hostname = "10.10.10.12"
    }
    db_host {
        hostname = "10.10.10.13"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
```

(continues on next page)

(continued from previous page)

```
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

2.10.10 MongoDB Sharded Cluster

Deploys a new MongoDB Sharded Cluster. The database cluster will be automatically added to ClusterControl once deployed. For production deployment, it is recommended to have at least 8 nodes for high availability setup:

- 2 nodes for the mongos (router),
- 3 nodes for the config server (replica set),
- 3 nodes per shard (replica set).

For a very minimal setup (suitable for testing and experimenting), you can start with two nodes and deploy it as below:

- 1 node for config server (one-node replica set) and mongos (co-locate together),
- 1 node for shard (one-node replica set).

The following vendors and versions are supported for a new deployment:

- MongoDB Enterprise - 7.0 and 8.0
- MongoDB Community - 7.0 and 8.0
- Percona Server for MongoDB - 7.0 and 8.0

Default configuration

By default, ClusterControl deploys MongoDB Sharded Cluster with the following configurations:

- Configure `setParameter.enableLocalhostAuthBypass: true` inside MongoDB configuration file.
- ClusterControl will create and grant necessary privileges for an additional MongoDB user - `admin.cmon_backup` for backup and restore purposes.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

MongoDB Sharded Cluster does not support `mongodump` backup method. Users will be asked to install Percona Backup for MongoDB (PBM) when creating or scheduling a backup for this cluster type after the deployment completes. Note that PBM requires a shared remote backup storage (e.g, NFS) accessible on all MongoDB nodes.

Deployment steps

ClusterControl GUI

1. To create a new MongoDB sharded cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MongoDB Shards".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy MongoDB Shards cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
    - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
    - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
    - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted the built-in root role.
    - **Password**: The password for **User**.
    - **Configuration template**: The dropdown loads all MongoDB configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
    - **Router configuration template**: The dropdown loads all MongoDB mongos configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
    - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
    - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
9. Click **Continue** to proceed to the next step.
10. Under the **Configuration Servers and Routers** section, you can specify the MongoDB config and mongos servers information:
    - **Add configuration server**: Specify the IP address or hostname of the MongoDB config servers. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. For high availability, at least 3 nodes are required.
        - **Port**: MongoDB server port for MongoDB config server (config). The default is 27019.
    - **Node router server**: Specify the IP address or hostname of the MongoDB mongos servers. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. It is possible to co-locate mongos and config servers together.
        - **Port**: MongoDB server port for MongoDB router server (mongos). The default is 27017.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Under the **Shard** section, you can specify the target MongoDB database nodes and configure the replica set and shards:

```
- **ReplicaSet name**: Specify the name of the replica set, similar to␣
↪`replication.replSetName` option in MongoDB.
- **Port**: MongoDB server port for MongoDB shard server (mongod). The␣
↪default is 27018.
- **Add nodes to the shard**: Specify the IP address or hostname of the␣
↪primary database node. Press ++enter++ to add the node, where␣
↪ClusterControl will perform a pre-deployment check to verify if the node␣
↪is reachable via SSH key-based authentication. If the target node has␣
↪more than one network interface, you will be able to select or enter a␣
↪separate IP address to be used only for database traffic. Only one␣
↪primary node is allowed for a replica set configuration.
- **Replica nodes**: Specify the IP address or hostname of the replica␣
↪database node. Press ++enter++ to add the node, where ClusterControl will␣
↪perform a pre-deployment check to verify if the node is reachable via SSH␣
↪key-based authentication. If the target node has more than one network␣
↪interface, you will be able to select or enter a separate IP address to␣
↪be used only for database traffic. You can specify zero or more replica␣
↪nodes. If this field is left blank, ClusterControl will configure the␣
↪shard with a single primary node.

    - **Replication delay** - Specify the amount of time the secondary␣
↪member should be delayed in seconds.
    - **Priority** - Set the secondary member priority. A bigger integer␣
↪represents a higher priority. A delayed secondary member should be set to␣
↪0 because it is not fit to become primary and should be hidden from the␣
↪application.

- **Add another shard**: Add another shard section to define another␣
↪replica set of the shard. The same **Shard** form will be presented and␣
↪you can add as many as 50 shards per cluster.
- **Remove this shard**: Discard the current shard.

:::{note} Note
```

After deployment completes, it is the database administrator job to enable sharding at the database and collection level. ::: 12. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

13. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a MongoDB Community sharded cluster 7.0 which consists of 3 mongos, 3 MongoDB config, and one shard consists of a three-node replica set called 'replset2' (with `priority` and `slave_delay` options), operating system user "root" and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=mongodb \
    --nodes=
↪"mongos://192.168.1.11;mongos://192.168.1.12;mongos://192.168.1.12;mongocfg://192.
↪\
    --vendor=mongodb \
    --provider-version='7.0' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --db-admin='admin' \
    --db-admin-passwd='MyS3cr3tPass' \
    --cluster-name='MongoDB ReplicaSet 7.0' \
    --wait
```

- Create a minimal 2-node MongoDB sharded cluster 6.0, using operating system user "root" and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=mongodb \
    --nodes=
↪"mongos://192.168.5.11:27017;mongocfg://192.168.5.11:27019;192.168.5.12:27018"␣
↪\
    --vendor=mongodb \
    --provider-version='6.0' \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='MongoDB Sharded Cluster 6.0 - minimal - devtest' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a 11-node MongoDB Sharded Cluster 7.0, with a 3-node config server replica set, a 6-node database nodes (2 replica sets, 3 nodes per shard) and 2-node mongos with operating system user "ubuntu" (the first node under `db_replica_set` will be the primary):

```
resource "clustercontrol_db_cluster" "my-mongodb-sharded-cluster" {

    db_cluster_create = true
    db_cluster_name = "MongoDB Sharded Cluster Set 7.0"
    db_cluster_type = "mongodb"
    db_version = "7.0"
    db_vendor = "mongodb"
    db_admin_user_password = var.cc_api_user_password
    db_config_server {
```

(continues on next page)

(continued from previous page)

```
        rs = "rs_config"
        member {
            hostname = "192.168.10.201"
        }
        member {
            hostname = "192.168.10.202"
        }
        member {
            hostname = "192.168.10.203"
        }
    }

    db_mongos_server {
        hostname = "192.168.10.101"
    }

    db_mongos_server {
        hostname = "192.168.10.102"
    }

    db_replica_set {
        rs = "rs1"
        member {
            hostname = "192.168.10.111"
        }
        member {
            hostname = "192.168.10.112"
        }
        member {
            hostname = "192.168.10.113"
        }
    }

    db_replica_set {
        rs = "rs2"
        member {
            hostname = "192.168.10.121"
        }
        member {
            hostname = "192.168.10.122"
        }
        member {
            hostname = "192.168.10.123"
        }
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
```

(continues on next page)

(continued from previous page)

```
    disable_selinux = true
    db_install_software = true

}
```

2.10.11 Valkey Sentinel

Deploys new Valkey replication instances with Valkey Sentinel. A minimum of 3 nodes is recommended for high availability and automatic failover. The following vendors and versions are supported for a new deployment:

- Valkey Sentinel - 7 and 8
- Canonical - 7

For a very minimal setup (suitable for testing and experimenting), it is possible to deploy a single-node Valkey instance with Sentinel (both services co-located on the same node), and later can be scaled out with more Valkey database and Sentinel nodes.

Default configuration

By default, ClusterControl deploys Valkey instances with the following configurations:

- Valkey Sentinel (default port is 26379) will be co-located with the Valkey instances (default port is 6379).
- ClusterControl will configure the Valkey instance with `appendonly` enabled.
- ClusterControl will secure the instance with authentication enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the database nodes, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Valkey being killed by OOM.

!!! Tips

```
Valkey Sentinel requires 3 nodes for automatic primary promotion. Sentinel␣
↪can be co-located on the ClusterControl server if you want to deploy a␣
↪two-node Valkey replication cluster (Sentinel will be co-located on each␣
↪database instance).
```

Deployment steps

ClusterControl GUI

1. To create a new Valkey replication with Sentinel, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Valkey Sentinel".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.

5. You will be presented with a **Deploy Valkey Sentinel Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:

   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:

   - **Valkey port**: The database server port that ClusterControl shall configure on all Valkey database nodes.
   - **Valkey Sentinel port**: The database server port that ClusterControl shall configure on all Valkey Sentinel nodes.

- **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
- **Password**: The Valkey admin password that ClusterControl shall configure on all database nodes. The value will be used to configure the `requirepass` and `masterauth` options.
- **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.

9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for a replication with Sentinel configuration.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify two or more replica nodes. If this field is left blank, ClusterControl will deploy the primary node as a standalone node.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node Valkey Sentinel 8.0, with operating system user "ubuntu" (the first node is the primary), where the admin user password will be auto-generated, let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=redis \
    --nodes=
↪"redis://10.0.0.11;redis://10.0.0.12;redis://10.0.0.13;redis-sentinel://10.0.0.11;
↪\
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --vendor=valkey \
    --provider-version=8  \
    --log \
    --cluster-name="My Valkey Sentinel v8"
```

- Create a standalone Valkey server (with Sentinel) 7.0, using operating system user "root" and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=redis \
    --nodes="redis://10.0.0.11;redis-sentinel://10.0.0.11" \
    --vendor=valkey \
    --provider-version=7 \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='Valkey 7 - standalone - devtest' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a three-node Valkey with Sentinel 7.0, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-valkey-sentinel" {

    db_cluster_create = true
    db_cluster_name = "Valkey Sentinel 7"
    db_cluster_type = "redis"
    db_version = "7"
    db_vendor = "valkey"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
    db_host {
        hostname = "10.10.10.12"
    }
    db_host {
        hostname = "10.10.10.13"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
```

(continues on next page)

(continued from previous page)

```
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

2.10.12 Redis Sentinel

Deploys new Redis replication instances with Redis. A minimum of 3 nodes is recommended for high availability and automatic failover. The following vendors and versions are supported for a new deployment:

- Redis Sentinel - 6, 7 and 8

For a very minimal setup (suitable for testing and experimenting), it is possible to deploy a single-node Redis instance with Sentinel (both services co-located on the same node), and later can be scaled out with more Redis database and Sentinel nodes.

Info

ClusterControl and Severalnines do not use or reproduce the official Redis® logo, which is a registered trademark of Redis Ltd. To comply with Redis trademark and branding guidelines, we have introduced our own distinct branding for Redis-related features within ClusterControl. This logo and name are original creations of Severalnines and are used solely as internal branding to represent Redis.

Default configuration

By default, ClusterControl deploys Redis instances with the following configurations:

- Redis Sentinel (default port is 26379) will be co-located with the Redis instances (default port is 6379).
- ClusterControl will configure the Redis instance with `appendonly` enabled.
- ClusterControl will secure the instance with authentication enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the database nodes, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.

!!! Tips

```
Redis Sentinel requires 3 nodes for automatic primary promotion. Sentinel␣
↪can be co-located on the ClusterControl server if you want to deploy a␣
↪two-node Redis replication cluster (Sentinel will be co-located on each␣
↪database instance).
```

Deployment steps

ClusterControl GUI

1. To create a new Redis replication with Sentinel, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Redis Sentinel".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy Redis Sentinel Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
    - **Redis port**: The database server port that ClusterControl shall configure on all Redis database nodes.
    - **Redis Sentinel port**: The database server port that ClusterControl shall configure on all Redis Sentinel nodes.
    - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
    - **Password**: The Redis admin password that ClusterControl shall configure on all database nodes. The value will be used to configure the `requirepass` and `masterauth` options.
    - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for a replication with Sentinel configuration.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify two or more replica nodes. If this field is left blank, ClusterControl will deploy the primary node as a standalone node.

    Note

    Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a three-node Redis Sentinel 7.0, with operating system user "ubuntu" (the first node is the primary), where the admin user password will be auto-generated, let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=redis \
    --nodes=
↪"redis://10.0.0.11;redis://10.0.0.12;redis://10.0.0.13;redis-sentinel://10.0.0.11;
↪\
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --vendor=redis \
    --provider-version=7  \
    --log \
    --cluster-name="My Redis Sentinel v7"
```

- Create a standalone Redis server (with Sentinel) 6.0, using operating system user "root" and let the deployment job run in the foreground:

```
s9s cluster --create \
    --cluster-type=redis \
    --nodes="redis://10.0.0.11;redis-sentinel://10.0.0.11" \
    --vendor=redis \
    --provider-version=6 \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='Redis 6 - standalone - devtest' \
    --wait
```

Terraform Provider for ClusterControl

- Deploy a three-node Redis Sentinel 7.0, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-redis-sentinel" {

    db_cluster_create = true
    db_cluster_name = "Redis with Sentinel 7"
    db_cluster_type = "redis"
    db_version = "7"
    db_vendor = "redis"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
```

(continues on next page)

(continued from previous page)

```
    db_host {
        hostname = "10.10.10.12"
    }
    db_host {
        hostname = "10.10.10.13"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

2.10.13 Valkey Cluster

Deploy a new Valkey Cluster. A minimum of 3 nodes is recommended for automatic failover. Valkey Cluster requires at least 3 nodes for a standard cluster setup with 3 shards. However, each shard should have its replica, hence 6 nodes are recommended (1 primary + 1 replica per shard) for production deployment:

- 2 nodes (1 primary + 1 replica) for shard 1
- 2 nodes (1 primary + 1 replica) for shard 2
- 2 nodes (1 primary + 1 replica) for shard 3

The following vendors and versions are supported for a new deployment:

- Valkey Cluster - 7 and 8
- Canonical - 7

Default configuration

By default, ClusterControl deploys Valkey cluster instances with the following configurations:

- ClusterControl will configure the Valkey instances with `appendonly` enabled.
- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Valkey being killed by OOM.

Deployment steps

ClusterControl GUI

1. To create a new Valkey Cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Valkey Cluster".

2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy Valkey Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

> **Tip**
>
> A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.
7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:

- **Valkey port**: The database server port that ClusterControl shall configure on all Valkey database nodes.
- **Cluster bus port**: The database server port that ClusterControl shall configure for Valkey cluster communication.
- **Node timeout (ms)**: The maximum amount of time a Valkey Cluster node can be unavailable, without it being considered as failing. If a master node is not reachable for more than the specified amount of time, it will be failed over by its replicas. This parameter controls other important things in the Valkey Cluster. Notably, every node that can't reach the majority of master nodes for the specified amount of time, will stop accepting queries.
- **Replica validity factor**: Specify the replication validity factor to consider a replica that is disconnected from the primary for more than **Node timeout** multiplied by this value. Set the factor to 0 to always consider a replica valid to failover. If the value is positive, a maximum disconnection time is calculated as the node timeout value multiplied by the factor provided with this option, and if the node is a replica, it will not try to start a failover if the master link was disconnected for more than the specified amount of time.
- **Configuration template**: The dropdown loads all Valkey configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
- **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
- **Username**: The Valkey admin username that ClusterControl shall configure on all database nodes. The value will be used to configure the `requirepass` and `masterauth` options.
- **Password**: The Valkey admin password that ClusterControl shall configure on all database nodes. The value will be used to configure the `requirepass` and `masterauth` options.

9. Click **Continue** to proceed to the next step.
10. Under the **Shard 1** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for a replication with Sentinel configuration.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node for this particular shard. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes. If no replica nodes, ClusterControl will configure the shard as a single primary setup.

- **Add another shard**: Add another shard section to define another replica set of the shard. A minimum of 3 shards is required. A similar **Shard** section will be presented where you can add another set of primary and replica nodes.
- **Remove this shard**: Discard the current shard.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a 6-node Valkey Cluster 7.0, with operating system user "root", let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=redis-sharded \
    --redis-port=6379 \
    --redis-bus-port=16379 \
    --vendor=valkey \
    --provider-version=7 \
    --node-timeout-ms=5000 \
    --replica-validity-factor=10 \
    --db-admin=admin \
    --db-admin-pass='Str0ngpa$$w0rd' \
    --nodes=
↪"redis-primary://192.168.0.101:6479;redis-replica://192.168.0.102:6479;redis-prima
↪\
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name="My Valkey Cluster v7" \
    --log
```

- Create a 6-node Valkey Cluster 7.0, with operating system user "ubuntu", where the admin user password will be auto-generated, let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=redis-sharded \
    --redis-port=6379 \
    --redis-bus-port=16379 \
    --vendor=valkey \
    --provider-version=7 \
```

(continues on next page)

(continued from previous page)

```
    --node-timeout-ms=5000 \
    --replica-validity-factor=10 \
    --db-admin=admin \
    --nodes=
↪"redis-primary://192.168.0.101:6479;redis-replica://192.168.0.102:6479;redis-prima
↪\
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name="My Valkey Cluster v7" \
    --log
```

2.10.14 Redis Cluster

Deploy a new Redis Cluster. A minimum of 3 nodes is recommended for automatic failover. Redis Cluster requires at least 3 nodes for a standard cluster setup with 3 shards. However, each shard should have its replica, hence 6 nodes are recommended (1 primary + 1 replica per shard) for production deployment:

- 2 nodes (1 primary + 1 replica) for shard 1
- 2 nodes (1 primary + 1 replica) for shard 2
- 2 nodes (1 primary + 1 replica) for shard 3

The following vendors and versions are supported for a new deployment:

- Redis Cluster - 6, 7 and 8

Info

ClusterControl and Severalnines do not use or reproduce the official Redis® logo, which is a registered trademark of Redis Ltd. To comply with Redis trademark and branding guidelines, we have introduced our own distinct branding for Redis-related features within ClusterControl. This logo and name are original creations of Severalnines and are used solely as internal branding to represent Redis.

Default configuration

By default, ClusterControl deploys Redis cluster instances with the following configurations:

- ClusterControl will configure the Redis instances with `appendonly` enabled.
- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.

Deployment steps

ClusterControl GUI

1. To create a new Redis Cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Redis Cluster".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy Redis Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

> **Tip**
>
> A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
    - **Redis port**: The database server port that ClusterControl shall configure on all Redis database nodes.
    - **Cluster bus port**: The database server port that ClusterControl shall configure for Redis cluster communication.
    - **Node timeout (ms)**: The maximum amount of time a Redis Cluster node can be unavailable, without it being considered as failing. If a master node is not reachable for more than the specified amount of time, it will be failed over by its replicas. This parameter controls other important things in the Redis Cluster. Notably, every node that can't reach the majority of master nodes for the specified amount of time, will stop accepting queries.
    - **Replica validity factor**: Specify the replication validity factor to consider a replica that is disconnected from the primary for more than **Node timeout** multiplied by this value. Set the factor to 0 to always consider a replica valid to failover. If the value is positive, a maximum disconnection time is calculated as the node timeout value multiplied by the factor provided with this option, and if the node is a replica, it will not try to start a failover if the master link was disconnected for more than the specified amount of time.
    - **Configuration template**: The dropdown loads all Redis or Valkey configuration template files under `/etc/cmon/templates` or `/usr/share/cmon/templates` of the ClusterControl node for this cluster type. Choose one configuration template for the deployment. See *Configuration Template*.
    - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
    - **Username**: The Redis admin username that ClusterControl shall configure on all database nodes. The value will be used to configure the `requirepass` and `masterauth` options.
    - **Password**: The Redis admin password that ClusterControl shall configure on all database nodes. The value will be used to configure the `requirepass` and `masterauth` options.
9. Click **Continue** to proceed to the next step.
10. Under the **Shard 1** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for a replication with Sentinel configuration.
    - **Replica nodes**: Specify the IP address or hostname of the replica database node for this particular shard. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the

target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes. If no replica nodes, ClusterControl will configure the shard as a single primary setup.

- **Add another shard**: Add another shard section to define another replica set of the shard. A minimum of 3 shards is required. A similar **Shard** section will be presented where you can add another set of primary and replica nodes.
- **Remove this shard**: Discard the current shard.

Note

Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a 6-node Redis Cluster 6.0, with operating system user "root", let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=redis-sharded \
    --redis-port=6379 \
    --redis-bus-port=16379 \
    --vendor=redis \
    --provider-version=6 \
    --node-timeout-ms=5000 \
    --replica-validity-factor=10 \
    --db-admin=admin \
    --db-admin-pass='Str0ngpa$$w0rd' \
    --nodes=
↪"redis-primary://192.168.0.101:6479;redis-replica://192.168.0.102:6479;redis-prima
↪\
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name="My Redis Cluster v6" \
    --log \
    --print-request
```

- Create a 6-node Redis Cluster 7.0, with operating system user "ubuntu", where the admin user password will be auto-generated, let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=redis-sharded \
```

(continues on next page)

(continued from previous page)

```
    --redis-port=6379 \
    --redis-bus-port=16379 \
    --vendor=redis \
    --provider-version=7 \
    --node-timeout-ms=5000 \
    --replica-validity-factor=10 \
    --db-admin=admin \
    --nodes=
↪"redis-primary://192.168.0.101:6479;redis-replica://192.168.0.102:6479;redis-prima
↪\
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name="My Valkey Cluster v7" \
    --log \
    --print-request
```

2.10.15 SQL Server

Deploys a new standalone or SQL Server AlwaysOn high availability setup. If only one database node is specified, ClusterControl will deploy it as standalone SQL Server. The following vendors and versions are supported for a new deployment:

- Microsoft SQL Server for Linux - 2019 and 2022

Warning

Attention

Only hostname or FQDN is supported. IP addresses are not accepted and will result in an error during the node addition process. Ensure proper host naming and mapping is configured beforehand, utilizing either `/etc/hosts` or a DNS resolver.

License

Deploying SQL Server with ClusterControl does not change how you must license SQL Server. Each node where SQL Server is running requires appropriate licensing, just as it would if you deployed SQL Server manually. The presence of ClusterControl as an orchestration or management layer has no bearing on Microsoft's licensing rules. In a new deployment scenario, ClusterControl configures the SQL Server instance with the Evaluation edition, which is free to use for 180 days.

When deploying SQL Server in a clustered environment, licensing rules still apply to each server running SQL Server. However, the specifics differ depending on whether the node is actively running SQL Server or is a passive failover node:

- Active Nodes: Each active node that runs SQL Server services needs to be fully licensed.
- Passive Failover Nodes: Under current SQL Server licensing rules, you can typically have one passive failover node per licensed instance without needing an additional license for that passive node. The passive node is considered a "standby" server that can take over if the active server fails. It must truly remain passive, running no active workloads except those required for maintaining the standby state.

Obtain your SQL Server license directly from Microsoft, a certified Microsoft Partner, or a cloud marketplace (if applicable).

Default configuration

By default, ClusterControl deploys SQL Server with the following configurations:

- All database nodes must use fully qualified domain name (FQDN). IP address is not supported.
- Minimum available RAM is 1800 MB per database node.
- Enforces the SQL Server user's password policy as shown here.
- At the moment, ClusterControl only deploys AlwaysOn with asynchronous-commit mode, where it does not wait for any secondary replica to write incoming transaction log records to disk.
- ClusterControl will set up the SQL Server using the Evaluation edition (free, 180-day limit).

Deployment steps

ClusterControl GUI

1. To create a new SQL Server, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "SQL Server".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy SQL Server Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.

- **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
- **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Admin username**: Specify the SQL Server admin username.
   - **Admin password**: The password for **Admin username**. The password must comply to SQL Server password policy. See Password Policy. Click on the eye icon to view the unmasked password. Copy this value to somewhere safe, or you may generate a new one. It is recommended that you use the generated password as it meets the minimum requirements.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
   - **Enable SSL encryption**: Toggle on (default) will instruct ClusterControl to configure database SSL encryption for both client-server and replication. Toggle off will make ClusterControl skip this step.
9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Primary node**: Specify hostname or FQDN of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for SQL Server AlwaysOn availability group configuration.
    - **Replica nodes**: Specify hostname or FQDN of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network

interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes. If no replica nodes, ClusterControl will configure the cluster as a standalone setup. You can add up to 8 replica nodes per availability group.

Note

Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

12. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a 3-node SQL Server 2022 with Always On availability group, with operating system user "root", let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=mssql_ao_async \
    --vendor=microsoft \
    --provider-version=2022 \
    --db-admin=SQLServerAdmin \
    --db-admin-pass='NqR$feGR68xli^#v' \
    --nodes=
↪"mssql://dsa-mssql-db1:1433;dsa-mssql-db2:1433;dsa-mssql-db3:1433;" \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name="MSSQL Server 2022 Always On" \
    --log
```

- Create a standalone SQL Server 2019, with operating system user "ubuntu", let the deployment job run in the foreground and show the job logs:

```
s9s cluster --create \
    --cluster-type=mssql_single \
    --vendor=microsoft \
    --provider-version=2019 \
    --db-admin=SQLServerAdmin \
    --db-admin-pass='NqR$feGR68xli^#v' \
    --nodes="mssql://dsa-mssql-db1:1433" \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name="Standalone SQL Server 2019" \
    --log
```

Terraform Provider for ClusterControl

- Deploy a three-node replica set of SQL Server 2022, with operating system user "ubuntu" (the first node will be the primary):

```
resource "clustercontrol_db_cluster" "my-mssql-2022" {
    db_cluster_create = true
    db_cluster_name = "Microsoft SQL Server 2022"
    db_cluster_type = "mssql_ao_async"
    db_version = "2022"
    db_vendor = "microsoft"
    db_admin_user = "SQLServerAdmin"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "10.10.10.11"
    }
    db_host {
        hostname = "10.10.10.12"
    }
    db_host {
        hostname = "10.10.10.13"
    }
    ssh_key_file = "/home/ubuntu/.ssh/id_rsa"
    ssh_user = "ubuntu"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true
}
```

2.10.16 Elasticsearch

Deploys a new Elasticsearch single-node cluster or high-availability cluster setup. For a high-availability cluster setup, 3 nodes are required with 3 eligible masters and at least 2 data nodes (co-located with the masters). The following vendor and versions are supported for a new deployment:

- Elastic - 7.1, 8.1 and 8.3

Default configuration

By default, ClusterControl deploys Elasticsearch with the following configurations:

- For high-availability cluster setup, ClusterControl will configure an NFS server on one of the Elasticsearch nodes and mount the shared filesystem on all data nodes. This is for snapshot backup and restoration.
- For single-node cluster setup, ClusterControl will create a local path for snapshot backup and restoration.

Warning

Attention

The minimum memory requirement for an Elasticsearch master node, data node, or master-data node is 1576 MB. ClusterControl will abort the deployment job if this requirement is not met. See Hardware Requirement for Elasticsearch.

Deployment steps

ClusterControl GUI

1. To create a new Elasticsearch cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Elasticsearch".
2. Choose a vendor from the **Vendor** dropdown. This will determine the version and package repository to use in the deployment.
3. Choose a major database version from the **Version** dropdown.
4. Click **Continue**.
5. You will be presented with a **Deploy Elasticsearch Cluster** deployment wizard. Specify the cluster name and tags. For cluster name, you may leave it blank and ClusterControl will generate a name for the cluster. For cluster tags, you can add multiple tags by pressing ++enter++ or choose from any existing tags and click **Continue** to the next step.

   Tip

   A proper tag configuration will help you group your database clusters and is commonly recommended in a large deployment.

6. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must physically secured and exist on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the database node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled,

ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.

- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

7. Click **Continue** to proceed to the next step.
8. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **HTTP port**: Elasticsearch HTTP port. The default is 9200 and not configurable.
   - **Transfer port**: Elasticsearch transfer port. The default is 9200 and not configurable.
   - **Admin user**: Specify the Elasticsearch admin username.
   - **Admin password**: Specify the password for **Admin user**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enteprise databases) and ClusterControl will skip the repository configuration part. Choose "Create new repository" to let ClusterControl create a local repository for the chosen vendor on the ClusterControl node.
   - **Enable SSL encryption**: This is not configurable as Elasticsearch enforces SSL encryption for high-availability cluster setup. If this is a single-node cluster setup, SSL encryption will be disabled automatically.
9. Click **Continue** to proceed to the next step.
10. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
    - **Eligible master**: Specify IP address, hostname, or FQDN of the eligible master node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for SQL Server AlwaysOn availability group configuration.
      - **Use as data node** - Toggle ON to configure the master as a data node. The role will be master-data. If OFF, the node will be configured as master only. You may add more data nodes under the ***Data nodes*** section.
    - **Data nodes**: IP address, hostname, or FQDN of the data node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes. If no replica nodes, ClusterControl will configure the server as a single-node cluster setup.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 11. Click **Continue** to proceed to the next step.

12. Under the **Snapshot storage configuration** section, you can configure Elasticsearch backup-related storage configuration:
    - **Repository name**: Specify the snapshot storage repository name.
    - **Storage host**: Host with physical file system storing snapshots and sharing with other cluster's nodes.
    - **Default storage location**: Location of the shared filesystem used to store and retrieve snapshots. This location will be registered in the `path.repo` setting on all master and data nodes in the cluster.
    - **Configure shared filesystem**: Toggle on to let ClusterControl install and configure the NFS shared filesystem on the chosen ***Storage host*** and ***Default storage location*** as the default cluster's snapshot repository. Toggle off to skip this configuration part. Users are responsible for configuring the NFS shared filesystem on the chosen ***Storage host*** and ***Default storage location***. The deployment job will fail if a shared filesystem is not configured.
13. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.
14. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

2.11 Import Database Cluster

The Import Database Cluster feature in ClusterControl enables users to bring existing database clusters under management with minimal effort.

With Import Database Cluster, you can:

- Easily connect single-node or multi-node database clusters to ClusterControl.
- Automatically detect and map existing topologies and replication setups.
- Apply custom configuration templates for unified management.
- Enable encryption and other security features for enhanced protection.

This feature streamlines the onboarding process, allowing database administrators to gain full visibility and control over existing infrastructure without the need for re-deployment. Whether you're consolidating environments or centralizing management, ClusterControl ensures consistency, security, and operational best practices.

2.11.1 Support Matrix

The following database cluster types, vendors and topology are supported:

Database	Vendor	Topology
MySQL	Percona, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
MySQL Group Replication	Oracle	Single-primary, multi-primary
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication, logical replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.11.2 Prerequisites

There are prerequisites that need to be fulfilled prior to the deployment for all database clusters:

- Make sure the target database nodes are running on a supported architecture platform and operating system. See *Hardware* and *Operating System*.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to all database nodes. See *SSH Key-based Authentication*.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.
- The target cluster must be in healthy state and not in a degraded state. For example, if you have a three-node Galera cluster, all nodes must be alive, accessible, and in sync.

Note

Note For more details, refer to the *Requirements* section. Each time you import an existing cluster or server, ClusterControl will trigger a job under ***ClusterControl GUI → Activity Center → Jobs***. You can see the progress and status on this page. Click on **…** of the importing job and click **Details**. A window will also appear with messages showing the progress.

2.11.3 MySQL Replication

ClusterControl can manage and monitor existing MySQL and MariaDB servers, whether standalone or in a replication setup. A minimum of two nodes is required for MySQL/MariaDB replication. If only one database IP address or hostname is provided, ClusterControl will deploy it as a standalone MySQL/MariaDB server with binary log enabled.

When adding individual hosts, listing them together will place them in the same server group. For this to work, ClusterControl expects all instances within a group to use the same MySQL root password. It will also automatically try to identify each server's role (primary, replica, multi, or standalone).

When importing an existing MySQL Replication, ClusterControl will do the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (primary, replica, multi, standalone).
- Pull the configuration files.
- Generate the authentication key and register the nodes into ClusterControl.

Default configuration

By default, ClusterControl deploys MySQL/MariaDB replication with the following configurations:

- MySQL GTID with `log_slave_updates` enabled (MySQL and Percona only).
- MariaDB GTID with `log_slave_updates` enabled (MariaDB only).
- All database nodes will be configured with `read_only=ON` and `super_read_only=ON` (if supported) for protection against accidental write. The chosen primary will be promoted by disabling the read-only in the runtime.
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- ClusterControl will create and grant necessary privileges for the following database users:

```
| Database user      | Purpose                                          |
| ------------------ | ------------------------------------------------ |
| `cmon`             | Management and automation.                       |
| `cmonexporter`     | Prometheus exporter for database monitoring.     |
| `cmonagent`        | ClusterControl query monitoring agent.           |
| `cmon_replication` | MySQL/MariaDB replication.                       |
| `backupuser`       | Backup and restore management.                   |
```

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

ClusterControl sets `read_only=ON` on all slaves but a privileged user (SUPER) can still write to a slave (except for MySQL versions that support `super_read_only`).

Import steps

ClusterControl GUI

1. To import an existing MySQL replication cluster, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "MySQL Replication".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.

- **Tags**: Add tags to search or group your database clusters

3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
   - **Admin/Root user**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPER privilege and the GRANT option for localhost only.
   - **Admin/Root password**: The password for **Admin/Root user**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
   - **information_schema queries**: By default, this is set to off. When enabled, this shall allow cmon or ClusterControl to query from information_schema schema to collect metrics from your MySQL database nodes.
   - **Cluster auto-recovery**: By default, this is set to off. Once enbled, the cluster auto-recovery will be enabled allowing for degraded or failing cluster will be attempted to be recovered by ClusterControl.
   - **Node auto-recovery**: By default, this is set to off. Once enbled, the node auto-recovery will be enabled allowing for failing nodes such as accidentally terminated system process will be tried to be started to get back to the cluster.
7. Click **Continue** to proceed to the next step.

8. Under the **Add nodes** section, you can specify your existing target database nodes and you are allowed to add more than one node depending on your existing MySQL Replication cluster you need to register to ClusterControl. Since this is an existing database cluster which means it is already setup, ClusterControl will be able to identify and scan the type of topology for your database cluster.

    - **Node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

    Note

    Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a three-node Oracle MySQL Replication 8.0 cluster, with operating system user "ubuntu". Regardless of the order, ClusterControl will be able to identify the primary node(s) through `read_only` and `super_read_only` variables:

```
s9s cluster --register \
    --cluster-type=mysqlreplication \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=oracle \
    --provider-version=8.0 \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name='PROD - MySQL Replication 8.0' \
    --wait \
    --log
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.11.4 MySQL Galera

ClusterControl can manage and monitor existing Galera Clusters, assuming all instances in the group use the same MySQL root password. Adding *asynchronous slaves* and *adding load balancers* connected to the Galera Cluster should be specified and imported separately after the cluster is imported into ClusterControl.

When importing an existing Galera Cluster, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Detect the remaining nodes in the cluster, only if Automatic node discovery is ON.
- Pull the configuration files.
- Generate the authentication key and register the nodes into ClusterControl.

A minimal setup is comprised of one Galera node (no high availability, but this can later be scaled with more nodes). However, a minimum of 3 nodes is recommended for high availability. Garbd (an arbitrator) can be added later after the deployment completes.

The following vendors and versions are supported for importing your existing cluster:

- Percona XtraDB Cluster - 8.0 and 8.4.
- Percona XtraDB Cluster Pro - 8.4.
- MariaDB Cluster - 10.4, 10.5, 10.6, 10.11 (LTS) and 11.4 (LTS).

Default configuration

By default, ClusterControl deploys Galera Cluster with the following configurations:

- Use `xtrabackup-v2` or `mariabackup` (depending on the vendor chosen) for `wsrep_sst_method`.
- Binary logging is enabled.
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	MySQL/MariaDB replication.
`backupuser`	Galera SST, backup and restore management.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Import steps

ClusterControl GUI

1. To import an existing MySQL Galera Cluster, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "MySQL Galera".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes. If the path does not exist, ClusterControl will create and configure it automatically.
   - **Admin/Root user**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPER privilege and the GRANT option for localhost only.
   - **Admin/Root password**: The password for **Admin/Root user**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
   - **information_schema queries**: By default, this is set to off. When enabled, this shall allow cmon or ClusterControl to query from information_schema schema to collect metrics from your MySQL database nodes.

- **Cluster auto-recovery**: By default, this is set to off. Once enbled, the cluster auto-recovery will be enabled allowing for degraded or failing cluster will be attempted to be recovered by ClusterControl.
- **Node auto-recovery**: By default, this is set to off. Once enbled, the node auto-recovery will be enabled allowing for failing nodes such as accidentally terminated system process will be tried to be started to get back to the cluster.
- **Automatic node discovery**: By default, this is set to on. Set this to off if `wsrep_node_incoming_address=AUTO` for all of your primary Galera nodes. If your ClusterControl controller node uses different IPs/hostnames to connect to Galera Cluster nodes for inter-node connectivity, then set it to off (or disable) Automatic Node Discovery and pass IPs/hostnames that ClusterControl can use to connect to Galera nodes.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify your existing target database nodes and you are allowed to add more than one node depending on your existing MySQL Galera Cluster you need to register to ClusterControl. Since this is an existing database cluster which means it is already setup, ClusterControl will be able to identify and scan the type of topology for your database cluster.
   - **Node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a three-node Percona XtraDB Cluster 8.0, with operating system user "ubuntu":

```
s9s cluster --register \
    --cluster-type=galera \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=percona \
    --provider-version=8.0 \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name='PROD - MySQL Replication 8.0' \
```

(continues on next page)

(continued from previous page)

```
    --wait \
    --log
```

- Create a three-node MariaDB Cluster 11.4 and let the deployment job run in the foreground:

```
s9s cluster --register \
    --cluster-type=galera \
    --nodes="192.168.5.11;192.168.5.12;192.168.5.13" \
    --vendor=mariadb \
    --provider-version=11.4 \
    --db-admin-passwd='Str0ngpa$$w0rd' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --cluster-name='System A - MariaDB Cluster 11.4' \
    --wait \
    --log
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.11.5 PostgreSQL Streaming

ClusterControl can manage and monitor existing PostgreSQL or TimeScaleDB clusters running streaming Replication version 12 or later. When importing hosts, those listed together will form a server group in the user interface. It is expected that all instances within the same group utilize the same database administrator password.

A minimum of two nodes is required for PostgreSQL streaming replication. If only one database IP address or hostname is provided, ClusterControl will deploy it as a standalone PostgreSQL server. The following vendors and versions are supported for a importing your existing cluster:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, 16 and 17.
- PostgreSQL from EDB repository - 12, 13, 14, 15, 16 and 17 (requires a valid EDB repository token).

When importing an existing PostgreSQL or TimescaleDB Streaming Replication, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (master, slave, TimescaleDB extension).
- Pull the configuration files.

- Generate the authentication key and register the nodes into ClusterControl.

Default configuration

By default, ClusterControl deploys PostgreSQL instances with the following configurations:

- Configure and load the `pg_stat_statements` module.
- The WAL level is set to `replica`.
- All replica will be set with `hot_standby`.
- Configure the `cluster_name` value.
- ClusterControl will configure the PostgreSQL instance with SSL encryption for client-server connections.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.
`backupuser`	Backup and restore management.

Import steps

ClusterControl GUI

1. To import an existing PostgreSQL streaming replication, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "PostgreSQL Streaming".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.

- **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.

5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
   - **Password**: The password for **User**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
   - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
   - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note If you have more than one secondary or replica nodes, make sure to add more under **Replica nodes** text field. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a three-node PostgreSQL 16 streaming replication, with operating system user "ubuntu" (the first node is the primary) with tags `production` and `postgres`. Then wait until the job is done while showing logs during the deployment process:

```
s9s cluster --register \
    --cluster-type=postgresql \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=postgresql \
    --provider-version=16 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --with-tags='production;postgres' \
    --cluster-name='PostgreSQL 16 - Streaming Replication' \
                --wait --log
    ```

- Create a four-node PostgreSQL EDB 16 streaming replication, using operating system user "root" (the first node is the primary) and let the deployment job run in the foreground:

    ```bash
    s9s cluster --register \
        --cluster-type=postgresql \
        --nodes="192.168.5.11;192.168.5.12;192.168.5.13;192.168.5.14" \
        --vendor=enterprisedb \
        --provider-version=11.4 \
        --db-admin='mydbuser' \
        --db-admin-passwd='mydbPassw0rd' \
        --enterprise-token='XXXXXXXXXXXXXXXXXXXXXXXXXX' \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --cluster-name='System A - PostgreSQL EDB 16 - Streaming Replication' \
        --wait
    ```
```

**Recommended Next Steps**

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*
```

2.11.6 TimescaleDB

Imports a new TimescaleDB standalone or streaming replication cluster from ClusterControl. A minimum of two nodes is required for TimescaleDB streaming replication. If only one database IP address or hostname is provided, ClusterControl will deploy it as a standalone TimescaleDB server. The following vendors and versions are supported for a importing your existing cluster:

- TimescaleDB - 12, 13, 14, 15, 16 and 17.

Default configuration

By default, ClusterControl deploys TimescaleDB with the following configurations:

- Configure and load the `pg_stat_statements` module and `timescaledb` extension.
- The WAL level is set to `replica`.
- All replica will be set with `hot_standby`.
- Configure the `cluster_name` value.
- ClusterControl will configure the TimescaleDB instance with SSL encryption for client-server connections.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.
`backupuser`	Backup and restore management.

Note

Note You can also import an existing PostgreSQL and convert it to TimescaleDB at a later stage. However, this action will be irreversible and ClusterControl will treat the cluster as TimescaleDB onwards.

Import steps

ClusterControl GUI

1. To import an existing TimescaleDB streaming replication, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "TimescaleDB".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:

- **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
- **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
- **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
- **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
- **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.

5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
   - **Password**: The password for **User**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
   - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
   - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note If you have more than one secondary or replica nodes, make sure to add more under **Replica nodes** text field. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the

summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a three-node TimescaleDB 16 streaming replication, with operating system user "ubuntu" (the first node is the primary) with tags `production` and `timescaledb`:

```
s9s cluster --register \
    --cluster-type=postgresql \
    --nodes="10.10.10.11;10.10.10.12;10.10.10.13" \
    --vendor=postgresql \
    --provider-version=16 \
    --db-admin='mydbuser' \
    --db-admin-passwd='mydbPassw0rd' \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --with-tags='production;timescaledb' \
    --cluster-name='TimescaleDB 16 - Streaming Replication' \
    --with-timescaledb
```

- Create a standalone TimescaleDB 15, with operating system user "root" with tags `test` and `timescaledb`, and wait until the job finishes:

```
s9s cluster --create \
    --cluster-type=postgresql \
    --nodes="192.168.99.11" \
    --vendor=postgresql \
    --provider-version=15 \
    --db-admin='postgres' \
    --db-admin-passwd='mYpa$$word' \
    --os-user=root \
    --os-key-file=/root/.ssh/id_rsa \
    --with-tags='test;timescaledb' \
    --cluster-name='TimescaleDB 16 - standalone' \
    --with-timescaledb \
    --wait
```

2.11.7 MongoDB Replica Set

ClusterControl is able to manage and monitor an existing MongoDB or Percona Server for MongoDB Replica Set.

When importing an existing MongoDB ReplicaSet, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (primary, secondary, arbiter).

- Pull the configuration files.
- Generate the authentication key and register the nodes into ClusterControl.

The database cluster will be automatically added to ClusterControl once import process is deployed successfully. If only one node is specified, ClusterControl will deploy it as a standalone MongoDB node. The following vendors and versions are supported for importing existing cluster:

- MongoDB Community - 4.4, 5.0, 6.0, 7.0 and 8.0
- MongoDB Enterprise - 4.4, 5.0, 6.0, 7.0 and 8.0
- Percona Server for MongoDB - 4.4, 5.0, 6.0, 7.0 and 8.0

Warning

Attention

If SSL/TLS is required, ClusterControl only supports a proper CAFile configuration, as shown in the MongoDB documentation, and does not support the `--allowInvalidCertificates` flag.

Default configuration

By default, ClusterControl deploys MongoDB Replica Set members with the following configurations:

- Configure `setParameter.enableLocalhostAuthBypass: true` inside MongoDB configuration file.
- ClusterControl will create and grant necessary roles for additional MongoDB users - `admin.cmon_backup` for backup and restore purposes and `admin.cmonexporter` for query monitoring.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

It is possible to deploy only 2 MongoDB nodes (without an arbiter). The caveat of this approach is no automatic failover. If the primary node goes down then manual failover is required to make the other server run as primary. Automatic failover works fine with 3 nodes and more.

Import steps

ClusterControl GUI

1. To import an existing MongoDB replica set, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose “MongoDB ReplicaSet”.
2. Under **Cluster details**, specify the cluster details you want to assign:
    - **Vendor**: This is under **Vendor and version** label in the panel. This is the kind of vendor that your database cluster binaries for MongoDB ReplicaSet is deployed. This is required and you need to choose from *Percona*, *MongoDB*, and *MongoDB Enterprise*.
    - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.

- **Tags**: Add tags to search or group your database clusters

3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
   - **Password**: The password for **User**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify the target database nodes and configure the database topology that you want to deploy:
   - **Node**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You

can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a 2-node MongoDB Replica Set using Percona binaries, you only need to specify the primary node. For example, if you have primary running on 192.168.40.190 and you have replica running on 192.168.40.191, no need to specify the replica as it will be auto-detected and will be imported by ClusterControl as well. The following command demonstrates this using `ubuntu` as it's OS username, using `ed25519` digital signature algorihtm and wait then print the logs of the job deployment:

```
        s9s cluster --register \
                --cluster-type=mongodb \
                --nodes="192.168.40.190" \
                --vendor=percona \
                --provider-version='7.0' \
                --os-user=ubuntu \
                --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
                --db-admin='mydbuser' \
                --db-admin-passwd='mydbPassw0rd' \
                --cluster-name='MongoDB Percona Server ReplicaSet 7.0' \
                --wait --log
    ```

- To import a three-node MongoDB Enterprise Replica Set 7.0, with operating system user "root" and let the deployment job run in the foreground:

    ```bash
    s9s cluster --register \
        --cluster-type=mongodbenterprise \
        --nodes="192.168.40.155" \
        --vendor=mongodbenterprise \
        --provider-version='7.0' \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --db-admin='mydbuser' \
        --db-admin-passwd='mydbPassw0rd' \
        --cluster-name='MongoDB Enterprise ReplicaSet 7.0'
    ```
```

## 2.11.8 MongoDB Sharded Cluster

ClusterControl is able to manage and monitor an existing MongoDB, Percona Server for MongoDB or MongoDB Enterprise 4.x ,5.x, 6.x, 7.x and 8.x sharded cluster setup. When importing an existing MongoDB Sharded Cluster, ClusterControl will perform the following:
```

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (mongos, config server, shard server - primary, secondary, arbiter).
- Pull the configuration files.
- Generate the authentication key and register the nodes into ClusterControl.

The following vendors and versions are supported for importing existing cluster:

- MongoDB Enterprise - 4.4, 5.0, 6.0, 7.0 and 8.0
- MongoDB Community - 4.4, 5.0, 6.0, 7.0 and 8.0
- Percona Server for MongoDB - 4.4, 5.0, 6.0, 7.0 and 8.0

Default configuration

By default, ClusterControl deploys MongoDB Sharded Cluster with the following configurations:

- Configure `setParameter.enableLocalhostAuthBypass: true` inside MongoDB configuration file.
- ClusterControl will create and grant necessary privileges for an additional MongoDB user - `admin.cmon_backup` for backup and restore purposes.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

MongoDB Sharded Cluster does not support `mongodump` backup method. Users will be asked to install Percona Backup for MongoDB (PBM) when creating or scheduling a backup for this cluster type after the deployment completes. Note that PBM requires a shared remote backup storage (e.g, NFS) accessible on all MongoDB nodes.

Import steps

ClusterControl GUI

1. To import an existing MongoDB Sharded Cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "MongoDB Shards".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Vendor**: This is under **Vendor and version** label in the panel. This is the kind of vendor that your database cluster binaries for MongoDB ReplicaSet is deployed. This is required and you need to choose from *Percona*, *MongoDB*, and *MongoDB Enterprise*.
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.

4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
    - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
    - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
    - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
    - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
    - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
    - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
    - **User**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
    - **Password**: The password for **User**.
    - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you will see the label **Router/Mongos servers**. This entails that you only have to specify the router/mongos server nodes. This is where you can specify the target database nodes that you want to deploy:
    - **Node router server**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note Specify only the MongoDB router servers (mongos) and ClusterControl will discover the rest of the config and shards members. If you have multiple MongoDB router servers, make sure to add them here if it's more than one node. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the

cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a MongoDB Shard Cluster using Percona binaries consisting of 3-node router nodes using `ubuntu` as it's OS username, using `ed25519` digital signature algorihtm and wait then print the logs of the job deployment:

```
        s9s cluster --register \
                --cluster-type=mongodb \
                --
↪nodes="mongos://192.168.1.11;mongos://192.168.1.12;mongos://192.168.1.12;"
                --vendor=percona \
                --provider-version='7.0' \
                --os-user=ubuntu \
                --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
                --db-admin='mydbuser' \
                --db-admin-passwd='mydbPassw0rd' \
                --cluster-name='MongoDB Percona Server Shards 7.0' \
                --wait --log
    ```
```

### 2.11.9 Valkey Sentinel

Import existing Valkey instances with Sentinel for v7 and v8. When importing existing Valkey instances, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host fenvironment and operating system.
- Discover the database role of each node (primary, replica, sentinel).

**Warning**

Attention

ClusterControl does not support importing Valkey instances without authentication. Authentication must be enabled and the provided admin username and password must have `~* +@all` (all keys, all commands) privilege.

The following vendors and versions are supported for a importing existing cluster:

- Valkey Sentinel - 7 and 8
- Canonical - 7
```

For a very minimal setup (suitable for testing and experimenting), it is possible to import a single-node Valkey instance with Sentinel (both services co-located on the same node), and later can be scaled out with more Valkey database and Sentinel nodes.

Default configuration

By default, ClusterControl deploys Valkey instances with the following configurations:

- Valkey Sentinel (default port is 26379) will be co-located with the Valkey instances (default port is 6379).
- ClusterControl will configure the Valkey instance with `appendonly` enabled.
- ClusterControl will secure the instance with authentication enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the database nodes, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Valkey being killed by OOM.

!!! Tips

```
Valkey Sentinel requires 3 nodes for automatic primary promotion. Sentinel␣
↪can be co-located on the ClusterControl server if you want to deploy a␣
↪two-node Valkey replication cluster (Sentinel will be co-located on each␣
↪database instance).
```

Import steps

ClusterControl GUI

1. To import a new Valkey Sentinel cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Valkey Sentinel".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.

- **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.

5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Valkey Port**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **Valkey Sentinel Port**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **Repository**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **Password**: Under the **Authentication** pane, this is the primary node password used for client/replica authentication.
   - **Replication user**: Under the **Authentication** pane, this is the replication user of your Valkey replication cluster.
   - **Replication password**: Under the **Authentication** pane, this is the replication password of your Valkey replication cluster.
   - **Sentinel password**: The password for your Sentinel user `sentinel-user`
   - **Use existing certificates**: Specify cmon's certificates directory to use on imported cluster. If ssl configuration is not completed scale up operations may fail.
   - **Cluster auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling cluster recovery enabled. Otherwise, off will not enable cluster recovery.
   - **Node auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling node recovery enabled. Otherwise, off will not enable node recovery.
7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, is where you can add your database nodes:
   - **Primary node**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.
   - **Replica nodes**: Only specify the IP address or hostname of your replica node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note If you have more than one replica nodes, make sure to add more under **Replica nodes** text field. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a Valkey Sentinel Cluster consisting of 3-nodes using `ubuntu` as it's OS username, using sentinel password as `sjl1215wi` and using `ed25519` digital signature algorihtm and wait then print the logs of the job deployment:

```
s9s cluster --register --cluster-type=valkey \
    --nodes="valkey://192.168.40.190:6379;␣
↪valkey://192.168.40.191:6379;valkey://192.168.40.192:6379; \
            ␣
↪valkey-sentinel://192.168.40.190:26379;valkey-sentinel://192.168.40.191:26379;␣
↪\
             valkey-sentinel://192.168.40.192:26379;" \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
    --sentinel-passwd sjl2l5wi\
    --vendor=valkey
    --provider-version=8  \
    --cluster-name="My Valkey Sentinel v8" \
    --log --wait
```

2.11.10 Valkey Cluster

Import existing Valkey Cluster instances for v7 and v8. When importing existing Valkey instances, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (primary, replica).

> **Warning**
>
> Attention
>
> ClusterControl does not support importing Valkey instances without authentication. Authentication must be enabled and the provided admin username and password must have `~* +@all` (all keys, all commands) privilege.

The following vendors and versions are supported for a importing existing cluster:

- Valkey Cluster - 7 and 8
- Canonical - 7

Default configuration

By default, ClusterControl deploys Valkey cluster instances with the following configurations:

- ClusterControl will configure the Valkey instances with `appendonly` enabled.
- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Valkey being killed by OOM.

Import steps

ClusterControl GUI

1. To import an existing Valkey Cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Valkey Cluster".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.

- **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
- **Username**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
- **Password**: The password for **User**.
- **Cluster auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling cluster recovery enabled. Otherwise, off will not enable cluster recovery.
- **Node auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling node recovery enabled. Otherwise, off will not enable node recovery.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you will see the label **Router/Mongos servers**. This entails that you only have to specify the router/mongos server nodes. This is where you can specify the target database nodes that you want to deploy:
   - **Cluster node**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note Specify one node and ClusterControl will discover the rest of members. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a Valkey Sharded Cluster, you only need to specify the primary node. For example, if you have primary running on 192.168.40.198 and you have replica running on 192.168.40.199, no need to specify the replica as it will be auto-detected and will be imported by ClusterControl as well. The following command demonstrates this using `ubuntu` as it's OS username, using `ed25519` digital signature algorihtm and wait then print the logs of the job deployment:

```
      s9s cluster --register \
              --cluster-type=valkey_sharded \
              --nodes="valkey://192.168.40.198;" \
              --os-user=ubuntu \
              --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
              --vendor=valkey \
              --provider-version=8  \
              --cluster-name="My Valkey Sharded Cluster" \
              --log --wait
  ```
```

### 2.11.11 Redis Sentinel

Import existing Redis instances with Sentinel for v7 and v8. When importing existing Redis instances, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host fenvironment and operating system.
- Discover the database role of each node (primary, replica, sentinel).

**Warning**

Attention

ClusterControl does not support importing Redis instances without authentication. Authentication must be enabled and the provided admin username and password must have `~* +@all` (all keys, all commands) privilege.

The following vendors and versions are supported for a importing existing cluster:

- Redis Sentinel - 6, 7 and 8

**Info**

ClusterControl and Severalnines do not use or reproduce the official Redis® logo, which is a registered trademark of Redis Ltd. To comply with Redis trademark and branding guidelines, we have introduced our own distinct branding for Redis-related features within ClusterControl. This logo and name are original creations of Severalnines and are used solely as internal branding to represent Redis.

#### Default configuration

By default, ClusterControl deploys Redis instances with the following configurations:

- Redis Sentinel (default port is 26379) will be co-located with the Redis instances (default port is 6379).
- ClusterControl will configure the Redis instance with `appendonly` enabled.
- ClusterControl will secure the instance with authentication enabled and configure the `requirepass` and `masterauth` options.
```

- ClusterControl will enable TLS encryption. To access the database nodes, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.

!!! Tips

```
Redis Sentinel requires 3 nodes for automatic primary promotion. Sentinel␣
↪can be co-located on the ClusterControl server if you want to deploy a␣
↪two-node Redis replication cluster (Sentinel will be co-located on each␣
↪database instance).
```

Import steps

ClusterControl GUI

1. To import a new Redis Sentinel cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Redis Sentinel".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Redis Port**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **Redis Sentinel Port**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.

- **Repository**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
- **Password**: Under the **Authentication** pane, this is the primary node password used for client/replica authentication.
- **Replication user**: Under the **Authentication** pane, this is the replication user of your Redis replication cluster.
- **Replication password**: Under the **Authentication** pane, this is the replication password of your Redis replication cluster.
- **Sentinel password**: The password for your Sentinel user `sentinel-user`
- **Use existing certificates**: Specify cmon's certificates directory to use on imported cluster. If ssl configuration is not completed scale up operations may fail.
- **Cluster auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling cluster recovery enabled. Otherwise, off will not enable cluster recovery.
- **Node auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling node recovery enabled. Otherwise, off will not enable node recovery.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, is where you can add your database nodes:
   - **Primary node**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.
   - **Replica nodes**: Only specify the IP address or hostname of your replica node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note If you have more than one replica nodes, make sure to add more under **Replica nodes** text field. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a Redis Sentinel Cluster consisting of 3-nodes using `ubuntu` as it's OS username, using sentinel password as `sjl1215wi` and using `ed25519` digital signature algorihtm and wait then print the logs of the job deployment:

```
s9s cluster --register --cluster-type=redis \
    --nodes="redis://192.168.40.190:6379;␣
↪redis://192.168.40.191:6379;redis://192.168.40.192:6379; \
            ␣
↪redis-sentinel://192.168.40.190:26379;redis-sentinel://192.168.40.191:26379;␣
↪\
             redis-sentinel://192.168.40.192:26379;" \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
    --sentinel-passwd sjl2l5wi\
    --vendor=redis
    --provider-version=8  \
    --cluster-name="My Redis Sentinel Cluster" \
    --log --wait
```

2.11.12 Redis Cluster

Import existing Redis Cluster instances for v7 and v8. When importing existing Redis instances, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (primary, replica).

Warning

Attention

ClusterControl does not support importing Redis instances without authentication. Authentication must be enabled and the provided admin username and password must have `~* +@all` (all keys, all commands) privilege.

The following vendors and versions are supported for a importing existing cluster:

- Redis Cluster - 6, 7 and 8
- Canonical - 7

Info

ClusterControl and Severalnines do not use or reproduce the official Redis® logo, which is a registered trademark of Redis Ltd. To comply with Redis trademark and branding guidelines, we have introduced our own distinct branding for Redis-related features within ClusterControl. This logo and name are original creations of Severalnines and are used solely as internal branding to represent Redis.

Default configuration

By default, ClusterControl deploys Redis cluster instances with the following configurations:

- ClusterControl will configure the Redis instances with `appendonly` enabled.
- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.

Import steps

ClusterControl GUI

1. To import an existing Redis Cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Redis Cluster".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not

setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.

- **Username**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPERUSER role and allowed for localhost only.
- **Password**: The password for **User**.
- **Cluster auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling cluster recovery enabled. Otherwise, off will not enable cluster recovery.
- **Node auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling node recovery enabled. Otherwise, off will not enable node recovery.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you will see the label **Router/Mongos servers**. This entails that you only have to specify the router/mongos server nodes. This is where you can specify the target database nodes that you want to deploy:
   - **Cluster node**: Only specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note Specify one node and ClusterControl will discover the rest of members. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- To import a Redis Sharded Cluster, you only need to specify the primary node. For example, if you have primary running on 192.168.40.188 and you have replica running on 192.168.40.189, no need to specify the replica as it will be auto-detected and will be imported by ClusterControl as well. The following command demonstrates this using `ubuntu` as it's OS username, using `ed25519` digital signature algorihtm and wait then print the logs of the job deployment:

```
s9s cluster --register \
        --cluster-type=redis_sharded \
```

(continues on next page)

(continued from previous page)

```
                --nodes="redis://192.168.40.188;" \
                --os-user=ubuntu \
                --os-key-file=/home/ubuntu/.ssh/id_ed25519 \
                --vendor=redis \
                --provider-version=8  \
                --cluster-name="My Redis Sharded Cluster" \
                --log --wait
    ```
```

### 2.11.13 Microsoft SQL Server

Import existing Microsoft SQL Server – 2019 and 2022 for Linux. When importing existing SQL Server instances, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (primary, replica).

The following vendors and versions are supported for a new deployment:

- Microsoft SQL Server for Linux - 2019 and 2022

**Warning**

Attention

The minimum memory requirement for a Microsoft SQL Server database node shall have at least 2000 MB of memory. ClusterControl will abort the deployment job if this requirement is not met.

#### Default configuration

By default, ClusterControl deploys SQL Server with the following configurations:

- All database nodes must use fully qualified domain name (FQDN). IP address is not supported.
- Minimum available RAM is 1800 MB per database node.
- Enforces the SQL Server user's password policy as shown here.
- At the moment, ClusterControl only deploys AlwaysOn with asynchronous-commit mode, where it does not wait for any secondary replica to write incoming transaction log records to disk.
- ClusterControl will set up the SQL Server using the Evaluation edition (free, 180-day limit).

#### Import steps

#### ClusterControl GUI

1. To import an existing SQL Server, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "SQL Server".
2. Under **Cluster details**, specify the cluster details you want to assign:
    - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
```

- **Tags**: Add tags to search or group your database clusters

3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **Admin Username**: The username of your admin user.
   - **Admin Password**: The password for the admin user.
   - **Server port**: This port is disabled but this serves as your reference to what port should be used during import.
   - **Cluster auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling cluster recovery enabled. Otherwise, off will not enable cluster recovery.
   - **Node auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling node recovery enabled. Otherwise, off will not enable node recovery.
7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you will see the label **Router/Mongos servers**. This entails that you only have to specify the router/mongos server nodes. This is where you can specify the target database nodes that you want to deploy:
   - **Primary node**: Specify the IP address or hostname of the primary database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.
   - **Replica nodes**: Specify the IP address or hostname of the replica database node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. You can specify zero or more replica nodes.

Note

Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note If you have more than one replica nodes, make sure to add more under **Replica nodes** text field. ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the cluster by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Importing is currently not implemented for SQL Server cluster

2.11.14 Elasticsearch

Import existing Elasticsearch Cluster instances for 7.x and 8.x. When importing existing Elasticsearch instances, ClusterControl will perform the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Discover the database role of each node (Master, data).

Warning

Attention

ClusterControl does not support importing Elasticsearch instances without authentication. Authentication must be enabled and the provided sysadmin username and password.

The following vendor and versions are supported for a new deployment:

- Elastic - 7.1, 8.1 and 8.3

Default configuration

By default, ClusterControl deploys Elasticsearch with the following configurations:

- For high-availability cluster setup, ClusterControl will configure an NFS server on one of the Elasticsearch nodes and mount the shared filesystem on all data nodes. This is for snapshot backup and restoration.
- For single-node cluster setup, ClusterControl will create a local path for snapshot backup and restoration.

Warning

Attention

The minimum memory requirement for an Elasticsearch master node, data node, or master-data node is 1576 MB. ClusterControl will abort the deployment job if this requirement is not met. See Hardware Requirement for Elasticsearch.

Import steps

ClusterControl GUI

1. To import a new Elasticsearch cluster, go to ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and under the **Database** dropdown, choose "Elasticsearch".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which lables with **Name your cluster**. ONce import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when deploying the cluster:
   - **HTTP port**: This port is disabled but this serves as your reference to what port should be used during import.
   - **Certificates Password**: If specified, the password for a preexisting CA private key
   - **Admin Username**: Enter the sysadmin username you use in your Elasticsearch cluster.
   - **Admin Password**: Enter the sysadmin password you use in your Elasticsearch cluster.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.

- **Use existing certificates**: Specify cmon's certificates directory to use on imported cluster. If empty, CC will search on configuration folder '/etc/elasticsearch' or default generation folder `/usr/share/elasticsearch` to get certificate files (like *elastic-certificates.p12*, *elastic-stack-ca.p12* and *elasticsearch-ssl-http.zip*) on node to configure certificates on new nodes in cluster. If ssl configuration is not completed scale up operations may fail.
- **Cluster auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling cluster recovery enabled. Otherwise, off will not enable cluster recovery.
- **Node auto-recovery**: Under **Advanced configuration**, you will see this option allowing you to turn it on/off. When on, it will import the cluster enabling node recovery enabled. Otherwise, off will not enable node recovery.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you will see the label **Router/Mongos servers**. This entails that you only have to specify the router/mongos server nodes. This is where you can specify the target database nodes that you want to deploy:
   - **Cluster node**: Specify the IP address or hostname of the primary or master-data node. Press ++enter++ to add the node, where ClusterControl will perform a pre-deployment check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic. Only one primary node is allowed for single-primary replication.

> **Note**
>
> Note

You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: :::{note} Note :::

```
                Specify one node and ClusterControl will discover the rest␣
↪of members. Specified node will be used to search for certificate files.

9. Click **Continue** to proceed to the **Preview** page. In this section,␣
↪you can see the summary of your deployment and if everything is correct,␣
↪you may proceed to deploy the cluster by clicking **Finish**. You can␣
↪always go back to any previous section to modify your configurations if␣
↪you wish. The deployment settings will be kept until you exit the␣
↪deployment wizard.

10. ClusterControl will trigger a deployment job and the progress can be␣
↪monitored under _**ClusterControl GUI → Activity Center → Jobs**_.
```

ClusterControl CLI

- Importing is currently not implemented for Elasticsearch cluster

2.11.15 MySQL Group Replication

ClusterControl can manage and monitor existing MySQL Group Replication clusters. A minimum of three nodes is required for MySQL Group Replication to achieve high availability and automatic failover. The cluster can be configured in either single-primary mode (one primary and multiple secondaries) or multi-primary mode (all nodes accept writes).

When importing an existing MySQL Group Replication cluster, ClusterControl will do the following:

- Verify SSH connectivity to all nodes.
- Detect the host environment and operating system.
- Pull the configuration files.
- Detect the remaining nodes in the cluster, only if Automatic node discovery is ON.
- Register the nodes into ClusterControl.

The following vendors and versions are supported for importing your existing cluster:

- Oracle MySQL 8.0 and 8.4

Default configuration

By default, ClusterControl deploys MySQL Group Replication with the following configurations:

- MySQL GTID is enabled with `log_slave_updates` enabled.
- Group Replication plugin is loaded and configured.
- Binary logging is enabled.
- All database nodes in multi-primary mode will be configured with `read_only=OFF` to allow writes on all nodes.
- The generated backup credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- ClusterControl will create and grant necessary privileges for the following database users:

```
| Database user      | Purpose                                          |
| ------------------ | ------------------------------------------------ |
| `cmon`             | Management and automation.                       |
| `cmonexporter`     | Prometheus exporter for database monitoring.     |
| `rpl_user`         | MySQL Group Replication.                         |
| `backupuser`       | Backup and restore management.                   |
```

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

MySQL Group Replication requires all nodes to be reachable by each other. Ensure that network connectivity between all nodes is properly configured before importing the cluster.

Import steps

ClusterControl GUI

1. To import an existing MySQL Group Replication cluster, go to ***ClusterControl GUI → Deploy a cluster → Import a database cluster*** and under the **Database** dropdown, choose "MySQL Group Replication".
2. Under **Cluster details**, specify the cluster details you want to assign:
   - **Name**: This is optional. This shall be the name of your cluster which labels with **Name your cluster**. Once import is done, ClusterControl will use this as its registry name of the cluster.
   - **Tags**: Add tags to search or group your database clusters
3. Click **Continue**.
4. Under the **SSH configuration** section, specify the SSH credentials that ClusterControl should use to connect to the database nodes:
   - **SSH user**: The SSH user that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH user.
   - **SSH user key path**: The SSH private key path that ClusterControl will use to perform SSH authentication to the database node. Relative path is not supported. The SSH private key must be physically secured and existed on the ClusterControl node.
   - **SSH port**: The SSH port that ClusterControl will use to perform SSH to the database node. ClusterControl assumes that all database nodes are using the same SSH port.
   - **SSH sudo password**: The sudo password if the **SSH user** requires a password for privilege escalation.
   - **SSH sudo / OS elevation command**: The OS elevation command (`sudo`, `doas`, `pbrun`) that ClusterControl will uses for non-root account.
5. Click **Continue** to proceed to the next step.
6. Under the **Node configuration** section, specify the database credentials and configurations that ClusterControl shall use when importing the cluster:
   - **Server port**: The database server port that ClusterControl shall configure on all database nodes.
   - **Server data directory**: The database server data directory path that ClusterControl shall configure on all database nodes.
   - **Admin/Root user**: The database admin username that ClusterControl shall configure on all database nodes. This user will be granted global SUPER privilege and the GRANT option for localhost only.
   - **Admin/Root password**: The password for **Admin/Root user**.
   - **Repository**: Choose "Use vendor repositories" (default) will let ClusterControl provision software by setting up and using the database vendor's preferred software repository. ClusterControl will always install the latest version of what is provided at that moment. Choose "Do not setup vendor repositories" if you have a special configuration from the vendor (commonly for enterprise databases) and ClusterControl will skip the repository configuration part.
   - **Replication user/Replication password**: The credentials configured for replication.

- **information_schema queries**: By default, this is set to off. When enabled, this shall allow ClusterControl to query from information_schema schema to collect metrics from your MySQL database nodes.
- **Cluster auto-recovery**: By default, this is set to off. Once enabled, the cluster auto-recovery will be enabled allowing for degraded or failing cluster will be attempted to be recovered by ClusterControl.
- **Node auto-recovery**: By default, this is set to off. Once enabled, the node auto-recovery will be enabled allowing for failing nodes such as accidentally terminated system process will be tried to be started to get back to the cluster.
- **Automatic node discovery**: By default, this is set to on. Set it to off (or disable) if you want to specify each IPs/hostnames that ClusterControl should use to connect to the database nodes.

7. Click **Continue** to proceed to the next step.
8. Under the **Add nodes** section, you can specify your existing target database nodes. Since this is an existing database cluster which means it is already setup, ClusterControl will be able to identify and scan the type of topology for your database cluster.
   - **Node**: Specify the IP address or hostname of any database node in the group. Press ++enter++ to add the node, where ClusterControl will perform a check to verify if the node is reachable via SSH key-based authentication. If the target node has more than one network interface, you will be able to select or enter a separate IP address to be used only for database traffic.

> **Note**
>
> Note

You can specify just one node and ClusterControl will automatically discover the remaining nodes in the Group Replication cluster. ::: :::{note} Note You can only proceed to the next step if all of the specified nodes are reachable (shown in green). ::: 9. Click **Continue** to proceed to the **Preview** page. In this section, you can see the summary of your deployment/import and if everything is correct, you may proceed by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The settings will be kept until you exit the wizard.

10. ClusterControl will trigger an import job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

- Import an Oracle MySQL Group Replication 8.4 cluster, with operating system user **ubuntu**:

```
```bash
s9s cluster --register \
    --cluster-type=group_replication \
    --nodes="10.10.10.10,10.10.10.11,10.10.10.12" \
    --vendor=oracle \
    --provider-version=8.4 \
    --db-admin-passwd="mYpa$$word" \
    --repl-user="rpl_user" \
    --repl-passwd="mYrepLpa$$word" \
    --os-user=ubuntu \
    --os-key-file=/home/ubuntu/.ssh/id_rsa \
    --cluster-name="MySQL Group Replication 8.4" \
```

(continues on next page)
```

(continued from previous page)

```
    --wait \
    --log
```

Recommended Next Steps

- *Create Load Balancer*
- *Database Status and Healthchecks*
- *Schedule Backup*
- *Restore External Backup*
- *Alerts and Notifications*

2.12 Create Load Balancer

ClusterControl simplifies the deployment of load balancers for database clusters by providing a guided workflow via its web UI and CLI. Load balancers are essential in distributing traffic across multiple database nodes, improving availability, fault tolerance, and performance.

With Create Load Balancer, you can:

- Deploy supported load balancers in front of your database clusters.
- Automatically configure health checks and failover mechanisms.
- Integrate with Keepalived to provide VIP-based failover.
- Manage multiple load balancer instances for redundancy.

2.12.1 Support Matrix

ClusterControl supports deploying the following load balancer types:

Application	Purpose	Supported Databases
ProxySQL	Load balancer, query router	MySQL, MariaDB, Galera Cluster
HAProxy	TCP load balancer	MySQL, MariaDB, Galera Cluster, PostgreSQL, TimescaleDB
MariaDB MaxScale	SQL load balancer, query router	MySQL, MariaDB, Galera Cluster
garbd	Galera arbitrator (vote-only)	Galera Cluster
PgBouncer	Lightweight connection pooler for PostgreSQL	PostgreSQL, TimescaleDB
Keepalived	Provides a floating virtual IP address (VIP) service	ProxySQL, HAProxy, MariaDB MaxScale

Info

A single set of load balancers should be dedicated to a single database cluster. ClusterControl does not support managing multiple database clusters sharing one set of load balancers.

2.12.2 Prerequisites

Before deploying a load balancer, ensure the following:

- You have a database cluster deployed and managed by ClusterControl.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to load balancer nodes. See *SSH Key-based Authentication*.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.
- The target node(s) are reachable from the ClusterControl server.

2.12.3 ProxySQL

ClusterControl deploys ProxySQL in read/write split mode using ProxySQL query rules where the read-only traffic will be sent to replicas while your writes will be sent to a writable primary by creating two host groups.

In case of primary failure, ProxySQL will detect the new writable primary and route writes to it automatically without any user intervention. ProxySQL offers a wide range of features and capabilities that make it suitable for various scenarios, including improving the scalability, availability, performance, and security of MySQL database infrastructures. You can use an existing database server or another new host by specifying the hostname or IP address.

With two or more ProxySQL nodes, you can then configure a virtual IP address service using Keepalived. See *Keepalived*.

See also

Database Load Balancing for MySQL and MariaDB with ProxySQL – Tutorial

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Create balancer**.
2. Choose the load balancer service **ProxySQL**.
3. Click **Continue**.
4. You will be presented with a **Create ProxySQL service** deployment wizard. Specify:
    - **Version**: Choose a supported version.
    - **Server address**: Specify the hostname or IP address of the host. This host must be accessible via passwordless SSH from the ClusterControl node.
    - **Admin port**: ProxySQL administration port. The default is 6032.
    - **Listening port**: ProxySQL for MySQL load balancing. The default is 6033. This is where the applications and clients will connect after ProxySQL is activated.

5. For **ProxySQL Configuration**, you can choose:

   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.
   - **Import configuration**: Deploys a new ProxySQL based on an existing ProxySQL instance. The source instance must be added first into ClusterControl. Once added, you can choose the source ProxySQL instance from a dropdown list.
   - **Use native clustering**: The ProxySQL Server will be created using ProxySQL native clustering and an entry will be created in the `proxysql_server` tab. It is recommended to enable this if you would like to have more than one ProxySQL node. Port 6032 must be reachable between all ProxySQL nodes.

   Then, click **Continue** to the next step.

6. Under the **Configuration** section, specify the credentials that ClusterControl should use to create the load balancer:

   - **Administration user**: ProxySQL administration user name.
   - **Administration password**: Password for the administration user.
   - **Monitor user**: ProxySQL monitor user.
   - **Monitor password**: Password for monitor user.
   - **Database user**: Create or use an existing database user.
     - For existing user:
       - **Database username**: Choose the existing user name from the dropdown.
       - **Database password**: Password for DB User.
     - For new user:
       - **Database username**: The database user name.
       - **Database password**: Password for DB Users.
       - **Database name**: Database name in “database.table” format.
       - **MySQL privilege(s)**: ClusterControl will load the privilege name along the keypress. Multiple privileges are possible.

Note

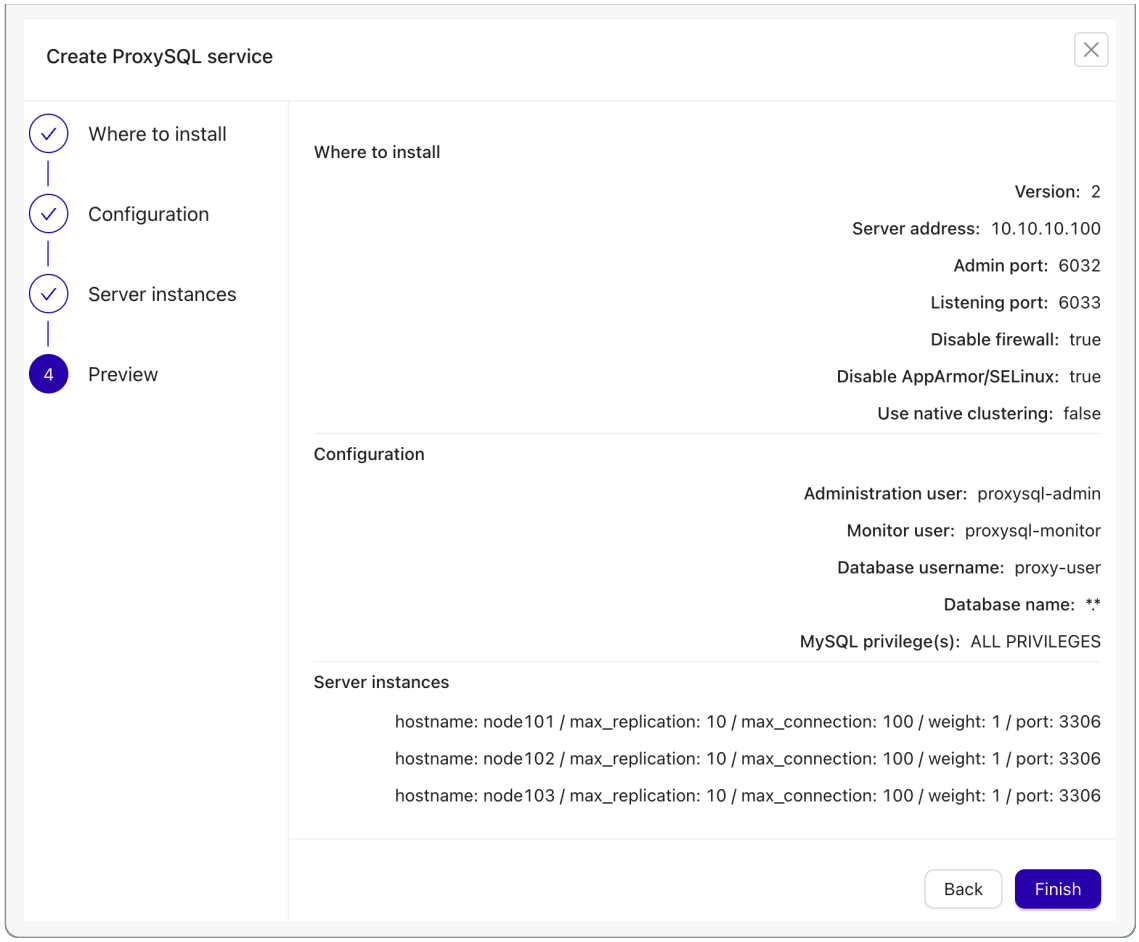
Note

Administration requires port 6032 to be open in the firewall/security group. ::: 7. In the **Server instances** section, specify what nodes are going to be added to the load balancer, its Max replication lag, Max connection, and Weight. Also, choose if you want to use implicit transactions - default value is Off.

```
- **Max replication lag**: How many seconds of replication lags should be␣
↪allowed before marking the node as unhealthy. The default value is 10.
- **Max connection**: Maximum connections are to be sent to the backend␣
↪servers. It's recommended to match or lower than the `max_connections`␣
↪value of the backend servers.
- **Weight**: This value is used to adjust the server's weight relative to␣
↪other servers. All servers will receive a load proportional to their␣
↪weight relative to the sum of all weights. The higher the weight, the␣
↪higher the priority.
- **Are you using implicit transactions?**: Choose **Off** if you rely on␣
↪`SET AUTOCOMMIT=0` to create a transaction. Choose **On** if you␣
↪explicitly use `BEGIN` or `START TRANSACTION` to create a transaction.␣
↪Choose **Off** if you are unsure of this part.
```

8. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

Example output



9. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a ProxySQL node:

```
s9s cluster \
    --cluster-id=2 \
    --add-node \
    --nodes=
↪"proxysql://10.10.10.103?db_username=proxysqluser&db_password=proxysqlpass&db_data
↪'*.*'&db_privs='ALL PRIVILEGES'" \
    --wait \
    --log
```

2.12.4 HAProxy

HAProxy is a TCP load balancer and can be used for MySQL/MariaDB replication, Galera Cluster, PostgreSQL and TimescaleDB.

ClusterControl will automatically install and configure HAProxy, install and configure the healthcheck script (mysqlchk for MySQL/MariaDB-based clusters and postgresqlchk for PostgreSQL-based clusters) on each of the database nodes using systemd service, and start the HAProxy service. If you set up read/write splitting for primary-replica replication, there will be two listening ports configured (one for read-write and

another one for read-only connections). You can use an existing database server or another new host by specifying the hostname or IPv4 address.

With two or more HAProxy nodes, you can then configure a virtual IP address service using Keepalived. See *Keepalived*.

See also

MySQL Load Balancing with HAProxy – Tutorial

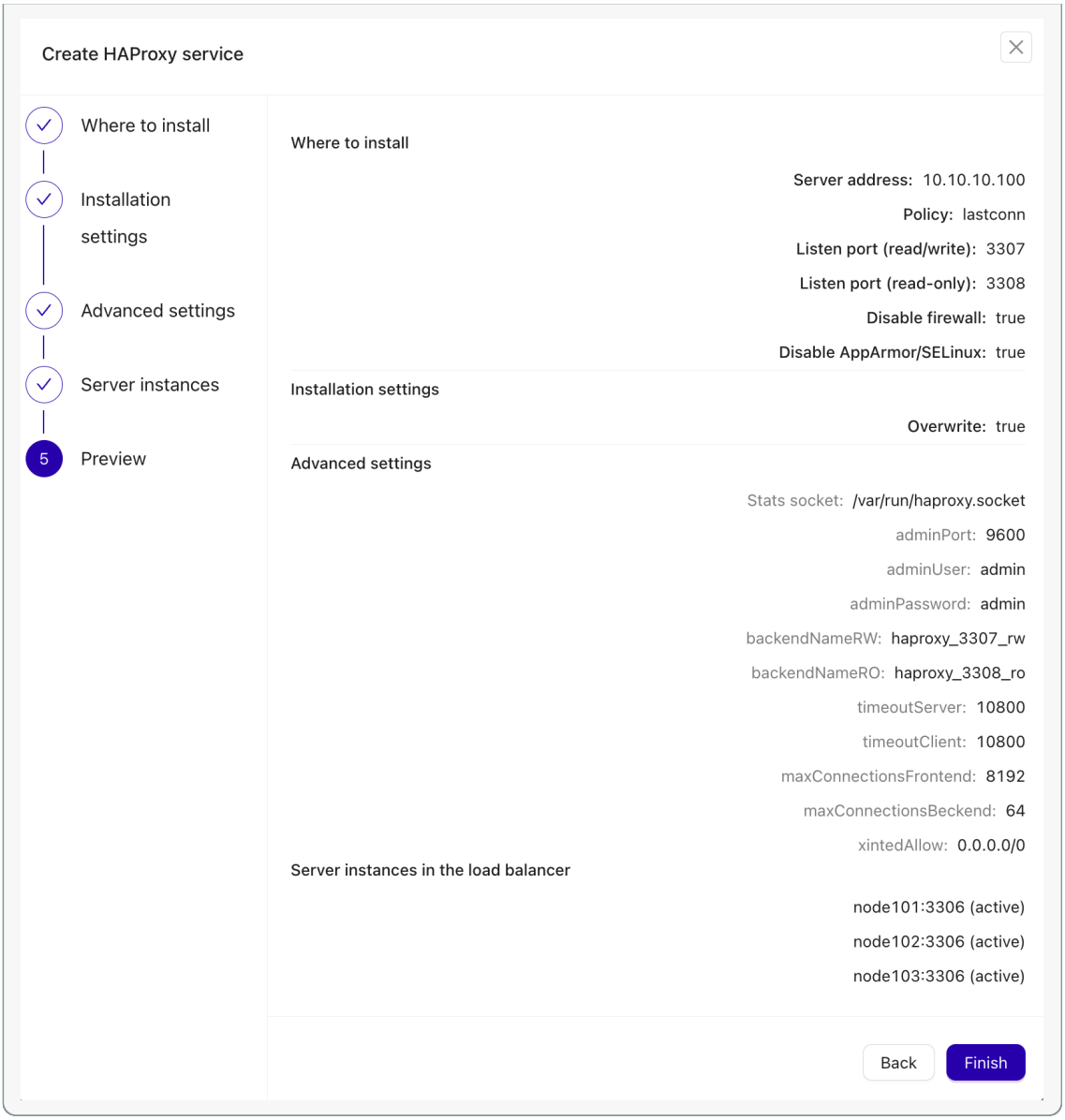
ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Create balancer**.
2. Choose the load balancer service **HAProxy**.
3. Click **Continue**.
4. You will be presented with a **Create HAProxy service** deployment wizard. Specify:
   - **Server address**: Select which host to add the load balancer. If the host is not provisioned by ClusterControl, type in the IP address or hostname. The required files will be installed on the new host. Note that ClusterControl will access the new host using passwordless SSH.
   - **Policy**: Choose one of these load-balancing algorithms:
     - **leastconn:** The server with the lowest number of connections receives the connection.
     - **roundrobin:** Each server is used in turns, according to their weights.
     - **source:** Same client IP address will always reach the same server as long as no server goes down.
   - **Listen port (read/write)**: The listening port that ClusterControl shall configure during the deployment process to receive read/write traffic.
   - **Listen port (read-only)**: The listening port that ClusterControl shall configure during the deployment process to receive read-only traffic.
   - **Install for read/write splitting (master-slave replication)**: Toggled On if you want HAProxy to use another listener port for read-only. A new text box for **Listen Port (Read Only)** will appear right next to the **Listen Port (Read/Write)** text box. You can then specify the port for read-only database connection port.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

   Then, click **Continue** to the next step.

5. Under the **Installation settings** section, specify if you want to Overwrite existing health check script on the node. Enabled by default.
6. In **Advanced settings** section, specify:
   - **Stats socket**: Specify the path to bind a UNIX socket for HAProxy statistics.
   - **Admin port**: Port to listen to the HAProxy statistic page.
   - **Admin user**: Admin username to access the HAProxy statistic page.
   - **Admin password**: Password for the admin user.
   - **Backend name (RW)**: Name of the read-write backend to be created in HAProxy.
   - **Backend name (RO)**: Name of the read-only backend to be created in HAProxy.
   - **Timeout server (in seconds)**: Sets the maximum inactivity time on the server side.
   - **Timeout client (in seconds)**: Sets the maximum inactivity time on the client side.
   - **Max connections frontend**: Sets the maximum per-process number of concurrent connections to the HAProxy instance.
   - **Max connections backend (per instance)**: Sets the maximum per-process number of concurrent connections per backend instance.
   - **xinetd allow connections from**: The specified subnet will be allowed to access the health check script `mysqlchk` or `mysqlchk_rw_split` for read/write splitting (MySQL-based clusters) or `postgreschk` and `postgreschk_rw_split` (PostgreSQL-based clusters) as xinetd service, which listens on port 9200 on every database node. To allow connections from all IP addresses, use the default value, "0.0.0.0/0".
7. For the **Server instances** section, switch on the instances you want to be included in the load balancer. As **Advanced options** you can set Role (`active` – the server is actively used in load balancing, or `backup` – the server is only used in load balancing when all other non-backup servers are unavailable) and **Connection address** (choose the IP address where HAProxy should be listening on the host) for each database node.
8. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

Example output



9. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create an HAProxy node:

```
s9s cluster \
    --add-node \
    --cluster-id=1 \
    --nodes="haproxy://10.10.10.101" \
    --wait \
    --log
```

2.12.5 MariaDB MaxScale

MaxScale is an intelligent proxy that allows the forwarding of database statements to one or more database servers using complex rules, a semantic understanding of the database statements, and the roles of the various servers within the backend cluster of databases.

You can deploy or import the existing MaxScale node as a load balancer and query router for your Galera Cluster and MariaDB replication. For new deployment using ClusterControl, by default, it will create two listeners:

- RW – Implements read-write split access (default port 4008)
- RR – Implements round-robin access (default port 4006)

With two MaxScale nodes, you can then configure a virtual IP address service using Keepalived. See *Keepalived*.

Warning

Attention

ClusterControl performs MariaDB MaxScale installation via direct package download without using the MariaDB repository. The package download URL is kept inside `/usr/share/cmon/templates/packages.conf` on the ClusterControl server under the `[maxscale]` section. Occasionally, the provided URL will be outdated as MariaDB releases a new minor version, and removes the older minor version for a specific MaxScale major version. If that is the case, a manual modification is required to update the download link in this file. The updated download URL is available on the MariaDB MaxScale website.

ClusterControl GUI

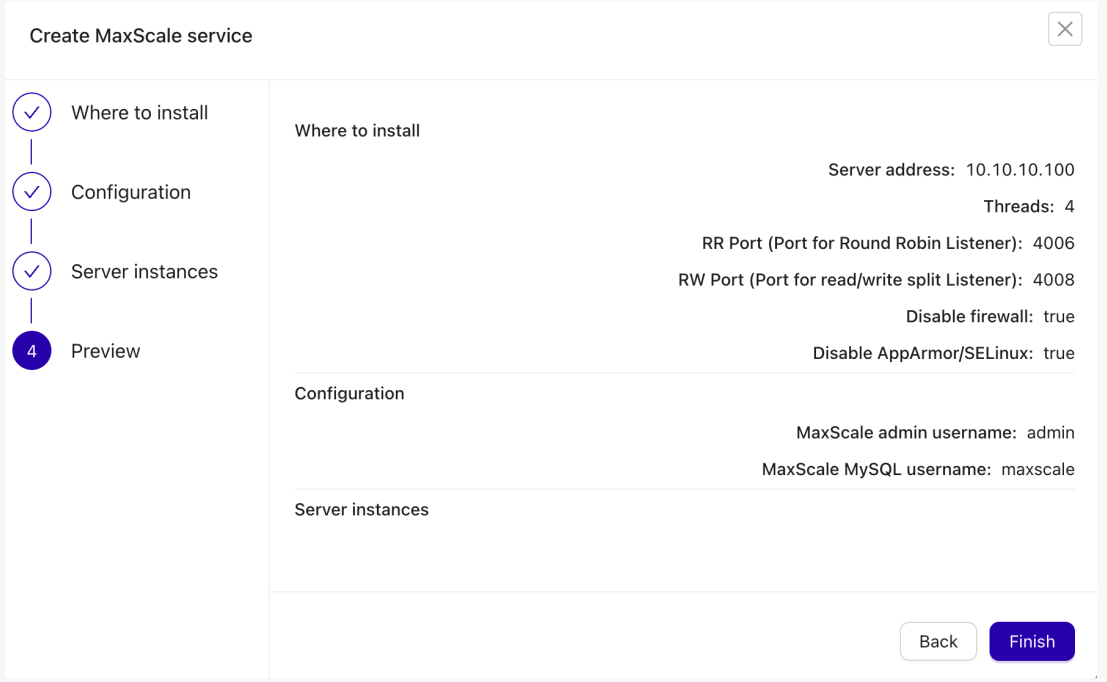
1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Create balancer**.
2. Choose the load balancer service **MariaDB MaxScale**.
3. Click **Continue**.
4. You will be presented with a **Create MaxScale service** deployment wizard. Specify:
   - **Server address**: The IP address of the node where MaxScale will be installed. ClusterControl must be able to perform passwordless SSH to this host.
   - **Threads**: How many threads MaxScale is allowed to use.
   - **RR Port**: Port for round-robin listener. The default is 4006.
   - **RW Post**: Port for the read-write split listener. The default is 4008.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded

to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

Then, click **Continue** to the next step.

5. Under the **Configuration** section, specify the credentials that ClusterControl should use to create the load balancer:
   - **MaxScale admin username**: MaxScale admin username. The default is 'admin'.
   - **MaxScale admin password**: MaxScale enforces that the admin password for admin user 'admin' is 'mariadb'. If you want to change or use another password you must create another user. The 'admin' user can later be dropped.
   - **MaxScale MySQL username**: MariaDB/MySQL user that will be used by MaxScale to access and monitor the MariaDB/MySQL nodes.
   - **MaxScale MySQL password**: Password of MaxScale MySQL Username.
6. In the **Server instances** section, switch on the instances you want to be included in the load balancer.
7. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

Example output



8. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a MaxScale node:

```
s9s cluster \
    --cluster-id=2 \
    --add-node \
    --nodes="maxscale://10.10.10.104" \
    --maxscale-mysql-user=maxscaleuser \
    --maxscale-mysql-password=maxscalepass \
    --wait \
    --log
```

2.12.6 Garbd

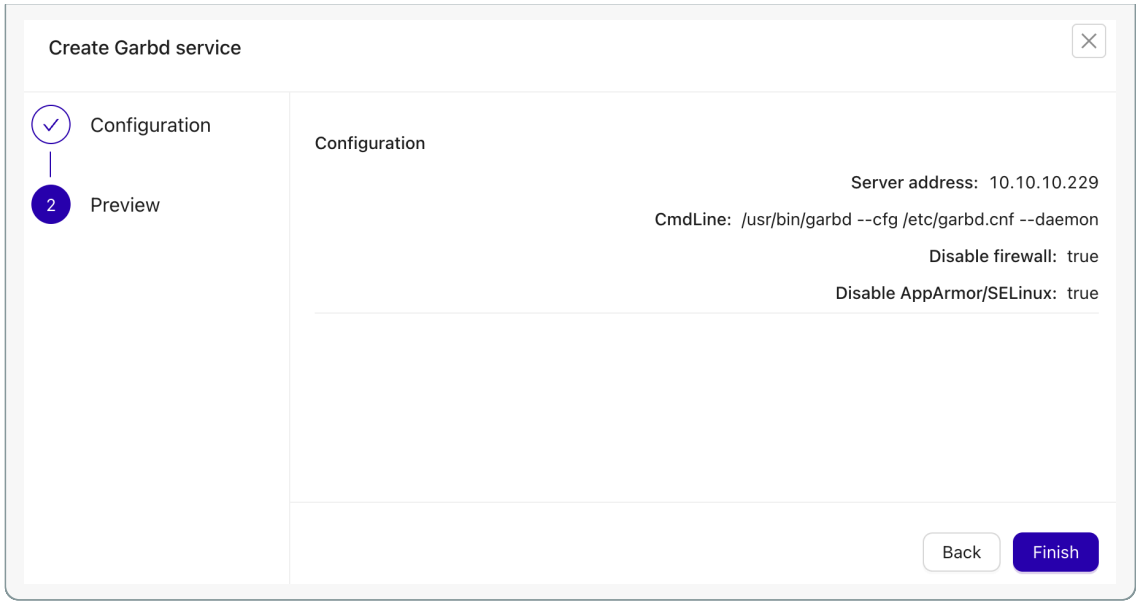
Exclusive for Percona XtraDB Cluster or MariaDB Galera Cluster. Galera arbitrator daemon (garbd) can be installed to avoid network partitioning or split-brain scenarios.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Create balancer**.
2. Choose the load balancer service **Garbd**.
3. Click **Continue**.
4. You will be presented with a **Create garbd service** deployment wizard. Specify:
   - **Server address**: The server address that ClusterControl will use to deploy the load balancer. ClusterControl must be able to perform passwordless SSH to this host.
   - **CmdLine**: Garbd command line to start garbd process on the target node.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

   Then, click **Continue** to the next step.
5. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

Example output



6. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

Warning

Attention

ClusterControl does not support and allow garbd deployment on a server where ClusterControl is also running and hosted. There is a tendency that the existing MySQL packages will be removed which is managed by the software packaging tools.

2.12.7 PgBouncer

This section is for PgBouncer nodes (PostgreSQL, TimescaleDB). PgBouncer is a lightweight connection pooler for PostgreSQL. It reduces PostgreSQL resource consumption (memory, backends, fork) and supports online restart or upgrade without dropping client connections.

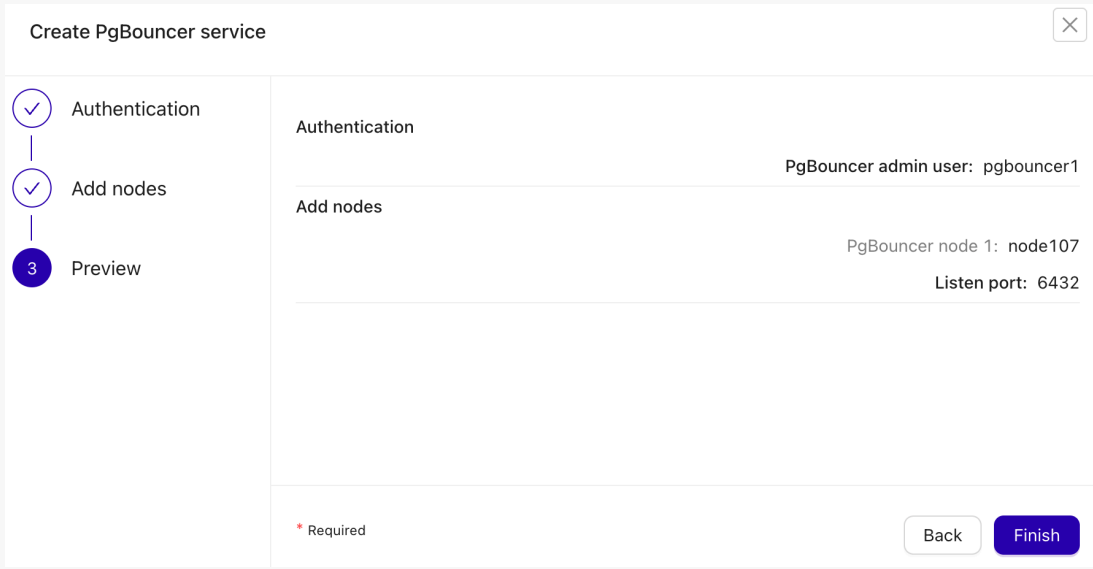
Using ClusterControl, you can manage PgBouncer on one or more nodes, manage multiple pools per node and support 3 pool modes:

- **session (default):** When a client connects, a server connection will be assigned to it for the whole duration the client stays connected. When the client disconnects, the server connection will be put back into the pool.
- **transaction:** A server connection is assigned to a client only during a transaction. When PgBouncer notices that the transaction is over, the server connection will be put back into the pool.
- **statement:** The server connection will be put back into the pool immediately after a query completes. Multi-statement transactions are disallowed in this mode as they would break.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Create balancer**.
2. Choose the load balancer service **PgBouncer**.
3. Click **Continue**.
4. You will be presented with a **Create PgBouncer service** deployment wizard. Specify:
   - **PgBouncer admin user**: PgBouncer admin username. ClusterControl will create the admin user if specified. If empty, ClusterControl will generate an admin user called "pgbadmin".
   - **PgBouncer admin password**: The password for the PgBouncer admin user.
5. Under the **Add nodes** section, specify the nodes where to deploy PgBouncer and the Listening port to be used (default is 6432).
6. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your deployment and if everything is correct, you may proceed to deploy the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The deployment settings will be kept until you exit the deployment wizard.

Example output



7. ClusterControl will trigger a deployment job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Create a ProxySQL node:

```
s9s cluster \
    --cluster-id=4 \
    --add-node \
    --nodes="pgbouncer://10.10.10.107" \
```

(continues on next page)

(continued from previous page)

```
    --wait \
    --log
```

2.12.8 Keepalived

Keepalived requires two or more HAProxy, ProxySQL, or MariaDB MaxScale instances to provide virtual IP address failover. Keepalived uses the IP Virtual Server (IPVS) kernel module to provide transport layer (Layer 4) load balancing, redirecting requests for network-based services to individual members of a server cluster.

Initially, the virtual IP address is assigned to the instance designated as 'Keepalived 1'. In the event of a failure of this node, the IP address is automatically failed over to 'Keepalived 2'.

See also

To understand further how ClusterControl configures Keepalived, see this blog post: How ClusterControl Configures Virtual IP and What to Expect During Failover

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Create balancer**.
2. Choose the load balancer service **Keepalived**.
3. Click **Continue**.
4. You will be presented with a **Create Keepalived service** deployment wizard:
    - **Load balancer type**: HAProxy, ProxySQL, MaxScale.
    - **Keepalived**: Choose the servers from the list or type IP address or hostname of the nodes where to install Keepalived.
    - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during deployment. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this deployment otherwise there is a risk of failure deployment. See *Firewall and Security Groups*.
    - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.
5. Under the **Host configuration** section, specify the virtual IP address and network interface where to add this virtual IP address. Specify a network interface to bind the virtual IP address of the load balancer host. This interface must be able to communicate with other Keepalived instances and support IP protocol 112 (VRRP) and unicast.

Warning

Attention

```
    Make sure the specified network interface matches the name of the␣
↪network interface in the node.
```

```
6.  Click **Continue** to proceed to the **Preview** step. In this section,␣
↪you can see the summary of your deployment and if everything is correct,␣
↪you may proceed to deploy the load balancer by clicking **Finish**. You␣
↪can always go back to any previous section to modify your configurations␣
↪if you wish. The deployment settings will be kept until you exit the␣
↪deployment wizard.

    :::{admonition} Example output
    :class: dropdown

    ![Keepalived Deployment Preview](../../images/keepalived_deployment.png)
    :::
7. ClusterControl will trigger a deployment job and the progress can be␣
↪monitored under _**ClusterControl GUI → Activity Center → Jobs**_.
```

2.13 Import Load Balancer

The Import Load Balancer feature in ClusterControl supports importing existing load balancer instances into its management console. It enables users to integrate external load balancers with imported database clusters, allowing centralized visibility, monitoring, and management from within ClusterControl.

With Import Load Balancer, it allows you to:

- Import supported load balancers in front of your database clusters.
- Monitor performance metrics such as throughput, latency, and active connections.
- Automatically configure health checks and failover mechanisms.
- Manage configuration changes.
- Integrate with Keepalived to provide VIP-based failover.
- Manage multiple load balancer instances for redundancy.

ClusterControl supports importing the following load balancer types:

- ProxySQL
- HAProxy
- MariaDB MaxScale
- Garbd
- PgBouncer
- Keepalived

2.13.1 Prerequisites

Before importing a load balancer, ensure the following:

- You have a database cluster imported and managed by ClusterControl. Every load balancer must be associated with a database cluster, and can't be shared with another cluster.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to load balancer nodes. See *SSH Key-based Authentication*.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.
- The target node(s) are reachable from the ClusterControl server.

2.13.2 Support Matrix

Load Balancer	Supported Databases
ProxySQL	MySQL, MariaDB, Galera Cluster
HAProxy	MySQL, MariaDB, Galera Cluster, PostgreSQL, TimescaleDB
MariaDB MaxScale	MySQL, MariaDB, Galera Cluster
Garbd	Galera Cluster
PgBouncer	PostgreSQL, TimescaleDB
Keepalived	For floating virtual IP address (VIP) setup. Support ProxySQL, HAProxy and MariaDB MaxScale

2.13.3 ProxySQL

This section is for ProxySQL nodes (MySQL/MariaDB replication, Galera Cluster). By default, ClusterControl imports ProxySQL in read/write split mode – your read-only traffic will be sent to replicas while your writes will be sent to a writable primary by creating two host groups. In case of primary failure, ProxySQL will detect the new writable primary and route writes to it automatically without any user intervention.

ProxySQL offers a wide range of features and capabilities that make it suitable for various scenarios, including improving the scalability, availability, performance, and security of MySQL database infrastructures. You can use an existing database server or another new host by specifying the hostname or IP address. With two or more ProxySQL nodes, you can then configure a virtual IP address service using Keepalived.

See also

Database Load Balancing for MySQL and MariaDB with ProxySQL – Tutorial

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Import balancer**.
2. Choose the load balancer service **ProxySQL**.
3. Click **Continue**.
4. You will be presented with a **Import ProxySQL service**, and on the step 1, it's on the Configuration:

- **Server address**: Specify the hostname or IP address of the host. This host must be accessible via passwordless SSH from the ClusterControl node.
- **Admin port**: ProxySQL administration port. The default is 6032.
- **Listening port**: ProxySQL for MySQL load balancing. The default is 6033. This is where the applications and clients will connect after ProxySQL is activated.
- **Import configuration**: By default, this is **Off**. If set to **On**, this means its enabled. A text field will show up allowing you to type the hostname of your source ProxySQL instance that contains the configuration.

5. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The import settings will be kept until you exit the import wizard.
6. ClusterControl will trigger an import job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import a ProxySQL node:

```
s9s cluster \
    --register \
    --cluster-id=2 \
    --nodes=
↪"proxysql://10.10.10.103?db_username=proxysqluser&db_password=proxysqlpass&db_data
↪'*.*'&db_privs='ALL PRIVILEGES'" \
    --wait \
    --log
```

2.13.4 HAProxy

This section is for HAProxy nodes (MySQL/MariaDB replication, Galera Cluster, PostgreSQL, TimescaleDB). ClusterControl will automatically install and configure HAProxy, install mysqlcheck script (for MySQL health checks) on each of the database nodes as part of xinetd service, and start the HAProxy service. If you set up read/write splitting for primary-replica replication, there will be two listening ports configured (one for read-write and another one for read-only connections). You can use an existing database server or another new host by specifying the hostname or IPv4 address. With two or more HAProxy nodes, you can then configure a virtual IP address service using Keepalived.

See also

MySQL Load Balancing with HAProxy – Tutorial

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Import balancer**.
2. Choose the load balancer service **HAProxy**.
3. Click **Continue**.

4. You will be presented with a **Import HAProxy service**, and on the step 1, it's on the Configuration:
   - **Server address**: Select which host to add the load balancer. If the host is not provisioned by ClusterControl, type in the IP address or hostname. The required files will be installed on the new host. Note that ClusterControl will access the new host using passwordless SSH.
   - **Port**: By default, this is set to 9600. This is the port to access stats for HAProxy. Unless you have different port, then change it to something else other than 9600.

   Then, click **Continue** to the next step.
5. In **Advanced settings** section, specify:
   - **Admin user**: Admin username to access the HAProxy statistic page.
   - **Admin password**: Password for the admin user.
   - **cmdline**: This is a string of command line that will be executed to run the daemon service or server. By default, the value is `/usr/sbin/haproxy -D -f /etc/haproxy/haproxy.cfg -sf $(cat /var/run/haproxy.pid)`
   - **LB name**: This shall be the load balancer name. By default, this is set to `lb_group1`
   - **Haproxy config**: The default value is `/etc/haproxy/haproxy.cfg`. This is the path of the configuration for your HAProxy server.
   - **Stats socket**: The default value is `/var/run/haproxy.socket`. This is the path of the socket file for your haproxy server.
6. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The import settings will be kept until you exit the import wizard.
7. ClusterControl will trigger an import job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import an existing HAProxy node:

```
s9s cluster \
    --register \
    --cluster-id=1 \
    --nodes="haproxy://10.10.10.101" \
    --wait \
    --log
```

2.13.5 MariaDB MaxScale

This section is for MariaDB MaxScale nodes (MySQL/MariaDB replication, Galera Cluster). MaxScale is an intelligent proxy that allows the forwarding of database statements to one or more database servers using complex rules, a semantic understanding of the database statements, and the roles of the various servers within the backend cluster of databases.

You can import the existing MaxScale node as a load balancer and query router for your Galera Cluster and MySQL/MariaDB replication. For new import using ClusterControl, by default, it will create two production services:

- RW – Implements read-write split access.
- RR – Implements round-robin access.

With two MaxScale nodes, you can then configure a virtual IP address service using Keepalived.

Warning

Attention

ClusterControl performs MariaDB MaxScale installation via direct package download without using the MariaDB repository. The package download URL is kept inside `/usr/share/cmon/templates/packages.conf` on the ClusterControl server under the `[maxscale]` section. Occasionally, the provided URL will be outdated as MariaDB releases a new minor version, and removes the older minor version for a specific MaxScale major version. If that is the case, a manual modification is required to update the download link in this file. The updated download URL is available on the MariaDB MaxScale website.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Import balancer**.
2. Choose the load balancer service **MariaDB MaxScale**.
3. Click **Continue**.
4. You will be presented with a **Import MaxScale service** import wizard. Specify:
   - **Server address**: The IP address of the node where MaxScale will be installed. ClusterControl must be able to perform passwordless SSH to this host.
   - **Threads**: How many threads MaxScale is allowed to use.
   - **RR Port**: Port for round-robin listener. The default is 4006.
   - **RW Post**: Port for the read-write split listener. The default is 4008.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during import. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this import otherwise there is a risk of failure import. See *Firewall and Security Groups*.
   - **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

   Then, click **Continue** to the next step.
5. Under the **Configuration** section, specify the credentials that ClusterControl should use to create the load balancer:
   - **MaxScale admin username**: MaxScale admin username. The default is 'admin'.
   - **MaxScale admin password**: MaxScale enforces that the admin password for admin user 'admin' is 'mariadb'. If you want to change or use another password you must create another user. The 'admin' user can later be dropped.

- **MaxScale MySQL username**: MariaDB/MySQL user that will be used by MaxScale to access and monitor the MariaDB/MySQL nodes.
- **MaxScale MySQL password**: Password of MaxScale MySQL Username.

6. In the **Server instances** section, switch on the instances you want to be included in the load balancer.
7. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The import settings will be kept until you exit the import wizard.
8. ClusterControl will trigger an import job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import an existing MaxScale node:

```
s9s node \
    --register \
    --cluster-id=2 \
    --nodes="maxscale://10.10.10.104" \
    --maxscale-mysql-user=maxscaleuser \
    --maxscale-mysql-password=maxscalepass \
    --wait \
    --log
```

2.13.6 Garbd

Exclusive for Galera Cluster. Galera arbitrator daemon (garbd) can be installed to avoid network partitioning or split-brain scenarios. If you already have an existing Galera arbitrator daemon running, you can import it into ClusterControl.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Import balancer**.
2. Choose the load balancer service **Garbd**.
3. Click **Continue**.
4. You will be presented with a **Import garbd service** import wizard. Specify:
   - **Server address**: The server address that ClusterControl will use to import the load balancer. ClusterControl must be able to perform passwordless SSH to this host.
   - **CmdLine**: Garbd command line to start garbd process on the target node.
   - **Disable firewall**: Whether ClusterControl should disable firewall (`iptables`, `ufw`, `firewalld`) on the node during import. It is recomemnded to use the default setting (checked). If unchecked, this configuration task will be skipped. If the firewall is enabled, ensure you have configured the necessary ports prior to this import otherwise there is a risk of failure import. See *Firewall and Security Groups*.

- **Disable SELinux/AppArmor**: Whether ClusterControl should disable SELinux (Red Hat-based operating system) or AppArmor (Debian-based operating system). It is recomemnded to use the default setting (checked). If unchecked, ensure you have set a proper policy for the database-related processes and all of their dependencies.

Then, click **Continue** to the next step.

5. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The import settings will be kept until you exit the import wizard.
6. ClusterControl will trigger an import job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

Warning

Attention

ClusterControl does not support and allow garbd import on a server where ClusterControl is also running and hosted. There is a tendency that the existing MySQL packages will be removed which is managed by the software packaging tools.

2.13.7 PgBouncer

This section is for PgBouncer nodes (PostgreSQL, TimescaleDB). PgBouncer is a lightweight connection pooler for PostgreSQL. It reduces PostgreSQL resource consumption (memory, backends, fork) and supports online restart or upgrade without dropping client connections.

Using ClusterControl, you can manage PgBouncer on one or more nodes, manage multiple pools per node and support 3 pool modes:

- **session (default):** When a client connects, a server connection will be assigned to it for the whole duration the client stays connected. When the client disconnects, the server connection will be put back into the pool.
- **transaction:** A server connection is assigned to a client only during a transaction. When PgBouncer notices that the transaction is over, the server connection will be put back into the pool.
- **statement:** The server connection will be put back into the pool immediately after a query completes. Multi-statement transactions are disallowed in this mode as they would break.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Import balancer**.
2. Choose the load balancer service **PgBouncer**.
3. Click **Continue**.
4. You will be presented with a **Import PgBouncer service** import wizard. In the first step, under Authentication, specify the following fields below:
   - **PgBouncer admin user**: PgBouncer admin username you currently used in your PgBouncer
   - **PgBouncer admin password**: The password for the PgBouncer admin user.

5. Under the **Add nodes** section, specify the nodes where to import PgBouncer and the Listening port to be used (default is 6432).
6. Click **Continue** to proceed to the **Preview** step. In this section, you can see the summary of your import and if everything is correct, you may proceed to import the load balancer by clicking **Finish**. You can always go back to any previous section to modify your configurations if you wish. The import settings will be kept until you exit the import wizard.
7. ClusterControl will trigger an import job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Import an existing PgBouncer node:

```
s9s node \
    --register \
    --cluster-id=4 \
    --nodes="pgbouncer://10.10.10.107" \
    --wait \
    --log
```

2.13.8 Keepalived

This section is for Keepalived import. Keepalived requires two or more HAProxy, ProxySQL, or MariaDB MaxScale instances to provide virtual IP address failover. By default, the virtual IP address will be assigned to instance 'Keepalived 1'. If the node goes down, the IP address will automatically fail over to 'Keepalived 2' accordingly.

Keepalived uses the IP Virtual Server (IPVS) kernel module to provide transport layer (Layer 4) load balancing, redirecting requests for network-based services to individual members of a server cluster.

See also

How ClusterControl Configures Virtual IP and What to Expect During Failover

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Select a cluster → Cluster Actions → Add new → Load balancer*** and choose **Import balancer**.
2. Choose the load balancer service **Keepalived**.
3. Click **Continue**.
4. You will be presented with a **Import Keepalived service** import wizard:
    - **Keepalived**: Choose the servers that contains your Keepalived instance. Take note that importing a Keepalived requires that the host/node shall be existing in the cluster.
5. Click **Continue**.
6. Under the **Host configuration** section, specify the virtual IP address and network interface that is currently used by your existing Keepalived in your database cluster.

Warning

Attention

```
    Make sure the network interface matches the name of the network␣
↪interface in the nodes.
```

```
7.  Click **Continue** to proceed to the **Preview** step. In this section,␣
↪you can see the summary of your import and if everything is correct, you␣
↪may proceed to import the load balancer by clicking **Finish**. You can␣
↪always go back to any previous section to modify your configurations if␣
↪you wish. The import settings will be kept until you exit the import␣
↪wizard.

8. ClusterControl will trigger an import job and the progress can be␣
↪monitored under _**ClusterControl GUI → Activity Center → Jobs**_.
```

2.14 Observability

This section explores the essential strategies and tools for monitoring and analyzing your database clusters with ClusterControl. Learn how to gain full visibility into performance, detect issues proactively, and ensure optimal health of your database environment. You can also scale observability as your clusters grow.

See also

- *Database Status and Healthchecks →*
- *Host Monitoring →*
- *Query Monitoring →*
- *Database Performance →*
- *Database Logs →*
- *Database Audit Logs →*
- *Watchlist →*
- *Activity Center→*
- *Operational Reports →*

2.15 Database Status & Health Checks

There are several ways to perform database health checks to ensure it runs correctly. ClusterControl provides comprehensive features to monitor your database clusters. Here's how to effectively check database health and status:

2.15.1 Cluster Summary

The Cluster Summary page appears after login into the ClusterControl UI. The Cluster Summary describe the status of each cluster database either it is in `Operational`, `Failed`, `Offline`, `Shutdown`, `Recovering` or `Unreachable/Unknown`. There is also information about global alert in Cluster Summary which give information about Critical alerts across all clusters, Warning notifications, Informational messages.

There is a filter on the right side in Cluster Summary which you can filter the status of Cluster either Operational, Unknown State, Failed, or All.

How to access the Cluster Summary:

ClusterControl GUI

1. Login into the ClusterControl UI.
2. The landing page will go to the Home menu and you will see the status of each database cluster, nodes, and alarms too.

ClusterControl CLI

1. Check status of the cluster.

```
s9s cluster --list --long
```

2. Filter the cluster specific by `clusterID`.

```
s9s cluster --cluster-id=<clusterID> --list --long
```

2.15.2 Dashboard Metrics

The dashboard metrics provides a comprehensive set of real-time and historical metrics on its dashboard to monitor your database clusters from system-level metrics, database specific, cluster health status, performance advisors.

Cluster Load

The cluster-wide load metrics to help monitor the overall performance and health of entire database cluster (not just individual nodes). These metrics give a high-level view of how well the cluster is handling workloads and whether it's operating efficiently. It give insight related to `Total Queries Per Second (QPS)`, `Read vs. Write Ratio (%)`, `Slow Queries Count`, `Connection`, `Active Connections`.

Monitor the Cluster Load in ClusterControl:

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the **Cluster Overview**, and you will see the **Cluster Load** metrics.

Cluster Health Status

The Cluster Health metrics provide a holistic view of the database cluster's operational state, ensuring high availability and quick issue detection. The node status metrics display real-time condition on each servers, clearly indicating roles (primary/master for write operation or secondary/slave for read scaling), the replication status, and health states (online, offline, recovery).

Utilize the Cluster Health Metrics in ClusterControl using statistics information in the database overview:

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the `Cluster Overview`, and you will see information about status of Primary & Secondary nodes status of database. Hover the pointer into the each column will describe the ToolTip which describe information about the column.
4. The statistics can be expanded, there is icon in the right side to expand the data. For example in MariaDB/MySQL-based, information available in Primary Node are: Instance, ServerID, ExecutedGTID, Binlog File, Binlog Pos, Binlog do DB and Binlog ignore DB, Semisync status.

Note

Note Available in MySQL/MariaDB and PostgreSQL

There is information to check the status database.

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the `MySQL Server - General` or `MongoDB Server`, and it will see information about database Uptime, queries per second, percentage of connection used, and buffer hit ratio.

Note

Note Available in MySQL and MongoDB

Another metric that can be used to check the database status and health is the Load Average Ratio. The Load Average refers to the average number of processes that are either running on the CPU or waiting for CPU time.

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the `Cluster Overview`, and it will see information Load Average in 1 minute, 5 minutes, and 15 minutes. The default range of time metrics is 15 minutes.

2.15.3 Performance Advisors

The Performance Advisor in ClusterControl is an intelligent monitoring feature that automatically analyzes database clusters to detect performance issues, configuration problems, and potential risks, providing actionable recommendations for optimization.

There are a number of metrics monitored by the advisors, such as:

- CPU usage
- Disk Space usage
- Connection used
- Connection ever used
- InnoDB Dirty Pages
- Top Queries
- Numa checks
- Replication check

The thresholds, justifications, and advice are based on a custom configuration, which is flexible to manage. The advisors can be managed through:

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the ***Manage → Advisors***.
3. There is ***Action*** on each type of Advisories and some menu such as **Edit scripts** (modify the advisors script based on JavaScript), `Edit Param` (change the schedule and parameter argument), and **Disable** the advisor.

Monitor the Performance Advisors can be accessed in ClusterControl

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the ***Performance → Advisors***.
3. There will be information about type of Advisories, Host, Status, Justification, and Advice.

Host Monitoring in ClusterControl is designed to provide comprehensive insights into the health, performance, and availability of the hosts (servers) within the database cluster.

It can monitor status of the nodes and key metrics such as CPU usage, memory consumption, disk I/O, network activity, and many more.

2.16 Cluster Node Summary

The Cluster Node Summary is part of the Cluster Summary where is available after we login into ClusterControl. The Cluster Node Summary on the home page provides a quick overview of the current status of each node in database cluster.

Here is what it typically includes:

- Node Name/Hostname – Identifies the server/node.
- Status – Indicates if the node is Operational, Failed, Offline, Shutdown, Recovering or Unreachable/Unknown.
- Role – Shows the role of the node (e.g., Master, Slave, Primary, Secondary, depending on the database type).
- Type – Describe of the database engine running on the host.
- Cluster name – Identifies the cluster name to which the node belongs.

To get the cluster node summary:

ClusterControl GUI

1. Login into the ClusterControl UI.
2. In the **Home** section and you will see the **Nodes** group which appears in the **Status Overview** and **Nodes** itself.
3. The **Status Overview** inside the **Nodes** group shows the summary of the nodes. The green color indicates that the nodes status in operational while the red indicates the nodes is offline. If you rollover to one of the node, it will show more information about the node such as Hostname, Role, Type, Cluster Name, and Status.

ClusterControl CLI

1. Check status of the nodes and there will be information about cluster name, hostname, port, and status of the nodes.

```
s9s node --list --long
```

2. Filter the nodes specific by `clusterID`.

```
s9s node --cluster-id=<clusterID> --list --long
```

The ***clusterID (CID)*** is the unique identifier for each cluster.

2.17 Node Health Status

The node health status represents the current operational state of each individual server (or node) within a database cluster. It is a key part of monitoring and helps you quickly identify if a node is running as expected or experiencing issues.

2.17.1 CPU Utilization

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the **System Overview**, and you will see information about CPU Usage.
4. The CPU Usage metrics describe each status that is running in the nodes such as : Idle, User, System, Iowait, Irq, Streal.

2.17.2 Memory Utilization

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the **System Overview**, and you will see information about memory Usage.
4. The **Memory Usage** metrics show the **Free Memory** and **Used Memory** statistics.

2.17.3 Disk IO Utilization

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to specific cluster, and choose the Dashboard.
3. Choose the **System Overview**, and you will see information about disk I/O utilization.
4. The **Disk IO Util** metrics show percentage usage on each storage and mount points.

2.18 Node Menu

The Node Menu in ClusterControl is a context-specific menu accessible from the ClusterControl UI, typically by clicking on a specific database node listed in the cluster view or in the Nodes menu on the left side of the interface. The Node Menu serves as a key interface for accessing monitoring capabilities related to the status of services on each host.

It also provides a set of actions and detailed options for managing and monitoring the selected node.

ClusterControl GUI

1. Login into the ClusterControl UI.
2. Go to the ***Nodes*** menu in the left side of ClusterControl UI.
3. In the header, it appears the total of nodes in several categories (`Operational`, `Failed`, `Offline`, `Shutdown`, `Recovering`, `Unkown State`, `All`).
4. Beside the summary header, it appears list of the host with information such as: Type, Status, Role, version, Cluster, Last seen, and last one is Action menu. The actions menu provide some menu for managing the selected nodes (for example: Restart Nodes, Reboot Host, Stop Node, Remove Nodes, Rebuild Replication, Change Replication)

The Query Monitoring feature is a powerful tool designed to provide real-time visibility into the queries running on your database. It helps administrators identify performance bottlenecks, optimize query execution, and ensure the overall health of the database. This feature includes real-time query monitoring, top queries analysis, and query outlier detection, all of which work together to provide actionable insights into database performance.

Every database cluster managed by ClusterControl has it is own Query Monitoring. This will help monitor queries to the specific database cluster. Ideally, after the database cluster has been provisioned, administrators can set up the Slow Query Log. The purpose of the Slow Query Log is to capture slow queries based on a configured threshold.

2.19 Configure Slow Query Log

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Performance → Query Monitor***, go to the ***ettings***.
2. Enable the Query Monitor toggle and set the thresholds for long query time. The unit for long query time is in seconds. For example, we can set the threshold to 5 seconds.
3. In addition to the slow query threshold, we can enable the option to log queries that do not use indexes. This means any query performing a full table scan will also be logged.

ClusterControl CLI

1. Change the value of the long query time in s9s CLI is very straightforward. Define the `--cluster-id`, and the value of new slow query threshold. The long query time defined as seconds.

```
s9s cluster --change-config --cluster-id=<clusterID> --opt-
↪name=long_query_time --opt-value=5
```

2. Verify the config by executing the following command:

```
s9s cluster --list-config --cluster-id=<id> | grep -i long_query_time
```

   The clusterID is the unique identifier belong to the cluster.

2.20 Slow Query Monitor

Queries that exceed the slow query threshold and those that do not use indexes will appear in the Top Queries section.

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Performance → Query Monitor***, go to the ***Top Queries***.
2. It provides detailed information about each query, including the query text, execution time, user, database, and state (example: `running`, `idle`, `waiting`, or `completed`).

The next step after identifying the slow queries is to manually check the execution plan for each query (example: missing indexes, inefficient joins, and other issues) and make the necessary corrections.

2.21 RealTime Query Monitor

Administrators can monitor queries running in the database in real time, which is beneficial in the event of an incident. We can immediately check which queries are causing performance issues. Key aspects to watch include the query execution time, query status (active, idle, or waiting), the schema, and the user executing the queries.

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Performance → Query Monitor***, go to the DB Connections.
2. It provides detailed information about each query, including the query text, execution time, user, database, and state (example: `running`, `idle`, `waiting`, or `completed`).

2.22 ProxySQL for Query Monitor

Administrators can also monitor queries in real time through ProxySQL. It monitors queries from the perspective of the database load balancer. The ***Top Queries*** section in ProxySQL provides insights such as the number of queries executed (count_star), total query time (sum_time), and the timestamps for when queries were first and last seen. The ***Process List*** provides information about the queries being executed, including the host group, session ID, thread ID, user, and execution time.

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Nodes → Node List***, go to the ProxySQL node.
2. There is ***Actions*** menu in the right side, just click the menu and choose ***Node details***.
3. There are several tabs in the ProxySQL node list, and administrators can monitor the queries in the ***Top Queries***.

Note

Note ProxySQL is database load balancer that is currently supported for MySQL (Replication and Percona XtraDB Cluster), MariaDB (Replication & Galera Cluster)

2.23 Reset Query Monitoring

After the queries have been fixed, they may still appear in the ***Top Query Monitor***. Administrators can reset the query monitoring to start fresh.

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Performance → Query Monitor***, go to the Top Queries.
2. There is button ***Clear all queries*** to reset the query monitoring or enable ***Auto Clear Queries*** toggle in the Settings.

2.24 DB Growth

Provides a summary of your database and table growth on daily basis for the last 30 days. On the first line of the Top 25 Largest Databases grid, you should notice the actual size of the MySQL data directory (with a folder icon). This is useful to determine whether any other files that exist in the data directory may consume huge spaces e.g binary log, error log or MySQL general log.

Field	Description
DB growth	A summary of your database and table growth daily.
Top largest tables	A summary of the largest tables per database daily.
Update data in controller	Triggers a job to re-calculate the database size.
Actions → View	Shows the details per database on table count, row count, data size, index size, and database size.

Click on a database listed for further details on growth summary per table.

2.25 DB Status

Provides a comprehensive overview of your database's current state. It offers insights into various aspects of the database, such as connection status, data usage, and server performance. This feature is particularly useful for database administrators and developers as it allows them to monitor the health and performance of the database, identify potential issues, and make informed decisions to ensure optimal database operation.

By default, ClusterControl performs status collection every 30 seconds. This is configurable using the `db_stats_collection_interval` parameter inside the controller's configuration file.

Field	Description
Show differences	Shows a list of status and values that are not identical among each node (highlighted in red).
Filter Server	Specify the database server you want to view. Multiple values accepted.
Search	Filter the results.

2.26 DB Variables

Provides detailed information about the configuration of your database. It allows you to view and manage various parameters that control the behavior of your database system. These variables can include settings related to memory allocation, data storage, query optimization, and more. This feature is particularly beneficial for database administrators and developers, enabling them to customize the database environment to best suit their specific needs and application requirements.

By default, ClusterControl performs variable collection every 30 seconds. This is configurable using the `db_stats_collection_interval` parameter inside the controller's configuration file.

Field	Description
Show differences	Shows a list of variables and values that are not identical among each node (highlighted in red).
Filter Server	Specify the database server you want to view. Multiple values accepted.
Search	Filter the results.

2.27 Advisors

Lists of scheduled advisors' results created in ***Manage → Advisors*** and ***Manage → Scripts*** using ClusterControl DSL. You can think of it as a 'scheduled mini-program' that executes a script created inside ClusterControl and produces a result containing status, advice, and justification. You can get detailed schedules by rolling over to the Advisor's name.

Field	Description
Manage advisors	Opens the ***Manage → Advisors*** for advisor's management.

2.28 Schema Analyzer

Analyze your database schemas for missing primary keys, redundant indexes, and tables using the MyISAM storage engine. Galera Cluster needs an explicitly defined primary key on each table (unique key does not count). MyISAM tables are not recommended in Galera. ClusterControl will periodically check the schemas for these (default every 8 hours or every CMON restart), and raise an alert if necessary.

Field	Description
Tables without Primary Keys	List of tables without primary keys. A primary key is important in Galera. The `DELETE` operations are unsupported on tables without a primary key. Also, rows in tables without a primary key may appear in a different order on different nodes.
My ISAM Tables	MyISAM does not support transactions. However, the DMLs for MyISAM should also work but it's still experimental in Galera.
Redundant Indexes	Having duplicate keys in schemas can hurt the performance of the database:
	* They make the optimizer phase slower because MySQL needs to examine more query plans.
	* The storage engine needs to maintain, calculate and update more index statistics.
	* DML and even read queries can be slower because MySQL needs to update and fetch more data to Buffer Pool for the same load.
	* Data needs more disk space so the backups will be bigger and slower.

2.29 Transaction Deadlocks

Available for: MySQL-based clusters and MariaDB-based clusters

Lists out long-running transactions and deadlocks across database clusters where you can easily view what transactions are causing the deadlocks. The default query time threshold is 30 seconds. This is configurable in the CMON configuration file under `db_long_query_time_alarm` configuration option.

Click on the listed query to see the output of InnoDB status for detailed debugging.

Field	Description
Deadlock check interval	How often to check for deadlocks in seconds.
	0 means disabled (default).
	Deadlock detection will affect CPU usage on database nodes.

2.30 Running Operations

Available for: MongoDB-based clusters

Provides a view of current running operations similar to `db.currentOp()` command.

2.31 System Logs

Tree view of corresponding database-related logs across a cluster. Clicking on a log file will appear in a new tab in the viewing panel on the right-hand side. Browsing through the appropriate error logs is the key element of the troubleshooting process.

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Logs → System Log***.
2. It provides detailed log from the operating system, database logs, and load balancer logs for the database.

Click on ***Refresh logs*** will trigger a job for ClusterControl to perform log collection.

2.32 Audit Logs

Shows records of ClusterControl's events and changes:

- User authentication, cloud credentials, database configuration, and user changes.
- CMON configuration changes and jobs.
- ClusterControl user management changes (Users and ACLs), operational reports.

ClusterControl GUI

1. In the ClusterControl UI, go to the cluster and choose the tab ***Logs → Audit Log***.
2. Each log entry has the activity, type of activity, user, hostname or IP address (where the user comes from), cluster name, and timestamp. Rollover on the ***Cluster*** entry will pop up a cluster information card summarizing the cluster status. Rollover on the ***When*** entry will pop up a tooltip containing the exact timestamp with timezone.

2.33 Audit Logging

Enable audit logging on the cluster using policy-based monitoring and logging of connection and query activity for security and compliance purposes. It enables database administrators to track and monitor database operations, such as logins, queries, data modifications, and schema changes. This plugin records these activities in a structured format, making it easier to analyze and audit database usage. This feature will enable audit logging on all nodes in the cluster.

!!! Notes

```
Available in MariaDB Replication, MariaDB (Galera Cluster), PostgreSQL and␣
↪TimescaleDB.
```

2.33.1 MariaDB

For MariaDB audit logging, the following fields are presented:

Field	Description
Log Path	The filename to store the audit log in the log directory `/var/log/mysql/`. Alternatively, it can contain a path relative to the data directory or an absolute path. The default is `server_audit.log`.
Rotation Size in MB	Log file size in MB before log rotation happens. Changing this default value will require a cluster restart.
Rotation	Number of log files to keep after rotation.
Events	Specify MariaDB audit events that you would like to capture. ClusterControl preloads the audit events as you type. Multiple values are allowed.
Exclude Users	Exclude the specified MariaDB user(s) from the auditing. ClusterControl preloads all database users in a dropdown. Multiple values are allowed.

To enable audit logging for MariaDB-based clusters:

ClusterControl GUI

1. Go to the MariaDB cluster, Click on ***Actions Menu → Audit Logging***.
2. Just enable the toggle ***ON*** in the ***Audit Logging*** option.
3. There will be a dialogue box to fill related to the Log Path, Rotation, Events type, and Exclude the Users.

ClusterControl CLI

1. Enable audit logging through CLI as easy as the command below:

```
s9s cluster --setup-audit-logging --cluster-id=<clusterId>
```

   The `clusterId` indicate identifier of your MariaDB cluster.

2.33.2 PostgreSQL

For PostgreSQL/TimescaleDB audit logging, the following fields are presented:

Field	Description
Event	Specify PostgreSQL audit events that you would like to capture. ClusterControl preloads the audit events as you type. Multiple values are allowed.

To enable audit logging in PostgreSQL-based environment:

ClusterControl GUI

1. Go to the PostgreSQL/TimescaleDB cluster, Click on ***Actions Menu → Audit Logging***.
2. Just enable the toggle ***ON*** in the ***Audit Logging*** option.
3. There will be a dialogue box to fill the ***Events*** (eg. `ROLE`, `DDL`, `READ`, `WRITE`, `ALL`).

ClusterControl CLI

1. Enable audit logging through CLI as easily as the command below:

```
s9s cluster --setup-audit-logging --cluster-id=<clusterId>
```

2. Enable audit logging for specific roles or events through CLI:

```
s9s cluster --setup-audit-logging --cluster-id=<clusterId> --
↪audit-log-events-data=role,ddl
```

The `clusterId` indicates the identifier of your PostgreSQL cluster.

After enabling the audit log in the database, the output of the audit activity will be stored in the audit log file.

Warning

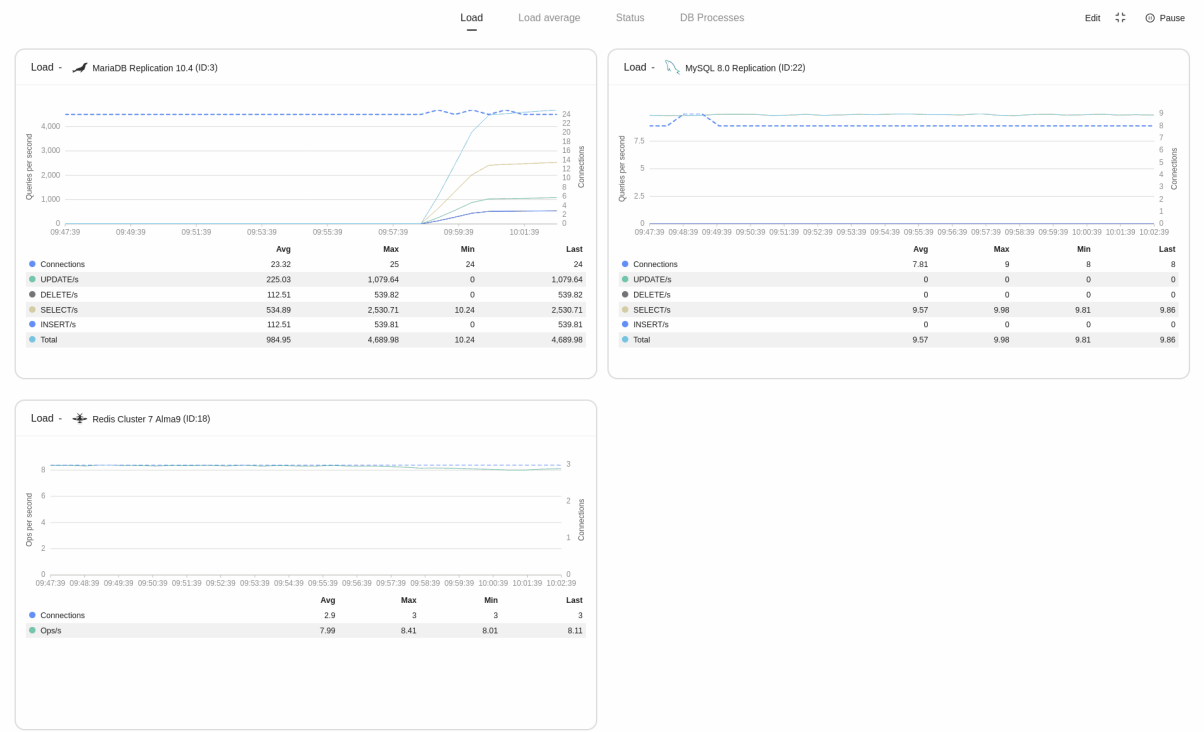
Attention

This feature will be enabled on all nodes of this cluster, cluster will be restarted to install the new plugin service.

2.34 Watchlist - Monitoring dashboards in kiosk-mode

Introduced in v2.3.2, Watchlist allows users to monitor the database clusters in a full-screen view (a.k.a kiosk-mode), suitable for displaying the monitoring dashboards on a dedicated screen in an operation center. Administrators can curate a watchlist based on topics, where ClusterControl will visualize the related monitoring panels based on the chosen topics and clusters in full screen view.

Every created watchlist has its own unique URL, where administrator can share the generated URL with other non-administrative users for active monitoring purposes (ClusterControl user login is required). Here is what it looks like in action (click to enlarge):



Info

This feature is still in beta and feedbacks are welcome. See *Give us feedback*.

2.34.1 Access Watchlist management section

ClusterControl GUI

Log in to your ***ClusterControl GUI → Activity center → Watchlists***. This will bring up a panel where you can view and edit watchlist.

2.34.2 Add clusters directly into watchlist

Apart from using the *Watchlist management section* to manage which clusters belongs to which watchlists, you can directly add clusters into a watchlist from the *Clusters Actions*.

ClusterControl GUI

1. To add a cluster into a new watchlist, log in to your ***ClusterControl GUI → Clusters → choose a cluster → Actions → Watchlists → Add to new watchlist***.
2. If you already have watchlists created, you can also add the chosen cluster into any existing watchlists by going to ***ClusterControl GUI → Clusters → choose a cluster → Actions → Watchlists → toggle On of the available watchlists***. This cluster will be added to all enabled watchlist view immediately. Toggling Off will remove the cluster from the corresponding watchlist.

2.34.3 Create watchlist

ClusterControl GUI

1. Click on the **Create watchlist** button. It will open a new **Add watchlist** panel.
2. Specify a unique name for the watchlist under the **Name** text field. This unique name will be used to generate a unique URL for the watchlist.
3. Choose one or more **Topics** from the list. Topics defines key metrics or dimensions users want to monitor in their watchlists — such as load, database growth, or status. These are visualized in dashboards and grid views when the watchlist is activated.
4. Choose the group of dashboards that will be displayed in one page, either by **Topic** or **Cluster**. If **Cluster**, all dashboards for the selected topics will be grouped per cluster in one page. If **Topic**, dashboards for all clusters will be grouped per topic in one page.
5. Choose a grid layout to align elements into columns and rows format (columns x rows). Depending on the screen size, you might want to choose the right grid layout to maximize the full-screen view. For a wide screen, 3x2 is usually better than 2x3.
6. Choose one or more clusters, or all clusters from the **Clusters** dropdown list.

2.34.4 Monitor in kiosk-mode

ClusterControl GUI

1. As an administrator, you can directly open the watchlist panel by going to ***Actions → Open***. A new tab will be opened specifically for the full-screen monitoring dashbards.
2. Click on the `[ ]` icon to enter the full-screen view. **Edit** to edit the watchlist settings and **Pause** to pause the animation in the current page.

Tip

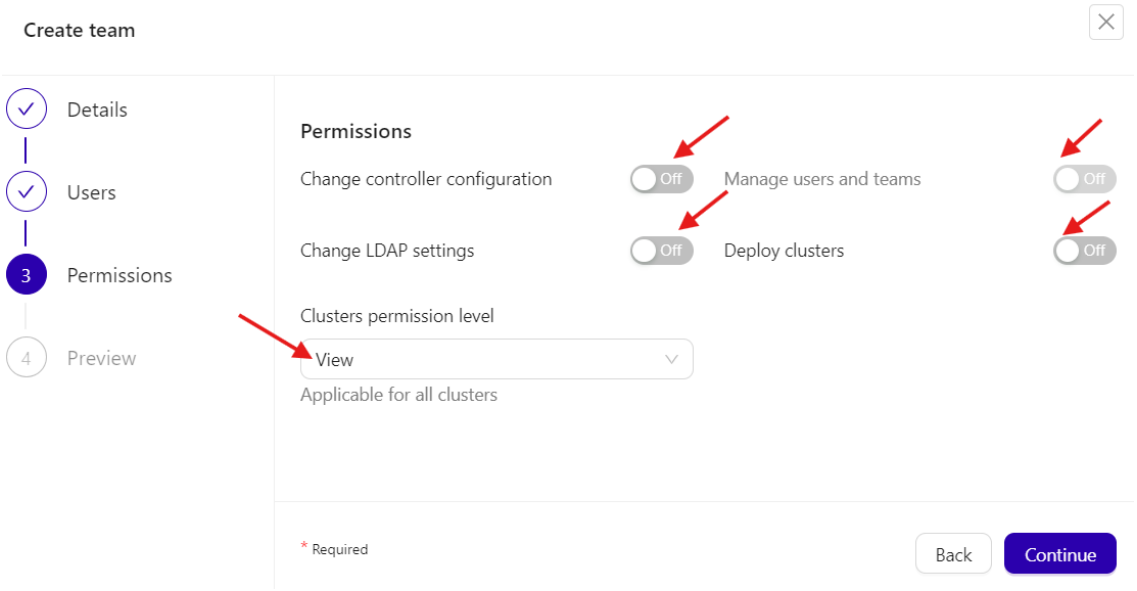
For non-administrative monitoring (commonly for NOC or SOC team), create a read-only user for this purpose. See *Share watchlist with other*.

2.34.5 Share watchlist with others

It is a common practice for the kiosk-mode monitoring to be passed over to the infrastructure monitoring team which has lower access level (can not edit the watchlist) to ClusterControl for monitoring-only purposes.

ClusterControl GUI

1. Create a read-only monitoring team called “monitor” by going to ***ClusterControl GUI → User management → Create user or team → Create team***. You may skip the **Users** part since we are creating a group first (will add the user later).
2. Under the **Permissions** section, toggle off all global permissions and choose “View” for cluster permission level, similar to the following example:



3. Create a user called "monitor" by going to ***ClusterControl GUI → User management → Create user or team → Create user***. Assign this user under group "monitor", which is a read-only group we created in the previous steps.
4. From the Watchlists tab, find the watchlist that you want to monitor in kiosk-mode ***Actions → Copy link***. Pass the "monitor" user credentials and the link to the operation team for monitoring. Use the provided link in the web browser and they are good to go.
5. Click on the `[ ]` icon to enter the full-screen view and **Pause** to pause the animation in the current page. Read-only user does not have the permission to edit the watchlist.

2.34.6 Delete watchlist

ClusterControl GUI

1. As an administrator, find the watchlist in the list and go to ***Actions → Delete***. Confirm the action and the watchlist will be deleted.

The Activity Center in ClusterControl is a centralized hub that provides a comprehensive overview of all system activities, events, and tasks. It consolidates critical information from alarms, jobs activity, and audit logs into a single, easy-to-navigate interface. This feature is designed to enhance visibility, streamline monitoring, and simplify troubleshooting by offering a unified view of what is happening within the database environment.

2.35 Alarms

Alarms indicate problems that could affect or degrade the cluster as a whole. Each alarm is categorized as:

- Cluster.
- Cluster recovery.
- Database health.
- Database performance.
- Host.
- Node.

- Network.

Accessing the alarms in ClusterControl can go to the following:

ClusterControl GUI

1. Click on ***Actions → Details***, to get a detailed explanation of the problem, together with the recommended action (if available) to resolve the problem.
2. An alarm can be acknowledged by clicking on ***Actions → Mute***. When muted, no notification will be sent via email. However, an alarm cannot be deleted or dismissed.

ClusterControl CLI

1. Check all the alarm through CLI

```
s9s alarm --list --long
```

2. Filter the alarms specific cluster.

```
s9s alarm --cluster-id=<clusterID> --list
```

3. Remove the alarm from CLI can be done by executing the following.

```
s9s alarm --delete --alarm-id=<alarmId>
```

The `alarmId` can be found from the list of alarm.

2.36 Jobs

Shows aggregated view of jobs that have been initiated and performed in a cluster by ClusterControl (e.g., promote replica, enable read-only, create/restore a backup, etc). Each job entry has a job title, job status, cluster name, the user who started the job, timestamp, and duration.

ClusterControl GUI

1. Rollover on the ***Cluster*** entry will pop up a cluster information card summarizing the cluster status. Rollover on the ***When*** entry will pop up a tooltip containing the exact timestamp with timezone.
2. Click on the ***Actions*** ** → ** ***Details*** to see the job specification and messages. Inside the ***Job details*** popup, you have the ability to see the full job definition sent to the controller service for that particular job by clicking on the ***jobSpecs*** under the job's name. Underneath it is the full job messages in descending order (newer first) returned by the controller service. The ***Copy to clipboard*** button will copy the content of the job messages to the clipboard. The job messages are commonly the first thing to look for when troubleshooting an issue that is happening to a cluster. It is highly recommended to include the job messages when opening a support request from us.

ClusterControl CLI

1. Check the history of the job

```
s9s jobs  --list --long
```

2. Dig the detailed logs of specific job

```
s9s job --log --job-id=<value>
```

The `job-id` can be found in the point `1` from the list of job are appear.

This allows administrators to monitor the progress of ongoing tasks, review the history of completed jobs, and troubleshoot any failed operations. The Jobs Activity feature is particularly useful for ensuring that automated tasks are executed as expected and for auditing changes made to the database environment.

2.37 Audit Log

Shows records of ClusterControl's events and changes:

- User authentication, cloud credentials, database configuration, and users changes
- CMON configuration changes and jobs
- ClusterControl user management changes (Users and ACLs), operational reports

Each log entry has the activity, type of activity, user, hostname or IP address (from where the user comes from), the corresponding cluster name, and timestamp.

ClusterControl GUI

1. In the ClusterControl UI, go to ***Activity Center → Audit Log***, check the activity of the users.

The Audit Log is essential for maintaining accountability, ensuring security, and facilitating compliance with operational and regulatory standards.

Operational Reports offer a detailed view of the cluster's health, performance, and outstanding issues, serving as a comprehensive reporting tool for administrators. This feature is designed to facilitate the creation of stakeholder reports on the cluster's condition over a specified time frame.

Reports can be generated manually or scheduled for automatic creation, with the option to include email recipients during the setup process. Only users belonging to the admin team are authorized to create or access Operational Reports. Non-admin users are strictly prohibited from performing these actions.

2.38 Report templates

ClusterControl offers several predefined report templates to address various operational requirements. These templates are:

Template	Description
Database Availability Reports	Details the uptime, downtime, and overall availability for each managed database cluster during the specified reporting period.
System Reports	Provides a summary of overall system health, covering node status, performance metrics, resource utilization, cluster events, and service availability over the reporting period.
Backup Reports	Summarizes the backup activities for each managed database cluster during the selected period.
Upgrade Reports	Offers an upgrade report for a managed cluster, including available package versions for the cluster (database packages, security packages, and libraries).
Database Growth Reports	Summarizes database growth over different timeframes, such as the last day, week, and month.
Schema Change Reports (MySQL-based systems only)	Provides reports for any modifications executed on the monitored databases. To utilize these predefined templates, you must configure the `schema_change_detection_address` and `schema_change_detection_databases` parameters in the ClusterControl settings. More information on these parameters can be found in the *references* section.

Note

Note Operational reports are produced in HTML format, with each report corresponding to a specific cluster.

Info

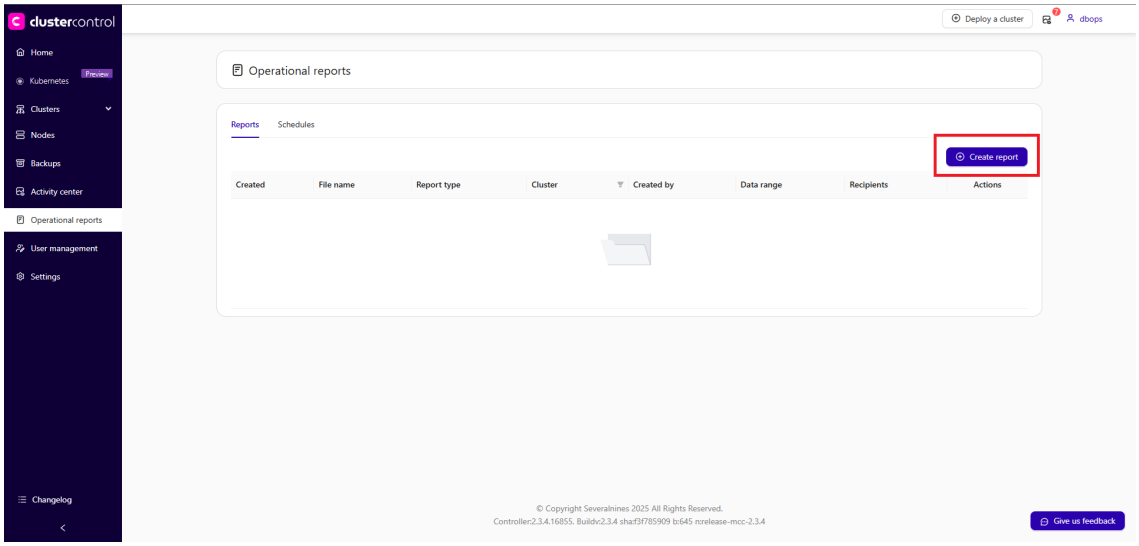
The reports generated may contain confidential information. It is essential to ensure that they are shared exclusively with the appropriate personnel.

2.39 Generating a report

The following steps show how to generate an operational report:

ClusterControl GUI

1. Go to ***Operational Reports → Reports → Create report***:



2. Under the **Create report** section, specify the following information:
   - **Cluster**: Select a cluster from the dropdown.
   - **Type**: Select a report to be generated from the dropdown for this cluster.
   - **Data range**: Specify the report's data range in days.
   - **Recipients**: Specify the email recipients. This field supports adding multiple email addresses after an ++enter++. You can specify any email address, not limited to what has been configured in ClusterControl.
3. Click **Create** to generate the report.
4. The generated report will be listed under the **Reports** tab.

ClusterControl CLI

1. List available template repots:

```
s9s report --list-templates
```

Example

```
$ s9s report --list-templates
report1
availability
backup
upgrade
schemachange
dbgrowth
```

2. Generate an availability report for cluster ID 37:

```
s9s report --create --cluster-id=37 --type='availability'
```

2.40 Listing and viewing reports

The following steps show how to list and view generated operational reports:

ClusterControl GUI

1. Go to ***Operational Reports → Reports*** to see the list of generated reports.
2. Each report can be viewed or deleted by clicking the action button and choose **View** or **Delete**. When viewing via the GUI, the report will be opened in a new web browser window.

ClusterControl CLI

1. List all generated operational reports:

```
s9s report --list
```

2. List all generated operational reports for cluster ID 37:

```
s9s report --list --cluster-id=37
```

3. View a generated report for cluster ID 37:

```
s9s report --cat --report-id=2 --cluster-id=37
```

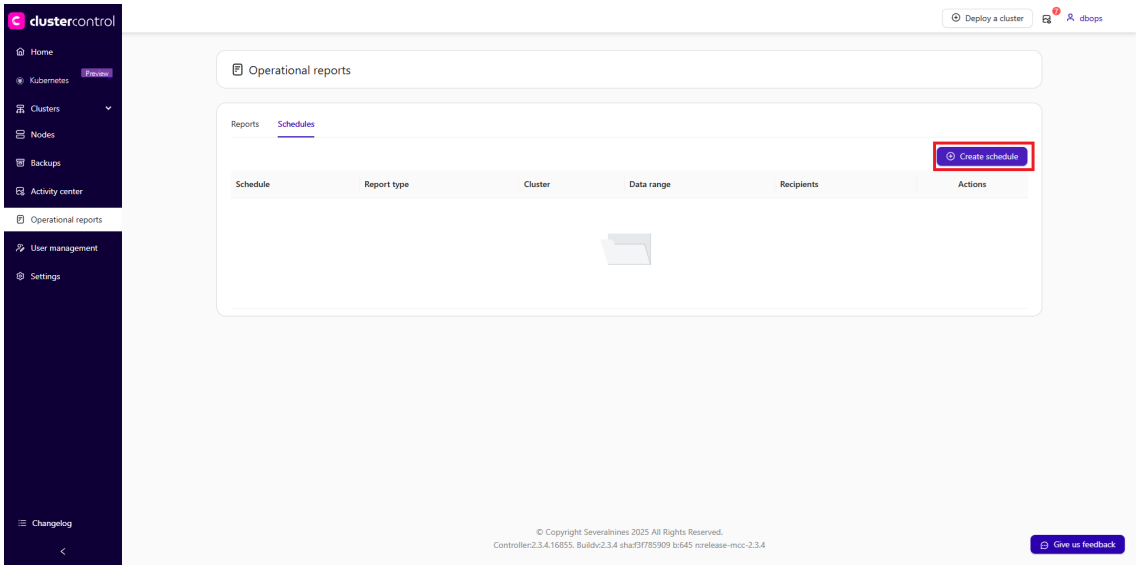
2.41 Scheduling a report

A report can be scheduled to be generated and delivered via email at specified intervals (e.g., daily, weekly, monthly).

The following steps show how to schedule an operational report:

ClusterControl GUI

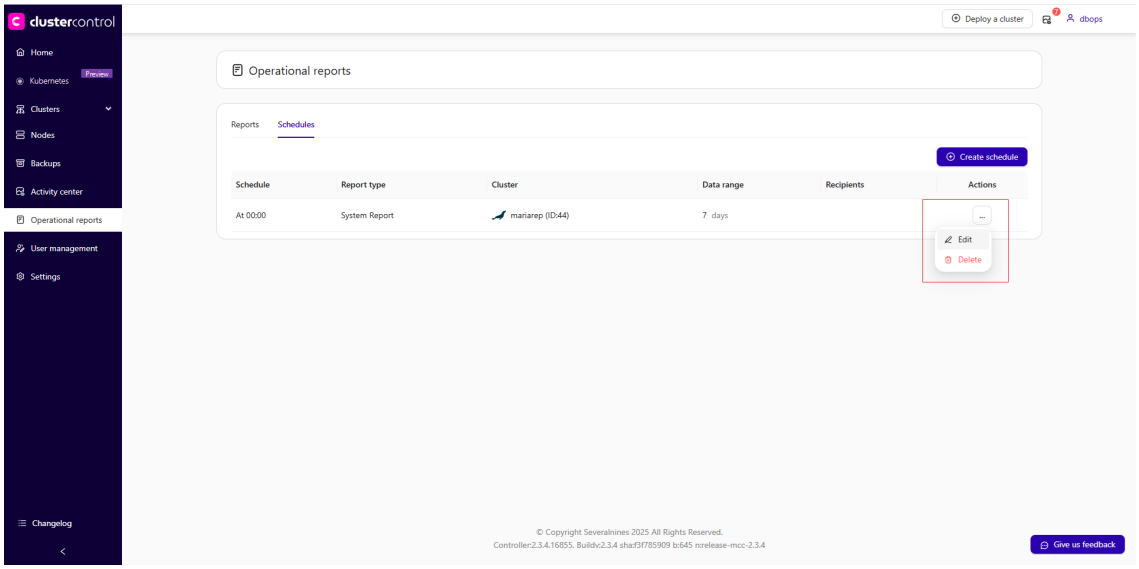
1. Go to ***Operational Reports → Schedules → Create schedule***:



2. Under the **Schedule report** section, specify the following information:

- **Cluster**: Select a cluster from the dropdown.
- **Type**: Select a report to be generated from the dropdown for this cluster.
- **Data range**: Specify the report's data range in days.
- **Recipients**: Specify the email recipients. This field supports adding multiple email addresses after an ++enter++. You can specify any email address, not limited to what has been configured in ClusterControl.
- **Set schedule**: Set the frequency or specific time to generate the report. The schedule can be configured to run minutely, hourly, daily, weekly, monthly, or yearly.

3. Click **Create** to save the schedule.
4. After the schedule is created, it can be edited or deleted by clicking the action button and choose **Edit** or **Delete**:



2.42 Management

This section highlights the key strategies and tools for managing your database clusters with ClusterControl. Learn how to simplify day-to-day operations, automate routine tasks, and maintain high availability and performance.

See also

- *Failover and Recovery →*
- *Cluster Management →*
- *Nodes Management →*
- *Configuration Management →*
- *Upgrades and Patches →*
- *Alerts and Notifications →*
- *Database Users and Privileges →*
- *Graceful Shutdown →*

- *Maintenance →*
- *Certificates →*

2.43 Failover and Recovery

ClusterControl is equipped with automated recovery algorithms designed to address common failures in database systems. It understands various database topologies and process management, enabling it to determine the optimal recovery strategy for a cluster.

ClusterControl offers two key recovery components:

1. **Node Recovery**: This attempts to restore a node to an operational state, covering issues such as a node being stopped outside of ClusterControl's awareness (e.g., via a user-initiated SSH stop command or an OOM process kill). See *Node Recovery*.
2. **Cluster Recovery**: This component aims to bring the entire cluster topology back to an operational state. See *Cluster Recovery*.

These two components are crucial for ensuring the highest possible service availability.

Warning

Attention

ClusterControl provides automatic failover and recovery for nodes within a single cluster. However, this functionality does not extend to replication between separate clusters. Consequently, automatic recovery is not supported for cluster-to-cluster replication. In the event that the primary cluster in Data Center A becomes unavailable, user intervention is required to promote the secondary cluster in Data Center B as the new active cluster. See *Create Replica Cluster*.

2.43.1 Node Recovery

ClusterControl ensures database node recovery from intermittent failures by monitoring process and connectivity. Similar to systemd, it keeps the database service running unless intentionally stopped via the ClusterControl GUI or CLI. The recovery includes all cluster-related services like ProxySQL, HAProxy, MaxScale, Keepalived, PgBouncer, Prometheus exporters, and garbd. Special attention to Prometheus exporters where ClusterControl will try to connect to the exporter's listening port for health check and verification. Thus, it's crucial to expose the exporter ports to ClusterControl and Prometheus server to make sure no false alarms during recovery. See *Firewall and security groups* for details.

Upon a node coming back online, ClusterControl re-establishes connection (using SSH persistent session) and performs necessary recovery actions:

1. It waits 30 seconds for systemd/init script to start monitored services/processes.
2. If services/processes remain down, ClusterControl automatically attempts to start the database service.
3. If ClusterControl is unable to recover the monitored services/processes, an alarm is raised.

Note

If a user initiates a database shutdown through ClusterControl GUI or CLI, ClusterControl will not automatically attempt to recover that specific node later. The user is expected to restart it either via the **Start Node** action in the ClusterControl GUI (or ClusterControl CLI), or by explicitly using an operating system command.

2.43.2 Cluster Recovery

ClusterControl excels at database recovery by understanding the topology and adhering to best practices. For clusters with built-in fault tolerance (like Galera, NDB Cluster, Redis Cluster, MongoDB Replicaset, and Elasticsearch), failover is automatic, managed by the database server through quorum calculations, heartbeats, and role switching. ClusterControl monitors this process, updating the **Topology** view and adjusting monitoring and management components for new roles, such as a new primary node in a replica set.

For database technologies lacking built-in fault tolerance and automatic recovery, such as MySQL/MariaDB Replication and PostgreSQL/TimescaleDB Streaming Replication, ClusterControl performs recovery procedures based on vendor best practices, as explained further down. Should recovery fail, user intervention is required, and an alarm notification will be issued.

In mixed or hybrid topologies, like an asynchronous replica attached to a Galera or NDB Cluster, ClusterControl will recover the node if cluster recovery is enabled.

While cluster recovery does not apply to standalone MySQL servers, it is still recommended to enable both node and cluster recoveries for this type of cluster in the ClusterControl GUI.

MySQL/MariaDB Replication

ClusterControl supports recovery of the following MySQL/MariaDB replication setup:

- Primary-replica with MySQL GTID
- Primary-replica with MariaDB GTID
- Primary-primary with MySQL GTID
- Primary-primary with MariaDB GTID
- Asynchronous replica attached to a Galera Cluster

ClusterControl will respect the following parameters when performing cluster recovery:

- `enable_cluster_autorecovery`
- `auto_manage_readonly`
- `repl_password`
- `repl_user`
- `replication_auto_rebuild_slave`
- `replication_check_binlog_filtration_bf_failover`
- `replication_check_external_bf_failover`
- `replication_failed_reslave_failover_script`
- `replication_failover_blacklist`

- replication_failover_events
- replication_failover_wait_to_apply_timeout
- replication_failover_whitelist
- replication_onfail_failover_script
- replication_post_failover_script
- replication_post_switchover_script
- replication_post_unsuccessful_failover_script
- replication_pre_failover_script
- replication_pre_switchover_script
- replication_skip_apply_missing_txs
- replication_stop_on_error

For more details on each of the parameters, see *Configuration Options*.

ClusterControl adheres to specific rules when monitoring and managing primary-replica replication:

- All nodes, regardless of their role, are started with `read_only=ON` and `super_read_only=ON`.
- Only one primary (`read_only=OFF`) is permitted at any given time.
- The MySQL variable `report_host` is used to map the topology.
- If multiple nodes have `read_only=OFF` concurrently, ClusterControl automatically sets `read_only=ON` on all of them to prevent accidental writes. Manual intervention is then required to designate the actual primary by disabling read-only.

In the event of an active primary going offline, ClusterControl attempts a primary failover with the following sequence:

1. An alarm is raised after 3 seconds of the primary being unreachable from multiple nodes including ClusterControl node, replica nodes or load balancer nodes.
2. Replica availability is checked, ensuring at least one replica is reachable by ClusterControl.
3. A replica is chosen as a candidate for promotion to primary.
4. If GTID is enabled, ClusterControl calculates the probability of errant transactions.
5. If no errant transactions are detected, the chosen replica is promoted as the new primary.
6. A replication user is created and granted for use by the replicas.
7. All replicas previously pointing to the old primary are reconfigured to point to the newly promoted primary.
8. The replicas are started.
9. Logs are flushed on all nodes.

If the replica promotion fails, the recovery job is aborted by ClusterControl. To re-trigger the recovery job, user intervention or a ClusterControl service restart is necessary.

When the old primary becomes available again, it is started in read-only mode and is no longer part of the replication. User intervention is required to reincorporate the old primary into the replication by restaging it as a replica, or to remove the node from the cluster using ClusterControl.

PostgreSQL/TimescaleDB Streaming Replication

ClusterControl supports recovery of the following PostgreSQL replication setup:

- PostgreSQL Streaming Replication
- TimescaleDB Streaming Replication

ClusterControl will respect the following parameters when performing cluster recovery:

- `enable_cluster_autorecovery`
- `repl_password`
- `repl_user`
- `replication_auto_rebuild_slave`
- `replication_failover_whitelist`
- `replication_failover_blacklist`

For more details on each of the parameters, see *Configuration Options*.

ClusterControl will obey the following rules for managing and monitoring a PostgreSQL streaming replication setup:

- `wal_level` is set to `replica` (or `hot_standby` depending on the PostgreSQL version).
- The parameter `archive_mode` is set to ON on the primary.
- Set `recovery.conf` or `recovery.signal` file on the replica nodes, which turns the node into a hot standby with read-only enabled.

In case the active primary goes down, ClusterControl will attempt to perform the cluster recovery in the following order:

1. After 10 seconds of primary unreachability, ClusterControl will raise an alarm.
2. Following a 10-second graceful waiting period, ClusterControl will initiate the primary failover job.
3. Sample the `replayLocation` and `receiveLocation` on all available nodes to identify the most advanced node.
4. Promote the most advanced node as the new primary.
5. Stop replicas.
6. Verify the synchronization state with `pg_rewind`.
7. Restarting replicas, pointing to the new primary.

If the replica promotion fails, ClusterControl will abort the recovery job. In such cases, user intervention or a ClusterControl service restart is required to re-trigger the recovery job.

Warning

Attention

When the old primary becomes available again, it will be forcefully shut down and will not rejoin the replication setup. User intervention is necessary. See further down.

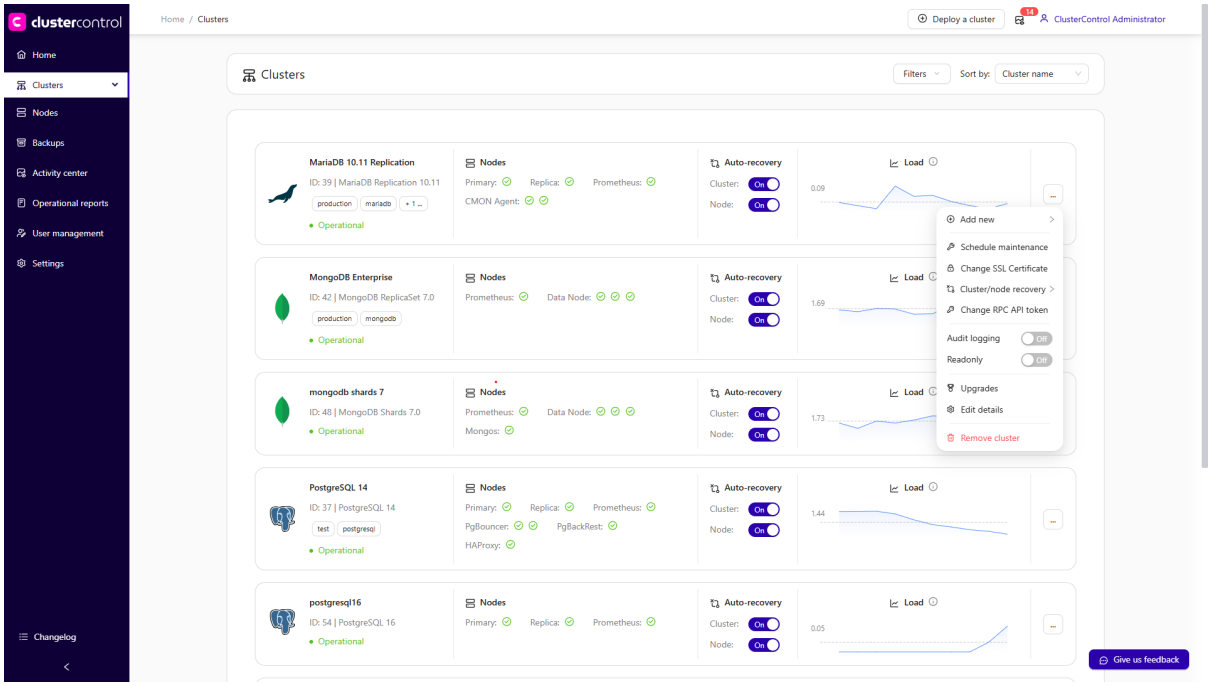
Upon the old primary coming back online, if the PostgreSQL service is running, ClusterControl will forcefully shut down the PostgreSQL service. This action prevents accidental writes, as the server would otherwise start without a recovery file (`recovery.conf` or `recovery.signal`) and thus be writable. You should expect the following lines will appear in `postgresql-{day}.log`:

```
2019-11-27 05:06:10.091 UTC [2392] LOG: database system is ready to accept␣
↪connections
2019-11-27 05:06:27.696 UTC [2392] LOG: received fast shutdown request
2019-11-27 05:06:27.700 UTC [2392] LOG: aborting any active transactions
2019-11-27 05:06:27.703 UTC [2766] FATAL: terminating connection due to␣
↪administrator command
2019-11-27 05:06:27.704 UTC [2758] FATAL: terminating connection due to␣
↪administrator command
2019-11-27 05:06:27.709 UTC [2392] LOG: background worker "logical␣
↪replication launcher" (PID 2419) exited with exit code 1
2019-11-27 05:06:27.709 UTC [2414] LOG: shutting down
2019-11-27 05:06:27.735 UTC [2392] LOG: database system is shut down
```

The PostgreSQL was started after the server was back online around 05:06:10 but ClusterControl performs a fast shutdown 17 seconds after that around 05:06:27. If this is something that you would not want it to be, you can disable node recovery for this cluster momentarily.

2.44 Cluster Management

This feature provides information and management options for database clusters managed by ClusterControl. Clicking on the ***Clusters*** (sidebar menu) will list out all database clusters on the main panel, each of the entries has a real-time summary of the cluster, together with the dropdown management menu (see *Cluster Actions*):



Each entry will have the following details:

- **Cluster name**: Configurable during import or deployment, also via the s9s command line.

- **Cluster ID**: Immutable ID assigned by ClusterControl after it is registered.
- **Cluster type**: The database cluster type, recognized by ClusterControl. See *Supported Databases*.
- **Nodes**: All nodes grouped under this particular cluster.
- **Auto recovery**: The ClusterControl automatic recovery settings. See *Failover and Recovery*.
- **Load**: Last 5 minutes of cluster load average.
- **...**: Every supported database cluster has its own set of dropdown menus. See *Cluster Actions*.

Choose a database cluster from the expandable list on the left-side menu to drill down on the cluster-specific features. The navigation breadcrumbs (top panel) will reflect with the corresponding cluster when selecting a cluster.

2.44.1 Cluster Actions

Provides shortcuts to the main cluster functionality. Each database cluster has its own set of functionality as described below:

MySQL/MariaDB/Percona (replication/standalone)

Feature	Description
Add new → Replication node	Adds a new or existing replica database node into the cluster. The new replica will automatically join and synchronize with the rest of the cluster. See *Add Replication Node*.
Add new → Load balancer	Adds a new or existing load balancer into the cluster. See *Create Load Balancer*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Encryption settings → Change SSL Certificate	Opens the SSL certificate management wizard which allows you to create or choose an existing certificate for database client-server encryption
Encryption settings → SSL encryption	Toggles On to enable client-server SSL encryption. This action will restart the whole cluster, one node at a time Toggles Off to disable client-server SSL encryption. This action will restart the whole cluster, one node at a time
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Audit logging	Enable audit logging (MariaDB-based clusters only) using policy-based monitoring and logging of connection and query activity. This feature will enable audit logging on all nodes in the cluster. See *Audit Logging*.
Readonly	Toggles On to enable cluster-wide read-only. ClusterControl will configure `read_only=ON` and `super_read_only=ON` (except MariaDB) on all database nodes in the cluster Toggles Off to disable cluster-wide read-only
Upgrades	Performs minor software upgrades for the database and load balancer software of this cluster, for example from 8.0.x to 8.0.y, in a rolling upgrade fashion. ClusterControl will perform the software upgrade based on what is available on the package repository for the particular vendor. See *Upgrades*
Create database	Create a new database schema on the primary node. For MySQL, the database name can contain letters, numbers, `$` and _ symbols. Maximum length is 64 characters.
Edit details	Edit the cluster's name and tags.
Remove cluster	Removing the cluster will delete all metadata we have on it. The database nodes will not be affected and will continue to operate as normal One has to explicitly type "REMOVE" in the text field before the cluster removal job is triggered. The cluster can be imported again in the future with fresh new metadata and configuration

Galera Cluster (Percona XtraDB/MariaDB)

Feature	Description
Add new → Node	Adds a new or existing database node into the cluster. You can scale out your cluster by adding more database nodes. The new node will automatically join and synchronize with the rest of the cluster. See *Add Node*.
Add new → Replication node	Adds a new or existing replica database node into the cluster. The new replica will automatically join and synchronize with the rest of the cluster. See *Add Replication Node*.
Add new → Load balancer	Adds a new or existing load balancer into the cluster. See *Create Load Balancer*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Encryption settings → Change SSL Certificate	Opens the SSL certificate management wizard which allows you to create or choose an existing certificate for database client-server encryption
Encryption settings → Change Galera SSL Certificate	Opens the SSL certificate management wizard which allows you to create or choose an existing certificate for Galera cluster nodes communication encryption
Encryption settings → Galera SSL encryption	Toggles On to enable Galera SSL encryption. This action will restart the whole cluster, one node at a time Toggles Off to disable Galera SSL encryption. This action will restart the whole cluster, one node at a time
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Readonly	Toggles On to enable cluster-wide read-only. ClusterControl will configure `read_only=ON` and `super_read_only=ON` (except MariaDB) on all database nodes in the cluster Toggles Off to disable cluster-wide read-only
Clone/Replicate the cluster → Clone	This feature has been deprecated. Use *Create cluster from backup* instead.

PostgreSQL/TimescaleDB (streaming replication/standalone)

Feature	Description
Add new → Replication node	Adds a new or existing replica database node into the cluster. The new replica will automatically join and synchronize with the rest of the cluster. See *Add Replication Node*.
Add new → Load balancer	Adds a new or existing load balancer into the cluster. See *Create Load Balancer*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Encryption settings → Change SSL Certificate	Opens the SSL certificate management wizard which allows you to create or choose an existing certificate for database client-server encryption
Encryption settings → SSL encryption	Toggles On to enable client-server SSL encryption. This action will restart the whole cluster, one node at a time Toggles Off to disable client-server SSL encryption. This action will restart the whole cluster, one node at a time
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Audit logging	Enable audit logging using policy-based monitoring and logging of connection and query activity. This feature will enable audit logging on all nodes in the cluster. See *Audit Logging*.
Create replica cluster	Create a new cluster that replicates this cluster. See *Create Replica Cluster*.

PostgreSQL Logical Replication

Feature	Description
Actions → Edit details	Edit the cluster's name and tags.
Actions → Remove cluster	Remove the logical replication grouping for all clusters connected to it. The PostgreSQL database nodes or streaming replication cluster will be ungrouped from the logical binding and will continue to operate as a normal standalone node or streaming replication cluster After the removal, the WAL level will remain as logical, which is usable for streaming replication as well (despite the WAL size will be bigger if compared to replica or hot_standby) One must explicitly type "REMOVE" in the text field before triggering the cluster removal job
	Pubs & Subs → Create publication
Publisher cluster	Choose a database cluster from the dropdown list as the publisher cluster.
Datab	Choose a database from the list. The target database and its corresponding objects like tables must be created first in the publisher cluster. This list will be auto-populated by ClusterControl depending on the ***Publisher cluster***
Publication name	Specify a publication name
Include all tables	Toggle on to include all tables under the chosen database Toggle off will open a dropdown editor where you can type the table name and press ++enter++. Multiple values are supported
	Pubs & Subs → Create subscription
Subscribe cluster	Choose a database cluster from the dropdown list as the subscriber cluster
Subscription name	Specify the subscription name
Publication	Choose an existing publication from the list
Datab	Choose a database from the list. The target database and its corresponding objects like tables must be created first in the subscriber cluster. This list will be auto-populated by ClusterControl depending on the Subscriber cluster
copy data	Toggle on to copy pre-existing data in the publications that are being subscribed to when the replication starts Toggle off will skip the initial snapshot stage. It is recommended for the user to manually perform the initial syncing before proceeding with this option
Origin	Make sure the subscription will only replicate changes that do not have an associated origin (i.e., changes made locally), while Any means the subscription will replicate all changes regardless of their origin, including those already replicated from other nodes This operation establishes a new logical cluster to manage publications and subscriptions. To enable bi-directional repli-

Warning

Attention

For PostgreSQL logical replication, setting `origin=any` can lead to a potential infinite loop in bi-directional replication setups, as changes replicated from one node can be sent back to the originating node. To avoid this, users should carefully configure subscriptions and use `origin=none` to prevent such behavior.

MongoDB

Feature	Description
Add node	Scale the current MongoDB Replica Set or Sharded Cluster deployment by adding a single shard, Mongos, or config server.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Convert to shard	Only for MongoDB Replica Set. Converts an existing MongoDB replica set to a sharded cluster by adding Mongos and config servers into the setup
SSL encryption	Toggles On to enable client-server SSL encryption. This action will restart the whole cluster, one node at a time Toggles Off to disable client-server SSL encryption. This action will restart the whole cluster, one node at a time
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Upgrades	Performs minor software upgrades for the database and load balancer software of this cluster, in a rolling upgrade fashion. ClusterControl will perform the software upgrade based on what is available on the package repository for the particular vendor. See *Upgrades*
Edit details	Edit the cluster's name and tags.
Remove cluster	Removing the cluster will delete all metadata we have on it. The database nodes will not be affected and will continue to operate as normal One has to explicitly type "REMOVE" in the text field before the cluster removal job is triggered. The cluster can be imported again in the future with fresh new metadata and configuration

Redis

Feature	Description
Add node	Adds a new or existing node into the cluster. You can scale out your cluster by adding more database nodes. The new node will automatically join and synchronize with the rest of the cluster. See *Add Node*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Upgrades	Performs minor software upgrades for the database and load balancer software of this cluster, for example from 7.0.x to 7.0.y, in a rolling upgrade fashion. ClusterControl will perform the software upgrade based on what is available on the package repository for the particular vendor. See *Upgrades*
Edit details	Edit the cluster's name and tags.
Remove cluster	Removing the cluster will delete all metadata we have on it. The database nodes will not be affected and will continue to operate as normal One has to explicitly type "REMOVE" in the text field before the cluster removal job is triggered. The cluster can be imported again in the future with fresh new metadata and configuration

Microsoft SQL Server

Feature	Description
Add node	Adds a new or existing node into the cluster. You can scale out your cluster by adding more database nodes. The new node will automatically join and synchronize with the rest of the cluster. See *Add Node*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Upgrades	This feature is not available yet for MSSQL-based clusters
Edit details	Edit the cluster's name and tags.
Remove cluster	Removing the cluster will delete all metadata we have on it. The database nodes will not be affected and will continue to operate as normal One has to explicitly type "REMOVE" in the text field before the cluster removal job is triggered. The cluster can be imported again in the future with fresh new metadata and configuration

Elasticsearch

Feature	Description
Add node	Adds a new Elasticsearch master, data, coordinator, or master-data node into the cluster. The new master or data node will automatically join and synchronize with the rest of the cluster. See *Add Node*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Upgrades	This feature is not available yet for Elasticsearch-based clusters
Edit details	Edit the cluster's name and tags.
Remove cluster	Removing the cluster will delete all metadata we have on it. The database nodes will not be affected and will continue to operate as normal One has to explicitly type "REMOVE" in the text field before the cluster removal job is triggered. The cluster can be imported again in the future with fresh new metadata and configuration

Valkey

Feature	Description
Add node	Adds a new or existing node into the cluster. You can scale out your cluster by adding more database nodes. The new node will automatically join and synchronize with the rest of the cluster. See *Add Node*.
Schedule maintenance	Schedules a cluster-wide maintenance mode, where ClusterControl will skip raising alarms and notifications while the mode is active All nodes in the cluster (regardless of the role) will be marked as under maintenance. A global banner will appear if there is upcoming maintenance for the corresponding cluster
Cluster/node recovery → Cluster recovery	Disables or enables cluster recovery If enabled, ClusterControl will attempt to recover the cluster if necessary. See *Cluster Recovery*.
Cluster/node recovery → Node recovery	Disables or enables node recovery If enabled, ClusterControl will attempt to recover the node if necessary. See *Node Recovery*.
Change RPC API token	Serves as the authentication string by ClusterControl UI to connect to the CMON RPC interface. Each cluster has its unique token
Upgrades	Performs minor software upgrades for the database and load balancer software of this cluster, for example from 7.0.x to 7.0.y, in a rolling upgrade fashion. ClusterControl will perform the software upgrade based on what is available on the package repository for the particular vendor. See *Upgrades*.
Edit details	Edit the cluster's name and tags.
Remove cluster	Removing the cluster will delete all metadata we have on it. The database nodes will not be affected and will continue to operate as normal. One has to explicitly type "REMOVE" in the text field before the cluster removal job is triggered. The cluster can be imported again in the future with fresh new metadata and configuration.

2.44.2 Cluster-specific actions

Bootstrap cluster

Bootstrap cluster refers to the process of bootstrapping or initializing a new Galera Cluster. When you're setting up a new Galera Cluster or performing a full backup restoration, you typically need to bootstrap it, which involves designating one node as the initial primary component of the cluster. This bootstrap node serves as the starting point for the cluster, and other nodes will then join this initial node to form the cluster.

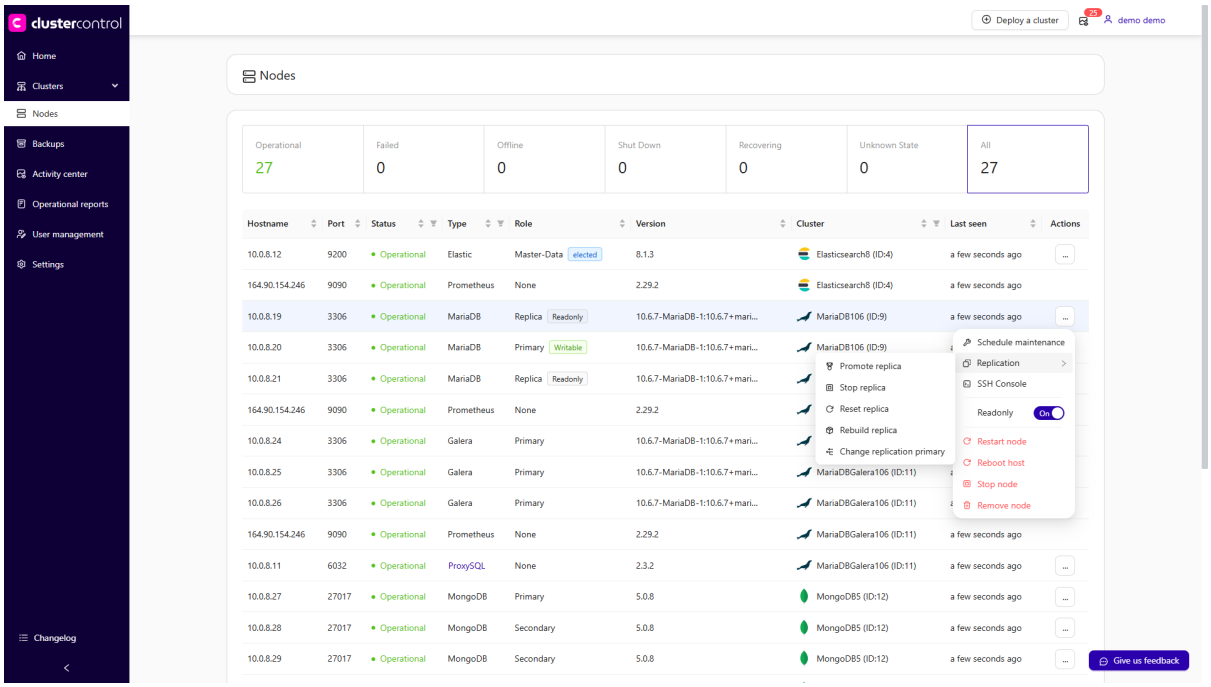
The following actions will be performed:

1. The cluster will be initialized from the selected node.
2. All nodes will be stopped unless they are already stopped.
3. When the bootstrap command is successful, the selected node will be Synced. The rest of the nodes will be started as joiners, one node at a time.

Field	Description
Bootstrap node	Choose a database node from the list to be the reference node. If you are unsure, you may choose Auto Select from the dropdown where ClusterControl will determine the most suitable node to bootstrap based on the Galera node's sequence number.
Graceful shutdown timeout (in seconds)	ClusterControl will wait for this timeout to finish before forcing the node to stop.
Force stop the nodes (after shutdown time)	Toggles On to force stop the database node if it takes longer than the shutdown time, specified in Graceful shutdown timeout (in seconds), defaulting to 1800 seconds (30 minutes). Commonly useful in the case of forcing a database cluster to bootstrap when database locking or long-running transactions are happening.
Clear MySQL datadir on Joining nodes	Toggles On to clear the MySQL data directory on the joining nodes. This is often necessary after a restoration.

2.45 Nodes Management

This feature provides information and management options for all nodes managed by ClusterControl. The Nodes view lists every managed node with a real-time summary of its status, last-seen timestamp, and a dropdown of available management actions.



Each entry displays the following details:

Column	Description
Hostname	The node name recognized by ClusterControl. If you used an IP address during cluster deployment or import, the IP address is shown instead.
Port	The active port for the corresponding host. Every managed node is identified by a `host:port` pair.
Status	The node's current operational status.
Type	The node's cluster type.
Role	The node's role within its cluster (e.g., primary, replica, arbiter).
Version	The database or service version running on the node.
Cluster	Rolling over the cluster name displays a pop-up with cluster details.
Last seen	The timestamp of the last successful check-in between ClusterControl and the node.
Actions	A context-sensitive dropdown menu with management actions available for the node. See *Node Actions* below.

2.45.1 Access the Nodes section

ClusterControl GUI

Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***. This will bring up the Nodes panel listing all managed nodes for the selected cluster.

ClusterControl CLI

Use the `s9s node` command to interact with nodes. Listing all nodes across all clusters:

```
s9s node --list --long
```

Listing nodes for a specific cluster (e.g., cluster ID 36):

```
s9s node --list --long --cluster-id=36
```

2.45.2 Node Actions

The **Actions** dropdown on each node row provides shortcuts to node-level operations. The available actions differ depending on the cluster type and the node's role. The sections below describe each action and how to perform it across the supported interfaces.

Restart node

Restarts the active monitored process of the selected node (e.g., the MySQL service, the HAProxy process). This is a service restart, not a system reboot. The action is only available when the service is currently running. The node enters maintenance mode automatically until the service is fully restarted.

Note

Note You can configure the **Graceful shutdown timeout** (default: 1800 seconds). If the node has not stopped within this period, you can toggle **Force stop the nodes (after shutdown time)** to send a `SIGKILL` signal and force the process down.

Applies to: MySQL/MariaDB, Galera Cluster, Group Replication, PostgreSQL/TimescaleDB, MongoDB, Redis, Microsoft SQL Server, Elasticsearch, Valkey, HAProxy, PGBouncer, Keepalived, ProxySQL, Prometheus

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the node you want to restart.
3. Click **Actions (…) → Restart node**.
4. Optionally, adjust the **Graceful shutdown timeout** or toggle **Force stop the nodes (after shutdown time)**.
5. Confirm the action to proceed.

ClusterControl CLI

Restart a MySQL node (e.g., 192.168.40.30:3306) in cluster ID 36:

```
s9s node \
    --restart \
    --cluster-id=36 \
    --nodes=1192.168.40.30:3306 \
    --wait
```

Restart a PostgreSQL node (e.g., 192.168.40.40:5432) in cluster ID 37:

```
s9s node \
    --restart \
    --cluster-id=37 \
    --nodes=192.168.40.40:5432 \
    --wait
```

Reboot host

Initiates a full operating system reboot of the selected node. Once initiated, ClusterControl monitors the reboot progress every 5 seconds for up to 10 minutes (600 seconds). If the node does not come back online within this window, the operation is declared failed.

Applies to: MySQL/MariaDB, Galera Cluster, Group Replication, PostgreSQL/TimescaleDB, MongoDB, Redis, Microsoft SQL Server, Elasticsearch, Valkey, HAProxy, PGBouncer, Keepalived, ProxySQL, Prometheus

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the node you want to reboot.
3. Click **Actions (…) → Reboot host**.
4. Confirm the action to proceed. ClusterControl will initiate the OS-level reboot and track its progress.

ClusterControl CLI

There is no dedicated `s9s` command for a full host reboot. Use an SSH-based approach or leverage an OS-level automation tool to reboot the host. You can monitor node availability afterwards with:

```
s9s node --list --long --cluster-id=<cluster_id>
```

Stop/Start node

Stops or starts the managed service on the selected node. For database nodes, ClusterControl stops or starts the database service bound to the given `host:port`. For load balancer nodes (HAProxy, ProxySQL), the load balancer process is stopped or started instead.

> **Note**
>
> Note By default, a graceful shutdown is attempted for 1800 seconds (30 minutes). Toggle **Force stop the nodes (after shutdown time)** to send `SIGKILL` if the node does not stop within this window. No data is lost during a graceful stop.

Applies to: MySQL/MariaDB, Galera Cluster, Group Replication, PostgreSQL/TimescaleDB, MongoDB, Redis, Microsoft SQL Server, Elasticsearch, Valkey, HAProxy, PGBouncer, Keepalived, ProxySQL, Prometheus

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the target node.
3. Click **Actions (…) → Stop node** (or **Start node** if the node is currently stopped).
4. Optionally, configure the **Graceful shutdown timeout** or enable **Force stop the nodes (after shutdown time)**.
5. Confirm the action to proceed.

ClusterControl CLI

Stop a MySQL node (e.g., 192.168.40.30:3306) in cluster ID 36:

```
s9s node \
    --stop \
    --cluster-id=36 \
```

(continues on next page)

(continued from previous page)

```
    --nodes=192.168.40.30:3306 \
    --wait
```

Start a previously stopped MySQL node:

```
s9s node \
    --start \
    --cluster-id=36 \
    --nodes=192.168.40.30:3306 \
    --wait
```

Stop a PostgreSQL node (e.g., 192.168.40.40:54 32) in cluster ID 37:

```
s9s node \
    --stop \
    --cluster-id=37 \
    --nodes=192.168.40.40:5432 \
    --wait
```

Remove node

Removes a node from ClusterControl management. Depending on the selected removal option, this can also stop the service or fully uninstall the software.

Warning

Warning!

Removing a node is a destructive operation with respect to ClusterControl's awareness of that node. Choose the appropriate removal option carefully.

Removal options:

Option	Behavior
Keep service running	The node is unregistered from ClusterControl but the database or load balancer service continues running on the host.
Stop service and keep files untouched	The node is unregistered and the service is stopped. Data files and configuration files are preserved on the host.
Stop and uninstall service	The node is unregistered, the service is stopped, and all related software packages are removed using the system package manager.

Applies to: MySQL/MariaDB, Galera Cluster, Group Replication, PostgreSQL/TimescaleDB, MongoDB, Redis, Microsoft SQL Server, Elasticsearch, Valkey, HAProxy, PGBouncer, Keepalived, ProxySQL, Prometheus

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the node you want to remove.
3. Click **Actions (…) → Remove node**.
4. Select the desired **removal option** from the panel.
5. Confirm the action to proceed.

ClusterControl CLI

Remove node 192.168.40.30:3306 from cluster ID 36 (the service will be stopped and uninstalled):

```
s9s node \
    --remove \
    --cluster-id=36 \
    --nodes=192.168.40.30:3306 \
    --wait
```

SSH Console

Opens a web-based SSH terminal in a new browser window, allowing you to execute shell commands directly on the node from your browser. The session authenticates as the configured `os_user`.

Applies to: MySQL/MariaDB, Galera Cluster, Group Replication, PostgreSQL/TimescaleDB, MongoDB, Redis, Microsoft SQL Server, Elasticsearch, Valkey, HAProxy, PGBouncer, Keepalived, ProxySQL, Prometheus

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the target node.
3. Click **Actions (…) → SSH Console**. A new browser tab or window will open with a terminal session on the selected host.

2.45.3 Replication actions

The following actions are specific to nodes that participate in replication topologies. Availability depends on the cluster type and the node's role.

Promote replica

Promotes the selected replica to become the new primary node. If the current primary is healthy, stop application traffic before promoting a replica to prevent data loss. Connections to the existing primary are terminated after a 10-second grace period.

Warning

Warning!

Toggle **Force promotion of replica** to ignore errors (such as errant transactions or long-running queries) during promotion. Without this option, ClusterControl aborts the failover if an errant transaction is detected on the replica.

Applies to: MySQL/MariaDB (replication), PostgreSQL/TimescaleDB, Microsoft SQL Server

ClusterControl GUI

MySQL/MariaDB and PostgreSQL

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the replica node you want to promote.
3. Click **Actions (…) → Replication → Promote replica**.
4. Optionally, enable **Force promotion of replica** to bypass error checks.
5. Confirm the action to proceed.

Redis / Valkey / Microsoft SQL Server

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the replica node.
3. Click **Actions (…) → Promote replica**.
4. Confirm the action to proceed.

ClusterControl CLI

Promote a MySQL replica (192.168.40.31:3306) in cluster ID 36:

```
s9s cluster \
    --promote-slave \
    --cluster-id=36 \
    --nodes=192.168.40.31:3306 \
    --wait
```

Promote a PostgreSQL replica (192.168.40.41:5432) in cluster ID 37:

```
s9s cluster \
    --promote-slave \
    --cluster-id=37 \
    --nodes=192.168.40.41:5432 \
    --wait
```

Stop/Start replica

Stops or starts the replication thread on the selected replica node without stopping the database service itself. This action is exclusive to replica nodes.

Applies to: MySQL/MariaDB (replication), Galera (if mix with a replica node(s))

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the replica node.
3. Click **Actions (…) → Replication → Stop replica** (or **Start replica** if the replication thread is currently stopped).
4. Confirm the action.

ClusterControl CLI

Stop the replication thread on replica 192.168.40.31:3306 in cluster ID 36:

```
s9s node \
    --stop \
    --cluster-id=36 \
    --nodes=192.168.40.31:3306 \
    --wait
```

Reset replica

Clears the replication state on the selected replica, causing it to forget its position in the primary's binary log and all connection parameters (primary host, port, user, and password). This is equivalent to running `RESET REPLICA ALL` on the replica. The replica service must be stopped before this action can be performed.

> **Note**
>
> Note Toggle **Force** to ignore errors encountered during the reset.

Applies to: MySQL/MariaDB (replication), Galera (if mix with a replica node(s))

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the replica node (ensure the replica thread is stopped first).
3. Click **Actions (…) → Replication → Reset replica**.
4. Optionally, toggle **Force** to bypass error checks.
5. Confirm the action.

Rebuild replica

Rebuilds the selected replica node by re-synchronizing its data from a primary node. This is used when a replica has drifted too far behind, has become corrupted, or needs to be brought back into the replication topology cleanly.

Two methods are available:

- **From a primary node** – ClusterControl uses Percona XtraBackup (MySQL) or `pg_basebackup` (PostgreSQL) to stream a fresh copy of data directly from the primary. This uses the primary's I/O resources during the sync.
- **From a backup** – Toggle **Rebuild from a backup** and select any PITR-compatible backup from the dropdown. This reduces the impact on the primary during the rebuild.

Applies to: MySQL/MariaDB (replication), PostgreSQL/TimescaleDB, Microsoft SQL Server

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the replica node you want to rebuild.
3. Click **Actions (…) → Replication → Rebuild replica**.
4. Choose whether to rebuild from the primary directly or toggle **Rebuild from a backup** to use an existing PITR-compatible backup.
5. If rebuilding from a backup, select the desired backup from the dropdown list.
6. Confirm the action to proceed.

Change replication primary

Reconfigures the replication topology so that all replicas point to a different primary node. This action can be initiated from either a replica or a primary node.

Applies to: MySQL/MariaDB (replication), Galera (if mix with a replica node(s)), PostgreSQL/TimescaleDB

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate any replica or primary node in the cluster.
3. Click **Actions (…) → Replication → Change replication primary**.
4. Select the new primary node from the presented list.
5. Confirm the action. ClusterControl will reconfigure all replicas to replicate from the new primary.

ClusterControl CLI

Change the replication primary for cluster ID 36 so that replicas now follow 192.168.40.30:3306:

```
s9s cluster \
    --change-config \
    --cluster-id=36 \
    --nodes=192.168.40.30:3306 \
    --wait
```

2.45.4 Read-only and binary logging

Disable/Enable read-only

Sets or unsets the read-only mode on a database node. When enabled, ClusterControl sets `read_only=ON` and `super_read_only=ON` (MySQL 8.0 and later) on the target node.

Applies to: MySQL/MariaDB, Galera Cluster, PostgreSQL/TimescaleDB

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the target node.
3. Click **Actions (…) → Disable readonly** (to enable write access) or **Actions (…) → Enable readonly** (to set the node as read-only).
4. Confirm the action.

ClusterControl CLI

Enable `read_only` on a MySQL node (192.168.40.31:3306) in cluster ID 36:

```
s9s node \
    --change-config \
    --cluster-id=36 \
    --nodes=192.168.40.31:3306 \
    --opt-group=mysqld \
    --opt-name=read_only \
    --opt-value=ON
```

Disable/Enable binary logging

Updates the node's configuration to enable or disable MySQL binary logging. Enabling binary logging allows a replica to be added to the node and makes point-in-time recovery (PITR) possible. Disabling binary logging stops any attached replicas and prevents new replicas from being added. A server restart is required for either change to take effect.

> **Note**
>
> Note Toggle **Restart the node** to have ClusterControl immediately restart the node after the configuration update, applying the change without a separate step.

Applies to: Galera Cluster

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the Galera node.
3. Click **Actions (…) → Disable binary logging** or **Actions (…) → Enable binary logging**.
4. Optionally, toggle **Restart the node** to apply the change immediately.
5. Confirm the action.

2.45.5 Galera-specific actions

Resync node

Forces a full resync of the selected Galera node. ClusterControl removes all files in the node's data directory (`datadir`) and triggers a State Snapshot Transfer (SST) from a donor node. If a PITR-compatible full backup exists, ClusterControl will restore from it first to shorten the SST window and increase the probability of an Incremental State Transfer (IST), which is a non-blocking operation.

This action is typically used when a Galera node repeatedly fails to recover or when a filesystem issue is suspected.

> **Warning**
>
> Warning!
>
> Wait for this operation to complete before starting another Galera node with **Initial Start**, as concurrent SST operations can disrupt the cluster.

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the Galera node that needs a resync.
3. Click **Actions (…) → Resync node**.
4. Confirm the action. ClusterControl will wipe the data directory and initiate an SST or IST from a suitable donor.

Bootstrap Cluster

Bootstraps the Galera cluster from the selected node. ClusterControl stops all currently running Galera nodes before bootstrapping the cluster from the node you choose. Use this action when the entire cluster is down and needs to be started from a known-good node.

> **Warning**
>
> Warning!
>
> All running Galera nodes will be stopped before the bootstrap begins. Ensure that the selected node has the most up-to-date data (highest `seqno`) to avoid data loss.

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the Galera node you want to bootstrap from (ideally the node with the highest sequence number).
3. Click **Actions (…) → Bootstrap Cluster**.
4. Confirm the action. ClusterControl will stop all running nodes and bootstrap the cluster from the selected node.

ClusterControl CLI

Bootstrap the Galera cluster (cluster ID 36) from node 192.168.40.30:3306:

```
s9s cluster \
    --bootstrap \
    --cluster-id=36 \
    --nodes=192.168.40.30:3306 \
    --wait
```

2.45.6 MongoDB-specific actions

Step down node

Steps down the selected MongoDB primary, causing it to transition to a secondary and become ineligible for election as primary for a configurable number of seconds. The remaining voting members of the replica set will elect a new primary, excluding the stepped-down node for the specified duration.

Applies to: MongoDB (replica set primary)

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the MongoDB primary node.
3. Click **Actions (…) → Step down node**.
4. Specify the number of seconds during which the node is ineligible for re-election.
5. Confirm the action.

Freeze node

Prevents the selected MongoDB replica set member from seeking election for a specified number of seconds. To unfreeze the node before the duration elapses, issue the command again with a value of `0` seconds.

Applies to: MongoDB (replica set members)

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the MongoDB node.
3. Click **Actions (…) → Freeze node**.
4. Enter the number of seconds to freeze the node.
5. Confirm the action.

2.45.7 PostgreSQL-specific actions

Configure WAL

The Write-Ahead Log (WAL) is PostgreSQL's REDO log. WAL records all changes made to the database and is essential for replication, online backup, and point-in-time recovery (PITR). Enabling WAL archiving prevents the primary from recycling WAL segments before replicas have applied them, which would otherwise force a full replica rebuild.

!!! danger "Warning"

```
Changing the WAL archive mode or directory causes the loss of the␣
↪previously collected WAL archive, eliminating the ability to perform␣
↪point-in-time recovery (PITR) for any prior time frames.
```

WAL configuration options:

Option	Description
Archive Mode	**Off** – WAL logs are not archived; PITR is not possible. **On** – WAL logs are archived only when the node is in primary mode. **Always** – WAL logs are archived regardless of the node's role (primary or replica).
Compressed WAL Archive	Enables compression for archived WAL logs. Enabled by default.
Custom WAL Archive Directory	Path to the WAL archive directory. Defaults to `/var/lib/pgsql/13/wal_archive` if left empty.
Nodes	WAL archiving can be configured on all database nodes simultaneously or on an individual node.

Applies to: PostgreSQL/TimescaleDB

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the PostgreSQL node.
3. Click **Actions (…) → Configure WAL**. This opens the WAL configuration panel.

4. Set the **Archive Mode**, toggle **Compressed WAL Archive** as desired, and specify a **Custom WAL Archive Directory** if needed.
5. Select whether the change applies to **All nodes** or the individual node.
6. Confirm the action to apply the WAL configuration.

Synchronous replication

Enables or disables synchronous streaming replication between the primary and the selected replica node. When enabled, the primary waits for the replica to confirm receipt of WAL data before acknowledging a transaction commit. Synchronous replication can be enabled per individual replica node but carries a notable performance overhead.

Applies to: PostgreSQL/TimescaleDB (replica nodes only)

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the PostgreSQL replica node.
3. Click **Actions (…) → Synchronous Replication**.
4. Toggle synchronous replication on or off for the selected replica.
5. Confirm the action.

2.45.8 ProxySQL-specific actions

Synchronize Instances

Synchronizes the ProxySQL configuration across multiple ProxySQL instances to ensure they remain identical. This operation overwrites the target's existing configuration.

Two synchronization modes are available:

- **Synchronize existing instances** – Exports the configuration from a source ProxySQL instance and imports it to one or more target instances.
- **Import Configuration file** – Restores a previously exported ProxySQL configuration backup to the selected instance.

Applies to: ProxySQL

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the ProxySQL node.
3. Click **Actions (…) → Synchronize Instances**.
4. Choose between **Synchronize existing instances** or **Import Configuration file**.
5. Select the source instance or the configuration backup file as appropriate.
6. Confirm the action.

Export configuration

Exports the full ProxySQL configuration from the selected instance to a backup file. The exported backup can be imported to any ProxySQL instance using the **Synchronize Instances → Import Configuration file** option.

The following configuration data is included in the export:

- Query Rules
- Host Groups and Servers
- Users and corresponding MySQL users
- Global Variables
- Scheduler
- `proxysql.cnf`

Applies to: ProxySQL

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the ProxySQL node.
3. Click **Actions (…) → Export configuration**.
4. Confirm the action. ClusterControl will generate and download the configuration backup file.

2.45.9 Schedule maintenance for a node

Schedules a maintenance window for an individual node, suppressing alarms and notifications for the node during the specified period. While in maintenance mode, ClusterControl will not generate alerts for node events (e.g., threshold breaches, health check failures).

For full details on configuring, checking, and deactivating maintenance mode, see *Maintenance*.

Warning

Attention

If automatic node recovery is enabled, ClusterControl will still attempt to recover the node regardless of maintenance mode status. Disable node recovery before starting maintenance if you do not want ClusterControl to interfere.

Applies to: MySQL/MariaDB, Galera Cluster, PostgreSQL/TimescaleDB, MongoDB, Redis, Microsoft SQL Server, Elasticsearch, Valkey, HAProxy, ProxySQL, Prometheus

ClusterControl GUI

Schedule maintenance:

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the target node.

3. Click **Actions (…) → Schedule maintenance**.
4. Specify the maintenance reason, define the duration, or set a custom start and end date/time.
5. Confirm to activate the maintenance window.

Manage or stop active maintenance:

1. Log in to your ***ClusterControl GUI → choose a database cluster → Nodes***.
2. Locate the node currently in maintenance (a blue wrench icon is shown next to its status).
3. Click **Actions (…) → Maintenance → Manage scheduled maintenance** to review the schedule details or delete it.
4. To stop an active maintenance window immediately, click **Actions (…) → Maintenance → Stop current maintenance** and confirm.

ClusterControl CLI

Schedule a one-day maintenance window for PostgreSQL node 192.168.40.40:5432 (ClusterControl expects UTC time):

```
s9s maintenance \
    --create \
    --nodes=192.168.40.40:5432 \
    --begin=2024-05-15T05:44:55.000Z \
    --end=2024-05-16T05:44:55.000Z \
    --reason='Upgrading RAM' \
    --batch
```

Schedule maintenance for a MySQL node 192.168.40.30:3306 starting tomorrow for one hour:

```
s9s maintenance \
    --create \
    --nodes=192.168.40.30:3306 \
    --begin="$(date --date='now + 1 day' --utc '+%Y-%m-%d %H:%M:%S')" \
    --end="$(date --date='now + 1 day + 1 hour' --utc '+%Y-%m-%d %H:%M:%S')
↪" \
    --reason="OS patching"
```

List all nodes currently under a maintenance period:

```
s9s maintenance --list --long
```

Delete (stop) a maintenance period by UUID:

```
s9s maintenance --delete --uuid=<maintenance_uuid>
```

2.46 Configuration Management

This feature enables administrators to define, apply, and monitor configurations for entire clusters or individual database nodes, ensuring consistency and compliance with best practices. With configuration management, users can automate updates, track configuration changes, and revert to previous versions if

needed, simplifying maintenance and reducing the risk of misconfigurations that could impact performance or security.

2.46.1 Access the configuration management section

ClusterControl GUI

Log in to your ***ClusterControl GUI → choose a database cluster → Manage → Configurations***. This will bring up a panel where you can view and edit configurations.

ClusterControl CLI

For CLI, the context of the configuration management is bounded at the node level. Hence, the `--nodes` flag is mandatory with a proper `<host>:<port>` value.

- List all database configuration options for a MySQL database node, 192.168.99.3:

```
s9s node --list-config --cluster-id=36 --nodes=192.168.99.3:3306
```

- List all database configuration options for a PostgreSQL database node, 192.168.99.6:

```
s9s node --list-config --cluster-id=37 --nodes=192.168.99.6:5432
```

2.46.2 Refresh configuration files

ClusterControl GUI

Click on **Refresh config** to get the latest configuration files from all nodes managed by ClusterControl. This will trigger a job to pull the latest configuration files from all nodes. The configuration files output listed will be refreshed after the job is successfully executed.

ClusterControl CLI

Suppose the cluster ID is 36, and wait for the refresh configuration job to finish:

```
s9s cluster --import-config --cluster-id=36 --wait
```

2.46.3 View and edit configuration files

ClusterControl GUI

1. In the Configuration Management panel, you will see a list of configuration files for the selected database technology (e.g., `my.cnf` for MySQL, `postgresql.conf` for PostgreSQL, `proxysql.cnf` for ProxySQL).
2. Select a configuration file to view its current parameters. Selecting a configuration will appear as a new tab in the editor area.
3. ClusterControl allows you to edit configurations directly within this interface, similar to a text editor. Make your changes as needed, adjusting parameters to meet your performance, security, or operational needs.
4. Click **Save** to apply the change. ClusterControl will push the changes into the configuration file on the corresponding database node. If you would like to change a configuration across all nodes in the cluster, use the **Edit params** button as explained in the next section.

!!! Info

```
Editing the configuration file with the text editor will only push the␣
↪configurations to the database configuration file, and not the␣
↪database runtime. Restart the database service to apply those␣
↪changes into runtime, or use the **Edit params** button.
```

ClusterControl CLI

For CLI, the context of the configuration management is bounded at the node level. Hence, the `--nodes` flag is mandatory with a proper `<host>:<port>` value.

- Change a MySQL configuration option for a MySQL database node 192.168.99.3 called `max_connections` which is located under the `[mysqld]` section:

```
s9s node \
    --change-config \
    --cluster-id=36 \
    --nodes='192.168.99.3:3306' \
    --opt-group=mysqld \
    --opt-name=max_connections \
    --opt-value=300
```

- Change a PostgreSQL configuration option for a PostgreSQL database node, 192.168.99.6 called `log_line_prefix`:

```
s9s node \
    --change-config \
    --cluster-id=37 \
    --nodes='192.168.99.6:5432' \
    --opt-name=log_line_prefix \
    --opt-value='%m '
```

2.46.4 Compare new configuration with existing configuration

ClusterControl GUI

Use **Show diff** button to compare the new configuration with the existing configuration before hitting the **Save** button. ClusterControl will open a new panel to compare these two configuration versions side-by-side before and after changes.

The left panel displays the existing configuration (changes are highlighted in red) while the right-panel displays the currently modified (yet to apply) configuration (changes are highlighted in green).

Click **Revert Changes** to rollback and discard the change, or click **Save** to commit and push the new configuration to the database node. ClusterControl will perform a file-system replace to the existing configuration file on the database node with the new version.

Note

Note ClusterControl does not store configuration changes history so there is no file versioning after saving. Only one version exists at one time on the managed node.

2.46.5 Apply configuration changes across nodes

ClusterControl GUI

Use **Edit params** if you want to change a specific configuration option across database nodes in the cluster.

1. Click on **Edit params** button and ClusterControl will load all configuration options in a table format.
2. Look up the configuration option in the filtering field and you can edit the corresponding configuration option by clicking on the pencil icon.
3. After making changes, press Enter and the configuration will be pushed to all database nodes' configuration file and runtime (only if applicable).

ClusterControl CLI

For CLI, the command must be repeated multiple times for every database node.

- Change a MySQL configuration option for multiple MySQL database nodes in cluster ID 36 (192.168.99.3, 192.168.99.4, 192.168.99.5) called `max_connections` which is located under the `[mysqld]` section:

```
s9s node \
    --change-config \
    --cluster-id=36 \
    --nodes='192.168.99.3:3306' \
    --opt-group=mysqld \
    --opt-name=max_connections \
    --opt-value=300

s9s node \
    --change-config \
    --cluster-id=36 \
    --nodes='192.168.99.4:3306' \
    --opt-group=mysqld \
    --opt-name=max_connections \
    --opt-value=300

s9s node \
    --change-config \
    --cluster-id=36 \
    --nodes='192.168.99.5:3306' \
    --opt-group=mysqld \
    --opt-name=max_connections \
    --opt-value=300
```

- Change a PostgreSQL configuration option for multiple PostgreSQL database nodes in cluster ID 37, (192.168.99.6, 192.168.99.7) called `log_line_prefix`:

```
s9s node \
    --change-config \
    --cluster-id=37 \
    --nodes='192.168.99.6:5432' \
    --opt-name=log_line_prefix \
```

(continues on next page)

(continued from previous page)

```
    --opt-value='%m '

s9s node \
    --change-config \
    --cluster-id=37 \
    --nodes='192.168.99.7:5432' \
    --opt-name=log_line_prefix \
    --opt-value='%m '
```

2.46.6 Monitor configuration consistency

ClusterControl GUI

ClusterControl allows you to monitor and verify that configurations are consistent across your cluster. Go to ***ClusterControl GUI → choose a database cluster → Performance → DB Variables*** to identify configuration drift (differences in configurations between nodes).

Entries highlighted in red indicate the configuration options are not identical across nodes. Note that some configuration options may be distinct across nodes and does not necessary mean a poor configuration.

ClusterControl CLI

To compare output between two commands, we can use `diff -y` with side-by-side output.

- Compare the current database configurations (from the configuration file) for two MySQL database nodes, 192.168.99.3 and 192.168.99.4:

```
diff -y <(s9s node --list-config --nodes=192.168.99.3:3306) <(s9s node --
↪list-config --nodes=192.168.99.4:3306)
```

- Compare the current database configurations (from the configuration file) for two PostgreSQL database nodes, 192.168.99.7 and 192.168.99.8:

```
diff -y <(s9s node --list-config --nodes=192.168.99.7:5432) <(s9s node --
↪list-config --nodes=192.168.99.8:5432)
```

2.46.7 Configuration template

All services configured by ClusterControl use a base configuration template available under `/usr/share/cmon/templates` on the ClusterControl node. You can directly modify the file to suit your deployment policy however, this directory will be replaced after a package upgrade.

To make sure your custom configuration template files persist across upgrades, store your template files under the `/etc/cmon/templates` directory (ClusterControl 1.6.2 and later). When ClusterControl loads up the template files for deployment, files under `/etc/cmon/templates` will always have higher priority over the files under `/usr/share/cmon/templates`. If two files having identical names exist on both directories, the one located under `/etc/cmon/templates` will be used.

Here is an example of how one would create a custom configuration template file:

```
# on the ClusterControl node, as privileged user
mkdir -p /etc/cmon/templates
```

(continues on next page)

(continued from previous page)

```
cp /usr/share/cmon/templates/my.cnf.repl80 /
↪etc/cmon/templates/my.cnf.repl80-custom
vi /etc/cmon/templates/my.cnf.repl80-custom     # make your changes and save
```

Go to the ***ClusterControl GUI → Deploy a cluster → Create a database cluster*** and start deploying a database cluster. Choose the created configuration file above from the **Configuration template** dropdown.

Dynamic variables

Inside the provided template files, many configuration variables are configurable dynamically (depending on the detected host specifications and user's input in the deployment wizard) by ClusterControl during deployment. These variables are represented with a capital letter enclosed by `@`, for example, `@DATADIR@`. The following shows the list of variables supported by the ClusterControl, grouped by the cluster type:

MySQL/MariaDB/Percona Server

Variable	Description
@BASEDIR@	The default is /usr. Value specified during cluster deployment takes precedence.
@DATADIR@	The default is /var/lib/mysql. Value specified during cluster deployment takes precedence.
@MYSQL_PORT@	The default is 3306. Value specified during cluster deployment takes precedence.
@BUFFER_POOL_SIZE@	Automatically configured based on the host's RAM.
@LOG_FILE_SIZE@	Automatically configured based on the host's RAM.
@LOG_BUFFER_SIZE@	Automatically configured based on the host's RAM.
@BUFFER_POOL_INSTAN	Automatically configured based on the host's CPU.
@SERVER_ID@	Automatically generated based on member's server-id.
@SKIP_NAME_RESOLVE@	Automatically configured based on MySQL variables.
@MAX_CONNECTIONS@	Automatically configured based on the host's RAM.
@ENABLE_PERF_SCHEMA@	The default is disabled. Value specified during cluster deployment takes precedence.
@WSREP_PROVIDER@	Automatically configured based on the Galera vendor.
@HOST@	Automatically configured based on hostname/IP address.
@GCACHE_SIZE@	Automatically configured based on disk space.
@SEGMENTID@	The default is 0. Value specified during cluster deployment takes precedence.
@WSREP_CLUSTER_ADDRESS	Automatically configured based on members in the cluster.
@WSREP_SST_METHOD@	Automatically configured based on the Galera vendor.
@BACKUP_USER@	Default is backupuser.
@BACKUP_PASSWORD@	Automatically generated and configured for backupuser.
@GARBD_OPTIONS@	Automatically configured based on garbd options.
@READ_ONLY@	Automatically configured based on replication role.
@SEMISYNC@	The default is disabled. Value specified during cluster deployment takes precedence.
@NDB_CONNECTION_POO	Automatically configured based on the host's CPU.
@NDB_CONNECTSTRING@	Automatically configured based on members in the MySQL cluster.
@LOCAL_ADDRESS@	Automatically configured based on the host's address.
@GROUP_NAME@	Default is grouprepl. Value specified during cluster deployment takes precedence.
@PEERS@	Automatically configured based on members in the Group Replication cluster.

MongoDB

Variable	Description
@DATADIR@	The default is /var/lib/mongodb. Value specified during cluster deployment takes precedence.
@MONGODB_PORT@	The default is 27017, 27018, 27019 (depending on the cluster type). Value specified during cluster deployment takes precedence.
@LOGDIR@	Automatically configured based on vendor.
@HOST@	Automatically configured based on hostname/IP address.
@SMALLFILES@	Automatically configured based on disk space.
@PIDFILEPATH@	Automatically configured based on MongoDB data directory.
@REPLICASET_NAME@	The default is my_mongodb_N. Value specified during cluster deployment takes precedence.

Redis Cluster/Redis Sentinel/Valkey Cluster

Variable	Description
@BIND_ADDRESS@	Automatically configured based on the host's address.
@PORT@	The default is 6379. Value specified during cluster deployment takes precedence.
@DATADIR@	The default is /var/lib/redis. Value specified during cluster deployment takes precedence.
@REPLICATION_PASSWORD@	Automatically configured based on the value specified during the deployment.
@REPLICA_OF@	Automatically configured based on members in the topology.
@CLUSTER_CONFIG_FILE@	Automatically configured based on cluster type.
@CLUSTER_NODE_TIMEOUT@	The default is 15000 in milliseconds. Value specified during cluster deployment takes precedence.
@CLUSTER_BUS_PORT@	The default is 16379. Value specified during cluster deployment takes precedence.
@CLUSTER_REPLICA_VALIDITY_FAC	The default is 10. Value specified during cluster deployment takes precedence.
@ACL_DB_ADMIN_USER@	Automatically configured based on members in the topology.
@SENTINEL_PASSWORD@	Automatically configured based on the value specified during the deployment.
@PASSWORD@	Automatically configured based on the value specified during the deployment.
@MAXMEMORY_BYTES@	Automatically configured based on the host's RAM.
@MAXMEMORY_POLICY@	The default is allkeys-lru.
@CLUSTER_ENABLED@	Automatically configured based on cluster type.

Elasticsearch

Variable	Description
@CLUSTER_NAME@	Automatically configured based on the value specified during the deployment.
@NODE_NAME@	Automatically configured based on hostname/IP address.
@NODE_ROLES@	Automatically configured based on node's role.
@DATADIR@	The default is /var/lib/elasticsearch. Value specified during cluster deployment takes precedence.
@SNAPSHOTS_LOCATION@	Automatically configured based on the value specified during the deployment.
@BIND_ADDRESS@	Automatically configured based on hostname/IP address.
@PUBLISHED_ADDRESS@	Automatically configured based on hostname/IP address.
@HTTP_PORT@	The default is 9200. Value specified during cluster deployment takes precedence.
@DISCOVERY_SEED_HOSTS@	Automatically configured based on hostname/IP address of the members in the topology.
@CLUSTER_INITIAL_MASTER_NOI	Automatically configured based on members in the topology.
@SECURITY_ENABLED@	Automatically configured based on the value specified during the deployment.

2.47 Upgrades and Patches

Performs database and load-balancer-related package upgrades.

ClusterControl upgrades database and load balancer packages by utilizing the vendor's package repository. The upgrade process depends on the availability of updated packages in this repository. If the vendor repository is outdated or inaccessible (due to network restrictions or proxy configurations), ClusterControl cannot perform the upgrade as new packages will not be available in the server's repository list.

!!! danger "Warning"

```
A database upgrade is a risky operation and ClusterControl does not support␣
↪an upgrade rollback operation. Please perform this activity on a␣
↪test/staging cluster first before proceeding with the actual upgrade.␣
↪Upgrades should only be performed when there is as little traffic as␣
↪possible on the cluster. Performing a full database backup (or taking a␣
↪full VM snapshot) before an upgrade will increase the chance of recovery␣
↪in case a rollback is needed. Taking a backup (before the upgrade) or␣
↪restoring a backup (to rollback) is not part of this particular upgrade␣
↪job.
```

2.47.1 Upgrade procedures

Database cluster upgrade procedures vary based on the cluster's topology, as detailed below.

Primary-replica replication

ClusterControl upgrades primary-replica setups by updating replica/secondary nodes first (in random order), followed by the primary node. Expect a service interruption during the primary node upgrade until it restarts.

Warning

Attention

ClusterControl does not perform automatic switchover before upgrading the primary node, which is always upgraded last. Always test upgrade procedures in a non-production environment before applying them to production databases.

Multi-primary

In a multi-primary setup, ClusterControl upgrades database nodes randomly, one node at a time. If an upgrade fails, the entire process stops, necessitating manual intervention for node recovery or reinstallation. Consequently, a suitable maintenance window with minimal cluster traffic is crucial for upgrades.

Tip

If database connections are load-balanced, performing the entire upgrade without database service downtime may be possible. Always test upgrade procedures in a test environment before applying them to a production database.

2.47.2 Minor upgrade

Performs minor version software upgrades for database and load balancer software. Minor upgrade version formatting is notably different depending on the database type and vendor (from `x` to `y`):

- MySQL/Percona Server for MySQL – 8.0.x to 8.0.y
- MariaDB – 11.4.x to 11.4.y
- MongoDB – 7.0.x to 7.0.y
- PostgreSQL/TimescaleDB/EntepriseDB – 16.x to 16.y
- Redis Cluster/Sentinel – 7.x to 7.y
- ProxySQL – 2.6.x to 2.6.y
- PgBouncer – 1.22.x to 1.22.y
- HAProxy – 2.1.x to 2.1.y
- MariaDB MaxScale – 22.08.x to 22.08.y

Checking for available upgrades

To determine if a newer version is available for your database or load balancer, follow the instructions below for either the ClusterControl GUI or CLI.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Clusters***
2. Select the desired cluster from the list.
3. Go to ***Actions (…) → Upgrades → Minor upgrades***.
4. ClusterControl will display a list of database and load balancer nodes within the cluster that can be upgraded.
5. Click **Check for upgrades**. This initiates a background job to query the package repositories of the database and load balancer nodes for the latest available versions.
6. Once the job is complete, verify if a new version is available.

ClusterControl CLI

- To initiate a check for newer package versions on cluster ID 17 and wait for the process to finish, execute the following command:

```
s9s cluster --check-pkg-upgrades --cluster-id=17 --wait
```

Performing minor version upgrade

Minor upgrades are performed online on the selected node(s), one node at a time. The node will be stopped, then the software will be updated, and then the node will be started again. If a node fails to upgrade, the upgrade process is aborted and manual intervention is required to recover or reinstall the node.

To perform a minor version upgrade, either for the database cluster or load balancer, or both, follow the instructions below for either the ClusterControl GUI or CLI.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Clusters***
2. Select the desired cluster from the list.
3. Go to ***Actions (…) → Upgrades → Minor upgrades***.
4. ClusterControl will display a list of database and load balancer nodes within the cluster that can be upgraded.
5. Check all the boxes for the nodes that you want to upgrade. Clicking on the top checkbox will automatically check all the boxes underneath it.
6. Select **Upgrade**. ClusterControl will start an upgrade process even if a newer version is not available. If no upgrade is needed, the job will check package versions and report "Nothing to upgrade."
7. Monitor the job log messages closely.

ClusterControl CLI

- Update database and load balancer packages to the newest minor version across all nodes in cluster ID 1:

```
s9s cluster \
    --upgrade-cluster \
    --cluster-id=1 \
    --wait
```

- Update database packages to the newest minor version for all MySQL database nodes in cluster ID 1:

```
s9s cluster \
    --upgrade-cluster \
    --cluster-id=1 \
    --nodes="192.168.0.81:3306;192.168.0.82:3306;192.168.0.83:3306" \
    --wait
```

2.47.3 Major upgrade

Info

ClusterControl only supports PostgreSQL-based clusters for major version upgrade, for example from PostgreSQL 16.x to PostgreSQL 17.x.

Performs major software upgrades for the database software. Only one major version upgrade is supported at a time, for example, from PostgreSQL 13.x to 14.x. At the moment, only PostgreSQL-based clusters are supported. If you want to upgrade from PostgreSQL 12.x to 14.x, you have to upgrade stage-by-stage to PostgreSQL 13.x first, followed by PostgreSQL 14.x.

For a primary-replica replication setup, ClusterControl will perform the upgrade starting from the replica/secondary nodes and will eventually perform the upgrade on the primary node of a cluster (secondary first, primary last). During the eventual primary node upgrade, expect a service disruption to the database cluster service until the primary node is online again after restarting. If a node fails to be upgraded, the upgrade job is aborted and manual intervention is required to recover or reinstall the node. Due to this fact, it is important to have a proper maintenance window, which should only be performed when there is as little traffic as possible to the cluster.

Performing major version upgrade

Major upgrades are performed online on all database nodes in the cluster, one node at a time. The node will be stopped, then the software will be updated, and then the node will be started again. If a node fails to upgrade, the upgrade process is aborted and manual intervention is required to recover or reinstall the node.

Warning

Attention

Depending on the upgrade method chosen, you might require the storage capacity to hold 2 data sets.

To perform a major version upgrade for the database cluster, follow the instructions below for either the ClusterControl GUI or CLI.

ClusterControl GUI

1. Navigate to ***ClusterControl GUI → Clusters***
2. Select the desired cluster from the list. Only PostgreSQL-based clusters are supported.
3. Go to ***Actions (…) → Upgrades → Major upgrades***.
4. Under the **Configuration** tab, ClusterControl will display the database vendor, current major version and the upgrade major version. This is informational and not configurable. ClusterControl only allows one major version upgrade at a time.
5. Choose one of the upgrade method from the dropdown:
   - **copy** – This is the default method with `pg_upgrade`. The `--copy` option is used which will copy "files at a time" which is faster than doing regular backup and restore. This requires disk storage capacity to hold both the old and the new files. Note that data stored in user-defined tablespaces is not being copied to the new cluster during an upgrade. It remains in its original file system location but is duplicated into a subdirectory indicating the version number of the new cluster. To manually relocate files that are stored in a tablespace after upgrading, move the files to the new location and update the symbolic links to point to the files.
   - **link** – This method uses the `--link` option with `pg_ugprade`. Hard links will be used instead of copying (`--copy`) files which is faster and requires no extra disk storage. It is crucial to refrain from starting the old cluster after this step to avert any potential data corruption with the new cluster since they share the data.
   - **pgdumpall** – This is an alternative method to `pg_upgrade` and uses `pgdumpall` to copy files by performing a backup on the old cluster and then restoring it on the new cluster. This requires disk storage capacity to hold the backup for the old and new files.
6. Accept the disclaimer to proceed with the upgrade.
7. Under the **Advanced** tab, configure the following:
   - **Temporary master port** - The temporary PostgreSQL port of the older version. ClusterControl will reconfigure the PostgreSQL port of the older instances to this port for the duration of the upgrade, since the common port 5432, will be taken over by the newer PostgreSQL instance after the upgrading job completes.
   - **Temporary upgrade candidate port** - The temporary PostgreSQL port of the newer version while the upgrade job is running. ClusterControl distinguishes every node with a "hostname:port" pair. This provides a temporary entity for the newer database instance for the duration of the upgrade before it takes over the actual PostgreSQL port. This port will be released after the upgrade job is completed.
   - **Delete old node** - Toggle on to delete the older PostgreSQL instance data if the upgrade succeeds. Note that all previous data will be deleted and this action cannot be rolled back.
   - **Preserve configuration settings** - When enabled, important PostgreSQL configuration parameters will be preserved during major version upgrades instead of using default values for the new version.
8. Proceed to the **Preview** section and click **Submit**. ClusterControl will trigger a major upgrade job and the progress can be monitored under ***ClusterControl GUI → Activity Center → Jobs***.

ClusterControl CLI

- Perform major upgrade from PostgreSQL 14 to PostgreSQL 15 for cluster ID 21:

```
s9s cluster \
    --upgrade-cluster \
    --upgrade-to-version=15 \
    --cluster-id=21 \
    --log
```

- Perform major upgrade from PostgreSQL 15 to 16 for cluster ID 1, preserve the configuration and use upgrade method `--link`:

```
s9s cluster \
    --upgrade-cluster \
    --upgrade-to-version=16 \
    --upgrade-method=link \
    --preserve-configuration \
    --cluster-id=1 \
    --log
```

2.48 Alerts and Notifications

ClusterControl provides multiple ways to view and send alerts when an alarm raised. The alarms can be viewed within ClusterControl GUI and CLI, and the corresponding alerts can be sent via email, messaging platform (Slack, Telegram), incident management systems (PagerDuty, OpsGenie, VictorOps, ilert) or web hooks.

Email notification is built-in into ClusterControl logic and requires a *mail server configuration* and *recipients configuration*. For other alerting endpoints like messaging platform, incident management and web hooks, ClusterControl relies on another module call `cmon-events`, provided by package `clustercontrol-notifications`. This module allows ClusterControl alarms to be forwarded to third-party services and easily configurable via *Notification Services*.

See also

To learn more about ClusterControl notification module, see *ClusterControl Notifications*.

2.48.1 Alarms

The alarms page provides administrators with an aggregated view of alarms raised for all clusters monitored by ClusterControl. Each alarm entry has a header, details on the issue, severity level, category, corresponding cluster name, corresponding host, and timestamp. Alarms indicate problems that could affect or degrade the cluster as a whole. Each alarms is categorized as:

- Cluster
- Cluster recovery
- Database health
- Database performance

- Host
- Node
- Network

View alarms

ClusterControl GUI

1. Log in to your ***ClusterControl GUI → Clusters → choose a database cluster → Alarms***. This will open a page where you can view alarms for the particular cluster.
2. Alternatively, you can view all alarms for all clusters by going to ***ClusterControl GUI → Activity Center → Alarms***.
3. Click on ***Actions → Details***, to get a detailed explanation of the problem, together with the recommended action (if available) to resolve the problem.
4. An alarm can be acknowledged by clicking on ***Actions → Mute***. When muted, no notification will be sent out for this type of alarm until it is unmuted. An alarm cannot be deleted or dismissed.
5. You can also see the most recent alarms summarized in the **Home** page under the **Recent alarms** section (bottom right).

Tip

Rollover on the ***Cluster*** entry will pop up a cluster information card summarizing the cluster status.

ClusterControl CLI

For CLI:

- List all alarms for all clusters:

```
s9s alarm --list
```

- List all alarms for cluster ID 1:

```
s9s alarm --cluster-id=1 --list
```

2.48.2 Email notifications

When an alarm is raised, ClusterControl can send an alert to any mail recipients. To receive an email from ClusterControl, one has to configure a remote SMTP mail server (or use local MTA) and enable email notifications for the particular cluster, as shown in the following sections.

Configure SMTP mail server

After installation, by default, ClusterControl does not send out email until a mail server is configured, or a local mail transfer agent (Postfix, Sendmail or Exim) is installed on activated on the ClusterControl server.

ClusterControl GUI

1. To configure an SMTP mail server, go to ***ClusterControl GUI → Settings → Configure now***. It will open a wizard called "Configure mail server". Choose **SMTP Server**.
2. In the "Configure mail server: SMTP" window, configure the following:
   - **Server Address**: The SMTP mail server address that you are going to use to send the email.
   - **Port**: The SMTP port for the mail server. Usually, this value is 25 or 587, depending on your SMTP mail server configuration.
   - **User name**: The SMTP user name. Leave empty if no authentication required.
   - **Password**: The SMTP password for **User name**. Leave empty if no authentication required.
   - **Reply to/from**: Specify the sender of the email. This will appear in the 'From' field of the mail header.
   - **TLS/SSL required**: Check this box if ou want to use TLS/SSL for extra security. The mail server must support TLS/SSL.
3. Check the **Send test email** to test out the SMTP configuration. If you receive an email from ClusterControl, click **Save** and proceed to configure the mail recipients. See *Configure mail recipients*.

Configure local MTA

To use local MTA to send out emails, make sure the local MTA is installed and verified using the test email button available at ***ClusterControl GUI → Settings → Configure now → Sendmail → Send test email***.

Sendmail

On ClusterControl server, install the following packages:

RHEL/CentOS/AlmaLinux/Rocky Linux

```
yum install sendmail mailx
```

Debian/Ubuntu

```
apt-get install sendmail mailutils
```

Start the Sendmail service:

```
systemctl start sendmail
```

Verify if it works:

```
$ echo "test message" | mail -s "test subject" myemail@example.com
```

Replace `myemail@example.com` with your email address.

Postfix

Many Linux distributions come with Sendmail as the default MTA. To replace Sendmail and use other MTA, e.g Postfix, you just need to uninstall Sendmail, install Postfix and start the service. Following example shows commands that need to be executed on the ClusterControl node as the root user for RockyLinux 9:

```
systemctl stop sendmail
yum remove sendmail -y
yum install postfix mailx cronie -y
systemctl enable postfix
systemctl start postfix
```

Configure mail recipients

After a mail server is configured, or a MTA is installed and activated, one has to enable email notifications for the clusters.

ClusterControl GUI

1. Go to ***ClusterControl GUI → Clusters → choose a database cluster → Settings → Email notifications***. It will open a wizard to help you configure the email recipients. This page configures email notifications for alarms generated for the corresponding database cluster.
2. Each recipient (chosen from a user group) must be enabled before you can configure the delivery settings. Choose one group under **User group** (only groups that has at least read permission to the cluster), select a user under the **Users in the selected group** dropdown and click **Enable**.
3. After enabled, configure the **Digest delivery times** which when to send a digested (summary) email every day to the selected recipient. The timezone will follow the user's timezone and can be configured under the **User profile** setting.
4. Configure **Digest delivery times** which is the maximum number of non-digest email notifications should be sent per day to the selected recipient. The default is 100. This is to protect spamming the recipients for events that are set to "Deliver". See *Events*.
5. Next, configure the frequency of the emails to be sent for each event's category. See *Events*.
6. You can add an external email address (a user that does not exist in ClusterControl) by clicking on **Add an external email**.

Events

On every event severity, you may choose the frequency of raised alarms/events to be delivered to the recipient:

Action	Description
Ignore	Ignore if an alarm is raised.
Deliver	Send notification immediately via email once an alarm is raised.
Digest	Send a summary of alarms raised every day at **Digest delivery time**.

The following table shows the event category that you can forward to a specific recipient:

Event	Description
Network related logs	Network-related messages, e.g. host unreachable, SSH issues.
Cmon database logs	Internal CMON database-related messages.
Mail logs	Mail system-related messages.
Cluster logs	Cluster-related messages, e.g. cluster failed.
Cluster configuration logs	Cluster configuration messages, e.g. software configuration messages.
Recovery logs	Recovery messages like cluster or node recovery failures.
Node related logs	Message related to nodes, e.g. node disconnected, missing GRANT, failed to start HAProxy, failed to start NDB cluster nodes.
Host logs	Host-related messages, e.g. CPU/disk/RAM/swap alarms.
Health logs	Database health-related messages, e.g. memory usage of MySQL servers, connections.
Database performance	Alarms for long-running transactions and deadlocks.
Installation logs	Software installation-related messages.
Backup logs	Messages about backups.
Other logsp	Other uncategorized messages.

2.48.3 Forward alarms to third-party services

Alarms can be forwarded to third-party services like messaging platforms (Slack, Telegram), incident management systems (PagerDuty, ServiceNow, OpsGenie, VictorOps, ilert) and web hooks. See *Integration → Notification Services*.

2.49 Database Users and Privileges

ClusterControl offers a centralized feature for managing database users and their privileges, providing a comprehensive summary for your cluster. Any changes made through this feature are automatically synchronized across all database nodes in the cluster.

Supported database clusters:

- MySQL/MariaDB replication & standalone
- MySQL/MariaDB Galera Cluster
- MySQL Group Replication
- PostgreSQL/TimescaleDB
- Redis/Valkey

2.49.1 Best practices

When utilizing this feature, adhere to the following recommendations:

- Operational notes:
  - Changes made via ClusterControl are applied to all nodes in the cluster
  - Privilege changes take effect immediately

    - Ensure proper replication state before making user changes
- Security and administration recommendations:
    - Avoid granting global privileges unless necessary
    - Prefer database-specific privileges for application users
    - Regularly audit unused accounts
    - Use strong passwords and limit host access
    - Do not modify ClusterControl system users unless required

2.49.2 Access the database users management

ClusterControl GUI

Log in to your ***ClusterControl GUI → choose a database cluster → Manage → DB Users*** (for Redis-based clusters, it is ***Access Control***). You will see a panel where you can view and manage database users and privileges.

The panel view is different depending on the cluster type. See below:

- *MySQL/MariaDB-based clusters*
- *PostgreSQL-based clusters*
- *Redis-based clusters*

ClusterControl CLI

To manage database users using CLI, use the `account` or `accounts` command (both are aliases).

To view the manual or help pages, use the `--help` flag or, `man` command:

```
s9s account --help
s9s accounts --help
man s9s account
man s9s-account
```

2.49.3 MySQL/MariaDB-based clusters

ClusterControl provides a centralized interface to manage MySQL and MariaDB database users across your cluster. This feature allows administrators to view, create, modify, and audit database users and their privileges directly from the GUI without manual SQL commands.

Info

ClusterControl understands the MySQL/MariaDB topology and creates database users on all nodes where applicable in the cluster. This includes the supported load balancer nodes like ProxySQL.

Common system users for MySQL

ClusterControl automatically manages several system users required for cluster operation:

User	Purpose
cmon	Internal cluster management
cmonagent	ClusterControl query monitoring agent
backupuser	Database backup and restore
cmonexporter	Database metrics collection (Prometheus exporter)
proxysql-monitor	ProxySQL monitoring
rpl_user	Database replication
maxscale	MariaDB MaxScale monitoring

Warning

Warning!

Modifying these users may impact cluster functionality.

List MySQL/MariaDB database users

ClusterControl GUI

The user list provides a tabular overview of all database users. In the main panel, you can filter the list by filing up the textbox with the magnifying glasss icon.

Column	Description
User	MySQL username and host (e.g., `user@host`)
Active	Indicates whether the account is currently active
Privileges	Displays granted privileges

Click on **Refresh** instructs ClusterControl to read the database user information for all hosts, and aggregate them in one view.

Privileges are grouped for clarity in boxes. The left-side of the box contains the granted scope of the database objects, for example:

- `*.*` - Global privileges. The granted privilege applies to all databases and all tables.
- `performance_schema` - The granted privilege applies to all tables/views for database `performance_schema`.
- `performance_schema.hosts` - The granted privilege applies to table/view of `hosts`, under database `performance_schema`.

The right-side of the box contains the comma-separted list of granted privileges, for example, `ALTER`, `CREATE`, `ALTER ROUTINE`, `REPLICATION CLIENT PROCESS` and `SELECT`. Administrative privileges such as `GRANT OPTION` or `PROXY` are also shown when applicable.

ClusterControl CLI

- Lists all database users for cluster ID 1:

```
s9s account \
    --list \
    --cluster-id=1
```

- Lists all database users for cluster ID 1, complete with the privileges for every database objects:

```
s9s account \
    --list \
    --long \
    --cluster-id=1
```

- Show all database users `myapp` from cluster ID 1, complete with their privileges:

```
s9s account --list --long --cluster-id=1 | grep myapp
```

ClusterControl RPC API

- Get all database user accounts for cluster ID 1, authenticated as ClusterControl user `ccreader` that has view-only permission level:

```
curl -k -XPOST \
-d '{
    "operation": "listAccounts",
    "cluster_id": 1,
    "authenticate":
        {
        "username": "ccreader",
        "password": "thesupers3curepassw0rd"
        }
    }' \
https://localhost:9501/v2/clusters
```

Create a MySQL/MariaDB user

ClusterControl GUI

1. Click on **Create DB User** to open the user creation wizard.
2. In the **Create database user** panel, specify the following:
    - **Username**: Database username.
    - **Password**: Password for ***Username***. The minimum requirement is 4 characters.
    - **Hostname**: Hostname, IP address, or IP address range of the user or client. For the IP address range, use the SQL wildcard `%`, for example, `192.168.10.%` to allow 192.168.10.0/24.

- **Privileges**: Use the statement builder to add more grant statements. Click on **Add Privileges** to add another grant statement builder for this user.
  - **Grant**: Specify the privilege for this user. If the Privileges text box is active, it list all possible privileges of the server.
  - **on**: Specify the database or table name. It can be in `*.*`,`{database_name}`, `{database_name}.*` or `{database_name}.{table_name}` format.
- **Advanced**: Includes fine-grained control over the usage of this user.
  - **Max Queries Per Hour**: Maximum queries this user can perform in an hour. Default is 0 (unlimited).
  - **Max Updates Per Hour**: Maximum update operations this user can perform in an hour. Default is 0 (unlimited).
  - **Max Connections Per Hour**: Maximum connections allowed for this user in an hour. Default is 0 (unlimited).
  - **Max User Connections**: Maximum connections allowed for this user ever. Default is 0 (unlimited).
  - **Requires SSL**: Toggle **On** if the user must be authenticated using SSL.

3. To create the user, click **Create**. ClusterControl manages the entire process where it ensures the changes are made on the correct database node (the primary) and any relevant load balancer nodes, and then it propagates the updated user to all other database nodes, including flushing and loading the changes for runtime effect.
4. A toast message will appear on the top-right indicating the status of the user creation. If no error, the user will be listed in the User List view.

ClusterControl CLI

- Create a new MySQL user account called `myuser` with password `secr3tP4ss`, and allow it to have `ALL PRIVILEGES` on database `shop_db` while `SELECT` only on table `account_db.payments`:

```
s9s account \
    --create \
    --cluster-id=1 \
    --account="myuser:secr3tP4ss@192.168.0.100" \
    --privileges="shop_db.*:ALL;account_db.payments:SELECT"
```

- Create a new MariaDB user account called `mydbuser` on cluster ID 50, and allow this user to access from all hosts in network 192.168.0.0/24 to database `mydbshop`:

```
s9s account \
    --create \
    --cluster-id=50 \
    --account='mydbuser:Passw0rd999@192.168.0.%' \
    --privileges="mydbshop.*:ALL"
```

- Create a new MySQL user account called `myapp` on cluster ID 1, and create a database with the same name `myapp` and assign the user with `ALL PRIVILEGES` to the database:

```
s9s account \
    --create \
    --cluster-id=1 \
    --account='myapp:Passw0rd999@192.168.0.%' \
    --with-database
```

Example

```
$ s9s account \
    --create \
    --cluster-id=1 \
    --account='myapp:Passw0rd999@192.168.0.%' \
    --with-database

Using plugin ''.
192.168.73.11:3306: Server supports new MySQL authentication syntax.␣
↪Server has version 8.4.5-5.1.
Account 'myapp' created.
Database 'myapp' created.
Access for 'myapp' to 'myapp' granted.
Importing account on ProxySql host(s).
Created.
```

ClusterControl RPC API

- Create a new database user called `mydbuser2` for cluster ID 1 and allow it to access from 192.168.10.0/24 with `ALL PRIVILEGES` for database `mydb2` and SELECT only for database `myapp`, authenticated as ClusterControl user `ccrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "createAccount",
    "cluster_id": 1,
    "account":
        {
       "user_name": "mydbuser2",
       "password": "pass123",
       "host_allow": "192.168.10.%",
       "grants": "mydb2:ALL;myapp:SELECT"
        },
    "authenticate":
        {
        "username": "ccrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

!!! Info

```
  If the same `createAccount` operation is called twice with an␣
↪identical account information, the latter call will be automatically␣
↪converted to an `ALTER` operation (edit an existing database user).
```

Modify MySQL/MariaDB users

ClusterControl GUI

1. Click on the action button **... → *Edit*** to edit an existing user.
2. In the **Edit database user** panel, specify the following:
   - **New Password**: Password for the database user. The minimum requirement is 4 characters.
   - **Privileges**: Use the statement builder to add more grant statements. Click on **Add Privileges** to add another grant statement builder for this user.
     - **Grant**: Specify the privilege for this user. If the Privileges text box is active, it list all possible privileges of the server.
     - **on**: Specify the database or table name. It can be in `*.*`,`{database_name}`, `{database_name}.*` or `{database_name}.{table_name}` format.
   - **Advanced**: Includes fine-grained control over the usage of this user.
     - **Max Queries Per Hour**: Maximum queries this user can perform in an hour. Default is 0 (unlimited).
     - **Max Updates Per Hour**: Maximum update operations this user can perform in an hour. Default is 0 (unlimited).
     - **Max Connections Per Hour**: Maximum connections allowed for this user in an hour. Default is 0 (unlimited).
     - **Max User Connections**: Maximum connections allowed for this user ever. Default is 0 (unlimited).
     - **Requires SSL**: Toggle **On** if the user must be authenticated using SSL.
3. To modify the user's settings, click **Save**. ClusterControl manages the entire process where it ensures the changes are made on the correct database node (the primary) and any relevant load balancer nodes, and then it propagates the updated user to all other database nodes, including flushing and loading the changes for runtime effect.
4. A toast message will appear on the top-right indicating the status of the user modification. If no error, the user will be listed in the User List view.

ClusterControl CLI

- List all database users to find the one that you want to edit:

```
s9s account \
    --list \
    --long \
    --cluster-id=1
```

- Modify an existing database user `'dataadmin'@'%'` by adding another MySQL administrative privilege called `PROCESS`:

```
s9s account \
    --grant \
    --cluster-id=1 \
    --account='dataadmin@%' \
    --privileges="PROCESS"
```

- Reset the password for user `'mydb'@'192.168.73.15'` to `MySuperDuperS3cureP455w0rd`:

```
s9s account \
    --create \
    --cluster-id=1 \
    --account='mydb:MySuperDuperS3cureP455w0rd@192.168.73.15'
```

Example

```
$ s9s account \
    --create \
    --cluster-id=1 \
    --account='mydb:MySuperDuperS3cureP455w0rd@192.168.73.15'

192.168.73.11:3306: Found auth plugin 'caching_sha2_password' for␣
↪account 'mydb'@'192.168.73.15'.
Updating account 'mydb'@'192.168.73.15'.
192.168.73.11:3306: Alter user processed: ALTER USER 'mydb'@
↪'192.168.73.15' IDENTIFIED WITH caching_sha2_password BY 'xxx xx'
Importing account on ProxySql host(s).
Created.
```

!!! Info

```
  The `--create` flag is capable of updating an existing user, as well␣
↪as creating a new user if does not exist.
```

- Modify an existing database user `'user1'@'192.168.10.14'` by adding a number of database privileges on table `db1.reports`:

```
s9s account \
    --grant \
    --cluster-id=1 \
    --account='user1@192.168.10.14' \
    --privileges="db1.reports:SELECT,INSERT,UPDATE"
```

- Modify an existing database user `'user1'@'192.168.10.14'` by revoking `INSERT` and `UPDATE` database privileges from table `db1.reports`:

```
s9s account \
    --revoke \
    --cluster-id=1 \
    --account='user1@192.168.10.14' \
    --privileges="db1.reports:INSERT,UPDATE"
```

ClusterControl RPC API

- Reset the password for an existing database user `'mydbuser2'@'192.168.73.14'` on cluster ID 1 to `mySuperSecr3tP455` and limit the database user connections to 30, authenticated as ClusterControl user `myrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "updateAccount",
    "cluster_id": 1,
    "account":
        {
       "user_name": "mydbuser2",
       "password": "mySuperSecr3tP455",
       "host_allow": "192.168.73.14",
       "max_user_connections": 30
        },
    "authenticate":
        {
        "username": "myrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

- Modify privileges for an existing database user `'mydbuser2'@'192.168.73.14'` on cluster ID 1 by adding `DELETE` and `TRUNCATE` to database `testDB`, authenticated as ClusterControl user `myrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "grantPrivileges",
    "cluster_id": 1,
    "privileges": "testDB.*:DELETE,TRUNCATE",
    "account":
        {
       "user_name": "mydbuser2",
       "host_allow": "192.168.73.14"
        },
    "authenticate":
        {
        "username": "myrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

Drop MySQL/MariaDB users

When dropping a database user, ClusterControl manages the entire process where it ensures the removals are made on the primary database node, or one of the primary nodes if it is a multi-primary cluster.

ClusterControl GUI

1. Click on the action button **…** → ***Delete*** and click **Delete** to confirm removing an existing user.
2. A toast message will appear on the top-right indicating the status of the user deletion.

ClusterControl CLI

- Drop a database user `'joe'@'%'` on cluster ID 1:

```
s9s account \
    --delete \
    --cluster-id=1 \
    --account="joe"
```

> **Tip**
>
> If a host value is missing in the `--account` option, the host value will default to `%`.

- Drop a database user `'mydb2'@'192.168.40.%'` on cluster ID 1:

```
s9s account \
    --delete \
    --cluster-id=1 \
    --account='mydb2@192.168.40.%'
```

ClusterControl RPC API

- Drop a database user `'mydbuser2'@'192.168.73.15'` for cluster ID 1, authenticated as ClusterControl user `myrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "deleteAccount",
    "cluster_id": 1,
    "account":
        {
       "user_name": "mydbuser2",
       "host_allow": "192.168.73.15"
        },
    "authenticate":
        {
        "username": "myrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

Warning

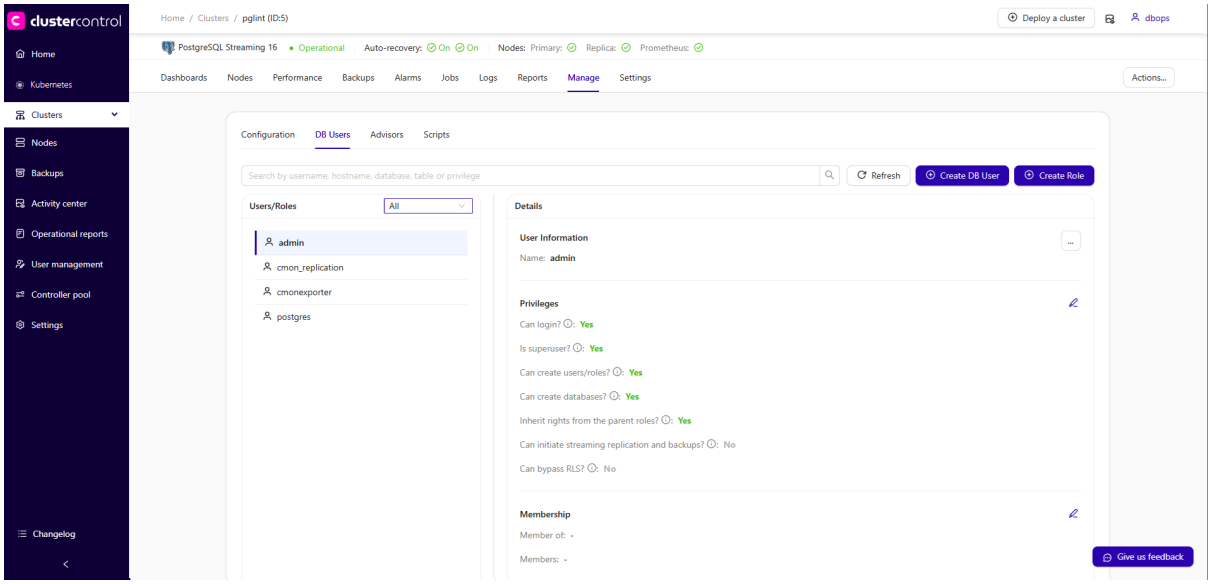
Attention

If the database cluster has a ProxySQL load balancer, ClusterControl would create the corresponding user on the ProxySQL as well. Deleting the database user here does not reflect the ProxySQL user. Separate deletion on ProxySQL is necessary for a full user removal.

2.49.4 PostgreSQL-based clusters

Since the release of ClusterControl 2.4.0, you can manage roles, enforce password policies, and configure fine grained permissions (down to the table level) directly from the GUI. It's also improved the user experience and enhanced `pg_hba.conf` synchronization to ensure consistent security settings across nodes and reduce connectivity issues.

To get started, navigate to ***Cluster → Manage → DB Users***. From there, you can create or modify roles, assign privileges, and adjust password policies through a guided interface.



Common system users for PostgreSQL

ClusterControl automatically manages several system users required for cluster operation:

User	Purpose
cmon	Internal cluster management
cmonagent	ClusterControl query monitoring agent
backupuser	Database backup and restore
cmonexporter	Database metrics collection (Prometheus exporter)
rpl_user	Database replication

Warning

Warning!

Modifying these users may impact cluster functionality.

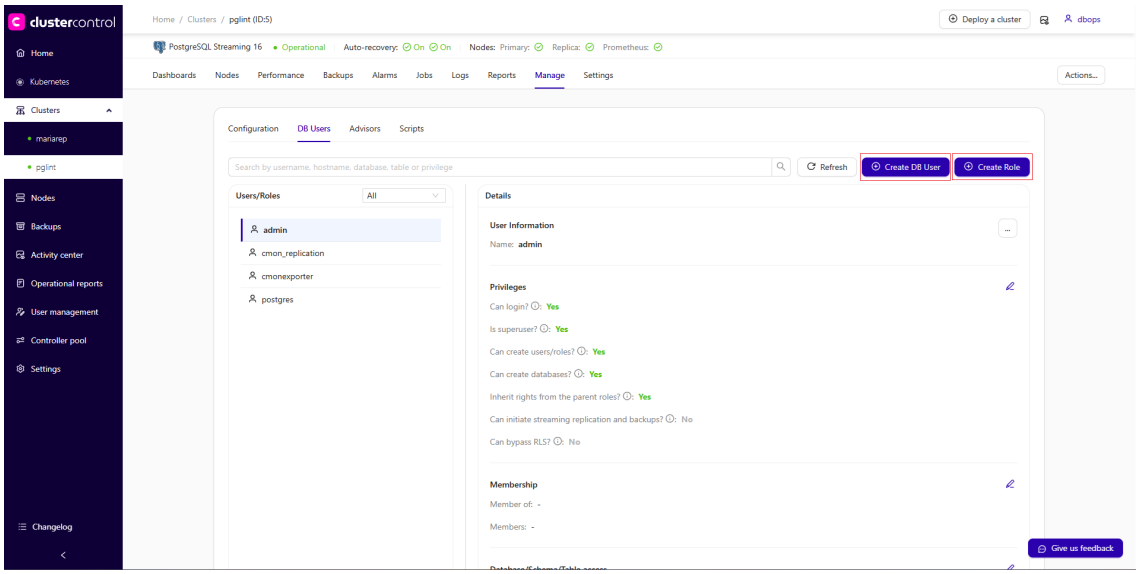
Roles and Users creation

PostgreSQL role and user creation can be fully managed from the GUI. As a best practice, we recommend creating roles first, defining the required privileges at the role level, and then assigning those roles to users. This approach improves access control management, simplifies permission updates, and helps maintain consistency across your PostgreSQL environment.

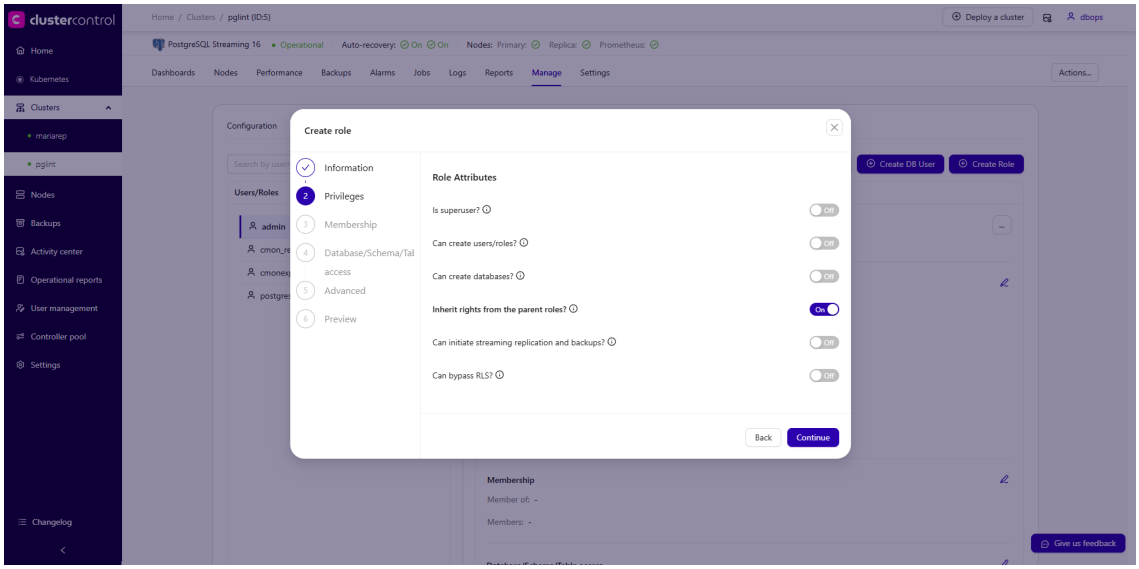
To create and configure roles or users, go to the *DB Users* section in ClusterControl, where you can define role attributes, assign privileges, and manage role memberships.

ClusterControl GUI

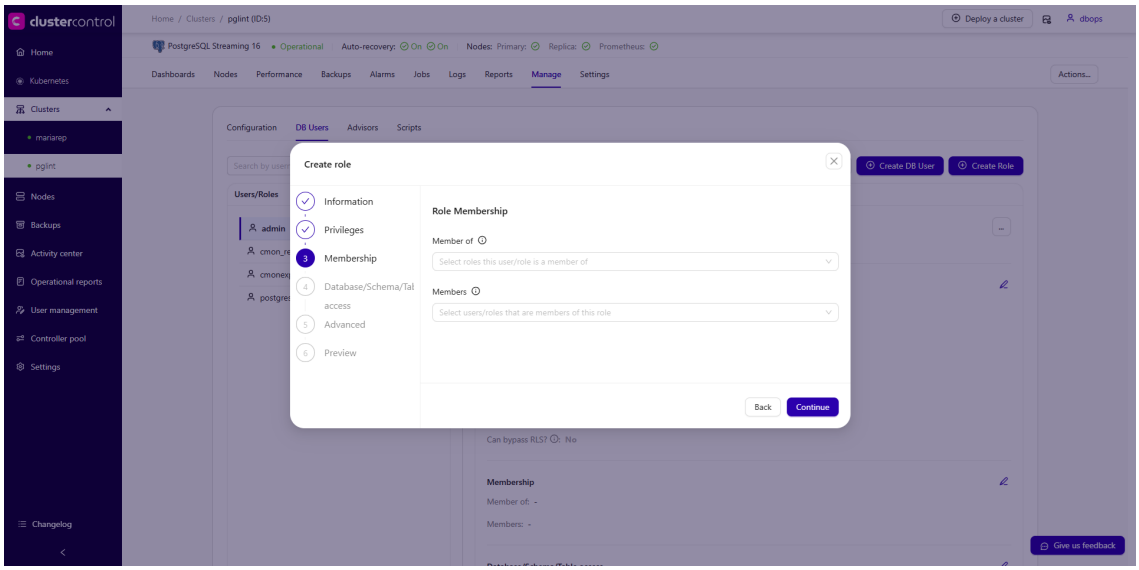
1. On the **DB Users** menu, click **Create Role** to create a new role and click **Create DB User** to create a new user.



2. A dialog will appear. Complete the following steps.
3. **Information**
   - For role creation, enter the role name in the **Role Name** input field.
   - For user creation, enter the **Username** and **Password** in the respective input fields.
4. **Role Privileges:** Configure the appropriate privileges for the role.

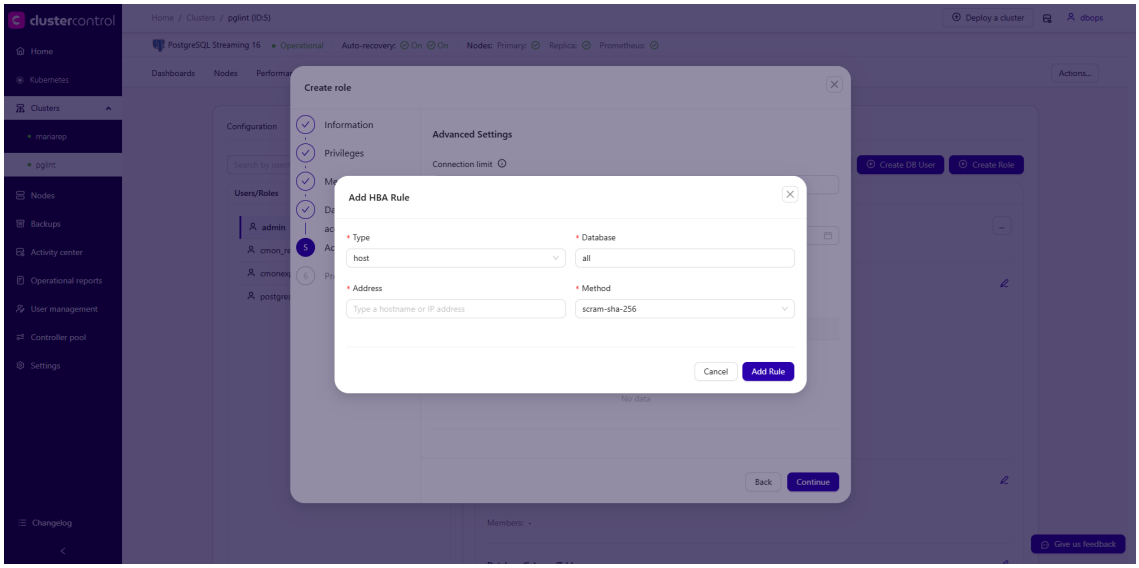


5. **Role Membership:** Configure role memberships or inheritance as needed. In this section, you can define both "Member of" and "Members" relationships.



- `Member of`: Assign this role to existing roles so it inherits their privileges.
- `Members`: Add other roles or users as members of this role, allowing them to inherit the privileges defined here.

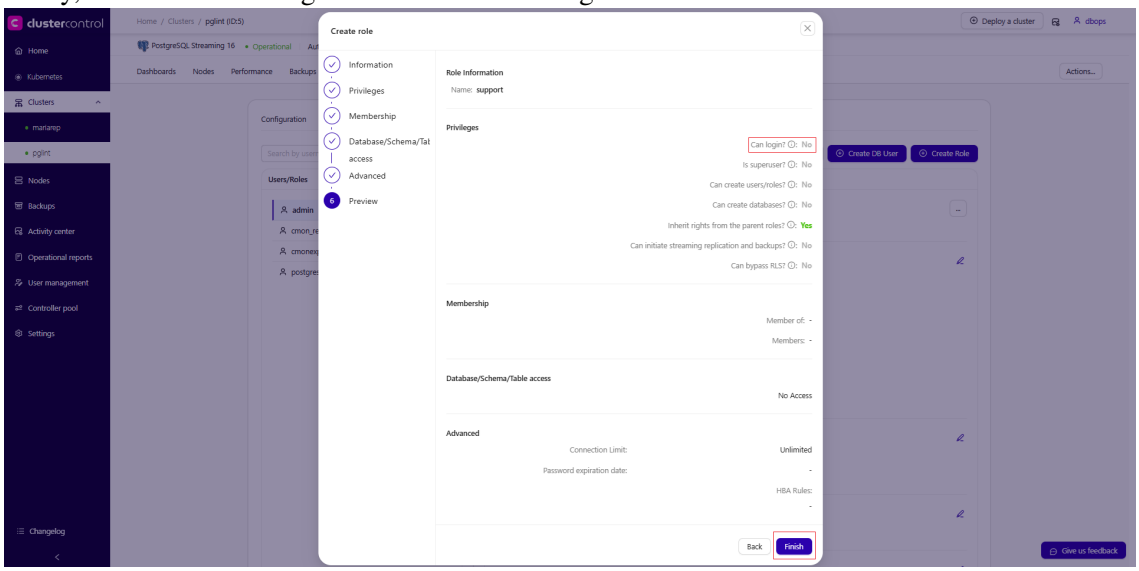
6. **Object Access:** Configure access at the database, schema, or table level.
7. **Advanced:** Configure additional options, including pg_hba settings and other advanced parameters.



- **Type:** Specifies the connection type. Common options:
  - `host`: Matches TCP/IP connections (IPv4 and IPv6) and standard network connections
  - `hostssl`: Matches TCP/IP connections that require SSL/TLS
  - `local`: Matches Unix socket connections (local machine only)
- **Database:** Specifies which database(s) the rule applies to.
  - `all`: Applies to all databases (default)
  - A specific database name (for example: `postgres`, `template1`, `product`)
- **Address:** Defines the client source address allowed to connect. Supported formats:
  - Single IP address (`10.10.0.25/32`)
  - Subnet or CIDR notation (`10.10.0.0/24`)
  - All IPv4 addresses (`0.0.0.0/0`): *not recommended*
  - IPv6
- **Method:** Specifies the authentication method used to validate client connections. Common methods:

Method	Description	Typical Use Case	Security Level
`trust`	Allows connection without authentication.	Testing or internal environments only.	Not secure
`reject`	Explicitly denies a connection attempt.	Blocking specific IPs or subnets.	-
`scram-sh`	Secure password-based authentication using SCRAM.	Recommended for production environments.	High
`md5`	Legacy password authentication using MD5 hashing.	Backward compatibility with older clients.	Medium
`password`	Sends password in clear text (use only with SSL).	Legacy systems requiring plain password auth.	Low (without SSL)
`gss`	Uses GSSAPI (Kerberos).	Enterprise environments with Kerberos SSO.	High
`sspi`	Windows SSPI authentication (Active Directory).	Windows domain environments.	High
`ident`	Uses ident protocol to verify OS user over TCP/IP.	Legacy or controlled internal networks.	Low Medium
`peer`	Authenticates based on matching OS user (Unix socket only).	Local Linux connections.	High (local only)
`ldap`	Authenticates against an LDAP server.	Centralized enterprise authentication.	High (depends on LDAP config)
`radius`	Uses a RADIUS server for authentication.	Enterprise centralized authentication.	High
`cert`	Uses SSL client certificate authentication.	High security environments requiring mutual TLS.	Very High
`pam`	Uses Pluggable Authentication Modules (PAM).	Linux systems integrating with system authentication.	High (depends on PAM setup)

8. Lastly, review the configuration before clicking the Finish button in the **Preview** section.



ClusterControl CLI

Create a new PostgreSQL role called `vendor`, and allowed the host in the network subnet 10.10.20.0/24 to access the database `postgres` with readonly privilege:

```
s9s account \
    --create \
    --cluster-id=5 \
    --account='vendor@10.10.20.0/24' \
    --privileges="postgres.*:SELECT"
```

Create a new PostgreSQL user called `vender_abc`

```
s9s account \
    --create \
    --cluster-id=5 \
    --account='vender_abc:Passw0rd999'
```

The difference between a Role and a User in s9s cli is the password. During creation, an account created with a password will be flagged as a User, while an account created without a password will be flagged as a Role.

ClusterControl RPC API

This request creates a role or user in PostgreSQL User Management (v2). For example, it creates a role named `vendor` with allowed host `10.10.20.0/24` and `readonly` privileges on tables in the `postgres` database.

```
curl -k -XPOST \
-d '{
    "cluster_id": 5,
    "operation": "createAccount",
    "account": {
    "class_name": "CmonAccount",
    "user_name": "vendor",
    "ssl_type": "NONE",
    "max_user_connections": 0,
    "max_questions": 0,
    "max_updates": 0,
    "max_connections": -1,
    "member_of": [],
    "members": [],
    "allowed_hosts": [
        {
        "type": "host",
        "database": "all",
        "user": "vendor",
        "address": "10.10.20.0/24",
        "method": "scram-sha-256",
        "inherited": false
        }
    ],
```

(continues on next page)

(continued from previous page)

```
    "grants":
↪"NOSUPERUSER,NOLOGIN,INHERIT,NOCREATEROLE,NOCREATEDB,NOREPLICATION,NOBYPASSRLS;postgres
    },
    "authenticate": {
    "username": "myrpcusr",
    "password": "mySuperStr0ngP@ssw0rd999"
    }
}' \
https://10.10.15.100:9501/v2/clusters
```

Note

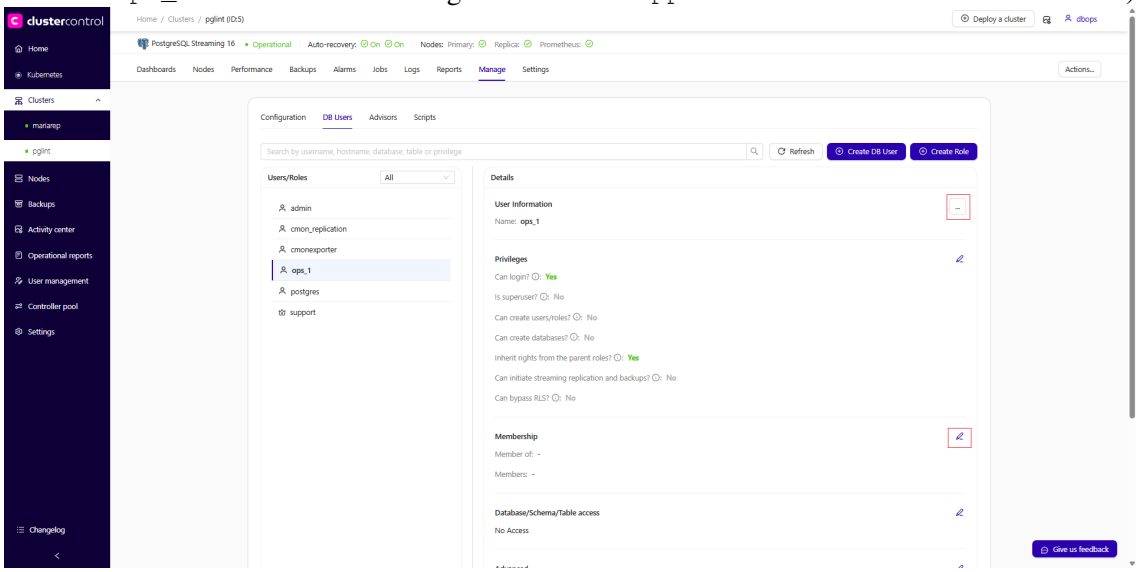
Note Roles created from the UI cannot log in to the database by default. You can verify this in the **Preview section**, where it shows: `Can login? : No`

Roles and Users modification

After creating roles and users, you can modify them as needed. This includes changing passwords, updating privileges (LOGIN, SUPERUSER, CREATEDB, CREATEROLE), managing role memberships, and updating `pg_hba.conf` policies. To make changes, navigate to the management section in the UI.

ClusterControl GUI

1. Navigate to **Cluster → Manage → DB Users**.
2. Select the role or user you want to modify.
3. Click on the action button **… → Edit** or click the **edit (pencil)** icon on each section.
4. Modify the user or role. (for example, modifying the user `ops_1` and attaching the `support` role to the user)



5. Review the changes and ensure everything is correct
6. Click **Save** to apply the updates.

ClusterControl CLI

Coming soon (To be updated once available)

ClusterControl RPC API

This request modifies a role or user in PostgreSQL User Management (v2). For example, it modifies user `vendor_abc` to become a member of role `vendor`.

```
curl -k -XPOST \
-d '{
    "cluster_id": 5,
    "operation": "updateAccount",
    "account": {
    "class_name": "CmonAccount",
    "user_name": "vendor_abc",
    "ssl_type": "NONE",
    "max_connections": -1,
    "member_of": [
        {
        "name": "vendor",
        "with_admin": false
        }
    ],
    "members": [],
    "grants":
↪"NOSUPERUSER,LOGIN,INHERIT,NOCREATEROLE,NOCREATEDB,NOREPLICATION,NOBYPASSRLS"
    },
    "authenticate": {
    "username": "myrpcusr",
    "password": "mySuperStr0ngP@ssw0rd999"
    }
}' \
https://10.10.15.100:9501/v2/clusters
```

2.49.5 Redis-based clusters

The Access Control feature in ClusterControl for Redis-based clusters (including Valkey cluster, Redis Sentinel and Valkey Sentinel setups) allows administrators to manage Redis ACL users, permissions, and security policies. This functionality is built upon Redis's native permissions and Access Control List (ACL) mechanism, which is detailed in the Redis documentation and Valkey documentation. For ClusterControl command-line users, the same management features are accessible via the `account` command.

This feature simplifies the configuration of Redis authentication and authorization by providing a centralized interface to define:

- Users
- Command permissions
- Key access patterns
- Channel access
- Categories

Common system users for Redis

ClusterControl automatically manages several system users required for Redis/Valkey cluster operation:

User	Purpose
default	Default Redis user
replica-user	Database replication
sentinel-user	Redis/Valkey Sentinel user for failover coordination

Warning

Warning!

Modifying these users may impact cluster functionality.

View Access Control List (ACL)

ClusterControl GUI

The access control list provides a tabular overview of all database users. In the main panel, you can filter the list by filing up the textbox with the magnifying glasss icon.

The ACL table displays the following information for each user:

Field	Description
User	Redis ACL username
Status	Whether the user is enabled or disabled
Categories	Command categories assigned (e.g., `+@all`, `-@all`)
Channels	Pub/Sub channel access patterns
Keys	Key pattern access (e.g., `~*`)
Commands	Specific allowed commands (e.g., `+get`, `+set`)

Click on **Refresh** instructs ClusterControl to read the database user information for all hosts, and aggregate them in one view.

ClusterControl CLI

- Lists all database users for cluster ID 1.

```
s9s account \
    --list \
    --cluster-id=1
```

- Lists all database users for cluster ID 1, complete with the privileges for every database objects:

```
s9s account \
    --list \
    --long \
    --cluster-id=1
```

Create a Redis user

ClusterControl GUI

1. Click on **Create ACL rule** to open the user creation wizard.
2. In the **Create ACL rule** panel, specify the following:
   - **Username**: Database username. This field is mandatory.
   - **Password**: Password for ***Username***. If no password is specified user will be set with `nopass` special string. It means that it will be possible to authenticate as such user with any password.
   - **Categories**: Predefined groups of commands in Redis ACLs used to manage user permissions. Examples include `+@admin` for administrative commands, `+@read` for read-only commands, and `+@write` for write commands. This field is mandatory.
   - **Channels**: Custom channels in Redis Pub/Sub messaging that users can be allowed or denied access to via ACLs. Channels facilitate the broadcasting of messages to multiple subscribers.
   - **Keys**: Custom channels in Redis Pub/Sub messaging that users can be allowed or denied access to via ACLs. Channels facilitate the broadcasting of messages to multiple subscribers.
   - **Commands**: Specific Redis commands that users can be allowed or denied access to in ACLs. Commands perform various operations such as setting values (`set`), retrieving data (`get`) or deleting keys (`del`). Prefix the command with plus '+' to allow or '–' to deny.
3. To create the user, click **Create**. ClusterControl manages the entire process where it ensures the changes are made on the correct database node (the primary), and ensure the user is propagated to all other database nodes.
4. A toast message will appear on the top-right indicating the status of the user creation. If no error, the user will be listed in the list view.

ClusterControl CLI

- Create a new Redis user account called `cache1` with password `secr3tP4ss` on cluster ID 4, and allow this user all commands except the dangerous ones for all channels (`&*`) and keys(`~*`):

```
s9s account \
    --create \
    --cluster-id=4 \
    --account='cache1:secr3tP4ss' \
    --privileges='+@all -@dangerous'
```

- Create a new non-password Redis user called `nopassuser` on cluster ID 4, and allow this user to run all list-related commands on all channels and keys:

```
s9s account \
    --create \
    --cluster-id=4 \
    --account='nopassuser' \
    --privileges='+@list'
```

ClusterControl RPC API

- Create a new Redis user called `mycache` for cluster ID 1 and allow this user to run all admin-related commands except the dangerous ones on all keys and channels, authenticated as ClusterControl user `ccrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "createAccount",
    "cluster_id": 1,
    "account":
        {
       "user_name": "mycache",
       "password": "passW0rd123",
       "host_allow": "%",
       "grants": "+@admin -@dangerous"
        },
    "authenticate":
        {
        "username": "ccrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

!!! Info

```
  If the same `createAccount` operation is called twice with an␣
↪identical account information, the latter call will be automatically␣
↪converted to an `ALTER` operation (edit an existing database user).
```

Modify Redis users

ClusterControl GUI

1. Click on the action button **…** → ***Edit*** to edit an existing ACL rule.
2. In the **Edit ACL rule** panel, specify the following:
   - **Password**: New password for the user. If no password is specified user, the current password will be kept. If you want to set the user without password (with `nopass` special string), create a new user with blank password instead.
   - **Categories**: Predefined groups of commands in Redis ACLs used to manage user permissions. Examples include `+@admin` for administrative commands, `+@read` for read-only commands, and `+@write` for write commands. This field is mandatory.
   - **Channels**: Custom channels in Redis Pub/Sub messaging that users can be allowed or denied access to via ACLs. Channels facilitate the broadcasting of messages to multiple subscribers.
   - **Keys**: Custom channels in Redis Pub/Sub messaging that users can be allowed or denied access to via ACLs. Channels facilitate the broadcasting of messages to multiple subscribers.
   - **Commands**: Specific Redis commands that users can be allowed or denied access to in ACLs. Commands perform various operations such as setting values (`set`), retrieving data (`get`) or deleting keys (`del`). Prefix the command with plus '+' to allow or '–' to deny.

3. To modify the user's settings, click **Save**. ClusterControl manages the entire process where it ensures the changes are made on the correct database node (the primary), and ensure the user is propagated to all other database nodes.
4. A toast message will appear on the top-right indicating the status of the user modification. If no error, the user will be listed in the User List view.

ClusterControl CLI

- List all database users to find the one that you want to edit:

```
s9s account \
    --list \
    --long \
    --cluster-id=1
```

- Modify an existing database user `dataadmin` by allowing administrative commands called `@admin`:

```
s9s account \
    --grant \
    --cluster-id=1 \
    --account='dataadmin' \
    --privileges="+@admin"
```

- Reset the password for user `mydb` to `MySuperDuperS3cureP455w0rd`:

```
s9s account \
    --create \
    --cluster-id=1 \
    --account='mydb:MySuperDuperS3cureP455w0rd'
```

!!! Info

```
  The `--create` flag is capable of updating an existing user, as well␣
↪as creating a new user if does not exist.
```

ClusterControl RPC API

- Reset the password for an existing database user `mydbuser` on cluster ID 1 to `mySuperSecr3tP455`, authenticated as ClusterControl user `myrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "updateAccount",
    "cluster_id": 1,
    "account":
        {
       "user_name": "mydbuser2",
       "password": "mySuperSecr3tP455"
        },
    "authenticate":
```

(continues on next page)

(continued from previous page)

```
        {
        "username": "myrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

- Modify privileges for an existing database user `mydbuser2` on cluster ID 1 by allowing `@admin` and denying `@dangerous` commands to all databases, authenticated as ClusterControl user `myrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "grantPrivileges",
    "cluster_id": 1,
    "privileges": "+@admin -@dangerous",
    "account":
        {
       "user_name": "mydbuser2"
        },
    "authenticate":
        {
        "username": "myrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

Drop Redis users

When dropping a database user, ClusterControl manages the entire process where it ensures the removals are made on the primary database node, or one of the primary nodes if it is a multi-primary cluster.

ClusterControl GUI

1. Click on the action button **…** → ***Delete*** and click **Delete** to confirm removing an existing user.
2. A toast message will appear on the top-right indicating the status of the user deletion.

ClusterControl CLI

- Drop a Redis user 'joe' on cluster ID 1:

```
s9s account \
    --delete \
    --cluster-id=1 \
    --account="joe"
```

ClusterControl RPC API

- Drop a Redis user 'mycache' for cluster ID 1, authenticated as ClusterControl user `myrpcadmin` that has MANAGE permission level:

```
curl -k -XPOST \
-d '{
    "operation": "deleteAccount",
    "cluster_id": 1,
    "account":
        {
       "user_name": "mycache"
        },
    "authenticate":
        {
        "username": "myrpcadmin",
        "password": "mySuperStr0ngP@ssw0rd999"
        }
    }' \
https://localhost:9501/v2/clusters
```

In a testing environment, you might need to perform a shutdown on ClusterControl and monitored database hosts in a graceful way. Depending on the clustering technology, the order of startup and shutting down is vital to keep the whole cluster in sync and ensure a smooth start-up operation in the future.

It's recommended to let the ClusterControl node to be the last one to shut down since it needs to oversee the state of the monitored hosts and saves it into the CMON database. When starting up the database cluster at the later stage, ClusterControl will perform a proper start-up procedure based on the last known state of the monitored hosts.

ClusterControl needs to know whether the database cluster that you are shutting down was shutdown outside of the ClusterControl domain. The following subsections provide details on the order of steps to perform in order to perform a graceful database cluster/server shutdown using ClusterControl.

2.50 MySQL/MariaDB Replication

Shutting down:

1. Shutdown the application manually. This usually outside of the ClusterControl domain.
2. Shutdown the Keepalived (if exists) by doing system shut down or through ClusterControl UI.
3. Shutdown the load balancer service (if exists) by doing system shut down or through ClusterControl UI.
4. Shutdown the replicas by using ***ClusterControl → Nodes → choose the server → Shutdown Node → Execute***.
5. Shutdown the primary by using ***ClusterControl → Nodes → choose the server → Shutdown Node → Execute***.
6. Shutdown ClusterControl host by doing system shut down.

Starting up:

1. Power on the ClusterControl host. Ensure you are able to connect to the UI and see the last state of the database cluster.

2. Power on the primary. If auto recover is turned on, this primary will be started automatically.
3. Once the primary is started, power on the remaining replicas.
4. Disable read-only on the primary by using ***ClusterControl → Nodes → choose the primary → Disable Read-Only → Execute***.
5. Power on the load balancer service (if exists).
6. Start the application manually. This usually outside of the ClusterControl domain.

2.51 MySQL Galera

Shutting down:

1. Shutdown the application manually. This usually outside of the ClusterControl domain.
2. Shutdown the Keepalived (if exists) by doing system shut down or through ClusterControl UI.
3. Shutdown the load balancer service (if exists) by doing system shut down or through ClusterControl UI.
4. Shutdown the database cluster by using ***ClusterControl → Cluster Actions → Stop Cluster → Proceed***.
5. Shutdown ClusterControl host by doing system shut down.

Starting up:

1. Power on the ClusterControl host. Ensure you are able to connect to the UI and see the last state of the database cluster.
2. If auto-recovery is turned on, the cluster will be started automatically. Otherwise, go to ***ClusterControl → Cluster Actions → Bootstrap Cluster → Proceed***.
3. If auto-recovery is turned on, the load balancer and virtual IP service will be started automatically. Otherwise, start the load balancer service and virtual IP service accordingly.
4. Start the application manually. This usually outside of the ClusterControl domain.

2.52 MySQL Cluster (NDB)

Shutting down:

1. Shutdown the application manually. This usually outside of the ClusterControl domain.
2. Shutdown the Keepalived (if exists) by doing system shut down or through ClusterControl UI.
3. Shutdown the load balancer service (if exists) by doing system shut down or through ClusterControl UI.
4. Shutdown the MySQL API servers by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
5. Shutdown the Data (NDB) servers by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
6. Shutdown the MySQL Cluster management servers by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
7. Shutdown ClusterControl host by doing system shut down.

Starting up:

1. Start ClusterControl host. Ensure you are able to connect to the UI and see the last state of the database cluster.
2. If auto recover is turned on, the cluster will be started automatically. Otherwise, start the nodes in this order - MySQL management, MySQL data, MySQL API.
3. Start the load balancer service (if exists).
4. Start the virtual IP service (if exists).
5. Start the application manually. This usually outside of the ClusterControl domain.

2.53 MongoDB ReplicaSet

Shutting down:

1. Shutdown the application manually. This usually outside of the ClusterControl domain.
2. Shutdown the secondaries by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
3. Shutdown the primary by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
4. Shutdown ClusterControl host by doing a system shut down.

Starting up:

1. Start ClusterControl host. Ensure you are able to connect to the UI and see the last state of the database cluster.
2. Start the primary. If auto recover is turned on, this primary will be started automatically.
3. Once the primary is started, start the remaining secondaries.
4. Start the application manually. This usually outside of the ClusterControl domain.

2.54 PostgreSQL/TimeScaleDB Replication

Shutting down:

1. Shutdown the application manually. This usually outside of the ClusterControl domain.
2. Shutdown the slaves by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
3. Shutdown the master by using ***ClusterControl → Nodes → pick the server → Shutdown Node → Execute***.
4. Shutdown ClusterControl host by doing system shut down.

Starting up:

1. Start ClusterControl host. Ensure you are able to connect to the UI and see the last state of the database cluster.
2. Start the master. If auto recover is turned on, this master will be started automatically.
3. Once the master is started, start the remaining slaves.

4. Start the application manually. This usually outside of the ClusterControl domain.

Note

Note If the database server was being shut down gracefully outside of ClusterControl knowledge (through command line init script, systemd or `kill -15`), ClusterControl would still attempt to recover the database node if Node AutoRecovery is turned on. Unless the node is marked as 'Under Maintenance'.

2.55 Maintenance

ClusterControl offers a maintenance mode for individual nodes or entire clusters. This feature suppresses alarms and notifications for a set or scheduled duration. Users can also document the reason for the maintenance, such as RAM upgrades or OS patching, for auditing. While in maintenance mode, ClusterControl will not automatically alter the node's state unless manual actions are taken.

Warning

Attention

If automatic cluster/node recovery is enabled, ClusterControl will always recover a node/cluster regardless of the maintenance mode status. Do not forget to disable cluster/node recovery to avoid ClusterControl interfering with your maintenance tasks.

2.55.1 Type of maintenance mode

ClusterControl has a built-in logic to handle two type of maintenance mode:

- Cluster-wide maintenance
- Individual node maintenance

Cluster-wide

Cluster-wide maintenance mode lets you schedule a maintenance window for the entire cluster rather than on a per-node basis. This is suitable if you want to perform maintenance on all nodes in the cluster in the same maintenance window. When active:

- Alarms and notifications are suppressed; ClusterControl will not generate alerts (e.g., threshold breaches, failover notifications) for cluster events during the maintenance period.
- All nodes are marked "under maintenance". Every node regardless of role (primary, replica, proxy, etc.) is shown with the maintenance badge in ClusterControl GUI, and a global banner flags the upcoming or active window.
- Automatic recovery can be disabled (optional). By default recovery/failover remains enabled, but you can turn it off to avoid "false" repairs during planned work.

Individual node

Schedule a maintenance window for a specific cluster node when targeted maintenance is required. While active:

- Alarms and notifications are suppressed; ClusterControl will not generate alerts (e.g., threshold breaches, failover notifications) for node events during the maintenance period.
- Only the selected node is marked "under maintenance" and will display a maintenance badge in the ClusterControl GUI.
- Automatic recovery can be disabled (optional). By default recovery/failover remains enabled, but you can turn it off to avoid "false" repairs during planned work.

2.55.2 Configuring maintenance mode

Maintenance mode can be configured from ClusterControl GUI and also using ClusterControl CLI.

ClusterControl GUI

Cluster-wide maintenance mode

1. Log in to your ***ClusterControl GUI → choose a database cluster → Actions → Schedule maintenance***.
2. Choose a database cluster.
3. Go to ***Actions → Schedule maintenance***. This will bring up a panel where you can activate a cluster-wide maintenance mode.

Node maintenance mode

1. Log in to your ***ClusterControl GUI → Clusters***.
2. Choose a database cluster.
3. Go to ***Nodes → select the node → Actions → Schedule maintenance***. This will bring up a panel where you can activate maintenance mode for the corresponding node.

Alarms and notifications will be reactivated once the maintenance period is over, or the operator explicitly disables it by going to ***Nodes → select the node → Actions → Disable Maintenance Mode*** or **_choose a database cluster → Actions → Disable Maintenance Mode_**.

ClusterControl CLI

- Create a maintenance period for PostgreSQL node 10.35.112.21, starting at 05:44:55 AM for one full day (cmon expects UTC time to create a maintenance):

```
s9s maintenance \
     --create \
     --nodes=10.35.112.21:5432 \
     --begin=2024-05-15T05:44:55.000Z \
     --end=2024-05-16T05:44:55.000Z \
     --reason='Upgrading RAM' \
     --batch
```

- Create a new maintenance period for 192.168.1.121 which shall start tomorrow and finish an hour later:

```
s9s maintenance --create \
      --nodes=192.168.1.121 \
      --begin="$(date --date='now + 1 day' --utc '+%Y-%m-%d %H:%M:%S')
↪'" \
      --end="$(date --date='now + 1day + 1 hour' --utc '+%Y-%m-%d
↪%H:%M:%S')" \
      --reason="Upgrading software."
```

2.55.3 Checking maintenance status

Check the maintenance status allowing you to quickly determine if maintenance mode is active. Alarms and notifications will be deactivated if the maintenance mode is active.

ClusterControl GUI

Cluster-wide maintenance mode

1. Log in to your ***ClusterControl GUI → Clusters***.
2. Choose a database cluster.
3. If you see a blue wrench icon next to the cluster status, it means the maintenance mode is active for the cluster. Roll over on the wrench icon will show a pop-up card with more details on the maintenance like status, end time, who iniatiated the maintenance and the reason.

Node maintenance mode

1. Log in to your ***ClusterControl GUI → Nodes***.
2. Choose a node from the list.
3. If you see a blue wrench icon next to the node status, it means the maintenance mode is active for the node. Roll over on the wrench icon will show a pop-up card with more details on the maintenance like status, end time, who iniatiated the maintenance and the reason.

ClusterControl CLI

- List out all clusters or nodes that are under maintenance period:

```
s9s maintenance --list --long
```

2.55.4 Deactivating maintenance mode

Maintenance mode automatically ends after the set end time. You can also manually deactivate it earlier once maintenance is finished and the server is ready. Alarms and notifications will be reactivated once the maintenance period is over.

To deactivate the maintenance mode:

ClusterControl GUI

Cluster-wide maintenance mode

1. Log in to your ***ClusterControl GUI → Clusters***.
2. Choose the database cluster.

3. Go to ***Actions → Maintenance → Stop current maintenance***. Confirm the action in the confirmation box to proceed.

Node maintenance mode

1. Log in to your ***ClusterControl GUI → Clusters***.
2. Choose the database cluster.
3. Go to ***Nodes → select the node → Actions → Maintenance → Stop current maintenance***. Confirm the action in the confirmation box to proceed.

ClusterControl CLI

- Find the item UUID by listing out all nodes that are under maintenance:

```
s9s maintenance --list --long
```

- Delete a maintenance period for UUID 70346c3:

```
s9s maintenance --delete --uuid=70346c3
```

2.56 Certificate Management

Certificate Management allows you to manage a set of SSL certificates and keys that can be provisioned on your clusters. This feature allows you to create Certificate Authority (CA) and/or self-signed certificates and keys. Then, it can be easily enabled and disabled for MySQL and PostgreSQL client-server connections using the SSL encryption feature. You can access it by going to ***ClusterControl GUI → Settings → Certificate management***. This will bring up a tab where you can view and manage certificates.

By default, the generated keys and certificates will be created under the default repository at `/var/lib/cmon/ca`.

Field	Description
Create Folder	Create a new directory under the default repository.
Refresh	Refresh the certificate and directory list.
More…	Generate Client/Server Certificate. Generate Self-signed Certificate. Import certificate.

2.56.1 Generate Self-signed Certificate

Generate a self-signed Certificate Authority and key. You can use this Certificate Authority (CA) to sign your client and server certificates.

Field	Description
Path	Certification repository path. To change the path, click on the file browser left-side menu. Default value is `/var/lib/cmon/ca`.
Certificate Authority and Key Name	Enter a name without an extension. For example MyCA, ca-cert.
Description	Put some description for the certificate authority.
Country	Choose a country name from the dropdown menu.
State	State or province name.
Localityr	City name.
Organization	Organization name.
Organization unit	Unit or department name.
Common name	Specify server fully-qualified domain name (FQDN) or your name. Common Name value used for the server and client certificates/keys must each differ from the Common Name value used for the CA certificate. Otherwise, the certificate and the key files will not work for the servers compiled using OpenSSL.
Email	Email address.
Key length (bits)	The key length in bits. 2048 and higher is recommended. The larger the public and private key length, the harder it is to crack
Expiration Date (days)	Certificate expiration in days.
Generate	Generate certificate and key.
Reset	Reset the form.

2.56.2 Generate Client/Server Certificate

Sign with an existing CA or generate a self-signed certificate. ClusterControl generates certificates and keys depending on the type, server or client. The generated server's key and certificate can then be used by the Enable SSL Encryption feature.

Field	Description
Certificate Authority	Select an existing CA (by clicking on any existing CA on the left-hand side menu) or leave it empty to generate a self-signed certificate.
Type	Server – Generate a certificate for server usage. Client – Generate a certificate for client usage.
Certificate and Key Name	Enter the certificate and key name. The same name will be used by ClusterControl to generate certificates and keys. For example, if you specify the name as ***Severalnines***, ClusterControl will generate `severalnines.key` and `severalnines.crt` respectively.
Description	Put some description for the certificate authority.
Country	Choose a country name from the dropdown menu.
State	State or province name.
Localityr	City name.
Organization	Organization name.
Organization unit	Unit or department name.
Common name	Specify server fully-qualified domain name (FQDN) or your name. Common Name value used for the server and client certificates/keys must each differ from the Common Name value used for the CA certificate. Otherwise, the certificate and the key files will not work for the servers compiled using OpenSSL.
Email	Email address.
Key length (bits)	The key length in bits. 2048 and higher is recommended. The larger the public and private key length, the harder it is to crack.
Expiration Date (days)	Certificate expiration in days.
Generate	Generate certificate and key.
Reset	Reset the form.

2.56.3 Import

Import keys and certificates into ClusterControl's certificate repository. The imported keys and certificates can then be used to enable SSL encryption for server-client connection, replication, or backup at a later stage. Before you perform the important action, bear in mind to:

1. Upload your certificate and key to a directory in the ClusterControl Controller host

2. Uncheck the Self-signed Certificate checkbox if the certificate is not self-signed
3. You need to also provide a CA certificate if the certificate is not self-signed
4. Duplicate certificates will not be created

Field	Description
Destination Path	Where you want the certificate to be imported to. Click on the file explorer window on the left to change the path.
Save As	Certificate name.
Certificate File	Absolute path to the certificate file. For example: `/home/user/ssl/file.crt`.
Private Key File	Absolute path to the key file. For example: `/home/user/ssl/file.key`.
Self-signed Certificate	Uncheck the checkbox if the certificate is not self-signed.
Import	Start the import process.

2.57 Backup and Restore

This section focuses on essential strategies and tools for backing up and restoring your database clusters with ClusterControl. Learn how to ensure data safety and enable quick recovery from potential data loss. You can also scale your cluster with a backup.

See also

- *Backup & Restore Overview →*
- *Create Backup →*
- *Schedule Backup →*
- *Restore Backup →*
- *Restore External Backup →*
- *Backup Verification →*
- *Create Cluster from Backup →*
- *Upload Backup to Cloud →*
- *Download Backup from Cloud →*

2.58 Overview

ClusterControl provides comprehensive backup and restore capabilities to ensure the continued availability and integrity of your database systems. This overview outlines how ClusterControl simplifies the process of creating reliable backups, managing your backup strategy, and performing efficient data restoration for various database technologies.

2.58.1 What you can do

Here are the list of things you can do or what you will get with backups and restore features in ClusterControl, across its supported database engines:

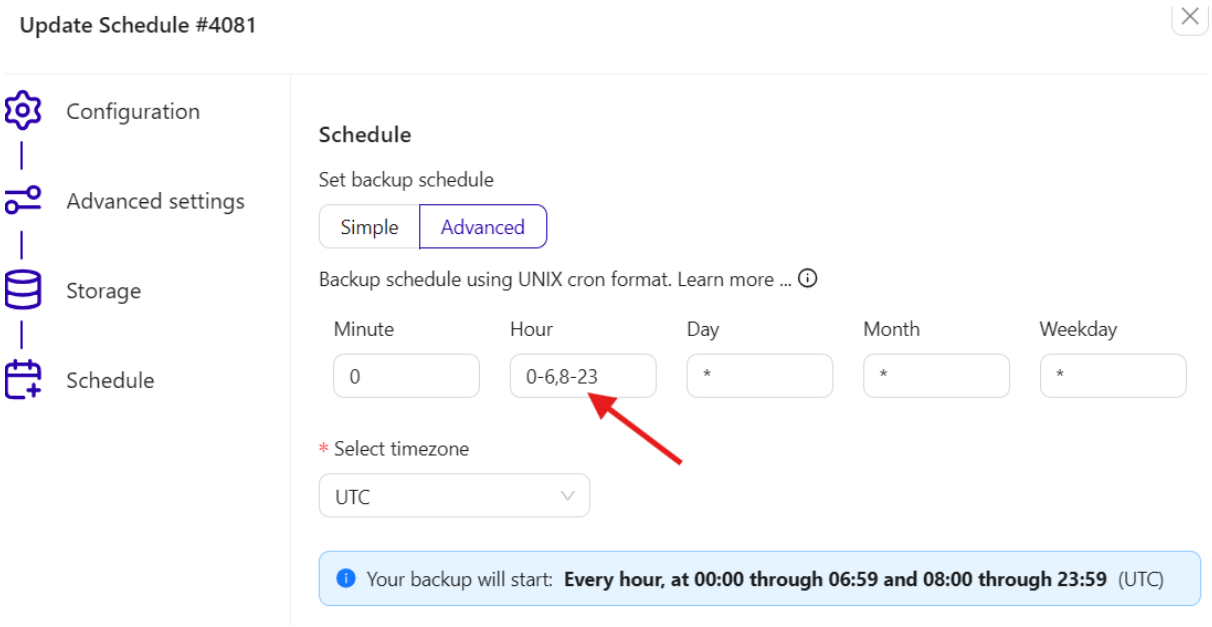
- Centralized backup management platform for all your database clusters - Create, schedule, verify, store, and restore backups, all from a unified dashboard in ClusterControl GUI or via the ClusterControl CLI.
- Create or schedule physical or logical database backup on any database node.
- Create or schedule full, incremental, or partial (specific schema/table).
- Define rotation and retention policies for local backups or cloud backups. The old backups will be automatically purged.
- Get alerts via email, PagerDuty, Slack or custom webhooks on any backup-related events.
- Create or schedule operational reports for backup status.
- Backups can be stored on local storage, controller storage, public cloud object storage, or any private cloud object storage that supports S3-compatible storage.
- Automatic backup verification.
- Ability to restore to a temporary database server, not part of the cluster.
- Create a new cluster from an existing backup.
- Backup encryption.
- Backup compression with configurable compression level.
- Point-in-time recovery for MySQL, MariaDB and PostgreSQL.
- Partial backup and restore.
- Backup history.
- Manage backups via ClusterControl CLI.
- Incremental or differential backup chaining - Backups are grouped together based on the backup type.
- Auto select backup host and backup failover.
- Backup throttling.
- Configure the backup file name and its subdirectory format.
- Upload/Download existing backups to/from cloud.

2.58.2 Backup jobs

The backup process performed by ClusterControl is running as a background thread (RUNNING3) which doesn't block any other non-backup jobs in the queue. If the backup job takes hours to complete, other non-backup jobs can still run simultaneously via the main thread (RUNNING). You can see the job progress at ***ClusterControl GUI → Activity center → Jobs***.

Note that ClusterControl executes **a single backup job per cluster** at any given time. If multiple scheduled backups for a cluster overlap, the subsequent schedules will not start until the active backup job is finished. For instance, if a full daily backup is set for 7:00 AM, and incremental backups are scheduled hourly, the 7:00 AM incremental backup will likely wait for the full backup to complete before it begins. You may

avoid this by excluding certain hours in the schedule by using the scheduler advanced editor, which follow the UNIX cron format, as shown in the following example:



2.58.3 Backup encryption and decryption

If the encryption option is enabled for a particular backup, ClusterControl will use OpenSSL to encrypt the backup using the AES-256 CBC algorithm. Encryption happens on the backup node. If you choose to store the backup on the controller node, the backup files are streamed over in encrypted format through `socat` or `netcat`.

If compression is enabled, the backup is first compressed and then encrypted resulting in smaller backup sizes. The encryption key will be generated automatically (if not exists) and stored inside the CMON configuration for the particular cluster under `backup_encryption_key` the option. This key is stored with base64 encoded and should be decoded first before using it as an argument to pass when decrypting the backup. The following command shows how to decode the key:

```
cat /etc/cmon.d/cmon_X.cnf | grep ^backup_encryption_key | cut -d"'" -f2 |␣
↪base64 -d > keyfile.key
```

Where X is the cluster ID. The above command will read the `backup_encryption_key` value and decode the value to a binary output. Thus, it is important to redirect the output to a file, as in the example, we redirected the output to `keyfile.key`. The key file which stores the actual encryption key can be used in the OpenSSL command to decrypt the backup, for example:

```
$ cat {BACKUPFILE}.aes256 | openssl enc -d -aes-256-cbc -pass␣
↪file:/path/to/keyfile.key > backup_file.xbstream.gz
```

Or, you can pass the stdin to the respective restore command chain, for example:

```
$ cat {BACKUPFILE}.aes256 | openssl enc -d -aes-256-cbc -pass␣
↪file:/path/to/keyfile.key | gzip -dc | xbstream -x -C /var/lib/mysql
```

2.58.4 Backup retention

Backup retention can be configured at ***ClusterControl GUI → choose a cluster → Backups → … → Backup Settings → Default backups retention period*** (default is 31 days) to the number of days ClusterControl keeps the existing backups. Backups older than the value defined will be deleted. You can also customize the retention period per backup (default, custom or keep forever) under the **Backup Retention** field when creating or scheduling a backup.

The purging is based on the following conditions:

1. When a new backup is successfully created, and if no verified backup is requested, the older backups will be checked and removed.
2. When the verified backup is successfully created, the older backups will be checked and removed.
3. The backup housekeeping job remains executed every 24 hours. Thus, if no backups are created and no backups are verified, the backup retention still will be done every 24 hours

For cloud-storage backups, they follow the **Cloud backup retention period** setting (default is 180 days).

!!! Danger

```
Backups exceeding the retention period will be deleted. Once the backups␣
↪are deleted, they can't be recovered.
```

2.58.5 Supported backup methods

ClusterControl offers support for various backup tools and methods, tailored to the specific database clusters in use. The table below details these options:

Cluster type	Supported backup methods
MySQL-based clusters	mysqldumpPercona Xtrabackup (full, incremental)
MariaDB-based clusters	mariadb-dumpMariaDB Backup (full, incremental)
PostgreSQL-based clusters	pg_dumppg_dumpallpg_basebackuppgBackRest (full, incremental, differential)
MongoDB-based clusters	mongodumpPercona Backup for MongoDB
Redis-based clusters	RDBAOF
SQL Server	FullDifferentialTransaction log
Elasticsearch	Elasticsearch snapshot

Each database backup method is explained in the following sections.

mysqldump

ClusterControl performs mysqldump against all or selected databases by using the `--single-transaction` option. It automatically performs mysqldump with `--master-data=2` if it detects binary logging is enabled on the particular node to generate a binary log file and position statement in the dump file. ClusterControl generates a set of 4 mysqldump files with the following suffixes:

- `_data.sql.gz` – Schemas' data.
- `_schema.sql.gz` – Schemas' structure.
- `_mysqldb.sql.gz` – MySQL system database.
- `_triggerseventroutines.sql.gz` – MySQL triggers, events, and routines.

Percona Xtrabackup

Percona Xtrabackup is an open-source MySQL hot backup utility from Percona. It is a combination of xtrabackup (built in C) and innobackupex (built on Perl) and can back up data from InnoDB, XtraDB, and MyISAM tables. Xtrabackup does not lock your database during the backup process. For large databases (100+ GB), it provides a much better restoration time as compared to mysqldump. The restoration process involves preparing MySQL data from the backup files before replacing or switching it with the current data directory on the target node.

Since its ability to create full and incremental MySQL backups, ClusterControl manages incremental backups, and groups the combination of full and incremental backups in a backup set. A backup set has an ID based on the latest full backup ID. All incremental backups after a full backup will be part of the same backup set. The backup set can then be restored as one single unit using the **Restore Backup** feature.

Warning

Attention

Without a full backup to start from, the incremental backups are useless.

mariadb-dump

The mariadb-dump is a logical backup utility included with MariaDB Server. It's essentially MariaDB's fork of the original MySQL mysqldump tool and is a replacement to mysqldump since MariaDB 10.4. It supports MariaDB system-versioned tables, sequence, virtual columns and MariaDB-specific privilege grants which are different than the MySQL implementation.

On all supported versions for MariaDB, ClusterControl will default to mariadb-dump as the preferred logical backup method.

MariaDB Backup

MariaDB Backup is a fork of Percona XtraBackup with added support for compression and data-at-rest encryption available in MariaDB, included in MariaDB 10.1.23 and later. It is an open-source tool provided by MariaDB for performing physical online backups of InnoDB, Aria, and MyISAM tables. MariaDB Backup is available on Linux and Windows.

On all supported versions for MariaDB, ClusterControl will default to MariaDB Backup as the preferred backup method and snapshot state transfer (SST) method.

pg_dump/pg_dumpall

Depending on the database backup target configuration, ClusterControl will automatically choose the suitable method to create the logical backup using either `pg_dump` or `pg_dumpall` backup tool. Commonly, `pg_dump` will be chosen for a partial logical backup, while `pg_dumpall` for a full logical backup.

ClusterControl performs `pg_dumpall` against all databases together with `--clean` the option, which includes SQL commands to clean (drop) databases before recreating them. DROP commands for roles and tablespaces are added as well. The output will be `.sql.gz` extension and the file name contains the timestamp of the backup.

pg_basebackup

pg_basebackup is used to take base backups of a running PostgreSQL database cluster. These are taken without affecting other clients to the database and can be used both for point-in-time recovery and as the starting point for a log shipping or streaming replication standby server. It makes a binary copy of the database cluster files while making sure the system is put in and out of backup mode automatically. Backups are always taken of the entire database cluster; it is not possible to back up individual databases or database objects.

ClusterControl connects to the replication stream using the replication user (default is `cmon_replication`) with `--wal-method=fetch` option when creating the backup. The output will be `base.tar.gz` inside the backup directory.

pgBackRest

pgBackRest is an open-source software developed to perform efficient backup on PostgreSQL databases that measure in tens of terabytes and greater. It supports per-file checksums, compression, partial/failed backup resume, high-performance parallel transfer, asynchronous archiving, tablespaces, expiration, full/differential/incremental, local/remote operation via SSH, hard-linking, restore, and more. The tool is written in Perl and does not depend on rsync or tar but instead performs its own deltas which give it maximum flexibility.

Starting from ClusterControl 1.9.0, pgBackRest can be configured as follows:

1. **Primary**: Install on the current master but not on slaves. The backup repository (host to store the backup data) will be configured to be on the master. There will be no SSH configuration for PgBackRest.
2. **All database nodes**: Install on all database nodes. The backup repository will be created on the current master. The backup will be made by using a standby node. PgBackRest will use SSH for communication between hosts.
3. **All database nodes and a dedicated repository host**: Install on all PostgreSQL database nodes. The backup repository will be made on a specified host. The backup will be made by using a standby node. PgBackRest will use SSH for communication between hosts.

Note

Note The pgBackRest backup directory cannot be reused or shared with other backup methods.

During the first attempt of making pgBackRest backup, ClusterControl will re-configure the node to install and configure pgBackRest. Take note that this operation requires a database restart and might introduce downtime to your database. A configuration file will be created at `/etc/pgbackrest.conf` and will be configured according to the version used and location of the PostgreSQL data. The pgBackRest default repository path is located at `/var/lib/pgbackrest`. Additionally, ClusterControl will configure the following lines inside postgresql.conf (which explains why it requires restart during the first run):

```
archive_mode = on
archive_command = 'pgbackrest --stanza=clustercontrol-stanza archive-push %p
↪'
```

Warning

Attention

PITR using `pg_basebackup` will no longer be possible if switching the WAL archiving method from "local WAL archiving" to "pgBackRest".

Info

In the ClusterControl GUI, pgBackRest nodes are listed with a fake port number (200000 + cluster ID), due to the internal CmonHost object requirement for a host:port combination. As a matter of fact, the pgBackRest process does not listen, nor need a port number.

Full backup

Full backup of pgBackRest copies the entire contents of the database cluster to the backup. The first backup of the database cluster is always a Full Backup. pgBackRest is always able to restore a full backup directly. The full backup does not depend on any files outside of the full backup for consistency.

Differential backup

For differential backup, pgBackRest copies only those database cluster files that have changed since the last full backup. pgBackRest restores a differential backup by copying all of the files in the chosen differential backup and the appropriate unchanged files from the previous full backup. The advantage of a differential backup is that it requires less disk space than a full backup, however, the differential backup and the full backup must both be valid to restore the differential backup.

For example, if a full backup is taken on Sunday and the following daily differential backups are scheduled, the data that is backed up will be:

- Monday – data from Sunday to Monday
- Tuesday – data from Sunday to Tuesday
- Wednesday – data from Sunday to Wednesday
- Thursday – data from Sunday to Thursday

Incremental backup

For incremental backup, pgBackRest copies only those database cluster files that have changed since the last backup (which can be another incremental backup, a differential backup, or a full backup). As an incremental backup only includes those files changed since the prior backup, they are generally much smaller than full or differential backups. As with the differential backup, the incremental backup depends on other backups to be valid to restore the incremental backup. Since the incremental backup includes only those files since the last backup, all prior incremental backups back to the prior differential, the prior differential backup, and the prior full backup must all be valid to perform a restore of the incremental backup. If no differential backup exists then all prior incremental backups back to the prior full backup, which must exist, and the full backup itself must be valid to restore the incremental backup.

For example, if a full backup is taken on Sunday and the following daily incremental backups are scheduled, the data that is backed up will be:

- Monday – data from Sunday to Monday
- Tuesday – data from Monday to Tuesday
- Wednesday – data from Tuesday to Wednesday
- Thursday – data from Wednesday to Thursday

mongodump

ClusterControl uses the standard command to perform mongodump with `--journal`, which allows mongodump operations to use the durability journal to ensure that the export is in a consistent state against shards.

Info

mongodump (and mongorestore) is not available for MongoDB Sharded Cluster 4 and later that have sharded transactions in progress, as backups created with mongodump do not maintain the atomicity guarantees of transactions across shards.

Percona Backup for MongoDB

Percona Backup for MongoDB is a distributed, low-impact solution for achieving consistent backups of MongoDB sharded clusters and replica sets. Percona Backup for MongoDB supports Percona Server for MongoDB and MongoDB Community v3.6 or higher with MongoDB Replication enabled (standalone is not supported due to the dependency on MongoDB's oplog). The Percona Backup for MongoDB project inherited from and replaces `mongodb-consistent-backup`, which is no longer actively developed or supported.

Percona Backup for MongoDB requires an extra step for installation and configuration, and it is not enabled by default. You can use ClusterControl to install this tool by going to ***ClusterControl → GUI Clusters → choose the MongoDB cluster → Cluster Action → Install Percona Backup***, or simply choose the backup method percona-backup-mongodb from the dropdown list when configuring a backup method for a backup. If the tool is not installed, ClusterControl will advise on installing the backup tool first before moving forward to the next step.

Warning

Attention

Percona Backup for MongoDB requires a remote file server mounted to a local directory. It is the responsibility of the server administrators to guarantee that the same remote directory is mounted at exactly the same local path on all servers in the MongoDB cluster or non-sharded replica set. If the path is accidentally a normal local directory, errors will eventually occur, most likely during a restore attempt.

Redis/Valkey RDB & AOF

RDB is Valkey Database Backup file. It is a dump of all user data stored in an internal, compressed serialization format at a particular timestamp which is used for point-in-time recovery (recovery from a timestamp). AOF stands for Append Only File, which is actually a persistence technique in which an RDB file is generated once and all the data is appended to it as it comes.

When backing up Redis or Valkey, ClusterControl backs up both RDB and AOF (if enabled) files on the selected database node.

MSSQL Backup

ClusterControl performs a backup routine on a Microsoft SQL Server by using the `sqlcmd` client to connect to the SQL Server and take backups. For a full backup, ClusterControl will iterate against all databases and create a standard full backup. Full database backups represent the database at the time the backup is finished.

For differential backup, it only backs up the data that has changed since the last full backup. This type of backup requires you to work with fewer data than a full database backup, while also shortening the time required to complete a backup. All the differential backups will be grouped under the last full backup on the **Backup** page.

Minimally, one must have created at least one full backup before one can create any log backups. After that, the transaction log can be backed up at any time unless the log is already being backed up. It is recommended to take log backups frequently, both to minimize work loss exposure and to truncate the transaction log.

A database administrator typically creates a full database backup occasionally, such as weekly, and, optionally, creates a series of differential database backups at shorter intervals, such as daily. Independent of the database backups, the database administrator backs up the transaction log at frequent intervals.

One of the limitations of using MSSQL Backup is backups created by a more recent version of SQL Server cannot be restored in earlier versions of SQL Server.

Elasticsearch Snapshot

An Elasticsearch snapshot is a way to back up your Elasticsearch indices and data. The process involves creating a snapshot of one or more indices and storing it in a designated repository. This snapshot can then be used to restore the indices to their previous state in case of data loss or other issues. The snapshot process is incremental, meaning that only changes made since the last snapshot are stored. Snapshots can be taken manually or automatically on a schedule, and can also be used to migrate data between clusters or to create new indices.

Note that during cluster deployment, ClusterControl configures Elasticsearch snapshot storage by requiring the specification of the repository name, storage location, and file system path early in the process. The snapshot repository will then be available under the **Backups** section of the Elascticsearch cluster.

2.59 Create Backup

ClusterControl provides an automated and centralized approach to managing backups for your database clusters. Creating a backup can be achieve by **Backup on Demand** and ***Schedule a Backup***. In this page, the Create Backup focuses on **Backup on Demand**.

2.59.1 Overview of Backup on Demand

Backup on demand can do the following actions:

- Create a single backup instantly
- Store backups on premise or in the cloud
- Compress and encrypt backups for secure storage

2.59.2 Support Matrix

All database clusters that are supported by ClusterControl allows you to create a backup. For a complete list of our supported database clusters, see below:

Database	Vendor	Topology
MySQL	Percona, Percona Pro, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.59.3 Prerequisites

When creating a backup, certain requirements should be met in order to create a backup. See below:

- Only backup methods that ClusterControl supports can be use to create a backup
- Backup created in ClusterControl can only be stored locally either on the ClusterControl controller or to the registered database nodes available on that cluster. If you have external backup host but not registered in ClusterControl, you can use NFS and share the volume where your backup shall be stored
- You have sufficient storage space for the backup.
- You have appropriate user permissions to manage backups.

2.59.4 Creating the backup

The steps on creating a backup can differ depending on the type of database cluster. Make sure you follow the instructions and guide based on the following sections per database cluster you are going to create the backup.

MySQL Replication

The following vendors and versions are supported for creating a backup:

- Oracle MySQL - 8.0.
- Percona Server for MySQL - 8.0.
- Percona Server for MySQL Pro - 8.0.
- MariaDB Server - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backup method supported are: mysqldump, xtrabackup by Percona for Percona Server and Oracle MySQL database clusters, mariabackup by MariaDB for MariaDB database clusters.
- for mysqldump supports dump type for *Schema only*, *MySQL DB Only*, *Data Only*, *Schema And Data*, *Complete PITR-compatible*
- supports upload to the cloud of the following providers: *Amazon Web Services*, *Google Cloud*, *Microsoft Azure*, *S3 Compatible storage provider* (***upload backups to any AWS S3 compatible cloud storage provider by creating a custom S3 cloud storage credential***)
- compression is set to **On**
- retention days is set to `31`
- compression level is set to `6 (System Default)`
- *Use extended insert* is set to **On** when *mysqldump* is choosen as the backup method
- *Backup locks* is set to **On** when *xtrabackup* is choosen as the backup method
- *Xtrabackup parallel copy threads* is set to `1`
- storage location is set to `Store on controller`
- storage directory is set to `/home/vagrant/backups` as well as its sub `Backup subdirectory` is set to `BACKUP-%I`

Steps to Create a Backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MySQL Replication cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `mysqldump`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Cannot be set to **Off**.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.

3. **Compression level** is set to `6 (System default)`
4. **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.
5. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
6. **Use extended insert** is set to **On**. If you do not want the extended inserts to generate in your `mysqldump` file, then set this to *Off*.
7. **One dump file per DB** is set to **Off**. If you want `mysqldump` to generate backup files per database, set this to **On**.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using mysqldump as backup method

```
s9s backup \
    --create \
    --cluster-id=127 \
    --nodes=192.168.40.54 \
    --backup-method=mysqldump \
    --wait \
```

```
                --log \
   ```


   - where `--create` is the backup command which means to create a␣
↪mysqldump backup
   - where `--cluster-id=127` targets the cluster having the cluster ID of␣
↪127.
   - where `--nodes=192.168.40.54` specifies the target node of your MySQL␣
↪Replication cluster (MySQL/Percona Server/MariaDB) that the backup will␣
↪be executed and originate
   - where `--backup-method=mysqldump` will be the backup method to be␣
↪used.
   - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

### Steps to Create a Backup (Backup method: xtrabackup or mariabackup (full))

### ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MySQL Replication cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
```

2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
3. **Compression level** is set to `6 (System default)`
4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

6. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
7. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
8. **Desync node during backup** is set to **Off**. When enabled or **On**, this will desync the node that serves as the source of the backup. This is helpful especially when you do not want your cluster to experience large writesets and that let the flow control be manipulated and the node will catch up later once the backup is finished and `wsrep_desync=OFF` shall be set.

```
       9. **Backup locks** is set to **On** by default. When enabled,␣
↪Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR␣
↪BACKUP" where it is supported when making a backup. Backup Locks set to␣
↪**Off**, it sets --no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG␣
↪TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.

   10. **Lock DDL per table** is set to **Off** by default. When enabled,␣
↪it sets --lock-ddl-per-table. Enable this flag if you have backup errors␣
↪like `InnoDB: An optimized (without redo logging) DDLoperation has been␣
↪performed.` This option is only applied to xtrabackup 2.4.8 and later␣
↪version. Setting this flag may cause blocked updates to tables for highly␣
↪loaded servers under some circumstances. Please see␣
↪https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-
↪and https://jira.mariadb.org/browse/MDEV-14095 and␣
↪https://jira.mariadb.org/browse/MDEV-5336

   11. **Xtrabackup parallel copy threads** is set to 1. If you want more␣
↪parallel threads especially for streaming the backup during copy,␣
↪increase the threads > 1.

   12. **Network streaming throttle rate (MB/s)** is 0. If you want to␣
↪throttle the rate, specify the MB/s. Throttling the streaming for backup␣
↪will use the "pv" tool.

7. Click **Continue**.

8. Third step is **Local storage**.

   1. In **Storage location**, you can choose from `Store on controller`␣
↪or `Store on node`, where the node pertains to the source of the backup␣
↪data being taken and executed.
```

(continues on next page)

(continued from previous page)

```
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a␣
↪different path if you desire it somewhere else or on a volume where it␣
↪has large storage space or an NFS volume.

    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value␣
↪here is a sub-directory of the path set in **Storage directory** where `
↪%I` is an integer that is the backup ID assigned for every process of␣
↪backup jobs.

9. Click **Continue**.

10. Fourth step is **Cloud storage**. This step is only visible whenever␣
↪**Upload backup to cloud** is set. Otherwise, this step is skipped and␣
↪fourth step shall proceed to **Preview**, the last step.

    1. In **Credentials** field is the available credentials you've created␣
↪in **Settings → Cloud storage credentials**.

    2. Specify in the **Bucket** field for your target bucket in your S3␣
↪storage. For new bucket, make sure to hit the **+** (plus) sign button␣
↪and make sure to confirm and hit **OK** button.

    3. Specify the **Location path** which location you want the backup to␣
↪be stored under your **Bucket** assigned.

    4. **Retention period** is set to **On**. Specify the number of days␣
↪you want the retention of your backup. Otherwise, set to **Off** if you␣
↪do not need retention of your backup.

    5. **Delete after upload** is set to **Off**. For some instances, you␣
↪want your backup stored locally to be removed or wipe immediately to␣
↪reclaim the disk space since backup is already stored safely in the␣
↪cloud. Turn it **On** if that is the condition of your environment and␣
↪setup.

11. Click **Continue** to proceed the last step **Preview**.

12. The last step is **Preview**, this is the last step where it shows all␣
↪the values and choosen options for the ClusterControl to use for setting␣
↪up the backup.
```

ClusterControl CLI

- Creating a backup using xtrabackup (full)

```
s9s backup \
    --create \
    --cluster-id=99 \
    --nodes="192.168.40.60:3306" \
```

(continues on next page)

(continued from previous page)

```
      --backup-method=xtrabackupfull \
      --wait \
```

```
                --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=99` targets the cluster having the cluster ID of␣
↪99.
    - where `--nodes="192.168.40.60:3306"` specifies the target node of the␣
↪MySQL Replication cluster (MySQL/Percona) and its port that the backup␣
↪will be executed and originate
    - where `--backup-method=xtrabackupfull` will be the backup method to␣
↪be used.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.

- Creating a backup using mariabackup (full)

    ```bash
      s9s backup \
          --create \
          --cluster-id=126 \
          --nodes=192.168.40.52 \
          --backup-method=mariabackupfull \
          --wait \
                  --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=126` targets the cluster having the cluster ID of␣
↪126.
    - where `--nodes=192.168.40.54` specifies the target node of the␣
↪MariaDB Replication cluster that the backup will be executed and originate
    - where `--backup-method=mariabackupfull` will be the backup method to␣
↪be used for full backup.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

### Steps to Create a Backup (Backup method: xtrabackup or mariabackup (incremental))

### ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MySQL Replication cluster chosen.
```

2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
3. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.

5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

    5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    6. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
    7. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
    8. **Desync node during backup** is set to **Off**. When enabled or **On**, this will desync the node that serves as the source of the backup. This is helpful especially when you do not want your cluster to experience large writesets and that let the flow control be manipulated and the node will catch up later once the backup is finished and `wsrep_desync=OFF` shall be set.
    9. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed.` This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336
    10. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
    11. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.

8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using xtrabackup (incremental)

```
s9s backup \
    --create \
    --cluster-id=99 \
    --nodes="192.168.40.60:3306" \
    --backup-method=xtrabackupincr \
    --wait \
```

```
                --log \
  ```

  - where `--create` is the backup command which means to create a backup
  - where `--cluster-id=99` targets the cluster having the cluster ID of␣
↪99.
  - where `--nodes="192.168.40.60:3306"` specifies the target node of the␣
↪MySQL Replication cluster (MySQL/Percona) and its port that the backup␣
```

(continues on next page)
```

(continued from previous page)

```
↪will be executed and originate
    - where `--backup-method=xtrabackupincr` will be the backup method to␣
↪be used for incremental backup
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.

- Creating a backup using mariabackup (incremental)

    ```bash
      s9s backup \
          --create \
          --cluster-id=126 \
          --nodes=192.168.40.52 \
          --backup-method=mariabackupincr \
          --wait \
                  --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=126` targets the cluster having the cluster ID of␣
↪127.
    - where `--nodes=192.168.40.52` specifies the target node of the␣
↪MariaDB Replication cluster that the backup will be executed and originate
    - where `--backup-method=mariabackupincr` will be the backup method to␣
↪be used for incrementals
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

MySQL Galera

The following vendors and versions are supported for creating a backup:

- Percona XtraDB Cluster - 8.0.
- MariaDB Cluster - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backup method supported are: mysqldump, xtrabackup by Percona for Percona Server and Oracle MySQL database clusters, mariabackup by MariaDB for MariaDB database clusters.
- for mysqldump supports dump type for *Schema only*, *MySQL DB Only*, *Data Only*, *Schema And Data*, *Complete PITR-compatible*
- supports upload to the cloud of the following providers: *Amazon Web Services*, *Google Cloud*, *Microsoft Azure*, *S3 Compatible storage provider* (***upload backups to any AWS S3 compatible cloud storage provider by creating a custom S3 cloud storage credential****)
- compression is set to **On**
- retention days is set to `31`
- compression level is set to `6 (System Default)`

- *Use extended insert* is set to **On** when *mysqldump* is choosen as the backup method
- *Backup locks* is set to **On** when *xtrabackup* is choosen as the backup method
- *Xtrabackup parallel copy threads* is set to `1`
- storage location is set to `Store on controller`
- storage directory is set to `/home/vagrant/backups` as well as its sub `Backup subdirectory` is set to `BACKUP-%I`

Steps to Create a Backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MySQL Galera cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `mysqldump`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Cannot be set to **Off**.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.
    5. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
    6. **Use extended insert** is set to **On**. If you do not want the extended inserts to generate in your `mysqldump` file, then set this to *Off*.
    7. **One dump file per DB** is set to **Off**. If you want `mysqldump` to generate backup files per database, set this to **On**.
7. Click **Continue**.
8. Third step is **Local storage**.

1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.

9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using mysqldump as backup method

```
s9s backup \
    --create \
    --cluster-id=127 \
    --nodes=192.168.40.54 \
    --backup-method=mysqldump \
    --wait \
```

```
                --log \
   ```

   - where `--create` is the backup command which means to create a␣
↪mysqldump backup
   - where `--cluster-id=127` targets the MySQL Galera cluster (MariaDB or␣
↪Percona Server) having the cluster ID of 127.
   - where `--nodes=192.168.40.54` specifies the target node that the␣
↪backup will be executed and originate
```

(continues on next page)
```

(continued from previous page)

```
    - where `--backup-method=mysqldump` will be the backup method to be␣
↪used.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

Steps to Create a Backup (Backup method: xtrabackup or mariabackup (full))

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MySQL Galera cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

    5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    6. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
    7. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
    8. **Desync node during backup** is set to **Off**. When enabled or **On**, this will desync the node that serves as the source of the backup. This is helpful especially when you do not want your cluster to experience large writesets and that let the flow control be manipulated and the node will catch up later once the backup is finished and `wsrep_desync=OFF` shall be set.

9. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
10. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed.` This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336
11. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
12. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using xtrabackup (full)

```
  s9s backup \
      --create \
      --cluster-id=124 \
      --nodes="192.168.40.67:3306" \
      --backup-method=xtrabackupfull \
      --wait \
```

```
                --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=124` targets the cluster having the cluster ID of␣
↪127.
    - where `--nodes="192.168.40.67:3306"` specifies the target node, which␣
↪is your Percona XtraDB Cluster node, and its port. This will be the ndoe␣
↪where backup shall be executed and originate
    - where `--backup-method=xtrabackupfull` will be the backup method to␣
↪be used for full backup.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.

- Creating a backup using mariabackup (full)

    ```bash
      s9s backup \
          --create \
          --cluster-id=127 \
          --nodes=192.168.40.54 \
          --backup-method=mariabackupfull \
          --wait \
                  --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=127` targets the cluster having the cluster ID of␣
↪127.
    - where `--nodes=192.168.40.54` specifies the target node, which is␣
↪your MariaDB Galera cluster node, that the backup will be executed and␣
↪originate
    - where `--backup-method=mariabackupfull` will be the backup method to␣
↪be used for full backup.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```
```

Steps to Create a Backup (Backup method: xtrabackup or mariabackup (incremental))

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MySQL Galera cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

    5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    6. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
    7. **Desync node during backup** is set to **Off**. When enabled or **On**, this will desync the node that serves as the source of the backup. This is helpful especially when you do not want your cluster to experience large writesets and that let the flow control be manipulated and the node will catch up later once the backup is finished and `wsrep_desync=OFF` shall be set.
    8. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
    9. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed`. This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded

servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336

10. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
11. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using xtrabackup (incremental)

```
s9s backup \
    --create \
    --cluster-id=124 \
    --nodes="192.168.40.67:3306" \
```

(continues on next page)

(continued from previous page)

```
      --backup-method=xtrabackupincr \
      --wait \
```

```
                --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=124` targets the cluster having the cluster ID of␣
↪124.
    - where `--nodes="192.168.40.67:3306"` specifies the target node, which␣
↪is your Percona XtraDB cluster node, and its port. This will be the ndoe␣
↪where backup shall be executed and originate.
    - where `--backup-method=xtrabackupincr` will be the backup method to␣
↪be used for incrementals.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.

- Creating a backup using mariabackup (incremental)

    ```bash
      s9s backup \
          --create \
          --cluster-id=127 \
          --nodes="192.168.40.54:3306" \
          --backup-method=mariabackupincr \
          --wait \
                  --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=127` targets the cluster having the cluster ID of␣
↪127.
    - where `--nodes="192.168.40.54:3306"` specifies the target node, which␣
↪is your MariaDB Galera cluster node, and its port. This will be the ndoe␣
↪where backup shall be executed and originate
    - where `--backup-method=mariabackupincr` will be the backup method to␣
↪be used for incrementals.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

### PostgreSQL/TimescaleDB

The following vendors and versions are supported for creating a backup:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, and 16.
- PostgreSQL from EDB repository - 12, 13, 14, 15, and 16 (requires a valid EDB repository token).
```

Default Configuration

- Backup method supported are: `pgdumpall` for logical backup, and for binary or physical backups `pg_basebackup`, `pgbackrestfull`, `pgbackrestdiff`, `pgbackrestincr`
- with `pgdumpall`, PITR is not possible with the selected backup method.
- `pg_basebackup` PITR is disabled when `pgbackuprest` is configured and setup to manage WAL
- upload to cloud is not supported if `pgbasebackup` is selected
- supports upload to the cloud of the following providers: *Amazon Web Services*, *Google Cloud*, *Microsoft Azure*, *S3 Compatible storage provider* (***upload backups to any AWS S3 compatible cloud storage provider by creating a custom S3 cloud storage credential***)
- compression is set to **On**
- retention days is set to `31`
- compression level is set to `6 (System Default)`
- storage location is set to `Store on controller`
- storage directory is set to `/home/vagrant/backups` as well as its sub `Backup subdirectory` is set to `BACKUP-%I`

Steps to Create a Backup (Backup method: pgdumpall)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the PostgreSQL/TimescaleDB cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `pgdumpall`. Other option to choose from are physical backups: `pg_basebackup`, `pgbackrestfull` (full backup), `pgbackrestdiff` (differential backup), `pgbackrestincr` (incremental backup)
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

4 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

5. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using pgdumpall as backup method

```
s9s backup \
    --create \
    --cluster-id=125 \
    --nodes="192.168.40.63:5432" \
    --backup-method=pgdumpall \
    --wait \
```

```
                --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=125` targets the PostgreSQL Cluster with␣
↪cluster-id of 125
    - where `--nodes="192.168.40.63:5432"` specifies the target node and␣
↪its port that the backup will be executed and originate
    - where `--backup-method=pgdumpall` will be the backup method to be␣
↪used.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

### Steps to Create a Backup (Backup method: pg_basebackup)

#### ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the PostgreSQL/TimescaleDB cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `pg_basebackup`. Other option to choose from are logical backup `pgdumpall`, and physical backups `pg_basebackup`, `pgbackrestfull` (full backup), `pgbackrestdiff` (differential backup), `pgbackrestincr` (incremental backup)
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

    4 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.
7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
```

2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.

9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using pg_basebackup as backup method

```
s9s backup \
    --create \
    --cluster-id=125 \
    --nodes="192.168.40.63:5432" \
    --backup-method=pg_basebackup \
    --wait \
```

```
                --log \
   ```


   - where `--create` is the backup command which means to create a backup
   - where `--cluster-id=125` targets the PostgreSQL Cluster with␣
↪cluster-id of 125
   - where `--nodes="192.168.40.63:5432"` specifies the target node and␣
↪its port that the backup will be executed and originate
   - where `--backup-method=pg_basebackup` will be the backup method to be␣
↪used.
   - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```
```

Steps to Create a Backup (Backup method: pgbackrestfull, pgbackrestdiff, pgbackrestincr)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the PostgreSQL/TimescaleDB cluster chosen.
    2. **Backup method**. Choose `pgbackrestfull` if you want full backup, `pgbackrestdiff` for differential, and `pgbackrestincr` for incremental. Other option to choose from are logical backup `pgdumpall`, and physical backups `pg_basebackup`.
        1. If `pgbackrest` is not installed, it will show and display a panel to install the tool. Click **Continue** button to install the tool
            1. **Configuration method** shows the following options for your target installation.
                1. **Primary**. Install on the current master but not on slaves. The backup repository (storing backup data) will be configured to be on the master. There will be no ssh configuration for PgBackRest.
                2. **All DB nodes**. Install on all database hosts. The backup repository will be created on the current master. Backup will be made by using standby node. PgBackRest will use ssh for communicating between hosts.
                3. **All DB nodes and dedicated repository host**. All DB nodes and dedicated repository host
                    1. Specify your **Dedicated repository host** using IP or valid hostname or FQDN of your target dedicated repository/backup host.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.

    2 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    3. **Compression level** is set to `6 (System default)`
7. Click **Continue** to proceed the last step **Preview**.
8. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using pgbackrest full backup method

```
s9s backup \
    --create \
    --cluster-id=125 \
```

(continues on next page)

(continued from previous page)

```
      --nodes="192.168.40.63:5432" \
      --backup-method=pgbackrestfull \
      --wait \
```

```
                --log \
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=125` targets the PostgreSQL Cluster with␣
↪cluster-id of 125
    - where `--nodes="192.168.40.63:5432"` specifies the target node and␣
↪its port that the backup will be executed and originate
    - where `--backup-method=pg_basebackup` will be the backup method to be␣
↪used.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.

- For differential and incrementals, see the following below:

    ```bash
      s9s backup \
          --create \
          --cluster-id=125 \
          --nodes="192.168.40.63:5432" \
          --backup-method=pgbackrestdiff \
          --wait \
                  --log \
    ```

    ```bash
      s9s backup \
          --create \
          --cluster-id=125 \
          --nodes="192.168.40.63:5432" \
          --backup-method=pgbackrestincr\
          --wait \
                  --log \
    ```
```

### MongoDB Replica Set

The following vendors and versions are supported for creating a backup:

- MongoDB Enterprise - 4.4, 5.0, 6.0 and 7.0
- MongoDB Community - 4.4, 5.0, 6.0 and 7.0
- Percona Server for MongoDB - 4.4, 5.0, 6.0 and 7.0
```

Default Configuration

- supports only `mongodump` as backup method
- retention days is enabled and defaults to 31 days
- storage location is set to `Store on controller`
- storage directory is set to `/home/vagrant/backups` as well as its sub `Backup subdirectory` is set to `BACKUP-%I`

Steps to Create a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MongoDB Replica Set cluster chosen.
    2. **Backup method** cannot be changed but defaults to `mongodump` as the sole backup method supported.
    3. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Retention** is set to **On** and defaults to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    2. **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.
7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from `Store on controller` or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.

1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.

11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup using mongodump backup method

```
  s9s backup \
      --create \
      --cluster-id=79 \
      --nodes="192.168.40.160:27017" \
      --backup-method=mongodump \
      --wait \
```

```
                --log \
    ```


    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=79` targets the PostgreSQL Cluster with␣
↪cluster-id of 79
    - where `--nodes="192.168.40.160:27017"` specifies the target node and␣
↪its port that the backup will be executed and originate
    - where `--backup-method=mongodump` will be the backup method to be␣
↪used.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

### MongoDB Sharded Cluster

The following vendors and versions are supported for creating a backup:

- MongoDB - 4.4, 5.0, 6.0 and 7.0
- MongoDB Enterprise - 4.4., 5.0, 6.0 and 7.0
- Percona - 4.4, 5.0, 6.0 and 7.0
```

Default Configuration

- supports only `percona-backup-mongodb` or PBM as backup method. `PBM` is Percona Backup for MongoDB is available for MongoDB 4.0 or later versions.
- retention days is enabled and defaults to 31 days

Steps to Create a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the MongoDB Sharded cluster chosen.
    2. **Backup method**. Defaults to `percona-backup-mongodb` and is the sole backup method. Option to choose is disabled. By default, it displays a **Continue** button allowing you to install the backup tool package.
        1. Click **Continue** button to proceed on installing the `percona-backup-mongodb` backup tool
            1. Specify the **Backup directory**. Make sure that the backup directory path is existent. Once ready, hit **Install** button. Hit **Cancel** if you are not ready to install yet. If installing the tool, make sure that the job has to be done successfully.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Retention** is set to **On** and defaults to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
7. Click **Continue** to proceed the last step **Preview**.
8. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Install percona-backup-mongodb agents first,

```
s9s cluster \
   --add-node \
   --nodes="pbmagent://*?backup_dir=/opt/pbm/backups/" \
   --cluster-id=28 \
   --wait \
   --log
```

- where `--add-node` is the command to add a node
- where `--nodes="pbmagent://*?backup_dir=/opt/pbm/backups/"` targets all the nodes in the cluster that shall participate on the replication and path for backup directory is `/opt/pbm/backups/`. Change it to your desired path.

- where `--cluster-id=28` the cluster id of your MongoDB Sharded Cluser
- where the `--wait` and `--log` as the name suggest to print the job logs while waiting.

- Creating a backup using percona-backup-mongodb backup method

```
s9s backup \
    --create \
    --cluster-id=28 \
    --nodes="192.168.40.170:27019" \
    --backup-method=percona-backup-mongodb \
    --backup-retention=3 \
    --wait \
```

```
                --log
    ```

    - where `--create` is the backup command which means to create a backup
    - where `--cluster-id=28` targets the PostgreSQL Cluster with␣
↪cluster-id of 79
    - where `--nodes="192.168.40.170:27019"` specifies the target node and␣
↪the Replica Set Config port that the backup will be executed and originate
    - where `--backup-method=percona-backup-mongodb` will be the backup␣
↪method to be used (i.e. percona-backup-mongodb)
    - where `--backup-retention=3` specifies the days of the backup shall␣
↪be retained before it can be wiped out or deleted.
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

### Redis Sentinel Cluster

The following vendors and versions are supported for creating a backup:

- Redis Sentinel - 6 and 7

### Default Configuration

- uses `rdb` and `aof` as backup method
- backup host points to the primary node as default selection
- backup runs on the cluster and saves the backup copy in the ClusterControl controller host. You will determine the directory of the backup location in the backup logs once its requested to run
- compression is set to `true` or **On**
- uses `socat` command to transfer the file using port `9999` as the default port for transfer or copy. The port `9999` can be changed if you need other port number by going to the **Backup Settings**.

### Steps to Create a Backup

### ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
```

3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the Redis/Valkey cluster chosen.
    2. **Backup host** is set to the primary node. You can opt to choose from primary to replica nodes.
    3. **Backup method** is set to `rdb, aof` as it's the sole backup method and is disabled for selection.
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

```
       4. **Use PIGZ for parallel gzip** field is set to **Off** by␣
↪default. Setting to **On** will use PIGZ and this shall speed up your␣
↪compression especially if your data to be back up is huge.

   5. **Enable encryption** is set to **Off**. When set to **On**,␣
↪ClusterControl will automate the generation of encryption key. This key␣
↪shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

7. Click **Continue**.

8. Third step is **Local storage**.

   1. In **Storage location**, you can choose from `Store on controller`␣
↪or `Store on node`, where the node pertains to the source of the backup␣
↪data being taken and executed.

   2. **Storage directory** is set to `/home/vagrant/backups`. Set to a␣
↪different path if you desire it somewhere else or on a volume where it␣
↪has large storage space or an NFS volume.

   3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value␣
↪here is a sub-directory of the path set in **Storage directory** where `
↪%I` is an integer that is the backup ID assigned for every process of␣
↪backup jobs.

9. Click **Continue**.

10. Fourth step is **Cloud storage**. This step is only visible whenever␣
↪**Upload backup to cloud** is set. Otherwise, this step is skipped and␣
↪fourth step shall proceed to **Preview**, the last step.

   1. In **Credentials** field is the available credentials you've created␣
```

(continues on next page)

(continued from previous page)

```
↪in **Settings → Cloud storage credentials**.

   2. Specify in the **Bucket** field for your target bucket in your S3␣
↪storage. For new bucket, make sure to hit the **+** (plus) sign button␣
↪and make sure to confirm and hit **OK** button.

   3. Specify the **Location path** which location you want the backup to␣
↪be stored under your **Bucket** assigned.

   4. **Retention period** is set to **On**. Specify the number of days␣
↪you want the retention of your backup. Otherwise, set to **Off** if you␣
↪do not need retention of your backup.

   5. **Delete after upload** is set to **Off**. For some instances, you␣
↪want your backup stored locally to be removed or wipe immediately to␣
↪reclaim the disk space since backup is already stored safely in the␣
↪cloud. Turn it **On** if that is the condition of your environment and␣
↪setup.

11. Click **Continue** to proceed the last step **Preview**.

12. The last step is **Preview**, this is the last step where it shows all␣
↪the values and choosen options for the ClusterControl to use for setting␣
↪up the backup.
```

ClusterControl CLI

- Creating a backup for Redis Sentinel Cluster. Without specifying a backup method, cmon shall be able to determine the origin and the kind of the cluster.

```
s9s backup \
    --create \
    --cluster-id=84 \
    --nodes="192.168.40.50:6379" \
    --backup-retention=3 \
    --wait \
    --log
```

  - where `--create` is the backup command which means to create a backup
  - where `--cluster-id=84` targets the PostgreSQL Cluster with cluster-id of 84
  - where `--nodes="192.168.40.50:6379"` specifies the target node and the Redis Sentinel server port that the backup will be executed and originate
  - where `--backup-retention=3` specifies the days of the backup shall be retained before it can be wiped out or deleted.
  - where the `--wait` and `--log` as the name suggest to print the job logs while waiting.

Redis/Valkey Cluster

The following vendors and versions are supported for creating a backup:

- Redis Cluster - 6 and 7
- Valkey Cluster - 7 and 8 (only RHEL 8/9 are supported)

Default Configuration

- uses `rdb` and `aof` as backup method
- backup runs on the cluster and saves the backup copy in the ClusterControl controller host. You will determine the directory of the backup location in the backup logs once its requested to run
- compression is set to `true` or **On**
- uses `socat` command to transfer the file using port `9999` as the default port for transfer or copy. The port `9999` can be changed if you need other port number by going to the **Backup Settings**.

Steps to Create a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the Redis/Valkey cluster chosen.
    2. **Backup method** is set to `rdb, aof` as it's the sole backup method and is disabled for selection.
    3. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

```
       4. **Use PIGZ for parallel gzip** field is set to **Off** by␣
↪default. Setting to **On** will use PIGZ and this shall speed up your␣
↪compression especially if your data to be back up is huge.

   5. **Enable encryption** is set to **Off**. When set to **On**,␣
↪ClusterControl will automate the generation of encryption key. This key␣
↪shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

7. Click **Continue**.
```

(continues on next page)

(continued from previous page)

```
8. Third step is **Local storage**.

    1. In **Storage location**, you can choose from `Store on controller`␣
↪or `Store on node`, where the node pertains to the source of the backup␣
↪data being taken and executed.

    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a␣
↪different path if you desire it somewhere else or on a volume where it␣
↪has large storage space or an NFS volume.

    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value␣
↪here is a sub-directory of the path set in **Storage directory** where `
↪%I` is an integer that is the backup ID assigned for every process of␣
↪backup jobs.

9. Click **Continue**.

10. Fourth step is **Cloud storage**. This step is only visible whenever␣
↪**Upload backup to cloud** is set. Otherwise, this step is skipped and␣
↪fourth step shall proceed to **Preview**, the last step.

    1. In **Credentials** field is the available credentials you've created␣
↪in **Settings → Cloud storage credentials**.

    2. Specify in the **Bucket** field for your target bucket in your S3␣
↪storage. For new bucket, make sure to hit the **+** (plus) sign button␣
↪and make sure to confirm and hit **OK** button.

    3. Specify the **Location path** which location you want the backup to␣
↪be stored under your **Bucket** assigned.

    4. **Retention period** is set to **On**. Specify the number of days␣
↪you want the retention of your backup. Otherwise, set to **Off** if you␣
↪do not need retention of your backup.

    5. **Delete after upload** is set to **Off**. For some instances, you␣
↪want your backup stored locally to be removed or wipe immediately to␣
↪reclaim the disk space since backup is already stored safely in the␣
↪cloud. Turn it **On** if that is the condition of your environment and␣
↪setup.

11. Click **Continue** to proceed the last step **Preview**.

12. The last step is **Preview**, this is the last step where it shows all␣
↪the values and choosen options for the ClusterControl to use for setting␣
↪up the backup.
```

ClusterControl CLI

- Creating a backup for Redis/Valkey Cluster. Without specifying a backup method, cmon shall be able to determine the origin and the kind of the cluster.

```
s9s backup \
   --create \
   --cluster-id=80 \
   --nodes="192.168.40.164:6379" \
   --backup-directory=/opt/backup/redis/ \
   --storage-host="192.168.40.164" \
   --backup-retention=3 \
   --wait --log
```

  - where `--create` is the backup command which means to create a backup
  - where `--cluster-id=80` targets the PostgreSQL Cluster with cluster-id of 80
  - where `--nodes="192.168.40.164:6379"` specifies the target node and the Redis/Valke server port that the backup will be executed and originate
  - where `--backup-directory=/opt/backup/redis/` specifies the target backup directory that the backup shall be stored
  - where `--storage-host="192.168.40.164"` specifies the target storage host that the backup shall be stored.
  - where `--backup-retention=3` specifies the days of the backup shall be retained before it can be wiped out or deleted.
  - where the `--wait` and `--log` as the name suggest to print the job logs while waiting.

SQL Server

The following vendors and versions are supported for creating a backup:

- Microsoft SQL Server for Linux - 2019 and 2022

ClusterControl does not use external tool to backup. It just relies on the internal backup engine from Microsoft SQL Server itself.

Default Configuration

- uses internal backup engine
- can create backup using methods `Full`, `Differential`, and `Transaction Log`
- runs backup on the primary node regardless of backup method choosen
- stores the backup directory in /var/lib/backups
- stores backup data only in the primary node
- cannot run backup if primary is down as it cannot find a candidate server
- if backup method choosen not equal to `Full`, and directory is does not find any previous full backup, it will upgrade its method to `Full`, making it as the first full backup for that job request

Steps to Create a Backup (Full, Differential, Transaction Log)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the Microsoft SQL Server cluster chosen.
    2. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    3. **Backup method**. Choose `Full` if you want full backup. Choose `Differential` if you want a differential backup. Lastly, choose `Transaction Log` if you want transaction log backup.
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Include system databases** is set to **On** by default. Set to **Off** if you do not want.
7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location** field is disabled as backup storage location defaults to the primary node.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.

4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.

11. Click **Continue** to proceed the last step **Preview**.
12. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup for SQL Server using full backup

```
s9s backup \
   --create \
   --cluster-id=42 \
   --nodes="pupnode195:1433" \
   --backup-method=mssqlfull \
   --backup-retention=3 \
   --wait --log
```

  - where `--create` is the backup command which means to create a backup
  - where `--cluster-id=80` targets the PostgreSQL Cluster with cluster-id of 80
  - where `--nodes="pupnode195:1433"` specifies the target host and its SQL Server port for which the backup will originate and will execute
  - where `--backup-method=mssqlfull` specifies the backup method to be used which is a full backup copy.
  - where `--backup-retention=3` specifies the days of the backup shall be retained before it can be wiped out or deleted.
  - where the `--wait` and `--log` as the name suggest to print the job logs while waiting.

- Creating a backup for SQL Server using differential backup

```
s9s backup \
   --create \
   --cluster-id=42 \
   --nodes="pupnode195:1433" \
   --backup-method=mssqldiff \
   --backup-retention=3 \
   --wait --log
```

- Creating a backup for SQL Server using transaction log backup

```
s9s backup \
   --create \
   --cluster-id=42 \
   --nodes="pupnode195:1433" \
   --backup-method=mssqllog \
```

(continues on next page)

(continued from previous page)

```
    --backup-retention=3 \
    --wait --log
```

Elasticsearch

The following vendors and versions are supported for creating a backup:

- Elasticsearch by Elastic versions 7.17, 8.3, 8.15

ClusterControl does not use external tool to backup. It just relies on `elasticsearch-snapshot` tool to backup the data.

Default Configuration

- uses Elasticsearch built-in backup engine `elasticsearch-snapshot`
- repository defaults to the repository assigned or created during deployment or importing of the cluster. This can also be found and modified using parameter `elastic_snapshot_repository` on the ClusterControl controller host through config file `/etc/cmon.d/cmon_<CLUSTER_ID>.cnf`, where *&ltCLUSTER_ID>* is the cluster id of the Elasticsearch cluster.
- retention days is set to `31`
- backup data shall be stored on the shared location set in the config file `/etc/cmon.d/cmon_<CLUSTER_ID>.cnf` through parameter `elastic_shared_filesystem`.

Steps to Create a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Backup on Demand**
4. First step is **Configuration**.
    1. **Cluster** is set to the name of the Elasticsearch cluster chosen.
    2. **Repository**. By default, points to your repository assigned in your `/etc/cmon.d/cmon_<CLUSTER_ID>.cnf` config file through parameter `elastic_snapshot_repository`. If you have other repository created, this will show up in the drop-down list.
    3. **Backup method**. This is constant and sets to `elastlicsearch-snapshot`.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
7. Click **Continue** to proceed the last step **Preview**.
8. The last step is **Preview**, this is the last step where it shows all the values and choosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

- Creating a backup for Elasticsearch cluster

```
s9s backup \
   --create \
   --cluster-id=8 \
   --snapshot-repo-type=fs-nfs \
   --snapshot-repository=s9s-es-repo \
   --backup-retention=4 \
   --wait --log
```

  - where `--create` is the backup command which means to create a backup
  - where `--cluster-id=8` targets the Elasticsearch cluster with cluster-id of 8
  - where `--snapshot-repo-type=fs-nfs"` specifies the snapshot repo type is based on `fs-nfs`. Other option you can take are `fs` and `s3`
  - where `-snapshot-repository=s9s-es-repo` specifies the snapshot repository to be used is the `s9s-es-repo`. This value is initiated during the creation of cluster. This can also be found in `/etc/cmon.d/cmon_<CID>.cnf` where `CID` is your cluster id in the controller host.
  - where `--backup-retention=4` specifies the days of the backup shall be retained before it can be wiped out or deleted.
  - where the `--wait` and `--log` as the name suggest to print the job logs while waiting.

2.60 Schedule Backup

The functionalities of backup scheduling are almost identical to *Create a Backup*. However, there are some additional functionalities that you might need guidance on how to use it. Below are the list of recipe on achieving a scheduled backup:

- Define a schedule to periodically create backups
- Store backups on premise or in the cloud
- Compress and encrypt backups for secure storage
- Automate backup verification

This guide provides step-by-step instructions on how to schedule a backup for your database using ClusterControl.

2.60.1 Support Matrix

All database clusters that are supported by ClusterControl allows you to schedule a backup. For a complete list of our supported database clusters, see below:

Database	Vendor	Topology
MySQL	Percona, Percona Pro, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.60.2 Prerequisites

When scheduling a backup, certain requirements should be met in order to schedule a backup. See below:

- Only backup methods that ClusterControl supports can be use to schedule a backup.
- Backup created and scheduled in ClusterControl can only be stored locally either on the ClusterControl controller or to the registered database nodes available on that cluster. If you have external backup host but not registered in ClusterControl, you can use NFS and share the volume where your backup shall be stored.
- You have sufficient storage space for the backup.
- You have appropriate user permissions to manage backups.

2.60.3 Scheduling the backup

The steps on scheduling a backup can differ depending on the type of database cluster. Make sure you follow the instructions and guide based on the following sections per database cluster you are going to schedule the backup.

MySQL Replication

The following vendors and versions are supported for scheduling a backup:

- Oracle MySQL - 8.0.
- Percona Server for MySQL - 8.0.
- Percona Server for MySQL Pro - 8.0.
- MariaDB Server - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backup method supported are: mysqldump, xtrabackup by Percona for Percona Server and Oracle MySQL database clusters, mariabackup by MariaDB for MariaDB database clusters.
- For mysqldump supports dump type for *Schema only*, *MySQL DB Only*, *Data Only*, *Schema And Data*, *Complete PITR-compatible*

- Supports upload to the cloud of the following providers: *Amazon Web Services*, *Google Cloud*, *Microsoft Azure*, *S3 Compatible storage provider* (***upload backups to any AWS S3 compatible cloud storage provider by creating a custom S3 cloud storage credential***)
- Compression is set to **On**
- Retention days is set to 31
- Compression level is set to `6 (System Default)`
- **Use extended insert** is set to **On** when *mysqldump* is chosen as the backup method
- *Backup locks* is set to **On** when *xtrabackup* is chosen as the backup method
- *Xtrabackup parallel copy threads* is set to 1
- Storage location is set to **Store on controller**
- Storage directory is set to `/home/vagrant/backups` as well as its sub **Backup subdirectory** is set to `BACKUP-%I`.

Steps to schedule a backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MySQL Replication cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose `mysqldump`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Cannot be set to **Off**.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

5. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
6. **Use extended insert** is set to **On**. If you do not want the extended inserts to generate in your `mysqldump` file, then set this to *Off*.
7. **One dump file per DB** is set to **Off**. If you want `mysqldump` to generate backup files per database, set this to **On**.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.

14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

Steps to schedule a backup (Backup method: xtrabackup or mariabackup (full))

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MySQL Replication cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

    5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    6. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
    7. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.

8. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
9. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed.` This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336
10. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
11. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.

1. If **Simple** is selected (default view), it contains the following elements:
    1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.

13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

Steps to schedule a backup (Backup method: xtrabackup or mariabackup (incremental))

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MySQL Replication cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

6. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
7. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
8. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed`. This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336
9. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
10. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.

12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

MySQL Galera

The following vendors and versions are supported for scheduling a backup:

- Percona XtraDB Cluster - 8.0.
- MariaDB Cluster - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backup method supported are: mysqldump, xtrabackup by Percona for Percona Server and Oracle MySQL database clusters, mariabackup by MariaDB for MariaDB database clusters.
- For mysqldump supports dump type for *Schema only*, *MySQL DB Only*, *Data Only*, *Schema And Data*, *Complete PITR-compatible*.
- Supports upload to the cloud of the following providers: *Amazon Web Services*, *Google Cloud*, *Microsoft Azure*, *S3 Compatible storage provider* (***upload backups to any AWS S3 compatible cloud storage provider by creating a custom S3 cloud storage credential***)
- Compression is set to **On**.
- Retention days is set to 31.
- Compression level is set to `6 (System Default)`.
- *Use extended insert* is set to **On** when *mysqldump* is chosen as the backup method.
- *Backup locks* is set to **On** when *xtrabackup* is chosen as the backup method.
- *Xtrabackup parallel copy threads* is set to 1.
- Storage location is set to **Store on controller**.
- Storage directory is set to `/home/vagrant/backups` as well as its sub **Backup subdirectory** is set to `BACKUP-%I`.

Steps to schedule a backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MySQL Galera cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose `mysqldump`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Cannot be set to **Off**.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.
    5. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
    6. **Use extended insert** is set to **On**. If you do not want the extended inserts to generate in your `mysqldump` file, then set this to *Off*.
    7. **One dump file per DB** is set to **Off**. If you want `mysqldump` to generate backup files per database, set this to **On**.
7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.

3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.

9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

Steps to schedule a backup (Backup method: xtrabackup or mariabackup (full))

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.

1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
2. **Cluster** is set to the name of the MySQL Galera cluster chosen.
3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
4. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.

5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    6. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
    7. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
    8. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
    9. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed`. This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336
    10. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
    11. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

Steps to schedule a backup (Backup method: xtrabackup or mariabackup (incremental))

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MySQL Galera cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose `xtrabackup (full)`, or `mariabackup (full)`. Other option to choose from the backup method available are logical backup [`mysqldump`] and physical backup [`xtrabackup` (full), `xtrabackup` (incremental), `mariabackup` (full), `mariabackup` (incremental)]
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`
    4. **Use PIGZ for parallel gzip** field is set to **Off** by default. Setting to **On** will use PIGZ and this shall speed up your compression especially if your data to be back up is huge.

    5 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    6. **Use ZSTD compression** is set to **Off**. When enabled, will use Zstandard compression. For MySQL/Percona Server versions <8.0.34, it uses QPress.
    7. **Backup locks** is set to **On** by default. When enabled, Backup locks means that `xtrabackup` behavior uses "LOCK TABLES FOR BACKUP" where it is supported when making a backup. Backup Locks set to **Off**, it sets –no-backup-locks which use "FLUSH NO_WRITE_TO_BINLOG TABLES" and "FLUSH TABLES WITH READ LOCK" when making a backup.
    8. **Lock DDL per table** is set to **Off** by default. When enabled, it sets –lock-ddl-per-table. Enable this flag if you have backup errors like `InnoDB: An optimized (without redo logging) DDLoperation has been performed.` This option is only applied to xtrabackup 2.4.8 and later version. Setting this flag may cause blocked updates to tables for highly loaded servers under some circumstances. Please see https://www.percona.com/blog/2017/08/08/avoiding-the-an-optimized-without-redo-logging-ddloperation-has-been-performed-error-with-percona-xtra

and https://jira.mariadb.org/browse/MDEV-14095 and https://jira.mariadb.org/browse/MDEV-5336

9. **Xtrabackup parallel copy threads** is set to 1. If you want more parallel threads especially for streaming the backup during copy, increase the threads > 1.
10. **Network streaming throttle rate (MB/s)** is 0. If you want to throttle the rate, specify the MB/s. Throttling the streaming for backup will use the "pv" tool.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.

14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

PostgreSQL/TimescaleDB

The following vendors and versions are supported for scheduling a backup:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, and 16.
- PostgreSQL from EDB repository - 12, 13, 14, 15, and 16 (requires a valid EDB repository token).

Default Configuration

- Backup method supported are: `pgdumpall` for logical backup, and for binary or physical backups `pg_basebackup`, `pgbackrestfull`, `pgbackrestdiff`, `pgbackrestincr`
- with `pgdumpall`, PITR is not possible with the selected backup method.
- `pg_basebackup` PITR is disabled when `pgbackuprest` is configured and setup to manage WAL
- upload to cloud is not supported if `pgbasebackup` is selected
- supports upload to the cloud of the following providers: *Amazon Web Services*, *Google Cloud*, *Microsoft Azure*, *S3 Compatible storage provider* (***upload backups to any AWS S3 compatible cloud storage provider by creating a custom S3 cloud storage credential***)
- Compression is set to **On**
- Retention days is set to 31
- Compression level is set to `6 (System Default)`
- storage location is set to **Store on controller**
- storage directory is set to `/home/vagrant/backups` as well as its sub **Backup subdirectory** is set to `BACKUP-%I`

Steps to schedule a backup (Backup method: pgdumpall)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the PostgreSQL/TimescaleDB cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.

4. **Backup method**. Choose `pgdumpall`. Other option to choose from are physical backups: `pg_basebackup`, `pgbackrestfull` (full backup), `pgbackrestdiff` (differential backup), `pgbackrestincr` (incremental backup)
5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.

5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

    4 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    5. **Enable partial backup**. If you want to have a partial backup, set it to **On**. This will show a field where you can fill-in the databases you want to backup only or select `All Databases`.
7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.

12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

Steps to schedule a backup (Backup method: pg_basebackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the PostgreSQL/TimescaleDB cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose `pg_basebackup`. Other option to choose from are logical backup `pgdumpall`, and physical backups `pg_basebackup`, `pgbackrestfull` (full backup), `pgbackrestdiff` (differential backup), `pgbackrestincr` (incremental backup)
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

4 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.
    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.
11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

Steps to schedule a backup (Backup method: pgbackrestfull, pgbackrestdiff, pgbackrestincr)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the PostgreSQL/TimescaleDB cluster chosen.
    3. **Backup method**. Choose `pgbackrestfull` if you want full backup, `pgbackrestdiff` for differential, and `pgbackrestincr` for incremental. Other option to choose from are logical backup `pgdumpall`, and physical backups `pg_basebackup`.
        1. If `pgbackrest` is not installed, it will show and display a panel to install the tool. Click **Continue** button to install the tool
            1. **Configuration method** shows the following options for your target installation.
                1. **Primary**. Install on the current master but not on slaves. The backup repository (storing backup data) will be configured to be on the master. There will be no ssh configuration for PgBackRest.
                2. **All DB nodes**. Install on all database hosts. The backup repository will be created on the current master. Backup will be made by using standby node. PgBackRest will use ssh for communicating between hosts.
                3. **All DB nodes and dedicated repository host**. All DB nodes and dedicated repository host
                    1. Specify your **Dedicated repository host** using IP or valid hostname or FQDN of your target dedicated repository/backup host.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.

    2 **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

    3. **Compression level** is set to `6 (System default)`
7. Click **Continue** to proceed the last step **Preview**.
8. Third step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:

1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.

2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.

9. Click **Continue** to proceed the last step **Preview**.

10. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

MongoDB Replica Set

The following vendors and versions are supported for scheduling a backup:

- MongoDB Enterprise - 4.4, 5.0, 6.0 and 7.0
- MongoDB Community - 4.4, 5.0, 6.0 and 7.0
- Percona Server for MongoDB - 4.4, 5.0, 6.0 and 7.0

Default Configuration

- Supports only `mongodump` as backup method.
- Retention days is enabled and defaults to 31 days.
- Storage location is set to **Store on controller**.
- Storage directory is set to `/home/vagrant/backups` as well as its sub **Backup subdirectory** is set to `BACKUP-%I`.

Steps to schedule a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MongoDB Replica Set cluster chosen.
    3. **Backup method** cannot be changed but defaults to `mongodump` as the sole backup method supported.

4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.

5. Click **Continue**.

6. Second step is **Advanced settings**.

    1. **Retention** is set to **On** and defaults to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.

    2. **Enable encryption** is set to **Off**. When set to **On**, ClusterControl will automate the generation of encryption key. This key shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

7. Click **Continue**.

8. Third step is **Local storage**.

    1. In **Storage location**, you can choose from **Store on controller** or `Store on node`, where the node pertains to the source of the backup data being taken and executed.

    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.

    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.

9. Click **Continue**.

10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.

    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.

    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.

    3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.

    4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.

    5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.

11. Click **Continue** to proceed the last step **Preview**.

12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.

    1. If **Simple** is selected (default view), it contains the following elements:

        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.

2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.

13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

MongoDB Sharded Cluster

The following vendors and versions are supported for scheduling a backup:

- MongoDB - 4.4, 5.0, 6.0 and 7.0
- MongoDB Enterprise - 4.4., 5.0, 6.0 and 7.0
- Percona - 4.4, 5.0, 6.0 and 7.0

Default Configuration

- Supports only `percona-backup-mongodb` or PBM as backup method. PBM is Percona Backup for MongoDB is available for MongoDB 4.0 or later versions.
- Retention days is enabled and defaults to 31 days.

Steps to schedule a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the MongoDB Sharded cluster chosen.
    3. **Backup method**. Defaults to `percona-backup-mongodb` and is the sole backup method. Option to choose is disabled. By default, it displays a **Continue** button allowing you to install the backup tool package.
        1. Click **Continue** button to proceed on installing the `percona-backup-mongodb` backup tool
            1. Specify the **Backup directory**. Make sure that the backup directory path is existent. Once ready, hit **Install** button. Hit **Cancel** if you are not ready to install yet. If installing the tool, make sure that the job has to be done successfully.
5. Click **Continue**.

6. Second step is **Advanced settings**.
    1. **Retention** is set to **On** and defaults to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
7. Click **Continue** to proceed the last step **Preview**.
8. Third step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
9. Click **Continue** to proceed the last step **Preview**.
10. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

Redis Sentinel Cluster

The following vendors and versions are supported for scheduling a backup:

- Redis Sentinel - 6 and 7

Default Configuration

- Uses `rdb` and `aof` as backup method.
- Backup host points to the primary node as default selection.
- Backup runs on the cluster and saves the backup copy in the ClusterControl controller host. You will determine the directory of the backup location in the backup logs once its requested to run.
- compression is set to `true` or **On**.
- Uses `socat` command to transfer the file using port `9999` as the default port for transfer or copy. The port `9999` can be changed if you need other port number by going to the **Backup Settings**.

Steps to schedule a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**

4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the Redis/Valkey cluster chosen.
    3. **Backup host** is set to the primary node. You can opt to choose from primary to replica nodes.
    4. **Backup method** is set to `rdb, aof` as it's the sole backup method and is disabled for selection.
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

```
        4. **Use PIGZ for parallel gzip** field is set to **Off** by␣
↪default. Setting to **On** will use PIGZ and this shall speed up your␣
↪compression especially if your data to be back up is huge.

    5. **Enable encryption** is set to **Off**. When set to **On**,␣
↪ClusterControl will automate the generation of encryption key. This key␣
↪shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

7. Click **Continue**.

8. Third step is **Local storage**.

    1. In **Storage location**, you can choose from **Store on controller**␣
↪or `Store on node`, where the node pertains to the source of the backup␣
↪data being taken and executed.

    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a␣
↪different path if you desire it somewhere else or on a volume where it␣
↪has large storage space or an NFS volume.

    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value␣
↪here is a sub-directory of the path set in **Storage directory** where `
↪%I` is an integer that is the backup ID assigned for every process of␣
↪backup jobs.

9. Click **Continue**.

10. Fourth step is **Cloud storage**. This step is only visible whenever␣
↪**Upload backup to cloud** is set. Otherwise, this step is skipped and␣
↪fourth step shall proceed to **Preview**, the last step.
```

(continues on next page)

(continued from previous page)

```
    1. In **Credentials** field is the available credentials you've created␣
↪in **Settings → Cloud storage credentials**.

    2. Specify in the **Bucket** field for your target bucket in your S3␣
↪storage. For new bucket, make sure to hit the **+** (plus) sign button␣
↪and make sure to confirm and hit **OK** button.

    3. Specify the **Location path** which location you want the backup to␣
↪be stored under your **Bucket** assigned.

    4. **Retention period** is set to **On**. Specify the number of days␣
↪you want the retention of your backup. Otherwise, set to **Off** if you␣
↪do not need retention of your backup.

    5. **Delete after upload** is set to **Off**. For some instances, you␣
↪want your backup stored locally to be removed or wipe immediately to␣
↪reclaim the disk space since backup is already stored safely in the␣
↪cloud. Turn it **On** if that is the condition of your environment and␣
↪setup.

11. Click **Continue** to proceed the last step **Preview**.

12. The next step is **Schedule**. It contains **Simple** and **Advanced**␣
↪tabs to choose for creating a scheduled backup.

    1. If **Simple** is selected (default view), it contains the following␣
↪elements:

        1. Setup periodically by `minute` (default selection). Other␣
↪options are `hour`, `day`, `week`, `month`, `year`. Every options, except␣
↪`minute` has its own fields to refine the type of period you would like␣
↪to set. For example, choosing `hour` would show a drop down to choose␣
↪your desired hour and minute to schedule the backup.

    2. **Select timezone** for your desired timezone especially if the␣
↪server is located on a different timezone than you have. It is very␣
↪important to determine the correct timezone to avoid running your backup␣
↪during high-peak period.

13. Click **Continue** to proceed the last step **Preview**.

14. The last step is **Preview**, this is the last step where it shows all␣
↪the values and chosen options for the ClusterControl to use for setting␣
↪up the backup.
```

ClusterControl CLI

To be added soon

Redis/Valkey Cluster

The following vendors and versions are supported for scheduling a backup:

- Redis Cluster - 6 and 7
- Valkey Cluster - 7 and 8 (only RHEL 8/9 are supported)

Default Configuration

- Uses `rdb` and `aof` as backup method.
- Backup runs on the cluster and saves the backup copy in the ClusterControl controller host. You will determine the directory of the backup location in the backup logs once its requested to run.
- Compression is set to `true` or **On**.
- Uses `socat` command to transfer the file using port `9999` as the default port for transfer or copy. The port `9999` can be changed if you need other port number by going to the **Backup Settings**.

Steps to schedule a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the Redis/Valkey cluster chosen.
    3. **Backup method** is set to `rdb, aof` as it's the sole backup method and is disabled for selection.
    4. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Compression level** is set to `6 (System default)`

```
    4. **Use PIGZ for parallel gzip** field is set to **Off** by␣
↪default. Setting to **On** will use PIGZ and this shall speed up your␣
↪compression especially if your data to be back up is huge.
```

(continues on next page)

(continued from previous page)

```
    5. **Enable encryption** is set to **Off**. When set to **On**,␣
↪ClusterControl will automate the generation of encryption key. This key␣
↪shall be in the `/etc/cmon.d/cmon_<cluster_id>.cnf` config file.

7. Click **Continue**.

8. Third step is **Local storage**.

    1. In **Storage location**, you can choose from **Store on controller**␣
↪or `Store on node`, where the node pertains to the source of the backup␣
↪data being taken and executed.

    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a␣
↪different path if you desire it somewhere else or on a volume where it␣
↪has large storage space or an NFS volume.

    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value␣
↪here is a sub-directory of the path set in **Storage directory** where `
↪%I` is an integer that is the backup ID assigned for every process of␣
↪backup jobs.

9. Click **Continue**.

10. Fourth step is **Cloud storage**. This step is only visible whenever␣
↪**Upload backup to cloud** is set. Otherwise, this step is skipped and␣
↪fourth step shall proceed to **Preview**, the last step.

    1. In **Credentials** field is the available credentials you've created␣
↪in **Settings → Cloud storage credentials**.

    2. Specify in the **Bucket** field for your target bucket in your S3␣
↪storage. For new bucket, make sure to hit the **+** (plus) sign button␣
↪and make sure to confirm and hit **OK** button.

    3. Specify the **Location path** which location you want the backup to␣
↪be stored under your **Bucket** assigned.

    4. **Retention period** is set to **On**. Specify the number of days␣
↪you want the retention of your backup. Otherwise, set to **Off** if you␣
↪do not need retention of your backup.

    5. **Delete after upload** is set to **Off**. For some instances, you␣
↪want your backup stored locally to be removed or wipe immediately to␣
↪reclaim the disk space since backup is already stored safely in the␣
↪cloud. Turn it **On** if that is the condition of your environment and␣
↪setup.

11. Click **Continue** to proceed the last step **Preview**.
```

(continues on next page)

(continued from previous page)

```
12. The next step is **Schedule**. It contains **Simple** and **Advanced**␣
↪tabs to choose for creating a scheduled backup.

    1. If **Simple** is selected (default view), it contains the following␣
↪elements:

        1. Setup periodically by `minute` (default selection). Other␣
↪options are `hour`, `day`, `week`, `month`, `year`. Every options, except␣
↪`minute` has its own fields to refine the type of period you would like␣
↪to set. For example, choosing `hour` would show a drop down to choose␣
↪your desired hour and minute to schedule the backup.

    2. **Select timezone** for your desired timezone especially if the␣
↪server is located on a different timezone than you have. It is very␣
↪important to determine the correct timezone to avoid running your backup␣
↪during high-peak period.

13. Click **Continue** to proceed the last step **Preview**.

1. The last step is **Preview**, this is the last step where it shows all␣
↪the values and chosen options for the ClusterControl to use for setting␣
↪up the backup.
```

ClusterControl CLI

To be added soon

SQL Server

The following vendors and versions are supported for scheduling a backup:

- Microsoft SQL Server for Linux - 2019 and 2022

ClusterControl does not use external tool to backup. It relies on the internal backup engine from Microsoft SQL Server itself.

Default Configuration

- Uses internal backup engine.
- Can create backup using methods **Full**, **Differential**, and **Transaction Log**.
- Runs backup on the primary node regardless of backup method chosen.
- Stores the backup directory in `/var/lib/backups`
- Stores backup data only in the primary node.
- Cannot run backup if primary is down as it cannot find a candidate server.
- If backup method chosen not equal to **Full**, and directory is does not find any previous full backup, it will upgrade its method to **Full**, making it as the first full backup for that job request.

Steps to schedule a backup (Full, Differential, Transaction Log)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the Microsoft SQL Server cluster chosen.
    3. **Backup host**. It is a drop-down element where you can choose where the backup to be taken and executed. Optionally, you can also choose **Auto Select** which it will choose the best node to execute the backup with least traffic and low load.
    4. **Backup method**. Choose **Full** if you want full backup. Choose **Differential** if you want a differential backup. Lastly, choose **Transaction Log** if you want transaction log backup.
    5. **Upload backup to cloud** is set to **Off**. If you already setup cloud storage credentials and want to upload to the cloud we support, set this to **On**.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Compression** is set to **On** as default. Setting to **Off** will not compress the backup when stored.
    2. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
    3. **Include system databases** is set to **On** by default. Set to **Off** if you do not want.
7. Click **Continue**.
8. Third step is **Local storage**.
    1. In **Storage location** field is disabled as backup storage location defaults to the primary node.
    2. **Storage directory** is set to `/home/vagrant/backups`. Set to a different path if you desire it somewhere else or on a volume where it has large storage space or an NFS volume.
    3. **Backup subdirectory** is set to `BACKUP-%I`. Basically, the value here is a sub-directory of the path set in **Storage directory** where `%I` is an integer that is the backup ID assigned for every process of backup jobs.
9. Click **Continue**.
10. Fourth step is **Cloud storage**. This step is only visible whenever **Upload backup to cloud** is set. Otherwise, this step is skipped and fourth step shall proceed to **Preview**, the last step.
    1. In **Credentials** field is the available credentials you've created in **Settings → Cloud storage credentials**.
    2. Specify in the **Bucket** field for your target bucket in your S3 storage. For new bucket, make sure to hit the **+** (plus) sign button and make sure to confirm and hit **OK** button.

3. Specify the **Location path** which location you want the backup to be stored under your **Bucket** assigned.
4. **Retention period** is set to **On**. Specify the number of days you want the retention of your backup. Otherwise, set to **Off** if you do not need retention of your backup.
5. **Delete after upload** is set to **Off**. For some instances, you want your backup stored locally to be removed or wipe immediately to reclaim the disk space since backup is already stored safely in the cloud. Turn it **On** if that is the condition of your environment and setup.

11. Click **Continue** to proceed the last step **Preview**.
12. The next step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
13. Click **Continue** to proceed the last step **Preview**.
14. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

ElasticSearch

The following vendors and versions are supported for scheduling a backup:

- Elasticsearch by Elastic versions 7.17, 8.3, 8.15

ClusterControl does not use external tool to backup. It relies on `elasticsearch-snapshot` tool to backup the data.

Default Configuration

- Uses Elasticsearch built-in backup engine `elasticsearch-snapshot`.
- Repository defaults to the repository assigned or created during deployment or importing of the cluster. This can also be found and modified using parameter `elastic_snapshot_repository` on the ClusterControl controller host through config file `/etc/cmon.d/cmon_<CLUSTER_ID>.cnf`, where `<CLUSTER_ID>` is the cluster id of the Elasticsearch cluster.
- Retention days is set to 31.
- Backup data shall be stored on the shared location set in the config file `/etc/cmon.d/cmon_<CLUSTER_ID>.cnf` through parameter `elastic_shared_filesystem`.

Steps to create a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and click **Create backup** button.
3. Choose **Schedule a Backup**
4. First step is **Configuration**.
    1. Fill-in the **Schedule name** field for your desired title or name of this scheduled backup. This serves as your identifier for your scheduled backups.
    2. **Cluster** is set to the name of the Elasticsearch cluster chosen.
    3. **Repository**. By default, points to your repository assigned in your `/etc/cmon.d/cmon_<CLUSTER_ID>.cnf` config file through parameter `elastic_snapshot_repository`. If you have other repository created, this will show up in the drop-down list.
    4. **Backup method**. This is constant and sets to `elastlicsearch-snapshot`.
5. Click **Continue**.
6. Second step is **Advanced settings**.
    1. **Retention** is set to `31 days`. Input your desired number if you want less than or greater than the default. Also set to **Off** if you do not want retention.
7. Click **Continue**.
8. Third step is **Schedule**. It contains **Simple** and **Advanced** tabs to choose for creating a scheduled backup.
    1. If **Simple** is selected (default view), it contains the following elements:
        1. Setup periodically by `minute` (default selection). Other options are `hour`, `day`, `week`, `month`, `year`. Every options, except `minute` has its own fields to refine the type of period you would like to set. For example, choosing `hour` would show a drop down to choose your desired hour and minute to schedule the backup.
    2. **Select timezone** for your desired timezone especially if the server is located on a different timezone than you have. It is very important to determine the correct timezone to avoid running your backup during high-peak period.
9. Click **Continue** to proceed the last step **Preview**.
10. The last step is **Preview**, this is the last step where it shows all the values and chosen options for the ClusterControl to use for setting up the backup.

ClusterControl CLI

To be added soon

2.61 Restore Backup

There are two key reasons for engaging with database backups:

- Recovering a database through restoring from a previous backup
- Verifying the authenticity and reliability of the backup.

Restoring a backup aids corruption or accidental data deletion or do point-in-time recovery to undo a bad transaction or operation. It can also beneficial and useful for testing or cloning environment. See *Create Cluster from Backup* to learn more about creating a cluster from backup.

Verifying is particularly important for meeting compliance requirements and ensuring a reliable recovery when a disaster strikes, which is essential for returning the system to a healthy state. See *Backup Verification* for more details on how to verify the backup.

The Restore Backup feature in ClusterControl tackles these concerns providing a safe, efficient, and user-friendly way to manage your backup and do the appropriate actions you need for your recovery plan.

2.61.1 Overview of Restore Backup

ClusterControl provides two ways to restore the backup.

1. Restore the backup within the cluster. This is very important especially if your database cluster encounters a disaster such as data loss or data corruption.
2. Verifying the backup on a single node or external node.

ClusterControl also offers *backup restoration taken externally*. This means, backups that are created without relying or using ClusterControl supported backup methods. However, only MySQL Replication or MySQL Galera built clusters have this feature. To know more about this feature, checkout *Restore External Backup*

On this page, the Restore Backup feature focuses on restoring backups that were made or taken using ClusterControl supported backup methods. Restoring the backup means that it can only be restored to its source cluster where the backup data originated. If your cluster experiences data corruption or data loss and you need to restore from its previous state, this page will guide you through the process of recovering your data.

This guide provides step-by-step instructions on how to restore your backup to its own cluster using ClusterControl.

2.61.2 Support Matrix

All database clusters supported by ClusterControl include the backup restore feature. See below for the complete list:

Database	Vendor	Topology
MySQL	Percona, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.61.3 Prerequisites

- Backup should be created using ClusterControl and from an existing cluster.
- You have sufficient storage space for the target environment. Make sure that the new environment or target nodes has enough disk space to copy and restore the backup.
- Network interoperability between ClusterControl controller and the source cluster where backup was taken.
- Ensure that you have appropriate user permissions and privileges on the target environment. If security kernel modules such as AppArmor or SELinux must be enabled, make sure you have appropriate configuration to allow the operation successful.
- Ensure that the state of your topology are intact. Since restore backup is to recover from failure, restoration means that nodes currently down should be able to get back on normal state once backup restoration process is done.

2.61.4 MySQL Replication

The following vendors and versions are supported for restoring a backup:

- Oracle MySQL - 8.0.
- Percona Server for MySQL - 8.0.
- Percona Server for MySQL Pro - 8.0.
- MariaDB Server - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster. By default, it picks the primary node but you can opt to restore the backup and select on the replica nodes
- Only successful backups can be listed and can be stored
- Bootstrap cluster from restored node is enabled or set to **On**

Steps to Restore a Backup (Backup method: mysqldump, mariabackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Incremental backups**: *(Only for **mariabackup** created backups)*. This is a drop down element that lists down the incremental backups link if the backup to be restored is a `mariabackup-full` method.
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - **Restore on node**: Make sure you select this option since this is you are going to restore your backup for recovery of your cluster.
   - **Restore and verify on standalone host**: This is for restoring the backup to verify if data is good or not. This shall be discussed on *Backup Verification*
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: When restoring the backup data, on most cases, you may to revert your cluster back to its previous health especially when disaster had just happened. Make sure you selected the *Primary* node. Otherwise, if you are just picking up some data for recovery or checking the contets that has been dropped or gone, you can try to restore it to the *Replica* node.
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. It must be as big as the datadir
   - **Restore system database**: *(Displays only for **mysqldump**)*. This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
   - **Make a copy of the datadir before restoring the backup**: *(Only for **mariabackup** created backups)*.This is turned off by default. Turn it **On** if you want to replace the system database when restoring the backup to its own cluster.
   - **Bootstrap cluster from restored node**: By default, this is turned **On**. This is the nature of storing the backup to its own cluster since it requires to bootstrap and make sure the primary starts first to avoid data drift or avoid deltas between your cluster nodes.

- **Point in time recovery (PITR)**: *(Displays only for **mysqldump** with PITR enabled and **mariabackup** created backups)*. This is turned off by default. If you are storing it with PITR, make sure to enable this feature.
  - **Time**: This tab is displayed when **PITR** is enabled or **On**.
    - **Restore time in host TZ**: When time is selected, this field is displayed and you need to specify the period when you want the PITR to stop. This means that recover the data up until the date and time given by Restore Time (Event time - stop date&time).
  - **Position**: This tab is displayed when **PITR** is enabled or **On**.
    - **Binary log name**: This is the binary log name for which the PITR shall based on what file will check to based upon until the PITR stops.
    - **Log stop position**: This is the log position for which the PITR shall based on the binary log name and which log position in that binlog file until the PITR stops.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

Steps to Restore a Backup (Backup method: xtrabackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - **Restore on node**: Make sure you select this option since this is you are going to restore your backup for recovery of your cluster.
   - **Restore and verify on standalone host**: This is for restoring the backup to verify if data is good or not. This shall be discussed on *Backup Verification*
8. Click **Continue**.
9. Third step is **Settings**.

- **Restore backup on**: When restoring the backup data, on most cases, you may to revert your cluster back to its previous health especially when disaster had just happened. Make sure you selected the *Primary* node. Otherwise, if you are just picking up some data for recovery or checking the contets that has been dropped or gone, you can try to restore it to the *Replica* node.
- **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. It must be as big as the datadir
- **xtrabackup –use-memory (MiB)**: This option affects how much memory is allocated for preparing a backup with `xtrabackup --prepare`, or analyzing statistics with `xtrabackup --stats`. Its purpose is similar to `innodb_buffer_pool_size`. It does not do the same thing as the similarly named option in Oracle's InnoDB Hot Backup tool. The default value is 100MB, and if you have enough available memory, 1024MB to 2048MB is a good recommended value.
- **Bootstrap cluster from restored node**: By default, this is turned **On**. This is the nature of storing the backup to its own cluster since it requires to bootstrap and make sure the primary starts first to avoid data drift or avoid deltas between your cluster nodes.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.5 MySQL Galera

The following vendors and versions are supported for restoring a backup:

- Percona XtraDB Cluster - 8.0.
- MariaDB Cluster - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster. By default, it picks the primary node but you can opt to restore the backup and select on the replica nodes
- Only successful backups can be listed and can be stored
- Bootstrap cluster from restored node is enabled or set to **On**

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).

4. Select and click **Restore** option.
5. First step is **Configuration**.
    - **Backup**: This is selected backup to be restored to the cluster
    - **Incremental backups**: *(Only for **mariabackup** and **xtrabackup** created backups with incrementals)*. This is a drop down element that lists down the incremental backups link if the backup to be restored is a `mariabackupfull` method.
    - **Restore this backup from**: This is the path or storage location of the backup
    - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
    - **Restore on node**: Make sure you select this option since this is you are going to restore your backup for recovery of your cluster.
    - **Restore and verify on standalone host**: This is for restoring the backup to verify if data is good or not. This shall be discussed on *Backup Verification*
8. Click **Continue**.
9. Third step is **Settings**.
    - **Restore backup on**: When restoring the backup data, on most cases, you may to revert your cluster back to its previous health especially when disaster had just happened. Make sure you selected the *Primary* node. Otherwise, if you are just picking up some data for recovery or checking the contets that has been dropped or gone, you can try to restore it to the *Replica* node.
    - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. It must be as big as the datadir
    - **xtrabackup –use-memory (MiB)**: *(Displays only for **xtrabackup**)*.This option affects how much memory is allocated for preparing a backup with `xtrabackup --prepare`, or analyzing statistics with `xtrabackup --stats`. Its purpose is similar to `innodb_buffer_pool_size`. It does not do the same thing as the similarly named option in Oracle's InnoDB Hot Backup tool. The default value is 100MB, and if you have enough available memory, 1024MB to 2048MB is a good recommended value.
    - **Restore system database**: *(Displays only for **mysqldump**)*. This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
    - **Make a copy of the datadir before restoring the backup**: *(Only for **mariabackup** created backups)*.This is turned off by default. Turn it **On** if you want to replace the system database when restoring the backup to its own cluster.
    - **Bootstrap cluster from restored node**: By default, this is turned **On**. This is the nature of storing the backup to its own cluster since it requires to bootstrap and make sure the primary starts first to avoid data drift or avoid deltas between your cluster nodes.
    - **Point in time recovery (PITR)**: *(Displays only for **mysqldump** with PITR enabled and **mariabackup** created backups)*. This is turned off by default. If you are storing it with PITR, make

sure to enable this feature.

- **Time**: This tab is displayed when **PITR** is enabled or **On**.
  - **Restore time in host TZ**: When time is selected, this field is displayed and you need to specify the period when you want the PITR to stop. This means that recover the data up until the date and time given by Restore Time (Event time - stop date&time).
- **Position**: This tab is displayed when **PITR** is enabled or **On**.
  - **Binary log name**: This is the binary log name for which the PITR shall based on what file will check to based upon until the PITR stops.
  - **Log stop position**: This is the log position for which the PITR shall based on the binary log name and which log position in that binlog file until the PITR stops.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.6 PostgreSQL/TimescaleDB

The following vendors and versions are supported for restoring a backup:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, and 16.
- PostgreSQL from EDB repository - 12, 13, 14, 15, and 16 (requires a valid EDB repository token).

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored
- Bootstrap cluster from restored node is enabled or set to **On**

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup

- **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken

6. Click **Continue**.
7. Second step is **How to restore**.
    - **Restore on node**: Make sure you select this option since this is you are going to restore your backup for recovery of your cluster.
    - **Restore and verify on standalone host**: This is for restoring the backup to verify if data is good or not. This shall be discussed on *Backup Verification*
8. Click **Continue**.
9. Third step is **Settings**.
    - **Restore backup on**: This drop-down element is disabled and defaults to the *Primary* node for which the restore shall happen.
    - **Make a copy of the datadir before restoring the backup**: *(Displays only for **pgbackrest** created backups)*. This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
    - **Point in time recovery (PITR)**: *(Displays only for **pgbackrest** created backups)*. This is turned off by default. If you are storing it with PITR, make sure to enable this feature.
        - **Restore Time**: This tab is displayed when **PITR** is enabled or **On**. This is the time until what the PostgreSQL forward (the oldest possible timestamp is the backup creation timestamp, or the current timestamp minus the "PITR_RETENTION_HOURS" configuration value if it is greater than 1.)
    - **Bootstrap cluster from restored node**: By default, this is turned **On**. This is the nature of storing the backup to its own cluster since it requires to bootstrap and make sure the primary starts first to avoid data drift or avoid deltas between your cluster nodes.
10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.7 MongoDB Replica Set

The following vendors and versions are supported for restoring a backup:

- MongoDB Enterprise - 4.4, 5.0, 6.0 and 7.0
- MongoDB Community - 4.4, 5.0, 6.0 and 7.0
- Percona Server for MongoDB - 4.4, 5.0, 6.0 and 7.0

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored
- Only **mongodump** backups can be attempted for backup restore

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.8 MongoDB Sharded Cluster

The following vendors and versions are supported for restoring a backup:

- MongoDB - 4.4, 5.0, 6.0 and 7.0
- MongoDB Enterprise - 4.4., 5.0, 6.0 and 7.0
- Percona - 4.4, 5.0, 6.0 and 7.0

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored
- Only **percona-backup-mongodb** backups can be restored for backup restore

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **Settings**.
   - **Make a copy of the datadir before restoring the backup**: This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
8. Click **Continue**to proceed the last step **Summary**.
9. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
10. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.9 Redis Sentinel Cluster

The following vendors and versions are supported for restoring a backup:

- Redis Sentinel - 6 and 7

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored
- Only **rdb, aof** backups can be restored for backup restore

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - **Restore on node**: Make sure you select this option since this is you are going to restore your backup for recovery of your cluster.
   - **Restore and verify on standalone host**: This is for restoring the backup to verify if data is good or not. This shall be discussed on *Backup Verification*
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: This drop-down element is disabled and defaults to the *Primary* node for which the restore shall happen.
   - **Make a copy of the datadir before restoring the backup**: This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.10 Redis/Valkey Cluster

The following vendors and versions are supported for restoring a backup:

- Redis Cluster - 6 and 7
- Valkey Cluster - 7 and 8 (only RHEL 8/9 are supported)

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored
- Only **rdb, aof** backups can be restored for backup restore

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Third step is **Settings**.
   - **Restore backup on**: This drop-down element is disabled and defaults to the *Primary* node for which the restore shall happen.
   - **Make a copy of the datadir before restoring the backup**: This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
8. Click **Continue**to proceed the last step **Summary**.
9. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
10. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.11 SQL Server

The following vendors and versions are supported for restoring a backup:

- Microsoft SQL Server for Linux - 2019 and 2022

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Incremental backups**: (Display only if the backup to be restored is a `mssqlfull` that has incrementals). This is a drop down element that lists down the incremental backups link to the complete `mssqlfull backup.
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.61.12 ElasticSearch

The following vendors and versions are supported for restoring a backup:

- Elasticsearch by Elastic versions 7.17, 8.3, 8.15

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore
- Restore can only be done on the current cluster and on its primary node
- Only successful backups can be listed and can be stored
- Only **elasticsearch-snapshot** backups can be restored for backup restore

Steps to Restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).

3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
    - **Backup**: This is selected backup to be restored to the cluster
    - **Restore this backup from**: This is the path or storage location of the backup
    - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **Settings**.
    - **Make a copy of the datadir before restoring the backup**: This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
8. Click **Continue**to proceed the last step **Summary**.
9. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
10. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.62 Restore External Backup

Before you end up with this guide, we expect that you have already check out and read *Restore Backup*. The cluster supported are MySQL Replication and Galera built clusters.

Similar to *Restore Backup* feature, they share the same fundamentals of restoring data from backup to its exisitng cluster. Data recovery, verifying your backup data to your target cluster, or restoring the previous health of your cluster.

2.62.1 Difference with restoring internal vs external backup

We have discussed the purpose of restoring a backup in *Restore Backup*. Primarily, the difference between internal and external backup are how they are stored and accessed by ClusterControl to perform a backup restoration. Internal backups are stored with metadata involved such that ClusterControl knows its location and how it was created. Whereas, external backups, you need to make sure what backup method is this created.

If you're looking to understand the difference between restoring an internal backup and restoring from an external backup, here's a breakdown of the key distinctions between the two:

1. External Backup Restore
    - Primarily, you can store a backup that was taken without using ClusterControl or backups that have been taken outside ClusterControl or tools provided by ClusterControl technology.
    - Only in the primrary node you can restore an external backup.

- Backup you can restore are either located on ClusterControl controller host, or from any of your database nodes registered in that cluster as long as its directory location is accessible in the host.
- Only `mysqldump` and `xtrabackup`/`mariabackup` are supported for external backup restore.

2. Internal Backup Restore
   - Only backups taken using ClusterControl can be restored.
   - You can restore the backup on the primary database server.
   - You can restore the backup on the replica database server.
   - If a backup verification server is already existent or setup, then you can also restore the backup on that backup verification server.
   - Backup you can restore are either located on ClusterControl controller host, or from any of your database nodes registered in that cluster as long as its directory location is accessible in the host.
   - You can restore your internal backup in the primary nodes. If you have a MySQL Replication Cluster, you can restore either from a primary or to a replica node.
   - You can restore with more options to chose from such as PITR and restore your system database.

2.62.2 Support Matrix

All database clusters supported by ClusterControl include the backup restore feature. Checkout below for the complete list:

Database	Vendor	Topology
MySQL Replication Cluster	Percona, Oracle, MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication

2.62.3 Prerequisites

- Backup should be supported by ClusterControl backup method.
- You have sufficient storage space for the target environment. Make sure that the new environment or target nodes has enough disk space to copy and restore the backup.
- Backup should be accessible via file system and make sure your directory. If shared, should have enough permission to access the directory and files in the network.
- Network interoperability between ClusterControl controller and the source cluster where backup was taken.
- Ensure that you have appropriate user permissions and privileges on the target environment. If security kernel modules such as AppArmor or SELinux must be enabled, make sure you have appropriate configuration to allow the operation successful.
- Ensure that the state of your topology are intact. Since restore backup is to recover from failure, restoration means that nodes currently down should be able to get back on normal state once backup restoration process is done.

2.62.4 MySQL Replication

The following vendors and versions are supported for restoring an external backup:

- Oracle MySQL - 8.0.
- Percona Server for MySQL - 8.0.
- Percona Server for MySQL Pro - 8.0.
- MariaDB Server - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backup should be supported by ClusterControl backup method. Currently, you can restore backups created using `mysqqldump`, `xtrabackup`, or `mariabackup`
- Restore can only be done on the current cluster. Only on the primary database node you can target to restore the external backup.
- Bootstrap cluster from restored node is enabled or set to **On**.

Steps to restore an external backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and find **More…** button just beside after the **Create backup** button.
3. Click **More…** button and choose **Restore backup** in the drop-down selection button.
4. A command prompt will be displayed and select **Restore an external backup** by clicking the option.
5. First step is **Configuration**.
    - **Cluster**: This is your selected database cluster in ClusterControl on which your target external backup restore shall be applied.
    - **Restore backup on**: This is the primary server where the external backup shall initiate from.
    - **Backup Method**: This is a drop-down list. You can choose from `mysqldump` or `xtrabackup/mariabackup`(for MariaDB databases). For this step, make sure you choose `mysqldump`
    - **Storage Host**: Select the host where the backuup file is stored
    - **Backup Path (absolute path)**: This is required. You need to specify the absolute path of your backup. In `mysqldump` created backup, supported extensions: sql.gz, sql.bz2, aes, aes256, aes128
    - **Temporary Directory**: The tmpdir is used as temporary storage area for the backup files during restore. It must be as big as the datadir

```
      - **The dump file sets the database to restore the data into**:␣
↪This is a checkbox button and is enabled by default. When check this␣
↪means that dump includes database. If the dump does **not** include␣
↪databases then set this off.
   - **Reset master before restore**: This is a checkbox button and is␣
↪disabled or unchecked by default. Enable or check since this maybe needed␣
```

(continues on next page)

(continued from previous page)

```
↪if the dumpfile contains GTID information.

6. Click **Continue** to proceed the last step **Summary**.

7. The last step is **Summary**. This summarizes the action to be taken␣
↪when restoring an external backup backup to your cluster. This gives you␣
↪the time to review your actions before executing the restore process.

8. Click **Finish** button to execute the job.
```

ClusterControl CLI

To be added soon

Steps to restore an external backup (Backup method: xtrabackup or mariabackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and find **More…** button just beside after the **Create backup** button.
3. Click **More…** button and choose **Restore backup** in the drop-down selection button.
4. A command prompt will be displayed and select **Restore an external backup** by clicking the option.
5. First step is **Configuration**.
   - **Cluster**: This is your selected database cluster in ClusterControl on which your target external backup restore shall be applied.
   - **Restore backup on**: This is the primary server where the external backup shall initiate from.
   - **Backup Method**: This is a drop-down list. You can choose from `mysqldump` or `xtrabackup/mariabackup`(for MariaDB databases). For this step, make sure you choose `mysqldump`
   - **Storage Host**: Select the host where the backuup file is stored
   - **Backup Path (absolute path)**: This is required. You need to specify the absolute path of your backup. In `mysqldump` created backup, supported extensions: sql.gz, sql.bz2, aes, aes256, aes128
   - **Temporary Directory**: The tmpdir is used as temporary storage area for the backup files during restore. It must be as big as the datadir

```
      - **The dump file sets the database to restore the data into**:␣
↪This is a checkbox button and is enabled by default. When check this␣
↪means that dump includes database. If the dump does **not** include␣
↪databases then set this off.
   - **Reset master before restore**: This is a checkbox button and is␣
↪disabled or unchecked by default. Enable or check since this maybe needed␣
↪if the dumpfile contains GTID information.

6. Click **Continue** to proceed the last step **Summary**.
```

(continues on next page)

(continued from previous page)

```
7. The last step is **Summary**. This summarizes the action to be taken␣
↪when restoring an external backup backup to your cluster. This gives you␣
↪the time to review your actions before executing the restore process.

8. Click **Finish** button to execute the job.
```

ClusterControl CLI

To be added soon

2.62.5 MySQL Galera

The following vendors and versions are supported for restoring an external backup:

- Percona XtraDB Cluster - 8.0.
- MariaDB Cluster - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore.
- Restore can only be done on the current cluster. By default, it picks the primary node but you can opt to restore the backup and select on the replica nodes.
- Only successful backups can be listed and can be stored.
- Bootstrap cluster from restored node is enabled or set to **On**.

Steps to restore a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Incremental backups**: *(Only for **mariabackup** and **xtrabackup** created backups with incrementals)*. This is a drop down element that lists down the incremental backups link if the backup to be restored is a `mariabackupfull` method.
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.

- **Restore on node**: Make sure you select this option since this is you are going to restore your backup for recovery of your cluster.
- **Restore and verify on standalone host**: This is for restoring the backup to verify if data is good or not. This shall be discussed on *Backup Verification*.

8. Click **Continue**.
9. Third step is **Settings**.
    - **Restore backup on**: When restoring the backup data, on most cases, you may to revert your cluster back to its previous health especially when disaster had just happened. Make sure you selected the *Primary* node. Otherwise, if you are just picking up some data for recovery or checking the contets that has been dropped or gone, you can try to restore it to the *Replica* node.
    - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. It must be as big as the datadir
    - **xtrabackup –use-memory (MiB)**: *(Displays only for **xtrabackup**)*.This option affects how much memory is allocated for preparing a backup with `xtrabackup --prepare`, or analyzing statistics with `xtrabackup --stats`. Its purpose is similar to `innodb_buffer_pool_size`. It does not do the same thing as the similarly named option in Oracle's InnoDB Hot Backup tool. The default value is 100MB, and if you have enough available memory, 1024MB to 2048MB is a good recommended value.
    - **Restore system database**: *(Displays only for **mysqldump**)*. This is turned off by default. Turn it **On** if your data directory is not corrupted and the purpose of storing the backup is just for testing or verification purposes within your existing cluster. Other than that, you might also have valuable data in data directory that you do not want to be wipe out.
    - **Make a copy of the datadir before restoring the backup**: *(Only for **mariabackup** created backups)*.This is turned off by default. Turn it **On** if you want to replace the system database when restoring the backup to its own cluster.
    - **Bootstrap cluster from restored node**: By default, this is turned **On**. This is the nature of storing the backup to its own cluster since it requires to bootstrap and make sure the primary starts first to avoid data drift or avoid deltas between your cluster nodes.
    - **Point in time recovery (PITR)**: *(Displays only for **mysqldump** with PITR enabled and **mariabackup** created backups)*. This is turned off by default. If you are storing it with PITR, make sure to enable this feature.
        - **Time**: This tab is displayed when **PITR** is enabled or **On**.
            - **Restore time in host TZ**: When time is selected, this field is displayed and you need to specify the period when you want the PITR to stop. This means that recover the data up until the date and time given by Restore Time (Event time - stop date&time).
        - **Position**: This tab is displayed when **PITR** is enabled or **On**.
            - **Binary log name**: This is the binary log name for which the PITR shall based on what file will check to based upon until the PITR stops.
            - **Log stop position**: This is the log position for which the PITR shall based on the binary log name and which log position in that binlog file until the PITR stops.
10. Click **Continue** to proceed the last step **Summary**.

11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.63 Backup Verification

As what we have discussed in *Restoring Backup* user guide, one importance of restoring a backup is to verify the backup. Does it comply with standards or comply with your data protection and business continuity? Does it meet with regulatory frameworks (such as GDPR, HIPAA, or US – SOX)? How ready are you when disaster strikes? Are your backups consistent and reliable once the backup is created and ready for restoration? For organizations and companies that has strict compliance and disaster recovery planning, backup verification feature of ClusterControl is helpful.

Backup verification is one of the very important process when managing backups. A successful backup might sound important and useful, but the big question that database administrators cannot determine is, how reliable the backup can be? Is it useful and reliable or is it corrupted? You won't truly be prepared for a disaster until you can successfully verify and restore the backup on a single node that is completely isolated from your production or pre-production environments, such as UAT or staging — especially when restoring your data becomes imperative.

Verifying backups isn't just about the data itself — it also involves ensuring that the configuration settings in your backup verification environment align with those in production. This helps confirm that any changes in configuration won't impact the ability to successfully restore the backup.

This page will help you guide on how to verify and restore your backup to provide you and your organization the sense of actual readiness and true sense of security.

2.63.1 Overview of Backup Verification

ClusterControl's Backup Verification is only available for Enterprise license. It allows you to choose your desired standalone environment that is setup for ClusterControl to communicate and access the server to perform necessary steps for the planned process of verifying backup.

Backup verification in ClusterControl is critical to avoid a false sense of security. In that regard, there are two ways you can achieve to verify your backup:

1. You can manually verify existing backups. This is ideal when you want to do the following:
   - Restore data to a target node to recover loss data.
   - Restore data to a target node to verify the integrity of your data.
   - Test and verify if using the data you have can work on a particular server and configuration changes does not damage or put your database server in havoc.
2. You can schedula a backup, then automate verifying your backup data either immediately after the backup is done or set an hour-basis up to not less than 24 hours. This is helpful especially if your systems rely heavily on the availability and integrity of data. Usually, this is the preferred method when creating a backup in order to determine that your backup is reliable and useful in the future whenever disaster happens and you need to restore your data immediately.

2.63.2 Support Matrix

Most database clusters supported by ClusterControl include the backup verification feature. For the complete list, checkout below:

Database	Vendor	Topology
MySQL	Percona, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
Redis	Redis	Sentinel
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.63.3 Prerequisites

- Backup should be created using ClusterControl and from an existing cluster.
- Target **Backup Verification Server (BVS)** should be accessible by ClusterControl controller, same access per OS user on how your database nodes are setup.
- You have sufficient storage space for the target environment. Make sure that the new environment or target nodes has enough disk space to copy and restore the backup. This also includes the BVS.
- Network interoperability between ClusterControl controller and the source cluster where backup was taken.
- Ensure that you have appropriate user permissions and privileges on the target environment. If security kernel modules such as AppArmor or SELinux must be enabled, make sure you have appropriate configuration to allow the operation successful.
- Requires to have a successful backup creation prior to verifying the backup.
- To ensure compatibility, it is recommended that the target BVS environment matches the operating system and version of the source database server where the backup was originally created.

2.63.4 MySQL Replication

The following vendors and versions are supported for backup verification:

- Oracle MySQL - 8.0.
- Percona Server for MySQL - 8.0.
- Percona Server for MySQL Pro - 8.0.
- MariaDB Server - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Restores backup on a standalone host.
- Backups created using ClusterControl's supported backup method shall be available for restore and backup verification.
- Only successful backups can be listed and can be attempted for restore and backup verification.

- ClusterControl will configure the target Backup Verification Server (BVS) and that covers the database configuration, package repositories, and required package dependencies.
- **Install software** shall be set to **On**.
- **Disable firewall** is enabled.
- **Disable SELinux/AppArmor** is enabled.
- When **Shutdown the server after the backup is restored** is checked, the BVS will be terminated when the **Verify Backup** job finishes. The `datadir` and configuration files will be removed. This also means that **Install Software** must be specified for the next *Verify Backup* job to set up the `datadir` and configuration files.

Steps to restore a backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.

- **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.
- **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When **On**, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

Steps to restore a backup (Backup method: xtrabackup and mariabackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (...).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Incremental backups**: Shows up only when there are incremental backups linked to a `mariabackupfull` or `xtrabackupfull` backup. This is a drop down element that lists down the incremental backups link to the full backup (`xtrabackup` or `mariabackup`)
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).

- **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."
- **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
- **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.
- **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.
- **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When **On**, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.63.5 MySQL Galera

The following vendors and versions are supported for backup verification:

- Percona XtraDB Cluster - 8.0.
- MariaDB Cluster - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Default Configuration

- Restores backup on a standalone host.
- Backups created using ClusterControl's supported backup method shall be available for restore and backup verification
- Only successful backups can be listed and can be attempted for restore and backup verification.
- ClusterControl will configure the target Backup Verification Server (BVS) and that covers the database configuration, package repositories, and required package dependencies.
- **Install software** shall be set to **On**.

- **Disable firewall** is enabled.
- **Disable SELinux/AppArmor** is enabled.
- When **Shutdown the server after the backup is restored** is checked, the BVS will be terminated when the **Verify Backup** job finishes. The `datadir` and configuration files will be removed. This also means that **Install Software** must be specified for the next *Verify Backup* job to set up the `datadir` and configuration files.

Steps to restore a backup (Backup method: mysqldump)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.
   - **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.

- **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When **On**, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

Steps to restore a backup (Backup method: xtrabackup and mariabackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Incremental backups**: Shows up only when there are incremental backups linked to a `mariabackupfull` or `xtrabackupfull` backup. This is a drop down element that lists down the incremental backups link to the full backup (`xtrabackup` or `mariabackup`)
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."

- **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
- **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.
- **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.
- **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When **On**, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.63.6 PostgreSQL/TimescaleDB

The following vendors and versions are supported for backup verification:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, and 16.
- PostgreSQL from EDB repository - 12, 13, 14, 15, and 16 (requires a valid EDB repository token).

Default Configuration

- Restores backup on a standalone host
- Backups created using ClusterControl's supported backup method shall be available for restore and backup verification
- Only successful backups can be listed and can be attempted for restore and backup verification
- ClusterControl will configure the target Backup Verification Server (BVS) and that covers the database configuration, package repositories, and required package dependencies
- **Install software** shall be set to **On**
- **Disable firewall** is enabled
- **Disable SELinux/AppArmor** is enabled

- When **Shutdown the server after the backup is restored** is checked, the BVS will be terminated when the **Verify Backup** job finishes. The `datadir` and configuration files will be removed. This also means that **Install Software** must be specified for the next *Verify Backup* job to set up the `datadir` and configuration files.

Steps to restore a backup (Backup method: pgdumpall and pg_basebackup)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.
   - **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.
   - **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When

On, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

Steps to restore a backup (Backup method: pgbackrest)

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Incremental backups**: Shows up only when there are incremental backups linked to a `pgbackrestfull` backup. This is a drop down element that lists down the incremental backups link to the full backup created using `pgbackrest`.
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages

manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.

- **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.
- **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.
- **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When **On**, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.

10. Click **Continue**to proceed the last step **Summary**.
11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.63.7 Redis Sentinel Cluster

The following vendors and versions are supported for backup verification:

- Redis Sentinel - 6 and 7

Default Configuration

- Restores backup on a standalone host.
- Backups created using ClusterControl's supported backup method shall be available for restore and backup verification.
- Only successful backups can be listed and can be attempted for restore and backup verification.
- ClusterControl will configure the target Backup Verification Server (BVS) and that covers the database configuration, package repositories, and required package dependencies.
- **Install software** shall be set to **On**.
- **Disable firewall** is enabled.
- **Disable SELinux/AppArmor** is enabled.
- When **Shutdown the server after the backup is restored** is checked, the BVS will be terminated when the **Verify Backup** job finishes. The `datadir` and configuration files will be removed. This also means that **Install Software** must be specified for the next *Verify Backup* job to set up the `datadir` and configuration files.

Steps to restore a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
   - **Backup**: This is selected backup to be restored to the cluster
   - **Restore this backup from**: This is the path or storage location of the backup
   - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken
6. Click **Continue**.
7. Second step is **How to restore**.
   - Choose **Restore and verify on standalone host**
8. Click **Continue**.
9. Third step is **Settings**.
   - **Restore backup on**: Enter a valid Fully Qualified Domain Name (FQDN) or valid IP-address of your target Backup Verification Server (BVS).
   - **Temporary directory**: The tmpdir is used as temporary storage area for the backup files during restore. Make sure it has enough space or assure you have a bit larger disk space than your datadir total size."
   - **Install software**: Toogle on (default) will instruct ClusterControl to install the database software from the package repository during deployment. Toggling off will skip the database installation part. In some cases, where you have a custom database build or in an offline environment without a proper offline repository set up, you may install the database server packages manually beforehand and set this option to off. ClusterControl will then skip the installation part and proceed to the database configuration part.
   - **Disable firewall** is enabled. Leave check if you need to **Disable firewall** (default). Otherwise uncheck if you have filtered entry for your firewall.
   - **Disable SELinux/AppArmor** is enabled. Leave check if you need to **Disable SELinux/AppArmor**. If you have uncheck **Disable SELinux/AppArmor**, make sure that you have configured properly to allow creating files and opening sockets for the database to run properly.
   - **Shutdown the server after the backup is restored** is set to **Off** (recommended). When **Off**, this assures that the database server is accessible and state is online. Its data directory and config files are also intact and packages are retained and not removed/uninstalled. When **On**, the BVS will be terminated and when the verify backup job finishes, the `datadir` and configuration files will be removed. This means that **Install Software** must be enabled for the next *Verify Backup* job to set up the `datadir` and configuration files.
10. Click **Continue**to proceed the last step **Summary**.

11. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
12. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.63.8 ElasticSearch

In Elasticsearch, verifying a backup has a different approach compared to the other supported database clusters for verifying a backup. Elasticsearch do not need a standalone server or host to verify the backup. Instead, it just run a HTTP request `_snapshot/_verify` to verify if the snapshot is healthy or not. Although it does not verify the backup/snapshot by restoring it, but it does the following:

- Access the repository (e.g., a shared filesystem, S3 bucket, etc.),
- Read and write to the repository
- Ensure all nodes in the cluster can access it (if it's a shared location)
- Confirm the repository is usable for snapshot and restore operations.

The reason that it does not verify the content of a specific snapshot (that is the backup you made is intact or restorable), Elasticsearch are used to have humungous amount of data and that could be complicated to determine and restore a backup to identify its ability if its restorable or not.

The following vendors and versions are supported for backup verification:

- Elasticsearch by Elastic versions 7.17, 8.3, 8.15

Default Configuration

- Backups created using ClusterControl's supported backup method shall be available for restore.
- Only successful backups can be listed and can be stored.
- Only **elasticsearch-snapshot** backups can be verified.
- Sends a HTTP request `_snapshot/_verify` to verify the state of the snapshot or backup.

Steps to restore a backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Restore** option.
5. First step is **Configuration**.
    - **Backup**: This is selected backup to be restored to the cluster
    - **Restore this backup from**: This is the path or storage location of the backup
    - **Method**: This is a panel view just displays information of the backup information such as backup method, size, when it was created, and host where backup was taken

6. Click **Continue**.
7. Second step is **How to restore**.
    - Choose **Verify snapshot**. This does a snapshot checksum to be validated
8. Click **Continue**to proceed the last step **Summary**.
9. The last step is **Summary**. This summarizes the action to be taken when restoring the backup to your cluster. This gives you the time to review your actions before executing the restore process.
10. Click **Finish** button to execute the job.

ClusterControl CLI

To be added soon

2.64 Create Cluster from Backup

The **Create Cluster from Backup** feature is a feature in ClusterControl with relevant approach to how you will deploy using *Create Database Cluster* method. With this method, users are given the option to easily create a new cluster from a backup, eliminating hassle, reducing time consumption, and minimizing errors. It also enables easy scalability and ensures idempotency with just a few clicks. This feature provides an intuitive workflow for configuring high-availability setups, replication, load balancing, and more.

Similar to *Create Database Cluster*, you can:

- Provisioning of a new cluster.
- Clone an exising cluster and re-deploy to a new cluster. Helpful especially you need to build a quick QA, staging, or dev environment.
- Create a copy of an existing cluster and migrate to a new cloud environment.
- Built-in validation to test on a different environment or operating system to achieve a rapid outcome.

This guide provides step-by-step instructions on how to create a cluster from backup using ClusterControl.

2.64.1 Support Matrix

Currently, using the GUI, there's only one suupported cluster in ClusterControl. See below:

Database	Vendor	Topology
Galera Cluster	MariaDB, Percona	Galera certification-based replication

2.64.2 Prerequisites

- Backup should be created using ClusterControl and from an existing cluster.
- Backup should be from a Galera Cluster.
- Only MariaDB Galera and Percona XtraDB Cluster are the types of cluster supported.
- You have sufficient storage space for the target environment. Make sure that the new environment or target nodes has enough disk space to copy and restore the backup.
- Network interoperability between the source of backup (either from an existing cluster node or from the ClusterControl controller host) down to your target environment.

- Ensure that you have appropriate user permissions and privileges on the target environment. If security kernel modules such as AppArmor or SELinux must be enabled, make sure you have appropriate configuration to allow the operation successful.

2.64.3 Creating the Cluster from Backup

There is only one method to create a cluster from backup using ClusterControl. As mentioned above, only Galera Cluster is supported to achieve this job.

Galera Cluster

There is only one method to accomplish the job. The step-by-step in the following section will guide you on hwo to achieve this.

Deployment Steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab.
3. By default, the **All Backups** view shall be displayed. Otherwise, make sure you click the **All Backups** view.
4. In the list of backups, select the backup you desire to be the source of data to stage the cluster and click the ellipsis button (…).
5. Select and click **Create cluster**.

```
6. First step is **Cluster details**. This provides you the backup␣
↪information, fields to define the name of your cluster, and tags.

    1. **Backup**. This field shows the selected backup that was created␣
↪successfully.

    2. **Method**. This is a panel view where it displays the information␣
↪of the backup such as, the backup method used, size, host taken, and when␣
↪it was created.

    3. Under **Name your cluster** panel, there are two fields you can␣
↪specify:

        1. Fill-in the **Name** of the cluster you want to assign.␣
↪(Optional)

        2. Add **Tags** to search or group your database clusters.␣
↪(Optional)

7. Click **Continue**.

8. Second step is **SSH Configuration**. This is where you have to define␣
↪the SSH connection details to have network interoparibility between the␣
↪ClusterControl controller, existing cluster, and the target environment␣
```

(continues on next page)

(continued from previous page)

```
↪for your new cluster.

   1. **SSH user**. Specify root if you have root credentials if you use␣
↪sudo to execute system commands, specify the username that you wish to␣
↪use here. The user must exists on all nodes.

   2. **SSH user key path**. Specify the full path of SSH key (the key␣
↪must exist in ClusterControl node) that will be used by SSH User to␣
↪perform passwordless SSH.

   3. **SSH port**. Specify the SSH port for target nodes. ClusterControl␣
↪assumes SSH is running on the same port on all nodes.

   4. **SSH sudo password**. Specify the password if the SSH user that you␣
↪specified under SSH User requires sudo password to run super-privileged␣
↪commands. Ignore this if SSH User is root or have no sudo password.

   5. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

   6. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

9. Click **Continue**.

10. Third step is **Node configuration**.

   1. Specify the **Server Port** of your MySQL service. Default is set to␣
↪3306.
   2. Specify the **Server data directory** is set to `/var/lib/mysql`. If␣
↪you have prepared a separate path or on a different volume, replace the␣
↪value with your custom path.
   3. **Admin/Root user**. The database admin username that ClusterControl␣
↪shall configure on all database nodes. This user will be granted global␣
↪SUPER privilege and the GRANT option for localhost only.
   4. **Admin/Root password**. The password for **Admin/Root user**.
   5. **Repository**. Choose "Use vendor repositories" (default) will let␣
↪ClusterControl provision software by setting up and using the database␣
↪vendor's preferred software repository. ClusterControl will always␣
↪install the latest version of what is provided at that moment. Choose "Do␣
↪not setup vendor repositories" if you have a special configuration from␣
```

(continues on next page)

(continued from previous page)

```
↪the vendor (commonly for enterprise databases) and ClusterControl will␣
↪skip the repository configuration part.
       6. **Version**. Make sure that the cluster of the source backup␣
↪should match the version specified here.
   7. **Configuration template**. The dropdown loads all MySQL␣
↪configuration template files under `/etc/cmon/templates` or `/
↪usr/share/cmon/templates` of the ClusterControl node for this cluster␣
↪type. Choose one configuration template for the deployment. See [Configu-
ration␣
↪Template](../management/configuration-management.md#configuration-template).
   8. **Enable SSL encryption**. Toggle on (default) will instruct␣
↪ClusterControl to configure database SSL encryption for both␣
↪client-server and replication. Toggle off will make ClusterControl skip␣
↪this step.

11. Fourth step is **Add nodes**.

    1. **Galera node**. Type the IP address/hostname or valid FQDN of the␣
↪nodes you want to assign for this new cluster. Please note that an odd␣
↪number of nodes is recommended, i.e., 3, 5, 7, etc.

12. Click **Continue** to proceed the last step **Preview**.

13. The last step is **Preview**, this is the last step where it shows all␣
↪the values and choosen options for the ClusterControl to use for setting␣
↪up the backup.
```

ClusterControl CLI

- Create a three-node Percona XtraDB Cluster using version 8.0 from an existing backup.

```
    s9s cluster \
        --create \
        --cluster-type=galera \
        --
↪nodes="192.168.40.67?master;192.168.40.68?master;192.168.40.69?master" \
        --vendor=percona \
        --cluster-name=pxc_stage_env \
        --provider-version=8.0 \
        --backup-id=466 \
        --os-user=vagrant \
        --os-key-file=/home/vagrant/.ssh/id_rsa \
        --wait \
        --log
    ```

    - where `--
```

(continues on next page)
```

(continued from previous page)

```
↪nodes="192.168.40.67?master;192.168.40.68?master;192.168.40.69?master"`␣
↪are the ip address of the target nodes.
    - `--cluster-name=pxc_stage_env` parameter is the name of the cluster.
    - `--provider-version=8.0` defines the version of the PXC to be␣
↪installed.
    - `--backup-id=466` defines the selected backup ID for the source␣
↪backup to be used to create the cluster. The backup ID can be located in␣
↪the **All Backups** list view.
    - `--os-user=vagrant` and `--os-key-file=/home/vagrant/.ssh/id_rsa`␣
↪defines the OS user and its path of its public key
    - where the `--wait` and `--log` as the name suggest to print the job␣
↪logs while waiting.
```

2.65 Upload Backup to Cloud

Automated backups created using ClusterControl are supported to be offloaded from own premise to external cloud storage. By offloading your database backups to the cloud, you are ensured of redundancy, data durability, and potentially easier disaster recovery.

Supported clouds available are the following:

- Amazon S3
- Google Cloud Storage (GCS)
- Azure Blob Storage (through an S3-compatible interface)
- Other S3-compatible storage (for example, MinIO)

Enabling you to upload your backups to your cloud environment requires that you must have done integrating your *Cloud Providers*. If you have not done yet, please make sure you have added and setup integration for your *cloud storage credentials*.

2.65.1 Uploading backups to the cloud

All supported database clusters by ClusterControl shares the same approach when dealing with uploading of backups to the cloud. These procedures are tailored by ClusterControl to make it straightforward with usability through GUI clicks replacing those automated playbooks that DevOps or SysOps might have dealt with.

With that said, the step-by-step guide are provided thru the succeeding sections. Before moving forward thru the steps, there are important pointers for uploading backups to the cloud using ClusterControl as it has its limits. There are certain backup methods that are created using ClusterControl that does not provide the feature and this is tabulated in the next section.

2.65.2 Support Matrix

Most database clusters supported by ClusterControl include the backup verification feature. For the complete list, checkout below:

Database	Vendor	Topology	Backup Methods
MySQL	Percona, Oracle	Standalone, replication	`mysqldump`, `xtrabackup`
MariaDB	MariaDB	Standalone, replication	`mysqldump`, `mariabackup`
Galera Cluster	MariaDB, Percona	Galera certification-based replication	`mysqldump`, `xtrabackup`/`mariabackup`
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication	`pgdumpall`, `pg_basebackup`
TimescaleDB	TimescaleDB	Standalone, streaming replication	`pgdumpall`, `pg_basebackup`
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica Set	`rdb, aof`
Redis	Redis, Valkey	Sentinel, cluster	`rdb, aof`
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group	`mssqlfull`

2.65.3 Prerequisites

- Backup should be created using ClusterControl and from an existing cluster.
- Must have done setting up your *cloud storage credentials*
- supported cloud providers for object storage services are only AWS S3, Google Cloud Storage (GCS), Azure Blob Storage, and other S3-compatible storage (MinIO, DigitalOcean Spaces, etc.)
- Ensure proper network connectivity and interoperability between the controller, your database clusters, and your cloud providers.
- Ensure that you have appropriate user permissions and privileges to your bucket or target object storage location.

2.65.4 Default Configuration

- Retention period is set to 180 days if not specified.

2.65.5 Steps to Upload a Backup

As mentioned above, all supported database clusters shares the same approach and the steps to do it are applicable for all of them based on the *Support Matrix*. Although, if a backup is uploaded to the cloud, the UI applies a slight change.

This slight change shows a different approach in the step-by-step guide which is tackled in the next section of uploading a backup.

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Upload** option. This will display a prompt with all the procedures in one display to upload the backup.

5. In the prompt, the following elements are displayed:

    - **Backup ID**: This displays the backup ID assigned for the selected backup and also when this was created.
    - **Cloud Storage**: In this section, it will cover the elements required for all cloud storage information
        - **Credentials**: Select your named cloud credentials profile to use for authentication. This is the credentials you have already setup during setting up your integrations in *Cloud storage credentials* section.
        - **Bucket**: Select your existing bucket or type to create new.

```
                - **Location path**: Directory under bucket where backup␣
↪will be stored. Backup will be stored in bucket root directory if␣
↪location path is empty
                - **Retention period**: By default, it will set to 180␣
↪days. Specify a number to set the number of days you want your backup to␣
↪retain.
        - **Delete after upload**: This is **Off** by default. When␣
↪enabled, this means that delete local backup copy after it has been␣
↪uploaded to the cloud.

6. Click **Upload** button to execute and upload the backup to the cloud.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Backup Verification*
- *Upload Backup Cloud*

2.65.6 Steps to Re-upload a Backup (for backups already in the cloud)

As mentioned in the previous step-by-step section, there are slight changes when a backup is already uploaded and present in the cloud. This also shows that backups can be *downloaded* as an option. Backups that are already in the cloud can still be re-uploaded. Take for an example, if the backup that is uploaded already in the cloud but was accidentally deleted, the backup copy stored in the ClusterControl premise means that it is still be re-uploaded to overwrite its existing directory or file.

As showed earlier in the *Support Matrix*, most of the clusters supported upload to the cloud. However, one cluster that does not affected by this is **Microsoft SQL Server for Linux** clusters. Although it supports uploading to the cloud but download options is not yet currently supported. See *Download Backup From Cloud* for more details.

Make sure to follow the steps below to get through the uploading or re-uploading of backup to the cloud.

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (...).
4. Select and click **Cloud options**, then select and click **Upload** option button.
5. In the prompt, the following elements are displayed:
    - **Backup ID**: This displays the backup ID assigned for the selected backup and also when this was created.
    - **Cloud Storage**: In this section, it will cover the elements required for all cloud storage information
        - **Credentials**: Select your named cloud credentials profile to use for authentication. This is the credentials you have already setup during setting up your integrations in *Cloud storage credentials* section.
        - **Bucket**: Select your existing bucket or type to create new.

```
                - **Location path**: Directory under bucket where backup␣
↪will be stored. Backup will be stored in bucket root directory if␣
↪location path is empty
                - **Retention period**: By default, it will set to 180␣
↪days. Specify a number to set the number of days you want your backup to␣
↪retain.
        - **Delete after upload**: This is **Off** by default. When␣
↪enabled, this means that delete local backup copy after it has been␣
↪uploaded to the cloud.

6. Click **Upload** button to execute and upload the backup to the cloud.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Backup Verification*
- *Upload Backup Cloud*

2.66 Download Backup from Cloud

ClusterControl offers downloading backup from the cloud. Only existing backups that are allowed to be downloaded from the cloud. This means that downloading of backups are tightly coupled or source-bounded. It is locked only to that uploaded backuptaken using ClusterControl.

2.66.1 Support Matrix

Most database clusters supported by ClusterControl include the backup verification feature. For the complete list, checkout below:

Database	Vendor	Topology	Backup Methods
MySQL	Percona, Oracle	Standalone, replication	`mysqldump`, `xtrabackup`
MariaDB	MariaDB	Standalone, replication	`mysqldump`, `mariabackup`
Galera Cluster	MariaDB, Percona	Galera certification-based replication	`mysqldump`, `xtrabackup`/`mariabackup`
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication	`pgdumpall`, `pg_basebackup`
TimescaleDB	TimescaleDB	Standalone, streaming replication	`pgdumpall`, `pg_basebackup`
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica Set	`rdb, aof`
Redis	Redis, Valkey	Sentinel, cluster	`rdb, aof`

2.66.2 Prerequisites

- Backups uploaded from ClusterControl can only be downloaded.
- Download only to the same cluster where the backup was uploaded.
- Supported cloud providers for object storage services are only AWS S3, Google Cloud Storage (GCS), Azure Blob Storage, and other S3-compatible storage (MinIO, DigitalOcean Spaces, etc.).
- Ensure proper network connectivity and interoperability between the controller, your database clusters, and your cloud providers.
- Ensure that you have appropriate user permissions and privileges to your bucket or target object storage location.

2.66.3 Default Configuration

- Retention period is set to 180 days if not specified.

2.66.4 Steps to Download a Backup

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster then click.
2. Go to the **Backups** tab and make sure you click or set the view to **All Backups** (default).
3. Choose a backup you would like to restore and click the ellipsis button (…).
4. Select and click **Cloud options**, then select and click **Download** option button.
5. In the prompt, the following elements are displayed:
    - **Backup ID**: This displays the backup ID assigned for the selected backup and also when this was created.
    - **Download from**: This shows where the backup was uploaded and to what cloud it was stored too, which might also linked to your *cloud storage credentials* already setup.

- **Download to**: This is a drop-down element which shows you the nodes in your cluster including the controller. This allows you to choose what node shall store the backup to be downloaded.
- **Backup dir in host**: Specify the location of your backup directory to be stored. For example, `/opt/backps/aws-backups-download/`
- **Retention period**: By default, it will set to 180 days. Specify a number to set the number of days you want your backup to retain.

6. Click **Download** button to execute and download the backup from the cloud.

Recommended Next Steps

- *Backup Verification*
- *Upload Backup Cloud*

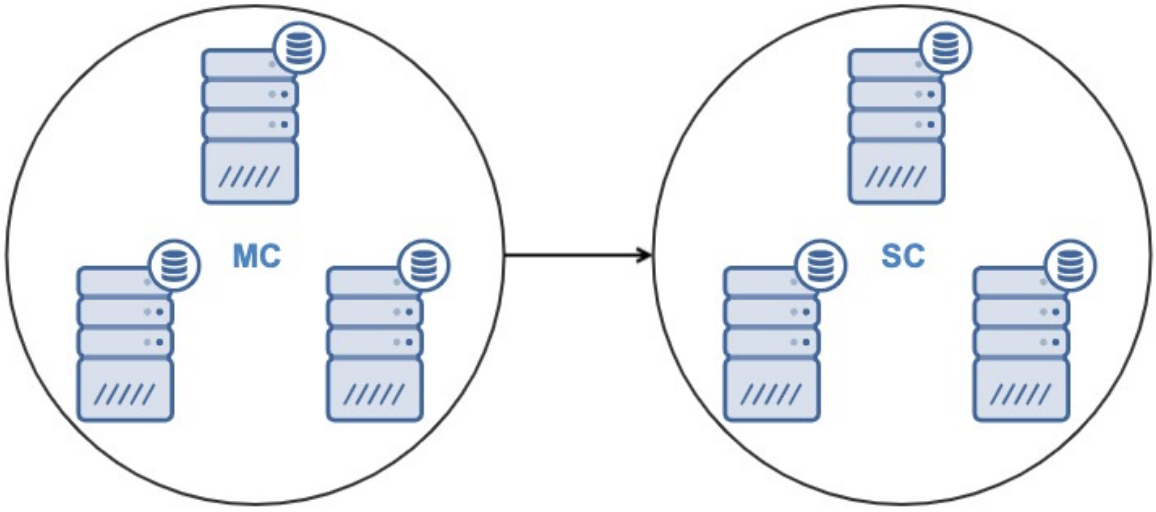
2.67 Create Replica Cluster

Create a replica cluster is basically cloning a cluster but it sets up a replication layer afterwards from your Source Cluster going to your target cluster (Replica Cluster). Creating a Replica Cluster does copy (or clones) the data from source including its configuration but ensuring that any required parameters such as hostname or IP address are replaced accordingly to have the replica cluster deployment running functional. After the data and configuration is setup on the Slave Cluster, cmon will handle the replication setup to link both clusters to communicate (that is bidirectional replication).

With this feature:

- One node in each cluster will replicate from another node in the other cluster.
- As mentioned earlier, replication will be bi-directional (or uni-directional) between the clusters.
- All nodes in the replica cluster will be read-only by default.
- Active-active clustering is only recommended if applications are only touching disjoint data sets on either cluster since the engine doesn't offer any conflict detection or resolution.

The following diagram shows you how does it look like for an ideal scenario of a Cluster-to-Cluster replication, where MC as Source Cluster and SC as Replica Cluster:



2.67.1 Support Matrix

The following database cluster types, vendors and topology are supported:

Database	Vendor	Topology	Version Supported
Galera Cluster	Percona XtraDB Cluster (PXC)	Galera certification-based replication	5.6.x and later
Galera Cluster	MariaDB Galera	Galera certification-based replication	5.6.x and later
PostgreSQL	PostgreSQL	Standalone, streaming replication	9.6 and later
TimescaleDB	TimescaleDB	Standalone, streaming replication	9.6 and later

2.67.2 Prerequisites

There are prerequisites that need to be fulfilled prior to the deployment of a replica cluster. See list below:

- Make sure the target database nodes are running on a supported architecture platform and operating system. See *Hardware* and *Operating System*.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to all database nodes. See *SSH Key-based Authentication*.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.
- The database credentials for admin user/password should be the same from Source Cluster.

2.67.3 Create Replica Cluster Options

When creating a replica cluster, you have two options to choose from:

- **Stream data from primary cluster**. This option allows you to setup your desired job to clone or copy the data from the primary or Source Cluster via streaming. This means that, copying of data is happening on-demand and this shall add traffic and load to your Source Cluster.
- **Stage cluster from a backup**. For large dataset or a cluster serving on a high-traffic load, creating a cluster by staging from a backup would be ideal.

2.67.4 Galera Cluster

A minimal setup is comprised of one Galera node (no high availability). Recommended setup should be at least 3-nodes at bare minimum for high availability.

The following vendors and versions are supported for a new deployment:

- Percona XtraDB Cluster - 8.0.
- MariaDB Cluster - 10.5, 10.6, 10.11, 11.4 and 11.8.

Choosing stream data from primary cluster

Streaming data from primary cluster on a Galera Cluster shall do the following:

- For Percona XtraDB Cluster, it will use Percona Xtrabackup to stream data from its source copying its data to the target node, that is the replica cluster.

- For MariaDB Galera, it will use Mariabackup to stream data from its source copying its data to the target node, that is the replica cluster.

Default configuration

Creating a replica cluster, the ClusterControl will simply copy the configuration from the Source Cluster, leaving most everything set based on the changes or its default values given during deployment of the cluster (see *Default Configuration* for deploying a cluster). Other than that, create replica cluster shall have these default configuration but can be modified or changed in the GUI. See the following list below:

- Bi-directional replication is enabled or **"On"**
- Disable firewall is checked
- Disable SELinux/AppArmor is checked
- Default SSH port is 22
- Default database port is 3306
- Shall use vendor repositories when pulling and instaling packages
- After Replica Cluster is deployed, auto-recovery (Node and Cluster) are enabled.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Clone/Replicate the cluster  → Create replica cluster**

3. Choose **Stream data from primary cluster** to open the GUI for␣
↪**Replicate from** **&lt;cluster_name&gt;** GUI..

4. First step is **Cluster details**.

    1. Specify on the **Select primary** at this step. In the **Cluster␣
↪replication primary**, choose the target primary node where replication␣
↪link shall be established.

    2. **Bi-directional replication** is set to **On**. You can turn it off␣
↪or disable it to have only unidirectional replication.

    3. **Name your cluster**. In the **Name** field, it is optional but␣
↪fill-in here the desired name of the cluster.

    4. **Tags** is a drop-down field but can accept input fields. Any␣
↪existing tags in the drop-down will show up, otherwise, input the new tag␣
↪to be added.

5. Click **Continue**.

6. Second step is **SSH configuration**.
```

(continues on next page)

(continued from previous page)

```
   1. Specify the **SSH Credentials** at this step. In the **SSH user**,␣
↪input the SSH user, then **SSH user key path** for the key path (for␣
↪example /home/clustercontrol/id_rsa.pub).
       Specify the **SSH port** if the port is not using the default 22.␣
↪Specify the **SSH sudo password** if there is a sudo password for the SSH␣
↪user being used here. Specify the option for OS elevation command (`sudo`,
↪ `doas`, `pbrun`).

   2. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

   3. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

7. Click **Continue**.

8. Third step is **Node configuration**.

   1. Specify the **Node configuration** at this step. Specify the␣
↪**Server port** if it's not using default port 3306.
       Specify the **Server data directory** if it's not using default `/
↪var/lib/mysql`.

   2. Specify the database admin user/password. Make sure to have the␣
↪value of the admin user and password from the source cluster.
       Specify the database admin user in **Admin/Root user** field.␣
↪Specify the database admin password in *Admin/Root password** field.

   3. Choose the repository in the **Repository** section. Default is set␣
↪to **Use vendor repositories**. Other options are `Do not setup vendor␣
↪repositories` and `Create new repository`.
       If choosing `Do not setup vendor repositories`, this shall not␣
↪install the vendor repository leaving to use AppStream or your main␣
↪repository.
       If choosing `Create new repository`, this shall create/mirror the␣
↪vendor's repositories. Deploy with the mirrored repositories.

   4. Specify the **Version** of your desired database. When creating a␣
↪replica, it is best to determine the source version including the minor␣
↪version number of the Source Cluster.
```

(continues on next page)

(continued from previous page)

```
    5. Select the **Configuration template**. ClusterControl will show the␣
↪templates that has been added or configured in `/etc/cmon/templates`. For␣
↪more details see [Configuration␣
↪template](../management/configuration-management.md#configuration-template)

    6. By default, **Enable SSL encryption** is enabled or **On**. If setup␣
↪does not require SSL (not recommended), opt-out to disable it.

    7. By default, **Create cluster as readonly** is enabled or **On**.␣
↪Disable it if Replica Cluster is writable.

9. Click **Continue**.

10. Fourth step is **Add nodes**.

    1. Specify the **Galera node** that will be added and setup as the␣
↪Replica Cluster. Please note that an odd number of nodes is recommended,␣
↪i.e., 3, 5, 7, etc.
       On this part, input the hostname or IP address of the node.␣
↪Ideally, the cmon will setup the first node added as the node to be␣
↪acting as a replica of the primary node from the Source Cluster.
       Make sure that this node has enough resources to handle the␣
↪replication link from the Source Cluster.

11. Click **Continue** to proceed the last step **Preview**.

12. Fifth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the Replica Cluster.

13. Click **Finish** button to deploy the job.
```

2.67.5 PostgreSQL

A minimal setup is comprised of one PostgreSQL standalone primary node (no high availability). For most setup, recommended setup should have at least one replica or standby node. The following vendors and versions are supported for a new deployment:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, 16 and 17.
- PostgreSQL from EDB repository - 12, 13, 14, 15, 16 and 17 (requires a valid EDB repository token).

Choosing stream data from primary cluster

Streaming data from the Source Clusters on a PostgreSQL should be established in the primary node since it will act as the primary node for replication. ClusterControl will use pg_basebackup to copy and stream from primary node (the primary node from your Source Cluster) to the replica node (primary node in the Replica Cluster).

Default configuration

Creating a replica cluster, the ClusterControl will simply copy the configuration from the Source Cluster, leaving most everything set based on the changes or its default values given during deployment of the cluster (see *Default Configuration* for deploying a cluster). Other than that, create replica cluster shall have these default configuration but can be modified or changed in the GUI. See the following list below:

- Bi-directional replication is enabled or **"On"**
- Disable firewall is checked
- Disable SELinux/AppArmor is checked
- Default SSH port is 22
- Default database port is 5432
- Shall use vendor repositories when pulling and instaling packages
- After Replica Cluster is deployed, auto-recovery (Node and Cluster) are enabled.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Clone/Replicate the cluster  → Create replica cluster**

3. Choose **Stream data from primary cluster** to open the GUI for␣
↪**Replicate from** **&lt;cluster_name&gt;** GUI..

4. First step is **Cluster details**.

    1. Specify on the **Select primary** at this step. In the **Cluster␣
↪replication primary**, choose the target primary node where replication␣
↪link shall be established.

    2. **Bi-directional replication** is set to **On**. You can turn it off␣
↪or disable it to have only unidirectional replication.

    3. **Name your cluster**. In the **Name** field, it is optional but␣
↪fill-in here the desired name of the cluster.

    4. **Tags** is a drop-down field but can accept input fields. Any␣
↪existing tags in the drop-down will show up, otherwise, input the new tag␣
↪to be added.

5. Click **Continue**.

6. Second step is **SSH configuration**.

    1. Specify the **SSH Credentials** at this step. In the **SSH user**,␣
↪input the SSH user, then **SSH user key path** for the key path (for␣
↪example /home/clustercontrol/id_rsa.pub).
```

(continues on next page)

(continued from previous page)

```
       Specify the **SSH port** if the port is not using the default 22.␣
↪Specify the **SSH sudo password** if there is a sudo password for the SSH␣
↪user being used here. Specify the option for OS elevation command (`sudo`,
↪ `doas`, `pbrun`).

   2. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

   3. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

7. Click **Continue**.

8. Third step is **Node configuration**.

   1. Specify the **Node configuration** at this step. Specify the␣
↪**Server port** if it's not using default port 3306.
       Specify the **Server data directory** if it's not using default `/
↪var/lib/mysql`.

   2. Specify the database admin user/password. Make sure to have the␣
↪value of the admin user and password from the source cluster.
       Specify the database admin user in **Admin/Root user** field.␣
↪Specify the database admin password in *Admin/Root password** field.

   3. Choose the repository in the **Repository** section. Default is set␣
↪to **Use vendor repositories**. Other options are `Do not setup vendor␣
↪repositories` and `Create new repository`.
       If choosing `Do not setup vendor repositories`, this shall not␣
↪install the vendor repository leaving to use AppStream or your main␣
↪repository.
       If choosing `Create new repository`, this shall create/mirror the␣
↪vendor's repositories. Deploy with the mirrored repositories.

   4. By default, **Enable SSL encryption** is enabled or **On**. If setup␣
↪does not require SSL (not recommended), opt-out to disable it.

9. Click **Continue**.

10. Fourth step is **Add nodes**.
```

(continues on next page)

(continued from previous page)

```
    1. Specify the **Primary node** that will be added and setup as the␣
↪Replica Cluster.

        2. If there are replicas to be setup and added for the Replica␣
↪Cluster, fill in the hostname or IP address in the **Replica nodes**␣
↪field. Make sure to hit enter or
        click the **+** button.

11. Fifth step is **Extensions**. By default, **pgvector** and **PostGIS**␣
↪extensions that ClusterControl supports are disabled.

    1. If you want **pgvector** to be enabled, switch the toggle button to␣
↪**On**

        2. If you want **PostGIS** to be enabled, switch the toggle button␣
↪to **On**

12. Click **Continue** to proceed the last step **Preview**.

13. Sixth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the Replica Cluster.

14. Click **Finish** button to deploy the job.
```

2.67.6 TimescaleDB

A minimal setup is comprised of one TimescaleDB standalone primary node (no high availability). Recommended setup should have at least one replica or standby node. The following vendors and versions are supported for a new deployment:

- TimescaleDB - 12, 13, 14, 15 and 16.

Choosing stream data from primary cluster

Streaming data from the Source Clusters on a TimescaleDB should be established in the primary node since it will act as the primary node for replication. ClusterControl will use pg_basebackup to copy and stream from primary node (the primary node from your Source Cluster) to the replica node (primary node in the Replica Cluster).

Default configuration

Creating a replica cluster, the ClusterControl will simply copy the configuration from the Source Cluster, leaving most everything set based on the changes or its default values given during deployment of the cluster (see *Default Configuration* for deploying a cluster). Other than that, create replica cluster shall have these default configuration but can be modified or changed in the GUI. See the following list below:

- Bi-directional replication is enabled or **"On"**
- Disable firewall is checked
- Disable SELinux/AppArmor is checked

- Default SSH port is 22
- Default database port is 5432
- Shall use vendor repositories when pulling and instaling packages
- After Replica Cluster is deployed, auto-recovery (Node and Cluster) are enabled.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Clone/Replicate the cluster  → Create replica cluster**

3. Choose **Stream data from primary cluster** to open the GUI for␣
↪**Replicate from** **&lt;cluster_name&gt;** GUI..

4. First step is **Cluster details**.

    1. Specify on the **Select primary** at this step. In the **Cluster␣
↪replication primary**, choose the target primary node where replication␣
↪link shall be established.

    2. **Bi-directional replication** is set to **On**. You can turn it off␣
↪or disable it to have only unidirectional replication.

    3. **Name your cluster**. In the **Name** field, it is optional but␣
↪fill-in here the desired name of the cluster.

    4. **Tags** is a drop-down field but can accept input fields. Any␣
↪existing tags in the drop-down will show up, otherwise, input the new tag␣
↪to be added.

5. Click **Continue**.

6. Second step is **SSH configuration**.

    1. Specify the **SSH Credentials** at this step. In the **SSH user**,␣
↪input the SSH user, then **SSH user key path** for the key path (for␣
↪example /home/clustercontrol/id_rsa.pub).
       Specify the **SSH port** if the port is not using the default 22.␣
↪Specify the **SSH sudo password** if there is a sudo password for the SSH␣
↪user being used here. Specify the option for OS elevation command (`sudo`,
↪ `doas`, `pbrun`).

    2. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    3. **Security configuration** to either disable firewall or␣
```

(continues on next page)

(continued from previous page)

```
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

7. Click **Continue**.

8. Third step is **Node configuration**.

    1. Specify the **Node configuration** at this step. Specify the␣
↪**Server port** if it's not using default port 3306.
       Specify the **Server data directory** if it's not using default `/
↪var/lib/mysql`.

    2. Specify the database admin user/password. Make sure to have the␣
↪value of the admin user and password from the source cluster.
       Specify the database admin user in **Admin/Root user** field.␣
↪Specify the database admin password in *Admin/Root password** field.

    3. Choose the repository in the **Repository** section. Default is set␣
↪to **Use vendor repositories**. Other options are `Do not setup vendor␣
↪repositories` and `Create new repository`.
       If choosing `Do not setup vendor repositories`, this shall not␣
↪install the vendor repository leaving to use AppStream or your main␣
↪repository.
       If choosing `Create new repository`, this shall create/mirror the␣
↪vendor's repositories. Deploy with the mirrored repositories.

    4. By default, **Enable SSL encryption** is enabled or **On**. If setup␣
↪does not require SSL (not recommended), opt-out to disable it.

9. Click **Continue**.

10. Fourth step is **Add nodes**.

    1. Specify the **Primary node** that will be added and setup as the␣
↪Replica Cluster.

       2. If there are replicas to be setup and added for the Replica␣
↪Cluster, fill in the hostname or IP address in the **Replica nodes**␣
↪field. Make sure to hit enter or
       click the **+** button.

11. Fifth step is **Extensions**. By default, **pgvector** and **PostGIS**␣
↪extensions that ClusterControl supports are disabled.
```

(continues on next page)

(continued from previous page)

```
    1. If you want **pgvector** to be enabled, switch the toggle button to␣
↪**On**

        2. If you want **PostGIS** to be enabled, switch the toggle button␣
↪to **On**

12. Click **Continue** to proceed the last step **Preview**.

13. Sixth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the Replica Cluster.

14. Click **Finish** button to deploy the job.
```

2.68 Add Database Node

Adding a database node in ClusterControl is similar to adding a database node or adding a primary node. To simplify the terminology in this page, the name **Add Node** is simply adding a primary database node or denoting *database node* is simply a primary database node.

A primary node denotes that it is a database node ready for accepting read and writes. In contrast with *Add Replica Node*, the Add Database Node refers only to a primary database node. If you are looking to add a replica instead, see *Add Replica Node*.

Adding a database node is a common goal in ClusterControl for improving the resilience, read scalability, and availability of your database systems; especially for having a disaster recovery located on a separate region, collocation, or in different geography. ClusterControl simplifies this process significantly by eliminating the hassle and manual work.

Info

Adding a database node stepwise, and that you can only add one node in the course of action. You can run an iterative process by automating this with ClusterControl CLI.

2.68.1 Support Matrix

The following database cluster types, vendors and topology supports adding a database node:

Database	Vendor	Topology
Galera Cluster	MariaDB, Percona	Galera certification-based replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.68.2 Prerequisites

When ading a node, it is very important to determine and prepare your host to have the requirements such as hardware, network, and user that is necessary to be accessed by ClusterControl and other database nodes.

There are prerequisites that need to be fulfilled prior to the deployment for all database clusters:

- Make sure the target database nodes are running on a supported architecture platform and operating system. See *Hardware* and *Operating System*.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to all database nodes. See *SSH Key-based Authentication*.
- Make sure that the OS user you setup for passwordless login is the same user your ClusterControl and its cluster used during deployment.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.

2.68.3 MySQL Galera

The following vendors and versions are supported for adding database node:

- Percona XtraDB Cluster - 8.0.
- MariaDB Galera - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS) and 11.4 (LTS).

Adding a database node requires that your existing cluster must have at least one node running. When the job is in progress, ClusterControl will rely on the Galera's mechanism to do a full SST from source to the target node (database node to be setup and added). This means that ClusterControl relies on binary backup to stream data to copy from source to the target database. Binary backup does not lock your primary node once a new node is added. However, If you are concerned of the traffic and/or the data to be copied from your Primary node is big, we recommended to add it during low peak period or use the *backup created* taken using ClusterControl. This shall be discussed on how to rebuild from backup in the following section below.

Below is how ClusterControl relies on Galera's mechanism to apply a full SST:

- For Percona XtraDB Cluster, it will use Percona XtraBackup to stream data from its existing primary to the target node.
- For MariaDB Server, it will use Mariabackup to stream data from its its existing primary to the target node.

This is set through `wsrep_sst_method` upon deployment.

Default configuration

By default, ClusterControl deploys MySQL/MariaDB database with the following configurations:

- During the job process, ClusterControl disables cluster and node recovery. It will re-instate its former values accordingly.
- ClusterControl sets `wsrep_sst_method` to `xtrabackup` for Perocna XtraDB Cluster or `mariabackup` for MariaDB Galera.
- MySQL GTID with `log_slave_updates` enabled (Oracle MySQL and Percona only).
- MariaDB GTID with `log_slave_updates` enabled (MariaDB only).
- a database node shall act as a primary having `read_only=OFF` and `super_read_only=OFF` (if supported).

- `performance_schema` is enabled.
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.

Note

Note If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Primary node**

3. Choose **Create a primary node**

4. First step is **Node configuration**.

    1. **Data directory** is set to `/var/lib/mysql`. If you have prepared␣
↪a separate path or on a different volume, replace the value with your␣
↪custom path.

    2. **Galera Segment** is set to `0` by default. If you are adding a␣
↪node into different segment, set the value here i.e. > 0. See␣
↪[here](https://severalnines.com/blog/using-mysql-galera-cluster-replication-create-geo-
↪for more about [segments in␣
↪Galera](https://severalnines.com/blog/using-mysql-galera-cluster-replication-create-geo

    3. **Configuration template** is set to `my.cnf.[galera-version]galera`␣
↪where _[galera-version]_ entails the vendor and version of Galera␣
↪ClusterControl intends to add this node. If you have your own custom␣
↪template, ClusterControl will show the templates that has been added or␣
↪configured in `/etc/cmon/templates`. For more details see [Configuration␣
↪template](../management/configuration-management.md#configuration-template)

    4. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package version instead.

    5. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
```

(continues on next page)

(continued from previous page)

```
↪run properly.

5. Click **Continue**.

6. Second step is **Advanced Settings**.

    1. **Rebuild from a backup** is disabled. If you desire to rebuild your␣
↪data instead of streaming from the Primary node, toggle it to **On**.␣
↪This shall reveal a drop-down list with all full-backup created using␣
↪Clustercontrol.

    2. **Include in LoadBalancer set (if exists)** is disabled. Switch it␣
↪to **On* if you want ClusterControl to include this new node in the␣
↪existing node load balancing set.

7. Click **Continue**.

8. Third step is **Add node**.

    1. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target node. If you intend to add␣
↪another one, this will just overwrite what was previously inputted and␣
↪added since it only allows one node at a time.

9. Click **Continue** to proceed the last step **Preview**.

10. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

11. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Add Replica Node*
- *Create Replica Cluster*

2.68.4 MongoDB Replica Set

The following vendors and versions are supported for adding database node:

- MongoDB Enterprise - 4.4, 5.0, 6.0, 7.0 and 8.0
- MongoDB Community - 4.4, 5.0, 6.0, 7.0 and 8.0
- Percona Server for MongoDB - 4.4, 5.0, 6.0, 7.0 and 8.0

Adding a database node requires that your existing cluster must have at least one node running as primary. If no primary found, ClusterControl will eventually fail the job process since it will fail to identify an elibible primary in that replica set.

When the job is in progress, ClusterControl will execute the `rs.add()` function to add the new node to its target replica set. If you are concerned of the traffic and/or the data to be copied from your Primary node is big, we recommended to add it during low peak period.

Once the adding a node job is done, the new node shall have **Role** of **Secondary** once viewed in the **Nodes** tab.

Default configuration

By default, ClusterControl deploys MongoDB Replica Set members with the following configurations:

- Configure `setParameter.enableLocalhostAuthBypass: true` inside MongoDB configuration file.
- ClusterControl will create and grant necessary roles for additional MongoDB users - `admin.cmon_backup` for backup and restore purposes and `admin.cmonexporter` for query monitoring.
- Configure `sharding.clusterRole: shardsvr` inside MongoDB configuration file.

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template* for details.

Warning

Attention

It is possible to deploy only a DB server only that has one primary database node already in the replica set and no other nodes or arbiter. The caveat of this approach is no automatic failover. If the primary node goes down then manual failover is required to make the remaining secondary node run as primary. Automatic failover works fine with 3 nodes and more.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Primary node**

3. Choose **Create a primary node**

4. First step is **Node configuration**.

    1. **Node type** has values `Db server` and `Arbiter`. Choose `Db␣
↪server` to add a primary database node.

    2. **Replica set** is a drop-down list. When click this shall reveal␣
↪replica set created in the cluster. Make sure you know the replica set␣
↪you are intending this node to be assigned and choose it as you desire.
```

(continues on next page)

(continued from previous page)

```
    3. Specify the **Port** of your target replica node. Default is set to␣
↪27017.

    4. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    5. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

5. Click **Continue**.

6. Third step is **Add node**.

    1. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target node. If you intend to add␣
↪another one, this will just overwrite what was previously inputted and␣
↪added since it only allows one node at a time.

7. Click **Continue** to proceed the last step **Preview**.

8. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

9. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

2.68.5 MongoDB Sharded Cluster

The following vendors and versions are supported for adding database node:

- MongoDB - 4.4, 5.0, 6.0, 7.0 and 8.0
- MongoDB Enterprise - 4.4., 5.0, 6.0, 7.0 and 8.0
- Percona - 4.4, 5.0, 6.0, 7.0 and 8.0

Adding a database node requires that your existing MongoDB Sharded Cluster must have at least one node running as primary. If no primary found, ClusterControl will eventually fail the job process since it will fail to identify an eligible primary in that replica set. Whether this replica set is a Config Server or Shard Server, it requires a primary node to run healthy so that the new node can join the cluster.

When the job is in progress, ClusterControl will execute the `rs.add()` function to add the new node to its target replica set. If you are concerned of the traffic and/or the data to be copied from your Primary node is big, we recommended to add it during low peak period. If you are going to add a router or `mongos`, this can be quick since router will act as your load balancer and does not store data. ClusterControl can configure quickly and does not have any impact at all to your database nodes or existing MongoDB cluster.

Once the adding a node job is done, the following roles shall be assigned:

- **Config Server**. This role is assigned if you added a Config server node
- **Mongos**. This role is assigned if you added a router/mongos node. This is your load balancer.
- **Primary/Secondary**. If you have added the node as part of your sharded server, then primarily, the new node shall have **Secondary** since your existing cluster shall have an assigned primary role assigned.

Default configuration

By default, ClusterControl deploys MongoDB Sharded Cluster with the following configurations:

- Configure `setParameter.enableLocalhostAuthBypass: true` inside MongoDB configuration file.
- ClusterControl will create and grant necessary roles for additional MongoDB users - `admin.cmon_backup` for backup and restore purposes and `admin.cmonexporter` for query monitoring.
- Configure `sharding.clusterRole: shardsvr` inside MongoDB configuration file if node added is a DB server and part of the shard.
- Configure `sharding.clusterRole: configsvr` inside MongoDB configuration file if node added is a Configuration server.
- sets the `sharding.configDB` to the replica set config servers when adding a router/mongos.
- sets `security.keyFile: /etc/mongo-cluster.key` when adding a router/mongos.

Warning

Attention

It is possible to deploy nodes either router, config, or database node with less than 3 nodes. However, for high availability and no single-poit of failure (SPOF), we recommend to make sure to have at least 2 or 3 nodes per shard, at least 2 nods for config server, and 2 nods for your router/mongos.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (...)

```
2. Choose **Add new  → Node**

3. Choose **Create a database node**

4. First step is **Node configuration**.
```

(continues on next page)

(continued from previous page)

```
   1. **Create router/mongos**. By default, this is `Off`.

   2. **Node type** has values `Db server`, `Config server`, and `Arbiter`.
↪ Choose `Db server` to add a primary database node with shardsvr role.␣
↪Choose `Config server` if you add it as a `configsvr` role, or arbiter as␣
↪the name suggest.

   2. **Replica set** is a drop-down list. When click this shall reveal␣
↪replica set created in the cluster. Make sure you know the replica set␣
↪you are intending this node to be assigned and choose it as you desire.

   3. Specify the **Port** of your target replica node. Default is set to␣
↪27017. Make sure that you set the port correctly to which port you are␣
↪aiming or to make it uniform as what your existing node roles are using,␣
↪but not required instead it's recommended.

   4. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

   5. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

5. Click **Continue**.

6. Third step is **Add node**.

   1. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target node. If you intend to add␣
↪another one, this will just overwrite what was previously inputted and␣
↪added since it only allows one node at a time.

7. Click **Continue** to proceed the last step **Preview**.

8. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

9. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

2.68.6 Redis Sentinel

The following vendors and versions are supported for adding database node:

- Redis Sentinel - 6 and 7

Essentially, as mentioned earlier, adding a database node in this page is simply adding a primary database node. However, in this case, action goes for adding a node but the end result, the newly database node added shall act as a replica, instead of a primary. In this manner, adding a database node requires a primary node in the course of action in order to complete the job successfully.

If the existing primary is down, job can still be done successfully. However, the primary node needs to be up sometime so that the replica can send a `PSYNC` to the primary node when its up and it can start syncing its data from the primary going to be replicated down to the replica node (the node that had just been added).

Apart from adding your database node as secondary, ClusterControl will also deploy Sentinel node which shall run on port 26379.

Default Configuration

By default, ClusterControl deploys Redis cluster instances with the following configurations:

- Default port points to `6379` for the Redis server and port `26379` for Sentinel service.
- ClusterControl will configure the Redis instances with `appendonly` enabled.
- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.
- Deploys sentinel node co-located within the same node (Redis instance) being added.

!!! Tips

```
Redis Sentinel requires 3 nodes for automatic primary promotion. When you␣
↪have only one-single Redis Sentinel node, make sure you add two or make␣
↪sure to have an odd total number of nodes per cluster.
```

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add node**

3. First step is **Node configuration**.

```

(continues on next page)

(continued from previous page)

```
    1. Specify the **Port** of your target database (replica) node. Default␣
↪is set to `6379`.

    2. Specify the **Redis sentinel port** for your sentinel service.␣
↪Default is set to `26379`.

    3. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    4. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

4. Click **Continue**.

5. Third step is **Add node**.

    1. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target node. If you intend to add␣
↪another one, this will just overwrite what was previously inputted and␣
↪added since it only allows one node at a time.

6. Click **Continue** to proceed the last step **Preview**.

7. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

8. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

2.68.7 Redis/Valkey Cluster

The following vendors and versions are supported for adding a database node:

- Redis Cluster - 6 and 7
- Valkey Cluster - 7 and 8 (only RHEL 8/9 are supported)

Adding a database node (primary database node) requires a running and healthy primary node in the course of action in order to complete the job successfully. When the job is in progress, ClusterControl will setup

the database node then executes CLUSTER MEET and rebalances the slots in the cluster to ensure a more even distribution across all nodes.

Default Configuration

By default, ClusterControl deploys Redis cluster instances with the following configurations:

- ClusterControl will configure the Redis/Valkey instances with `appendonly` enabled.
- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.
- ClusterControl uses cluster bus port i.e. main-port + 1000 unless manually specified during *cluster deployment*.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new → Primary Node**

3. First step is **Node configuration**.

    1. Specify the **Port** of your target database node. Default is set to␣
↪`6379`.

    2. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    3. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
        uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

4. Click **Continue**.

5. Third step is **Add node**.

    1. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target node. If you intend to add␣
```

(continues on next page)

(continued from previous page)

```
↪another one, this will just overwrite what was previously inputted and␣
↪added since it only allows one node at a time.

6. Click **Continue** to proceed the last step **Preview**.

7. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

8. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

2.68.8 SQL Server

The following versions for Microsoft SQL Server are supported for adding a database node:

- Microsoft SQL Server for Linux - 2019 and 2022

Adding a database node will deploy a secondary database node. In this action, ClusterControl will setup the node as part of the Always On Availability Groups (AGs). This requires to have a running and healthy primary node in the course of action in order to complete the job successfully.

Warning

Attention

Only hostname or FQDN is supported. Therefore, proper host naming and mapping must be performed beforehand. You may use `/etc/hosts` or DNS mapping to achieve this. When adding a database node in ClusterControl, entering an IP address will produce an error.

License

Deploying SQL Server with ClusterControl does not change how you must license SQL Server. Each node where SQL Server is running requires appropriate licensing, just as it would if you deployed SQL Server manually. The presence of ClusterControl as an orchestration or management layer has no bearing on Microsoft's licensing rules.

When deploying SQL Server in a clustered environment, licensing rules still apply to each server running SQL Server. However, the specifics differ depending on whether the node is actively running SQL Server or is a passive failover node:

- Active Nodes: Each active node that runs SQL Server services needs to be fully licensed.
- Passive Failover Nodes: Under current SQL Server licensing rules, you can typically have one passive failover node per licensed instance without needing an additional license for that passive node. The passive node is considered a "standby" server that can take over if the active server fails. It must truly remain passive, running no active workloads except those required for maintaining the standby state.

Obtain your SQL Server license directly from Microsoft, a certified Microsoft Partner, or a cloud marketplace (if applicable).

Default configuration

By default, ClusterControl deploys SQL Server with the following configurations:

- All database nodes must use fully qualified domain name (FQDN). IP address is not supported.
- Enforces the SQL Server user's password policy as shown here.
- At the moment, ClusterControl only deploys AlwaysOn with asynchronous-commit mode, where it does not wait for any secondary replica to write incoming transaction log records to disk.
- Sets `hadrenabled = 1` in `/var/opt/mssql/mssql.conf` config.
- Enables sqlagent in `/var/opt/mssql/mssql.conf` config.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add node**

3. First step is **Node configuration**.

    1. **Port** of your target database node. It is set to `1433` and␣
↪currently, this is disabled.

    2. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    3. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
        uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

4. Click **Continue**.

5. Third step is **Add node**.

    1. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target node. If you intend to add␣
↪another one, this will just overwrite what was previously inputted and␣
↪added since it only allows one node at a time.

6. Click **Continue** to proceed the last step **Preview**.

7. Fourth and the last step is **Preview**, this is the last step where it␣
```

(continues on next page)

(continued from previous page)

```
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

8. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

2.69 Add Replica Node

In ClusterControl, scaling an existing cluster to add more nodes for high availability can be done by adding a replica. Adding a replica in ClusterControl is a common task for improving the resilience, read scalability, and availability of your database systems.

ClusterControl simplifies this process significantly eliminating hassle and manual work. This page will guide you add replica node based on the database cluster that ClusterControl supports.

2.69.1 Support Matrix

The following database cluster types, vendors and topology are supported:

Database	Vendor	Topology
MySQL	Percona, Percona Server for MySQL Pro, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
Redis	Redis, Valkey	Sentinel, cluster

2.69.2 Prerequisites

Same as deploying a new cluster, nodes needs to be provisioned based on the requirements that ClusterControl requires in order to have a deployment for adding node successful. See list below:

- Make sure the target database nodes are running on a supported architecture platform and operating system. See *Hardware* and *Operating System*.
- Passwordless SSH (SSH using key-based authentication) is configured from the ClusterControl node to all database nodes. See *SSH Key-based Authentication*.
- Make sure that the OS user you setup for passwordless login is the same user your ClusterControl and its cluster used during deployment.
- Verify that sudo is working properly if you are using a non-root user. See *Operating System User*.

Percona Server for MySQL Pro - 8.0.

Make sure you have acquired the Percona Client ID and Percona Access Token.

PostgreSQL Entperise DB (EDB)

Make sure you have acquired a valid EDB repository token.

2.69.3 MySQL Replication

The following vendors and versions are supported for adding replica node:

- Oracle MySQL - 8.0.
- Percona Server for MySQL - 8.0.
- Percona Server for MySQL Pro - 8.0.
- MariaDB Server - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS), 11.4 (LTS) and 11.8.

Adding a replication requires a running and healthy primary node in the course of action in order to complete the job successfully. When the job is in progress, ClusterControl will copy the data from the primary but this shall not lock the primary node since ClusterControl uses binary backup as explained below:

- For Oracle MySQL and Percona Server (either Community edition or Pro), it will use Percona Xtrabackup to stream data from its primary to the target replica node.
- For MariaDB Server, it will use Mariabackup to stream data from its primary to the target replica node.

If you are concerned of the traffic and/or the data to be copied from your Primary node is large, it is recommended to use it during low peak period or rebuild your replica node using *backup created* and taken using ClusterControl. This shall be discussed on how to rebuild from backup in the following section below.

Default configuration

By default, ClusterControl deploys MySQL/MariaDB replication with the following configurations:

- ClusterControl uses `netcat` or `nc` to stream data using a binary backup (`xtrabackup or mariabackup`). It uses port 9999, or if used, it will search between 9990-9998 range and will use the next port available.
- MySQL GTID with `log_slave_updates` enabled (Oracle MySQL and Percona only).
- MariaDB GTID with `log_slave_updates` enabled (MariaDB only).
- Replica node shall be configured with `read_only=ON` and `super_read_only=ON` (if supported) for protection against accidental write.
- Semi-sync replication is enabled
- `performance_schema` is enabled
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- Since ClusterControl will copy its data from the Primary node which means users will be copied as well, ClusterControl will add one user with its IP address or hostname to grant necessary privileges for replication:

```
| Database user      | Purpose                                          |
| ------------------ | ------------------------------------------------ |
| `rpl_user`         | MySQL/MariaDB replication.                       |
```

Note

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template*.

Warning

Attention

ClusterControl sets `read_only=ON` on all slaves but a privileged user (SUPER) can still write to a slave (except for MySQL versions that support `super_read_only`).

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Replica node**

3. Choose **Create a replica node**

4. First step is **Node configuration**.

    1. Specify the **Port** of your target replica node. Default is set to␣
↪3306.

    2. **Data directory** is set to `/var/lib/mysql`. If you have prepared␣
↪a separate path or on a different volume, replace the value with your␣
↪custom path.

    3. **Configuration template** is set to `my.cnf.repl84`. If you have␣
↪your own custom template, ClusterControl will show the templates that has␣
↪been added or configured in `/etc/cmon/templates`. For more details see␣
↪[Configuration␣
↪template](../management/configuration-management.md/#configuration-template)

    4. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    5. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
```

(continues on next page)

(continued from previous page)

```
↪(default). Otherwise uncheck if any of the two needs not be disabled. If␣
↪you have uncheck **Disable SELinux/AppArmor**, make sure that you have␣
↪configured properly to allow creating files and opening sockets for the␣
↪database to run properly.

5. Click **Continue**.

6. Second step is **Advanced Settings**.

    1. **Rebuild from a backup** is disabled. If you desire to rebuild your␣
↪data instead of streaming from the Primary node, toggle it to **On**.␣
↪This shall reveal a drop-down list with all full-backup created using␣
↪Clustercontrol.

    2. **Delay the replica node** is disabled. If you desire to delay the␣
↪replica especially for data recovery purposes, switch this to **On**.

    3. **Include in LoadBalancer set (if exists)** is disabled. Switch it␣
↪to **On* if you want ClusterControl to include this new node in the␣
↪existing node load balancing set.

    4. **Semi-synchronous replication** is enabled by default. Switch to␣
↪**Off** if you only need the traditional asynchronous replication.

7. Click **Continue**.

8. Third step is **Add node**.

    1. Choose and select your **Primary node**.

    2. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target replica node. If you intend to␣
↪add another one, this will just overwrite what was previously inputted␣
↪and added since it only allows one node at a time.

9. Click **Continue** to proceed the last step **Preview**.

10. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

11. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Add Database Node*
- *Create Load Balancer*
- *Create Replica Cluster*

2.69.4 Galera Cluster

The following vendors and versions are supported for adding replica node:

- Percona XtraDB Cluster - 8.0.
- MariaDB Galera - 10.4, 10.5, 10.6 (LTS), 10.11 (LTS), 11.4 (LTS) and 11.8.

Adding a replication requires a running and healthy primary node in the course of action in order to complete the job successfully. A galera cluster can have multiple primary nodes so you can choose and assign which is the desired primary node to be used as a primary of the target replica node. When the job is in progress, ClusterControl will copy the data from the primary but this shall not lock the primary node since ClusterControl uses binary backup as explained below:

- For Percona XtraDB Cluster, it will use Percona Xtrabackup to stream data from its primary to the target replica node.
- For MariaDB Server, it will use Mariabackup to stream data from its primary to the target replica node.

If you are concerned of the traffic and/or the data to be copied from your Primary node is large, it is recommended to use it during low peak period or rebuild your replica node using *backup created* and taken using ClusterControl. This shall be discussed on how to rebuild from backup in the following section below.

Default configuration

By default, ClusterControl deploys MySQL/MariaDB replication with the following configurations:

- ClusterControl uses `netcat` or `nc` to stream data using a binary backup (`xtrabackup` or `mariabackup`). It uses port 9999, or if used, it will search between 9990-9998 range and will use the next port available.
- MySQL GTID with `log_slave_updates` enabled (Oracle MySQL and Percona only).
- MariaDB GTID with `log_slave_updates` enabled (MariaDB only).
- Replica node shall be configured with `read_only=ON` and `super_read_only=ON` (if supported) for protection against accidental write.
- Semi-sync replication is enabled
- `performance_schema` is enabled
- The generated account credentials are stored inside `secrets-backup.cnf` under the MySQL configuration directory.
- Since ClusterControl will copy its data from the Primary node which means users will be copied as well, ClusterControl will add one user with its IP address or hostname to grant necessary privileges for replication:

```
| Database user      | Purpose                                          |
| ------------------ | ------------------------------------------------ |
| `rpl_user`         | MySQL/MariaDB replication.                       |
```

If you would like to customize the above configurations, modify the template base file to suit your needs before proceeding to the deployment. See *Configuration Template*.

Warning

Attention

ClusterControl sets `read_only=ON` on all slaves but a privileged user (SUPER) can still write to a slave (except for MySQL versions that support `super_read_only`).

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Replica node**

3. Choose **Create a replica node**

4. First step is **Node configuration**.

    1. Specify the **Port** of your target replica node. Default is set to␣
↪3306.

    2. **Data directory** is set to `/var/lib/mysql`. If you have prepared␣
↪a separate path or on a different volume, replace the value with your␣
↪custom path.

    3. **Configuration template** is set to `my.cnf.repl84`. If you have␣
↪your own custom template, ClusterControl will show the templates that has␣
↪been added or configured in `/etc/cmon/templates`. For more details see␣
↪[Configuration␣
↪template](../management/configuration-management.md/#configuration-template)

    4. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    5. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
```

(continues on next page)

(continued from previous page)

```
↪run properly.

5. Click **Continue**.

6. Second step is **Advanced Settings**.

    1. **Rebuild from a backup** is disabled. If you desire to rebuild your␣
↪data instead of streaming from the Primary node, toggle it to **On**.␣
↪This shall reveal a drop-down list with all full-backup created using␣
↪Clustercontrol.

    2. **Delay the replica node** is disabled. If you desire to delay the␣
↪replica especially for data recovery purposes, switch this to **On**.

    3. **Include in LoadBalancer set (if exists)** is disabled. Switch it␣
↪to **On* if you want ClusterControl to include this new node in the␣
↪existing node load balancing set.

    4. **Semi-synchronous replication** is enabled by default. Switch to␣
↪**Off** if you only need the traditional asynchronous replication.

7. Click **Continue**.

8. Third step is **Add node**.

    1. Choose and select your **Primary node**.

    2. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target replica node. If you intend to␣
↪add another one, this will just overwrite what was previously inputted␣
↪and added since it only allows one node at a time.

9. Click **Continue** to proceed the last step **Preview**.

10. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Add Database Node*
- *Create Load Balancer*
- *Create Replica Cluster*

2.69.5 PostgreSQL

The replica will be staged with data from the primary. The primary's configuration will be altered to allow the replica to join the primary. The replica server must be reachable by SSH (key-based auth.) from the controller.

The following vendors and versions are supported for a new deployment:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, and 16.
- PostgreSQL from EDB repository - 12, 13, 14, 15, and 16

Adding a replication requires a running and healthy primary node in the course of action in order to complete the job successfully. When the job is in progress, ClusterControl will copy the data through streaming from the primary node by using `pg_basebackup`. `pg_basebackup` is a binary backup which does not lock the primary node. If data is huge and streaming the data to the target replica node can take time, the process can create a significant amount of I/O load and impact performance. If you are concerned of such impact and overhead from your primary, it is recommended to use it during low peak period when adding a replica.

Default configuration

By default, ClusterControl deploys PostgreSQL instances with the following configurations:

- Configure and load the `pg_stat_statements` module.
- The WAL level is set to `replica`.
- All replica will be set with `hot_standby`.
- Configure the `cluster_name` value in postgresql.conf given if **Instance name** is provided n the GUI under **Advanced settngs**.
- ClusterControl will configure the PostgreSQL instance with SSL encryption for clients and primary-replica connections.
- The **Disable firewall** and **Disable SELinux/AppArmor** is hidden in the background but functionality is set to enabled.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Replica node**

3. Choose **Create a replica node**

4. First step is **Node configuration**.

    1. Specify the **Port** of your target replica node. Default is set to␣
↪3306.

    2. **Data directory** is set to `/var/lib/postgresql/<version>/main` in␣
↪Ubuntu/Debian, whereas  `/var/lib/pgsql/<version>/data` in RHEL-variant␣
↪Linux OS. If you have prepared a separate path or on a different volume,␣
↪set **Use package default for datadir** to `Off` and specify the custom␣
↪full path.

    3. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

5. Click **Continue**.

6. Second step is **Advanced Settings**.

    1. **Include in LoadBalancer set (if exists)** is disabled. Switch it␣
↪to **On* if you want ClusterControl to include this new node in the␣
↪existing node load balancing set.

    2. **Synchronous replication** is `Off` by default. Switch to `On` to␣
↪enable synchronous replication.

    3. Specify the **Instance name** which shall be the value of␣
↪**cluster_name** in the `postgresql.conf`. Otherwise, you can leave it␣
↪empty.

7. Click **Continue**.

8. Third step is **Add node**.

    1. Choose and select your **Primary node**.

    2. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target replica node. If you intend to␣
↪add another one, this will just overwrite what was previously inputted␣
↪and added since it only allows one node at a time.
```

(continues on next page)

(continued from previous page)

```
9. Click **Continue** to proceed the last step **Preview**.

10. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

11. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Add Database Node*
- *Create Load Balancer*
- *Create Replica Cluster*

2.69.6 TimescaleDB

The replica will be staged with data from the primary. The primary's configuration will be altered to allow the replica to join the primary. The replica server must be reachable by SSH (key-based auth.) from the controller.

The following vendors and versions are supported for a new deployment:

- PostgreSQL from postgresql.org repository - 12, 13, 14, 15, and 16.
- PostgreSQL from EDB repository - 12, 13, 14, 15, and 16

Adding a replication requires a running and healthy primary node in the course of action in order to complete the job successfully. When the job is in progress, ClusterControl will copy the data through streaming from the primary node by using `pg_basebackup`. `pg_basebackup` is a binary backup which does not lock the primary node. If data is huge and streaming the data to the target replica node can take time, the process can create a significant amount of I/O load and impact performance. If you are concerned of such impact and overhead from your primary, it is recommended to use it during low peak period when adding a replica.

Default configuration

By default, ClusterControl deploys PostgreSQL instances with the following configurations:

- Configure and load the `pg_stat_statements` module.
- The WAL level is set to `replica`.
- All replica will be set with `hot_standby`.
- Configure the `cluster_name` value in postgresql.conf given if **Instance name** is provided n the GUI under **Advanced settngs**.
- ClusterControl will configure the PostgreSQL instance with SSL encryption for clients and primary-replica connections.

- The **Disable firewall** and **Disable SELinux/AppArmor** is hidden in the background but functionality is set to enabled.
- ClusterControl will create and grant necessary privileges for the following database users:

Database user	Purpose
`cmon`	Management and automation.
`cmonexporter`	Prometheus exporter for database monitoring.
`cmonagent`	ClusterControl query monitoring agent.
`cmon_replication`	PostgreSQL streaming replication.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Replica node**

3. Choose **Create a replica node**

4. First step is **Node configuration**.

    1. Specify the **Port** of your target replica node. Default is set to␣
↪3306.

    2. **Data directory** is set to `/var/lib/postgresql/<version>/main` in␣
↪Ubuntu/Debian, whereas  `/var/lib/pgsql/<version>/data` in RHEL-variant␣
↪Linux OS. If you have prepared a separate path or on a different volume,␣
↪set **Use package default for datadir** to `Off` and specify the custom␣
↪full path.

    3. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

5. Click **Continue**.

6. Second step is **Advanced Settings**.

    1. **Include in LoadBalancer set (if exists)** is disabled. Switch it␣
↪to **On* if you want ClusterControl to include this new node in the␣
↪existing node load balancing set.

    2. **Synchronous replication** is `Off` by default. Switch to `On` to␣
↪enable synchronous replication.

    3. Specify the **Instance name** which shall be the value of␣
↪**cluster_name** in the `postgresql.conf`. Otherwise, you can leave it␣
↪empty.
```

(continues on next page)

(continued from previous page)

```
7. Click **Continue**.

8. Third step is **Add node**.

    1. Choose and select your **Primary node**.

    2. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target replica node. If you intend to␣
↪add another one, this will just overwrite what was previously inputted␣
↪and added since it only allows one node at a time.

9. Click **Continue** to proceed the last step **Preview**.

10. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.

11. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Add Database Node*
- *Create Load Balancer*
- *Create Replica Cluster*

2.69.7 Redis/Valkey Cluster

The following vendors and versions are supported for adding replica node:

- Redis Cluster - 6 and 7
- Valkey Cluster - 7 and 8 (only RHEL 8/9 are supported)

Adding a replication requires a running and healthy primary node in the course of action in order to complete the job successfully. When the job is in progress, ClusterControl will setup the replica running full sync or incremental resynchronization running and executing `CLUSTER MEET` and `CLUSTER REPLICATE` commands. These commands update cluster metadata and replication roles without causing the master to become unresponsive.

Default Configuration

By default, ClusterControl deploys Redis cluster instances with the following configurations:

- ClusterControl will configure the Redis/Valkey instances with `appendonly` enabled.

- ClusterControl will secure the instance with AUTH enabled and configure the `requirepass` and `masterauth` options.
- ClusterControl will enable TLS encryption. To access the cluster, one must use the `--tls` flag to connect.
- The configuration `maxMemory` (70% of node's RAM, rounded to the nearest power of 2) and `maxMemoryPolicy=allkeys-lru` will be set, to reduce the risk of Redis being killed by OOM.

Deployment steps

ClusterControl GUI

1. Go to the **Clusters** dashboard. Choose your target source cluster and click the ellipsis button (…)

```
2. Choose **Add new  → Replica node**

3. First step is **Node configuration**.

    1. Specify the **Port** of your target replica node. Default is set to␣
↪6379.

    2. **Install software** is set to **On**. You can disable this if you␣
↪do not want to install the software if the package is already installed.␣
↪If set to **On**, ClusterControl will uninstall any existing package␣
↪found and install the right package and version of the database instead.

    3. **Security configuration** to either disable firewall or␣
↪SELinux/AppArmor. Leave check if you need to **Disable firewall**␣
↪(default). Leave check if you need to **Disable SELinux/AppArmor**␣
↪(default). Otherwise
       uncheck if any of the two needs not be disabled. If you have␣
↪uncheck **Disable SELinux/AppArmor**, make sure that you have configured␣
↪properly to allow creating files and opening sockets for the database to␣
↪run properly.

4. Click **Continue**.

5. Third step is **Add node**.

    1. Choose and select your **Primary node**.

    2. In the **Node** field, this is where you have to specify and input␣
↪the IP address or hostname of your target replica node. If you intend to␣
↪add another one, this will just overwrite what was previously inputted␣
↪and added since it only allows one node at a time.

9. Click **Continue** to proceed the last step **Preview**.

10. Fourth and the last step is **Preview**, this is the last step where it␣
↪shows all the values and choosen options for the ClusterControl to use␣
↪for setting up the node.
```

(continues on next page)

(continued from previous page)

```
11. Click **Finish** button to deploy the job.
```

ClusterControl CLI

To be added soon

Recommended Next Steps

- *Add Database Node*
- *Create Load Balancer*
- *Create Replica Cluster*

2.69.8 Post Deployment

Once a replica is added in the cluster, ClusterControl will immediately install Prometheus exporters on the database node and add it into the Prometheus monitoring list.

2.70 Remove Database Node (or Remove Node)

Removing a database node or removing a node helps you scale down your cluster. Scaling down a database cluster is useful when optimizing costs, reducing complexity, and maintaining an efficient infrastructure without over-provisioning resources. ClusterControl makes this straightforward providing you options to either just unregister the node in the ClusterControl, or stop the service, or stop and remove its configuration and the package installed. This is important especially if you are also going to reuse the node as a new resource for your other productivity or business use.

Removing a node does not convey its a destructive in nature such as wiping out the data or totally scrap the server itself ready for decomissioning. Removing a node is a term used here to manage the node by unregistering from ClusterControl management and services. It also offers the means of managing the configuration or service to just stop and normally terminates itself, does a graceful exit within the cluster to avoid abnormal consequences with regards to its cluster health.

Info

In this page, the term **Node** here refers to any node that is registered in ClusterControl such as database node, a load balancer, or even a backup verification server.

This page will guide you how to remove the node with ClusterControl.

2.70.1 Support Matrix

All database clusters that are supported by ClusterControl allows you to remove a node without affecting the other service. For a complete list of our supported database clusters, see below:

Database	Vendor	Topology
MySQL	Percona, Oracle	Standalone, replication
MariaDB	MariaDB	Standalone, replication
Galera Cluster	MariaDB, Percona	Galera certification-based replication
PostgreSQL	PostgreSQL, EnterpriseDB	Standalone, streaming replication
TimescaleDB	TimescaleDB	Standalone, streaming replication
MongoDB	MongoDB, Percona, MongoDB Enterprise	Replica set, sharded cluster
Redis	Redis, Valkey	Sentinel, cluster
Microsoft SQL Server for Linux	Microsoft	Standalone, Always On availability group
Elasticsearch	Elastic	Single-node cluster, high availability cluster

2.70.2 Prerequisites

Only those registered nodes that are imported or deployed using ClusterControl can be managed to remove a node.

2.70.3 Deployment steps

The recipe for removing a database node is consistent and standardized to all database clusters we support.

ClusterControl GUI

1. In the **Home** dashboard page, Go to the **Clusters** view and choose your target database cluster.
2. Go to the **Nodes** tab.
3. Choose your desired node to be removed and click the ellipsis button (…) under the **Actions** field or column
4. Choose **Remove node** and this shall open a new dialog view
    1. Under the **Remove node** dialog view, you have three options to choose from what actions should you desire:
        1. **Keep service running**. If you choose this action, the following actions will be performed:
            1. Node will be unregistered from ClusterControl but the service will be kept running
            2. Node will be unregistered from load balancer (if applicable)
        2. **Stop service and keep files untouched**. If you choose this action, the following actions will be performed:
            1. Node will be unregistered from ClusterControl and service will be stopped
        3. **Stop and uninstall service**. If you choose his action, the following actions will be performed:
            1. Node will be unregistered, stopped and all software will be uninstalled
5. Click **Remove** to perform and execute the action

ClusterControl CLI

- Remove a HAProxy node (that is running on port `9600`).

```
s9s cluster \
    --cluster-id 99 \
    --remove-node \
    --nodes="192.168.40.60:9600"  \
    --wait --log \
```

  - where `--cluster-id=99` targets the cluster having the cluster ID of 99.
  - where `--remove-node` defines the action to take
  - where `--nodes="192.168.40.60:9600"` will be the node to be removed.
  - where the `--wait` and `--log` as the name suggest to print the job logs while waiting.

- To delete multiple nodes, you can simply achieve this by running a "for" loop. For example:

```

```

```
        for i in 192.168.40.60:9600 192.168.40.61:9600 192.168.40.60:112␣
↪192.168.40.61:112;
        do
            s9s cluster --remove-node --cluster-id 99 --nodes="${i}"  --
↪print-request --print-json --wait --log;
                sleep 0.9;
                printf "0.%s" `seq 5`;
        done;
    ```

    - `--print-request` parameter means to print the HTTP request for this␣
↪action and `--print-json` will print the logs in JSON format
```

**Recommended Next Steps**

- *Add Replica Node*
- *Create Load Balancer*
- *Create Replica Cluster*

ClusterControl Operations Center (CC Ops-C) is a platform management designed to simplify and enhance the management of database clusters, providing a single interface for multiple ClusterControl instances (controllers) for handling complex database operations and ensuring the reliability, performance, and security of database systems. You can use the ClusterControl Operation center to handle large-volume environments of 1000+ of nodes.

Horizontal scaling of ClusterControl is accomplished by registering or adding additional ClusterControl installations to the designated primary ClusterControl installation. CMON controllers are manually registered or added to this primary installation. It also provides multi-tenancy isolation where each tenant is assigned their own CMON Controller.
```

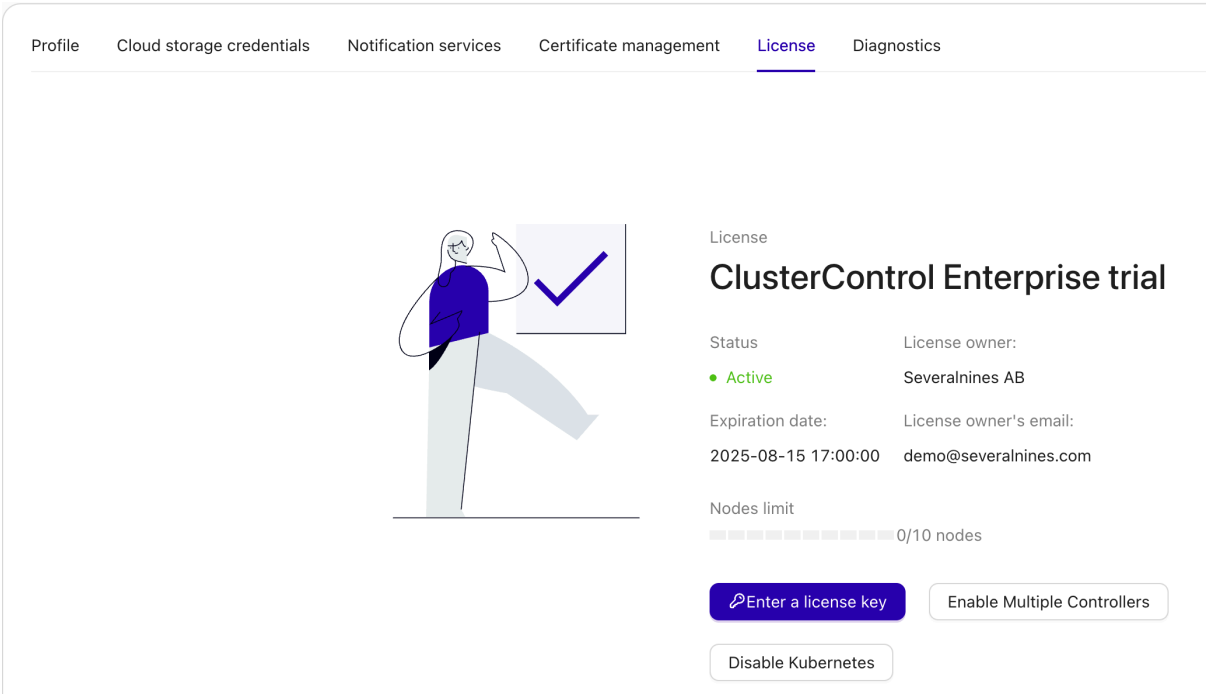
2.71 Single-Controller vs Multi-Controller

Mode	Description
Single-Controller	Traditional setup using a single CMON controller to manage clusters.
Multi-Controller (Ops-C)	Ops-C manages multiple CMON controllers via the **ClusterControl Manager** (`cmon-proxy`), providing consolidated control and visibility across different environments.

In a multi-controller architecture, each deployment can have its own isolated CMON controller, registered to a primary Ops-C instance.

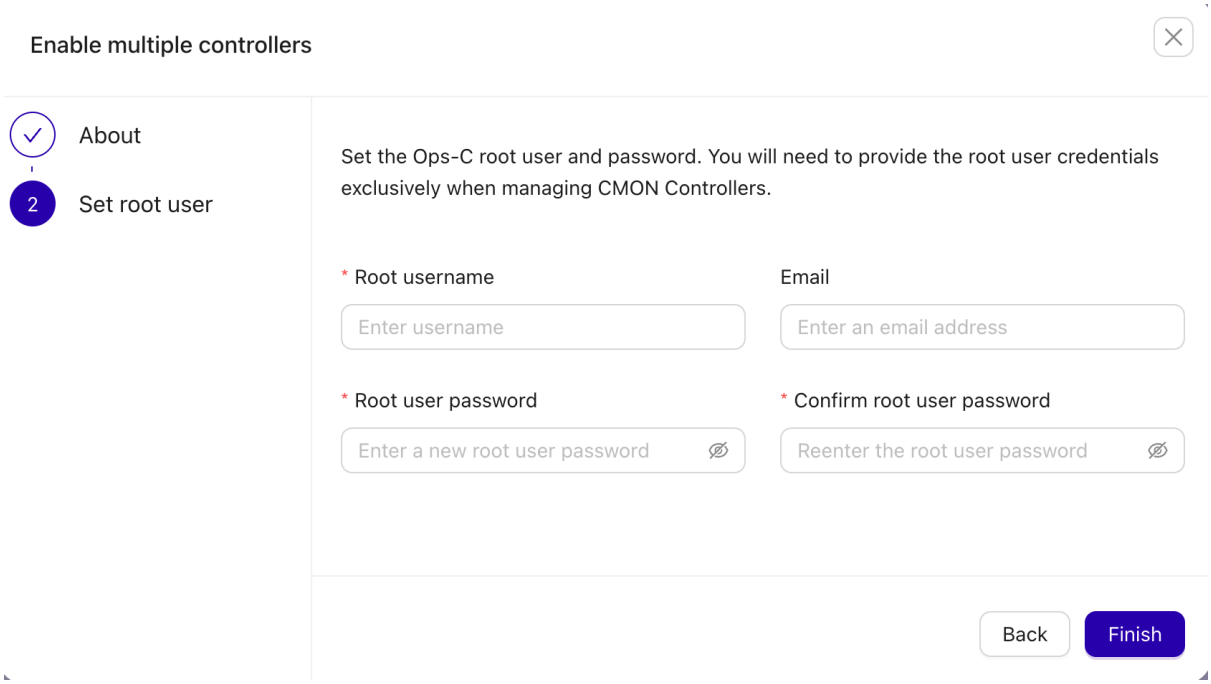
2.72 Activating ClusterControl Ops-C

Since ClusterControl version 2.3.2, ClusterControl Ops-C is installed by default so you only need to enable it from ClusterControl -> Settings -> License -> Enable Multiple Controllers.

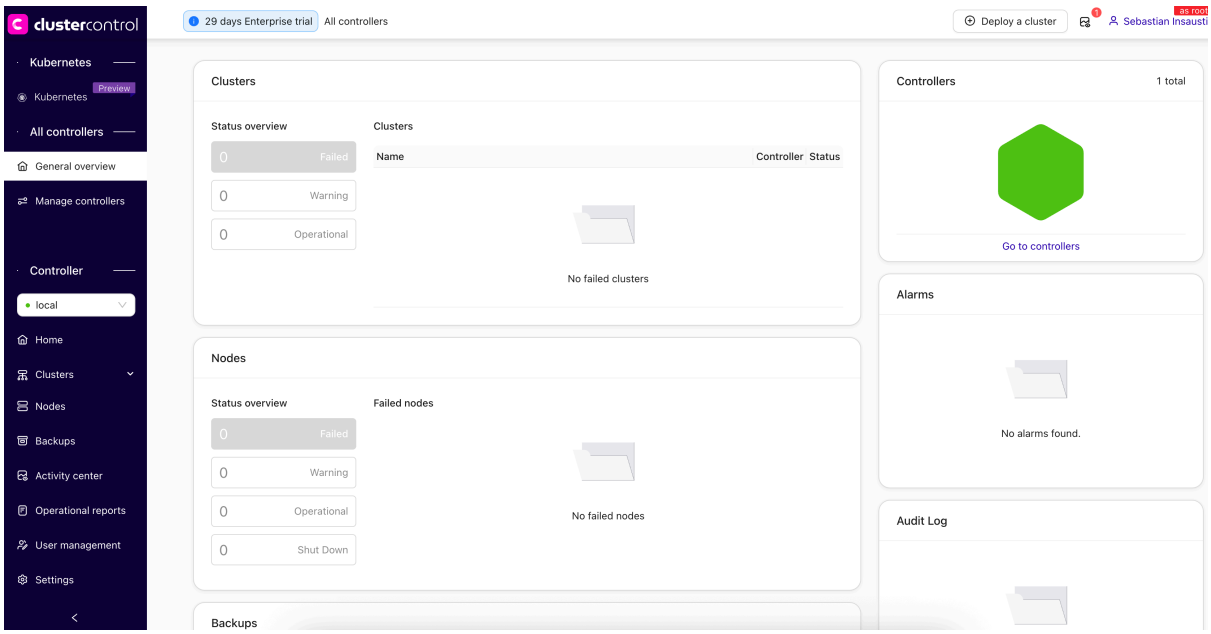


Info

When you enable ClusterControl Ops-C, it will ask you to create a **root** user and **password**. This root user is used only for managing CMON Controllers and serves as an extra layer of security.



After you enable it, you will see the corresponding UI.



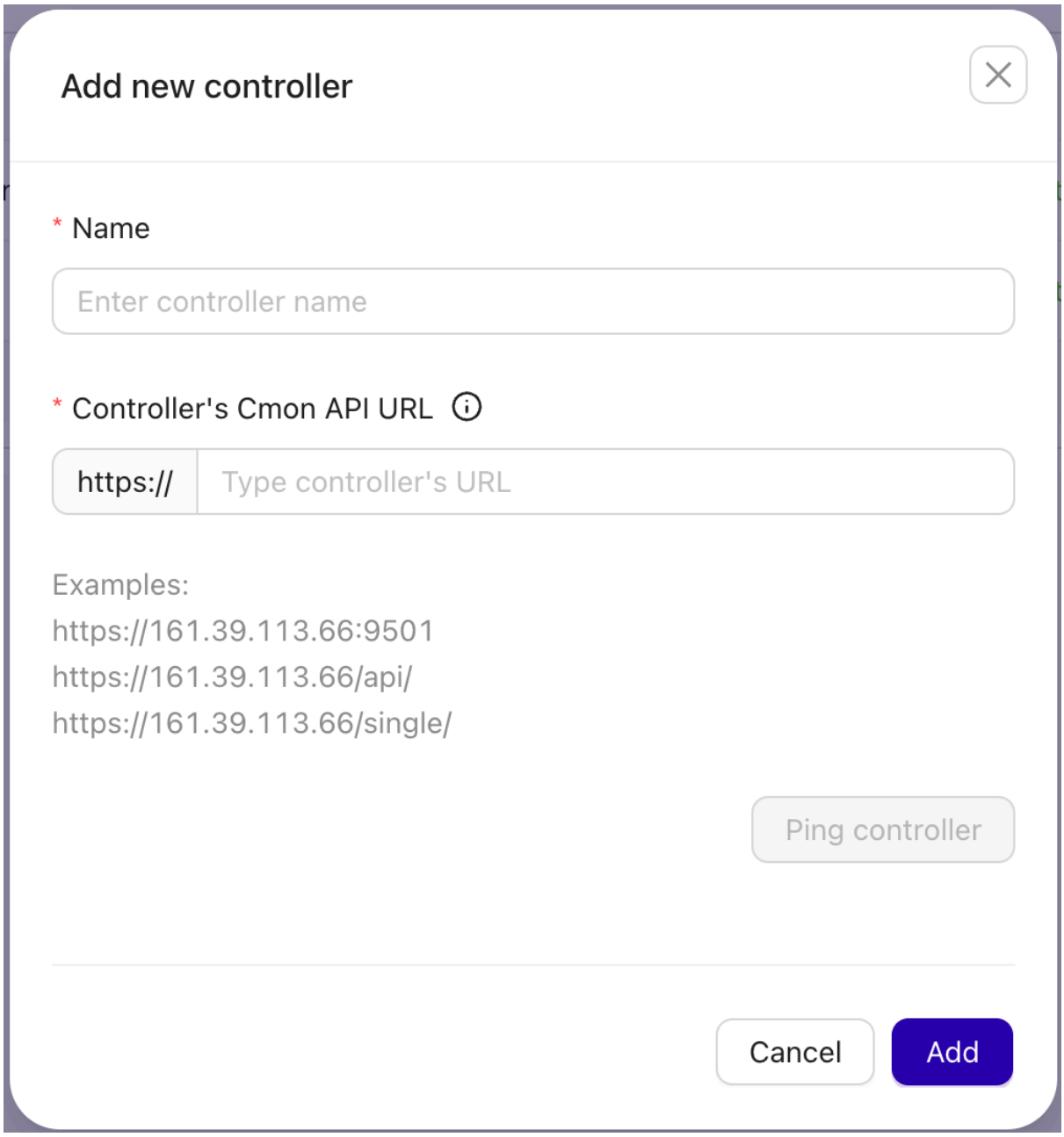
2.73 Controller Management

Ops-C enables full lifecycle management of CMON controllers through the web UI.

2.73.1 Add Controller

Adding a new ClusterControl Controller into the ClusterControl Ops-C is straightforward. Below are the instructions:

1. Navigate to Manage Controllers -> Add new controller. It will ask you to authenticate with the **root** user **password** created when you enabled ClusterControl Ops-C.
2. Provide the controller name and Controller's Cmon API URL. For example, `https://161.39.113.66/api/`
3. Test the connection by using the `Ping controller` button, and press on `Add`.
4. The new controller will appear in the controller list.



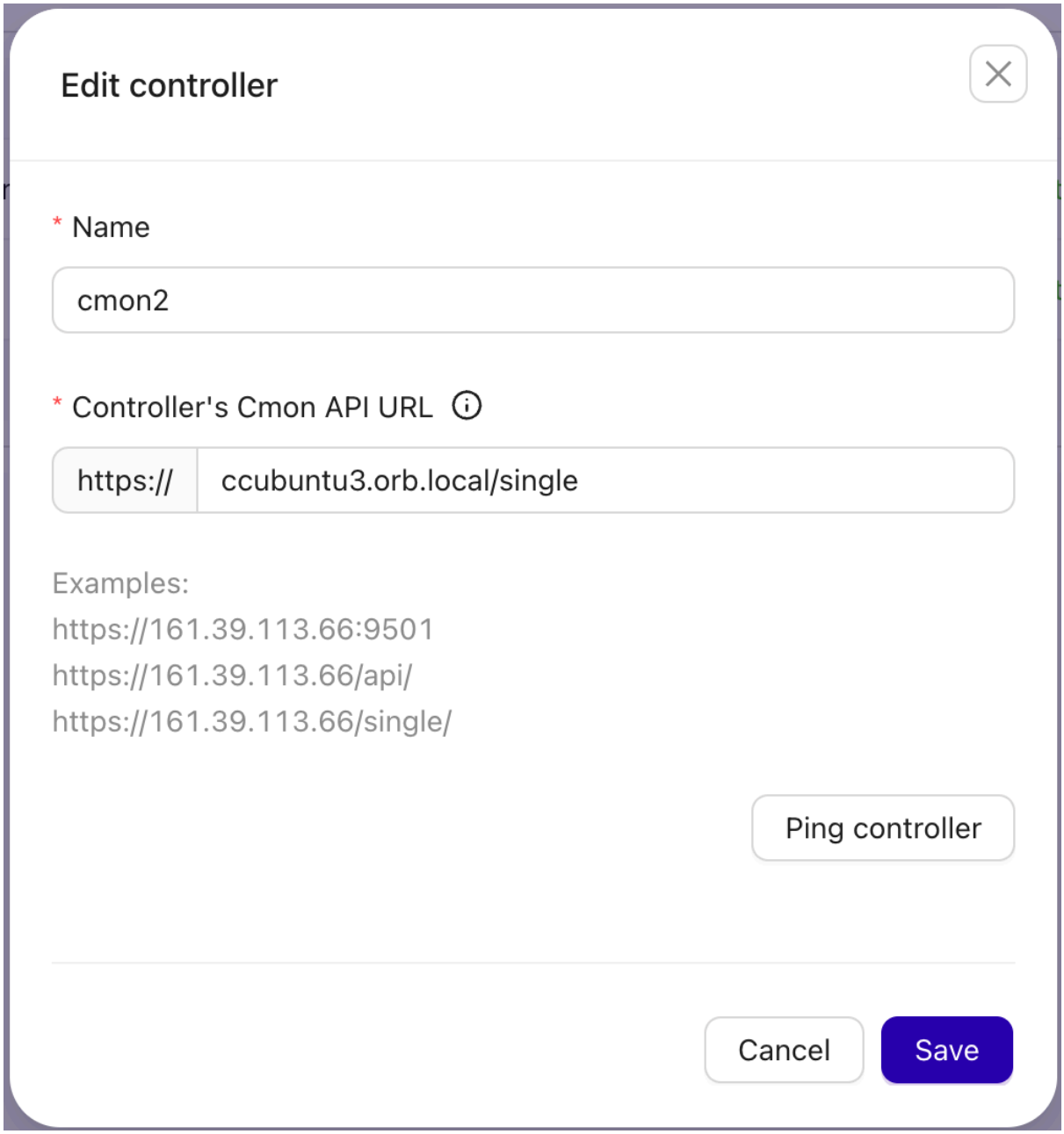
Info

To display aggregated data from multiple controllers, Ops-C app will attempt to log in to all registered controllers using your current credentials upon login. For consistent access, ensure this user exists on all controllers with the same credentials and permissions.

2.73.2 Edit Controller

To update the configuration of an existing controller:

1. Go to Manage Controllers.
2. Click the `Edit` button in the `Actions` section.
3. Modify properties such as name or API URL.
4. Save changes.



2.73.3 Remove Controller

To remove a controller from Ops-C:

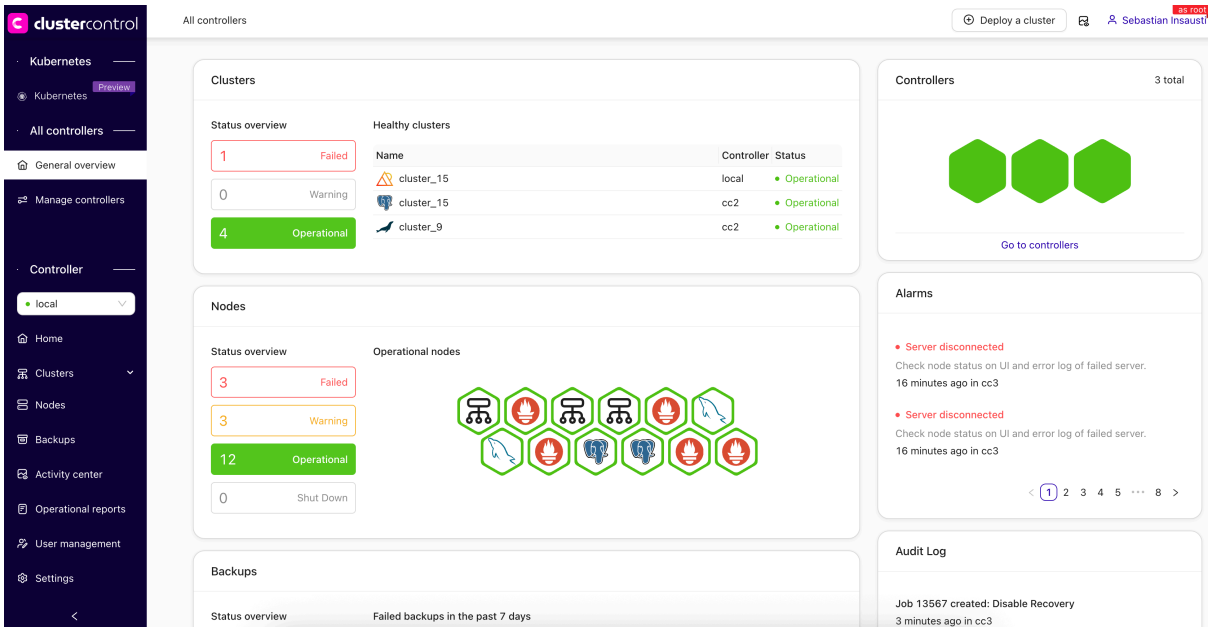
1. Navigate to Manage Controllers.
2. Click the `Remove` button in the `Actions` section.
3. Confirm removal.

The controller will be unlinked but not uninstalled from the remote host.

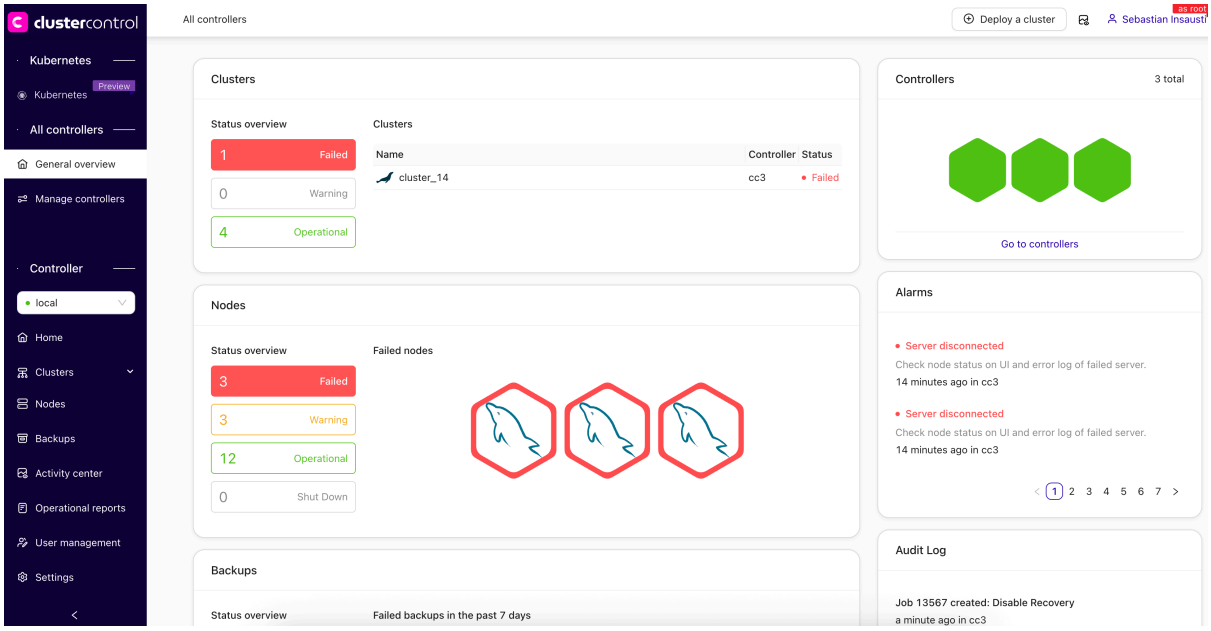
2.74 Monitoring Dashboard

The Controllers' Overview dashboard provides a global view of:

- Controller health and job status
- Cluster alarms and backups
- Node-level availability
- Active jobs per controller

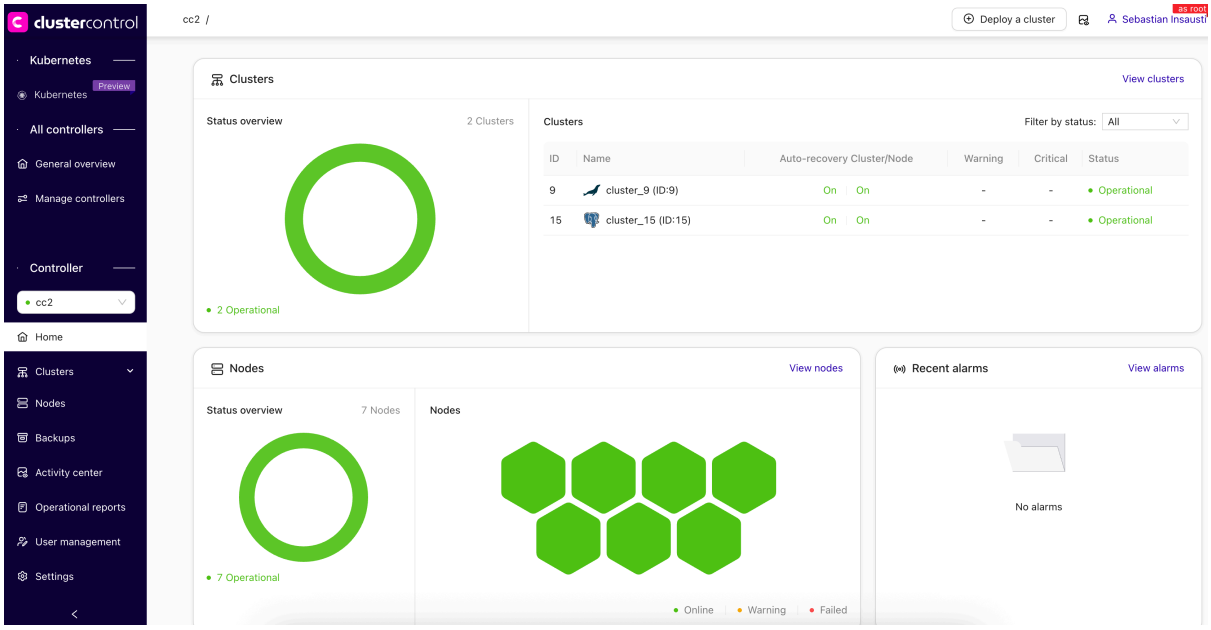


You cna filter by status Failed, Warning, Operational for both Clusters and Nodes.

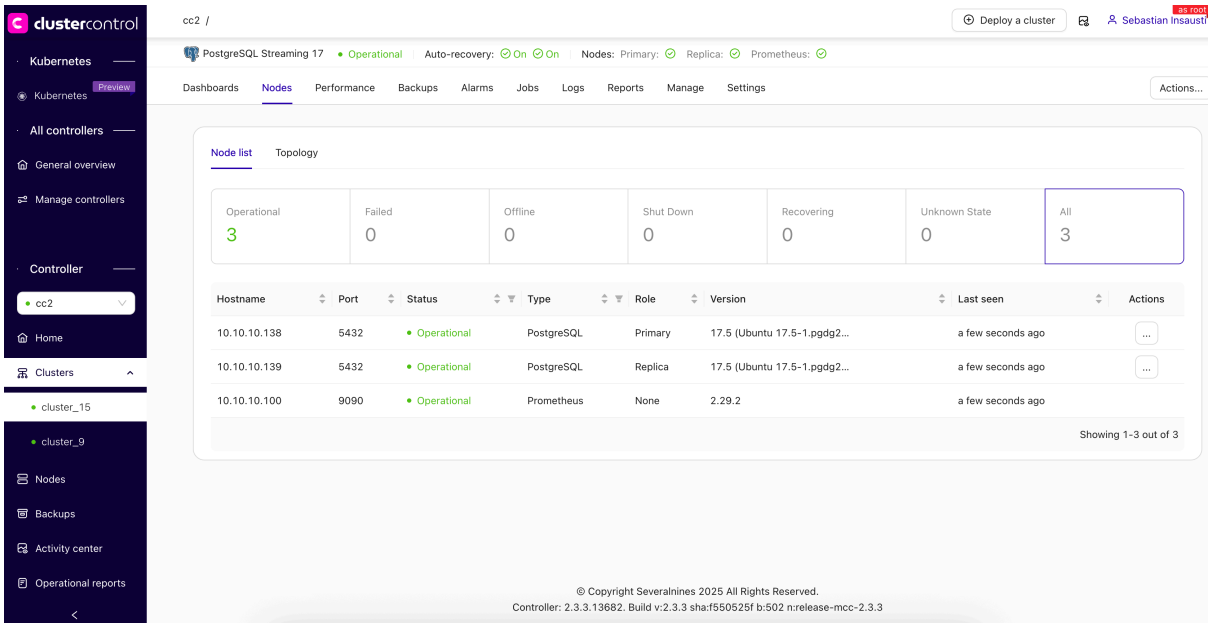


Users can switch between controllers using the Active CMON Controller selector. The application context is scoped to the selected controller for all operations outside the overview.

Discover the status of Clusters and Nodes in specific controllers as shown below:



Go through the node list for specific clusters in the selected controller:



2.75 ClusterControl Manager

A new process, the **ClusterControl Manager** (`ccmgr`), is responsible for managing, collecting data, and monitoring the CMON controllers in this configuration:

- The Controllers' Overview page features a dashboard that presents aggregated information on registered or managed CMON controllers, providing insights into the status of controllers, ongoing jobs, alarms, and backups, as well as the status of the database clusters and nodes.
- The web user interface allows users to select the "Active CMON Controller", which determines the information displayed throughout the application (excluding the Controllers' Overview) and directs user actions for clusters or nodes to the selected CMON Controller.

Warning

Attention

Starting from clustercontrol-mcc 2.2.4 (July 2024), `clustercontrol-mcc` web application doesn't require an Apache server any longer. There is no dependency for the Apache server and the web application is served by the `ccmgr` process (systemd unit is called `cmon-proxy`) instead. So, with an existing ClusterControl environment/installation, the Apache server and the CCv2 UI can be completely removed prior to installing ClusterControl Ops-C.

Functions of **ClusterControl Manager** (`ccmgr`) include:

- Routing API and UI traffic
- Aggregating data from multiple controllers
- Handling multi-tenant authentication
- Exposing Ops-C UI on a configurable port (default: 19052)

Ensure the service is active:

```
systemctl status cmon-proxy
```

To restart after changes:

```
systemctl restart cmon-proxy
```

ClusterControl allows DevOps and SRE teams to declaratively deploy, scale, tune, and backup open-source databases with Kubernetes database operators and ClusterControl's integration with Argo CD and GitHub-based GitOps workflows.

There are two primary methods for managing the database cluster on Kubernetes with ClusterControl:

1. Direct database cluster management using supported Kubernetes operators via the ClusterControl GUI.
2. The GitOps approach. See *GitOps* for details.

2.76 ClusterControl Kubernetes proxy

Integration with the Kubernetes operator is managed through a new process called `kuber-proxy`. This proxy is installed alongside ClusterControl Ops-C (the new GUI replacing ClusterControl GUI v2, starting from version 2.3.3) using the package name `clustercontrol-kuber-proxy`. The service is managed by systemd under the unit file `kuber-proxy`.

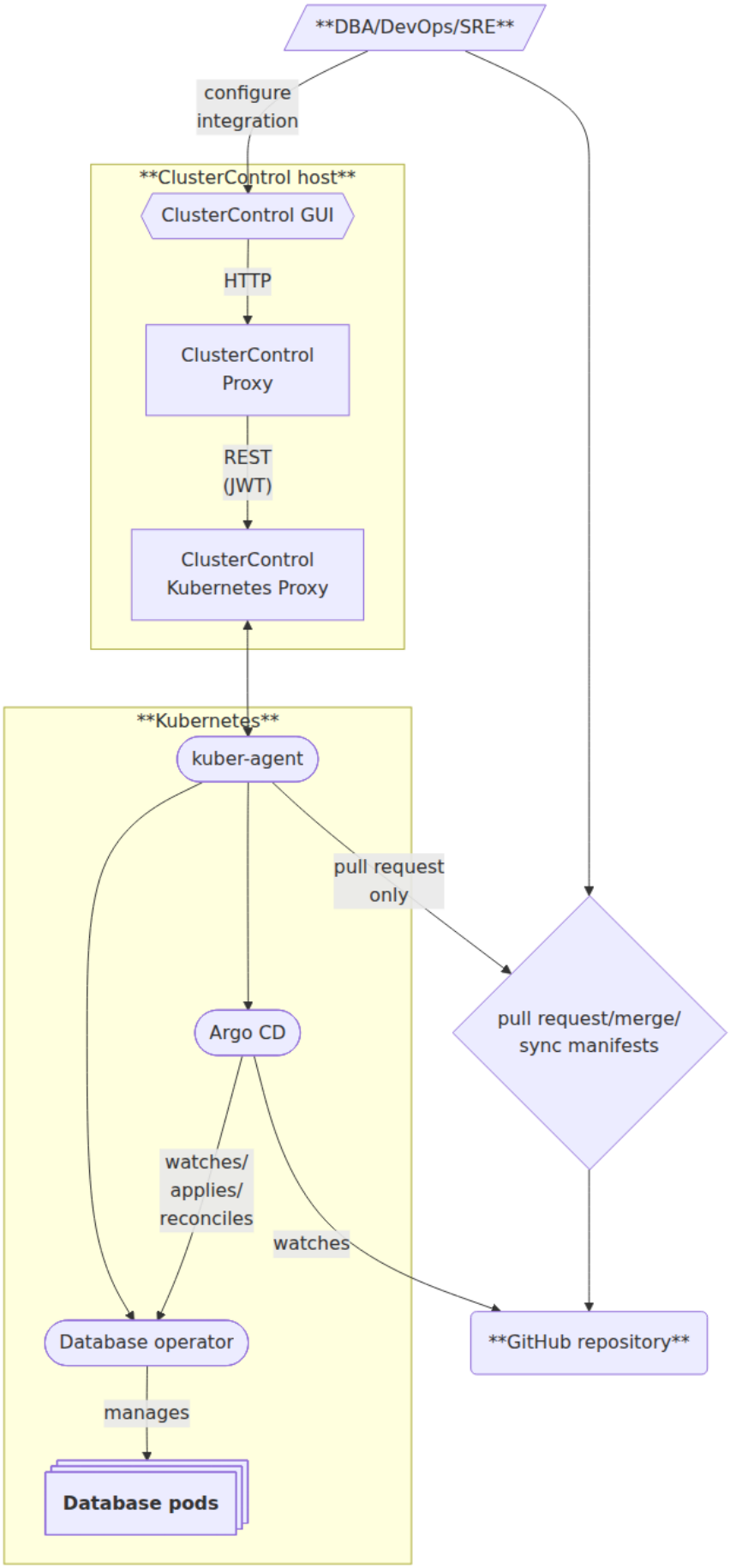
2.77 GitOps

ClusterControl serves as the operational layer that sits above Kubernetes and Argo CD. It facilitates traceable database operations by transforming declarative GitOps workflows. This approach establishes a modern operational model where Git is the single source of truth for declaratively managing applications and infrastructure. Configurations are version-controlled in Git and continuously reconciled with the live infrastructure.

The core principles of GitOps include:

- Declarative infrastructure using YAML.
- Versioned source of truth (Git).
- Automated reconciliation (e.g., via Argo CD).
- Continuous delivery achieved through Pull Requests.
- Security and governance inherited from the Git workflow.

The following flowchart illustrates the workflow and how the components communicate if GitOps integration is enabled:


DBA/DevOps/SRE
configure integration
ClusterControl host
ClusterControl GUI
HTTP
ClusterControl Proxy
REST (JWT)
ClusterControl Kubernetes Proxy
Kubernetes
kuber-agent
pull request only
pull request/merge/ sync manifests
Argo CD
watches/ applies/ reconciles
watches
Database operator
GitHub repository
manages
Database pods

2.77.1 Argo CD integration

Argo CD is a GitOps-based, declarative continuous delivery tool specifically for Kubernetes. It automates the deployment of desired application states into specified target environments. Deployment updates can be tracked via branches or tags, or pinned to a specific Git commit version of the manifests. Argo CD mandates the use of Git repositories as the source of truth for defining the desired application state.

ClusterControl offers integrated support for Argo CD by providing:

- Automatic Installation: Simple deployment and configuration of Argo CD within the target Kubernetes environment.
- External Integration: The ability to connect to pre-existing Argo CD instances, ensuring continuity of established CI/CD and governance policies.

2.77.2 GitHub integration

ClusterControl integrates with GitHub using Fine-Grained Personal Access Tokens (PATs) for secure, least-privilege authentication.

These tokens restrict access to specific repositories and operations, minimizing exposure. To create a fine-grained PAT:

1. Go to ***GitHub → Settings → Developer Settings → Personal Access Tokens → Fine-grained tokens***.
2. Click **Generate new token**.
3. Under **Resource owner**, select your user or organization.
4. Under **Repository access**, choose **Only select repositories** and pick your GitOps repository.
5. Under Permissions, enable:
    - Contents: Read and write
    - Metadata: Read
    - Pull requests: Read and Write
6. Set an expiration (recommended: 90 days or less).
7. Click **Generate token** and copy it.
8. Paste the token in ***ClusterControl GUI → Kubernetes → GitOps Integration → Repository Settings*** to authenticate securely.

Tip

Store tokens as Kubernetes Secrets or integrate with HashiCorp Vault. Rotate regularly and assign repository-specific permissions only.

2.78 Deploy your first Kubernetes cluster

To set up your first database in Kubernetes using ClusterControl, follow the following steps.

2.78.1 1. Install ClusterControl Ops-C

ClusterControl Ops-C is now installed by default (it is the default ClusterControl GUI, replacing ClusterControl GUI v2) so you just need to follow the instructions in the **Installation section** to install it. See *Online Installation*.

Check that the `kuber-proxy` process is reachable on port 50051 (default) either using a public or private address depending on your setup (public managed Kubernetes cluster versus on-premise, for example).

2.78.2 2. Install the required tools

The kubectl tool is required to use this feature. Install it on the ClusterControl host and make sure `kubectl` is installed in the `PATH`.

2.78.3 3. Create a Kubernetes cluster

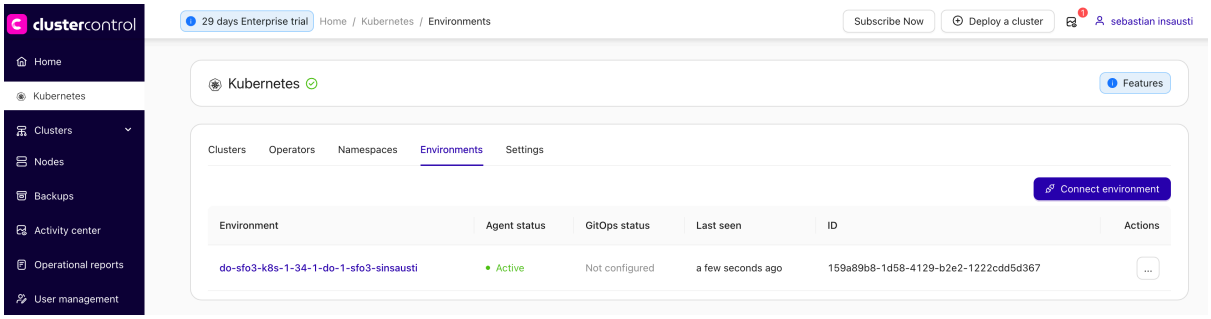
You must create a Kubernetes Cluster if you do not already have one.

The cluster can be a self-hosted on-premise installation, a public managed service, or a development environment like Docker Desktop or OrbStack. The only requirement is that the Kubernetes cluster can successfully connect to the kuber-proxy process on port 50051 of the ClusterControl host.

2.78.4 4. Connect to a Kubernetes cluster

After creating the Kubernetes cluster (or if it was already created), you have to connect to the environment by installing the agent-operator. The Kubernetes agent-operator establishes the connection to your Kubernetes resources. It acts as a bridge between ClusterControl and the Kubernetes environment, allowing you to monitor and perform operations directly through ClusterControl.

The Kubernetes integration is accessible from a new dedicated page, **Kubernetes**, where you can manage all things related to Kubernetes like your database clusters, backups, database operators and connections to different Kubernetes clusters.



You can add any Kubernetes cluster whether it is an on-premise self-hosted, a public managed service, Docker Desktop or OrbStack development environment as long as the agent-operator is able to connect to the kuber-proxy process using the provided GRPC address to the ClusterControl host where the proxy process is running. The default port that kuber-proxy is listening on is 50051.

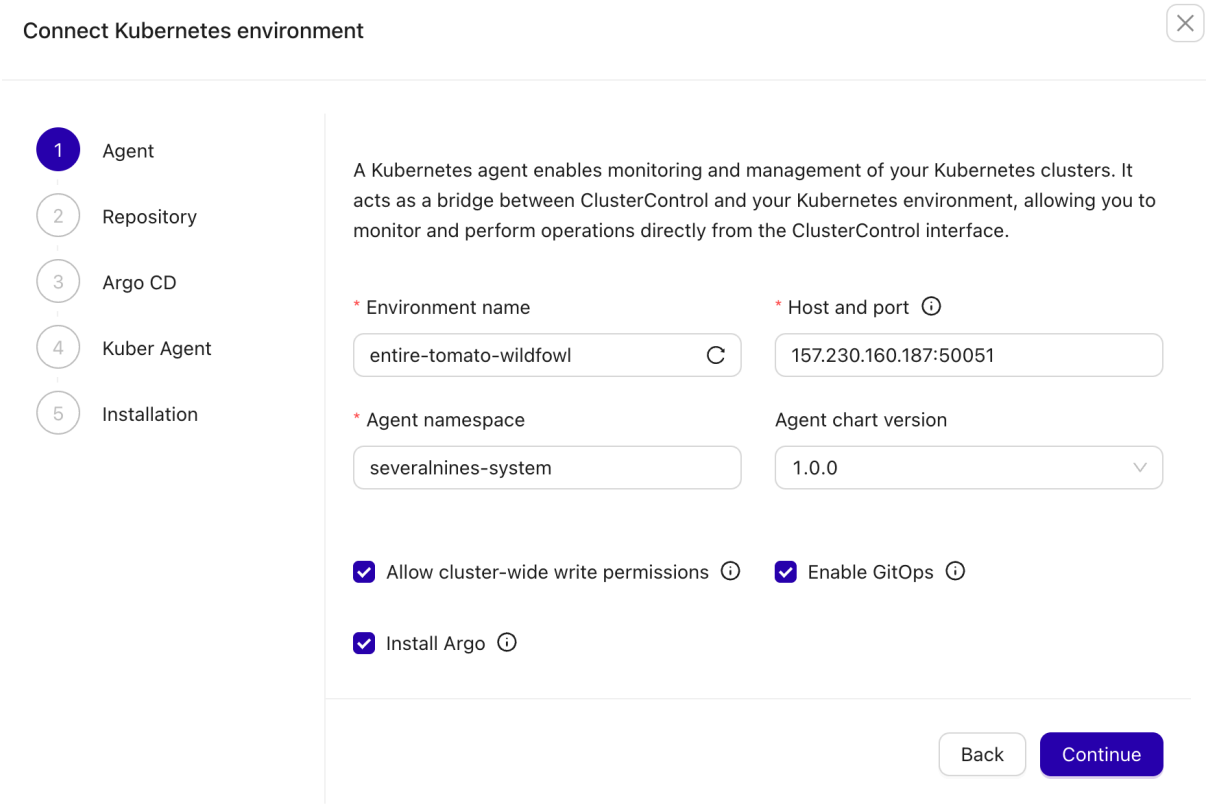
You can check which Kubernetes cluster that you are currently established with:

```
kubectl config current-context
```

After you have verified that this is the Kubernetes cluster that you want to use, then connect this Kubernetes cluster with ClusterControl by clicking on **Connect environment**.

2.78.5 5. Configure the agent

Begin by specifying the environment details.



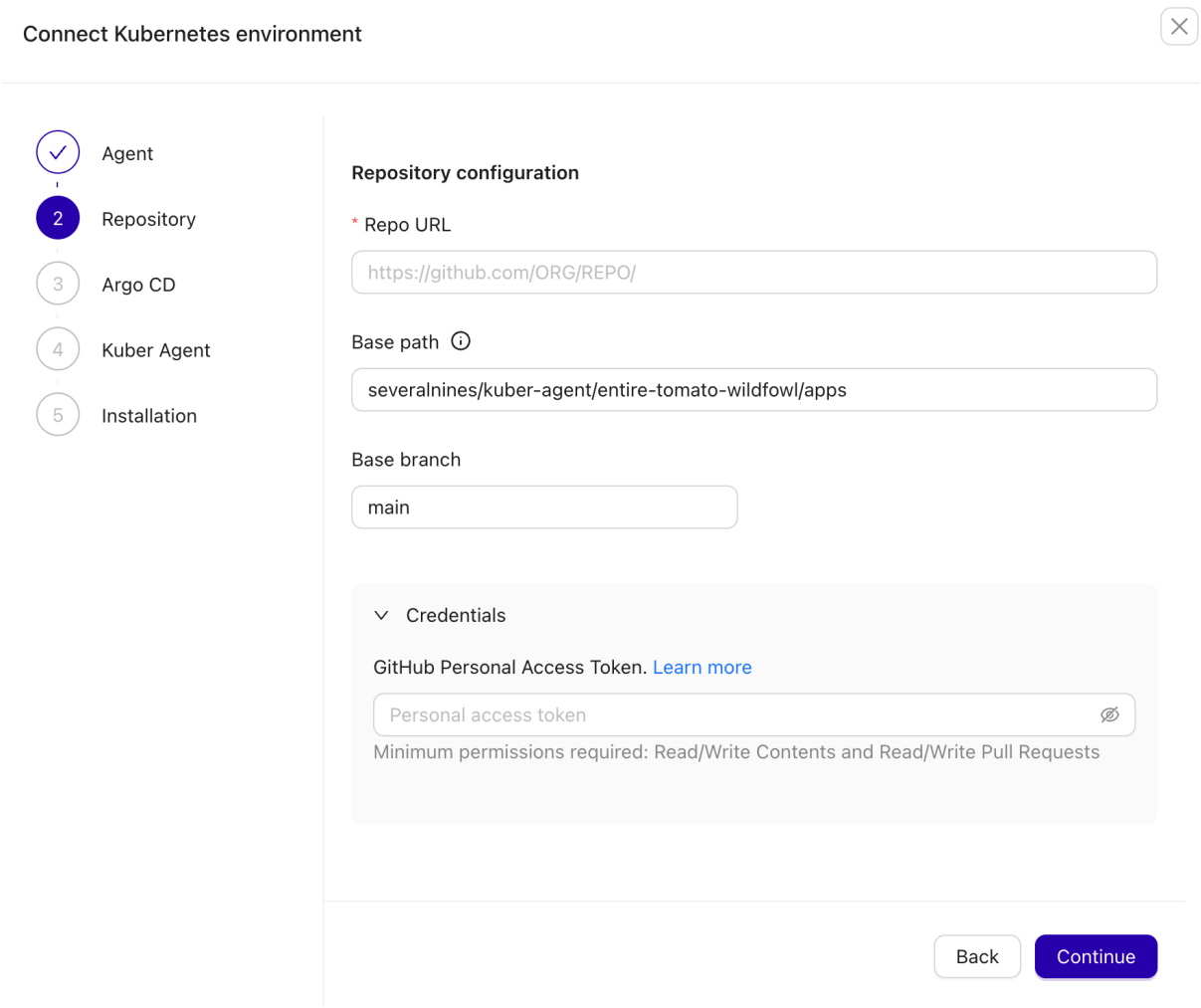
In this section, you have to complete the following information:

- **Environment name**: Enter an environment name.
- **Host and port**: ClusterControl Kubernetes Proxy host and GRPC port. The Kubernetes cluster must be able to reach this endpoint.
- **Agent namespace**: Specify the agent's namespace (e.g., severalnines-system).
- **Agent chart version**: Select the desired agent's Helm chart version (e.g., 1.0.0).
- **Allow cluster-wide write permissions**: Grants this agent permission to create, update, patch, and delete Kubernetes resources defined by the operator. Leave unchecked to confine write permissions to the namespaces you list below. Those namespaces can be used to deploy database operators and clusters.
- **Enable GitOps**: Use git repository as the single source of truth for desired cluster state.
- **Install Argo**: Install Argo CD to manage agent and database operators lifecycle.

2.78.6 6. Set up the repository

*Skip this step if **Enable GitOps** is unchecked in step 5. This step is for GitOps integration.*

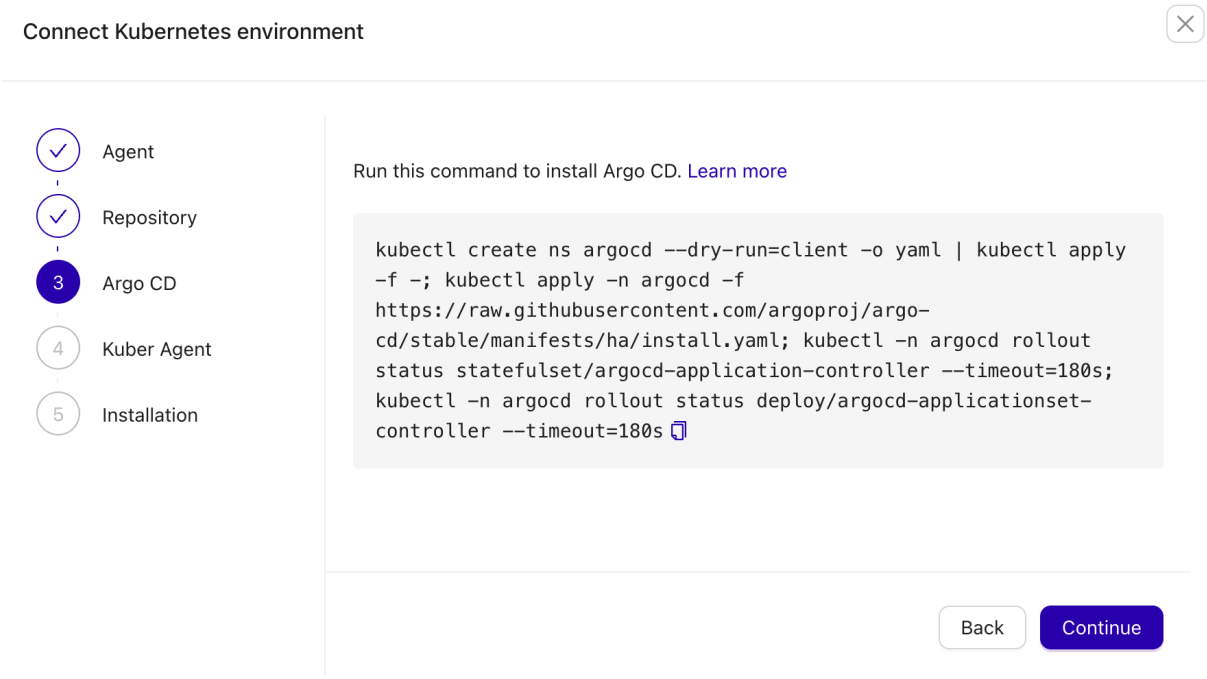
Provide the GitHub repository URL that will store the YAML manifests. Specify the **Base path** and **Base branch**, and authenticate using your GitHub Personal Access Token (PAT) to allow ClusterControl to push configuration changes automatically.



2.78.7 7. Install Argo CD

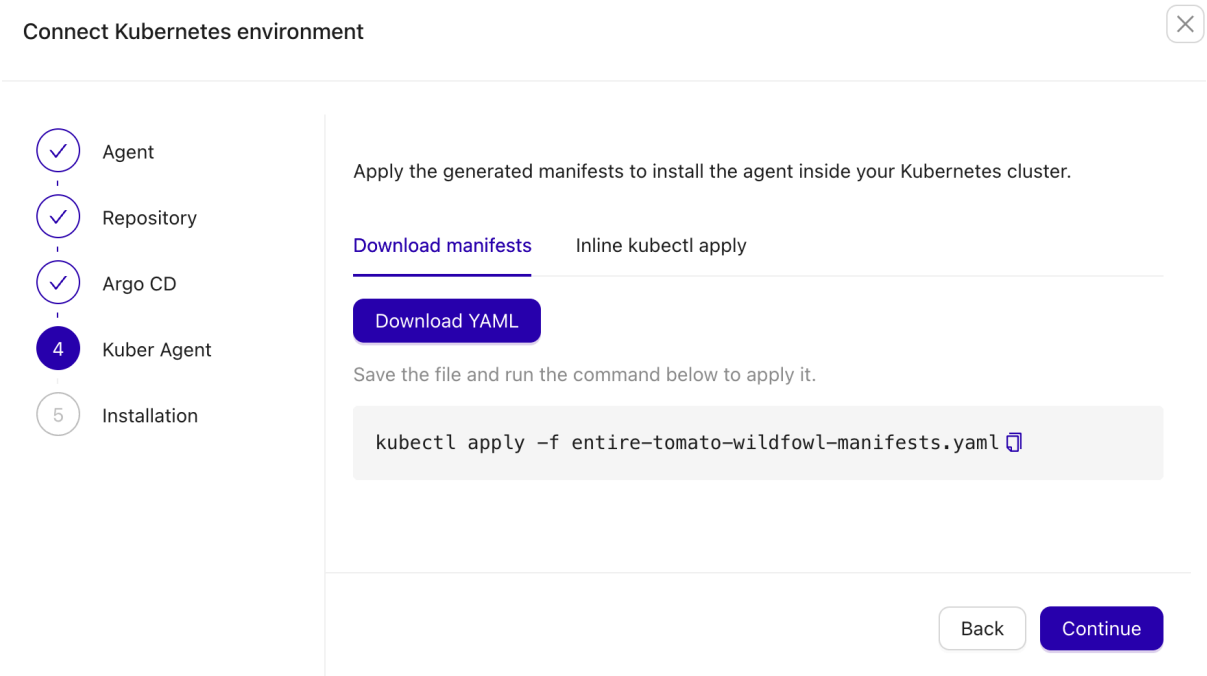
*Skip this step if **Enable GitOps** and **Install Argo** are unchecked in step 5. This step is for GitOps integration.*

The wizard generates a `kubectl` command to install Argo CD. Copy and paste this command into your Kubernetes management terminal. This installs the GitOps controller components needed for synchronization with your GitHub repository.



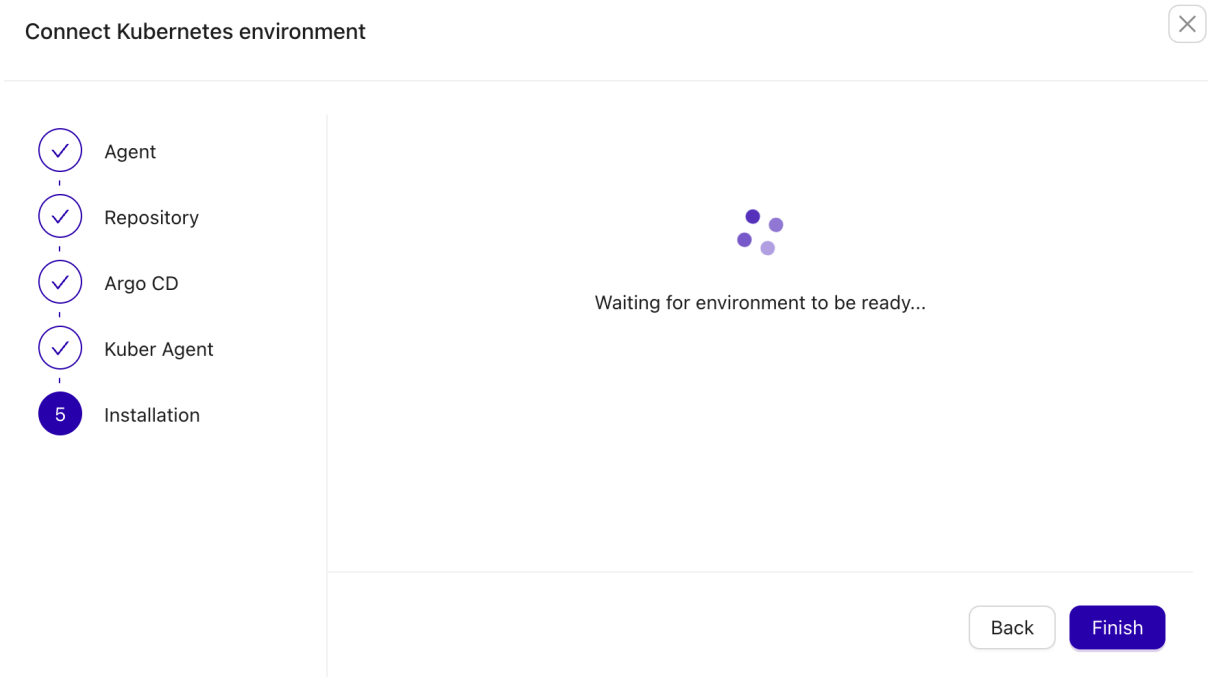
2.78.8 8. Install the agent

Next, you can download and execute the YAML file or you can copy and execute the command provided in the inline tab. This exports the necessary environment variables and installs the ClusterControl Kubernetes agent, which connects your cluster back to ClusterControl for management and monitoring.



2.78.9 9. Wait for environment detection

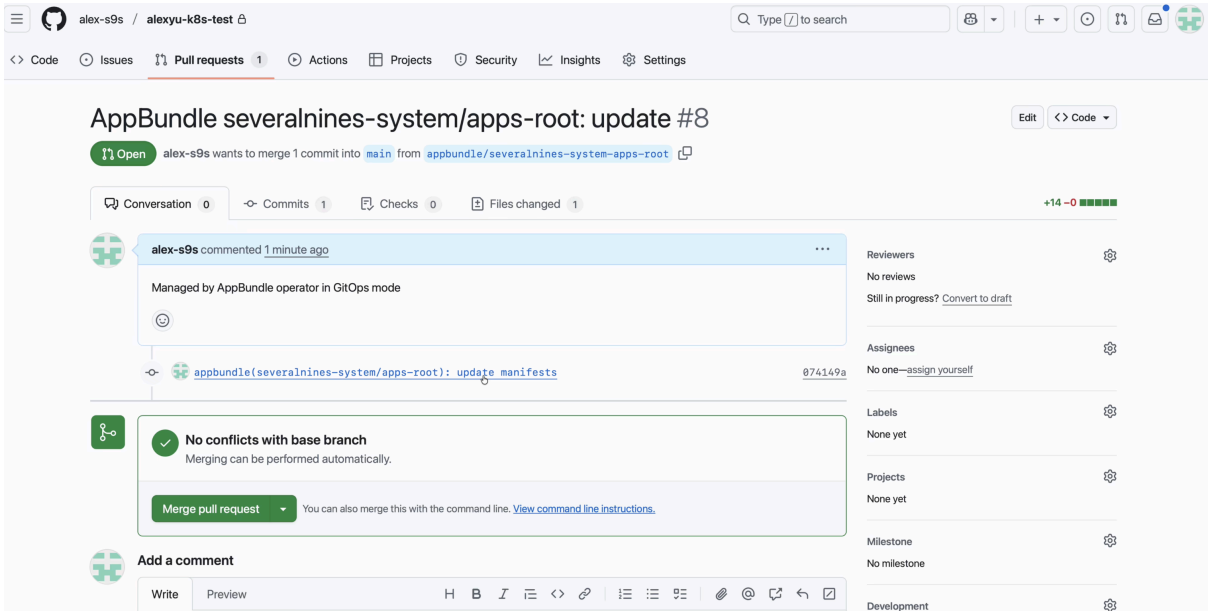
Once installation is complete, return to ClusterControl and wait for the new environment to appear in the dashboard.



2.78.10 10. Pull requests

*Skip this step if **Enable GitOps** and **Install Argo** are unchecked in step 5. This step is for GitOps integration.*

ClusterControl automatically creates a GitHub Pull Request for configuration deployment. Click **View PR**, review the YAML and Helm manifests, and merge it into your main branch.



2.78.11 11. GitOps synchronization

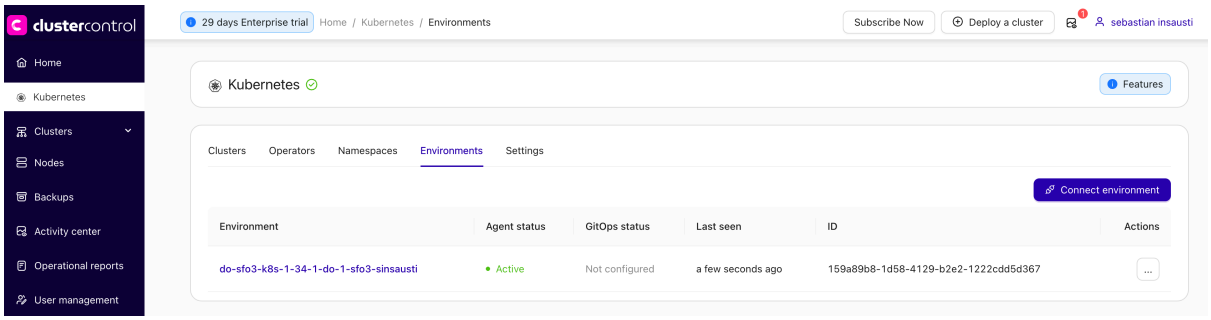
*Skip this step if **Enable GitOps** and **Install Argo** are unchecked in step 5. This step is for GitOps integration.*

Wait for Argo CD to reconcile the merged configuration with the cluster and deploy the agent components.

2.78.12 12. Confirm activation

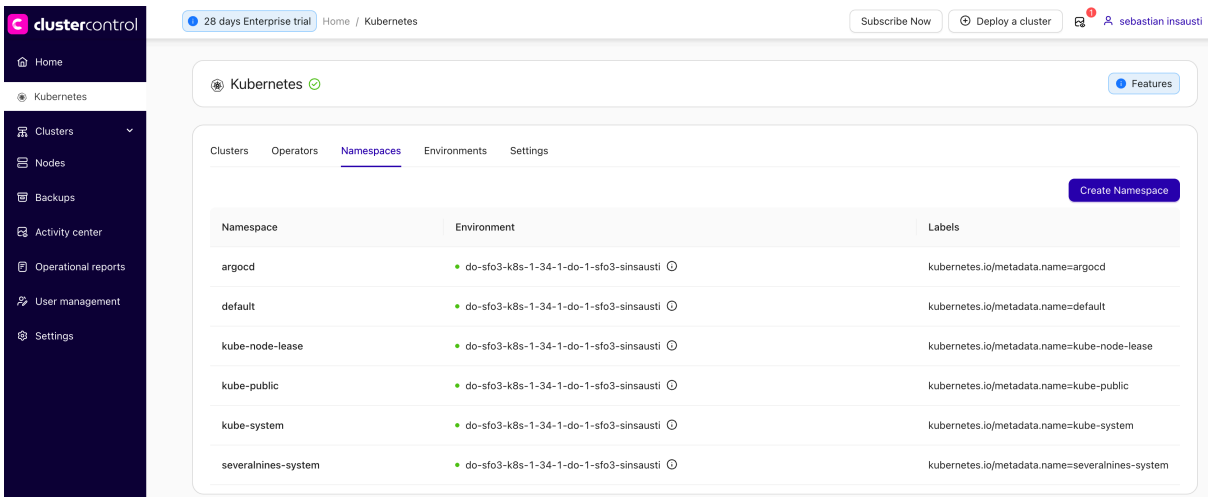
*Skip this step if **Enable GitOps** and **Install Argo** are unchecked in step 5. This step is for GitOps integration.*

Finally, verify that the Kubernetes environment's status changes to "Active". This indicates successful connection and readiness for ClusterControl operations such as monitoring, app deployment, and GitOps-based management.



2.79 Create a Kubernetes namespace

Go to ***ClusterControl GUI → Kubernetes → Namespaces*** and click on **Create Namespace**. Create the namespace and wait for the **View PR** to be shown.

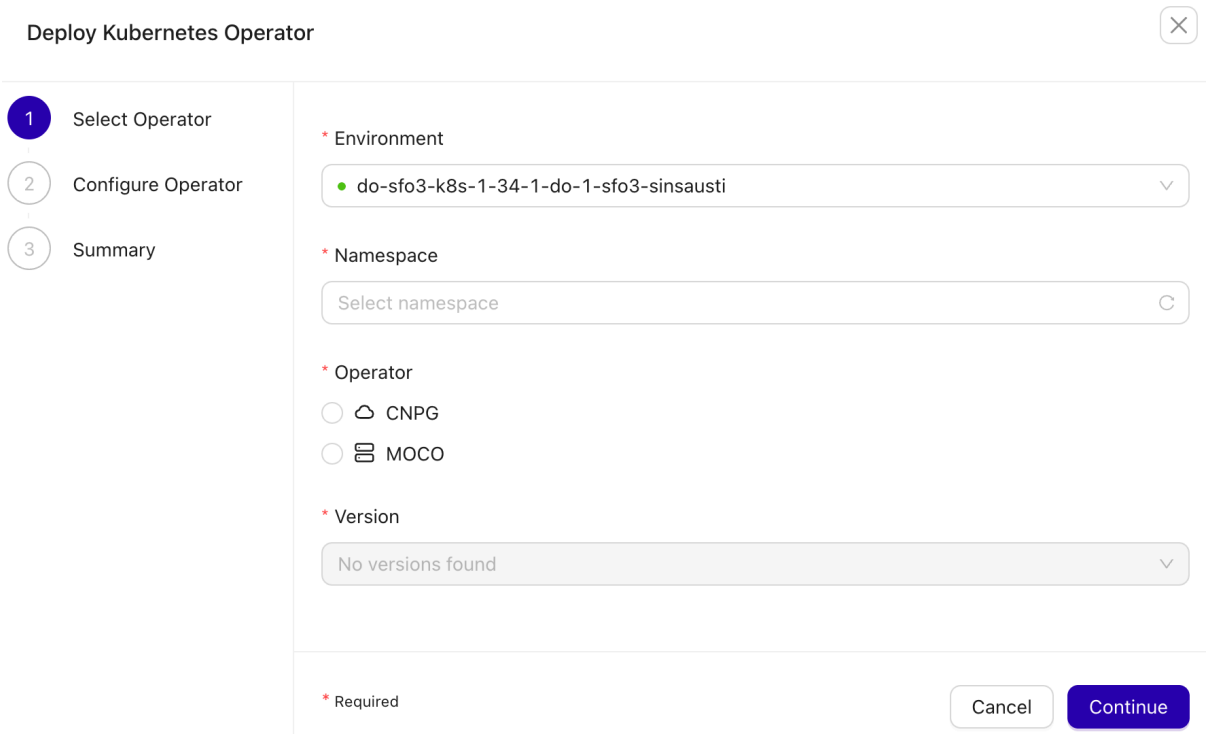


Click on **View PR** to open a page to GitHub to review and merge the Pull Request. A new namespace has been created that can be used to install a database operator.

2.80 Deploy a database operator

Before deploying a database cluster, a corresponding database operator for a specific database vendor or technology must be installed in the Kubernetes cluster that you want to use.

Go to ***ClusterControl GUI → Kubernetes → Operators*** and click on **Deploy operator** and select the namespace to use and the database operator to install.



2.80.1 Supported operators

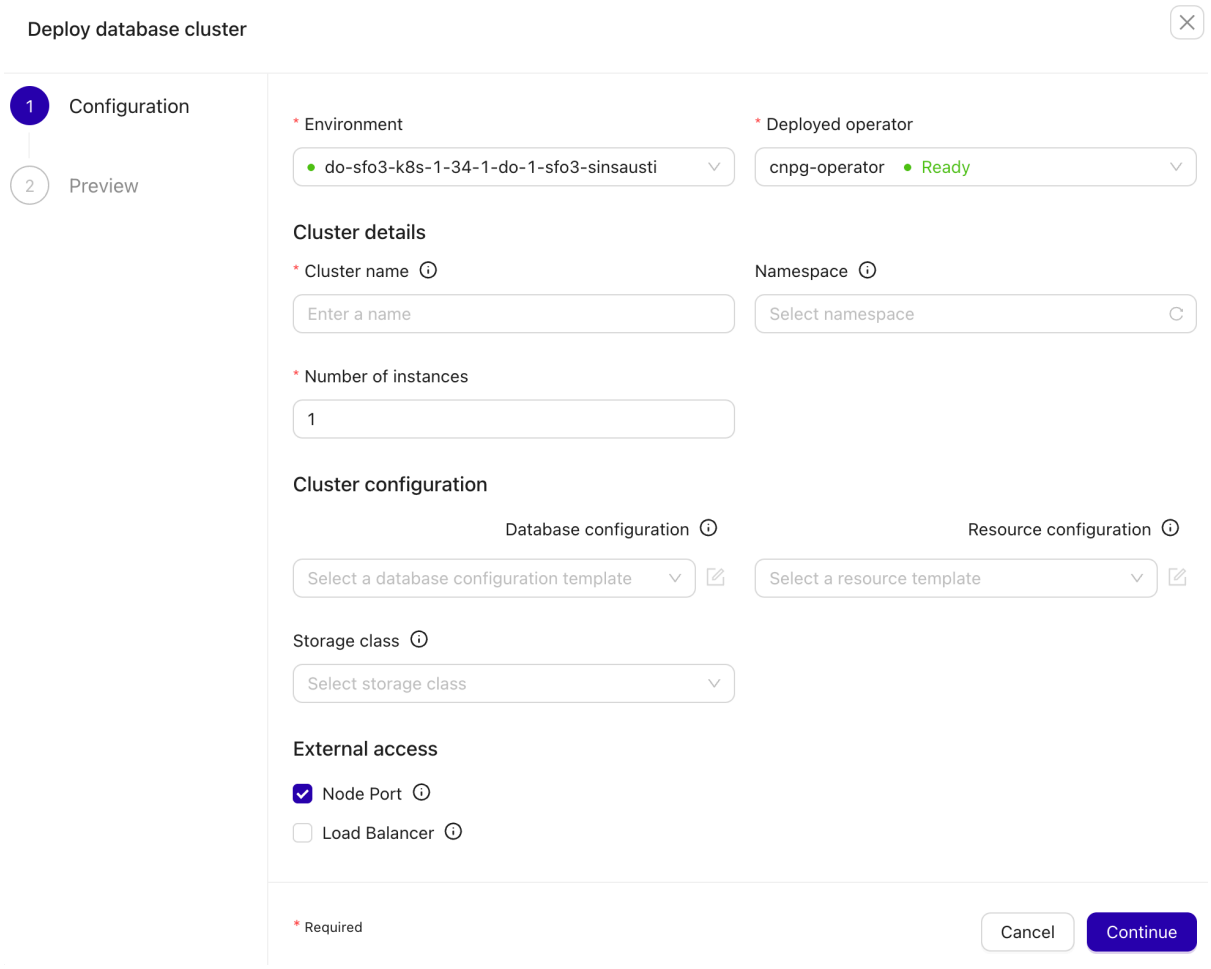
The following database operators are currently supported:

- PostgreSQL standalone and streaming replication: Cloudnative-pg
- MySQL standalone and replication: Moco

2.81 Create a new database cluster

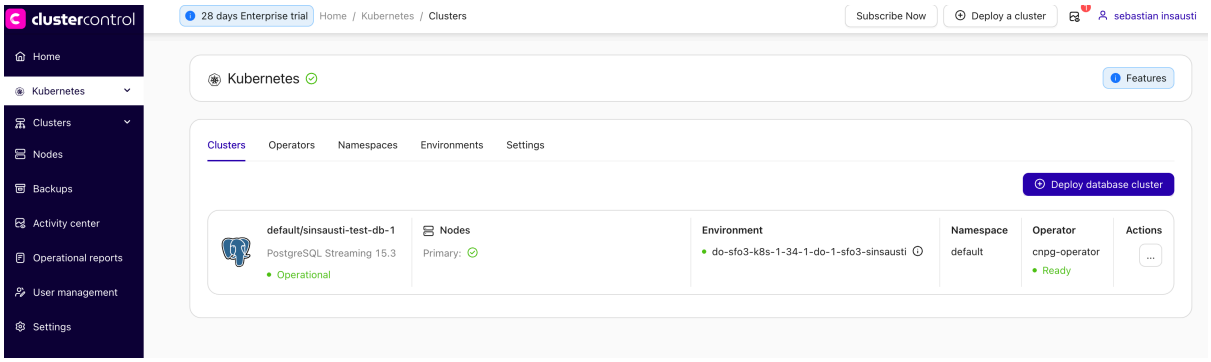
Once the desired database operators have been deployed, the next step is to deploy a new database cluster using one of those operators.

Go to ***ClusterControl GUI → Kubernetes → Clusters*** and click on **Deploy database cluster**.



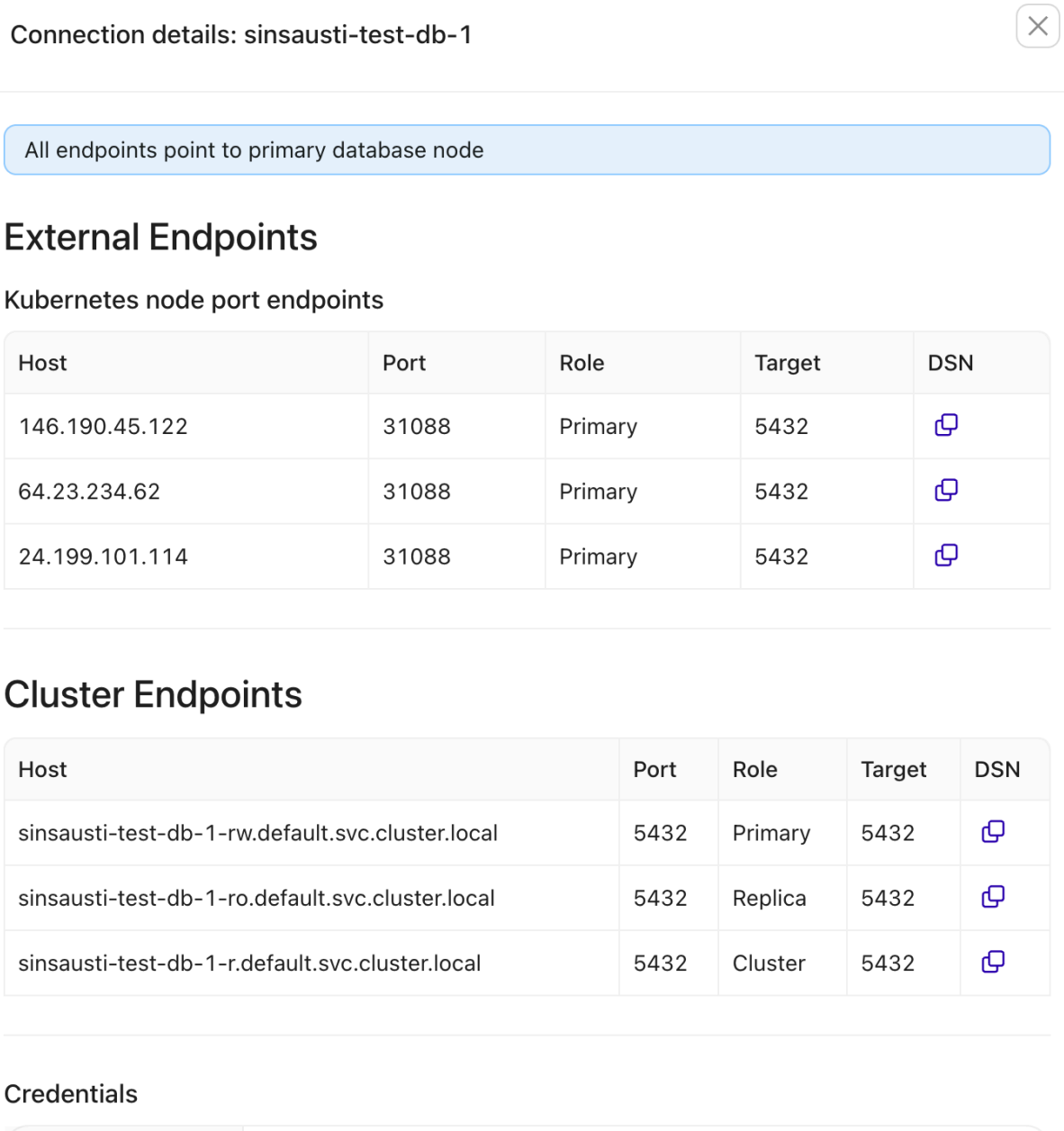
In this section, complete the required information:

- Select an environment where to deploy it.
- Select which database operator to use.
- Set a cluster name and which namespace to use.
- Number of nodes to deploy.
- The database configuration template to use.
- The resource template to use for CPU, Memory and Storage.
- Define how to access the database cluster, i.e., via a port on the node or launching a load balancer (which can incur cost depending on the Kubernetes cluster provider).



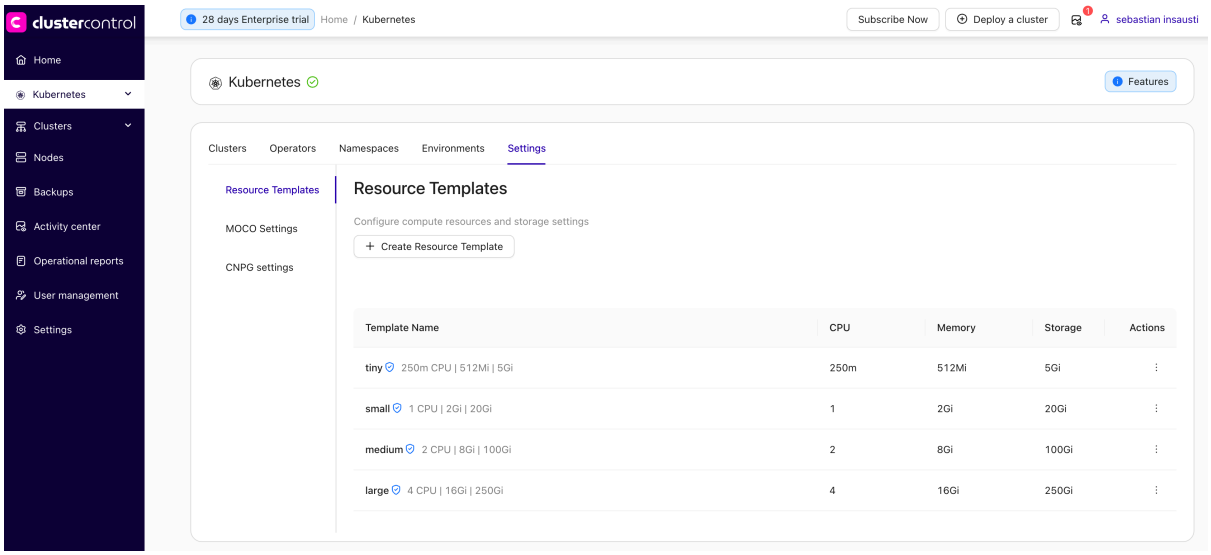
2.81.1 Database connection details

To enable your applications to connect to the database, you need the connection details for the database cluster. These details can be retrieved by selecting the cluster and using the **Details** menu action.



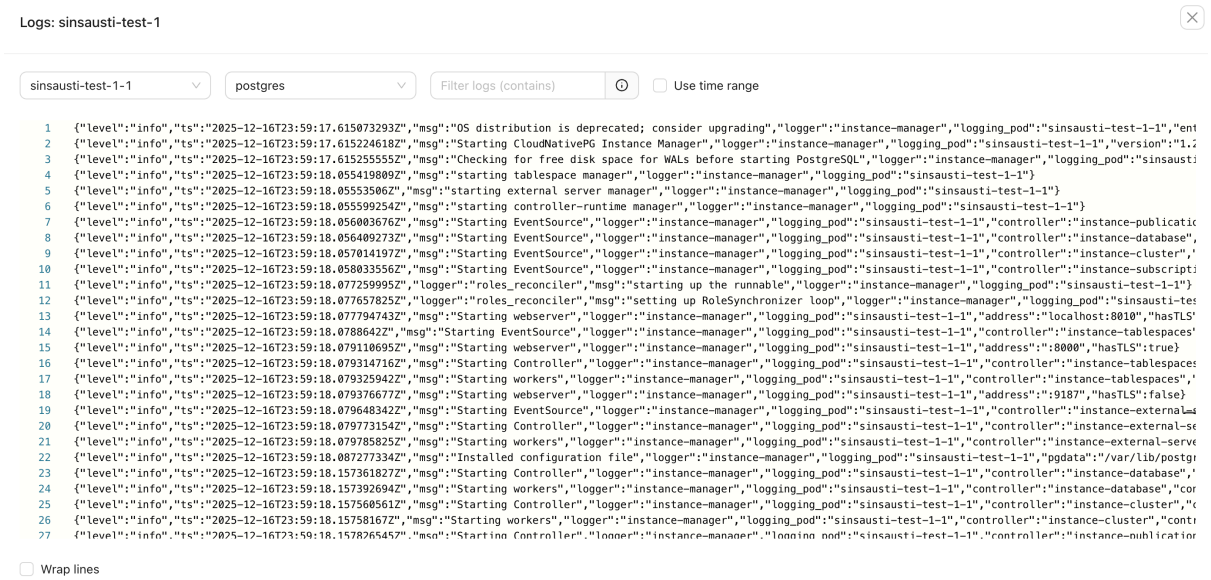
2.82 Resource and database configuration templates

ClusterControl supports creating templates that specifies what resources limits to set and different database configurations that can be quickly applied at deployment.



2.83 Retrieve and view logs

To access the logs, go to each section (**Clusters**, **Operators**, **Environments**) and click on the **Logs** option under the **Actions** menu.



2.84 Purge resources

- Clusters: Go to ***ClusterControl GUI → Kubernetes → Clusters → Cluster Actions → Delete***.
- Operators: Go to ***ClusterControl GUI → Kubernetes → Operators → Operator Actions → Delete***.
- Environments: Go to ***ClusterControl GUI → Kubernetes → Environments → Environment Actions → Delete***.

2.85 Troubleshooting issues

Use k9s to interact with your Kubernetes cluster. It makes it easier to navigate and visualize the resources deployed.

2.85.1 Restart agent

Sometimes you may need to restart agent when issues occurring. Known issues which are fixed by restart are:

- Agent is in a disconnected state and doesn't attempt to connect to proxy.
- Moco operator backups are successful but can't see backups in UI.

To restart, run the following command:

```
kubectl rollout restart deployment/kuber-agent-controller-manager -n␣
↪severalnines-system
```

2.85.2 Agent with debug log

To see debug logs when installing or upgrading agent add `--set debug.logLevel=debug` to helm install or upgrade command.

- Watch agent logs:

```
kubectl logs -l app.kubernetes.io/instance=kuber-agent -n␣
↪severalnines-system -f
```

- Search in logs:

```
kubectl logs -l app.kubernetes.io/instance=kuber-agent -n␣
↪severalnines-system --tail=-1 | grep "error"
```

2.85.3 Basic troubleshooting commands

- View pods in a specific namespace:

```
kubectl get pods -n <namespace>
```

- Get detailed information about a resource:

```
kubectl describe <resource> <resource-name> -n <namespace>
```

- View pod logs:

```
kubectl logs <pod-name> -n <namespace>
```

- List all pods across namespaces:

```
kubectl get pods --all-namespaces
```

2.86 Example issues and solutions

Let's see some examples about different issues and its possible solutions.

2.86.1 Failed database operators

- **Moco operator failure:** Check the operator status:

```
kubectl get databaseoperator
NAME         TYPE         STATUS   VERSION   AGE
cnpg         cnpg         Ready              38m
moco         moco         Error              37m
stackgres    stackgres    Ready              35m
```

```
kubectl get pods --all-namespaces
NAMESPACE             NAME                                                ␣
↪READY   STATUS              RESTARTS      AGE
cert-manager          cert-manager-86d7c7b689-znbz5                       ␣
↪1/1     Running             0             13m
cert-manager          cert-manager-cainjector-77894f5f57-djr94            ␣
↪1/1     Running             0             13m
cert-manager          cert-manager-webhook-6cf469dbd6-tkvnd               ␣
↪1/1     Running             0             13m
cnpg-system           cnpg-controller-manager-8db87d769-jcvp2             ␣
↪1/1     Running             1 (13m ago)   13m
default               cnpg-cluster-1                                      ␣
↪1/1     Running             0             11m
default               cnpg-cluster-2                                      ␣
↪1/1     Running             0             10m
default               cnpg-cluster-3                                      ␣
↪1/1     Running             0             10m
severalnines-system   kuber-agent-controller-manager-c6b7d9489-259gp      ␣
↪1/1     Running             0             19h
stackgres             stackgres-operator-5bfff484d8-55dmj                 ␣
↪0/1     ContainerCreating   0             10m
stackgres             stackgres-operator-set-crd-version-n2rm7            ␣
↪0/1     Completed           0             10m
stackgres             stackgres-restapi-6554545f7b-mmpmc                  ␣
↪0/2     ContainerCreating   0             10m
```

- **Diagnosis:** Moco operator requires cert-manager and operator pods. While cert-manager is running, the operator pods have not been deployed.
- **Check agent logs for details:**

```
kubectl logs kuber-agent-controller-manager-c6b7d0000-000gp -n␣
↪severalnines-system -f
```

```
  ```
  $ kubectl logs kuber-agent-controller-manager-c6b7d0000-000gp -n␣
↪severalnines-system -f
```

(continues on next page)
```

(continued from previous page)

```
 2025-03-19T04:27:24Z  ERROR   Failed to install operator      {
↪"controller": "databaseoperator", "controllerGroup":
↪"agent.severalnines.com", "controllerKind": "DatabaseOperator",
↪"DatabaseOperator": {"name":"moco","namespace":"default"},
↪"namespace": "default", "name": "moco", "reconcileID":
↪"af00009c-0000-4b9a-0000-faadfbf70000", "error": "failed to install␣
↪cert-manager: failed to apply manifest: [failed to apply resource␣
↪moco-system/cert-manager-edit:␣
↪clusterroles.rbac.authorization.k8s.io \"cert-manager-edit\" is␣
↪forbidden: user \
↪"system:serviceaccount:severalnines-system:agent-operator-controller-manager\"␣
↪(groups=[\"system:serviceaccounts\" \
↪"system:serviceaccounts:severalnines-system\" \
↪"system:authenticated\"]) is attempting to grant RBAC permissions␣
↪not currently held:\n{APIGroups:[\"acme.cert-manager.io\"],␣
↪Resources:[\"challenges\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"acme.cert-manager.io\"],␣
↪Resources:[\"orders\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"certificaterequests\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"certificates\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"issuers\"], Verbs:[\"deletecollection\"]}, failed to␣
↪apply resource␣
↪moco-system/cert-manager-controller-approve:cert-manager-io:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-approve:cert-manager-io\" is forbidden:␣
↪user \
↪"system:serviceaccount:severalnines-system:agent-operator-controller-manager\"␣
↪(groups=[\"system:serviceaccounts\" \
↪"system:serviceaccounts:severalnines-system\" \
↪"system:authenticated\"]) is attempting to grant RBAC permissions␣
↪not currently held:\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"clusterissuers.cert-manager.io/*\"],␣
↪Verbs:[\"approve\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"issuers.cert-manager.io/*\"], Verbs:[\"approve\"]},␣
↪failed to apply resource␣
↪moco-system/cert-manager-controller-certificatesigningrequests:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-certificatesigningrequests\" is forbidden:␣
↪user \
↪"system:serviceaccount:severalnines-system:agent-operator-controller-manager\"␣
↪(groups=[\"system:serviceaccounts\" \
↪"system:serviceaccounts:severalnines-system\" \
↪"system:authenticated\"]) is attempting to grant RBAC permissions␣
↪not currently held:\n{APIGroups:[\"certificates.k8s.io\"],␣
```

(continues on next page)

(continued from previous page)

```
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"clusterissuers.cert-manager.io/*\"],␣
↪Verbs:[\"sign\"]}\n{APIGroups:[\"certificates.k8s.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"issuers.cert-manager.io/*\"], Verbs:[\"sign\"]},␣
↪failed to apply resource␣
↪moco-system/cert-manager-controller-approve:cert-manager-io:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-approve:cert-manager-io\" not found, failed␣
↪to apply resource␣
↪moco-system/cert-manager-controller-certificatesigningrequests:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-certificatesigningrequests\" not found]"}
␣
↪github.com/severalnines/clustercontrol-k8s/agent-operator/internal/controller.(*Da
        /
↪app/agent-operator/internal/controller/databaseoperator_controller.go:275
␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).Reconcil
        /
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).reconcil
        /
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).processN
        /
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).Start.fu
        /
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
 2025-03-19T04:27:24Z  ERROR   Reconciler error        {"controller":
↪"databaseoperator", "controllerGroup": "agent.severalnines.com",
↪"controllerKind": "DatabaseOperator", "DatabaseOperator": {
↪"name":"moco","namespace":"default"}, "namespace": "default",
↪"name": "moco", "reconcileID":
↪"af00009c-0000-4b9a-0000-faadfbf70000", "error": "failed to install␣
↪cert-manager: failed to apply manifest: [failed to apply resource␣
↪moco-system/cert-manager-edit:␣
↪clusterroles.rbac.authorization.k8s.io \"cert-manager-edit\" is␣
↪forbidden: user \
↪"system:serviceaccount:severalnines-system:agent-operator-controller-manager\"␣
↪(groups=[\"system:serviceaccounts\" \
↪"system:serviceaccounts:severalnines-system\" \
↪"system:authenticated\"]) is attempting to grant RBAC permissions␣
↪not currently held:\n{APIGroups:[\"acme.cert-manager.io\"],␣
↪Resources:[\"challenges\"],␣
```

(continues on next page)

(continued from previous page)

```
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"acme.cert-manager.io\"],␣
↪Resources:[\"orders\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"certificaterequests\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"certificates\"],␣
↪Verbs:[\"deletecollection\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"issuers\"], Verbs:[\"deletecollection\"]}, failed to␣
↪apply resource␣
↪moco-system/cert-manager-controller-approve:cert-manager-io:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-approve:cert-manager-io\" is forbidden:␣
↪user \
↪"system:serviceaccount:severalnines-system:agent-operator-controller-manager\"␣
↪(groups=[\"system:serviceaccounts\" \
↪"system:serviceaccounts:severalnines-system\" \
↪"system:authenticated\"]) is attempting to grant RBAC permissions␣
↪not currently held:\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"clusterissuers.cert-manager.io/*\"],␣
↪Verbs:[\"approve\"]}\n{APIGroups:[\"cert-manager.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"issuers.cert-manager.io/*\"], Verbs:[\"approve\"]},␣
↪failed to apply resource␣
↪moco-system/cert-manager-controller-certificatesigningrequests:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-certificatesigningrequests\" is forbidden:␣
↪user \
↪"system:serviceaccount:severalnines-system:agent-operator-controller-manager\"␣
↪(groups=[\"system:serviceaccounts\" \
↪"system:serviceaccounts:severalnines-system\" \
↪"system:authenticated\"]) is attempting to grant RBAC permissions␣
↪not currently held:\n{APIGroups:[\"certificates.k8s.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"clusterissuers.cert-manager.io/*\"],␣
↪Verbs:[\"sign\"]}\n{APIGroups:[\"certificates.k8s.io\"],␣
↪Resources:[\"signers\"],␣
↪ResourceNames:[\"issuers.cert-manager.io/*\"], Verbs:[\"sign\"]},␣
↪failed to apply resource␣
↪moco-system/cert-manager-controller-approve:cert-manager-io:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-approve:cert-manager-io\" not found, failed␣
↪to apply resource␣
↪moco-system/cert-manager-controller-certificatesigningrequests:␣
↪clusterroles.rbac.authorization.k8s.io \
↪"cert-manager-controller-certificatesigningrequests\" not found]"}
 ␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).reconcil
       /
```

(continues on next page)

(continued from previous page)

```
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).processN
       /
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
␣
↪sigs.k8s.io/controller-runtime/pkg/internal/controller.(*Controller[...]).Start.fu
       /
↪go/pkg/mod/sigs.k8s.io/controller-runtime@v0.19.4/pkg/internal/controller/controll
  2025-03-19T04:27:24Z  INFO    status-predicate        DIAGNOSTIC:␣
↪Detected status-only change, filtering out reconciliation   {"name":
↪"moco", "kind": "", "oldGeneration": 1, "newGeneration": 1,
↪"resourceVersion": "24302778", "oldResourceVersion": "24302764"}
  2025/03/19 04:27:25 Received message from proxy: {
↪"group":"agent.severalnines.com","version":"v1alpha1","kind":"DatabaseOperator","na
↪(list_resources)
  ```
```

### 2.86.2 Failed backups

- **List database backups:**

```
kubectl get databasebackup
NAME                                                                STATUS␣
↪  AGE   STARTED   COMPLETED
cnpg-cluster-backup-cnpg-cluster-backup-backup-20250319055800   Failed␣
↪  26s
```

- **Check CNPG backup status:**

```
kubectl get backup
NAME                                              AGE   CLUSTER       ␣
↪METHOD              PHASE    ERROR
cnpg-cluster-backup-backup-20250319055800   94s   cnpg-cluster  ␣
↪barmanObjectStore   failed   can't execute backup: cmd: [/
↪controller/manager backup cnpg-cluster-backup-backup-20250319055800].
↪..
```

- **Get backup details:**

```
kubectl describe backup cnpg-cluster-backup-backup-20250319055800
```

- **Error:**

```
Status:
Error:  can't execute backup: cmd: [/controller/manager backup␣
↪cnpg-cluster-backup-backup-20250319055800]
error: command terminated with exit code 1
stdErr: {"level":"info","ts":"2025-03-19T05:58:22.131051022Z","msg":
↪"Error while requesting backup","logging_pod":"cnpg-cluster-2",
```

(continues on next page)
```

(continued from previous page)

```
↪"backupURL":"http://localhost:8010/pg/backup","statusCode":500,
↪"body":"error while requesting backup: while starting backup: cannot␣
↪recover backup credentials: while getting secret s3-credentials:␣
↪secrets \"s3-credentials\" not found\n"}
```

2.86.3 Node maintenance and troubleshooting

- **Check node status**

```
kubectl get nodes
NAME                    STATUS   ROLES    AGE   VERSION
pool-n97uu1q8u-edf02    Ready    <none>   57d   v1.31.1
```

- **Debug node issues**

  Start a debug container on a node:

```
kubectl debug node/pool-n97uu1q8u-edf02 -it --image=ubuntu:latest -- /
↪bin/bash
```

- **Manage container runtime**

  Access the host filesystem:

```
chroot /host
```

  List container images:

```
crictl images ls
```

  List all Containers (including stopped):

```
crictl ps -a
```

- **Disk usage management**

  Check log usage:

```
du -sh /var/log/*
```

  Check container runtime storage:

```
du -sh /var/lib/containerd/*
```

- **Cleanup tasks**

  Remove stopped containers:

```
crictl ps -a -o json | jq -r '.containers[] |␣
↪select(.state=="CONTAINER_EXITED") | .id' | xargs -r crictl rm
```

  Remove dangling images:

```
crictl images -o json | jq -r '.images[] | select((.repoTags == null)␣
↪or ((.repoTags | length)==0)) | .id' | xargs -r crictl rmi
```

2.87 Cloud providers

ClusterControl supports integration with S3-compatible storage solutions from multiple providers:

- Amazon Web Services (AWS)
- Google Cloud (via Google Cloud Storage)
- Microsoft Azure (via an S3-compatible interface or directly via credentials)
- Other S3-Compatible storage providers (e.g., MinIO, DigitalOcean Spaces, OpenStack, etc.)

Once configured, you can use S3 storage as an offsite storage for backups created by ClusterControl.

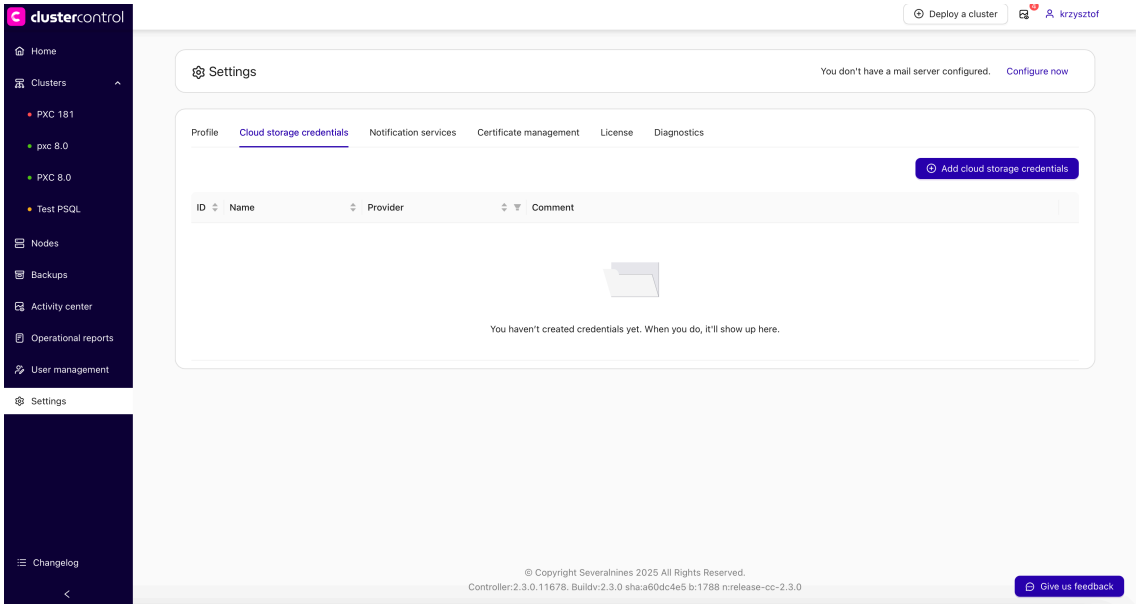
2.87.1 Prerequisites

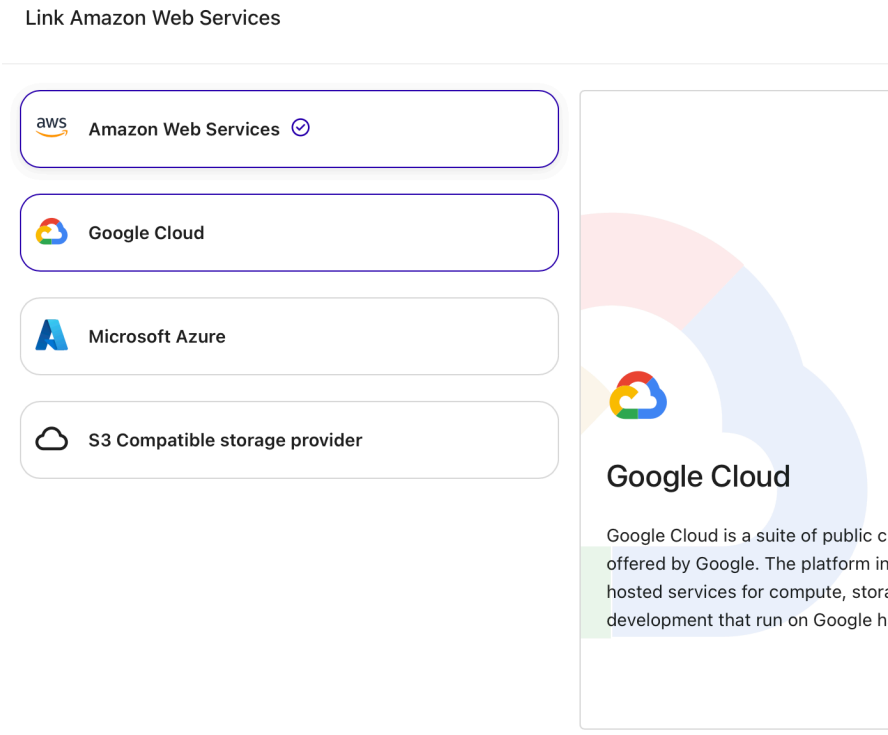
1. Appropriate Access Credentials
   - AWS: Access key ID and secret access key.
   - Google Cloud: Service account JSON key file.
   - Microsoft Azure: Service principal credentials, or an S3-compatible credential file (depending on your configuration).* Other S3 Providers: Typically an access key, secret key, region, and a custom endpoint.
2. Correct Region or Endpoint
   - AWS: Region selection (e.g., eu-west-2).
   - Google Cloud: Not strictly a region input—this is handled by the JSON credentials.
   - Azure: May require an endpoint or rely on the JSON credentials if you have an S3-compatible gateway set up.
   - Other S3: Custom endpoint (e.g., https://nyc3.digitaloceanspaces.com), plus region if required.
3. Permissions in the Platform
   - Ensure you have privileges to create/edit integrations.

2.87.2 Creating a New S3 Integration

Access the Integrations Section

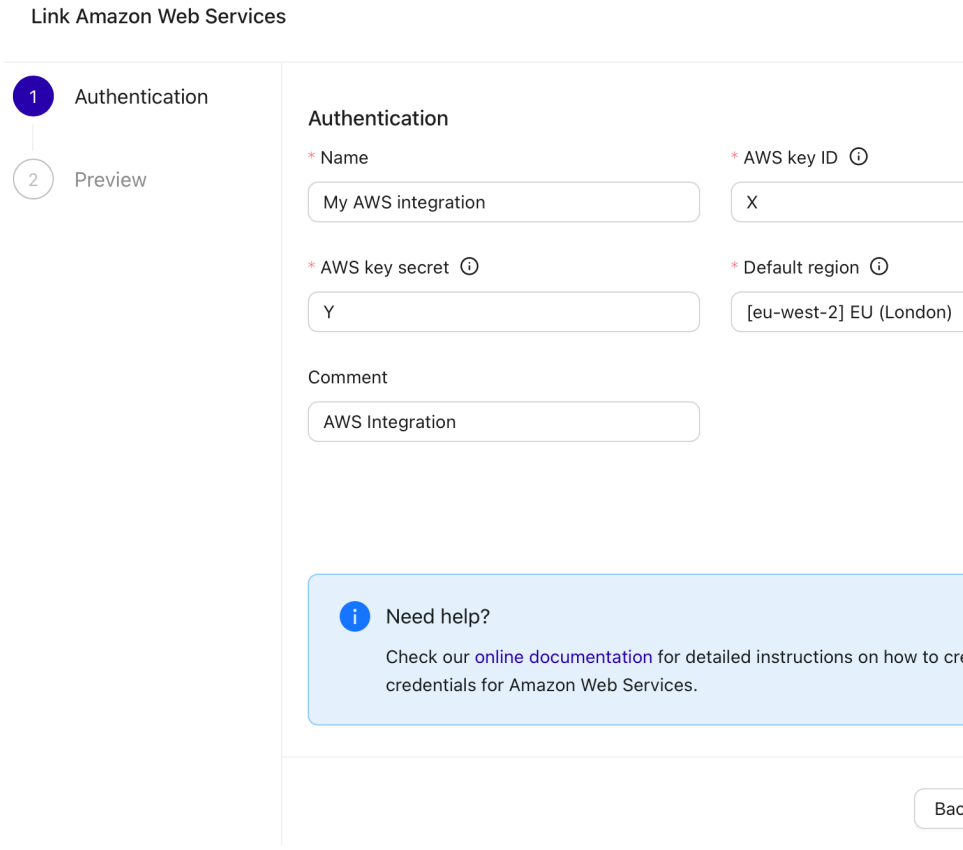
1. In ClusterControl, navigate to the ***Settings→Cloud storage credentials*** section. There is an option to **Add cloud storage credentials**.



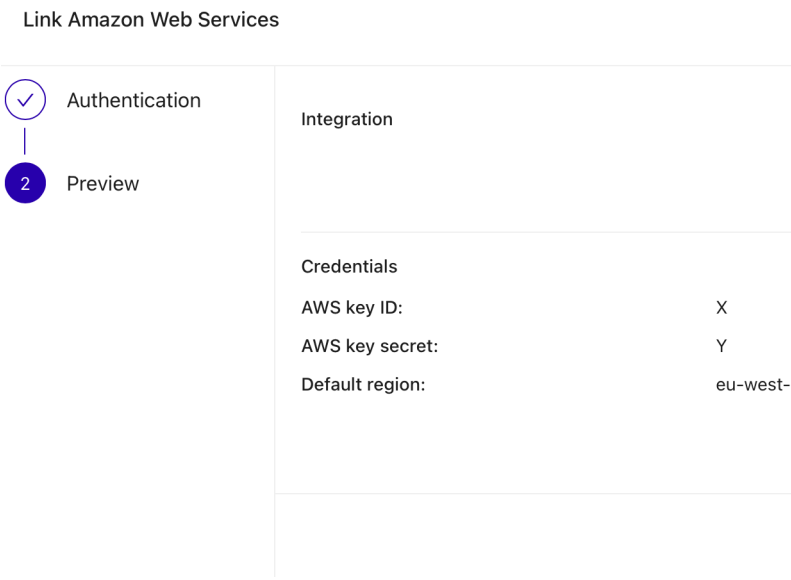


2. Click **Add cloud storage credentials**.
   You will see a list of supported providers:
   - Amazon Web Services
   - Google Cloud
   - Microsoft Azure
   - S3 Compatible storage provider
3. Select the one you want to use.

Setting up Amazon Web Services S3

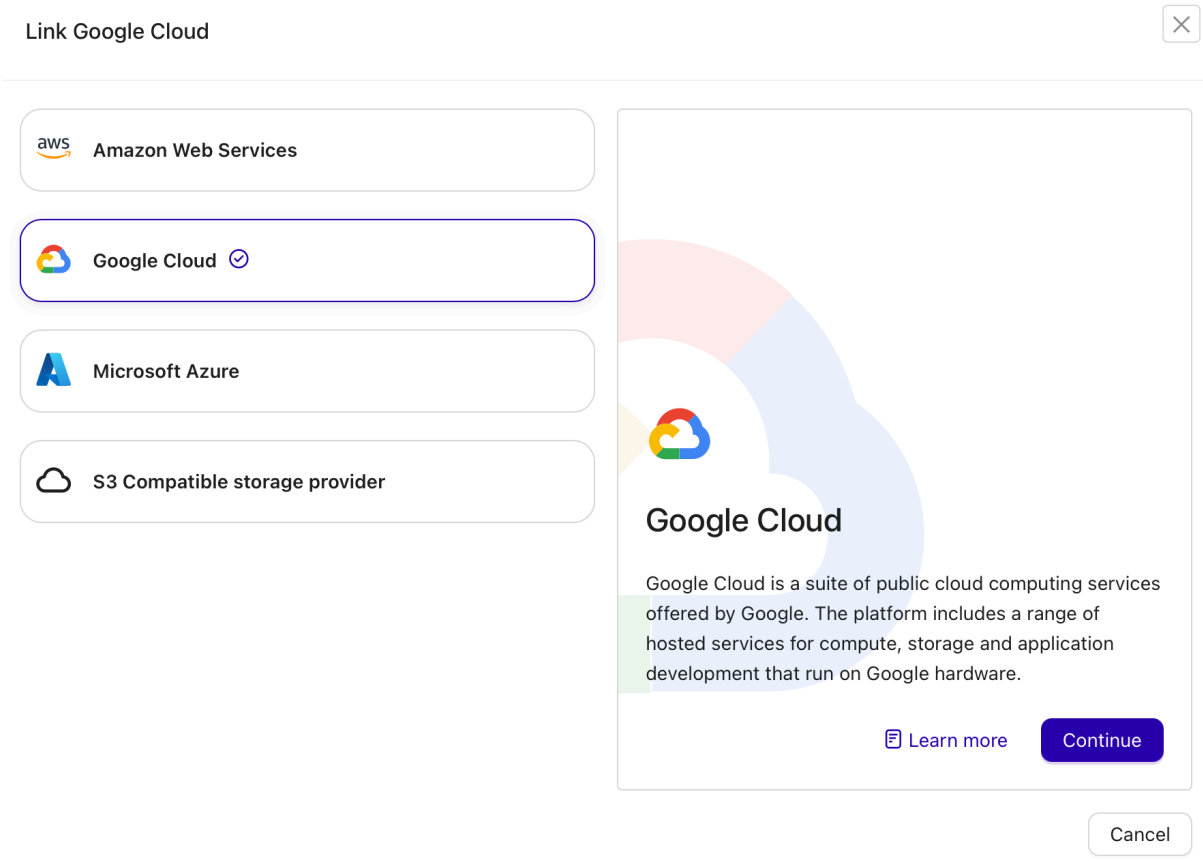


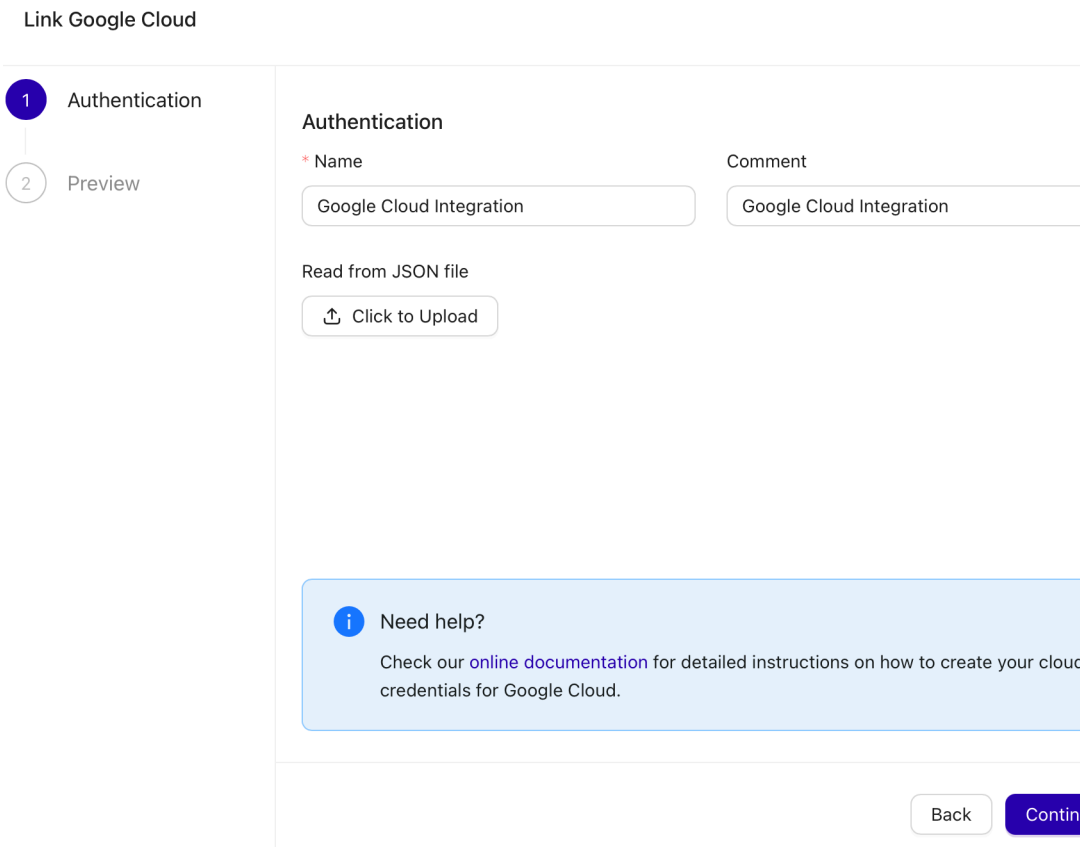
1. Select “Amazon Web Services”.
2. Enter Credentials in the Authentication step:
   - Name: A descriptive label, e.g. My AWS Integration.
   - AWS key ID: Your AWS access key ID.
   - AWS key secret: Your AWS secret access key.
   - Default region: Choose your primary region, e.g. [eu-west-2] EU (London).
   - Comment: Optional internal notes (e.g., AWS integration).



3. Click **Continue** to move to the Preview screen.
4. Review the details (key ID, region, etc.) and click **Finish** to create the integration.

Setting up Google Cloud





1. Select Google Cloud.
2. Enter Credentials in the Authentication step:

- Name: Enter a descriptive label, e.g. Google Cloud Integration.
- Comment: Optional internal description.
- Read from JSON file: Click the **Click to Upload** button to browse for and upload your Google Cloud service account JSON file.

This JSON file contains the keys and project info needed to authenticate with Google Cloud Storage. For details on creating this JSON file, see the Google Cloud documentation on service accounts.

3. Click **Continue** to view the Preview screen.
4. Verify the integration name and any other details you entered. If correct, Click **Finish** to save the integration.

Setting up Microsoft Azure

1. Select "Microsoft Azure" from the provider list.
2. Enter Credentials in the Authentication step:
   - Name: e.g. Azure Integration.
   - Comment: e.g. Azure integration for S3 storage.
   - Read from JSON file: Upload your Azure credential JSON.

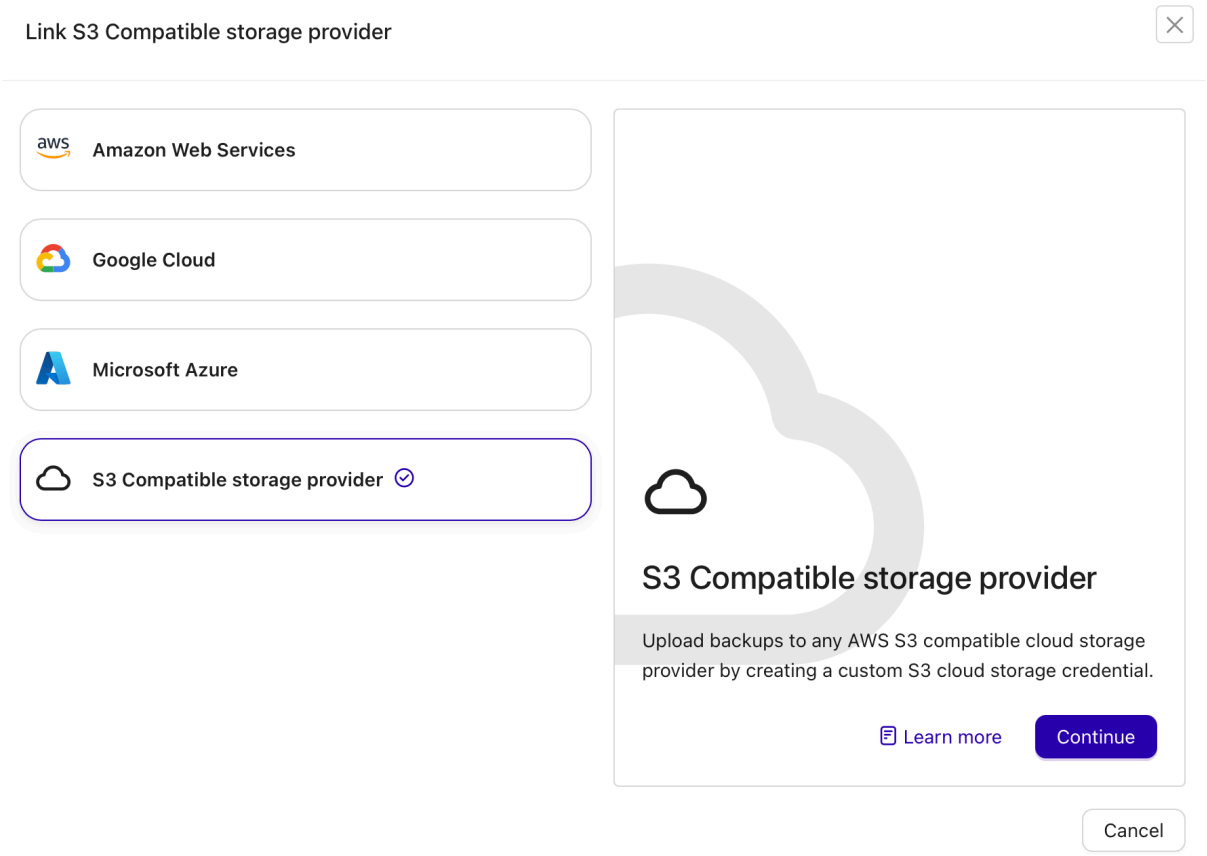
   The contents and format of this file depend on your Azure setup. It may include your storage account, tenant ID, client ID, secret, etc.
3. Click **Continue**, then Preview the integration details.

4. Verify the integration name and any other details you entered. If correct, Click **Finish** to save the integration.

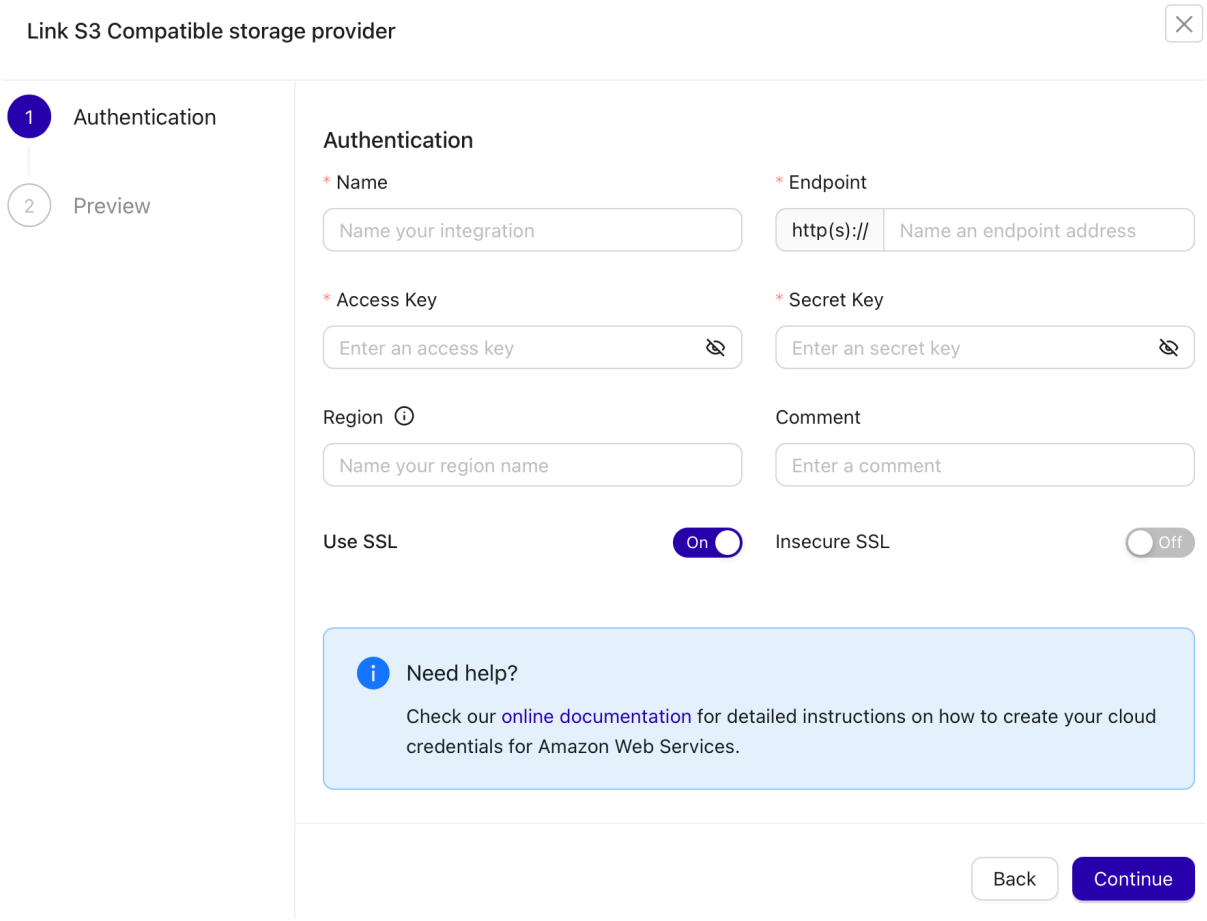
Tip

If you are using an S3-compatible gateway for Azure (like Blob storage with S3 proxy), you might instead follow the Other S3 steps and provide the custom endpoint, access key, and secret that the gateway supplies.

Setting up Other S3-Compatible Providers



Select "S3 Compatible storage provider" if your preferred provider is not explicitly listed. This covers services like MinIO, DigitalOcean Spaces, OpenStack etc.



1. Select “S3 Compatible storage provider” from the provider list.
2. Enter required data in the Authentication step:
   - Name: e.g. My MinIO Integration.
   - Endpoint: The HTTPS URL of your service (e.g. https://nyc3.digitaloceanspaces.com).
   - Access Key: The access key from your S3-compatible account.
   - Secret Key: The secret key from your S3-compatible account.
   - Region: If your provider requires a specific region name (e.g. us-east-1, us-central), enter it here. Not all providers strictly require it.
   - Comment: Optional notes about this integration.
   - Use SSL: On by default. If your provider uses SSL, keep it on. Otherwise, switch to “Off” or “Insecure SSL” if you must connect insecurely (not recommended).
3. Click **Continue**, then Preview the integration details.
4. Verify the integration name and any other details you entered. If correct, Click **Finish** to save the integration.

2.88 Modifying existing integrations

Once you've created an integration (AWS, Google Cloud, Azure, or Other S3), it will appear in your Integrations list.

If you encounter issues, check:

- Credentials: Are the keys and secrets correct (no typos)?
- Endpoint/Region: Did you select the correct region or endpoint for your service?
- Permissions: Does your IAM policy, its equivalent or service account allow read/write access?
- File Format: For Google Cloud and Azure, verify the JSON file you uploaded is the correct one for authentication.

Beginning with ClusterControl 1.8.0, it is possible to integrate HashiCorp's Vault for storing credentials used in ClusterControl. When the controller starts up, it will connect to the Vault server and load the credentials.

See also

See ClusterControl's Vault Integration Adds Dynamic Data Security Protections on example configuration on ClusterControl with HashiCorp Vault.

2.89 Requirements

The following is required in order to use the Vault Integration:

- A running Vault Server. It must be reachable from the Controller. This has been tested with Vault v1.4.3.
- ClusterControl version 1.8.0 or later.

2.90 Configuration

The controller must be configured to use Vault. The following parameters are set in `/etc/cmon.cnf` (do not set in `cmon.d/cmon_X.cnf`):

Parameter	Description
`vault_t`	A token used by the controller to authorize itself against the Vault server. The Vault token is created by the Vault server. When the vault_token is present credentials for new clusters will be stored in Vault and not in the cmon configuration files.
`vault_a`	The full URL (including protocol and port) where the Vault server is listening. You can set it to the same as you would set `VAULT_ADDR` environment variable for vault CLI. Default is `https://localhost:8200`.
`vault_p`	Default value is clustercontrol. The controller is going to store the data within this path, e.g `clustercontrol/controller_UUID/cluster_ID/mysql_password`. Each ClusterControl instance and controller, and cluster can have its unique namespace.
`vault_a`	Enable this to auto-move all existing secrets from existing cmon configuration files to the configured Vault instance.

Warning

Warning!

Passwords will no longer be present in the CMON configuration files after this operation. Backup the configuration files and ensure that everything is working as expected before removing the backed up config files. The main `/etc/cmon.cnf` file is not modified.

2.91 Starting up

The Vault integration feature is enabled as soon as the `vault_token` configuration parameter is set. At this stage, provided there is a Vault server listening on the `vault_addr`, then any new cluster that is created will store its credentials in Vault.

Credentials of existing clusters are not affected until `vault_auto_migrate` have been enabled.

Since the Vault settings are applied to `/etc/cmon.cnf`, then these are global for all clusters. Hence, it is not possible to migrate one cluster at a time.

2.92 Reverting the Vault integration

Should you want to revert this process, you should follow these steps:

1. Get all the keys and values from the Vault for all of the clusters.
2. Stop cmon process.
3. Remove Vault configuration from `/etc/cmon.cnf`.
4. Add the keys and values you extracted from the Vault into respective configuration files in `/etc/cmon.d`. You can use the backup copy of the configuration from before Vault has been used but please keep in mind that some additional passwords might have been added in the meantime – double-check that your old configuration files contain all the data that is stored in the Vault.

See also

See ClusterControl's Vault Integration Adds Dynamic Data Security Protections on example configuration on ClusterControl with HashiCorp Vault.

2.93 Limitations

- In this version, ClusterControl only supports the KV v2 secrets engine in Vault.
- Currently, the Controller must be restarted to re-read credentials from Vault.
- Updating a credential in Vault will not change the underlying credential on the database node. Thus, when changing a password, then the account must be updated on the database node(s).
- Not been tested with CMON HA.
- The cmon-events, cmon-cloud, cmon-ssh services have not been migrated to use Vault yet. Hence the passwords are still present in `/etc/cmon.cnf`. You can ensure that the config files used by these services are reconfigured and then drop the passwords completely from `/etc/cmon.cnf`.

The Terraform Provider for ClusterControl offers a seamless integration between the powerful infrastructure management tool, Terraform, and the robust database management capabilities of ClusterControl. This provider empowers users to effortlessly provision, configure, and manage database clusters through Infrastructure as Code (IaC), streamlining the deployment process and enhancing the efficiency of managing complex database environments.

With a range of customizable configurations and automation capabilities, this provider simplifies the orchestration of database infrastructure, enabling teams to focus on innovation and development rather than mundane operational tasks. Leveraging the flexibility and scalability of Terraform alongside the comprehensive management features of Severalnines ClusterControl, organizations can achieve greater agility, reliability, and scalability in their database operations.

This project is hosted in Github, terraform-provider-clustercontrol.

2.94 Requirements

Name	Version
Terraform	>= 0.13.x
ClusterControl	>= 1.9.8

2.95 Providers

Name	Version
Terraform ClusterControl Provider	>= 0.1

2.96 Resources

Name
clustercontrol_db_cluster
clustercontrol_db_cluster_backup
clustercontrol_db_cluster_backup_schedule
clustercontrol_db_cluster_maintenance

2.97 Quick start

2.97.1 Installing and configuring ClusterControl for API access

To get started, you may follow the following steps (if you already have ClusterControl running in your env, skip to #4):

1. *Getting started with ClusterControl*
2. *ClusterControl requirements*
3. *Install ClusterControl*

4. Configure ClusterControl – Enable ClusterControl for API access by editing `/etc/default/cmon` with a text editor and set the `RPC_BIND_ADDRESSES` as shown below:

```
RPC_BIND_ADDRESSES="10.0.0.5,127.0.0.1"
```

   Where `10.0.0.5` is the private IP of the ClusterControl host. Restart the ClusterControl service to apply the changes:

```
sudo systemctl restart cmon
```

5. Run a quick test to make sure you can access ClusterControl via its REST API (using curl or Postman):

```
curl -k 'https://10.0.0.5:9501/v2/clusters' -XPOST -d '{"operation":
↪"getAllClusterInfo", "authenticate": {"username":
↪"CHANGE-ME","password": "CHANGE-ME"}}'
```

   Where `username` and `password` are valid login credentials for ClusterControl. The output should be a load of JSON text returned with details of the clusters managed by ClusterControl.

2.97.2 Setting up the Terraform provider for ClusterControl

In your environment, create a `terraform.tfvars` file to store your ClusterControl secrets. In the file, add the following secrets:

```
cc_api_url="https://<cc-host-private_ip>:9501/v2"
cc_api_user="CHANGE-ME"
cc_api_user_password="CHANGE-ME"
```

Create a file (e.g `main.tf`) to store the ClusterControl Terraform provider configuration and the resources you will create. In the file, add the following configuration:

```
terraform {
  required_providers {
    clustercontrol = {
      source = "severalnines/clustercontrol"
      version = "0.2.16" // refer to terraform registry for latest version
    }
  }
}

variable "cc_api_url" {
  type = string
}

variable "cc_api_user" {
  type = string
}

variable "cc_api_user_password" {
  type = string
}
```

(continues on next page)

(continued from previous page)

```
provider "clustercontrol" {
  cc_api_url = var.cc_api_url
  cc_api_user= var.cc_api_user
  cc_api_user_password= var.cc_api_user_password
}
```

The above Terraform code will install the Terraform Provider for ClusterControl, import the secrets from the `terraform.tfvars` and configure the provider for use across the environment.

Run `terraform init` to initialize the directory and install the provider. You can also verify the correctness of the configuration with `terraform validate` command.

With your ClusterControl Terraform provider configuration validated, you can now deploy database clusters.

2.97.3 Deploying database clusters using Terraform

To deploy a database cluster with the ClusterControl Terraform provider, you use the *clustercontrol_db_cluster* resource. The following example `clustercontrol_db_cluster` resource schema deploys a PostgreSQL cluster:

```
resource "clustercontrol_db_cluster" "test-postgresql" {

    db_cluster_create = true
    db_cluster_name = "PostgreSQL Cluster"
    db_cluster_type = "pg-replication"
    db_version = "15"
    db_vendor = "postgresql"
    db_admin_user_password = var.cc_api_user_password
    db_host {
        hostname = "test-primary"
    }
    db_host {
        hostname = "test-replica"
    }
    ssh_key_file = "/root/.ssh/id_rsa"
    ssh_user = "root"
    db_deploy_agents = true
    disable_firewall = true
    disable_selinux = true
    db_install_software = true

}
```

In the above resource:

- The `db_cluster_type`, `db_vendor` and `db_version` attributes define the type of cluster, vendor and version respectively.
- `db_cluster_create` tells the provider to create a new cluster. If you are importing an already existing cluster, you will use the `db_cluster_import` attribute.

- The `db_host` blocks are the list of nodes in the cluster. The hostname of the DB node can also be its IP address as well. You can configure your `db_host` node configurations with its nested schema.
- ClusterControl uses passwordless SSH to deploy and manage the database cluster automatically. The `ssh_user` attribute holds the SSH user ClusterControl will use to SSH to the DB nodes from the ClusterControl node, and `ssh_key_file` the path to the private key file for the Sudo user on the ClusterControl node.
- The `db_deploy_agents` attribute set to "true", automatically deploys prometheus and other relevant agents for observability after setting up the initial database cluster.
- If the nodes are new without any database package pre-installed, this resource has the `db_install_software` attribute set to "true". This tell ClusterControl to install all the database packages it needs from their respective repos, and `disable_firewall` and `disable_selinux` attributes disables security configurations on the node OS to enable ClusterControl download and installs the DB packages.

To apply the above resource run the following commands:

```
terraform plan    # to preview the configuration changes
terraform apply   # to apply the configuration
```

After a few minutes, the cluster creation will complete and you should see a final output as below.

```
clustercontrol_db_cluster.test-postgresql: Creation complete after 9m20s␣
↪[id=1]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

You can verify that the cluster is using the *ClusterControl GUI* or the *ClusterControl CLI (s9s-tools)* which both come with your ClusterControl installation.

For more examples, see the examples documentation on deploying other types of database clusters – MySQL/MariaDB replication or Galera with ProxySQL, MongoDB replica set and/or sharded, Redis Sentinel, etc. Navigate to the docs directory for generated documentation on the Terraform provider plugin for ClusterControl.

2.97.4 Destroying a deployed database cluster

After navigating to the appropriate directory which was used to deploy a cluster:

```
terraform destroy
```

ClusterControl supports integration with directory services like Active Directory, FreeIPA, and OpenLDAP authenticatio using lightweight directory access protocol (LDAP). This allows users to log into ClusterControl by using their corporate credentials instead of a separate password. LDAP groups can be mapped onto ClusterControl user groups to apply privileges to the entire group. This ensures that ClusterControl authorizes the logged-in user based on the group assigned. It supports up to the LDAPv3 protocol based on RFC2307.

To integrate with a directory service, one has to perform the following steps:

1. Fill up the ***LDAP Settings*** configuration wizard. See *LDAP Settings*.
2. Save the settings. At this point, LDAP is saved but not activated because no group mapping has been created yet.

3. Create at least one group mapping entry by going to ***Map LDAP Group***. See *Map LDAP Group*.
4. Enable the LDAP authentication by toggling on the ***LDAP Settings → Enable LDAP Authentication***.
5. Log in to ClusterControl by using the value of LDAP ***Username Attributes*** with the respective password. The user will be authenticated and redirected to the ClusterControl dashboard page based on the assigned group.
6. From this point, both ClusterControl native authentication and LDAP authentications are enabled.

Warning

Attention

The last entry of the group mapping can not be deleted while LDAP authentication is enabled and activated. See *Delete mapping group*.

2.98 LDAP settings

Field	Description
General Settings	
Enable LDAP Authentication	Enables LDAP authentication. The native authentication in ClusterControl will also work. Refer to the steps mentioned above, on enabling LDAP authentication.
LDAP URI	Enter the LDAP or LDAPS Uniform Resource Identifier (URI), with the port number (if applicable). An example is `ldaps://ad.s9s.com:636`. For LDAPS, provide the certificates and key files under **SSL/TLS Settings** section.
Login DN	The Distinguished Name is used to bind the LDAP server. This user requires read access to all LDAP users and group entries to work correctly. ClusterControl must perform an LDAP search using the DN before any user can log in. This field is case-sensitive. An example is `cn=Administrator,cn=Users,dc=s9s,dc=com`.
Login DN Password	The password for Login DN.
User Base DN	The Distinguished Name (DN) to locate the users' information. This field is case-sensitive. An example is `cn=Users,dc=s9s,dc=com`.
Group Base DN	The Distinguished Name (DN) to locate the group information. ClusterControl does not support LDAP users that do not belong to at least one LDAP group. An example is `ou=Groups,dc=s9s,dc=com`.
Advanced	
Prefill for	Fill up all required fields based on common settings for Active Directory or OpenLDAP. Clicking on **Reset** will clear all textfields.
User Base Filter	Filter the object class of the LDAP users. If empty, all object classes will be returned.
Username Attribute	The LDAP attributes which hold the username, separated by a comma (whitespace value is not allowed). For Active Directory, this is commonly `sAMAccountName` and `uid` for OpenLDAP. When authenticating with ClusterControl (via ClusterControl GUI login page, ClusterControl CLI or RPC API), use the value of this attribute. If more than one attribute is specified, ClusterControl will attempt to look up all of them with the first non-empty reply (in the particular order) used for the login username.
Real Name Attribute	The LDAP attributes which hold the full name of the user, separated by a comma. For Active Directory, this is commonly `displayName` and `cn` for OpenLDAP. This value of this attribute will be used as the first and last name of the user. If more than one attribute is specified, ClusterControl will attempt to look up all of them with the first non-empty reply (in the particular order) used for the user's full name.
Email Attribute	The LDAP attributes which hold the email address of the user, separated by a comma. For Active Directory, this is commonly `userPrincipalName` and `mail` for OpenLDAP. This value of this attribute will be used as the email address of the user. If more than one attribute is specified, ClusterControl will attempt to look up all of them with the first non-empty reply

2.99 Map LDAP group

ClusterControl requires at least one LDAP group mapping entry to be defined before the LDAP authentication can be activated. Otherwise, ClusterControl would only save the LDAP settings without activating them.

To create a group mapping:

1. Click on the ***Map LDAP Group*** button.
2. Choose an existing team from the dropdown and specify the value of the LDAP's ***Group Name Attributes*** that you want to be mapped with the ClusterControl team. For single-tier RDN, specify the attribute value for example "DBA". ClusterControl will prepend the **Group name attribute** configured under the **LDAP Settings** with the value specified, and append it with the **Group base DN** value. The final constructed DN will be `{Group name attribute}={value specified}, {Group base DN}`. For multi-tier RDNs, specify the attribute name and value (for multiple RDNs, delimited by a comma) for example `cn=DBA,cn=IT,ou=org1`. ClusterControl will append the value specified with the Group base DN value under the LDAP Settings. The final constructed DN will be `{value specified},{Group base DN}`.
3. After creating the first entry, you may proceed to activate the LDAP authentication by going to ***LDAP Settings → Enable LDAP Authentication***.

All LDAP configurations and mappings will be stored inside a configuration file at `/etc/cmon-ldap.cnf`. It is recommended to configure LDAP settings and group mappings via the ClusterControl GUI because any changes to this file will require a reload to the controller process, which is triggered automatically when configuring LDAP via the GUI.

Note

Note It is also possible to modify the configuration file directly, however, the `cmon` service must be reloaded manually by using the `systemctl restart cmon` command, or `service cmon restart`.

2.100 Delete mapping group

Click on **Delete** of the entry in the list to delete it. The last entry of the group mapping can not be deleted while LDAP authentication is enabled and activated (the button will be greyed out). To delete the last entry, set **Enable LDAP Authentication** to OFF and you will notice the **Remove Group** button is now clickable, allowing you to remove the last group mapping entry and effectively deactivating LDAP authentication for ClusterControl.

Configures third-party notifications on events triggered by ClusterControl. This feature allows users to integrate ClusterControl into your organization's communication channels, incident response systems, and workflow applications. To prevent users from receiving too many notifications, ClusterControl integrations also allow users to send out only specific critical or warning alerts (see *Warning Events* and *Critical Events*).

See also

Introducing the ClusterControl Alerting Integrations.

2.101 Supported integrations

Supported services are:

Service type	Service provider
Incident management	PagerDuty Splunk On-Call (formerly known as VictorOps) OpsGenie ServiceNow ilert
Messaging platform	Slack Telegram
Others	Webhook

2.102 Notification behavior

Within ClusterControl internal states, there are 3 types of events:

- CREATED - An alarm was raised.
- CHANGED - Something changed in the alarm event.
- ENDED - The alarm was resolved.

However, by default, the ClusterControl Notification process (`cmon-events`) will only listen to 2 events - **CREATED** and **ENDED** events - and pass them to the configured services. This can be configured with `allowed_events` CLI parameter.

The notification behavior is depending on the type of services that receives the event, as shown below:

Service	Notification behavior
Email	CREATED - Will send an email ENDED - Will send an email
Incident management	CREATED - Will create an incident ENDED - Will resolve an incident
Messaging platform	CREATED - Will send a message ENDED - Will send a message
Webhook	CREATED - Will send a HTTP POST request to the webhook ENDED - Will send a HTTP POST request to the webhook See *Webhook integration* for examples.

Warning

Attention

If you already have an alarm raised and then you create, for example, a Slack (or any other) integration, you will never see events for the alarms already raised/created before you created the Slack integration. You will only see alarms created after the integration is set up.

2.103 Set up notification

Every supported service has different requirements and configurations as shown in the following subsections.

2.103.1 Slack

Slack is a cloud-based collaboration platform designed for teams to communicate and work together in real time. For Slack integration, the following information are needed:

- **Slack workspace URL** - The webhook URLs for your workspace. Commonly it looks like this: `https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXXXXXXXXXXXX`
- **Channel** - The Slack's channel name.

The following are the steps to get the required information:

1. Create a Slack app by going to Your Apps.
2. Go to ***Features → Incoming Webhooks*** and toggle on **Activate Incoming Webhooks**.
3. Click on **Add New Webhook to Workspace** and choose a Slack channel from the workspace in the dropdown and click **Allow**. Take note on the channel name that you choose.
4. Copy the generated **Webhook URL**. This is the Slack workspace URL that will be used in the integration.

Tip

See Slack documentation - Sending messages using incoming webhooks.

To configure Slack integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → Slack***.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give a name to this integration.
   - **Workspace URL**: The webhook URL of your workspace, taken from the Slack app page.
   - **Channel**: The channel that ClusterControl will post the events, prefixed it with a hash tag (#).
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.103.2 Telegram

Telegram is a cloud-based instant messaging app available on mobile, desktop, and web that lets you send text and voice messages, make voice and video calls, and share photos, videos, and files of any type. For Telegram integration, the following information are needed:

- **Telegram bot token** - The telegram bot API authentication token. Commonly it looks like this: `999999999:AAHfiqksKZ8WmR2zSjiQ7_v4TMAKdiHm8T0`.
- **Channel** - The Telegram's channel name.

The following are the steps to get the required information:

1. Open Telegram and create a new bot by contacting `@BotFather`.
2. Give the bot a unique name and copy the bot token. This is the Telegram bot token that is required.
3. Create a new Telegram channel (or use an existing channel).
4. Add the created bot to the channel as Administrator.
5. Allow your bot to send messages to the channel.
6. Copy channel ID from the channel link. For example, if the link is `t.me/mypowerfulchannel`, the channel ID is `mypowerfulchannel`.

Tip

See Telegram documentation - Bots.

To configure Telegram integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → Telegram***.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give a name to this integration.
   - **Telegram BOT token**: The telegram bot API authentication token, provided by Telegram bot `@BotFather`.
   - **Channel**: The channel that ClusterControl will post the events, prefixed it with an alias (`@`).
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.103.3 ServiceNow

ServiceNow is a cloud-based platform for digital workflow automation and enterprise service management. For ServiceNow integration, the following information are needed:

- **Username** - The instance username.
- **Password** - The password for the instance user.
- **Service** - The Service name that you want to monitor.
- **Configuration item** - The ServiceNow components that need to be tracked.
- **Instance** - The instance name.

Tip

See ServiceNow documentation.

To configure ServiceNow integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → ServiceNow***.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give a name to this integration.
   - **Username** - The instance username.
   - **Password** - The password for the instance user.
   - **Service** - The Service name that you want to monitor.
   - **Configuration item** - The ServiceNow components that need to be tracked.
   - **Instance** - The instance name.
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.103.4 PagerDuty

PagerDuty is an incident management platform for IT operations, enabling teams to detect and respond to service disruptions and outages in real time. For PagerDuty integration, the following information is needed:

- **Service key** - Integration key for PagerDuty Events API v2. Commonly it looks like this: `aaaaa75294fe496b8ba9a87f71bddddd`.

The following are the steps to get the required information:

1. Login to your PagerDuty account.
2. Create a new service by going to ***Services → Service Directory → New Service***.
3. Give the Service a name and generate an (or select an existing) escalation policy.
4. In the **Integration** section, choose "Events API V2". Generate a new **Integration Key** for this integration and this is the **Service key** that is required.

Tip

See PagerDuty documentation - Services and Integrations.

To configure PagerDuty integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → PagerDuty***.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give any name to this integration.
   - **Service Key**: The PagerDuty Events API v2 integration key of the chosen service.
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.103.5 Splunk On-Call (formerly known as VictorOps)

Splunk On-Call (formerly known as VictorOps), is an incident management platform for IT operations, enabling teams to detect and respond to service disruptions and outages in real time. For Splunk On-Call integration, the following information is needed:

- **REST Integration URL** - The Splunk On-Call Service API Endpoint you get when you enable the REST integration. Commonly it looks like this: `https://alert.victorops.com/integrations/generic/<YOUR_REST_ENDPOINT_KEY>/alert/<YOUR_ROUTING_KEY>`.

The following are the steps to get the required information:

1. Login to your Splunk On-Call/VictorOps account.
2. Create a new integration by going to ***Settings → Integrations → 3rd Party Integrations → REST – Generic***.
3. Click on the **Enable** button to generate an endpoint destination URL.

4. Routing keys in Splunk On-Call can be set up and associated by clicking on ***Settings → Routing Keys***. Routing key must be included as part of the **REST integration URL**.

Tip

See Splunk On-Call documentation - REST Endpoint Integration Guide.

To configure Splunk On-Call integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → VictorOps***.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give any name to this integration.
   - **REST Integration URL**: The Splunk On-Call integration URL for REST API. The URL shall include the REST endpoint key and routing key.
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.103.6 OpsGenie

Opsgenie is a cloud-based incident management and on-call scheduling platform (part of the Atlassian suite) that centralizes alerts from your monitoring, ticketing, and collaboration tools. For Opsgenie integration, the following information are needed:

- **Region** - The Opsgenie data-center your account lives in, so your HTTP calls go to the correct endpoint. It is either US or EU.
- **Teams** - The Opsgenie team identifiers or names that should receive and handle the alerts generated by this integration.
- **API key** - The unique "GenieKey" token created when you add an API integration under ***Settings → Integrations → API***.

The following are the steps to get the required information:

1. Login to your OpsGenie account.
2. Create a new integration by going to ***Settings → Integrations → API***.
3. Click **Continue**, expand **Steps to configure the integration**, then copy the **API key**.
4. Click **Turn on integration** to enable it.

Tip

See Opsgenie Support - Create an API Integration.

To configure OpsGenie integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → OpsGenie***.
2. In the **Service Configuration** section, specify the following:
   - **Region**: Choose whether US or EU region.
   - **Teams**: The Splunk On-Call integration URL for REST API. The URL shall include the REST endpoint key and routing key.
   - **API Key**: The unique "GenieKey" token created when you add the API integration.
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.103.7 ilert

ilert is an AI-first company offering an all-in-one incident management tool for alerting, on-call management, and incident communication to help companies increase their digital uptime. B2C and B2B companies from across the globe, including well-known brands such as IKEA, Lufthansa Systems, and NTT Data, trust ilert to empower their operations teams and ensure everything is running smoothly. For ilert integration, the following information is needed:

- **Alert source URL** - The ilert API alert source URL generated in the ClusterControl integration page for ilert. Commonly it looks like this: `https://api.ilert.com/api/v1/events/clustercontrol/xxxxxxxxxxxxxxxxxxxxxxxxxxxx`.

The following are the steps to get the required information:

1. Login to your ilert account.
2. Go to ***Alert sources → Alert sources*** and click ***Create new alert source***.
3. Search for "ClusterControl" in the search field, click the ClusterControl tile, and then **Next**.
4. Give your alert source a name, optionally assign teams, and click **Next**.
5. Select an escalation policy by creating a new one or assigning an existing one.
6. On the final page, an ClusterControl URL will be generated. Use this information for the **URL** when you configure ClusterControl integration.

Tip

See ilert Documentation - ClusterControl Integration.

To configure ilert integration:

ClusterControl GUI

1. Go to *ClusterControl GUI → Settings → Notification Services → Add new integration → ilert*.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give any name to this integration.
   - **URL**: The ilert ClusterControl integration URL generated from the alert source integration dashboard.
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.104 Webhook integration

A webhook is a lightweight API that powers one-way data sharing triggered by events. Together, they enable applications to share data and functionality, and turn the web into something greater than the sum of its parts. APIs and webhooks both allow different software systems to sync up and share information. For webhook integration, the following information is needed:

- **URL** - The HTTP endpoint of the service that should receive the notifications. Make sure the service is properly set up to handle webhook events.

For webhook integration, ClusterControl will raise an alarm by sending the JSON data using the HTTP POST method to the configured endpoint. The webhook endpoint should receive the following example event if a new alarm is created:

```
{
  "id": 470,
  "status": "CREATED",
  "component": "Node",
  "hostname": "192.168.20.62",
  "title": "Server disconnected",
  "message": "PostgreSQL server on 192.168.20.62:5432 disconnected: Connect␣
↪failure: Watchdog: Failed connection to postgres@192.168.20.62:5432.␣
↪timeout expired\n",
  "recommendation": "Check node status on UI and error log of failed␣
↪server.",
```

(continues on next page)

(continued from previous page)

```
  "severity": "CRITICAL"
}
```

The above event was created and sent out to the webhook endpoint after ClusterControl has detected that one of our database servers (192.168.20.62) was down. After the server came back online, ClusterControl would then send another event with the same alarm ID (470) with a different status "ENDED", indicating the above alarm has ended and the node is back operational:

```
{
  "id": 470,
  "status": "ENDED",
  "component": "Node",
  "hostname": "192.168.20.62",
  "title": "Server disconnected",
  "message": "PostgreSQL server on 192.168.20.62:5432 disconnected: Connect␣
↪failure: Watchdog: Failed connection to postgres@192.168.20.62:5432.␣
↪timeout expired\n",
  "recommendation": "Check node status on UI and error log of failed␣
↪server.",
  "severity": "CRITICAL"
}
```

Commonly, the "ended" event has an almost identical response text with the "created" event except for the "status" value.

Tip

You may use https://webhook.site/ to test out the webhook integration with ClusterControl.

To configure webhook integration:

ClusterControl GUI

1. Go to ***ClusterControl GUI → Settings → Notification Services → Add new integration → Webhook***.
2. In the **Service Configuration** section, specify the following:
   - **Integration name**: Give any name to this integration.
   - **URL**: The HTTP endpoint of the service that should receive the notifications.
3. Then click on **Test credentials**. Ensure you receive a test notification from ClusterControl before proceeding to the next step.
4. In the **Notification settings** section, specify the following:
   - **Clusters** - Choose one or more clusters to be notified. Multiple values are accepted.
   - **Events** - Choose one or more ClusterControl events that will trigger the notification. Multiple values are accepted. Details on the events are described in *Warning Events* and *Critical Events*.
5. Click **Finish** to complete the configuration.

2.105 Events

The following table lists out all event groups supported by ClusterControl:

Event	Description
All Events	All ClusterControl events including warning and critical events.
All Warning Events	All ClusterControl warning events, e.g. cluster degradation, network glitch. See *Warning Events*.
All Critical Events	All ClusterControl critical events, e.g. cluster failed, host failed. See *Critical Events*.
Network	Network-related events, e.g. host unreachable, SSH issues.
CMON Database	Internal CMON database-related events, e.g. unable to connect to CMON database, `datadir` mounted as read-only.
Mail	Mail system-related events, e.g. unable to send mail, mail server unreachable.
Cluster	Cluster-related events, e.g. cluster failure, cluster degradation, time drifting.
Cluster Configuration	Cluster configuration events, e.g. SST account mismatch.
Cluster Recovery	Recovery events, e.g. cluster or node recovery failures.
Node	Node-related events, e.g. node disconnected, missing GRANT, failed to start HAProxy, failed to start NDB cluster nodes.
Host	Host-related messages, e.g. CPU/disk/RAM/swap exceeds thresholds, memory full.
Database Health	Database health-related events, e.g. memory usage of MySQL servers, connections, missing primary key.
Database Performance	Alarms for long-running transactions, replication lag, and deadlocks.
Software Installation	Software installation-related events, e.g. license expiration.
Backup	Backups related events, e.g. backup failed.

2.105.1 Warning Events

Note

Note ClusterControl uses the same alarms' code name for other database clusters which produces a similar result. For example, if a PostgreSQL slave is lagging behind, the alarms internal code name is also called "MySqlReplicationLag". However, the actual alarm's response will have proper PostgreSQL relevant texts. This is handled internally.

Area	Alarms	Severity	Description
Node	MySqlReplicationLag	Warning	MySQL replication slave lag, default 10 seconds.
	MySqlReplicationBroken	Warning	The SQL thread has stopped. For PostgreSQL, it means the slave gets disconnected from the master.
	CertificateExpiration	Warning	SSL certificate expiration time (<=31 days, >7 days).
	MySqlAdvisor	Warning	Raised by `wsrep_sst_method.js` and `wsrep_node_name.js` advisors.
	MySqlTableAnalyzer	Warning	Raised by `schema_check_nopk.js` advisor.
	StorageMyIsam	Warning	Raised by `schema_check_myisam.js` advisor.
	MySqlIndexAnalyzer	Warning	Raised by `schema_check_dupl_index.js` advisor.
	MySqlReplicationLooseServer	Warning	A slave is found but its master can't be determined, or it is not part of the cluster.
Host	HostSwapV2	Warning	If a configurable number of pages has been swapped in/out during a configurable period of time. Default 20 pages in 10 minutes.
	HostSwapping	Warning	>5% swap space has been used.
	HostCpuUsage	Warning	>80%, <90% CPU used.
	HostRamUsage	Warning	>80%, <90% RAM used.
	HostDiskUsage	Warning	>80%, <90% disk space used on a monitored_mountpoint.
	ProcessCpuUsage	Warning	>95 % CPU used on average by a process for 15 minutes.
Back	BackupFailed	Warning	The backup job fails.
Recovery	GaleraWsrepMissing	Warning	`wsrep_cluster_address` or `wsrep_provider` is missing.
	GaleraSstAut	Warning	SST settings (user/pass are wrong).
Network	HostFirewall	Warning	The host is not responding to ping after 3 cycles.
	HostSshSlow	Warning	It takes 6-12 seconds to SSH into a host.
Cluster	ClusterTimeDrift	Warning	Time drift between ClusterControl and database nodes.
	ClusterLicenseExpire	Warning	The license is about to expire.
	ClusterInconsistentView	Warning	The load balancer or ClusterControl sees a different set of working nodes (master is down from ClusterControl point-of-view, while load balancer or the slave reports the master working.)

2.105.2 Critical Events

Note

Note ClusterControl uses the same alarms' code name for other database clusters which produces a similar result. For example, if a PostgreSQL server goes down, the alarms internal code name is also called "MySqlDisconnected". However, the actual alarm's response will have proper PostgreSQL relevant texts. This is handled internally.

Area	Alarms	Severity	Description
Node	MySqlDisconnected	Critical	The database server cannot be reached.
	MySqlGrantMissing	Critical	Node does not have the correct privileges set for the cmon user.
	MySqlLongRunningQuery	Critical	If queries are running for too long time. Only used if configured, by default it is not.
	ProcFailedRestart	Critical	A process (HAProxy, ProxySQL, Garbd, MaxScale) could not be restarted after a failure.
	CertificateExpiration	Critical	(<= 7 days), SSL Certificates expiration time.
	MySqlReplicationMultiMaster	Critical	Multiple writable masters detected.
Host	HostSwapV2	Critical	If a configurable number of pages has been swapped in/out during a configurable period of time. Default 20 pages in 10 minutes.
	HostSwapping	Critical	>20% swap space has been used.
	HostCpuUsage	Critical	>90% CPU used.
	HostRamUsage	Critical	>90% RAM used.
	HostDiskUsage	Critical	>90% disk space used on a `monitored_mountpoint`.
	ProcessCpuUsage	Critical	>99 % CPU used on average by a process for 15 minutes.
Backu	BackupVerificationFailed	Critical	Backup verification fails.
Recovery	GaleraWsrepMissing	Critical	`wsrep_cluster_address` or `wsrep_provider` is missing, and still missing after 20 sample cycles (which are ~100 seconds in this case)
	GaleraClusterSplit	Critical	There is a split brain.
	ClusterRecoveryFail	Critical	Recovery has failed.
	GaleraConfigProblem1	Critical	A configuration issue preventing the node to start.
	GaleraNodeRecoveryFail	Critical	Automatic recovery has failed 3 consecutive times.
	ReplicationFailoverBlacklistError	Critical	In the case of automatic failover, the only possible candidate is blacklisted, then this alarm is raised with critical severity.
Network	HostUnreachable	Critical	The host is not responding to ping after 3 cycles.
	HostSshFailed	Critical	Please check SSH access to the host. The host may also be down.
	HostSshAuth	Critical	Please check whether the configured SSH key is properly configured and can be authenticated on the host.
	HostSudoError	Critical	`sudo` command error on the host.
	HostSshSlow	Critical	It takes more than 12 seconds to SSH into a host.
Cluster	ClusterFailure	Critical	Cluster is failed.
	ClusterLicense-	Critical	The license is expired.

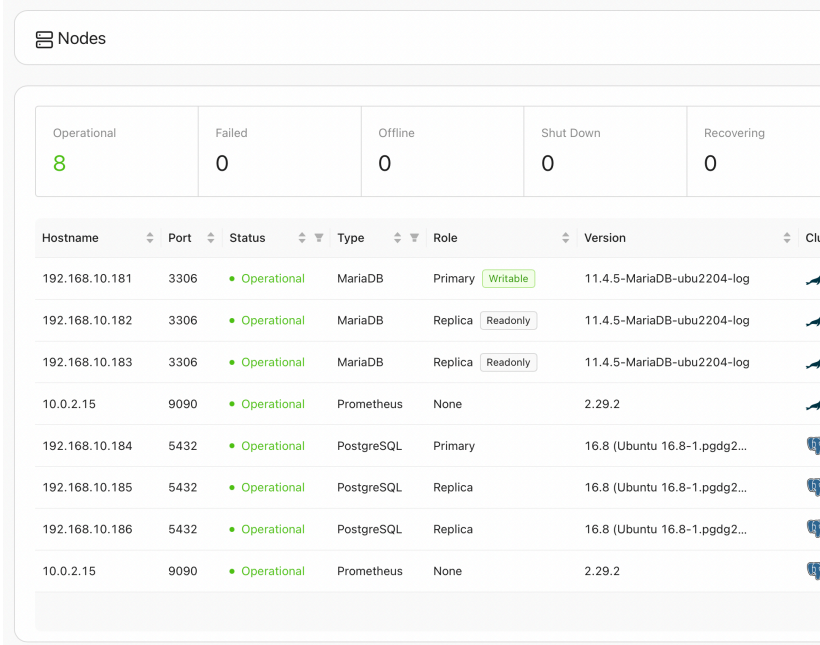
2.106 Setting up dedicated Prometheus node

When deploying the first cluster, ClusterControl sets up a Prometheus instance on the ClusterControl node. That instance serves as a time-series database to keep the data collected from nodes by exporters. That Prometheus instance, by default, is reused for all other clusters that will be deployed in ClusterControl.

In some cases this setup may be not enough:

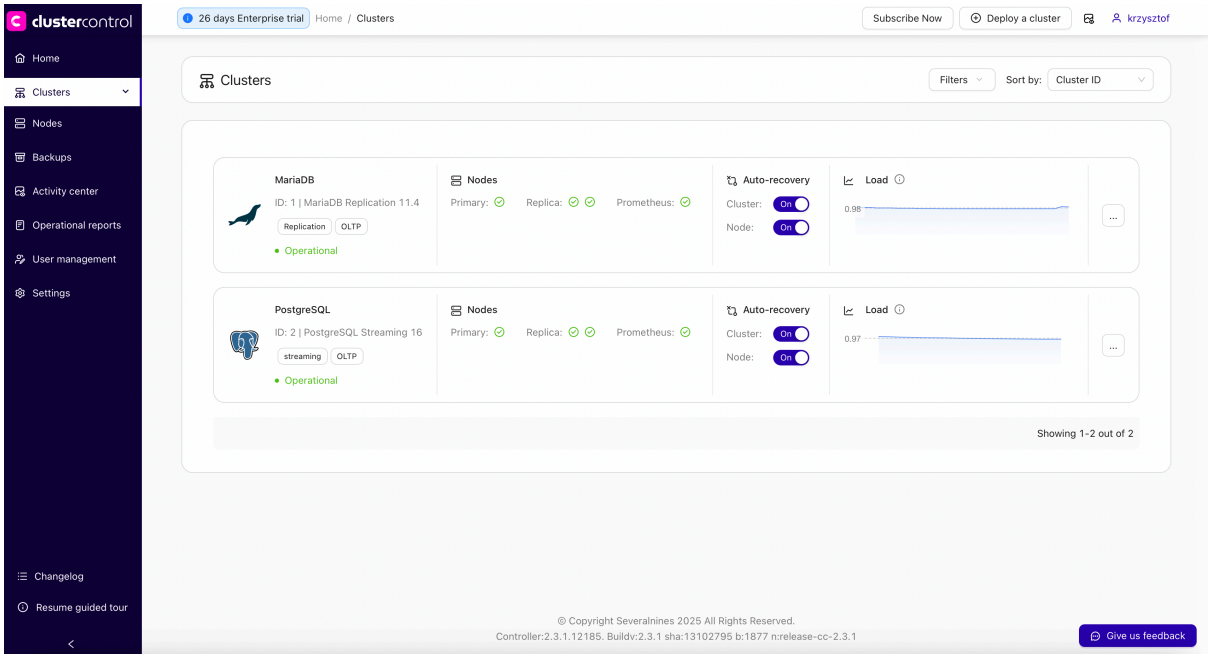
- You may have very large cluster that is generating significant amount of metrics
- You may have many clusters and, in total, the amount of data is big
- You may want to increase the retention period from default 15 days to something longer, causing more data to be stored

In all those cases you may find that you could use a dedicated node for Prometheus - faster, with more disk space available for data storage. Such setup is possible in ClusterControl.

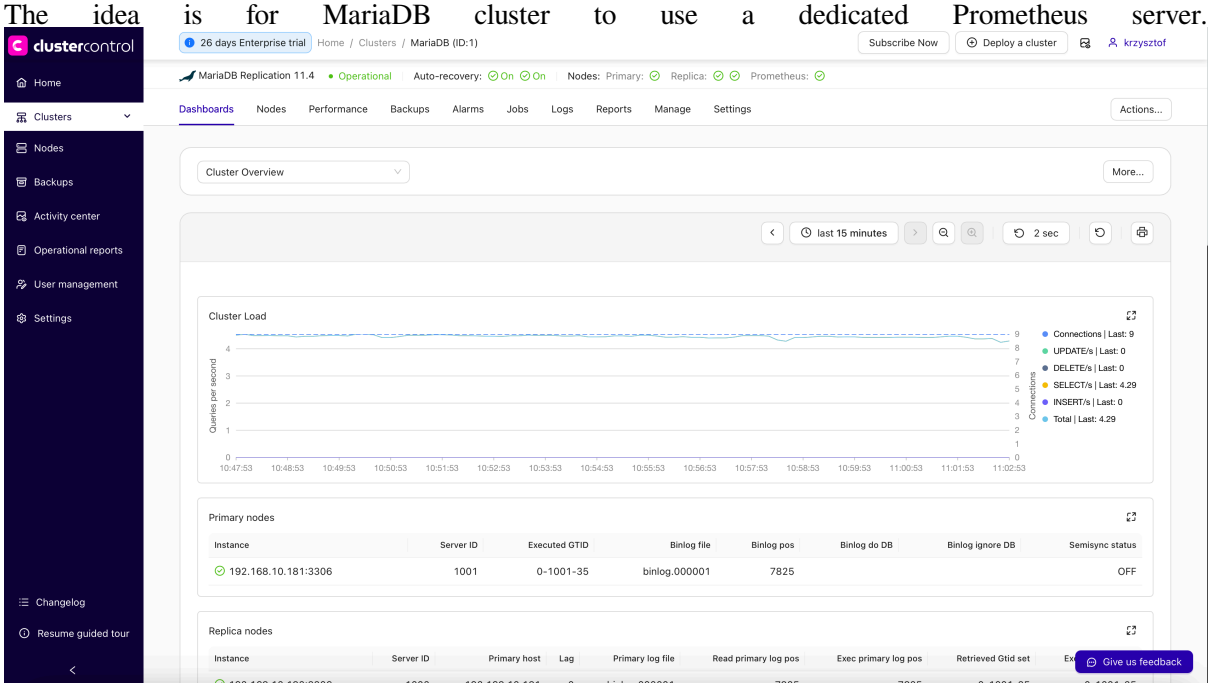


How to do this? Assume that there are two clusters:

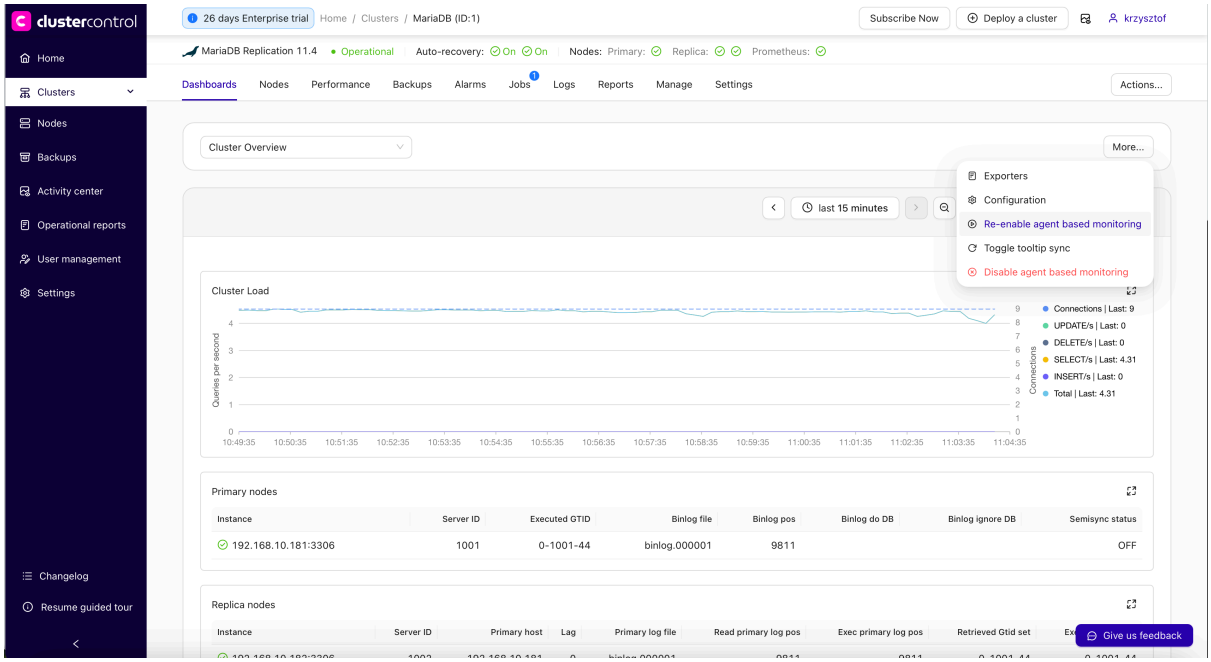
They are sharing the same Prometheus server on host 10.0.2.15 and port 9090.



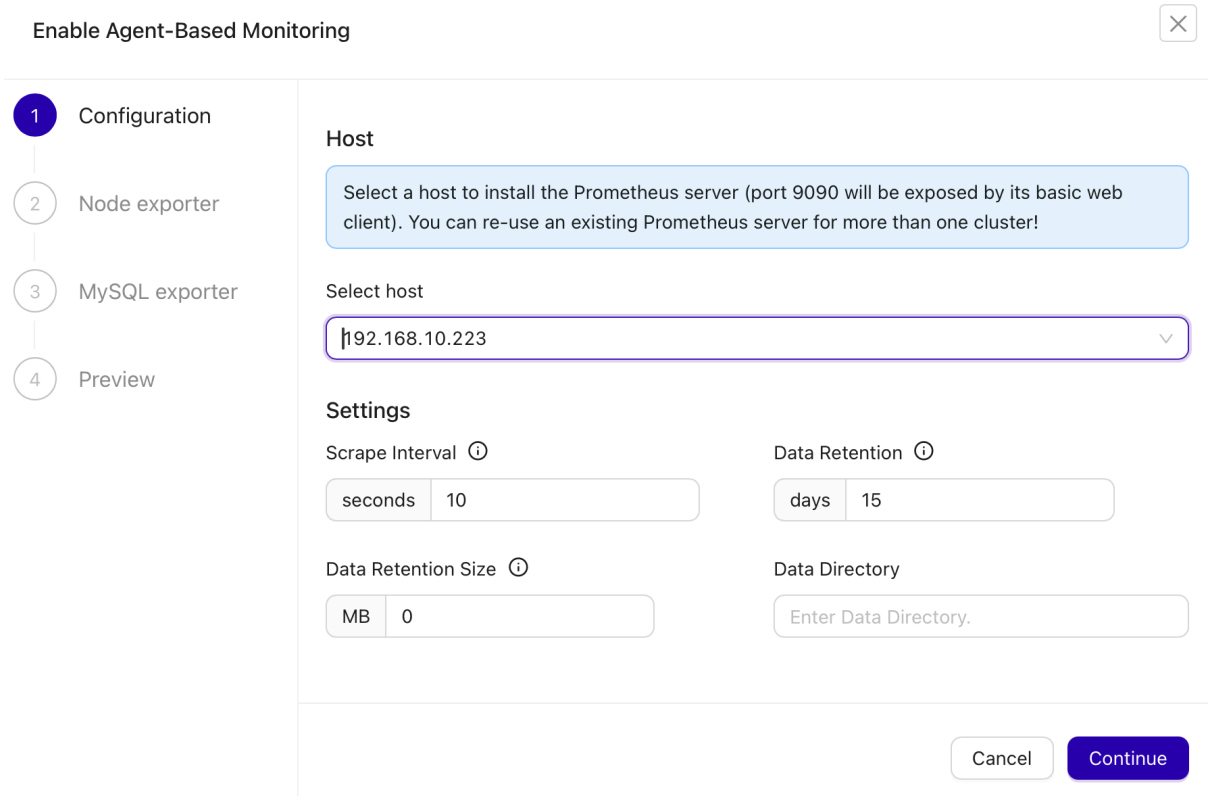
The idea is for MariaDB cluster to use a dedicated Prometheus server.



To do that, open MariaDB cluster and go to Dashboards tab. From there you pick re-enable and reconfigure agent based monitoring.



More→Re-enable agent based monitoring. This opens a window to perform the reconfiguration.



There are settings to configure:

2.106.1 Host Selection

Determines on which host the Prometheus server will be installed. You can ether pick existing Prometheus installation or put in a new hostname or IP address for a new Prometheus installation.

2.106.2 Scrape Interval

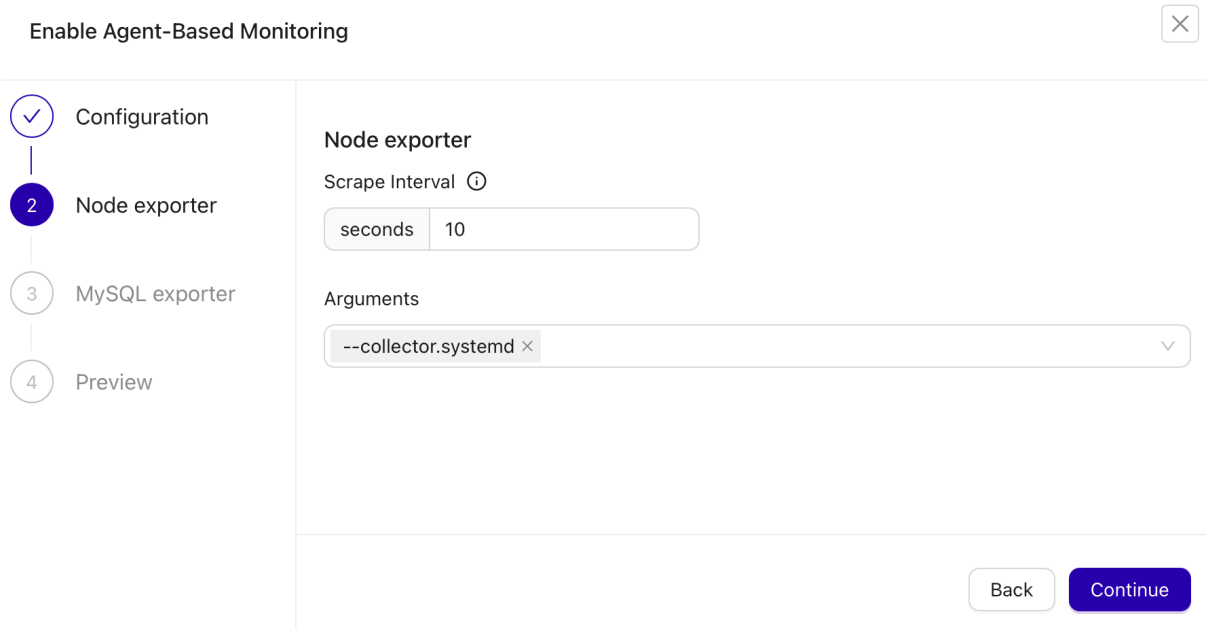
Defines how frequently Prometheus collects (scrapes) metrics from the targets. Default is set to 10 seconds.

2.106.3 Data Retention Size

Indicates the maximum amount of storage (MB) allocated for metrics data. Default (0 MB) means no limit.

2.106.4 Data Directory

Specifies the location on disk where Prometheus will store metrics data. Optional field (can be left unspecified).



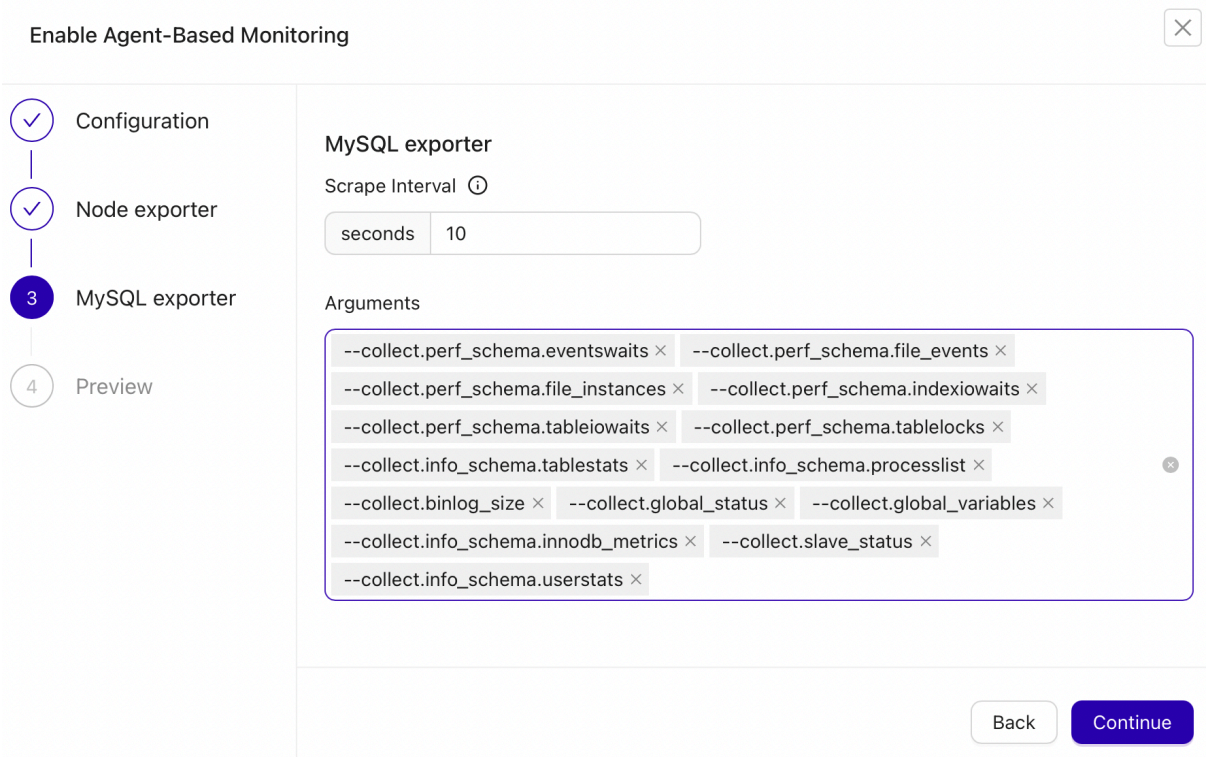
In this step you can configure settings related to node exporter.

2.106.5 Scrape Interval

Frequency at which Prometheus retrieves system metrics from the node exporter (10 seconds by default).

2.106.6 Arguments

Custom parameters for Node Exporter functionality, for instance: `--collector.systemd` enables collecting metrics from the systemd service manager.



Here are settings related to MySQL exporter

2.106.7 Scrape Interval

How often Prometheus scrapes metrics from the MySQL exporter (10 seconds).

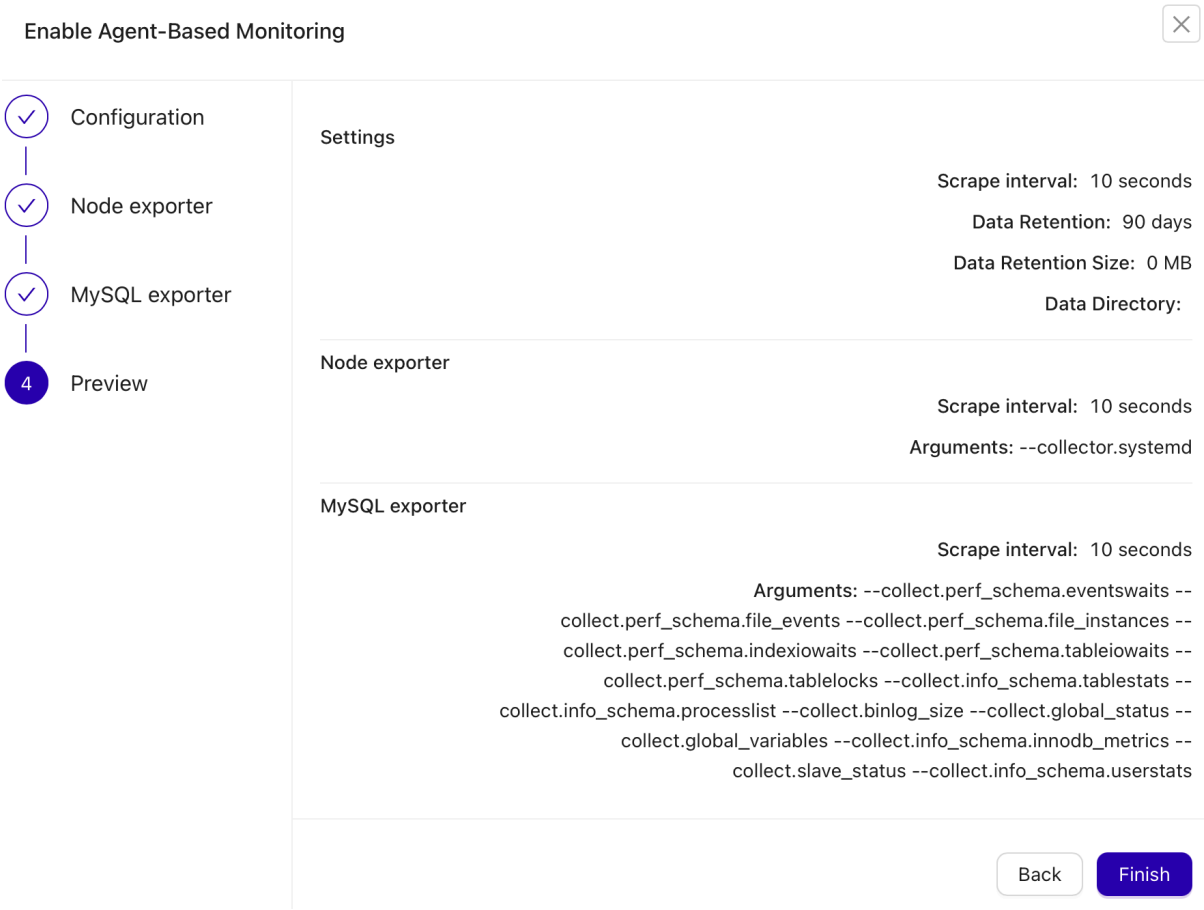
2.106.8 Arguments

Additional parameters to specify exactly which MySQL metrics should be collected, such as:

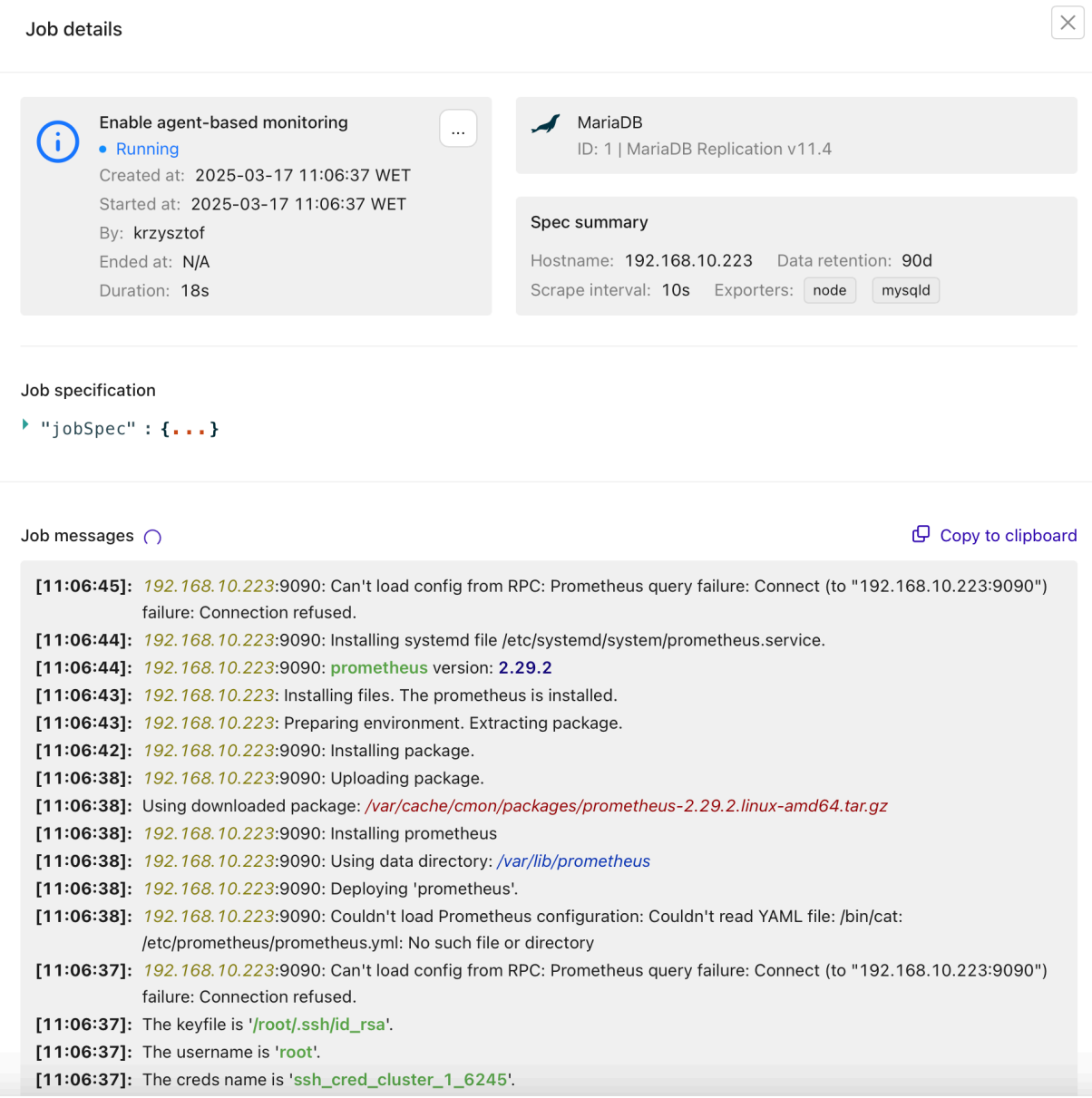
- `--collect.perf_schema.eventswaits` (Wait events)
- `--collect.perf_schema.file_events` (File operation events)
- `--collect.perf_schema.indexiowaits` (Index I/O waits)
- `--collect.perf_schema.tableiowaits` (Table I/O waits)
- `--collect.info_schema.processlist` (Active MySQL processes)
- `--collect.binlog_size` (Binary log size)
- `--collect.global_status` (Global status variables)
- `--collect.global_variables` (Global MySQL variables)
- `--collect.slave_status` (Replication slave status)

and other schema-related metrics.

Once all settings are defined, verify that everything is exactly as you want it to be in the Preview:



After the job is started, you can track it in the Activity center:



This section provides information is your go-to resource for diagnosing and resolving issues that may arise within the ClusterControl platform.

Note that this troubleshooting guide only covers the latest version of ClusterControl. We highly advise you to upgrade to the latest version before troubleshooting any kind of issues related to ClusterControl due to the following reasons:

- ClusterControl has a short release cycle if compared to other software applications (commonly a new major release every quarter of the year).
- We regularly release maintenance build of the latest stable version (if there is). See Release Notes.
- ClusterControl has to keep up with all the latest changes and modifications introduced by supported database and application vendors.
- Some of the issues you encountered probably have been fixed in the latest maintenance patch or major release.

To upgrade ClusterControl to the latest version, see Upgrading ClusterControl.

2.107 Logging

ClusterControl consists of a number of components that write to their own log file. These files reside on the ClusterControl node. By default, `cmon` runs without the debug option enabled. See *Debugging* on how to get the run in debug mode.

If you encounter any problems with ClusterControl, it is highly recommended to examine the related log files.

Log Type	Default location (RedHat/CentOS)	Default location (Debian/Ubuntu)
CMON process log	`/var/log/cmon.log or /var/log/cmon_<cluster_id>.log`	`/var/log/cmon.log` or `/var/log/cmon_<cluster_id>.log`
CMON cloud log	`/var/log/cmon-cloud.log`	`/var/log/cmon-cloud.log`
CMON events log	`/var/log/cmon-events.log`	`/var/log/cmon-events.log`
CMON audit log	`/var/log/cmon_audit.log`	`/var/log/cmon_audit.log`
CMON upgrade log	`/var/log/cmon_upgrade.log`	`/var/log/cmon_upgrade.log`
CMON discovery log	`/var/log/cmon_discovery.log`	`/var/log/cmon_discovery.log`
Apache error log	- `/var/log/httpd/ssl_error_log` - `/var/log/httpd/error_log`	`/var/log/apache2/error.log`
Apache access log	- `/var/log/httpd/access_log` - `/var/log/httpd/ssl_access_log`	- `/var/log/apache2/access.log` - `/var/log/apache2/ssl_access.log`

2.108 Debugging

The debugging section describes more advanced troubleshooting in ClusterControl. It's usually used for cases related to `cmon` crashes and requires in-depth investigation.

2.108.1 Debugging ClusterControl Controller (CMON)

In case if you encounter a CMON crash, please install the debuginfo package and the necessary packages as shown below.

2.108.2 Install debugging components

Red Hat/CentOS

1. Enable the debug repo under `/etc/yum.repos.d/CentOS-Debuginfo.repo` and set `enabled=1`.
2. Install yum utilities:

```
yum -y install yum-utils
```

3. Install ClusterControl debuginfo and gdb:

```
yum -y install clustercontrol-controller-debuginfo gdb
```

4. Then, run:

```
debuginfo-install clustercontrol-controller
```

Debian/Ubuntu

1. Install ClusterControl debuginfo package and gdb:

```
apt-get install clustercontrol-controller-dbg gdb
```

Optionally, you can

2. Install the debugging components' library:

```
apt-get install libstdc++6-4.8-dbg libc6-dbg
```

However, this totally depends on the libstdc++6 version installed. Print the shared object dependencies using `ldd`:

```
ldd /usr/sbin/cmon | grep libstdc
        libstdc++.so.6 => /usr/lib/x86_64-linux-gnu/libstdc++.so.6␣
↪(0x00007ff508001000)
```

Based on the library path, locate the package name that provides this library:

```
dpkg -S /usr/lib/x86_64-linux-gnu/libstdc++.so.6
libstdc++6:amd64: /usr/lib/x86_64-linux-gnu/libstdc++.so.6
```

Then, find the package's version:

```
dpkg -l | grep libstdc++6
ii libstdc++6:amd64 4.9.2-10 amd64 GNU Standard C++ Library v3
```

In this case, we have version "4.9" installed for libstc++6. Finally, install the corresponding debug packages:

```
apt-get install gdb libc6-dbg libstdc++-6-4.9-dbg
```

2.108.3 Debugging Steps

Debugging is a program that produces a core dump. It consists of the recorded state of the working memory of a computer program at a specific time, generally when the program has crashed or otherwise terminated abnormally. ClusterControl Controller (CMON) package comes with a crown file installed under `/etc/cron.d/` which will auto-restart if the cmon process is terminated abnormally. Typically, you may notice if the cmon process has crashed by looking at the `dmesg` output.

In such cases, generating a core dump is the only way to backtrace the issue. Make sure you have the debugging components installed as described in the previous section beforehand. On ClusterControl node as the root user, increase the CPU limit, adjust the kernel's core pattern value and run CMON on the foreground:

```
systemctl stop cmon
service cmon stop # for systemd only, to disable /etc/cron.d/cmon
ulimit -c unlimited
echo "/tmp/core.%e.%p.%h.%t" > /proc/sys/kernel/core_pattern
cmon -d
```

Or, if you want to run CMON as the background process, we need to create a unit override file for the cmon service:

```
systemctl edit cmon
```

And add the following lines:

```
[Service]
LimitCORE=infinity  # only when debugging
```

Then, you may start/stop the service using the `systemctl` command:

```
systemctl daemon-reload
systemctl start cmon
```

Warning

Warning!

Only set `LimitCORE=infinity` for debugging purposes. To revert back to the standard setting, remove `/etc/systemd/system/cmon.service.d/override.conf` and reload systemd using the `systemctl daemon-reload` command.

When `cmon` crashes, there will now be a core file under `/tmp`. Compress the core dump (gzip is recommended) and attach it to a support ticket so we can take a look and perform the necessary fix. Alternatively, you can send only the backtrace in a support ticket by using the following command:

```
gdb /usr/sbin/cmon /tmp/<corefile>
thread apply all bt full
```

Attach the full output and potentially replace sensitive information with "XXXXXXXXX". Traces may contain password information.

2.109 CMON on Foreground

If you would like to run cmon as a foreground process, you can do that by invoking `-d` option:

```
$ service cmon stop
$ CMON_DEBUG=1 cmon -d
```

CMON will enable `LOG_DEBUG` messages and print detailed information on the screen (stdout) as well as `/var/log/cmon.log` or `/var/log/cmon_{cluster ID}.log`. Press ++ctrl++ + C to terminate the process. In some cases, this type of CMON output might be needed to get insight into the problem.

2.110 ClusterControl GUI diagnostic

For ClusterControl GUI-related issue, you can use **Diagnostic** feature available at ***ClusterControl GUI → Settings → Diagnostics*** and toggle ON **UI audit data collection**. This allows user to record and capture the necessary data while reproducing an issue in the user interface. Once the issue has been replicated, click on "Download audit data" for analysis or to share with our Support team for troubleshooting and debugging.

Warning

Warning!

Remember to switch off the setting afterward to stop audit data collection.

Note

Note Note that we are not collecting any personal data during this process.

This section describes on how to troubleshoot the database cluster. It can be a new deployment, import a cluster or the existing database cluster managed by ClusterControl.

2.111 New deployment abd import cluster

The new deployment of database cluster through ClusterControl will trigger a new job either it is succesfull or failed. How to check if the deployment of new cluster is failed and what is the reason of the failed can be checked in the jobs.

ClusterControl GUI

1. In the ClusterControl UI, go to ***Activity Center → Jobs***, check the failed jobs for the deployment cluster.
2. There is an Action menu in the jobs, expand the details for detailed errors.

ClusterControl CLI

1. Check the history of the job

```
s9s jobs --list --show-failed
```

2. Dig the detailed error of failed job

```
s9s job --log --job-id=<value>
```

The `job-id` can be found from the job list output above.

2.112 Existing database cluster

Failure can occur in the database cluster due to various reasons. It may stem from issues in the network, disk failures, or problems within the operating system itself. It is important to know how to monitor and assess what is happening within the database cluster.

Below are steps to check the database cluster in ClusterControl.

ClusterControl GUI

1. In the ClusterControl UI, go to **Home** page of ClusterControl, it will shown what cluster and nodes that is degraded.
2. Go to the failure Cluster, check the triggered Alarm under the **Alarm**.
3. If the failure are related to the database, check the database log under the ***Logs → System Logs***

ClusterControl CLI

1. Run the following command in ClusterControl host. The cluster list give information about the list of the cluster including the state (`STARTED`, `FAILURE`, `UNKNOWN`).

   ```
   s9s cluster --list --long
   ```

   The node list show information of which node that is failure in the cluster.

   ```
   s9s nodes --list --cluster-id=<clusterId>
   ```

   The `clusterId` value can be found when executing the cluster list command.

2. Check the triggered alarm in the CLI.

   ```
   s9s alarm --list  --long
   ```

2.113 Overview

This section describe the Error Report utility in ClusterControl. It is a compressed file (tarball) containing diagnostic information about the cluster (`cmon` logs, database error logs, and the contents of some important CMON database tables). This is very useful for troubleshooting purposes, especially when working with the Severalnines Support team on a support issue. Attach the generated compressed file to your support request.

2.114 Generating an Error Report

Follow the following steps to generate an error report:

ClusterControl GUI

1. To create a new error report for a cluster, go to ***ClusterControl GUI → Clusters → choose a cluster → Reports***, click the **Create Error Report** button.
2. There will be an option to mask passwords so that if any are contained in the configuration, ClusterControl will mask them. The default is always ON.
3. Download the tarball file through ***Action → Download***.

ClusterControl CLI

Execute the following command in ClusterControl host as root/sudo user:

```
s9s_error_reporter -i <cluster_id>
```

The `<cluster_id>` is the identifier of the cluster where the error report is to be generated. Where 1 is the cluster ID of the corresponding cluster. For other clusters, just change the cluster ID accordingly. The cluster ID can be retrieved from ***ClusterControl GUI → Clusters → locate the cluster → ID***, using `s9s cluster --list` command or by looking into the CMON configuration file for the respective cluster inside `/etc/cmon.d`.

Example

The following command generates an error report for cluster ID 15:

```
s9s_error_reporter -i 15
```

At the end of the execution, it will print out something like this:

```
Executing tar -C /var/tmp/cmon-003522-de42b82f69aba63c -czf '/
↪root/s9s_tmp/error-report-2025-05-23_090757-cluster16.tar.gz'␣
↪error-report-2025-05-23_090757-cluster16
Please attach /
↪root/s9s_tmp/error-report-2025-05-23_090757-cluster16.tar.gz to the␣
↪support issue.
```

Tip

If you have no cluster managed by ClusterControl, you might want to specify `0` instead, where 0 has a special meaning in ClusterControl – collect information about ClusterControl installation, configurations, and logs on the local node only.

Please attach the generated tarball to your support ticket. If the tarball exceeds the 20MB attachment limit of our support system, please upload it to a cloud storage service such as Google Drive, Dropbox, or WeTransfer. Then, share the HTTPS download link in your support ticket. Be aware that the error report might contain sensitive and confidential information, so restrict public access and only share the download URL with us.

Tip

We also recommend you to take screenshots showing the area of the problem, e.g, the Overview page from the UI allows us to understand the current state of nodes and clusters.

2.115 ClusterControl GUI diagnostic

For ClusterControl GUI-related issue, you can use the **Diagnostic** feature available at ***ClusterControl GUI → Settings → Diagnostics*** and toggle ON **UI audit data collection**. This allows user to record and capture the necessary data while reproducing an issue in the user interface.

After replicating the issue, immediately disable the setting to halt audit data collection. Then, click **Download audit data** for your analysis or to share with our Support team for troubleshooting and debugging. This audit data allows our team to replay the exact error users encountered in their specific configurations and environments, eliminating the need for screen sharing.

Warning

Warning!

Remember to switch off the setting afterward to stop audit data collection otherwise it will impact the web browser's performance after a period of time.

Note

Note Note that we are not collecting any personal data during this process.

2.116 Common Issues

This section covers common issues when configuring and dealing with ClusterControl components, with possible troubleshooting steps and solutions. There is also a community forum available with knowledge base sections for public reference.

If you need further assistance, please contact us via our support channel by submitting a support request or post a new thread in our community help forum.

2.116.1 ClusterControl Installation

This section covers common issues encountered during ClusterControl installation and the installer script.

Failed to start MySQL Server during ClusterControl installation

Description:

During installation, the installer script fails to start the MySQL/MariaDB server on ClusterControl host and returns the following lines:

```
=> Starting database. This may take a couple of minutes. Do NOT press any␣
↪key.
mysqld: unrecognized service
=> Failed to start the MySQL Server. ...
Please contact Severalnines support at http://support.severalnines.com if␣
↪you have installation issues that cannot be resolved.
```

Troubleshooting steps:

1. Examine the MySQL error log on possible reasons why does MySQL fails to start. Typically, the log file is located under `/var/log/mysqld.log` or `/var/lib/mysql/error.log`.
2. Try starting the MySQL server manually by using `systemctl` or `service` command.

Solutions:

If you are running on the older operating system, ClusterControl might not support the distribution and some issues are expected to show up. See *Requirements* for details. Once the MySQL server is up and running, restart the ClusterControl installer script again:

```
$ ./install-cc
```

ClusterControl Controller (CMON)

This section covers common issues encountered related to ClusterControl Controller (CMON).

User '{username}' suspended for previous authentication failures.

Description:

When trying to execute an s9s command line, the command returns the current s9s user is suspended due to authentication failures. This happens if the key is no longer recognized by the CMON controller when processing a request.

Example error:

When running an s9s command, the following error occurs:

```
$ s9s user --list
User 'admin' suspended for previous authentication failures.
```

Solution:

1. Remove all the s9s user configuration files (/etc/s9s.conf, ~/.s9s/s9s.conf):

   ```
   $ mv ~/.s9s/s9s.conf ~/.s9s/s9s.conf.bak
   $ mv /etc/s9s.conf /etc/s9s.conf.bak
   ```

2. Create another user for s9s CLI, in this example, we created a user called dba, under group "admins":

   ```
   $ s9s user --create --generate-key --controller=
   ↪"https://localhost:9501" --group=admins dba
   Grant user 'dba' succeeded
   ```

3. Use it to unlock the "original" suspended user (assuming the suspended user is "admin"):

   ```
   $ s9s user --cmon-user=dba --enable admin
   OK.
   ```

By performing the above, it will clear the "disabled" flag of the user so that the user will be able to log in again. The "suspended" flag will also be cleared, the failed login counter set to 0 and the date and time of the last failed login gets deleted, so users who are suspended for failed login attempts will also be able to log in.

CMON is unable to restart MySQL using the service command

Description:

When scheduling a start/restart job, ClusterControl fails to start the node with the error "Command not found".

Example error:

```
galera1.domain.com: Starting mysqld failed: Error: Command not found (host:␣
↪galera1.domain.com): service mysql restart
galera1.domain.com: Starting mysqld
```

Troubleshooting steps:

1. SSH into the DB node and check the user's environment path variable:

```
$ ssh -tt -i /home/admin/.ssh/id_rsa root@192.168.10.101 "sudo env | grep␣
↪PATH"
PATH=/usr/local/bin:/bin:/usr/bin
```

2. Look at the `PATH` output.

Solution:

- Ensure the `/sbin` the path is there. Otherwise, ClusterControl won't be able to automatically locate and run the "service" command.
- If the `/sbin` the path is not listed in the PATH, add it by using the following command:

```
PATH=$PATH:/sbin
export PATH
```

However, the above won't persist if the user logs out from the terminal. To make it persistent, add those lines into `/home/<SSH user>/.bash_profile` or `/home/<SSH user>/.bashrc`

CMON always tries to recover failed database nodes during my maintenance window.

Description:

By default, CMON is configured to perform recovery of failed nodes or clusters. This behavior can be overridden by disabling automatic recovery or enabling maintenance mode for the node.

Solution:

1. Enable maintenance mode for selected nodes (recommended). To enable the maintenance window, go to ***Nodes → select the node → Schedule Maintenance Mode → Enable***. You have to specify the reason and duration of the maintenance window. During this period, any alarms and notifications raised for this node will be disabled.
2. Disable automatic recovery. To disable automatic recovery temporarily, turn off automatic recovery via ***ClusterControl GUI → Clusters → choose a cluster → Actions → Disable Node/Cluster recovery***. you can just click on the 'power' icon for node and cluster. Red means automatic recovery is turned off while green indicates recovery is turned on. This behavior will not persistent if CMON is restarted. To make the above change persistent, disable node or cluster auto-recovery by specifying the following line inside the CMON configuration file of the respective cluster. For example, if you want to disable automatic recovery for cluster ID 1, inside `/etc/cmon.d/cmon_1.cnf`, set the following line: `enable_autorecovery=0`

CMON process dies with "Critical error (mysql error code 1)"

Description:

After starting CMON service, it stops and `/var/log/cmon.log` shows the following error:

```
(ERROR) Critical error (mysql error code 1) occurred - shutting down
```

Troubleshooting steps:

1. Run the following command on the ClusterControl host to check if it has the ability to connect to the DB host with current credentials:

```
$ mysql -ucmon -p -h<database node IP> -P<MySQL port> -e 'SHOW STATUS'
```

2. Check GRANT for the cmon user on each database host:

```
mysql> SHOW GRANTS FOR 'cmon'@'<ClusterControl IP address>';
```

Solution:

It is not recommended to mix public IP addresses and internal IP addresses. For the GRANT, try to use the IP address that your database nodes use to communicate with each other. If the `SHOW STATUS` statement returns `ERROR 1130 (HY000): Host '<ClusterControl IP address>' is not allowed to connect to this`, the database host is missing the cmon user grant. Run the following command to reset the cmon user privileges:

```
mysql> GRANT ALL PRIVILEGES ON *.* TO 'cmon'@'<ClusterControl IP>'␣
↪IDENTIFIED BY '<cmon password>' WITH GRANT OPTION;
mysql> FLUSH PRIVILEGES;
```

Where, is ClusterControl IP address and is `mysql_password` value inside the CMON configuration file.

Job fails with 'host is already in another cluster' error

Description:

When deploying a new node, or adding a node into an existing cluster managed by ClusterControl, the deployment fails with the following error:

```
Host 1.2.3.4:nnnn is already in another cluster.
```

Solution:

A host can only exist in one cluster at a time. Check if you have an `/etc/cmon.d/cmon_X.cnf` file (where X is an integer) that contains the hostname and remove the file if the corresponding cluster does no exist in the UI (be careful to not remove the wrong `cmon_X.cnf` file):

```
$ rm /etc/cmon.d/cmon_X.cnf
```

Otherwise, delete the host from `server_node`, `mysql_server`, and `hosts` tables:

```
mysql> DELETE FROM cmon.server_node WHERE hostname='1.2.3.4';
mysql> DELETE FROM cmon.mysql_server WHERE hostname='1.2.3.4';
mysql> DELETE FROM cmon.hosts WHERE hostname='1.2.3.4';
```

Restart CMON to load the new changes:

```
$ systemctl restart cmon
```

You may have to execute the deletion several times for each hostname/IP of the cluster you are trying to add.

Failed to deploy Prometheus service

Description:

The deployment of prometheus service failed due to the `prometheus.service` does not exist as follows:

```
Failed to enable prometheus.service: Command exited with return code 1 on␣
↪host xxxx. Command: systemctl --no-pask-password enable prometheus.
↪service stdErr: Failed to enable unit: Unit file prometheus.service does␣
↪not exist. stderr: Failed to enable unit: Unit file prometheus.service␣
↪does not exist
```

Troubleshooting steps:

Run the following command to verify the following command in the ClusterControl host to check for the detail why the prometheus won't start:

```
$ journalctl -t setroubleshoot
```

If the result is related to the `[20764]: SELinux is preventing systemd from read access on the file prometheus.service.`, check the status of SELinux.

```
$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
```

Solution:

In this case, you need to update the parameter `SELINUX` in SELinux configuration for CentOS/Redhat based. The configuration can be found at `/etc/selinux/config`. Change the value become `PERMISSIVE` or `DISABLED`. While for the Debian/Ubuntu based, you need to stop the AppArmor service.

Unable to authenticate to LDAP server

Description:

Unable to log in to ClusterControl using the LDAP user after LDAP Settings is configured.

Troubleshooting steps:

1. Make sure that you have mapped ClusterControl's Team with the respective LDAP Group Name under ***ClusterControl → User Management → LDAP → Map LDAP Group***.

2. Verify if the configured ***LDAP Settings*** are still correct. Go to ***ClusterControl → User Management → LDAP → LDAP Settings*** and make sure hit the **Enable LDAP authentication** is toggled ON. When toggling ON, ClusterControl will perform a simple call to the configured LDAP server to verify the connection info. You will see a pop-up error if the connection info is incorrect. Note that Windows Active directory DNs are case-sensitive.
3. For failure LDAP events, ClusterControl will capture the error log under `/var/log/cmon.log`. Examine this log to look for any clues why LDAP authentication fails.
4. On ClusterControl node, try to list out the directory branch by using the admin DN's and password using `ldapsearch` the command (OpenLDAP-clients package is required):

```
$ ldapsearch -H ldaps://ad.company.com -x -b
↪'CN=DBA,OU=Groups,DC=ad,DC=company,DC=com' -D
↪'CN=Administrator,OU=Users,DC=ad,DC=company,DC=com' -W
```

Solutions:

ClusterControl supports Active Directory, FreeIPA, and OpenLDAP authentication, see *LDAP Integration* for details. If the above troubleshooting steps do not help you solve the issue, please contact us via our support channel for further assistance.

2.117 Known Limitations

ClusterControl is a complex and sophisticated product. It consists of several components and it is recommended to run it on a dedicated host. Here are some noteworthy and known limitations that we have gathered so far:

2.117.1 General

- ClusterControl can not reside on the same host as the database nodes that you want to deploy or import. ClusterControl must be running on an independent node away from your database clusters/servers.
- For good long-term support, do not co-locate ClusterControl with other applications outside of ClusterControl's control. For example, do not co-locate ClusterControl with a DNS server, an email server, or on a hosting server controlled by a control panel manager. A dedicated host is recommended.
- Not all functionalities are available on both user interfaces, *ClusterControl GUI* and *ClusterControl CLI*. The command-line client is focused on automation and adoption in management, deployment and scaling operations while the web UI client, focuses more on structural visualization with a guided approach.
- In terms of Linux security, ClusterControl does not manage firewalls (iptables, ufw, firewalld) or any Linux kernel security modules like SELinux or AppArmor. ClusterControl only offers a switch for this feature to be turned on or off while deployment to reduce the risk of deployment errors. The SysAdmin is responsible for these aspects.

2.117.2 SSH

- ClusterControl requires the SSH user to be a sudo user without restrictions or simply a root user.
- ClusterControl only supports OpenSSH. Other SSH implementations like Dropbear and TinySSH are not supported.

2.117.3 MySQL/MariaDB

- ClusterControl does not support managing multiple MySQL and MariaDB instances running on a single host. This includes multiple MySQL/MariaDB instances running on Docker container or running MySQL/MariaDB instances via mysqld_multi, MySQL Sandbox or DBdployer.
- ClusterControl only supports GTID-based replication (both MySQL's and MariaDB's) for automatic failover and recovery. If you are running on a position-based replication, ClusterControl is still able to manage and monitor it without supporting the automatic failover.
- When importing an existing MySQL/MariaDB server/cluster into ClusterControl, some non-destructive configurations will be appended:
  - CMON database user.
  - Backup database user credentials.
  - Some systemd configuration for MySQL/MariaDB will be overridden e.g, Galera Cluster to allow long SST.
  - Read-only flag (MySQL/MariaDB replication).
- When automatic recovery is enabled, ClusterControl always tries to be as conservative as possible when recovering a cluster or a database node. For example, it will always start/restart node one node at a time (not multiple at once), it will always force one master at a time (MySQL replication) or in case of network flapping, ClusterControl will attempt to recover the cluster once and if it keeps failing instantaneously, it will back off on the second attempt and raise an alarm. ClusterControl provides many hooks for you to extend the recovery and failover behavior, as shown on the *Configuration Options*.

2.117.4 Containerization

- Although Severalnines offers ClusterCluster as a Docker image, it is not intended for production usage. ClusterControl product direction is never intended to run on a container environment due to its internal logic and system design. We are maintaining the Docker image on a best-effort basis, and it is not part of the product development projection and pipeline.
- Supporting on database instances running on the Docker container is fairly limited. ClusterControl requires an SSH server to communicate and control the instance, which commonly does not come with the database's Docker image (which defeats the purpose of microservices). You either have to install an SSH server on the container or use a special image that comes with SSH like centos-ssh.

2.117.5 Monitoring

- When agent-based monitoring is activated, ClusterControl will control the Prometheus and all exporters' processes to avoid false positives when observing the node and cluster state. Hence, ClusterControl does not support importing an existing Prometheus configured and deployed outside of ClusterControl.

CHAPTER

THREE

REFERENCE MANUALS

This reference manual provides detailed information on every feature, setting, and command available within ClusterControl, ensuring you have the knowledge needed to maximize the platform's capabilities.

3.1 What You'll Find in This Manual

This manual serves as an essential resource for Database Administrators (DBAs), DevOps Engineers, IT Managers, and Developers looking to optimize their database clusters and streamline their operational workflows. Here's an overview of what each section covers:

3.1.1 ClusterControl Components

A detailed documentation on all ClusterControl components on configuring ClusterControl, including requirements, supported environments, and integration options. From initial setup to configuring essential components, this section helps you build a strong foundation.

3.1.2 Feature Reference

An in-depth look at ClusterControl's features, from cluster deployment and scaling to monitoring, backup, and failover. Each feature is broken down to show how it works, its benefits, and example use cases to help you understand when and why to use each function.

3.1.3 Command and API Reference

Detailed documentation on every available command and API endpoint within ClusterControl. This is especially useful for users looking to integrate ClusterControl with other tools or automate tasks, as well as those utilizing the CLI for greater control over their environment.

3.1.4 Troubleshooting and Support

Practical advice and solutions for common issues, error codes, and known limitations. This section also provides guidance on accessing support resources and engaging with the ClusterControl community for additional help.

3.2 Components

This section introduces the core components that make up ClusterControl. Learn how each component works together to provide deployment, management, monitoring, and automation capabilities for your database clusters.

See also

- *ClusterControl Controller →*
- *ClusterControl GUI →*
- *ClusterControl CLI →*
- *ClusterControl Web SSH →*
- *ClusterControl Notifications →*
- *ClusterControl Cloud →*
- *ClusterControl Cloud File Manager →*
- *ClusterControl Proxy →*
- *ClusterControl Agent →*

3.3 ClusterControl Controller

ClusterControl Controller (cmon) is the core backend process that performs all automation and management procedures. It is installed as `/usr/sbin/cmon` and comes with a collection of helper scripts in `/usr/bin` the directory (prefixed with `s9s_`) to complete specific tasks. However, some of the scripts have been deprecated due to the corresponding tasks are now being handled by the CMON core process.

ClusterControl Controller packages are available at Severalnines Repository page. The packages are also available at the Severalnines download site. RedHat-based systems should download and install the RPM package while Debian-based systems should download and install the DEB package. The package name is formatted as:

- RPM package (RedHat-based systems): `clustercontrol-controller-{version}-{build number}-{architecture}.rpm`
- DEB package (Debian-based systems): `clustercontrol-controller-{version}-{build number}-{architecture}.deb`

A configuration file `/etc/cmon.cnf` is required to initially run a `cmon` process. It is possible to have several configuration files for multiple clusters as described in the Configuration Files.

3.3.1 Command Line Arguments

By default, if you just run `cmon` (without any arguments), `cmon` defaults to run in the background. The binary supports several command-line options as shown below:

Shorthand, Option	Description
-h, --help	Print help.
--help-con	Print the manual for configuration parameters. See *Configuration Options*.
--help-ini	Shows the special options for --init.
-v, --version	Prints out the version number and build info.
--debug	Print more messages into the log.
--logfile=	The path of the log file to be used.
--log-form	Format string for log files.
--log-form	Format string for printing log messages.
--no-color	Do not use colors when printing log to stderr.
--no-log-d	Do not store cmon logs in database.
--upload-j	To upload JS scripts from archive files to DB.
--disable-	Disables jobs for all scheduled DSL Advisors.
--mysql-co	The connect retries to cmonDB before quitting.
-s, --syslog	Log also to syslog.
-b, --bind-add ip2..'	Bind Remote Procedure Call (RPC) to IP addresses (default is 127.0.0.1,::1). By default cmon binds to 127.0.0.1 and ::1. If another bind-address is needed, then it is possible to define the bind addresses in the file /etc/default/cmon. See *Startup File*.
-c, --cloud-se	A custom clustercontrol-cloud service URL.
-d, --nodaemon	Run in the foreground. Ctrl + C to exit.
-e, --events-c	Additional RPC URL where backend sends events.
-g, --grant	Create grants.
-i, --init	Creates configuration file and database.
--log-rpc	Log every RPC call (very verbose).
--no-safet	Do not check if another cmon is connected.
-p, --rpc-port	Listen on RPC port. The default is 9500.
-r, --director	Running directory.
-u, --upgrade-	Try to upgrade the CMON schema (Supported from CMON version 1.2.12 and later).

3.3.2 Startup File

To customize the cmon startup process, you can define the command line arguments in a custom file, instead of hacking up the init script directly. The cmon init script (or systemd) will append all configuration options defined inside /etc/default/cmon when starting up the cmon process and translates those options into command line arguments. For example:

```
$ cat /etc/default/cmon
RPC_PORT=9500
RPC_BIND_ADDRESSES="10.10.10.13,192.168.33.1,127.0.0.1"
```

(continues on next page)

(continued from previous page)

```
EVENTS_CLIENT=http://127.0.0.1:9510
CLOUD_SERVICE=http://127.0.0.1:9518
```

In the example above, `cmon` will bind into those IP addresses and listen to port 9500 once started. If you filter out the `ps` output from the server, you should get the following:

```
/usr/sbin/cmon --rpc-port=9500 \
    --bind-addr='10.10.10.13,192.168.33.1,127.0.0.1' \
    --events-client='http://127.0.0.1:9510' \
    --cloud-service='http://127.0.0.1:9518
```

3.3.3 Configuration Options

Values that consist of special characters must be enclosed with a single-quote. Any value change to the configuration options that shall be applied to the `cmon` configuration file or indirectly via GUI or through CLI command, requires a service restart before they are applied.

Following is the list of configuration options inside the CMON configuration file. You can also see them by using `--help-config` the parameter in the terminal:

```
$ cmon --help-config
```

General Configurations

Option	Description
clus- ter_id=<in	Cluster identifier. This will be used by CMON to indicate which cluster to provision. It must be unique, two clusters can not share the same ID. Example: `cluster_id=1`
name=<stri	Cluster name. The cluster name configured under ***ClusterControl → Clusters → choose a clutser → Actions → Edit details → Name*** precedes this. Example: `name='Galera Cluster'` Another alias: `cluster_name`
type=<stri	Cluster type. Supported values are "galera", "mysql_single", "mysqlcluster", "mongodb", "postgresql_single", "replication", "group_replication", "elasticsearch", "sqlserver", "redis-cluster". Example: `type=galera` Another alias: `cluster_type`
tags=<stri	A set of strings the user can specify to tag a cluster. - Example: `tag="production;galera;MySQL;app1"`
cre- ated_by_jo	The ID of the job created this cluster. This is usually automatically generated by ClusterControl. Example: `created_by_job=13`
ven- dor=<strin	Database vendor name. ClusterControl needs to know this in order to distinguish the vendor's relevant naming convention especially for package name, daemon name, deployment steps, recovery procedures, and lots more. Supported value at the moment is "oracle", "percona", "codership", "mariadb", "10gen". Example: `vendor=percona`
use_intern integer>	Setting which disabled the 3rd party repository to be set up. Default is 0 (false).
en- able_html_ integer>	Enables sending of HTML e-mails. Default is 1 (true).
fron- tend_url=<	The ClusterControl URL to be embedded inside e-mail notifications. The URL sent in the emails to direct the recipient to the ClusterControl UI. Example: `frontend_url='https://monitor.domain.com/clustercontrol'`
er- ror_report_	The storage location of error reports on the controller node.
ops_report_	How many days to keep operational reports. Operational Reports matching retention period are removed. Default is 31.
ccx_mode=<	If this cluster is configured from CCX.
ccxad- min_passwo	The password used for the ccxadmin user.
cloud_only_	When enabled, backups configured to be stored in cloud storage (e.g. S3) will be directly uploaded to cloud storage without intermediate node storage. This only applies to streaming backups such as `mariabackup`/`xtrabackup`/`pg_basebackup`. Default: false.
database_g	Setting for database growth monitoring. Default: true.
database_g	Specifies how many of the largest databases will be included in the database growth analysis. Default: 20.
db_growth_	Determines how long the controller should retain data related to database growth monitoring. Default: 32.
email_subj	Custom e-mail subject prefix.
force_http	Forces HTTP repository URLs to be replaced with HTTPS. Default: false.
enter- prise_toke	Customer's Repo/Download Token for Enterprise versions of Databases.
snmp_commu	The SNMP community string to be used on sent SNMP traps.
snmp_proto	The SNMP protocol to be used.
snmp_trap_	The host to which SNMP traps will be sent when an alarm occurs.
snmp_trap_	The target port for SNMP traps when an alarm occurs.
use_preins	Use pre-installed SSL certs instead of generating new ones. Certificates must exist under `/var/lib/cmon/ccx/` named (`ca.pem`, `server.pem`, `server_key.pem`). Default: false

CMON

Option	Description
hostname=<string>	Hostname or IP address of the controller host. Example: hostname=192.168.0.10
controller_id=<integer>	An arbitrary identifier string of this controller instance. Example: controller_id=1
mode=<string>	CMON role. Supported values are "controller", "dual", "hostonly". Example: mode=controller
agentless=<boolean integer>	CMON controller mode (deprecated). Agents are no longer supported. 0 for agent-based or 1 for agentless (default). Example: agentless=1
logfile=<path>	CMON log file location. This is where CMON logs its activity. The file will be automatically generated if it doesn't exist. CMON will write to Syslog by default. Example: logfile=/var/log/cmon.log
pidfile=<path>	CMON process identifier file directory. Keep the default value is recommended. Example: pidfile=/var/run
mysql_hostname=<string>	The MySQL hostname or IP address where the CMON database resides. Using an IP address is recommended. Default is 127.0.0.1. Example: mysql_hostname=192.168.0.5 Other aliases: cmon_mysql_hostname, cmondb_hostname, local_mysql_hostname, cmon_local_mysql_hostname.
mysql_password=<string>	The MySQL password for user cmon to connect to CMON database. Example: mysql_password='cM%^nP4ss' Other aliases: cmon_mysql_password, cmondb_password
mysql_port=<integer>	The MySQL port used by CMON to connect to the CMON database. Example: mysql_port=3306 Other aliases: cmon_mysql_port, cmondb_port
mysql_socket=<boolean integer>	The local UNIX socket file to be used for MySQL connection (if set, TCP will not be used). Example: mysql_socket=/var/lib/mysql/mysqld.sock
mysql_basedir=<path>	The MySQL base directory used by CMON to find MySQL client-related binaries. Example: mysql_basedir=/usr Another alias: basedir
mysql_bindir=<path>	The MySQL binary directory used by CMON to find MySQL client-related binaries. Another alias: bindir Example: mysql_bindir=/usr/bin
mysql_scriptdir=<path>	The scripts dir of MySQL installation.
config_file_path=<path>	The config file path (read-only) for CMON instance.

continues on next page

Table 3.1 – continued from previous page

Option	Description
cmon_db=<string>	CMON database name. Default to "cmon". Example: cmon_db=cmon Another alias: cmondb_database
cmon_user=<string>	The username for the CMON database. Default to "cmon". Example: cmon_user=cmon Another alias: cmondb_user
cmondb_ssl_key=<path>	Path to SSL key, for SSL encryption between the CMON process and the CMON database. Example: cmondb_ssl_key=/etc/ssl/mysql/client-key.pem
cmondb_ssl_cert=<path>	Path to SSL certificate, for SSL encryption between the CMON process and the CMON database. Example: cmondb_ssl_cert=/etc/ssl/mysql/client-cert.pem
cmondb_ssl_ca=<path>	Path to SSL CA, for SSL encryption between the CMON process and the CMON database. Example: cmondb_ssl_ca=/etc/ssl/mysql/ca-cert.pem
cluster_ssl_key=<path>	Path to SSL key, for SSL encryption between the CMON process and managed MySQL Servers. Example: cluster_ssl_key=/etc/ssl/mysql/client-key.pem
cluster_ssl_cert=<path>	Path to SSL certificate, for SSL encryption between the CMON process and managed MySQL Servers. Example: cluster_ssl_cert=/etc/ssl/mysql/client-cert.pem
cluster_ssl_ca=<path>	Path to SSL CA, for SSL encryption between CMON and managed MySQL Servers. Example: cluster_ssl_ca=/etc/ssl/mysql/ca-cert.pem
cluster_certs_store=<path>	Directory path to store SSL-related files. This is required when you want to add a new node in an encrypted Galera cluster. Example: cluster_certs_store=/etc/ssl/galera/cluster_1
cluster_ssl_enforce=<boolean integer>	If set to 1 (true), then SSL is required. Default is 0 (false). CMON is going to raise a critical alarm if an SSL connection can not be made to the node. Example: cluster_ssl_enforce=1

continues on next page

Table 3.1 – continued from previous page

Option	Description
`rpc_key=<string>`	Unique secret token for authentication. To interact with individual clusters via CMON RPC interface (port 9500), one must use this key, or else you would get 'HTTP/1.1 403 Access denied'. ClusterControl UI needs this key stored as RPC API Token to communicate with the CMON RPC interface. Each cluster should be configured with a different`rpc_key` value. This value is automatically generated when a new cluster/server is created or added into ClusterControl. Example: `rpc_key=VJZKhr5CvEGI32dP` Another alias: `rpc_api_key`
`rpc_version=<string>`	The RPC version created the cluster.
`net_read_timeout=<integer>`	Network read timeout value in seconds (DB connections). Default is 10.
`net_write_timeout=<integer>`	Network write timeout value in seconds (DB connections). Default is 10.
`connect_timeout=<integer>`	Network connect timeout value in seconds (DB connections). Default is 10.
`owner=<integer>`	The CMON user ID of the owner.
`group_owner=<integer>`	The CMON group ID of the group owner.
`cdt_path=<string>`	The location in the CMON Directory Tree (CDT). See *s9s-tree*.
`acl=<string>`	The Access Control List is a string controlling the access to the cluster object. See *ACL Text Forms*.
`staging_dir=<path>`	Staging path for temporary files. Example: `staging_dir=/home/ubuntu/s9s_tmp` Another alias: `stagingdir`
`plugin_dir=<path>`	Directory path for CMON plugins. Default to `/var/cmon/plugins` Another alias: `plugin_path`
`cmon_use_mail=<boolean integer>`	Setting to use the `mail` command for sending out emails. Default is 0 (false). Example: `cmon_use_mail=1`
`cmon_mail_sender=<email>`	The sender email address when sending out emails. Example: `cmon_mail_sender=info@example.com`
`cmon_pool_size=<integer>`	The connection pool size to the cmon database. Default is 64. Example: `cmon_pool_size=32`
`cmon_ha_heartbeat_interval_millis=<in`	CMON HA heartbeat interval measured in milliseconds. Default is 1000.
`cmon_ha_heartbeat_millis_rand_max=<in`	CMON HA timeout random component max value. Default is 2500.
`cmon_ha_heartbeat_network_timeout=<in`	CMON HA heartbeat network connection timeout measured in seconds. Default is 5.
`cmon_ha_heartbeat_timeout_millis=<int`	CMON HA timeout for heartbeats in milliseconds. Default is 5000.

continues on next page

Table 3.1 – continued from previous page

Option	Description
cmon_ha_max_send_retry=<integer>	CMON HA log send maximum retry count. Default is 15.
cmon_ha_send_retry_delay=<integer>	CMON HA log send retry delay in seconds. Default is 3.
vault_addr=<URI>	URI address of the Vault server. See *Hashicorp Vault Integration*. Example: vault_addr=https://localhost:8200
vault_auto_migrate=<boolean integer>	Enable this to auto-move all the secrets to the configured Vault instance. Default is 0 (false). See *Hashicorp Vault Integration*. Example: vault_auto_migrate=1
vault_path=<path>	The secrets path prefix for ClusterControl credentials. Default value is clustercontrol. The controller is going to store the data within this path, e.g clustercontrol/controller_UUID/cluster_ID/mysql_password. Each ClusterControl instance and controller, and cluster can have its unique namespace. See *Hashicorp Vault Integration*. Example: vault_path="staging/clustercontrol"
vault_token=<string>	A token used by the controller to authorize itself against the Vault server. The Vault token is created by the Vault server. When the vault_token is present credentials for new clusters will be stored in Vault and not in the cmon configuration files. See *Hashicorp Vault Integration*.
audit_ignore_users=<string>	Skip certain users from audit logging. Comma-separated list. Default: system;ccrpc.
audit_logfile=<path>	Path to CMON's audit log file. Default: /var/log/cmon_audit.log.
connect_heartbeat_when_faulty=<intege	MySQL connections blocking period (in seconds) when a faulty situation (errNo=3159) is detected to prevent resource overuse. Default: 30.
mail_queue_disabled=<boolean>	Disable CMON mail feature. Default: false.
prometheus_hostname=<string>	A Prometheus node hostname for which CMON shall use custom SSH credentials.
prometheus_ssh_user=<string>	SSH username used for accessing the Prometheus node.
prometheus_ssh_port=<integer>	Custom SSH port for Prometheus node. Default: 22.
prometheus_ssh_password=<string>	SSH password for connecting to Prometheus node.
prometheus_ssh_keypath=<path>	SSH key file for connecting to Prometheus node.
prometheus_ssh_keydata=<string>	Base64-encoded SSH key for Prometheus connection.

Operating System

Option	Description
os=<strin	The operating system distribution runs across the cluster, including ClusterControl host. 'redhat' for RedHat-based distributions (CentOS/Red Hat Enterprise Linux/Oracle Linux) or 'debian' for Debian-based distributions (Debian/Ubuntu). Example: `os=redhat`
os-user=<str	Operating system user that will be used by CMON to perform automation tasks like cluster recovery, backups and upgrades. This user must be able to perform super-user activities. Using root is recommended. Example: `os_user=root` Other aliases: `os_user`, `ssh_user`
sudo=<com	The command used to obtain superuser permissions. If sudo user requires a password, specify the sudo command with sudo password here. The sudo command must be trimmed by redirecting stderr to somewhere else. Therefore, it is compulsory to have `-S 2>/dev/null` appended in the `sudo` command. Example: `sudo="echo 'My5ud0'`
os-user_home	The home directory of the user used on nodes. Example: `osuser_home=/home/ubuntu/` Other aliases: `os_user_home`, `ssh_user_home`
software_pack	This is the storage location of software packages, i.e, all necessary files to successfully install a node, if there is no yum/apt repository available, must be placed here. Applies mainly to MySQL Cluster or older Codership/Galera installations.
local_repo_	The used local repository names for cluster deployment. Example: `local_repo_name=epel`
init_name	The OS service name used for starting/stopping the database servers. Another alias: `init_service_name` Example: `init_name=postgresql-9.6`
disable_numa integer>	Not to use NUMA support dependent features. Default is 1 (enabled).

SSH

Option	Description
ssh_identi	The SSH key or key pair file will be used by CMON to connect managed nodes (including ClusterControl node) passwordless. If undefined, CMON will use the home directory of `os_user` and look for `.ssh/id_rsa` file. Example: `ssh_identity=/root/.ssh/id_rsa` Other aliases: `ssh_keypath`, `identity_file`
ssh_port=<	The SSH port used by CMON to connect to managed nodes. If undefined, default to 22. Example: `ssh_port=22`
ssh_acquir integer>	Setting for `libssh` - should it acquire a remote tty. Default is 1 (true). Example: `ssh_acquire_tty=1`
ssh_passwo	The SSH password used for connection to nodes. This is not recommended, hence ClusterControl enforces the passwordless SSH instead.
ssh_timeou	Network timeout value in seconds for SSH connections. Default is 30. Example: `ssh_timeout=30` Another alias: `libssh_timeout=<integer>`
libssh_loglev	Setting for `libssh` logging verbosity to stdout. Accepted values are 0 (NONE), 1 (WARN), 2 (INFO), 3 (DEBUG), 4 (TRACE). Example: `libssh_loglevel=2`
slow_ssh_w	A warning alarm will be raised if it takes longer than the specified time to set up an SSH connection (secs). Default is 6 seconds. Example: `slow_ssh_warning=6`
slow_ssh_c	A critical alarm will be raised if it takes longer than the specified time to set up an SSH connection (secs). Default is 12 seconds. - Example: `slow_ssh_critical=12`

Automatic Recovery

Option	Description
enable_cluster_a integer>	If undefined, CMON defaults to 0 (false) and will NOT perform automatic recovery if it detects cluster failure. The supported value is 1 (cluster recovery is enabled) or 0 (cluster recovery is disabled).
enable_node_auto integer>	If undefined, CMON defaults to 0 (false) and will NOT perform automatic recovery if it detects node failure. The supported value is 1 (node recovery is enabled) or 0 (node recovery is disabled).
enable_autorecov integer>	If undefined, CMON defaults to 0 (false) and will NOT perform automatic recovery if it detects node or cluster failure. The supported value is 0 (cluster and node recovery are disabled) or 1 (cluster and node recovery are enabled). This setting will internally set enable_node_autorecovery and enable_cluster_autorecovery to the specified value.
node_recovery_	Specify a lock file and if present on a node, the node will not recover. The administrator is responsible to create/remove the file. Example: node_recovery_lock_file=/root/do_not_recover
node_recovery_	A timeout value for waiting the new node to reach the synchronized state. Default : 28800.
add_cluster_ti	Timeout value for cluster start during deployment (currently NDB only). Default : 28800.
send_clear_clu integer>	Send notification email if a cluster failure event is cleared (meaning the cluster is recovered). Default is true. Another alias: send_clear_alarm
replication_stage_fai	Automatically stops a slave node if any cluster's stage job fails. Default: false.
replication_failover_	Number of extra db_stats_collection_interval rounds to wait before proceeding with failover. Default: 0.

Monitoring and Thresholds

Option	Description
monitored_mountpoints=<paths>	The MySQL/MongoDB/PostgreSQL data directory used by database nodes for disk performance in a comma-separated list. Example: monitored_mountpoints=/var/lib/mysql,/mnt/data/mysql. Another alias: monitored_mount_points
monitored_nics=<string>	List of network interface names to be monitored for network performance in a comma-separated list. Example: monitored_nics=eth1,eth2
db_stats_collection_interval=<integer	Database metrics sampling interval in seconds. The lowest value is 1. Default is 30 seconds. Example: db_stats_collection_interval=30
host_stats_collection_interval=<integ	Host metrics sampling interval in seconds. The lowest value is 1. Default is 30 seconds. Example: host_stats_collection_interval=30

continues on next page

Table 3.2 – continued from previous page

Option	Description
host_stats_window_size=<integer>	Setting the window size (in seconds) to examine (CPU, memory..) stats to raise/clear host stats alarms. The default is 180 seconds. Example: host_stats_window_size=300
lb_stats_collection_interval=<integer>	Load balancer stats collection interval. Default is 15. Example: lb_stats_collection_interval=30
db_schema_stats_collection_interval=<	How often database growth and table checks are performed in seconds. This translates to information_schema queries. Default is 10800 seconds (3 hours). 0 means disabled. Example: db_schema_stats_collection_interval=10800
db_proc_stats_collection_interval=<in	Setting for database process stats collection interval. The default is 3 seconds. The minimum allowed value is 1 second. Example: db_proc_stats_collection_interval=5
db_log_collection_interval=<integer>	Database log files collection interval. Default is 600. Example: db_log_collection_interval=600
db_deadlock_check_interval=<integer>	How often to check for deadlocks in seconds. Deadlock detection will affect CPU usage on database nodes. Default is 0, which means disabled. Example: db_deadlock_check_interval=600
db_exporter_user=<string>	Database user for Prometheus exporter. Default is db_exporter_user=cmonexporter
db_exporter_password=<string>	Password for db_exporter_user Example: db_exporter_password=myS3cret
db_exporter_use_nonlocal_address=<boo integer>	Specifies if Prometheus exporter should connect to the non-local address of the DB services, instead of 127.0.0.1. Default is 0 (false). Example: db_exporter_use_nonlocal_address=1
db_hourly_stats_collection_interval=<	Database statistic collections interval in seconds. Default is 5. Example: db_hourly_stats_collection_interval=5
swap_warning=<integer>	Warning alarm threshold for swap usage. Default is 5. Example: swap_warning=20
swap_critical=<integer>	Critical alarm threshold for swap usage. Default is 20. Example: swap_critical=40
swap_inout_period=<integer>	The interval for swap I/O alarms in seconds. 0 means disabled. Default is 600 (10 minutes). Example:swap_inout_period=120
swap_inout_warning=<integer>	The number of pages swapped I/O for warning in the specified swap_inout_period. The default is 10240. To determine the page size for the host, use getconf PAGESIZE. Example: swap_inout_warning=51200

continues on next page

Table 3.2 – continued from previous page

Option	Description
swap_inout_critical=<integer>	The number of pages swapped I/O for critical in the specified swap_inout_period. The default is 102400. To determine the page size for the host, use getconf PAGESIZE. Example: swap_inout_critical=102400
save_history_days=<integer>	How many days the controller shall keep monitoring data collected by ClusterControl like database growth, host/database stats and logs. Default is 7. 0 means disabled.
mysqlmemory_warning=<integer>	Warning alarm threshold for MySQL memory. Default is 80.
mysqlmemory_critical=<integer>	Critical alarm threshold for MySQL memory. Default is 90.
ram_warning=<integer>	Warning alarm threshold for RAM usage. Default is 80.
ram_critical=<integer>	Critical alarm threshold for RAM usage. Default is 90.
diskspace_warning=<integer>	Warning alarm threshold for disk usage. Default is 80.
diskspace_critical=<integer>	Critical alarm threshold for disk usage. Default is 90.
cpu_warning=<integer>	Warning alarm threshold for CPU usage. Default is 80.
cpu_critical=<integer>	Critical alarm threshold for CPU usage. Default is 90.
cpu_steal_warning=<integer>	Warning alarm threshold for CPU steal. Default is 10.
cpu_steal_critical=<integer>	Critical alarm threshold for CPU steal. Default is 20.
cpu_iowait_warning=<integer>	Warning alarm threshold for CPU IO Wait. Default is 50.
cpu_iowait_critical=<integer>	Critical alarm threshold for CPU IO Wait. Default is 60.
monitor_cpu_temperature=<boolean integer>	Whether to monitor CPU temperature. Default is 0 (false).
redobuffer_warning=<integer>	Warning alarm threshold for redo buffer usage. Default is 80.
redobuffer_critical=<integer>	Critical alarm threshold for redo buffer usage. Default is 90.
indexmemory_warning=<integer>	Warning alarm threshold for index memory usage. Default is 80.
indexmemory_critical=<integer>	Critical alarm threshold for index memory usage. Default is 90.
datamemory_warning=<integer>	Warning alarm threshold for data memory usage. Default is 80.
datamemory_critical=<integer>	Critical alarm threshold for data memory usage. Default is 90.

continues on next page

Table 3.2 – continued from previous page

Option	Description
tablespace_warning=<integer>	Warning alarm threshold for table space buffer memory usage. Default is 80.
tablespace_critical=<integer>	Critical alarm threshold for table space buffer memory usage. Default is 90.
redolog_warning=<integer>	Warning alarm threshold for redo log usage. Default is 80.
redolog_critical=<integer>	Critical alarm threshold for redo log usage. Default is 90.
enable_is_queries=<boolean integer>	Specifies whether queries to the information_schema will be executed or not. Queries to the information_schema may not be suitable when having many schema objects (hundreds of databases, hundreds of tables in each database, triggers, users, events, stored procedures). If disabled, the query that would be executed will be logged so it can be determined if the query is suitable in your environment. Default is 1 (enabled). Disable with 0.
max_replication_lag=<integer>	Max allowed replication lag in seconds before sending an alarm. Default is 10.
enable_icmp_ping=<boolean integer>	Toggles if the controller shall measure the ICMP ping times to the host. Default is 1 (true).
galera_flush_status_interval=<integer	Interval in seconds for executing FLUSH STATUS on Galera to reset status variables. Default: 0 (disabled).
lb_pre_install_script=<path>	Script executed before load balancer installation/setup.
lb_post_install_script=<path>	Script executed after load balancer installation/setup.
lb_pre_start_script=<path>	Script executed before load balancer node start.
lb_post_start_script=<path>	Script executed after load balancer node start.
lb_backend_draining_grace_sec=<intege	Max time (seconds) a load-balancer backend can stay in draining/OFFLINE_SOFT before alarm. 0 disables. Default: 300.

Query Monitor

Option	Description
long_query_time=<f	The threshold value for slow query checking. Default 0.5. Example: long_query_time=0.0003
log_queries_not_us integer>	Set query monitor to detect queries not using indexes. Default is 0 (false).
query_monitor_use_ integer>	Do not override the local settings of MySQL's long_query_time and log_queries_not_using_indexes. Default is 0 (false).
enable_query_monitor	Setting for query monitor interval in seconds. Default is 1 means enabled. -1 means disable. Another alias: query_monitor_enabled
enable_query_monitor integer>	If enabled the Performance_Schema table events_statements_summary_by_digest will be auto-purged (using TRUNCATE TABLE) every hour. By default, this is disabled (false). Enable by setting it to '1' (true). Another alias: query_monitor_auto_purge_ps
query_monitor_long	Raises an alarm if a query executes for longer than this value in milliseconds. The default is 10000. The minimum value is 1000.
query_monitor_aler integer>	Raises an alarm if a query executes for longer than query_monitor_long_running_query_ms. Default is 0 (disabled). Enable with 1.
query_monitor_kill integer>	Kill the query if the query executes for longer than query_monitor_long_running_query_ms. Default is 0 (Disabled). Enable with 1.
query_monitor_long	Match only queries with an "Info" only matching this regex. No default value means match any Info. Example: `query_monitor_long_running_query_matching_info=(?i)(^SELECT.*
query_monitor_long	The opposite matching of query_monitor_long_running_query_matching_info. Example: `query_monitor_long_running_query_matching_info_negate=(?i)(^ALTER TABLE.*
query_monitor_long	Match only queries with a "Host" only matching this regex. No default value means match any Host. Example: `query_monitor_long_running_query_matching_host=(192.168.10..*
query_monitor_long	Match only queries with a "Db" only matching this regex. No default value means match any Database. Example: query_monitor_long_running_query_matching_db=^joomla_.*
query_monitor_long	Match only queries with a "User" only matching this regex. No default value means match any User. Example: `query_monitor_long_running_query_matching_user=(dbuser1
query_monitor_long	The opposite matching of query_monitor_long_running_query_matching_user. Example: `query_monitor_long_running_query_matching_user_negate=(monitoring_user
query_monitor_long	Match only queries with a "Command" only matching this regex. Defaults to "Query". Example: `query_monitor_long_running_query_matching_command=(?i)(QUERY

Backup

Option	Description
net-cat_port=<str	List of netcat/socat ports and port ranges used to stream backups. The first value before a comma is the preferred port. The next value is a port range where ClusterControl should pick on. Defaults to '9999,9990-9998' which means port 9999 will be preferred if available, otherwise pick the next available port in the defined range. Example: `netcat_port=9999,9990-9998`
backup_user=<	The database username for managing backups. Example: `backup_user=backupuser`
backup_user_p	The database password for the `backup_user`. Example: `backup_user_password=MyS3cretC`
backup_encryp	The AES encryption key used for backups in base64.
back-updir=<path>	The default backup directory, to be pre-filled in ClusterControl UI. Example:`backupdir=/storage/backup`
backup_subdir	Set the name of the backup subdirectory. For more details on the formatting, see *Backup Subdirectory Variables*. Default is "`BACKUP-%I`". Example: `backup_subdir=BACKUP-%I-%D`
backup_retent	How many days to keep the backups. Backups matching retention period are removed. Default is 31. Example: `backup_retention=15`
backup_cloud_	How many days to keep the uploaded backups to the cloud. Backups matching retention period are removed. Default is 180. Example: `backup_cloud_retention=90`
backup_n_safe	How many completed full backups will be kept regardless of their retention status. Default is 1. Example: `backup_n_safety_copies=3`
datadir_backu	During restore/rebuild operations a backup (filesystem copy) of the existing data directory (datadir) may be performed (user decides). Unless specified, the default data directory backup location is `DATADIR_bak`, e.g `/var/lib/mysql_bak` if the `datadir` is `/var/lib/mysql`.
dis-able_backup_e integer>	This setting controls if emails are sent or not if a backup finished or failed. This feature is disabled by default, meaning backup emails are sent. Example: `disable_backup_email=1` Another alias: `disable_backup_emails`
backup_create integer>	Configures CMON if it has to calculate MD5 checksum on the created backup files and verify them. Default is 1 (true). Checksumming is a resource-intensive process and disabling this process might save some resources, especially on a huge backup. Example: `backup_create_hash=0`
backup_pre_sc	Script executed before backup happens. If it returns non-zero, the backup will be aborted. If the script does not exist, the backup will be aborted. Arguments passed: backup id, method, type, hostname, storage hostname, storage dir.
backup_post_s	Script executed after backup happens. If it returns non-zero, the backup will be aborted. If the script does not exist, the backup will be aborted. Arguments passed: backup id, method, type, hostname, storage hostname, storage dir, status.
backup_disksp	Sets the lower bound of free disk space (%) required for backups to start. If below this threshold, backup fails. Default: 0.
backup_delete	Max number of attempts for retrying DELETE_ALL_BACKUPS jobs (triggered on cluster drop). Default: 3.
backup_delete	Delay (in seconds) between retries for DELETE_ALL_BACKUPS jobs. Default: 1.
pgback-rest_cipher_p	The AES key to encrypt the PgBackRest backup repository.
pgback-rest_cipher_t	Encryption cipher used for the PgBackRest repository.
pgback-rest_repo_hos	Repository host for storing PgBackRest backup data.
pgback-rest_repo_pat	Directory path for PgBackRest repository data.

MySQL, MariaDB Nodes

Option	Description
mysql_server_addresses=<string>	A comma-separated list of MySQL hostnames or IP addresses (with or without port is supported). For MySQL Cluster, this should be the list of MySQL API node IP addresses. In the case of Galera Cluster, you can add?slave or ?bvs (backup verification server) to the URL so ClusterControl will register the node accordingly. Example: mysql_server_addresses=192.168.0.11:3306,192.168.0.12:3306,192.168.0.13:3306,192.168.0.14:3306?slave
monitored_mysql_port=<integer>	MySQL port for the managed cluster. ClusterControl assumes all DB nodes are listening on the same port. Default is 3306. Example: monitored_mysql_port=3306 Other aliases: cmon_local_mysql_port,local_mysql_port
monitored_mysql_root_user=<string>	MySQL root user for the managed cluster. ClusterControl assumes all DB nodes share the same root user. The user must have the same privileges as root (SUPER with GRANT OPTION). This is required when you want to scale your cluster by adding a new database node or replica. Default is "root". Example: monitored_mysql_root_user=dbadmin
monitored_mysql_root_password=<string>	MySQL root password for the managed cluster. ClusterControl assumes all DB nodes share the same root password. This is required when you want to scale your cluster by adding a new database node or replica. Example: monitored_mysql_root_password='r00tP$@^%sw0rd'
skip_name_resolve=<boolean integer>	Setting to disable name resolution. Default is 1 means skip_name_resolve is enabled.
mysql_version=<string>	The MySQL/MariaDB major version. Example: mysql_version='8.0' Other alias: server_version
galera_port=<integer>	Exclusive for Galera Cluster. The Galera communication port to be used when adding nodes/garbd, and constructing wsrep_cluster_address. Default is 4567.
galera_version=<string>	Exclusive for Galera Cluster. The Galera API major version number. Example: galera_version=3.x
galera_vendor=<string>	Exclusive for Galera Cluster. The database vendor name. Supported values are "percona", "codership" and "mariadb". Example: galera_vendor=mariadb

continues on next page

Table 3.3 – continued from previous page

Option	Description
repl_user=<string>	The MySQL replication user. Example: repl_user=repluser
repl_password=<string>	Password for repl_user Example: repl_password='ZG04Z2Jnk0MUWAZK'
replication_failover_whitelist=<strin	A comma-separated list of MySQL slaves that should be used as potential master candidates. If no server on the whitelist is available (up/connected) the failover will fail. If this variable is set, only those hosts will be considered. This parameter takes precedence over replication_failover_blacklist Example: replication_failover_whitelist=192.168.1.11,192.168.1.12
replication_failover_blacklist=<strin	A comma-separated list of MySQL replicas that will never be considered a primary candidate. You can use it to list replicas that are used for backups or analytical queries. If the hardware varies between replicas, you may want to put here the replica which use slower hardware. replication_failover_whitelist takes precedence over this parameter if it is set. Example: replication_failover_blacklist=192.168.1.101,192.168.1.102
replication_skip_apply_missing_txs=<b integer>	Default is 0. Skip the check process for additional missing transactions before promoting a slave to a master and just use the most advanced slave. Such a process may result in a serious problem though - if an errant transaction is found, replication may be broken. Example: replication_skip_apply_missing_txs=1
replication_stop_on_error=<boolean integer>	Controls if the failover/switchover procedures should fail if errors are encountered that may cause data loss. Default : true.
replication_failover_wait_to_apply_ti	The candidate waits up to this many seconds to apply outstanding relay log events (retrieved_gtids) before failing over. Default is -1, which means ClusterControl will wait indefinitely for it to apply all missing transactions from its relay logs. This is safe, but, if for some reason, the most up-to-date replica is lagging badly, failover may take hours to complete. If set to 0, failover happens immediately, no matter if the master candidate is lagging or not. Default -1 seconds (wait forever). A value higher than 0 means ClusterControl will wait for the specified seconds before failover happens. Example: replication_failover_wait_to_apply_timeout=0

continues on next page

Table 3.3 – continued from previous page

Option	Description
replication_auto_rebuild_slave=<boole integer>	If the SQL THREAD is stopped and the error code is non-zero then the replica will be automatically rebuilt. 1 means enable, default is 0 (false). Example: replication_auto_rebuild_slave=1
replication_onfail_failover_script=<p	Path to the failover script on ClusterControl node. This script executes as soon as it has been discovered that failover is needed. If the script returns non-zero it will force the failover to abort. If the script is defined but not found, the failover will be aborted. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: failover_script.sh "arg1" "arg2" "arg3" "arg4". The script must be accessible on the controller and executable. Example: replication_onfail_failover_script=/usr/local/bin/failover_script.sh
replication_pre_failover_script=<path	Path to the failover script on ClusterControl node. This script executes before the failover happens, but after a candidate has been elected and it is possible to continue the failover process. If the script returns non-zero it will force the failover to abort. If the script is defined but not found, the failover will be aborted. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: pre_failover_script.sh "arg1" "arg2" "arg3" "arg4". The script must be accessible on the controller and executable. Example: replication_pre_failover_script=/usr/local/bin/pre_failover_script.sh

continues on next page

Table 3.3 – continued from previous page

Option	Description
replication_post_failover_script=<pat	Path to the failover script on ClusterControl node. This script executes after the failover has happened. If the script returns non-zero, a warning will be written in the job log. The script must be accessible and executable on the controller. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: `post_failover_script.sh "arg1" "arg2" "arg3" "arg4"`. The script must be accessible on the controller and executable. Example: `replication_post_failover_script=/usr/local/bin/post_failover_script.sh`
replication_post_unsuccessful_failove	Path to the script on the ClusterControl node. This script is executed after the failover attempt failed. If the script returns non-zero a Warning will be written in the job log. The script must be accessible on the controller and executable. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: `post_fail_failover_script.sh "arg1" "arg2" "arg3" "arg4"`. The script must be accessible on the controller and executable. Example: `replication_post_unsuccessful_failover_script=post sh`
replication_failed_reslave_failover_s	Path to the script on the ClusterControl node. This script is executed after that a new master has been promoted and if the enslaving of the slaves to the new master fails. If the script returns non-zero a Warning will be written in the job log. The script must be accessible on the controller and executable. Example: `replication_failed_reslave_failover_script=/usr/local/bin/fail_reslave_failover_script.sh`

continues on next page

Table 3.3 – continued from previous page

Option	Description
replication_pre_switchover_script=<pa	Path to the switchover script on ClusterControl node. This script executes before the switchover happens. If the script returns non-zero it will force the switchover to fail. If the script is defined but not found, the switchover will be aborted. The script must be accessible on the controller and executable. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: `pre_switchover_script.sh "arg1" "arg2" "arg3" "arg4"`. The script must be accessible on the controller and executable. Example:`replication_pre_switchover_script=/usr/local/bin/pre_switchover_script.sh`
replication_post_switchover_script=<p	Path to the switchover script on ClusterControl node. This script executes after the switchover happened. If the script returns non-zero a Warning will be written in the job log. The script must be accessible on the controller and executable. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: `post_switchover_script.sh "arg1" "arg2" "arg3" "arg4"`. The script must be accessible on the controller and executable. Example: `replication_post_switchover_script=/usr/local/bin/post_switchover_script.sh`
replication_check_external_bf_failove integer>	Before attempting a failover, perform extended checks by checking the slave status to detect if the master is truly down, and also check if ProxySQL (if installed) can still see the master. If the master is detected to be functioning, then no failover will be performed. Default is 0 (false) meaning the checks are disabled. 1 means enable. Example: `replication_check_external_bf_failover=0`
replication_check_binlog_filtration_b integer>	Before attempting a failover, verify filtration (`binlog_do`/`ignore_db`) and `replication_*` are identically configured on the candidate and the slaves. Default is 0 (false) meaning the checks are disabled. 1 means enable. Example: `replication_check_binlog_filtration_bf_failover=1`

continues on next page

Table 3.3 – continued from previous page

Option	Description
replication_failover_events=<boolean integer>	Automatically failover events (SLAVE-SIDE_DISABLED) and enable the event_scheduler after a replication failover/switchover action. Default is disabled. Enabled by setting it to 1. Example: replication_failover_events=1
auto_manage_readonly=<boolean integer>	Enable/Disable automatic management of the MySQL server read_only variable. Default is 1 (true), which means ClusterControl will set the read_only=ON if the MySQL replication role is a slave. Default is 1 (enabled). Example: auto_manage_readonly=0
schema_change_detection_address=<stri	This option must be defined to use "Operational Report for Schema Change". Creating a report of thousands of database objects (schemas, tables, etc) will take some time (about 5-10 minutes) depending on the hardware. It's recommended to configure a specific host to run this job for example on a replication slave or an asynchronous slave connected to e.g a Galera or Group Replication Cluster. For NDB this option should be set to a MySQL server used for admin purposes. Example: schema_change_detection_address=192.168.111.53
schema_change_detection_pause_time_ms=	Throttle the detection process by pausing every this value, in milliseconds. For example, if defined as 3000, ClusterControl will pause the operation for every 3 seconds. Example: schema_change_detection_pause_time_ms=3000
schema_change_detection_databases=<st	A comma-separated string of database names and also supports wildcards. For example "DB%", will evaluate all databases starting with "DB". Example:schema_change_detection_databases=mydb%,shops_db,mymonitoring
datanode_addresses=<string>	Exclusive for MySQL Cluster. Comma-separated list of data node hostnames or IP addresses. Example: datanode_addresses=192.168.0.41,192.168.0.42
mgmnode_addresses=<string>	Exclusive for MySQL Cluster. Comma-separated list of management node hostnames or IP addresses. Example: mgmnode_addresses=192.168.0.51,192.168.0.5.2
ndb_connectstring=<string>	Exclusive for MySQL Cluster. NDB connection string for the cluster. Example: ndb_connectstring=192.168.0.51:1186,192.168.0.52:1186

continues on next page

Table 3.3 – continued from previous page

Option	Description
ndb_binary=<string>	Exclusive for MySQL Cluster. NDB binary for data node. Supported values are ndbd or ndbmtd. Example: ndb_binary=ndbmtd
ndbd_datadir=<path>	Exclusive for MySQL Cluster. The datadir of the NDBD nodes. Example: ndbd_datadir=/var/lib/mysqlcluster
mgmd_datadir=<path>	Exclusive for MySQL Cluster. The datadir of the NDB MGMD nodes. Example: mgmd_datadir=/var/lib/mysql
db_configdir=<string>	Exclusive for MySQL Cluster. The datadir of the NDB MGMD nodes. Example: db_configdir=/etc/mysql-cluster
percona_client_id=<string>	Percona client identifier.
percona_pro_token=<string>	Token to access Percona Pro repository.

MongoDB Nodes

Option	Description
mongodb_server_addr	Comma-separated list of MongoDB shard or replica IP addresses with port. Example: mongodb_server_addresses=192.168.0.11:27017,192.168.0.12:27017,192.168.0.13:27017
mongoarbiter_server_add	Comma-separated list of MongoDB arbiter IP addresses with port. Example: mongoarbiter_server_addresses=192.168.0.11:27019,192.168.0.12:27019,192.168.0.13:27019
mongocfg_server_add	Comma-separated list of MongoDB config server IP addresses with port. Example: mongocfg_server_addresses=192.168.0.11:27018,192.168.0.12:27018,192.168.0.13:27018
mongos_server_addre	Comma-separated list of MongoDB mongos IP addresses with port. Example: mongos_server_addresses=192.168.0.11:27017,192.168.0.12:27017,192.168.0.13:27017
mongodb_basedir=<pa	Location MongoDB base directory to find the client-related binaries. Example: mongodb_basedir=/usr
mongodb_user=<strin	MongoDB admin/root username. Example: mongodb_user=root
mongodb_password=<s	Password for mongodb_user Example: mongodb_password='kPo123^^#*'
mongodb_authdb=<str	The database containing user information to use for authentication. Default is admin. Example: mongodb_authdb=admin
mongodb_cluster_key	The cluster's nodes authenticating to each other using this key. Example: mongodb_cluster_key=/etc/repl.key
server_selection	Mongo connection URI option. Defines the timeout value in milliseconds till the mongo driver should try to do a successful server selection operation. The default is 30000. Example: server_selection_timeout_ms=40000
server_selection integer>	Mongo connection URI option. Defines if server selection should be repeated on failure until a server selection timeout expires, or just return with failure at once. Default is 1 (true). Example: server_selection_try_once=0

PostgreSQL, TimescaleDB Nodes

Option	Description
post-gresql_s	Comma separated list of PostgreSQL instances with port. Example: postgresql_server_addresses=192.168.10.100:5432,192.168.10.101:5432 Other alias: postgre_server_addresses
post-gresql_u	The PostgreSQL admin user name. Default is Postgres. Example: post-gresql_user=postgres Other alias: postgre_user
post-gresql_p	The PostgreSQL admin password. Example: post-gresql_password='p4s$^#0rd123' Other alias: postgre_password
wal_rete inte-ger>	Retention hours to erase old WAL archive logs for PITR. Default is 0, which means WAL archive logs are kept forever. Other alias: pitr_retention_hours
repli-ca-tion_aut inte-ger>	If the SQL THREAD is stopped and the error code is non-zero then the slave will be automatically rebuilt. 1 means enable, default is 0 (false). Example: replica-tion_auto_rebuild_slave=1
repl_use	The PostgreSQL replication user. Example: repl_user=repluser
repl_pas	Password for repl_user Example: repl_password='ZG04Z2Jnk0MUWAZK'
repli-ca-tion_fai	Comma-separated list of PostgreSQL slaves which should be used as potential master candidates. If no server on the whitelist is available (up/connected) the failover will fail. If this variable is set, only those hosts will be considered. This parameter takes precedence over replication_failover_blacklist. Example: replication_failover_whitelist=192.168.1.11,192.168.1.12
repli-ca-tion_fai	Comma-separated list of PostgreSQL slaves which will never be considered a master candidate. You can use it to list slaves that are used for backups or analytical queries. If the hardware varies between slaves, you may want to put here the slaves which use slower hardware. replication_failover_whitelist takes precedence over this parameter if it is set. Example: replication_failover_blacklist=192.168.1.101,192.168.1.102
repli-ca-tion_pre	Path to the failover script on ClusterControl node. This script executes before the failover happens, but after a candidate has been elected and it is possible to continue the failover process. If the script returns non-zero it will force the failover to abort. If the script is defined but not found, the failover will be aborted. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: pre_failover_script.sh "arg1" "arg2" "arg3" "arg4". The script must be accessible on the controller and executable. Example: replication_pre_failover_script=/usr/local/bin/pre_failover_script.sh
repli-ca-tion_pos	Path to the failover script on ClusterControl node. This script executes after the failover has happened. If the script returns non-zero, a warning will be written in the job log. The script must be accessible and executable on the controller. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: post_failover_script.sh "arg1" "arg2" "arg3" "arg4". The script must be accessible on the controller and executable. Example: replication_post_failover_script=/usr/local/bin/post_failover_script.sh
repli-ca-tion_pos	Path to the script on the ClusterControl node. This script is executed after the failover attempt failed. If the script returns non-zero a Warning will be written in the job log. The script must be accessible on the controller and executable. 4 arguments are supplied to the script: - arg1="All servers in the replication" - arg2="The failed master" - arg3="Selected candidate" - arg4="Slaves of old master" The arguments will be passed like this: post_fail_failover_script.sh "arg1" "arg2" "arg3"

Elasticsearch Nodes

Option	Description
elastic_server_addresses=<string>	Comma-separated list of Elasticsearch server instances.
elastic_user=<string>	Elasticsearch user name.
elastic_password=<string>	Password for the Elasticsearch user.
elastic_https=<boolean>	Indicates whether ClusterControl communicates with Elasticsearch nodes via HTTPS.
elastic_certs_password=<string>	Password for TLS/HTTPS certificates generation.
elastic_force_built_in_user=<boole	Force use of Elasticsearch built-in user credentials.
elastic_shared_filesystem=<string>	Configured shared filesystems in cluster. Format: :,:.
elastic_snapshot_repository=<strir	Configured Elasticsearch snapshot repository name.
elastic_snapshot_repository_type=<	Snapshot repo type: fs (shared filesystem) or fs-nfs (filesystem with managed NFS).

MSSQL Nodes

Option	Description
license=<string>	License applied to MSSQL deployments.
mssql_server_addresses=<strin	Comma-separated list of MSSQL instances.
mssql_sa_username=<string>	The MSSQL sa username.
mssql_sa_password=<string>	The MSSQL sa password.
mssql_sysadmin_username=<stri	External sysadmin username allowed to connect.
mssql_sysadmin_password=<stri	Password for external sysadmin user.
mssql_backup_cert_password=<s	Password used to encrypt the MSSQL backup certificate created by CMON.
mssql_master_key_password=<st	Password for the MSSQL master key created by CMON.

3.3.4 LDAP

ClusterControl has a built-in support for LDAP authentication and requires another configuration file called cmon-ldap.cnf. The configuration file resides under the /etc/cmon.d directory and comes by default with the package. When an LDAP configuration change event happens in ***ClusterControl GUI → User management → LDAP → LDAP Settings***, ClusterControl will commit the change into the controller's runtime and save the configuration directly into cmon-ldap.cnf.

LDAP integration is only available in the Enterprise edition.

Example output

The following is an example configuration if one has an Active Directory server integrated with ClusterControl:

```
#
```

```
# Enables LDAP authentication. The native authentication in ClusterControl
# will also work.
#
enabled = true

#
# Enter the LDAP or LDAPS Uniform Resource Identifier (URI), with the port
# number (if applicable)
#
ldapServerUri = "ldap://support-ad.severalnines.com"

#
# The "Distinguished Name" is used to bind the LDAP server. This user
# requires read access to all LDAP user and group entries to work
# correctly. ClusterControl must perform a LDAP search using the DN before
# any user can log in. This field is case-sensitive.
#
ldapAdminUser = "CN=Administrator,CN=Users,DC=severalnines,DC=com"

#
# Enter the password for the "Distinguished Name" login.
#
ldapAdminPassword = "<redacted>"

#
# Enter the user's "Relative Distinguished Name" (RDN) to locate this
# user's information. This field is case-sensitive.
#
ldapUserSearchRoot = "CN=Users,DC=severalnines,DC=com"

#
# The group's "Relative Distinguished Name" (RDN) is used to locate the
# group information. ClusterControl does not support LDAP users that do
# not belong to at least one LDAP group.
#
ldapGroupSearchRoot = "OU=Groups,DC=severalnines,DC=com"

[ldap_settings]
#
# This is used to filter the object class of the LDAP users. If empty,
# all object classes will be returned.
#
# ldapUserClassName = "objectclass=*"

#
# The LDAP attribute which should contain the username, separated by a
# comma (whitespace value is not allowed). For Active Directory, this is
# commonly "sAMAccountName" and "uid" for OpenLDAP. If more than one
# attribute is specified ClusterControl will attempt to look up all of␣
↪them
```

```
# with the first non-empty reply (in the particular order) used for the
# login username.
#
ldapUsernameAttributes = "sAMAccountName"

#
# The LDAP attribute which should contain the full name of the user,
# separated by a comma. For Active Directory, this is commonly
# "displayName" and "cn" for OpenLDAP. If more than one attribute is
# specified ClusterControl will attempt to look up all of them with the
# first non-empty reply (in the particular order) used for the user's full
# name.
#
ldapRealnameAttributes = "displayName"

#
# The LDAP attribute which should contain the email address of the user,
# separated by a comma. For Active Directory, this is commonly
# "userPrincipalName" and "mail" for OpenLDAP. If more than one attribute
# is specified, ClusterControl will attempt to look up all of them with␣
↪the
# first non-empty reply (in the particular order) used for the user's␣
↪email
# address.
#
ldapEmailAttributes = "userPrincipalName"

#
# Filter the object class for the LDAP groups. If empty, all object␣
↪classes
# will be returned.
#
# ldapGroupClassName = ""

#
# The LDAP attribute which represents the group's members, separated by
# a comma. For Active Directory, this is commonly "member" and "memberUid"
# for OpenLDAP. These attributes will be used to create combinations with
# the value of Group mapping attributes when querying the LDAP server to
# retrieve the group's information.
#
ldapMemberAttributes = "member"

#
# The LDAP attribute of which name holds the group membership, separated
# by a comma. For Active Directory, this is commonly "dn" and "uid" for
# OpenLDAP. The value of these attributes will be used to create
# combinations with Static member attributes to query the LDAP server to
# retrieve the group.
#
```

```
ldapGroupIdAttributes = "dn"

#
# The LDAP attribute where the name represents the group's name. This is
# commonly "cn". The value of this attribute will be used in the group
# mapping with "ClusterControl's Team" for authorization purposes.
#
ldapGroupNameAttribute = "cn"

#
# This variable specifies the connection timeout in seconds if the LDAP
# server is unreachable or takes too long to respond.
#
ldapNetworkTimeout = 5

#
# This is the current LDAP protocol version being used.
#
ldapProtocolVersion = 3

#
# This variable specifies the limit in seconds for an LDAP query to␣
↪finish,
# any query which takes longer will be aborted.
#
ldapQueryTimeLimit = 5

[security]

#
# This is the location of the CA certification file. (Only for LDAPS).
#
# caCertFile = "/etc/ssl/certs/ldap-ca.crt"

#
# This is the location of the certification file. (Only for LDAPS).
#
# certFile = "/etc/ssl/certs/ldap.crt"

#
# This is the location of the certificate key file. (Only for LDAPS).
#
# keyFile = "/etc/ssl/certs/ldap.pem"
[DBAadmins]
ldapGroupId   = "DBA"
cmonGroupName = "admins"
```

3.3.5 CMON Database

The CMON database is the persistent store for all monitoring data collected from the managed nodes, as well as all ClusterControl metadata (e.g. what jobs there are in the queue, backup schedules, backup statuses, etc.). The ClusterControl Controller requires a MySQL database running on `mysql_hostname` as defined in the configuration file. The database name and the user is "cmon" and is immutable. The CMON database dump files are shipped together with the ClusterControl Controller package and can be found under `/usr/share/cmon` once it is installed.

MySQL user "cmon" needs to have proper access to the CMON database by performing the following grant:

Grant all privileges to "cmon" at `hostname` value (as defined in CMON configuration file) on ClusterControl host:

```
GRANT ALL PRIVILEGES ON *.* TO 'cmon'@'<hostname>' IDENTIFIED BY '
↪<mysql_password>' WITH GRANT OPTION;
```

Grant all privileges for "cmon" at 127.0.0.1 on ClusterControl host:

```
GRANT ALL PRIVILEGES ON *.* TO 'cmon'@'127.0.0.1' IDENTIFIED BY '
↪<mysql_password>' WITH GRANT OPTION;
```

On every managed database server, grant all privileges to cmon at the controller's `hostname` value (as defined in CMON configuration file) on each of the managed database host:

```
GRANT ALL PRIVILEGES ON *.* TO 'cmon'@'<hostname>' IDENTIFIED BY '
↪<mysql_password>' WITH GRANT OPTION;
```

If one deploys a cluster using ClusterControl deployment wizard, the above privileges will be configured automatically.

3.4 ClusterControl GUI

3.4.1 Introduction

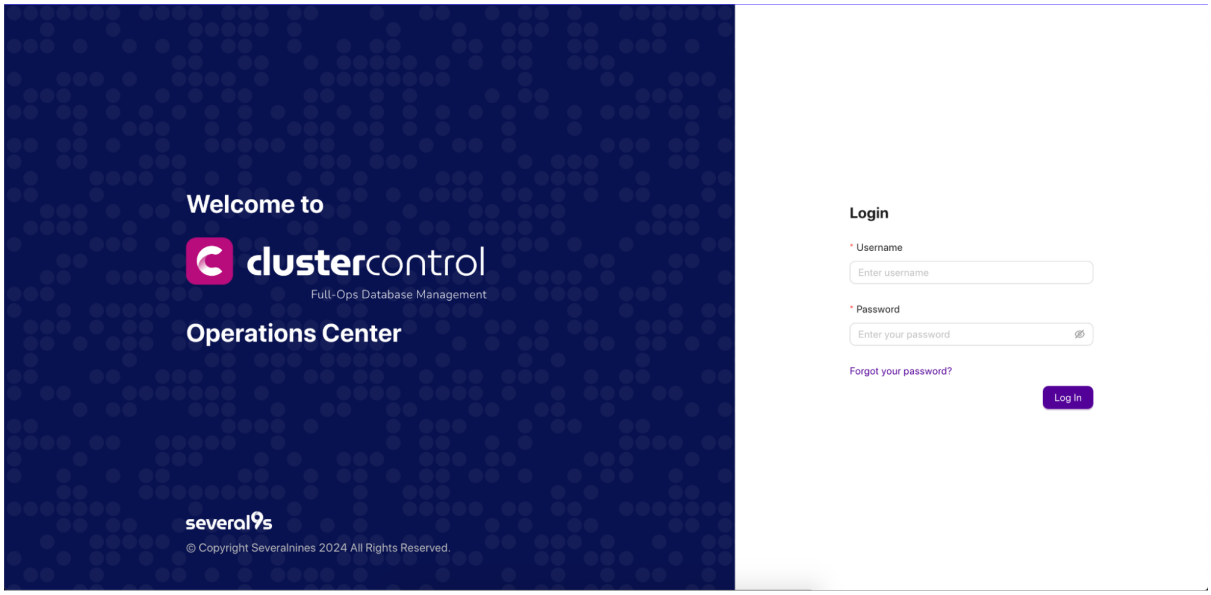
ClusterControl's Graphical User Interface (GUI) serves as a unified platform for managing database clusters. It streamlines complex database operations and ensures the reliability, performance, and security of database systems by providing a single interface to control multiple ClusterControl instances (controllers). The GUI is designed to handle large-scale environments, supporting over 1000 nodes.

The ClusterControl GUI is built upon the closely integrated `clustercontrol-mcc` and `clustercontrol-proxy` packages. This newer version of the `clustercontrol-mcc` web application no longer relies on an Apache server. The `cmon-proxy` service now serves the web application instead via a process called `ccmgr`, also known as **ClusterControl Manager**. By default, a ClusterControl installation registers the local controller.

Info

Prior to ClusterControl v2.3.2 (April 2025), ClusterControl GUI was provided by a different package called `clustercontrol2`. This package is still available in case users would like to use Apache as the web server (e.g, security compliance, advanced proxying and TLS, etc). However, `clustercontrol2` does not have the multi-controller and Kubernetes functionalities within.

ClusterControl GUI can be accessed via `https://<ClusterControl_host>` (or any other port passed during the initial setup). Use one of the ClusterControl users from the existing ClusterControl installation/environment to log in:



3.4.2 Files and location

The `clustercontrol-mcc` package installs the `clustercontrol-proxy` package as a dependency (the web server). The web application is located at `/var/www/html/clustercontrol-mcc` and the configuration file is located at `/var/www/html/clustercontrol-mcc/config.js`. Here is an example content:

```
window.FEAS_ENV = {
  CMON_API_URL: '/',
  GIT_SHA: 'f457257a3',
  GIT_BRANCH: 'release-mcc-2.3.3',
  VERSION: '2.3.3',
  BUILD_NUMBER: '541',
  USER_REGISTRATION: 1,
}
```

The service name for the web server is called `cmon-proxy` and the systemd unit definition file is located at `/etc/systemd/system/cmon-proxy.service`. The `cmon-proxy` is also responsible for managing, collecting data, and monitoring the local and external ClusterControl controllers. See *ClusterControl Proxy* for details.

3.4.3 Managing multiple controllers

A single ClusterControl GUI can be used to manage multiple ClusterControl controllers. ClusterControl's horizontal scaling is achieved through the ClusterControl Operation Center (Ops-C), which enables a single ClusterControl GUI to manage multiple ClusterControl controllers. This is ideal for large database farms with thousands of nodes.

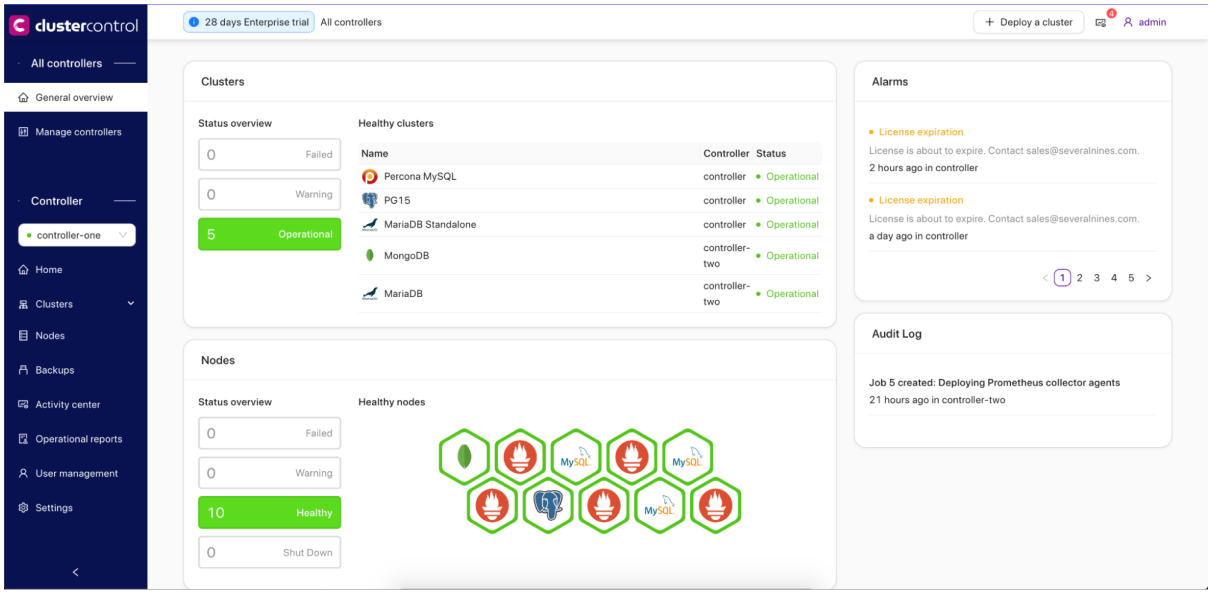
To set up Ops-C, you need to install additional ClusterControl servers, import or add clusters to them as you would with a standard ClusterControl operation. Then, activate the multi-controller GUI mode (Ops-C), allowing you to integrate these additional controllers into the same GUI for remote monitoring within a single interface.

By default, each ClusterControl GUI connects to its local controller in single-controller GUI mode. For details on activating multi-controller GUI mode (Ops-C), refer to *Integration → ClusterControl Ops-C*.

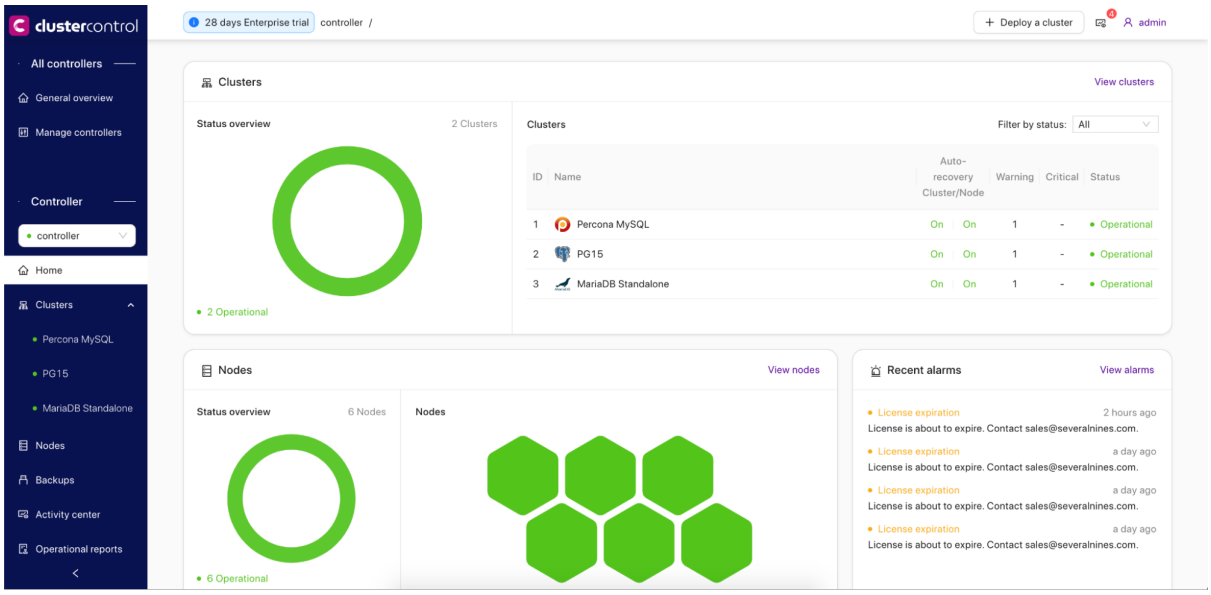
If multi-controller interface is enabled (default is disabled):

- The Controllers' Overview page features a dashboard that presents aggregated information on registered or managed CMON controllers, providing insights into the status of controllers, ongoing jobs, alarms, and backups, as well as the status of the database clusters and nodes.
- The web user interface allows users to select the "Active CMON Controller", which determines the information displayed throughout the application (excluding the Controllers' Overview) and directs user actions for clusters or nodes to the selected CMON Controller.

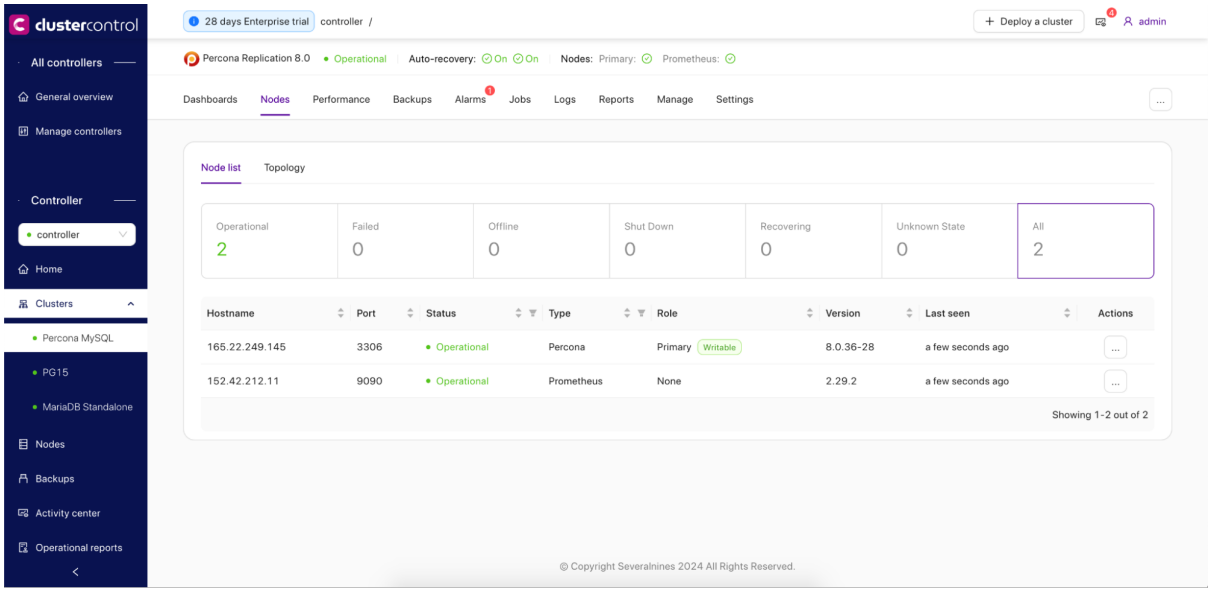
After all the ClusterControl has been added to the ClusterControl GUI, the Operations Center can give information about the status related to the Clusters, Nodes, and Controllers in the Overview Page:



Discover the status of Clusters and Nodes in specific controllers as shown below:



Go through the node list for specific clusters in the selected controller:



See also

- *Integration → ClusterControl Ops-C*
- *ClusterControl Proxy*

3.5 ClusterControl CLI

Also known as **s9s-tools**, this optional package is introduced in ClusterControl version 1.4.1, which contains a binary called `s9s`. It is a command-line tool to interact, control, and manage database clusters using the ClusterControl Database Platform. Starting from version 1.4.1, the installer script will automatically install this package on the ClusterControl node. You can also install it on another computer or workstation to manage the database cluster remotely. Communication between this client and the CMON controller is encrypted and secure through TLS. This command-line project is open source and publicly available at GitHub.

The command-line tool is invoked by executing a binary called `s9s`. The commands are basically JSON messages being sent over to the ClusterControl Controller (CMON) RPC interface. Communication between the s9s (the command-line tool) and the cmon process (ClusterControl Controller) is encrypted using TLS and requires port 9501 to be opened on the controller and the client host.

Following is the list of supported commands and options for ClusterControl CLI. You can also get the same information by accessing the manual page or using the `--help` flag in the terminal:

```
man s9s
s9s --help
```

3.5.1 s9s-account

Manage user accounts on clusters. The "account" term in this section can be treated as a database user account of a managed database server/cluster.

Usage

```
s9s account {command} {options}
```

Commands

Name, shorthand	Description
`--create`	Creates a new account on the cluster. Note that the account is an account of the cluster and not a user of the Cmon system.
`--delete`	Removes an account.
`--list, -L`	Lists the accounts on the cluster.
`--grant`	Grant privileges for an account on one or more databases.

Options

Name, shorthand	Description
`--accoun`	The account to be used or created on the cluster. The command-line option argument may contain a username, a password for the user and a hostname identifying the host from where the user may log in. The `s9s` command-line tool will handle the command line option argument as an URL encoded string, so if the password for example contains an `@` character, it should be encoded as `%40`. URL encoded parts are supported anywhere in the string, usernames and passwords (and even hostnames) may also have special characters.
`-l, --long`	This option will change its format to show more details.
`--privat`	Create a secure, more restricted account on the cluster. The actual interpretation of this flag depends on the controller, the current version is restricting access to the ProxySQL servers. The account that is created with the `--private` option will not be imported into the ProxySQL to have access through the ProxySQL server immediately after they are created on the cluster.
`--privil`	Privileges to be granted to a user account on the server. See *Privilege Expression*.
`--with-d`	Creates a database for the new account while creating a new user account on the cluster. The name of the database will be the same as the name of the account and all access rights will be granted for the account to use the database.

Privilege Expression

The privileges are specified using a simple language that is interpreted by the CMON Controller. The language is specified as follows:

```
expression: specification[;...]
specification: [object[,...]:]privilege[,...]
object: {
  *
  | *.*
```

(continues on next page)

(continued from previous page)

```
  | database_name.*
  | database_name.table_name
  | database_name
}
```

Note that an object name on itself is a database name (and not a table name) and multiple objects can be enumerated by using the `,` as a separator. It is also important that multiple specifications can be enumerated using the semicolon (`;`) as a separator.

The expression `MyDb:INSERT,UPDATE;Other:SELECT` for example, defines `INSERT` and `UPDATE` privileges on the MyDb database and `SELECT` privilege on the Other database. The expression `INSERT,UPDATE` on the other hand, would specify `INSERT` and `UPDATE` privileges on all databases and all tables.

Examples

- Create a new MySQL user account `myuser` with password `secr3tP4ss`, and allow it to have `ALL PRIVILEGES` on database `shop_db` while `SELECT` on the table `account_db.payments`:

```
s9s account \
  --create \
  --cluster-id=1 \
  --account="myuser:Passw0rd999@192.168.0.100" \
  --privileges="shop_db.*:ALL;account_db.payments:SELECT"
```

- Create a new PostgreSQL user account called `mydbuser`, and allowed the host in the network subnet 192.168.0.0/24 to access the database `mydbshop`:

```
s9s account --create \
  --cluster-id=50 \
  --account='mydbuser:k#Passw0rd999@192.168.0.0/24' \
  --privileges="mydbshop.*:ALL"
```

- Delete a database user called `joe`:

```
s9s account \
  --delete \
  --cluster-id=1 \
  --account="joe"
```

- Lists the accounts on the cluster.

```
s9s account \
  --list \
  --long \
  --cluster-id=1
```

- Modify the existing MariaDB privileges of a user called `admin1@%` for cluster ID 4:

```
s9s account \
    --grant \
    --cluster-id=4 \
```

(continues on next page)

(continued from previous page)

```
    --account="admin1@%" \
    --privileges="REPLICATION SLAVE,PROCESS,CREATE USER,REPLICATION␣
↪SLAVE ADMIN,SLAVE MONITOR;mydb:ALL PRIVILEGES,GRANT OPTION;db1:ALL␣
↪PRIVILEGES,GRANT OPTION;db2:ALL PRIVILEGES,GRANT OPTION;db3:ALL␣
↪PRIVILEGES,GRANT OPTION;performance_schema:SELECT"
```

3.5.2 s9s-alarm

Manage alarms.

Usage

```
s9s alarm {command} {options}
```

Commands

Name, shorthand	Description
`--delete`	Sets the alarm to be ignored. This will not in fact delete the alarm, but it will make the alarm to disappear from the active alarms list, hence the name of the option.
`--list`	Lists the active alarms.
`--stat`	Prints a simple list of the number of alarms.

Options

Name, shorthand	Description
`--cluster-id=ID, -i`	The ID of the cluster to manipulate.
`--cluster-name=NAM -n`	Sets the cluster name. If the operation creates a new cluster this will be the name of the new cluster.
`--alarm-id=ID`	The ID of the alarm to manipulate.

Examples

- List out all alarms generated by ClusterControl for a database cluster named “PostgreSQL Cluster”:

```
s9s alarm --cluster-name="PostgreSQL Cluster" --list
```

- Delete an alarm:

```
s9s alarm --delete --alarm-id=1015
```

- Print a simple list of the number of alarms:

```
s9s alarm --stat
```

Example output

```
1,0,0
```

The list will contain three numbers for every line. The first is the cluster ID, the second is the number of critical alarms and the third is the warning level alarms for the given cluster.

3.5.3 s9s-backup

View and create database backups. Three backup methods are supported:

- mysqldump
- xtrabackup (full)
- xtrabackup (incremental)
- mariabackup (full)
- mariabackup (incremental)
- mongodump
- pg_dumpall
- pgdump

The s9s client also needs to know:

- The cluster ID (or cluster name) of the cluster to backup.
- The node to backup.
- The databases that should be included (default all databases).

By default, the backups will be stored on the controller node. If you wish to store the backup on the data node, you can set the flag `--on-node`.

Note

Note If you are using Percona Xtrabackup, an incremental backup requires that there is already a full backup made of the same databases (all or individually specified). Otherwise, the incremental backup will be upgraded to a full backup.

Usage

```
s9s backup {command} {options}
```

Commands

Name, shorthand	Description
`--create`	Creates a new backup. This command-line option will initiate a new job that will create a new backup.
`--crea`	Creates a backup schedule, a backup that is repeated. Please note that there are two ways to create a repeated backup. A job that creates a backup can be scheduled and repeated and with this option a backup schedule can be created to repeat the creation of a backup.
`--dele`	Deletes an existing backup.
`--dele`	Initiates a job that checks for expired backups and removes them from the system.
`--dele`	Delete all backups (filtering by some property as snapshot repository name). This command line option will initiate a job that checks for all backups and removes them from the system.db_cluster_id property is used as it could be that no cluster exists but there is data on DB. With the forced option, it will delete a backup record on the DB even when backup deletion failed.
`--dele`	Creates a job to generate a backup only stored in the cloud, as after creating a backup on the storage location filesystem, the copy will be deleted after uploading it to the cloud. This option has to be set to indicate the deletion of the backup local copy after uploading it to the cloud.
`--dele`	Delete the backup schedule specified on the job id.
`--list`	Lists the backups. When listing the backups with the –long option, more detailed columns are listed. See Backup List.
`--list`	Lists the backups in database view format. This format is designed to show the archived databases in the backups.
`--list`	Lists the backups in file view format. This format is designed to show the archive files of the backups.
`--list`	Lists the backup schedules.
`--restore`	Restores an existing backup.
`--rest`	Restores the information the controller has about a cluster from a previously created archive file.
`--rest`	Restores the entire controller from a previously created tarball (created by using the –save-controller option).
`--pitr`	Timestamp specification for doing point-in-time backup recovery.
`--psql`	By default PostgreSQL when started in recovery mode after backup restoration applies the available complete WAL stream as well to provide the most up-to-date state of the database. By this option, we can ask PostgreSQL not to apply WAL or at least as much of it as it is a must for database consistency and to finish recovery as soon as possible.
`--save`	Saves information about one cluster.
`--save`	Saves the entire controller into a file.
`--verify`	Creates a job to verify a backup. When this main option is used the `--backup-id` the option has to be used to identify a backup and the `--test-server` is also necessary to provide a server where the backup will be tested
`--clou`	Creates a job to generate a backup streaming it directly to the cloud, without intermediate files `--cloud-only` option has to be set to indicate direct streaming of backup to the cloud container
`--crea`	Creates a job to create a snapshot repository on elasticsearch cluster. When this main option is used the `--cluster-id` option has to be used to identify the cluster, the `--snapshot-repository-type` defines the allowed type (example: “s3”) `--snapshot-repository` to specify the repository name, `--credential-id` to specify the cloud credentials to use, `--s3-bucket` is also necessary to provide s3 bucket to use for this repository. `--s3-region` is also necessary to provide s3 region to use for this repository.
`--list`	Creates a job to list the snapshot repositories on an elasticsearch cluster. When this main option is used the `--cluster-id` option has to be used to identify the cluster.
`--dele`	Creates a job to delete a snapshot repository on elasticsearch cluster. When this

Options

Name, shorthand	Description
`--backup-directory=DIR`	The directory where the backup is placed.
`--backup-format[=FORMATSTRING]`	The string that controls the format of the printed information about the backups. See *Backup Format*.
`--backup-id=ID`	The ID of the backup.
`--backup-method=METHOD`	Controls what backup software is going to be used to create the backup. The controller currently supports the following methods: ndb, mysqldump, xtrabackupfull, xtrabackupincr, mariabackupfull, mariabackupincr, mongodump, pg_dump, pg_basebackup, mysqlpump.
`--backup-password=PASSWORD`	The password for the SQL account that will create the backup. This command-line option is not mandatory.
`--backup-retention=DAYS`	Controls a custom retention period for the backup, otherwise, the default global setting will be used. Specifying a positive number value here can control how long (in days) the taken backups will be preserved, -1 has a special meaning, it means the backup will be kept forever, while value 0 is the default, means prefer the global setting (configurable on UI).
`--backup-user=USERNAME`	The username for the SQL account that will create the backup.
`--cloud-retention=DAYS`	Retention used when the backup is on a cloud.
`--cluster-id=ID`	The ID of the cluster.
`--compression-level`	Compression level (used threads) to apply on the backup compression process. Value must be between1 and 9.
`--databases=LIST`	A comma-separated list of database names. This argument controls which databases are going to be archived into the backup file. By default, all the databases are going to be archived.
`--data-only`	Backup data only, no structure (PostgreSQL only).
`--encrypt-backup`	When this option is specified ClusterControl will attempt to encrypt the backup files using AES-256 encryption (the key will be auto-generated if not exists yet and stored in a cluster configuration file).
`--exclude-schemas=LIST`	Exclude specific schemas (PostgreSQL only).
`--full-path`	Print the full path of the files.
`--memory=MEGABYTES`	Controls how much memory the archiver process should use while restoring an archive. Currently, only the xtrabackup supports this option.
`--no-compression`	Do not compress the archive file.
`--no-owner`	Skip ownership information (PostgreSQL only).
`--no-privileges`	Skip privilege information (PostgreSQL only).

continues on next page

Table 3.4 – continued from previous page

Name, shorthand	Description
`--nodes=NODELIST`	The list of nodes involved in the backup. See *Node List.*
`--on-node`	Do not copy the created archive file to the controller, store it on the node where it was created.
`--on-controller`	Stream and store the created backup files on the controller.
`--parallelism=N`	Controls how many threads are used while creating a backup. Please note that not all the backup methods support multi-thread operations.
`--pitr-compatible`	Creates PITR-compatible backup.
`--pitr-stop-time`	Timestamp specification for doing point in time backup recovery.
`--recurrence=STRING`	Schedule time and frequency in cron format.
`--safety-copies=N`	Controls how many safety backups should be kept while deleting old backups. This command-line option can be used together with the `--delete-old` option.
`--schemas=LIST`	Include specific schemas (PostgreSQL only).
`--schemas-only`	Backup structure only, no data (PostgreSQL only).
`--subdirectory=MARKUPSTRING`	Sets the name of the subdirectory that holds the newly created backup files. The command-line option argument is considered to be a subpath that may contain the field specifiers using the usual %X format. See *Backup Subdirectory Variables.*
`--temp-dir-path=DIR`	By default, s9s backup creates temporary backup files to `/var/tmp/cmon-%` path. Specify this option with your desired path if you want to target to another location.
`--keep-temp-dir`	Specify this option if you want to retain your archive files from the temporary directory.
`--test-server=HOSTNAME`	Use the given server to verify the backup. If this option is provided while creating a new backup after the backup is created a new job is going to be created to verify the backup. During the verification, the SQL software will be installed on the test server and the backup will be restored on this server. The verification job will be successful if the backup is successfully restored.
`--title=STRING`	A short human-readable string that helps the user to identify the backup later.
`--to-individual-files`	Archive every database into individual files. Currently, only the mysqldump backup method supports this option.
`--use-pigz`	Use the pigz program to compress the archive.

Backup Subdirectory Variables

Variable	Description
B	The date and time when the backup creation was beginning.
H	The name of the backup host, the host that created the backup.
i	The numerical ID of the cluster.
I	The numerical ID of the backup.
J	The numerical ID of the job that created the backup.
M	The backup method (e.g. "mysqldump").
O	The name of the user who initiated the backup job.
S	The name of the storage host, the host that stores the backup files.
%	The percent sign itself. Use two percent signs, `%%` the same way the standard `printf()` function interprets it as a one percent sign.

Backup List

Column	Description
ID	The numerical ID of the backup.
PI	The numerical ID of the parent backup if there is a parent backup for the given entry.
CID	The numerical ID of the cluster to which the backup belongs.
V	The verification status. Here `V` means the backup is verified, and – which means the backup is not verified.
I	The flag showing if the backup is incremental or not. Here F means the backup is a full backup, `I` means the backup is incremental, – means the backup contains no incremental or full backup files (because for example, the backup failed) and B means the backup contains both full and incremental backups files (which is impossible).
STAT	The state of the backup. Here "COMPLETED" means the backup is completed, "FAILED" means the backup has failed, and "RUNNING" means the backup is being created.
OWN	The name of the Cmon user that owns the backup.
HOST NAM	The name of the host where the backup was created.
CRE· ATEI	The date and time when the backup was created.
SIZE	The total size of the created backup files.
TITLE	The name or title of the backup. This is a human-readable string that helps identify the backup.

Backup Format

When the option `--backup-format` is used the specified information will be printed instead of the default columns. The format string uses the `%` character to mark variable fields and flag characters as they are specified in the standard `printf()` C library functions. The `%` specifiers are ended by field name letters to refer to various properties of the backups.

The `%+12I` format string for example has the `+12` flag characters in it with the standard meaning: the field will be 12 characters wide and the `+` or – sign will always be printed with the number. The properties of

the backup are encoded by letters. The`%16H` for example the letter`H` encodes the `hostname`. Standard `\` notation is also available, `\n` for example, encodes a new-line character.

The s9s-tools support the following fields:

Character	Description
B	The date and time when the backup creation was beginning. The format used to print the dates and times can be set using the –date-format.
C	The backup file creation date and time. The format used to print the dates and times can be set using the –date-format.
d	The names of the databases in a comma-separated string-list.
D	The description of the backup. If the c the modifier is used (e.g. `%cD`) the configured description is shown.
e	The word “ENCRYPTED” or “UNENCRYPTED” depending on the encryption status of the backup.
E	The date and time when the backup creation was ended. The format used to print the dates and times can be set using the –date-format.
F	The archive file name.
H	The backup host (the host that created the backup). If the c modifier is used (e.g. %cH) the configured backup host is shown.
I	The numerical ID of the backup.
i	The numerical ID of the cluster to which the backup belongs.
J	The numerical ID of the job that created the backup.
M	The backup method used. If the c modifier is used the configured backup method will be shown.
O	The name of the owner of the backup.
P	The full path of the archive file.
R	The root directory of the backup.
S	The name of the storage host, the host where the backup was stored.
s	The size of the backup file measured in bytes.
t	The title of the backup. They can be added when the backup is created, it helps to identify the backup later.
v	The verification status of the backup. Possible values are “Unverified”, “Verified” and “Failed”.
%	The percent sign itself. Use two percent signs, `%%` the same way the standard `printf()` function interprets it as a one percent sign.

Examples

- Suppose we have a data node on 10.10.10.20 (port 3306) on cluster-id 2, and we want to backup all databases using mysqldump and store the backup on ClusterControl server:

```
s9s backup --create \
        --backup-method=mysqldump \
        --cluster-id=2 \
        --nodes=10.10.10.20:3306 \
        --on-controller \
        --backup-directory=/storage/backups
```

- Create a mongodump backup on 10.0.0.148 for cluster named ‘MongoDB ReplicaSet 5.0’ and store

the backup on the database node:

```
s9s backup --create \
        --backup-method=mongodump \
        --cluster-name='MongoDB ReplicaSet 5.0' \
        --nodes=10.0.0.148 \
        --backup-directory=/storage/backups
```

- Schedule a full backup using MariaDB backup every midnight at 1:10 AM:

```
s9s backup --create \
        --backup-method=mariabackupfull \
        --nodes=10.10.10.19:3306 \
        --cluster-name=MDB101 \
        --backup-dir=/home/vagrant/backups \
        --on-controller \
        --recurrence='10 1 * * *'
```

- Schedule an incremental backup using MariaDB backup every day at 1:30 AM:

```
s9s backup --create \
        --backup-method=mariabackupincr \
        --nodes=10.10.10.19:3306 \
        --cluster-name=MDB101 \
        --backup-dir=/home/vagrant/backups \
        --on-controller \
        --recurrence='30 1 * * *'
```

- Create a pg_dumpall backup on PostgreSQL master server and store the backup on ClusterControl server:

```
s9s backup --create \
        --backup-method=pgdump \
        --nodes=192.168.0.81:5432 \
        --cluster-id=43 \
        --backup-dir=/home/vagrant/backups  \
        --on-controller \
        --log
```

- Create a PostgreSQL partial backup using pgdump:

```
 s9s backup   --create \
              --cluster-id=1 \
              --backup-method=pgdump \
              --databases="app_db" \
              --schemas="public,app_data" \
              --backup-dir=/tmp
```

- Create a PostgreSQL partial backup with schema exclusions using pgdump:

```
 s9s backup   --create \
              --cluster-id=1 \
```

(continues on next page)

(continued from previous page)

```
              --backup-method=pgdump \
              --databases="app_db" \
              --exclude-schemas="temp,audit" \
              --backup-dir=/var/backups
```

- Create a PostgreSQL partial structure-only backup with removed ownership using pgdump:

```
s9s backup   --create \
             --cluster-id=1 \
             --backup-method=pgdump \
             --databases="app_db" \
             --schema-only \
             --no-owner \
             --no-privileges \
             --backup-format=custom \
             --backup-dir=/var/schema-backups
```

- List all backups for cluster ID 2:

```
s9s backup --list \
        --cluster-id=2 \
        --long \
        --human-readable
```

Tip

Omit the`--cluster-id=2` to see the backup records for all clusters.

- Restore backup ID 3 on cluster ID 2:

```
s9s backup --restore \
        --cluster-id=2 \
        --backup-id=3 \
        --wait
```

???+ Note

```
  If the backup is encrypted, it will be automatically decrypted when␣
↪restoring.
```

- Create a job to verify the given backup identified by the backup ID. The job will attempt to install MySQL on the test server using the same settings as for the given cluster, then restore the backup on this test server. The job returns OK only if the backup is successfully restored on the test server:

```
s9s backup --verify \
        --log \
        --backup-id=1 \
        --test-server=192.168.0.55 \
        --cluster-id=1
```

- Delete the local backup copy after uploading it to the cloud for cluster ID 1:

```
s9s backup \
        --create \
        --cluster-id=1 \
        --nodes=192.168.0.209 \
        --delete-after-upload \
        --cloud-provider="aws" \
        --s3-bucket="my-aws-bucket-eu-west-1" \
        --credential-id=2 \
        --backup-method=xtrabackupfull \
        --wait
```

- Delete old backups for cluster ID 1 that are longer than 7 days, but do not delete at least 3 of the latest backups:

```
s9s backup --delete-old \
        --cluster-id=1 \
        --backup-retention=7 \
        --safety-copies=3 \
        --log
```

- Delete a backup schedule with job ID 1:

```
s9s backup \
        --delete-schedules \
        --job-id=1
```

- Perform a point-in-time restoration at a particular date and time for backup ID 10 on cluster ID 1:

```
s9s backup \
        --restore \
        --cluster-id=1 \
        --backup-id=10 \
        --pitr-stop-time="2020-07-14 14:27:04" \
        --log
```

- Perform a PostgreSQL restoration without applying WAL backup ID 10 on cluster ID 1:

```
s9s backup \
        --restore \
        --cluster-id=1 \
        --backup-id=10 \
        --psql-immediate \
        --log
```

- Create a snapshot repository on an Elasticsearch cluster with cluster ID 1:

```
s9s backup \
      --create-snapshot-repository \
      --cluster-id=1 \
      --snapshot-repo-type=s3 \
```

(continues on next page)

(continued from previous page)

```
      --snapshot-repository=mySnapshotRepository \
      --credential-id=1 \
      --s3-bucket=elastic-s3-test \
      --s3-region=eu-west-3 \
      --wait
```

- List a snapshot repository on a elasticsearch cluster with cluster ID 1:

```
s9s backup \
      --list-snapshot-repository \
      --cluster-id=1 \
```

- Delete a snapshot repository on a elasticsearch cluster with cluster ID 1:

```
s9s backup \
      --delete-snapshot-repository \
      --cluster-id=1 \
      --snapshot-repository=mySnapshotRepository \
      --wait
```

- Generate a backup streaming it directly to the cloud with cluster ID 1:

```
s9s backup \
    --create \
    --cluster-id=1 \
    --nodes=192.168.0.209 \
    --cloud-only \
    --cloud-provider="aws" \
    --s3-bucket="my-aws-bucket-eu-west-1" \
    --credential-id=2 \
    --backup-method=xtrabackupfull \
    --wait
```

3.5.4 s9s-cluster

Create, manage, and manipulate clusters.

Usage

```
s9s cluster {command} {options}
```

Commands

Name, shorthand	Description
`--add-node`	Adds a new node (server) to the cluster or to be more precise creates a new job that will eventually add a new node to the cluster. The name (or IP address) of the node should be specified using the `--nodes` command-line option. See *Node List.*
`--available-upgrades`	Shows the available packages to upgrade the cluster with.
`--change-config`	Changes the configuration values for the cluster. The cluster configuration in this context is the Cmon Controller's configuration for the given cluster.
`--check-hosts`	Checks the hosts before installing a cluster.
`--check-pkg-upgrades`	Checks available cluster packages upgrades.
`--collect-logs`	Creates a job that will collect the log files from the nodes of the cluster.
`--create`	Creates a new cluster. When this command-line option is provided the program will contact the controller and register a new job that will eventually create a new cluster.
`--create-account`	Creates a new account to be used on the cluster to access the database(s).
`--create-database`	Creates a database on the cluster.
`--create-report`	When this command-line option is provided a new job will be started that will create a report. After the job is executed the report will be available on the controller. If the –output-dir the command-line option is provided the report will be created in the given directory on the controller host.
`--delete-account`	Deletes an existing account from the cluster.
`--delete-database`	Creates a new job that will delete a database from the cluster.
`--deploy-agents`	Starts a job to deploy agent-based monitoring using Prometheus and exporters to the nodes.
`--deploy-cmonagents`	Starts a job to deploy Cmon Agents (TopQuery Monitoring) to the nodes.
`--disable-recovery`	Creates a new job that will disable the auto-recovery for the cluster (both cluster auto-recovery and node auto-recovery). The job can optionally be used to also register a maintenance period for the cluster.
`--disable-ssl`	Disable SSL connections on the nodes.
`--drop`	Drops cluster from the controller.
`--enable-recovery`	Creates a job that will enable auto-recovery for both the cluster and the nodes in the cluster.
`--enable-ssl`	Enable SSL connections on the nodes.
`--import-config`	Creates a job that will import all the configuration files from the nodes of the cluster.

continues on next page

Table 3.5 – continued from previous page

Name, shorthand	Description
`--import-sql-users`	Imports SQL users to the load balancer. Depending on the actual load balancer, this can be only import or complete update of the user authentication information known by the load balancer. This is only supported by PgBouncer at the moment. Adds a new node (server) to the cluster, or to be more precise creates a new job that will eventually add a new node to the cluster. The load balancer nodes where the users are to be imported shall be specified using the `--nodes` command line option. See *Node List.*
`--list, -L`	Lists the clusters.
`--list-config`	This command-line option can be used to print the configuration values for the cluster. The cluster configuration in this context is the Cmon Controller's configuration for the given cluster.
`--list-databases`	Lists the databases found on the cluster. Please note that if the cluster has a lot of databases, this option might not show some of them. Sampling a huge number of databases would generate a high load and so the controller has an upper limit built into it.
`--ping`	Checks the connection to the controller.
`--preserve-configuration`	When performing a major version upgrade, this option controls whether configuration settings are migrated from the old PostgreSQL instance to the new one. This includes performance, memory, and security parameters. The default value is false, allowing new PostgreSQL versions to use their version-specific default settings unless explicitly overridden.
`--promote-slave`	Promotes a slave node to become a master. This main option will, of course, work only on clusters where it is meaningful, where there are slaves and masters are possible.
`--reconfigure-node`	Reconfigures specified nodes of the cluster.
`--reinstall-node`	Reinstalls software and also reconfigures it on nodes.
`--register`	Registers an existing cluster in the controller. This option is very similar to the –create option, but it, of course, will not install a new cluster; it just registers one.
`--remove-node`	Removes a node from the cluster (creates a new job that will remove the node from the cluster). The name (or IP address) of the node should be specified using the –nodes command line option. See *Node List.*

continues on next page

Table 3.5 – continued from previous page

Name, shorthand	Description
`--rolling-restart`	Restarts the nodes (one node at a time) without stopping the cluster.
`--set-read-only`	Creates a job that, when executed, will set the entire cluster into read-only mode. Please note that not every cluster type supports the read-only mode.
`--setup-audit-logging`	Creates a job that will set up audit logging on the cluster nodes. This option is used to configure audit logging for database clusters, particularly PostgreSQL clusters. Additional audit event data can be specified using the –audit-log-events-data option.
`--start`	Creates a new job to start the cluster.
`--stat`	Prints the details of one or more clusters.
`--stop`	Creates and registers a new job that will stop the cluster when executed.
`--sync`	Synchronize cluster list with UI Frontend.
`--upgrade-cluster`	Upgrades cluster packages while keeping the same major version.
`--upgrade-to-version`	The new newer major version to upgrade to. Without this, only a minor upgrade will be done.
`--upgrade-method`	Strategy for doing a major upgrade. For PostgreSQL, there are two methods supported: copy and link. The default value is the copy method. The copy method will copy all data by doing a backup of the old version and restoring it to the new version. With the link method, the data files won't be copied, instead, hard links will be created to the old version's data files.
`--uninstall-cmonagents`	Starts a job to remove Cmon Agents (TopQuery Monitoring) from the nodes.

Options

Name, shorthand	Description
`--access-check-cmd=NAME`	The command that will be called on hosts to confirm ability to execute commands on the node. The command specified must return 0 on successful execution.
`--add-publication`	Creates a new PostgreSQL publication for logical replication cluster.
`--add-subscription`	Creates a new PostgreSQL subscription for logical replication.

continues on next page

Table 3.6 – continued from previous page

Name, shorthand	Description
--all-pool	Include all cluster pool information when listing clusters. This option is used with the –list operation to retrieve extended cluster pool details from all controllers in pool. This option is mutually exclusive with –cluster-id when using the –list operation.
--audit-log-evenets-data=STR	Set the audit events data for PostgreSQL audit logging. This option is used in conjunction with the –setup-audit-logging main option to specify which database events should be audited. The value is passed as-is to the audit logging configuration.
--backup-id=NUMBER	The ID of a backup to be restored on the newly created cluster.
--batch	Print no messages. If the application created a job print only the job ID number and exit. If the command prints data do not use syntax highlight, headers, or totals, only the pure table to be processed using filters.
--cluster-format=FORMATSTRING	The string controls the format of the printed information about clusters. See *Cluster Format.*
--cluster-id=ID, -i	The ID of the cluster to manipulate.
--cluster-name=NAME, -n	Sets the cluster name. If the operation creates a new cluster, this will be the name of the new cluster.
--cluster-type=TYPENAME	The type of cluster to install. Currently, the following types are supported: galera, mysqlreplication, groupreplication (or group_replication), ndb (or ndbcluster), mongodb (MongoDB ReplicaSet only), and postgresql.
--config-template=FILENAME	Use the specified file as a configuration template to create the configuration file for the new cluster.
--create-local-repository	Create a local software (APT/YUM) repository mirror when installing software packages. Using this command line option, it is possible to deploy clusters and add nodes offline, without a working internet connection.
--datadir=DIRECTORY	The directory on the node(s) that will hold the data. The primary use for this command-line option is to set the data directory path when a cluster is created.
--db-admin=USERNAME	The user name of the database administrator (e.g., 'root').
--db-admin-passwd=PASSWD	The password for the database admin.
--donor=ADDRESS	Currently, this option is used when starting a cluster. It can be used to control which node will be started first and used for the others as donors.
--drop-publication	Drops an existing PostgreSQL publication.
--drop-subscription	Drops an existing PostgreSQL subscription.

continues on next page

Table 3.6 – continued from previous page

Name, shorthand	Description
`--enterprise-token=TOKEN`	The customer's Repo/Download Token for an Enterprise Database.
`--extensions= LIST`	For Postgres, a comma-separated list of PostgreSQL extensions (pgvector and PostGIS).
`--job-tags=LIST`	Tags for the job if a job is created.
`--keep-firewall`	When not specified the CLI will pass the disable firewall option to create cluster and node addition operations. To keep your firewall settings you may pass this option. This option can also be set in the s9s configuration file using the keep_firewall keyword (check s9s.conf(5) for further details).
`--list-publications`	Lists all publications in a PostgreSQL Logical Replication cluster.
`--list-subscriptions`	Lists all subscriptions in a PostgreSQL Logical Replication cluster.
`--local-repository=NAME`	Use a local repository mirror created by ClusterControl for software deployment.
`--long`, `-l`	Print the detailed list.
`--modify-publication`	Modifies an existing PostgreSQL publication.
`--modify-subscription`	Modifies an existing PostgreSQL subscription. The subscription can be enabled or disabled using the –enable and –disable options, respectively.
`--no-header`	Do not print headers for tables.
`--no-install`	Skip the cluster software installation part. Assume all software is installed on the node(s). This command-line option is considered when installing a new cluster or adding a new node to an existing cluster.
`--nodes=NODE_LIST`	List of nodes to work with. See *Node List.*
`--os-user=USERNAME`	The name of the remote user that is used to gain SSH access on the remote nodes. If this command-line option is omitted the name of the local user will be used on the remote host too.
`--os-elevation=NAME`	he method for authorizing superuser access on the node, allowing the execution of commands with elevated privileges. Supported options include sudo (default) for traditional Unix-like systems, doas for a simpler alternative, and pbrun for privilege escalation in specific environments.
`--os-key-file=PATH`	The path of the SSH key to install on a new container to allow the user to log in. This command-line option can be passed when a new container is created, the argument of the option should be the path of the private key stored on the controller. Although the path of the private key file is passed only the public key will be uploaded to the new container.

continues on next page

Table 3.6 – continued from previous page

Name, shorthand	Description
`--output-dir=DIR`	The directory where the files are created. Use in conjunction with–create-report command.
`--percona-client-id=CLIENTID`	The client ID for the Percona Pro repository.
`--percona-pro-token=TOKEN`	The token for the Percona Pro repository.
`--provider-version=VERSION`	The version string of the software to be installed.
`--remote-cluster-id=ID`	The remote cluster-ID is used for cluster creation when cluster-to-cluster replication is to be installed. Please note that not all the cluster types support cluster-to-cluster replication.
`--semi-sync=[true/false]`	Specifies the semi-sync mode for MySQL Replication.
`--use-internal-repos`	Use internal repositories when installing software packages. Using this command-line option it is possible to deploy clusters and add nodes offline, without a working internet connection. The internal repositories have to be set up in advance.
`--vendor=VENDOR`	The name of the software vendor to be installed.
`--wait`	Waits for the specified job to end. While waiting, a progress bar will be shown unless the silent mode is set.
`--with-ssl`	Sets up SSL while creating a new cluster.
`--with-database`	Create a new database for the account when creating a new database user account.
`--without-ssl`	If this option is provided SSL will not be set up while creating a new cluster.
`--with-tags=LIST`	Limit the list of printed clusters by tags. See *Cluster Tagging.*
`--without-tags=LIST`	Limit the list of printed clusters by tags. See *Cluster Tagging.*
`--with-timescaledb`	Install the TimescaleDB option when creating a new cluster. This is currently only supported on PostgreSQL systems.
ACCOUNT, DATABASE & CONFIGURATION MANAGEMENT	
`--account=NAME[:PASSWD][@HOST`	Account to be created on the cluster.
`--db-name=NAME`	The name of the database.
`--opt-group=NAME`	The option group for configuration.
`--opt-name=NAME`	The name of the configuration item.
`--opt-value=VALUE`	The value for the configuration item.
`--with-database`	Create a database for the user too.
CONTAINER & CLOUD	
`--cloud=PROVIDER`	This option can be used when new container(s) are created. The name of the cloud provider where the new container will be created. This command-line option can also be used to filter the list of the containers when used together with one of the `--list` or `--stat` options.

continues on next page

Table 3.6 – continued from previous page

Name, shorthand	Description
`--containers=LIST`	A list of containers to be created and used by the created job. This command-line option can be used to create containers (virtual machines) and install clusters on them or just add them to an existing cluster as nodes. See *s9s container for details.*
`--credential-id=ID`	The cloud credential ID that should be used when creating a new container. This is an optional value, if not provided the controller will find the credential to be used by the cloud name and the chosen region.
`--firewalls=LIST`	List of firewall (security groups) IDs separated by, or; to be used for newly created containers. See *s9s-container for further details.*
`--generate-key`	Create a new SSH key pair when creating new containers. If this command-line option was provided a new SSH key pair will be created and registered for a new user account to provide SSH access to the new container(s). If the command creates more than one container the same one keypair will be registered for all. The username will be the username of the authenticated cmon-user. This can be overruled by the `--os-user` command line option. When the job creates a new cluster the generated keypair will be registered for the cluster and the file path will be saved into the cluster's Cmon configuration file. When adding a node to such a cluster this `--generate-key` option should not be passed, the controller will automatically re-use the previously created key pair.
`--image=NAME`	The name of the image from which the new container will be created. This option is not mandatory, when a new container is created the controller can choose an image if it is needed. To find out what images are supported by the registered container servers please issue the s9s server `--list-images` command.
`--image-os-user=NAME`	The name of the initial OS user is defined in the image for the first login. Use this option to create containers based on custom images.
`--os-password=PASSWORD`	This command-line option can be passed when creating new containers to set the password for the user that will be created on the container. Please note that some virtualization backends might not support passwords, only keys.

continues on next page

Table 3.6 – continued from previous page

Name, shorthand	Description
`--subnet-id=ID`	This option can be used when new containers are created to set the subnet ID for the container. To find out what subnets are supported by the registered container servers please issue the s9s server `--list-subnets` command.
`--template=NAME`	The name of the container template. See *Container Template.*
`--volumes=LIST`	When a new container is created this command-line option can be used to pass a list of volumes that will be created for the container. The list can contain one or more volumes separated by the `;` character. Every volume consists of three properties separated by the `:` character, a volume name, the volume size in gigabytes, and a volume type that is either "HDD" or "SSD". The string vol1:5:hdd;vol2:10:hdd for example defines two hard-disk volumes, one 5GByte, and one 10GByte. For convenience the volume name and the type can be omitted, so that automatically generated volume names are used.
`--vpc-id=ID`	This option can be used when new containers are created to set the VPC ID for the container. To find out what VPCs are supported by the registered container servers please issue the s9s server `--list-subnets --long` command.
LOGICAL REPLICATION	
`--subcluster-id=ID`	The PostgreSQL sub-cluster ID where the publication will be created.
`--subcluster-name=NAME`	The PostgreSQL sub-cluster name where the publication will be created.
`--pub-name=NAME`	The name of the publication to create or manage.
`--new-pub-name=NAME`	The new name for the publication when modifying an existing publication.
`--sub-name=NAME`	The name of the subscription to create or manage.
`--new-sub-name=NAME`	The new name for the subscription when modifying an existing subscription.
`--include-all-tables`	Include all tables in the publication.
`--db-tables=LIST`	Comma-separated list of tables to include in the publication. Not compatible with –include-all-tables.
LOAD BALANCER	
`--admin-password=PASSWORD`	The password for the administrator of load balancers.
`--admin-user=USERNAME`	The username for the administrator of load balancers.

continues on next page

Table 3.6 – continued from previous page

Name, shorthand	Description
`--dont-import-accounts`	If this option is provided the database accounts will not be imported after the load balancer is installed and added to the cluster. The accounts can be imported later, but it is not going to be part of the load balancer installation performed by the controller.
`--haproxy-config-template=FILENAME`	Configuration template for the HAProxy installation.
`--monitor-password=PASSWORD`	The password of the monitoring use of the load balancer.
`--monitor-user=USERNAME`	The username of the monitoring use of the load balancer.
`--maxscale-mysql-user=USERNAME`	The MySQL username of the Maxscale balancer.
`--maxscale-mysql-password=PASSWORD`	The password of the MySQL user of the Maxscale balancer.
SSL	
`--ssl-ca=PATH`	The SSL CA file path on the controller.
`--ssl-cert=PATH`	The SSL certificate file path on the controller.
`--ssl-key=PATH`	The SSL key file path on the controller.
`--ssl-pass=PASSWD`	The password for an existing CA private key when registering a cluster.
`--move-certs-dir=PATH`	The path to the directory where the SSL certificates are stored will be moved on the imported cluster. (Please ommit initial /var/lib/cmon/ca).

Cluster List

Using the `--list` and `--long` command-line options a detailed list of the clusters can be printed. Here is an example of such a list:

```
$ s9s cluster --list --long
ID STATE TYPE OWNER GROUP NAME COMMENT
1 STARTED replication pipas users mysqlrep All nodes are operational.
Total: 1
```

The list contains the following fields:

Field	Description
ID	The cluster ID of the given cluster.
STATE	A short string describing the state of the cluster. Possible values are `MGMD_NO_CONTACT`, `STARTED`, `NOT_STARTED`, `DEGRADED`, `FAILURE`, `SHUTTING_DOWN`, `RECOVERING`, `STARTING`, `UNKNOWN`, `STOPPED`.
TYPE	The type of the cluster. Possible values are `mysqlcluster`, `replication`, `galera`, `group_repl`, `mongodb`, `mysql_single`, `postgresql_single`.
OWNE	The user name of the owner of the cluster.
GROU	The group owner's name.
NAME	The name of the cluster.
COMMENT	A short human-readable description of the current state of the cluster.

Node List

The list of nodes or hosts is enumerated in a special string using a semicolon as a field separator (e.g. 192.168.1.1;192.168.1.2). The strings in the node list are URLs that can have the following protocols:

URI	Description
`mysql://`	The protocol to install and handle MySQL servers.
`ndbd://`	The protocol for MySQL Cluster (NDB) data node servers.
`ndb_mgmd://`	The protocol for MySQL Cluster (NDB) management node servers. The mgmd:// notation is also accepted.
`haproxy://`	Used to create and manipulate HaProxy servers.
`proxysql://`	Use this to install and handle ProxySql servers.
`maxscale://`	The protocol to install and handle MaxScale servers.
`mongos://`	The protocol to install and handle mongo router servers.
`mongocfg://`	The protocol to install and handle mongo config servers
`mongodb://`	The protocol to install and handle mongo data servers.
`pgbackrest:,`	The protocol to install and handle the PgBackRest backup tool.
`pgbouncer://`	The protocol to install and handle PgBouncer servers.
`pbmagent://`	The protocol to install and handle PBMagent (Percona Backup for MongoDB agent) servers.

Cluster Format

The string controls the format of the printed information about clusters. When this command-line option is used, the specified information will be printed instead of the default columns. The format string uses the `%` character to mark variable fields and flag characters as they are specified in the standard `printf()` C library functions. The `%` specifiers are ended by field name letters to refer to various properties of the clusters.

The `%+12I` format string for example has the `+12` flag characters in it with the standard meaning: the field will be 12 characters wide and the + or − the sign will always be printed with the number. The properties

of the message are encoded by letters. The in the `%-5I` for example the letter `I` encodes the "cluster ID" field, so the numerical ID of the cluster will be substituted. Standard `\` a notation is also available, `\n` for example encodes a new-line character.

The s9s-tools support the following fields:

Field	Description
a	The number of active alarms on the cluster.
C	The configuration file for the cluster.
c	The total number of CPU cores in the cluster. Please note that this number may be affected by hyper-threading. When a computer has 2 identical CPUs, with four cores each, and uses 2x hyper-threading it will count as 2x4x2 = 16.
D	The domain name of the controller of the cluster. This is the string one would get if executed the "domain name" command on the controller host.
G	The name of the group owner of the cluster.
H	The hostname of the controller of the cluster. This is the string one would get if executed the "hostname" command on the controller host.
h	The number of the hosts in the cluster including the controller itself.
I	The numerical ID of the cluster.
i	The total number of monitored disk devices (partitions) in the cluster.
k	The total number of disk bytes found on the monitored devices in the cluster. This is a double-precision floating-point number measured in Terabytes. With the `f` modifier (e.g. `%6.2fk`) this will report the free disk space in TeraBytes.
L	The log file of the cluster.
M	A human-readable short message that describes the state of the cluster.
m	The size of the memory of all the hosts in the cluster added together, measured in GBytes. This value is represented by a double-precision floating-point number, so formatting it with precision (e.g. `%6.2m`) is possible. When used with the `f` modifier (e.g.`%6.2fm`) this reports the free memory, the memory that is available for allocation, used for cache, or used for buffers.
N	The name of the cluster.
n	The total number of monitored network interfaces in the cluster.
O	The name of the owner of the cluster.
P	The CDT path of the cluster.
S	The state of the cluster.
T	The type of the cluster
t	The total network traffic (both received and transmitted) measured in MBytes/seconds found in the cluster.
V	The vendor and the version of the main software (e.g. the MySQL server) on the node.
U	The number of physical CPUs on the host.
u	The CPU usage percent found on the cluster.
w	The total swap space found in the cluster measured in GigaBytes. With the f modifier (e.g. `%6.2fk`) this reports the free swap space in GigaBytes.
%	The `%` character itself.

Cluster Tagging

The concept is very similar to the hash-tags used by popular services such as Twitter and Instagram. A cluster can be created with tags, by using `--with-tags` the option:

```
s9s cluster --create \
        --cluster-name="MyMariaDBGaleraCluster" \
        --cluster-type=galera \
        --provider-version="10.5" \
        --vendor=mariadb  \
        --nodes="mysql://10.10.10.10?hostname_internal=2323" \
        --os-user=vagrant \
        --os-key-file=~/.ssh/id_rsa \
        --with-tags="MDB;DC1;PRODUCTION" \
        --log \
        --print-request
```

Multiple values are supported using a semi-colon as a delimiter. The tag values are case-sensitive.

An existing cluster can be tagged with `--add-tag` and `--tag` options under by specifying the CMON tree path, retrievable using the tree command. To retrieve the "tree path", one has to list out the CMON object tree and look for column "NAME", as shown in the following example:

```
$ s9s tree --list --long
MODE        SIZE OWNER   GROUP   NAME
crwxrwx---  -    system  admins  MariaDB Replication 10.3
srwxrwxrwx  -    system  admins  localhost
drwxrwxr--  1, 0 system  admins  groups
urwxr--r--  -    admin   admins  admin
urwxr--r--  -    nobody  admins  nobody
urwxr--r--  -    system  admins  system
```

Then, specify the tree path as `/` + tree name ("MariaDB Replication 10.4"), as shown in the following:

```
s9s tree --add-tag --tag="REPLICATION;PRODUCTION" "/MariaDB Replication␣
↪10.4"
```

To show all clusters having a certain tag:

```
s9s cluster --list --long --with-tags="PRODUCTION"
```

Show all cluster that does not have a certain tag:

```
s9s cluster --list --long --without-tags="PRODUCTION"
```

To filter using multiple tag values:

```
s9s cluster --list --long --with-tags="PRODUCTION;DEV"
```

Examples

- Create a three-node Percona XtraDB Cluster 5.7 cluster, with OS user vagrant:

```
s9s cluster --create \
        --cluster-type=galera \
        --nodes="10.10.10.10;10.10.10.11;10.10.10.12" \
        --vendor=percona \
```

(continues on next page)

(continued from previous page)

```
        --provider-version=8.0 \
        --db-admin-passwd='pa$word' \
        --os-user=vagrant \
        --os-key-file=/home/vagrant/.ssh/id_rsa \
        --cluster-name='Percona XtraDB Cluster 5.7'
```

- Create a three-node MongoDB Replica Set 5.0 by MongoDB Inc (formerly 10gen) and use the default `/root/.ssh/id_rsa` as the SSH key, and let the deployment job run in the foreground:

```
s9s cluster --create \
        --cluster-type=mongodb \
        --nodes="10.0.0.148;10.0.0.189;10.0.0.219" \
        --vendor=10gen \
        --provider-version='5.0' \
        --os-user=root \
        --db-admin='admin' \
        --db-admin-passwd='MyS3cr3tPass' \
        --cluster-name='MongoDB ReplicaSet 5.0' \
        --wait
```

- Create a MongoDB Sharded Cluster with 3 mongos, 3 mongo config, and one shard consists of a three-node replica set called 'replset2':

```
s9s cluster --create \
        --cluster-type=mongodb \
        --vendor=10gen \
        --provider-version=5.0 \
        --db-admin=adminuser \
        --db-admin-passwd=adminpwd \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --nodes=
↪"mongos://192.168.1.11;mongos://192.168.1.12;mongos://192.168.1.12;mongocfg://192.
```

- Import and existing Percona XtraDB Cluster 8.0 and let the deployment job running in the foreground (provided passwordless SSH from ClusterControl node to all database nodes have been set up correctly):

```
s9s cluster --register \
        --cluster-type=galera \
        --nodes="192.168.100.34;192.168.100.35;192.168.100.36" \
        --vendor=percona \
        --provider-version=8.0 \
        --db-admin="root" \
        --db-admin-passwd="root123" \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --cluster-name="My DB Cluster" \
        --wait
```

- Create a MySQL 8.0 replication cluster by Oracle with multiple master and slaves (note the `?` sign

to identify the node's role in the `--nodes` parameter):

```
s9s cluster --create \
        --cluster-type=mysqlreplication \
        --nodes=
↪"192.168.1.117?master;192.168.1.113?slave;192.168.1.115?slave;192.168.1.116?master
↪\
        --vendor=oracle \
        --db-admin="root" \
        --db-admin-passwd="root123" \
        --cluster-name=ft_replication_23986 \
        --provider-version=8.0 \
        --log
```

- Create a Redis Cluster v7 with 6 Redis nodes:

```
s9s cluster --create \
        --cluster-type=redis-sharded \
        --redis-port=6379
        --redis-bus-port=16579 \
        --vendor=redis \
        --provider-version=7 \
        --node-timeout-ms=5000 \
        --replica-validity-factor=10 \
        --db-admin=s9s-user \
        --db-admin-pass=s9ss9s \
        --nodes=
↪"redis-primary://rlc2:6479;redis-replica://rlc3:6479;redis-primary://rlc4:6479;red
↪\
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --cluster-name="My Redis Cluster v7" \
        --log \
        --print-request
```

- Create a Redis Sentinel v6 with one master and two slaves:

```
s9s cluster --create \
        --cluster-type=redis \
        --nodes=
↪"redis://rlc2;redis://rlc3;redis://rlc4;redis-sentinel://rlc2;redis-sentinel://rlc
↪\
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --vendor=redis \
        --provider-version=6  \
        --log \
        --print-request \
        --cluster-name="My Redis Sentinel v6"
```

- Import and existing Redis Cluster and let the deployment job run in the foreground (provided passwordless SSH from ClusterControl node to all database nodes have been set up correctly):

```
s9s cluster --register \
        --cluster-type=redis-sharded \
        --redis-port=6379 \
        --db-admin=s9s-user \
        --db-admin-passwd=s9ss9s \
        --os-user=root \
        --os-key-file=/root/.ssh/id_rsa \
        --nodes="redis-primary://rlc3:6479" \
        --vendor=redis \
        --log \
        --cluster-name="My Redis Cluster" \
        --print-request \
        --wait
```

- Create a PostgreSQL 12 streaming replication cluster with one master and two slaves (note the `?` sign to identify the node's role in the `--nodes` parameter):

```
s9s cluster --create \
        --cluster-type=postgresql \
        --nodes=
↪"192.168.1.81?master;192.168.1.82?slave;192.168.1.83?slave;" \
        --db-admin="postgres" \
        --db-admin-passwd="mySuperStongP455w0rd" \
        --cluster-name=ft_replication_23986 \
        --os-user=vagrant \
        --os-key-file=/home/vagrant/.ssh/id_rsa \
        --provider-version=12 \
        --log
```

- Create a new PostgreSQL publication for logical replication cluster on Cluster ID 1:

```
s9s cluster   --add-publication \
              --cluster-id=1 \
              --subcluster-id=2 \
              --pub-name=my_publication \
              --db-name=mydb \
              --db-tables="orders,customers,products"
```

- Creates a new PostgreSQL subscription for logical replication on Cluster ID 1:

```
s9s cluster   --add-subscription \
              --cluster-id=1 \
              --subcluster-id=3 \
              --sub-name=my_subscription \
              --sub-db-name=dest_db \
              --pub-name=source_pub \
              --pub-db-name=source_db \
              --origin=none \
              --copy-data=false
```

- Modify an existing PostgreSQL publication for logical replication on Cluster ID 1:

```
s9s cluster   --modify-publication \
              --cluster-id=1 \
              --subcluster-name=source_db \
              --pub-name=sales_pub \
              --db-name=sales_db \
              --include-all-tables
```

- Modify an existing PostgreSQL subscription for logical replication on Cluster ID 1:

```
s9s cluster   --modify-subscription \
              --cluster-id=1 \
              --subcluster-id=3 \
              --sub-name=old_sub_name \
              --new-sub-name=new_sub_name \
              --pub-name=new_pub_name \
              --db-name=mydb \
              --enable
```

- Drop an existing PostgreSQL publication for logical replication on Cluster ID 1:

```
s9s cluster   --drop-publication \
              --cluster-id=1 \
              --subcluster-id=2 \
              --pub-name=my_publication \
              --db-name=mydb
```

- Drop an existing PostgreSQL subscription for logical replication on Cluster ID 1:

```
s9s cluster   --drop-subscription \
              --cluster-id=1 \
              --subcluster-id=3 \
              --sub-name=my_subscription \
              --db-name=mydb
```

- List all clusters with more details:

```
s9s cluster --list --long
```

- Delete a cluster with cluster ID 1:

```
s9s cluster --delete --cluster-id=1
```

- Add a new database node on Cluster ID 1:

```
s9s cluster --add-node \
        --nodes=10.10.10.14 \
        --cluster-id=1 \
        --wait
```

- Delete a database from the cluster name galera_001.

```
s9s cluster \
        --delete-database \
        --print-request \
        --cluster-name="galera_001" \
        --db-name="my_database" \
        --log
```

- Add a data node to an existing MongoDB Sharded Cluster with cluster ID 12 having replica set name 'replset2':

```
s9s cluster --add-node \
        --cluster-id=12 \
        --nodes="mongodb://192.168.1.20?rs=replset2"
```

- Create an HAProxy load balancer, 192.168.55.198 on cluster ID 1:

```
s9s cluster --add-node \
        --cluster-id=1 \
        --nodes="haproxy://192.168.55.198" \
        --wait
```

- Remove a database node from cluster ID 1 as a background job:

```
s9s cluster --remove-node \
        --nodes=10.10.10.13 \
        --cluster-id=1
```

- Check if the hosts are part of other cluster and accessible from ClusterControl:

```
s9s cluster --check-hosts \
        --nodes="10.0.0.148;10.0.0.189;10.0.0.219"
```

- Schedule a rolling restart of the cluster 20 minutes from now:

```
s9s cluster --rolling-restart \
        --cluster-id=1 \
        --schedule="$(date -d 'now + 20 min')"
```

- Create a database on the cluster with the given name:

```
s9s cluster --create-database \
        --cluster-id=2 \
        --db-name=my_shopping_db
```

- Create a database account on the cluster and also create a new database to be used by the new user. Grant all access to the new database for the new user:

```
s9s cluster --create-account \
        --cluster-id=1 \
        --account="john:myPassword135" \
        --with-database
```

- Create a cluster and tag it using the `--with-tags` option:

```
s9s cluster --create \
        --cluster-name="TAGGED_MDB" \
        --cluster-type=galera \
        --provider-version="10.4" \
        --vendor=mariadb \
        --nodes="mysql://10.10.10.14:3306" \
        --os-user=vagrant \
        --with-tags="MDB;DC1;PRODUCTION" \
        --log
```

- List all databases for every cluster:

```
$ s9s cluster --list-databases --long
SIZE       #TBL #ROWS   OWNER  GROUP  CLUSTER              DATABASE
         0    0       - system admins MySQL Rep Oracle 8.0 sys
 381681664    7  690984 system admins MySQL Rep Oracle 8.0 db5
 381681664    7  690984 system admins MySQL Rep Oracle 8.0 db4
 599785472   11 1083295 system admins MySQL Rep Oracle 8.0 db3
 272629760    5  493560 system admins MySQL Rep Oracle 8.0 db2
 381681664    7  690984 system admins MySQL Rep Oracle 8.0 db1
   7340032    2       0 system admins PostgreSQL 13        postgres
   7340032    0       0 system admins PostgreSQL 13        template1
   7340032    0       0 system admins PostgreSQL 13        template0
   7340032    0       0 system admins PostgreSQL 13        db1
   7340032    0       0 system admins PostgreSQL 13        db2
Total: 11 databases, 2059403264, 39 tables.
```

- Show all clusters having a certain tag (an existing cluster can be tagged using s9s tree), with multiple tags separated by a semi-colon:

```
s9s cluster --list \
        --long \
        --with-tags="PRODUCTION;big_cluster"
```

- List out CMON configuration options for cluster ID 2:

```
s9s cluster --list-config --cluster-id=2
```

- Change a CMON configuration option called `diskspace_warning` for cluster ID 2 (the configuration change will be applied on CMON runtime and configuration file, `/etc/cmon.d/cmon_2.cnf`):

```
s9s cluster --change-config \
        --cluster-id=2 \
        --opt-name=diskspace_warning \
        --opt-value=70
```

- Upgrade cluster packages while keeping the major version for cluster ID 1.

```
s9s cluster \
        --upgrade-cluster \
```

(continues on next page)

(continued from previous page)

```
        --cluster-id=1 \
        --nodes="192.168.0.84:5433;192.168.0.85" \
        --wait
```

- Upgrade cluster packages to a newer major version for cluster ID 1.

```
s9s cluster \
        --upgrade-cluster \
        --upgrade-to-version=12 \
        --cluster-id=1 \
        -- log
```

- Upgrade cluster packages to a newer major version for cluster ID 1 with a specific upgrade method.

```
s9s cluster \
        --upgrade-cluster \
        --upgrade-to-version=12 \
        --upgrade-method=link \
        --cluster-id=1 \
        -- log
```

- Preserve configurations when performing a major version upgrade for cluster ID 1:

```
s9s cluster \
             --upgrade-cluster \
             --upgrade-to-version=15 \
             --upgrade-method=link \
             --preserve-configuration \
             --cluster-id=1 \
             --log
```

- Set up audit logging for cluster ID 1 with a specific log events:

```
s9s cluster \
             --setup-audit-logging \
             --audit-log-events-data="SELECT,INSERT,UPDATE,DELETE" \
             --cluster-id=1 \
             --wait
```

- Deploys CMON agents to all nodes in cluster ID 1. (This enables top query monitoring functionality for the cluster nodes.)

```
s9s cluster \
        --deploy-cmonagents \
        --cluster-id=1 \
        --print-request \
        --log \
        --wait
```

- Uninstalls CMON agents from specified nodes in cluster ID 1. (If no node is specified, CMON agents will be uninstalled from all nodes in the cluster.)

```
s9s cluster \
        --uninstall-cmonagents \
        --cluster-id=1 \
        --nodes=10.67.199.164 \
        --print-request \
        --log \
        --wait
```

3.5.5 s9s-container

Manage cloud and container virtualization. Multiple technologies (multiple virtualization backends) are supported (e.g. Linux LXC and AWS) providing various levels of virtualization. Throughout this documentation (and in fact in the command line options) s9s uses the word "container" to identify virtualized servers. The actual virtualization backend might use the term "virtual machine" or "Linux container" but s9s provides a high-level generic interface to interact with them, so the generic "container" term is used. So please note, the term "container" does not necessarily mean "Linux container", it means "a server that is running in some kind of virtualized environment".

In order to utilize the s9s command-line tool and the CMON Controller to manage virtualization, a virtualization host (container server) has to be installed first. The installation of such a container environment is documented in the s9s server.

Usage

```
s9s container {command} {options}
```

Commands

Name, shorthand	Description
--create	Creates and starts a new container or virtual machine. If this option is provided, the controller will create a new job that creates a container. By default the container will also be started, an account will be created, passwordless sudo granted and the controller will wait for the controller to obtain an IP address.
--del	Stop and delete the container or virtual machine.
--lis	Lists the containers. See *Container List*.
--sta	Starts an existing container.
--sta	Prints the details of a container.
--sto	Stops the container. This will not remove the container by default, but it will stop it. If the container is set to be deleted on stop (temporary) it will be deleted.

Options

Name, shorthand	Description
`--log`	Waits until the job is executed. While waiting the job logs will be shown unless the silent mode is set.
`--recurrence=`	This option can be used to create recurring jobs, jobs that are repeated over and over again until they are manually deleted. Every time the job is repeated a new job will be instantiated by copying the original recurring job and starting the copy. The options argument is a crontab style string defining the recurrence of the job. See *Crontab*.
`--schedule=DA`	The job will not be executed now but it is scheduled to execute later. The DateTime string is sent to the backend, so all the formats are supported by the controller.
`--timeout=SE`	Sets the timeout for the created job. If the execution of the job is not done before the timeout counted from the start time of the job expires the job will fail. Some jobs might not support the timeout feature, the controller might ignore this value.
`--wait`	Waits until the job is executed. While waiting a progress bar will be shown unless the silent mode is set.
`--clou`	This option can be used when a new container(s) created. The name of the cloud provider where the new container will be created. This command-line option can also be used to filter the list of the containers when used together with one of the `--list` or `--stat` options.
`--cont`	The string controls the format of the printed information about the containers. See *Container Format*.
`--containers=LI`	A list of containers to be created or managed. The containers can be passed as command-line options (suitable for simple commands) or as an optional argument for this command-line option. The `s9s container --stop node01` and the `s9s container --stop --containers=node01` commands for example are equivalent. See *Create Container List*.
`--crec`	The cloud credential ID that should be used when creating a new container. This is an optional value, if not provided the controller will find the credential to be used by the cloud name and the chosen region.
`--firewalls=`	List of firewall (AKA security groups) IDs separated by `,` or `;` to be used for newly created containers. This is not a mandatory option, if the virtualization server needs a firewall to be set one such a firewall will be automatically created. Containers created in the same job (for example in a create cluster operation) the containers will share the same firewall, so they will be able to communicate.If the container is created so that it will be added to an existing cluster (e.g. in an add node job) the controller will try to find the firewall of the existing nodes and if it exists will re-use the same ID, so that the nodes can reach each other.
`--gene`	Create a new SSH keypair when creating new containers. If this command line option was provided a new SSH keypair will be created and registered for a new user account to provide SSH access to the new container(s). If the command creates more than one containers the same one keypair will be registered for all. This command line option is actually useful for the cases when a new cluster is created together with the new containers.
`--image=NA`	The name of the image from which the new container will be created. This option is not mandatory, when a new container is created the controller can choose an image if it is needed. To find out what images are supported by the registered container servers please issue the `s9s server --list-images` command.
`--imag`	The name of the initial OS user defined in the image for the first login. Use this option to create containers based on custom images.
`--os-k`	The path of the SSH key to install on a new container to allow the user to log in. This command-line option can be passed when a new container is created, the argument of the option should be the path of the private key stored on the controller. Although the path of the private key file is passed only the public key will be uploaded to the new container.
`--os-p`	This command-line option can be passed when creating new containers to set the password for the user that will be created on the container. Please note that some virtualization backend might not support passwords, only keys.
`--os-u`	This option may be used when creating new containers to pass the name of the user that will

Container Format

The string controls the format of the printed information about the containers. When this command line option is used the specified information will be printed instead of the default columns. The format string uses the `%` character to mark variable fields and flag characters as they are specified in the standard `printf()` C library functions. The `%` specifiers are ended by field name letters to refer to various properties of the containers.

The `%+12i` format string for example has the `+12` flag characters in it with the standard meaning: the field will be 12 characters wide and the + or – sign will always be printed with the number. The properties of the container are encoded by letters. The in the `%16D` for example the letter `D` encodes the `data directory` field, so the full path of the data directory on the container will be substituted. Standard `\` notation is also available, `\n` for example, encodes a new-line character.

The s9s-tools support the following fields:

Field	Description
A	The IP address of the container. This is by default the public IPv4 address of the container. Containers being deleted/created might not have any IP addresses, then the – string is substituted.
a	The private IP address of the container if there is any or the – string.
C	The full path of the configuration file that stores the container settings if such a configuration file exists.
c	The cloud (sometimes mentioned as 'provider') of the container, for example, AWS or AZ as it is set in the credentials file `/var/lib/cmon/cloud_credentials.json`.
F	The name of the first firewall (security group) if the container has such a property set, the string – otherwise.
G	The name of the group owner of the node.
I	The ID of the container.
i	The name of the image that was used to create the container.
N	The name (alias) of the container.
O	The username of the owner of the container.
S	The state of the container as a string.
P	The CDT path of the user.
p	The name of the parent server, the container server that manages the container.
R	The name of the region in which the container is hosted.
r	The address range of the subnet the container belongs to in CIDR notation (e.g.`10.0.0.0/24`)
T	The type of container (e.g. `cmon-cloud` or `lxc`).
t	The name of the template that was used to create a container or the – string if no such template was used.
U	The ID of the subnet of the container.
V	The ID of the VPC for the container.
z	The class name of the container object.
%	The `%` character itself.

Container List

Using the `--list` and `--long` command-line options a detailed list of the containers can be printed. Here is an example of such a list:

```
$ s9s container --list --long
S TYPE TEMPLATE OWNER GROUP     NAME                  IP ADDRESS    SERVER
```

(continues on next page)

(continued from previous page)

```
- lxc  -        pipas testgroup bestw_controller     -             core1
u lxc  -        pipas testgroup dns1                 192.168.0.2   core1
u lxc  ubuntu   pipas testgroup ft_containers_35698  192.168.0.228 core1
u lxc  -        pipas testgroup mqtt                 192.168.0.5   core1
- lxc  -        pipas testgroup ubuntu               -             core1
u lxc  -        pipas testgroup www                  192.168.0.19  core1
Total: 6 containers, 4 running.
```

The list contains the following fields:

Field	Description
S	The abbreviated status information. This is `u` for a container that is up and running and – otherwise.
TYPE	Shows what kind of container or virtual machine shown in this line, the type of software that provides virtualization.
TEMPLATE	The name of the template that is used to create the container.
OWNER	The owner of the server object.
GROUP	The group owner of the server object.
NAME	The name of the container. This is not necessarily the hostname, this is a unique name to identify the container on the host.
IP ADDRESS	The IP address of the container or the – character if the container has no IP address.
SERVER	The server on which the container can be found.

Create Container List

The command-line option argument is one or more containers separated by the `;` character. Each container is an URL defining the container name (an alias for the container) and zero or more properties. The string `container05parent_server=core1;container06?parent_server=core2`, for example, defines two containers one on one server and the other on the other server.

To see what properties are supported in the controller for the containers, one may use the following command:

```
$ s9s metatype --list-properties --type=CmonContainer --long
ST NAME         UNIT DESCRIPTION
r- acl          -    The access control list.
r- alias        -    The name of the container.
r- architecture -    The processor architecture.
```

See Property List for details.

Container Template

Defining a template is an easy way to set a number of complex properties without actually enumerating them in the command line one by one. The actual interpretation of the template name is up to the virtualization backend that is the protocol of the container server. The lxc backend for example considers the template to be an already created container, it simply creates the new container by copying the template container so the new container inherits everything.

The template name can also be provided as a property name for the container, so the command `s9s container --create --containers="node02?template=ubuntu;node03"--log` for example will create two containers, one using a template, the other using the default settings.

Note that the `--template` the command-line option is not mandatory, if emitted suitable default values will be chosen, but if the template is provided and the template is not found the creation of the new container will fail.

Volume List

The list can contain one or more volumes separated by the `;` character. Every volume consists of three properties separated by the `:` character, a volume name, the volume size in gigabytes, and a volume type that is either "HDD" or "SSD". The string `vol1:5:hdd;vol2:10:hdd` for example defines two hard-disk volumes, one 5GByte, and one 10GByte.

For convenience, the volume name and the type can be omitted, so that automatically generated volume names are used.

Examples

- Create a container with no special information needed, every setting will use the default values. For this, of course, at least one container server has to be pre-registered and properly working:

```
s9s container --create --wait
```

- Using the default, automatically chosen container names might not be the easiest way, so here is an example that provides a container name:

```
s9s container --create --wait node01
```

- This is equivalent to the following example that provides the container name through a command-line option:

```
s9s container --create --wait --containers="node01"
```

3.5.6 s9s-controller

View and handle controller (CMON instance), allows building a highly available cluster of CMON instances to achieve ClusterControl's high availability.

???+ Note

```
CMON HA feature is still a beta feature.
```

This command can help setup CMON with high availability feature using the following simple steps:

1. Install a CMON Controller together with the CMON Database serving as permanent storage for the controller. The CMON HA will not replicate the CMON Database, so it has to be accessible from all the controllers and if necessary it has to provide redundancy on itself.
2. Enable the CMON HA subsystem using the `--enable-cmon-ha` on the running controller. This will create one CmonController class object. Check the object using the `--list` or `--stat` option. The CMON HA is now enabled, but there is no redundancy, only one controller is running. The one existing controller in this stage should be a leader although there are no followers.

3. Install additional Cmon Controllers one by one and start them the usual way. The next controllers should use the same CMON Database and should have the same configuration files. When the additional controllers are started they will find the leader in the CMON Database and will ask the leader to let them join. When the join is successful one more CmonController will be created for every joining controller.

Usage

```
s9s controller {command} {options}
```

Commands

Name short-hand	Description
`--cre`	Creates a job that will create a controller to controller snapshot of the Cmon HA subsystem. Creating a snapshot manually using this command-line option is not necessary for the Cmon HA to operate, this command-line option is made for testing and repairing.
`--ena`	Enabled the CMON HA subsystem. By default, Cmon HA is not enabled for compatibility reasons, so this command-line option is implemented to enable the controller to controller communication. When the CMON HA is enabled CmonController class objects will be created and used to implement the high availability features (e.g. the leader election). So if the controller at least one CmonController object, the CMON HA is enabled, if not, it is not enabled.
`--get`	Gets the LDAP configuration from the controller and prints it to the standard output.
`--lis`	Lists the CmonController type objects known to the controller. If the CMON HA is not enabled there will be no such objects, if it is enabled one or more controllers will be listed. With the `--long` option a more detailed list will be shown where the state of the controllers can be checked.
`--pin`	Sends a ping request to the controller and prints the information received. Please note that there is another ping request for the clusters, but this ping request is quite different from that. This request does not need a cluster, it is never redirected (follower controllers will also reply to this request) and it is replied with some basic information about the Cmon HA subsystem.
`--sta`	Prints more details about the controller objects.

Examples

- Create a controller to controller snapshot of CMON HA subsystem:

```
s9s controller \
    --create-snapshot \
    --log
```

- Enable CMON HA feature for the current CMON instance:

```
s9s controller --enable-cmon-ha
```

- The command line option itself is only a syntactic sugar to save a “true” boolean value into a CDT entry, so for example the following command can also be used:

```
echo "true" | \
s9s tree \
--save \
--cmon-user=system \
--password=secret \
--controller="https://10.10.1.23:9501" \
--batch \
/.runtime/cmon_ha/enabled
```

- Print out LDAP configuration from the controller:

```
s9s controller \
        --cmon-user="system" \
        --password="XXXXXXXX" \
        --get-ldap-config \
        --print-json
```

- List out all controllers participate in the CMON HA cluster:

```
s9s controller \
        --list \
        --long
```

- Send ping request to the controller and print the output in JSON format, with some text filtering on the output:

```
s9s controller \
        --ping \
        --print-json \
        --json-format='status: ${controller_status}\n'
```

- Print more details about the controller objects:

```
s9s controller --stat
```

3.5.7 s9s-job

View and manage jobs.

Usage

```
s9s job {command} {options}
```

Commands

Name, shorthand	Description
`--clo`	Creates a copy of the job to re-run it. The clone will have all the properties the original job had and will be executed the same way as new jobs are executed. If the `--cluster-id` command-line option is used the given cluster will execute the job, if not, the clone will have the same cluster the original job had.
`--del`	Deletes the job referenced by the job ID.
`--dis able`	This command line option can be used to disable (pause) a scheduled or recurring job instance.
`--en- able`	This command line option can be used to enable a disabled (scheduled or recurring) job instance.
`--fai`	Creates a job that does nothing and fails.
`--kil`	This command-line option can be used to send a signal to a running Cmon Job in order to abort its execution. The job subsystem is not preemptive in the controller, so the job only will be actually aborted if and when the job supports aborting what it is doing.
`--lis`	Lists the jobs. See *Job List*.
`--log`	Prints the job messages of the specified job.
`--suc cess`	Creates a job that does nothing and succeeds.
`--wai`	Waits for the specified job to end. While waiting, a progress bar will be shown unless the silent mode is set.

Options

Name, shorthand	Description
NEWLY CREATED JOB	
`--f/ --follow`	Combination of `--log` and `--wait`, s9s is going to attach to an existing running job and print out its job messages while it is running.
`--job-tags`	List one of more strings separated by either `,` or `;` to be added as tags to a newly created job if a job is indeed created.
`--log`	Waits for the specified job to end. While waiting, the job logs will be shown unless the silent mode is set.
`--recurrence=CRON`	Creates recurring jobs, jobs that are repeated over and over again until they are manually deleted. See *Crontab*.
`--schedule=DATETI`	The job will not be executed now but it is scheduled to execute later. The `date-time` string is sent to the backend, so all the formats are supported that is supported by the controller.
`--timeout=SECOND`	Sets the timeout for the created job. If the execution of the job is not done before the timeout counted from the start time of the job expires the job will fail. Some jobs might not support the timeout feature, and the controller might ignore this value.
`--wait`	Waits for the specified job to end. While waiting, a progress bar will be shown unless the silent mode is set.
JOB RELATED OPTIONS	
`--job-id=I`	The job ID of the job to handle or view.
`--from=DAT`	Controls the start time of the period that will be printed in the job list.
`--limit=NU`	Limits the number of jobs printed.
`--offset=NUMBER`	Controls the relative index of the first item printed.
`--show-abo`	Turn on the job state filtering and show jobs that are in an aborted state. This command-line option can be used while printing job lists together with the other `--show-*` options.
`--show-def`	Turn on the job state filtering and show jobs that are in a defined state. This command-line option can be used while printing job lists together with the other `--show-*` options.
`--show-fai`	Turn on the job state filtering and show jobs that are failed. This command-line option can be used while printing job lists together with the other `--show-*` options.
`--show-fin`	Turn on the job state filtering and show jobs that are finished. This command-line option can be used while printing job lists together with the other `--show-*` options.
`--show-run`	Turn on the job state filtering and show jobs that are running. This command-line option can be used while printing job lists together with the other `--show-*` options.
`--show-sch`	Turn on the job state filtering and show jobs that are scheduled. This command-line option can be used while printing job lists together with the other `--show-*` options.
`--until=DATE&T`	Controls the end time of the period that will be printed in the job list.
`--log-form`	The string that controls the format of the printed log and job messages. See *Log Format Variables*.
`--with-tag`	List of one of more strings separated by either `,` or `;` to be used as a filter when printing information about jobs. When this command-line option is provided only the jobs that have any of the tags will be printed.
`--without-`	List of one of more strings separated by either `,` or `;` to be used as a filter when printing information about jobs. When this command-line option is provided the jobs that have any of the tags will not be printed.

Crontab

Every time the job is repeated a new job will be instantiated by copying the original recurring job and starting the copy. The option argument is a crontab-style string defining the recurrence of the job.

The crontab string must have exactly five space-separated fields as follows:

Field	Value
minute	0 – 59
hour	0 – 23
day of the month	1 – 31
month	1 – 12
day of the week	0 – 7

All the fields may be a simple expression or a list of simple expressions separated by a comma (`,`). So to clarify the fields are separated by space can contain subfields separated by a comma.

The simple expression is either a star (`*`) representing "all the possible values", an integer number representing the given minute, hour, day or month (e.g. 5 for the fifth day of the month), or two numbers separated by a dash representing an interval (e.g. 8-16 representing every hour from 8 to 16). The simple expression can also define a "step" value, so, for example, `*/2` might stand for "every other hour" or `8-16/2` might stand for "every other hour between 8 and 16 or `*/2` might say "every other hour".

Please check the crontab man page for more details.

Job List

Using the `--list` command-line option a detailed list of jobs can be printed (the `--long` option results in even more details). Here is an example of such a list:

```
$ s9s job --list
ID CID STATE    OWNER  GROUP  CREATED             RDY  TITLE
1  0   FINISHED pipas  users  2017-04-25 14:12:31 100% Create MySQL Cluster
2  1   FINISHED system admins            03:00:15 100% Removing Old Backups
Total: 2
```

The list contains the following fields:

Field	Description
ID	The numerical ID of the job. The `--job-id` command-line option can be used to pass such ID numbers.
CID	The cluster ID. Most of the jobs are related to one specific cluster so those have a cluster ID in this field. Some of the jobs are not related to any cluster, so they are shown with cluster ID 0.
STATE	The state of the job. The possible values are `DEFINED`, `DEQUEUED`, `RUNNING`, `SCHEDULED`, `ABORTED`, `FINISHED` and `FAILED`.
OWNE	The user name of the user who owns the job.
GROU	The name of the group owner.
CREATED	The date and time show when the job was created. The format of this timestamp can be set using the `--date-format` command-line option.
RDY	A progress indicator showing how many percent of the job was done. Please note that some jobs have no estimation available and so this value remains 0% for the entire execution time.
TITLE	A short, human-readable description of the job.

Log Format Variables

The format string uses the `%` character to mark variable fields, flag characters as they are specified in the standard `printf()` C library functions and its own field name letters to refer to the various properties of the messages.

The `%+12I` format string for example has the `+12` flag characters in it with the standard meaning: the field will be 12 characters wide and the + or – sign will always be printed with the number. Standard `\` notation is also available, \n for example encodes a new-line character.

The properties of the message are encoded by letters. The in the `%-5L` for example the letter `L` encodes the "line-number" field, so the number of the source line that produced the message will be substituted. The program supports the following fields:

Variable	Description
B	The basename of the source file that produced the message.
C	The creation date and time that mark the exact moment when the message was created. The format of the date&time substituted can be set using the `--date-format` command-line option.
F	The name of the source file that created the message. This is similar to the `B` fields, but instead of the base name, the entire file name will be substituted.
I	The ID of the message, a numerical ID that can be used as a unique identifier for the message.
J	The Job ID.
L	The line number in the source file where the message was created. This property is implemented mostly for debugging purposes.
M	The message text.
S	The severity of the message in text format. This field can be "MESSAGE", "WARNING" or "FAILURE".
T	The creation time of the message. This is similar to the C field but shows only hours, minutes and seconds instead of the full date and time.
%	The `%` character itself.

Examples

- List jobs:

```
$ s9s job --list
10235 RUNNING dba 2017-01-09 10:10:17 2% Create Galera Cluster
10233 FAILED  dba 2017-01-09 10:09:41 0% Create Galera Cluster
```

- The s9s client will send a job that will be executed in the background by cmon. It will print out the job ID, for example: "Job with ID 57 registered." It is then possible to attach to the job to find out the progress:

```
s9s job --wait --job-id=57
```

- View job log messages of job ID 10235:

```
s9s job --log --job-id=10235
```

- Delete the job that has the job ID 42:

```
s9s job --delete --job-id=42
```

- Disable the job that has the job ID 102:

```
s9s job --disable --job-id=102
```

- Enable the job that has the job ID 102:

```
s9s job --enable --job-id=102
```

- Create a job that runs every 5 minutes and does nothing at all. This can be used for testing and demonstrating the recurring jobs without doing any significant or dangerous operations.

```
s9s job --success --recurrence="*/5 * * * *"
```

- Clone job ID 14, run it in as a new job, and see the job messages:

```
s9s job --clone \
  --job-id=14 \
  --log
  --clone
```

- Kill a running job with job ID 20:

```
s9s job --kill \
  --job-id=20
```

3.5.8 s9s-log

View logs.

Usage

```
s9s log {command} {options}
```

Commands

Name, shorthand	Description
`--list, -L`	Lists all log entries.

Options

Name, shorthand	Description
CLUSTER RELATED OPTIONS	
`--cluster-id=ID`	The ID of the cluster to check.
`--cluster-name=NAME`	The NAME of the cluster to check.
JOB RELATED OPTIONS	
`--limit=NUMBER`	Limits the number of log messages printed.
`--offset=NUMBER`	Controls the relative index of the first item printed.

Warning

Attention

The following log-related option has been deprecated since version 2.2.0 (Sept 2024):

- `--from=DATE&TIME`
- `--until=DATE&TIME`
- `--message-id=ID`
- `--log-format=FORMATSTRING`
- `--long`

Examples

- List log messages for Cluster ID 1:

```
s9s log \
    --list \
    --cluster-id=1
```

- List log messages for Cluster ID 1 except the 10 last log records:

```
s9s log \
    --list \
    --cluster-id=1 \
    --offset=10
```

- List only the last 20 log messages for Cluster ID 1:

```
s9s log \
    --list \
    --cluster-id=1 \
    --limit=20
```

- List 20 log messages skipping the latest 20 log records for Cluster ID 1:

```
s9s log \
    --list \
    --cluster-id=1 \
    --limit=20 \
    --offset=20
```

3.5.9 s9s-maintenance

View and manipulate maintenance periods.

Usage

```
s9s maintenance {command} {options}
```

Commands

Name, short hand	Description
--create	Creates a new maintenance period.
--current	Prints the active maintenance for a cluster or for a host. Prints nothing if no maintenance period is active.
--list	Lists the registered maintenance period. See *Maintenance List*.
--delete	Deletes an existing maintenance period. The maintenance periods are identified by their UUID strings. The UUID strings by default, are shown in an abbreviated format. The full-length UUID strings will be shown when the command line option is provided. Deleting a maintenance period is also possible by providing only the first few characters of the UUID when these first characters are unique and enough to identify the maintenance period.
--next	Prints information about the next maintenance period for a cluster or host. Prints nothing if no maintenance is registered to be started in the future.

Options

Name, shorthand	Description
`--be-gin=DATETI`	A string representation of the date and time when the maintenance period will start.
`--cluster-`	The cluster for cluster maintenance.
`--end=DATE`	A string representation of the date and time when the maintenance period will end.
`--full-uui`	Print the full UUID.
`--nodes=NO`	The nodes for the node maintenance. See *Node List*.
`--rea-son=STRING`	The reason for the maintenance.
`--start=DA`	A string representation of the date and time when the maintenance period will start. This option is deprecated, please use the `--begin` option instead.
`--uuid=UUI`	The UUID to identify the maintenance period.

Maintenance List

Using the `--list` and `--long` command-line options a detailed list of the registered maintenance periods can be printed:

```
$ s9s maintenance --list --long
ST UUID    OWNER  GROUP  START    END      HOST/CLUSTER  REASON
Ah a7e037a system admins 11:21:24 11:41:24 192.168.1.113 Rolling restart.
Total: 1
```

The list contains the following fields:

Field	Description
ST	The short status information, where at the first character position `A` stands for 'active' and – stands for 'inactive'. In the second character position `h` stands for 'host-related maintenance' and `c` stands for 'cluster-related maintenance'.
UUID	The unique string that identifies the maintenance period. Normally only the first few characters of the UUID is shown, but if the `--full-uuid` the command-line option is provided the full-length string will be printed.
OWNER	The name of the owner of the given maintenance period.
GROUP	The name of the group owner of the maintenance period.
START	The date and time when the maintenance period starts.
END	The date and time when the maintenance period expires.
HOST/C	The name of the cluster or host under maintenance.
REA-SON	A short human-readable description showing why maintenance is required.

Examples

- Create a maintenance period for PostgreSQL node 10.35.112.21, starting at 05:44:55 AM for one full day (cmon expects UTC time to create a maintenance):

```
s9s maintenance \
      --create \
      --nodes=10.35.112.21:5432 \
      --begin=2024-05-15T05:44:55.000Z \
      --end=2024-05-16T05:44:55.000Z \
      --reason='Upgrading RAM' \
      --batch
```

- Create a new maintenance period for 192.168.1.121 which shall start tomorrow and finish an hour later:

```
s9s maintenance --create \
      --nodes=192.168.1.121 \
      --begin="$(date --date='now + 1 day' --utc '+%Y-%m-%d %H:%M:%S')"
↪" \
      --end="$(date --date='now + 1day + 1 hour' --utc '+%Y-%m-%d %H:
↪%M:%S')" \
      --reason="Upgrading software."
```

- List out all nodes that are under maintenance period:

```
$ s9s maintenance --list --long
ST UUID    OWNER GROUP START    END      HOST/CLUSTER REASON
-h 70346c3 dba   admin 07:42:18 08:42:18 10.0.0.209   Upgrading RAM
Total: 1
```

- Delete a maintenance period for UUID 70346c3:

```
s9s maintenance --delete --uuid=70346c3
```

- Check if there is any ongoing maintenance period for cluster ID 1:

```
s9s maintenance --current --cluster-id=1
```

- Check the next maintenance period scheduled for node 192.168.0.227 for cluster ID 1:

```
s9s maintenance \
      --next \
      --cluster-id=1 \
      --nodes="192.168.0.227"
```

3.5.10 s9s-metatype

Lists meta-types supported by the controller.

Usage

```
s9s metatype {command} {options}
```

Commands

Name, shorthand	Description
`--list, -L`	Lists the names of the types the controller supports. See *Property List*.
`--list-cluster-`	Lists the cluster types the controller supports. With the `--long` option also lists the vendors and the versions.
`--list-properti`	Lists the properties the given metatype has. Use the `--type` command-line option to pass the name of the metatype.

Options

Name, shorthand	Description
`--type=TYPENAME`	The name of the type.

Property List

Using the `--list-properties` and `--long` command line options a detailed list of the metatype properties can be printed:

```
$ s9s metatype --list-properties --type=CmonUser --long
ST NAME          UNIT DESCRIPTION
rw email_address -    The email address of the user.
rw first_name    -    The first name of the user.
rw groups        -    The list of groups for the user.
rw job_title     -    The job title of the user.
r- last_login    -    The date&time of the last successful login.
```

The list contains the following fields:

Field	Description
ST	The short status information, where at the first character position `r` stands for 'readable' and – shows that the property is not readable by the client program. At the second character position `w` stands for 'writable' and – shows that the property is not writable by the client.
NAME	The name of the property.
UNIT	The unit in which the given property is measured (e.g. 'byte'). This field shows a single – character if the unit is not applicable.
DESCRIPTION	The short human-readable description of the property.

Examples

- List the metatypes the controller supports:

```
s9s metatype --list
```

- List a detailed list of the properties the CmonUser type has:

```
s9s metatype \
        --list-properties \
        --type=CmonUser \
        --long
```

- List all cluster types currently managed by this controller:

```
s9s metatype \
        --list-cluster-types \
        --long
```

3.5.11 s9s-node

View and handle nodes.

Usage

```
s9s node {command} {options}
```

Commands

Name, shorthand	Description
--change-co	Changes configuration values for the given node.
--enable-bi	Creates a job to enable binary logging on a specific node. Not all clusters support this feature (MySQL does). One needs to enable binary logging in order to set up a cluster to cluster replication.
--list, -L	List nodes. Default to all clusters.
--list-conf	Lists the configuration values for the given node.
--pull-conf	Copy the configuration file(s) from the node to the local computer. Use the --output-dir to control where the files will be created.
--push-conf	Copy the configuration file(s) to a node.
--register	Registers a node that already is working.
--restart	Restarts the node. This means the process that provides the main functionality on the node (e.g. the MySQL daemon on a Galera node) will be stopped then start again.
--set	Sets various properties of the specified node/host.
--set-confi	Changes configuration values for the given node.
--set-read-	Creates a job that sets the node to read-only mode. Please note that not all cluster types support read-only mode.
--set-read-	Creates a job that sets the node to read-write mode if it was previously set to read-only mode. Please note that not all cluster types support read-only mode.
--start	Starts the node. This means the process that provides the main functionality on the node (e.g. the MySQL daemon on a Galera node) will be started.
--stat	Prints detailed node information. It can be used in conjunction with --graph to produce statistical data. See *Graph Options*.
--stop	Stops the node. This means the process that provides the main functionality on the node (e.g. the MySQL daemon on a Galera node) will be stopped.
--unregis-ter	Unregisters the node from ClusterControl.

Options

Name, short-hand	Description
`--cluste`	The ID of the cluster in which the node is.
`--cluste`	Name of the cluster to list.
`--nodes=`	The nodes to list or manipulate. See *Node List*.
`--node-f`	The format string used to print nodes.
`--opt-gr`	The configuration option group.
`--opt-na`	The name of the configuration option.
`--opt-va`	The value of the configuration option.
`--output`	The directory where the files are created.
`--graph=`	The name of the graph to show. See *Graph Options*.
`--be-gin=TIME`	The start of the graph interval.
`--end=TI`	The end of the graph interval.
`--force`	Force the execution of potentially dangerous operations like restarting a master node in a MySQL Replication cluster.
`--be-gin=TIME`	The start time of the graph (the X-axis).
`--den-sity`	If this option is provided will be a probability density function (or histogram) instead of a timeline. The X-axis shows the measured values (e.g. MByte/s) while the Y-axis shows how many percent of the measurements contain the value. If for example, the CPU usage is between 0% and 1% at 90% of the time the graph will show a 90% bump at the lower end.
`--end=TI`	The end of the graph.
`--node-f`	The string that controls the format of the printed information about the nodes. See *Node Format*.
`--prop-er-ties=ASS`	One or more assignments specifying property names and values. The assignment operator is the = character (e.g. `--properties='alias="newname"'`), multiple assignments are separated by the semicolon `;`.
`--output`	The directory where the output files will be created on the local computer.

Graph Options

When providing a valid graph name together with the `--stat` option a graph will be printed with statistical data. Currently, the following graphs are available:

Option	Description
cpughz	The graph will show the CPU clock frequency measured in GHz.
cpu-load	Shows the average CPU load of the host computer.
cpusys	Percent of time the CPU spent in kernel mode.
cpuidle	Percent of time the CPU is idle on the host.
cpuiowa	Percent of time the CPU is waiting for IO operations.
cputemp	The temperature of the CPU measured in degree Celsius. Please note that to measure the CPU temperature some kernel module might be needed (e.g. it might be necessary to run `sudo modprobe coretemp`). On multiprocessor systems, the graph might show only the first processor.
cpu-user	Percent of time the CPU is running user space programs.
disk-free	The amount of free disk space measured in GBytes.
diskrea speed	Disk read speed measured in MBytes/sec.
diskrea write-speed	Disk read and write speed measured in MBytes/sec.
diskwri speed	Disk write speed measured in MBytes/sec.
disku-ti-liza-tion	The bandwidth utilization for the device in percent.
mem-free	The amount of the free memory measure in GBytes.
memu-til	The memory utilization of the host measured in percent.
neter-rors	The number of receive and transmit errors on the network interface.
ne-tre-ceived-speed	Network read speed in MByte/sec.
ne-tre-ceiveer rors	The number of packets received with error on the given network interface.
net-trans-miter-rors	The number of packets failed to transmit.
net-sentspe	Network write speed in MByte/sec.
net-speed	Network read and write speed in MByte/sec.
sql-com-mands	Shows the number of SQL commands executed measured in 1/s.
sql-com-mits	The number of commits measured in 1/s.

Node List

Using the `--list` and `--long` command-line options a detailed list of the nodes can be printed. Here is an example of such a list:

```
$ s9s node --list --long '192.168.1.1*'
STAT VERSION CID CLUSTER HOST PORT COMMENT
poM- 9.6.2 1 ft_postgresql_11794 192.168.1.117 8089 Up and running
coC- 1.4.2 1 ft_postgresql_11794 192.168.1.127 9555 Up and running
Total: 3
```

???+ Note

```
The list in the example is created using a filter (that is '192.168.1.1\*'␣
↪in the example). The last line shows a 3 as the total, the number of␣
↪nodes maintained by the controller, but only two of the nodes are printed␣
↪in the list because of the filter.
```

The list contains the following fields:

Field	Description
STAT	**Nodetype**This is the type of node. It can be `c` for controller, `g` for Galera node, `x` for MaxScale node, `k` for Keepalived node, `p` for PostgreSQL, `m` for Mongo, `e` for MemCached, `y` for ProxySql, `h` for HaProxy, `b` for PgBouncer, `B` for PgBackRest, `t` for PBMAgent, `a` for Garbd, `r` for group replication host, `A` for cmon agent, `P` for Prometheus, `s` for generic MySQL nodes, `S` for Redis sentinel, `R` for Redis, `E` for Elasticsearch, and `?` for unknown nodes. **Hoststatus**The status of the node. It can be `o` for online, `l` for off-line, `f` for failed nodes, `r` for nodes performing recovery, `-` for nodes that are shut down and `?` for nodes in an unknown state.**Role** This field shows the role of the node in the cluster. This can be `M` for primary, `S` for replica, `U` for multi (primary and replica), `C` for controller, `V` for backup verification node, `A` for arbiter, `R` for backup repository host, `D` for Elasticsearch data host `c` for Elasticsearch coordinator only host and `-` for everything else.**Maintenance** This field shows if the node is in maintenance mode. The character is `M` for nodes in maintenance mode and `-` for nodes that are not in maintenance mode.
VERSION	This field shows the version string of the software that provides the service represented in the given line.
CID	The cluster ID of the cluster that holds the node as a member. Every node belongs to exactly one cluster.
CLUSTER	The name of the cluster that holds the node as a member.
HOST	The hostname of the host. This can be a real DNS hostname, the IP address, or the Cmon alias name of the node depending on the configuration and the command line options. The cluster is usually configured to use IP addresses (the Cmon configuration file contains IP addresses) so this field usually shows IP addresses.
PORT	The IP port on which the node accepts requests. The same DNS hostname or IP address can be added multiple times to the same or to multiple clusters, but the host:port pair must be unique. In other words, the same host with the same port can not be added to the same Cmon controller twice. Since the hostname:port pair is unique the nodes are identified by this and every line of the node list is representing a hostname:port node. There is one exception to this rule: the Cmon Controller can manage multiple clusters and so be a part of more than one cluster with the same hostname and port.
COMMEN	A short human-readable description that the Cmon Controller sets automatically to describe the host state. A single `-` character is shown if the controller did not set the message.

Node Format

The string that controls the format of the printed information about the nodes. When this command-line option is used the specified information will be printed instead of the default columns. The format string uses the `%` character to mark variable fields and flag characters as they are specified in the standard `printf()` C library functions. The `%` specifiers are ended by field name letters to refer to various properties of the nodes.

The `%+12i` format string for example has the `+12` flag characters in it with the standard meaning the field will be 12 character-wide and the `+` or `-` sign will always be printed with the number. The properties of the node are encoded by letters. In the `%16D` for example, the letter `D` encodes the "data directory" field, so the full path of the data directory on the node will be substituted. Standard `\` notation is also available, `\n` for example encodes a new-line character.

The s9s-tools support the following fields:

Field	Description
A	The IP address of the node.
a	Maintenance mode flag. If the node is in maintenance mode a letter `M`, otherwise `-`.
b	The master host for this slave if it is applicable.
C	The configuration file for the most important process on the node (e.g. the configuration file of the MySQL daemon on a Galera node).
c	The total number of CPU cores in the host. Please note that this number may be affected by hyper-threading. When a computer has 2 identical CPUs, with four cores each and uses 2x hyper-threading it will count as 2x4x2 = 16.
D	The data directory of the node. This is usually the data directory of the SQL server.
d	The PID file on the node.
G	The name of the group owner of the node.
g	The log file on the node.
h	The CDT path of the node.
I	The numerical ID of the node.
i	The total number of monitored disk devices (partitions) in the cluster.
k	The total number of disk bytes found on the monitored devices in the node. This is a double-precision floating-point number measured in Terabytes.
L	The replay location. This field currently only has valid value in PostgreSQL clusters.
l	The received location. This field currently only has valid value in PostgreSQL clusters.
M	A message, describing the node's status in a human-readable format.
m	The total memory size found in the host, measured in GBytes. This value is represented by a double-precision floating-point number, so formatting it with precision (e.g. `%6.2m`) is possible. When used with the f modifier (e.g. `%6.2fm`) this reports the free memory, the memory that available for allocation, used for cache or used for buffers.
N	The name of the node. If the node has an alias that is used, otherwise the name of the node is used. If the node is registered using the IP address, the IP address is the name.
n	The total number of monitored network interfaces in the host.
O	The user name of the owner of the cluster that holds the node.
o	The name and version of the operating system together with the codename.

continues on next page

Table 3.7 – continued from previous page

Field	Description
P	The port on which the most important service is waiting for requests.
p	The PID (process ID) on the node that presents the service (e.g. the PID of the MySQL daemon on a Galera node).
R	The role of the node (e.g. “controller”, “master”, “slave” or “none”).
r	The work read-only or read-write indicating if the server is in read-only mode or not.
S	The status of the host (e.g. CmonHostUnknown, CmonHostOnline, CmonHostOffLine, CmonHostFailed, CmonHostRecovery, CmonHostShutDown).
s	The list of slaves of the given host in one string.
T	The type of the node, e.g. “controller”, “galera”, “postgres”.
t	The total network traffic (both received and transmitted) measured in MBytes/seconds.
U	The number of physical CPUs on the host.
u	The CPU usage percent found on the host.
V	The version string of the most important software (e.g. the version of the PostgreSQL installed on a PostgreSQL node).
v	The ID of the container/VM in “CLOUD/ID” format. The – string if no container ID is set for the node.
w	The total swap space found in the host measured in GigaBytes. With the f modifier (e.g. `%6.2fw`) this reports the free swap space in GigaBytes.
Z	The name of the CPU model. Should the host have multiple CPUs, this will return the model name of the first CPU.
%	The `%` character itself.

Examples

- List all nodes:

```
$ s9s node --list --long
ST  VERSION                  CID CLUSTER        HOST       PORT COMMENT
go- 10.1.22-MariaDB-1~xenial   1 MariaDB Galera 10.0.0.185 3306 Up and␣
↪running
co- 1.4.1.1856                 1 MariaDB Galera 10.0.0.205 9500 Up and␣
↪running
go- 10.1.22-MariaDB-1~xenial   1 MariaDB Galera 10.0.0.209 3306 Up and␣
↪running
go- 10.1.22-MariaDB-1~xenial   1 MariaDB Galera 10.0.0.82  3306 Up and␣
↪running
```

(continues on next page)

(continued from previous page)

```
Total: 4
```

- Print the configuration for a node:

```
$ s9s node --list-config --nodes=10.0.0.3
...
mysqldump   max_allowed_packet                      512M
mysqldump   user                                    backupuser
mysqldump   password                                nWC6NSm7PnnF8zQ9
xtrabackup  user                                    backupuser
xtrabackup  password                                nWC6NSm7PnnF8zQ9
MYSQLD_SAFE pid-file                                /
 ↪var/lib/mysql/mysql.pid
MYSQLD_SAFE basedir                                 /usr/
Total: 71
```

- The following example shows how a node in a given cluster can be restarted. When this command executed a new job will be created to restart a node. The command-line tool will stop and show the job messages until the job is finished:

```
s9s node \
        --restart \
        --cluster-id=1 \
        --nodes=192.168.1.117 \
        --log
```

- Change a configuration value for a PostgreSQL server:

```
s9s node \
        --change-config \
        --nodes=192.168.1.115 \
        --opt-name=log_line_prefix \
        --opt-value='%m '
```

- Push a configuration option inside `my.cnf` (`max_connections=500`) on node 10.0.0.3:

```
s9s node \
        --change-config \
        --nodes=10.0.0.3 \
        --opt-group=mysqld \
        --opt-name=max_connections \
        --opt-value=500
```

- Import two existing Keepalived nodes (192.168.20.56 and 192.168.20.57) where the virtual IP is 192.168.20.59 into cluster ID 13:

```
# import primary keepalived
s9s node \
        --register \
        --nodes="keepalived://192.168.20.56" \
        --virtual-ip=192.168.20.59 \
```

(continues on next page)

(continued from previous page)

```
        --cluster-id=13 \
        --wait

# import secondary keepalived
s9s node \
        --register \
        --nodes="keepalived://192.168.20.57" \
        --virtual-ip=192.168.20.59 \
        --cluster-id=13 \
        --wait
```

- Listing the Galera hosts. This can be done by filtering the list of nodes by their properties:

```
s9s node \
        --list \
        --long \
        --properties="class_name=CmonGaleraHost"
```

- Create a set of graphs, one for each node shown in the terminal about the load on the hosts. If the terminal is wide enough the graphs will be shown side by side for a compact view:

```
s9s node \
        --stat \
        --cluster-id=1 \
        --begin="08:00" \
        --end="14:00" \
        --graph=load
```

- The density function can also be printed to show what were the typical values for the given statistical data. The following example shows what was the typical values for the user mode CPU usage percent:

```
s9s node \
        --stat \
        --cluster-id=2 \
        --begin=00:00 \
        --end=16:00 \
        --density \
        --graph=cpuuser
```

- The following example shows how a custom list can be created to show some information about the CPU(s) in some specific hosts:

```
s9s node \
        --list \
        --node-format="%N %U CPU %c Cores %6.2u%% %Z\n" 192.168.1.191␣
↪192.168.1.195
192.168.1.191 2 CPU 16 Cores 22.54% Intel(R) Xeon(R) CPU L5520 @␣
↪2.27GHz
192.168.1.195 2 CPU 16 Cores 23.12% Intel(R) Xeon(R) CPU L5520 @␣
↪2.27GHz
```

- The following list shows some information about the memory, the total memory and the memory available for the applications to allocate (including cache and buffer with the free memory):

```
s9s node \
        --list \
        --node-format="%4.2m GBytes %4.2fm GBytes %N\n"
16.00 GBytes 15.53 GBytes 192.168.1.191
47.16 GBytes 38.83 GBytes 192.168.1.127
```

- Set a node to read-write mode if it was previously set to read-only mode:

```
s9s node \
        --set-read-write \
        --cluster-id=1 \
        --nodes=192.168.0.78 \
        --log
```

- Copy configuration file(s) from a PostgreSQL server 192.168.0.232 into the localhost:

```
s9s node \
        --pull-config \
        --nodes="192.168.0.232" \
        --output-dir="tmp"
```

3.5.12 s9s-process

View processes running on nodes.

Usage

```
s9s process {command} {options}
```

Commands

Name, short-hand	Description
`--list`	Lists the processes.
`--list`	Prints statement digests together with statistical data showing how long it took them to be executed. The printed list will not contain individual SQL statements but patterns that collect multiple statements of similar form merged into groups by the similarities.
`--list`	Lists the queries, internal SQL processes of the cluster.
`--top-`	Continuously showing the internal SQL processes in an interactive UI, similar *ClusterControl GUI Top Queries* page.
`--top`	Continuously showing the processes in an interactive UI like the well-known `top` utility. Please note that if the terminal program supports the UI can be controlled with the mouse.

Options

Name, shorthand	Description
`--cluster-id=ID`	The ID of the cluster to show.
`--client=PATTERN`	Shows only the processes that originate from clients that match the given pattern.
`--limit=N`	Limits the number of processes shown in the list.
`--server=PATTERN`	Shows only the processes that are executed by servers that match the given pattern.
`--sort-by-memory`	Sorts the processes by resident memory size instead of CPU usage.
`--sort-by-time`	Sorts the SQL queries by their runtime. The longer running queries are going to be on top.
`--update-freq=INTE(`	Update frequency for screen refresh in seconds.

Examples

- Continuously print aggregated view of processes (similar to `top` output) of all nodes for cluster ID 1:

```
s9s process --top --cluster-id=1
```

- List aggregated view of processes (similar to `ps` output) of all nodes for cluster ID 1:

```
s9s process --list --cluster-id=1
```

- Print out aggregated digested SQL statements on all MySQL nodes in cluster ID 1:

```
s9s process \
        --list-digests \
        --cluster-id=1 \
        --human-readable \
        --limit=10 \
        '*:3306'
```

- Print aggregated list of database top queries which contains the string "INSERT" on all nodes in cluster ID 1 with refresh rate every 1 second:

```
s9s process \
        --top-queries \
        --cluster-id=1 \
        --update-freq=1 \
        'INSERT*'
```

- Print all database queries on cluster ID 1 that is coming from a client with IP address 192.168.0.127 which having only the "INSERT" string:

```
s9s process \
        --list-queries \
        --cluster-id=1 \
        --client='192.168.0.127:*' \
        'INSERT*'
```

- Print all database queries on cluster ID 1 that are reaching the database server 192.168.0.81:

```
s9s process \
        --list-queries \
        --cluster-id=1 \
        --server='192.168.0.81:*'
```

3.5.13 s9s-replication

Manage database replication related functions.

???+ Note

```
Only applicable for supported database clusters namely MySQL/MariaDB␣
↪Replication and PostgreSQL Streaming Replication.
```

Usage

```
s9s replication {command} {options}
```

Commands

Name, shorthand	Description
`--failover`	Takes over the role of master from a failed master.
`--list`	List the replication links.
`--promote`	Promotes a slave to become a master.
`--stage`	Rebuilds or stages a replication slave.
`--start`	Starts a replication slave previously stopped using the `--stop` option.
`--stop`	Make the slave stop replicating. This option will create a job that does not stop the server but stops the replication on it.

Options

Name, shorthand	Description
`--fail-stop-slave`	When this option is specified slave node will be stopped.
`--link-format=FORMA`	The format string controls the format of the printed information about the replication links. See *Link Format*.
`--master=NODE`	The replication master.
`--remote-cluster-id`	Remote cluster ID for the c2c replication.
`--replication-maste`	This is the same as the `--master` option.
`--slave=NODE`	The replication slave.
`--replication-slave`	This is the same as the `--slave` option.

Link Format

The format string controls the format of the printed information about the links. When this command-line option is used the specified information will be printed instead of the default columns. The format string uses the `%` character to mark variable fields and flag characters as they are specified in the standard `printf()` C library functions. The `%` specifiers are ended by field name letters to refer to various properties of the replication.

The `%+12p` format string for example has the `+12` flag characters in it with the standard meaning: the field will be 12 characters wide and the `+` or `-` the sign will always be printed with the number. The properties of the links are encoded by letters. The in the `%4p` for example the letter `p` encodes the `slave port` field, so the port number of the slave node will be substituted. Standard `\` the notation is also available, `\n` for example encodes a new-line character.

The following example prints out a customized format to show the replication link among nodes together with the position of master/slave in the replication log for PostgreSQL streaming replication, MySQL GTID and MariaDB GTID:

```
```
$ s9s replication \
        --list \
        --long \
        --link-format="%16h %4p <- %H %2P %o %O\n"
192.168.0.81 5432 <- 192.168.0.83 5432 0/71001BA8
192.168.0.82 5432 <- 192.168.0.83 5432 0/71001BA8
192.168.0.42 3306 <- 192.168.0.41 3306␣
↪dca7e205-90db-11ea-9143-5254008afee6:1-10 3157
192.168.0.43 3306 <- 192.168.0.41 3306␣
↪dca7e205-90db-11ea-9143-5254008afee6:1-10 3157
192.168.0.92 3306 <- 192.168.0.91 3306 0-54001-49 10285
192.168.0.93 3306 <- 192.168.0.91 3306 0-54001-49 10285
```
```

The s9s-tools support the following fields:

Field	Description
c	The cluster ID of the cluster where the slave node can be found.
C	The master cluster-ID property of the slave host. This shows in which cluster the master of the represented replication link can be found.
d	This format specifier denotes the “seconds behind the master” property of the slave.
h	The hostname of the slave node in the link.
H	The hostname of the master node in the link.
o	The position of the slave in the replication log.
O	The position of the master in the replication log.
p	The port number of the slave.
P	The port number of the master node.
s	A short string representing the link status, e.g. “Online” when everything is ok.
m	A slightly longer, human-readable string representing the state of the link. This is actually the `slave_io_state` property of the slave node.

Examples

- Print all database replication links for clusters that fall under the replication category:

```
s9s replication --list
```

- Promote a slave 192.168.0.164 to become a new master for a database cluster named "MySQL Rep 5.7 – Production":

```
s9s replication \
        --promote \
        --cluster-name="MySQL Rep 5.7 - Production" \
        --slave="192.168.0.164:3306" \
        --wait
```

- Rebuild the replication slave of 192.168.0.83 from the master, 192.168.0.76, and tag the job as "stage" for cluster ID 1:

```
s9s replication \
        --stage \
        --cluster-id=1 \
        --job-tags="stage" \
        --slave="192.168.0.83:3306" \
        --master="192.168.0.76:3306" \
        --wait
```

- Stop the replication slave on node 192.168.0.80:

```
s9s replication \
        --stop \
        --cluster-id=1 \
        --slave="192.168.0.80:3306" \
        --wait
```

- Start the replication slave on node 192.168.0.80:

```
s9s replication \
        --start \
        --cluster-id=1 \
        --slave="192.168.0.80:3306" \
        --wait
```

3.5.14 s9s-report

Manage operational reports.

Usage

```
s9s report {command} {options}
```

Commands

Name, shorthand	Description
`--cat`	Prints the report text to the standard output.
`--create`	Creates and prints a new report. Please provide the `--type` command-line option with the report type (that was chosen from the template list printed with the `--list-templates` option) and the `--cluster-id` with the cluster ID.
`--delet`	Deletes a report that already exists. The report ID should be used to specify which report deleting.
`--list`	Lists the reports that already created.
`--list-`	Lists the report templates showing what kind of reports can be created. This command-line option is usually used together with the `--long` option.

Options

Name, shorthand	Description
`--cluster`	This command-line option passes the numerical ID of the cluster.
`--report-`	This command-line option passes the numerical ID of the report to manage. When a report is deleted (`--delete` option) or printed (`--cat` option) this option is mandatory.
`--type=NA`	This command-line option controls the type of report. Use the `--list-templates` to list what types are available.

Examples

- Lists already created reports for cluster ID 1:

```
s9s report \
        --list \
        --long \
        --cluster-id=1
```

- Print the report ID 1 for cluster ID 1:

```
s9s report \
        --cat \
        --report-id=1 \
        --cluster-id=1
```

- Create a database growth report for cluster ID 1:

```
s9s report \
        --create \
        --type=dbgrowth \
        --cluster-id=1
```

- Delete an operation report with report ID 11:

```
s9s report \
        --delete \
        --report-id=11
```

- Print the supported templates that can be used with `type` value when creating a new report:

```
s9s report \
        --list-templates \
        --long
```

3.5.15 s9s-script

Manage and execute Advisor scripts.

Usage

```
s9s script {command} {options}
```

Commands

Name, shorthand	Description
`--execute`	Executes a script from a local file.
`--system`	Executes a shell command or an entire shell script on the nodes of the cluster showing the output of the command as job messages. See *Shell Commands*.
`--tree`	Print the names of the scripts stored on the controller in tree format.

Options

Name, shorthand	Description
`--cluster-i`	The target cluster-ID.
`--cluster-n`	The target cluster name.
`--shell-com`	Executes a shell command with `--system` flag. See *Shell Commands*.
`--timeout`	The timeout value when executing the command/script in seconds, default is 10 seconds. If the command/script keeps running, the job will be failed and the execution of the command(s) is aborted.

Shell Commands

If the `--shell-command` option is provided, the options argument will be executed. If a file name is provided as a command-line argument the content of the file will be executed as a shell script. If neither found in the command line the s9s will read its standard input and execute the lines found as a shell script on the nodes (this is the way to implement self-contained remote executed shell scripts using a shebang).

Please note that this will be a job and it can be blocked by other jobs running on the same cluster.

```
s9s script \
        --system \
        --log \
        --cluster-id=1 \
        --shell-command="df -h"
```

The command/script will be executed using the pre-configured OS user of the cluster. If that user has no superuser privileges the Sudo utility can be used in the command or the script to gain superuser privileges.

By default the command/script will be executed on all nodes, all members of the cluster except the controller. This can be changed by providing a node list using the `--nodes` command-line option.

To execute shell commands the authenticated user has to have executed privileges on the nodes. If the executive privilege is not granted the credentials of another Cmon User can be passed in the command line or the privileges can be changed (see *s9s-tree* for details about owners, privileges, and ACLs).

```
s9s script \
        --system \
        --cmon-user=system \
        --password=mysecret \
        --log \
        --cluster-id=1 \
        --timeout=2 \
        --nodes='192.168.0.127;192.168.0.44' \
        --shell-command="df -h"
```

Please note that the job will by default has a 10 seconds timeout, so if the command/script keeps running the job will be failed and the execution of the command(s) aborted. The timeout value can be set using the `--timeout` command-line option.

Examples

- Print the scripts available for cluster ID 1:

```
s9s script --tree --cluster-id=1
```

- Execute a script called `test_script.sh` on all nodes in the cluster with ID 1:

```
s9s script \
        --system \
        --log \
        --log-format="%M\n" \
        --timeout=20 \
        --cluster-id=1 \
        test_script.sh
```

- Get the disk space summary of host 192.168.0.127 and 192.168.0.44 in cluster ID 1:

```
s9s script \
        --system \
        --log \
        --cluster-id=1 \
```

(continues on next page)

(continued from previous page)

```
        --timeout=2 \
        --nodes='192.168.0.127;192.168.0.44' \
        --shell-command="df -h"
```

3.5.16 s9s-server

Provisions and manages virtualization host to be used to host database containers.

???+ Note

```
The supported virtualization platforms are LXC/LXD and Amazon Web Service␣
↪EC2 instances. Docker is not supported at the moment.
```

LXC Containers

Handling lxc containers is a new feature added to the CMON Controller and the s9s command-line tool. The basic functionality is available and tested, containers can be created, started, stopped, deleted, and even creating containers on the fly while installing clusters or cluster nodes is possible.

For the lxc containers, one needs a container server, a computer that has the lxc software installed and configured, and of course needs a proper account to access the container server from the CMON Controller. One can set up an lxc container server in two easy and one not so easy steps:

1. Install a Linux server and set it up so that the root user can ssh in from the Cmon Controller with a key, without a password. Creating such access for the superuser is of course not the only way, it is just the easiest.
2. Register the server as a container server on the CMON Controller by issuing the `s9s server --register --servers="lxc://IP_ADDRESS"` command. This will install the necessary software and register the server as a content server to be used later.
3. The hard part is the network configuration on the container server. Most of the distributions by default have a network configuration that provides a local (host only) IP address for the newly created containers. In order to provide a public IP address for the containers, the container server must have some sort of bridging or NAT configured.

A possible way to configure the network for public IP is described in this blog, Converting eth0 to br0 and getting all your LXC or LXD onto your LAN.

CMON-Cloud Virtualization

The cmon-cloud containers are an experimental virtualization backend currently added to the CMON Controller as a brand new feature.

Usage

```
s9s server {command} {options}
```

Commands

Name, shorthand	Description
`--add-a`	Adds a new ACL entry to the server or modifies an existing ACL entry.
`--create`	Creates a new server. If this option is provided the controller will use SSH to discover the server and install the necessary software packages and modify the configuration if needed so that the server can host containers.
`--get-a`	List the ACL of a server.
`--list`	List the registered servers.
`--list-`	List disks found in one or more servers.
`--list-`	List the images available on one or more servers. With the `--long` command-line option a more detailed list is available.
`--list-`	List memory modules from one or more servers.
`--list-`	List network controllers from one or more servers.
`--list-`	List partitions from multiple servers.
`--list-`	List processors from one or more servers.
`--list-`	Prints the list of regions the server(s) support together with some important information (e.g. if the controller has credentials to use those regions or not).
`--list-`	List all the subnets that exist on one or more servers.
`--list-`	Lists the supported templates. Various virtualization technologies handle templates differently, some even use different terminology (for example "size" is one such synonym). In the case of LXC, any stopped container can be used as a template for new containers to be created.
`--register`	Registers an existing container server. If this command-line option is provided the controller will register the server to be used as a container server later. No software packages are installed or configurations changed.
`--start`	Boot up a server. This option will try to start up a server that is physically turned off (using e.g. the wake-on-LAN feature).
`--stat`	Prints details about one or more servers.
`--stop`	Shuts down and power off a server. When this command-line option is provided the controller will run the shutdown program on the server.
`--unregister`	Unregisters a container server, and simply removes it from the controller.

Options

Name, shorthand	Description
`--log`	If the s9s application created a job and this command-line option is provided it will wait until the job is executed. While waiting the job logs will be shown unless the silent mode is set.
`--recurrence=CR`	This option can be used to create recurring jobs, jobs that are repeated over and over again until they are manually deleted. Every time the job is repeated a new job will be instantiated by copying the original recurring job and starting the copy. The option's argument is a crontab-style string defining the recurrence of the job. See *Crontab*.
`--schedule=DATE`	The job will not be executed now but it is scheduled to execute later. The DateTime string is sent to the backend, so all the formats are supported by the controller.
`--timeout=SECO`	Sets the timeout for the created job. If the execution of the job is not done before the timeout counted from the start time of the job expires the job will fail. Some jobs might not support the timeout feature, the controller might ignore this value.
`--wait`	If the application created a job (e.g. to create a new cluster) and this command-line option is provided the s9s program will wait until the job is executed. While waiting a progress bar will be shown unless the silent mode is set.
`--acl=AC`	The ACL entry to set.
`--os-key`	The SSH key file to authenticate on the server. If none of the operating system authentication options are provided (`--os-key-file`, `--os-password`, `--os-user`) the controller will try to log in with the default settings.
`--os-pas`	The SSH password to authenticate on the server. If none of the operating system authentication options are provided (`--os-key-file`, `--os-password`, `--os-user`) the controller will try to log in with the default settings.
`--os-use`	The SSH username to authenticate on the server. If none of the operating system authentication options are provided (`--os-key-file`, `--os-password`, `--os-user`) the controller will try to log in with the default settings.
`--refresh`	Do not use cached data, collect information.
`--server`	List of servers.

Server List

Using the `--list` and `--long` command line options a detailed list of the servers can be printed. Here is an example of such a list:

```
$ s9s server --list --long
PRV VERSION #C OWNER GROUP     NAME         IP            COMMENT
lxc 2.0.8    5 pipas testgroup core1       192.168.0.4   Up and running.
lxc 2.0.8    5 pipas testgroup storage01   192.168.0.17  Up and running.
Total: 2 server(s)
```

The list contains the following fields:

Field	Description
PRV	The name of the provider software. The software that will handle containers or virtual machines on the server. One server can have only one such system, but multiple servers can be registered using one physical computer.
VERSION	The version of the provider software.
#C	The number of containers/virtual machines currently hosted by the server.
OWNE	The owner of the server object.
GROU	The group owner of the server object.
NAME	The hostname of the server.
IP	The IP address of the server.
COMMENT	A human-readable description of the server and its state.

Examples

- Register a virtualization host:

```
s9s server --register --servers=lxc://storage01
```

- Check the list of virtualization hosts:

```
s9s server --list --long
```

- Create a virtualization server with an operating system username and password to be used to host containers. The controller will try to access the server using the specified credentials:

```
s9s server \
        --create \
        --os-user=testuser \
        --os-password=passw0rd \
        --servers=lxc://192.168.0.250 \
        --log
```

3.5.17 s9s-tree

Create, manage and manipulate CMON Directory Tree (CDT).

Usage

```
s9s tree {command} {options}
```

Commands

Name shorthand	Description
`--access`	Check the access rights for the authenticated user to the given CDT entry. This main option is made to be used in shell scripts, the exit code of the s9s program shows if the user has the privileges specified in the command line.
`--add`	Add an access control list (ACL) entry to an object in the tree. Overwrites the ACL if the given type of ACL already exists.
`--add`	This main option can be used to add a new tag to the tag list of an existing object.
`--cat`	This option can be used to print the content of a CDT file entry similar way the `cat` standard utility is used to print normal files.
`--cho`	Change the ownership of an object. The new owner (and optionally the group owner) should be passed through the `--owner` command-line option.
`--del`	Remove CDT entries.
`--get`	Print the ACL of a CDT entry.
`--lis`	Print the Cmon Directory Tree in list format.
`--mkd`	Create a new directory in the tree.
`--mov`	Move an object to a new location in the tree or rename the entry. If the target contains the `/` character, it is assumed to be a directory and the source entry will be moved to that directory with its name unchanged. If the target contains no, it is assumed to be a new name and the source entry will be renamed while kept in the same directory.
`--rem`	Remove an ACL entry from the ACL of an object.
`--rem`	Remove a tag from the tag list of an existing object.
`--rmd`	Remove an empty directory from the tree.
`--sav`	Saves data (text) into an existing CDT entry that has the proper type (the type is a file).
`--tou`	Creates a CDT entry that is a file.
`--tre`	Print the tree in its original tree format.
`--wat`	Opens an interactive UI to watch and manipulate the CDT filesystem and its entries.

Options

Name, shorthand	Description
`--acl=ACLS`	An ACL entry in string format. See *ACL Text Forms*.
`--all`	The CDT entries that have a name starting with. considered to be hidden entries. These are only printed if the `--all` command-line option is provided.
`--owner=US`	The user name and group name of the owner.
`--recursive`	The print also the sub-items of the tree. The `--chown` will change the ownership for sub-items too. Please note that the `--tree` is always recursive, no need for this command-line option there.
`--refresh`	Recollect the data.
`--tag=TAGS`	Specify one single tag when adding or removing tags of a tag list that belongs to an object.

ACL Text Forms

A long and a short text form for representing ACLs is defined. In both forms, ACL entries are represented as three colon-separated fields – an ACL entry tag type, an ACL entry qualifier, and discretionary access permissions. The first field contains one of the following entry tag type keywords:

Tag type	Description
user	A user ACL entry specifies the access granted to either the file owner (entry tag type `ACL_USER_OBJ`) or a specified user (entry tag type `ACL_USER`).
group	A group ACL entry specifies the access granted to either the filegroup (entry tag type `ACL_GROUP_OBJ`) or a specified group (entry tag type `ACL_GROUP`).
mask	A mask ACL entry specifies the maximum access which can be granted by any ACL entry except the user entry for the file owner and the other entry (entry tag type `ACL_MASK`).
other	Another ACL entry specifies the access granted to any process that does not match any user or group ACL entries (entry tag type `ACL_OTHER`).

The second field contains the user or group identifier of the user or group associated with the ACL entry for entries of entry tag type `ACL_USER` or `ACL_GROUP`, and is empty for all other entries. A user identifier can be a user name or a user ID number in decimal form. A group identifier can be a group name or a group ID number in decimal form.

The third field contains discretionary access permissions. The read, write and search/execute permissions are represented by the `r`, `w`and `x` characters, in this order. Each of these characters is replaced by the `-` character to denote that permission is absent in the ACL entry. When converting from the text form to the internal representation, permissions that are absent need not be specified.

White space is permitted at the beginning and end of each ACL entry, and immediately before and after a field separator (the colon character).

LONG TEXT FORM

The long text form contains one ACL entry per line. In addition, a number sign (`#`) may start a comment that extends until the end of the line. If an `ACL_USER`, `ACL_GROUP_OBJ` or `ACL_GROUP` ACL entry contains permissions that are not also contained in the `ACL_MASK` entry, the entry is followed by a number sign, the string "effective:", and the effective access permissions defined by that entry. This is an example of the long text form:

```
user::rw-
user:lisa:rw-          #effective:r--
group::r--
group:toolies:rw-      #effective:r--
mask::r--
other::r--
```

SHORT TEXT FORM

The short text form is a sequence of ACL entries separated by commas and is used for input. Comments are not supported. Entry tag type keywords may either appear in their full unabbreviated form, or in their single letter abbreviated form. The abbreviation for a user is `u`, the abbreviation for a group is `g`, the abbreviation for the mask is `m`, and the abbreviation for others is `o`.

The permissions may contain at most one each of the following characters in any order: `r`, `w`, `x`. These are examples of the short text form:

```
u::rw-,u:lisa:rw-,g::r--,g:toolies:rw-,m::r--,o::r--
g:toolies:rw,u:lisa:rw,u::wr,g::r,o::r,m::r
```

For more information, check out the POSIX Access Control Lists documentation by running `man acl` command.

Examples

- List out all CDT objects in a tree structure view:

```
s9s tree \
        --tree --all
```

- Add an access control list to allow a group called "users" to have read (access the cluster) and write (make a modification to the cluster) to a cluster named "galera_001":

```
s9s tree \
        --add-acl \
        --acl="group:users:rw-" \
        galera_001
```

- Tag a cluster with a string "Production" for a cluster named `PostgreSQL_Cluster_001` which located at /PostgreSQL Cluster 1 (as a CDT object):

```
s9s tree \
        --add-tag \
        --tag="Production" \
        "/PostgreSQL Cluster 001"
```

- Change the ownership of a cluster called "MariaDB 2 QA" (located at `/MariaDB 2 QA` as a CDT object) to user john and group DBA:

```
s9s tree \
        --chown \
        --owner=john:DBA \
        --recursive \
        "/MariaDB 2 QA"
```

- Create a new directory inside the CDT tree:

```
s9s tree \
        --mkdir \
        /home/kirk
```

- List out all objects under `/MariaDB_Cluster_1` including the hidden objects:

```
s9s tree \
        --list \
        --long \
```

(continues on next page)

(continued from previous page)

```
        --recursive \
        --all \
        --full-path \
        /MariaDB_Cluster_1
```

3.5.18 s9s-user

Manage users.

Usage

```
s9s user {command} {options}
```

Commands

Name, shorthand	Description
--add-k	Registers a new public key for an existing user. After the command, the user will be able to authenticate with the private part of the registered public key, no password will be necessary.
--add-t	Adds the user to a group.
--chang	Modifies the password of an existing user. The password is not a simple property, so it can not be changed using the --set option, this special command-line option has to be used.
--create	Registers a new user (create a new user account) on the controller and grants access to the ClusterControl system. The user name of the new account should be the command line argument.
--delet	Deletes an existing user.
--disable	Disables the user (turn on the "disabled" flag of the user). The users that are disabled are not able to log in.
--enable	Enables the user. This will clear the "disabled" flag of the user so that the user will be able to log in again. The "suspended" flag will also be cleared, the failed login counter set to 0 and the date and time of the last failed login gets deleted, so users who are suspended for failed login attempts will also be able to log in.
--list	List the users registered for the ClusterControl controller.
--list-	List the user groups maintained by the ClusterControl controller.
--list-	Lists the public keys registered in the controller for the specified user. Please note that viewing the public keys require special privileges, ordinary users can not view the public keys of other users.
--prefe	To add/set/delete the preferences for a given --cmon-user. For example the preferences for UI. Is used with other input options like --preferences-to-set or --preferences-to-delete options.
--get-p	To get the preferences for a given --cmon-user.
--passv	Resets the password for the user using the "forgot password" email schema. This option must be used twice to change the password, once without a token to send an email about the password reset and once with the token received in the email.
--set	Changes the specified properties of the user.
--remov	Removes the user from a group.
--set-g	Sets the primary group for the specified user. The primary group is the first group the user belongs to. This option will remove the user from this primary group and add it to the group specified by the --group command-line option.
--stat	Prints detailed information about the specified user(s).
--whoam	Same as --list, but only lists the current user, the user that authenticated on the controller.

Options

Name, shorthand	Description
--cmon-user,	Username on the CMON system
--group=GROU	Set the name of the group. For example, when a new user is created this option can be used to control what will be the primary group of the new user. It is also possible to filter the users by the group name while listing them.
--create-grc	If this command-line option is provided and the group for the new user does not exist the group will be created together with the new user.
--first-name	Sets the first name of the user.
--last-name=	Sets the last name of the user.
--public-key	The name of the file where the public key is stored. Please note that this currently only works with the --add-key option.
--preference	List of a given --cmon-user preferences to insert/update to/in the cmon database. It is used with --preferences main option. The LIST consists of the key=value pairs separated by a semicolon (;).
--preference	List of a given --cmon-user preferences to delete from the cmon database. It is used with --preferences main option. The LIST consists of the keys separated by a semicolon (;).
--title=TITLE	The title prefix (e.g. Dr.) for the user.
--email-addr	The email address for the user.
--new-passwc	The new password when changing the password.
--old-passwc	The old password when changing the password.
--user-forma	The string controls the format of the printed information about the users. See *User Formatting*.
--without-ta	When listing the users this option can be used to limit the list to those users that have none of the specified tags set.
--with-tags=	When listing the users this option can be used to limit the list to those users that have at least one of the tags set.

Creating the First User

To use the s9s command line tool a Cmon User Account is needed to authenticate on the Cmon Controller. These user accounts can be created using the s9s program itself by either authenticating with a pre-existing user account or bootstrapping the user management to create the very first user. The following section describes the authentication and the creation of the first user in detail.

If there is a username specified either in the command line using the --cmon-user (or -u) options or in the configuration file (either the ~/.s9s/s9s.conf or the /etc/s9s.conf file using the cmon_user variable name) the program will try to authenticate with this username. Creating the very first user account is of course not possible this way. The --cmon-user option and the cmon_user the variable is not for specifying what user we want to create, it is for setting what user we want to use for the connection.

If no user name is set for the authentication and user creation is requested the s9s will try to send a request to the controller through a named pipe. This is the only way a user account can be created without authenticating with an existing user account, this is the only way the very first user can be created. Here is an example:

```
s9s user --create \
       --group=admins \
       --generate-key \
       --controller=https://192.168.1.127:9501 \
       --new-password="MyS3cr3tpass" \
       --email-address="user@example.com" \
       admin
```

Consider the following:

1. There is no `--cmon-user` specified, this is the first user we create, we do not have any pre-existing user account. This command line is for creating the very first user. Please check out the *Examples* section to see how to create additional users.
2. This is the first run, so we assume no `~/.s9s/s9s.conf` configuration file exists, there is no user name there either.
3. In the example, we create the user with the username "admin", and it is the command-line argument of the program.
4. The command specifies the controller to be at `https://192.168.1.127:9501`. The HTTPS protocol will be used later, but to create this first user the s9s will try to use SSH and sudo to access the controller's named pipe on the specified host. For this, to work the UNIX user running this command has to have a passwordless SSH and sudo set up to the remote host. If the specified host is the local host the user does not need SSH access, but still needs to be root or have a passwordless sudo access because of course, the named pipe is not accessible for anyone.
5. Since the UNIX user has no s9s configuration file it will be created. The controller URL and the user name will be stored in it under `~/.s9s/s9s.conf`. The next time this user runs the program, it will use this "admin" user unless another user name is set in the command line and will try to connect this controller unless another controller is set in the command line.
6. The password will be set for the user on the controller, but the password will never be stored in the configuration file.
7. The `--generate-key` the option is provided, so a new private/public key pair will be generated, then stored in the `~/s9s/` directory and the public key will be registered on the controller for the new user. The next time the program runs it will find the username in the configuration file, find the private key in place for the user, and will automatically authenticate without a password. The command-line options will always have precedence, so this automatic authentication is simply the default way, the password authentication is always available.
8. The group for the new user is set to "admins", so this user will have special privileges. It is always a good idea to create the very first user with special privileges, then other users can be created by this administrator account.

User List

Using the `--list` and `--long` command-line options a detailed list of the users can be printed. Here is an example of such a list:

```
$ s9s user --list --long worf jadzia
A ID UNAME  GNAME EMAIL           REALNAME
- 11 jadzia ds9   dax@ds9.com     Lt. Jadzia Dax
```

(continues on next page)

(continued from previous page)

```
A 12 worf   ds9   warrior@ds9.com Lt. Worf
Total: 12
```

Please note that there are a total of 12 users defined on the system, but only two of those are printed because we filtered the list with the command line arguments.

The list contains the following fields:

Field	Description
A	Shows the authentication status. If this field shows the letter 'A' the user is authenticated with the current connection.
ID	Shows the user ID, a unique numerical ID identifying the user.
UNAME	The username.
GNAME	The name of the primary group of the user. All user belongs to at least one group, the primary group.
EMAIL	The email address of the user.
REALNAME	The real name of the user consists of first name, last name, and some other parts, printed here as a single string composed of all the available components.

User Formatting

When this command-line option is used the specified information will be printed instead of the default columns. The format string uses the `%` character to mark variable fields and flag characters as they are specified in the standard `printf()` C library functions. The '`%`' specifiers are ended by field name letters to refer to various properties of the users.

The `%+12I` format string for example has the `+12` flag characters in it with the standard meaning: the field will be 12 characters wide and the + or − the sign will always be printed with the number. The properties of the user are encoded by letters. For example, the letter `N` encodes the username field, so `username` of the user will be substituted. Standard `\` the notation is also available, `\n` for example, encodes a new-line character. The `%16N` for example, the letter `N` encodes the username field, so `username` of the user will be substituted. Standard `\` notation is also available, `\n` for example, encodes a new-line character.

The `s9s` command supports the following fields:

Field	Description
d	The distinguished name of the user. This currently has meaning only for users originated from an LDAP server.
F	The full name of the user.
f	The first name of the user.
G	The names of groups the given user belongs to.
I	The unique numerical ID of the user.
j	The job title of the user.
l	The last name of the user.
M	The email address of the user.
m	The middle name of the user.
N	The username for the user.
o	The origin of the user, the place that used to store the original instance of the user. The possible values are "CmonDb" for users from the Cmon Database or "LDAP" for users from the LDAP server.
P	The CDT path of the user.
t	The title of the user (e.g. "Dr.").

Examples

- Create a remote s9s user and generate an SSH key for the user:

```
s9s user --create \
        --generate-key \
        --cmon-user=dba \
        --controller="https://10.0.1.12:9501"
```

- List out all users:

```
$ s9s user --list --long
A ID UNAME      GNAME  EMAIL REALNAME
-  1 system     admins -     System User
-  2 nobody     nobody -     -
A  3 dba        users  -     -
-  4 remote_dba users  -     -
```

- List users that have at least one of the tags set.

```
s9s user \
        --list \
        --long \
        --with-tags="ds9;tng"
```

- Register a new public key for the active user:

```
s9s user \
        --add-key \
        --public-key-file=/home/pipas/.ssh/id_rsa.pub \
        --public-key-name=The_SSH_key
```

- Add user "myuser" into group "admins":

```
s9s user \
        --add-to-group \
        --group=admins \
        myuser
```

- Set a new password for user "pipas":

```
s9s user \
        --change-password \
        --new-password="MyS3cr3tpass" \
        pipas
```

- Create a new user called "john" under group "dba":

```
s9s user \
        --create \
        --group=dba \
        --create-group \
        --generate-key \
        --new-password=s3cr3tP455 \
        --email-address=johndoe@somewhere.com \
        --first-name=John \
        --last-name=Doe \
        --batch \
        john
```

- Delete an existing user called "mydba":

```
s9s user \
        --delete \
        mydba
```

- Disable user nobody:

```
s9s user \
        --cmon-user=system \
        --password=secret \
        --disable \
        nobody
```

- Enable user nobody:

```
s9s user \
        --cmon-user=system \
        --password=secret \
        --enable \
        nobody
```

- Reset the password for a user called "dba". One has to obtain the one-time token which will be sent to the registered email address of the corresponding user, followed by the actual password modification command with `--token` and `--new-password` parameters:

```
s9s user \
        --password-reset \
        dba
## check the mail inbox of the respective user
s9s user \
        --password-reset \
        --token="98197ee4b5584cedba88ef1f583a1258" \
        --new-password="newp455w0rd"
        dba
```

- Set a primary group for user "dba":

```
s9s user \
        --set-group \
        --group=admins \
        dba
```

- To add/set/delete the preferences for a given `--cmon-user`.

```
s9s user --cmon-user=s9s --password=pass1299 --preferences --
↪preferences-to-set="key1=value1;key2=value2;key3=value3"
s9s user --cmon-user=s9s --password=pass1299 --preferences --
↪preferences-to-set="key2=QWERTY"
s9s user --cmon-user=s9s --password=pass1299 --preferences --
↪preferences-to-delete="key1;key2"
```

- To get the preferences for a given `--cmon-user`.

```
s9s user --cmon-user=s9s --password=***** --get-preferences
```

3.6 ClusterControl SSH

This optional package is introduced in ClusterControl v1.4.2. This module provides the ability to connect to the SSH console to any of your cluster hosts directly via ClusterControl UI. This can be very useful if you need to quickly log into a database server and access the command line.

The package installs a binary called `cmon-ssh` located under `/usr/sbin` directory and by default listens to port 9511 on the ClusterControl node. It interacts directly with the target host via SSH protocol using the credential (`os_user` and `ssh_identity`) configured when deploying or importing the cluster into ClusterControl.

Set up the RewriteRule inside the Apache configuration file `cc-frontend.conf` (above the `<Directory/>` definitions):

```
<Location /api/v2/cmon-ssh/cmon/ws/>
    RewriteEngine On
    RewriteCond %{REQUEST_URI} ^/cmon-ssh/cmon/ws/(.*)$
    RewriteCond %{REQUEST_URI} ^/api/v2/cmon-ssh/cmon/ws/(.*)$
    RewriteRule ^(.*)$ ws://127.0.0.1:9511/cmon/ws/%1 [P,L]
</Location>
<LocationMatch /api/v2/cmon-ssh/>
```

(continues on next page)

(continued from previous page)

```
    ProxyPass http://127.0.0.1:9511/
    ProxyPassReverse http://127.0.0.1:9511/
</LocationMatch>
```

Enable the following Apache modules:

```
$ a2enmod proxy proxy_http proxy_wstunnel
```

Communication is based on HTTPS, so it is possible to access your servers from behind a firewall that restricts Internet access to only port 443. Access to WebSSH is configurable by the ClusterControl admin through the GUI.

Warning

Warning!

ClusterControl does not provide extra layers of authentication and authorization when accessing the cluster from a web-based SSH terminal. The user who has access to the cluster in the ClusterControl UI may capable of accessing the terminal as a privileged user. Use the access privileges to limit them. See Users and Teams.

3.7 ClusterControl Notifications

This optional package is introduced in ClusterControl v1.4.2, deprecating the previous version of the ClusterControl NodeJS package (which served the same purpose). Alarms and events can now easily be sent to incident management services like PagerDuty, VictorOps, and OpsGenie. You can also run any command available in the *ClusterControl CLI* from your CCBot-enabled chat services like Slack and Telegram. Additionally, it provides a generic webhook if you want to integrate with other services to act on status changes in your clusters. The direct connections with these popular incident communication services allow you to customize how you are alerted from ClusterControl when something goes wrong with your database environments.

The package installs a binary called `cmon-events` located under `/usr/sbin` directory and by default listens to port 9510 on the ClusterControl node.

Additional resources on setting up integration with third-party tools are listed below:

Field	Description
Blogs	- Introducing the ClusterControl Alerting Integrations - Video: ClusterControl ChatOps Integrations – Product Demonstration
Documentation	- *Notification Services*

3.8 ClusterControl Cloud

This optional package is introduced in ClusterControl v1.5. The package name is `clustercontrol-cloud` and it provides a GUI extension in ClusterControl under *Cloud Provider Integrations*.

This package installs the following new files:

- `/etc/cron.d/cmon-cloud` – Cron's job to monitor `cmon-cloud` process.
- `/etc/rc.d/init.d/cmon-cloud` – sysvinit script.
- `/etc/systemd/system/cmon-cloud.service` – Systemd unit file.
- `/usr/sbin/cmon-cloud` – The executable binary file.

By default, this service will use port 9518 on localhost interface. Cloud credentials are stored under `/var/lib/cmon/` with permission set to root only.

Info

ClusterControl Cloud is a reintroduction of a feature called "Service Providers", available in ClusterControl 1.4 and older.

3.8.1 Commands

Name, Shorthand	Description
upload	Upload a file to the cloud.
download	Download a file from the cloud.
rm	Delete a file on the cloud.
clouds	List supported cloud providers.
help, h	Shows a list of commands or help for one command.

3.8.2 Options

Shorthand, Option	Description
`-cloud`	Use this cloud for the action.
`--service`	Use this service within the cloud.
`--credentials`	Raw credentials JSON.
`--credentials-file`	Take credentials from this file instead of using STDIN input.
`--auto-create-bucket`	Use this flag if the bucket should be auto-created if not exists.
`--help, -h`	Show help.
`--version`	Print the version.

3.9 ClusterControl Cloud File Manager

This optional package is complimentary of *ClusterControl Cloud* and is introduced in ClusterControl v1.5. The package name is `clustercontrol-clud` (an abbreviation of **CL**oud **U**pload **D**ownload) and is the command-line interface for ClusterControl to interact with the cloud providers when uploading and downloading backups. You can think of it as a cloud file manager. No extra configuration is required.

This package installs the following file:

- `/usr/sbin/clud` – The executable binary file.

This tool is used by ClusterControl in the following scenarios:

1. After an on-demand or scheduled database backup has been successfully taken and if it is configured to be uploaded to the cloud, when **Upload backup to cloud** is toggled on.
2. When you want to manually upload an existing backup to the cloud at ***Backups → choose a backup → Actions → Upload***.
3. When you want to download an existing backup from the cloud to the host (controller or database node) at ***Backups → choose a backup → Actions → Download***.

3.9.1 Usage

The general synopsis to execute commands using `clud` is:

```
clud {global options} command {command option} {arguments...}
```

3.9.2 Commands

Name, shorthand	Description
upload	Upload a file to the cloud.
download	Download a file from the cloud.
rm	Delete a file on the cloud.
clouds	List supported cloud providers.
help, h	Shows a list of commands or help for one command.

3.9.3 Options

Name, shorthand	Description
`--cloud`	Use this cloud for the action.
`--service`	Use this service within the cloud.
`--credentials`	Raw credentials JSON.
`--credentials-file`	Take credentials from this file instead of using STDIN input.
`--auto-create-bucket`	Use this flag if the bucket should be auto-created if not exists.
`--help, -h`	Show help.
`--version`	Print the version.

3.9.4 Examples

List out the supported cloud providers:

```
$ clud clouds
SUPPORTED CLOUD PROVIDERS:
     aws - Amazon Web Services     gce - Google Cloud     az - Microsoft␣
↪Azure     s3 - Any S3-Compatible Cloud
```

Commonly, ClusterControl performs the upload to cloud task by invoking the following command:

```
/usr/sbin/clud \
    --cloud <supported_cloud_provider> \
    --credentials-file '<path_to_credential.json>' \
```

(continues on next page)

(continued from previous page)

```
    --service unknown \
    --auto-create-bucket \
    upload <bucket_name> <source_file_path> <destination_file_path_in_cloud>
```

The following is an example to upload a file to S3-compatible object storage:

```
$ /usr/sbin/clud --cloud s3 --credentials-file 'mycreds.json' --service␣
↪unknown --auto-create-bucket upload mybucket1 my_data.sql␣
↪folder1/my_data.sql
UPLOAD COMPLETE
Bucket: mybucket1
Location: folder1/my_data.sql
```

Where the content of `mycreds.json` is:

```
{
  "access_key_id": "AKIAIOSFODNN7EXAMPLE",
  "access_key_secret": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
  "endpoint": "s3.us-east-1.amazonaws.com",
  "region": "us-east-1"
}
```

3.10 ClusterControl Proxy

ClusterControl Proxy is the next-generation ClusterControl manager which creates a unified view of multiple ClusterControl controllers and providing web services, effectively removing dependencies for Apache or nginx web servers to serve ClusterControl GUI components. It is developed to purposedly handle the ClusterControl GUI component provided by package `clustercontrol-mcc`.

Having said that, when installing ClusterControl GUI (package name: `clustercontrol-mcc`), ClusterControl Proxy (package name: `clustercontrol-proxy`) is expected be installed together.

3.10.1 ClusterControl Manager

The ClusterControl Manager binary called ccmgr is the main process (default port: 19501) which serves the web contents of ClusterControl GUI and acts as a proxying layer to all ClusterControl controllers added into it. The process is controlled by a systemd unit file located at `/etc/systemd/system/cmon-proxy.service`. Hence, the service management should follow the standard systemd format:

- Starting the service:

```
systemctl start cmon-proxy
```

- Stopping the service:

```
systemctl stop cmon-proxy
```

- Restart the service:

```
systemctl restart cmon-proxy
```

The main binary is installed at `/usr/bin/ccmgr` and comes with the following options:

Option	Description
`--debug-cmon-rpc`	Debug log RPC requests to cmon
`--debug-web-rpc`	Debug web RPC requests to cmon-proxy
`--basedir=`	The basedir of configuration (default: `/usr/share/ccmgr/`)

This binary is expected to be running as a service. Use *ClusterControl Manager Admin CLI* to manage the ClusterControl Manager.

3.10.2 ClusterControl Manager Admin CLI

The `ccmgradm` is a command-line interface to manage ClusterControl Manager users, passwords and controllers. Run `ccmgradm --help` to see all available commands.

Example

```
$ ccmgradm  --help
ClusterControl Manager - admin CLI v2.2
Usage: ccmgradm <command> [<args>]

Options:
--help, -h              display this help and exit

Commands:
dropuser
adduser
setpassword
dropcontroller
addcontroller
init
updatecontroller
listcontrollers
enableMcc
```

For every available command, there is a further help instruction:

Example

```
$ ccmgradm adduser --help
ClusterControl Manager - admin CLI v2.2
Usage: ccmgradm adduser [--email EMAIL] [USERNAME [PASSWORD]]

Positional arguments:
USERNAME
PASSWORD

Options:
```

```
--email EMAIL, -e EMAIL
--help, -h              display this help and exit
```

You may use the `ccmgradm` client to interact with the `ccmgr` service or use the ClusterControl Ops-C web GUI client to manage users and controllers.

3.10.3 Configuration file

By default, ClusterControl Proxy comes with a default configuration in YAML format, available at `/usr/share/ccmgr/ccmgr.yaml`:

```
filename: /usr/share/ccmgr/ccmgr.yaml
webapproot: /var/www
fetch_jobs_hours: 12
fetch_backups_days: 7
timeout: 30
logfile: /var/log/ccmgr.log
frontend_path: /app
port: 19051
tls_cert: /usr/share/ccmgr/server.crt
tls_key: /usr/share/ccmgr/server.key
session_ttl: 1800000000000
single_controller: ""
```

The following is an example content of the configuration file with one ClusterControl instance:

```
filename: ccmgr.yaml
instances: # List of controller
    - xid: chebjd8gfi863qqhrolg # Controller id. Generated automatically
      url: hostname:9443/api # Controller api URL
      name: Controller name
      username: admin
      password: password
      frontend_url: http://hostname:9443/ # Url of Cluster Control web␣
↪interface
      cmon_ssh_host: 127.0.0.1:9511 # cmon-ssh host and port, used for web␣
↪ssh console. Default for single controller - 127.0.0.1:9511
      cmon_ssh_secure: false # if true - use TLS for cmon-ssh connection
    - xid: cnoi4d3fo0o9e9m7hap0 # Controller id. Generated automatically
      # ....
timeout: 30 # request timeout in seconds. Default 30
logfile: ccmgr.log # cmon proxy log file. Default ccmgr.log
users: # Cmon proxy users, can be set via ccmgradm setpassword <username>
↪<password>
    - username: admin
      passwordhash: <hash>
frontend_path: /app # Path to static files that cmon proxy can serve
port: 19051
tls_cert: server.crt # Default server.crt can be set from env␣
↪TLS_CERTIFICATE_FILE variable
```

(continues on next page)

(continued from previous page)

```
tls_key: server.key # Default server.key can be set from env TLS_KEY_FILE␣
↪variable
session_ttl: 3600000000000 # Session time to live in nanoseconds. Default 1␣
↪hour
fetch_backups_days: 7 # How many days in the past cmon-proxy should look␣
↪for backup data. Default is 7 days
fetch_jobs_hours: 12 # How many hours in the past cmon-proxy should look␣
↪for jobs data. Default is 12 hours
single_controller: chebjd8gfi863qqhrolg # ID of single controller, if it is␣
↪set - multi-controller is disabled
k8s_proxy_url: http://127.0.0.1:8080 # Kubernetes proxy URL. Default is␣
↪http://127.0.0.1:8080
kubernetes_enabled: true # Enable Kubernetes proxy. Default is true
license_proxy_url: "https://severalnines.com/service/lic.php" # URL to␣
↪request demo license during registration. Default -␣
↪https://severalnines.com/service/lic.php
http_port: 80 # Port for the plain HTTP server used for ACME challenges and␣
↪redirection to HTTPS. Default is 80

# Let's Encrypt settings for automatic TLS certificates
acme_enabled: false # Enable Let's Encrypt. Default is false
acme_staging: false # Use Let's Encrypt staging environment. Default is␣
↪false
acme_domains: [] # List of domains to obtain certificates for.
acme_email: "" # Email address for Let's Encrypt registration and␣
↪notifications.
acme_cache_dir: "autocert-cache" # Directory to cache ACME certificates.␣
↪Default is "autocert-cache" in the base directory.
acme_directory_url: "" # The ACME directory URL. Defaults is empty (using␣
↪library pre-defined), or staging␣
↪(https://acme-staging-v02.api.letsencrypt.org/directory) if acme_staging␣
↪is true.
acme_accept_tos: true # Automatically accept the ACME provider's Terms of␣
↪Service. Default is true.
acme_renew_before: "720h" # Renewal window before certificate expiration (a␣
↪Go duration string, e.g., "720h" for 30 days). Default is 30 days (720h).
acme_host_policy_strict: false # If true, strictly enforce that certs are␣
↪only issued for domains in acme_domains. Recommended for production.␣
↪Default is false.
web_server:
  trusted_proxies: [] # List of trusted proxy IPs. Default is empty.
  trusted_platform: "" # Platform header for identifying client IPs (e.g.,
↪"X-Forwarded-For"). Default is empty.
  security:
    frame_deny: true # Sets X-Frame-Options to DENY. Default is true.
    sts_seconds: 31536000 # HSTS max-age in seconds. Default is 31536000 (1␣
↪year).
    sts_include_subdomains: true # Include subdomains in HSTS policy.␣
↪Default is true.
```

(continues on next page)

(continued from previous page)

```
    sts_preload: true # Enable HSTS preload. Default is true.
    force_sts_header: true # Force HSTS header even for non-TLS requests.␣
↪Default is true.
    content_type_nosniff: true # Sets X-Content-Type-Options to nosniff.␣
↪Default is true.
    browser_xss_filter: false # Disables the browser's XSS filter. Default␣
↪is false.
    content_security_policy: "default-src 'self'; ..." # Content Security␣
↪Policy. See below for details.
    content_security_policy_report_only: true # Use CSP in report-only␣
↪mode. Default is true.
    referrer_policy: "strict-origin-when-cross-origin" # Referrer-Policy␣
↪header. Default is "strict-origin-when-cross-origin".
    permissions_policy: "camera=(), microphone=(), geolocation=()" #␣
↪Permissions-Policy header. Default is "camera=(), microphone=(),␣
↪geolocation=()".
  cors:
    allow_origins: [] # List of allowed origins for CORS. Default is empty.
    allow_methods: [] # List of allowed methods for CORS. Default is empty.
    allow_headers: [] # List of allowed headers for CORS. Default is empty.
    expose_headers: [] # List of headers to expose for CORS. Default is␣
↪empty.
    allow_credentials: true # Allow credentials for CORS. Default is true.
    max_age_seconds: 0 # Max age for preflight requests. Default is 0.
  gzip:
    level: 1 # Gzip compression level (1 to 9). Default is 1 (BestSpeed).
  frontend:
    nonce_replacement_files: ["index.html"] # Files to replace CSP nonce␣
↪in. Default is ["index.html"].
```

3.10.4 Configure custom SSL certificates

The default directory `/usr/share/ccmgr` shall contain the configuration files and TLS certificates and keys, as well as the logs of the `cmon-proxy` application. You can put your own TLS cert (a combined full chain, so the main cert and all the CAs together, or just a self signed one) and key into your data directory using the following file names - `server.crt` and `server.key`.

3.10.5 Web server: headers and simple examples

The ClusterControl configuration supports security headers which can be configured in the `ccmgr.yaml`. This section simplifies `web_server` settings and details the headers they add. These settings do not change the app logic; they only affect how requests are interpreted and what security headers are sent.

Content Security Policy (CSP)

The Content Security Policy (CSP) limits where scripts, styles, and images can load from. The inline scripts can be safely allowed with support for a per-response nonce and it can run in report-only mode while you test your policy (report-only by default).

How to apply the configuration:

- Configure `web_server.security.content_security_policy` as a string. Use `{{nonce}}` where a nonce should appear (e.g., in `script-src`).
- For HTML files listed in `web_server.frontend.nonce_replacement_files` (default: index.html), the server:
  1. Generates a fresh nonce per request
  2. Replaces **NONCE** in your HTML with that nonce
  3. Sends CSP header with {{nonce}} replaced
  4. Disables caching for those HTML responses
- If `content_security_policy_report_only: true`, the header is sent as `Content-Security-Policy-Report-Only` and the directive `upgrade-insecure-requests` is removed automatically for compatibility.

Example configuration and usage:

```
web_server:
  security:
    content_security_policy: "default-src 'self'; script-src 'self'
↪'nonce-{{nonce}}' 'strict-dynamic'; style-src 'self' 'unsafe-inline';␣
↪img-src 'self' data:;"
    content_security_policy_report_only: true
  frontend:
    nonce_replacement_files: ["index.html"]
```

```
<!-- index.html -->
<script nonce="__NONCE__">/* inline boot script */</script>
```

Resulting header (report-only example):

```
Content-Security-Policy-Report-Only: default-src 'self'; script-src 'self'
↪'nonce-<random>' 'strict-dynamic'; style-src 'self' 'unsafe-inline';␣
↪img-src 'self' data:
```

CORS (Cross-Origin Resource Sharing)

The CORS is a security mechanism built into web browsers that controls how resources (like APIs, images, fonts, or scripts) can be requested from a domain different from the one that served the original web page. If the frontend (browser app) is served from a different origin (domain/port) than this API.

Example configuration:

```
web_server:
  cors:
    allow_origins: ["https://ui.example.com"]
    allow_methods: ["GET", "POST", "PUT", "PATCH", "DELETE"]
    allow_headers: ["Content-Type", "Authorization"]
    expose_headers: ["X-Request-Id"]
    allow_credentials: true
    max_age_seconds: 600
```

Typical preflight (OPTIONS) response:

```
Access-Control-Allow-Origin: https://ui.clustercontrol.com
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE
Access-Control-Allow-Headers: Content-Type,Authorization
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 600
```

HTTPS enforcement (HSTS)

The HSTS is a web security policy mechanism that tells browsers to always use HTTPS (encrypted connection) when communicating with a website. Without HSTS, if a client types http://example.com (or clicks an old http link), the browser will first connect over HTTP, then get redirected to HTTPS. That short HTTP step is vulnerable to `man-in-the-middle (MITM)` attacks, where an attacker could intercept or modify the redirect.

Example of headers sent:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

The `max-age` for 1 year (31536000) indicate the browser must only use HTTPS for the site and `includeSubDomains` configuration apply to all the subdomains.

3.11 ClusterControl Agent

Also known as the **query monitoring agent**. Deploys agents for monitoring the database nodes, particularly the database queries. It is only available for MySQL and PostgreSQL database systems.

Features you can use with Query Monitor/Agent are the following:

- Install/remove query monitor agents on each database node. You can remove these agents later on.
- Start/stop collecting query stats with the agents.
- New 'Query Workload' overview showing query digests, latency, throughput, and concurrency with a scatter chart.

3.11.1 Query Monitor Agents

ClusterControl GUI Install agent

Log in to your ***ClusterControl GUI → choose a database cluster → Performance → Query Monitor/Agents***.When you click the Install monitoring agent button, ClusterControl will set up and install an agent called `cmnd` to all of the database nodes in that particular cluster.

ClusterControl CLI Install agent

For CLI, we will use the `s9s cluster` option: Deploys CMON agents to all nodes in cluster ID 1. (This enables TopQuery Monitoring functionality for the cluster nodes.):

```
s9s cluster \
        --deploy-cmonagents \
        --cluster-id=1 \
        --print-request \
        --log \
        --wait
```

Uninstalls CMON agents from specified nodes in cluster ID 1. (If no node is specified, CMON agents will be uninstalled from all nodes in the cluster.)

```
s9s cluster \
        --uninstall-cmonagents \
        --cluster-id=1 \
        --nodes=10.67.199.164 \
        --print-request \
        --log \
        --wait
```

Agent Configuration

The agent has the following configuration which you can find in `/etc/cmnd.conf`. This is for the agent configuration. `/etc/cmnd.d` directory for the Top Query configuration. The agent configuration file `/etc/cmnd.conf` has all the details you can find here. By default, these are the following options that are set:

```
$cat /etc/cmnd.conf | sed -e 's/#.*$//' -e '/^$/d' -e '/^$/N;/^\n$/D' | sed
↪'/^\t*$/d'

[cmnd]
data_directory = "/var/lib/cmnd"
log_file = "/var/log/cmnd.log"
pid_file = "/var/run/cmnd.pid"
plugin_names = [
"libpluginMySql.so",
"libpluginSqlite.so",
"libpluginPgSql.so"
]
port = 4433
cert_file = "/etc/ssl/cmnd/cert.pem"
key_file = "/etc/ssl/cmnd/key.pem"
```

You can change the port to your desired port number based on the firewall policy you have in your organization. So take note, you need to have port **4433** open by default to make it work properly.

You can find more of its configuration files under these files/directories:

```
$ find /etc/ -name "cmn*"

/etc/ssl/cmnd
/etc/logrotate.d/cmnd
/etc/cmnd.conf
/etc/cmnd.d
/etc/cmn.passwd
```

The Query Monitor Agents are installed using systemd, allowing you to start and stop the service using the `systemctl` command. To determine if the agents are running, you can do the following:

```
systemctl status cmon-daemon
```

Or verify it with the `netstat` just like below:

```
$ netstat -ntlvp46 | grep cmnd

tcp 0 0 0.0.0.0:4433 0.0.0.0:* LISTEN 8456/cmnd
```

Query Monitoring Workflow

The agent collects samples from the `performance_schema` (MySQL/MariaDB) or `pg_stat_statements` (PostgreSQL/TimescaleDB). These metrics which is collected by a database client and send the query inquiry to the agent. Then, the agent that reads the input data will produce the query statistics and make it available to the Query Monitor's Overview dashboard query metrics. That means all collected information from the Query Monitoring shall depend upon the following conditions:

1. The reply should be sent in a timely fashion no matter how big the input data is. Should the computer be very slow or the processing is too complicated, the query statistics should be produced and made available in a progressive manner so a responsive UI can be implemented.
2. The agent should be able to update the *query statistics* on itself when the *input data* is updated by the database server. A simple query or RPC call is invoked so that the agent keeps monitoring the input data and keeps the data updated.
3. The *query statistics* should be according to the *query inquiry* and the *input data*. In simpler words, the produced statistical data should be in sync with the input data and the query the user sent. Some query statistics may be derived from the input data.

3.12 ClusterControl CMON RPC v2 API Reference

3.12.1 Introduction

This is the documentation of the RPC v2, a new RPC interface for the Cmon Controller that exists beside the original (or v1) interface. The two versions of the RPC interface are independent from each other, one can be used without the services of the other.

Please note that the RPC v2 API calls are available only using secure (TLS) connections, encryption is mandatory. The RPC v2 also uses an authentication call, very little of the calls are available to be used without authenticating first.

By default the encrypted communication can be used through port 9501 and the RPC v2 requests are served on the `/v2` path (e.g. `/v2/clusters`).

List of RPC v2 paths:

- `/v2/auth`: Authentication.
- `/v2/alarm`: Information about alarms.
- `/v2/users`: Handling Cmon users.
- `/v2/backup`: Creating and restoring backups.
- `/v2/clusters`: Information about clusters.
- `/v2/host`: Information about hosts.
- `/v2/stat`: Statistical information, measurements.
- `/v2/jobs`: Creating and monitoring jobs.
- `/v2/maintenance`: Maintenance periods.

- `/v2/metatype`: Information about types used by the controller.
- `/v2/config`: Configuration.
- `/v2/process`: Processes.
- `/v2/proxysql`: ProxySQL
- `/v2/log`: Logging.
- `/v2/imperative`: Maintaining and running scripts.
- `/v2/discovery`: Collecting real-time data.
- `/v2/cloud`: Managing cloud credentials and proxying to cmon-cloud.
- `/v2/audit`: List the audit log of the controller.
- `/v2/ca`: Certificate authority.
- `/v2/dbversions`: Get supported database versions for specific cluster type & vendor.
- `/v2/performance`: Get information about MySQL databaseperformance.
- `/v2/logical_replication`: PostgreSQL Logical Replication.
- `/v2/watchlists`: Manipulate whatchlists to visualize clusters according to user configuration.

Here is an example showing a typical RPC v2 request:

```
{
    "cluster_name": "ft_galera_18262",
    "operation": "getClusterInfo",
    "request_id": 2,
    "request_created": "2017-06-22T06:13:46.393Z",
}
```

Most of the requests contain at least the following fields:

- `operation`: This field identifies what kind of action the controller should take to serve the request.
- `request_id`: Optional field that will be sent back with the reply.
- `request_created`: This is an optional field which will be sent back to the client with the reply.
- `cluster_name`: Many of the requests are closely related to one cluster and that cluster can be identified by the cluster ID or the cluster ID.
- `cluster_name`: Many of the requests are closely related to one cluster and that cluster can be identified by the cluster ID or the cluster ID.

The RPC v2 reply usually contains many fields, but some fields can be found in all the replies. Here is an example showing some of the fields that are always sent with the reply:

```
{
    "request_id": 2,
    "request_created": "2017-06-22T06:13:46.339Z",
    "request_processed": "2017-06-22T06:13:46.385Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `messages`: A list of strings with human readable messages describing what happened while processing the request.
- `error_string`: A human readable error message describing the error if an error happened. Should a request result in more than one message strings the "messages" field will be sent.
- `request_id`: If the request held this field it will be simply sent back in the reply.
- `request_created`: This is just a field that is sent back by the controller without modification.
- `request_processed`: The timestamp shows what time the reply was created.
- `request_status`: A string showing if the request was successfully processed and the error code if not. The possible values are:
  - Ok: The request was successfully processed.
  - InvalidRequest: Something was fundamentally wrong with the request.
  - ObjectNotFound: The referenced object (e.g. the cluster) was not found.
  - TryAgain: The request cannot be processed at this time.
  - UnknownError: The exact error could not be identified.
  - AccessDenied: The authenticated user has insufficient rights.
  - AuthRequired: The client has to authenticate first.
- `request_user_id`: The numeric ID of the authenticated user. Using this field the client can identify who sent the request.

3.12.2 Alarms

getStatistics

The "getStatistics` returns statistical information about alarms for one or more clusters, including a count of active critical and warning alarms. This is a quick way to check whether there are active alarms without fetching the full alarm list.

Most importantly this is a quick way to check if there are active alarms on the cluster.

```
{
    "cluster_id": 1,
    "operation": "getStatistics",
    "request_created": "2017-08-02T05:22:29.566Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster to check. The cluster can also be identified by the cluster name.
- `cluster_name`: The name of the cluster to check. The cluster ID can also be used to identify the cluster.
- `cluster_ids`: Multiple clusters can also be requested in one call.

```
{
    "alarm_statistics": [
    {
```

(continues on next page)

(continued from previous page)


```
        "class_name": "CmonAlarmStatistics",
        "cluster_id": 1,
        "critical": 2,
        "warning": 1
    } ],
    "request_created": "2017-08-02T05:22:29.566Z",
    "request_id": 3,
    "request_processed": "2017-08-02T05:22:29.615Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


getAlarm

Get information about one particular alarm.


```
{
    "alarm_id": 1,
    "operation": "getAlarm",
    "request_created": "2017-08-02T04:52:33.983Z",
    "request_id": 3
}
```


- `alarm_id`: The numerical ID of the alarm to return.
- `cluster_id`: This field is not required for the call, but if provided it will control which cluster to read.
- `cluster_name`: This field is not required for the call, but if provided it will control which cluster to read.


```
{
    "alarm":
    {
        "alarm_id": 1,
        "class_name": "CmonAlarm",
        "cluster_id": 1,
        "component": 3,
        "component_name": "Cluster",
        "counter": 1,
        "created": "2017-08-02T04:50:52.000Z",
        "ignored": 0,
        "measured": 0,
        "message": "System time is drifting between servers and the␣
↪distance between the highest system time and lowest is more than one␣
↪minute.",
        "recommendation": "Synchronize the system clock on the servers␣
↪using e.g NTP or make sure NTP is working. Time drifting may lead to␣
↪unexpected failures or problems, and makes debugging hard.\n\nThe last␣
↪seen host time values (in controller's time-zone):\n192.168.1.127: Aug 02␣
↪06:49:55\n192.168.1.199: Aug 02 06:51:10\n",
```


(continues on next page)

(continued from previous page)


```
        "reported": "2017-08-02T04:50:52.000Z",
        "severity": 1,
        "severity_name": "ALARM_WARNING",
        "title": "System time is drifting",
        "type": 40000,
        "type_name": "ClusterTimeDrift"
    },
    "request_created": "2017-08-02T04:52:33.983Z",
    "request_id": 3,
    "request_processed": "2017-08-02T04:52:34.030Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `alarm`: A `CmonAlarm` object with the details of one particular alarm.

getAlarms

Get information about multiple alarms. Here is an example request:


```
{
  "class_name": "CmonRpcRequest",
  "cluster_id": 1,
  "operation": "getAlarms",
  "request_created": "2021-06-03T05:25:49.100Z",
  "request_id": 3
}
```


Here is an example reply.


```
{
    "alarms": [
    {
        "alarm_id": 1,
        "class_name": "CmonAlarm",
        "cluster_id": 1,
        "component": 3,
        "component_name": "Cluster",
        "counter": 1,
        "created": "2017-07-27T06:33:38.000Z",
        "ignored": 0,
        "measured": 0,
        "message": "System time is drifting between servers and the␣
↪distance between the highest system time and lowest is more than one␣
↪minute.",
        "recommendation": "Synchronize the system clock on the servers␣
↪using e.g NTP or make sure NTP is working. Time drifting may lead to␣
↪unexpected failures or problems, and makes debugging hard.\n\nThe last␣
↪seen host time values (in controller's time-zone):\n192.168.1.127: Jul 27␣
↪08:32:40\n192.168.1.169: Jul 27 08:33:44\n",
```


(continues on next page)

(continued from previous page)


```
        "reported": "2017-07-27T06:33:38.000Z",
        "severity": 1,
        "severity_name": "ALARM_WARNING",
        "title": "System time is drifting",
        "type": 40000,
        "type_name": "ClusterTimeDrift"
    },
    . . .
    {
        "alarm_id": 3,
        "class_name": "CmonAlarm",
        "cluster_id": 1,
        "component": 0,
        "component_name": "Network",
        "counter": 12,
        "created": "2017-07-27T06:35:56.000Z",
        "host_id": 1,
        "hostname": "192.168.1.169",
        "ignored": 0,
        "measured": 0,
        "message": "Server 192.168.1.169 reports: Host 192.168.1.169 is not␣
↪responding to ping after 3 cycles, the host is most likely unreachable.",
        "recommendation": "Restart failed host, check firewall.",
        "reported": "2017-07-27T06:35:56.000Z",
        "severity": 2,
        "severity_name": "ALARM_CRITICAL",
        "title": "Host is not responding",
        "type": 10006,
        "type_name": "HostUnreachable"
    } ],
    "cluster_id": 1,
    "request_created": "2017-07-27T06:36:03.627Z",
    "request_id": 3,
    "request_processed": "2017-07-27T06:36:03.675Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `alarms`: A list of `CmonAlarm` objects with the details of the alarms.

ignoreAlarm

Sets the ignored state of an alarm. When `ignore` is `true` the alarm is suppressed; set it to `false` to re-enable a previously suppressed alarm.


```
{
    "alarm_id": 5,
    "ignore": true,
    "operation": "ignoreAlarm",
    "request_created": "2018-09-25T08:03:17.416Z",
```


(continues on next page)

(continued from previous page)

```
    "request_id": 3
}
```


```
{
    "alarm":
    {
        "alarm_id": 5,
        "class_name": "CmonAlarm",
        "cluster_id": 1,
        "component": 0,
        "component_name": "Network",
        "counter": 1,
        "created": "2018-09-25T08:03:17.000Z",
        "host_id": 1,
        "ignored": 1,
        "measured": 0,
        "message": "Server 192.168.0.79 reports: SSH failure to␣
↪192.168.0.79: No route to host.",
        "recommendation": "Please check SSH access to host. The host may␣
↪also be down.",
        "reported": "2018-09-25T08:03:17.000Z",
        "severity": 2,
        "severity_name": "ALARM_CRITICAL",
        "title": "SSH failed",
        "type": 10008,
        "type_name": "HostSshFailed"
    },
    "request_created": "2018-09-25T08:03:17.416Z",
    "request_id": 3,
    "request_processed": "2018-09-25T08:03:17.425Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```


3.12.3 Information about Controller's audit log

The `/v2/audit` path can be used to retrieve audit log entries for one or more clusters.

getEntries

Returns audit log entries for one or more clusters. Results can be sorted ascending or descending and filtered by cluster ID.

```
{
    "ascending": true,
    "cluster_ids": [1,3],
    "operation": "getEntries",
    "request_created": "2022-02-26T09:07:48.374Z",
    "request_id": 3
}
```

```
{
  "request_processed": "2022-03-03T18:24:38.304Z",
  "request_status": "Ok",
  "total": 4,
  "audit_entries":
  [
      {
          "class_name": "CmonAuditEntry",
          "client_hostname": "127.0.0.1",
          "cluster_id": 0,
          "entry_type": "users",
          "id": 1000,
          "message_text": "Deleted user 'user.name.123.",
          "report_ts": "2022-03-03T18:11:40.800Z",
          "username": "kedz"
      },
      {
          "class_name": "CmonAuditEntry",
          "client_hostname": "127.0.0.1",
          "cluster_id": 0,
          "entry_type": "users",
          "id": 1001,
          "message_text": "Group removed 'fancygroup.",
          "report_ts": "2022-03-03T18:11:35.498Z",
          "username": "kedz"
      },
      {
          "class_name": "CmonAuditEntry",
          "client_hostname": "127.0.0.1",
          "cluster_id": 0,
          "entry_type": "users",
          "id": 1002,
          "message_text": "User 'user.name.123' has been removed from group
↪'fancygroup.",
          "report_ts": "2022-03-03T18:11:29.324Z",
          "username": "kedz"
      },
      {
          "class_name": "CmonAuditEntry",
          "client_hostname": "127.0.0.1",
          "cluster_id": 0,
          "entry_type": "users",
          "id": 1003,
          "message_text": "User 'user.name.123' has been added to group
↪'fancygroup.",
          "report_ts": "2022-03-03T18:11:18.168Z",
          "username": "kedz"
      }
  ]
}
```

3.12.4 User Authentication

The client authentication methods are available on this path (`/v2/auth`).

There are currently two separate way is available for user authentication, one that uses a public/private key pair and one that uses a password. The key-based authentication has two phases, the password based authentication uses one request.

You may send `with_extended_privileges: true` to this request to get extended user information in case of successful authentication.

The password based authentication is implemented using the "authenticateWithPassword" request:

```
{
    "operation": "authenticateWithPassword",
    "password": "secret",
    "request_created": "2017-06-29T05:46:10.622Z",
    "request_id": 1,
    "user_name": "system"
}
```

The key based authentication starts with the client sending an "authenticate" request to the controller:

```
{
  "operation": "authenticate",
  "request_created": "2017-06-22T07:50:18.117Z",
  "request_id": 1,
  "user_name": "worf"
}
```

The controller replies by sending a challenge to the client:

```
{
  "challenge": "3694b3af-e2f0-e49e-4f32-1549fd824ea9",
  "request_created": "2017-06-22T07:50:18.117Z",
  "request_id": 1,
  "request_processed": "2017-06-22T07:50:18.163Z",
  "request_status": "Ok",
  "request_user_id": 0,
  "user_name": "worf"
}
```

The client then encrypts the challenge string with its private key and sends it back in a new request:

```
{
  "operation": "authenticateresponse",
  "request_created": "2017-06-22T07:50:18.169Z",
  "request_id": 2,
  "signature":
↪"XjiGlQNbq/yN9vzgS5uPjJfKw8DLXGD/uwlw+cec/iC9GJdgs1rZO6GNQzR2Mk6StgopX7gCDQwNVF426v3kGV
}
```

Should the signature match the controller sends back a reply notifying the client that it is now authenticated for the session:

```
{
    "is_superuser": true,
    "request_created": "2017-08-23T09:22:36.857Z",
    "request_id": 2,
    "request_processed": "2017-08-23T09:22:36.906Z",
    "request_status": "Ok",
    "user":
    {
        "class_name": "CmonUser",
        "email_address": "laszlo@severalnines.com",
        "first_name": "Laszlo",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 1,
            "group_name": "admins"
        } ],
        "last_name": "Pere",
        "title": "",
        "user_id": 3,
        "user_name": "admin"
    }
}
```


- `is_superuser`: Shows if the authenticated user actually has the superuser privileges.
- `request_status`: Shows that authentication was successful and the session now has an authenticated user.
- `request_user_id`: The request that produced this reply had no authenticated user ID, so the reply will not have this field.
- `user`: The details about the user authenticated for the session.

The client can terminate the session when it is no longer needed, on server side using the `logout` operation on auth.


```
{
  "operation": "logout",
  "request_created": "2017-06-22T07:50:18.117Z",
  "request_id": 4,
}
```


authenticate

The first step in the process where the client initiates the authentication by sending the `authenticate` request.


```
{
  "operation": "authenticate",
  "user_name": "worf"
}
```

This request should be called with the following arguments:

- `user_name`: The user name of the Cmon user.

The controller then will reply back with a security challenge text to the client:


```
{
  "challenge": "3694b3af-e2f0-e49e-4f32-1549fd824ea9",
  "request_processed": "2017-06-22T07:50:18.163Z",
  "request_status": "Ok",
  "request_user_id": 0,
  "user_name": "worf"
}
```


The reply contains the following fields:

- `challenge`: The string that should be encrypted by the client with the private key.
- `request_user_id`: The user ID of the currently authenticated user. If no authentication was finished before this numerical ID is 0.
- `user_name`: The name of the user which is going to be authenticated.

The next expected request from client is a 'response' call, with the signature created using the client private key.

passwordReset

Resets a user's password using a two-step process. The first request triggers a password-reset email to be sent to the user's registered address. The second request applies the new password using the token received in that email. Both requests use the same operation name; the first contains only `user_name` (and optionally `base_url`), while the second adds `password_reset_token` and `new_password`:


```
{
    "operation": "passwordReset",
    "request_created": "2019-05-31T10:14:30.457Z",
    "request_id": 1,
    "user_name": "system"
}
```


- `user_name`: The name of the user for whom the password reset email will be sent. The email address can not be in the request for security reasons, the controller will use the pre-registered email address of the user or if no such email address is registered the call will fail.
- `base_url`: The first part of the URL that will be sent in the email.


```
{
    "new_password": "newpassword",
    "operation": "passwordReset",
    "request_created": "2019-05-31T10:14:30.540Z",
    "request_id": 1,
    "password_reset_token": "5a018ede8d9e486aadc700fc241e9590",
    "user_name": "system"
}
```


- `new_password`: The new password that will be set for the user.

- `token`: The password change token that was received in the email.

authenticateResponse

This method shall be called by the client once it receives a security challenge (in reply to "authenticate" call) from the server.

```
{
  "operation": "authenticateresponse",
  "request_created": "2017-06-22T07:50:18.169Z",
  "request_id": 2,
  "signature":
↪"XjiGlQNbq/yN9vzgS5uPjJfKw8DLXGD/uwlw+cec/iC9GJdgs1rZO6GNQzR2Mk6StgopX7gCDQwNVF426v3kGV
}
```

This method should be called with the following arguments:

- `signature`: A base64 RSA-SHA256 signature using the client's private key of the 'challenge' text.

In reply backend sends either an error message if authentication failed, or just a success message showing that the authentication is finished and the user is authenticated for the session.

Here is an example for a reply that sent after the authentication succeeded:

```
{
    "request_created": "2017-06-29T13:25:32.885Z",
    "request_id": 2,
    "request_processed": "2017-06-29T13:25:32.936Z",
    "request_status": "Ok",
    "user":
    {
        "class_name": "CmonUser",
        "email_address": "warrior@ds9.com",
        "first_name": "Worf",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 4,
            "group_name": "ds9"
        } ],
        "last_login": "2017-06-29T13:25:21.906Z",
        "last_name": "",
        "title": "Lt.",
        "user_id": 12,
        "user_name": "worf"
    }
}
```

authenticateWithPassword

It is possible to authenticate the user using the user's own password. Here is an example showing the `authenticateWithPassword` request:

```
{
    "operation": "authenticateWithPassword",
    "password": "secret",
    "request_created": "2017-06-29T05:46:10.622Z",
    "request_id": 1,
    "user_name": "system"
}
```

- `user_name`: The username of the user to authenticate.
- `password`: The plain password of the user. Please note that the plain password is not a property of the CmonUser class, it is not stored by the controller, it is only used temporarily.

Here is a reply that sent after the authentication request succeeded:

```
{
    "request_created": "2017-06-29T13:22:58.592Z",
    "request_id": 1,
    "request_processed": "2017-06-29T13:22:58.645Z",
    "request_status": "Ok",
    "user":
    {
        "class_name": "CmonUser",
        "first_name": "System",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 1,
            "group_name": "admins"
        } ],
        "last_login": "2017-06-29T13:22:42.624Z",
        "last_name": "User",
        "user_id": 1,
        "user_name": "system"
    }
}
```

And here is a reply that sent about a failed attempt:

```
{
    "error_string": "Wrong username or password.",
    "reply_received": "2017-06-29T05:57:27.128Z",
    "request_created": "2017-06-29T05:57:27.118Z",
    "request_id": 1,
    "request_processed": "2017-06-29T05:57:27.193Z",
    "request_status": "AccessDenied"
}
```

Please note that the reply contains no `request_user_id` field because the session is not authenticated and there is no known user to associate with it. It is also worth noting that the error string does not specify what exactly was the problem with the authentication. Since there is no known user we only send a generic error message. In this case the user was found, but the password did not match, but we are not sending this

information with the reply.

3.12.5 Information about Backups

The `/v2/backup` path can be used to get information about backups.

Please note that creating and restoring a backup is a job and so is not implemented here.

getBackups

An RPC call that returns the backups of a cluster. For backup locations use `backup_record_version: 2`.

```
{
    "ascending": true,
    "cluster_id": 1,
    "operation": "getBackups",
    "backup_record_version": 1,
    "request_created": "2017-06-26T09:07:48.374Z",
    "request_id": 3
}
```

```
{
    "backup_records": [
    {
        "backup": [
        {
            "db": "all",
            "files": [
            {
                "class_name": "CmonBackupFile",
                "created": "2017-06-26T09:07:35.000Z",
                "hash": "md5:938bacc9699b362eb1be355eace1281d",
                "path": "pg_dump_2017-06-26_110732.sql.gz",
                "size": 796,
                "type": "full"
            } ],
            "start_time": "2017-06-26T09:07:35.000Z"
        } ],
        "backup_host": "192.168.1.198",
        "chain_up": 0,
        "cid": 1,
        "class_name": "CmonBackupRecord",
        "compressed": true,
        "config":
        {
            "backupDir": "/tmp",
            "backupHost": "192.168.1.198",
            "backupMethod": "",
            "backupToIndividualFiles": false,
            "backup_failover": false,
            "backup_failover_host": "",
```

(continues on next page)

(continued from previous page)


```
            "ccStorage": true,
            "compression": true,
            "createdBy": "pipas",
            "description": "Backup created by s9s-tools.",
            "includeDatabases": "",
            "netcat_port": 9999,
            "origBackupDir": "/tmp",
            "port": null,
            "scheduleId": 0,
            "set_gtid_purged_off": true,
            "storageHost": "",
            "throttle_rate_iops": 0,
            "throttle_rate_netbw": 0,
            "usePigz": false,
            "wsrep_desync": false,
            "xtrabackupParallellism": 1,
            "xtrabackup_locks": false
        },
        "created": "2017-06-26T09:07:32.000Z",
        "created_by": "",
        "description": "",
        "finished": "2017-06-26T09:07:35.930Z",
        "id": 1,
        "job_id": 2,
        "log_file": "",
        "lsn": 0,
        "method": "pgdump",
        "parent_id": 0,
        "root_dir": "/tmp/BACKUP-1",
        "schedule_id": 0,
        "status": "Completed",
        "storage_host": "192.168.1.127",
        "total_datadir_size": 38959683,
        "verified":
        {
            "message": "",
            "status": "Unverified",
            "verified_time": "1969-12-31T23:59:59.000Z"
        }
    },

    . . .

    ],
    "request_created": "2017-06-26T09:07:48.374Z",
    "request_id": 3,
    "request_processed": "2017-06-26T09:07:48.423Z",
    "request_status": "Ok",
    "request_user_id": 3,
```


(continues on next page)

(continued from previous page)

```
    "total": 2
}
```

getBackupSet

An RPC call that returns the backup set of a backup id, i.e a full backup including its increments are returned. If the backup id belongs to an incremental backup, then all the returned list of chained backups up to the parent (full backup) is returned. If the backup id belongs to a full backup, only the full backup is returned. For backup locations use `backup_record_version: 2`.

Example request:

```
{
    "cluster_id": 1,
    "operation": "getBackupSet",
    "backup_record_version": 1,
    "request_created": "2017-06-26T09:07:48.374Z",
    "request_id": 3
}
```

Example response:

```
{
    "backup_records": [
    {
        "backup": [
        {
            "db": "all",
            "files": [
            {
                "class_name": "CmonBackupFile",
                "created": "2017-06-26T09:07:35.000Z",
                "hash": "md5:938bacc9699b362eb1be355eace1281d",
                "path": "pg_dump_2017-06-26_110732.sql.gz",
                "size": 796,
                "type": "full"
            } ],
            "start_time": "2017-06-26T09:07:35.000Z"
        } ],
        "backup_host": "192.168.1.198",
        "chain_up": 0,
        "cid": 1,
        "class_name": "CmonBackupRecord",
        "compressed": true,
        "config":
        {
            "backupDir": "/tmp",
            "backupHost": "192.168.1.198",
            "backupMethod": "",
            "backupToIndividualFiles": false,
```

(continues on next page)

(continued from previous page)


```
            "backup_failover": false,
            "backup_failover_host": "",
            "ccStorage": true,
            "compression": true,
            "createdBy": "pipas",
            "description": "Backup created by s9s-tools.",
            "includeDatabases": "",
            "netcat_port": 9999,
            "origBackupDir": "/tmp",
            "port": null,
            "scheduleId": 0,
            "set_gtid_purged_off": true,
            "storageHost": "",
            "throttle_rate_iops": 0,
            "throttle_rate_netbw": 0,
            "usePigz": false,
            "wsrep_desync": false,
            "xtrabackupParallellism": 1,
            "xtrabackup_locks": false
        },
        "created": "2017-06-26T09:07:32.000Z",
        "created_by": "",
        "description": "",
        "finished": "2017-06-26T09:07:35.930Z",
        "id": 1,
        "job_id": 2,
        "log_file": "",
        "lsn": 0,
        "method": "pgdump",
        "parent_id": 0,
        "root_dir": "/tmp/BACKUP-1",
        "schedule_id": 0,
        "status": "Completed",
        "storage_host": "192.168.1.127",
        "total_datadir_size": 38959683,
        "verified":
        {
            "message": "",
            "status": "Unverified",
            "verified_time": "1969-12-31T23:59:59.000Z"
        }
    },

    . . .

    ],
    "request_created": "2017-06-26T09:07:48.374Z",
    "request_id": 3,
    "request_processed": "2017-06-26T09:07:48.423Z",
```


(continues on next page)

(continued from previous page)

```
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 2
}
```

getBackupSchedules

An RPC call to get the backup schedules from the controller.

Please note that the backup schedules are deprecated. Instead of creating scheduled backups creating of scheduled jobs (that create backups) is advisable. This RPC call was only created for backward compatibility.

Here is how a request should look like:

```
{
    "operation": "getBackupSchedules",
    "request_created": "2020-01-29T11:01:25.887Z",
    "request_id": 3
}
```

And here is how the reply should look like:

```
{
  "backup_schedules": [ {
      "class_name": "CmonBackupSchedule",
      "cluster_id": 1,
      "enabled": true,
      "id": 1,
      "job": {
        "command": "backup",
        "job_data": {
          "description": "Backup created by s9s-tools."
        },
        "scheduleId": 1
      },
      "lastExec": "1970-01-01T00:00:00.000Z",
      "schedule": "0 12 * * 5"
    }  ],
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 3,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
  "request_user_id": 4,
  "total": 1
}
```

scheduleBackup

Creates or updates a backup schedule for a cluster. The `schedule` field uses a standard cron expression to define when the backup job runs. The `job` field within the schedule object defines the backup command and its parameters.

Example request:

```
{
    "cluster_id": 1,
    "operation": "scheduleBackup",
    "request_created": "2020-01-29T10:57:52.916Z",
    "request_id": 3,
    "schedule":
    {
        "class_name": "CmonBackupSchedule",
        "enabled": true,
        "job":
        {
            "command": "backup",
            "job_data":
            {
                "description": "Backup created by s9s-tools."
            }
        },
        "schedule": "0 12 * * 5"
    }
}
```

Example response:

```
{
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
  "reply_received": "2020-01-29T10:57:52.927Z",
  "request_created": "2020-01-29T10:57:52.916Z",
  "request_id": 3,
  "request_processed": "2020-01-29T10:57:52.926Z",
  "request_status": "Ok",
  "request_user_id": 4,
  "schedule": {
    "class_name": "CmonBackupSchedule",
    "cluster_id": 1,
    "enabled": true,
    "id": 1,
    "job": {
      "command": "backup",
      "job_data": {
        "description": "Backup created by s9s-tools."
      },
      "scheduleId": 1
    },
    "lastExec": null,
```

(continues on next page)

(continued from previous page)

```
    "schedule": "0 12 * * 5"
  }
}
```

deleteBackupRecord

Deletes a single backup record identified by `backup_id`. This removes the backup metadata from the controller database but does not automatically delete the backup files from storage.

Example request:

```
{
    "backup_record":
    {
        "backup_id": 2
    },
    "operation": "deleteBackupRecord",
    "request_created": "2019-09-20T12:15:50.682Z",
    "request_id": 3
}
```

deleteBackupList

Deletes multiple backup records in a single call by providing a comma-separated list of backup IDs in the `backup_id_list` field. More efficient than calling `deleteBackupRecord` repeatedly.

Example request:

```
{
    "operation": "deleteBackupList",
    "backup_id_list": "2,3,4",
    "request_created": "2024-12-20T12:15:50.682Z",
    "request_id": 3
}
```

snapshotRepositories

Manages Elasticsearch snapshot repositories used for backup storage. Supports creating a new snapshot repository (`snapshotRepository`), listing all repositories (`getAllSnapshotRepositories`), retrieving repositories for a specific cluster (`getSnapshotRepositories`), and deleting an existing repository (`deleteSnapshotRepository`).

Example request:

```
{
  "class_name": "CmonRpcRequest",
  "cluster_id": 108,
  "operation": "snapshotRepository",
  "request_created": "2022-05-24T09:36:57.784Z",
  "request_id": 2,
  "repository": {
```

(continues on next page)

(continued from previous page)

```
    "class_name": "CmonBackupSnapshotRepository",
    "job": {
      "command": "backup",
      "description": "Snapshot repository created by s9s-tools on␣
↪elasticsearch cluster",
      "job_data": {
        "credential_id": 1,
        "description": "Backup created by s9s-tools.",
        "s3_bucket": "elastic-s3-test",
        "snapshot_repository": "cliRepo",
        "snapshot_repository_type": "s3"
      }
    }
  }
}
```

```

```

getAllSnapshotRepositories

List all Elasticsearch snapshot repositories.

Example request:

```
{
    "operation": "getAllSnapshotRepositories",
    "request_created": "2022-05-26T09:42:31.844Z",
    "request_id": 3
}
```

Example response:

```
{
  "controller_id": "1084dffd-be4d-4c03-bca4-00accbcc200b",
  "reply_received": "2022-07-08T10:04:17.952Z",
  "request_created": "2022-07-08T10:04:17.846Z",
  "request_id": 2,
  "request_processed": "2022-07-08T10:04:17.952Z",
  "request_status": "Ok",
  "request_user_id": 5,
  "total": 2,
  "debug_messages": [ "RPC V2 authenticated user is 'ccalvaro'."  ],
  "messages": [ "Snapshot repositories on all clusters has been retrieved" ␣
↪],
  "snapshot_repositories_list": {
    "215": {
      "fsRepo": {
        "type": "fs",
        "settings": {
          "location": "/mnt/data/backups/es-snapshot-repositories"
```

(continues on next page)

(continued from previous page)


```
        }
      },
      "test_s3_repo": {
        "type": "s3",
        "uuid": "QbQ00rxEQJqwnzcgoK_OAg",
        "settings": {
          "bucket": "elastic-s3-test",
          "client": "secondary",
          "region": "eu-west-3"
        }
      }
    },
    "216": {
      "fsRepo": {
        "type": "fs",
        "storage_host": "kvm2",
        "settings": {
          "location": "/mnt/data/backups/es-snapshot-repositories"
        }
      },
      "s3Repo": {
        "type": "s3",
        "storage_host": "s3",
        "uuid": "QbQ00rxEQJqwnzcgoK_OAg",
        "settings": {
          "bucket": "elastic-s3-test",
          "client": "secondary",
          "region": "eu-west-3"
        }
      }
    }
  }
}
```


getSnapshotRepositories

Retrieve Elasticsearch repositories for a specific cluster.

Example request:


```
{
    "cluster_id": 108,
    "operation": "getSnapshotRepositories",
    "request_created": "2022-05-26T09:42:31.844Z",
    "request_id": 3
}
```


Example response:


```
{
```


(continues on next page)

(continued from previous page)


```
  "controller_id": "1084dffd-be4d-4c03-bca4-00accbcc200b",
  "reply_received": "2022-05-26T09:42:31.898Z",
  "request_created": "2022-05-26T09:42:31.844Z",
  "request_id": 2,
  "request_processed": "2022-05-26T09:42:31.898Z",
  "request_status": "Ok",
  "request_user_id": 5,
  "total": 1,
  "debug_messages": [ "RPC V2 authenticated user is 'ccalvaro'."  ],
  "messages": [ "Snapshot repositories has been retrieved"  ],
  "snapshot_repositories": {
    "sharedFs": {
      "type": "fs",
      "storage_host": "s3",
      "uuid": "yGaLnQU5TFaAX96ZBAgN_Q",
      "settings": {
        "location": "/mnt/data/"
      }
    }
  }
}
```


deleteSnapshotRepository

Delete an existing Elasticsearch repository.

Example request:


```
 {
  "class_name": "CmonRpcRequest",
  "cluster_id": 108,
  "operation": "deleteSnapshotRepository",
  "request_created": "2022-05-26T09:17:13.385Z",
  "request_id": 2,
  "snapshot_repository": "s3_cli_repo"
}
```


3.12.6 Certificate Authority (CA)

With these API calls you can list and manage your certificates managed by ClusterControl. These methods can be found on path `/v2/ca`.

listCerts

A method to list the (CA, server and client) certificates

Here is how a listCerts request should look like:


```
{ "operation": "listCerts" }
```


And an example response:

```
{
   "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
   "is_superuser": true,
   "request_processed": "2023-02-16T14:28:37.281Z",
   "request_status": "Ok",
   "total": 1,
   "data":
   [
       {
           "certfile": "galera/cluster_1/server_ca.crt",
           "id": 1,
           "isCA": true,
           "isClient": true,
           "isServer": true,
           "issued": 1620992185,
           "issuerId": 0,
           "keybits": 2048,
           "keyfile": "galera/cluster_1/server_ca.key",
           "serialNumber": 1,
           "status": "Issued",
           "validFrom": 1620905785,
           "validUntil": 1936352185,
           "inUseBy":
           {
               "hosts": [],
               "clusters": []
           },
           "subjectName":
           {
              "CN": "ClusterControl_AutoGenerated_CA_Certificate",
              "basicConstraints": "ca",
              "serialNumber": "1",
              "extendedKeyUsage":
              [
                  "ClientAuth",
                  "ServerAuth",
                  "OCSPSigning"
              ],
              "keyUsage":
              [
                  "DigitalSignature",
                  "NonRepudiation",
                  "KeyEncipherment",
                  "KeyAgreement",
                  "KeyCertificateSign",
                  "CRLSign"
              ]
          }
       }
  ]
```


(continues on next page)

(continued from previous page)

```
}
```

certInfo

A method to give details about a certificate.

Here is how a request should look like

```
{
  "operation": "certInfo",
  "id": 2
}
```

And an example response:

```
{
    "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
    "is_superuser": true,
    "request_processed": "2023-02-16T14:57:24.882Z",
    "request_status": "Ok",
    "total": 1,
    "data":
    {
        "certfile": "galera/cluster_1/server.crt",
        "id": 2,
        "isCA": false,
        "isClient": true,
        "isServer": true,
        "issued": 1620992185,
        "issuerId": 1,
        "keybits": 2048,
        "keyfile": "galera/cluster_1/server.key",
        "serialNumber": 2,
        "status": "Issued",
        "validFrom": 1620905785,
        "validUntil": 1936352185,
        "inUseBy":
        {
            "hosts": [],
            "clusters": []
        },
        "subjectName":
        {
            "CN": "MySQL_Server_Cmon_Auto_Generated_Server_Certificate",
            "description": "Generated by ClusterControl",
            "serialNumber": "2",
            "extendedKeyUsage":
            [
                "ClientAuth",
                "ServerAuth"
```

(continues on next page)

(continued from previous page)

```
            ],
            "keyUsage":
            [
                "DigitalSignature",
                "NonRepudiation",
                "KeyEncipherment",
                "KeyAgreement"
            ]
        }
    }
}
```

create

Creates a signed (self or CA signed) certificate.

Here is how a request should look like

```
{
   "operation":"create",
   "type":"ca",
   "name":"exampleCa1",
   "validity":3650,
   "data":{
     "keybits":4096,
     "CN":"My CA certificate.",
     "description":"This is an example CA certificate.",
     "emailAddress":"john.doe@severalnines.com"
   }
}
```

And an example response:

```
{
   "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
   "is_superuser": true,
   "request_processed": "2023-02-23T08:55:21.030Z",
   "request_status": "Ok",
   "total": 1,
   "data":
   {
       "certfile": "exampleCa1.crt",
       "id": 228,
       "isCA": true,
       "isClient": true,
       "isServer": true,
       "issued": 1677142521,
       "issuerId": 0,
       "keybits": 4096,
       "keyfile": "exampleCa1.key",
```

(continues on next page)

(continued from previous page)


```
        "requesterUserId": 68,
        "serialNumber": 228,
        "status": "Issued",
        "validFrom": 1677052521,
        "validUntil": 1992498921,
        "inUseBy":
        {
            "hosts": [],
            "clusters": []
        },
        "subjectName":
        {
            "CN": "My CA certificate.",
            "basicConstraints": "ca",
            "description": "This is an example CA certificate.",
            "emailAddress": "john.doe@severalnines.com",
            "serialNumber": "228",
            "extendedKeyUsage":
            [
                "ClientAuth",
                "ServerAuth",
                "OCSPSigning"
            ],
            "keyUsage":
            [
                "DigitalSignature",
                "NonRepudiation",
                "KeyEncipherment",
                "KeyAgreement",
                "KeyCertificateSign",
                "CRLSign"
            ]
        }
    }
}
```


revoke

Marks the specified certificate as revoked. You might need to regenerate the CRL list of the issuer (CA) certificate after this operation.

Here is how a request should look like


```
{
    "operation":"revoke",
    "id": 227
}
```


And an example response:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-02-23T14:12:39.292Z",
  "request_status": "Ok",
  "total": 1,
  "data":
  {
      "certfile": "replication/cluster_117/client.crt",
      "id": 227,
      "isCA": false,
      "isClient": true,
      "isServer": false,
      "issued": 1674744928,
      "issuerId": 225,
      "keybits": 2048,
      "keyfile": "replication/cluster_117/client.key",
      "revoked": 1677161559,
      "revokerUserId": 68,
      "serialNumber": 227,
      "status": "Revoked",
      "validFrom": 1674654928,
      "validUntil": 1990101328,
      "inUseBy":
      {
          "hosts": [],
          "clusters": []
      },
      "subjectName":
      {
          "CN": "MySQL_Server_Cmon_Auto_Generated_Client_Certificate",
          "description": "Generated by ClusterControl",
          "serialNumber": "227",
          "extendedKeyUsage":
          [
              "ClientAuth"
          ],
          "keyUsage":
          [
              "DigitalSignature",
              "NonRepudiation",
              "KeyEncipherment",
              "KeyAgreement"
          ]
      }
  }
}
```

delete

Deletes the certificate (cert and private key) files. Please note that this is an unusual and dangerous operation, please use revoke instead of this.

Here is how a request should look like

```
{
   "operation":"delete",
   "id": 1001
}
```

And an example response:

```
{
   "request_status": "Ok"
}
```

move

Moves a certificate within the ClusterControl CA storage.

Here is how a request should look like

```
{
   "operation":"move",
   "id": 228,
   "name": "examples/demoCA/cert01"
}
```

And an example response:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-02-23T13:32:09.206Z",
  "request_status": "Ok",
  "total": 1,
  "data":
  [
      "certfile": "examples/demoCA/cert01.crt",
      "id": 228,
      "isCA": true,
      "isClient": true,
      "isServer": true,
      "issued": 1677142521,
      "issuerId": 0,
      "keybits": 4096,
      "keyfile": "examples/demoCA/cert01.key",
      "requesterUserId": 68,
      "serialNumber": 228,
      "status": "Issued",
      "validFrom": 1677052521,
```

(continues on next page)

(continued from previous page)

```
        "validUntil": 1992498921,
        "inUseBy":
        {
            "hosts": [],
            "clusters": []
        },
        "subjectName":
        {
            "CN": "My CA certificate.",
            "basicConstraints": "ca",
            "description": "This is an example CA certificate.",
            "emailAddress": "john.doe@severalnines.com",
            "serialNumber": "228",
            "extendedKeyUsage":
            [
                "ClientAuth",
                "ServerAuth",
                "OCSPSigning"
            ],
            "keyUsage":
            [
                "DigitalSignature",
                "NonRepudiation",
                "KeyEncipherment",
                "KeyAgreement",
                "KeyCertificateSign",
                "CRLSign"
            ]
        }
    }
}
```

crl

Creates a certificate revokation list (CRL) signed by the specified CA.

Here is how a request should look like

```
{
   "operation":"crl",
   "id": 229
}
```

And an example response:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-02-23T13:29:00.685Z",
  "request_status": "Ok",
```

(continues on next page)

(continued from previous page)

```
  "total": 1,
  "data":
  {
      "certfile": "replication/cluster_118/server_ca.crt",
      "crlfile": "replication/cluster_118/server_ca.crl"
  }
}
```

importlocal

Imports a certificate (.crt and .key file) from controller host. CA is optional in the request, as not needed for example in case of the certs are self-signed or CA certs.

Here is how a request should look like

```
{
   "operation":"importlocal",
   "cert_file": "/tmp/TestCLIENT.crt",
   "key_file": "/tmp/TestCLIENT.key",
   "ca_file": "/tmp/TestingCA007.crt",
   "name": "testing/import/testClient",
   "name_ca": "testing/import/testCA"
}
```

And an example response:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-02-23T14:20:39.058Z",
  "request_status": "Ok",
  "total": 1,
  "data":
  {
      "certfile": "testing/import/testClient.crt",
      "id": 236,
      "isCA": false,
      "isClient": true,
      "isServer": false,
      "issued": 1677162039,
      "issuerId": 235,
      "keybits": 2048,
      "keyfile": "testing/import/testClient.key",
      "serialNumber": 234,
      "status": "Issued",
      "validFrom": 1677071895,
      "validUntil": 1992518295,
      "wasCaKnown": false,
      "inUseBy":
      {
```

(continues on next page)

(continued from previous page)

```
            "hosts": [],
            "clusters": []
        },
        "subjectName":
        {
            "description": "fdsfdsf",
            "serialNumber": "234",
            "extendedKeyUsage":
            [
                "ClientAuth"
            ],
            "keyUsage":
            [
                "DigitalSignature",
                "NonRepudiation",
                "KeyEncipherment",
                "KeyAgreement"
            ]
        }
    }
}
```

3.12.7 Cloud credentials API

The `/v2/cloud` path provides calls to manage the cloud-credentials in cmon.

proxy

Proxies a request to the cmon-cloud service and returns the raw response. Use this to interact with cloud provider APIs through the controller without exposing credentials directly to the client.

```
{
  "operation":"proxy",
  "uri":"/aws/vm/sizes?type=all&mem_size_mb=2048",
  "method":"GET",
  "provider":"aws",
  "credentials_id":1,
  "body": "",
}
```

```
{
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 10,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
  "response":{
    "reason_phrase":"OK",
    "status_code":200,
```

(continues on next page)

(continued from previous page)

```
    "headers":{
      "content-length":"1016",
      "content-type":"application\/json; charset=UTF-8",
      "date":"Tue, 30 Nov 2021 12:40:43 GMT"
    },
    "json":["t3.small","t3.medium","t3.large"],
  }
}
```

verifyCredentials

Verifies that a set of cloud provider credentials are valid by issuing a test request (such as listing storage buckets) through the cmon-cloud service.

```
{
  "operation": "verifyCredentials",
  "provider":"aws",
  "credentials":
  {
    "access_key_id":"DEADBAADDEADBAAD0x0x0x0x",
    "access_key_secret":"SECRET1SECRET0",
    "access_key_region":"ap-southeast-2"
  }
}
```

```
{
  "added_id": 12313,
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 3,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
  "request_user_id": 4
}
```

listCredentials

Returns all stored cloud provider credentials, grouped by provider. Ensure this call is made over a secure (TLS) connection as credential secrets are included in the response.

```
{
  "operation": "listCredentials"
}
```

```
{
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 10,
  "request_processed": "2020-01-29T11:01:25.893Z",
```

(continues on next page)

(continued from previous page)


```
  "request_status": "Ok",
  "request_user_id": 4,
  "result":{
    "aws":[
      {
        "name":"MyAWS",
        "comment":"this is a comment",
        "id":1,
        "credentials":
        {
          "access_key_id":"DEADBAADDEADBAAD0x0x0x0x",
          "access_key_region":"ap-southeast-2",
          "access_key_secret":"SECRET1SECRET0"
        }
      }
    ],
    "gce":[
      {
         "name":"GCE1",
         "comment":"DDDD",
         "id":2,
         "credentials":
         {
           "auth_provider_x509_cert_url":
↪"https:\/\/www.googleapis.com\/oauth2\/v1\/certs",
           "auth_uri":"https:\/\/accounts.google.com\/o\/oauth2\/auth",
           "client_email":
↪"490013373999-compute@developer.gserviceaccount.com",
           "client_id":"100000568876229481 11",
           "client_x509_cert_url":
↪"https:\/\/www.googleapis.com\/robot\/v1\/metadata\/x509\/1234-compute%40developer.gser
↪
           "private_key":"-----BEGIN PRIVATE␣
↪KEY-----\nFDSFSFDSFSDFDSFDSFDS\n-----END PRIVATE KEY-----\n",
           "private_key_id":"b0a4df1c54fff4410c33c9bca57bd0fb01679000",
           "project_id":"brave-night-121210",
           "token_uri":"https:\/\/accounts.google.com\/o\/oauth2\/token",
           "type":"service_account"
         }
       }
     ]
  }
}
```


getCredentials

Returns a specific set of stored cloud credentials identified by `id` and `provider`.

```
{
  "operation": "getCredentials",
  "id": 10,
  "provider": "aws"
}
```

```
{
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 10,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
  "request_user_id": 4,
  "data":{
    "name":"MyAWS",
    "comment":"this is a comment",
    "id":1,
    "credentials":
    {
      "access_key_id":"DEADBAADDEADBAAD0x0x0x0x",
      "access_key_region":"ap-southeast-2",
      "access_key_secret":"SECRET1SECRET0"
    }
  }
}
```

addCredentials

Stores a new set of cloud provider credentials in the controller. Returns the assigned `added_id` on success.

```
{
  "operation": "addCredentials",
  "provider":"aws",
  "name":"MyAWS",
  "credentials":
  {
    "access_key_id":"DEADBAADDEADBAAD0x0x0x0x",
    "access_key_secret":"SECRET1SECRET0",
    "access_key_region":"ap-southeast-2"
  },
  "comment":"this is a comment"
}
```

```
{
  "added_id": 12313,
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 3,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
```

(continues on next page)

(continued from previous page)

```
  "request_user_id": 4
}
```

updateCredentials

Updates an existing set of cloud provider credentials identified by `id`. All credential fields are replaced with the supplied values.

```
{
  "operation": "updateCredentials",
  "id": 12313,
  "provider":"aws",
  "name":"MyAWS",
  "credentials":
  {
    "access_key_id":"DEADBAADDEADBAAD0x0x0x0x",
    "access_key_secret":"SECRET1SECRET0",
    "access_key_region":"ap-southeast-2"
  },
  "comment":"this is a comment"
}
```

```
{
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 4,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
  "request_user_id": 4
}
```

removeCredentials

Removes a stored set of cloud provider credentials identified by `id` and `provider`. Clusters or resources that depend on these credentials will no longer be able to access them.

```
{
  "operation": "removeCredentials",
  "id": 12313,
  "provider":"aws",
}
```

```
{
  "reply_received": "2020-01-29T11:01:25.893Z",
  "request_created": "2020-01-29T11:01:25.887Z",
  "request_id": 4,
  "request_processed": "2020-01-29T11:01:25.893Z",
  "request_status": "Ok",
```

(continues on next page)

(continued from previous page)

```
  "request_user_id": 4
}
```

3.12.8 Cluster Information and Manipulation

Cluster information and cluster wide operations can be found on the path `/v2/clusters`. Information about hosts, alarms, license, statistics can also be sent back for convenience.

getConfig

This is the "getConfig" call for clusters. Please note that there is an other "getConfig" call for nodes and these two should not be mixed up. There are some similarities since the controller is itself a node, but this is an entirely different call because it is on a different path.

Here is how a getConfig call should look like:

```
{
    "cluster_id": 1,
    "operation": "getConfig",
    "request_created": "2019-09-25T06:53:50.012Z",
    "request_id": 3
}
```

```
{
    "cluster_id": 1,
    "operation": "getConfigGrouped",
    "request_created": "2019-09-25T06:53:50.012Z",
    "request_id": 3
}
```

Here is how a reply to a getConfig call looks like:

```
{
  "configuration": {
    "values": [
      {
        "current_value": "",
        "default_value": null,
        "description": "An arbitrary identifier string of this controller␣
↪instance.",
        "keys": [ "controller_id"  ],
        "name": "controller_id",
        "sql_keys": [ "CONTROLLER_ID"  ]
      },

      . . .

      {
        "current_value": null,
        "default_value": null,
```

(continues on next page)

(continued from previous page)

```
        "description": "Specify a lock file and if present on a node,  the␣
↪node will not recover. It is the responsibilty of the administrator to␣
↪create/remove the file.",
        "keys": [ "node_recovery_lock_file"  ],
        "name": "node_recovery_lock_file",
        "sql_keys": [ "NODE_RECOVERY_LOCK_FILE"  ]
      }
    ]
  },
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
  "reply_received": "2019-09-25T12:21:30.909Z",
  "request_created": "2019-09-25T12:21:30.904Z",
  "request_id": 3,
  "request_processed": "2019-09-25T12:21:30.944Z",
  "request_status": "Ok",
  "request_user_id": 4
}
```

As it is shown the "configuration" field holds the cluster configuration and inside it there is a list "values" showing the possible settings. The structure of the data here might be enhanced later, we might want to send back more information.

setConfig

This is the "setConfig" call for clusters. There is a similar call for nodes, but that has slightly different requests and replies.

Here is how a setConfig request should look like:

```
{
    "cluster_id": 1,
    "configuration": [
    {
        "name": "host_stats_collection_interval",
        "value": "60"
    } ],
    "operation": "setConfig",
    "request_created": "2019-09-25T07:53:11.644Z",
    "request_id": 3
}
```

Here the changes are passed in a list with names and values. The names can be either any of the keys or any of the sql_keys for the configuration item as they are returned for the getConfig call. So one can issue a getConfig call, study, find the key to be changed and send back for the setConfig request.

One needs write access on the configuration (same as the write access on the cluster) to change the configuration.

Here is how a reply looks like (it is just a standard 'ok' reply anyway):

```
{
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
```

(continues on next page)

(continued from previous page)

```
  "reply_received": "2019-09-25T12:19:52.392Z",
  "request_created": "2019-09-25T12:19:52.387Z",
  "request_id": 3,
  "request_processed": "2019-09-25T12:19:52.430Z",
  "request_status": "Ok",
  "request_user_id": 4
}
```

getClusterInfo

This request is for reading information about one cluster. The cluster can be identified by the cluster ID or the cluster name.

- `with_hosts`: Send also the list of hosts.
- `with_sheet_info`: Send also the information from the sheet manager.
- `with_databases`: Send also the databases we found on the cluster.
- `with_license_check`: Send also the license info.
- `cluster_id`: The numerical ID of the cluster to get. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster to get. The cluster can also be identified by its numerical ID.

getAllClusterInfo

This request can be used to get information about one or more clusters. The clusters can be identified by their cluster ID passed as a list of integers in the request.

```
{
   "operation": "getAllClusterInfo",
   "request_created": "2017-06-20T06:19:58.451Z",
   "with_hosts": true,
   "with_sheet_info": true
}
```

- `cluster_ids`: An optional list of cluster IDs represented as integers to control which clusters should be detailed in the reply. If this argument is not provided the controller will return information about clusters that the user can see (has the proper privileges and location to see).
- `with_hosts`: Send also the list of hosts.
- `with_containers`: Send also the information about the containers/VM-s.
- `with_sheet_info`: Send also the information from the sheet manager.
- `with_databases`: Send also the databases we found on the cluster.
- `with_license_check`: Send also the license info.
- `with_tags`: The request may contain this field to limit the list of the returned clusters to those that have at least one of the tags passed here.
- `without_tags`: The request may contain this field to limit the list of the returned users to those that has none of the tags passed here.

This call will return information about clusters the authenticated user "can see". This means the return can not hold information about a cluster the user has no read access according to the effective ACL and the owner/group owner of the cluster object. Read access for the folder where the cluster is located is also necessary, so if the user has no read access for the folder will not be visible. And finally the chroot environment is also considered. If the user in an other branch of the CDT and so the cluster is not under "his" subtree the cluster will not be visible.

Here is an example of the reply:


```
{
    "clusters": [
    {
        "alarm_statistics":
        {
            "class_name": "CmonAlarmStatistics",
            "cluster_id": 1,
            "critical": 2,
            "warning": 1
        },
        "class_name": "CmonClusterInfo",
        "cluster_auto_recovery": true,
        "cluster_id": 1,
        "cluster_name": "ft_postgresql_59238",
        "cluster_type": "POSTGRESQL_SINGLE",
        "configuration_file": "/tmp/cmon_1.cnf",
        "group_owner":
        {
            "class_name": "CmonGroup",
            "group_id": 4,
            "group_name": "testgroup"
        },
        "hosts": [
        {
            "class_name": "CmonPostgreSqlHost",
            "clusterid": 1,
            "configfile": [ "/etc/postgresql/9.6/main/postgresql.conf" ],
            "connected": true,
            "container": true,
            "data_directory": "/var/lib/postgresql/9.6/main",
            "datadir": "/var/lib/postgresql/9.6/main",
            "description": "",
            "distribution":
            {
                "codename": "xenial",
                "name": "ubuntu",
                "release": "16.04",
                "type": "debian"
            },
            "hostId": 1,
            "hostname": "192.168.1.177",
            "hoststatus": "CmonHostOnline",
```


(continues on next page)

(continued from previous page)


```
            "hot_standby": true,
            "ip": "192.168.1.177",
            "lastseen": 1501216153,
            "logfile": "/var/log/postgresql/postgresql-9.6-main.log[0m",
            "maintenance_mode_active": false,
            "message": "Up and running",
            "nodetype": "postgres",
            "pid": 5248,
            "pidfile": "/var/run/postgresql/9.6-main.pid",
            "pingstatus": 0,
            "pingtime": 0,
            "port": 8089,
            "readonly": false,
            "received_location": "0/1520838",
            "replay_location": "0/1520838",
            "role": "master",
            "sshfailcount": 0,
            "ssl_certs":
            {
                "server":
                {
                    "ca": "",
                    "id": 0,
                    "key": "/etc/ssl/private/ssl-cert-snakeoil.key",
                    "path": "/etc/ssl/certs/ssl-cert-snakeoil.pem"
                }
            },
            "timestamp": 1501216153,
            "unique_id": 1,
            "uptime": 220,
            "version": "9.6.3",
            "wallclock": 1501216175,
            "wallclocktimestamp": 1501216110
        },
        {
            "class_name": "CmonHost",
            "clusterid": 1,
            "configfile": "/tmp/cmon_1.cnf",
            "connected": true,
            "container": false,
            "distribution":
            {
                "codename": "trusty",
                "name": "ubuntu",
                "release": "14.04",
                "type": "debian"
            },
            "hostId": 2,
            "hostname": "192.168.1.127",
```


(continues on next page)

(continued from previous page)


```
            "hoststatus": "CmonHostOnline",
            "ip": "192.168.1.127",
            "lastseen": 1501216153,
            "logfile": "/tmp/cmon_1.log",
            "maintenance_mode_active": false,
            "message": "Up and running",
            "nodetype": "controller",
            "pid": 59196,
            "pingstatus": 0,
            "pingtime": 0,
            "port": 9555,
            "role": "controller",
            "timestamp": 1501216153,
            "unique_id": 2,
            "uptime": 413,
            "version": "1.4.3",
            "wallclock": 1501216110,
            "wallclocktimestamp": 1501216110
        } ],
        "info":
        {
            "cluster.status": 2,
            "cluster.statustext": "Cluster started.",
            "cmon.domainname": "",
            "cmon.hostname": "t7500",
            "cmon.running": true,
            "cmon.starttime": 1501215924,
            "cmon.uptime": 227,
            "conf.backup_retention": 31,
            "conf.clusterid": 1,
            "conf.clustername": "ft_postgresql_59238",
            "conf.clustertype": 5,
            "conf.configfile": "/tmp/cmon_1.cnf",
            "conf.hostname": "192.168.1.127",
            "conf.os": "debian",
            "conf.statustext": "Configuration loaded.",
            "host.1.connected": true,
            "host.1.cpu_io_wait_percent": 0,
            "host.1.cpu_steal_percent": 0,
            "host.1.cpu_usage_percent": 7.92751,
            "host.1.cpucores": 16,
            "host.1.cpuinfo": [
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 4,
                "cpumaxmhz": 2.268e+06,
                "cpumhz": 1600,
                "cpumodel": "Intel(R) Xeon(R) CPU           L5520  @␣
↪2.27GHz",
```


(continues on next page)

(continued from previous page)


```
                "cputemp": 54.5,
                "hostid": 1,
                "physical_cpu_id": 0,
                "siblings": 8,
                "vendor": "GenuineIntel"
            },
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 4,
                "cpumaxmhz": 2.268e+06,
                "cpumhz": 1600,
                "cpumodel": "Intel(R) Xeon(R) CPU           L5520  @␣
↪2.27GHz",
                "cputemp": 54.5,
                "hostid": 1,
                "physical_cpu_id": 1,
                "siblings": 8,
                "vendor": "GenuineIntel"
            } ],
            "host.1.cpumaxmhz": 2268,
            "host.1.cpumhz": 1600,
            "host.1.cpumodel": "Intel(R) Xeon(R) CPU           L5520  @␣
↪2.27GHz",
            "host.1.cputemp": 54.5,
            "host.1.devices": [ "/dev/mapper/core1--vg-root" ],
            "host.1.free_disk_bytes": 156237512704,
            "host.1.hostname": "192.168.1.177",
            "host.1.interfaces": [ "eth0" ],
            "host.1.ip": "192.168.1.177",
            "host.1.membuffer": 0,
            "host.1.memcached": 3,
            "host.1.memfree": 16450528,
            "host.1.memtotal": 16777216,
            "host.1.network_interfaces": [
            {
                "interface_name": "eth0",
                "rx_bytes_per_sec": 10438.1,
                "tx_bytes_per_sec": 15937.2
            } ],
            "host.1.pingdelay": -1,
            "host.1.pingstatustext": "Creating ICMP socket (to ping
↪'192.168.1.177') failed: Operation not permitted.",
            "host.1.port": 8089,
            "host.1.rx_bytes_per_second": 10438.1,
            "host.1.swapfree": 0,
            "host.1.swaptotal": 0,
            "host.1.total_disk_bytes": 208033853440,
            "host.1.tx_bytes_per_second": 15937.2,
            "host.1.uptime": 390,
```


(continues on next page)

(continued from previous page)


```
            "host.1.wallclock": 1501216175,
            "host.1.wallclocksampled": 1501216110,
            "host.2.class_name": "controller",
            "host.2.connected": true,
            "host.2.cpu_io_wait_percent": 4.14259,
            "host.2.cpu_steal_percent": 0,
            "host.2.cpu_usage_percent": 178.599,
            "host.2.cpucores": 24,
            "host.2.cpuinfo": [
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 6,
                "cpumaxmhz": 2.661e+06,
                "cpumhz": 2660,
                "cpumodel": "Intel(R) Xeon(R) CPU           X5650  @␣
↪2.67GHz",
                "cputemp": 0,
                "hostid": 2,
                "physical_cpu_id": 0,
                "siblings": 12,
                "vendor": "GenuineIntel"
            },
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 6,
                "cpumaxmhz": 2.661e+06,
                "cpumhz": 2661,
                "cpumodel": "Intel(R) Xeon(R) CPU           X5650  @␣
↪2.67GHz",
                "cputemp": 0,
                "hostid": 2,
                "physical_cpu_id": 1,
                "siblings": 12,
                "vendor": "GenuineIntel"
            } ],
            "host.2.cpumaxmhz": 2661,
            "host.2.cpumhz": 2660,
            "host.2.cpumodel": "Intel(R) Xeon(R) CPU           X5650  @␣
↪2.67GHz",
            "host.2.cputemp": 0,
            "host.2.devices": [ "/dev/sda1" ],
            "host.2.free_disk_bytes": 1404706975744,
            "host.2.hostname": "192.168.1.127",
            "host.2.interfaces": [ "eth0" ],
            "host.2.ip": "192.168.1.127",
            "host.2.membuffer": 232240,
            "host.2.memcached": 13097476,
            "host.2.memfree": 14108052,
            "host.2.memtotal": 49453276,
```


(continues on next page)

(continued from previous page)


```
            "host.2.network_interfaces": [
            {
                "interface_name": "eth0",
                "rx_bytes_per_sec": 17268.2,
                "tx_bytes_per_sec": 11158.5
            } ],
            "host.2.pingdelay": -1,
            "host.2.pingstatustext": "Creating ICMP socket (to ping
↪'192.168.1.127') failed: Operation not permitted.",
            "host.2.port": 9555,
            "host.2.rx_bytes_per_second": 17268.2,
            "host.2.swapfree": 0,
            "host.2.swaptotal": 0,
            "host.2.total_disk_bytes": 2125259440128,
            "host.2.tx_bytes_per_second": 11158.5,
            "host.2.uptime": 2.65439e+06,
            "host.2.version": "1.4.3",
            "host.2.wallclock": 1501216110,
            "host.2.wallclocksampled": 1501216110,
            "license.expires": -1,
            "license.status": false,
            "license.statustext": "No license found.",
            "mail.statustext": "Created.",
            "netStat.1.eth0.rxBytes": 5238632,
            "netStat.1.eth0.txBytes": 33144284,
            "netStat.2.eth0.rxBytes": 531337507751,
            "netStat.2.eth0.txBytes": 505661940612
        },
        "job_statistics":
        {
            "by_state":
            {
                "ABORTED": 0,
                "DEFINED": 0,
                "DEQUEUED": 0,
                "FAILED": 0,
                "FINISHED": 0,
                "RUNNING": 0
            },
            "class_name": "CmonJobStatistics",
            "cluster_id": 1
        },
        "log_file": "/tmp/cmon_1.log",
        "maintenance_mode_active": false,
        "managed": true,
        "node_auto_recovery": true,
        "owner":
        {
            "class_name": "CmonUser",
```


(continues on next page)

(continued from previous page)


```
            "email_address": "laszlo@severalnines.com",
            "first_name": "Laszlo",
            "groups": [
            {
                "class_name": "CmonGroup",
                "group_id": 4,
                "group_name": "testgroup"
            } ],
            "last_login": "2017-07-28T04:29:14.185Z",
            "last_name": "Pere",
            "title": "",
            "user_id": 3,
            "user_name": "pipas"
        },
        "state": "STARTED",
        "status_text": "All nodes are operational.",
        "vendor": "postgres",
        "version": "9.6"
    } ],
    "reply_received": "2017-07-28T04:29:14.197Z",
    "request_created": "2017-07-28T04:29:14.193Z",
    "request_id": 3,
    "request_processed": "2017-07-28T04:29:14.241Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 1
}
```


For the "getAllClusterInfo" request the controller is able to send back a lot of information. A full view of the cluster can be implemented using the reply. The following main categories can be found here:

- `cluster`: A list of clusters. All the details are categorised under these cmonclusterinfo objects.
- `hosts`: The list of hosts that are part of the cluster. This piece of information is sent if the request had the "with_hosts" property set to true.
- `info`: The information that the sheet manager collected about the cluster.
- `alarm_statistics`: Shows the number of alarms on the syste. further information can be found in the CmonSimpleObject documentation.
- `job_statistics`: Statistical information about the jobs the cluster has. Further information can be found in the CmonJobStatistics page.
- `total`: The total number of clusters managed by the controller.

getAllClusterInfoWithAllPool

This request behaves exactly like getAllClusterInfo but aggregates data from all active controllers in the pool. It applies the same filtering logic, access controls, and generates the same response structure as the normal getAllClusterInfo method.

The difference is that instead of getting data from the local controller only, it retrieves cluster information from all active controllers and presents them in a unified response.

createAccount

This request can be used to create an account on the cluster. Please note that this is not going to be a Cmon User, this RPC is for creating SQL users.

```
{
    "account":
    {
        "class_name": "CmonAccount",
        "grants": "john:ALL;pipas:INSERT",
        "host_allow": "1.2.3.4",
        "own_database": "laszlo",
        "password": "passwd",
        "user_name": "laszlo"
    },
    "cluster_id": 1,
    "operation": "createAccount",
    "request_created": "2017-01-27T14:03:09.491Z"
}
```

- `account`: A CmonAccount class object that defines the properties of the account that will be created.

  To get a comprehensive list of the properties the controller supports in the CmonAccount class one can run the following command (or check the cmonaccount section):

  ```
  s9s metatype --list-properties --type=CmonAccount --long
  ```

- `cluster_id`: The numerical ID of the cluster on which the account will be created. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster on which the new account will be created. The cluster can also be identified by its numerical ID.

updateAccount

This request can be used to update an account on the cluster. Please note that this is not going to be a Cmon User, this RPC is for creating SQL users.

```
{
    "account":
    {
        "class_name": "CmonAccount",
        "grants": "john:ALL;pipas:INSERT",
        "host_allow": "1.2.3.4",
        "own_database": "laszlo",
        "password": "passwd",
        "user_name": "laszlo",
        "ssl_type": "SSL",
        "max_user_connections": 0,
        "max_questions": 0,
        "max_updates": 0,
        "max_connections": 0
    },
```

(continues on next page)

(continued from previous page)

```
    "cluster_id": 1,
    "operation": "updateAccount",
    "request_created": "2017-01-27T14:03:09.491Z"
}
```

- `account`: A CmonAccount class object that defines the properties of the account that will be updated.

  To get a comprehensive list of the properties the controller supports in the CmonAccount class one can run the following command (or check the cmonaccount section):

  ```
  s9s metatype --list-properties --type=CmonAccount --long
  ```

- `cluster_id`: The numerical ID of the cluster on which the account will be updated. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster on which the account will be updated. The cluster can also be identified by its numerical ID.

getAccounts

The getAccounts call can be used to get the accounts from the cluster. Here is how a simple call look like:

```
{
    "cluster_id": 1,
    "operation": "getAccounts",
    "request_created": "2017-08-10T03:03:49.592Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster to get. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster to get. The cluster can also be identified by its numerical ID.
- `limit`: Limits the number of accounts sent back in the reply.
- `offset`: The offset of the first account sent back in the reply.

Here is an example of the reply:

```
{
    "accounts": [
    {
        "class_name": "CmonAccount",
        "connections": 1,
        "grants": "CREATE USER,CREATE␣
↪DATABASE,LOGIN,REPLICATION,SUPER;template1:CREATE,TEMPORARY,CONNECT;template0:CREATE,TE
↪
        "host_allow": "",
        "max_connections": 0,
        "password": "",
        "user_name": "postgres"
```

(continues on next page)

(continued from previous page)

```
    },

    . . .

    {
        "class_name": "CmonAccount",
        "grants": "LOGIN;joe:CREATE,TEMPORARY,CONNECT",
        "host_allow": "",
        "max_connections": 0,
        "password": "",
        "user_name": "joe"
    } ],
    "request_created": "2017-08-10T03:03:49.592Z",
    "request_id": 3,
    "request_processed": "2017-08-10T03:03:49.639Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 5
}
```

- `accounts`: A list of cmonaccount objects, the accounts found on the cluster.
- `total`: The total number of accounts found on the cluster regardless the number of accounts sent back in the reply.

listAccounts

The listAccounts call can be used to get the accounts from the cluster. Here is how a simple call look like:

```
{
    "cluster_id": 1,
    "operation": "listAccounts",
    "request_created": "2017-08-10T03:03:49.592Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster to get. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster to get. The cluster can also be identified by its numerical ID.
- `limit`: Limits the number of accounts sent back in the reply.
- `offset`: The offset of the first account sent back in the reply.

Here is an example of the reply:

```
{
    "queryResults": [
      {
        "accounts": [
         {
             "class_name": "CmonAccount",
```

(continues on next page)

(continued from previous page)


```
            "connections": 1,
            "grants": "CREATE USER,CREATE␣
↪DATABASE,LOGIN,REPLICATION,SUPER;template1:CREATE,TEMPORARY,CONNECT;template0:CREATE,TE
↪
            "host_allow": "",
            "max_connections": 0,
            "password": "",
            "user_name": "postgres"
        },

        . . .

        {
            "class_name": "CmonAccount",
            "grants": "LOGIN;joe:CREATE,TEMPORARY,CONNECT",
            "host_allow": "",
            "max_connections": 0,
            "password": "",
            "user_name": "joe"
        } ],
        "hostname": "masterHostName123",
        "port": 3306
      }
  ],
  "request_created": "2017-08-10T03:03:49.592Z",
  "request_id": 3,
  "request_processed": "2017-08-10T03:03:49.639Z",
  "request_status": "Ok",
  "request_user_id": 3,
  "total": 5
}
```


- `accounts`: A list of cmonaccount objects, the accounts found on the cluster.
- `total`: The total number of accounts found on the cluster regardless the number of accounts sent back in the reply.

deleteAccount

This request is for deleting a user account from a cluster.


```
{
    "account":
    {
        "class_name": "CmonAccount",
        "user_name": "pipas"
    },
    "cluster_id": 1,
    "operation": "deleteAccount",
    "request_created": "2017-01-30T13:12:30.192Z"
}
```

- `account`: The account that should be deleted. To get a comprehensive list of the properties the controller supports in the CmonAccount class one can run the following command (or check the cmonaccount section):

```
s9s metatype --list-properties --type=CmonAccount --long
```

- `cluster_id`: The numerical ID of the cluster from which the account should be removed. The cluster can also be identified by the cluster name.
- `cluster_name`: The name of the cluster from which the account will be removed. The cluster can also be identified by its numerical ID.

grantPrivileges

The request can be used to grant privileges for a existing user to an existing database.

```
{
    "account":
    {
        "class_name": "CmonAccount",
        "user_name": "pipas"
    },
    "cluster_id": 1,
    "operation": "grantPrivileges",
    "privileges": "testCreateDatabase.*:DELETE,TRUNCATE",
}
```

- `account`: The account that will be granted some privileges on some database. To get a comprehensive list of the properties the controller supports in the CmonAccount class one can run the following command (or check the cmonaccount section):

```
s9s metatype --list-properties --type=CmonAccount --long
```

- `cluster_id`: The numerical ID of the cluster on which the privileges will be granted. The cluster can also be identified by name.
- `cluster_name`: The name of the cluster on which the privileges will be granted. The cluster can also be identified by the cluster ID.
- `privileges`: The privileges to grant. This property should be a string in pdl_language format.

```
{
    "messages": [ "Grant processed: GRANT DELETE,TRUNCATE ON DATABASE␣
↪testCreateDatabase TO pipas" ],
    "request_processed": "2017-06-22T08:49:25.548Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

revokePrivileges

The request can be used to revoke privileges from an existing user to an existing database.

```
{
    "account":
    {
        "class_name": "CmonAccount",
        "user_name": "pipas"
    },
    "cluster_id": 1,
    "operation": "revokePrivileges",
    "privileges": "testCreateDatabase.*:DELETE,TRUNCATE",
}
```

- `account`: The account from privileges will be revoked on some database. To get a comprehensive list of the properties the controller supports in the CmonAccount class one can run the following command (or check the cmonaccount section):

  ```
  s9s metatype --list-properties --type=CmonAccount --long
  ```

- `cluster_id`: The numerical ID of the cluster on which the privileges will be granted. The cluster can also be identified by name.
- `cluster_name`: The name of the cluster on which the privileges will be granted. The cluster can also be identified by the cluster ID.
- `privileges`: The privileges to grant. This property should be a string in pdl_language format.

```
{
    "messages": [ "Revoke processed: REVOKE DELETE,TRUNCATE ON DATABASE␣
↪testCreateDatabase FROM pipas" ],
    "request_processed": "2017-06-22T08:49:25.548Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

revokeAllPrivileges

The request can be used to revoke all privileges from an existing user.

```
{
    "account":
    {
        "class_name": "CmonAccount",
        "user_name": "pipas"
    },
    "cluster_id": 1,
    "operation": "revokePrivileges"
}
```

- `account`: The account from all privileges will be revoked. To get a comprehensive list of the properties the controller supports in the CmonAccount class one can run the following command (or check the cmonaccount section):

```
s9s metatype --list-properties --type=CmonAccount --long
```

- `cluster_id`: The numerical ID of the cluster on which the privileges will be granted. The cluster can also be identified by name.
- `cluster_name`: The name of the cluster on which the privileges will be granted. The cluster can also be identified by the cluster ID.


```
{
    "messages": [ "Revoke processed: REVOKE ALL PRIVILEGES ON DATABASE␣
↪testCreateDatabase FROM pipas" ],
    "request_processed": "2017-06-22T08:49:25.548Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


getSqlProcesses

Returns the list of currently active SQL processes (connections and running queries) across all nodes in a cluster. Useful for monitoring active workloads and identifying long-running or blocking queries.


```
{
    "cluster_id": 1,
    "operation": "getSqlProcesses",
    "request_created": "2020-02-28T06:24:36.355Z",
    "request_id": 3
}
```



```
{
    "debug_messages": [ "RPC V2 authenticated user is 'pipas'." ],
    "processes": [
    {
        "blocked_by_trx_id": "",
        "client": "",
        "command": "Sleep",
        "currentTime": 1582871076,
        "db": "",
        "duration": 139914683312432,
        "host": "",
        "hostId": 1,
        "hostname": "192.168.0.227",
        "info": "",
        "innodb_status": "",
        "innodb_trx_id": "",
        "instance": "192.168.0.227:3306",
        "lastseen": 139914683312176,
        "message": "",
        "mysql_trx_id": 1,
        "pid": 1,
        "query": "",
        "reportTs": 1582871076,
```


(continues on next page)

(continued from previous page)


```
        "sql": "",
        "state": "wsrep aborter idle",
        "time": 664,
        "user": "system user"
    },
    {
        "blocked_by_trx_id": "",
        "client": "",
        "command": "Sleep",
        "currentTime": 1582871076,
        "db": "",
        "duration": 139914683312432,
        "host": "",
        "hostId": 3,
        "hostname": "192.168.0.220",
        "info": "",
        "innodb_status": "",
        "innodb_trx_id": "",
        "instance": "192.168.0.220:3306",
        "lastseen": 139914683312176,
        "message": "",
        "mysql_trx_id": 5,
        "pid": 5,
        "query": "",
        "reportTs": 1582871076,
        "sql": "",
        "state": "",
        "time": 130,
        "user": "system user"
    } ],
    "request_created": "2020-02-28T06:24:36.355Z",
    "request_id": 3,
    "request_processed": "2020-02-28T06:24:36.378Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```


getTopQueries

This request can be used to get the top queries, actually the top query patterns (AKA digests).

Here is how a request should look like:


```
{
    "cluster_id": 1,
    "filterStrings": [ "*:3306" ],
    "limit": 10,
    "offset": 10,
    "operation": "getTopQueries",
    "request_created": "2020-03-30T09:57:12.971Z",
```


(continues on next page)

(continued from previous page)

```
    "request_id": 3
}
```

- `limit`: Optional number that limits the number of items returned.
- `offset`: Optional number, the offset of the first item returned.
- filterString: Optional list of filters to be applied on the server name and port (the "instance" of the entry) the same way the file name patterns are handled in the UNIX shell.

Here is an example reply for th getTopQueries request.


```
{
    "debug_messages": [ "RPC V2 authenticated user is 'pipas'." ],
    "digests": [
    {
        "affectedRows": 9020,
        "class_name": "CmonStatementDigest",
        "count": 9020,
        "databaseName": "pipas1_3198",
        "dtFirstSeen": "2020-03-25T14:37:24.000Z",
        "dtLastSeen": "2020-03-25T14:44:02.000Z",
        "instance": "192.168.0.215:6032",
        "statementPattern": "INSERT INTO test_table(NAME, VALUE) VALUES(?, ?
↪)",
        "waitMillisMax": 610.891,
        "waitMillisMin": 8.935,
        "waitMillisSum": 324158
    },
    {
        "affectedRows": 9020,
        "class_name": "CmonStatementDigest",
        "count": 9020,
        "databaseName": "pipas1_3198",
        "dtFirstSeen": "2020-03-25T14:37:24.000Z",
        "dtLastSeen": "2020-03-25T14:44:02.000Z",
        "instance": "192.168.0.227:3306",
        "statementPattern": "INSERT INTO `test_table` ( NAME , VALUE )␣
↪VALUES (...) ",
        "waitMillisAvg": 35.7737,
        "waitMillisMax": 610.721,
        "waitMillisMin": 8.78106,
        "waitMillisSum": 322679
    } ],
    "request_created": "2020-03-26T08:33:01.146Z",
    "request_id": 3,
    "request_processed": "2020-03-26T08:33:01.202Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "total": 37
}
```

createDatabase

This request can be used to create a database on the cluster.

```
{
    "cluster_id": 1,
    "database":
    {
        "class_name": "CmonDataBase",
        "database_name": "dbname"
    },
    "operation": "createDatabase",
    "request_created": "2017-06-20T07:24:15.815Z"
}
```

- `cluster_id`: The numerical ID of the cluster on which the new database will be created. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster on which the new database will be created. The cluster can also be identified by its name.
- `database`: The database to be created.

```
{
    "database":
    {
        "class_name": "CmonDataBase",
        "database_name": "testDatabase"
    },
    "messages": [ "Database 'testDatabase' created." ],
    "reply_received": "2017-08-10T08:16:17.949Z",
    "request_created": "2017-08-10T08:16:17.945Z",
    "request_id": 3,
    "request_processed": "2017-08-10T08:16:18.098Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

ping

The "ping" request can be used to check the connection to the Cmon Controller or keep it alive. Every ping request will be replied with a simple message that holds some general information about the controller and the connection.

Here is an example showing a ping request and the reply:

```
{
    "operation": "ping",
    "request_created": "2017-06-22T11:55:39.854Z",
    "request_id": 3
}
```

```
{
    "authenticated": true,
    "package_bugreport": "support@severalnines.com",
    "package_name": "cmon",
    "package_version": "1.4.3",
    "request_created": "2017-06-22T11:55:39.854Z",
    "request_id": 3,
    "request_processed": "2017-06-22T11:55:39.900Z",
    "request_status": "Ok",
    "request_user_id": 12,
    "rpc_version": "2.0"
}
```


availableUpgrades

This request is for reading a list of available package upgrades for the cluster. The cluster can be identified by the cluster ID or the cluster name.

```
{
    "cluster_id": 1,
    "operation": "availableUpgrades",
}
```

- `nodes`: List of nodes (hosts) to query available package upgrades on.
- `cluster_id`: The numerical ID of the cluster to get. The cluster can also be identified by its name.
- `cluster_name`: The name of the cluster to get. The cluster can also be identified by its numerical ID.

3.12.9 Configuration

The `/v2/config` path provides access to various configuration files.

getConfig

This call is for getting the configuration of one specific node in a cluster.

The call looks like this:

```
{
    "hostname": "192.168.1.127",
    "operation": "getConfig",
    "request_created": "2017-07-28T09:48:30.491Z",
    "request_id": 3
}
```

Access rights: The authenticated user has to have read access on both the host and the cluster to get successful reply on this call.

The reply contains the configuration if the node has some configuration file(s):

```
{
    "config":
    {
        "className": "CmonClusterConfig",
        "files": [
        {
            "filename": "cmon_1.cnf",
            "values": [
            {
                "filepath": "/tmp/cmon_1.cnf",
                "linenumber": 1,
                "section": "",
                "value": "1",
                "variablename": "cluster_id"
            },

            . . .

            {
                "filepath": "/tmp/cmon_1.cnf",
                "linenumber": 29,
                "section": "",
                "value": "postgres",
                "variablename": "vendor"
            } ]
        } ],
        "host":
        {
            "class_name": "CmonHost",
            "hostname": "192.168.1.127",
            "port": 9555
        }
    },
    "files": [
    {
        "content":
↪"cluster_id=1\ncluster_type=postgresql_single\ncmon_db=ft_install_testdb\ncmon_user=roo
↪'p'\nmysql_port=3306\nname='ft_postgresql_63564'\
↪nos=debian\nosuser=pipas\nowner=3\npidfile=/var/run\npostgresql_password=passwd12\npost
↪
        "filename": "cmon_1.cnf",
        "path": "/tmp/cmon_1.cnf",
        "syntax": "MySqlConfigSyntax"
    } ],
    "request_created": "2017-07-28T09:48:30.491Z",
    "request_id": 3,
    "request_processed": "2017-07-28T09:48:30.541Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `config`: This contains the configuration of the host.
- `files`: One host can have multiple configuration files, so this part contains a list one element for every configuration file.
- `values`: This is the parsed version of the configuration. Every item is one configuration value in some configuration file.
- `content`: The literal content of the configuration file escaped for JSON format.
- `path`: The full path of the configuration file.
- `syntax`: The syntax of the configuration file. This can be GenericConfigSyntax, MySqlConfigSyntax, ProxySqlConfigSyntax, HaProxyConfigSyntax, YamlSyntax, UnknownSyntax or MongoConfigSyntax which is the same as YamlSyntax.

getConfigs

A method to obtain all configuration files of a cluster.

The call looks like this:

```
{
    "operation": "getConfigs",
}
```

Access rights: The authenticated user has to have read access on the cluster to get successful reply on this call.

The reply contains the configurations of the nodes. file(s):

```
{
    "configs":
    [
      {"config": {
        "className": "CmonClusterConfig",
        "files": [
        {
            "filename": "cmon_1.cnf",
            "values": [
            {
                "filepath": "/tmp/cmon_1.cnf",
                "linenumber": 1,
                "section": "",
                "value": "1",
                "variablename": "cluster_id"
            },

            . . .

            {
                "filepath": "/tmp/cmon_1.cnf",
                "linenumber": 29,
                "section": "",
                "value": "postgres",
```

(continues on next page)

(continued from previous page)


```
                "variablename": "vendor"
            } ]
        } ],
        "host":
        {
            "class_name": "CmonHost",
            "hostname": "192.168.1.127",
            "port": 9555
        }
      },
      "files": [ {
        "content":
↪"cluster_id=1\ncluster_type=postgresql_single\ncmon_db=ft_install_testdb\ncmon_user=roo
↪'p'\nmysql_port=3306\nname='ft_postgresql_63564'\
↪nos=debian\nosuser=pipas\nowner=3\npidfile=/var/run\npostgresql_password=passwd12\npost
↪
        "filename": "cmon_1.cnf",
        "path": "/tmp/cmon_1.cnf",
        "syntax": "MySqlConfigSyntax"
      } ]},
    ],
    "request_created": "2023-02-23T09:48:30.491Z",
    "request_id": 3,
    "request_processed": "2023-02-23T09:48:30.541Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


setConfig

This call is for changing individual configuration values for a host. Multiple values can be changed by sending one request.

Access rights: The authenticated user has to have write access on both the host and the cluster to get successful reply on this call.

Here is how the request looks like:


```
{
    "configuration": [
    {
        "name": "log_line_prefix",
        "value": "'%m'"
    } ],
    "hostname": "192.168.1.127",
    "operation": "setConfig",
    "port": 9555,
    "request_created": "2017-07-28T12:06:53.093Z",
    "request_id": 3
}
```

- cluster-id: This field is not mandatory but might be used to identify the cluster
- `configuration`: A list of values to be changed.
- `hostname`: The name of the host where the configuration should be changed.
- `port`: If the hostname is not enough to identify the node the sender can provide the port number to identify the node. The port is not mandatory, but might be needed.

The reply will look something like this:

```
{
    "messages": [ "Node restart is required for change to take effect." ],
    "request_created": "2017-07-28T12:06:53.093Z",
    "request_id": 3,
    "request_processed": "2017-07-28T12:06:53.157Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `messages`: This call can send multiple messages back. Changing the configuration is a complex step and may result in more than one messages.

setConfig

This call is for unsetting, commenting out individual configuration values for a host. Multiple values can be unset by sending one request.

Access rights: The authenticated user has to have write access on both the host and the cluster to get successful reply on this call.

Here is how the request looks like:

```
{
    "configuration": [
    {
        "group": "groupname",
        "name": "log_line_prefix",
    } ],
    "hostname": "192.168.1.127",
    "operation": "unsetConfig",
    "port": 9555,
    "request_created": "2017-07-28T12:06:53.093Z",
    "request_id": 3
}
```

- cluster-id: This field is not mandatory but might be used to identify the cluster
- `configuration`: A list of variable names to be changed.
- `hostname`: The name of the host where the configuration should be changed.
- `port`: If the hostname is not enough to identify the node the sender can provide the port number to identify the node. The port is not mandatory, but might be needed.

The reply will look something like this:

```
{
    "messages": [ "Node restart is required for change to take effect." ],
    "request_created": "2017-07-28T12:06:53.093Z",
    "request_id": 3,
    "request_processed": "2017-07-28T12:06:53.157Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `messages`: This call can send multiple messages back. Changing the configuration is a complex step and may result in more than one messages.

getLdapConfig

This call is for reading the LDAP configuration. Access to the LDAP configuration is controlled by the access rights for the "/.runtime/LDAP/configuration" CDT entry, whoever has read access to that CDT entry can get the LDAP configuration through this call.

Here is how a request should look like:


```
{
  "operation": "getLdapConfig",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2
}
```


And here is a reply containing the LDAP configuration.


```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "effective_privileges": "rwx",
  "reply_received": "2021-01-13T07:22:33.878Z",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2,
  "request_processed": "2021-01-13T07:22:33.877Z",
  "request_status": "Ok",
  "request_user_id": 1,
  "debug_messages": [ "RPC V2 authenticated user is 'system'."  ],
  "ldap_configuration": {
    "enabled": false,
    "ldapAdminPassword": null,
    "ldapAdminUser": null,
    "ldapGroupSearchRoot": null,
    "ldapServerUri": null,
    "ldapUserSearchRoot": null,
    "defaults": {
      "enabled": false,
      "ldapAdminPassword": "",
      "ldapAdminUser":
↪"CN=Administrator,CN=Users,DC=ldap,DC=mydomain,DC=com",
      "ldapGroupSearchRoot": "CN=Groups,OU=IT,DC=ldap,DC=mydomain,DC=com",
```


(continues on next page)

(continued from previous page)


```
      "ldapServerUri": "ldaps://ldap.mydomain.com:686",
      "ldapUserSearchRoot": "CN=Users,DC=ldap,DC=mydomain,DC=com",
      "groupMappings": [  ],
      "ldapSettings": {
        "ldapEmailAttributes": "mail,userPrincipalName",
        "ldapGroupClassName": "",
        "ldapGroupIdAttributes": "dn,uid",
        "ldapGroupNameAttribute": "cn",
        "ldapGroupSearchRoot": "CN=Groups,OU=IT,DC=ldap,DC=mydomain,DC=com",
        "ldapMemberAttributes": "member,memberUid",
        "ldapNetworkTimeout": 5,
        "ldapProtocolVersion": 3,
        "ldapQueryTimeLimit": 5,
        "ldapRealnameAttributes": "cn,displayName",
        "ldapUserClassName": "objectclass=*",
        "ldapUsernameAttributes": "uid,sAMAccountName"
      },
      "security": {
        "caCertFile": "/etc/ssl/certs/ldap-ca.crt",
        "certFile": "/etc/ssl/certs/ldap.crt",
        "keyFile": "/etc/ssl/certs/ldap.pem"
      }
    },
    "groupMappings": [  ],
    "ldapSettings": {
      "ldapEmailAttributes": null,
      "ldapGroupClassName": null,
      "ldapGroupIdAttributes": null,
      "ldapGroupNameAttribute": null,
      "ldapMemberAttributes": null,
      "ldapNetworkTimeout": null,
      "ldapProtocolVersion": null,
      "ldapQueryTimeLimit": null,
      "ldapRealnameAttributes": null,
      "ldapUserClassName": null,
      "ldapUsernameAttributes": null
    },
    "security": {
      "caCertFile": null,
      "certFile": null,
      "keyFile": null
    }
  }
}
```


- `effective_privileges`: This field shows what access privileges the authenticated user has. So with this field the caller can immediately decide if the user has the rights to change the settings or not.
- `ldap_configuration`: The LDAP configuration.

setLdapConfig

This call is for writing the LDAP configuration. Access to the LDAP configuration is controlled by the access rights for the "/.runtime/LDAP/configuration" CDT entry, whoever has write access to that CDT entry can set the LDAP configuration through this call.

Here is how a request should look like:

```
{
  "operation": "setLdapConfig",
  "request_created": "2021-01-13T07:22:33.966Z",
  "request_id": 2,
  "ldap_configuration": {
    "enabled": true,
    "ldapAdminPassword": "p",
    "ldapAdminUser": "cn=admin,dc=homelab,dc=local",
    "ldapGroupSearchRoot": "dc=homelab,dc=local",
    "ldapServerUri": "ldap://ldap.homelab.local:389",
    "ldapUserSearchRoot": "dc=homelab,dc=local",
    "groupMappings": [ {
        "cmonGroupName": "ldapgroup",
        "ldapGroupId": "ldapgroup",
        "sectionName": "mapping1"
      }  ],
    "ldapSettings": {
      "ldapEmailAttributes": "mail",
      "ldapGroupClassName": null,
      "ldapGroupIdAttributes": null,
      "ldapGroupNameAttribute": null,
      "ldapMemberAttributes": "memberUid",
      "ldapNetworkTimeout": null,
      "ldapProtocolVersion": null,
      "ldapQueryTimeLimit": null,
      "ldapRealnameAttributes": "displayName,cn",
      "ldapUserClassName": null,
      "ldapUsernameAttributes": "cn"
    },
    "security": {
      "caCertFile": null,
      "certFile": null,
      "keyFile": null
    }
  }
}
```

- `ldap_configuration`: The LDAP configuration.

And here is a reply:

```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "reply_received": "2021-01-13T07:22:33.980Z",
```

(continues on next page)

(continued from previous page)

```
  "request_created": "2021-01-13T07:22:33.966Z",
  "request_id": 2,
  "request_processed": "2021-01-13T07:22:33.979Z",
  "request_status": "Ok",
  "request_user_id": 1,
  "debug_messages": [ "RPC V2 authenticated user is 'system'."  ]
}
```

listTemplates

This call is to list the ClusterControl provided configuration templates.

Here is how a request should look like:

```
{
  "operation": "listTemplates",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2
}
```

And here is a reply containing the configuration templates.

```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "reply_received": "2021-01-13T07:22:33.878Z",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2,
  "request_processed": "2021-01-13T07:22:33.877Z",
  "request_status": "Ok",
  "data":
  {
      "cluster_type": "replication",
      "config_templates": [ "my.cnf.gtid_replication" ],
      "vendor": "percona",
      "version": "5.6",
      "requestStatus": "ok",
  }
}
```

setLicense

immediately.

Here is how a request should look like:

```
{
  "operation": "setLicense",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2,
  "licensedata":
```

(continues on next page)

(continued from previous page)

```
↪"ewogICAgImNvbXBhbnkiOiAiU2FudGEgQ2xhdXNlIEluYy4iLAogICAgImVtYWlsX2FkZHJlc3Mi\nOiAic2Fu
}
```

And here is a reply containing the confirmed license.


```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "reply_received": "2021-01-13T07:22:33.878Z",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2,
  "request_processed": "2021-01-13T07:22:33.877Z",
  "request_status": "Ok",
  "data":
  {
     "hasLicense":true,
     "licenseExpires":387,
     "licenseStatus":"License found.",
     "license":{
         "class_name":"CmonLicense",
         "company":"Santa Clause Inc.",
         "days_left":16,
         "email_address":"santa@hoho.ho",
         "expiration_date":"2021-12-24T00:00:00.001Z",
         "licensed_nodes":7,
         "type":"Demo",
         "used_nodes":89,
         "valid_date":true
     }
  }
}
```


getLicense

Returns the current license information registered with the controller, including the license type, expiration date, licensed node count, and current usage.

```
{
  "operation": "getLicense",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 3,
}
```

And here is a reply containing the confirmed license.


```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "reply_received": "2021-01-13T07:22:33.878Z",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2,
  "request_processed": "2021-01-13T07:22:33.877Z",
```


(continues on next page)

(continued from previous page)


```
  "request_status": "Ok",
  "data":
  {
     "hasLicense":true,
     "licenseExpires":387,
     "licenseStatus":"License found.",
     "license":{
         "class_name":"CmonLicense",
         "company":"Santa Clause Inc.",
         "days_left":16,
         "email_address":"santa@hoho.ho",
         "expiration_date":"2021-12-24T00:00:00.001Z",
         "licensed_nodes":7,
         "type":"Demo",
         "used_nodes":89,
         "valid_date":true
     }
  }
}
```


setmailserver

Here is how a request should look like:

```
{
  "operation": "setmailserver",
  "smtp_server":
  {
    "hostname": "my.smtp.server",
    "password": "password",
    "port": 587,
    "sender": "hullala@smtp.server.tld",
    "use_tls": false,
    "username": "hullala@smtp.server.tld"
  }
}
```

A response is just simply a generic one:


```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "reply_received": "2021-01-13T07:22:33.878Z",
  "request_created": "2021-01-13T07:22:33.872Z",
  "request_id": 2,
  "request_processed": "2021-01-13T07:22:33.877Z",
  "request_status": "Ok",
}
```

getmailserver

Here is how a request should look like:

```
{
  "operation": "getmailserver",
}
```

And the response gives back the SMTP server details:

```
{
  "controller_id": "ae03400a-b1d1-42eb-98ad-3196acc1ec23",
  "reply_received": "2022-01-13T07:22:33.878Z",
  "request_created": "2022-01-13T07:22:33.872Z",
  "request_id": 3,
  "request_processed": "2022-01-13T07:22:33.877Z",
  "request_status": "Ok",
  "smtp_server":
  {
      "hostname": "my.smtp.server",
      "password": "password",
      "port": 587,
      "sender": "hullala@smtp.server.tld",
      "use_tls": false,
      "username": "hollala@smtp.server.tld"
  }
}
```

setContent

This call can be used to overwrite a configuration file content in a free way.

Here is how the request looks like:

```
{
    "content": "[mysql]\\nport=322\\nlisten_address=\"0.0.0.0\"\\n\\n",
    "cluster_id": 114,
    "hostname": "192.168.1.127",
    "operation": "setContent",
    "filename": "/etc/my.cnf",
    "port": 9555,
    "request_created": "2023-02-06T12:06:53.093Z",
    "request_id": 3
}
```

The reply will look something like this:

```
{
    "messages": [ "Node restart is required for change to take effect." ],
    "request_created": "2017-07-28T12:06:53.093Z",
    "request_id": 3,
    "request_processed": "2017-07-28T12:06:53.157Z",
```

(continues on next page)

(continued from previous page)

```
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `messages`: This call can send multiple messages back. Changing the configuration is a complex step and may result in more than one messages.

addIntegration

This call can be used to add a cmon-events (clustercontrol-notifications package) external integration.

Here is how the request looks like:

```
{
   "operation": "addintegration",
   "integration": {
       "name": "webhook",
       "service_id": "webhook",
       "is_active": true,
       "config": "{\
↪"clusters\":[-1],\"triggers\":[{\"trigger\":\"ALL\",\"severity\":\"ALL\"}],\"name\":\"w
   }
}
```

updateIntegration

This call can be used to update (by id) a cmon-events (clustercontrol-notifications package) external integration.

Here is how the request looks like:

```
  {
     "operation": "updateintegration",
     "integration": {
         "id": 1,
         "name": "webhook",
         "service_id": "webhook",
         "is_active": true,
         "config": "{\
↪"clusters\":[-1],\"triggers\":[{\"trigger\":\"ALL\",\"severity\":\"ALL\"}],\"name\":\"w
     }
  }
```

The reply will look something like this:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-05-05T12:16:31.637Z",
  "request_status": "Ok",
  "debug_messages":
```

(continues on next page)

(continued from previous page)


```
  [
      "RPC V2 authenticated user is 'alex'."
  ],
  "integration":
  {
      "name": "webhook",
      "config": "{\
↪"clusters\":[-1],\"triggers\":[{\"trigger\":\"ALL\",\"severity\":\"ALL\"}],\"name\":\"w
↪
      "id": 1,
      "is_active": true,
      "service_id": "webhook"
  }
}
```


The other operation is 'updateIntegration' then you must supply an existing 'id' otherwise the request will fail.

removeIntegration

This call can be used to remove an external integration.

Here is how the request looks like:


```
{
   "operation": "removeintegration",
   "id": 1
}
```


The reply will look something like this:


```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-05-05T12:16:31.637Z",
  "request_status": "Ok",
  "debug_messages":
  [
      "RPC V2 authenticated user is 'alex'."
  ]
}
```


listIntegrations

This call can be used to list external integrations, you may use pagination (limit, offset) arguments here.

Here is how the request looks like:


```
{
   "operation": "listintegrations"
}
```

The reply will look something like this:


```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-05-05T12:16:31.637Z",
  "request_status": "Ok",
  "total": 1,
  "debug_messages":
  [
      "RPC V2 authenticated user is 'alex'."
  ],
  "integrations":
  [
      {
          "name": "webhook",
          "config": "{\
↪"clusters\":[-1],\"triggers\":[{\"trigger\":\"ALL\",\"severity\":\"ALL\"}],\"name\":\"w
↪
          "id": 1,
          "is_active": false,
          "service_id": "webhook"
      }
  ]
}
```


getcomponentmeta

A method to obtain human readable description of e-mail notification components.

Here is how the request looks like:

```
{
   "operation": "getcomponentmeta"
}
```

The reply will look something like this:


```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-05-05T12:16:31.637Z",
  "request_status": "Ok",
  "total": 1,
  "debug_messages":
  [
      "RPC V2 authenticated user is 'alex'."
  ],
  "data":
  [
    {
        "alarm_name": "Network alarms",
```


(continues on next page)

(continued from previous page)

```
        "component": "Network",
        "log_name": "Network related logs",
        "message_name": "Network related messages, e.g host unreachable,␣
↪SSH issues."
    },
    {
        "alarm_name": "Cmon Internal alarms",
        "component": "CmonDatabase",
        "log_name": "Cmon database logs",
        "message_name": "Internal Cmon database related messages."
     }
  ]
}
```

getrecipients

Obtain the recipients list.

```
{
   "operation": "getrecipients",
   "cluster_id": 101
}
```

The reply will look something like this:

```
{
  "request_processed": "2023-05-19T15:03:23.080Z",
  "request_status": "Ok",
  "total": 3,
  "data":
  [
      "testing1000@aaaa.bbb",
      "testing100@aaaa.bbb",
      "testing10@aaaa.bbb"
  ]
}
```

setnotificationsettings

Sets/updates the notification settings for the specified email.

```
{
  "cluster_id": 200,
  "daily_limit": 1000,
  "digest_hour": 7,
  "email": "testing1000@aaaa.bbb",
  "operation": "setnotificationsettings",
  "settings":
  {
      "Cluster":
```

(continues on next page)

(continued from previous page)

```
        {
            "CRITICAL": "Ignore",
            "INFO": "Ignore",
            "WARNING": "Ignore"
        },
        "Network":
        {
            "CRITICAL": "Digest",
            "INFO": "Digest",
            "WARNING": "Digest"
        }
    },
    "time_zone": 0
}
```

The reply will look something like this:

```
{
    "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
    "is_superuser": true,
    "request_processed": "2023-05-05T12:16:31.637Z",
    "request_status": "Ok",
    "total": 1,
    "data":
    [
        {
            "daily_limit": 1000,
            "digest_hour": 7,
            "recipient": "testing1000@aaaa.bbb",
            "time_zone": 0,
            "settings":
            {
                "Backup":
                {
                    "CRITICAL": "Digest",
                    "INFO": "Digest",
                    "WARNING": "Digest"
                },
                "Cluster":
                {
                    "CRITICAL": "Ignore",
                    "INFO": "Ignore",
                    "WARNING": "Ignore"
                },
                "ClusterConfiguration":
                {
                    "CRITICAL": "Digest",
                    "INFO": "Digest",
                    "WARNING": "Digest"
                },
```

(continues on next page)

(continued from previous page)

```
            "ClusterRecovery":
            {
                "CRITICAL": "Deliver",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "CmonDatabase":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "DbHealth":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "DbPerformance":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Host":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Mail":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Network":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Node":
            {
                "CRITICAL": "Deliver",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "SoftwareInstallation":
```

(continues on next page)

(continued from previous page)


```
                {
                    "CRITICAL": "Digest",
                    "INFO": "Deliver",
                    "WARNING": "Digest"
                },
                "Unknown":
                {
                    "CRITICAL": "Digest",
                    "INFO": "Digest",
                    "WARNING": "Digest"
                }
            }
        }
    ]
}
```


getnotificationsettings

Returns the notification settings of a specified recipient.


```
{
   "operation": "getnotificationsettings",
   "email": "testing1000@aaaa.bbb",
   "cluster_id": 101
}
```


The reply will look something like this:


```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-05-05T12:16:31.637Z",
  "request_status": "Ok",
  "total": 1,
  "data":
  [
      {
          "daily_limit": 1000,
          "digest_hour": 7,
          "recipient": "testing1000@aaaa.bbb",
          "time_zone": 0,
          "settings":
          {
              "Backup":
              {
                  "CRITICAL": "Digest",
                  "INFO": "Digest",
                  "WARNING": "Digest"
              },
              "Cluster":
```


(continues on next page)

(continued from previous page)

```
            {
                "CRITICAL": "Ignore",
                "INFO": "Ignore",
                "WARNING": "Ignore"
            },
            "ClusterConfiguration":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "ClusterRecovery":
            {
                "CRITICAL": "Deliver",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "CmonDatabase":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "DbHealth":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "DbPerformance":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Host":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Mail":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Network":
            {
```

(continues on next page)

(continued from previous page)

```
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "Node":
            {
                "CRITICAL": "Deliver",
                "INFO": "Digest",
                "WARNING": "Digest"
            },
            "SoftwareInstallation":
            {
                "CRITICAL": "Digest",
                "INFO": "Deliver",
                "WARNING": "Digest"
            },
            "Unknown":
            {
                "CRITICAL": "Digest",
                "INFO": "Digest",
                "WARNING": "Digest"
            }
        }
    }
  ]
}
```

removerecipient

Removes an e-mail recipient.

```
{
   "operation": "removerecipient",
   "email": "someuser@mailhost.tld",
   "cluster_id": 101
}
```

The reply will look something like this:

```
{
  "request_processed": "2023-05-19T15:03:23.080Z",
  "request_status": "Ok"
}
```

3.12.10 Controllers and HA

The `/v2/controller` path is for managing controller and operate the Cmon HA subsystem.

ping

This call is created so that the client application can check on a controller even when that controller is not the leader. This call should be as fast as possible and should not do any locking on the Cmon HA subsystem.

```
{
    "operation": "ping",
    "request_created": "2020-01-02T10:01:08.035Z",
    "request_id": 1
}
```

```
{
    "cmon_ha_supported": true,
    "controller_status": "CmonControllerLeader",
    "term": 0,
    "prev_log_index": 0,
    "prev_log_term": 0,
    "request_created": "2020-01-02T10:01:08.035Z",
    "request_id": 1,
    "request_processed": "2020-01-02T10:01:08.044Z",
    "request_status": "Ok"
}
```

JoinRequest

An internal HA call used by a follower controller to request joining the controller pool managed by the leader. This is part of the CMON HA protocol and is not intended for direct use by external clients.

HeartBeat

An internal HA call exchanged between controllers to maintain cluster membership and detect node failures. The payload includes the current leader, committed log index, and known controller membership. This is part of the CMON HA protocol and is not intended for direct use by external clients.

```
{
    "client_ip_address": "192.168.0.127",
    "controller_key": "6a03558f-5d7a-41ed-9af1-9e0ff162bcb8",
    "failed_controllers":
    {
    },
    "follower_controllers":
    {
        "192.168.0.127:10001":
        {
            "class_name": "CmonControllerHost",
            "hostname": "192.168.0.127",
            "ip": "192.168.0.127",
            "port": 10001
        },
        "192.168.0.127:20001":
        {
```

(continues on next page)

(continued from previous page)


```
            "class_name": "CmonControllerHost",
            "hostname": "192.168.0.127",
            "ip": "192.168.0.127",
            "port": 20001
        }
    },
    "committed_index": 6,
    "leader_controller":
    {
        "class_name": "CmonControllerHost",
        "hostname": "192.168.0.127",
        "ip": "192.168.0.127",
        "port": 9556
    },
    "operation": "heartBeat",
    "request_created": "2019-03-19T09:43:58.444Z",
    "term": 0
}
```



```
{
    "cmon_ha_supported": true,
    "pid": 41918,
    "request_created": "2019-03-19T09:43:58.444Z",
    "request_processed": "2019-03-19T09:43:58.489Z",
    "request_status": "Ok",
    "wallclock": 1552988638
}
```


appendEntries

An internal Raft consensus call used to replicate log entries from the leader controller to its followers. Each entry carries a payload instruction — such as saving a configuration file — that the follower applies locally. This is part of the CMON HA replication protocol and is not intended for direct use by external clients.


```
{
    "client_ip_address": "192.168.0.127",
    "controller_key": "6a03558f-5d7a-41ed-9af1-9e0ff162bcb8",
    "log_entry":
    {
        "class_name": "CmonHaLogEntry",
        "comment": "",
        "committed": false,
        "executed": false,
        "index": 11,
        "prev_log_index": 10,
        "prev_log_term": 0,
        "payload":
        {
            "files": [
```


(continues on next page)

(continued from previous page)


```
            {
                "class_name": "CmonFile",
                "content":
↪"basedir=/usr\ncdt_path=/\ncluster_id=1\ncmon_db=ft_install_testdb\ncmon_user=root\ncre
↪'password'\
↪nmysql_bindir=/usr/bin/\nmysql_hostname=127.0.0.1\nmysql_password='p'\
↪nmysql_port=3306\nmysql_server_addresses=192.168.0.232:3306\nmysql_version=5.6\nname=
↪'ft_cmonhabasic_41388'\
↪nos=debian\nowner=1\npidfile=/var/run\nquery_monitor_use_local_settings=false\nssh_iden
↪-n 2>/dev/null\ntype=galera\nvendor=percona\n",
                "full_path": "/tmp/cmon_1.cnf"
            } ],
            "instruction": "SaveLocalFile"
        },
        "term": 0
    },
    "operation": "appendEntries",
    "request_created": "2019-03-19T09:48:32.864Z"
}
```



```
{
    "request_created": "2019-03-19T09:48:32.864Z",
    "request_processed": "2019-03-19T09:48:32.909Z",
    "request_status": "Ok"
}
```


requestVote

An internal Raft consensus call sent during leader election. A candidate controller sends this to request votes from the other controllers in the pool. This is part of the CMON HA protocol and is not intended for direct use by external clients.


```
{
    "client_ip_address": "192.168.0.127",
    "prev_log_index": 23,
    "prev_log_term": 42,
    "operation": "requestVote",
    "request_created": "2019-02-28T07:32:22.095Z",
    "term": 43
}
```

3.12.11 Discovery

The `/v2/discovery` path provides a set of calls helping the UI to check various input values during the interaction with the user.

checkClusterName

This call can be used to check if a specific cluster name is acceptable to be used to create a new cluster.

```
{
    "new_cluster_name": "Cluster_01",
    "operation": "checkClusterName",
    "request_created": "2017-08-15T09:32:29.823Z",
    "request_id": 3
}
```

- `new_cluster_name`: The new cluster name the user plans to use to name a new cluster.

```
{
    "is_acceptable": true,
    "message": "The cluster name 'Cluster_01' is acceptable.",
    "request_created": "2017-08-15T09:32:29.823Z",
    "request_id": 3,
    "request_processed": "2017-08-15T09:32:29.868Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `is_acceptable`: Shows if the cluster name is acceptable as a name for a new cluster. If the name is not valid for this purpose (e.g. the name already used for some other cluster) this value will be false.
- `message`: Shows why the name is not acceptable. This string is human readable, it can be shown to the user.
- `request_status`: Please note that the request status will be "Ok" even if the reply is negative, when the new name is not acceptable. This field shows if the request is processed, it is not the reply.

getSupportedClusterTypes

The getSupportedClusterTypes returns some metadata about the cluster types supported by the controller. This is a very important call, using it the GUI can fill up comboboxes, lists and can make sure that the cluster creation will not fail because the user requested some invalid combination. It is also worth mentioning that if the UI uses this call the UI source doesn't need to be changed when we start supporting a new version or vendor.

Here is how the call look like:

```
{
    "operation": "getSupportedClusterTypes",
    "request_created": "2017-08-15T08:37:30.150Z",
    "request_id": 3
}
```

And here is the reply. Please note that this call is under construction, we might add new fields if we find some other important information.

```
{
    "cluster_type_names": [ "galera", "replication", "groupreplication",
↪"myqlcluster", "postgresql", "mongodb" ],
    "cluster_type_properties":
    {
        "galera":
        {
            "intro_url": "http://galeracluster.com/products/",
            "long_name": "Galera Cluster for MySQL",
            "short_name": "Galera",
            "type_name": "galera",
            "vendors": [
            {
                "name": "mariadb",
                "versions": [ "5.5", "10.1", "10.2" ]
            },
            {
                "name": "codership",
                "versions": [ "5.5", "5.6" ]
            },
            {
                "name": "percona",
                "versions": [ "5.5", "5.6", "5.7" ]
            } ]
        },
        "groupreplication":
        {
            "intro_url":
↪"http://mysqlhighavailability.com/gr/doc/index.html",
            "long_name": "MySQL Group Replication",
            "short_name": "Group Replication",
            "type_name": "groupreplication",
            "vendors": [
            {
                "name": "oracle",
                "versions": [ "5.7" ]
            } ]
        },
        "mongodb":
        {
            "intro_url": "https://www.mongodb.com/what-is-mongodb",
            "long_name": "MongoDB Cluster",
            "short_name": "MongoDB",
            "type_name": "mongodb",
            "vendors": [
            {
                "name": "10gen",
                "versions": [ "3.2", "3.4" ]
```

(continues on next page)

(continued from previous page)

```
            },
            {
                "name": "percona",
                "versions": [ "3.2" ]
            } ]
        },
        "myqlcluster":
        {
            "long_name": "MySQL NDB Cluster",
            "nick_name": "ndb",
            "short_name": "NDB Cluster",
            "type_name": "myqlcluster",
            "vendors": [
            {
                "name": "oracle",
                "versions": [ "7.4" ]
            } ]
        },
        "postgresql":
        {
            "intro_url": "https://www.postgresql.org/about/",
            "long_name": "PostgreSQL Cluster",
            "short_name": "PostgreSQL",
            "type_name": "postgresql",
            "vendors": [
            {
                "name": "postgresql",
                "versions": [ "9.4", "9.5", "9.6", "10" ]
            } ]
        },
        "replication":
        {
            "long_name": "MySQL Replication",
            "nick_name": "mysqlreplication",
            "short_name": "Replication",
            "type_name": "replication",
            "vendors": [
            {
                "name": "mariadb",
                "versions": [ "5.5", "10.1", "10.2" ]
            },
            {
                "name": "codership",
                "versions": [ "5.5", "5.6" ]
            },
            {
                "name": "percona",
                "versions": [ "5.5", "5.6", "5.7" ]
            } ]
```

(continues on next page)

(continued from previous page)

```
        }
    },
    "request_created": "2017-08-15T08:37:30.150Z",
    "request_id": 3,
    "request_processed": "2017-08-15T08:37:30.200Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 6
}
```

getSshCredentials

Using the getSshCredentials the caller can get the SSH credentials used when the controller authenticates on the nodes of the cluster. This cluster is made for the UI that creates a new cluster: here the user can check and re-use the SSH credentials used on some existing cluster.

```
{
    "cluster_id": 1,
    "operation": "getSshCredentials",
    "request_created": "2017-08-14T08:25:20.298Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster to read. The cluster name also can be used to identify the cluster.
- `cluster_name`: The name of the cluster to read. The cluster ID can also be used to identify the cluster.

The privilege check provides access if the authenticated user is either the system user or the owner of the existing cluster. No ACLs are implemented and other than the owner can't access the sensitive information of the existing clusters

A successful reply will look like this:

```
{
    "cluster_id": 1,
    "request_created": "2017-08-14T08:25:20.298Z",
    "request_id": 3,
    "request_processed": "2017-08-14T08:25:20.344Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "ssh_credentials":
    {
        "class_name": "CmonSshCredentials",
        "password": "",
        "port": 22,
        "ssh_keyfile": "./configs/testing_id_rsa",
        "timeout": 30,
        "tty_for_sudo": true,
        "user_name": "pipas"
```

(continues on next page)

(continued from previous page)

```
    }
}
```

- `cluster_id`: The numerical ID of the cluster is always sent back even if the cluster was identified by its name in the request.
- `ssh_credentials`: The credentials used to authenticate on the nodes of the cluster.

checkHosts

The checkHosts request performs a set of tests on the given hosts and returns various hardware information about the hosts themselves. The call can specify what kind of tests it needs at the given stage, e.g. if the user already provided an ssh password the access to the host through SSH can be checked.

Here is how these checks can be performed using the s9s command line tool:

```
s9s cluster --check-hosts --nodes="192.168.1.113" --cluster-type=galera --
↪provider-version=5.7 --vendor=codership
```

Here is an example of the request:

```
{
    "check_if_already_registered": true,
    "nodes": [
    {
        "class_name": "CmonHost",
        "hostname": "192.168.1.212",
        "port": 8089
    } ],
    "operation": "checkHosts",
    "request_created": "2017-08-11T12:32:58.547Z",
    "request_id": 3
}
```

- `check_if_already_registered`: If true the controller will check if the hosts are already registered for other clusters.

  If the port is set the code will check if the same host with the same name is already registered in some other cluster. The same port on the same host can't be used twice and the test will check this.

  If the class_name is set the backend will perform an other check. We do not support running two instances of the same daemon on one single host, so if for example a mysqld daemon is running on a host we can't put an other mysqld daemon to the same host even if we use a different port number. So if the class name is set in the request the backend checks if the host is already registered with the same purpose/class. Currently the following class names are supported:

  - CmonHost: A generic host. Use this if there is no information about what role the user plans for the host and no check of host type will be performed.
  - CmonMySqlHost: For hosts that will hold a MySQL daemon.
  - CmonPostgreSqlHost: For hosts that will hold a PostgreSql daemon.
  - CmonProxySqlHost: For hosts that will be holding a ProxySql installation.

    - CmonHaProxyHost: For HAProxy hosts.

- `check_ssh_sudo`: This check will see that the host is actually accessible through ssh as superuser. For this test either the SSH credentials or an existing cluster must be specified.
- `check_job`: When this property is true the caller can send a "job" field with the "create_cluster" job for a preliminary check. The job will not be of course executed, but some checks will be performed to see if the job with the passed properties is valid.
- `cluster_id`: The cluster can be specified through the numerical ID of the cluster. If the cluster is specified the SSH credentials can be accessed from the cluster configuration, so the user don't need to enter those. If the cluster is not specified the user has to enter the information needed for the SSH access to perform ssh test.
- `cluster_name`: Same as cluster_id, but here the cluster can be referenced by the cluster name.
- `ssh_credentials`: Fields providing SSH access to the nodes. The SSH credentials can come from the existing cluster and from this field, but this field has precedence.

Here is an other request that is slightly different. Note that this request will not check if the host is already registered simple because the "check_if_already_registered" field is not set to true. This reques does not indentify an existing cluster either, it contains the "ssh_credentials" field so that the controller can gain access to the nodes. Should the cluster be also referenced the "ssh_credentials" have priority over the credentials found in an existing cluster.

```
{
    "check_ssh_sudo": true,
    "nodes": [
    {
        "class_name": "CmonHost",
        "hostname": "192.168.1.113",
        "port": 8089
    } ],
    "operation": "checkHosts",
    "request_created": "2017-08-14T10:48:50.978Z",
    "request_id": 4,
    "ssh_credentials":
    {
        "class_name": "CmonSshCredentials",
        "password": "",
        "ssh_keyfile": "./configs/testing_id_rsa",
        "user_name": "pipas"
    }
}
```

And here is a call that will also check the passed "create_cluster" job:

```
{
    "check_if_already_registered": true,
    "check_job": true,
    "check_ssh_sudo": true,
    "job":
    {
        "class_name": "CmonJobInstance",
```

(continues on next page)

(continued from previous page)

```
        "job_spec":
        {
            "command": "create_cluster",
            "job_data":
            {
                "cluster_type": "group_replication",
                "mysql_version": "",
                "nodes": [
                {
                    "class_name": "CmonHost",
                    "hostname": "192.168.1.113"
                } ],
                "vendor": ""
            }
        }
    },
    "nodes": [
    {
        "class_name": "CmonHost",
        "hostname": "192.168.1.113"
    } ],
    "operation": "checkHosts",
    "request_created": "2017-10-10T07:55:59.653Z",
    "request_id": 3
}
```

Here is how a simple reply telling that the tests are all passed looks like:

```
{
    "checked_hosts": [
    {
        "host":
        {
            "class_name": "CmonHost",
            "hostname": "192.168.1.112",
            "port": 8089
        },
        "status":
        {
            "error_code": "HostIsOk"
        }
    } ],
    "request_created": "2017-08-11T13:49:10.052Z",
    "request_id": 3,
    "request_processed": "2017-08-11T13:49:10.097Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 1,
    "total_failed": 0
}
```

- `checked_hosts`: The detailed list with the hosts checked. Every host has the status showing what was the result of the check on that particular host.
- `total`: The toal number of hosts checked.
- `total_failed`: The number of hosts that failed the checks. This can be used as a quick checks, but the details of the failure is in the reply too.

The "error_code" field of the "status" can be one of the following strings:

- HostFoundInOtherCluster: The host is already part of some other cluster.
- SshConnectionFailed: The SSH connection failed.
- SshAuthenticationFailed: The SSH connected, but the authentication failed.
- SudoFailed: Failed to gain root access.
- InvalidRequest: The request is invalid, check could not be performed.
- JobWouldFail: The job would fail if it is sent seriously.
- HostIsOk: The host passed all tests.

When the "check_ssh_sudo" test is performed the reply shows some information about the hosts checked. Please note that these tests are under development, later more information might be added.


```
{
    "checked_hosts": [
    {
        "hardware":
        {
            "cpu":
            {
                "class_name": "CmonCpuSet",
                "cpus": [
                {
                    "cores": 4,
                    "cpu_max_ghz": 2.268,
                    "id": 0,
                    "model": "Intel(R) Xeon(R) CPU L5520 @ 2.27GHz",
                    "siblings": 8,
                    "vendor": "GenuineIntel"
                },
                {
                    "cores": 4,
                    "cpu_max_ghz": 2.268,
                    "id": 1,
                    "model": "Intel(R) Xeon(R) CPU L5520 @ 2.27GHz",
                    "siblings": 8,
                    "vendor": "GenuineIntel"
                } ],
                "total_cores": 8,
                "total_cpus": 2,
                "total_siblings": 16
            },
```


(continues on next page)

(continued from previous page)


```
            "memory":
            {
                "class_name": "CmonMemoryInfo",
                "memory_available_mb": 16085,
                "memory_free_mb": 16085,
                "memory_total_mb": 16384,
                "swap_free_mb": 0,
                "swap_total_mb": 0
            },
            "mounted_partitions": [
            {
                "device": "/dev/mapper/core1--vg-root",
                "filesystem": "ext4",
                "free_gb": 142.631,
                "free_mb": 146053,
                "mountpoint": "/",
                "total_gb": 162.286,
                "total_mb": 166180
            } ],
            "network_interfaces":
            {
                "eth0": "192.168.1.113"
            }
        },
        "host":
        {
            "class_name": "CmonHost",
            "hostname": "192.168.1.113",
            "port": 8089
        },
        "software":
        {
            "firewall":
            {
                "firewalld_installed": false,
                "ip_tables_installed": false,
                "iptables_installed": false,
                "ufw_installed": false
            },
            "os_version":
            {
                "class_name": "CmonOsVersion",
                "codename": "xenial",
                "name": "ubuntu",
                "release": "16.04",
                "type": "debian"
            },
            "security":
            {
```


(continues on next page)

(continued from previous page)

```
                "apparmor_installed": false,
                "selinux_installed": false,
                "sudo_installed": true
            }

        },
        "status":
        {
            "error_code": "HostIsOk"
        }
    } ],
    "request_created": "2017-08-16T10:21:19.122Z",
    "request_id": 4,
    "request_processed": "2017-08-16T10:21:20.571Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 1,
    "total_failed": 0
}
```

- `hardware/cpu`: Some information about the CPU(s) found in the host. This can be shown to the user te help recognize the hardware. A total can also be calculated for the cluster so that the user can see whet resources the cluster will have.
- `hardware/memory`: Information about the memory configuration.
- `hardware/mounted_partitions`: Very important piece of information that shows the available storage capacity.
- `hardware/network_interfaces`: This actually could show more, like we could show the type of connection.
- `software/firewall`: Shows what firewall software is installed
- `software/os_version`: The distribution and its version

Well, it is clear, we need to send more info, but the basics are there. Takes a lot of work add new things here.

3.12.12 Hosts and Containers

The `/v2/host` path is for managing servers, hosts and virtual machines or containers.

startServers

This call is for starting up (or booting) servers.

```
{
    "operation": "startServers",
    "request_created": "2017-10-16T07:48:21.779Z",
    "request_id": 3,
    "servers": [
    {
```

(continues on next page)

(continued from previous page)

```
        "class_name": "CmonContainerServer",
        "hostname": "host04"
    } ]
}
```

```
{
    "messages": [ "Started server 'host04'." ],
    "request_created": "2017-10-16T07:48:21.779Z",
    "request_id": 3,
    "request_processed": "2017-10-16T07:48:21.943Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

shutDownServers

Shuts down (powers off) one or more container servers. The servers must be registered with the controller. This is the counterpart to `startServers`.

registerServers

This call can be used to register a container server in the Cmon Controller. This is the fast way, but there is a job called "create_server" that does a very similar registration. The job will actually install software, this call just registers an existing server.

Here is how a request will be sent:

```
{
    "operation": "registerServers",
    "request_created": "2017-08-31T07:53:41.158Z",
    "request_id": 3,
    "servers": [
    {
        "class_name": "CmonContainerServer",
        "cdt_path": "myservers",
        "hostname": "storage01"
    } ]
}
```

Here is a reply that shows the server was registered, but the server is actually turned off:

```
{
    "reply_received": "2017-10-16T11:33:24.501Z",
    "request_created": "2017-10-16T11:33:24.496Z",
    "request_id": 3,
    "request_processed": "2017-10-16T11:33:27.606Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "servers": [
    {
```

(continues on next page)

(continued from previous page)

```
        "cdt_path": "/",
        "class_name": "CmonContainerServer",
        "clusterid": 0,
        "connected": false,
        "hostId": 6,
        "hostname": "storage01",
        "hoststatus": "CmonHostOffLine",
        "ip": "192.168.1.17",
        "maintenance_mode_active": false,
        "message": "SSH connection failed.",
        "owner_group_id": 2,
        "owner_group_name": "users",
        "owner_user_id": 3,
        "owner_user_name": "pipas",
        "protocol": "lxc",
        "timestamp": 1508153607,
        "unique_id": 6
    } ]
}
```

unregisterHost

This call will drop the given host from our database entirely. The computer the host represents will not be touched in any way, software will not be uninstalled, service will not be stopped, nada!

```
{
    "cluster_id": 1,
    "dry_run": true,
    "host":
    {
        "class_name": "CmonHost",
        "hostname": "192.168.1.127",
        "port": 9555
    },
    "operation": "unregisterHost",
    "request_created": "2017-09-08T09:42:49.137Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster that will execute the job. The cluster can also be referenced using the cluster name. For RPC v2 identifying the cluster is not mandatory, but it can be usefull if the same host is part of multiple clusters.
- `cluster_name`: The name of the cluster that will execute the job. The cluster can also be referenced using the cluster ID. For RPC v2 identifying the cluster is not mandatory, but it can be usefull if the same host is part of multiple clusters.
- `dry_run`: If this field is provided (with any value at all) the host will not be really unregistered. All the checks will be done, a success will be reported back, only the error message will show that this was just a drill.
- `host`: The host that shall be unregistered.

- `class_name`: The class name of the host will not be strictly checked (because the client might not know this), just send "CmonHost" that'll do.
- `hostname`: This is mandatory to identify the host.
- `port`: The port of the host. RPC v2 allows the usage of hosts without a port, but for most cases the port will be required.

So there are multiple fields to identify the host, there is teh hostname, the port, the cluster ID, maybe the cluster name. The backend will use whatever it is provided and find the host. If multiple hosts are found with the given data the request will be rejected.

Here is what we get for a dry run:

```
{
    "error_string": "Dry run was requested, not unregistering host.",
    "request_created": "2017-09-08T09:42:49.137Z",
    "request_id": 3,
    "request_processed": "2017-09-08T09:42:49.183Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

unregisterServers

This call is for dropping a container server from the scope of the controller. The server will not seize to exist, but the controller will forget everything about it.

```
{
    "operation": "unregisterServers",
    "request_created": "2017-09-01T08:42:22.756Z",
    "request_id": 3,
    "servers": [
    {
        "class_name": "CmonContainerServer",
        "hostname": "core1"
    } ]
}
```

- `servers`: A list of CmonContainerServer class objects to unregister. In the objects only the class name and the host name has to be provided.

```
{
    "messages": [ "Unregistered server 'core1'." ],
    "request_created": "2017-09-01T08:42:22.756Z",
    "request_id": 3,
    "request_processed": "2017-09-01T08:42:22.810Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

getContainers

Returns the list of containers known by the controller. The RPC v2 call also checks the rights of the authenticated user, if the user has no right to read the container object it will not appear in the list.

```
{
    "operation": "getContainers",
    "request_created": "2017-09-07T05:09:16.895Z",
    "request_id": 3
}
```

```
{
    "containers": [
    {
        "alias": "debian",
        "class_name": "CmonContainer",
        "hostname": "debian",
        "owner_group_id": 4,
        "owner_group_name": "testgroup",
        "owner_user_id": 3,
        "owner_user_name": "pipas",
        "parent_server": "storage01",
        "status": "STOPPED",
        "type": "lxc"
    },
    . . .
    {
        "alias": "vnc_server",
        "class_name": "CmonContainer",
        "hostname": "192.168.1.210",
        "ip": "192.168.1.210",
        "ipv4_addresses": [ "192.168.1.210" ],
        "owner_group_id": 4,
        "owner_group_name": "testgroup",
        "owner_user_id": 3,
        "owner_user_name": "pipas",
        "parent_server": "storage01",
        "status": "RUNNING",
        "template": "ubuntu",
        "type": "lxc"
    } ],
    "request_created": "2017-09-07T05:09:16.895Z",
    "request_id": 3,
    "request_processed": "2017-09-07T05:09:16.947Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 5
}
```

- `containers`: A list of #CmonContainer objects. The following command can be used to print the properties of this class:

```
s9s metatype --list-properties --type=CmonContainer --long
```

- `total`: The total number of containers known by the controller.

getServers

Returns the container servers and their properties. Here is a request:

```
{
    "operation": "getServers",
    "request_created": "2017-10-03T08:19:58.769Z",
    "request_id": 3
}
```

And here is the reply. It is rather complex, but we have a lot of data.

```
{
    "request_created": "2017-10-03T08:19:58.769Z",
    "request_id": 3,
    "request_processed": "2017-10-03T08:19:58.824Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "servers": [
    {
        "cdt_path": "/",
        "class_name": "CmonContainerServer",
        "clusterid": 0,
        "connected": false,
        "containers": [
        {
            "acl": "user::rwx,user:nobody:r--,group::rw-,other::---",
            "alias": "bestw_controller",
            "cdt_path": "/core1/containers",
            "class_name": "CmonContainer",
            "container_id": 1,
            "hostname": "192.168.1.182",
            "ip": "192.168.1.182",
            "ipv4_addresses": [ "192.168.1.182" ],
            "owner_group_id": 2,
            "owner_group_name": "users",
            "owner_user_id": 3,
            "owner_user_name": "pipas",
            "parent_server": "core1",
            "status": "RUNNING",
            "type": "lxc",
            "version": 25
        },
        . . .
        {
            "acl": "user::rwx,group::rw-,other::---",
            "alias": "ubuntu",
```

(continues on next page)

(continued from previous page)


```
            "cdt_path": "/core1/containers",
            "class_name": "CmonContainer",
            "container_id": 5,
            "hostname": "ubuntu",
            "owner_group_id": 2,
            "owner_group_name": "users",
            "owner_user_id": 3,
            "owner_user_name": "pipas",
            "parent_server": "core1",
            "status": "STOPPED",
            "type": "lxc",
            "version": 25
        } ],
        "disk_devices": [
        {
            "class_name": "CmonDiskDevice",
            "device": "/dev/mapper/core1--vg-root",
            "filesystem": "ext4",
            "free_mb": 141617,
            "mountpoint": "/",
            "total_mb": 166180
        },
        . . .
        {
            "device": "/dev/sdc",
            "is_hardware_storage": false,
            "model": "LSILOGIC Logical Volume",
            "total_mb": 170230,
            "volumes": [
            {
                "description": "Linux filesystem partition",
                "device": "/dev/sdc1",
                "filesystem": "ext2",
                "free_mb": 295,
                "mount_point": "/boot",
                "total_mb": 487
            },
            {
                "description": "Extended partition",
                "device": "/dev/sdc2",
                "filesystem": "",
                "mount_point": "",
                "total_mb": 169741,
                "volumes": [
                {
                    "description": "Linux LVM Physical Volume partition",
                    "device": "/dev/sdc5",
                    "filesystem": "",
                    "mount_point": "",
```


(continues on next page)

(continued from previous page)


```
                "total_mb": 169741
            } ]
        } ]
    },
    . . .
    {
        "device": "/dev/sda",
        "is_hardware_storage": false,
        "model": "SCSI Disk",
        "total_mb": 15272.1,
        "volumes": [
        {
            "description": "EXT4 volume",
            "device": "/dev/sda1",
            "filesystem": "ext4",
            "mount_point": "",
            "total_mb": 15271.1
        } ]
    } ],
    "distribution":
    {
        "codename": "xenial",
        "name": "ubuntu",
        "release": "16.04",
        "type": "debian"
    },
    "hostId": 1,
    "hostname": "core1",
    "hoststatus": "CmonHostOffLine",
    "ip": "192.168.1.4",
    "last_container_collect": 1506791691,
    "last_hw_collect": 1507017024,
    "lastseen": 1506791691,
    "maintenance_mode_active": false,
    "memory":
    {
        "banks": [
        {
            "bank": "0",
            "name": "DIMM 800 MHz (1.2 ns)",
            "size": 4294967296
        },
        . . .
        {
            "bank": "7",
            "name": "DIMM 800 MHz (1.2 ns)",
            "size": 4294967296
        } ],
        "class_name": "CmonMemoryInfo",
```


(continues on next page)

(continued from previous page)


```
            "memory_available_mb": 54091,
            "memory_free_mb": 41919,
            "memory_total_mb": 64421,
            "swap_free_mb": 0,
            "swap_total_mb": 0
        },
        "message": "SSH connection failed.",
        "model": "SUN FIRE X4170 SERVER (4583256-1)",
        "network_interfaces": [
        {
            "gbits": 1,
            "link": true,
            "mac": "00:21:28:76:06:2a",
            "model": "82575EB Gigabit Network Connection",
            "name": "enp1s0f0"
        },
        . . .
        {
            "gbits": 0,
            "ip": "192.168.122.1",
            "mac": "",
            "model": "",
            "name": "virbr0"
        } ],
        "owner_group_id": 2,
        "owner_group_name": "users",
        "owner_user_id": 3,
        "owner_user_name": "pipas",
        "processors": [
        {
            "cores": 4,
            "cpu_max_ghz": 1.6,
            "id": 5,
            "model": "Intel(R) Xeon(R) CPU L5520 @ 2.27GHz",
            "siblings": 8,
            "vendor": "Intel Corp."
        },
        {
            "cores": 4,
            "cpu_max_ghz": 1.6,
            "id": 9,
            "model": "Intel(R) Xeon(R) CPU L5520 @ 2.27GHz",
            "siblings": 8,
            "vendor": "Intel Corp."
        } ],
        "timestamp": 1507017024,
        "unique_id": 1,
        "version": "2.17"
    },
```


(continues on next page)

(continued from previous page)

```
    . . .
    ],
    "total": 8
}
```

3.12.13 Job Information and Manipulation

Creating and manipulating jobs, reading information about jobs is available on the `/v2/jobs` path.

createJobInstance

Arguably the most important call of the whole RPC 2.0 is the "createJobInstance" call where new jobs can be started. Jobs are very powerfull and complex and so this RPC call is very important and very often used.

The request contains a job description that will be created as a new job and then the new job will be described in detail returned in the reply. This way the caller immediately know what properties the controller added to the new job.

```
{
    "cluster_id": 1,
    "job":
    {
        "class_name": "CmonJobInstance",
        "job_spec":
        {
            "command": "rolling_restart"
        },
        "title": "Rolling Restart",
    },
    "operation": "createJobInstance",
    "request_created": "2017-06-22T12:46:05.054Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster that will execute the job. The cluster can also be referenced using the cluster name.
- `cluster_name`: The name of the cluster that will execute the job. The cluster can also be referenced using the cluster ID.
- `job`: The job that will be created. Here the caller can set various properties while other properties will be set by the controller automatically. The most important properties the caller can send are the following:
  - `class_name`: This should always be "CmonJobInstance".
  - `job_spec`: Speicifies what the job will do.
  - `recurrence`: A crontab string that describes when the job should be repeated. When this is provided a recurring (repeated) job will be created.
  - `scheduled`: If this date and time is specified the job will be executed at a fixed time later. Recurring jobs are scheduled by the recurrence, so a job can not contain both the recurrence and the scheduled properties simultaneously.

- `title`: The human readable title of the job. If this property is not specified the controller will choose a title that is appropriate.

To get a comprehensive list of the properties the controller supports one can run the following command (or check the jobclasses page):

```
s9s metatype --list-properties --type=CmonJobInstance --long
```

Here is an example showing a reply to the "createJobInstance" request:

```
{
    "job":
    {
        "can_be_aborted": false,
        "can_be_deleted": true,
        "class_name": "CmonJobInstance",
        "cluster_id": 1,
        "created": "2017-06-22T12:46:05.000Z",
        "exit_code": 0,
        "group_id": 2,
        "group_name": "users",
        "job_id": 2,
        "job_spec":
        {
            "command": "rolling_restart"
        },
        "status": "DEFINED",
        "status_text": "Waiting",
        "title": "Rolling Restart",
        "user_id": 3,
        "user_name": "pipas"
    },
    "request_created": "2017-06-22T12:46:05.054Z",
    "request_id": 3,
    "request_processed": "2017-06-22T12:46:05.105Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

Here is an example that registers a recurring job:

```
{
    "cluster_id": 1,
    "job":
    {
        "class_name": "CmonJobInstance",
        "job_spec":
        {
            "command": "success",
            "job_data":
            {
            }
```

(continues on next page)

(continued from previous page)

```
        },
        "recurrence": "2 * * * *",
        "tags": [ "testRecurringJob" ],
        "title": "Simulated Success"
    },
    "operation": "createJobInstance",
    "request_created": "2019-11-06T06:16:23.859Z",
    "request_id": 3
}
```

updateJobInstance

An RPC 2.0 method call to update an existing (non-closed) job instance.

The request contains the updated job description and status then the updated job will be described in detail returned in the reply. This way the caller immediately know what properties the controller has updated added to the existing job.

```
{
    "cluster_id": 1,
    "job":
    {
        "class_name": "CmonJobInstance",
        "status": "PAUSED",
        "title": "Rolling Restart",
        "job_spec":
        {
            "command": "rolling_restart"
        },
    },
    "operation": "updateJobInstance",
    "request_created": "2020-05-05T12:46:05.054Z",
    "request_id": 3
}
```

- `cluster_id`: The numerical ID of the cluster that will execute the job. The cluster can also be referenced using the cluster name.
- `cluster_name`: The name of the cluster that will execute the job. The cluster can also be referenced using the cluster ID.
- `job`: The job that will be created. Here the caller can set various properties while other properties will be set by the controller automatically. The most important properties the caller can send are the following:
  - `class_name`: This should always be "CmonJobInstance".
  - `job_spec`: Speicifies what the job will do.
  - `status`: a new job status, for example one can pause or resume a job by switching between PAUSED and SCHEDULED states.
  - `recurrence`: A crontab string that describes when the job should be repeated. When this is provided a recurring (repeated) job will be created.

- `scheduled`: If this date and time is specified the job will be executed at a fixed time later. Recurring jobs are scheduled by the recurrence, so a job can not contain both the recurrence and the scheduled properties simultaneously.
- `title`: The human readable title of the job. If this property is not specified the controller will choose a title that is appropriate.

To get a comprehensive list of the properties the controller supports one can run the following command (or check the jobclasses page):

```
s9s metatype --list-properties --type=CmonJobInstance --long
```

Here is an example showing a reply to the "updateJobInstance" request:


```
{
    "job":
    {
        "can_be_aborted": false,
        "can_be_deleted": true,
        "class_name": "CmonJobInstance",
        "cluster_id": 1,
        "created": "2020-05-05T12:46:05.000Z",
        "exit_code": 0,
        "group_id": 2,
        "group_name": "users",
        "job_id": 2,
        "job_spec":
        {
            "command": "rolling_restart"
        },
        "status": "DEFINED",
        "status_text": "Waiting",
        "title": "Rolling Restart",
        "user_id": 3,
        "user_name": "pipas"
    },
    "request_created": "2020-05-05T12:46:05.054Z",
    "request_id": 3,
    "request_processed": "2020-05-05T12:46:05.105Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


deleteJobInstance

Deletes an existing job instance record from the controller. Only closed jobs — those with a status of `FINISHED`, `FAILED`, or `ABORTED` — can be deleted. Use `killJobInstance` to abort a running job before deleting it.


```
{
    "job_id": 4,
    "operation": "deleteJobInstance",
```


(continues on next page)

(continued from previous page)

```
    "request_created": "2017-10-20T12:55:08.277Z",
    "request_id": 3
}
```

```
{
    "request_created": "2017-10-20T12:55:08.277Z",
    "request_id": 3,
    "request_processed": "2017-10-20T12:55:08.335Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

killJobInstance

This call was made for killing/aborting jobs running on the controller. To be more precise killing a Cmon job works like killing a UNIX process (it is not a real UNIX process though): we deliver a signal, the job code recognizes the job signal and aborts itself. In short we could say this is not preemptive, jobs that don't want to be aborted will not be aborted.

Here is a request to abort a job:

```
{
    "job_id": 1,
    "operation": "killJobInstance",
    "request_created": "2019-05-22T06:41:18.944Z",
    "request_id": 3,
    "signal": 15
}
```

- `job_id`: The numerical ID of the job that should receive the signal.
- `signal`: The signal itself as a single integer number. In the current implementation there is very little difference between the various signals, it is a good practice to send 15 now.

Here is the reply, it is just a notification about the signal being delivered. The job will be aborted later (if the job supports aborting at all).

```
{
    "request_created": "2019-05-22T06:41:18.944Z",
    "request_id": 3,
    "request_processed": "2019-05-22T06:41:18.953Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```

cloneJobInstance

This call will clone (copy) an existing job instance in order to execute it again. All the properties are going to be inherited from the existing job and the newly created copy will be executed the same way new jobs are always executed. Here is how the request looks like:

```
{
    "cluster_id": 1,
    "job_id": 2,
    "operation": "cloneJobInstance",
    "request_created": "2018-09-19T12:01:57.142Z",
    "request_id": 3
}
```

The following fields are important:

- `job_id`: The ID of the job that will be cloned. The original job.
- cluster-id: If this field is provided the job will be executed by this cluster. If this field is not provided the job will be executed by the same cluster that executed the original job.

getJobInstance

This call is very similar to the "getJobInstances" call, but this one returns information about one job.

```
{
    "job_id": 1,
    "operation": "getJobInstance",
    "request_created": "2017-06-23T08:29:47.266Z",
    "request_id": 4
}
```

- `job_id`: The numerical ID of the job that should be returned.

```
{
    "job":
    {
        "can_be_aborted": false,
        "can_be_deleted": false,
        "class_name": "CmonJobInstance",
        "cluster_id": 0,
        "created": "2017-06-23T08:29:19.000Z",
        "exit_code": 0,
        "group_id": 2,
        "group_name": "users",
        "has_progress": true,
        "job_id": 1,
        "job_spec":
        {
            "command": "create_cluster",
            "job_data":
            {
                "cluster_name": "ft_postgresql_63712",
                "cluster_type": "postgresql_single",
                "enable_uninstall": true,
                "hostname": "192.168.1.192",
                "port": 8089,
                "postgre_password": "passwd12",
```

(continues on next page)

(continued from previous page)

```
                "postgre_user": "postmaster",
                "ssh_user": "pipas",
                "type": "postgresql"
            }
        },
        "progress_percent": 20,
        "started": "2017-06-23T08:29:19.000Z",
        "status": "RUNNING",
        "status_text": "Installing software",
        "title": "Setup PostgreSQL Server",
        "user_id": 3,
        "user_name": "pipas"
    },
    "request_created": "2017-06-23T08:31:23.490Z",
    "request_id": 3,
    "request_processed": "2017-06-23T08:31:23.537Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `job`: The job instance itself.

getJobInstances

This call can be used to get the job instances from the controller. The request is rather simple:

```
{
  "operation": "getJobInstances",
  "request_created": "2017-06-23T05:25:17.891Z",
  "request_id": 3
}
```

This can hold the following fields:

- `ascending`: Normally the records will be sent in descending order of time, the newest first. If this boolean is true the order will be descending.
- `cluster_id`: The ID of the cluster from which the jobs will be received. The cluster name can also be used to identify the cluster, but neither of them are mandatory. It is possible to receive a mixed job list with multiple clusters.
- `cluster_ids`: A list of integers, the IDs of the clusters from which we will get the jobs. No cluster ID or cluster name is mandatory, this call works without any identification of the clusters.
- `cluster_name`: The name of the cluster from which the job list will be received. The cluster ID can also be used to identify the cluster, but neither of them are mandatory.
- `show_defined`: Turn on state filtering and add jobs in defined state.
- `show_running`: Turn on state filtering and add jobs that are running.
- `show_scheduled`: Turn on state filtering and add jobs that are scheduled.
- `show_aborted`: Turn on state filtering and add jobs that are aborted.

- `show_finished`: Turn on state filtering and add jobs that are finished.
- `show_failed`: Turn on state filtering and add jobs that are failed.
- `tags`: A list of short strings (tags) to filter the job list. If the list is empty no tags will be considered.
- `limit`: Limits the number of records the same way the SQL LIMIT statement does.
- `offset`: The offset of the first record returned. This works the same way the SQL OFFSET statement works.

If no cluster_id, cluster_ids or cluster_name is sent the controller will send the list of jobs of all the clusters (considering the limit and offset filters of course). The access rights of the authenticated user should also filter the records sent back, but that is unfortunately not yet implemented.

Here is an example showing a reply sent to the getJobInstances request:


```
{
    "jobs": [
    {
        "can_be_aborted": false,
        "can_be_deleted": true,
        "class_name": "CmonJobInstance",
        "cluster_id": 1,
        "created": "2017-06-23T01:00:00.000Z",
        "ended": "2017-06-23T01:00:00.000Z",
        "exit_code": 0,
        "group_id": 1,
        "group_name": "admins",
        "ip_address": "127.0.0.1",
        "job_id": 2,
        "job_spec": "Removing Old Backups",
        "started": "2017-06-23T01:00:00.000Z",
        "status": "FINISHED",
        "status_text": "Command ok",
        "title": "Removing Old Backups",
        "user_id": 0,
        "user_name": "system"
    },
    {
        "can_be_aborted": false,
        "can_be_deleted": true,
        "class_name": "CmonJobInstance",
        "cluster_id": 0,
        "created": "2017-06-22T12:56:33.000Z",
        "ended": "2017-06-22T12:58:18.000Z",
        "exit_code": 0,
        "group_id": 2,
        "group_name": "users",
        "has_progress": true,
        "job_id": 1,
        "job_spec":
        {
            "command": "create_cluster",
```


(continues on next page)

(continued from previous page)


```
            "job_data":
            {
                "cluster_name": "ft_postgresql_4357",
                "cluster_type": "postgresql_single",
                "enable_uninstall": true,
                "hostname": "192.168.1.191",
                "port": 8089,
                "postgre_password": "passwd12",
                "postgre_user": "postmaster",
                "ssh_user": "pipas",
                "type": "postgresql"
            }
        },
        "progress_percent": 100,
        "started": "2017-06-22T12:56:33.000Z",
        "status": "FINISHED",
        "status_text": "Job finished.",
        "title": "Setup PostgreSQL Server",
        "user_id": 3,
        "user_name": "pipas"
    } ],
    "request_created": "2017-06-23T05:25:17.891Z",
    "request_id": 3,
    "request_processed": "2017-06-23T05:25:17.939Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 2
}
```


The most important fields of the reply are as follows.

- `jobs`: This is of course the list of job instances found. The elements in the list are CmonJobInstance class objects.

  To get a comprehensive list of the properties the controller supports in the CmonJobInstance class please run the following command (or check the jobclasses page):

  ```
  s9s metatype --list-properties --type=CmonJobInstance --long
  ```

- `total`: This is the total number of records found in the database. The caller can implement a pager using this "total", the "limit" and the "offset" properties.

getJobLog

This RPC call is for reading the job status and job messages from the controller. The reply will have the job and the job message in it, so the caller can show the job together with all the messages.

Here is an example of the RPC call:


```
{
    "ascending": true,
    "job_id": 1,
```


(continues on next page)

(continued from previous page)

```
    "limit": 1,
    "log_level": "DEBUG",
    "offset": 100,
    "operation": "getJobLog",
    "request_created": "2019-07-17T05:23:50.262Z",
    "request_id": 3
}
```

- `ascending`: Normally the job messages returned so that the first is the newest. When this property is set to true in the request the first message will be the oldest, the last will be the newest.
- `job_id`: The numerical ID of the job to return.
- `limit`: Optional argument, setting the limit for the log messages sent back.
- `log_level`: Optional argument controlling the severity of the messages that will be sent back.

  Valid values are "DEBUG", "SUCCESS", "WARNING" or "FAILED" (or as an alternative "JOB_DEBUG", "JOB_SUCCESS", "JOB_WARNING" or "JOB_FAILED").

  The "DEBUG" will send back every message, "SUCCESS" will not send debug messages, "WARNING" will send warning and failure messages while "FAILURE" will send only failure messages.
- `offset`: Optional argument, setting the offset of the first log message to be sent back.

Here is an example of the reply:

```
{
    "job":
    {
        "can_be_aborted": false,
        "can_be_deleted": true,
        "class_name": "CmonJobInstance",
        "cluster_id": 0,
        "created": "2017-06-23T08:29:19.000Z",
        "ended": "2017-06-23T08:32:31.000Z",
        "exit_code": 0,
        "group_id": 2,
        "group_name": "users",
        "has_progress": true,
        "job_id": 1,
        "job_spec":
        {
            "command": "create_cluster",
            "job_data":
            {
                "cluster_name": "ft_postgresql_63712",
                "cluster_type": "postgresql_single",
                "enable_uninstall": true,
                "hostname": "192.168.1.192",
                "port": 8089,
                "postgre_password": "passwd12",
                "postgre_user": "postmaster",
```

(continues on next page)

(continued from previous page)


```
                "ssh_user": "pipas",
                "type": "postgresql"
            }
        },
        "progress_percent": 100,
        "started": "2017-06-23T08:29:19.000Z",
        "status": "FINISHED",
        "status_text": "Job finished.",
        "title": "Setup PostgreSQL Server",
        "user_id": 3,
        "user_name": "pipas"
    },
    "messages": [
    {
        "class_name": "CmonJobMessage",
        "created": "2017-06-23T08:29:19.000Z",
        "file_name": "CmonCommandHandlerWorker.cpp",
        "job_id": 1,
        "line_number": 2820,
        "message_id": 1,
        "message_status": "JOB_SUCCESS",
        "message_text": "Started create cluster job."
    },

    . . .

    {
        "class_name": "CmonJobMessage",
        "created": "2017-06-23T08:32:31.000Z",
        "file_name": "CmonCommandHandlerWorker.cpp",
        "job_id": 1,
        "line_number": 945,
        "message_id": 79,
        "message_status": "JOB_WARNING",
        "message_text": "Try Settings -> Cluster Registrations and press
↪'Synchronize Clusters'."
    } ],
    "reply_received": "2017-06-23T08:38:01.052Z",
    "request_created": "2017-06-23T08:38:01.046Z",
    "request_id": 3,
    "request_processed": "2017-06-23T08:38:01.102Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `job`: The info about the job instance as it is described in the jobclasses page.
- `messages`: The job messages of the given job. This is a list of cmonjobmessage objects.

3.12.14 Log Related Calls

The `/v2/log` provides calls to access the log handled by the Cmon controller. This is not for the job messages, this is about the logs that are stored in ASCII files.

getLogEntries

This call can be used to get the log messages from the controller.

```
{
    "ascending": true,
    "cluster_name": "ft_postgresql_31001",
    "limit": 3,
    "operation": "getLogEntries",
    "request_created": "2017-07-26T07:22:25.767Z",
    "request_id": 200
}
```

- `ascending`: Boolean value controlling the order of the log messages.
- `cluster_name`: Identifies the cluster from which the log entries will be returned. The cluster ID can also be to identify the cluster.
- `cluster_id`: Identifies the cluster from which the log entries will be returned. The cluster name can also be used.
- `component`: Identifies the component that sent the log message. Currently this string can be 'Network', 'CmonDatabase', 'Mail', 'Cluster', 'ClusterConfiguration', 'ClusterRecovery', 'Node', 'Host', 'DbHealth', 'DbPerformance', 'SoftwareInstallation', 'Backup' or 'Unknown'.
- `created_before`: If this date&time string is provided it will filter the messages by their creation time.
- `created_after`: If this date&time string is provided it fill filter the messages by their creation time.
- `limit`: If this provided it limits the number of log entries sent back.
- `offset`: The index of the first item. If this value is provided together with the limit they can be used to implement a pager.
- `severity`: This string can be one of the "LOG_EMERG", "LOG_ALERT", "LOG_CRIT", "LOG_ERR", "LOG_WARNING", "LOG_NOTICE", "LOG_INFO", "LOG_DEBUG" to filter the messages by their severity.

```
{
    "log_entries": [
    {
        "class_name": "CmonLogMessage",
        "component": "ClusterConfiguration",
        "created": "2017-07-26T08:35:40.704Z",
        "log_class": "LogMessage",
        "log_id": 110,
        "log_origins":
        {
            "sender_binary": "cmon",
```

(continues on next page)

(continued from previous page)


```
            "sender_file": "../../src/cmonhostmanager.cpp",
            "sender_line": 594,
            "sender_pid": 40818,
            "tv_nsec": 704637455,
            "tv_sec": 1501058140
        },
        "log_specifics":
        {
            "cluster_id": 1,
            "message_text": "Registering CmonPostgreSqlHost:␣
↪192.168.1.167:8089"
        },
        "severity": "LOG_DEBUG"
    },

    . . .

    {
        "class_name": "CmonLogMessage",
        "created": "2017-07-26T08:38:11.697Z",
        "log_class": "LogMessage",
        "log_id": 351,
        "log_origins":
        {
            "sender_binary": "cmon",
            "sender_file": "../../src/cmonalarmdb.cpp",
            "sender_line": 1467,
            "sender_pid": 40818,
            "tv_nsec": 697060177,
            "tv_sec": 1501058291
        },
        "log_specifics":
        {
            "cluster_id": 1,
            "message_text": "hostId: 1, nodeId: 0, alarm: Host is not␣
↪responding"
        },
        "severity": "LOG_DEBUG"
    } ],
    "log_entry_counts":
    {
        "component":
        {
            "Cluster": 1,
            "ClusterConfiguration": 2,
            "Node": 3,
            "Unknown": 194
        },
        "hostname":
```


(continues on next page)

(continued from previous page)


```
        {
            "": 199,
            "192.168.1.167": 1
        },
        "sender_binary":
        {
            "cmon": 200
        },
        "severity":
        {
            "LOG_CRIT": 2,
            "LOG_DEBUG": 192,
            "LOG_ERR": 1,
            "LOG_INFO": 2,
            "LOG_WARNING": 3
        }
    },
    "reply_received": "2017-07-26T08:39:53.936Z",
    "request_created": "2017-07-26T08:39:53.931Z",
    "request_id": 3,
    "request_processed": "2017-07-26T08:39:54.019Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 258
}
```


- `log_entries`: A list that holds the log entries returned as result. The log entries returned as CmonLogMessage objects.
- `log_entry_counts`: A simple statistics showing how many messages are in the reply with various properties.
- `total`: The total number of records with the given filters. This with the limit and offset request parameters can be used to implement a pager.

getLogEntry

This call is made for retrieving one message by its message ID. Here is how a request looks like:


```
{
    "cluster_id": -1,
    "message_id": 32,
    "operation": "getLogEntry",
    "request_created": "2019-08-13T05:22:18.882Z",
    "request_id": 3
}
```


And here is an example for the reply:


```
{
    "debug_messages": [ "RPC V2 authenticated user is 'pipas'." ],
```


(continues on next page)

(continued from previous page)


```
    "log_entry":
    {
        "class_name": "CmonLogMessage",
        "created": "2019-08-13T04:00:38.375Z",
        "log_class": "JobStarted",
        "log_id": 32,
        "log_origins":
        {
            "sender_binary": "cmon",
            "sender_file": "CmonCommandHandlerWorker.cpp",
            "sender_line": 332,
            "sender_pid": 49080,
            "tv_nsec": 375158422,
            "tv_sec": 1565668838
        },
        "log_specifics":
        {
            "cluster_id": 0,
            "message_text": "Simulated Failure",
            "reason": "Simulated Failure"
        },
        "severity": "LOG_INFO"
    },
    "request_created": "2019-08-13T05:22:18.882Z",
    "request_id": 3,
    "request_processed": "2019-08-13T05:22:18.891Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```


getLogStatistics

This call can be used to get some statistical information about the subsystem that handles log messages. This is mostly for debugging purposes, but it also can be used to get some information that is shown to the end-user.

Here is an example for the request:


```
{
    "operation": "getLogStatistics",
    "request_created": "2017-07-26T10:58:35.447Z",
    "request_id": 3
}
```


Here is an example for the reply:


```
{
    "log_statistics":
    {
        "cluster_log_statistics": [
```


(continues on next page)

(continued from previous page)


```
        {
            "cluster_id": -1,
            "disabled": false,
            "entries_received": 3,
            "format_string": "%C : (%S) %M",
            "last_error_message": "",
            "lines_written": 0,
            "log_file_name": "",
            "max_log_file_size": 5242880,
            "messages_per_sec": 0.05,
            "syslog_enabled": false,
            "write_cycle_counter": 1
        },
        {
            "cluster_id": 0,
            "disabled": false,
            "entries_received": 93,
            "format_string": "%C : (%S) %M",
            "last_error_message": "Success.",
            "lines_written": 93,
            "log_file_name": "./cmon-ft-install.log",
            "max_log_file_size": 5242880,
            "messages_per_sec": 0.05,
            "syslog_enabled": false,
            "write_cycle_counter": 3
        },
        {
            "cluster_id": 1,
            "disabled": false,
            "entries_received": 218,
            "format_string": "%C : (%S) %M",
            "last_error_message": "Success.",
            "lines_written": 218,
            "log_file_name": "/tmp/cmon_1.log",
            "max_log_file_size": 5242880,
            "messages_per_sec": 0.85,
            "syslog_enabled": false,
            "write_cycle_counter": 11
        } ],
        "current_time": "2017-07-26T11:01:08.884Z",
        "entries_statistics":
        {
            "entries_received": 319,
            "entries_written_to_cmondb": 314
        },
        "has_cmondb": true,
        "last_error_message": "Success.",
        "last_flush_time": "2017-07-26T11:00:59.543Z",
        "log_debug_enabled": false,
```


(continues on next page)

(continued from previous page)

```
        "writer_thread_running": true,
        "writer_thread_started": "2017-07-26T10:55:29.410Z"
    },
    "request_created": "2017-07-26T11:01:08.840Z",
    "request_id": 3,
    "request_processed": "2017-07-26T11:01:08.884Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

listCollectedLogs

This call retrieves logs from all hosts then store in the local file system. The default folder is /var/lib/cmon/host_logs. Each subfolder represents the cluster which contains some nodes. Nodes/hosts are named using the format: hostname_port.

Here is an example for the request:

```
{
    "cluster_id": 10,
    "operation": "listCollectedLogs",
}
```

Here is an example for the reply:

```
{
    "files": [
    {
        "created": "1662022801",
        "filename": "postgresql-2022-09-01_064920.log",
        "hostname": "10.0.8.11",
        "length": "2030",
        "path": "/
↪var/lib/cmon/host_logs/10/10.0.8.11_5432/postgresql-2022-09-01_064920.log"
    },
    {
        "created": "1663655401",
        "filename": "prometheus.log",
        "hostname": "178.128.85.233",
        "length": "313625",
        "path": "/
↪var/lib/cmon/host_logs/10/178.128.85.233_9090/prometheus.log"
    },
    {
        "created": "1663655400",
        "filename": "agent_internal_logs.log",
        "hostname": "10.0.8.11",
        "length": "3568",
        "path": "/
↪var/lib/cmon/host_logs/10/10.0.8.11_4433/agent_internal_logs.log"
```

(continues on next page)

(continued from previous page)


```
    },
    {
        "created": "1663632001",
        "filename": "cmnd.log",
        "hostname": "10.0.8.11",
        "length": "105914",
        "path": "/var/lib/cmon/host_logs/10/10.0.8.11_4433/cmnd.log"
    }
]
}
```


getCollectedLog

This call retrieves the details of a selected log from the UI. Logs ared in the local file system and 'read' when user clicks on the particular log.

Here is an example for the request:


```
{
    "cluster_id": 10,
    "filename": "postgresql-2022-08-31_035553.log",
    "hostname": "10.0.8.11",
    "operation": "getCollectedLog",
}
```


Here is an example for the reply:


```
{
    "data": {
        "content": "2022-06-02 05:16:44.319 UTC [499] LOG:  starting␣
↪PostgreSQL 14.2 (Ubuntu 14.2-1.pgdg20.04+1+b1) on x86_64-pc-linux-gnu,␣
↪compiled by gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0,␣
↪64-bit\r\n2022-06-02 05:16:44.319 UTC [499] LOG:  listening on IPv4␣
↪address \"0.0.0.0\", port 5432\r\n2022-06-02 05:16:44.319 UTC [499] LOG: ␣
↪listening on IPv6 address \"::\", port 5432\r\n2022-06-02 05:16:44.339␣
↪UTC [499] LOG:  listening on Unix socket \"/
↪var/run/postgresql/.s.PGSQL.5432\"\r\n2022-06-02 05:16:44.451 UTC [508]␣
↪LOG:  database system was shut down at 2022-06-02 05:14:41␣
↪UTC\r\n2022-06-02 05:16:44.653 UTC [499] LOG:  database system is ready␣
↪to accept connections\r\n",
        "created": "1661841000",
        "filename": "postgresql-2022-06-02_051644.log",
        "hostname": "10.0.8.11",
        "length": "642"
    }
}
```

3.12.15 PostgreSQL Logical Replication

The `/v2/logical_replication` is for managing logical replication clusters.

createPublication

This call is for creating a publication for logical replication.

Here is an example for the request:

```
{
     "operation": "createPublication",
     "cluster_id": 1,
     "subcluster_id": 3,
     "subcluster_name": "p1",
     "pub_name": "pub1",
     "pub_db_name": "postgres",
     "include_all_tables": true,
     "db_tables": ["t1", "t2"]
 }
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID to create the publication on.
- `subcluster_name`: The sub-cluster name to create the publication on.
- `pub_name`: The name of the new publication.
- `{pub_db_name|db_name}`: The database to create the publication on.
- `include_all_tables`: Whether to replicate changes for all tables in the database (including future tables).
- `db_tables`: List of specific tables to include in the publication.

  Remarks: Either `subcluster_id` or `subcluster_name` must be provided. Either `include_all_tables` must be `true` or `db_tables` must contain at least one table. The user must have the necessary privileges to manage replication.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Created publication 'pub1' in database 'postgres' on␣
↪sub-cluster '3'"
}
```

dropPublication

Removes an existing publication from logical replication.

Here is an example for the request:

```
{
    "operation": "dropPublication",
    "cluster_id": 1,
```

(continues on next page)

(continued from previous page)

```
    "subcluster_id": 3,
    "subcluster_name": "p1",
    "pub_name": "pub1",
    "pub_db_name": "postgres"
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID containing the publication.
- `subcluster_name`: The sub-cluster name containing the publication.
- `pub_name`: The name of the publication to drop.
- `{pub_db_name|db_name}`: The database containing the publication.

  Remarks: Either `subcluster_id` or `subcluster_name` must be provided. The user must have the necessary privileges to manage replication.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Dropped publication 'pub1' from database 'postgres' on␣
↪sub-cluster 3"
}
```

modifyPublication

Modifies an existing publication in logical replication.

Here is an example for the request:

```
{
    "operation": "modifyPublication",
    "cluster_id": 1,
    "subcluster_id": 3,
    "pub_name": "old_pub_name",
    "pub_db_name": "postgres",
    "new_pub_name": "new_pub_name",
    "include_all_tables": true,
    "db_tables": ["t1", "t2"]
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID containing the publication.
- `pub_name`: The current name of the publication.
- `{pub_db_name|db_name}`: The database containing the publication.
- `new_pub_name`: Optional new name for the publication.
- `include_all_tables`: Whether to replicate all tables (including future ones).
- `db_tables`: List of specific tables to include in the publication.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Modified publication 'old_pub_name' to 'new_pub_name' in␣
↪database 'postgres'"
}
```

getPublications

Retrieves list of all publications configured for logical replication on the specified host.

Here is an example for the request:

```
{
    "operation": "getPublications",
    "cluster_id": 1,
    "subcluster_id": 3,
    "subcluster_name": "p1",
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID to query publications from.
- `subcluster_name`: The sub-cluster name to query publications from.

  Remarks: Either `subcluster_id` or `subcluster_name` must be provided.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "publications": [
        {
            "publication_name": "pub1",
            "database": "postgres",
            "all_tables": true,
            "tables": []
        },
        {
            "publication_name": "pub2",
            "database": "mydb",
            "all_tables": false,
            "tables": ["table1", "table2"]
        }
    ]
}
```

createSubscription

Creates a subscription for logical replication. A subscription represents a replication connection to the publisher.

Here is an example for the request:

```
{
    "operation": "createSubscription",
    "cluster_id": 1,
    "subcluster_id": 4,
    "subcluster_name": "p2",
    "sub_name": "sub1",
    "pub_name": "pub1",
    "pub_db_name": "mydb",
    "pub_origin": "any",
    "sub_db_name": "postgres",
    "copy_data": false
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID to create the subscription on.
- `subcluster_name`: The sub-cluster name to create the subscription on.
- `sub_name`: The name of the new subscription.
- `pub_name`: The name of the publication to subscribe to.
- `pub_db_name`: Optional: the database name of the publication.
- `pub_origin`: Specifies whether to request changes with no origin ("none") or all changes regardless of origin ("any").
- `sub_db_name`: The database to create the subscription in (preferred field name).
- `db_name`: Deprecated: use `sub_db_name` instead. The database to create the subscription in.
- `copy_data`: Specifies whether to copy pre-existing data in the publications when replication starts.

  Remarks: Either `subcluster_id` or `subcluster_name` must be provided. The subscription name must be distinct from existing subscriptions in the database. The `pub_db_name` is optional and can be ignored, yet with multiple publications having the same name an error will be reported. In this case, it must be provided. The default value for `pub_origin` is `"any"`, and for `copy_data` is `true`. When `copy_data=true` and `origin=none`, initial sync copies data directly from publisher, which may include data from upstream sources. The user must have the necessary privileges to manage replication.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Created subscription 'sub1' for publication 'pub1' in␣
↪database 'postgres' on sub-cluster 4"
}
```

dropSubscription

Removes an existing subscription from logical replication.

Here is an example for the request:

```
{
    "operation": "dropSubscription",
    "cluster_id": 1,
    "subcluster_id": 4,
    "subcluster_name": "p2",
    "sub_name": "sub1",
    "sub_db_name": "postgres"
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID containing the subscription.
- `subcluster_name`: The sub-cluster name containing the subscription.
- `sub_name`: The name of the subscription to drop.
- `{sub_db_name|db_name}`: The database containing the subscription.

  Remarks: Either `subcluster_id` or `subcluster_name` must be provided. The user must have the necessary privileges to manage replication.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Dropped subscription 'sub1' from database 'postgres' on␣
↪sub-cluster 4"
}
```

modifySubscription

Modifies an existing subscription in logical replication.

Here is an example for the request:

```
{
    "operation": "modifySubscription",
    "cluster_id": 1,
    "subcluster_id": 4,
    "sub_name": "old_sub_name",
    "sub_db_name": "postgres",
    "new_sub_name": "new_sub_name",
    "pub_origin": "none",
    "enabled": true
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID containing the subscription.

- `sub_name`: The current name of the subscription.
- `sub_db_name`: The database containing the subscription. (preferred field name).
- `db_name`: Deprecated: use `sub_db_name` instead. The database containing the subscription.
- `new_sub_name`: Optional new name for the subscription.
- `pub_origin`: Optional new publication origin setting ('any' or 'none') for the subscription.
- `enabled`: Optional flag to enable/disable the subscription.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Modified subscription 'old_sub_name' to 'new_sub_name' in␣
↪database 'postgres'"
}
```

setSubscriptionState

Sets the state (enabled/disabled) of a subscription.

Here is an example for the request:

```
{
    "operation": "setSubscriptionState",
    "cluster_id": 1,
    "subcluster_id": 4,
    "sub_name": "sub1",
    "sub_db_name": "postgres",
    "enabled": true
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID containing the subscription.
- `sub_name`: The name of the subscription to modify.
- `{sub_db_name|db_name}`: The database containing the subscription.
- `enabled`: Whether to enable (true) or disable (false) the subscription.

  Remarks: The user must have the necessary privileges to manage replication.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "message": "Subscription 'sub1' in database 'postgres' on sub-cluster 4␣
↪has been enabled"
}
```

getSubscriptions

Retrieves list of all subscriptions configured for logical replication on the specified host.

Here is an example for the request:

```
{
    "operation": "getSubscriptions",
    "cluster_id": 1,
    "subcluster_id": 4,
    "subcluster_name": "p2"
}
```

- `cluster_id`: The logical replication cluster ID.
- `subcluster_id`: The sub-cluster ID to query subscriptions from.
- `subcluster_name`: The sub-cluster name to query subscriptions from.

  Remarks: Either `subcluster_id` or `subcluster_name` must be provided.

Here is an example successful response:

```
{
    "request_status": "Ok",
    "subscriptions": [
        {
            "subscription_name": "sub1",
            "database": "postgres",
            "publication_name": "pub1",
            "publication_database": "mydb",
            "publication_origin": "any",
            "enabled": true
        },
        {
            "subscription_name": "sub2",
            "database": "mydb",
            "publication_name": "pub2",
            "publication_database": "postgres",
            "publication_origin": "none",
            "enabled": false
        }
    ]
}
```

3.12.16 Maintenance

The "v2/maintenance" is for creating, deleting and reading maintenance periods.

addMaintenance

The addMaintenance call is for adding new maintenance periods for clusters and hosts. A maintenance period is either belong to an entire cluster, in which case the cluster must be identified or it belongs to a host, so that the host name must be provided.

```
{
    "deadline": "2017-06-23T10:46:37.000Z",
    "hostname": "192.168.1.195",
    "initiate": "2017-06-23T10:46:27.000Z",
    "operation": "addMaintenance",
    "reason": "test_31643_maintenance",
    "request_created": "2017-06-23T10:46:27.754Z",
    "request_id": 3
}
```


- `deadline`: Marks the end of the maintenance period. At this time the maintenance period will automatically expire.
- `cluster_id`: The numerical ID of the cluster if this is a cluster maintenance period.
- `hostname`: The name of the host if this is a host maintenance.
- `initiate`: The start of the maintenance period.
- `reason`: A human readable string which the users will see.


```
{
    "UUID": "366372d3-2633-f126-bf8b-c69418274d69",
    "request_created": "2017-06-23T10:46:27.754Z",
    "request_id": 3,
    "request_processed": "2017-06-23T10:46:27.800Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- UUID: A unique string which can be used to identify the maintenance period.

removeMaintenance

This RPC call can be used to delete maintenance periods before their time. Deleting a maintenance period will make them expire immediately.


```
{
    "UUID": "a1f2f274-b151-e286-0eee-199efaa5b444",
    "operation": "removeMaintenance",
    "request_created": "2017-06-23T13:23:28.770Z",
    "request_id": 3
}
```


- UUID: Indentifies which maintenance period is going to be deleted.


```
{
    "request_created": "2017-06-23T13:23:28.770Z",
    "request_id": 3,
    "request_processed": "2017-06-23T13:23:28.816Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

getMaintenance

This call either sends back cluster maintenance periods when it received a cluster ID or host maintenance periods when it received one or more hostnames. When no cluster ID provided and no host names are sent either, both cluster maintenance and host maintenance periods are sent back.

The idea behind this call is that when the UI shows a host, it gets the maintenance periods of that host to show the details, when it shows a cluster it gets the maintenance periods of that cluster to show the details. If however the UI needs to know what nodes are in a cluster, that's an other call, that's not a "getMaintenance" call at all.

```
{
    "operation": "getMaintenance",
    "request_created": "2017-06-23T10:46:27.913Z",
    "request_id": 3
}
```

Here is an example of an s9s run. It shows two maintenance records, one for a host and one for a cluster:

```
$ s9s maintenance --list --long
ST UUID    OWNER GROUP     START               END                 ␣
↪HOST/CLUSTER  *         REASON
-h 1da4955 pipas testgroup 2019-11-10 13:05:40 2019-11-10 15:05:40 192.
↪168.0.58         testMaintenanceJob4
Ac 2358876 pipas testgroup 13:05:37            14:05:37            ␣
↪ft_maintenance_36279 testMaintenanceJob1
-c d89319c pipas testgroup 2019-11-09 13:05:39 2019-11-09 15:05:39␣
↪ft_maintenance_36279 testMaintenanceJob3
Total: 2
```

But the two records has three maintenance periods in it, because the same cluster has two maintenance periods. Now these looks like this in a reply:

```
{
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
  "maintenance_records": [
    {
      "class_name": "CmonMaintenanceInfo",
      "hostname": "192.168.0.58",
      "is_active": false,
      "maintenance_periods": [ {
          "UUID": "1da49552-dd68-4604-ab16-a340c312cd9c",
          "deadline": "2019-11-10T14:05:40.000Z",
          "groupid": 4,
          "groupname": "testgroup",
          "initiate": "2019-11-10T12:05:40.000Z",
          "is_active": false,
          "reason": "testMaintenanceJob4",
          "userid": 4,
          "username": "pipas"
        }  ]
    },
```

(continues on next page)

(continued from previous page)


```
    {
      "class_name": "CmonMaintenanceInfo",
      "cluster": {
        "acl": "user::rwx,group::rwx,other::---",
        "alarm_statistics": {
          "class_name": "CmonAlarmStatistics"
        },
        "cdt_path": "/",
        "class_name": "CmonClusterInfo",
        "cluster_auto_recovery": true,
        "cluster_id": 1,
        "cluster_name": "ft_maintenance_36279",
        "cluster_type": "POSTGRESQL_SINGLE",
        "configuration_file": "/tmp/cmon_1.cnf",
        "effective_privileges": "",
        "group_owner": {
          "acl": "user::rwx,group::rwx,other::---",
          "cdt_path": "/groups",
          "class_name": "CmonGroup",
          "created": "2019-11-08T12:02:37.983Z",
          "group_id": 4,
          "group_name": "testgroup",
          "owner_group_id": 1,
          "owner_group_name": "admins",
          "owner_user_id": 1,
          "owner_user_name": "system"
        },
        "job_statistics": {
          "class_name": "CmonJobStatistics"
        },
        "log_file": "/tmp/cmon_1.log",
        "maintenance_mode_active": true,
        "managed": true,
        "node_auto_recovery": true,
        "owner": {
          "acl": "user::rwx,group::r--,other::r--",
          "cdt_path": "/",
          "class_name": "CmonUser",
          "created": "2019-11-08T12:02:37.995Z",
          "disabled": false,
          "email_address": "laszlo@severalnines.com",
          "first_name": "Laszlo",
          "groups": [ {
              "acl": "user::rwx,group::rwx,other::---",
              "cdt_path": "/groups",
              "class_name": "CmonGroup",
              "created": "2019-11-08T12:02:37.983Z",
              "group_id": 4,
              "group_name": "testgroup",
```


(continues on next page)

(continued from previous page)


```
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "owner_user_id": 1,
            "owner_user_name": "system"
          }  ],
        "last_failed_login": "",
        "last_login": "2019-11-08T12:24:56.456Z",
        "last_name": "Pere",
        "n_failed_logins": 0,
        "origin": "CmonDb",
        "owner_group_id": 1,
        "owner_group_name": "admins",
        "owner_user_id": 4,
        "owner_user_name": "pipas",
        "suspended": false,
        "timezone": {
          "abbreviation": "CET",
          "class_name": "CmonTimeZone",
          "name": "Central European Time",
          "offset": -3600,
          "has_dst_rules": true
        },
        "user_id": 4,
        "user_name": "pipas"
      },
      "state": "STARTED",
      "status_text": "All nodes are operational.",
      "vendor": "postgres",
      "version": "9.6"
    },
    "cluster_id": 1,
    "is_active": true,
    "maintenance_periods": [
      {
        "UUID": "2358876c-8eec-42c8-b149-69e722044b3f",
        "deadline": "2019-11-08T13:05:37.178Z",
        "groupid": 4,
        "groupname": "testgroup",
        "initiate": "2019-11-08T12:05:37.178Z",
        "is_active": true,
        "reason": "testMaintenanceJob1",
        "userid": 4,
        "username": "pipas"
      },
      {
        "UUID": "d89319c9-3e7c-462e-9f54-4052682d2daa",
        "deadline": "2019-11-09T14:05:39.000Z",
        "groupid": 4,
        "groupname": "testgroup",
```


(continues on next page)

(continued from previous page)


```
            "initiate": "2019-11-09T12:05:39.000Z",
            "is_active": false,
            "reason": "testMaintenanceJob3",
            "userid": 4,
            "username": "pipas"
          }
        ]
      }
  ],
  "reply_received": "2019-11-08T12:24:56.475Z",
  "request_created": "2019-11-08T12:24:56.464Z",
  "request_id": 3,
  "request_processed": "2019-11-08T12:24:56.475Z",
  "request_status": "Ok",
  "request_user_id": 4,
  "total": 2
}
```


getCurrentMaintenance

This RPC call was made so that the client program has a quick and easy way to get information about the active maintanence for a cluster or a host.

If this call finds a host maintenance that is active it will return that, if it finds a cluster maintenance that will be returned.

Here is a call to search for a cluster wide maintenance (no host maintenance is returned ever).


```
{
    "cluster_id": 1,
    "operation": "getCurrentMaintenance",
    "request_created": "2019-11-11T15:07:37.539Z",
    "request_id": 3
}
```


Here is a call that returns either the cluster maintenance or the host maintenance or no maintenance if neither of those found:


```
{
    "cluster_id": 1,
    "hostname": "10.10.1.8",
    "operation": "getCurrentMaintenance",
    "request_created": "2019-11-12T08:55:58.507Z",
    "request_id": 3
}
```


And here is how a reply should look like when a maintenance period is found:


```
{
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
  "found_maintenance": true,
```


(continues on next page)

(continued from previous page)


```
  "maintenance_period": {
    "UUID": "9dec403d-0940-4603-b93f-3c58fa773ce1",
    "class_name": "CmonMaintenanceDate",
    "deadline": "2019-11-11T16:07:12.969Z",
    "groupid": 4,
    "groupname": "testgroup",
    "initiate": "2019-11-11T15:07:12.969Z",
    "reason": "testMaintenanceJob1",
    "userid": 4,
    "username": "pipas"
  },
  "reply_received": "2019-11-11T15:07:37.546Z",
  "request_created": "2019-11-11T15:07:37.539Z",
  "request_id": 3,
  "request_processed": "2019-11-11T15:07:37.546Z",
  "request_status": "Ok",
  "request_user_id": 4,
  "ui_string": "Cluster 1 is under maintenance: testMaintenanceJob1"
}
```


getNextMaintenance

This RPC call was made so that the client program has a quick and easy way to get information about the next coming maintenance periods.

If this call finds a host maintenance that is active it will return that, if it finds a cluster maintenance that will be returned.

Here is a call to search for a cluster wide maintenance (no host maintenance is returned ever).

```
{
    "cluster_id": 1,
    "operation": "getNextMaintenance",
    "request_created": "2019-11-12T08:07:29.966Z",
    "request_id": 3
}
```

Here is a call that returns either the cluster maintenance or the host maintenance or no maintenance if neither of those found:

```
{
    "cluster_id": 1,
    "hostname": "10.10.1.8",
    "operation": "getNextMaintenance",
    "request_created": "2019-11-12T08:55:15.628Z",
    "request_id": 3
}
```

And here is how a reply should look like when a maintenance period is found:

```
{
  "debug_messages": [ "RPC V2 authenticated user is 'pipas'."  ],
  "found_maintenance": true,
  "maintenance_period": {
    "UUID": "8a812d58-18a2-4d55-9702-2e5e05de1a41",
    "class_name": "CmonMaintenanceDate",
    "deadline": "2019-11-13T09:07:15.000Z",
    "groupid": 4,
    "groupname": "testgroup",
    "initiate": "2019-11-13T08:07:15.000Z",
    "reason": "testMaintenanceJob1",
    "userid": 4,
    "username": "pipas"
  },
  "reply_received": "2019-11-12T08:07:29.973Z",
  "request_created": "2019-11-12T08:07:29.966Z",
  "request_id": 3,
  "request_processed": "2019-11-12T08:07:29.973Z",
  "request_status": "Ok",
  "request_user_id": 4,
  "ui_string": "Cluster 1 maintenance starts Nov 13 08:07:15:␣
↪testMaintenanceJob1"
}
```


3.12.17 Mysql databaseperformance

noPrimaryKeys

An RPC call that returns a list of tables that do not have primary keys.

```
{
    "cluster_id": 1,
    "operation": "noPrimaryKeys",
}
```


```
{
    "controller_id": "5bfb8a65-8bbf-4762-9c48-e6603377b26e",
    "request_processed": "2024-08-01T12:16:59.926Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "total": 2,
    "data":
    [
        {
            "database": "mysql",
            "engine": "myEngine",
            "table": "AAA"
        },
        {
            "database": "mysql",
```


(continues on next page)

(continued from previous page)


```
            "engine": "myEngine",
            "table": "BBB"
        }
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 'Adam'."
    ]
}
```


myIsamTables

An RPC call that returns a list of tables that are using MyISAM engine.


```
{
    "cluster_id": 1,
    "operation": "myisamTables",
}
```

```
{
    "controller_id": "5bfb8a65-8bbf-4762-9c48-e6603377b26e",
    "request_processed": "2024-08-01T12:16:59.926Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "total": 1,
    "data":
    [
        {
            "database": "mysql",
            "engine": "myIsam",
            "table": "BBB"
        }
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 'Adam'."
    ]
}
```


redundantIndexes

An RPC call that returns a list of tables that have redundant indexes.


```
{
    "cluster_id": 1,
    "operation": "redundantIndexes",
}
```

```
{
    "controller_id": "5bfb8a65-8bbf-4762-9c48-e6603377b26e",
    "request_processed": "2024-08-01T12:16:59.926Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "total": 1,
    "data":
    [
       {
          "advise": "Do not touch",
          "database": "myXdb",
          "index": "66",
          "indexColumns": "88",
          "rediдxColumns": "55",
          "redundantIndex": "111",
          "table": "myTable"
      }
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 'Adam'."
    ]
}
```


3.12.18 Reports

The `/v2/reports` provides access to reports generated by the Cmon controller.

Download report

Pass the basename of the report after path `/v2/reports/`, and you must specify the cluster-id as well via the ?cluster_id= query argument.

```
curl -k -b cookies.jar
↪'https://127.0.0.1:9501/v2/reports/default_2023-01-31_071953.html?cluster_id=103
↪'
```

```
<!DOCTYPE html>
<html><body>
...
<p class="text"><br/><b>How do you like this report?</b><br/>What else do␣
↪you like to see or not see? We'd <a href=
↪"mailto:info@severalnines.com?subject=Operation Reports Feedback">
↪appreciate your feedback</a>. Thanks!<br/></p>

</body>
</html>
```

generateReport

Generates a named report for a cluster and returns its content inline. The `report_type` field specifies which report to generate, and `text_format` controls the output format (`AnsiTerminal`, `Html`, etc.). The generated report content is returned in `report.content` and `report.document`.

```
{
    "cluster_id": 1,
    "operation": "generateReport",
    "report":
    {
        "class_name": "CmonReport",
        "recipients": "laszlo@severalnines.com",
        "report_type": "testreport",
        "text_format": "AnsiTerminal"
    },
    "request_created": "2017-08-04T11:06:32.148Z",
    "request_id": 3
}
```

```
{
    "report":
    {
        "class_name": "CmonReport",
        "content": "  Simple Test Report\n\n    This report was created to␣
↪test the report subsystem. The document was\n  created in the␣
↪CmonReporter::buildTestReport() method.\n\n",
        "created": "2017-08-04T11:06:32.000Z",
        "document":
        {
            "className": "CmonDocument",
            "elements": [
            {
                "className": "CmonTitle",
                "text": "Simple Test Report",
                "titleLevel": 1
            },
            {
                "className": "CmonParagraph",
                "text": "This report was created to test the report␣
↪subsystem. The document was created in the␣
↪CmonReporter::buildTestReport() method."
            } ]
        },
        "owner": "pipas",
        "path": "/
↪home/pipas/s9s_tmp/1/postgresql_single/cmon-reports/testreport_2017-08-04_130632.html",
↪
        "recipients": "laszlo@severalnines.com",
        "report_id": 1,
        "report_type": "testreport",
```

(continues on next page)

(continued from previous page)

```
        "text_format": "AnsiTerminal",
        "title": "Simple Test Report"
    },
    "request_created": "2017-08-04T11:06:32.148Z",
    "request_id": 3,
    "request_processed": "2017-08-04T11:06:32.200Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

- `report`: The report that has been created as a CmonReport class object.

  To get a comprehensive list of the properties the controller supports in the CmonReport class one can run the following command:

  ```
  s9s metatype --list-properties --type=CmonReport --long
  ```

getReport

The request format for this call has recently been changed. Instead of passing just a single ID we need to pass a CmonReport class object. Here is how:

```
{
    "operation": "getReport",
    "report":
    {
        "class_name": "CmonReport",
        "report_id": 1,
        "text_format": "AnsiTerminal"
    },
    "request_created": "2019-10-04T07:39:40.138Z",
    "request_id": 3
}
```

getReportTemplates

This call is to return the report templates describing what kind of reports are available for the user when generating reports. Report templates are just simple CmonReport objects with a few properties set to reflect the properties that the report will have when a report with the same report type is used.

So template name is just an other way of saying `report_type`.

Here is an example showing how a request should look like:

```
{
    "operation": "getReportTemplates",
    "request_created": "2019-07-31T06:45:30.083Z",
    "request_id": 3
}
```

And here is an example showing how the reply will look like:

```
{
    "reports": [
    {
        "class_name": "CmonReport",
        "report_type": "default",
        "title": "System Report"
    },

    . . .

    {
        "class_name": "CmonReport",
        "report_type": "capacity",
        "title": "Database Growth Report"
    } ],
    "request_created": "2019-07-31T06:45:30.083Z",
    "request_id": 3,
    "request_processed": "2019-07-31T06:45:30.089Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```


deleteReport

A call to delete an existing report from the controller.


```
{
    "cluster_id": 1,
    "operation": "deleteReport",
    "report":
    {
        "class_name": "CmonReport",
        "report_id": 1
    },
    "request_created": "2017-08-04T07:11:38.745Z",
    "request_id": 4
}
```



```
{
    "request_created": "2017-08-04T07:11:38.745Z",
    "request_id": 4,
    "request_processed": "2017-08-04T07:11:38.793Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

getReports

A call to get existing reports from the controller.

```
{
    "cluster_id": 1,
    "operation": "getReports",
    "request_created": "2017-08-02T12:34:59.892Z",
    "request_id": 4
}
```

```
{
    "reports": [
    {
        "class_name": "CmonReportInfo",
        "cluster_id": 1,
        "created": "2017-08-02T13:01:44.000Z",
        "name": "availability_2017-08-02_143459.html",
        "owner": "pipas",
        "path": "/
↪home/pipas/s9s_tmp/1/postgresql_single/cmon-reports/availability_2017-08-02_143459.html
↪
        "recipients": "",
        "report_id": 1,
        "report_type": "availability"
    } ],
    "request_created": "2017-08-02T12:34:59.892Z",
    "request_id": 4,
    "request_processed": "2017-08-02T12:34:59.936Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 1
}
```

listErrorReports

A call to list the generated error reports.

```
{
    "cluster_id": 90,
    "operation": "listErrorReports",
}
```

```
{
 "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
 "is_superuser": true,
 "request_processed": "2022-09-26T08:21:43.405Z",
 "request_status": "Ok",
 "total": 1,
 "data":
 [
```

(continues on next page)

(continued from previous page)

```
    {
        "created": "2022-09-26T08:21:18.000Z",
        "id": 13,
        "path": "/
↪home/cmon_user/s9s_tmp/error-report-2022-09-26_082116-cluster90.tar.gz",
        "size": "7.63 MiB",
        "www": false
    }
 ]
}
```

removeErrorReport

A call to list the generated error reports.

```
{
    "cluster_id": 90,
    "operation": "removeErrorReport",
    "id": 13
}
```

Returns:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2022-09-26T08:58:28.004Z",
  "request_status": "Ok"
}
```

downloadErrorReport

A method to download a specific error report. Please NOTE this method will return the generated `.tar.gz` file directly (so don't expect a JSON reply).

```
{
    "cluster_id": 90,
    "operation": "downloadErrorReport",
    "id": 13
}
```

The return value is an `application/octet-stream` of the `tar.gz` file directly. Or in case of error a normal JSON RPC reply including the error reason.

addSchedule

A call to schedule an operational report.

```
{
    "cluster_id": 90,
```

(continues on next page)

(continued from previous page)

```
    "operation": "addSchedule",
    "name": "default",
    "schedule": "*0 10 * * *",
    "recipients": "kedazo@severalnines.com"
}
```

Returns:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2022-09-26T08:58:28.004Z",
  "request_status": "Ok",
  "id": 103
}
```

removeSchedule

A call to remove an operational report schedule.

```
{
    "id": 112,
    "operation": "removeSchedule"
}
```

Returns:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2022-09-26T08:58:28.004Z",
  "request_status": "Ok"
}
```

listSchedules

Lists the currently active operational reports schedules.

```
{
    "operation": "listSchedules"
}
```

Returns:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2023-01-26T14:16:16.603Z",
  "request_status": "Ok",
  "total": 2,
```

(continues on next page)

(continued from previous page)


```
"data":
[
    {
        "arguments": "days=7",
        "command": "default",
        "id": 1,
        "recipients": "",
        "schedule": "1 6 * * *"
    },
    {
        "arguments": "days=7",
        "command": "upgrade",
        "id": 2,
        "recipients": "kedazo@severalnines.com",
        "schedule": "1 22 10 * *"
    }
],
"debug_messages":
[
    "RPC V2 authenticated user is 'alex'."
],
"user":
{
    "class_name": "CmonUser",
    "cdt_path": "/",
    "owner_user_id": 68,
    "owner_user_name": "alex",
    "owner_group_id": 1,
    "owner_group_name": "admins",
    "acl": "user::rwx,group::r--,other::r--",
    "created": "2022-12-16T15:21:36.542Z",
    "disabled": false,
    "distinguished_name": "uid=alex,ou=People,dc=severalnines,dc=org",
    "email_address": "alex@severalnines.com",
    "first_name": "alex",
    "last_failed_login": "",
    "last_login": "2023-01-26T14:16:16.601Z",
    "n_failed_logins": 0,
    "origin": "LDAP",
    "suspended": false,
    "user_id": 68,
    "user_name": "alex",
    "groups":
    [
        {
            "class_name": "CmonGroup",
            "cdt_path": "/groups",
            "owner_user_id": 1,
            "owner_user_name": "system",
```


(continues on next page)

(continued from previous page)


```
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "acl": "user::rwx,group::rwx,other::---",
            "created": "2022-02-23T11:01:14.628Z",
            "group_id": 1,
            "group_name": "admins"
        }
    ],
    "timezone":
    {
        "class_name": "CmonTimeZone",
        "name": "Central European Time",
        "abbreviation": "CET",
        "offset": -3600,
        "has_dst_rules": false
    }
  }
}
```


3.12.19 Repositories

The `/v2/repositories` path provides calls to create and manage local software repositories and get information about supported software.

getRepositories

Returns available local repositories.


```
{
    "cluster_id": 1,
    "operation": "getRepositories",
    "request_created": "2017-07-31T07:56:02.214Z",
    "request_id": 3
}
```


- `cluster_type`: filter by cluster-type (galera, mongodb, postgresql, …)
- `vendor`: filter by vendor (percona, mariadb, …)
- `db_version`: filter by database(major.minor) version (5.6, 10.1, …)


```
{
    "repositories": [
    {
        "class_name": "CmonRepository",
        "cluster_type": "galera",
        "db_version": "5.6",
        "modified": "2017-08-01T08:25:29.000Z",
        "name": "percona-5.6-apt-precise",
        "os":
        {
```


(continues on next page)

(continued from previous page)


```
            "distribution/codename": "unknown",
            "distribution/name": "unknown",
            "distribution/release": "unknown",
            "distribution/type": "debian"
        },
        "path": "/var/tmp/unammed-cmon-repo",
        "used_by_cluster": "",
        "vendor": "percona"
    } ],
    "request_created": "2017-08-01T08:25:35.414Z",
    "request_id": 4,
    "request_processed": "2017-08-01T08:25:35.462Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 1
}
```


deleteRepository

Removes a software repository registration from the controller and deletes the repository directory from disk. This operation is irreversible.

getRepositorySetup

Prints out the (semi-)generated repository filename and contents to use the repository manually. You have to substitute the CMON-HOSTNAME string with the controllers IP address before u use it.

getSupportedSetups

This call will return what possible setups can be used to create/install a new node.

The request does not need a cluster ID or cluster name, the only field that is needed is the "operation" field:


```
{
    "operation": "getSupportedSetups",
    "request_created": "2017-07-31T09:57:07.145Z",
    "request_id": 3
}
```


Here is how the reply will look like:


```
{
    "request_created": "2017-07-31T09:57:07.145Z",
    "request_id": 3,
    "request_processed": "2017-07-31T09:57:07.188Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "supported_setups": [
    {
        "cluster_type": "mongodb",
        "db_version": "3.2",
```


(continues on next page)

(continued from previous page)


```
        "os_version": "5",
        "vendor": "10gen"
    },

    . . .

    {
        "cluster_type": "replication",
        "db_version": "5.7",
        "os_version": "xenial",
        "vendor": "oracle"
    } ],
    "total": 206
}
```


3.12.20 Cmon Events

The `/v2/events` path holds proxied requests to cmon-events.

test

This request proxies to the `/test` endpoint in cmon-events.


```
{
   "operation": "test",
   "request_created": "2017-06-20T06:19:58.451Z",
   "name": "test",
   "credentials": {
   }
}
```


getSpreadsheetNames

Returns the list of spreadsheet names available for a cluster. Spreadsheets are used for capacity planning and other analytical reports. Use the names returned here with `getSpreadsheet` to retrieve the actual data.


```
{
    "cluster_id": 1,
    "operation": "getSpreadsheetNames",
    "request_created": "2018-10-01T10:25:27.120Z",
    "request_id": 3
}
```


getSpreadsheet

Returns the contents of a named spreadsheet for a cluster. Use `getSpreadsheetNames` first to discover the spreadsheet names available for a given cluster.

```
{
    "cluster_id": 1,
    "operation": "getSpreadsheet",
    "request_created": "2018-10-02T07:12:36.353Z",
    "request_id": 3,
    "spreadsheet_name": "Capacity Planning"
}
```


3.12.21 Statistical Data Calls

The `/v2/stat` provides calls to access various statistical information about the cluster.

getInfo

This call is for getting the data that the sheet manager collected from the system. Here is an example of the call:


```
{
    "operation": "getInfo",
    "request_created": "2017-07-27T12:58:00.283Z",
    "request_id": 3
}
```



```
{
    "collected_info": [
    {
        "cluster_id": 1,
        "info":
        {
            "cluster.status": 2,
            "cluster.statustext": "Cluster started.",
            "cmon.domainname": "",
            "cmon.hostname": "t7500",
            "cmon.running": true,
            "cmon.starttime": 1501144642,
            "cmon.uptime": 15636,
            "conf.backup_retention": 31,
            "conf.clusterid": 1,
            "conf.clustername": "ft_postgresql_48154",
            "conf.clustertype": 5,
            "conf.configfile": "/tmp/cmon_1.cnf",
            "conf.hostname": "192.168.1.127",
            "conf.os": "debian",
            "conf.statustext": "Configuration loaded.",
            "host.1.connected": true,
            "host.1.cpu_io_wait_percent": 0.353357,
            "host.1.cpu_steal_percent": 0,
            "host.1.cpu_usage_percent": 15.5331,
            "host.1.cpucores": 16,
            "host.1.cpuinfo": [
```


(continues on next page)

(continued from previous page)


```
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 4,
                "cpumaxmhz": 2.268e+06,
                "cpumhz": 1600,
                "cpumodel": "Intel(R) Xeon(R) CPU           L5520  @␣
↪2.27GHz",
                "cputemp": 55.5,
                "hostid": 1,
                "physical_cpu_id": 0,
                "siblings": 8,
                "vendor": "GenuineIntel"
            },
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 4,
                "cpumaxmhz": 2.268e+06,
                "cpumhz": 1733,
                "cpumodel": "Intel(R) Xeon(R) CPU           L5520  @␣
↪2.27GHz",
                "cputemp": 55.5,
                "hostid": 1,
                "physical_cpu_id": 1,
                "siblings": 8,
                "vendor": "GenuineIntel"
            } ],
            "host.1.cpumaxmhz": 2268,
            "host.1.cpumhz": 1733,
            "host.1.cpumodel": "Intel(R) Xeon(R) CPU           L5520  @␣
↪2.27GHz",
            "host.1.cputemp": 55.5,
            "host.1.devices": [ "/dev/mapper/core1--vg-root" ],
            "host.1.free_disk_bytes": 155814936576,
            "host.1.hostname": "192.168.1.174",
            "host.1.interfaces": [ "eth0" ],
            "host.1.ip": "192.168.1.174",
            "host.1.membuffer": 0,
            "host.1.memcached": 1,
            "host.1.memfree": 16031900,
            "host.1.memtotal": 16777216,
            "host.1.network_interfaces": [
            {
                "interface_name": "eth0",
                "rx_bytes_per_sec": 11775.8,
                "tx_bytes_per_sec": 17652.3
            } ],
            "host.1.pingdelay": -1,
            "host.1.pingstatustext": "Creating ICMP socket (to ping
↪'192.168.1.174') failed: Operation not permitted.",
```


(continues on next page)

(continued from previous page)


```
            "host.1.port": 8089,
            "host.1.rx_bytes_per_second": 11775.8,
            "host.1.swapfree": 0,
            "host.1.swaptotal": 0,
            "host.1.total_disk_bytes": 208033853440,
            "host.1.tx_bytes_per_second": 17652.3,
            "host.1.uptime": 15779,
            "host.1.wallclock": 1501160299,
            "host.1.wallclocksampled": 1501160235,
            "host.2.class_name": "controller",
            "host.2.connected": true,
            "host.2.cpu_io_wait_percent": 2.89087,
            "host.2.cpu_steal_percent": 0,
            "host.2.cpu_usage_percent": 204.378,
            "host.2.cpucores": 24,
            "host.2.cpuinfo": [
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 6,
                "cpumaxmhz": 2.661e+06,
                "cpumhz": 1596,
                "cpumodel": "Intel(R) Xeon(R) CPU           X5650  @␣
↪2.67GHz",
                "cputemp": 0,
                "hostid": 2,
                "physical_cpu_id": 0,
                "siblings": 12,
                "vendor": "GenuineIntel"
            },
            {
                "class_name": "CmonCpuInfo",
                "cpucores": 6,
                "cpumaxmhz": 2.661e+06,
                "cpumhz": 1596,
                "cpumodel": "Intel(R) Xeon(R) CPU           X5650  @␣
↪2.67GHz",
                "cputemp": 0,
                "hostid": 2,
                "physical_cpu_id": 1,
                "siblings": 12,
                "vendor": "GenuineIntel"
            } ],
            "host.2.cpumaxmhz": 2661,
            "host.2.cpumhz": 1596,
            "host.2.cpumodel": "Intel(R) Xeon(R) CPU           X5650  @␣
↪2.67GHz",
            "host.2.cputemp": 0,
            "host.2.devices": [ "/dev/sda1" ],
            "host.2.free_disk_bytes": 1409452683264,
```


(continues on next page)

(continued from previous page)


```
            "host.2.hostname": "192.168.1.127",
            "host.2.interfaces": [ "eth0" ],
            "host.2.ip": "192.168.1.127",
            "host.2.membuffer": 318624,
            "host.2.memcached": 19415480,
            "host.2.memfree": 9243460,
            "host.2.memtotal": 49453276,
            "host.2.network_interfaces": [
            {
                "interface_name": "eth0",
                "rx_bytes_per_sec": 29150.4,
                "tx_bytes_per_sec": 18520.8
            } ],
            "host.2.pingdelay": -1,
            "host.2.pingstatustext": "Creating ICMP socket (to ping
↪'192.168.1.127') failed: Operation not permitted.",
            "host.2.port": 9555,
            "host.2.rx_bytes_per_second": 29150.4,
            "host.2.swapfree": 0,
            "host.2.swaptotal": 0,
            "host.2.total_disk_bytes": 2125259440128,
            "host.2.tx_bytes_per_second": 18520.8,
            "host.2.uptime": 2.59852e+06,
            "host.2.version": "1.4.3",
            "host.2.wallclock": 1501160235,
            "host.2.wallclocksampled": 1501160235,
            "license.expires": -1,
            "license.status": false,
            "license.statustext": "No license found.",
            "mail.statustext": "Created.",
            "netStat.1.eth0.rxBytes": 287358005,
            "netStat.1.eth0.txBytes": 205682361,
            "netStat.2.eth0.rxBytes": 531084878595,
            "netStat.2.eth0.txBytes": 503628558465
        }
    } ],
    "request_created": "2017-07-27T12:58:00.283Z",
    "request_id": 3,
    "request_processed": "2017-07-27T12:58:00.328Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


statByName

Returns time-series statistics samples for a named metric across all nodes in a cluster. Common metric names include `cpustat`, `memstat`, and `diskstat`. Filter results to a specific time window using `start_datetime` and `end_datetime`. Set `with_hosts` to `true` to include host metadata alongside each sample, and `calculate_per_sec` to `true` to normalise counter-type values to per-second rates.

```
{
    "cluster_id": 1,
    "end_datetime": "15:21",
    "name": "cpustat",
    "operation": "statByName",
    "request_created": "2017-07-27T13:30:41.866Z",
    "request_id": 3,
    "start_datetime": "15:20",
    "with_hosts": true,
    "calculate_per_sec": true
}
```



```
{
    "data": [
    {
        "busy": 0.00555556,
        "cpuid": 0,
        "cpumhz": 1600,
        "cpumodelname": "Intel(R) Xeon(R) CPU           L5520  @ 2.27GHz",
        "cpuphysicalid": 0,
        "cputemp": 55.5,
        "created": 1501161605,
        "hostid": 1,
        "idle": 0.994444,
        "interval": 5473,
        "iowait": 0,
        "loadavg1": 0.14,
        "loadavg15": 0.09,
        "loadavg5": 0.11,
        "sampleends": 1501161608,
        "samplekey": "CmonCpuStats-1-0",
        "steal": 0,
        "sys": 0.0037037,
        "uptime": 17107,
        "user": 0.00185185
    },

    . . .

    {
        "busy": 0.0980228,
        "cpuid": 23,
        "cpumhz": 1596,
        "cpumodelname": "Intel(R) Xeon(R) CPU           X5650  @ 2.67GHz",
        "cpuphysicalid": 0,
        "cputemp": 0,
        "created": 1501161658,
        "hostid": 2,
        "idle": 0.901977,
        "interval": 5379,
```


(continues on next page)

(continued from previous page)


```
        "iowait": 0,
        "loadavg1": 2.35,
        "loadavg15": 2.23,
        "loadavg5": 2.33,
        "sampleends": 1501161661,
        "samplekey": "CmonCpuStats-2-23",
        "steal": 0,
        "sys": 0.0247148,
        "uptime": 2.5999e+06,
        "user": 0.073308
    } ],
    "hosts": [
    {
        "class_name": "CmonPostgreSqlHost",
        "clusterid": 1,
        "configfile": [ "/etc/postgresql/9.6/main/postgresql.conf" ],
        "connected": true,
        "data_directory": "/var/lib/postgresql/9.6/main",
        "datadir": "/var/lib/postgresql/9.6/main",
        "description": "",
        "distribution":
        {
            "codename": "xenial",
            "name": "ubuntu",
            "release": "16.04",
            "type": "debian"
        },
        "hostId": 1,
        "hostname": "192.168.1.174",
        "hoststatus": "CmonHostOnline",
        "hot_standby": true,
        "ip": "192.168.1.174",
        "lastseen": 1501162240,
        "logfile": "/var/log/postgresql/postgresql-9.6-main.log[0m",
        "maintenance_mode_active": false,
        "message": "Up and running",
        "nodetype": "postgres",
        "pid": 5247,
        "pidfile": "/var/run/postgresql/9.6-main.pid",
        "pingstatus": 0,
        "pingtime": 0,
        "port": 8089,
        "readonly": false,
        "received_location": "0/1523B10",
        "replay_location": "0/1523B10",
        "role": "master",
        "sshfailcount": 0,
        "ssl_certs":
        {
```


(continues on next page)

(continued from previous page)


```
            "server":
            {
                "ca": "",
                "id": 0,
                "key": "/etc/ssl/private/ssl-cert-snakeoil.key",
                "path": "/etc/ssl/certs/ssl-cert-snakeoil.pem"
            }
        },
        "timestamp": 1501162240,
        "unique_id": 1,
        "uptime": 17592,
        "version": "9.6.3",
        "wallclock": 1501162260,
        "wallclocktimestamp": 1501162197
    },
    {
        "class_name": "CmonHost",
        "clusterid": 1,
        "configfile": "/tmp/cmon_1.cnf",
        "connected": true,
        "distribution":
        {
            "codename": "trusty",
            "name": "ubuntu",
            "release": "14.04",
            "type": "debian"
        },
        "hostId": 2,
        "hostname": "192.168.1.127",
        "hoststatus": "CmonHostOnline",
        "ip": "192.168.1.127",
        "lastseen": 1501162240,
        "logfile": "/tmp/cmon_1.log",
        "maintenance_mode_active": false,
        "message": "Up and running",
        "nodetype": "controller",
        "pid": 48111,
        "pingstatus": 0,
        "pingtime": 0,
        "port": 9555,
        "role": "controller",
        "timestamp": 1501162240,
        "unique_id": 2,
        "uptime": 17759,
        "version": "1.4.3",
        "wallclock": 1501162197,
        "wallclocktimestamp": 1501162197
    } ],
    "reply_received": "2017-07-27T13:30:41.871Z",
```


(continues on next page)

(continued from previous page)


```
    "request_created": "2017-07-27T13:30:41.866Z",
    "request_id": 3,
    "request_processed": "2017-07-27T13:30:41.929Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 384
}
```


getCpuPhysicalInfo

Returns physical CPU hardware information for each node in a cluster. Each entry in the `data` array represents one physical CPU socket on one host, and includes the CPU model name, core count, clock speed, socket topology, and temperature reading.


```
{
    "cluster_id": 1,
    "operation": "getCpuPhysicalInfo",
    "request_created": "2017-07-27T13:43:47.802Z",
    "request_id": 3
}
```



```
{
    "data": [
    {
        "class_name": "CmonCpuInfo",
        "cpucores": 4,
        "cpumaxmhz": 2.268e+06,
        "cpumhz": 1600,
        "cpumodel": "Intel(R) Xeon(R) CPU           L5520  @ 2.27GHz",
        "cputemp": 55.5,
        "hostid": 1,
        "physical_cpu_id": 0,
        "siblings": 8,
        "vendor": "GenuineIntel"
    },
    {
        "class_name": "CmonCpuInfo",
        "cpucores": 4,
        "cpumaxmhz": 2.268e+06,
        "cpumhz": 1600,
        "cpumodel": "Intel(R) Xeon(R) CPU           L5520  @ 2.27GHz",
        "cputemp": 55.5,
        "hostid": 1,
        "physical_cpu_id": 1,
        "siblings": 8,
        "vendor": "GenuineIntel"
    } ],
    "request_created": "2017-07-27T13:43:47.802Z",
    "request_id": 3,
```


(continues on next page)

(continued from previous page)

```
    "request_processed": "2017-07-27T13:43:47.848Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 2
}
```

The caller should specify the hostname and cluster id where cmonagent is deployed; Along with limit (maximum number of data to retrieve), origin e.g. "timeline", and begin and end date - range of queries to retrieve.

Example:

```
curl -k 'https://127.0.0.1:9501/v2/stat' -XPOST -d '{"hostname":
↪"10.10.10.10", "cluster_id": 123, "operation": "gettopqueries", "limit":␣
↪10, "begin": "2021-06-01T09:48:31.970Z", "end" :
↪"2021-06-07T09:48:31.970Z", "origin" : "timeline"}' -b cookies.jar
```

```
{"hostname": "10.10.10.10", "cluster_id": 123, "operation": "getTopQueries",
↪ "limit": 10, "begin": "2021-06-01T09:48:31.970Z", "end" :
↪"2021-06-07T09:48:31.970Z", "origin" : "timeline"}
```

getTopQueries

Sample request:

```
{
    "operation": "getTopQueries",
    "hostname": "dbhost1",
    "className": "CmnRequest",
    "requestCreated": "2021-06-07T09:41:43.636Z",
    "requestType": "RequestRead",
    "requestData":
    {
        "begin": "021-06-01T09:48:31.970Z",
        "className": "CmnReadParam",
        "end": "021-06-07T09:48:31.970Z",
        "limit": 10,
        "mimeType": "application/json",
        "objectName": "/topQueryMonitor",
        "origin": "timeline"
    }
}
```

Sample response:

```
{
    "className": "CmnReply",
    "requestCreated": "2021-06-07T09:51:44.255Z",
    "requestProcessed": "2021-06-07T09:51:44.243+00:00",
    "requestReceived": "2021-06-07T09:51:44.232+00:00",
```

(continues on next page)

(continued from previous page)


```
    "requestStatus": "OK",
    "result":
    {
        "className": "CmnReadResult",
        "readParam":
        {
            "begin": "2021-06-01T09:48:31.970Z",
            "className": "CmnReadParam",
            "end": "2021-06-07T09:48:31.970Z",
            "limit": 1,
            "mimeType": "application/json",
            "objectName": "/topQueryMonitor",
            "origin": "timeline"
        },
        "resultData":
        {
            "className": "CmnSqlResult",
            "created": "2021-06-07T09:51:44.233+00:00",
            "databaseName": ".storageConnection",
            "sqlServerName": "localhost",
            "columnNames":
            [
                "created",
                "server",
                "schema",
                "count",
                "sumTime",
                "sumIoTime",
                "shrdBlcks",
                "localBlcks",
                "tempBlcks"
            ],
            "records":
            [
                [
                    "2021-06-07T09:41:12.915Z",
                    "127.0.0.1",
                    "sbtest",
                    1,
                    773514,
                    0,
                    0,
                    0,
                    0
                ]
            ]
        }
    }
}
```

Returns historical database size statistics for a cluster, which returns the growth trend of a database in order to assist with capacity planning.

getDbGrowth

Returns historical database size statistics for a cluster, allowing you to track data growth over time. Results include per-host datadir size samples with timestamps and an optional per-database breakdown. Filter the time window using `startdate` and `enddate`, or request a specific day with `dayofyear` and `year`.

Parameter	Description
request	- An RPC request, which contains the following parameters: cluster_id, startdate, enddate, dayofyear, year, include_tables and db.

```
{"operation":"getDbGrowth", "cluster_id": 1}
```

Parameter	Description
reply	- A reply, which contains a dictionary with the list of data and a total value


```
{
    "controller_id": "a19271f7-3dd7-4f6b-89a4-7b84b6fb0bf2",
    "request_processed": "2021-09-27T18:44:32.327Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 1,
    "data":
    [
        {
            "class_name": "CmonDbStats",
            "created": "Sep 18 15:13:36",
            "database_count": 0,
            "datadir": "/var/lib/mysql/",
            "free_datadir_size": 406810349568,
            "hostname": "10.63.228.75",
            "port": 3306,
            "total_datadir_size": 458595631104,
            "year": 2021,
            "yearday": 261,
            "dbs": []
        }
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 'admin'."
    ]
}
```

The caller could specify/select here a Prometheus monitor instance to be used for processing the query, if not specified the code will takes the first Online CmonPrometheusHost instance to process the query.

prometheus

The following URL path (`/v2/stat/prometheus`) will be proxied to the prometheus daemon, you have to pass additional URI paths like `/query`.

Mandatory argument: `cluster_id` or `cluster_name` to identify the cluster.

Note

Note Only the following arguments will be sent to Prometheus: “query”, “time”, “timeout”, “start”, “end”, “step”, “match[]”

```
{
"operation": "/prometheus",
"queries":[
   {
      "query":
↪"mysql_global_status_wsrep_cert_deps_distance{instance=~\"10.117.12.165(:[0-9]+)?\"}",
↪
      "step":10
   }
],
"start":1622105617,
"end":1622108317,
"step":10,
"returnfrom":0,
"cluster_id":1
}
```

```
{
 "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
 "request_processed": "2021-05-27T09:51:45.311Z",
 "request_status": "Ok",
 "request_user_id": 6,
 "total": 1,
 "data":
 [
     {"status":"success","data":{"resultType":"vector","result":[{"metric":{
↪"__name__":"mysql_global_status_wsrep_cert_deps_distance","instance":
↪"10.117.12.165:9104","job":"mysqld","monitors":"10.117.12.165:3306"},
↪"value":[1622109105.311,"0"]}]}}
 ],
 "debug_messages":
 [
     "RPC V2 authenticated user is 'kedz'."
 ]
}
```

qm_topqueries

Returns the top queries captured by the Query Monitor for a cluster, ranked by total execution time. Supports filtering by `username` and `db_name`, and pagination via `limit` and `offset`. Pass `-1` for `limit` or `offset` to return all results without pagination.

Example request:

```
{
  "operation": "qm_topqueries",
  "cluster_id": 107,
  "limit": -1,
  "offset": -1,
  "username": "root",
  "db_name": "test"
}
```

Example response:

```
{
   "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
   "is_superuser": true,
   "request_processed": "2022-12-05T14:48:48.548Z",
   "request_status": "Ok",
   "total": 2,
   "available_databases": ["aaa", "bbb"],
   "available_users": ["root", "admin"],
   "controllers": {},
   "data": {
       "total_query_exec_time_us": 3587694,
       "queries": [
           {
               "class_name": "CmonQueryMonRecord",
               "avg_query_time": 620268,
               "canonical": "COMMIT",
               "command": "",
               "count": 3,
               "db": "dcps",
               "host": "localhost",
               "host_id": 1101,
               "hostid": 1101,
               "info": "COMMIT",
               "last_seen": 1634133907,
               "lock_time": 0,
               "max_query_time": 685060,
               "min_query_time": 0,
               "query_id": 10201897797591768868,
               "query_time": 685060,
               "rows_examined": 0,
               "rows_sent": 0,
               "state": "",
               "stddev": 48187,
```

(continues on next page)

(continued from previous page)


```
                "sum_created_tmp_disk_tables": 0,
                "sum_created_tmp_tables": 0,
                "sum_lock_time": 0,
                "sum_no_good_index_used": 0,
                "sum_no_index_used": 0,
                "sum_query_time": 1860804,
                "sum_rows_examined": 0,
                "sum_rows_sent": 0,
                "user": "cmon[cmon]",
                "variance": 2322006804
            },
            {
                "class_name": "CmonQueryMonRecord",
                "avg_query_time": 575630,
                "canonical": "UPDATE `dcps`.`cake_sessions` SET `id` = ?,␣
↪`data` = ?, `expires` = ? ",
                "command": "",
                "count": 3,
                "db": "dcps",
                "host": "localhost",
                "host_id": 1101,
                "hostid": 1101,
                "info": "UPDATE `dcps`.`cake_sessions` SET `id` =
↪'mqivh99fpnav209eomfouo4mi1', `data`",
                "last_seen": 1634133910,
                "lock_time": 160,
                "max_query_time": 602311,
                "min_query_time": 0,
                "query_id": 9449288033917105119,
                "query_time": 602311,
                "rows_examined": 1,
                "rows_sent": 0,
                "state": "",
                "stddev": 10766,
                "sum_created_tmp_disk_tables": 0,
                "sum_created_tmp_tables": 0,
                "sum_lock_time": 160,
                "sum_no_good_index_used": 0,
                "sum_no_index_used": 0,
                "sum_query_time": 1726890,
                "sum_rows_examined": 2,
                "sum_rows_sent": 0,
                "user": "cmon[cmon]",
                "variance": 115915538
            }
        ]
    }
}
```

qm_queryoutliers

Returns individual query executions that exceeded the outlier threshold configured in the Query Monitor. Filter results by time range using `starttime` and `endttime` (Unix epoch timestamps), and paginate using `limit` and `offset`.

Example request:

```
{
  "operation": "qm_queryoutliers",
  "cluster_id": 107,
  "starttime": 1670000000,
  "endttime": 1670252422,
  "limit": -1,
  "offset": -1
}
```

Example reply:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2022-12-05T15:00:58.305Z",
  "request_status": "Ok",
  "total": 0,
  "controllers": { },
  "data":
  {
      "queries": []
  }
}

### qm_purge

Purges all slow-query data collected by the Query Monitor for a cluster. This clears the accumulated query history. Use with caution — the data cannot be recovered after purging.

Example request:

```json
{
  "operation": "qm_purge",
  "cluster_id": 107,
}
```

Example response:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2022-12-05T14:46:24.323Z",
```

(continues on next page)
```

(continued from previous page)

```
  "request_status": "Ok"
}

### qm_killprocess

Terminates a specific database backend process identified by its `pid` on␣
↪the node given by `host_port`.

Example request:

```json
{
  "operation": "qm_killprocess",
  "cluster_id": 107,
  "pid": 916805,
  "host_port": [ "10.228.47.220:5432" ]
}
```

Example response:

```
{
  "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
  "is_superuser": true,
  "request_processed": "2022-12-05T14:46:24.323Z",
  "request_status": "Ok"
}
```

qm_processes

Returns the list of currently active database backend processes for a cluster as seen by the Query Monitor. Supports pagination via `limit` and `offset`.

Example request:

```
{"operation":"qm_processes","cluster_id": 107,"limit":1,"offset":0}
```

Example response:

```
{
     "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
     "is_superuser": true,
     "request_processed": "2022-12-05T14:37:36.259Z",
     "request_status": "Ok",
     "total": 6,
     "controllers": { },
     "data":
     [
         {
             "class_name": "CmonPostgreSqlDbProcess",
             "appName": "",
```

(continues on next page)
```

(continued from previous page)


```
            "backendPid": 440,
            "backendStart": "2022-11-17 15:22:04.460071+01",
            "client": "localhost:0",
            "databaseName": "",
            "elapsedTime": "",
            "hostId": 7006,
            "hostname": "10.228.47.220",
            "node": "10.228.47.220:5432",
            "port": 5432,
            "query": "",
            "queryStart": "",
            "reportTs": 1670251052,
            "state": "",
            "userName": "postgres",
            "waiting": "LogicalLauncherMain",
            "xactStart": ""
        }
    ]
}
```


gettxdeadlock

Returns recent transaction deadlock events detected across the cluster nodes. Each entry describes the SQL statement involved, the host and instance where the deadlock occurred, and the IDs of the conflicting transactions.

Example request:


```
{"operation":"gettxdeadlock","cluster_id": 107,"limit":1,"offset":0}
```


Example response:


```
{
    "controller_id": "7f399ff3-7484-4a93-a47a-a5c0bba42fca",
     "is_superuser": true,
     "request_processed": "2022-12-05T14:37:36.259Z",
     "request_status": "Ok",
     "total": 6,
     "controllers": { },
     "data": [
         {
             "blocked_by_trx_id": "",
             "db": "classicmodels",
             "duration": "45",
             "host": "localhost",
             "hostId": 2,
             "info": "DELETE FROM t WHERE i = 1",
             "innodb_trx_id": "",
             "instance": "10.0.8.12:3306",
             "internal_trx_id": "",
```


(continues on next page)

(continued from previous page)

```
            "lastseen": 1696620029,
            "message": "DELETE FROM t WHERE i = 1",
            "mysql_trx_id": "23",
            "port": 3306,
            "sql": "DELETE FROM t WHERE i = 1",
            "state": "updating",
            "user": "root"
        }
    ]
}
```

nodevariables

Returns the current runtime variables for each node in a cluster (equivalent to `SHOW VARIABLES` for MySQL/MariaDB nodes). Nodes that have not yet reported their variables appear in the list with a `null report_ts`.

Example request:

```
{"operation": "nodevariables", "cluster_id": 399}
```

Example response:

```
{
    "controller_id": "42e4370a-cae2-4b51-aa29-63d7fbfeacb2",
    "request_processed": "2023-10-16T15:56:58.857Z",
    "request_status": "Ok",
    "request_user_id": 5,
    "total": 14,
    "data":
    [
        {
            "hostId": 2611,
            "hostname": "10.238.5.169",
            "port": 27017,
            "report_ts": null,
            "unique_id": 1251
        },
        {
            "hostId": 2612,
            "hostname": "10.238.5.25",
            "port": 27017,
            "report_ts": null,
            "unique_id": 1252
        },
        {
            "hostId": 2613,
            "hostname": "10.238.5.5",
            "port": 27017,
            "report_ts": null,
```

(continues on next page)

(continued from previous page)


```
            "unique_id": 1253
        },
        {
            "hostId": 2675,
            "hostname": "10.238.5.17",
            "port": 3306,
            "report_ts": 1697471809,
            "unique_id": 1275,
            "variables":
            {
                "allow_suspicious_udfs": "OFF",
                "alter_algorithm": "DEFAULT",
                "analyze_sample_percentage": "100.000000",
                ".......": ".........",
                ".......": ".........",
                "wsrep_trx_fragment_size": "0",
                "wsrep_trx_fragment_unit": "bytes"
            }
        },
        {
            "hostId": 2676,
            "hostname": "10.238.5.112",
            "port": 3306,
            "report_ts": 1697471809,
            "unique_id": 1276,
            "variables":
            {
                "allow_suspicious_udfs": "OFF",
                "alter_algorithm": "DEFAULT",
                "analyze_sample_percentage": "100.000000",
                ".......": ".........",
                ".......": ".........",
                "wsrep_trx_fragment_size": "0",
                "wsrep_trx_fragment_unit": "bytes"
            }
        },
        {
            "hostId": 2677,
            "hostname": "10.238.5.23",
            "port": 3306,
            "report_ts": 1697471809,
            "unique_id": 1277,
            "variables":
            {
                "allow_suspicious_udfs": "OFF",
                "alter_algorithm": "DEFAULT",
                "analyze_sample_percentage": "100.000000",
                ".......": ".........",
                ".......": ".........",
```


(continues on next page)

(continued from previous page)

```
                "wsrep_trx_fragment_size": "0",
                "wsrep_trx_fragment_unit": "bytes"
            }
        }
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 's9s'."
    ]
}
```

getdbstatus

Returns the current runtime status variables for each node in a cluster (equivalent to `SHOW STATUS` for MySQL/MariaDB nodes). Useful for monitoring engine-level counters, replication state, and cache hit rates.

Example request:

```
{"operation": "getdbstatus", "cluster_id": 399}
```

Example response:

```
{
    "controller_id": "42e4370a-cae2-4b51-aa29-63d7fbfeacb2",
    "request_processed": "2023-10-16T15:57:20.493Z",
    "request_status": "Ok",
    "request_user_id": 5,
    "total": 3,
    "data":
    [
        {
            "hostId": 2675,
            "hostname": "10.238.5.17",
            "statuses":
            {
                "class_name": "CmonSqlVariables",
                "ACL_DATABASE_GRANTS": "2",
                ".......": ".........",
                ".......": ".........",
                "WSREP_LOCAL_INDEX": "18446744073709551615",
                "report_ts": 1697471840
            }
        },
        {
            "hostId": 2676,
            "hostname": "10.238.5.112",
            "statuses":
            {
                "class_name": "CmonSqlVariables",
```

(continues on next page)

(continued from previous page)


```
                "ACL_DATABASE_GRANTS": "2",
                ".......": ".........",
                ".......": ".........",
                "WSREP_LOCAL_INDEX": "18446744073709551615",
                "report_ts": 1697471840
            }
        },
        {
            "hostId": 2677,
            "hostname": "10.238.5.23",
            "statuses":
            {
                "class_name": "CmonSqlVariables",
                "ACL_DATABASE_GRANTS": "2",
                ".......": ".........",
                ".......": ".........",
                "WSREP_LOCAL_INDEX": "18446744073709551615",
                "report_ts": 1697471840
            }
        }
    ],
    "debug_messages":
    [
        "RPC V2 authenticated user is 's9s'."
    ]
}
```


3.12.22 CMON Directory Tree

The `/v2/tree` path is for managing objects in the CMON Directory Tree.

getTree

This call can be used to get the CDT (CMON Directory Tree).


```
{
    "operation": "getTree",
    "path": "/galera_001",
    "filters": [ "Cluster", "User", "/some/path" ],
    "request_created": "2017-10-03T07:12:13.682Z",
    "request_id": 3
}
```


- `path`: The path of the sub-tree to get. This is not mandatory, if it is not provided the entire tree will be returned (or at least the part that the user allowed to see).


```
{
    "cdt":
    {
        "item_acl": "user::rwx,group::rw-,other::---",
```


(continues on next page)

(continued from previous page)


```
        "item_name": "galera_001",
        "item_path": "/",
        "item_type": "Cluster",
        "object_id": 1,
        "owner_group_id": 2,
        "owner_group_name": "users",
        "owner_user_id": 3,
        "owner_user_name": "pipas",
        "sub_items": [
        {
            "item_acl": "user::rwx,group::rw-,other::---",
            "item_name": "192.168.1.212:3306",
            "item_path": "/galera_001",
            "item_type": "Node",
            "object_id": 2,
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "owner_user_id": 1,
            "owner_user_name": "system"
        },
        {
            "item_acl": "user::rwx,group::rw-,other::---",
            "item_name": "192.168.1.127:9555",
            "item_path": "/galera_001",
            "item_spec": "controller",
            "item_type": "Node",
            "object_id": 3,
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "owner_user_id": 1,
            "owner_user_name": "system"
        } ]
    },
    "request_created": "2017-10-03T07:12:13.682Z",
    "request_id": 3,
    "request_processed": "2017-10-03T07:12:13.750Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `cdt`: The CMON Directory Tree organized into a tree of objects. This is either the root node of the tree or the node representing the path that was requested.

move

This call can be used to move an object into a new location in the tree.


```
{
    "operation": "move",
    "request_created": "2017-10-03T07:51:53.285Z",
```


(continues on next page)

(continued from previous page)

```
    "request_id": 3,
    "source_path": "/storage01",
    "target_path": "/servers"
}
```

- `source_path`: The full path of the object to move.
- `target_path`: The new location where the object will be moved.

The reply of this request is quite simple, it just states the request was processed:

```
{
    "request_created": "2017-10-03T07:51:53.285Z",
    "request_id": 3,
    "request_processed": "2017-10-03T07:51:53.400Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

rename

Changes the name of a CDT entry. Here is how a request should look like.

```
{
    "operation": "rename",
    "request_created": "2019-01-08T09:18:04.315Z",
    "request_id": 62,
    "source_path": "/groups/mygroup",
    "target_name": "newgroup"
}
```

So for example if one wants to change the name of a user group one needs to get the group. Then the "cdt_path" and the "group_name" property of the group has to be accessed to build the path of the group object and the "rename" request has to be sent.

Here is a reply given for the "rename" request if the operation was successful:

```
{
    "request_created": "2019-01-08T09:18:04.315Z",
    "request_id": 62,
    "request_processed": "2019-01-08T09:18:04.343Z",
    "request_status": "Ok",
    "request_user_id": 1
}
```

getAcl

This call can be used to get the ACL of an object from the CMON Directory Tree.

```
{
    "operation": "getAcl",
```

(continues on next page)

(continued from previous page)


```
    "path": "/core1/containers/bestw_controller",
    "request_created": "2017-10-03T06:39:59.289Z",
    "request_id": 3
}
```


- `path`: The full path of the object.


```
{
    "acl": "user::rwx,group::rw-,other::---",
    "object_name": "bestw_controller",
    "object_path": "/core1/containers",
    "owner_group_name": "users",
    "owner_user_name": "pipas",
    "request_created": "2017-10-03T06:39:59.289Z",
    "request_id": 3,
    "request_processed": "2017-10-03T06:40:02.358Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `acl`: The ACL for the requested object in string format.
- `object_name`: The name of the object.
- `object_path`: The path in the CDT where the object can be found.
- `owner_group_name`: The group name of the group owner for the object.
- `owner_user_name`: The user name of the owner of the group.

cat

This call reads the content of a CDT file.


```
{
    "operation": "cat",
    "path": "/.issue",
    "request_created": "2017-12-18T13:22:45.664Z",
    "request_id": 3
}
```



```
{
    "file_content": "Severalnines Clustercontrol Server.\n",
    "request_created": "2017-12-18T13:22:45.664Z",
    "request_id": 3,
    "request_processed": "2017-12-18T13:22:45.681Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

getObject

This call can be used to get the object using its full path as a reference.

This is how the request should be sent.

```
{
    "operation": "getObject",
    "path": "/.runtime/cluster_manager",
    "request_created": "2018-11-29T09:09:22.836Z",
    "request_id": 2
}
```

Here is how a reply looks like.

```
{
    "object":
    {
        "acl": "user::r--,group::---,other::---",
        "cdt_path": "/.runtime",
        "class_name": "CmonCdtEntry",
        "content": "        cluster_manager_instance : 0x7fad58001380\n\n",
        "dirty": false,
        "major_device_number": 10,
        "minor_device_number": 1,
        "name": "cluster_manager",
        "owner_group_id": 1,
        "owner_group_name": "admins",
        "owner_user_id": 1,
        "owner_user_name": "system",
        "type": "CmonCdtFile"
    },
    "reply_received": "2018-11-29T09:09:22.841Z",
    "request_created": "2018-11-29T09:09:22.836Z",
    "request_id": 2,
    "request_processed": "2018-11-29T09:09:22.864Z",
    "request_status": "Ok",
    "request_user_id": 1
}
```

addAcl

This call can be used to add an ACL entry to an object. Please note that if the given entry is already added to the ACL it will be overwritten. This is the intuitive thing to do.

```
{
    "acl": "user:nobody:r--",
    "operation": "addAcl",
    "path": "/core1/containers/bestw_controller",
    "request_created": "2017-10-03T06:43:10.458Z",
    "request_id": 3
}
```

- `acl`: The ACL to add. This should be the text format of one ACL entry.
- `path`: The full path of the object to change.


```
{
    "acl": "user::rwx,user:nobody:r--,group::rw-,other::---",
    "object_name": "bestw_controller",
    "object_path": "/core1/containers",
    "owner_group_name": "users",
    "owner_user_name": "pipas",
    "request_created": "2017-10-03T06:46:42.233Z",
    "request_id": 3,
    "request_processed": "2017-10-03T06:46:42.322Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


- `acl`: The ACL of the object after the change.
- `object_name`: The name of the object.
- `object_path`: The path in the CDT where the object can be found.
- `owner_group_name`: The group name of the group owner for the object.
- `owner_user_name`: The user name of the owner of the group.

removeAcl

This call can be used to remove an ACL entry from the ACL of an object. Currently only one single ACL entry can be removed at a a time.

Please note that certain ACL entries are mandatory and can not be removed only changed. If the call tries to remove such a mandatory entry an error will be reported.


```
{
    "acl": "user:pipas:rwx",
    "operation": "removeAcl",
    "path": "/",
    "request_created": "2017-10-04T15:07:50.427Z",
    "request_id": 3
}
```


Here is a reply notifying the caller about the successful removal of the entry.


```
{
    "acl": "user::rwx,group::rwx,other::rwx",
    "object_name": "",
    "object_path": "/",
    "owner_group_name": "admins",
    "owner_user_name": "system",
    "request_created": "2017-10-04T15:07:50.427Z",
    "request_id": 3,
    "request_processed": "2017-10-04T15:07:50.504Z",
```


(continues on next page)

(continued from previous page)

```
    "request_status": "Ok",
    "request_user_id": 3
}
```

checkAccess

This RPC call can be used to check if the authenticated user has access to the given object. Here is the request:

```
{
    "operation": "checkAccess",
    "path": "/pipas",
    "privileges": "rw-",
    "request_created": "2017-12-06T12:08:53.704Z",
    "request_id": 3
}
```

The reply will simply be an "Ok" reply if the user has the access or an "AccessDenied" if the the user is not provided with the given access right(s). Other errors also might be possible, for example the request might be mallformad of the object might be missing. Here is the reply showing the user has the given rights:

```
{
    "request_created": "2017-12-06T12:08:53.704Z",
    "request_id": 3,
    "request_processed": "2017-12-06T12:08:53.717Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

chown

Changes the ownership of object(s).

```
{
    "operation": "chown",
    "owner_group_name": "users",
    "owner_user_name": "pipas",
    "path": "/core2/containers",
    "recursive": true,
    "request_created": "2017-10-05T07:51:05.133Z",
    "request_id": 3
}
```

- `owner_group_name`: The group name of the new group owner. One of the owner_user_name and owner_group_name is mandatory.
- `owner_user_name`: The username of the new owner. One of the owner_user_name and owner_group_name is mandatory.
- `path`: The full path of the object to change.

- `recursive`: If this propertyis true all the subitems of the tree will be changed too. If any of the changes fail with access denied no change will be done on any of the tree items.

The reply to a successful chown request looks like this:


```
{
    "acl": "user::rwx,group::rwx,other::rwx",
    "object_name": "containers",
    "object_path": "/core2/",
    "owner_group_name": "users",
    "owner_user_name": "pipas",
    "request_created": "2017-10-05T07:51:05.133Z",
    "request_id": 3,
    "request_processed": "2017-10-05T07:51:05.208Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


mkdir

This call is for creating folders in the CMON Directory Tree.


```
{
    "operation": "mkdir",
    "path": "/foldername",
    "request_created": "2017-10-09T07:42:17.091Z",
    "request_id": 3
}
```


- `path`: The full path of the folder to create.


```
{
    "request_created": "2017-10-09T07:42:17.091Z",
    "request_id": 3,
    "request_processed": "2017-10-09T07:42:17.143Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


rmdir

This call is for removing empty folders. This call will be returning with an error if the object at the given path is not an empty folder. To remove objects other than a folder please consider using the `delete` call. The `delete` call is also able to remove empty folders.


```
{
    "operation": "rmdir",
    "path": "/foldername",
    "request_created": "2017-10-09T07:13:54.410Z",
    "request_id": 3
}
```

- `path`: The full path of the folder to remove.


```
{
    "request_created": "2017-10-09T07:13:54.410Z",
    "request_id": 3,
    "request_processed": "2017-10-09T07:13:54.470Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```


mkfile

This call can be used to create a file type CDT entry. Here is a call:


```
{
    "operation": "mkfile",
    "path": "/tmp/filename.txt",
    "request_created": "2019-01-04T09:05:27.349Z",
    "request_id": 17
}
```


Here is a reply showing a successful file operation:


```
{
    "request_created": "2019-01-04T09:05:27.349Z",
    "request_id": 17,
    "request_processed": "2019-01-04T09:05:27.364Z",
    "request_status": "Ok",
    "request_user_id": 1
}
```


setContent

This call can be used to change the content string of a CDT entry. Currently only the file type CDT entries can have contents handled in this call. Here is an example call:


```
{
    "content": "This is something.\n\n",
    "operation": "setcontent",
    "path": "/tmp/ak.txt",
    "request_created": "2019-01-04T09:03:34.657Z",
    "request_id": 13
}
```


Here is a reply reporting a successful operation:


```
{
    "request_created": "2019-01-04T09:03:34.657Z",
    "request_id": 13,
    "request_processed": "2019-01-04T09:03:34.670Z",
    "request_status": "Ok",
```


(continues on next page)

(continued from previous page)

```
    "request_user_id": 1
}
```

delete

This call is for deleting object from the tree. Once an object is deleted, it is deleted for good. If for example one performs a delete operation on a container it is deleted from the server. It is gone, it is nonexisting, it is a late container.

```
{
    "operation": "delete",
    "path": "/host04/containers/temporary",
    "request_created": "2017-10-16T13:58:53.186Z",
    "request_id": 3
}
```

- `path`: The full path of the object to delete.

```
{
    "request_created": "2017-10-16T13:58:53.186Z",
    "request_id": 3,
    "request_processed": "2017-10-16T13:58:56.179Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

3.12.23 User Database manipulation

The `/v2/users` path is to manipulate Cmon Users, users that are maintained by the Cmon controller. These users are used while setting up encrypted communication for the RPC v2 interface, so without this interface it is not possible to use the RPC v2 itself.

createUser

The createUser RPC call can be used to create a new Cmon User.

In the call both the user's password and public kay can be specified so that the user can log in after the account is created.

```
{
    "create_group": true,
    "new_password": "p",
    "operation": "createUser",
    "user":
    {
        "class_name": "CmonUser",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_name": "testgroup"
```

(continues on next page)

(continued from previous page)

```
        } ],
        "public_keys": [
        {
            "key": "-----BEGIN RSA PUBLIC␣
↪KEY-----\nMIIBCgKCAQEAxupGXCUCtoytVYBD1NKeAB+oWauxJzGYvunMGnVIqhjF/o8gokNf\nV0mcad022SJ
↪RSA PUBLIC KEY-----\n",
            "name": "No Name"
        } ],
        "user_name": "pipas"
    }
}
```

Here are the most important elements of the request:

- `create_group`: If the groups of the user does not exist the controller will try to create the group on the fly. If the group could be created the user will be added to the group, if not the request will be failed.
- `new_password`: The plain text password that the user can actually use to log in. This is not mandatory, if the new password is not provided no password will be set and the user won't be able to log in with a password until a valid password is set Please note that the `new_password` is not a property of the cmonuser. The plain password will not be stored, only the encoded version of the password is a property of the CmonUser object, but even that is a hidden property, not seen from outside of the controller code.
- `groups`: The user groups the user belongs to. This is not a mandatory argument, if the `create_group` is set to true, but no group is provided the controller will try to create a group that has the same name as the user and add the new user to this new group. It is important to note that every user must belong to at least one group the so called "primary group". If the user belongs to multiple groups the first group is the primary group. Theoretically this call can accept multiple groups, but this feature is not yet implemented.
- `public_keys`: Zero or more keys the user can use to log in. This is not a mandatory argument, if no key is provided the user will not able to log in with a key, but might still be able to use the system with a password. The `public_keys` is actually a property of the cmonuser, but it is a hidden property, not readable only using special calls like the getKeys call.
- `key`: The actual public key string.
- `name`: The name of the key. The name is just a human readable string the user provided as a reminder.

The reply to a createUser request looks like this:

```
{
    "request_created": "2017-07-04T12:49:50.896Z",
    "request_id": 2,
    "request_processed": "2017-07-04T12:49:50.954Z",
    "request_status": "Ok",
    "request_user_id": 1,
    "user":
    {
        "class_name": "CmonUser",
```

(continues on next page)

(continued from previous page)

```
        "email_address": "rpc@email.com",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 6,
            "group_name": "testgroup"
        } ],
        "user_id": 11,
        "user_name": "rpc_user"
    }
}
```

- `request_user_id`: This is just the user ID of the user who sent the request, the same as in any other replies.
- `user`: All the details of the user that has been created.
- `user_id`: The user ID of the user that just has been created. The only inmutable property of the user that can not be ever changed.

Please note that there is no password or public key in the reply. We handle those as a secret.

Here is an example showing a stored user with the internal properties that are not visible from the outside (but might be set like the "password_encrypted"), for example:

```
{
    "class_name": "CmonUser",
    "email_address": "laszlo@severalnines.com",
    "first_name": "Laszlo",
    "groups": [
    {
        "class_name": "CmonGroup",
        "group_id": 4,
        "group_name": "testgroup"
    } ],
    "last_login": "2017-09-06T08:45:55.588Z",
    "last_name": "Pere",
    "password_encrypted":
↪"3babe525f514e13a421b1e2870824d8d3e08531c5db461b088aa8c4298a1eca8",
    "password_format": "sha256",
    "password_salt": "7cc254f8-1be8-e78d-765a-2e63339fc99a",
    "public_keys": [
    {
        "key": "-----BEGIN RSA PUBLIC␣
↪KEY-----\nMIIBCgKCAQEAxupGXCUCtoytVYBD1NKeAB+oWauxJzGYvunMGnVIqhjF/o8gokNf\nV0mcad022SJ
↪RSA PUBLIC KEY-----\n",
        "name": "No Name"
    } ],
    "user_id": 3,
    "user_name": "pipas"
}
```

It is also important to know what the backend checks to see if the authenticated user actually has the proper privileges to create the new user in the given situation.

1. The controller checks if the `/.runtime/user_manager` entry exists and the authenticated user has execute access on it. If not the request will be rejected.
2. The backend checks if a user with the same name already exists. If so the creation of the user with the same name will be denied.
3. The controller checks if the passed new user has a group set. If so and the group does not exist the controller checks if the request asks for a new group to be created. If so the controller tries to create a new group.
4. Then the controller checks if the authenticated user has write access on the group. If the group was just created the authenticated user will be able to write it, but if it is pre-existing the authenticated user might not have write access to it. If this happens the request will be rejected.
5. Then the controller checks if there is already an object (not a cmon user) at the same path and name. If so the request will be rejected.
6. Then the new user is saved and an OK reply is sent.

whoAmI

This call returns information about the authenticated user. The client application can send this call periodically to update the information about the user and the privileges the user has.

The request shall look like this:

```
{
    "operation": "whoAmI",
    "request_created": "2018-12-07T08:51:14.324Z",
    "request_id": 3,
    "with_extended_privileges": true
}
```

Here the elements of the request are:

- `with_extended_privileges`: If the request has this field set to true the reply will contain some extra information about the user's rigts.

The reply is as follows:

```
{
    "request_created": "2018-12-07T08:51:14.324Z",
    "request_id": 3,
    "request_processed": "2018-12-07T08:51:14.332Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "user":
    {
        "acl": "user::rwx,group::r--,other::r--",
        "cdt_path": "/",
        "class_name": "CmonUser",
        "created": "2018-12-07T07:22:49.165Z",
        "disabled": false,
```

(continues on next page)

(continued from previous page)


```
        "email_address": "laszlo@severalnines.com",
        "first_name": "Laszlo",
        "groups": [
        {
            "acl": "user::rwx,group::rw-,other::---",
            "cdt_path": "/groups",
            "class_name": "CmonGroup",
            "created": "2018-12-07T07:22:49.153Z",
            "group_id": 4,
            "group_name": "testgroup",
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "owner_user_id": 1,
            "owner_user_name": "system"
        } ],
        "last_failed_login": "",
        "last_login": "2018-12-07T08:51:14.317Z",
        "last_name": "Pere",
        "n_failed_logins": 0,
        "owner_group_id": 1,
        "owner_group_name": "admins",
        "owner_user_id": 4,
        "owner_user_name": "pipas",
        "suspended": false,
        "timezone":
        {
            "class_name": "CmonTimeZone",
            "name": "CET",
            "offset": -3600,
            "has_dst_rules": true
        },
        "user_id": 4,
        "user_name": "pipas"
    },
    "user_extended_privileges":
    {
        "can_create_clusters": true,
        "can_execute_jobs": true,
        "class_name": "CmonExtendedPrivileges"
    }
}
```


The reply shows the user object and some extended privileges of that user. These privileges are not the property of the user but are controlled by the access rights of other CDT entries on the controller. So these extended privileges is a convenience check the return information about the effective privileges of several CDT entries.

The list of the extended privileges are under construction, the caller may expect more than shown in this example.

getUsers

A call to read the user database. In its basic form the RPC call looks like this:

```
{
    "operation": "getUsers",
    "request_created": "2020-05-13T06:16:11.197Z",
    "request_id": 3,
    "with_tags": [ "testUser" ]
}
```

- `with_tags`: The request may contain this field to limit the list of the returned users to those that have at least one of the tags passed here.
- `without_tags`: The request may contain this field to limit the list of the returned users to those that has none of the tags passed here.

The reply contains the users managed by the controller:

```
{
    "request_created": "2020-05-13T06:16:11.197Z",
    "request_id": 3,
    "request_processed": "2020-05-13T06:16:11.216Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "total": 16,
    "debug_messages":
    [
        "RPC V2 authenticated user is 'pipas'."
    ],
    "users":
    [
        {
            "class_name": "CmonUser",
            "cdt_path": "/",
            "owner_user_id": 4,
            "owner_user_name": "pipas",
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "acl": "user::rwx,group::r--,other::r--",
            "tags":
            [
                "testUser"
            ],
            "created": "2020-05-13T06:08:24.564Z",
            "disabled": false,
            "email_address": "laszlo@severalnines.com",
            "first_name": "Laszlo",
            "last_failed_login": "",
            "last_login": "2020-05-13T06:16:11.186Z",
            "last_name": "Pere",
            "n_failed_logins": 0,
```

(continues on next page)

(continued from previous page)


```
            "origin": "CmonDb",
            "suspended": false,
            "user_id": 4,
            "user_name": "pipas",
            "groups":
            [
                {
                    "class_name": "CmonGroup",
                    "cdt_path": "/groups",
                    "owner_user_id": 1,
                    "owner_user_name": "system",
                    "owner_group_id": 1,
                    "owner_group_name": "admins",
                    "acl": "user::rwx,group::rwx,other::---",
                    "created": "2020-05-13T06:08:13.521Z",
                    "group_id": 1,
                    "group_name": "admins"
                }
            ],
            "timezone":
            {
                "class_name": "CmonTimeZone",
                "name": "Central European Time",
                "abbreviation": "CET",
                "offset": -3600,
                "has_dst_rules": true
            }
        }
    ]
}
```


- `total`: The total number of users managed by the controller.
- `users`: A list of CmonUser class objects. To get more information about the user objects please check the cmonuser and cmongroup sections.

setUser

The setUser call can be used to modify certain properties of existing Cmon users. Here us an example showing how to modify the email address of a user:


```
{
    "operation": "setUser",
    "request_created": "2017-06-26T07:25:16.198Z",
    "request_id": 3,
    "user":
    {
        "class_name": "CmonUser",
        "email_address": "system@mydomain.com",
        "user_name": "system"
```


(continues on next page)

(continued from previous page)

```
    }
}
```

- `user`: Identifies the user and the properties to change. The "class_name" of this map should be "CmonUser" and the "user_name" should identify an existing user. Any other fields are properties the caller wants to change. Please check out the cmonuser section to see what properties can be changed.

The reply to this request will contain the request status and, if an error occurred, the error string. If the request was successfully processed the controller will send back the user object with all of its properties after the change:

```
{
    "request_created": "2017-06-26T07:37:51.522Z",
    "request_id": 3,
    "request_processed": "2017-06-26T07:37:51.575Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "user":
    {
        "class_name": "CmonUser",
        "email_address": "system@mydomain.com",
        "first_name": "System",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 1,
            "group_name": "admins"
        } ],
        "last_name": "User",
        "user_id": 1,
        "user_name": "system"
    }
}
```

- `user`: Shows the properties of the user after the change.

disable

This call will set the "disabled" flag for the user and so prevent the user from logging in.

The request goes like this:

```
{
    "operation": "disable",
    "request_created": "2017-10-30T08:25:15.685Z",
    "request_id": 3,
    "user":
    {
        "class_name": "CmonUser",
        "user_name": "nobody"
```

(continues on next page)

(continued from previous page)

```
    }
}
```

The reply contains the user object the "disabled" flag set to true.


```
{
    "request_created": "2017-10-30T08:25:15.685Z",
    "request_id": 3,
    "request_processed": "2017-10-30T08:25:16.535Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "user":
    {
        "cdt_path": "/",
        "class_name": "CmonUser",
        "created": "2017-10-27T12:34:47.333Z",
        "disabled": true,
        "first_name": "Default",
        "groups": [
        {
            "cdt_path": "/groups",
            "class_name": "CmonGroup",
            "group_id": 3,
            "group_name": "nobody"
        } ],
        "last_failed_login": "",
        "last_name": "User",
        "n_failed_logins": 0,
        "owner_group_id": 1,
        "owner_group_name": "admins",
        "owner_user_id": 2,
        "owner_user_name": "nobody",
        "suspended": false,
        "user_id": 2,
        "user_name": "nobody"
    }
}
```


deleteUser

Permanently deletes an existing CMON user. The user to delete is identified by `user_name` inside the `user` object. All associated SSH keys, group memberships, and preferences for that user are also removed.

```
{
    "operation": "deleteUser",
    "request_created": "2018-11-26T14:19:29.891Z",
    "request_id": 3,
    "user":
    {
        "class_name": "CmonUser",
```

(continues on next page)

(continued from previous page)

```
        "user_name": "laszlo"
    }
}
```

enable

This call will clear the "disabled" flag for the user so the user will be able to log in. The "suspended" flag will also be cleared, the failed login counter set to 0 and the date&time of the last failed login gets deleted, so users who are suspended for failed login attempts will also be able to log in.

The request looks like this:

```
{
    "operation": "enable",
    "request_created": "2017-10-30T08:28:29.852Z",
    "request_id": 3,
    "user":
    {
        "class_name": "CmonUser",
        "user_name": "nobody"
    }
}
```

And the reply:

```
{
    "request_created": "2017-10-30T08:28:29.852Z",
    "request_id": 3,
    "request_processed": "2017-10-30T08:28:29.912Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "user":
    {
        "cdt_path": "/",
        "class_name": "CmonUser",
        "created": "2017-10-27T12:34:47.333Z",
        "disabled": false,
        "first_name": "Default",
        "groups": [
        {
            "cdt_path": "/groups",
            "class_name": "CmonGroup",
            "group_id": 3,
            "group_name": "nobody"
        } ],
        "last_failed_login": "",
        "last_name": "User",
        "n_failed_logins": 0,
        "owner_group_id": 1,
        "owner_group_name": "admins",
```

(continues on next page)

(continued from previous page)

```
        "owner_user_id": 2,
        "owner_user_name": "nobody",
        "suspended": false,
        "user_id": 2,
        "user_name": "nobody"
    }
}
```

changePassword

This call is of course for changing the password of an existing user.

```
{
    "new_password": "newpass",
    "old_password": "secret",
    "operation": "changePassword",
    "request_created": "2017-06-29T07:56:17.710Z",
    "request_id": 2,
    "user":
    {
        "class_name": "CmonUser",
        "user_name": "system"
    }
}
```

- `new_password`: The new password in plain string format.
- `old_password`: The current password in plain format. The user's old password that will be checked using the usually password authentication algorithms. Please note that a superuser can change the passwords for any user without providing the old password, but if the old password is sent it will be checked even if the user that sent it is a superuser.
- `user`: Identifies the user that should have the new password.

```
{
    "request_created": "2017-06-29T07:56:17.710Z",
    "request_id": 2,
    "request_processed": "2017-06-29T07:56:17.761Z",
    "request_status": "Ok",
    "request_user_id": 1
}
```

getKeys

A call to return the public keys of a user. Here is an example showing a getKeys request:

```
{
    "operation": "getKeys",
    "request_created": "2017-07-05T09:23:04.988Z",
    "request_id": 2,
    "user":
```

(continues on next page)

(continued from previous page)

```
    {
        "class_name": "CmonUser",
        "user_name": "pipas"
    }
}
```

- `user`: This identifies the user whose public keys will be returned. Only the class name and the user name needed here. Please note that the controller handles the public keys securely, normal users can not read the public keys of other users. Users can always read their own keys of course.

Here is an example of the reply to the getKeys request:

```
{
    "public_keys": [
    {
        "key": "-----BEGIN RSA PUBLIC␣
↪KEY-----\nMIIBCgKCAQEAxupGXCUCtoytVYBD1NKeAB+oWauxJzGYvunMGnVIqhjF/o8gokNf\nV0mcad022SJ
↪RSA PUBLIC KEY-----\n",
        "name": "Unnamed key"
    } ],
    "request_created": "2017-07-05T09:23:04.988Z",
    "request_id": 2,
    "request_processed": "2017-07-05T09:23:05.034Z",
    "request_status": "Ok",
    "request_user_id": 1,
    "total": 1,
    "user":
    {
        "class_name": "CmonUser",
        "email_address": "laszlo@severalnines.com",
        "first_name": "Laszlo",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 4,
            "group_name": "testgroup"
        } ],
        "last_login": "2017-07-05T09:22:55.587Z",
        "last_name": "Pere",
        "title": "",
        "user_id": 3,
        "user_name": "pipas"
    }
}
```

- `public_keys`: Contains the public keys of the user.
  - `key`: The actual key string.
  - `name`: The human readable name of the key, a reminder that the user set when adding the key.
- `total`: Shows how many keys has been found.

- `user`: The user that has these keys. This field is sent back for convenience.

addKey

A call to add/register a new public key for a user.

```
{
    "operation": "addKey",
    "public_key":
    {
        "key": "-----BEGIN RSA PUBLIC␣
↪KEY-----\nMIIBCgKCAQEAr6KS5KsnwVuoOzFGJsF2oPhUokBEEdrcGfVPX5XbdlIn0E/Gn44i\nx4HBt0rDKNe
↪RSA PUBLIC KEY-----\n",
        "name": ""
    },
    "request_created": "2017-07-05T11:55:57.885Z",
    "request_id": 3,
    "user":
    {
        "class_name": "CmonUser",
        "user_name": "pipas"
    }
}
```

- `public_key`: The public key to add.
- `user`: The user to whom the key will be added. Only the class name and the user name has to be specified here.

The reply to this request will look like this:

```
{
    "request_created": "2017-07-05T11:55:57.885Z",
    "request_id": 3,
    "request_processed": "2017-07-05T11:55:57.939Z",
    "request_status": "Ok",
    "request_user_id": 3
}
```

deleteKey

This call can be used to delete a public key by its name. The call looks very similar to the "addKey" call:

```
{
    "operation": "deleteKey",
    "public_key":
    {
        "name": "The name"
    },
    "request_created": "2017-07-05T12:35:43.135Z",
    "request_id": 3,
    "user":
```

(continues on next page)

(continued from previous page)

```
    {
        "class_name": "CmonUser",
        "user_name": "pipas"
    }
}
```

createGroup

Creates a new CMON user group. Groups are used to organise users and manage access control collectively. The group is identified by `group_name` inside the `group` object.

```
{
    "operation": "createGroup",
    "group":
    {
        "class_name": "CmonGroup",
        "group_name": "GroupName"
    }
}
```

deleteGroup

This call is for deleting an existing Cmon Group. Groups can only be deleted if they are empty (no Cmon User is assigned to them) and the authenticateduser has a write access to the actual group itself.

Currently only the group name is used to identify the group that will be deleted.

```
{
    "group":
    {
        "class_name": "CmonGroup",
        "group_name": "pip"
    },
    "operation": "deleteGroup",
    "request_created": "2019-08-05T07:44:39.972Z",
    "request_id": 3
}
```

```
{
    "request_created": "2019-08-05T07:44:39.972Z",
    "request_id": 3,
    "request_processed": "2019-08-05T07:44:39.995Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```

getGroups

A call to read the user database. In its basic form the RPC call looks like this:

```
{
    "operation": "getGroups",
    "request_created": "2017-06-26T06:42:12.902Z",
    "request_id": 3
}
```

```
{
    "groups": [
    {
        "class_name": "CmonGroup",
        "group_id": 1,
        "group_name": "admins"
    },
    {
        "class_name": "CmonGroup",
        "group_id": 2,
        "group_name": "users"
    },

    . . .

    {
        "class_name": "CmonGroup",
        "group_id": 6,
        "group_name": "rpc_group"
    } ],
    "reply_received": "2017-07-24T08:53:33.498Z",
    "request_created": "2017-07-24T08:53:33.493Z",
    "request_id": 3,
    "request_processed": "2017-07-24T08:53:33.538Z",
    "request_status": "Ok",
    "request_user_id": 3,
    "total": 6
}
```

- `groups`: The list of user groups the controller has.
- `total`: The total number of groups the controller has.

addToGroup

Adds the user to a new group or moves the user from the current primary group to a new group.

```
{
    "group_name": "admins",
    "operation": "addToGroup",
    "replace_primary_group": true,
    "request_created": "2018-01-12T11:07:48.963Z",
```

(continues on next page)

(continued from previous page)

```
    "request_id": 2,
    "user":
    {
        "class_name": "CmonUser",
        "user_name": "pipas"
    }
}
```

```
{
    "request_created": "2018-01-12T11:07:48.963Z",
    "request_id": 2,
    "request_processed": "2018-01-12T11:07:48.984Z",
    "request_status": "Ok",
    "request_user_id": 1,
    "user":
    {
        "cdt_path": "/",
        "class_name": "CmonUser",
        "created": "2018-01-12T07:26:13.677Z",
        "disabled": false,
        "email_address": "laszlo@severalnines.com",
        "first_name": "Laszlo",
        "groups": [
        {
            "cdt_path": "/groups",
            "class_name": "CmonGroup",
            "group_id": 1,
            "group_name": "admins"
        } ],
        "last_failed_login": "",
        "last_login": "2018-01-12T10:54:53.222Z",
        "last_name": "Pere",
        "n_failed_logins": 0,
        "owner_group_id": 1,
        "owner_group_name": "admins",
        "owner_user_id": 3,
        "owner_user_name": "pipas",
        "suspended": false,
        "user_id": 3,
        "user_name": "pipas"
    }
}
```

removeFromGroup

Removes the user from a group if it is indeed the member of that group. Here is how the rquest should look like:

```
{
    "group_name": "admins",
    "operation": "removeFromGroup",
    "request_created": "2018-10-08T06:03:36.983Z",
    "request_id": 3,
    "user":
    {
        "class_name": "CmonUser",
        "user_name": "sisko"
    }
}
```

And here is the reply:

```
{
    "request_created": "2018-10-08T06:03:36.983Z",
    "request_id": 3,
    "request_processed": "2018-10-08T06:03:37.006Z",
    "request_status": "Ok",
    "request_user_id": 4,
    "user":
    {
        "acl": "user::rwx,user:pipas:rwx,group::r--,other::r--",
        "cdt_path": "/",
        "class_name": "CmonUser",
        "created": "2018-10-05T14:09:14.660Z",
        "disabled": false,
        "email_address": "sisko@ds9.com",
        "first_name": "Benjamin",
        "groups": [
        {
            "cdt_path": "/groups",
            "class_name": "CmonGroup",
            "group_id": 5,
            "group_name": "ds9",
            "owner_group_id": 1,
            "owner_group_name": "admins",
            "owner_user_id": 1,
            "owner_user_name": "system"
        } ],
        "last_failed_login": "",
        "last_login": "2018-10-05T14:09:18.111Z",
        "last_name": "Sisko",
        "n_failed_logins": 0,
        "owner_group_id": 1,
        "owner_group_name": "admins",
        "owner_user_id": 5,
        "owner_user_name": "sisko",
        "suspended": false,
        "title": "Captain",
```

(continues on next page)

(continued from previous page)

```
        "user_id": 5,
        "user_name": "sisko"
    }
}
```

canCreateUser

This call can be used to find out if the currently authenticated user is allowed to create a new user. For the user creation the authenticated user must be the superuser or has write access to a user group and a folder.

```
{
    "operation": "canCreateUser",
    "request_created": "2017-10-19T08:02:53.276Z",
    "request_id": 3
}
```

Here is the answer. Please note that the request will be successfull (the request_status will be "Ok") even if the user can not possibly create a new cmon user.

```
{
    "can_create_user": true,
    "folders_available_for_write": [ "/", "/core1/containers" ],
    "groups_available_for_write": [ "admins", "users", "nobody" ],
    "request_created": "2017-10-19T08:02:53.276Z",
    "request_id": 3,
    "request_processed": "2017-10-19T08:02:56.341Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```

- `can_create_user`: True if the authenticated user has a chance to create a new user.
- `folders_available_for_write`: The folders that are writable and so can hold a new user. The full path of the folders are enumerated in a list.
- `groups_available_for_write`: The names of the groups that are writable for the user and so can hold the new user.

canCreateGroup

This call is to decide if the authenticated user is allowed to create user groups.

```
{
    "operation": "canCreateGroup",
    "request_created": "2017-10-19T09:49:43.632Z",
    "request_id": 3
}
```

```
{
    "can_create_group": true,
    "folders_available_for_write": [ "/", "/core1/containers", "/
```

(continues on next page)

(continued from previous page)

```
↪galera_001/databases" ],
    "request_created": "2017-10-19T09:49:43.632Z",
    "request_id": 3,
    "request_processed": "2017-10-19T09:49:46.692Z",
    "request_status": "Ok",
    "request_user_id": 4
}
```

- `can_create_group`: True if the user is allowed to create cmon user groups.
- `folders_available_for_write`: The list of folders the user has write access.

updateUser

The updateUser RPC call can be used to update the current logged in user.

In the call you may define a new password as well in the `new_password` field.

In the call both the user's password and public kay can be specified so that the user can log in after the account is created.

```
{
    "new_password": "p",
    "operation": "updateUser",
    "user":
    {
        "class_name": "CmonUser",
        "user_id": 1234,
        "first_name": "Firstname",
        "last_name": "Lastname",
        "email_address": "something@xy.com",
        "user_name": "newusername"
    }
}
```

The reply to an updateUser request looks like this:

```
{
    "request_created": "2017-07-04T12:49:50.896Z",
    "request_id": 2,
    "request_processed": "2017-07-04T12:49:50.954Z",
    "request_status": "Ok",
    "request_user_id": 1,
    "user":
    {
        "class_name": "CmonUser",
        "email_address": "rpc@email.com",
        "groups": [
        {
            "class_name": "CmonGroup",
            "group_id": 6,
            "group_name": "testgroup"
```

(continues on next page)

(continued from previous page)

```
        } ],
        "user_id": 11,
        "user_name": "rpc_user"
    }
}
```

- `request_user_id`: This is just the user ID of the user who sent the request, the same as in any other replies.
- `user`: All the details of the user that has been updated.
- `user_id`: The user ID of the user that just has been created. The only inmutable property of the user that can not be ever changed.

Please note that there is no password or public key in the reply. We handle those as a secret.

setUserPreferences

A call to set the user's preferences.

Possible RPC request command:

To add new preferences:

```
$ s9s user --cmon-user=s9s --password=***** --preferences --
↪preferences-to-set="key1=value1;key2=value2;key3=value3"
```

To update existing preferences:

```
$ s9s user --cmon-user=s9s --password=***** --preferences --
↪preferences-to-set="key2=QWERTY;key3=UIOP"
```

In its basic form the RPC request looks like this:

```
{
  "operation": "setUserPreferences",
  "request_created": "2023-01-25T14:57:37.155Z",
  "request_id": 2,
  "user": {
    "class_name": "CmonUser",
    "user_name": "s9s",
    "preferences": {
      "param7": "val7"
    }
  }
}
```

- `user`: The ClusterControl user's data in the request to set his/her properties.
- `user.user_name`: Must be a name of a valid logged in user.
- `user.preferences`: The list of one or more user's preferences to store to the DB.

In its basic form the RPC response looks like this:

```
{
  "controller_id": "b195b016-ed96-49e4-a646-596b52d9d8f3",
  "reply_received": "2023-01-25T14:57:37.168Z",
  "request_created": "2023-01-25T14:57:37.155Z",
  "request_id": 2,
  "request_processed": "2023-01-25T14:57:37.167Z",
  "request_status": "Ok",
  "request_user_id": 3,
  "debug_messages": [ "RPC V2 authenticated user is 'admin'."  ],
  "user": {
    "class_name": "CmonUser",
    "user_id": 3,
    "user_name": "s9s",
    "preferences": {
      "param11": "val11",
      "param12": "val12",
      "param7": "val7"
    }
  }
}
```


- `user`: The ClusterControl user's data, including his/her actual properties.
- `user.preferences`: The list of all user's preferences stored in the databaseafter the request is completed.

getUserPreferences

A call to get the user's preferences from DB.

Possible RPC request command with s9s tools:

```
$ s9s user --cmon-user=s9s --password=***** --get-preferences
```

In its basic form the RPC request looks like this:

```
{
  "operation": "getUserPreferences",
  "request_created": "2023-01-25T13:55:07.068Z",
  "request_id": 2,
  "user": {
    "class_name": "CmonUser",
    "user_name": "s9s"
  }
}
```

- `user`: The ClusterControl user's data in the request to get his/her properties.
- `user.user_name`: Must be a name of a valid logged in user.

In its basic form the RPC response looks like this:

```
{
  "controller_id": "b195b016-ed96-49e4-a646-596b52d9d8f3",
  "reply_received": "2023-01-25T13:55:07.073Z",
  "request_created": "2023-01-25T13:55:07.068Z",
  "request_id": 2,
  "request_processed": "2023-01-25T13:55:07.072Z",
  "request_status": "Ok",
  "request_user_id": 3,
  "debug_messages": [ "RPC V2 authenticated user is 'admin'."  ],
  "user": {
    "class_name": "CmonUser",
    "user_id": 3,
    "user_name": "s9s",
    "preferences": {
      "pref01": "value01",
      "pref02": "value02",
      "pref03": "value03"
    }
  }
}
```


- `user`: The ClusterControl user's data, including his/her actual properties.
- `user.preferences`: The list of all user's preferences stored in the databaseafter the request is completed.

deleteUserPreferences

A call to read the user's preferences.

Possible RPC request command:

```
$ s9s user --cmon-user=s9s --password=***** --preferences --
↪preferences-to-delete="key1;key2"
```

In its basic form the RPC request looks like this:


```
{
  "operation": "deleteUserPreferences",
  "request_created": "2023-01-25T14:57:46.274Z",
  "request_id": 2,
  "user": {
    "class_name": "CmonUser",
    "user_name": "s9s",
    "preferences": {
      "param7": ""
    }
  }
}
```


- `user`: The ClusterControl user's data in the request to set his/her properties.
- `user.user_name`: Must be a name of a valid logged in user.

- `user.preferences`: The list of one or more user's preferences to delete from the DB.

In its basic form the RPC response looks like this:


```
{
  "controller_id": "b195b016-ed96-49e4-a646-596b52d9d8f3",
  "reply_received": "2023-01-25T14:57:46.284Z",
  "request_created": "2023-01-25T14:57:46.274Z",
  "request_id": 2,
  "request_processed": "2023-01-25T14:57:46.283Z",
  "request_status": "Ok",
  "request_user_id": 3,
  "debug_messages": [ "RPC V2 authenticated user is 'admin'."  ],
  "user": {
    "class_name": "CmonUser",
    "user_id": 3,
    "user_name": "s9s",
    "preferences": {
      "param11": "val11",
      "param12": "val12"
    }
  }
}
```


- `user`: The ClusterControl user's data, including his/her actual properties.
- `user.preferences`: The list of all user's preferences from the databaseafter the request is completed.

getDbVersions

Lists the available 1-2 or 3 digits database versions supported by ClusterControl for a given cluster type and vendor. Returned values can be used as values on "provider_version" field on the cluster creation jobs.


```
{
  "cluster_type": "galera",
  "operation": "getDbVersions",
  "request_created": "2024-01-04T17:59:22.892Z",
  "request_id": 2,
  "vendor": "mariadb"
}
```


The controller then will reply back with supported 1-2 digits versions for supported databases on specified `cluster_type` and `vendor` client:


```
{
  "authenticated": true,
  "cluster_type": "GALERA",
  "controller_id": "449bf7e9-a274-41db-bec6-85c915e619c5",
  "reply_received": "2024-01-04T17:59:22.895Z",
  "request_created": "2024-01-04T17:59:22.892Z",
  "request_id": 2,
  "request_processed": "2024-01-04T17:59:22.894Z",
```


(continues on next page)

(continued from previous page)


```
  "request_status": "Ok",
  "request_user_id": 6,
  "rpc_version": "2.0",
  "vendor": "mariadb",
  "db_versions": [
    "10.4",
    "10.5",
    "10.6",
    "10.7",
    "10.8",
    "10.9",
    "10.10",
    "10.11"
  ],
}
```


The reply contains the following fields:

- `db_versions`: Possible values on `provider_version` field on deployment jobs

It is also possible to specify the `patch_number` option on some clusters (for example: "galera") The default value for this option is false.


```
{
  "cluster_type": "galera",
  "operation": "getDbVersions",
  "patch_number": true,
  "request_created": "2024-01-04T18:06:00.767Z",
  "request_id": 2,
  "vendor": "percona"
}
```


The controller then will reply back with supported 3 digits versions only for supported databases on specified `cluster_type` and `vendor` client:


```
{
  "authenticated": true,
  "cluster_type": "GALERA",
  "controller_id": "449bf7e9-a274-41db-bec6-85c915e619c5",
  "reply_received": "2024-01-04T18:06:00.771Z",
  "request_created": "2024-01-04T18:06:00.767Z",
  "request_id": 2,
  "request_processed": "2024-01-04T18:06:00.771Z",
  "request_status": "Ok",
  "request_user_id": 6,
  "rpc_version": "2.0",
  "use_vendor_api": false,
  "vendor": "percona",
  "db_versions": [
    "8.0.21",
    "8.0.22",
```


(continues on next page)

(continued from previous page)

```
    "8.0.23",
    "8.0.25",
    "8.0.26",
    "8.0.27",
    "8.0.28",
    "8.0.29",
    "8.0.30",
    "8.0.31",
    "8.0.32",
    "8.0.33",
    "8.0.34"
  ],
}
```

The reply contains the following fields:

- `db_versions`: possible values on `provider_version` field on deployment jobs

The source of the versions can be the stored values of ClusterControl or the vendor's API. The `use_vendor_api` option can be used to specify the source of the versions. (default value is false)

- ```
  {
    "cluster_type": "galera",
    "operation": "getDbVersions",
    "patch_number": true,
    "request_created": "2024-01-04T18:06:00.767Z",
    "request_id": 2,
    "use_vendor_api": true,
    "vendor": "percona"
  }
  ```

The controller then will query vendor's api to reply back with supported 3 digits versions only for supported databases on specified 'cluster_type' and 'vendor' client:

```
{
  "authenticated": true,
  "cluster_type": "GALERA",
  "controller_id": "449bf7e9-a274-41db-bec6-85c915e619c5",
  "reply_received": "2024-01-04T18:06:00.771Z",
  "request_created": "2024-01-04T18:06:00.767Z",
  "request_id": 2,
  "request_processed": "2024-01-04T18:06:00.771Z",
  "request_status": "Ok",
  "request_user_id": 6,
  "rpc_version": "2.0",
  "use_vendor_api": false,
  "vendor": "percona",
  "db_versions": [
    "8.0.21",
    "8.0.22",
    "8.0.23",
```

(continues on next page)
```

(continued from previous page)

```
    "8.0.25",
    "8.0.26",
    "8.0.27",
    "8.0.28",
    "8.0.29",
    "8.0.30",
    "8.0.31",
    "8.0.32",
    "8.0.33",
    "8.0.34"
  ],
}
```

getClusterTypes

Gets the list of supported clusters types. Returned values can be used as an argument to the `cluster_type` option (upper or lower case).

```
{
  "operation": "getClusterTypes",
  "request_created": "2024-01-04T17:43:14.776Z",
  "request_id": 2
}
```

The controller then will reply back with supported cluster types client:

```
{
  "authenticated": true,
  "controller_id": "449bf7e9-a274-41db-bec6-85c915e619c5",
  "reply_received": "2024-01-04T17:43:14.779Z",
  "request_created": "2024-01-04T17:43:14.776Z",
  "request_id": 2,
  "request_processed": "2024-01-04T17:43:14.779Z",
  "request_status": "Ok",
  "request_user_id": 6,
  "rpc_version": "2.0",
  "cluster_types": [
    "MYSQLCLUSTER",
    "REPLICATION",
    "MYSQL_SINGLE",
    "MONGODB",
    "POSTGRESQL_SINGLE",
    "GROUP_REPL",
    "REDIS",
    "MSSQL_SINGLE",
    "MSSQL_AO_ASYNC",
    "MSSQL_AO_SYNC",
    "ELASTIC",
    "GALERA"
  ],
```

(continues on next page)

(continued from previous page)

```
}
```

The reply contains the following fields:

- `cluster_types`: possible cluster type supported

getVendors

Gets the list of supported vendors on all supported clusters. If no version is retrieved for a specific cluster type-vendor, it means that it is not supported.

```
{
  "operation": "getVendors",
  "request_created": "2024-01-04T17:49:46.012Z",
  "request_id": 2
}
```

The controller then will reply back with supported vendors on all cluster types client:

```
{
  "authenticated": true,
  "controller_id": "449bf7e9-a274-41db-bec6-85c915e619c5",
  "reply_received": "2024-01-04T17:49:46.016Z",
  "request_created": "2024-01-04T17:49:46.012Z",
  "request_id": 2,
  "request_processed": "2024-01-04T17:49:46.015Z",
  "request_status": "Ok",
  "request_user_id": 6,
  "rpc_version": "2.0",
  "debug_messages": [ "RPC V2 authenticated user is 'ccalvaro'."  ],
  "vendors": [
    "percona",
    "mariadb",
    "oracle",
    "codership",
    "10gen",
    "mongodbenterprise",
    "microsoft",
    "elasticsearch",
    "redis",
    "postgresql",
    "enterprisedb"
  ]
}
```

The reply contains the following fields:

- `vendors`: Possible vendor values on `vendor` field.

3.12.24 Controllers manipulation on database

The `/v2/poolcontrollers` path is to retrieve controllers managed by the system.

listPoolControllers

The getControllers RPC call can be used to retrieve controllers from the DB. By default, only controllers that are not in 'stopped' status are returned.

Example request:

```
{
  "operation": "listcontrollers",
  "controller_id": 0,
}
```

- `controller_id`: Optional. If provided and non-zero, only the controller with this id is returned.
- `all_controllers`: Optional. If set to true, all controllers including stopped ones are returned.

Example reply:

```
{
  "controllers": [ {
    "controller_id": 1,
    "hostname": "controller1",
    "port": 3306,
    "status": "active",
    "properties": "full_controller",
    "report_ts": "2025-06-28 15:54:00",
    "clusters": [ 1, 2, 3 ]
  }, {
    "controller_id": 2,
    "hostname": "controller2",
    "port": 3306,
    "status": "stopped",
    "properties": "full_controller",
    "report_ts": "2025-06-28 15:54:00",
    "clusters": [ 4, 5, 6 ]
  } ],
  "total": 2
}
```

startController

Starts a controller in the pool remotely via SSH. This operation is only allowed when the controller is in pool mode and requires superuser privileges.

Example request:

```
{
  "operation": "startcontroller",
  "controller_id": 2
}
```

- `controller_id`: Required. The ID of the controller to start.

Example reply:

```
{
  "controller": {
    "controller_id": 2,
    "hostname": "controller2",
    "port": 9500,
    "status": "active",
    "properties": "full_controller",
    "clusters": [ 4, 5 ]
  },
  "message": "Controller start command sent successfully."
}
```

stopController

Stops a controller in the pool remotely via SSH. This operation is only allowed when the controller is in pool mode and requires superuser privileges. Cannot be used to stop the local controller.

Example request:

```
{
  "operation": "stopcontroller",
  "controller_id": 2
}
```

- `controller_id`: Required. The ID of the controller to stop.

Example reply:

```
{
  "controller": {
    "controller_id": 2,
    "hostname": "controller2",
    "port": 9500,
    "status": "stopped",
    "properties": "full_controller",
    "clusters": [ 4, 5 ]
  },
  "message": "Controller stop command sent successfully."
}
```

setPoolMode

Changes the pool mode state on the controller. When the state changes, the controller will restart automatically. This operation is only allowed on the main controller and requires superuser privileges.

Example request:

```
{
  "operation": "setPoolMode",
```

(continues on next page)

(continued from previous page)

```
  "pool_mode": true,
  "conf_storage": "nfs"
}
```

- `pool_mode`: Optional. Boolean indicating whether to enable or disable pool mode. Defaults to true.
- `conf_storage`: Optional. Storage type for pool configuration. Valid values: "nfs" or "k8s". Defaults to "nfs".

Example reply:

```
{
  "pool_mode": true,
  "conf_storage": "nfs",
  "controller": {
    "controller_id": 1,
    "hostname": "controller1",
    "port": 9500,
    "status": "restarting",
    "properties": "main_controller",
    "clusters": [ 1, 2, 3 ]
  }
}
```

3.12.25 Watchlists manipulation on database

The `/v2/watchlists` path is to manipulate whatchlists to visualize clusters according to user configuration.

createWatchlist

The `createWatchlist` RPC call can be used to create a new Watchlist.

```
{
    "operation": "createWatchlist",
    "watchlist":
    {
          "watchlist_name": "my watch",
          "topics": "topic1, topic3",
          "clusters": "11, 13, 14",
          "page_by": "cluster",
          "grid": "2x2",
          "owner_id": 1
    }
}
```

Here are the most important elements of the request:

- `name`: name given by the user to identify watchlist
- `topics`: themes for the different information types to be visualized
- `clusters`: List of clusters IDs to be visualized on watchlist

- `paged_by`: pagination to be used on watchlist. Initially only 'cluster' or 'topic' are supported
- `grid`: The grid layout to be used on watchlist.
- `owner_id`: user ID of the owner (creator) of the watchlist

The other parameters are the same as in the request.

updateWatchlist

The `updateWatchlist` RPC call can be used to update an existing watchlist.

```
{
    "operation": "updateWatchlist",
    "watchlist":
    {
         "watchlist_id" : 2,
         "watchlist_name": "my watch",
         "topics": "topic1, topic3",
         "clusters": "11, 13, 14",
         "page_by": "cluster",
         "grid": "2x2",
         "owner_id": 1
    }
}
```

Here are the most important elements of the request:

- `watchlist_id`: The internal ID of the watchlist once it has been persisted.
- `name`: name given by the user to identify watchlist
- `topics`: themes for the different information types to be visualized
- `clusters`: List of clusters IDs to be visualized on watchlist
- `paged_by`: pagination to be used on watchlist. Initially only 'cluster' or 'topic' are supported
- `grid`: The grid layout to be used on watchlist. Initially only '2x2' | '2x3' | '3x2' | '3x3' are allowed
- `owner_id`: user ID of the owner (creator) of the watchlist

listWatchlists

A call to read the watchlists on controller database. In its basic form the RPC call looks like this:

```
{
  "operation": "listWatchlists",
  "request_created": "2025-04-01T08:35:17.696Z",
  "request_id": 2,
  "watchlist_id": 0
}
```

- `watchlist_id`: The request may contain this field to limit the list of the returned watchlists to those that have this id. Use '0' to get all watchlists.
- `watchlist_name`: The request may contain this field to limit the list of the returned users to those that has none of the tags passed here.

The reply contains the users managed by the controller:


```
{
  "controller_id": "3d797309-1bfe-4b6d-be07-e65d65f3b411",
  "reply_received": "2025-04-01T08:35:23.397Z",
  "request_created": "2025-04-01T08:35:17.696Z",
  "request_id": 2,
  "request_processed": "2025-04-01T08:35:23.396Z",
  "request_status": "Ok",
  "request_user_id": 7,
  "total": 2,
  "debug_messages": [ "RPC V2 authenticated user is 'ccalvaro2'."  ],
  "watchlists": [
    {
      "name": "dbWatchlist",
      "id": 1,
      "owner_id": 3,
      "page_by": "cluster",
      "properties": "empty",
      "clusters": [
        "12",
        "13",
        "14"
      ],
      "grid": [ "2x2"  ],
      "topics": [
        "topic1",
        "topic3"
      ]
    },
    {
      "name": "w1",
      "id": 2,
      "owner_id": 0,
      "page_by": "cluster",
      "properties": "",
      "clusters": [ "1"  ],
      "grid": [  ],
      "topics": [ "t1"  ]
    }
  ]
}
```


- `total`: The total number of watchlists stored on controller db.
- `watchlists`: A list of watchlists class objects. To get more information about the user objects please check the watchlists sections.

deleteWatchlist

This call can be used to delete an existing watchlist.

```
{
  "operation": "deleteWatchlist",
  "request_created": "2025-04-01T08:49:50.860Z",
  "request_id": 2,
  "watchlist_id": 2
}
```

3.13 ClusterControl Domain Specific Language (DSL)

The DSL syntax is similar to JavaScript, with extensions to provide access to ClusterControl's internal data structures and functions. The CCDSL allows you to execute SQL statements, run shell commands/programs across all your cluster hosts, and retrieve results to be processed for advisors/alerts or any other actions.

3.13.1 Introduction

This language will be provided for the users, to implement monitoring related code in a language similar to JavaScript. The code is executed on the Cmon Controller and will have access to the cluster nodes through the network.

The language is different from JavaScript in some ways. Here is a list of the most important differences:

- Semicolons are mandatory like in C or C++.
- Not all numbers are handled as double-precision floating-point values, we have integers and even unsigned long integers. We need those to handle disk sizes and network traffic measured in bytes.
- There are associative arrays with the data type `Map`.
- The arrays are two-dimensional, but they can be used as one-dimensional arrays (e.g. `a[10, 11]` and `a[10]` are also valid).
- We have a `List` type.
- JavaScript uses a period in function names like `JSON.parse(text)`, here we use the C++ notation like `JSON::parse(text)`.
- New variables created on-the-fly in functions are local variables and not globals.
- The language implements a C like #include preprocessor directive.

Supported ClusterControl Controller (CMON) version: 1.7.0.

3.13.2 Language basics

Preprocessor directives

This system has no preprocessor, the parsing is done in one stage. Some expressions however are implemented the same way it is implemented in most of the C/C++ language preprocessors. Here is a list of the supported preprocessor directives.

- `#include "filepath"`
  Preprocessor directive to include a source file the same way it is used for the C/C++ languages.

- `#pragma once` Preprocessor directive to protect an include file to be included multiple times. The same as in the C/C++ languages.

Types

Here is an example of the new operator.

```
var a = new Int;
var b = new CmonHost();
passed =
  a === 0 &&
  b.typeName() == "CmonHost";
```

- Int
- Bool
- Double
- Ulonglong
- String
- Error
- Map
- List
- Array
- CmonHost
- CmonMySqlHost
- CmonPostgreSqlHost
- CmonGaleraHost
- CmonMongoHost
- CmonMaxScaleHost
- CmonAdvice
- CmonClusterConfig
- CmonGraph
- CmonJob

Literals

- **Boolean literals**. These are just simply `true` and `false`.
- **String literals**. The string literals are handled the usual way, they can be enclosed in single or double quotes, single-quoted strings may contain double quotes, the double-quoted strings can contain single quotes.

```
var carName1 = "Volvo XC60";
var carName2 = 'Volvo XC60';
var answer1 = "It's alright";
var answer2 = "He is called 'Johnny'";
var answer3 = 'He is called "Johnny"';
```

Strings literals can also be concatenated in compile time as it is seen in the C/C++ languages:

```
var a =
    "one "
    "two "
    "three";
passed = a == "one two three";
```

- **Integer literals**. Integer literals are integer numbers that are fit to be stored in the host computer's "int" type. Here is an example:

```
var1 = 0xff;
var2 = 0XFFFF;
passed = var1 == 255 && var2 == 65535;
```

- **Unsigned long long literals**. If an integer literal is too big to fit on an "int" type but fits in an unsigned long it is automatically stored in an unsigned long (or "Ulonglong" type). If the number is prefixed with "ull" it is also considered to have the Ulonglong type.

```
var a = 91872698761001;
var b = 10ull;
passed =
    a.typeName() == "Ulonglong" &&
    b.typeName() == "Ulonglong";
```

- **Double literals**. All the numbers that are not fit any integer types will be stored in a double type as the usual double format strings.

```
var a = 10.2;
var b = 10.8E11;
var c = 2.8e-10;
passed =
    a.typeName() == "Double" &&
    b.typeName() == "Double" &&
    c.typeName() == "Double";
```

- **Error literals**.

```
var a = #ARGS!;
passed = a === #ARGS!;
```

The available error literals are the following:

- `#NULL`: Null value error.
- `#DIV/0!`: Division by zero.
- `#VALUE!`: Type mismatch error e.g. log of a string.

- `#REF!`: Invalid variable reference, missing variable.
- `#NAME?`: The name was not found, e.g. a function name is invalid.
- `#NUM!`: Invalid numerical value e.g. sqrt(-1).
- `#N/A`: Value is not available.
- `#SYNTAX!`: Syntax error in the formula.
- `#ARGS!`: Argument number for a function is invalid.

- **Map literals**. Map literals are associative arrays that can hold any type of value (even maps) and can be indexed by strings. Here is an example of how to create and use a map:

```
var a = {};
a["one"] = {};
a["one"]["two"] = "value";
passed =
    a.typeName() == "Map" &&
    a["one"]["two"] == "value";
```

The map keys could be listed using the `.keys()` method:

```
// an example iteration on the map:
var testmap = {};
testmap["key1"] = "test";
testmap["key2"] = "test";
keys = testmap.keys();

for (i = 0; i < keys.size(); ++i)
{
    print (keys[i] + ": " + testmap[keys[i]]);
}
```

- **Regular expression literals**. Regular expression literals are supported the same way they are supported by the JavaScript language. Here is an example that demonstrates the usage of such literals:

```
var regexp = /([0-9]+)x([0-9]+)/i;
var string = "s: 640x480";
string.replace(regexp, "$2x$1");
// string === "s: 480x640";
```

Regular expressions have the type `CmonRegExp`.

3.13.3 Functions

If the function is called with the wrong number of arguments the return value will be an `#ARGS!` error (the type of the return value will be "Error").

JSON Functions

- `Map JSON::parse(text)`
  Parses a JSon string and returns it in a Map format.

- `String JSon::toString(map)` Converts object (a constructed Map object) to a well-formatted JSON string.
- `String JSON::postRequest(url, map)`
  Sends a POST request to the specified URL using JSonized value of the 'map'.

An example reply/request:

```
var req = new Map;
req["operation"] = "clusters";
var retval = JSON::postRequest("http://localhost:9500/0/clusters", req);
print ("retval of clusters:\n" + retval);
```

And the raw reply:

```
{
    "cc_timestamp": 1447936009,
        "requestStatus": "ok",
        "results": {
            "exitStatus": null,
            "fileName": "/rpc-client-test.js",
            "messages": [
            {
                "message": "retval of clusters:\\n{\n    \"cc_timestamp\":␣
↪1447936009,\n    \"clusters\": [ \n    {\n        \
↪"clusterAutorecovery\": true,\n        \"configFile\": \"/
↪etc/cmon.d/cmon_1.cnf\",\n        \"id\": 1,\n        \"logFile\": \"/
↪var/log/cmon_1.log\",\n        \"name\": \"cluster_1\",\n        \
↪"nodeAutorecovery\": true,\n        \"running\": true,\n        \
↪"status\": 0,\n        \"statusText\": \"\",\n        \"type\": \
↪"galera\"\n    } ],\n    \"info\": \n    {\n        \"hasLicense\":␣
↪true,\n        \"licenseExpires\": 13,\n        \"licenseStatus\": \
↪"License will expire in 13 days.\",\n        \"version\": \"1.2.12\"\n  ␣
↪},\n    \"requestStatus\": \"ok\"\n}"
            }
            ],
            "status": "Ended"
        },
        "success": true
}
```

Controller Functions

- `abort()`
  Aborts the execution of the script and presents the backtrace showing where exactly the abort() function was executed.
- `exit(exitstatus)`
  Ends the execution of the script and returns the exit status to the Cmon environment. Ends the script with normal program termination.
- `main(...)`
  If a `main()` function is provided once the program lines outside the functions are executed the `main()` function will be executed. The arguments of the `main()` function will be passed from the

running environment and the return value of the `main()` will be sent back as the exit status. Except if the `exit()` function is used to set the exit status.

```
var global1 = 10;
function main(arg1)
{
    return
        arg1 == "UtCmonImperative" &&
        global1 === 10 &&
        global2 === 11;
}
var global2 = 11;
```

Input/Output Functions

- `string print([value]...)`
  Prints all the values as one message with the severity set to 'info'. Also returns the printed string.
- `string warning([value]...)`
  Prints all the values as one message with the severity set to 'warning'. Also returns the printed string.
- `string error([value]...)`
  Prints all the values as one message with the severity set to 'critical'. Also returns the printed string.

General Tag Functions

- `string value.typeName()`
  Returns the type name of the value.
- `string value.toString([formatid])`
  Returns the value converted to a string. If the format ID is specified the string will be formatted accordingly. The available format specifiers are defined in `cmon/io.h`.

```
/*
* Converting a double to string using various formats.
*/
#include "cmon/io.h"

var theDouble = 42.0;
var str1      = theDouble.toString(TwoDecimalNumber);
var str2      = theDouble.toString(FourDecimalNumber);
var str3      = theDouble.toString(DateTime);

passed =
    str1 == "42.00" &&
    str2 == "42.0000" &&
    str3 == "Thu Jan  1 01:00:42 1970";
```

- `boolean value.empty()`
  Returns true if the value is empty. The strings are empty when no characters are in them, the container objects (e.g. maps or lists) are empty when there are no items in them.
- `int value.size()`
  The size of the strings is the number of characters in them, container objects hold the number of

items as size. One mentionable exception is the Array objects that return the number of the columns as size, so it is easy to use them as single-dimensional arrays (sometimes called vectors).

- `boolean value.isNull()`
  Returns true if the value is a null string (e.g. an unset value from an SQL server).
- `boolean value.isInvalid()`
  Returns true if the value is invalid, e.g. a variable with no value set before.
- `boolean value.isString()`
  Returns true if the value is a string.
- `int value.isInt()`
  Returns true if the value is an integer.
- `boolean value.isULongLong()`
  Returns true if the value is an unsigned long long.
- `boolean value.isDouble()`
  Returns true if the value is a double.
- `boolean value.isBoolean()`
  Returns true if the value is a boolean.
- `boolean value.isNumber()`
  Returns true if the type of the value is one of the number formats (e.g. int, ulonglong or double).
- `boolean value.isError()`
  Returns true if the value is an error.
- `boolean value.isMap()`
  Returns true if the value is a map.
- `boolean value.isList()`
  Returns true if the value is a list.
- `boolean value.isArray()`
  Returns true if the value is an array.
- `int value.toInt()`
  Converts the value to the host computer's integer format.
- `ulonglong value.toULongLong()` Converts the value to unsigned long long.
- `double value.toDouble()`
  Converts the value to double, strings with the usual number formats will be recognized.
- `boolean value.toBoolean()`
  Converts the value into boolean. String like "true" and "false" will be recognized, integer values will have true value when they are not equal to zero.

Mathematical Functions

- `number rand()`
  Creates a random number between 0 and 1.
- `number pi()`
  Returns π, a mathematical constant, the ratio of a circle's circumference to its diameter.
- `number degrees(number)`
  Converts radians to degrees.

- `number radians(number)`
  Converts degrees to radians.
- `number sign(number)`
  Returns -1 if the number is negative, +1 if not.
- `number sin(number)`
  Returns the sine of an angle.
- `number asin(number)`
  Returns the inverse hyperbolic sine of a number.
- `number sinh(number)`
  Returns the hyperbolic sine of a number.
- `number cos(number)`
  Returns the cosine of an angle.
- `number acos(number)`
  Returns the inverse cosine of a number.
- `number cosh(number)`
  Returns the hyperbolic cosine of a number.
- `number acosh(number)`
  Returns the inverse hyperbolic cosine of a number.
- `number fisher(number)`
  Returns the Fisher transformation of a number.
- `number fisherinv(number)`
  Returns the inverse of the Fisher transformation of a number.
- `number tan(number)`
  Returns the tangent of a number.
- `number atan(number)`
  Returns the inverse tangent of a number.
- `number tanh(number)`
  Returns the hyperbolic tangent of a number.
- `number atanh(number)`
  Returns the inverse hyperbolic tangent of a number.
- `number log(number, [base])`
  Returns the logarithm of a number to a specified base or to base 10 if the base is not specified.
- `number sqrt(number)`
  Returns the square root of a number.
- `number abs(number)`
  Returns the absolute value of a number. Also works with arrays.
- `number exp(number)`
  Returns e raised to the power of the given number.
- `number floor(number, [significance])`
  Returns the number rounded down to the multiple of the given significance.

- `number ceiling(number, [significance])`
  Returns the number rounded up to the multiple of the given significance.
- `number round(number, digits)`
  Returns the number rounded to the given number of digits.
- `number roundup(number, digits)` Returns the number rounded up to the given number of digits.
- `number rounddown(number, digits)`
  Returns the number rounded down to the given number of digits.
- `number mround(number, multiple)`
  Returns the number rounded to the given multiple.
- `number even(number)`
  Returns the number rounded to the nearest even number.
- `number iseven(number)`
  Returns true if the number is even.
- `number odd(number)`
  Returns the number rounded to the nearest odd number.
- `number convert(number, from, to)`
  Converts between units. Supported units are byte, kbyte, mbyte, gbyte, tbyte, celsius, kelvin, fahrenheit, hz, mhz, ghz. When the 'units' utility program is installed more units are available for CONVERT().
- `number isodd(number)`
  Returns true if the number is odd.

Functions providing information about values

- `boolean iserr(value)`
  Returns true if the value is an error.
- `boolean isnumeric(value)`
  Returns true if the value is a number.
- `boolean istext(value)`
  Returns true if the value is a string.
- `boolean isnumber(value)`
  Returns true if the value is a number.
- `boolean isarray(value)`
  Returns true if the value is an array.

String Functions

- `integer asc(text)`
  Returns the ASCII value of the first character in the string.
- `text char(number)`
  Returns the character that has the given number as ASCII value.
- `text chr(number)`
  Returns the character that has the given number as ASCII value.

- `text left(text, number)`
  Returns the leftmost characters of string.
- `text right(text, number)`
  Returns the rightmost characters of a string.
- `boolean startswith(text1, text2)` Returns true if text1 starts with text2.
- `boolean endwith(text1, text2)`
  Returns true if text1 ends with text2.
- `text mid(text, start, length)`
  Returns the substring of a text that starts at the given location has at most the given length.
- `text escape(text)`
  Returns the text escaped with backslash characters.
- `text unescape(text)`
  Returns the text after the escaping characters has been removed.
- `text upper(text)`
  Returns the text converted to uppercase.
- `text lower(text)`
  Returns the text converted to lowercase.
- `text trim(text)` Returns the text after removing the white characters from the beginning and the end.
- `number len(text)`
  Returns the length of a string.
- `text concatenate(text, [text]...)`
  Returns a string that holds all the arguments concatenated.
- `number int(value)`
  Returns the value converted into an integer number.
- `number cbool(value)`
  Returns the value converted into a logical (boolean) value.
- `number cstr(value)`
  Returns the value converted into a string.
- `number cdbl(value)`
  Returns the value converted into a floating-point double-precision number.

String Tag Functions

- `int string.length()`
  Returns the length of the string.
- `int string.indexOf(substring, [start])`
  Returns the position of the first occurrence of the substring in the string. Returns -1 of the substring was not found.
- `array string.split(separatorstring)`
  Returns an array that contains all the substrings separated by the given separator in the original string.

- `string string.substr(begin, length)`
  Extracts parts of a string, beginning at the character at the specified position. Returns the specified number of characters.
- `string string.trim()`
  Returns the string without the leading and tailing whitespace characters.
- `string string.toLowerCase()`
  Returns the string converted to lower case letters.
- `string string.toUpperCase()`
  Returns the string converted to upper case characters.
- `boolean string.contains(string)`
  Returns true if the string contains the argument as substring.
- `string string.replace(string, string)`
  Returns the string that has the given substring replaced to the second argument.
- `boolean string.looksInteger()`
  Returns true if the string represents an integer number that can be stored as an Int type value.
- `integer string.toInteger()`
  Converts the string to an Int type integer number.
- `boolean string.looksULongLong()`
  Returns true if the string converts to an integer fits on an Ulonglong but will not fit on an Int type.
- `boolean string.looksDouble()`
  Returns true if the string can be converted to a Double type number.
- `boolean string.looksBoolean()`
  Returns true if the string is a textual representation of a boolean value (e.g. “true” or “false”).
- `integer string.toULongLong()`
  Converts the string to an Ulonglong type integer number.
- `boolean string.looksEmail()`
  Returns true if the string represents a valid e-mail address.
- `boolean string.looksIpAddress()`
  Returns true if the string represents a valid IPv4 address.

General Array Functions

- `value choose(position, value, [value]...)`
  Returns the value at the given position of the values. The first value is returned when a position is 1.
- `array transpose(array)`
  Returns the transposed version of an array where rows are converted to columns and columns are converted to rows.
- `array filter rows(array, column, value)`
  Returns an array that contains only those rows matching to a specific value in the specified column.
- `number columns(array)`
  Returns how many columns the array has.
- `number rows(array)`
  Returns how many rows the array has.

- `value vlookup(value, array, column, [notExact])`
  Performs a vertical lookup in the leftmost column of the array and returns the value of the found row from another column.
- `value hlookup(value, array, column, [notExact])`
  Performs a horizontal lookup in the leftmost column of the array and returns the value of the found column from another row.
- `value match(value, array, [matchType])`
  Searches for a value and returns the relative position of the item found.

Statistical Functions

Statistical functions will provide statistical calculations on number sets. The values for most functions can be passed through individual arguments or using arrays. Here is an example:

```
b = [ 10, 8, 5 ];
c = average(b);
d = average(10, 11, 12);
```

- `number count(value, [value]...)`
  Returns how many of the values and array elements contains a number.
- `number countblank(value, [value]...)`
  Returns how many of the values and array elements are empty.
- `number min(value, [value]...)`
  Returns the smallest number.
- `number max(value, [value]...)`
  Returns the largest number.
- `number sum(value, [value]...)`
  Returns the sum of all number.
- `number sumsq(value, [value]...)`
  Returns the sum of the squares of the numbers.
- `number product(value, [value]...)`
  Returns the product of all the numbers.
- `number average(arg1, [arg2]...)`
  Calculates the average, the arithmetic mean value for a set of numbers or arrays.
- `number geomean(arg1, [arg2]...)`
  Calculates the geometric mean of a set of positive numbers.
- `number mode(value1, [value2]...)`
  Returns the most frequently occurring value of a data set.
- `number emaverage(alpha, value1, [value2]...)`
  Calculates the exponential moving average.
- `number median(value1, [value2]...)`
  Returns the median of the numbers.
- `number percentile(array, [n])`
  Returns the nth percentile of the numbers in the array.

- `number small(array, n)`
  Returns the nth smallest number of an array.
- `number large(array, n)`
  Returns the nth largest number of an array.
- `number stdev(value1, [value2]...)`
  Returns the standard deviation of a sample.
- `number avedev(value1, [value2]...)`
  Returns the average of the absolute deviations of the number.
- `number pearson(array1, array2)`
  Returns the Pearson product-moment correlation coefficient between two sets of numbers.
- `number correl(array1, array2)`
  Returns the correlation coefficient of the array1 and array2.
- `number covar(array1, array2)`
  Returns the average of the products of deviations for each data pair.
- `number devsq(value1, [value2]...)`
  Returns the sum of the squares of deviations from the average.
- `number var(value1, [value2]...)`
  Returns the variance of a set of numbers.
- `number forecast(x, knownYValues, knownXValues)`
  Estimates future values from existing data using the linear regression method.
- `number linest(knownYValues, knownXValues)`
  Uses the "least squares" method to find the linear equation that fits the data. Returns an array with the slope and height of the equation.
- `number slope(knownYValues, knownXValues)`
  Uses the "least squares" method to find the linear equation that fits the data. Returns the slope of the equation.
- `number intercept(knownYValues, knownXValues)`
  Uses the "least squares" method to find the linear equation that fits the data. Returns the y-axis intersection point of the line.

Regular Expression Functions

Regular expressions has the type CmonReExp with the following tag functions:

- `Bool regexp.test(String)`
  Tests if the regular expression matches a string, returns true or false accordingly.
- `Int regexp.lastIndex()`
  Returns the index of the string where the next match will be checked. Returns 0 if the "global" modifier is not set.
- `List regexp.match(String)`
  Tests if the regular expression matches a string, returns a list that contains the matched text (at index 0) and all the matched text for subexpressions (from index 1).
- `Map regexp.exec(String)`
  Checks the match on a string and returns a number of information in a map.

When the toString() function is called the CmonRegExp type it supports the following format specifiers:

- `%r` – The regular expression string itself.
- `%j` – The regular expression and the modifiers in JavaScript notation (e.g. `/[0-9]+/ig`).
- `%m` – The matched string if there is any.
- `%nm` – The nth matched sub-expression where n is an integer number.

Date and Time Functions

This type is different from the JavaScript Date type.

- `CmonDateTime CmonDateTime::currentDateTime()`
  Returns the real-time clock time from the host computer (controller).
- `CmonDateTime CmonDateTime::fromUnixTime(time)`
  Converts the unix time (seconds elapsed from epoch) a CmonDateTime value.
- `CmonDateTime CmonDateTime::fromString(string)`
  Converts the string to a date&time format value. The recognized string formats are somewhat limited.
- `int CmonDateTime::timeZone()`
  Returns how many seconds must be added to the local time to get UTC (Coordinated Universal Time). CET for example is 1 hour ahead of UTC and so the return value is -3600.
- `int CmonDateTime::dayLight()`
  Returns how many seconds must be added to the local time because of the daylight saving time
- `CmonDateTime dateTime::toString([format])`
  Converts the date&time to string. The available formats are defined in the `cmon/io.h` header file. Here are some examples:

  When the `toString()` function is called on a CmonDateTime function and a format string is passed as the first argument the CmonDateTime will support all the format specifiers supported by the `strftime()` standard C library function. Please check the documentation of the `strftime()` for further details.

```
#include "cmon/io.h"

var dateTime = CmonDateTime::fromUnixTime(1424686003);

str01 = dateTime.toString(FileNameFormat);
str02 = dateTime.toString(ShortDayFormat);
str03 = dateTime.toString(LogFileFormat);
str04 = dateTime.toString(MySqlLogFileFormat);
str05 = dateTime.toString(MySqlShortLogFormat);
str05 = dateTime.toString(MySqlLogFileDateFormat);
str06 = dateTime.toString(MySqlShortLogDateFormat);
str07 = dateTime.toString(ShortTimeFormat);
str08 = dateTime.toString(LongTimeFormat);
str09 = dateTime.toString(ShortDateFormat);
str10 = dateTime.toString(LocalDateTimeFormat);
str11 = dateTime.toString(EmailDateTimeFormat);
```

(continues on next page)

(continued from previous page)

```
passed =
str01 == "2015-02-23_110643" &&
str02 == "150223" &&
str03 == "Feb 23 11:06:43" &&
str04 == "2015-02-23 11:06:43" &&
str05 == "2015-02-23" &&
str06 == "150223" &&
str07 == "11:06" &&
str08 == "11:06:43" &&
str09 == "02/23/15" &&
str10 == "Mon Feb 23 11:06:43 2015" &&
str11 == "Mon, 23 Feb 2015 11:06:43 +0100";
```

- `int dateTime.second()`
  Returns the 'seconds' part of the time.
- `int dateTime.minute()`
  Returns the 'minute' part of the time.
- `int dateTime.hour()`
  Returns the 'hours' part of the time.
- `int dateTime.hour()`
  Returns the 'hours' part of the time.
- `int dateTime.month()`
  Returns the month in the year between 1 and 12.
- `int dateTime.year()`
  Returns the year of the date like 2014.
- `int dateTime.weekday()`
  Sunday = 1, Monday = 2,... Saturday = 7.

CmonHost Tag Functions

Here is an example for the CmonHost tag functions. The variable host1 here has the CmonHost object type.

```
hosts       = cluster::hosts();
host1       = hosts[0];

hostName    = host1.hostName();
port        = host1.port();
```

- `string CmonHost::hostname()`
  Returns the name of the host as it was provided by the user. If the Cmon configuration file for example holds the hostname as IP address this function will return the hostname as a string representation of that address.
- `int CmonHost::port()`
  Returns the port number of the host. This is usually the port number of the SQL server.

- `int CmonHost::clusterId()`
  Returns the cluster ID of the cluster of the host. The Cluster ID is a unique ID number Cmon uses to identify the cluster.
- `string CmonHost::ipAddress()`
  Returns the string that holds the IPv4 address of the host.
- `boolean CmonHost::connected()`
  FIXME: Documentation.
- `string CmonHost::message()`
  Returns a human readable string that describes the status of the host.
- `string CmonHost::dataDir()`
  The data directory of the specified host instance.
- `string CmonHost::description()`
  FIXME: Documentation.
- `string CmonHost::distributionName()`
  Returns the name of the OS distribution running on the host.
- `string CmonHost::distributionCodeName()`
  Returns the code name of the OS distribution running on the host.
- `string CmonHost::distributionRelease()`
  Returns the release number of the OS distribution running on the host.
- `string CmonHost::nodeType()`
  Returns the nodeType of the host (controller, galera, mysql, postgresql ..)
- `string CmonHost::role()`
  Returns the role of the host (master, slave, …)
- `int CmonHost::pingDelay()`
  FIXME: Documentation.
- `string CmonHost::serverVersion()`
  For hosts running used as SQL servers returns the version number of the SQL server software, for a controller returns the Cmon software version number.
- `string CmonHost::toJSonString()`
  Converts the host to a JSON string.
- `map CmonHost::toMap()`
  Converts the host object to a map where all properties of the host are held and accessible with string keys. Convert the host to JSON message to see what properties are available and what keys are used.
- `string host.checkValue(type, value)`
  Checks the given value according the rules of the given alarm type, activates an alarm related to the given host or clears the alarm acordingly. Please check the Alarms section for examples.
- `string host.raiseAlarm(type, severity, [message])`
  Activates an alarm related to the given host. Please check the *Alarms* section for examples.
- string host.clearAlarm(type)
  Clears an alarm related to the given host. Please check the *Alarms* section for examples.
- `list host.alarms()`
  Returns all the active alarms related to the given host. Please check the *Alarms* section for examples.

- `map host.memoryInfo()`
  Returns a map with the latest memory information statistics of the host. The fields in the map are described in the CmonMemoryStats properties section. If the requested data is not collected for some reason the #N/A error is returned.

- `list host.memoryStats(startTime, endTime)`
  Returns a list with the memory information statistics of the host. The fields in the map are described in the CmonMemoryStats properties section. If the requested data is not collected for some reason the #N/A error is returned. See the *Obtaining and processing statistical information* section for some examples.

- `map host.sqlInfo()`
  Returns a map with the latest sql server statistics of the host. The fields in the map are described in the CmonSqlStats properties section. If the requested data is not collected for some reason the #N/A error is returned.

- `list host.sqlStats(startTime, endTime)`
  Returns a list with the sql server statistics of the host. The fields in the map are described in the CmonSqlStats properties section. If the requested data is not collected for some reason the #N/A error is returned. See the *Obtaining and processing statistical information* section for some examples.

- `list host.mongoStats(startTime, endTime)`
  Returns a list with the mongo server statistics of the host. The fields in the map are described in the CmonSqlStats properties section. If the requested data is not collected for some reason the #N/A error is returned. See the *Obtaining and processing statistical information* section for some examples.

- `list host.networkInfo()`
  Returns a list of maps, one list item for each monitored network interface. The fields in the map are documented in the CmonNetworkStats properties section.

- `list host.diskInfo()`
  Returns a list of maps, one list item for each monitored disk partition. The fields in the maps are documented in the CmonDiskStat properties section.

- `list host.diskStats(startTime, endTime, [devicename])`
  Returns a list of maps, one list item for each disk stats sample. If the third option is provided returns only samples from that device. The fields in the maps are documented in the CmonDiskStat properties section.

- `list host.cpuInfo()`
  Returns a list of maps, one list item for each CPU cores. The fields in the maps are documented in the CmonCpuStats properties section.

- `list host.cpuStats(startTime, endTime, [coreid])`
  Returns a list of maps, one list item for each CPU statistical sample in the given period. If the third argument is provided returns only samples for the cpu with the given ID. The fields in the maps are documented in the CmonCpuStats properties section.

- `map host.system(command)`
  Executes a shell command on the host. Returns a map that contains information about the return value and the standard output of the executed process.

```
function main()
{
var hosts      = cluster::hosts();
var host       = hosts[0];
```

(continues on next page)

(continued from previous page)

```
var retval;
retval = host.system("ls -lha /home");
if (!retval["success"])
{
    error("ERROR: ", retval["errorMessage"]);
}
print("Result: ", retval["result"]);
return retval["success"];
}
```

- `map host.sqlSystemVariables()`
  Returns the map of the cached SQL system variables, (`SHOW GLOBAL VARIABLES` for MySQL, `SHOW ALL` for PostgreSQL).
- `value host.sqlSystemVariable(name)`
  Returns the cached value of an SQL variable, (`SHOW GLOBAL VARIABLES` for MySQL, `SHOW ALL` for PostgreSQL).
- `map host.sqlStatusVariables()`
  Returns the map of the cached SQL system status variables, (`SHOW GLOBAL STATUS` for MySQL).
- `value host.sqlStatusVariable(name)`
  Returns the cached value of an SQL status (`SHOW GLOBAL STATUS` for MySQL).
- `CmonClusterConfig host.config([fileName])`
  Loads the configuration from the host (together with the include files and the files from the include directories) and returns a CmonClusterConfig object that holds all the information from the files.
- `map host.executeSqlQuery(query)`
  Executes the SQL query (an SQL expression that has return data, e.g. `SELECT`) and returns the results. The returned map will have a value for "success" to show if the operation was successful or not, an "errorMessage" that holds a human readable error message and a "result" field that holds an array with all the data the SQL server sent. Please check *Executing SQL commands and queries* section for some examples. A similar function (CmonDb::executeSqlQuery()) is available to execute an SQL query on the Cmon Database.
- `map host.executeMongoQuery(dbname, query)`
  Executes the mongo query (an JS expression that has return data, e.g. { serverStatus : 1 }) on the specified database (dbname) and returns the results. The returned map will have a value for "success" to show if the operation was successful or not, an "errorMessage" that holds a human readable error message and a "result" field that holds an map with all the data the mongo server sent.
- `map host.executeSqlCommand(sqlCommand)`
  Executes the SQL command (an SQL expression that has no return data, e.g. an INSERT or an UPDATE) and returns the status. The returned map will have a value for "success" to show if the operation was successful or not and an "errorMessage" that holds a human readable error message. Please check *Executing SQL commands and queries section* for some examples.
- `boolean host.sqlPing([timeout])`
  Executes a neutral SQL command (e.g. `SELECT` 1;) on the host to see if the SQL server up and able to run queries. Returns true if the SQL server returns a valid reply. If the argument is provided it controls how many seconds the the function will try to reach the server.

CmonMySqlHost Tag Functions

The CmonMySqlHost inherits all the properties and tag functions of the CmonHost.

- `boolean host.isGalera()`
  Returns true if the MySQL host is a Galera host.
- `ulonglong host.uptime()`
  Returns the 'uptime' status variable that shows the SQL server uptime in seconds.
- `boolean host.readOnly()`
  Returns the value of the 'read_only' SQL variable.

CmonClusterConfig Tag Functions

The CmonClusterConfig is a class that represents a set of configuration files found on one or more hosts of the cluster.

- `string config.errorMessage()`
  Returns a human readable error message that describe the state of the last operation.
- `list config.variable([variableName])`
  Returns a list of variables found in the configuration. If the variable name is not provided returns all the variables defined in the configuration file. Every list element is a Map that holds the following keys: "variablename", "linenumber", "value", "filepath" and "section".
- `value config.setVariable(section, variableName, value)`
  Sets the variable in the given section to the given value. If the variable or the section is not in the configuration it will be added. Please note that this function changes the configuration object, the change to has an effect the cluster configuration has to be saved.
- `map config.save()`
  Saves the configuration to the original host(s) using the original filename(s). The return map shall have the "success" and the "errorMessage" set to reflect if the operation was successful.

CmonAdvice Tag Functions

CmonAdvice is a class that represents an action to be taken by the administrator of the cluster advised by the advisor, a code that executed by the Cmon Controller. An advice is mostly constructed of human readable descriptions together with some information that help tracking where and when the advice was created.

- `void advice.setTitle(title)` Sets the title for the advice. The title should be a short description for the advice.
- `string advice.title()`
  Returns the title of the advice.
- `void advice.setCreator(creator)`
  Sets the name of the owner that created the advice. This is automatically set to the source file.
- string advice.creator()
  Returns the name of the creator.
- `void advice.setJustification(justification)`
  Sets the justification for the advice. The justification is a detailed description about the reason why the advisor decided there should be an action taken. The justification usually contain measured values if human readable form.

- `string advice.justification()`
  Returns the justification for the advice.
- `void advice.setAdvice(string)`
  Sets the detailed description of the advice.
- `string advice.advice()`
  Returns the detailed description of the advice.
- `void advice.setSeverity(Severity)`
  Sets the severity level for the advice. Severity level for an advice is the same as the severity levels for the alarms as it is defined in alarms.h.
- `Severity advice.severity()`
  Returns the severity level for the advice.
- `void advice.setHost(host)`
  Sets the host for the advice so the user will know which host was investigated when the advice was given.

When the `toString()` function is called the CmonAdvice supports the following format specifiers:

- `%t` – The title of the advice.
- `%j` – The justification for the advice.
- `%a` – The description, the advice itself.
- `%c` – The creator of the advice.
- `%h` – The name of the host if there is a host set for the advice.
- `%E` – A multi line description of the advice that contain multiple properties.

CmonJob Functions

The CmonJob type represents a job or a task that the Cmon controller can execute. These jobs are usually executed asynchronously. The script creates a job, stores all the necessary information in the CmonJob object, and pushes it into the execution queue. Then the controller executes the job and sends its results to the UI where the user can check what happened.

The CmonJob type is supported from version 1.2.11 of Cmon.

General Job Functions

- `Bool job.enqueue()`
  Checks the job for consistency and sends it to the execution queue. Returns true if everything went well.
- `String job.errorString()`
  Returns the human-readable error string describing the error or the empty string if there were no errors.
- `Int job.jobId()`
  Returns the unique numerical ID for the job. Only jobs that are enqueued for execution has valid (greater than 0) IDs.
- `CmonJob CmonJob::getJob(jobid)`
  Reads the job with the specified job ID from the Cmon database. Returns `#N/A` if the job was not found.

Backup Handling

The following methods are related to jobs that create backups.

- `CmonJob CmonJob::createBackupJob(host, dir)`
  This function creates a CmonJob that will create a backup of the data found in a database or in all databases of a specific database server.
- `CmonJob CmonJob::createDoCheckJob()`
  Creates a job that will trigger some checks on the controller. These checks can be used to see if there are duplicate indexes, missing indexes, database schema issues on the production system. More checks will be added later.
- `Bool job.setBackupMethod(method)`
  Sets what kind of backup will be created, what software will be used to create the backup file. Returns true if the job is valid, all the properties are in order. Currently the following methods are supported:
- - null – If the backup method is missing one will be chosen.
  - "auto" – To use the default backup software for the given cluster type. (This means mysqldump now).
  - "none" – This is the same as "auto".
  - "mysqldump" – Use the mysqldump program to create a backup.
  - "xtrabackupfull" – Use the xtrabackup program to create a full (not incremental) backup.
  - "xtrabackupincr" – Use the xtrabackup program to create an incremental backup.
  - "pgdump" – Backup method for PostgreSQL hosts using the pg_dump or pg_dumpall programs.
- `Bool job.setCompression(compression)`
  Sets if the created backup file should be compressed.
- `Bool job.setIncludeDatabases(value)`
  Sets which databases should be processed.
- `Bool job.setIsCcStorage(value)`
  Sets if the Cmon Controller should store the backup file.
- `Bool job.setNetcatPort(value)`
  Sets the netcat port that is used when copying the backup file through the network.

The following example shows how easy and simple to create a job that will create a backup of the data found on one specific host.

```
//
// This program will create a backup job using two strings, one for␣
↪hostname and
// one for directory name. Then the backup method is set (teh default value␣
↪is
// "auto") and the job is send for execution.
//
var job = CmonJob::createBackupJob("127.0.0.1", "/var/tmp");
var passed;
job.setBackupMethod("mysqldump");
```

(continues on next page)

(continued from previous page)

```
passed = job.enqueue();
if (passed !== true)
{
    error("ERROR: ", job.errorString());
    exit(false);
}
```

Cluster Configuration Jobs

Some functions change the state or configuration of the cluster. Adding and removing nodes, starting and stopping nodes or the entire cluster are the most important jobs in this section.

- `CmonJob CmonJob::createAddNodeJob(hostName, [install], [configFile])`

Creates a job that ultimately will add a new node to the cluster. The node is identified by the host name passed as the first argument. If the second argument is true the database software is also installed on the new node and so the third argument must be the file name of the configuration file template.

CmonGraph Tag Functions

A CmonGraph consists of one or more plots. These plots are usually shown as lines, lines with points on them or bars to represent a number of values in a two dimensional (x/y) coordinate system. Various properties of these plots can be set using the plot index, that is a number referencing the plots from 1 to the last plot.

- `void advice.setTitle(title)`
  Sets the text that is shown on the top of the graph image.
- `boolean graph.setSize(width, height)`
  Sets the size of the generated graph in pixels.
- `boolean graph.setXDataIsTime([boolean])`
  Sets if the X axis values should be printed as date and time value.
- `boolean graph.setPlotLegend(plotIdx, legend)`
  Sets the text that is shown for the given plot as legend.
- `boolean graph.setPlotColumn(plotIdx, xColumn, yColumn)`
  Sets which data array column is used as data for the given plot.
- `boolean graph.setPlotStyle(plotIdx, style)`
  Sets what style will be used to plot the data. Check the `cmon/graph.h` include file for the available styles.

Cluster Functions

- `array cluster::hosts()`
  Returns all the CmonHosts that are considered as part of he cluster. This function also returns the host of the Cmon controller.
- `array cluster::mySqlNodes()`
  Returns all the CmonMySqlHosts that are considered as part of the cluster. The CmonMySqlHost inherits the properties and functions of the CmonHost, so where a CmonHost can be used a CmonMySqlHost is also accepted (e.g. CmonHost::executeSqlQuery() also works for CmonMySqlHost).

- `array cluster::galeraNodes()`
  Returns the Galera nodes of the cluster. The returned list holds CmonGaleraHost type items.
- `array cluster::postgreSqlNodes()`
  Returns the MySQL nodes of the cluster. The returned list holds CmonMySqlHost type items.
- `array cluster::mongoNodes()`
  Returns the MongoDb nodes of the cluster. The returned list holds CmonMongoNode type items.
- `array cluster::maxscaleNodes()`
  Returns the MaxScale nodes of the cluster. The returned list holds CmonMaxScaleHost type items.
- `array cluster::ndbdNodes()`
  Returns the NDB nodes of the cluster. The returned list holds CmonNdbHost type items.
- `string cluster::statustext()`
  Returns the human readable description of the cluster status.
- `int cluster::state()`
  FIXME: documentation.
- `bool cluster::rollingRestart()`
  Restarts the nodes without stopping the cluster.

Cmon Functions

- `text cmon::version()`
  Returns the Cmon version as a string.
- `text cmon::build()`
  Returns the Cmon build number as a string.
- `number cmon::uptime()`
  Returns how many seconds ago Cmon started to manage this cluster.
- `boolean cmon::running()`
  Returns true if Cmon is managing this cluster.
- `text cmon::hostname()`
  Returns the hostname of the computer running the Cmon controller.
- `text cmon::domainname()`
  Returns the domain name of the computer running the Cmon controller.

General alarm functions

- int Alarm::alarmId(category, isHost, title, message, recommendation)
  Registers a new alarm type if an alarm type with the same properties is not registered already. Returns the alarm type that can be used to raise and clear alarms with these properties. Use this function to implement custom alarms for the Cmon system. Please check the *Alarms* section for examples.
- `int Alarm::checkId(category, isHost, warningLevel, criticalLevel, title, message, recommendation)`
  Registers a new check type if a check type with the same properties is not registered already. Checks are in reality alarms that have warning and critical levels so they can easily be used to check numerical values. Returns the alarm type that can be used to raise and clear alarms with these properties calling the `host.checkValue()` function.

Mail functions

- `Map Mail::sendMail(subject, body, [component])`
  Appends a new mail message to the outgoing folder of the Cmon system. The third (optional) argument controls the component which will ultimately be used to decide what recipients will get the email. The possible values are defined in the ComponentType enum in the `cmon/alarms.h` file.

License functions

- `text license::statustext()`
  Returns a short string describing the status of the license.
- `bool license::status()`
  Returns true if there is a valid license for the cluster.
- `int license::expires()`
  A negative value indicates the Cmon license expired or not found, positive values show how many days the license has left.

Cmon Database functions

The Cmon Database is the SQL database where the Cmon stores all its internal data. This database is accessible from the JS programs.

- `Map CmonDb::executeSqlQuery(query)`
  Executes the SQL query on the Cmon Database and returns the results. The returned map will have values for the keys "success", "errorMessage" and "result" where the value for the "result" is a two-dimensional array that holds the values from the SQL query.

3.13.4 Examples

Executing SQL commands and queries

The following example demonstrates how to execute an SQL query on an arbitrary host of a cluster and receive the results in an array. The return value of the `host.executeSqlQuery()` holds the success/failed status of the query, the error message and also the results in a two dimensional array.

```
function getSqlVariable(host, variableName)
{
  var query = "SHOW GLOBAL STATUS LIKE '$1'";
  var retval;
  var value;
  if (host.typeName() != "CmonHost")
    return #ARGS!;
  query.replace("$1", variableName);
  retval = host.executeSqlQuery(query);
  if (!retval["success"])
  {
    print("ERROR:", retval["errorMessage"]);
    return #N/A;
  }
  value = retval["result"][0, 1];
  if (value.looksInteger())
    return value.toInt();
```

(continues on next page)

(continued from previous page)

```
  else if (value.looksULongLong())
    return value.toULongLong();
  else if (value.looksDouble())
    return value.toDouble();
  return value;
}
function main()
{
  var hosts = cluster::hosts();
  var value = getSqlVariable(hosts[0], "COM_SELECT");
  print("*** value: ", value);
  return value.isInt();
}
```

Here is an example that shows how to execute an SQL query on the Cmon Database:

```
var retval = CmonDb::executeSqlQuery("select * from mysql_states;");
var passed = true;
if (!retval["success"])
{
  error("Executing SQL query failed: ", retval["errorMessage"]);
}

for (idx = 0; idx < retval["result"].rows(); ++idx)
{
  var string = retval["result"][idx, 2];
  print(retval["result"][idx, 2]);
  if (string.empty())
  {
    error("Value at idx = ", idx, " is empty.");
  }
}
```

Executing shell commands

```
function listFiles(host)
{
  var retval = host.system("ls -lha /home");
  if (!retval["success"])
    error("ERROR: ", retval["errorMessage"]);
  print("Host    : ", host.hostName());
  print("Result  : ", retval["result"]);
  print("Success : ", retval["success"]);
  return retval["success"];
}
```

Obtaining and processing statistical information

The following example shows how to obtain statistical data from a specific host about a specific time interval and how to process the data using low-level indexing operators. For the most task there are more efficient high-level statistical functions that can be used without looping through the data, but low-level access can be also beneficial for custom calculations.

```
//
// Going through the memory statistics of the last 10 minutes and printing␣
↪the
// size of the free memory with the time.
//
#include "cmon/io.h"
function toGigaBytes(value)
{
  return value / (1024 * 1024 * 1024);
}
function main()
{
  var host = cluster::hosts()
[0];
  var endTime = CmonDateTime::currentDateTime();
  var startTime = endTime - 10 * 60;
  var stats = host.memoryStats(startTime, endTime);
  var retval = true;
  for (idx = 0; idx < stats.size(); ++idx)
  {
    map = stats[idx];
    created = CmonDateTime::fromUnixTime(map["created"]);
    ramfree = toGigaBytes(map["ramfree"]);
    print(
      created.toString(LongTimeFormat),
      " ", ramfree.toString(TwoDecimalNumber), "GBytes");
  }
  return retval;
}
```

The next example shows a sophisticated recipe to process some of the statistical data using high-level statistical functions. It first gets the data calling the `host.memoryStats()` function, then it filters all the memory utilization information into an array. This array then can be processed by statistical functions like `min(),max()` or `percentile()`.

```
//
// Printing the min, the ninth percentile and the max of the memory␣
↪utilization
// in the last ten minutes for every host. Prints something like this:
//
// MEMORY UTILIZATION
// HOST MIN NINTH MAX
// 127.0.0.1 42.00% - 42.58% - 42.64%
//
```

(continues on next page)

(continued from previous page)

```
#include "cmon/io.h"
function printUtil(host, startTime, endTime)
{
  var list = host.memoryStats(startTime, endTime);
  var array = list.toArray("memoryutilization");
  var min = min(array);
  var max = max(array);
  var ninth = percentile(array, 0.9);
  print(host.hostName(),
    " ",
    min.toString(TwoDecimalPercent), " - ",
    ninth.toString(TwoDecimalPercent), " - ",
    max.toString(TwoDecimalPercent));

  return true;
}
function main()
{
  var endTime = CmonDateTime::currentDateTime();
  var startTime = endTime - 10 * 60;
  var hosts = cluster::hosts();

  print(" MEMORY UTILIZATION ");
  print("HOST MIN NINTH MAX");
  for (idx = 0; idx < hosts.size(); ++idx)
    printUtil(hosts[idx], startTime, endTime);
  return true;
}
```

Alarms

```
#include "cmon/alarms.h"
var hosts   = cluster::hosts();
var host    = hosts[0];
var alarms;
var found   = false;
//
// Raising an alarm
//
host.raiseAlarm(MySqlAdvisor, Critical, "Some message.");
//
// Reading the active alarms and searching for the same alarm.
//
alarms = host.alarms();
for (idx = 0; idx < alarms.size(); ++idx)
{
  if (alarms[idx]["title"] == "MySQL advisor alarm")
  {
    found = true;
```

(continues on next page)

(continued from previous page)

```
    break;
  }
}
The following example demonstrates how to create a custom alarm type and␣
↪raise an alarm with the custom alarm.

//
// Demonstrating custom alarms.
//
#include "cmon/alarms.h"
//
// This function returns an alarm type for a custom alarm with some
// properties encoded into the function.
//
function myAlarm()
{
  return Alarm::alarmId(
        Node, true,
        "Computer is on fire",
        "The computer is on fire, it is on flames.",
        "Pour some water on it.");
}
var myAlarmId = myAlarm();
var hosts     = cluster::hosts();
var host      = hosts[0];
var sentMessage;
sentMessage = host.raiseAlarm(myAlarmId, Critical);
//
// An alarm is raised and sentMessage should be:
// Server 127.0.0.1 reports: The computer is on fire, it is on flames.
//
```

Configuration files

```
function getConfiguredClientPort(host)
{
  var config      = host.config();
  var variable    = config.variable("port");
  for (idx = 0; idx < variable.size(); ++idx)
  {
    print("*** section  : ", variable[idx]["section"]);
    print("*** value    : ", variable[idx]["value"]);
    print("*** location : ",
          variable[idx]["filepath"], ":", variable[idx]["linenumber"]);
    if (variable[idx]["section"] == "client")
        return variable[idx]["value"].toInt();
  }
  return #N/A;
}
```

Creating graphs

The following example shows how to create a graph, set up with statistical data and return to the UI to be shown as an image.

```
//
// This is a test program that prints a graph on the sql statistics.
//
#include "cmon/graph.h"
var hosts     = cluster::hosts();
var host      = hosts[0];
var endTime   = CmonDateTime::currentDateTime();
var startTime = endTime - 10 * 60;
var stats     = host.sqlStats(startTime, endTime);
var array     = stats.toArray(
  "created,interval,COM_SELECT,COM_INSERT");
//
// Calculating some values from the statistics
//
for (idx = 0; idx < array.columns(); idx++)
{
  array[5, idx] = 1000 * array[2, idx] / array[1, idx];
  array[6, idx] = 1000 * array[3, idx] / array[1, idx];
}
var graph     = new CmonGraph;
graph.setXDataIsTime();
graph.setTitle("SQL Statistics " + host.toString());
graph.setSize(800, 600);
// This graph contains two plots, we set the various properties for them␣
↪here
// here. The plot index will be 1 and 2.
graph.setPlotLegend(1, "Select (1/s)");
graph.setPlotColumn(1, 0, 5);
graph.setPlotStyle(1, Impulses);
graph.setPlotLegend(2, "Insert (1/s)");
graph.setPlotColumn(2, 0, 6);
graph.setPlotStyle(2, Impulses);
graph.setData(array);
exit(graph);
```

The following example shows how to interact with MongoDB:

```
// Mongo JS example
function main()
{
    var hosts   = cluster::mongoNodes();
    for (i = 0; i < hosts.size(); i++)
    {
        host        = hosts[i];
        print(host.hostName());
        var config      = host.config();
```

(continues on next page)

(continued from previous page)

```
        var variable    = config.variable("port");
        for (idx = 0; idx < variable.size(); ++idx)
        {
            print("*** section  : ", variable[idx]["section"]);
            print("*** value    : ", variable[idx]["value"]);
            print("*** location : ",
                    variable[idx]["filepath"], ":", variable[idx][
↪"linenumber"]);
            if (variable[idx]["section"] == "client")
                return variable[idx]["value"].toInt();
        }


        var res= host.executeMongoQuery("{ serverStatus : 1 }");
        print(res["result"]["host"]);
        print(res["result"]["storageEngine"]["name"]);

        var endTime = CmonDateTime::currentDateTime();
        var startTime = endTime - 10 * 60;
        var stats     = host.mongoStats(startTime, endTime);
        var array     = stats.toArray(
↪"created,interval,opcounters.command");
        for (idx = 0; idx < array.columns(); idx++)
        {
            var x= 1000 * array[2, idx] / array[1, idx];
            //print(x);
        }
        break;
    }
}
```

3.14 ClusterControl Client SDK

ClusterControl has a HTTPS based RPC API that provides complete access and control of the CMON Controller which is its workflow engine. The ClusterCOntrol CMON API documentation is readily available *here*.

Using the CMON RPCv2 API directly though requires that clients are passing JSON documents of jobs specs to be executed. Creating these CMON JSON job specs are straightforward however cumbersome given its text based nature where correct syntax and manual validations is needed in order for the jobs to be executed.

A more user (developer) friendly way of using the CMON RPCv2 API is to use the OpenAPI specification to document and generate API client libraries for various system languages. The OpenAPI documentation is machine readable and makes it easy formally describe an API and also use tools such as the openapi-generator to generate client libraries.

You can use your favourite programming language such as Go or Python to create applications which integrates with ClusterControl to manage and monitor your databases.

This project is hosted in Github called clustercontrol-client-sdk.

3.14.1 SDK for various programming languages

ClusterControl API (v2) SDK has bindings for various programming languages.

API binding for Java

API binding for Java is available in the `java` subdirectory. It requires Apache Maven to build. Here are the build instructions.

Setting up Apache Maven Here are instructions on how to setup Maven.

1. Download the latest maven from here and extract it to your machine.
2. Install JDK 8 and set `JAVA_HOME` to the directory where JDK is installed.

```
export JAVA_HOME=<jdk-install-dir>
```

3. Update `PATH` environment variable to include the Maven and JDK `bin` directories.

```
export PATH=<maven-bin-dir>:$JAVA_HOME/bin:$PATH
```

4. Building Java bindings:

```
git clone https://github.com/severalnines/clustercontrol-client-sdk.git
cd clustercontrol-client-sdk/java
mvn -DskipTests clean package
```

We also provide an API Abstraction Layer above the underlying low-level API bindings. This makes it super easy to use the API SDK. See the instructions here.

API binding for Python

API binding for Python is available in the `python` subdirectory.

We hope to provide in the very near future an API Abstraction Layer above the underlying low-level API bindings. This makes it super easy to use the API SDK. See the instructions here.

API binding for GoLang

API binding for GoLang is available in the `go` subdirectory.

We hope to provide in the very near future an API Abstraction Layer above the underlying low-level API bindings. This makes it super easy to use the API SDK. See the instructions here.

3.14.2 Generating Sources from the API definition

Note

Note It is not required for you to generate new sources from the API definition. However, if you happend to modify the API definition, then you will have to generate new sources from the definition in order for the sources to reflect changes to the definition.

1. Install Java Runtime 8. Skip these steps if you already have the latest generated language bindings:
   - Follow the instructions for Linux.

- Follow the instructions for MacOS.

2. Download and setup the openapi-generator:

```
$ cd clustercontrol-client-sdk
$ mkdir -p $PWD/bin/openapitools
$ curl -o $PWD/bin/openapitools/openapi-generator-cli.sh␣
↪https://raw.githubusercontent.com/OpenAPITools/openapi-generator/master/bin/utils/
$ chmod +x $PWD/bin/openapitools/openapi-generator-cli.sh
$ export PATH=$PATH:$PWD/bin/openapitools
```

3. Install Apache Maven. Follow the instructions here for your favourite OS:

```
$ curl -o apache-maven-3.8.6-bin.tar.gz␣
↪https://dlcdn.apache.org/maven/maven-3/3.8.6/binaries/apache-maven-3.8.6-bin.tar.g
$ tar xzvf apache-maven-3.8.6-bin.tar.gz
$ export PATH=apache-maven-3.8.6/bin:$PATH
```

4. Check if `openapi-generator.sh` is setup properly:

```
$ openapi-generator.sh version
6.0.1
```

5. Generate API language bindings:

```
# install make with 'apt install make'
#make python
#make java
make go
...
[main] INFO  o.o.codegen.TemplateManager - writing file /
↪root/clustercontrol-client-sdk/./generated/go/.openapi-generator/FILES
################################################################################
↪#########
# Thanks for using OpenAPI Generator.                                          ␣
↪          #
# Please consider donation to help us maintain this project                    ␣
↪       #
# https://opencollective.com/openapi_generator/donate                          ␣
↪          #
################################################################################
↪#########
```

The next step is to write your first applicaton which calls upon these generated client API stubs. Checkout the respective binding subdiirectories for `java`, `python` and `go`.

The GitHub repository contains examples of deploying database clusters of various types (MySQL/MariaDB replication or galera with ProxySQL, PostgreSQL replication, MongoDB replica set and/or sharded, Redis sentinel, Microsoft SQL server, and Elasticsearch)

Navigate to the repository docs folder for generated documentation on the terraform provider plugin for ClusterControl.

The sub-folders contain examples of the following:

Database type	Description
MySQL/MariaDB	MySQL and/or MariaDB database (both Master/Slave and Galera multi-master)
ProxySQL	ProxySQL load balancer with MySQL/MariaDB database clusters
PostgreSQL	Postgres (Primary with Hot-Standby clusters)
MongoDB	Both sharded clusters and single Replicaset clusters
Redis	Redis sentinel clusters
Microsoft SQL Server	Both standalone and hot-standby cluster with one hot-standby (async)
Elasticsearch	Elasticsearch clusters

3.15 Provider Configuration

Use the following to configure the Terraform provider for ClusterControl:

```
terraform {
  required_providers {
    clustercontrol = {
      source = "severalnines/clustercontrol"
      version = "0.2.15"
    }
  }
}

provider "clustercontrol" {
  # Configuration options:
  # cc_api_url = ""
  # cc_api_user = ""
  # cc_api_user_password = ""
}
```

3.16 Resources

Name
clustercontrol_db_cluster
clustercontrol_db_cluster_backup
clustercontrol_db_cluster_backup_schedule
clustercontrol_db_cluster_maintenance

3.16.1 clustercontrol_db_cluster

Required

- `db_admin_user_password` (String, Sensitive) Password for the admin/root user for the database. Note that this may show up in logs, and it will be stored in the state file
- `db_cluster_name` (String) The name of the database cluster.

- db_cluster_type (String) Type of cluster – replication, galera, postgresql_single (single is a misnomer), etc
- db_vendor (String) Database vendor – oracle, percona, mariadb, 10gen, microsoft, redis, elasticsearch, for postgresql it is defaultetc
- db_version (String) The database version
- ssh_key_file (String) SSH Key file. The path to the private key file for the Sudo user on the ClusterControl host
- ssh_user (String) The SSH user ClusterControl will use to SSH to the DB host from the ClusterControl host

Optional

- db_admin_username (String) Name for the admin/root user for the database
- db_auto_recovery (Boolean) Have cluster auto-recovery on (or off)
- db_cluster_create (Boolean) Whether to create this resource or not?
- db_cluster_import (Boolean) Whether to import this resource or not?
- db_config_server (Block List) Specification for the MongoDB Configuration Server. (see *below for nested schema*)
- db_data_directory (String) The data directory for the database data files. If not set explicily, the default for the respective DB vendor will be chosen
- db_deploy_agents (Boolean) Automatically deploy prometheus and other relevant agents after setting up the intial DB cluster.
- db_elasticsearch_http_port (String) The port on which Elasticsearch will accept client http connections
- db_elasticsearch_transfer_port (String) The port on which Elasticsearch will accept client connections for data transfer(?)
- db_enable_pbm_agent (Boolean) Enable percona backup for mongodb.
- db_enable_pgbackrest_agent (Boolean) Enable PgBackRest for Postgres.
- db_enable_ssl (Boolean) Enable SSL based comms between the cluster nodes and client access to node.
- db_enable_timescale (Boolean) Whether to setup TimescaleDB extension or not
- db_enable_uninstall (Boolean) When removing DB cluster from ClusterControl, enable uinstalling DB packages.
- db_host (Block List) The list of nodes/hosts that make up the cluster (see *below for nested schema*)
- db_install_software (Boolean) Install DB packages from respective repos
- db_load_balancer (Block List) The list of nodes/hosts that make up the cluster (see *below for nested schema*)
- db_mongo_auth_db (String) The mongodb database to use for authentication purposes
- db_mongo_config_server_port (String) The port on which MongoDB config server will accept client connections. MongoS server will use same port number as db_mongo_port

- `db_mongo_port` (String) The port on which MongoDB will accept client connections
- `db_mongos_server` (Block List) Specification for the MongoDB mongos Server. (see *below for nested schema*)
- `db_mssqlserver_port` (String) The port on which MSSQL will accept client connections
- `db_mysql_port` (String) The port on which MySQL will accept client connections
- `db_pbm_backup_dir` (String) Backup dir, nfs mounted directory/path for PBM backup.
- `db_postgres_port` (String) The port on which PostgreSql will accept client connections
- `db_redis_port` (String) The port on which Redis will accept client connections
- `db_replica_set` (Block List) The hosts that make up the cluster. (see *below for nested schema*)
- `db_semi_sync_replication` (Boolean) Semi-synchronous replication for MySQL and MariaDB non-galera clusters
- `db_sentinel_port` (String) The port Redis Sentinel uses to communicate
- `db_snapshot_host` (String) Elasticsearch snapshot host
- `db_snapshot_location` (String) Elasticsearch snapshot location
- `db_snapshot_repository` (String) Elasticsearch snapshot repository
- `db_tags` (Set of String) Tags to associate with a DB cluster. The tags are only relevant in the ClusterControl domain.
- `db_topology` (Block List) Only applicable to MySQL/MariaDB non-galera clusters. A way to specify Master and Slave(s). (see *below for nested schema*)
- `disable_firewall` (Boolean) Disable firewall on the host OS when installing DB packages.
- `disable_selinux` (Boolean) Disable SELinux on the host OS when installing DB packages.
- `ssh_port` (String) The ssh port.
- `ssh_user_password` (String, Sensitive) Sudo user's password. If sudo user doesn't have a password, leave this field blank

Read-Only

- `db_cluster_id` (String) TODO
- `id` (String) The ID of this resource.
- `last_updated` (String) TODO

Nested Schema for `db_config_server`

Required:

- `member` (Block List, Min: 1) The host that make up the replicaset member. (see *below for nested schema*)
- `rs` (String) The replicaset's name.

Nested Schema for `db_config_server.member`

Required:

- `hostname` (String) Hostname of the DB host. Can be IP address as well.

Optional:

- `hostname_data` (String) Hostname/IP used for data comms (may be legacy ClusterControl).
- `hostname_internal` (String) If there's a private net that all DB hosts can communicate, use it here.

Nested Schema for `db_host`

Required:

- `hostname` (String) Hostname of the DB host. Can be IP address as well

Optional:

- `datadir` (String) The data directory for the database data files. If not set explicily, default for the DB type will be used, or inherited from earlier/top-level specification.
- `hostname_data` (String) Hostname/IP used for data comms (may be legacy ClusterControl).
- `hostname_internal` (String) If there's a private net that all DB hosts can communicate, use it here.
- `protocol` (String) TODO.
- `roles` (String) Applicable to Elasticsearch – the role of this host (master-data: host will be designated as the master node and a data node, etc)
- `sync_replication` (Boolean) Applicable to PostgreSQL hot-standby nodes only. Use synchronous replication (or not)

Nested Schema for `db_load_balancer`

Required:

- `db_lb_admin_user_password` (String, Sensitive) The load balancer admin user's password
- `db_lb_monitor_user_password` (String, Sensitive) The load balancer monitor user's password
- `db_lb_type` (String) The load balancer type (e.g., proxysql, haproxy, etc)
- `db_lb_version` (String) Software version
- `ssh_user` (String) The SSH user ClusterControl will use to SSH to the DB host from the ClusterControl host

Optional:

- `db_lb_admin_port` (String) The load balancer admin port that will be used to administer it.
- `db_lb_admin_username` (String) The load balancer admin user
- `db_lb_enable_uninstall` (Boolean) When removing DB cluster from ClusterControl, enable uinstalling DB packages.
- `db_lb_install_software` (Boolean) Install DB packages from respective repos
- `db_lb_monitor_username` (String) The load balancer monitor user (only applicable to proxysql)

- `db_lb_port` (String) The load balancer port that it will accept connections on behalf of the database it is front-ending.
- `db_lb_use_clustering` (Boolean) Whether to use ProxySQL clustering or not. Only applicable to ProxySQL at this time
- `db_lb_use_rw_splitting` (Boolean) Whether to Read/Write splitting for queries or not?
- `db_my_host` (Block List) The load balancer host in question (i.e, self) (see *below for nested schema*)
- `disable_firewall` (Boolean) Disable firewall on the host OS when installing DB packages.
- `disable_selinux` (Boolean) Disable SELinux on the host OS when installing DB packages.
- `ssh_key_file` (String) SSH Key file. The path to the private key file for the Sudo user on the ClusterControl host.
- `ssh_port` (String) The ssh port.
- `ssh_user_password` (String, Sensitive) Sudo user's password. If sudo user doesn't have a password, leave this field blank

Nested Schema for `db_load_balancer.db_my_host`

Required:

- `hostname` (String) Hostname/IP of this load balancer. Can be IP address as well.

Nested Schema for `db_mongos_server`

Required:

- `hostname` (String) Hostname of the DB host. Can be IP address as well.

Optional:

- `hostname_data` (String) Hostname/IP used for data comms (may be legacy ClusterControl).
- `hostname_internal` (String) If there's a private net that all DB hosts can communicate, use it here.

Nested Schema for `db_replica_set`

Required:

- `member` (Block List, Min: 1) The hosts that make up the replicaset HA nodes. (see *below for nested schema*)
- `rs` (String) The replicaset's name.

Nested Schema for `db_replica_set.member`

Required:

- `hostname` (String) Hostname of the DB host. Can be IP address as well

Optional:

- `arbiter_only` (Boolean) The host is acting as an arbiter only.
- `hidden` (Boolean) TODO.

- `hostname_data` (String) Hostname/IP used for data comms (may be legacy ClusterControl).
- `hostname_internal` (String) If there's a private net that all DB hosts can communicate, use it here.
- `priority` (Number) Priority of the host in the mongo replication setup.
- `slave_delay` (String) Used in non-galera MySQL/MariaDB standby setup. Specifies the lag for the slave.

Nested Schema for `db_topology`

Optional:

- `primary` (String) The Master host
- `replica` (String) The Slave host.

3.16.2 clustercontrol_db_cluster_backup

Required

- `db_backup_method` (String) mariabackup, xtrabackup, …
- `db_cluster_id` (String) The database cluster ID for which this LB is being deployed to.

Optional

- `db_backup_compression` (Boolean) Whether to compress backups or not
- `db_backup_compression_level` (Number) Compression level
- `db_backup_dir` (String) Base direcory where backups is to be stored
- `db_backup_encrypt` (Boolean) Whether to encrypt or not
- `db_backup_failover_host` (String) If backup failover is enabled, which host to use as backup host in the event of failure of first choice host.
- `db_backup_host` (String) Where there are multiple hosts, which host to choose to create backup from.
- `db_backup_retention` (Number) Backup retention period in days
- `db_backup_storage_controller` (Boolean) Whether to store the backup on CMON controller host or not
- `db_backup_storage_host` (String) Which host to store the backup on. Typically, used with mongodump backup method.
- `db_backup_subdir` (String) Sub-dir for backups – default: "BACKUP-%I"
- `db_backup_system_db` (Boolean) Whether to enable backup failover to another host in case the host crashes
- `db_enable_backup_failover` (Boolean) Whether to enable backup failover to another host in case the host crashes

Read-Only

- `db_resource_id` (String) TODO
- `id` (String) The ID of this resource.
- `last_updated` (String) TODO

3.16.3 clustercontrol_db_cluster_backup_schedule

Required

- `db_backup_method` (String) mariabackup, xtrabackup, …
- `db_backup_sched_time` (String) The time to kick off a backup (e.g. 'TZ=UTC 0 0 * * *')
- `db_cluster_id` (String) The database cluster ID for which this LB is being deployed to.

Optional

- `db_backup_compression` (Boolean) Whether to compress backups or not
- `db_backup_compression_level` (Number) Compression level
- `db_backup_dir` (String) Base direcory where backups is to be stored
- `db_backup_encrypt` (Boolean) Whether to encrypt or not
- `db_backup_failover_host` (String) If backup failover is enabled, which host to use as backup host in the event of failure of first choice host.
- `db_backup_host` (String) Where there are multiple hosts, which host to choose to create backup from.
- `db_backup_retention` (Number) Backup retention period in days
- `db_backup_sched_title` (String) A title for the backup schedule (e.g., Daily full, Hourly incremental, etc)
- `db_backup_storage_controller` (Boolean) Whether to store the backup on CMON controller host or not
- `db_backup_storage_host` (String) Which host to store the backup on. Typically, used with mongodump backup method.
- `db_backup_subdir` (String) Sub-dir for backups – default: "BACKUP-%I"
- `db_backup_system_db` (Boolean) Whether to enable backup failover to another host in case the host crashes
- `db_enable_backup_failover` (Boolean) Whether to enable backup failover to another host in case the host crashes

Read-Only

- `db_resource_id` (String) TODO
- `id` (String) The ID of this resource.
- `last_updated` (String) TODO

3.16.4 clustercontrol_db_cluster_maintenance

Required

- `db_cluster_id` (String) The database cluster ID for which this LB is being deployed to.
- `db_maint_start_time` (String) Format: `Jan-02-2006T15:04`
- `db_maint_stop_time` (String) Format: `Jan-02-2006T15:04`

Optional

- `db_maint_reason` (String) TODO

Read-Only

- `db_resource_id` (String) TODO
- `id` (String) The ID of this resource.
- `last_updated` (String) TODO

3.17 Common fields in the resource definition

3.17.1 Resource – clustercontrol_db_cluster

`db_host`

The `db_host` block inside the `clsutercontrol_db_cluster` resource specifies the hosts that make up the cluster. Each host that makes up the DB cluster should have one of these blocks. The mandatory attribute for each `db_host` block is the **hostname**.

Example:

```
resource "clustercontrol_db_cluster" "this" {
    ...
    db_host {
        hostname = "host-1"
    }
    db_host {
        hostname = "host-2"
    }
    ...

}
```

3.17.2 Scheduling Backups using the clustercontrol_db_cluster_backup_scheduler resource

The backup schedule resource allows you to create a backup schedule for a cluster in ClusterControl through the terraform provider. Here's an example of a daily full backup schedule using `xtrabackup`. As can be seen the `clustercontrol_db_cluster_backup_schedule`resource depends on the `clustercontrol_db_cluster` resource.

```
resource "clustercontrol_db_cluster_backup_schedule" "full-1" {
  depends_on                    = [clustercontrol_db_cluster.this]
  db_backup_sched_title         = "Daily full"
  db_backup_sched_time          = "TZ=UTC 0 0 * * *"
  db_cluster_id                 = clustercontrol_db_cluster.this.id
  db_backup_method              = "xtrabackupfull"
  db_backup_dir                 = var.db_backup_dir
  db_backup_subdir              = var.db_backup_subdir
  db_backup_encrypt             = var.db_backup_encrypt
  db_backup_host                = var.db_backup_host
  db_backup_storage_controller = var.db_backup_storage_controller
  db_backup_compression         = var.db_backup_compression
  db_backup_compression_level   = var.db_backup_compression_level
  db_backup_retention           = var.db_backup_retention
}
```

3.17.3 Taking adhoc backups using the clustercontrol_db_cluster_backup resource

You can a maintenance window for a cluster using the `clustercontrol_db_cluster_backup` resource. Here's an example of a full backup using `xtrabackup`.

```
resource "clustercontrol_db_cluster_backup" "full-1" {
  depends_on                    = [clustercontrol_db_cluster.this]
  db_cluster_id                 = clustercontrol_db_cluster.this.id
  db_backup_method              = "xtrabackupfull"
  db_backup_dir                 = var.db_backup_dir
  db_backup_subdir              = var.db_backup_subdir
  db_backup_encrypt             = var.db_backup_encrypt
  db_backup_host                = var.db_backup_host
  db_backup_storage_controller = var.db_backup_storage_controller
  db_backup_compression         = var.db_backup_compression
  db_backup_compression_level   = var.db_backup_compression_level
  db_backup_retention           = var.db_backup_retention
}
```

3.17.4 Setting a maintenance window using the clustercontrol_db_cluster_maintenance resource

You can take adhoc backups (full or incremental) of a cluster using the `clustercontrol_db_cluster_backup` resource. Here's an example of a full backup using `xtrabackup`.

```
resource "clustercontrol_db_cluster_maintenance" "server-upgrade-03312024"
↪{
  depends_on = [clustercontrol_db_cluster.this]
  db_cluster_id       = clustercontrol_db_cluster.this.id
  db_maint_start_time = "Mar-31-2024T00:00"
  db_maint_stop_time  = "Mar-31-2024T23:30"
  db_maint_reason     = "Hardware refresh March 31, 2024"
}
```

Note

Note The `db_maint_start_time` and `db_maint_stop_time` should be specified in local time (without the timezone).

3.18 Supported backup methods for the respective database types (and vendors)

The following types are supported:

Database type	Vendor	Backup method
MySQL	Oracle, Percona	xtrabackupfull, xtrabackupincr, mysqldump
MariaDB	MariaDB	mariabackupfull, mariabackupincr, mysqldump
PostgreSQL	Community	pg_basebackup, pgdumpall, pgbackrest(full,incr,diff)
MongoDB	MongoDB	mongodump, percona-backup-mongodb
Redis	Redis	Use the value “” to indicate (aof – default redis)
SQL Server	Microsoft	mssql_full
Elasticsearch	Elastic	TBD – default snapshot

3.19 Toggling cluster auto-recovery option

You can toggle the cluster-auto-recovery feature in ClusterControl using the `db_auto_recovery` field of the `clustercontrol_db_cluster`resource.

```
resource "clustercontrol_db_cluster" "this" {
...
  db_auto_recovery          = true
...
}
```

cmon_exporter is a tool intended to extract and expose metrics collected by ClusterControl. This project is hosted in Github, cmon_exporter.

3.20 Build

```
make
```

3.21 Create a CMON user for the cmon_exporter

```
s9s user \
    --create \
    --group=admins \
    --generate-key \
    --controller=https://127.0.0.1:9501 \
    --new-password="SECRET" \
```

(continues on next page)

(continued from previous page)

```
    --email-address="admin@example.com" \
    cmon_exporter
```

3.22 How to run

3.22.1 Running in foreground

```
CMON_USERNAME=cmon_exporter CMON_PASSWORD=SECRET␣
↪CMON_ENDPOINT=https://127.0.0.1:9501 ./cmon_exporter
```

3.22.2 Systemd

1. Build or download a released version from here.
2. Copy the `cmon_exporter` binary to `/usr/local/bin`
3. Download cmon_exporter.service and place it at `/etc/systemd/system/cmon_exporter.service`.
4. Change `USER` and `SECRET` to a ClusterControl (cmon user) that has access to all database clusters.
5. Enable and start:

```
systemctl enable cmon_exporter
systemctl restart cmon_exporter
systemctl status cmon_exporter
```

3.22.3 Docker

```
sudo docker run --net=host -it  -e CMON_USERNAME=cmon_exporter -e␣
↪CMON_PASSWORD=SECRET -e CMON_ENDPOINT=https://127.0.0.1:9501 ␣
↪severalnines/cmon_exporter
```

3.23 Access the metrics

```
curl 127.0.0.1:9954/metrics
```

3.24 Current metrics

```
# HELP cmon_alarms_backup_failed_total Total number of failed backups␣
↪alarms.
# TYPE cmon_alarms_backup_failed_total counter
cmon_alarms_backup_failed_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_alarms_critical_total Total number of clusters in unknown state.
# TYPE cmon_alarms_critical_total counter
cmon_alarms_critical_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
```

(continues on next page)

(continued from previous page)

```
# HELP cmon_cluster_backup_failed Is there an active backup failed alarm on␣
↪the cluster.
# TYPE cmon_cluster_backup_failed counter
cmon_cluster_backup_failed{ClusterID="397",ClusterName=
↪"251635de-08f5-4099-8cf0-6637c4fbeddc",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
cmon_cluster_backup_failed{ClusterID="402",ClusterName=
↪"d34fe2fd-17a4-4c1f-8cb9-730331c305bd",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_backup_upload_failed Is there an active backup failed␣
↪alarm on the cluster.
# TYPE cmon_cluster_backup_upload_failed counter
cmon_cluster_backup_upload_failed{ClusterID="397",ClusterName=
↪"251635de-08f5-4099-8cf0-6637c4fbeddc",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
cmon_cluster_backup_upload_failed{ClusterID="402",ClusterName=
↪"d34fe2fd-17a4-4c1f-8cb9-730331c305bd",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_degraded Is the cluster up (DEGRADED) or not.
# TYPE cmon_cluster_degraded counter
cmon_cluster_degraded{ClusterID="397",ClusterName=
↪"251635de-08f5-4099-8cf0-6637c4fbeddc",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_degraded_total Total number of clusters in degraded␣
↪state.
# TYPE cmon_cluster_degraded_total counter
cmon_cluster_degraded_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_failed_init Cluster failed to initialize.
# TYPE cmon_cluster_failed_init counter
cmon_cluster_failed_init{ClusterID="397",ClusterName=
↪"251635de-08f5-4099-8cf0-6637c4fbeddc",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
cmon_cluster_failed_init{ClusterID="402",ClusterName=
↪"d34fe2fd-17a4-4c1f-8cb9-730331c305bd",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_failed_init_total Total number of clusters that failed␣
↪to initialize..
# TYPE cmon_cluster_failed_init_total counter
cmon_cluster_failed_init_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_failed_total Total number of clusters in failed state.
# TYPE cmon_cluster_failed_total counter
cmon_cluster_failed_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 1
# HELP cmon_cluster_failure Is the cluster up (FAILURE) or not.
# TYPE cmon_cluster_failure counter
cmon_cluster_failure{ClusterID="397",ClusterName=
↪"251635de-08f5-4099-8cf0-6637c4fbeddc",ControllerId=
```

(continues on next page)

(continued from previous page)

```
↪"00000000-0000-0000-0000-000000000000"} 0
cmon_cluster_failure{ClusterID="402",ClusterName=
↪"d34fe2fd-17a4-4c1f-8cb9-730331c305bd",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 1
# HELP cmon_cluster_started_total Total number of clusters in started state.
# TYPE cmon_cluster_started_total counter
cmon_cluster_started_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 1
# HELP cmon_cluster_stopped_total Total number of clusters in stopped state.
# TYPE cmon_cluster_stopped_total counter
cmon_cluster_stopped_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_total Total number of clusters.
# TYPE cmon_cluster_total counter
cmon_cluster_total{ControllerId="00000000-0000-0000-0000-000000000000"} 2
# HELP cmon_cluster_unknown_total Total number of clusters in unknown state.
# TYPE cmon_cluster_unknown_total counter
cmon_cluster_unknown_total{ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_cluster_up Is the cluster up (STARTED) or not.
# TYPE cmon_cluster_up counter
cmon_cluster_up{ClusterID="397",ClusterName=
↪"251635de-08f5-4099-8cf0-6637c4fbeddc",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 1
cmon_cluster_up{ClusterID="402",ClusterName=
↪"d34fe2fd-17a4-4c1f-8cb9-730331c305bd",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 0
# HELP cmon_up Was the last  CMON query successful.
# TYPE cmon_up gauge
cmon_up{CmonVersion="1.9.8.7039",ControllerId=
↪"00000000-0000-0000-0000-000000000000"} 1
```

The `cmon_sd` service simplifies Prometheus target discovery for all nodes managed by ClusterControl by exposing an HTTP endpoint. This feature is particularly useful for external Prometheus instances not managed by ClusterControl. The service works by monitoring ClusterControl's internal objects and converting them into readily discoverable Prometheus targets.

This project is publicly hosted in Github, cmon_sd.

Read more

- HTTP-based Service Discovery Configuration for Prometheus

3.25 Install

Navigate to `/usr/local/bin` directory, download and extract the latest binary from here. At the moment, only amd64/x86-64/x64 architecture is supported:

```
cd /usr/local/bin
wget␣
↪https://github.com/severalnines/cmon-sd/releases/download/v0.0.8/cmon_sd_0.0.8_linux_am
tar -xzf cmon_sd_*_linux_amd64.tar.gz
rm -Rf cmon_sd_*_linux_amd64.tar.gz      # clean up
```

3.26 Options

Name, shorthand	Description
`--port`	Listening port, defaults to 8080.

3.27 Usage

1. Create a ClusterControl control user that can log in into ClusterControl. The user must have a global view of all clusters and can be a read-only user. In this example, we will create a user called `cmonsd` which belongs to a global read-only group called `global-ro`:

ClusterControl GUI

1. To create a team, go to ***ClusterControl GUI → User management → Create user or team → Create team***.
2. In the **Team name** field, specify the group name: `global-ro`. Click **Continue**.
3. Under the **Users** section, skip adding a user first as we will add it later. Click **Continue**.
4. Under the **Permissions** section, toggle off the following:
   - Change controller configuration
   - Change LDAP settings
   - Deploy clusters
5. Under the **Clusters permission level** dropdown, choose "View". This only allows users in this group to have read-only permission for all clusters. Click **Continue** and **Finish**.
6. To create a user, go to ***ClusterControl GUI → User management → Create user or team → Create user***.
7. Under the **Details** section, specify the all necessary information. For username, we can specify "cmonsd". Toggle off **Force change password** and click **Continue**.

8. Under the **Team** section, choose "global-ro" from the dropdown. Click **Continue** and **Finish** to create the user.

2. Pass `CMON_USERNAME` and `CMON_PASSWORD` environment variables to `cmon_sd` binary:

```
CMON_USERNAME='cmonsd' CMON_PASSWORD='SuperSecret$$$P455' ./cmon_sd -p␣
↪9909
```

3. Verify the HTTP response:

```
$ curl http://127.0.0.1:9909
[
    {
        "targets": [
            "10.10.10.17:9100",
            "10.10.10.17:9011",
            "10.10.10.17:9104",
            "10.10.10.16:9100",
            "10.10.10.16:9011",
            "10.10.10.16:9104",
            "10.10.10.18:9100",
            "10.10.10.18:9011",
            "10.10.10.18:9104"
        ],
        "labels": {
            "ClusterID": "641",
            "ClusterName": "PXC57",
            "ClusterType": "GALERA",
            "cid": "641"
        }
    }
]
```

4. Alternatively, you can configure it as a systemd unit. See *Run as systemd*.

3.28 Run as systemd

1. To create a systemd unit file for `cmon_sd` service, save the following content to `/etc/systemd/system/cmon_sd.service`:

```
[Unit]
Description=CMON Service Discovery
# Ensures cmon starts before this service
After=cmon.service
# Makes this service dependent on cmon
Requires=cmon.service

[Service]
Type=simple
ExecStart=/usr/local/bin/cmon_sd -p 9909
WorkingDirectory=/usr/local/bin/
```

(continues on next page)

(continued from previous page)

```
# Environment variables
Environment="CMON_USERNAME=cmonsd"
Environment="CMON_PASSWORD=SuperSecret$$$P455"

# Security and Reliability
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

2. Apply the configuration by running:

```
sudo systemctl daemon-reload
```

3. Start the service immediately and enable it to run on boot:

```
sudo systemctl start cmon_sd
sudo systemctl enable cmon_sd
```

3.29 Sample prometheus.yml

On your Prometheus instance, configure the `http_sd_configs` accordingly:

```
---
alerting:
  alertmanagers:
  - static_configs:
    - targets:
global:
  evaluation_interval: 10s
  external_labels:
    monitor: clustercontrol
  scrape_interval: 10s
  scrape_timeout: 10s
rule_files:
scrape_configs:
- http_sd_configs:
  - url: http://10.10.10.1:9909
  job_name: clustercontrol
...
```

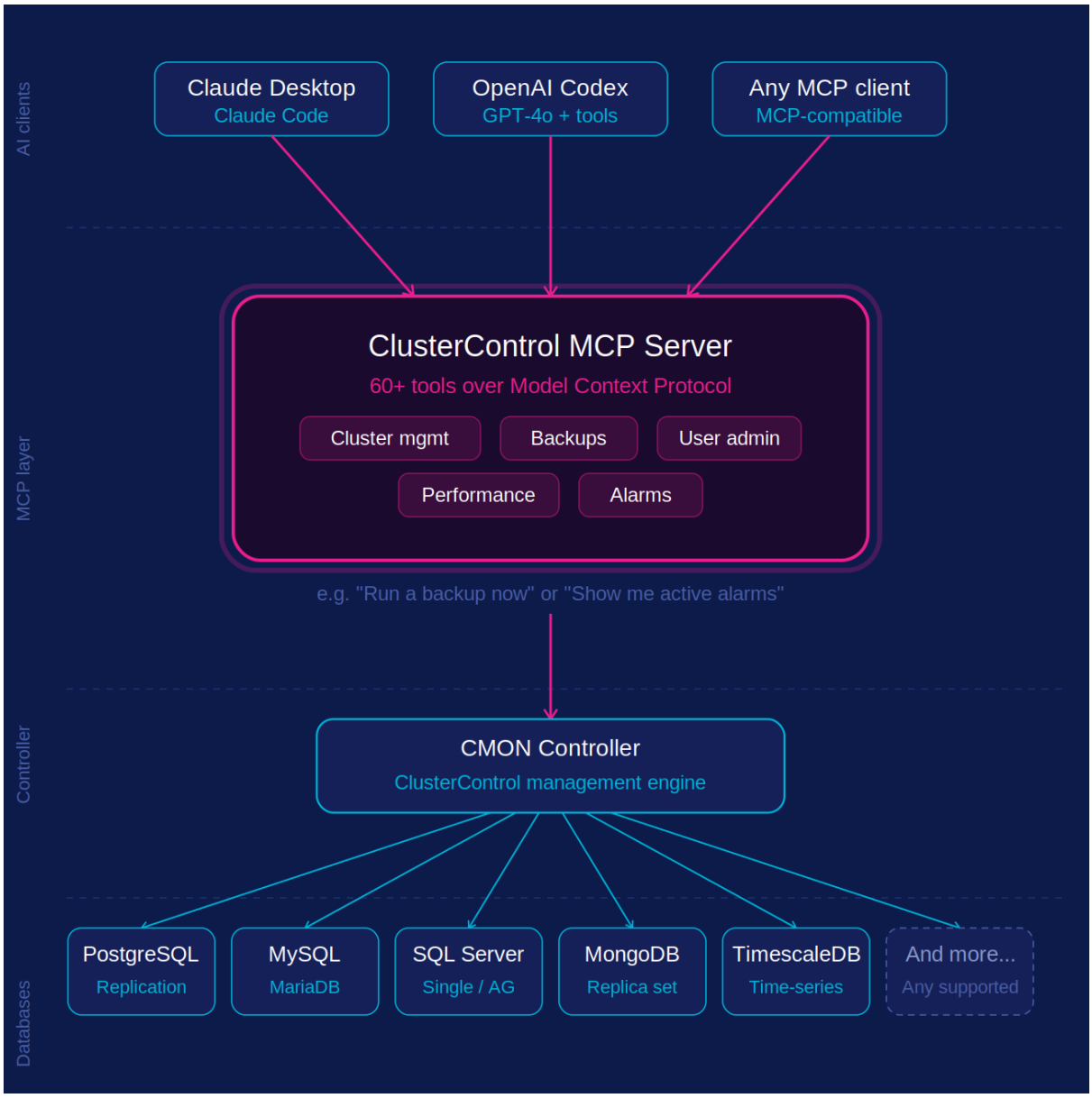
Restart Prometheus to apply the change. The Prometheus should start scraping all targets managed by ClusterControl.

3.30 ClusterControl MCP

The ClusterControl MCP Server is a bridge between your AI assistant and your databases. You type what you need in plain English or other languages, and it figures out the rest. No dashboards to click through, no commands to memorize. It works with AI tools like Claude Desktop, Claude Code, and OpenAI Codex, and supports more than 50 actions and counting, covering backups, cluster management, user accounts, and performance monitoring. It works with MySQL, MariaDB, PostgreSQL, MongoDB, SQL Server, and TimescaleDB.

3.30.1 How It Works

The diagram below shows how everything connects. Your AI tool sits at the top. When you type a request, it goes through the MCP Server, which translates it into the right action and passes it to the CMON Controller. The controller then talks to whichever database cluster you are working with.



Before you get started, make sure you have the following in place:

- A running ClusterControl installation with CMON Controller accessible over the network (CMON

controller reachable on port 9501)

- A CMON user account with access to the clusters you want to query
- An MCP-compatible AI client such as Claude Desktop, Claude Code, or OpenAI Codex

Once you have confirmed all the requirements above are in place, you are ready to install and configure the ClusterControl MCP Server. Packages are published to the Severalnines repository alongside other ClusterControl components.

RHEL / Rocky / AlmaLinux

```
sudo yum install clustercontrol-mcp
```

Debian / Ubuntu

```
sudo apt-get install clustercontrol-mcp
```

The binary installs to `/usr/bin/cmon-mcp`. A systemd service unit and default configuration file are also installed:

- `/etc/systemd/system/cmon-mcp.service` : service unit
- `/etc/default/cmon-mcp` : environment file for credentials and transport options

3.30.2 Configuration

The ClusterControl MCP Server is configured through the `/etc/default/cmon-mcp` file on your CMON host. Open the file and edit the values to match your environment:

```
sudo vi /etc/default/cmon-mcp
```

Configuration Parameters

CMON Controller Connection

Parameter	Description	Example
`CMON_ENDPOINT`	The URL of your CMON Controller API	`https://127.0.0.1:9501`
`CMON_USERNAME`	The ClusterControl username	`admin`
`CMON_PASSWORD`	Password for the user (leave empty if using key auth)	`severalNINES999`
`CMON_KEY_FILE`	Path to RSA private key (alternative to password)	`/etc/clustercontrol/id_rsa`

You can authenticate using either a password or an RSA key file, not both at the same time. If `CMON_PASSWORD` is set, leave `CMON_KEY_FILE` empty, and vice versa.

The CMON user you specify here is the identity the MCP Server uses for every action it performs. We recommend creating a dedicated CMON user with only the privileges your team actually needs and please avoid reusing your main admin account.

MCP Server

Parameter	Description	Example
MCP_BIND_ADDRESS	Address and port the MCP Server listens on	0.0.0.0:3000
MCP_BASE_URL	Public URL that AI clients use to connect	http://your-cc-host:3000
MCP_AUTH_TOKEN	Bearer token to secure all incoming connections	your-strong-random-token

For MCP_BIND_ADDRESS, use 127.0.0.1:3000 if you only need local access, or 0.0.0.0:3000 to allow remote AI clients to connect.

MCP Transport Endpoints

The MCP Server exposes two transports on the same port: /sse and /message is SSE transport, used by Claude Desktop and Claude Code. /mcp is Streamable HTTP transport, used by OpenAI Codex

Complete Configuration Example

The following shows the ClusterControl MCP Server example configuration after it has been set up through the /etc/default/cmon-mcp file on your CMON host.

```
# CMON Controller connection
CMON_ENDPOINT=https://127.0.0.1:9501
CMON_USERNAME=admin
CMON_PASSWORD='severalNINES999'
# RSA key auth alternative (leave CMON_PASSWORD empty if using this):
# CMON_KEY_FILE=/etc/clustercontrol/id_rsa

# MCP Server
MCP_BIND_ADDRESS=0.0.0.0:3000
MCP_BASE_URL=http://your-cc-host:3000
MCP_AUTH_TOKEN=
```

Generating a Secure Token

It is strongly recommended to protect your MCP Server with a bearer token, especially when MCP_BIND_ADDRESS is set to 0.0.0.0. Generate one with:

```
openssl rand -hex 32
```

Copy the output and set it as the value of MCP_AUTH_TOKEN.

Starting the Service

Save the configuration file, then restart the service:

```
systemctl restart cmon-mcp
```

Confirm it started successfully:

```
journalctl -u cmon-mcp -n 20
```

You should see output confirming the MCP Server is running and listening on the address defined in `MCP_BIND_ADDRESS`.

3.30.3 Connecting to AI Clients

Once the MCP Server is running, you need to register it with your AI client. The steps differ slightly depending on which client you use.

Claude Desktop

Open your Claude Desktop configuration file:

- **macOS:** `~/Library/Application Support/Claude/claude_desktop_config.json`
- **Windows:** `%APPDATA%\Claude\claude_desktop_config.json`

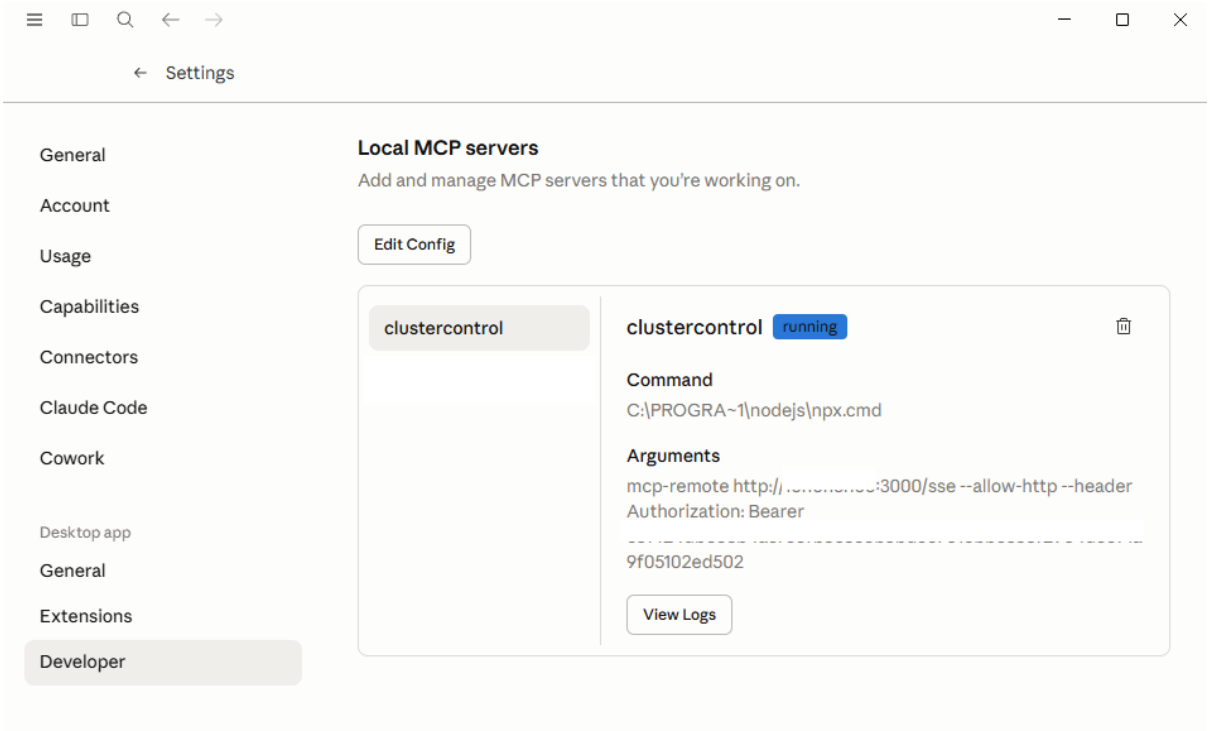
Add the following entry under `mcpServers` so that it looks like this :

```
{
  "mcpServers": {
    "clustercontrol": {
      "type": "sse",
      "url": "http://your-cc-host:3000/sse",
      "headers": {
        "Authorization": "Bearer your-auth-token"
      }
    }
  }
}
```

Alternatively, you can use mcp-remote with Node.js installed on your environment with the following JSON configuration:

```
{
  "mcpServers": {
    "clustercontrol": {
      "command": "npx",
      "args": [
        "mcp-remote",
        "http://your-cc-host:3000/sse",
        "--allow-http",
        "--header",
        "Authorization: Bearer your-auth-token"
      ]
    }
  }
}
```

Save the file and restart Claude Desktop. You should see clustercontrol appear in the File → Settings → developer → Local MCP servers panel like this after restart Claude Desktop.



Claude Code

Run the following command in your terminal:

```
claude mcp add clustercontrol -- cmon-mcp \
  -endpoint https://your-cc-host:9501 \
  -username admin \
  -password your-password
```

To verify it was added:

```
claude mcp list
```

OpenAI Codex

Add the MCP Server to your Codex tool configuration using the `/mcp` endpoint:

```
{
  "tools": [
    {
      "type": "mcp",
      "server_label": "clustercontrol",
      "server_url": "http://your-cc-host:3000/mcp",
      "headers": {
        "Authorization": "Bearer your-auth-token"
      },
      "require_approval": "never"
    }
  ]
}
```

3.30.4 Verifying the Connection

Once connected, ask your AI assistant the following to confirm everything is working:

```
List all my database clusters
```

If the setup is correct, it will return the list of clusters managed by your ClusterControl instance.